

Microsoft®

# РАЗРАБОТКА ПРИЛОЖЕНИЙ НА ОСНОВЕ

Microsoft®

# SQL Server® 2008



Леонард Лобел  
Эндрю Дж. Браст  
Стивен Форте

**twentysix**  
NEW YORK



РУССКАЯ РЕДАКЦИЯ

**Microsoft®**

# Programming Microsoft® SQL Server® 2008

*Leonard Lobel  
Andrew J. Brust  
Stephen Forte*

**Леонард Лобел  
Эндрю Дж. Браст  
Стивен Форте**

**РАЗРАБОТКА ПРИЛОЖЕНИЙ  
НА ОСНОВЕ  
Microsoft®  
SQL Server® 2008**

«Русская Редакция»

«БХВ-Петербург»

2010

УДК 681.3.06  
ББК 32.973.26-018.2  
Л68

**Лобел, Л.**

Л68 Разработка приложений на основе Microsoft® SQL Server® 2008 / Л. Лобел, Э. Дж. Браст, С. Форте: Пер. с англ. — М.: Русская редакция; СПб.: БХВ-Петербург, 2010. — 1024 с.: ил.

ISBN 978-5-7502-0393-2 ("Русская редакция")

ISBN 978-5-9775-0452-2 ("БХВ-Петербург")

В практическом руководстве по разработке приложений на основе SQL Server 2008 подробно излагаются основные технологии SQL Server: улучшения языка T-SQL, расширенные возможности программирования в кодах SQL CLR (на таких .NET-языках, как Microsoft Visual Basic .NET и C#), управление сервером, безопасность. Рассматриваются вопросы использования нереляционных данных в мире реляционных баз данных, технологии создания приложений, работающих с базами данных и улучшающих их досягаемость, а также вопросы по управлению базами данных и программированию с точки зрения стратегического анализа, а именно: создание хранилищ данных и использование их с помощью OLAP, интеллектуального анализа данных и служб отчетности.

*Для разработчиков*

УДК 681.3.06  
ББК 32.973.26-018.2

© 2010-2012, Translation Russian Edition Publishers

Authorized Russian translation of the English edition of Programming Microsoft® SQL Server® 2008, ISBN 978-0-7356-2599-0 © Andrew Brust, Leonard Lobel, and Stephen Forte.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

© 2010-2012, перевод ООО «Издательство «Русская редакция», издательство «БХВ-Петербург».

Авторизованный перевод с английского на русский язык произведения Programming Microsoft® SQL Server® 2008, ISBN 978-0-7356-2599-0 © Andrew Brust, Leonard Lobel, and Stephen Forte.

Этот перевод оригинального издания публикуется и продается с разрешения O'Reilly Media, Inc., которая владеет или распоряжается всеми правами на его публикацию и продажу.

© 2010-2012, оформление и подготовка к изданию, ООО «Издательство «Русская редакция», издательство «БХВ-Петербург».

Microsoft, а также товарные знаки, перечисленные в списке, расположенном по адресу: <http://www.microsoft.com/about/legal/en/us/IntellectualProperty/Trademarks/EN-US.aspx> являются товарными знаками или охраняемыми товарными знаками корпорации Microsoft в США и/или других странах. Все другие товарные знаки являются собственностью соответствующих фирм.

Все названия компаний, организаций и продуктов, а также имена лиц, используемые в примерах, вымышлены и не имеют никакого отношения к реальным компаниям, организациям, продуктам и лицам.

**Лобел Леонард, Браст Эндрю Дж., Форте Стивен**  
**Разработка приложений на основе Microsoft® SQL Server® 2008**  
**Перевод с английского языка Андрея Лашкевича**

Совместный проект издательства "Русская редакция" и издательства "БХВ-Петербург"

 РУССКАЯ РЕДАКЦИЯ

 **bhv®**



# Оглавление

- ПРОГРАММИРОВАНИЕ MICROSOFT® SQL SERVER® 2008 ..... 15
- БЛАГОДАРНОСТИ ..... 16
- ВВЕДЕНИЕ ..... 19
  - Для кого написана эта книга ..... 20
  - Как организована эта книга..... 22
  - Примеры кодов и Web-сайт этой книги ..... 24
  - Использование базы данных *Northwind* ..... 25
  - Использование баз данных *AdventureWorks2008* ..... 26
  - Системные требования ..... 27
  - Поддержка этой книги ..... 28
- ЧАСТЬ I. ФУНДАМЕНТАЛЬНЫЕ ОСНОВЫ ..... 29
- ГЛАВА 1. ОБЗОР..... 30
  - Насколько он значителен ..... 31
  - Книга для разработчиков..... 33
  - Книга, которая написана разработчиками ..... 34
  - Книга, которая укажет вам путь..... 35
    - Фундаментальные технологии ..... 35
    - За пределами реляционности ..... 37
    - Улучшаем досягаемость ..... 38
    - Стратегии бизнес-аналитики ..... 40
  - Резюме..... 42
- ГЛАВА 2. УЛУЧШЕНИЯ ЯЗЫКА T-SQL ..... 43
  - Обобщенные табличные выражения ..... 44
    - Создание рекурсивных запросов при помощи выражения CTE ..... 49
  - Операторы *PIVOT* и *UNPIVOT* ..... 53
    - Использование оператора *UNPIVOT*..... 54
    - Динамическое сведение столбцов ..... 56
    - Оператор *APPLY* ..... 58
  - Улучшения опции *TOP* ..... 59
  - Ранжирование функций ..... 61
    - Функция *ROW\_NUMBER*..... 62
    - Функция *RANK* ..... 66
    - Функции *DENSE\_RANK* и *NTILE*..... 68
    - Совместное использование всех функций ранжирования ..... 71
    - Ранжирование по группам при помощи *PARTITION BY* ..... 72
  - Обработка исключительных состояний в транзакциях..... 76

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| Тип данных <i>varchar(max)</i> .....                                        | 79         |
| Инструкция <i>WAITFOR</i> .....                                             | 79         |
| Триггеры DDL .....                                                          | 80         |
| Изоляция <i>SNAPSHOT</i> .....                                              | 82         |
| Возвращающие табличные значение параметры .....                             | 82         |
| Не просто еще одна временная таблица .....                                  | 83         |
| Работаем с набором из большого количества строк .....                       | 85         |
| Использование TVP для массовых вставок и обновлений .....                   | 87         |
| Работаем с единственной строкой данных .....                                | 89         |
| Создание TVP в стиле словаря .....                                          | 93         |
| Передача TVP при помощи ADO.NET .....                                       | 95         |
| Ограничения TVP .....                                                       | 98         |
| Новые типы данных для даты и времени .....                                  | 99         |
| Разделение даты и времени .....                                             | 99         |
| Дополнительные переносимые дата и время .....                               | 100        |
| Часовые пояса .....                                                         | 101        |
| Дата и время: точность, хранение и формат .....                             | 103        |
| Новые и измененные функции .....                                            | 106        |
| Инструкция <i>MERGE</i> .....                                               | 109        |
| Определение источника и назначения <i>MERGE</i> .....                       | 112        |
| Выражение <i>WHEN MATCHED</i> .....                                         | 113        |
| Выражение <i>WHEN NOT MATCHED BY TARGET</i> .....                           | 114        |
| Использование <i>MERGE</i> для репликации таблиц .....                      | 115        |
| Вывод в инструкции <i>MERGE</i> .....                                       | 119        |
| Выбор метода соединения .....                                               | 120        |
| Поведение инструкции <i>MERGE</i> в DML .....                               | 122        |
| Выполнение вставки/обновления .....                                         | 125        |
| Синтаксис <i>INSERT OVER DML</i> .....                                      | 136        |
| Расширение инструкции <i>OUTPUT...INTO</i> .....                            | 137        |
| Использование ключевого слова <i>CHANGES</i> .....                          | 142        |
| Резюме .....                                                                | 145        |
| <b>ГЛАВА 3. ИЗУЧАЕМ SQL CLR .....</b>                                       | <b>146</b> |
| Приступаем: включаем интеграцию с CLR .....                                 | 148        |
| Интеграция Visual Studio/SQL Server .....                                   | 149        |
| Проекты SQL Server Project в Visual Studio .....                            | 149        |
| Автоматическое развертывание .....                                          | 153        |
| Атрибуты кода SQL CLR .....                                                 | 153        |
| Ваша первая хранимая процедура SQL CLR .....                                | 155        |
| Хранимые процедуры CLR и доступ к данным на стороне сервера .....           | 157        |
| Пересылка данных при помощи <i>SqlDataRecord</i> и <i>SqlMetaData</i> ..... | 160        |
| Развертывание .....                                                         | 163        |
| Развертывание вашей сборки .....                                            | 163        |
| Развертывание ваших хранимых процедур .....                                 | 166        |
| Тестирование хранимых процедур .....                                        | 167        |
| Функции CLR .....                                                           | 170        |

|                                                                   |            |
|-------------------------------------------------------------------|------------|
| Триггеры CLR .....                                                | 176        |
| Агрегаты CLR.....                                                 | 182        |
| Типы SQL CLR.....                                                 | 188        |
| Безопасность.....                                                 | 194        |
| Изучение типов SQL CLR в базе данных и управление ими.....        | 197        |
| Лучшие практики использования SQL CLR .....                       | 207        |
| Резюме.....                                                       | 208        |
| <b>ГЛАВА 4. УПРАВЛЕНИЕ СЕРВЕРОМ.....</b>                          | <b>209</b> |
| Что такое SMO .....                                               | 210        |
| По поводу SQL-DMO.....                                            | 211        |
| Новейшие функциональные возможности SMO .....                     | 217        |
| Работа с SMO в Microsoft Visual Studio .....                      | 218        |
| Перебор доступных серверов.....                                   | 221        |
| Получение настроек сервера .....                                  | 223        |
| Создание приложений резервного копирования и восстановления ..... | 228        |
| Выполнение программных функций DBCC при помощи SMO.....           | 235        |
| Управление на основе политик .....                                | 238        |
| Простая политика.....                                             | 239        |
| Резюме.....                                                       | 244        |
| <b>ГЛАВА 5. БЕЗОПАСНОСТЬ В SQL SERVER 2008 .....</b>              | <b>245</b> |
| Четыре темы инфраструктуры безопасности .....                     | 246        |
| Конструктивная безопасность.....                                  | 246        |
| Безопасность по умолчанию .....                                   | 246        |
| Безопасность по развертыванию .....                               | 247        |
| Безопасный обмен.....                                             | 247        |
| Обзор безопасности SQL Server 2008.....                           | 248        |
| Регистрационные записи SQL Server .....                           | 249        |
| Пользователи базы данных .....                                    | 251        |
| Пользовательская учетная запись <i>guest</i> .....                | 252        |
| Аутентификация и авторизация .....                                | 253        |
| Как клиенты устанавливают подключение .....                       | 254        |
| Политики паролей.....                                             | 256        |
| Разделение пользователь—схема .....                               | 258        |
| Контекст выполнения .....                                         | 260        |
| Поддержка шифрования в SQL Server .....                           | 264        |
| Шифрование данных при их перемещении .....                        | 266        |
| Шифрование данных в покое.....                                    | 268        |
| Прозрачное шифрование данных в SQL Server 2008.....               | 273        |
| Создание ключей и сертификатов .....                              | 274        |
| Включение TDE .....                                               | 275        |
| Запросы к представлениям TDE .....                                | 276        |
| Резервное копирование сертификата .....                           | 278        |
| Восстановление зашифрованной базы данных.....                     | 278        |
| Аудит SQL Server.....                                             | 279        |

|                                        |     |
|----------------------------------------|-----|
| Создание объекта аудита .....          | 280 |
| Опции аудита .....                     | 281 |
| Запись аудита в файловую систему ..... | 283 |
| Резюме .....                           | 284 |

## **ЧАСТЬ II. ЗА ПРЕДЕЛАМИ РЕЛЯЦИОННОСТИ .....285**

### **ГЛАВА 6. XML И РЕЛЯЦИОННАЯ БАЗА ДАННЫХ .....286**

|                                                     |     |
|-----------------------------------------------------|-----|
| XML в SQL Server 2000 .....                         | 288 |
| XML в SQL Server 2008 — тип данных <i>xml</i> ..... | 290 |
| Работаем с типом <i>xml</i> как с переменной .....  | 290 |
| Работаем с XML в таблицах .....                     | 292 |
| Значения по умолчанию и ограничения .....           | 293 |
| XML-схемы .....                                     | 293 |
| XML-индексы .....                                   | 302 |
| Команды <i>FOR XML</i> .....                        | 306 |
| Синтаксис <i>FOR XML RAW</i> .....                  | 306 |
| Синтаксис <i>FOR XML AUTO</i> .....                 | 307 |
| Синтаксис <i>FOR XML EXPLICIT</i> .....             | 309 |
| Улучшения инструкции <i>FOR XML</i> .....           | 314 |
| Улучшения <i>OPENXML</i> в SQL Server 2008 .....    | 323 |
| XML Bulk Load .....                                 | 325 |
| Запрашиваем XML-данные при помощи XQuery .....      | 326 |
| Разбираемся с выражениями XQuery и XPath .....      | 326 |
| XQuery сервера SQL Server 2008 в действии .....     | 329 |
| Расширения XQuery в SQL Server .....                | 339 |
| Язык XML DML .....                                  | 341 |
| Преобразование столбца в XML .....                  | 342 |
| Резюме .....                                        | 344 |

### **ГЛАВА 7. ИЕРАРХИЧЕСКИЕ ДАННЫЕ И РЕЛЯЦИОННАЯ БАЗА ДАННЫХ .....345**

|                                                     |     |
|-----------------------------------------------------|-----|
| Тип данных <i>hierarchyid</i> .....                 | 346 |
| Создание иерархической таблицы .....                | 348 |
| Метод <i>GetLevel</i> .....                         | 349 |
| Заполнение иерархии .....                           | 350 |
| Метод <i>GetRoot</i> .....                          | 351 |
| Метод <i>GetDescendant</i> .....                    | 352 |
| Метод <i>ToString</i> .....                         | 353 |
| Метод <i>GetAncestor</i> .....                      | 359 |
| Стратегии индексирования иерархических таблиц ..... | 364 |
| Индексирование в глубину .....                      | 365 |
| Индексирование в ширину .....                       | 365 |
| Запросы к иерархическим таблицам .....              | 366 |
| Изменение порядка узлов в иерархии .....            | 369 |
| Трансплантация поддеревьев .....                    | 371 |
| Другие методы <i>hierarchyid</i> .....              | 373 |
| Резюме .....                                        | 374 |

|                                                                      |            |
|----------------------------------------------------------------------|------------|
| <b>ГЛАВА 8. ХРАНЕНИЕ НЕСТРУКТУРИРОВАННЫХ ДАННЫХ .....</b>            | <b>375</b> |
| BLOB в базе данных .....                                             | 376        |
| BLOB в файловой системе .....                                        | 377        |
| Что такое атрибут.....                                               | 378        |
| Включение FILESTREAM.....                                            | 379        |
| Включение FILESTREAM для компьютера .....                            | 380        |
| Включение FILESTREAM для экземпляра сервера .....                    | 381        |
| Создание базы данных с использованием FILESTREAM .....               | 382        |
| Создание таблицы со столбцами FILESTREAM .....                       | 383        |
| Интерфейс прикладного программирования неуправляемых клиентов        |            |
| <i>OpenSqlFilestream</i> .....                                       | 386        |
| Потоковая передача файлов в .NET .....                               | 388        |
| Разбираемся с доступом FILESTREAM к данным .....                     | 389        |
| Резюме.....                                                          | 400        |
| <b>ГЛАВА 9. ГЕОПРОСТРАНСТВЕННЫЕ ТИПЫ ДАННЫХ .....</b>                | <b>401</b> |
| SQL Server 2008 увеличивает возможности .....                        | 402        |
| Пространственные модели .....                                        | 403        |
| Планарная модель "плоской Земли" .....                               | 403        |
| Геодезическая модель "круглой Земли" .....                           | 404        |
| Пространственные типы данных .....                                   | 405        |
| Определяем пространство при помощи Well-Known Text .....             | 405        |
| Резюме.....                                                          | 406        |
| <b>ЧАСТЬ III. ТЕХНОЛОГИИ ДОСТУПА .....</b>                           | <b>407</b> |
| <b>ГЛАВА 10. MICROSOFT DATA ACCESS MACHINE .....</b>                 | <b>408</b> |
| ADO.NET и типизированные наборы <i>DataSet</i> .....                 | 409        |
| Основы типизированных объектов <i>DataSet</i> .....                  | 409        |
| Объекты <i>TableAdapter</i> .....                                    | 412        |
| Управление строкой подключения .....                                 | 413        |
| Использование мастера <i>TableAdapter Configuration Wizard</i> ..... | 414        |
| Дополнительно о запросах и параметрах.....                           | 419        |
| Методы <i>DBDirect</i> и подключенное использование типизированных   |            |
| объектов <i>DataSet</i> .....                                        | 421        |
| ADO.NET: работаем в коде .....                                       | 421        |
| Простейший запрос.....                                               | 422        |
| LINQ: новый синтаксический подход для доступа к данным .....         | 428        |
| LINQ для <i>DataSet</i> .....                                        | 429        |
| Синтаксис LINQ .....                                                 | 430        |
| LiNQ для SQL и ADO.NET Entity Framework: ORM приходит в .NET .....   | 432        |
| Почему нельзя ограничиться технологией ADO.NET .....                 | 433        |
| Создаем модель L2S .....                                             | 434        |
| Изучение модели .....                                                | 436        |
| Резюме: так много инструментов, так мало времени.....                | 440        |

|                                                                        |                |
|------------------------------------------------------------------------|----------------|
| <b>ГЛАВА 11. МНОГОГРАННОСТЬ СВЯЗЫВАНИЯ ДАННЫХ В .NET .....</b>         | <b>443</b>     |
| Связывание данных в Windows Forms — золотой стандарт.....              | 444            |
| Подготовка.....                                                        | 445            |
| Генерирование пользовательского интерфейса.....                        | 446            |
| Изучаем выходные данные.....                                           | 448            |
| Конвертация в LINQ для SQL .....                                       | 450            |
| Преобразование к Entity Framework .....                                | 451            |
| Преобразование для ADO.NET Data Services.....                          | 453            |
| Связывание данных в Web-приложениях при помощи ASP.NET .....           | 454            |
| L2S и EF — это просто .....                                            | 455            |
| Не только простые сетки .....                                          | 456            |
| Связывание данных при помощи разметки .....                            | 457            |
| Использование AJAX для простого доступа к данным .....                 | 458            |
| Динамические данные ASP.NET .....                                      | 464            |
| Привязка данных в Windows Presentation Foundation .....                | 467            |
| Трудности на этапе конструирования .....                               | 468            |
| Изучение XAML.....                                                     | 472            |
| Грандиозный финал: Silverlight .....                                   | 476            |
| Резюме.....                                                            | 479            |
| <br><b>ГЛАВА 12. ТРАНЗАКЦИИ .....</b>                                  | <br><b>480</b> |
| Что такое транзакция .....                                             | 481            |
| Свойства ACID.....                                                     | 482            |
| Поддержка локальных транзакций в SQL Server 2008.....                  | 485            |
| Режим транзакций Autocommit.....                                       | 485            |
| Режим транзакций Explicit .....                                        | 485            |
| Режим транзакций Implicit .....                                        | 489            |
| Пакетный режим транзакций .....                                        | 490            |
| Использование локальных транзакций в ADO.NET .....                     | 492            |
| Терминология транзакций.....                                           | 495            |
| Уровни изоляции.....                                                   | 495            |
| Уровни изоляции в SQL Server 2008 .....                                | 496            |
| Уровни изоляции в ADO.NET.....                                         | 502            |
| Распределенные транзакции.....                                         | 503            |
| Терминология распределенных транзакций .....                           | 504            |
| Правила и методы прикрепления.....                                     | 506            |
| Распределенные транзакции в SQL Server 2008 .....                      | 508            |
| Распределенные транзакции в .NET Framework.....                        | 509            |
| Резюме.....                                                            | 515            |
| <br><b>ГЛАВА 13. РАЗРАБОТКА ПЕРИОДИЧЕСКИ ПОДКЛЮЧЕННЫХ СИСТЕМ .....</b> | <br><b>516</b> |
| Сравниваем службы Sync Services и Merge Replication .....              | 518            |
| Компоненты периодически подключенной системы .....                     | 519            |
| Merge Replication.....                                                 | 520            |
| Знакомимся с Merge Replication .....                                   | 521            |

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| Создание периодически подключаемого приложения<br>с использованием Merge Replication..... | 522        |
| Конфигурирование Merge Replication .....                                                  | 526        |
| Создание мобильного приложения при помощи Microsoft Visual Studio 2008 .....              | 555        |
| Службы синхронизации Sync Services для ADO.NET.....                                       | 572        |
| Объектная модель Sync Services .....                                                      | 573        |
| Сбор изменений для синхронизации .....                                                    | 578        |
| Создание приложения при помощи служб Sync Services .....                                  | 585        |
| Дополнительные соображения.....                                                           | 603        |
| Резюме.....                                                                               | 608        |
| <b>ЧАСТЬ IV. БИЗНЕС-АНАЛИТИКА.....</b>                                                    | <b>609</b> |
| <b>ГЛАВА 14. ХРАНИЛИЩА ДАННЫХ.....</b>                                                    | <b>610</b> |
| Определение хранилищ данных.....                                                          | 610        |
| Важность хранилищ данных .....                                                            | 611        |
| Что предшествовало хранилищам данных.....                                                 | 613        |
| Отсутствие интеграции с предприятием .....                                                | 614        |
| Отсутствие стандартизированных справочных данных .....                                    | 616        |
| Отсутствие ретроспективных данных .....                                                   | 616        |
| Данные не оптимизированы для анализа .....                                                | 617        |
| Проектирование хранилищ данных .....                                                      | 619        |
| Подход сверху вниз.....                                                                   | 621        |
| Подход снизу вверх.....                                                                   | 622        |
| Чем хранилище данных не является.....                                                     | 631        |
| OLAP .....                                                                                | 631        |
| Интеллектуальный анализ данных .....                                                      | 632        |
| Бизнес-аналитика .....                                                                    | 633        |
| Панели и системы показателей .....                                                        | 634        |
| Управление производительностью .....                                                      | 637        |
| Практические советы по хранилищам данных .....                                            | 638        |
| Предвидим и поощряем изменение операционных процессов .....                               | 638        |
| Поощряем отказ от управления .....                                                        | 638        |
| Прототип может не помочь при "продаже идеи" .....                                         | 639        |
| Проблемы суррогатных ключей .....                                                         | 639        |
| Проблемы конвертирования валют .....                                                      | 640        |
| События и моментальные снимки .....                                                       | 641        |
| SQL Server 2008 и хранилища данных .....                                                  | 642        |
| Инструкция <i>MERGE</i> языка T-SQL.....                                                  | 643        |
| Отслеживание изменений Change Data Capture.....                                           | 646        |
| Параллелизм секционированных таблиц .....                                                 | 657        |
| Оптимизация запросов для соединений типа "звезда" .....                                   | 660        |
| Резюме.....                                                                               | 661        |
| <b>ГЛАВА 15. ОСНОВЫ OLAP.....</b>                                                         | <b>662</b> |
| Для чего существует бизнес-аналитика.....                                                 | 662        |
| Начальный курс OLAP .....                                                                 | 664        |

|                                                                             |            |
|-----------------------------------------------------------------------------|------------|
| Словарь OLAP .....                                                          | 666        |
| Измерения, оси, звезды и снежинки .....                                     | 667        |
| Построение вашего первого куба .....                                        | 670        |
| Подготовка объектов схемы звезды .....                                      | 670        |
| Инструмент (как бы его ни называли).....                                    | 671        |
| Создание проекта .....                                                      | 673        |
| Добавление представления источника данных.....                              | 675        |
| Создание куба при помощи мастера Cube Wizard.....                           | 681        |
| Использование визуального конструктора Cube Designer.....                   | 683        |
| Использование мастера измерений Dimension Wizard .....                      | 687        |
| Использование визуального конструктора измерений Dimension Designer.....    | 690        |
| Работаем с окном свойств <i>Properties</i> и <i>Solution Explorer</i> ..... | 693        |
| Обработка куба.....                                                         | 694        |
| Выполнение запросов .....                                                   | 695        |
| Резюме.....                                                                 | 697        |
| <b>Глава 16. РАСШИРЕННЫЙ OLAP.....</b>                                      | <b>698</b> |
| Что мы расскажем в этой главе.....                                          | 699        |
| MDX в контексте.....                                                        | 700        |
| А теперь несколько слов о нашем спонсоре.....                               | 700        |
| Расширенные измерения и меры .....                                          | 701        |
| Ключи и названия.....                                                       | 701        |
| Изменение элемента <i>All</i> .....                                         | 704        |
| Добавление именованного запроса в представление источника данных .....      | 706        |
| Измерения родитель/потомок .....                                            | 709        |
| Группирование элементов.....                                                | 714        |
| Пользовательские таблицы и типизирование "измерение/атрибут" .....          | 715        |
| Серверные измерения времени .....                                           | 726        |
| Измерения фактов.....                                                       | 728        |
| Ролевые измерения .....                                                     | 732        |
| Расширенные меры .....                                                      | 733        |
| Вычисления .....                                                            | 735        |
| Вычисляемые элементы .....                                                  | 736        |
| Именованные наборы .....                                                    | 743        |
| Дополнительная информация по представлению скриптов .....                   | 744        |
| Ключевые индикаторы производительности.....                                 | 747        |
| Визуализация KPI: состояние и тенденция.....                                | 748        |
| Конкретный KPI.....                                                         | 749        |
| Тестирование KPI в представлении браузера.....                              | 752        |
| Интерпретация результатов и модификация среза.....                          | 753        |
| Запросы KPI в Management Studio .....                                       | 755        |
| Прочие фокусы бизнес-аналитики в Management Studio .....                    | 760        |
| Действия.....                                                               | 762        |
| Действия определяются просто .....                                          | 762        |
| Проектирование действий.....                                                | 763        |
| Тестирование действий.....                                                  | 765        |



|                                                                          |            |
|--------------------------------------------------------------------------|------------|
| Секции, настройки хранения и упреждающее кэширование.....                | 767        |
| Редактирование и создание секций .....                                   | 768        |
| Опции хранения секции.....                                               | 771        |
| Упреждающее кэширование .....                                            | 773        |
| Дополнительные функциональные возможности и советы.....                  | 775        |
| Агрегаты .....                                                           | 776        |
| Проектирование алгоритмического агрегирования .....                      | 777        |
| Проектирование агрегатов на основе использования .....                   | 778        |
| Проектирование агрегатов вручную.....                                    | 779        |
| Управление проектированием агрегирования .....                           | 781        |
| Схема агрегирования и Management Studio .....                            | 783        |
| Перспективы .....                                                        | 783        |
| Резюме.....                                                              | 786        |
| <b>ГЛАВА 17. OLAP: ЗАПРОСЫ, ИНСТРУМЕНТЫ И РАЗРАБОТКА ПРИЛОЖЕНИЙ.....</b> | <b>787</b> |
| Использование Excel.....                                                 | 789        |
| Подключение к службам Analysis Services .....                            | 790        |
| Создание PivotTable .....                                                | 796        |
| Изучение данных PivotTable .....                                         | 798        |
| Системы показателей.....                                                 | 800        |
| Создание и настройка диаграмм.....                                       | 804        |
| Запросы к кубам внутри формул .....                                      | 807        |
| Функция <i>Convert To Formulas</i> .....                                 | 809        |
| Excel предоставляет технологию IntelliSense .....                        | 809        |
| Форматирование таблиц.....                                               | 812        |
| Фильтрация при помощи именованных ячеек.....                             | 813        |
| Инструменты Visual Studio Tools for Office и надстройки Excel .....      | 814        |
| Средства Excel Services .....                                            | 815        |
| Разработка OLAP при помощи .NET.....                                     | 822        |
| MDX и интерфейсы прикладного программирования Analysis Services.....     | 823        |
| Переходим к MDX .....                                                    | 824        |
| Management Studio как клиент MDX .....                                   | 824        |
| Богатая поддержка MDX.....                                               | 827        |
| Помимо синтаксиса.....                                                   | 833        |
| KPI, вычисляемые элементы и именованные наборы .....                     | 839        |
| Разработка OLAP при помощи ADO MD.NET.....                               | 841        |
| Использование ваших навыков работы с ADO.NET.....                        | 842        |
| Выполнение запросов MDX.....                                             | 843        |
| Резюме.....                                                              | 844        |
| <b>ГЛАВА 18. РАСШИРЕНИЕ БИЗНЕС-АНАЛИТИКИ ПРИ ПОМОЩИ</b>                  |            |
| <b>ИНТЕЛЛЕКТУАЛЬНОГО АНАЛИЗА ДАННЫХ.....</b>                             | <b>845</b> |
| Зачем делать интеллектуальный анализ данных .....                        | 845        |
| Вспоминаем статистику.....                                               | 849        |
| Расширения интеллектуального анализа данных в SQL Server 2008 .....      | 850        |
| Начало построения.....                                                   | 851        |

|                                                                                                      |            |
|------------------------------------------------------------------------------------------------------|------------|
| Подготовка исходных данных .....                                                                     | 852        |
| Создание проекта Analysis Services .....                                                             | 854        |
| Data Mining Wizard и визуальный конструктор .....                                                    | 857        |
| Создание структуры интеллектуального анализа данных .....                                            | 860        |
| Создание модели интеллектуального анализа .....                                                      | 861        |
| Редактирование и добавление моделей интеллектуального анализа данных .....                           | 868        |
| Развертывание и обработка объектов интеллектуального анализа данных .....                            | 877        |
| Просмотр моделей интеллектуального анализа данных .....                                              | 879        |
| Проверка и сравнение моделей интеллектуального анализа данных .....                                  | 891        |
| Вложенные таблицы .....                                                                              | 894        |
| Использование расширений интеллектуального анализа данных .....                                      | 902        |
| Моделирование интеллектуального анализа данных при помощи DMX .....                                  | 903        |
| Прогнозы интеллектуального анализа данных при помощи DMX .....                                       | 917        |
| Шаблоны DMX .....                                                                                    | 928        |
| Применение интеллектуального анализа данных .....                                                    | 929        |
| Интеллектуальный анализ данных и программирование при помощи API .....                               | 930        |
| Использование элементов управления средствами просмотра содержимого<br>моделей в Windows Forms ..... | 931        |
| Выполнение прогнозирующих запросов при помощи ADO MD.NET .....                                       | 934        |
| Запросы содержимого модели .....                                                                     | 935        |
| ADO MD.NET и ASP.NET .....                                                                           | 935        |
| Использование веб-элементов управления для интеллектуального<br>анализа данных .....                 | 936        |
| Разработка управляемых хранимых процедур .....                                                       | 938        |
| XMLA и интеллектуальный анализ данных .....                                                          | 940        |
| Надстройки интеллектуального анализа данных для Excel 2007 .....                                     | 943        |
| Резюме .....                                                                                         | 945        |
| <b>ГЛАВА 19. СЛУЖБЫ ОТЧЕТНОСТИ REPORTING SERVICES .....</b>                                          | <b>947</b> |
| Использование конструктора отчетов .....                                                             | 948        |
| Создание простого отчета .....                                                                       | 951        |
| Применение форматирования отчетов .....                                                              | 957        |
| Форматирование области данных .....                                                                  | 959        |
| Добавление в отчет групп .....                                                                       | 961        |
| Работа с параметрами .....                                                                           | 963        |
| Написание кода пользовательского отчета .....                                                        | 970        |
| Создание отчета OLAP .....                                                                           | 974        |
| Создание отчета при помощи матричной области данных .....                                            | 981        |
| Объясняем тип <i>tablix</i> .....                                                                    | 986        |
| Добавление области данных Chart Data Region .....                                                    | 992        |
| Делаем отчет интерактивным .....                                                                     | 995        |
| Доставка отчетов .....                                                                               | 998        |
| Развертывание на сервер отчетности .....                                                             | 998        |
| Программный доступ к отчетам .....                                                                   | 1009       |
| Администрирование служб Reporting Services .....                                                     | 1020       |
| Использование Reporting Services Configuration Manager .....                                         | 1021       |
| Резюме .....                                                                                         | 1024       |

# Программирование Microsoft<sup>®</sup> SQL Server<sup>®</sup> 2008

Leonard Lobel

Andrew J. Brust

Stephen Forte

Моему партнеру Марку и детям — Адаму, Жаклин и Джошуа — за то, что вы остаетесь со мной на всех поворотах судьбы.

*Леонард Лобел*

Моей жене Лорин и сыновьям Шону и Майлзу. Спасибо за вашу любовь, вашу поддержку и ваше смирение с безрассудством.

*Эндрю Браст*

Кэтлин, спасибо за твою поддержку и за то, что ты заставляешь меня бегать марафоны, что гораздо труднее, чем писать книги и создавать бета-машины.

*Стивен Форте*

# Благодарности

Работа над этой книгой была самым полезным опытом моей профессиональной карьеры до настоящего момента, и я хочу поблагодарить многих людей, которые сделали это возможным.

Впервые я встретил Эндрю Браста около 10 лет тому назад, и в течение последних семи лет мы поддерживали тесные рабочие отношения. Я даже не могу сосчитать, сколько раз Эндрю давал мне возможность участвовать в проектах, писать и выступать — и вот теперь эта книга. Эндрю представил меня Стивену Форте в 2004 году, и после 30 лет работы в этой области я научился находить новое удовольствие в разработке программного обеспечения благодаря энтузиазму Стивена. Эндрю и Стивен значительно облегчили реализацию этого проекта, они создали прообраз этой книги — прекрасное издание Microsoft SQL Server 2005, которое стало стартовой точкой для этого нового издания 2008 года. Искл​ючительно волнующе и большая честь для меня, что они пригласили меня присоединиться к ним на этот раз и взять на себя роль ведущего автора нового издания. Спасибо вам обоим за то, что вы доверили мне эту ответственность, а также и за вашу усердную работу над этим изданием.

Мы никогда не смогли бы создать такую широкую по своему охвату книгу без дополнительной помощи других соавторов. Элси Пен, Поль Декоглиано, Марк Фроли и Джефф Болтон проделали выдающуюся работу, а Элси также переработала материал последнего издания. Сердечная благодарность также и Кенну Скрибнеру, который сделал исключительно тщательную техническую рецензию всей книги, и особенно — за помощь при обновлении важного материала. Я очень благодарен им всем за профессионализм и считаю большой привилегией возможность работать с каждым из них. Я хотел бы также поблагодарить Джей Рабин за постоянную поддержку и поощрение всего этого проекта.

Мне посчастливилось тесно сотрудничать с Microsoft Press и Waypoint Press. Превосходная редакторская работа, планирование проекта и общее руководство были весьма существенными при создании этой книги. Двойная благодарность Салли, которая никогда не отказывала мне в помощи (и в выходные тоже), в которой я очень нуждался как начинающий писатель. А помощь

многих людей из различных команд разработчиков компании Microsoft помогла справиться с описанием новых программных продуктов по мере их прохождения через этапы бета-версий. Поэтому спасибо Стиву Ласкеру за помощь с Compact and Sync Services, а также Роджеру Догерти и Айзеку Кунену за поддержку при описании функций "за пределами реляционности". В частности Роджер вдохновил создание нескольких демо-приложений (для FILESTREAM и геопространственных), которые имеются в этих главах. Джордж Согби также оказал большую помощь с геопространственными приложениями, он даже подготовил демо-приложение с совместно используемым рабочим столом для Нью-Йорка и Каира (и большое спасибо также Стивену Форте за введение).

Конечно, все это было бы совершенно нереально без любви и поддержки моей семьи. Почти все мое свободное время в течение последнего года было потрачено (в той или иной форме) на работу над этим проектом — исследования и написание нового материала, редактирование всей книги, координирование ее производства, что иногда превращало меня в отсутствующего партнера и отца. Я выражаю огромную благодарность моему замечательному партнеру Марку и моим почтительным детям — Адаму, Жаклин и Джош — за то, что они с этим справились. Спасибо, друзья. Я снова дома! И самое большое спасибо моей дорогой маме (благослови Господи ее душу) за то, что она всегда советовала мне писать "с выражением" (с самого первого класса).

*Леонард Лобел*

Писать книги тяжело, особенно для тех людей, кто в этом процессе героически поддерживает авторов. Писать второе издание также тяжело, особенно для тех людей, кто поддерживает авторов снова.

С учетом всего этого я хочу поблагодарить мою жену Лорин (которая вынесла этот проект, причем параллельно получила свою степень магистра и была изумительной матерью для двух наших мальчиков). Я благодарю также наших мальчиков: Майлза (который несмотря на свои четыре года от роду является ветераном обоих изданий этой книги) и Шона (которому в возрасте 18 месяцев пришлось столкнуться с тем, что его старший брат пережил первым). Все трое терпели мои непозволительные отлучки из их жизни. Каждому из них я благодарен за терпение и понимание.

Я хочу также поблагодарить всех сотрудников twenty-six New York (особенно Джей Рабин) за то, что они дали мне период спокойного времени (небывалой продолжительности) для завершения работы над этим изданием.

И, наконец, я хотел бы поблагодарить Леонарда Лобела за то, что он стоял у штурвала этого издания книги. Если бы он этого не сделал, у нас просто не было бы этого издания. Ленни склонен благодарить меня за предоставленную ему возможность. Однако он не понимает, что в очередной раз добившись успеха, он просто подчеркивает мой хороший вкус, проявившийся в том, что я его рекомендовал.

*Эндрю Брост*

Очень приятно видеть свое имя на обложке книги, но без помощи разных людей эта книга никогда не увидела бы свет. Я начну с Эндрю и Ленни, моих замечательных соавторов, с которыми легко работать, и которые очень терпеливы ко мне. Было очень приятно также работать с людьми из Microsoft Press, которые очень энергичны и проявляют большую гибкость.

Я не смог бы работать над этим проектом, если бы не имел поддержки со стороны Telerik, Triton Works и Dash Soft (это три компании, с которыми я очень тесно сотрудничаю). Я хотел бы особо поблагодарить руководителей этих компаний — за их понимание и поддержку в тот период, когда сроки сдачи книги стали поджимать. У нас было также очень много отличных рецензентов.

Я начал эту книгу в первый день занятий моего второго года обучения на MBA. Иногда мне приходилось принимать решение, писать ли мне об управлении на основе политик или о макроэкономических тенденциях в Китае и Индии. Судьбе было так угодно, что я пишу эти строки в свой последний день обучения. Окончание книги и обучения на MBA в одни и те же дни — это очень подходящие друг к другу события.

*Стивен Форте*

# Введение

Добро пожаловать, разработчик!

Эта книга, подобно самому Microsoft SQL Server 2008, основана на отличном предыдущем издании. Сервер SQL Server 2005 был в архитектурном смысле революционной модернизацией, по сравнению с более ранними версиями продукта, а издание этой книги по версии 2005 г. стало новым печатным источником, который дал всестороннее изложение модернизированной платформы. Это новое издание включает в себя тщательно обновленное изложение наиболее важных тем из предыдущего издания плюс описание всех новых функциональных возможностей SQL Server 2008. Как и в издании 2005 г., мы поставили себе цель создать самую лучшую книгу для тех разработчиков, которым необходимо программировать SQL Server 2008 всеми теми способами, которыми его можно запрограммировать.

Чтобы лучше понять наш подход, представим себе SQL Server 2008 в виде воскресной газеты. Воскресная газета состоит из множества разделов, каждый из которых пишется отдельно и обращается к своей аудитории. Разделы имеют кое-какое общее содержимое и некоторых общих читателей, но большинство людей не прочитывает всю газету (да это им и не нужно). В то же время газета считается единым изданием, и те, кто ее читает, считают себя ее читателями (а не читателями ее разделов). Аналогичным же образом SQL Server имеет в своем составе много частей: мало кто использует их все, и обычно их изучают постепенно (по мере возникновения бизнес-потребностей).

Наша книга отражает эту действительность и во многих отношениях повторяет структуру воскресной газеты. Во-первых, в ее создании участвовало много авторов, использовавших свой опыт по конкретным темам написанных ими глав. Во-вторых, контекст разных глав сильно отличается. Некоторые главы подробно описывают специфические темы. Другие главы охватывают более широкий диапазон материала (и на более общем уровне). Этот подход отличается от того, который мы использовали в издании 2005 г. Но именно этот подход мы сочли наиболее эффективным к моменту завершения того издания, и именно его мы используем в этом новом издании по SQL Server 2008. Мы обнаружили, что он облегчает восприятие этого огромного набора технологий и делает процесс обучения более модульным.

Однако поймите нас правильно, общее видение этой книги вполне цельное: это изучение программирования SQL Server 2008 и описание функциональных возможностей продукта. Прочитаете ли вы всю книгу или только некоторые ее главы, будете ли вы читать ее главы по порядку или нет, наша цель — обеспечить вас практической информацией и множеством полезных примеров, а также комбинировать изложение на общем уровне и подробное обсуждение в соответствии с нашим пониманием.

Точно так же, как воскресная газета не охватывает всего, что происходит в мире, эта книга не сможет научить вас всему, что есть в SQL Server. Например, мы не описываем такие функции высокой готовности и отказоустойчивости, как репликация, кластеризация или зеркалирование. Мы не обсуждаем планы запросов и оптимизацию, а также не изучаем SQL Server Profiler, SQL Trace или Database Engine Tuning Advisor. Некоторые описанные в издании 2005 г. функции существенных изменений в SQL Server 2008 не претерпели (такие как XML Web Services, Service Broker, Integration Services и перекрестная отладка). Эти темы тоже не рассматриваются, чтобы оставить больше места для новых функциональных возможностей SQL Server 2008. Вы можете скачать главы с описанием этих тем в издании 2005 г. с Web-сайта данной книги.

При создании этой книги мы обнаружили, что описание всего, что есть в этом продукте, приведет к появлению книги огромного размера и неприемлемого стиля. Мы надеемся, что нам удалось достичь нужного баланса, обеспечив вполне усваиваемый объем информации (с достаточным для разработчиков количеством подробностей) и указатели на другие материалы, которые помогут вам стать опытным профессионалом по работе с SQL Server.

## Для кого написана эта книга

После того как мы установили, что описывается в этой книге, и чего в ней нет, мы хотели бы прояснить, кого она больше всего заинтересует, и кому лучше всего послужит. Коротко говоря, эта книга — для .NET-разработчиков и для разработчиков под SQL Server, которые работают с базами данных и доступом к данным на уровне бизнес-логики и на уровне приложений.

Возможно, это несколько эгоистичное представление о мире разработчиков, но мы считаем, что сюда относится большинство .NET-разработчиков; тем не менее, некоторые разработчики больше заинтересованы в программировании баз данных (и SQL Server), и именно к ним мы и обращаемся.

Мы предполагаем, что вы имеете базовые знания по .NET-программированию на клиенте и программированию на языке Transact-SQL (T-SQL) на сто-



роне сервера, однако вполне подойдет и опыт работы с SQL на любой платформе. Мы также предполагаем, что вы уверенно чувствуете себя в основах создания таблиц, представлений и хранимых процедур на сервере. Мы предполагаем, что (на клиентской стороне) вы знакомы с предыдущим поколением инструментов разработки SQL Server и .NET. Если вы уже работали с SQL Server Management Studio в SQL Server 2005, то вы будете чувствовать себя как дома и в версии 2008, которая была расширена для поддержки новых функций сервера (и теперь даже имеет поддержку технологии IntelliSense для T-SQL). Если вы по-прежнему используете SQL Server 2000 или более раннюю версию, то вы определенно оцените внесенные в SQL Server Management Studio улучшения (по сравнению с двумя предшествовавшими ей инструментами — Enterprise Manager и Query Analyzer). SQL Server Management Studio по существу представляет собой сплав этих двух инструментов, упакованный в современный пользовательский интерфейс (очень похожий на Microsoft Visual Studio), имеющий настраиваемые меню и панели инструментов, плавающие и закрепленные панели, решения и проекты. Основным инструментом .NET-разработчика является, конечно же, Visual Studio 2008, знакомство с любой версией которого вам также пригодится.

То есть наши предварительные требования являются весьма свободными. Например, если вы только начали заниматься SQL и .NET — все в порядке (если у вас есть желание осваивать новое при чтении). Большая часть наших примеров кодов не очень сложна. Однако наши пояснения подразумевают наличие у вас определенных базовых знаний, и в некоторых случаях вам может понадобиться что-то изучить самостоятельно.

### **ПРИМЕЧАНИЕ**

Из соображений единообразия .NET-код в этой книге написан на языке C#. (Единственное исключение из этого правила имеется в *главе 19* про службы Reporting Services, поскольку вывод в скрипты развертываний и выражений отчетов поддерживается только на языке Visual Basic .NET). Однако в предоставленных .NET-кодах нет ничего специфичного для языка C#. Коды примеров не очень сложные, и если вы имеете больше опыта работы с Visual Basic .NET (чем с C#), не должно возникнуть проблем с переводом кодов на Visual Basic .NET прямо во время чтения.

В дополнение к описанию реляционного ядра SQL Server, его новейшего поколения возможностей "за пределами реляционности" и вспомогательных служб эта книга также дает глубокое изложение функциональных возможностей бизнес-аналитики SQL Server (в том числе служб отчетности Reporting Services), оперативной аналитической обработки (OLAP), а также компонентов интеллектуального анализа данных служб Analysis Services. Несмотря на

то, что это не книга по бизнес-аналитике, мы твердо уверены в том, что все эти функциональные возможности должен понимать каждый разработчик баз данных. Бизнес-аналитика — это один из краеугольных камней SQL Server 2008, поэтому пришло время разработчикам "переселиться" в мир бизнес-аналитики.

Понимая, что эти технологии (особенно OLAP и интеллектуальный анализ данных) будут неизведанной территорией для многих читателей, мы не предполагаем наличия у вас знаний о них. Любой читатель, который соответствует уже обсуждавшимся предварительным требованиям, должен чувствовать себя вполне комфортно при чтении описания этих функций бизнес-аналитики и, скорее всего, будет полон желания начать работать с ними по завершении изучения соответствующих глав.

## Как организована эта книга

Эта книга разбита на четыре части. Каждая часть посвящена определенной теме SQL Server (если так можно выразиться).

*Часть I* начинается с обзора, который кратко описывает разбивку на главы всех четырех частей книги. Затем мы погружаемся в описание основных технологий SQL Server. Мы изучаем множество сделанных в языке Transact-SQL (T-SQL) улучшений как в SQL Server 2005, так и в SQL Server 2008, причем именно в таком порядке, а также представляем возможности интеграции с .NET Common Language Runtime (CLR), которые затрагивают обсуждения типов данных и программирования на серверной стороне. Вы узнаете, как программно администрировать сервер при помощи Server Management Objects (SMO) и как использовать новую административную инфраструктуру под названием Policy-Based Management (PBM) в SQL Server 2008. Затем мы займемся безопасностью. После краткого описания основных концепций безопасности SQL Server мы покажем, как шифровать ваши данные и при передаче (по сети), и при нахождении их в покое (на диске). Мы также обучим вас самым последним функциям безопасности SQL Server 2008 (в том числе Transparent Data Encryption (TDE) и SQL Server Audit), которые будут исключительно полезны в сегодняшнем мире для выполнения требований регулирующих органов.

*Часть II* посвящается теме "за пределами реляционности", которая описывает работу с полуструктурированными и неструктурированными данными. Это концепция, которая расширяет наше традиционное представление о реляционных базах данных, заставляя нас думать в терминах всех разнообраз-

ных типов данных, которыми SQL Server может управлять, манипулировать и запрашивать. Мы начинаем с главы о поддержке XML (которая появилась в SQL Server 2005) и даем подробное описание, в которое включены последние улучшения XML (сделанные в SQL Server 2008). Остальные главы *части II* описывают нереляционные функции, которые являются совершенно новыми в SQL Server 2008. Сюда входят: иерархические таблицы, потоковый вывод файлов, геопространственные возможности. Эти функции были спроектированы для обогащения реляционного ядра беспрецедентными интеллектуальными возможностями и удобствами программирования на уровне базы данных.

В *части III* мы отходим от сервера и обсуждаем концепции, которые относятся к разработке программного обеспечения баз данных (как на промежуточном уровне, так и на уровне приложений). Сюда входят: доступ к данным при помощи "традиционной" технологии ADO.NET, интегрированные в язык запросы LINQ, инфраструктура ADO.NET Entity Framework, а также последние новинки — ADO.NET Data Services и SQL Server Data Services. После того как вы успешно получили доступ к вашим данным, вам нужно доставить эти данные вашим пользователям, а для этого нужна привязка данных. Мы займемся привязкой данных для приложений Microsoft Windows и Web-приложений ASP.NET, а также для новейших платформ пользовательского интерфейса (Windows Presentation Foundation (WPF) и Silverlight). Мы также опишем транзакции и прочие темы, такие как репликация слиянием, Sync Services for ADO.NET, а также разработка мобильных приложений баз данных при помощи SQL Server Compact 3.5.

*Часть IV* — это наш раздел бизнес-аналитики. В нем мы даем эффективное и практичное изложение служб Analysis Services и Reporting Services. Мы особенно гордимся этим разделом, поскольку не требуем от вас никаких знаний бизнес-аналитики или OLAP и, тем не менее, даем действительно глубокое изложение концепций бизнес-аналитики, ее функциональных возможностей и программирования. У нас есть глава, посвященная хранилищам данных. В ней вы увидите, как использовать новую функциональную возможность SQL Server 2008 под названием Change Data Capture (CDC) для облегчения инкрементального обновления больших хранилищ данных. Более того, мы описываем все новые функции бизнес-аналитики SQL Server 2008 (и в том числе новейшие надстройки интеллектуального анализа данных для Microsoft Office Excel 2007). Глава о службах Reporting Services была написана заново (из-за полностью переработанного и улучшенного визуального конструктора Report Designer); она учит вас многим способам программирования и управления службами Reporting Services.

Все эти четыре части книги дают вам полное описание множества ориентированных на разработчиков функций SQL Server 2008, а также концептуальный материал для их понимания. Мы не описываем весь SQL Server 2008, но вооружим вас значительным количеством базовых знаний и дадим ссылки для дальнейшего изучения продукта. Там, где это необходимо, мы отсылаем вас к SQL Server Books Online, которая доступна через меню **Start | Programs | Microsoft SQL Server 2008 | Documentation And Tutorials**.

## Примеры кодов и Web-сайт этой книги

Все обсуждаемые в этой книге примеры кодов можно скачать с Web-сайта данной книги по адресу **<http://www.microsoft.com/mspress/companion/9780735625990/>**.

### **ВАЖНО**

Данная книга и ее примеры были написаны по выпуску Release Candidate (RC0) версии SQL Server 2008 Developer (выпущен в июне 2008 г.). Если нами будут обнаружены какие-то несовместимости с окончательной версией продукта либо с выпущенными впоследствии пакетами обновлений, мы обновим коды примеров и разместим сообщения об ошибках на Web-сайте этой книги по адресу: **<http://www.microsoft.com/mspress/companion/9780735625990/>**. Пожалуйста, следите за обновлениями кодов и сообщениями об ошибках на этом сайте.

В дополнение к примерам кодов, на Web-сайте этой книги также имеется несколько глав этой книги издания 2005 г., которые не обновлялись для издания 2008 г. В их число входят: главы по XML Web Services и Service Broker (это функции, которые после их внедрения в SQL Server 2005 широко не использовались, но поддержка которых в SQL Server 2008 продолжается). На этом же сайте имеются главы по: SQL Server Management Studio (это основной графический инструмент, который вы будете использовать для большей части вашей разработки баз данных), версии SQL Server 2005 Express, службам Integration Services и отладке. После описания всех новых возможностей SQL Server 2008 у нас просто не осталось места на эти темы (которые в SQL Server 2008 существенно не изменились). И хотя мы даем описание самых последних технологий привязки данных, способы программирования ADO.NET описаны в издании 2005 г. (и также опубликованы на Web-сайте). Глава этой книги по разработке приложений OLAP была переработана и дополнена описанием Excel 2007, но версия 2005 г. также имеется на сайте (для

тех разработчиков, которые по-прежнему работают с кубами OLAP при помощи Excel 2003).

Поскольку это книга для разработчиков, мы часто включаем в состав кодов примеров проекты Visual Studio (в дополнение к проектам SQL Server Management Studio, содержащим T-SQL или файлы скриптов Analysis Services). Внутри родительского каталога с файлами примеров есть дочерний каталог для каждой главы. Каталог каждой главы в свою очередь содержит один из следующих каталогов: SSMS и VS (или оба). Первый содержит решение SQL Server Management Studio (файл с расширением smssln), а второй — решение Visual Studio (файл sln). Некоторые главы могут иметь по несколько решений Visual Studio. После инсталляции файлов примеров дважды щелкните по файлу решения для того, чтобы открыть скрипты или код в соответствующих интегрированных средах.

Поскольку большая часть кодов имеет пояснения в тексте, вы можете создать их сами (а не открывать готовую версию из кодов примеров). Однако готовая версия все равно пригодится в том случае, если вы сделаете по пути какую-то ошибку или захотите быстро запустить данный код перед тем, как читать описывающий его фрагмент.

Некоторые из проектов SQL Server Management Studio содержат встроенные подключения, которые настроены на локальный экземпляр SQL Server. Если на вашем локальном компьютере инсталлирован SQL Server 2008 (как экземпляр по умолчанию с интегрированной безопасностью Windows), а также база данных AdventureWorks2008, то большинство кодов примеров должно работать без всякой их модификации. Если это не так, вам нужно будет модифицировать название сервера, название экземпляра или учетные данные пользователя. Вам также понадобится инсталлировать AdventureWorks2008.

Некоторые главы основаны на популярных базах данных SQL Server. Сюда входят транзакционные базы данных Northwind и только что упомянутая AdventureWorks2008, база данных хранилища AdventureWorksDW2008, база данных Adventure Works DW 2008 служб Analysis Services. Ни в одном из наших примеров не используется база данных pubs, которая появилась задолго до SQL Server 2000.

## Использование базы данных *Northwind*

Вы можете использовать ту версию Northwind, которая поставлялась в составе SQL Server 2000 (если она у вас есть), и подключить ее к серверу SQL Server 2008. Компания Microsoft опубликовала также файл инсталлятора

Windows Installer (файл msi) базы данных Northwind на сервер (в комплект входит даже старая база pubs). Инсталлятор содержит основной файл базы данных и файл журнала, который может быть подключен напрямую, а также скрипты T-SQL. На момент выхода из печати этой книги страница скачивания инсталлятора находилась по адресу: <http://www.microsoft.com/downloads/details.aspx?FamilyID=06616212-0356-46a0-8da2-eebc53a68034&DisplayLang=en>. Ссылка на данную страницу имеется в кодах примеров этой главы. Если ссылка не работает, сделайте поиск по строке "Northwind and pubs Sample Databases for SQL Server 2000".

## Использование баз данных *AdventureWorks2008*

Начиная с версии SQL Server 2005, компания Microsoft предоставляет семейство баз данных AdventureWorks, которые были обновлены для SQL Server 2008 и не будут работать с версией SQL Server 2005. (Однако на момент написания этой книги доступны также и более старые версии баз данных AdventureWorks для SQL Server 2005).

На момент печати этой книги адрес для скачивания всех баз AdventureWorks2008 был таков: <http://www.codeplex.com/MSFTDBProdSamples>. На этой странице щелкните по вкладке **Releases** для выбора любой базы, чтобы скачать ее на ваш компьютер. Ссылка на эту страницу имеется на Web-сайте данной книги. Если эта ссылка не работает, попробуйте сделать поиск по строке "SQL Server 2008 product sample databases".

База данных оперативной обработки транзакций AdventureWorks2008 использует новую функцию FILESTREAM сервера SQL Server 2008, и поэтому необходимо, чтобы на том экземпляре, где инсталлирована база AdventureWorks2008, был включен FILESTREAM. Глава 8 посвящена описанию FILESTREAM, и показано, как включить FILESTREAM для поддержки базы AdventureWorks2008.

### **ВАЖНО**

Примеры этой книги выполнены на основе 32-битных версий баз AdventureWorks2008. Если вы используете 64-битные версии этих баз, то некоторые результаты ваших запросов могут слегка отличаться от показанных в примерах этой книги.

## Системные требования

Для того чтобы следить за нашим изложением и успешно выполнять коды примеров, мы рекомендуем вам установить версию Developer сервера SQL Server 2008, которую многие разработчики получили через подписку Microsoft's MSDN Premium. Вам понадобится также Visual Studio 2008; мы рекомендуем использовать версию Professional или одну из версий Team (они также доступны через соответствующие версии подписки MSDN Premium).

### **ВАЖНО**

Для того чтобы охватить самый широкий спектр функциональных возможностей, данная книга основана на версии Developer сервера SQL Server 2008. Версия Developer имеет тот же самый набор функциональных возможностей, что и версия Enterprise (однако условия лицензирования версии Developer не позволяют использовать ее в промышленных условиях). Обе версии — это профессиональные платформы. Мы считаем, что в интересах разработчиков необходимо описать весь диапазон функций SQL Server 2008, включая и те, которые имеются только в версиях Enterprise и Developer.

Большая часть описанных в этой книге функций программирования имеется в любой версии SQL Server 2008. Одним примечательным исключением является отсутствие поддержки служб Analysis Services в версиях Workgroup, Web и Express. Пользователям других промышленных версий (не Enterprise) следует посмотреть страницу сравнения функциональных возможностей версий SQL Server 2008 по адресу: <http://msdn.microsoft.com/en-us/library/cc645993.aspx>, чтобы понять, какие из описанных в этой книге функций будут доступны им в рабочей среде.

Для работы этих версий SQL Server и Visual Studio (а, следовательно, и примеров этой книги) вам понадобится следующее 32-битное аппаратное и программное обеспечение. (64-битное обеспечение мы здесь не приводим, но оно очень похоже):

- ☐ совместимый с Pentium III 600 МГц процессор или быстрее (минимум — 1 ГГц, однако рекомендуется 2 ГГц и более);
- ☐ Microsoft Windows 2000 Server Service Pack (SP) 4 или более поздний; Windows 2000 Professional Edition SP4 или более поздняя; Windows XP SP2 или более поздняя; Windows Server 2003 (любая версия) SP1 или более поздний; Windows Small Business Server 2003 SP1 или более поздний; Windows Server 2008 (любая версия);
- ☐ для SQL Server 2008 — по меньшей мере 512 Мбайт оперативной памяти (рекомендуется 1 Гбайт или более);

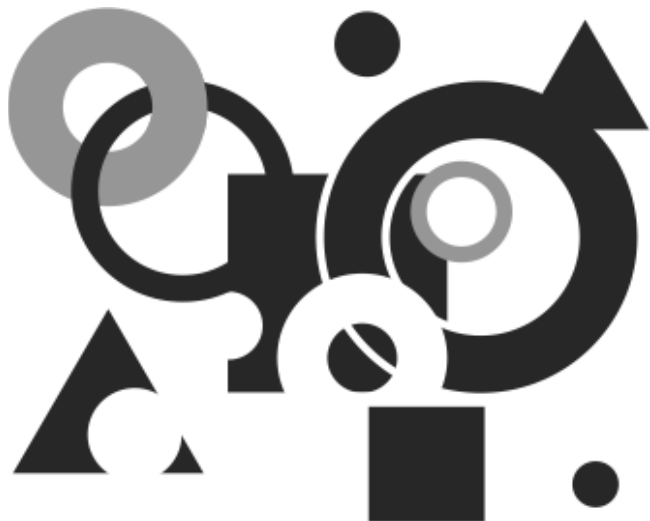
- ❑ для Visual Studio 2008 — 192 Мбайт (рекомендуется 256 Мбайт);
- ❑ для инсталляции SQL Server 2008 — примерно 1460 Мбайт дискового пространства. Примерно 200 Мбайт дополнительного дискового пространства для документации SQL Server Books Online;
- ❑ для Visual Studio 2008 — максимум 20 Гбайт дискового пространства. Сюда входит объем полной документации MSDN;
- ❑ интернет-подключение для скачивания примеров кодов с сайта этой книги. Некоторым примерам нужно интернет-подключение для их работы;
- ❑ рекомендуется наличие дисководов CD-ROM или DVD-ROM;
- ❑ рекомендуется видеоадаптер и монитор с разрешением Super VGA (1024×768) или лучший;
- ❑ рекомендуется наличие манипулятора Microsoft Mouse или совместимого с ним;
- ❑ Microsoft Internet Explorer 6.0 SP1 или более поздний. Рекомендуется наличие Microsoft Internet Explorer 7.0;
- ❑ для служб отчетности Reporting Services — сервер Microsoft Internet Information Services (IIS) 6.0 или более поздний, а также ASP.NET 2.0 или более поздняя версия.

## Поддержка этой книги

Мы приложили все усилия для того, чтобы обеспечить точность этой книги и сопровождающего ее содержимого. По мере сбора исправлений и изменений они будут добавлены в статью Microsoft Knowledge Base.

Издательство Microsoft Press предоставляет поддержку своих книг и сопровождающего им содержимого на сайте <http://www.microsoft.com/learning/support/books/>.

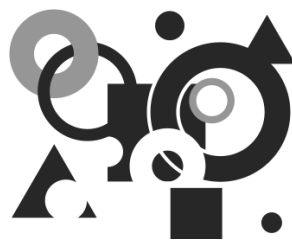




# **ЧАСТЬ I**

## **ФУНДАМЕНТАЛЬНЫЕ ОСНОВЫ**

# ГЛАВА 1



## Обзор

Эта книга про Microsoft SQL Server 2008 написана специально для разработчика.

Новейшая версия SQL Server компании Microsoft улучшена по сравнению со своими предшественницами (SQL Server 2005 и более ранними версиями) во всех ключевых областях, чего и следует ожидать от любой новой версии. В продукте имеется множество новых функциональных возможностей, которые дают существенные преимущества. Эти улучшения и новые функции безусловно укрепляют положение SQL Server как платформы базы данных промышленного уровня, способной справиться с самыми серьезными рабочими нагрузками. В этой книге центром нашего внимания является программирование для разработки баз данных — это та область, в которой SQL Server 2008 продолжает мощно развиваться и является самым программируемым сервером SQL по состоянию на сегодняшний день.

Возможности кластеризации, зеркалирования и способность добавлять дополнительную оперативную память и процессоры прямо "на ходу" (и это еще не все) безусловно важны в любой серьезной системе уровня предприятия. Но повторимся еще раз: это книга о базе данных, которая предназначена для разработчиков, а не для администраторов (несмотря на то, что у нас есть целая глава, посвященная администрированию SQL Server программным путем). Надежность, высокая готовность и масштабируемость — все это весьма впечатляющие и важные функциональные возможности, которые сами по себе делают SQL Server чрезвычайно надежной платформой, но они не очень программируемы, и поэтому в данной книге не рассматриваются.

Если же вы разработчик, то добро пожаловать! Эта книга именно для вас. Программируете ли вы для SQL Server 2008 на уровне базы данных или

уровнем выше (при помощи .NET), данная книга покажет вам верную дорогу. В этих главах вы найдете подробное изложение самых новых и наиболее важных программируемых функциональных возможностей SQL Server. Вместе с вами мы исследуем массу способов, которыми можно программировать SQL Server 2008, что позволит вам быстро разрабатывать богатые приложения для работы с базами данных и получать удовольствие от этого процесса.

## Насколько он значителен

С каждой новой версией SQL Server мы задаем одни и те же вопросы. Существенны ли изменения? Сколько функциональных возможностей добавлено, что это за возможности, и насколько они соответствуют моим потребностям? Какие ранее существовавшие ограничения были сняты? Есть ли радикальные изменения в самом ядре продукта? Что более не поддерживается? После некоторых исследований подобного рода мы обычно приходим к последнему вопросу: следует ли делать обновление версии?

Давайте внесем здесь ясность. SQL Server 2008 не является переломным явлением, как SQL Server 2005. В то время SQL Server 2005 действительно превратил данный продукт из ядра реляционной базы данных (которая уже была вполне зрелым продуктом еще в версии SQL Server 2000 за шесть лет до того) в нечто большее. Новые улучшения, в частности, службы бизнес-аналитики для извлечения, преобразования и загрузки (extraction, transformation, loading, ETL), а также службы отчетности и анализа, стали появляться, начиная с 1999 г., и продолжали стабильно выходить после появления SQL Server 2000. Эти возможности интегрировались в продукт бессистемно, в виде заплат и надстроек, мастеров и консолей управления. В результате с течением времени получилась система управления реляционной базой данных с коллекцией дополнительных функциональных возможностей, поддерживаемых несколько несогласованным набором инструментов.

Компания Microsoft изменила такое положение вещей с выходом SQL Server 2005, сделав полный структурный пересмотр платформы, буквально революционизировав этот продукт. Движущей силой такого создания радикально нового облика стали кардинальные изменения в отрасли программного обеспечения: распределенный характер растущего взаимодействия предприятий (business-to-business, B2B) через Интернет и растущий рынок бизнес-аналитики, в том числе оперативной аналитической обработки (online analytical processing, OLAP), интеллектуального анализа данных, а также от-

четности. Начиная с SQL Server 2005, ядро реляционной базы данных не является более центром внимания. Оно скорее располагается в общем ряду основных служб, которые (вместе взятые) и представляют собой данный продукт. В результате получился богатый и более согласованный набор функциональных возможностей и служб, которые встроены в платформу (а не притянуты к ней).

SQL Server 2008 является продуктом эволюции этой модели (основываясь на архитектуре SQL Server 2005 и расширяя ее). Таким образом, обновление с версии SQL Server 2000 (или более ранней) до версии 2005 или 2008 является настоящим скачком в смысле архитектуры платформы, а обновление с версии 2005 до версии 2008 является в этом смысле неким эволюционным изменением. В таком сравнении версия 2008 не очень значительна.

В другой стороны, SQL Server 2008 можно совершенно определенно рассматривать как существенно новую версию продукта. Компания Microsoft добавила в SQL Server 2008 новые беспрецедентные функциональные возможности: интеллектуальность, удобство и мощь программирования на уровне базы данных. SQL Server 2008 основан на интеграции с общезыковой исполняющей средой (Common Language Runtime, CLR), которая была реализована в SQL Server 2005 для введения нового поколения неуправляемых типов данных, основанных на определяемых пользователем типах (user-defined types, UDT) из CLR, что позволяет использовать в базе данных иерархические и геопространственные возможности.

Был также значительно улучшен язык Transact-SQL (T-SQL). Сжатие резервных копий, сжатие данных и отслеживание изменений в данных (Change Data Capture, CDC) помогают нам справиться с быстро растущими хранилищами данных. Новые возможности обеспечения безопасности, такие как прозрачное шифрование данных (Transparent Data Encryption, TDE) и подсистема аудита SQL Server Audit, облегчают выполнение все возрастающих требований нормативных документов. Мы также имеем потоковую запись между базой данных и файловой системой для хранения больших двоичных объектов (binary large object, BLOB) и новую модель синхронизации для репликации с периодически подключенными клиентскими системами. Есть также новые функциональные возможности бизнес-аналитики для анализа и отчетности. Учтите и повышение надежности, готовности и масштабируемости — и вам, вероятно, придется признать, что SQL Server 2008 является существенно новой версией. Перед нами не стоит задача убедить вас в этом (или в обратном), мы хотим помочь вам понять и научиться программировать так, чтобы получить от Microsoft SQL Server 2008 максимум возможного.

**ПРИМЕЧАНИЕ**

Последние опросы продолжают указывать на популярность SQL Server 2000. Для облегчения процесса обновления таких инсталляций компания Microsoft поддерживает прямую миграцию с SQL Server 2000 на SQL Server 2008 (без необходимости промежуточного обновления до версии SQL Server 2005).

**ВАЖНО**

Службы уведомления Notification Services, которые имелись в составе SQL Server 2005 в виде отдельной надстройки, а затем были интегрированы в ядро SQL Server 2005, в версии SQL Server 2008 больше не поддерживаются, это единственный компонент SQL Server 2005, который был ликвидирован компанией Microsoft в SQL Server 2008.

В соответствии с заявлениями компании Microsoft, поддержка служб Notification Services будет осуществляться, как составная часть поддержки продукта SQL Server 2005 в течение всего его жизненного цикла. Однако в дальнейшем поддержка уведомлений будет включена в службы отчетности SQL Server Reporting Services. Некоторые из потребностей в уведомлениях уже реализуются функциональными возможностями служб Reporting Services (такие как стандартные и управляемые данными подписки, которые обсуждаются в *главе 19*). В последующих версиях можно ожидать появления дополнительных возможностей уведомления.

## Книга для разработчиков

Если вы добрались до этого места, то мы уже можем считать установленным тот факт, что вы являетесь разработчиком (или, по крайней мере, это одна из ваших специальностей). При адаптации мы сделали относительно вас еще несколько других предположений.

Во-первых, мы ожидаем от вас хороших знаний концепций реляционных баз данных (это может быть опыт работы как с SQL Server, так и с платформами не от компании Microsoft). Вы должны знать о таблицах, представлениях, связях по первичному и внешнему ключам, хранимым процедурам, функциях и триггерах. Знание этих основ нами подразумевается, и в данной книге они не рассматриваются. Точно так же мы не обсуждаем проектирование реляционных баз данных (правила нормализации данных, стратегически важные схемы индексации и прочие фундаментальные основы реляционных баз данных). Мы также подразумеваем, что вы знакомы, по крайней мере, с основами синтаксиса инструкций SQL (либо T-SQL в SQL Server, либо диалектов SQL других платформ).

После того как мы установили эту точку отсчета, чего же мы ждем от вас (в смысле того, чему бы вы хотели научиться)? Это будет все то, что разработчик может пожелать узнать о сервере Microsoft SQL Server 2008! Начнем мы с самых мощных расширений T-SQL и ядра реляционной базы данных. Мы также ожидаем, что вы желаете получить знания и в областях за пределами реляционных технологий, таких как неструктурированные и слабоструктурированные данные, клиентский доступ к данным, безопасность, а также бизнес-аналитика с ее хранилищами данных, анализом и отчетностью.

Как мы уже начали объяснять, SQL Server 2005 был действительно новаторской версией, которая пересмотрела данный продукт и включила в него длинный список функциональных возможностей и служб (в то время как более ранние версии были традиционно специализированы в качестве ядра реляционной базы данных). Учитывая это обстоятельство, данная книга содержит обновленное изложение улучшений версии SQL Server 2005, а также самых новейших функций SQL Server 2008. Это делает ее идеальным источником информации при обновлении до SQL Server 2008 как с предыдущих версий SQL Server, так и с других платформ реляционных баз данных.

## **Книга, которая написана разработчиками**

Мы, авторы и соавторы этой книги, прежде всего являемся разработчиками (как и вы). Все мы сделали карьеру при помощи написания кодов, которые приводят в действие наши приложения (главным образом — приложения баз данных). Мы посвятили себя созданию решений, которые работают с данными и выдают эти данные в самом убедительном виде для бизнеса, партнеров и клиентов. И мы любим SQL Server!

Подобно самому SQL Server 2008, эта книга основана на предыдущем издании, написанном для SQL Server 2005, и является совокупным результатом многих лет труда, вложенного в нее множеством авторов. Нам повезло, что главу по безопасности написал менеджер продукта из группы разработки Microsoft SQL Server. Наши главы по интеллектуальному анализу данных, хранилищам данных, отчетности, репликации и транзакциям также были написаны специалистами по этим вопросам.

Наш подход — дополнить документацию по продукту путем предоставления ориентированного на разработчика исследования новых и улучшенных функциональных возможностей и служб SQL Server 2008. Поэтому в данной книге имеется изобилие примеров кодов, и в том числе — библиотека примеров проектов Visual Studio и SQL Server Management Studio, которую вы мо-

жете скачать с сайта этой книги. *(Подробности по скачиванию и использованию примеров кодов см. во введении).*

## Книга, которая укажет вам путь

Эта книга была тщательно спланирована таким образом, чтобы представить вам огромное количество новых ориентированных на разработчиков функций SQL Server 2008 в максимально логически последовательной манере. Для этого главы были организованы в четыре части, обзор которых дается в этой главе.

## Фундаментальные технологии

В *части 1* мы сосредоточимся на основных технологиях SQL Server. Сюда входят улучшения языка T-SQL, расширенные возможности программирования в кодах SQL CLR (на таких .NET-языках, как Microsoft Visual Basic .NET и C#), управление сервером, безопасность.

В *главе 2* мы исследуем существенные улучшения, внесенные в язык Transact-SQL (T-SQL), который по-прежнему остается лучшим инструментом программирования для использования как новых, так и старых функций SQL Server. Мы начинаем с улучшений SQL Server 2005, описываем преимущества и недостатки написания рекурсивных запросов с обобщенными табличными выражениями (common table expressions, CTE) и изучаем скалярные функции, которые обеспечивают основу ранжирования. Затем мы переходим к изучению обработки исключительных состояний и триггеров языка определения данных (data definition language, DDL).

Затем глава погружается в мощные расширения языка T-SQL, добавленные в SQL Server 2008. Возвращающие табличные значения параметры (table-valued parameters, TVP) позволяют передавать целые результирующие наборы между хранимыми процедурами и функциями на сервере, а также между клиентом и сервером при помощи Microsoft ADO.NET. Затем исследуются новые функции даты и времени (в том числе отдельные типы данных для даты и времени), осведомленность о временной зоне, а также улучшения диапазонов дат и времени, хранения и точности. Затем мы покажем множество способов использования MERGE, новой инструкции языка манипулирования данными (data manipulation language, DML), которая инкапсулирует все отдельные операции, которые обычно выполняются при слиянии. Там же вы узнаете о технологии INSERT OVER DML, которая повышает наши возможности захватывать изме-

нения данных из выражения `OUTPUT` любой инструкции `DML`. И, наконец, мы рассмотрим `GROUPING SETS`, расширение традиционного выражения `GROUP BY` выполнения срезов данных в запросах агрегирования.

В *главе 3* дано обстоятельное изложение программирования в среде `SQL CLR`, которое позволяет вам выполнять скомпилированный `.NET`-код на сервере `SQL Server`, а также дано руководство, когда и где это программирование использовать. Мы выходим за рамки обычных хранимых процедур, триггеров и функций и объясняем (а также демонстрируем) создание типов и агрегатов `CLR` — таких объектов, которые совершенно невозможно создать на языке `T-SQL`. Мы описываем различные способы создания объектов `SQL CLR` в `Microsoft Visual Studio 2008` и управление их развертыванием (из `Visual Studio`, из скриптов `T-SQL` в `SQL Server Management Studio` и вообще отовсюду).

В *главе 4* мы показываем вам, как выполнять административные задачи программным способом при помощи объектов `Server Management Objects (SMO)`, появившихся в `SQL Server 2005`. Вы научитесь использовать `SMO` для выполнения резервного копирования и восстановления баз данных, выполнения проверок непротиворечивости баз данных (`Database Consistency Check, DBCC`) и многого другого (и все это будет делаться из вашего кода). Вы узнаете также о функции управления на основе политик (`Policy-Based Management, PBM`) — новой функции `SQL Server 2008`, которая помогает разработчикам сотрудничать с администраторами с целью обеспечения соблюдения одинаковых конфигураций (настраиваемых политиками) на сервере разработки и на производственном сервере.

В *главе 5* подробно обсуждается безопасность сервера `SQL Server` и изучаются варианты обеспечения защиты данных от любопытных глаз. Мы начинаем с основных концепций безопасности, касающихся входа в систему, ролей, аутентификации и авторизации пользователей. Затем вы переходите к изучению поддержки шифрования при помощи ключей (введенной в `SQL Server 2005`), которая защищает ваши данные, как при передаче, так и тогда, когда они находятся в покое. Затем изучаются добавленные в `SQL Server 2008` новые функции безопасности, в том числе прозрачное шифрование данных (`Transparent Data Encryption, TDE`) и аудит `SQL Server Audit`. `TDE` позволяет вам шифровать в фоновом режиме целые базы данных (без написания специального кода). При помощи аудита `SQL Server Audit` практически любое предпринятое пользователем действие может быть записано либо в файловую систему, либо в журнал событий `Windows`. Глава завершается чрезвычайно важными указаниями по выполнению лучших практик во избежание стандартных проблем с безопасностью.



## За пределами реляционности

С выпуском версии SQL Server 2008 компания Microsoft продолжает изменять наше восприятие использования нереляционных данных в мире реляционных баз данных. Одной из ключевых идей версии SQL Server 2008 является "не только реляционность", и к тому времени, как вы закончите *часть II*, вы поймете и оцените те значительные достижения, которых компания Microsoft добилась в этой области.

SQL Server 2005 охватил слабоструктурированные данные посредством введения типа данных `xml` (и сопутствовавшей ему богатой поддержки XML). Это новшество было огромным улучшением по сравнению с использованием обычных столбцов типа `varchar` или `text` для хранения строк XML (что было обычным делом в более ранних версиях SQL Server), и буквально революционизировало хранение XML в реляционной базе данных. Оно позволило разработку приложений баз данных, которые работают с иерархическим XML сами — в среде системы управления реляционной базой данных, что раньше (при использовании обычных строковых столбцов) было невозможно. В *главе 6* мы описываем тип данных `xml`, расширения XQuery для T-SQL, коллекции XML Schema Definition (XSD) на серверной стороне, улучшения XML в версии SQL Server 2008, а также множество других функций XML, имеющихся в SQL Server.

Однако встроенная поддержка XML была только первым шагом того начинания, которое компания Microsoft продолжила еще более энергично в версии SQL Server 2008. Были добавлены новые функции для работы с более широким разнообразием нереляционных типов, в том числе неструктурированных данных и пространственных типов. В заключение *части II* мы исследуем эти новые функциональные возможности и показываем, как вы можете использовать их для создания современных приложений, которые требуют наличия унифицированных сервисов хранения в базе данных структурированных, слабоструктурированных и неструктурированных данных и манипулирования ими.

Начиная с версии SQL Server 2008, язык XML уже не является единственным вариантом работы с иерархическими данными в базе данных. В *главе 7* мы исследуем новый тип данных `hierarchyid`, который позволяет наложить иерархическую структуру на любую реляционную таблицу. Этот тип данных реализован как "системный CLR", просто определяемый пользователем (user-defined type, UDT) тип SQL CLR, аналогичный тем, которые мы научились создавать в *главе 3*, за исключением того, что для использования `hierarchyid` вам не нужно включать SQL CLR на сервере. В хранящемся в типе данных `hierarchyid` значении закодирован полный путь к данному узлу

древовидной структуры (начиная от корня и до конкретного порядкового номера среди остальных узлов-потомков, имеющих одного и того же родителя). При помощи предоставляемых этим новым типом методов вы можете эффективно строить, запрашивать и манипулировать данными древовидной структуры в ваших реляционных таблицах.

В *главе 8* вы узнаете все о новой функции FILESTREAM в SQL Server 2008 хранения неструктурированных BLOB-данных в базе данных — это становится все более актуальным с учетом взрывообразного увеличения объемов данных, характерного для нашего времени. Раньше нам пришлось бы выбирать между хранением BLOB-данных в базе данных при помощи столбцов `varbinary(max)` или вне базы данных в виде неструктурированных двоичных потоков (обычно в виде файлов в файловой системе). Оба подхода имеют существенные недостатки, и именно здесь нужен FILESTREAM, высокоэффективный уровень абстракции который позволяет логически обращаться с BLOB-данными как с неотъемлемой частью базы данных, а в это время SQL Server 2008 физически хранит BLOB-данные в файловой системе NTFS отдельно от базы данных. Он даже интегрирует для ваших BLOB-данных транзакции базы данных и транзакции файловой системы NTFS. В этой же главе вы научитесь использовать новую мощную функцию в ваших .NET-приложениях путем создания приложений Windows, Web-приложений и приложений Windows Presentation Foundation (WPF).

В *главе 9* изучаются новые типы данных `geometry` и `geography`. Эти новые системные типы CLR предоставляют в SQL Server 2008 геопространственные возможности на уровне базы данных. Типы данных `geometry` и `geography` поддерживают соответственно пространственные вычисления по двум геопространственным моделям поверхностей: плоской и геодезической (круглая планета). При помощи представленных координатами географических данных (которые хранятся в этих типах данных) вы можете легко определить пересечения и вычислить длину, площадь и расстояния. Данная глава сначала кратко описывает основы, а затем дает разборы, в которых мы строим несколько геопространственных приложений баз данных (в том числе и такое, которое интегрируется с Microsoft Virtual Earth). И хотя это огромная тема, которая могла бы занять целую отдельную книгу, в нашей главе описаны основы, которые понадобятся вам для работы с геопространственными данными.

## Улучшаем досягаемость

После того как мы дали так много материала о том, что вы можете делать на сервере и в вашей базе данных, мы переходим к *части III* данной книги, опи-

сывающей технологии создания приложений, работающих с базами данных и улучшающих их досягаемость.

Мы начинаем с *главы 10*, в которой сначала описывается Microsoft ADO.NET и функции доступа к данным Microsoft Visual Studio (в том числе типизированные объекты DataSet). После этого описания ADO.NET мы даем обзор новых методик доступа к данным, в том числе концепций и синтаксиса интегрированных в язык запросов (language-integrated query, LINQ). Мы рассматриваем технологии LINQ To SQL и ADO.NET Entity Framework Object Relational Mapping (ORM), ADO.NET Data Services и SQL Server Data Services.

В *главе 11* мы описываем привязку данных. Мы начинаем с Windows Forms (используется привязка перетаскиванием в Visual Studio). Затем мы рассматриваем привязку данных ASP.NET (при помощи визуальных конструкторов, кода и способов разметки). Затем мы переходим к использованию ASP.NET Asynchronous JavaScript и XML (AJAX) и представляем новый набор функций ASP.NET Dynamic Data. Заканчиваем мы привязкой данных в WPF и Silverlight 2.1 и показываем вам, чем они отличаются друг от друга и от других моделей привязки данных. Мы надеемся, что для любого типа приложения мы поможем вам без труда справиться с управлением данными.

Вне зависимости от того, как вы пишете и упаковываете свой код — будь это T-SQL или .NET-язык, имей он форму обычной хранимой процедуры, функции или Web-службы, будь он развернут на клиенте или сервере — вы должны обеспечить согласованность ваших данных для защиты их целостности. Ключом согласованности являются транзакции, и как прочими программируемыми функциями SQL Server, транзакциями можно управлять из нескольких мест. Если вы пишете код T-SQL или клиентский код при помощи поставщика ADO.NET SqlConnection или System.Transactions, то вам нужно знать различные поддерживаемые сервером SQL Server уровни изоляции транзакций, область действия ваших транзакций, а также лучшие практики по написанию транзакционного кода. В *главе 12* вы все это получите.

Мы не могли бы закончить изложение без описания репликации слиянием и синхронизации в среде распределенных баз данных. Мы начинаем *главу 13* разбором создания синхронизированного приложения клиент-сервер для работы с базой данных (при помощи просто SQL Server Merge Replication). Затем мы изучаем новую функцию Change Tracking, предназначенную для работы совместно с новыми службами Sync Services ADO.NET. Эти функциональные возможности расширяют приложения SQL Server Compact 3.5 мобильных устройств под управлением Windows. Эти устройства используются не только потребителями, но и удаленными сотрудниками различных организаций (причем как в автономном режиме, так и в режиме беспроводного

го подключения). Мобильные приложения часто переходят из подключенного в автономное состояние и обратно. Вы узнаете о том, как Merge Replication и Change Tracking с Sync Services для ADO.NET делают возможной реализацию мобильных клиентов и настольных приложений под Windows, которые работают в автономном режиме и синхронизируют свои данные при появлении подключения к серверу.

## Стратегии бизнес-аналитики

В *части IV* данной книги мы поможем вам поднять ваши знания по управлению базами данных и программированию на новый уровень стратегического анализа. Точнее говоря, мы научим вас, как создать настоящее хранилище данных, и как затем использовать его при помощи OLAP, интеллектуального анализа данных и служб отчетности.

В *главе 14* мы покажем вам основы хранилищ данных и объясним, как использовать несколько новых важных функциональных возможностей SQL Server 2008, которые поддерживают вашу бизнес-аналитику. Глава начинается с изложения некоторых важных основ, указаний по проектированию, а также практических советов по созданию хранилищ данных. Затем мы переходим к практическому описанию новых и улучшенных функциональных возможностей SQL Server 2008. Использование инструкции MERGE (которую мы описываем в *главе 2*), Change Data Capture (CDC), сжатие данных и резервных копий — и все это появилось в SQL Server 2008 впервые. CDC позволяет вам отследить изменение данных в таблицах без применения триггеров или модификации кода. Это, в свою очередь, облегчает и процессы ETL, которые обновляют большие хранилища данных инкрементальным образом. И поскольку хранилища данных продолжают становиться еще больше, чем раньше, то сжатие данных и резервных копий становится жизненно важным и чрезвычайно желательным. В этой главе рассматриваются также и другие расширенные функции хранилищ данных, такие как параллелизм секционированных таблиц, оптимизация запросов соединений типа "звезда", а также разреженные столбцы.

В остальных главах мы показываем вам, как использовать преобразованные репозитории данных в качестве основы для сложных баз данных аналитических служб SQL Server Analysis Services, которые поддерживают самые современные функции бизнес-аналитики. Многие книги по SQL Server не описывают аналитические службы по той причине, что это очень "специальная" тема, но мы с таким мнением согласиться не можем. Сама идея аналитических служб SQL Server 2008 состоит в массовой привлекательности и дос-

тупности бизнес-аналитики. Мы покажем вам, каким простым может быть программирование бизнес-аналитики, и как оно дополняет обычные реляционные базы данных и программирование для них.

В *главе 15* мы покажем вам основы создания кубов OLAP в Visual Studio. Мы покажем вам, как строить, развертывать и запрашивать кубы OLAP, которые поддерживают практический и детальный анализ ваших данных. Мы сделали эту главу очень короткой для того, чтобы дать некое "краткое введение" в бизнес-аналитику для вечно занятых разработчиков.

В *главе 16* мы берем простой куб, созданный в *главе 15*, и используем Visual Studio для реализации целого массива новых функциональных возможностей OLAP, предоставляемых вам аналитическими службами версии 2008 г. К концу этой главы ваш куб будет иметь несколько функциональных возможностей унифицированной модели измерений компании Microsoft. Эта глава длинная, но вы можете читать ее в своем темпе, практикуясь с новыми функциями сразу по прочтении каждого раздела.

В *главе 17* мы даем всестороннее изложение многих методик разработки приложений OLAP. Мы описываем создание пользовательских интерфейсов OLAP в Microsoft Office Excel 2007, использование PivotTables, диаграммы и новые формулы CUBE для ячеек. Затем мы описываем публикацию этих элементов на Web-панелях при помощи Excel Services (компонент Microsoft Office SharePoint Server).

Мы даем небольшое учебное пособие по запросам языка многомерных выражений (multidimensional expression language, MDX) и показываем вам, как выполнять запросы MDX из окна запросов в SQL Server Management Studio. Мы также покажем вам, как выполнять запросы MDX из ваших собственных приложений (посредством программирования на уровне интерфейса прикладного программирования ADOMD.NET). Мы описываем управление базами данных аналитических служб, как интерактивное посредством SQL Server Management Studio, так и программное при помощи объектов Analysis Management Objects (AMO). Мы представим вам стандарт XML for Analysis (XMLA), на котором построены как ADOMD.NET, так и AMO. Покажем, как создавать скрипты XMLA в Management Studio и программно манипулировать XMLA в .NET-коде. Мы также представим вам интеграцию аналитических служб с CLR, как создавать управляемые хранимые процедуры в .NET (при помощи ADOMD.NET на стороне сервера и AMO).

*Глава 18* целиком посвящена интеллектуальному анализу данных, она дает совершенно самостоятельное и полное изложение этой темы (включая новейшие функциональные возможности интеллектуального анализа данных, добавленные в SQL Server 2008). Мы начинаем с концептуального введения,

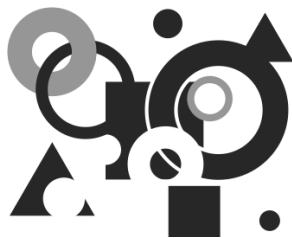
а затем даем учебное пособие по созданию, обучению, просмотру и развертыванию ваших структур интеллектуального анализа данных и моделей из Visual Studio. Затем мы переходим к SQL Server Management Studio и показываем вам интерактивное управление структурами анализа и моделями при помощи инструментов графического интерфейса (graphical user interface, GUI), программное — при помощи скриптов SQL Data Mining Extensions (DMX). Затем мы возвращаемся обратно к .NET-программированию и показываем, как встроить средства просмотра Data Mining Model Browsers аналитических служб в ваши приложения, как совместно использовать DMX, ADO.NET и привязку интеллектуального анализа данных (при помощи Windows Forms и ASP.NET), как использовать серверный ADOMD.NET для создания хранимых процедур DMX. Заканчиваем мы показом применения новых мощных надстроек интеллектуального анализа данных для Excel 2007.

В *главе 19* описаны службы отчетности Reporting Services, которые в SQL Server 2008 были значительно улучшены. Мы начинаем с учебного пособия по новому визуальному конструктору Report Designer в Visual Studio 2008, а затем показываем вам, как использовать службы отчетности для быстрого и простого создания сложных отчетов, как по реляционным базам данных, так и по базам OLAP. Вы увидите, как нужно создавать полномасштабные информационные системы отчетности. Мы покажем вам, как доставлять необходимые пользователям отчеты с гибкой компоновкой (посредством использования в ваших отчетах новой области tablix — гибрида table и matrix). Затем мы дадим вам обзор конфигурирования сервера отчетности и инструментов администрирования и научим вас развертывать, а также встраивать отчеты в клиентские приложения Windows Forms и ASP.NET. Мы также покажем вам, как использовать предоставляемые Web-службы для программного встраивания отчетности в приложения и скрипты развертывания.

## Резюме

Мы сравнили версию Microsoft SQL Server 2008 с более ранними версиями этого же продукта и обсудили широкий диапазон функций программирования. В этом процессе мы выделили различные главы и рассказали об их организации, а также дали обзор широкого набора функциональных возможностей SQL Server 2008 для разработчиков. Нет никакой необходимости читать эту книгу в каком-то определенном порядке. Читайте ее от начала и до конца (если хотите) или переходите напрямую к тем главам, которые вам необходимы больше всего. В любом случае вы получите практическую информацию.

## ГЛАВА 2



# Улучшения языка T-SQL

*Стивен Форте и Леонард Лобел*

Вы, вероятно, уже слышали, что можно писать хранимые процедуры Microsoft SQL Server, используя общезыковую исполняющую среду (common language runtime, CLR), такую как Microsoft Visual C# или Microsoft Visual Basic .NET. Это хорошая новость для тех, кто не владеет языком Transact-SQL (T-SQL), не так ли? Однако хранимые процедуры CLR не являются панацеей для всех задач программирования SQL Server. И слухи о смерти T-SQL были в значительной степени преувеличены. В большинстве случаев использование T-SQL для запросов и хранимых процедур более эффективно, чем написание хранимых процедур CLR.

Поскольку T-SQL жив и здоров, давайте посмотрим на то, как он изменился со времен SQL Server 2000. В этой главе мы начнем с исследования наиболее примечательных изменений в T-SQL, введенных в SQL Server 2005, таких как:

- ☐ обобщенные табличные выражения (CTE);
- ☐ операторы PIVOT и UNPIVOT;
- ☐ оператор APPLY;
- ☐ улучшения параметра TOP;
- ☐ функции ранжирования;
- ☐ обработка исключений при помощи блоков TRY и CATCH;
- ☐ тип данных `varchar(max)`;
- ☐ инструкция WAITFOR;
- ☐ триггеры языка определения данных (data definition language, DDL);
- ☐ уровень изоляции моментального снимка SNAPSHOT.

Продолжим знакомство мы подробным изложением новых функциональных возможностей T-SQL, которые появились в SQL Server 2008:

- ☐ возвращающие табличные значения параметры (table-valued parameters, TVP);
- ☐ новые типы данных для дат и времени;
- ☐ инструкция MERGE;
- ☐ синтаксис INSERT OVER DML;
- ☐ оператор GROUPING SETS;
- ☐ новый сокращенный синтаксис T-SQL.

Язык T-SQL предоставляет также некоторые типы данных "за пределами реляционности", такие как xml, hierarchyid, varbinary(max) FILESTREAM, geography и geometry. В главах с 6 по 9 дается подробное изложение этих специальных типов данных SQL Server 2008.

## Обобщенные табличные выражения

Обобщенное табличное выражение (CTE) очень напоминает непостоянное представление. Это временный именованный результирующий набор, который вы можете определить так, что он будет использован в выражении FROM запроса. CTE определяется только один раз (но на него можно сколько угодно ссылаться в области его действия) и живет столько же, сколько и запрос. Вы можете использовать CTE для выполнения рекурсивных операций. Вот синтаксис для создания CTE:

```
WITH <name of your CTE>(<column names>)
AS
(
    <actual query>
)
SELECT* FROM <name of your CTE>
```

### ПРИМЕЧАНИЕ

Многие наши примеры в этой главе используют базу данных AdventureWorks2008. Для выполнения этих примеров вам необходимо скачать и установить базу данных на ваш компьютер с сайта книги.



Пример простого выражения CTE с использованием базы AdventureWorks2008 показан в листинге 2.1.

### Листинг 2.1. Пример выражения CTE

```
USE AdventureWorks2008
GO
WITH AllMRContacts
AS
(
    SELECT* FROM Person.Person WHERE Title = 'Mr.'
)
SELECT LastName + ', ' + FirstName AS Contact
FROM AllMRContacts
ORDER BY LastName
```

Результаты таковы:

Contact

```
-----
Abbas, Syed
Achong, Gustavo
Adams, Jay
Adams, Ben
Adina, Ronald
Agcaoili, Samuel
...
```

Код, показанный в листинге 2.2, выдает количество всех продаж торгового представителя в системе заказов AdventureWorks2008, а затем выполняет простое внутреннее соединение с таблицей SalesPerson для выдачи дополнительной информации о продавце. Здесь показано, как выражение CTE соединяется с вызывающим запросом. Если вы часто создаете временную таблицу или одноразовое представление, теперь вы можете вместо них использовать CTE и предоставить ему все сложности агрегирования (что упрощает код).

### Листинг 2.2. Соединение выражения CTE и запроса

```
WITH OrderCountCTE(SalesPersonID, OrderCount)
AS
(
```

```
SELECT SalesPersonID, COUNT(*)
FROM Sales.SalesOrderHeader
WHERE SalesPersonID IS NOT NULL
GROUP BY SalesPersonID
)
SELECT
    sp.BusinessEntityID,
    FirstName + ' ' + LastName as SalesPerson,
    oc.OrderCount,
    sp.SalesYTD
FROM Sales.vSalesPerson AS sp
INNER JOIN OrderCountCTE AS oc ON oc.SalesPersonID = sp.BusinessEntityID
ORDER BY oc.OrderCount DESC
```

Результаты выглядят следующим образом:

| BusinessEntityID | SalesPerson              | OrderCount | SalesYTD     |
|------------------|--------------------------|------------|--------------|
| -----            | -----                    | -----      | -----        |
| 277              | Jillian Carson           | 473        | 3857163.6332 |
| 275              | Michael Blythe           | 450        | 4557045.0459 |
| 279              | Tsvi Reiter              | 429        | 2811012.7151 |
| 276              | Linda Mitchell           | 418        | 5200475.2313 |
| 289              | Jae Pak                  | 348        | 5015682.3752 |
| 282              | José Saraiva             | 271        | 3189356.2465 |
| 281              | Shu Ito                  | 242        | 3018725.4858 |
| 278              | Garrett Vargas           | 234        | 1764938.9859 |
| 283              | David Campbell           | 189        | 3587378.4257 |
| 290              | Ranjit Varkey Chudukatil | 175        | 3827950.238  |
| 284              | Tete Mensa-Annan         | 140        | 1931620.1835 |
| 288              | Rachel Valdez            | 130        | 2241204.0424 |
| 286              | Lynn Tsoflias            | 109        | 1758385.926  |
| 280              | Pamela Ansman-Wolfe      | 95         | 0.00         |
| 274              | Stephen Jiang            | 48         | 677558.4653  |
| 287              | Amy Alberts              | 39         | 636440.251   |
| 285              | Syed Abbas               | 16         | 219088.8836  |
| :                |                          |            |              |

Выражение СТЕ может также устранить самосоединения в запросах. Взгляните на пример листинга 2.3. Вы создадите таблицу с названием Products и вставите дубликаты в столбец ProductName.

**Листинг 2.3. Вставка дубликатов в столбец ProductName базы данных AdventureWorks2008**

```
CREATE TABLE Products
(
    ProductID int NOT NULL,
    ProductName varchar(25),
    Price money NULL,
    CONSTRAINT PK_Products PRIMARY KEY NONCLUSTERED (ProductID)
)
GO

INSERT INTO Products VALUES (1, 'Widgets', 25)
INSERT INTO Products VALUES (2, 'Gadgets', 50)
INSERT INTO Products VALUES (3, 'Thingies', 75)
INSERT INTO Products VALUES (4, 'Whoozits', 90)
INSERT INTO Products VALUES (5, 'Whatzits', 5)
INSERT INTO Products VALUES (6, 'Gizmos', 15)
INSERT INTO Products VALUES (7, 'Widgets', 24)
INSERT INTO Products VALUES (8, 'Gizmos', 36)
INSERT INTO Products VALUES (9, 'Gizmos', 36)
```

Обычной проблемой в базах данных являются дублированные названия с разными идентификаторами продуктов. Если вы запустите запрос поиска дубликатов, он вернет все записи (как правильные значения, так и дубликаты). Это осложняет автоматическое удаление дубликатов. Если вы хотите найти дубликаты ProductName без включения первого вхождения названия в таблицу, можно использовать самосоединение, как показано в листинге 2.4.

**Листинг 2.4. Самосоединение без выражения CTE**

```
SELECT* FROM Products WHERE ProductID NOT IN
(SELECT MIN(ProductID) FROM Products AS P
WHERE Products.ProductName = P.ProductName)
```

Самосоединение возвращает такие данные:

| ProductID | ProductName | Price |
|-----------|-------------|-------|
| 8         | Gizmos      | 36.00 |
| 9         | Gizmos      | 36.00 |
| 7         | Widgets     | 24.00 |

Вы можете также переписать ваш запрос при помощи выражения CTE для устранения этого путаного самосоединения и получите такие же результаты. Такой способ не дает выигрыша в производительности по сравнению с самосоединениями, но удобен в плане дальнейшего сопровождения кода. Предыдущий пример самосоединения переписан в листинге 2.5 как выражение CTE и дает те же самые результаты. Обратите внимание, что мы соединяем выражение CTE с таблицей `Products`.

#### Листинг 2.5. Самосоединение как выражение CTE

```
WITH MinProductRecords AS
(
    SELECT MIN(ProductID) AS ProductID, ProductName
    FROM Products
    GROUP BY ProductName
    HAVING COUNT(*) > 1
)
SELECT P.*
FROM Products AS P
    INNER JOIN MinProductRecords AS MP
    ON P.ProductName = MP.ProductName AND P.ProductID > MP.ProductID
```

После исследования дубликатов при помощи предыдущего выражения CTE вы, возможно, захотите удалить дублированные данные и обновить все внешние ключи в соответствующих таблицах (для использования исходного значения `ProductID`). Если ваши дублированные данные не имеют никаких дочерних строк в другой таблице (либо вы обновили правильное значение `ProductID`), то вы можете удалить дублированные данные, как показано в листинге 2.6 (заменяв оператор `SELECT*` на `DELETE`).

#### Листинг 2.6. Удаление дубликатов в выражении CTE

```
WITH MinProductRecords AS
(
    SELECT MIN(ProductID) AS ProductID, ProductName
    FROM Products
    GROUP BY ProductName
    HAVING COUNT(*) > 1
)
DELETE Products
```

```
FROM Products AS P
INNER JOIN MinProductRecords AS MP
ON P.ProductName = MP.ProductName AND P.ProductID > MP.ProductID
```

## Создание рекурсивных запросов при помощи выражения CTE

Вся мощь выражений CTE проявляется тогда, когда вы используете их рекурсивно для выполнения иерархических запросов по данным с древовидной структурой. Фактически именно по этой причине (в дополнение к совместимости с ANSI SQL-92) компания Microsoft и создала выражения CTE. Рекурсивное выражение CTE составлено минимум из двух запросов. Первый (закрепленный элемент) — это нерекурсивный запрос, второй (рекурсивный элемент) — это рекурсивный запрос. Внутри скобок CTE (после выражения AS) вы определяете запросы, которые являются независимыми или ссылаются на тот же самый элемент CTE. Закрепленный и рекурсивный элементы разделены инструкцией UNION ALL. Закрепленные элементы вызываются только один раз, а рекурсивные элементы вызываются повторно до тех пор, пока запрос перестанет возвращать строки. Вы можете соединять закрепленные элементы друг с другом при помощи операторов UNION или UNION ALL в зависимости от того, хотите ли вы устранять дубликаты или нет. Рекурсивные элементы надо соединять при помощи оператора UNION ALL. Вот синтаксис:

```
WITH SimpleRecursive(field names)
AS
(
    <Select Statement for the Anchor Member>
    UNION ALL
    <Select Statement for the Recursive Member>
)
SELECT* FROM SimpleRecursive
```

Пример в листинге 2.7 демонстрирует эту функциональную возможность. Мы создадим таблицу служащих и ссылающееся на себя поле ReportsTo. Затем мы напишем запрос, который возвращает всех служащих, подчиняющихся Стивену (Stephen, EmployeeID=2), и всех служащих, которые подчиняются подчиненным Стивена.

### Листинг 2.7. Пример таблицы для рекурсивных запросов выражения CTE

```
CREATE TABLE EmployeeTree
(EmployeeID int PRIMARY KEY,
```

```

EmployeeName nvarchar(50),
ReportsTo int)
GO
-- вставить некоторые данные, построить дерево подчиненности
INSERT INTO EmployeeTree VALUES(1, 'Richard', NULL)
INSERT INTO EmployeeTree VALUES(2, 'Stephen', 1)
INSERT INTO EmployeeTree VALUES(3, 'Clemens', 2)
INSERT INTO EmployeeTree VALUES(4, 'Malek', 2)
INSERT INTO EmployeeTree VALUES(5, 'Goksin', 4)
INSERT INTO EmployeeTree VALUES(6, 'Kimberly', 1)
INSERT INTO EmployeeTree VALUES(7, 'Ramesh', 5)

```

В листинге 2.8 показан рекурсивный запрос для определения тех служащих, которые подчиняются Стивену.

#### Листинг 2.8. Пример рекурсивного запроса

```

WITH SimpleRecursive(EmployeeID, EmployeeName, ReportsTo)
AS
(
    SELECT EmployeeID, EmployeeName, ReportsTo
    FROM EmployeeTree WHERE EmployeeID = 2
    UNION ALL
    SELECT p.EmployeeID, p.EmployeeName, p.ReportsTo
    FROM EmployeeTree AS P
    INNER JOIN SimpleRecursive A ON A.EmployeeID = P.ReportsTo
)
SELECT sr.EmployeeName AS Employee, et.EmployeeName AS Boss
FROM SimpleRecursive AS sr
INNER JOIN EmployeeTree AS et ON sr.ReportsTo = et.EmployeeID

```

Вот результаты:

| Employee | Boss    |
|----------|---------|
| -----    | -----   |
| Stephen  | Richard |
| Clemens  | Stephen |
| Malek    | Stephen |
| Goksin   | Malek   |
| Ramesh   | Goskin  |

Эта рекурсия начинается там, где `EmployeeID = 2` (закрепленный элемент или первый `SELECT`), выбирает запись и затем при помощи рекурсивного элемента (`SELECT` после `UNION ALL`) выбирает все записи, которые подчиняются Стивену, и потомков этой записи (`Goksin` подчиняется `Malek`, а `Malek` подчиняется `Stephen`). Каждая последующая рекурсия пытается найти других потомков, родителями которых являются служащие, найденные в предыдущей рекурсии. В конечном итоге очередная рекурсия не вернет никаких результатов, и именно это приводит к остановке рекурсии (поэтому не возвращается значение `Kimberly`). Если закрепленный элемент изменить на `EmployeeID = 1`, то `Kimberly` также встретится в результатах.

По идее, рекурсивный элемент продолжает искать потомков и может циклически работать до бесконечности. Если вы предполагаете, что таких циклов будет много, и вы хотите ограничить количество рекурсивных вызовов, можете указать опцию `MAXRECURSION` сразу же после внешнего запроса (при помощи выражения `OPTION`):

```
OPTION (MAXRECURSION 25)
```

Эта опция приводит к тому, что `SQL Server` выдает ошибку тогда, когда значение `CTE` превосходит указанный предел. По умолчанию предельное значение равно 100 (если вы не указали эту опцию). Для того чтобы не использовать эту опцию, вы должны установить `MAXRECURSION` в 0. Вы можете выполнить этот же самый запрос для поиска подчиненных только на один уровень вглубь (будут включены `Clemens`, `Malek` и его подчиненный `Goksin`, но пропущен `Ramesh` внизу на третьем уровне), как показано в листинге 2.9.

### Листинг 2.9. Рекурсивный запрос при помощи `MAXRECURSION`

```
WITH SimpleRecursive(EmployeeID, EmployeeName, ReportsTo)
AS
(
    SELECT EmployeeID, EmployeeName, ReportsTo
    FROM EmployeeTree WHERE EmployeeID = 2
    UNION ALL
    SELECT p.EmployeeID, p.EmployeeName, p.ReportsTo
    FROM EmployeeTree AS P
    INNER JOIN SimpleRecursive A ON A.EmployeeID = P.ReportsTo
)
SELECT sr.EmployeeName AS Employee, et.EmployeeName AS Boss
FROM SimpleRecursive AS sr
    INNER JOIN EmployeeTree AS et ON sr.ReportsTo = et.EmployeeID
OPTION (MAXRECURSION 2)
```

Вот результаты:

```
Employee    Boss
-----
Stephen     Richard
Clemens     Stephen
Malek       Stephen
Goksin      Malek
```

Вы увидите также, что запрос выдает следующее сообщение об ошибке:

```
Msg 530, Level 16, State 1, Line 2
```

```
The statement terminated. The maximum recursion 2 has been exhausted
before statement completion.
```

Можно избежать этого исключения, если использовать сгенерированный столбец для отслеживания уровня, на котором вы находитесь, и включить в выражение `WHERE` (вместо использования выражения `MAXRECURSION`). Переработанный пример в листинге 2.10 возвращает те же самые данные, что и предыдущий пример, но без ошибки.

#### Листинг 2.10. Управление рекурсией без выражения `MAXRECURSION`

```
WITH SimpleRecursive(EmployeeID, EmployeeName, ReportsTo, SubLevel)
AS
(
    SELECT EmployeeID, EmployeeName, ReportsTo, 0
    FROM EmployeeTree WHERE EmployeeID = 2
    UNION ALL
    SELECT p.EmployeeID, p.EmployeeName, p.ReportsTo, SubLevel + 1
    FROM EmployeeTree AS P
    INNER JOIN SimpleRecursive A ON A.EmployeeID = P.ReportsTo
    WHERE SubLevel <= 2
)
SELECT sr.EmployeeName AS Employee, et.EmployeeName AS Boss
FROM SimpleRecursive sr
INNER JOIN EmployeeTree AS et ON sr.ReportsTo = et.EmployeeID
```

#### ПРИМЕЧАНИЕ

SQL Server 2008 вводит новый тип данных `hierarchyid`, чтобы реализовать более надежную древовидную структуру (по сравнению с показанной в этих примерах рекурсивной таблицей с самосоединением). Тип данных `hierarchyid` описан в *главе 7*.



## Операторы *PIVOT* и *UNPIVOT*

Что и говорить, пользователи обычно хотят видеть данные в табличном формате, а это несколько затруднительно, поскольку данные в SQL Server чаще всего хранятся в реляционной форме. *PIVOT* — это оператор T-SQL, который вы можете указать в выражении *FROM* для обращения строк в столбцы и создания обычного перекрестного запроса.

Использовать оператор *PIVOT* несложно. В инструкции *SELECT* вы указываете значения, по которым вы хотите делать сведение. Следующий пример (по базе данных *AdventureWorks2008*) использует годы заказов (вычисленные при помощи функции *DatePart*) как столбцы. Выражение *FROM* выглядит совершенно обычно (за исключением инструкции *PIVOT*). Эта инструкция создает значение, которое вы хотите показать в строках вновь созданных столбцов. Данный пример использует агрегат *SUM of TotalDue* (вычисляемое поле в выражении *FROM*). Затем мы используем оператор *FOR* для перечисления значений, по которым мы хотим делать сведение (в столбце *OrderYear*). Пример показан в листинге 2.11.

### Листинг 2.11. Создание табличных результатов при помощи оператора *PIVOT*

```
SELECT
    CustomerID,
    [2001] AS Y2001, [2002] AS Y2002, [2003] AS Y2003, [2004] AS Y2004
FROM
    (
        SELECT CustomerID, DATEPART(yyyy, OrderDate) AS OrderYear, TotalDue
        FROM Sales.SalesOrderHeader
    ) AS piv
PIVOT
    (
        SUM(TotalDue) FOR OrderYear IN([2001], [2002], [2003], [2004])
    ) AS child
ORDER BY CustomerID
```

Вот результаты:

| CustomerID | Y2001      | Y2002       | Y2003       | Y2004      |
|------------|------------|-------------|-------------|------------|
| 1          | 40732.6067 | 72366.1284  | NULL        | NULL       |
| 2          | NULL       | 15653.6715  | 12118.0275  | 4962.2705  |
| 3          | 39752.8421 | 168393.7021 | 219434.4265 | 51925.3549 |

|    |      |             |             |             |
|----|------|-------------|-------------|-------------|
| 4  | NULL | 263025.3113 | 373484.299  | 143525.6018 |
| 5  | NULL | 33370.6901  | 60206.9999  | 20641.1106  |
| 6  | NULL | NULL        | 668.4861    | 2979.3473   |
| 7  | NULL | 6651.036    | 3718.7804   | NULL        |
| 8  | NULL | NULL        | 19439.2466  | 10900.0347  |
| 9  | NULL | 320.6283    | 11401.5975  | 5282.8652   |
| 10 | NULL | 96701.7401  | 291472.2172 | 204525.9634 |

...

Вот и все. Конечно, этот пример специально упрощен для того, чтобы показать вам концепцию. Возможны и другие, более сложные агрегаты, и вы можете даже использовать CTE в выражении FROM. В любом случае использовать PIVOT несложно.

## Использование оператора *UNPIVOT*

Вы можете использовать оператор UNPIVOT для нормализации данных, которые уже сведены. Например, предположим, что вы получили сведенные данные, которые показывают количество заказов, размещенных всеми служащими по всем поставщикам. Таковую таблицу создает код листинга 2.12.

### Листинг 2.12. Таблица со сведенными данными

```
CREATE TABLE VendorEmployee
(
  VendorID int,
  Emp1Orders int,
  Emp2Orders int,
  Emp3Orders int,
  Emp4Orders int,
  Emp5Orders int
)
GO
INSERT INTO VendorEmployee VALUES (1, 4, 3, 5, 4, 4)
INSERT INTO VendorEmployee VALUES (2, 4, 1, 5, 5, 5)
INSERT INTO VendorEmployee VALUES (3, 4, 3, 5, 4, 4)
INSERT INTO VendorEmployee VALUES (4, 4, 2, 5, 4, 4)
INSERT INTO VendorEmployee VALUES (5, 5, 1, 5, 5, 5)
```

Наша таблица выглядит следующим образом (табл. 2.1).

Таблица 2.1. Связанные данные

| VendorID | Emp1Orders | Emp2Orders | Emp3Orders | Emp4Orders | Emp5Orders |
|----------|------------|------------|------------|------------|------------|
| 1        | 4          | 3          | 5          | 4          | 4          |
| 2        | 4          | 1          | 5          | 5          | 5          |
| 3        | 4          | 3          | 5          | 4          | 4          |
| 4        | 4          | 2          | 5          | 4          | 4          |
| 5        | 5          | 1          | 5          | 5          | 5          |

Возможно, вы захотите отменить сведение данных. В листинге 2.13 показано, как использовать для достижения этой цели оператор UNPIVOT.

**Листинг 2.13. Использование оператора UNPIVOT**

```
SELECT VendorId, Employee, Orders AS NumberOfOrders
FROM
    (SELECT VendorId, Emp1Orders, Emp2Orders, Emp3Orders, Emp4Orders,
        Emp5Orders
    FROM VendorEmployee
    ) AS p
UNPIVOT
(
    Orders FOR Employee IN
        (Emp1Orders, Emp2Orders, Emp3Orders, Emp4Orders, Emp5Orders)
) AS unpvt
```

Вот результаты:

| VendorID | Employee   | NumberOfOrders |
|----------|------------|----------------|
| -----    |            |                |
| 1        | Emp1Orders | 4              |
| 1        | Emp2Orders | 3              |
| 1        | Emp3Orders | 5              |
| 1        | Emp4Orders | 4              |
| 1        | Emp5Orders | 4              |
| 2        | Emp1Orders | 4              |
| ...      |            |                |

## Динамическое сведение столбцов

Проблема с оператором `PIVOT` та же самая, что и с оператором `CASE`: вы должны указать столбцы. Рассмотрим код в листинге 2.14.

### Листинг 2.14. Статически управляемый `PIVOT`

```
SELECT*
  FROM (SELECT CustomerID, YEAR(OrderDate) AS OrderYear, TotalDue
        FROM Sales.SalesOrderHeader) AS header
PIVOT
(
  SUM(TotalDue) FOR orderyear IN([2002],[2003],[2004])
) AS piv
```

В результате мы имеем перекрестный запрос со столбцами по годам:

| CustomerID | 2002      | 2003      | 2004      |
|------------|-----------|-----------|-----------|
| 14324      | NULL      | 2264.2536 | 3394.9247 |
| 22814      | NULL      | 5.514     | NULL      |
| 11407      | NULL      | 59.659    | NULL      |
| 28387      | NULL      | NULL      | 645.2869  |
| 19897      | NULL      | NULL      | 659.6408  |
| 15675      | 2699.9018 | 2682.9953 | 2580.1529 |
| 24165      | NULL      | 2699.9018 | 666.8565  |
| ...        |           |           |           |

Эти данные доходят только до 2004 г. Что произойдет при добавлении в данные 2005 г.? Вы хотите внести изменения во все ваши запросы и добавить текущий год в выражение `IN`? Мы можем учесть их в запросах путем динамического построения выражения `IN` и последующего программного составления полной инструкции SQL. После того как вы динамически напишете инструкцию SQL, вы можете выполнить ее при помощи запроса `sp_executesql`, как показано в листинге 2.15. Все, что нам нужно сделать — это динамически сгенерировать выражение `IN`; и создать динамический оператор `PIVOT` в SQL Server гораздо проще, чем динамическую инструкцию `CASE`. Результаты будут точно такими же, как и в листинге 2.14, за исключением того, что по мере добавления в таблицу данных за новый год запрос динамически добавляет для них столбец. Генератор отчетности, скорее всего, не справится с новыми динамическими столбцами, однако элементы управления с привязкой к данным работать с ними смогут.

**Листинг 2.15. Динамически управляемый оператор PIVOT**

```

DECLARE @tblOrderDate AS TABLE(y int NOT NULL PRIMARY KEY)
INSERT INTO @tblOrderDate
    SELECT DISTINCT YEAR(OrderDate) FROM Sales.SalesOrderHeader
DECLARE @cols AS nvarchar(max)
DECLARE @years AS int
SET @years = (SELECT MIN(y) FROM @tblOrderDate)
SET @cols = N''
WHILE @years IS NOT NULL BEGIN
    SET @cols = @cols + N', [' + CAST(@years AS nvarchar(max)) + N'] '
    SET @years = (SELECT MIN(y) FROM @tblOrderDate WHERE y > @years)
END
SET @cols = SUBSTRING(@cols, 2, LEN(@cols))
DECLARE @sql AS nvarchar(max)
SET @sql = '
    SELECT*
    FROM
        (SELECT CustomerId, YEAR(OrderDate) AS OrderYear, TotalDue
         FROM Sales.SalesOrderHeader
        ) AS a
    PIVOT
    (
        SUM(TotalDue) FOR OrderYear IN(' + @cols + N')
    ) AS b
'
PRINT @sql -- for debugging
EXEC sp_executesql @sql

```

Вы можете получить те же самые результаты при помощи более нового синтаксиса CTE (вместо использования переменной table), как показано в листинге 2.16.

**Листинг 2.16. Динамически управляемый PIVOT с использованием CTE**

```

DECLARE @cols AS nvarchar(MAX)
WITH YearsCTE AS
    (SELECT DISTINCT YEAR(OrderDate) as [Year] FROM Sales.SalesOrderHeader)
SELECT @cols = ISNULL(@cols + ', [' + ']' + CAST([YEAR] AS nvarchar(10)) + ']'

```

```

FROM YearsCTE
ORDER BY [YEAR]
-- Сконструировать полную инструкцию T-SQL и динамически выполнить ее.
DECLARE @sql AS nvarchar(MAX)
SET @sql = '
    SELECT*
    FROM
        (SELECT CustomerId, YEAR(OrderDate) AS OrderYear, TotalDue
         FROM Sales.SalesOrderHeader
        ) AS a
    PIVOT
    (
        SUM(TotalDue) FOR OrderYear IN(' + @cols + N')
    ) AS b
'
PRINT @sql -- for debugging
EXEC sp_executesql @sql

```

## Оператор *APPLY*

*APPLY* — это оператор, который вы указываете в выражении *FROM* запроса. Он позволяет вам вызвать возвращающую табличное значение функцию (TVF) для каждой строки внешней таблицы. Гибкость оператора *APPLY* становится очевидной тогда, когда вы используете столбцы внешней таблицы в качестве аргументов вашей функции. Оператор *APPLY* имеет две формы: *CROSS APPLY* и *OUTER APPLY*. Так, *CROSS APPLY* не возвращает строки внешней таблицы в том случае, если TVF возвращает пустой набор, а *OUTER APPLY* возвращает строку со значениями *NULL* вместо столбцов функции.

Для того чтобы увидеть, как работает оператор *APPLY*, мы сначала создадим TVF. В листинге 2.17 показана простая функция, которая возвращает в виде таблицы верхние *n* строк из таблицы *SalesOrderHeader*.

### Листинг 2.17. Возвращение таблицы

```

CREATE FUNCTION fnGetCustomerOrders(@CustomerID int, @TopRecords bigint)
RETURNS TABLE
AS RETURN
    SELECT TOP (@TopRecords) *
    FROM Sales.SalesOrderHeader

```

```
WHERE CustomerID = @CustomerID  
ORDER BY OrderDate DESC
```

После создания `fnGetCustomerOrders` мы вызываем ее из запроса (как показано в листинге 2.18).

#### Листинг 2.18. Выполнение запроса при помощи оператора `APPLY`

```
SELECT* FROM Sales.Customer cust  
CROSS APPLY fnGetCustomerOrders(CustomerID, 100)
```

Запрос возвращает все записи из таблицы `Customers`, а затем (как дополнительные поля) те записи из таблицы `Orders`, которые соответствуют идентификатору пользователя, поскольку именно он передается динамически в качестве первого аргумента в `fnGetCustomerOrders`. Поскольку мы передали для второго параметра значение 100, то данным запросом будет сгенерировано и возвращено до 100 первых заказов данного клиента.

## Улучшения опции *TOP*

В SQL Server 2000 и более ранних версиях выражение `TOP` позволяет вам ограничить (числом или процентом) количество возвращаемых строк запроса `SELECT`. Начиная с SQL Server 2005, вы можете использовать `TOP` в запросах `DELETE`, `UPDATE`, `INSERT` и указывать количество (или процент) строк при помощи переменных или любого возвращающего числовое значение выражения (подзапрос). Главной причиной разрешения использовать `TOP` в запросах `DELETE`, `UPDATE` и `INSERT` было желание заменить инструкцию `SET ROWCOUNT`, которую SQL Server всегда не очень хорошо оптимизировал.

Вы можете указывать предел для `TOP` как число или как выражение. Если вы используете выражение, то вы должны заключить его в круглые скобки. Выражение должно иметь тип данных `bigint` (если вы не используете опцию `PERCENT`) или тип `float` в диапазоне от 0 до 100 (если вы используете опцию `PERCENT`). Возможно, вам покажется удобным создать выражение для `TOP` и сделать его параметром, который вы передаете в хранимую процедуру (как показано в листинге 2.19).

#### Листинг 2.19. Использование улучшений опции `TOP` в хранимой процедуре

```
CREATE PROCEDURE uspReturnTopOrders(@NumberOfRows bigint)  
AS
```

```
SELECT TOP (@NumberOfRows) SalesOrderID
FROM Sales.SalesOrderHeader
ORDER BY SalesOrderID
```

Выполнить хранимую процедуру просто. Передайте в нее количество необходимых вам записей (в данном случае это 100), как показано здесь:

```
EXEC uspReturnTopOrders @NumberOfRows = 100
```

Вот результаты:

```
SalesOrderID
-----
43659
43660
43661
43662
43663
:
(100 row(s) affected)
```

Использование подзапроса может быть весьма мощным средством, если вы делаете что-то "на ходу". В следующем примере показано, как получить верхние несколько заказов (в зависимости от количества строк в таблице SalesPerson):

```
SELECT TOP (SELECT COUNT(*) FROM Sales.SalesPerson)
SalesOrderID, RevisionNumber, OrderDate
FROM Sales.SalesOrderHeader
ORDER BY SalesOrderID
```

Поскольку в таблице SalesPerson имеется 17 строк, то запрос возвращает только 17 верхних строк из таблицы SalesOrderHeader:

| SalesOrderID | Revision | NumberOrderDate         |
|--------------|----------|-------------------------|
| -----        | -----    | -----                   |
| 43659        | 1        | 2001-07-01 00:00:00.000 |
| 43660        | 1        | 2001-07-01 00:00:00.000 |
| 43661        | 1        | 2001-07-01 00:00:00.000 |
| 43662        | 1        | 2001-07-01 00:00:00.000 |
| 43663        | 1        | 2001-07-01 00:00:00.000 |
| 43664        | 1        | 2001-07-01 00:00:00.000 |
| 43665        | 1        | 2001-07-01 00:00:00.000 |
| 43666        | 1        | 2001-07-01 00:00:00.000 |
| 43667        | 1        | 2001-07-01 00:00:00.000 |



```
43668          1          2001-07-01 00:00:00.000
43669          1          2001-07-01 00:00:00.000
43670          1          2001-07-01 00:00:00.000
43671          1          2001-07-01 00:00:00.000
43672          1          2001-07-01 00:00:00.000
43673          1          2001-07-01 00:00:00.000
43674          1          2001-07-01 00:00:00.000
43675          1          2001-07-01 00:00:00.000
(17 row(s) affected)
```

Использовать опцию `PERCENT` так же просто. Добавьте ключевое слово `PERCENT` и убедитесь, что переменная имеет тип `float`. В листинге 2.20 мы запрашиваем верхние 10% и получаем 3147 записей, поскольку таблица `SalesOrderHeader` базы данных `AdventureWorks2008` имеет 31 465 записей.

#### Листинг 2.20. Возвращение процента

```
DECLARE @NumberOfRows AS float
SET @NumberOfRows = 10
SELECT TOP (@NumberOfRows) PERCENT *
FROM Sales.SalesOrderHeader
ORDER BY OrderDate
```

## Ранжирование функций

Базы данных хранят данные. Пользователи иногда хотят использовать простые вычисления или алгоритмы для этих данных, чтобы ранжировать результаты в определенном порядке, например, чтобы получить золотые, серебряные и бронзовые медали Олимпийских игр, либо 10 самых важных клиентов по каждому региону. Начиная с SQL Server 2005, вам предоставлена функциональная возможность использования ранжирующих выражений для вашего результирующего набора. Вы можете выбрать несколько алгоритмов ранжирования, которые затем применяются к указанному вами столбцу (в области действия выполняющегося запроса). Если данные изменяются, то ваш алгоритм ранжирования при следующем выполнении возвратит другие данные. Это удобно использовать в приложениях Microsoft .NET Framework для разбивки на страницы и сортировки внутри сетки, а также и во многих других случаях.

## Функция **ROW\_NUMBER**

Самая простая ранжирующая функция — это `ROW_NUMBER`. Она возвращает столбец как выражение, которое содержит номер строки в результирующем наборе. Этот номер используется только в контексте результирующего набора, если результат меняется, то изменяется и `ROW_NUMBER`. Выражение `ROW_NUMBER` берет инструкцию `ORDER BY` со столбцом, который вы хотите использовать для количества строк, и оператор `OVER`, который связывает `ORDER BY` с конкретной используемой вами функцией ранжирования. `ORDER BY` в выражении `OVER` заменяет `ORDER BY` в конце инструкции `SQL`.

Простой код листинга 2.21 дает номер каждой строке результирующего набора, упорядочивая по `SalesOrderID`.

### Листинг 2.21. Ранжирование по номеру строки

```
SELECT
    SalesOrderID,
    CustomerID,
    ROW_NUMBER() OVER (ORDER BY SalesOrderID) AS RowNumber
FROM Sales.SalesOrderHeader
```

Вот результаты:

| SalesOrderID | CustomerID | RowNumber |
|--------------|------------|-----------|
| 43659        | 676        | 1         |
| 43660        | 117        | 2         |
| 43661        | 442        | 3         |
| 43662        | 227        | 4         |
| 43663        | 510        | 5         |
| 43664        | 397        | 6         |
| 43665        | 146        | 7         |
| 43666        | 511        | 8         |
| 43667        | 646        | 9         |

:

## Опции **ORDER BY**

Функции ранжирования упорядочивают результирующий набор по полям опции `ORDER BY`, содержащейся в выражении `OVER`. Вы можете включить дополнительную инструкцию `ORDER BY` в ваш результирующий набор, она от-

личается от выражения `ORDER BY` в выражении `OVER`. Но тогда результаты функции `ROW_NUMBER` отображаются в том порядке, в котором они определены дополнительной инструкцией `ORDER BY` (а не инструкцией `ORDER BY`, содержащейся в выражении `OVER`). Поэтому может получиться путаница. Для иллюстрации, если мы добавим дополнительное выражение `ORDER BY CustomerID` в только что выполнявшийся нами запрос, то мы получим такие "перемешанные" результаты:

| SalesOrderID | CustomerID | RowNumber |
|--------------|------------|-----------|
| 43860        | 1          | 202       |
| 44501        | 1          | 843       |
| 45283        | 1          | 1625      |
| 46042        | 1          | 2384      |
| 46976        | 2          | 3318      |
| 47997        | 2          | 4339      |
| 49054        | 2          | 5396      |
| 50216        | 2          | 6558      |
| 51728        | 2          | 8070      |
| 57044        | 2          | 13386     |
| 63198        | 2          | 19540     |
| 69488        | 2          | 25830     |
| 65310        | 3          | 21652     |
| 71889        | 3          | 28231     |
| 53616        | 3          | 9958      |

:

Как видно, полагая, что результаты будут отсортированы по инструкции `ORDER BY` выражения `OVER`, вы ждете ранжирования по `SalesOrderID` (в то время как фактически они упорядочены по `CustomerID`).

Если вы запустите функцию `ROW_NUMBER` по неуникальному столбцу, который содержит множество копий одного и того же значения (называемых также "узелками", это может быть одинаковое количество проданных предметов или одинаковое время, показанное в гонке), то `ROW_NUMBER` разбивает этот узелок и дает нарастающий счетчик (так что строки имеют разные номера). Например, в листинге 2.22 может повторяться `CustomerID`, что создает несколько узелков. SQL Server просто увеличивает текущий счетчик для каждой строки (независимо от количества существующих узелков).

**Листинг 2.22. Ранжирование номеров строк с узелками**

```

SELECT
    SalesOrderID,
    CustomerID,
    ROW_NUMBER() OVER (ORDER BY CustomerID) AS RowNumber
FROM Sales.SalesOrderHeader

```

Результаты показаны здесь:

| SalesOrderID | CustomerID | RowNumber |
|--------------|------------|-----------|
| 43860        | 1          | 1         |
| 44501        | 1          | 2         |
| 45283        | 1          | 3         |
| 46042        | 1          | 4         |
| 46976        | 2          | 5         |
| 47997        | 2          | 6         |
| 49054        | 2          | 7         |
| 50216        | 2          | 8         |
| 51728        | 2          | 9         |
| 57044        | 2          | 10        |
| 63198        | 2          | 11        |
| 69488        | 2          | 12        |
| 44124        | 3          | 13        |

:

## Группировка и фильтрация при помощи *ROW\_NUMBER*

Если вы хотите включить в ваш запрос функцию `GROUP BY`, то ранжирующие функции работать не будут. Для того чтобы без труда обойти это ограничение, необходимо создать `GROUP BY` в `CTE`, а затем выполнить ранжирование результатов, как показано в листинге 2.23.

**Листинг 2.23. Группирование по номеру строки**

```

WITH CustomerSum
AS
(
    SELECT CustomerID, SUM(TotalDue) AS TotalAmt

```

```

FROM Sales.SalesOrderHeader
GROUP BY CustomerID
)
-- здесь в конец набора данных добавляется номер строки
SELECT
    *,
    ROW_NUMBER() OVER (ORDER BY TotalAmt DESC) AS RowNumber
FROM CustomerSum

```

Вот результаты:

| CustomerID | TotalAmt     | RowNumber |
|------------|--------------|-----------|
| 678        | 1179857.4657 | 1         |
| 697        | 1179475.8399 | 2         |
| 170        | 1134747.4413 | 3         |
| 328        | 1084439.0265 | 4         |
| 514        | 1074154.3035 | 5         |
| 155        | 1045197.0498 | 6         |
| 72         | 1005539.7181 | 7         |

Для фильтрации по ROW\_NUMBER вы должны поместить в CTE функцию ROW\_NUMBER (листинг 2.24).

#### Листинг 2.24. Фильтрация по номеру строки

```

WITH NumberedRows AS
(
    SELECT
        SalesOrderID,
        CustomerID,
        ROW_NUMBER() OVER (ORDER BY SalesOrderID) AS RowNumber
    FROM Sales.SalesOrderHeader
)
SELECT* FROM NumberedRows
WHERE RowNumber BETWEEN 100 AND 200

```

Вот результаты:

| SalesOrderID | CustomerID | RowNumber |
|--------------|------------|-----------|
| 43759        | 13257      | 100       |
| 43760        | 16352      | 101       |

|       |       |     |
|-------|-------|-----|
| 43761 | 16493 | 102 |
| :     |       |     |
| 43857 | 533   | 199 |
| 43858 | 36    | 200 |

## Функция *RANK*

Чаше всего вы, наверное, будете использовать функцию `RANK`, которая ранжирует данные в выражении `ORDER BY` в указанном вами порядке. Функция `RANK` синтаксически похожа на `ROW_NUMBER`, но дает правильные результаты ранжирования. Она работает по олимпийскому принципу: когда результатом борьбы за золото становится ничья, то следующая награда — бронза. Например, если при использовании функции `RANK` четыре строки имеют значение 1, то следующее значение столбца ранжирования будет 5 (листинг 2.25).

**Листинг 2.25. Функция `RANK`**

```
SELECT
    SalesOrderID,
    CustomerID,
    RANK() OVER (ORDER BY CustomerID) AS Rank
FROM Sales.SalesOrderHeader
```

Вот результаты:

| SalesOrderID | CustomerID | Rank |
|--------------|------------|------|
| -----        |            |      |
| 43860        | 1          | 1    |
| 44501        | 1          | 1    |
| 45283        | 1          | 1    |
| 46042        | 1          | 1    |
| 46976        | 2          | 5    |
| 47997        | 2          | 5    |
| 49054        | 2          | 5    |
| 50216        | 2          | 5    |
| 51728        | 2          | 5    |
| 57044        | 2          | 5    |
| 63198        | 2          | 5    |
| 69488        | 2          | 5    |
| 44124        | 3          | 13   |
| :            |            |      |

Как и другим функциям ранжирования, функции RANK для работы с агрегатами нужна помощь CTE. Рассмотрим запрос, который ранжирует клиентов по объему покупок. Нам необходимо использовать CTE для того, чтобы сначала выполнить агрегирование, а затем ранжировать по новому выражению агрегирования (как показано в листинге 2.26).

**Листинг 2.26. Ранжированные агрегаты**

```
WITH CustomerSum AS
(
    SELECT CustomerID, SUM(TotalDue) AS TotalAmt
    FROM Sales.SalesOrderHeader
    GROUP BY CustomerID
)
SELECT
    *,
    RANK() OVER (ORDER BY TotalAmt DESC) AS Rank
FROM CustomerSum
```

Результаты показаны здесь. Обратите внимание, что на первом месте находится клиент 678:

| CustomerID | TotalAmt     | Rank |
|------------|--------------|------|
| 678        | 1179857.4657 | 1    |
| 697        | 1179475.8399 | 2    |
| 170        | 1134747.4413 | 3    |
| 328        | 1084439.0265 | 4    |
| 514        | 1074154.3035 | 5    |

Как уже указывалось ранее, важно помнить, что предоставляемые в SQL Server функции ранжирования действительны только для области действия выполняющегося запроса. Если данные изменятся, и после этого вы опять запустите тот же самый запрос, то вы получите другие результаты. Давайте, например, в листинге 2.27 изменим запись для клиента 697. Изменив одну из строк детализации по размещенному клиентом 697 заказу и увеличив количество заказанного, мы делаем клиента 697 нашим главным клиентом.

**Листинг 2.27. Изменение результатов ранга RANK при изменении данных**

```
UPDATE Sales.SalesOrderDetail
SET OrderQty = 50 -- исходное значение было равно 2
WHERE SalesOrderDetailID = 535
```

Теперь опять запустим тот же самый запрос. Обратите внимание, что клиент 697 теперь обошел клиента 678 и стал главным:

| CustomerID | TotalAmt     | Rank |
|------------|--------------|------|
| 697        | 1272595.5474 | 1    |
| 678        | 1179857.4657 | 2    |
| 170        | 1134747.4413 | 3    |
| 328        | 1084439.0265 | 4    |
| 514        | 1074154.3035 | 5    |

:

## Функции *DENSE\_RANK* и *NTILE*

Последние две ранжирующие функции, которые мы здесь рассмотрим — это *DENSE\_RANK* и *NTILE*. Функция *DENSE\_RANK* работает точно так же, как и *RANK*, за исключением того, что приращение делается только при изменениях ранга. Иначе говоря (в отличие от Олимпийских игр), после двух золотых медалей она присваивает серебро. Пример показан в листинге 2.28.

### Листинг 2.28. Ранжирование при помощи *DENSE\_RANK*

```
SELECT
    SalesOrderID,
    CustomerID,
    DENSE_RANK() OVER (ORDER BY CustomerID) AS DenseRank
FROM Sales.SalesOrderHeader
WHERE CustomerID > 100
```

Результаты показаны далее:

| SalesOrderID | CustomerID | DenseRank |
|--------------|------------|-----------|
| 46950        | 101        | 1         |
| 47979        | 101        | 1         |
| 49048        | 101        | 1         |
| 50200        | 101        | 1         |
| 51700        | 101        | 1         |
| 57022        | 101        | 1         |
| 63138        | 101        | 1         |
| 69400        | 101        | 1         |



|       |     |   |
|-------|-----|---|
| 43855 | 102 | 2 |
| 44498 | 102 | 2 |
| 45280 | 102 | 2 |
| 46038 | 102 | 2 |
| 46951 | 102 | 2 |
| 47978 | 102 | 2 |
| 49103 | 102 | 2 |
| 50199 | 102 | 2 |
| 51733 | 103 | 3 |
| 57058 | 103 | 3 |

:

В следующем примере показана разница между RANK и DENSE\_RANK. Мы округлим продажи по клиентам до ближайшей сотни (поскольку начальство любит видеть в отчетах круглые числа!) и посмотрим, что изменится в случае ничьей. Код показан в листинге 2.29.

**Листинг 2.29. Функция RANK в сравнении с DENSE\_RANK**

```
WITH CustomerSum AS
(
    SELECT
        CustomerID,
        ROUND(CONVERT(int, SUM(TotalDue)) / 100, 8) * 100 AS TotalAmt
    FROM Sales.SalesOrderHeader
    GROUP BY CustomerID
)
SELECT*,
    RANK() OVER (ORDER BY TotalAmt DESC) AS Rank,
    DENSE_RANK() OVER (ORDER BY TotalAmt DESC) AS DenseRank
FROM CustomerSum
```

Вот результаты:

| CustomerID | TotalAmt | Rank  | DenseRank |
|------------|----------|-------|-----------|
| -----      | -----    | ----- | -----     |
| 697        | 1272500  | 1     | 1         |
| 678        | 1179800  | 2     | 2         |
| 170        | 1134700  | 3     | 3         |
| 328        | 1084400  | 4     | 4         |

:

|     |        |     |     |
|-----|--------|-----|-----|
| 87  | 213300 | 170 | 170 |
| 667 | 210600 | 171 | 171 |
| 196 | 207700 | 172 | 172 |
| 451 | 206100 | 173 | 173 |
| 672 | 206100 | 173 | 173 |
| 27  | 205200 | 175 | 174 |
| 687 | 205200 | 175 | 174 |
| 163 | 204000 | 177 | 175 |
| 102 | 203900 | 178 | 176 |
| :   |        |     |     |

Обратите внимание, что между клиентами 451 и 672 ничья, суммы продаж по ним одинаковые. Им обоим присвоен ранг 173 как функцией `RANK`, так и функцией `DENSE_RANK`. Разница проявляется в том, что происходит после этого. Между клиентами 27 и 687 ничья в состязании за следующую позицию, и им обоим `RANK` присваивает 175, а `DENSE_RANK` — 174. Следующий клиент с номером 163 получает 177 от `RANK` и 175 от `DENSE_RANK`.

Функция `NTILE` делит возвращенные строки на группы примерно одинакового размера (количество которых вы указываете как параметр функции). Она присваивает каждому члену группы в результирующем наборе один и тот же номер. Отличным примером является процентильное ранжирование на университетских экзаменах или в дорожных гонках. Пример использования `NTILE` показан в листинге 2.30.

**Листинг 2.30. Ранжирование при помощи функции `NTILE`**

```
SELECT
    SalesOrderID,
    CustomerID,
    NTILE(10000) OVER (ORDER BY CustomerID) AS NTile
FROM Sales.SalesOrderHeader
```

Вот результаты:

| SalesOrderID | CustomerID | NTile |
|--------------|------------|-------|
| -----        |            |       |
| 43860        | 1          | 1     |
| 44501        | 1          | 1     |
| 45283        | 1          | 1     |
| 46042        | 1          | 1     |
| 46976        | 2          | 2     |
| 47997        | 2          | 2     |

```

49054          2          2
50216          2          2
51728          2          3
57044          2          3
63198          2          3
69488          2          3
44124          3          4
:
45024          29475       9998
45199          29476       9998
60449          29477       9998
60955          29478       9999
49617          29479       9999
62341          29480       9999
45427          29481      10000
49746          29482      10000
49665          29483      10000
(31465 row(s) affected)

```

## Совместное использование всех функций ранжирования

До настоящего момента мы рассматривали все ранжирующие функции по отдельности. Ранжирующие функции — это регулярные выражения SQL Server, поэтому в одной инструкции `SELECT` их может быть у вас столько, сколько захотите. В листинге 2.31 мы рассмотрим еще один, последний пример, который сводит все это в одну инструкцию SQL и показывает разницу между четырьмя функциями ранжирования.

### Листинг 2.31. Сравнение функций ранжирования в SQL Server

```

SELECT
    SalesOrderID AS OrderID,
    CustomerID,
    ROW_NUMBER() OVER (ORDER BY CustomerID) AS RowNumber,
    RANK() OVER (ORDER BY CustomerID) AS Rank,
    DENSE_RANK() OVER (ORDER BY CustomerID) AS DenseRank,
    NTILE(10000) OVER (ORDER BY CustomerID) AS NTile
FROM Sales.SalesOrderHeader

```

Результаты показаны здесь:

| OrderID | CustomerID | RowNumber | Rank | DenseRank | NTile |
|---------|------------|-----------|------|-----------|-------|
| 43860   | 1          | 1         | 1    | 1         | 1     |
| 44501   | 1          | 2         | 1    | 1         | 1     |
| 45283   | 1          | 3         | 1    | 1         | 1     |
| 46042   | 1          | 4         | 1    | 1         | 1     |
| 46976   | 2          | 5         | 5    | 2         | 2     |
| 47997   | 2          | 6         | 5    | 2         | 2     |
| 49054   | 2          | 7         | 5    | 2         | 2     |
| 50216   | 2          | 8         | 5    | 2         | 2     |
| 51728   | 2          | 9         | 5    | 2         | 3     |
| 57044   | 2          | 10        | 5    | 2         | 3     |
| 63198   | 2          | 11        | 5    | 2         | 3     |
| 69488   | 2          | 12        | 5    | 2         | 3     |
| 44124   | 3          | 13        | 13   | 3         | 4     |
| 44791   | 3          | 14        | 13   | 3         | 4     |

-----

:

## Ранжирование по группам при помощи *PARTITION BY*

Функции ранжирования можно комбинировать с *вырезающими функциями*. Вырезающая функция разделяет результирующий набор на равные сегменты в соответствии со значениями инструкции `PARTITION BY` и в сочетании с выражением `OVER` функции ранжирования. Это подобно применению опции `GROUP BY` к вашей функции ранжирования: вы получаете отдельный ранг для каждого сегмента. В листинге 2.32 использованы `ROW_NUMBER` и `PARTITION BY` для подсчета количества заказов по датам заказов и по продавцам. Мы делаем это при помощи использования команды `PARTITION BY SalesPersonID ORDER BY OrderDate`. Это можно сделать с любой из четырех функций ранжирования.

### Листинг 2.32. Ранжирование по группам при помощи *PARTITION BY*

```
SELECT
    SalesOrderID,
    SalesPersonID,
    OrderDate,
```

```

ROW_NUMBER() OVER (PARTITION BY SalesPersonID ORDER BY OrderDate) AS
    OrderRank
FROM Sales.SalesOrderHeader
WHERE SalesPersonID IS NOT NULL

```

Далее показаны результаты. Порядок ваших строк может несколько отличаться от показанных здесь результатов.

| SalesOrderID | SalesPersonID | OrderDate               | OrderRank |
|--------------|---------------|-------------------------|-----------|
| -----        | -----         | -----                   | -----     |
| :            |               |                         |           |
| 43659        | 279           | 2001-07-01 00:00:00.000 | 1         |
| 43660        | 279           | 2001-07-01 00:00:00.000 | 2         |
| 43681        | 279           | 2001-07-01 00:00:00.000 | 3         |
| 43684        | 279           | 2001-07-01 00:00:00.000 | 4         |
| 43685        | 279           | 2001-07-01 00:00:00.000 | 5         |
| 43694        | 279           | 2001-07-01 00:00:00.000 | 6         |
| 43695        | 279           | 2001-07-01 00:00:00.000 | 7         |
| 43696        | 279           | 2001-07-01 00:00:00.000 | 8         |
| 43845        | 279           | 2001-08-01 00:00:00.000 | 9         |
| 43861        | 279           | 2001-08-01 00:00:00.000 | 10        |
| :            |               |                         |           |
| 48079        | 287           | 2002.11-01 00:00:00.000 | 1         |
| 48064        | 287           | 2002.11-01 00:00:00.000 | 2         |
| 48057        | 287           | 2002.11-01 00:00:00.000 | 3         |
| 47998        | 287           | 2002.11-01 00:00:00.000 | 4         |
| 48001        | 287           | 2002.11-01 00:00:00.000 | 5         |
| 48014        | 287           | 2002.11-01 00:00:00.000 | 6         |
| 47982        | 287           | 2002.11-01 00:00:00.000 | 7         |
| 47992        | 287           | 2002.11-01 00:00:00.000 | 8         |
| 48390        | 287           | 2002.12.01 00:00:00.000 | 9         |
| 48308        | 287           | 2002.12.01 00:00:00.000 | 10        |
| :            |               |                         |           |

Давайте разобьем нашу функцию ранжирования по странам (как это показано в листинге 2.33). Мы создадим СТЕ для агрегирования продаж по клиентам и по странам. Затем мы применим функцию ранжирования по полю TotalAmt и по полю CustomerID (с разбиением по полю CountryName).

#### Листинг 2.33. Агрегаты с использованием PARTITION BY

```

WITH CTETerritory AS
(

```

```

SELECT
    cr.Name AS CountryName,
    CustomerID,
    SUM(TotalDue) AS TotalAmt
FROM
    Sales.SalesOrderHeader AS soh
    INNER JOIN Sales.SalesTerritory AS ter ON soh.TerritoryID =
        ter.TerritoryID
    INNER JOIN Person.CountryRegion AS cr ON cr.CountryRegionCode =
        ter.CountryRegionCode
GROUP BY
    cr.Name, CustomerID
)
SELECT
    *,
    RANK() OVER(PARTITION BY CountryName ORDER BY TotalAmt, CustomerID
DESC) AS Rank
FROM CTETerritory

```

Результаты выглядят следующим образом:

CountryName CustomerID TotalAmt Rank

```

-----
Australia      29083          4.409      1
Australia      29061          4.409      2
Australia      29290          5.514      3
Australia      29287          5.514      4
Australia      28924          5.514      5
:
Canada         29267          5.514      1
Canada         29230          5.514      2
Canada         28248          5.514      3
Canada         27628          5.514      4
Canada         27414          5.514      5
:
France         24538          4.409      1
France         24535          4.409      2
France         23623          4.409      3
France         23611          4.409      4
France         20961          4.409      5
:

```

PARTITION BY поддерживает и другие функции агрегирования SQL Server (в том числе MIN и MAX), а также ваши собственные скалярные функции. Давайте применим эту методику к текущему примеру (путем добавления столбца в наш результирующий набор посредством функции AVG, как показано в листинге 2.34). Мы получим те же самые результаты, но с дополнительным столбцом среднего значения по стране.

#### Листинг 2.34. Использование AVG вместе с PARTITION BY

```
WITH CTETerritory AS
(
    SELECT
        cr.Name AS CountryName,
        CustomerID,
        SUM(TotalDue) AS TotalAmt
    FROM
        Sales.SalesOrderHeader AS soh
        INNER JOIN Sales.SalesTerritory AS ter ON soh.TerritoryID =
            ter.TerritoryID
        INNER JOIN Person.CountryRegion AS cr ON cr.CountryRegionCode =
            ter.CountryRegionCode
    GROUP BY
        cr.Name, CustomerID
)
SELECT
    *,
    RANK() OVER (PARTITION BY CountryName ORDER BY TotalAmt,
        CustomerID DESC) AS Rank, AVG(TotalAmt) OVER(PARTITION BY Country-
Name)
        AS Average
FROM CTETerritory
```

Вот результаты:

| CountryName | CustomerID | TotalAmt | Rank | Average   |
|-------------|------------|----------|------|-----------|
| Australia   | 29083      | 4.409    | 1    | 3364.8318 |
| Australia   | 29061      | 4.409    | 2    | 3364.8318 |
| Australia   | 29290      | 5.514    | 3    | 3364.8318 |

:

|        |       |       |   |           |
|--------|-------|-------|---|-----------|
| Canada | 29267 | 5.514 | 1 | 12824.756 |
| Canada | 29230 | 5.514 | 2 | 12824.756 |
| Canada | 28248 | 5.514 | 3 | 12824.756 |

:

## Обработка исключительных состояний в транзакциях

Начиная с SQL Server 2005, вы можете перехватывать ошибки T-SQL и ошибки аварийного завершения транзакций при помощи модели TRY/CATCH без потери контекста транзакции. Типы ошибок, которые не может обработать конструкция TRY/CATCH — это те, которые вызывают завершение вашего сеанса (с кодом серьезности 21 и выше, ошибки аппаратного обеспечения). Синтаксис показан здесь:

```
BEGIN TRY
    -- инструкции sql
END TRY
BEGIN CATCH
    --инструкции sql для перехвата ваших ошибок
END CATCH
```

Если ошибка в явной транзакции происходит внутри блока TRY, то управление передается блоку CATCH (который следует сразу за TRY). Если ошибок не происходит, то блок CATCH полностью пропускается.

Вы можете изучить тип возникшей ошибки и соответствующим образом отреагировать. Для этого вы можете использовать функции ERROR\_ххх (для возвращения информации об ошибке в блоке CATCH, как это показано в листинге 2.35).

### Листинг 2.35. Пример обработки исключений T-SQL

```
BEGIN TRY
    SELECT 5/0
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER()    AS ErrNumber,
        ERROR_SEVERITY()  AS ErrSeverity,
        ERROR_STATE()     AS ErrState,
```



```

ERROR_PROCEDURE() AS ErrProc,
ERROR_LINE()      AS ErrLine,
ERROR_MESSAGE()   AS ErrMessage
END CATCH

```

Вы можете изучить значение, выданное любой из функций `ERROR_xxx`, чтобы решить, что же делать с потоком управления вашей процедуры, и нужно ли прерывать выполнение каких-либо транзакций. Так, в листинге 2.35 производится попытка деления на 0. (Функция `ERROR_PROCEDURE` в этом примере возвращает `NULL`, поскольку исключение произошло не в хранимой процедуре).

```

ErrNumber ErrSeverity ErrState ErrProc ErrLine ErrMessage
-----
8134      16          1      NULL    2      Divide by zero error
encountered.

```

Если аварийно завершается транзакция внутри блока `TRY`, то управление передается в блок `CATCH`. Затем транзакция переходит в состояние сбоя (при этом блокировки не освобождаются и сохраненная работа не отменяется до тех пор, пока вы явно не выдадите инструкцию `ROLLBACK`). Вам не позволяется выполнять никакие действия (которые требуют открытия явной или неявной транзакции) до тех пор, пока вы не выдадите инструкцию `ROLLBACK`.

Некоторые типы ошибок блоком `TRY/CATCH` не выявляются, и в итоге у вас остается необработанный исключение (даже в том случае, когда ошибка произошла внутри блока `TRY`). Два примера таких ошибок — синтаксические ошибки и ошибки перекомпилирования на уровне инструкций (например, выборка из несуществующей таблицы). Такие ошибки перехватываются на более низком уровне, когда вы выполняете динамический SQL или вызываете хранимую процедуру из блока `TRY`. Например, если внутри блока `TRY` синтаксическая ошибка, то вы получаете ошибку компилятора, и блок `CATCH` не выполняется, как показано далее:

```

-- синтаксическая ошибка не ловится
BEGIN TRY
    SELECT* * FROM Customer
END TRY
BEGIN CATCH
    PRINT 'Error'
END CATCH

```

Результатом будет ошибка SQL Server (а не блока `CATCH`), как показано далее:

```

Msg 102, Level 15, State 1, Line 2
Incorrect syntax near '*'.

```

Ошибки перекомпиляции на уровне инструкций также не ловятся блоками CATCH. Например, выборка несуществующего объекта инструкцией SELECT в блоке TRY порождает ошибку SQL Server, но ваш блок CATCH выполняться не будет:

```
-- перекомпиляция на уровне инструкции не ловится
BEGIN TRY
    SELECT* FROM NonExistentTable
END TRY
BEGIN CATCH
    PRINT 'Error'
END CATCH
```

В результате получится ошибка SQL Server:

```
Msg 208, Level 16, State 1, Line 3
Invalid object name 'NonExistentTable'.
```

В динамическом SQL или хранимой процедуре эти типы ошибок компиляции перехватываются, поскольку они являются частью текущего уровня выполнения. Каждый из показанных в листинге 2.36 блоков SQL будет выполнять блок CATCH.

**Листинг 2.36. Перехват синтаксических ошибок и ошибок перекомпиляции в динамическом SQL и вызовы хранимых процедур с обработчиками ошибок**

```
-- Пример динамического SQL
BEGIN TRY
    EXEC sp_executesql 'SELECT* * FROM Customer'
END TRY
BEGIN CATCH
    PRINT 'Error'
END CATCH
GO

-- Пример хранимой процедуры
CREATE PROCEDURE MyErrorProc
AS
    SELECT* FROM NonExistentTable
GO
BEGIN TRY
    EXEC MyErrorProc
END TRY
BEGIN CATCH
    PRINT 'Error'
END CATCH
```

## Тип данных *varchar(max)*

Типы данных *varchar(max)*, *nvarchar(max)* и *varbinary(max)* являются расширениями соответственно типов *varchar*, *nvarchar* и *varbinary* и могут хранить до 2 Гбайт данных. Они являются альтернативой типов *text*, *ntext* и *image* и используют спецификатор размера *max*. Использовать эти типы данных несложно: вы просто указываете их в инструкции *CREATE TABLE* (либо в декларации любой переменной) с идентификатором (*max*), как в этом примере:

```
CREATE TABLE TableWithMaxColumn  
(Customer_Id int, CustomerLifeStory varchar(max))
```

Все стандартные строковые функции T-SQL работают и с *varchar(max)*, в том числе и функции конкатенации *SUBSTRING*, *LEN* и *CONVERT*. Например, вы можете использовать функцию *SUBSTRING* для чтения частей строки (блоков).

Это огромное улучшение по сравнению с ограничениями SQL Server 2000 и более ранних версий, где для хранения данных этого типа использовались поля *text* и *image*, которые не годятся в качестве параметров хранимых процедур и нельзя обновлять напрямую.

## Инструкция *WAITFOR*

В SQL Server 2000 инструкция *WAITFOR* ожидала в течение указанного времени либо дожидалась указанного момента времени *datetime*. Начиная с SQL Server 2005, *WAITFOR* с выражением SQL (точно так же, как и *TOP*) ожидает, когда инструкция T-SQL затронет, по крайней мере, одну строку. (Вы можете также задать тайм-аут для этого выражения SQL). Вы можете указывать *WAITFOR* не только в инструкциях *SELECT*, но также и в *INSERT*, *UPDATE*, *DELETE* и *RECEIVE*. В итоге инструкция *SELECT* не будет завершена до тех пор, пока не будет выдана, по крайней мере, одна строка, а инструкции языка манипулирования данными (*data manipulation language*, *DML*) не будут завершены до тех пор, пока не будет затронута хотя бы одна строка.

Вот синтаксис:

```
WAITFOR (<statement>) [, TIMEOUT <timeout_value>]
```

Эта функция является альтернативой опроса. Например, вы можете использовать *WAITFOR* для выбора всех записей журнала или таблицы очереди, как показано здесь:

```
WAITFOR (SELECT* FROM MyQueue)
```

## Триггеры DDL

SQL поддерживает триггеры языка определения данных (data definition language, DDL). Таким образом, вы можете откатить назад действия DDL. Триггеры DDL работают синхронно (немедленно после вызывавшего срабатывание триггера события) аналогично тому, как работают триггеры DML. Триггеры DDL могут работать в пределах базы данных и реагировать на определенные типы DDL (либо на все DDL).

Триггеры DDL замечательны в том плане, что по запросу функции `EVENTDATA` вы можете получить информацию о контексте в формате XML (и в том числе о времени, подключении и пользователе), тип произошедшего события, прочие полезные данные. Если вы выдадите в триггере инструкцию `ROLLBACK`, то функция `EVENTDATA` информации не вернет. В этой ситуации до выдачи инструкции `ROLLBACK` вы должны сохранить информацию в переменной (чтобы к ней можно было обратиться позднее).

### ПРИМЕЧАНИЕ

SQL Server 2008 вводит новые функции безопасности, которые позволяют вам делать аудит действий DDL (а не использовать для этой цели триггеры DDL). Мы описываем аудит SQL Server в *главе 5*.

Показанный в листинге 2.37 триггер `AdventureWorks2008` создан на уровне базы данных и будет перехватывать попытки сделать `DROP TABLE`. Сначала мы создаем таблицу для журналирования всех наших данных по событиям (при помощи столбца XML), а затем мы создадим пустую таблицу, которую мы будем пытаться удалить (с целью проверки нашего триггера). Наш триггер будет выдавать `ROLLBACK` (что отменяет попытку `DROP TABLE`), а затем записывать данные о событии в таблицу.

#### Листинг 2.37. Перехват при помощи триггера попыток сделать `DROP TABLE`

```
-- Создать таблицу журнала
CREATE TABLE TriggerLog (LogInfo xml)

-- Создать формальную таблицу для последующего удаления
CREATE TABLE TableToDelete (Id int PRIMARY KEY)

-- Добавить некие формальные данные
INSERT INTO TableToDelete VALUES(1)

GO

-- Создать триггер, который предотвратит удаление таблицы
```

```

CREATE TRIGGER StopTableDrop ON DATABASE AFTER DROP_TABLE
AS
    DECLARE @EventData AS xml
    SET @EventData = EVENTDATA() -- must be captured *before* rollback
    ROLLBACK
    PRINT 'DROP TABLE attempt in database ' + DB_NAME() + '.'
    INSERT INTO TriggerLog VALUES(@EventData)

```

Предпринимается попытка удаления таблицы (которая благодаря триггеру DDL закончится неудачей), а затем делается запрос таблицы TriggerLog для изучения подробностей этой попытки:

```

-- Триггер в действии...
DROP TABLE TableToDelete
SELECT* FROM TriggerLog

```

Результаты выглядят следующим образом:

```

DROP TABLE attempt in database AdventureWorks2008.
(1 row(s) affected)
Msg 3609, Level 16, State 2, Line 2
The transaction ended in the trigger. The batch has been aborted.

```

Записанные (при выполнении скрипта листинга 2.37) в таблицу TriggerLog данные EventData (в формате XML) в нашем компьютере выглядели следующим образом:

```

<EVENT_INSTANCE>
  <EventType>DROP_TABLE</EventType>
  <PostTime>2008-06-11T22:07:42.910</PostTime>
  <SPID>55</SPID>
  <ServerName>SQL08DEV</ServerName>
  <LoginName>SQL08DEV\Administrator</LoginName>
  <UserName>dbo</UserName>
  <DatabaseName>AdventureWorks2008</DatabaseName>
  <SchemaName>dbo</SchemaName>
  <ObjectName>TableToDelete</ObjectName>
  <ObjectType>TABLE</ObjectType>
  <TSQLCommand>
    <SetOptions ANSI_NULLS="ON" ANSI_NULL_DEFAULT="ON" ANSI_PADDING="ON"
      QUOTED_IDENTIFIER="ON" ENCRYPTED="FALSE" />
    <CommandText>DROP TABLE TableToDelete</CommandText>
  </TSQLCommand>
</EVENT_INSTANCE>

```

## Изоляция SNAPSHOT

Для работы с транзакциями T-SQL в сервере SQL Server 2005 был введен новый уровень изоляции под названием SNAPSHOT, когда объекты записи не блокируют объекты чтения. Изоляция SNAPSHOT работает таким образом: SQL Server поддерживает в базе tempdb изменения строк и конструирует старую (зафиксированную) версию данных для доступа объектов чтения. Такой уровень изоляции полезен для оптимистического блокирования, при котором конфликты UPDATE происходят редко. Например, если процесс 1 делает выборку данных и затем пытается их модифицировать, а процесс 2 уже модифицировал эти же данные между выборкой и модификацией процесса 1, то SQL Server выдает ошибку тогда, когда процесс 1 пытается модифицировать данные (вследствие возникшего конфликта). После этого процесс 1 может попытаться перезапустить транзакцию. Такой режим работы может быть эффективным в тех ситуациях, когда конфликты UPDATE случаются нечасто.

Более подробное обсуждение уровней изоляции SQL Server и транзакций смотрите в *главе 12*.

## Возвращающие табличные значение параметры

В начале этой главы мы изучали обобщенные табличные выражения (CTE) и видели, что в SQL Server 2005 они обеспечивают альтернативу использованию временных таблиц и табличных переменных. В SQL Server 2008 возвращающие табличные значения параметры (TVP) дают нам еще один способ работы с набором строк как с единым объектом (который вы можете запрашивать и присоединять). Теперь стало очень просто отправить целый набор строк клиентских .NET-приложений на SQL Server всего за один цикл обмена с сервером (при помощи Microsoft ADO.NET). Вы увидите это ближе к концу нашего обсуждения TVP.

### **ПРИМЕЧАНИЕ**

TVP и все темы в оставшейся части данной главы — это совершенно новые функции SQL Server 2008.

## Не просто еще одна временная таблица

Легче всего будет разобраться с TVP в том случае, если сначала сравнить их с табличными переменными, временными таблицами и CTE, выявить сходства и различия. Все эти средства позволяют различными способами выполнять запросы и соединения для временного типизированного результирующего набора, что дает вам возможность во многих случаях обращаться с TVP, табличными переменными, временными таблицами и CTE точно так же, как и с обычными таблицами или представлениями.

CTE и табличные переменные хранят свои строки в памяти при условии, что эти наборы не переполняют выделенный для них кэш в оперативной памяти. Если же они его переполняют, то они вытесняются в базу tempdb. В отличие от них, TVP в SQL Server 2008 всегда хранятся в tempdb. Это требует дискового ввода-вывода (как обычные временные таблицы) и приводит к снижению производительности. В то же время это означает, что TVP лучше приспособлены к работе с большим количеством строк, чем CTE и табличные переменные, поскольку TVP можно индексировать, а CTE и табличные переменные — нет.

Реальная сила TVP в SQL Server 2008 состоит в их способности передавать целую таблицу (набор строк) как один параметр с клиента на сервер, а также между хранимыми процедурами T-SQL и определяемыми пользователем функциями. А табличные переменные и временные таблицы передавать как параметры нельзя. Область действия CTE ограничена инструкцией, следующей за его созданием (как упоминалось в начале данной главы), и поэтому CTE по своей природе не может передаваться как параметр.

Еще один побочный положительный эффект TVP — это их пригодность к повторному использованию. Схема TVP поддерживается централизованно (в случае табличных переменных, временных таблиц и CTE это не так). Вы определяете схему только один раз при помощи нового определяемого пользователем типа (user-defined type, UDT) в выражении AS TABLE к инструкции CREATE TYPE (как показано в листинге 2.38).

### Листинг 2.38. Определение схемы для определяемого пользователем табличного типа

```
CREATE TYPE CustomerUdt AS TABLE
    (Id int,
     CustomerName nvarchar(50),
     PostalCode nvarchar(50))
```

Эта инструкция создает новый определяемый пользователем табличный тип с названием `CustomerUdt` и тремя столбцами. Теперь можно объявлять переменные TVP типа `CustomerUdt` и заполнять их строками данных, соответствующими этой схеме (которые SQL Server будет сохранять в `tempdb`). Эти переменные можно свободно передавать между хранимыми процедурами. Когда переменные TVP, объявленные как `CustomerUdt`, выходят из области действия, и ссылок на них больше не делается, SQL Server автоматически удаляет соответствующие данные из `tempdb`.

### **СОВЕТ**

Схема этого табличного типа `CustomerUdt` теперь встроена в базу данных, что облегчает его многократное использование в вашем коде. Даже если вам не нужна способность TVP передавать параметры, определение этой способности облегчает быстрое объявление переменной табличного типа (без необходимости объявлять всю схему). Сравните это с табличными переменными и временными таблицами, где иногда приходится дублировать в вашем коде одну и ту же схему.

Итак, несмотря на то, что он называется возвращающим табличные значения параметром, фактически мы видим, что TVP — это по сути новый определяемый пользователем табличный тип. Этот новый табличный тип позволяет передавать свои заполненные экземпляры как параметры в хранимые процедуры и определяемые пользователем функции (а с обычными табличными переменными вы этого сделать не можете). Вы можете представить себе TVP как накачанные стероидами табличные переменные: они предоставляют схожие функциональные возможности (хранение временного набора строк), добавляют новые способности (могут передаваться как параметры) и влекут за собой дополнительные издержки (используют хранение на диске, а не в оперативной памяти).

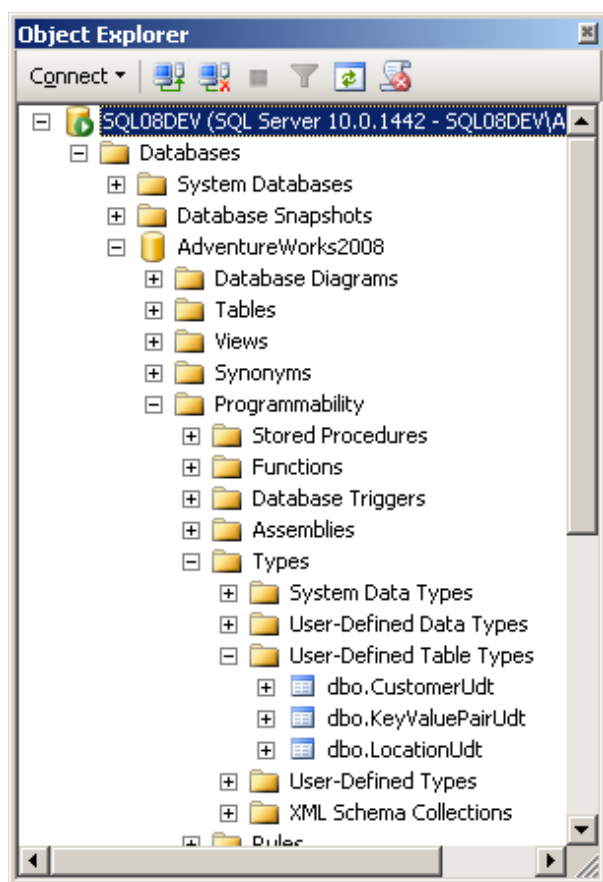
### **ПРИМЕЧАНИЕ**

В этом тексте термины "возвращающий табличные значения параметр" (TVP) и "определяемый пользователем табличный тип" используются взаимозаменяемо.

После определения табличного типа вы можете создавать хранимые процедуры с параметрами этого типа (для передачи целого набора строк при помощи TVP).

Типы TVP отображаются в окне **Object Explorer Management Studio** в новом узле **User-Defined Table Types** (как показано на рис. 2.1).





**Рис. 2.1.** Определяемые пользователем табличные типы в окне **Object Explorer**

Для этой только что появившейся возможности передавать целые наборы данных как параметры есть много практических применений, и в оставшейся части этого раздела мы изучим некоторые из них. Вы и сами, конечно же, сможете найти массу своих собственных вариантов использования TVP.

## Работаем с набором из большого количества строк

Самым обычным примером применения TVP является система приема заказов. Когда клиент размещает заказ, в базе данных необходимо создать строку

нового заказа и некоторое количество новых строк с подробностями о заказе. Традиционно это делается путем создания двух хранимых процедур: одна для вставки строки заказа, а вторая — для вставки строки с подробностями заказа. Приложение будет делать вызов хранимой процедуры для каждой отдельной строки, поэтому для заказа с 20 позициями будет сделан 21 вызов хранимых процедур (один для заказа и 20 — для подробностей). Могут быть, конечно, и заказы с более чем 20 позициями. В результате между приложением и базой данных производится множество циклов обмена, каждый из которых несет в себе всего одну строку данных. Потребуется также дополнительное программирование для обертывания всего набора операций вставки в единую атомарную транзакцию. Транзакция потребуется для обеспечения того, что заказ будет либо полностью вставлен в базу данных, либо не будет вставлен вовсе (в том случае, если в какой-то момент этого процесса произойдет ошибка).

Используем TVP. Теперь вы можете создать единственную хранимую процедуру с двумя TVP (один для строки заказа, другой — для строки подробностей заказа). Клиент теперь может делать один вызов этой хранимой процедуры, передавая ей (за один заход) и заказ, и подробности заказа (листинг 2.39). Более того, вся операция в целом либо пройдет успешно, либо закончится неудачно (избавляя от необходимости программировать для этого процесса транзакцию).

#### Листинг 2.39. Создание хранимой процедуры, которая принимает TVP

```
CREATE PROCEDURE uspInsertNewOrder
    (@OrderTvp AS OrderUdt READONLY,
     @OrderDetailsTvp AS OrderDetailUdt READONLY)
AS
    INSERT INTO [Order]
        SELECT* FROM @OrderTvp
    INSERT INTO [OrderDetail]
        SELECT* FROM @OrderDetailsTvp
```

Как вы видите, этот код вставлен в таблицы `Order` и `OrderDetails` непосредственно из строк, переданных через два TVP. По сути, мы выполнили одним вызовом операцию массовой вставки (вместо того чтобы делать множество отдельных вставок, обернутых в транзакцию). Обратите внимание, что код листинга 2.39 предполагает, что таблицы `Order` и `OrderDetails` уже существуют, а табличные типы `OrderUdt` и `OrderDetailUdt` уже были созданы (с такой схемой столбцов, которая совпадает с таблицами).

Теперь мы более подробно рассмотрим возможности массовой вставки для TVP, а также создание, объявление, заполнение и передачу TVP в T-SQL. Затем мы рассмотрим, как TVP можно использовать в качестве мощного механизма передачи параметров T-SQL. Затем мы покажем, как заполнить TVP и передать их по сети из клиентского .NET-приложения в хранимые процедуры (при помощи ADO.NET).

## Использование TVP для массовых вставок и обновлений

Здесь приведен пример хранимой процедуры, которую вы можете создать в базе данных AdventureWorks2008. Она принимает TVP и вставляет все строки (которые передаются через него) в таблицу `Product.Location`. Путем создания определенного пользователем табличного типа с названием `LocationUdt` (который описывает схему строк, переданных в хранимую процедуру) любой код может вызвать хранимую процедуру и передать ей набор строк для вставки в `Product.Location` при помощи единственного параметра (типизированного как `LocationUdt`).

Сначала мы создадим определенный пользователем табличный тип данных `LocationUdt` (как показано в листинге 2.40).

### Листинг 2.40. Создание табличного типа `LocationUdt` для использования в массовых операциях при помощи TVP

```
CREATE TYPE LocationUdt AS TABLE(  
    LocationName varchar(50),  
    CostRate int)
```

Теперь можно объявить переменную TVP этого типа (для хранения набора строк, с двумя столбцами `LocationName` и `CostRate`). Эти строки могут подаваться в хранимую процедуру. Хранимая процедура может затем делать выборку из TVP (точно так же, как из обычной таблицы или представления) и использовать в качестве источника для инструкции `INSERT INTO...SELECT`, которая добавляет все строки в таблицу `Product.Location`.

Добавленным в таблицу `Product.Location` строкам требуются не только два поля для названия местоположения и стоимости. Таблице нужны также значения по наличию и модифицированные поля дат (и пусть этим занимается хранимая процедура). Здесь мы определяем схему подмножества требуемых полей `Product.Location` (это `Name` и `CostRate`) для передачи множества строк

данных в хранимую процедуру, которая обеспечивает значения для оставшихся необходимых полей (*Availability* и *ModifiedDate*). В нашем примере хранимая процедура устанавливает значение *Availability* в 0, а значение *ModifiedDate* — в значение функции *GETDATE* для каждой строки данных, вставленных из TVP, переданного внутрь в качестве единственного параметра, которые предоставляют значения для *Name* и *CostRate* (как показано в листинге 2.41).

**Листинг 2.41. Создание хранимой процедуры для выполнения массовой вставки при помощи TVP, объявленного как табличный тип *LocationUdt***

```
CREATE PROCEDURE uspInsertProductionLocation
    (@TVP LocationUdt READONLY)
AS
    INSERT INTO [Production].[Location]
        ([Name], [CostRate], [Availability], [ModifiedDate])
    SELECT*, 0, GETDATE() FROM @TVP
```

Теперь у нас есть хранимая процедура, которая может принять TVP с набором строк названий местоположения и стоимости, и все делается при помощи единственного параметра и единственной инструкции *INSERT INTO...SELECT*! Процедура не знает и не заботится о том, как вызывающая сторона заполняет TVP перед тем, как он используется в качестве источника для инструкции *INSERT INTO...SELECT*. Например, вызывающая сторона может вручную добавлять по одной строке:

```
DECLARE @LocationTvp AS LocationUdt
INSERT INTO @LocationTvp VALUES('UK', 122.4)
INSERT INTO @LocationTvp VALUES('Paris', 359.73)
EXEC uspInsertProductionLocation @LocationTvp
```

Или же вызывающая сторона может сделать массовую вставку в TVP из другой исходной таблицы при помощи *INSERT INTO...SELECT* (как в нашем следующем примере). Мы заполним TVP из существующей таблицы *Person.StateProvince* значениями столбца *Name* для *LocationName* и 0 для *CostRate*. Передача этого TVP в нашу хранимую процедуру приведет к тому, что в *Product.Location* будет добавлен новый набор строк, у которых поле *Name* будет настроено в соответствии с именами из таблицы *Person.StateProvince*, значения *CostRate* и *Availability* будут установлены в 0, а значения их *ModifiedDate* будут установлены при помощи функции *GETDATE*, как показано здесь:

```
DECLARE @LocationTVP AS LocationUdt
INSERT INTO @LocationTVP
```

```
SELECT [Name], 0.00 FROM [Person].[StateProvince]  
EXEC uspInsertProductionLocation @LocationTVP
```

TVP может быть заполнен на клиенте при помощи ADO.NET (в конце данного раздела).

При помощи TVP можно делать также массовые обновления и удаления. Вы можете создать инструкцию UPDATE при помощи соединения TVP (которому вы должны дать псевдоним) с таблицей, которую вы хотите обновить. Обновляемые строки таблицы определяются совпадениями и могут быть установлены в новые значения, которые также содержатся в TVP. Например, вы можете передать TVP (заполненный идентификаторами категорий и названиями) для обновления таблицы Categories в базе данных (как показано в листинге 2.42). Посредством соединения TVP с таблицей Categories по идентификатору категории ID все совпадающие строки таблицы Categories могут быть обновлены новыми названиями категорий из TVP.

#### Листинг 2.42. Массовые обновления при помощи TVP

```
UPDATE Category  
SET Category.Name = ec.Name  
FROM Category INNER JOIN @EditedCategoriesTVP AS ec ON Category.Id =  
ec.Id
```

## Работаем с единственной строкой данных

Для того чтобы использовать преимущества TVP, вам не обязательно работать с множеством строк данных. TVP могут пригодиться даже тогда, когда вы имеете дело с единственной строкой, содержащей коллекцию значений столбцов типизированных параметров.

Теперь можно легко передать это множество значений из одной хранимой процедуры в другую при помощи использования единственного параметра, вместо написания и поддержки в вашем коде T-SQL дублированных сигнатур параметров. Как вы уже видели, схема TVP определяется только один раз (при помощи показанной ранее инструкции CREATE TYPE...AS TABLE) и поэтому поддерживается централизованно в одном месте для всей базы данных. Это облегчает многократное использование TVP в качестве структуры для обертывания списка типизированных параметров, передаваемой в коде как единственный параметр.

При помощи создания предыдущего TVP с названием CustomerUdt, который инкапсулирует три столбца, мы можем теперь передать три значения из од-

ной хранимой процедуры в другую при помощи одного TVP (а не использовать три отдельных параметра для каждого столбца). В случае более длинных списков параметров это может выглядеть как один параметр вместо десятка или более. Без использования TVP наша хранимая процедура могла бы выглядеть как код листинга 2.43.

**Листинг 2.43. Создание хранимой процедуры с обычным списком параметров**

```
CREATE PROCEDURE uspPassWithoutUdt (
    @Id int,
    @CustomerName nvarchar(50),
    @PostalCode nvarchar(50))
AS
BEGIN
    -- Использовать параметры
    SELECT @Id, @CustomerName, @PostalCode
END
```

Вызов этой хранимой процедуры выглядел бы примерно так:

```
EXEC uspPassWithoutUdt 1, 'Christian Hess', '23911'
```

Теперь представьте себе множество хранимых процедур с одинаковым списком параметров. При этом вам пришлось бы одни и те же три параметра копировать и вставлять, дублировать и поддерживать — и так в каждой хранимой процедуре. Использование TVP избавляет вас от этой обузы, поскольку схема определяется централизованно и только один раз (в определении табличного типа, определяемого пользователем, см. листинг 2.38).

После определения этого табличного типа наши хранимые процедуры уже имеют гораздо более простые сигнатуры и не требуют от нас дублирования списка параметров. Та же самая хранимая процедура может быть теперь реализована, как в листинге 2.44.

**Листинг 2.44. Создание хранимой процедуры со встроенными в TVP значениями параметров**

```
CREATE PROCEDURE uspPassWithUdt (@CustomerTvp CustomerUdt READONLY)
AS
BEGIN
    -- Извлечь значения из TVP
    SELECT Id, CustomerName, PostalCode FROM @CustomerTvp
END
```

Для вызова этой хранимой процедуры мы объявляем переменную TVP типа `CustomerUdt`, вставляем ее в единственную строку данных, содержащую три значения, а затем передаем единственную переменную в хранимую процедуру, как показано здесь:

```
DECLARE @CustomerTvp CustomerUdt
INSERT INTO @CustomerTvp(Id, CustomerName, PostalCode)
VALUES(1, 'Christian Hess', '23911')
EXEC uspPassWithUdt @CustomerTvp
```

Это по логике аналогично созданию (в мире .NET) экземпляра класса или структуры, заполнению его свойств или полей и передаче этого экземпляра в вызов метода, который затем извлекает необходимые ему значения из свойств или полей. Такой подход с "упаковкой параметров" обычно приводит к более четкому и более простому в обслуживании коду, независимо от того, написан ли он на .NET или на T-SQL (в особенности для длинных и часто используемых списков параметров).

Если вы позаботитесь об этом заранее, то сможете спроектировать эти списки параметров таким образом, чтобы они выносили в будущем добавление новых столбцов в схему определяемого пользователем табличного типа (без изменения существующего кода T-SQL, который ссылается на существующие столбцы). Для полной изоляции кода от расширений схемы списка параметров вам придется уделить особое внимание вашему стилю кодирования при заполнении значениями — избегать инструкций `INSERT`, которые не имеют явного списка столбцов. Инструкции `INSERT` для заполнения TVP должны всегда использовать явный список столбцов (чтобы добавляемые в будущем новые столбцы не влияли на их поведение и не вызывали ошибок).

### **СОВЕТ**

Вообще использование в инструкциях `INSERT` явных списков столбцов — хорошая практика.

Инструкции `INSERT` без явного списка столбцов адаптируются динамически по отношению к схеме таблицы, в которую происходит вставка. В результате добавления новых столбцов в таблицу инструкция `INSERT` без явного списка столбцов (которая раньше работала) теперь точно даст сбой, поскольку количество указанных в выражении `VALUES` значений больше не соответствует количеству столбцов таблицы. Для того чтобы обеспечить работоспособность вставки, невзирая на будущие расширения схемы, следует предоставлять явный список столбцов для установления соответствий указанных значений `VALUES`. Это позволит вам вставлять новые столбцы произвольно в

схеме (а не только добавлять в конец), поскольку наличие явного списка столбцов означает отсутствие привязки к количеству (или порядку) столбцов в табличном типе. Вам также придется обеспечить, чтобы добавляемые в схему новые столбцы либо допускали неопределенные значения, либо имели присвоенные им значения по умолчанию (чтобы существующие инструкции INSERT продолжали правильно работать без необходимости предоставления значений для добавленных новых столбцов).

Такая стратегия ставит вас в выгодное положение, поскольку теперь вы управляете ситуацией и можете сами решать, как, когда и где использовать значения, добавленные недавно в списки ваших типизированных параметров. Затем вы можете принять решение использовать новые значения только в новом коде T-SQL, который должен будет поддерживать их (и при этом вам не придется возвращаться к существующему коду или модифицировать его, поскольку он будет продолжать нормально работать). Вы можете также постепенно и выборочно обновить любой существующий код (по мере необходимости или желания).

Если вы кодировали именно таким безопасным способом (как мы только что описали), то вы готовы безопасно расширить определение таблицы CustomerUdt путем добавления четвертого столбца. Не существует такой инструкции, как ALTER TYPE...AS TABLE, поэтому вам придется удалить таблицу и потом вновь ее создать. Поскольку SQL Server отслеживает и обеспечивает целостность зависимостей объектов в базе данных, то сначала придется удалить все хранимые процедуры, которые ссылаются на табличный тип, затем удалить сам табличный тип, пересоздать табличный тип с новой схемой, а потом пересоздать удаленные хранимые процедуры. Это может (и, наверное, должно) охладить ваше стремление слишком часто расширять списки параметров в схемах ваших TVP, хотя задача облегчается возможностью без труда генерировать все необходимые скрипты для пересоздания всех программируемых объектов (хранимых процедур, TVP и т. д.) базы данных. Код листинга 2.45 удаляет и пересоздает табличный тип и хранимую процедуру.

#### **Листинг 2.45. Пересмотр схемы табличного типа CustomerUdt**

```
DROP PROCEDURE uspPassWithUdt
DROP TYPE CustomerUdt
GO
CREATE TYPE CustomerUdt AS TABLE
    (Id int,
     CustomerName nvarchar(50),
```



```
    HomePhone nvarchar(50), -- inserted new column
    PostalCode nvarchar(50))
GO
CREATE PROCEDURE uspPassWithUdt (@CustomerTvp CustomerUdt READONLY)
AS
BEGIN
    -- Извлечение значений из TVP
    SELECT Id, CustomerName, PostalCode FROM @CustomerTvp
END
```

Инструкция `SELECT` в хранимой процедуре не модифицировалась и поэтому продолжает использовать только исходные три столбца. Если бы инструкция `SELECT` была закодирована как `SELECT*` (то есть без явного списка столбцов), то теперь она выбирала бы и новый столбец `HomePhone` (без всякой модификации с вашей стороны). Вы можете в каждом конкретном случае решить, использовать ли вам `SELECT*` или `SELECT` с явным списком столбцов (`SELECT*` считается плохой практикой для промышленного кода). Для инструкций `INSERT` (как мы уже объясняли) вам следует всегда использовать явный список столбцов, это будет безопасным кодированием на случай расширений схемы TVP.

## Создание TVP в стиле словаря

Вы можете использовать TVP в реализации обобщенной словарной структуры (в качестве еще одного механизма гибкой передачи параметров). Этот способ аналогичен заполнению объекта `Dictionary` в .NET (строка в качестве ключа и объект, который может содержать данные любого типа, в качестве значения) и последующей передаче этого объекта словаря в другой метод. Достижение такого результата при помощи T-SQL в прошлом потребовало бы составления строки (содержащей все данные) с одним ограничителем, отделяющим каждую пару, и другим ограничителем, разделяющим ключ и значение внутри каждой пары. Хранимая процедура должна была бы принять эту строку, а затем разобрать по разделителям (для извлечения значений и приведения их к соответствующим типам). Использовать для этого встроенные функции манипулирования строками T-SQL утомительно и неудобно. Начиная с SQL Server 2005, ситуация несколько улучшилась путем использования типа данных `xml` и составления XML-массива, который передавался в хранимую процедуру, где этот XML разбирался при помощи `XQuery`. В такой ситуации TVP также может обеспечить гораздо более простое решение.

Сначала создайте простой определяемый пользователем табличный тип с названием `KeyValuePairUdt` (как показано в листинге 2.46).

**Листинг 2.46. Создание табличного типа для использования при хранении и передаче пар ключ—значение**

```
CREATE TYPE KeyValuePairUdt AS TABLE
    (K nvarchar(50),
     V sql_variant)
```

Этот табличный тип определяет два столбца `k` и `v`, которые могут содержать строковый ключ и значение любого типа. (Тип данных `sql_variant` является универсальным типом, который может содержать в SQL Server данные почти любого типа). Затем объявляем TVP, заполняем его несколькими парами ключ—значение и передаем его в вашу хранимую процедуру:

```
DECLARE @Username AS nvarchar(50)
DECLARE @DOB AS date
DECLARE @IsActive AS bit
SET @Username = 'Admin'
SET @DOB = '1/2/2006'
SET @IsActive = 1
DECLARE @ParamsPackage AS KeyValuePairUdt
INSERT INTO @ParamsPackage(K, V) VALUES('Username', @Username)
INSERT INTO @ParamsPackage(K, V) VALUES('DOB', @DOB)
INSERT INTO @ParamsPackage(K, V) VALUES('IsActive', @IsActive)
EXEC uspProcessEntry 392, @ParamsPackage
```

Здесь у нас есть три значения, присвоенные переменным (имя пользователя, дата рождения, флаг активности) с типами `nvarchar(50)`, `date` и `bit` соответственно. Затем мы объявляем TVP с названием `@ParamsPackage` (типизированный как наш новый табличный тип `KeyValuePairUdt`), в который мы вставляем все три значения разных типов с соответствующими строковыми ключами `Username`, `DOB` и `IsActive`. Поскольку ключевой столбец `k` в `KeyValuePairUdt` имеет тип `nvarchar`, то ключи должны быть строками. Однако путем объявления столбца значений `v` как `sql_variant` мы можем хранить смесь разных типов данных в качестве значений в каждой строке нашего TVP с названием `@ParamsPackage`.

Мы вызываем хранимую процедуру `uspProcessEntry` (как показано в листинге 2.47), передавая два параметра: обычное целое `EntryId` и TVP, содержащий наш словарь пар ключ—значение. Хранимая процедура принимает первый параметр как нормальное целое значение. Для извлечения элементов

из словаря (переданного вторым параметром) хранимая процедура выбирает их из TVP по одному по строковым ключам, настроенным вызывающей стороной (Username, DOB и IsActive). Вследствие использования `sql_variant` хранимая процедура может извлечь значения с разными типами данных.

**Листинг 2.47. Создание хранимой процедуры, которая принимает словарь значений при помощи TVP**

```
CREATE PROCEDURE uspProcessEntry(
    @EntryId AS int,
    @KeyValuePairTvp AS KeyValuePairUdt READONLY)
AS
BEGIN
    SET NOCOUNT ON
    DECLARE @Username AS nvarchar(max)
    DECLARE @DOB AS date
    DECLARE @IsActive AS bit
    SELECT @Username = CONVERT(nvarchar, V)
        FROM @KeyValuePairTvp WHERE K = 'Username'
    SELECT @DOB = CONVERT(date, V)
        FROM @KeyValuePairTvp WHERE K = 'DOB'
    SELECT @IsActive = CONVERT(bit, V)
        FROM @KeyValuePairTvp WHERE K = 'IsActive'
    PRINT 'EntryId: ' + CAST(@EntryId AS nvarchar)
    PRINT 'Username: ' + @Username
    PRINT 'DOB: ' + CAST(@DOB AS nvarchar)
    PRINT 'IsActive: ' + CAST(@IsActive as nvarchar)
END
```

## Передача TVP при помощи ADO.NET

Вероятно, наиболее впечатляющим аспектом TVP является его способность проводить множество строк данных из клиентского приложения в SQL Server 2008 (без необходимости выполнения большого количества циклов обмена или реализации на сервере специальной логики для обработки данных). Мы завершим наше обсуждение TVP новым перечислением `SqlDbType.Structured` в ADO.NET, которое облегчает этот процесс.

Подготовьте объект `SqlCommand` (как вы уже делали это раньше), настройте его свойство `CommandType` в `CommandType.StoredProcedure` и заполните его

коллекцию `Parameters` объектами `SqlParameter`. Для того чтобы обозначить `SqlParameter` как TVP, необходимо установить его свойство `SqlDbType` в значение `SqlDbType.Structured`. Затем вы сможете указать любой объект `DataTable`, `DbDataReader` или `ICollection<SqlDataRecord>` в качестве значения параметра (для передачи в хранимую процедуру на сервер одним вызовом).

В листинге 2.48 новый заказ клиента хранится в отдельных объектах `Order` и `OrderDetail` типа `DataTable` внутри `DataSet`. Эти две таблицы передаются в хранимую процедуру SQL Server, которая принимает их как TVP для вставки в таблицы `Order` и `OrderDetail` базы данных.

#### Листинг 2.48. Передача TVP в хранимую процедуру SQL Server из ADO.NET

```
// Предполагается, что conn — это открытый объект SqlConnection,
// а ds — это DataSet с таблицами Order и OrderDetails
using (conn)
{
    // Создать командный объект для вызова хранимой процедуры
    SqlCommand cmd = new SqlCommand("uspInsertNewOrder", conn);
    cmd.CommandType = CommandType.StoredProcedure;
    // Создать параметр для передачи TVP с названием Order
    SqlParameter headerParam = cmd.Parameters.AddWithValue(
        "@OrderTvp", ds.Tables["Order"]);
    // Создать параметр для передачи TVP с названием OrderDetails
    SqlParameter detailsParam = cmd.Parameters.AddWithValue(
        "@OrderDetailsTvp", ds.Tables["OrderDetail"]);
    // Установить тип SqlDbType параметров в значение Structured
    headerParam.SqlDbType = SqlDbType.Structured;
    detailsParam.SqlDbType = SqlDbType.Structured;
    // Выполнить хранимую процедуру
    cmd.ExecuteNonQuery();
}
```

Этот код вызывает хранимую процедуру SQL Server 2008 и передает ей заголовок заказа и полный набор подробностей заказа (за один цикл обмена с сервером в одной неявной транзакции). Замечательно то, что все это так просто.

Вы можете также послать набор строк непосредственно в параметризованную инструкцию SQL (без создания хранимой процедуры). Поскольку инструкция SQL динамически конструируется на клиенте, то не существует сигнатуры хранимой процедуры, которая указывала бы TVP название

определяемого пользователем табличного типа. Поэтому вам необходимо указать ADO.NET этот тип (путем настройки свойства `TypeName` в имя табличного типа, определенного на сервере). Например, код листинга 2.49 передает `DataTable` в параметризованную инструкцию SQL.

**Листинг 2.49. Передача TVP в параметризованную инструкцию SQL из ADO.NET**

```
// Определить INSERT INTO... SELECT инструкцию для вставки в Categories
const string TSqlStatement =
    "INSERT INTO Categories (CategoryID, CategoryName)" +
    " SELECT nc.CategoryID, nc.CategoryName" +
    " FROM @NewCategoriesTvp AS nc";
// Подразумевается, что conn — это открытый объект SqlConnection,
// а ds — это DataSet с таблицей Category
using(conn)
{
    // Настроить командный объект для инструкции
    SqlCommand cmd = new SqlCommand(TSqlStatement, conn);
    //добавить TVP, указывающий DataTable как значение параметра
    SqlParameter catParam = cmd.Parameters.AddWithValue
        ("@NewCategoriesTvp", ds.Tables["Category"]);
    catParam.SqlDbType = SqlDbType.Structured;
    catParam.TypeName = "dbo.CategoriesUdt";
    // Выполнить команду
    cmd.ExecuteNonQuery();
}
```

Настройка в этом коде свойства `TypeName` в значение `dbo.CategoriesUdt` означает, что у вас есть определяемый пользователем табличный тип с этим именем на сервере (созданный при помощи инструкции `CREATE TYPE...AS TABLE`, которая определяет столбцы `CategoryID` и `CategoryName`).

Вы можете также использовать любой производный от `DbDataReader` объект для направления потока строк данных в TVP. В листинге 2.50 мы сначала вызываем хранимую процедуру `Oracle` (для выборки из базы данных `Oracle` в `OracleDataReader`). Объект считывателя передается в хранимую процедуру `SQL Server 2008` как возвращающий табличные значения входной параметр, который затем может использовать данные `Oracle` из считывателя как источник для добавления новых строк в таблицу `Category` базы данных `SQL Server`.

**Листинг 2.50. Передача подключенного источника OracleDataReader как TVP в SQL Server**

```
// Настроить командный объект для выборки из Oracle
OracleCommand selCmd = new OracleCommand
    ("SELECT CategoryID, CategoryName FROM Categories;", oracleConn);
// Выполнить команду и вернуть результаты в подключенный считыватель,
// который автоматически закроет подключение по завершении
OracleDataReader rdr = selCmd.ExecuteReader
    (CommandBehavior.CloseConnection);
// Настроить командный объект для вставки в SQL Server
SqlCommand insCmd = new SqlCommand
    ("uspInsertCategories", connection);
insCmd.CommandType = CommandType.StoredProcedure;
// Добавить TVP, указывающий считыватель как значение параметра
SqlParameter catParam = cmd.Parameters.AddWithValue
    ("@NewCategoriesTvp", rdr);
catParam.SqlDbType = SqlDbType.Structured;
// Выполнить хранимую процедуру
insertCommand.ExecuteNonQuery();
```

## Ограничения TVP

TVP имеет некоторые ограничения, о которых вы должны знать. Прежде всего, TVP после начального заполнения и передачи можно использовать только для чтения и нельзя применять для возвращения данных. В сигнатурах ваших хранимых процедур для TVP необходимо применять ключевое слово `READONLY` (а иначе они не скомпилируются). Можно использовать также и ключевое слово `OUTPUT`. Вы не можете обновлять значения столбцов в строках TVP, вы не можете также вставлять или удалять строки. Если вам необходимо модифицировать данные в TVP, то для этого можно вставить данные из TVP во временную таблицу (или в табличную переменную) и уже там выполнить изменения.

Не существует инструкции `ALTER TABLE...AS TYPE`, которая поддерживала бы изменение схемы табличного типа TVP. Вместо этого перед удалением типа вы должны сначала удалить все хранимые процедуры (которые ссылаются на данный тип), затем пересоздать тип с новой схемой, а потом пересоздать хранимые процедуры. Индексирование также ограничено, поддерживаются только ограничения `PRIMARY KEY` и `UNIQUE`. SQL Server не поддерживает статистики по TVP.

## Новые типы данных для даты и времени

Типы `date`, `time`, `datetime2` и `datetimeoffset` — это четыре новых типа данных в SQL Server 2008 для хранения данных и времени, которые вам теперь следует использовать для разработки новых баз данных вместо традиционных типов данных `datetime` и `smalldatetime`. Новые типы гораздо лучше соответствуют стандартам .NET Framework, Microsoft Windows и SQL и имеют важные преимущества в диапазоне, точности и хранении.

Кроме того, SQL Server 2008 предоставляет поддержку поставщиков Open Database Connectivity (ODBC), OLE-DB и ADO.NET для всех этих четырех типов данных. Они совместимы со всеми остальными компонентами SQL Server, в том числе клиентскими инструментами, службами интеграции, репликацией, службами отчетности и аналитическими службами.

## Разделение даты и времени

Мы начнем с рассмотрения новых типов `date` и `time`. Разработчики давно требовали возможности хранить дату и время в виде отдельных типов данных, и, наконец, в SQL Server 2008 такая возможность обеспечена. Если вам необходимо хранить только дату (например, дату рождения), используйте новый тип `date`. Для хранения только значения времени (например, времени ежедневного приема лекарств) используйте новый тип `time`, как показано здесь:

```
DECLARE @DOB date
DECLARE @MedsAt time
```

Прежние типы `datetime` и `smalldatetime` включают и дату, и время. В тех случаях, когда нужна только дата или только время, лишняя составляющая только занимает место хранения. Кроме экономии места, использование `date` вместо `datetime` дает более высокую производительность при вычислении дат и манипуляциях с ними (поскольку нет необходимости обрабатывать составляющую времени).

### **ПРИМЕЧАНИЕ**

Появление отдельных типов данных `date` и `time` планируется и в следующей версии .NET Framework. До их появления новым типам `date` и `time` сервера SQL Server 2008 соответствуют типы `System.DateTime` и `System.TimeSpan`.

## Дополнительные переносимые дата и время

Для того чтобы продолжить хранение даты и времени как единого значения, используйте новый тип данных `datetime2`. Этот новый тип поддерживает такой же диапазон значений, что и тип данных `DateTime` в .NET Framework, поэтому он может хранить даты с 1/1/1001 (это значение `DateTime.MinValue` в .NET) и до 12/31/9999 (это `DateTime.MaxValue` в .NET) по григорианскому календарю. Сравните это с допустимыми значениями дат для обычного типа `datetime`, которые имеют диапазон с 1/1/1753 до 12/31/9999. Это означает, что данные в .NET с 1/1/1001 до 12/31/1752 не могут храниться в типе `datetime`. Эта проблема решается в SQL Server 2008 путем использования типов `date`, `datetime2` или `datetimeoffset`. Поскольку поддерживаемый диапазон дат теперь одинаков как в .NET, так и в SQL Server, то любую дату можно передать между клиентом и сервером без всяких дополнительных осложнений. Мы вам настоятельно советуем не использовать в дальнейшем старые типы данных `datetime` и `smalldatetime`, а использовать в новых разработках только типы `date` и `datetime2` (или тип `datetimeoffset` для часовых поясов, описанный далее).

### ПРИМЕЧАНИЕ

Стандарт SQL определяет тип данных `datetime2` как тип данных `timestamp`. К несчастью, компания Microsoft уже использовала понятие `timestamp` для специального типа данных (генерирующего уникальные двоичные значения), который часто используется для управления версиями строк, но фактически не содержит ни даты, ни времени. Здесь SQL Server 2008 отклоняется от стандарта SQL, поскольку называет свой новый тип `datetime2` и одновременно вводит новый синоним для неудачно выбранного типа `timestamp`, который теперь назван `rowversion`. Это препятствует дальнейшему использованию понятия `timestamp` в первоначальном значении, приданном ему компанией Microsoft.

Вы не одиноки, если вам кажется, что `datetime2` — это еще одно неудачное название нового типа, созданного как замена `datetime`. Не имея возможности использовать название `timestamp`, компания Microsoft явно испытывала затруднения с новым названием и в итоге просто остановилась на `datetime2`. Не удивляйтесь, если в будущем для этого типа появится новый синоним (когда кому-то в Редмонде посчастливится выбрать удачное название!).

Тип `datetime` имеет точность примерно в 3,33 мс, в то время как время в Windows и .NET имеет значительно более высокую точность (до 100 нс, это



одна десятимиллионная часть секунды). Тип `smalldatetime` даже не поддерживает секунды и имеет точность до минуты. Таким образом, сохранение времени в базе данных приводит к потере точности.

Новые типы `time`, `datetime2` и `datetimeoffset` соответствуют возможностям .NET (и других платформ) и предоставляют ту же 100-наносекундную точность. В результате у нас не происходит потери точности долей секунд при записи значений в базу данных. Конечно, большая точность времени влечет за собой увеличение размера хранения (что мы вскоре и обсудим).

## Часовые пояса

Четвертый и последний новый тип данных, введенный SQL Server 2008 в этой категории — это `datetimeoffset`. Этот тип определяет дату и время с тем же диапазоном значений и с той же точностью, что и `datetime2`, но он содержит также и значение смещения в диапазоне от `-14:00` до `+14:00` (которое определяет часовой пояс). В прошлом единственным практичным способом глобализации даты и времени в базе данных было сохранение их в формате универсального глобального времени (Coordinated Universal Time, UTC). Для этого требуется преобразование из местного времени в UTC на уровне приложения, для которого нужно писать код.

При помощи нового типа `datetimeoffset` вы можете теперь сохранять значения, которые представляют местную дату и время в различных регионах мира, и включать соответствующий сдвиг часового пояса в каждое значение. Поскольку сдвиг часового пояса (содержащийся в значении даты и времени) специфичен для конкретного языкового стандарта (locale), то SQL Server может выполнять сравнения даты и времени между разными стандартами (без всяких усилий по преобразованию с вашей стороны). Хотя значения `datetimeoffset` используются как местные для данного региона, однако для операций сравнения, сортировки и индексирования они внутренним образом преобразуются в формат UTC.

Поэтому вычисления и сравнения производятся правильно для даты и времени в базе данных (независимо от различных часовых поясов разных регионов). Простым добавлением сдвига часового пояса к значениям `datetimeoffset` сервер SQL Server осуществляет автоматическое преобразование в UTC и обратно в фоновом режиме. Что еще лучше, вы можете получить значение `datetimeoffset` как UTC или как местное время. Для тех баз данных, которым необходимо хранить различное местное время (или даже

просто даты) в различных регионах мира, это исключительно удобная новая функциональная возможность SQL Server 2008. Теперь база данных сама работает со всеми деталями, и разработчику не приходится этим заниматься. Функциональные возможности часовых поясов теперь просто доступны на уровне базы данных.

Например, база данных знает, что 9:15 утра в Нью-Йорке — это позднее, чем 10:30 утра в Лос-Анджелесе (если вы сохраните значения в типе данных `datetimeoffset` с соответствующими сдвигами часовых поясов). Поскольку нью-йоркское время указывает сдвиг часового пояса `-5:00`, а время Лос-Анджелеса имеет сдвиг `-8:00`, то SQL Server знает о трехчасовой разнице между этими двумя часовыми поясами и учитывает эту разницу во всех вычислениях даты/времени и манипуляциях с ними (листинг 2.51).

**Листинг 2.51. Вычисления часовых поясов при помощи `datetimeoffset`**

```
DECLARE @Time1 datetimeoffset
DECLARE @Time2 datetimeoffset
DECLARE @MinutesDiff int
SET @Time1 = '2007-11-10 09:15:00-05:00' -- NY time is UTC -05:00
SET @Time2 = '2007-11-10 10:30:00-08:00' -- LA time is UTC -08:00
SET @MinutesDiff = DATEDIFF(minute, @Time1, @Time2)
SELECT @MinutesDiff
```

Вот результат этого кода:

```
-----
255
(1 row(s) affected)
```

SQL Server четко учел разницу в три часа между часовыми поясами. Поскольку 10:30 утра в Лос-Анджелесе — это фактически 1:30 дня в Нью-Йорке, то разница между этим временем и временем 9:15 в Нью-Йорке была вычислена правильно (255 минут, то есть 4 часа и 15 минут).

### **ПРИМЕЧАНИЕ**

Имена часовых поясов не поддерживаются, нет также и поддержки летнего времени. К несчастью, эти функции не попали в финальную версию продукта (но они числятся в списке для следующей версии SQL Server). Часовые пояса можно выражать только в виде смещений в часах/минутах, и летнее время вам пока придется учитывать самостоятельно.

Все эти функциональные возможности .NET Framework теперь предоставляет в новом типе `System.DateTimeOffset`. Это означает, что клиентские приложения .NET и SQL Server могут передавать друг другу "осведомленные" о часовом поясе значения.

### **ПРИМЕЧАНИЕ**

SQL Server 2008 предоставляет также поддержку часовых поясов и для типа `xsd:dateTime`. Мы описываем это улучшение схемы Extensible Schema Definition (XSD) в *главе 6*.

## **Дата и время: точность, хранение и формат**

Хранимые в типах `date`, `datetime2` и `datetimeoffset` значения дат упаковываются в фиксированный размер 3 байта. Они используют оптимизированный формат, который на один байт короче, чем те 4 байта, которые занимает дата в старом типе `datetime` (поддерживается больший диапазон в меньшем размере).

Хранимые в типах `time`, `datetime2` и `datetimeoffset` значения времени по умолчанию занимают 5 байтов (для поддержания той же 100-наносекундной точности, что в Windows и в .NET). Однако специалисты компании Microsoft не стали отвергать потребностей (в смысле минимального размера хранения) тех разработчиков и администраторов баз данных, которым не нужна высочайшая точность с долями секунды. Для того чтобы выиграть в размере хранения, вы можете указать более низкую точность (необязательный параметр масштаба при объявлении переменных `time`, `datetime2` и `datetimeoffset`). Масштаб может варьироваться от 0 до 7, причем 0 не дает никаких долей секунды (и занимает всего 3 байта), а 7 (значение по умолчанию) дает максимальную точность в 100 нс (и занимает 5 байтов). Масштаб, по существу, определяет количество цифр, поддерживаемых после десятичной точки (значение 7 соответствует точности 100 нс, так как 100 нс соответствует 0,0000001 с).

В табл. 2.2 показан размер хранения и точность значений `time`, `datetime2` и `datetimeoffset` для разных значений масштаба (от 0 до 7). Значение в столбце точности показывает число символов, содержащихся в сформатированной по правилам организации International Organization for Standardization (ISO) строке, представляющей данный тип.

**Таблица 2.2.** Размер хранения и точность для новых типов даты и времени

| time              |       |          | datetime2 |          | datetimeoffset |          |
|-------------------|-------|----------|-----------|----------|----------------|----------|
| Значение масштаба | Байты | Точность | Байты     | Точность | Байты          | Точность |
| 0                 | 3     | 8        | 6         | 19       | 8              | 26       |
| 1                 | 3     | 10       | 6         | 21       | 8              | 28       |
| 2                 | 3     | 11       | 6         | 22       | 8              | 29       |
| 3                 | 4     | 12       | 7         | 23       | 9              | 30       |
| 4                 | 4     | 13       | 7         | 24       | 9              | 31       |
| 5                 | 5     | 14       | 8         | 25       | 10             | 32       |
| 6                 | 5     | 15       | 8         | 26       | 10             | 33       |
| 7                 | 5     | 16       | 8         | 27       | 10             | 34       |

Значение масштаба по умолчанию равно 7, что дает самую высокую точность (до 100 нс) и самый большой размер хранения. Объявление переменной как `time`, `datetime2` или `datetimeoffset` — это то же, что `time(7)`, `datetime2(7)` или `datetimeoffset(7)`. Поэтому следующие два оператора эквивалентны:

```
DECLARE @StartDateTime datetime2
```

```
DECLARE @StartDateTime datetime2(7)
```

Если вам не нужна точность в долях секунды, используйте для масштаба значение 0:

```
DECLARE @FeedingTime time(0)
```

Как показано в табл. 2.2, только 3 байта потребуется для хранения времени `@FeedingTime` — точность только до секунды.

Два значения с различными масштабами вполне сравнимы. SQL Server автоматически преобразует значение с более низким значением масштаба к значению с более высоким значением масштаба и сравнивает их.

Для представления дат и времени поддерживаются почти все стандартные форматы строковых литералов. Например, дата "15 мая 2008 года" может быть представлена в любом из форматов, показанных в табл. 2.3.

**Таблица 2.3. Форматы строковых литералов даты и времени**

| Формат     | Пример                            |
|------------|-----------------------------------|
| Числовой   | 5/15/2008, 15-05-2008, 05.15.2008 |
| Алфавитный | May 15, 2008                      |
| ISO8601    | 2008-05-15, 200805153             |
| ODBC       | {d'2008-05-15'}                   |
| W3C XML    | 2008-05-15Z                       |

Такая же гибкость есть и в отношении представления времени. Например, одно и то же значение времени может быть представлено как 23:30, 23:30:00, 23:30:00.0000 или 11:30:00 PM. Сдвиги часовых поясов выражаются в виде плюса или минуса, затем следует число часов и минут UTC для данного пояса — например, +02:00 для Иерусалима.

### **ПРИМЕЧАНИЕ**

Кроме указанных, есть и другие возможные варианты форматирования. Целью табл. 2.3 было показать гибкость SQL Server к вариантам синтаксиса даты и времени. Полную спецификацию поддерживаемых форматов литералов даты и времени можно получить в SQL Server Books Online.

Для извлечения при поиске из столбца `datetime2` только даты или только времени вы можете использовать опции `CAST` или `CONVERT`. Когда вы делаете такое преобразование для индексированного столбца `datetime2`, то серверу SQL Server нет необходимости делать последовательный просмотр таблицы, и он может выполнить гораздо более быстрый индексный поиск для нахождения конкретной даты или времени. Например, в листинге 2.52 определяется таблица с типом `datetime2`, который имеет кластеризованный индекс. Выборку только по дате или только по времени можно сделать при помощи опции `CONVERT` (продолжая использовать кластеризованный индекс для эффективности поиска).

#### **Листинг 2.52. Использование опции `CONVERT` для извлечения даты и времени из столбца `datetime2`**

```
CREATE TABLE DateList(MyDate datetime2)
CREATE CLUSTERED INDEX idx1 ON DateList (MyDate);
-- Вставить несколько строк в DateList...
```

```
SELECT MyDate FROM DateList WHERE CONVERT(date, MyDate) = '2005-04-07';  
SELECT MyDate FROM DateList WHERE CONVERT(time(0), MyDate) = '09:00:00';
```

## Новые и измененные функции

Все традиционные функции для работы с датами и временем (включая DATEADD, DATEDIFF, DATEPART и DATENAME) теперь поддерживают новые введенные сервером SQL Server 2008 типы даты и времени. Было добавлено также несколько новых функций. Мы завершаем наше обсуждение новых типов даты и времени рассмотрением расширений T-SQL, добавленных для их поддержки.

Новые функции SYSDATETIME и SYSUTCDATETIME возвращают дату и время на сервере в виде типов datetime2 (с полной точностью до 100 нс), а функции GETDATE и GETUTCDATE продолжают возвращать текущую дату и время в виде типов datetime. Еще одна новая функция SYSDATETIMEOFFSET возвращает дату и время на сервере в виде типа datetimeoffset, причем сдвиг часового пояса отражает местные настройки сервера (который имеет представление о местном летнем времени). Код в листинге 2.53 показывает разницу между различными функциями даты и времени.

### Листинг 2.53. Сравнение функций даты и времени на сервере

```
SET NOCOUNT ON  
SELECT GETDATE() AS 'GETDATE() datetime'  
SELECT GETUTCDATE() AS 'GETUTCDATE() datetime'  
SELECT SYSDATETIME() AS 'SYSDATETIME() datetime2'  
SELECT SYSUTCDATETIME() AS 'SYSUTCDATETIME() datetime2'  
SELECT SYSDATETIMEOFFSET() AS 'SYSDATETIMEOFFSET() datetimeoffset'
```

Выполнение этого кода в 8:20 вечера 10 ноября 2007 года в Нью-Йорке даст следующий вывод:

```
GETDATE() datetime  
-----  
2007-11-10 20:21:19.380  
GETUTCDATE() datetime  
-----  
2007-11-11 01:21:19.380  
SYSDATETIME() datetime2  
-----
```

```

2007-11-10 20:21:19.3807984
SYSUTCDATETIME() datetime2
-----
2007-11-11 01:21:19.3807984
SYSDATETIMEOFFSET() datetimeoffset
-----
2007-11-10 20:21:19.3807984 -05:00

```

Есть также новые функции `TODATETIMEOFFSET` и `SWITCHOFFSET`, которые позволяют выполнять манипуляции со сдвигами часовых поясов. `TODATETIMEOFFSET` преобразует любой тип даты или времени (не имеющий сдвига часового пояса) в тип `datetimeoffset` (путем применения к нему любого указанного вами сдвига часового пояса). `SWITCHOFFSET` облегчает определение времени в другом часовом поясе. Вы указываете `datetimeoffset` для исходного местоположения и сдвиг часового пояса для целевого местоположения, а `SWITCHOFFSET` возвращает `datetimeoffset`, представляющий соответствующие дату и время в целевом местоположении (как указано в листинге 2.54).

**Листинг 2.54. Выполнение манипуляций со сдвигом часовых поясов при помощи `TODATETIMEOFFSET` и `SWITCHOFFSET`**

```

DECLARE @TheTime datetime2
DECLARE @TheTimeInNY datetimeoffset
DECLARE @TheTimeInLA datetimeoffset
-- Время без указания часового пояса
SET @TheTime = '2007-11-10 7:35PM'
-- Преобразовать его во время с указанием часового пояса Нью-Йорка
SET @TheTimeInNY = TODATETIMEOFFSET(@TheTime, '-05:00')
-- Вычислить эквивалентное время в Лос-Анжелесе
SET @TheTimeInLA = SWITCHOFFSET(@TheTimeInNY, '-08:00')
SELECT @TheTime AS 'Any Time'
SELECT @TheTimeInNY AS 'NY Time'
SELECT @TheTimeInLA AS 'LA Time'

```

Вот получившийся результат:

```

Any Time
-----
2007-11-10 19:35:00.0000000
NY Time
-----

```

```
2007-11-10 19:35:00.0000000 -05:00
LA Time
-----
2007-11-10 16:35:00.0000000 -08:00
```

Вы можете использовать `TODATETIMEOFFSET` вместе с `INSERT INTO...SELECT` для массовой вставки значений даты и времени (без информации о часовом поясе) из исходной таблицы в таблицу назначения и применения сдвига часового пояса с целью получения значений `datetimeoffset` в таблице назначения. Например, следующий код копирует все значения строк из столбца `dt2` таблицы `test1` (типа `datetime2`, который не имеет информации о часовом поясе) в столбец `dto` таблицы `test2` (типа `datetimeoffset`) и применяет сдвиг часового пояса в `-05:00` для каждого скопированного значения:

```
INSERT INTO test2(dto)
SELECT TODATETIMEOFFSET(dt2, '-05:00') FROM test1
```

Следующий пример извлекает из столбца `dto` таблицы `test2` все значения `datetimeoffset`, которые могут содержать значения из многих различных часовых поясов. Посредством функции `SWITCHOFFSET` с указанием сдвига `-05:00` эти значения из любого сохраненного в `test2` часового пояса автоматически преобразуются к нью-йоркскому времени:

```
SELECT SWITCHOFFSET(dto, '-05:00') FROM test2
```

И наконец, существующие функции `DATEPART` и `DATENAME` были расширены для поддержки микросекунд (`mcs`), наносекунд (`ns`) и сдвигов часовых поясов (`tz`) для новых типов (как это показано в листинге 2.55).

#### Листинг 2.55. Использование дат в SQL Server 2008 при помощи `DATEPART` и `DATENAME`

```
SET NOCOUNT ON
DECLARE @TimeInNY datetimeoffset
SET @TimeInNY = SYSDATETIMEOFFSET()
-- Показать текущее время в Нью-Йорке
SELECT @TimeInNY AS 'Time in NY'
-- DATEPART с указанием tz получает значение часового пояса
SELECT DATEPART(tz, @TimeInNY) AS 'NY Time Zone Value'
-- DATENAME с указанием tz получает строку часового пояса
SELECT DATENAME(tz, @TimeInNY) AS 'NY Time Zone String'
-- И DATEPART, и DATENAME с указанием mcs получают микросекунды
SELECT DATEPART(mcs, @TimeInNY) AS 'NY Time Microseconds'
```



```
-- И DATEPART, и DATENAME с указанием ns получают наносекунды
SELECT DATEPART(ns, @TimeInNY) AS 'NY Time Nanoseconds'
```

Выполнение этого кода дает следующий результат:

```
Time in NY
-----
2007-11-10 20:50:55.7851424 -05:00
NY Time Zone Value
-----
-300
NY Time Zone String
-----
-05:00
NY Time Microseconds
-----
785142
NY Time Nanoseconds
-----
785142400
```

## Инструкция *MERGE*

Новая инструкция `MERGE` в SQL Server 2008 комбинирует обычные операции вставки, обновления и удаления (которые выполняются при обычном слиянии) и операцию выборки (которая обеспечивает исходные данные и данные назначения для слияния). Именно так она объединяет четыре инструкции в одну. Фактически при помощи выражения `OUTPUT` вы можете скомбинировать в одну даже пять инструкций и больше (техника `INSERT OVER DML`, как вы увидите далее в этой же главе).

Эта новая инструкция имеет гибкий синтаксис, который позволяет нам точно управлять соответствием источника и назначения, а также различными действиями языка DML в отношении назначения. В результате получается более простой код, который легче писать и обслуживать (и который быстрее выполняется), чем эквивалентный ему код с использованием отдельных инструкций.

## ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

В нашем обсуждении мы рассматриваем только самые актуальные ключевые слова инструкции MERGE. За полным описанием синтаксиса инструкции MERGE вам следует обратиться к справочной системе Books Online.

## ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Инструкция MERGE особенно хорошо приспособлена для хранилищ данных. Мы описываем концепции и методики хранилищ данных в *главе 14*.

Давайте рассмотрим первый пример эффективного управления нашими акциями и сделками. Мы начинаем с создания двух таблиц имеющих акций и ежедневных сделок, которые мы проводим (как показано в листинге 2.56).

### Листинг 2.56. Создание таблиц Stock и Trade

```
CREATE TABLE Stock(Symbol varchar(10) PRIMARY KEY,
                    Qty int CHECK (Qty > 0))
CREATE TABLE Trade(Symbol varchar(10) PRIMARY KEY,
                    Delta int CHECK (Delta <> 0))
```

Мы начинаем с 10 акций компании Adventure Works и 5 акций компании Blue Yonder Airlines. Они записываются в нашу таблицу Stock:

```
INSERT INTO Stock VALUES ('ADW', 10)
INSERT INTO Stock VALUES ('BYA', 5)
```

В течение дня мы делаем три сделки. Мы покупаем 5 новых акций Adventure Works, продаем 5 акций Blue Yonder Airlines и покупаем 3 акции для вложения денег в Northwind Traders. Все это сохраняется в таблице Trades:

```
INSERT INTO Trade VALUES ('ADW', 5)
INSERT INTO Trade VALUES ('BYA', -5)
INSERT INTO Trade VALUES ('NWT', 3)
```

Вот содержимое этих двух таблиц:

```
SELECT* FROM Stock
GO
Symbol Qty
-----
ADW 10
BYA 5
(2 row(s) affected)
```

```
SELECT* FROM Trade
GO
Symbol Delta
-----
ADVW 5
BYA -5
NWT 3
(3 row(s) affected)
```

В конце дня мы хотим обновить количество в нашей таблице `Stock` (чтобы отразить записанные нами в таблице `Trade` сделки этого дня). Количество наших акций `Adventure Works` увеличилось до 15, у нас больше нет акций `Blue Yonder Airlines` (мы продали те 5 акций, которыми владели), и теперь у нас есть 3 новых акции `Northwind Traders`. Потребуется соединение таблиц `Stock` и `Trades` для выявления изменений количества акций в результате наших сделок, а также операции вставки, обновления и удаления этих изменений в таблице `Stock`. Впервые в SQL Server всю эту логику и манипуляции можно выполнить единой инструкцией `MERGE` (как показано в листинге 2.57).

#### Листинг 2.57. Применение сделок к акциям при помощи `MERGE`

```
MERGE Stock
  USING Trade
  ON Stock.Symbol = Trade.Symbol
  WHEN MATCHED AND (Stock.Qty + Trade.Delta = 0) THEN
    -- удалить акции, если все они проданы
    DELETE
  WHEN MATCHED THEN
    -- обновить количество акций (удаление имеет преимущество
    -- перед обновлением)
    UPDATE SET Stock.Qty += Trade.Delta
  WHEN NOT MATCHED BY TARGET THEN
    -- добавить вновь купленные акции
    INSERT VALUES (Trade.Symbol, Trade.Delta);
```

Давайте исследуем эту инструкцию. Она начинается с ключевого слова `MERGE`, за ним следуют `USING` и `ON`, которые указывают соответственно источник и назначение операции слияния, а также *ключи соединения* источника и назначения друг с другом (для слияния). После синтаксиса

WHEN...THEN следуют три *выражения слияния*, а затем инструкция завершается точкой с запятой.

### **ВАЖНО**

Завершающая точка с запятой (часть стандарта SQL) в SQL Server требуется редко. Однако новая инструкция MERGE в SQL Server 2008 обязательно требует этой точки с запятой, и при ее отсутствии вы получите ошибку.

## **Определение источника и назначения MERGE**

Инструкция MERGE имеет в SQL Server 2008 весьма элегантную реализацию. Она работает с соединением между источником и назначением слияния точно так же, как любой стандартный предикат JOIN выражения FROM инструкции SELECT. Эту операцию SELECT описывает само ключевое слово MERGE, а также слова USING и ON, которые соответственно указывают назначение, источник и предикат соединения:

```
MERGE Stock
  USING Trade
  ON Stock.Symbol = Trade.Symbol
```

Назначением может быть любая таблица или обновляемое представление. Назначение указывается сразу за ключевым словом MERGE. В нашем примере это таблица Stock, которая получает изменения ежедневных сделок.

Источник (в нашем примере это таблица Trade) указывается при помощи ключевого слова USING сразу же за назначением. Все, что вы можете указать в выражении FROM обычной инструкции SELECT, поддерживается как источник для MERGE. Сюда входят не только обычные таблицы и представления, но и подзапросы, текстовые файлы (доступ к которым осуществляется при помощи инструкции OPENROWSETBYTES), удаленные таблицы, CTE, табличные переменные и TVP. Если вы можете записать что-то в выражение FROM инструкции SELECT, то вы можете записать это и в выражение USING инструкции MERGE. (Далее в этом же разделе вы увидите пример использования вложенного SELECT для выполнения подзапроса, который генерирует исходные данные "на ходу").

Указанный далее в виде ключевого слова ON предикат соединения определяет ключевой столбец (или столбцы), связывающий источник и назначение друг с другом (так же, как и в стандартном соединении таблиц). Все, что вы можете вписать в соединение SELECT, вы можете также указать и в соединении

MERGE при помощи ON, не больше и не меньше. Соединение определяет, какие записи считаются совпадающими или несовпадающими в источнике и назначении. В нашем текущем примере это столбец `Symbol`, связывающий акции со сделками. Он говорит SQL Server о том, какие акции существуют в обеих таблицах (чтобы мы могли соответственно вставлять, обновлять и удалять данные в таблице назначения). *Тип соединения* (внутреннее, левое внешнее, правое внешнее или полное внешнее) определяется тем, какое из выражений слияния применяется затем в инструкции MERGE.

## Выражение **WHEN MATCHED**

В нашем примере используются три выражения слияния: два выражения WHEN MATCHED и одно WHEN NOT MATCHED BY TARGET. Давайте подробно рассмотрим каждое.

Первое выражение WHEN MATCHED выполняется тогда, когда один и тот же символ обозначения акций найден и в таблице Stock, и в таблице Trade:

```
WHEN MATCHED AND (Stock.Qty + Trade.Delta = 0) THEN
    -- удалить акции, если они все распроданы
    DELETE
```

Такое совпадение символов означает, что необходимо откорректировать значение количества в таблице Stock. Однако в нашем сценарии мы хотим физически удалить строку из таблицы Stock (если значение после обновления в ней становится равным нулю, как в случае с Blue Yonder Airlines). Мы можем написать код для такого сценария, надо пометить выражение WHEN MATCHED дополнительным предикатом, который проверяет, не равно ли нулю количество акций после сделки. Имея возможность установить свои собственные критерии в виде предикатов для выражений слияния, вы получаете исключительную гибкость и можете применить свою собственную бизнес-логику для различных условий совпадения. В нашем конкретном случае мы хотим удалить строку из таблицы Stock при помощи инструкции DELETE (а не изменить ее значение на 0).

Следующее выражение слияния — это еще одно выражение WHEN MATCHED, но это второе выражение не имеет предикатов, квалифицирующих условие совпадения:

```
WHEN MATCHED THEN
    -- обновить количество акций (удаление имеет приоритет над обновлением)
    UPDATE SET Stock.Qty += Trade.Delta
```

Это выражение (второе) обрабатывает все остальные сделки по существующим акциям (которые не привели к нулевому значению) и соответствующим образом изменяет количество акций (при помощи инструкции UPDATE). Значения `Stock.Qty` будут увеличиваться или уменьшаться в зависимости от положительного или отрицательного значения `Trade.Delta`. В нашем примере мы хотим, чтобы количество акций Adventure Works возросло до 15 (отражая покупку 5 акций в дополнение к уже имевшимся у нас 10 акциям).

### **ПРИМЕЧАНИЕ**

Если мы попытаемся продать больше, чем нам принадлежит, произойдет ошибка. Фактически ошибка произойдет также и в том случае, если мы попытаемся продать все, что нам принадлежит (без предварительного перехвата этого условия путем удаления акций в предыдущем выражении слияния). Так происходит потому, что мы дали базе данных указание обеспечивать такие правила и не допускать нулевых или отрицательных значений `Qty` (путем описания проверочного ограничения для столбца `Qty` при создании таблицы `Stock` в самом начале нашего примера).

SQL Server 2008 очень точно определяет использование выражений слияния. Вы можете использовать одно или два выражения `WHEN MATCHED` — и не больше. Если таких выражений два, то первое должно быть квалифицировано условием `AND`. Более того, одно выражение должно указывать инструкцию `UPDATE`, а другое — `DELETE`. Как показано в нашем текущем примере, `MERGE` выберет для выполнения одно из двух выражений `WHEN MATCHED` в зависимости от значения условия `AND` для данной строки.

## **Выражение *WHEN NOT MATCHED BY TARGET***

Наше выражение слияния — это `WHEN NOT MATCHED BY TARGET`:

```
WHEN NOT MATCHED BY TARGET THEN
    -- добавить вновь приобретенные акции
    INSERT VALUES (Trade.Symbol, Trade.Delta);
```

Это выражение обрабатывает строки, обнаруженные в источнике, но отсутствующие в назначении. В нашем примере это относится к акциям, сделки по которым совершаются впервые (это новые акции Northwind Trader, которые еще не существуют в таблице назначения `Stock`). Выражение не имеет предиката (хотя и могло бы), поэтому дополнительных условий для выражения нет. Здесь мы просто добавляем в таблицу `Stock` обнаруженные в таблице `Trade` новые данные (при помощи инструкции `INSERT`).

**ПРИМЕЧАНИЕ**

Для этого выражения ключевые слова `BY TARGET` не обязательны. `WHEN NOT MATCHED` эквивалентно `WHEN NOT MATCHED BY TARGET`.

В инструкции `MERGE` допустимо только одно выражение `WHEN NOT MATCHED BY TARGET`. Оно может быть квалифицировано условием `AND` (как мы видели это раньше с `WHEN MATCHED`). В данном примере нет необходимости в условии `AND` для выражения `WHEN NOT MATCHED BY TARGET`.

После выполнения инструкции `MERGE` таблица `Stock` обновляется для отражения всех сделок за день, которые сливаются:

```
SELECT* FROM Stock
```

```
GO
```

```
Symbol      Qty
```

```
-----
```

```
ADVW        15
```

```
NWT         3
```

```
(2 row(s) affected)
```

Все, как мы хотели и ожидали: акции Blue Yonder Airlines исчезли, добавлены акции Northwind Traders и обновлены акции Adventure Works. Это весьма впечатляющий результат для всего лишь одной инструкции! Для ее написания потребовалось меньше кода, и для его обслуживания потребуется меньше усилий, чем в случае эквивалентных этому коду отдельных инструкций. Этот код и выполняется быстрее, поскольку он компилируется и выполняется как единая инструкция. Нет никаких дополнительных накладных расходов по той простой причине, что эта инструкция работает на тех же фундаментальных принципах, что и выражения `FROM` и `JOIN` обычной инструкции `SELECT`.

## Использование **MERGE** для репликации таблиц

Давайте перейдем к другому примеру, который показывает, как `MERGE` можно использовать в качестве инструмента простых репликаций. Сначала мы определяем две таблицы с одинаковыми схемами (`Original` и `Replica`). Затем мы создаем хранимую процедуру с инструкцией `MERGE`, которая будет повторять изменения таблицы `Original` в таблице `Replica` (как показано в листинге 2.58).

**Листинг 2.58. Создание двух таблиц и хранимой процедуры, которая использует MERGE для их синхронизации**

```
CREATE TABLE Original(PK int primary key, FName varchar(10), Number int)
CREATE TABLE Replica(PK int primary key, FName varchar(10), Number int)
GO
CREATE PROCEDURE uspSyncReplica AS
MERGE Replica AS R
  USING Original AS O ON O.PK = R.PK
  WHEN MATCHED AND (O.FName != R.FName OR O.Number != R.Number) THEN
    UPDATE SET R.FName = O.FName, R.Number = O.Number
  WHEN NOT MATCHED THEN
    INSERT VALUES(O.PK, O.FName, O.Number)
  WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

Инструкция MERGE в этой хранимой процедуре выполняет задачу репликации путем соединения двух таблиц по их первичным ключам (PK) и при помощи трех выражений слияния. Первое выражение обрабатывает обновления, как показано здесь:

```
WHEN MATCHED AND (O.FName != R.FName OR O.Number != R.Number) THEN
  UPDATE SET R.FName = O.FName, R.Number = O.Number
```

Здесь мы используем опцию WHEN MATCHED для нахождения всех записей, которые существуют как в оригинале, так и в реплике. Затем мы обновляем совпадающие строки на стороне реплики самыми свежими данными из оригинала (при помощи инструкции UPDATE). Выполнение такого обновления, когда данные реально не изменились, представляет собой расточительство ресурсов, поэтому мы применим предикат выражения слияния, чтобы оно применялось только в том случае, когда обнаружено изменение строки (отличие между оригиналом и репликой в любом из неключевых значений).

Второе выражение слияния обрабатывает вставки, как видно далее:

```
WHEN NOT MATCHED THEN
  INSERT VALUES(O.PK, O.FName, O.Number)
```

Как уже упоминалось, WHEN NOT MATCHED эквивалентно WHEN NOT MATCHED BY TARGET, которое возвращает все исходные строки, отсутствующие в таблице-реплике. Эти записи представляют собой новые строки, добавленные в исходную таблицу после последнего слияния, они добавляются и в реплику при помощи инструкции INSERT этого выражения.



## Выражение **WHEN NOT MATCHED BY SOURCE**

Третье и последнее слияние обрабатывает удаления, причем в конце имеется обязательный разделитель в виде точки с запятой:

```
WHEN NOT MATCHED BY SOURCE THEN  
    DELETE;
```

Выражение `WHEN NOT MATCHED BY SOURCE` возвращает строки назначения, которых нет в исходной таблице. Такой сценарий получается в результате удаления строк (после их репликации) из оригинальной таблицы. Удаленные из оригинальной таблицы строки необходимо удалить и из таблицы-реплики, что и делается в данном случае при помощи простой инструкции `DELETE`.

Разрешается иметь до двух выражений `WHEN NOT MATCHED BY SOURCE`. Если их у вас два, то первое должно быть квалифицировано условием `AND`. Более того, в одном выражении должно быть указано `UPDATE`, а в другом — `DELETE`. Если указаны два выражения `WHEN NOT MATCHED BY SOURCE`, то `MERGE` выберет для выполнения одно из них в зависимости от значения условия `AND`.

На данный момент обе таблицы пустые (что фактически означает их синхронность), поэтому мы сначала заполним исходную таблицу несколькими строками:

```
INSERT Original VALUES(1, 'Sara', 10)  
INSERT Original VALUES(2, 'Steven', 20)
```

Теперь давайте вызовем нашу хранимую процедуру `uspSyncReplica` обновления таблицы-реплики. При первом вызове хранимой процедуры будет выполнено только выражение `WHEN NOT MATCHED`. Фактически оно будет выполнено два раза, по разу для каждой из двух строк, которые нужно добавить в таблицу-реплику из исходной таблицы. Об этом сообщает счетчик обработанных строк, отображаемый после выполнения хранимой процедуры:

```
EXEC uspSyncReplica  
GO  
(2 row(s) affected)
```

Изучение обеих таблиц подтверждает, что реплика синхронизирована с оригиналом:

```
SELECT* FROM Original  
SELECT* FROM Replica  
GO  
PK FName Number  
-----
```

```

1 Sara 10
2 Steven 20
(2 row(s) affected)
PK FName Number
-----

```

```

1 Sara 10
2 Steven 20
(2 row(s) affected)

```

Теперь мы произведем дополнительные изменения в исходной таблице. Некий набор операций вставки, обновления и удаления опять нарушает синхронизацию таблиц:

```

INSERT INTO Original VALUES(3, 'Andrew', 100)
UPDATE Original SET FName = 'Stephen', Number += 10 WHERE PK = 2
DELETE FROM Original WHERE PK = 1
GO
SELECT* FROM Original
SELECT* FROM Replica
GO
PK FName Number
-----

```

```

2 Stephen 30
3 Andrew 100
(2 row(s) affected)
PK FName Number
-----

```

```

1 Sara 10
2 Steven 20
(2 row(s) affected)

```

Еще один запуск хранимой процедуры восстанавливает синхронизацию обеих таблиц (перенос изменений из исходной таблицы в таблицу-реплику):

```

EXEC uspSyncReplica
GO
(3 row(s) affected)

```

На этот раз мы видим, что в результате одной вставки (Andrew), одного обновления (Stephen) и одного удаления (Sara) были обработаны три строки.

Изучение обеих таблиц подтверждает, что реплика опять синхронизирована с оригиналом:

```
SELECT* FROM Original
```

```
SELECT* FROM Replica
```

```
GO
```

```
PK FName Number
```

```
-----  
2 Stephen 30
```

```
3 Andrew 100
```

```
(2 row(s) affected)
```

```
PK FName Number
```

```
-----  
2 Stephen 30
```

```
3 Andrew 100
```

```
(2 row(s) affected)
```

## Вывод в инструкции **MERGE**

Инструкция **MERGE** поддерживает выражение **OUTPUT**, которое было введено в SQL Server 2005 для инструкций **INSERT**, **UPDATE** и **DELETE**. Это выражение возвращает информацию об изменениях в каждой строке, обработанной операциями **DML** (в тех же самых псевдотаблицах **INSERTED** и **DELETED**, предоставляемых триггерами). Возможность получить эту информацию в выражении **OUTPUT** является более удачным вариантом, чем перехват ее в триггерах, поскольку вы не можете гарантировать, что триггеры в таблице будут всегда срабатывать в одном и том же порядке, а это часто является причиной неявных ошибок, которые очень сложно выявить. (Так что триггеров следует вообще избегать, насколько это возможно).

В дополнение к столбцам псевдотаблиц **INSERTED** и **DELETED** в выражении **OUTPUT** инструкции **MERGE** введен новый виртуальный столбец с названием **\$action**. Этот столбец будет возвращать одно из трех строковых значений — **'INSERT'**, **'UPDATE'** или **'DELETE'**, в зависимости от выполняемого с каждой строкой действия. В листинге 2.59 показана модифицированная версия нашей хранимой процедуры **uspSyncReplica**, которая содержит выражение **OUTPUT** в инструкции **MERGE**. Выражение **OUTPUT** выбирает виртуальный столбец **\$action** и все столбцы псевдотаблиц **INSERTED** и **DELETED** так, чтобы инструкция **MERGE** могла дать подробный отчет по каждой выполняемой ею операции **DML**.

**Листинг 2.59. Использование выражения OUTPUT и виртуального столбца \$action в MERGE**

```
CREATE PROCEDURE uspSyncReplica AS
MERGE Replica AS R
  USING Original AS O ON O.PK = R.PK
  WHEN MATCHED AND (O.FName != R.FName OR O.Number != R.Number) THEN
    UPDATE SET R.FName = O.FName, R.Number = O.Number
  WHEN NOT MATCHED THEN
    INSERT VALUES(O.PK, O.FName, O.Number)
  WHEN NOT MATCHED BY SOURCE THEN
    DELETE
    OUTPUT $action, INSERTED.*, DELETED.*;
```

Выполнение этой модифицированной версии инструкции MERGE для последнего набора изменений в нашем примере даст следующий вывод:

| \$action | PK   | FName   | Number | PK   | FName  | Number |
|----------|------|---------|--------|------|--------|--------|
| DELETE   | NULL | NULL    | NULL   | 1    | Sara   | 10     |
| UPDATE   | 2    | Stephen | 30     | 2    | Steven | 20     |
| INSERT   | 3    | Andrew  | 100    | NULL | NULL   | NULL   |

(3 row(s) affected)

Этот вывод показывает все действия, выполненные слиянием, и все вовлеченные в этот процесс данные. Конечно, вы можете использовать выражение OUTPUT...INTO (точно так же, как и с инструкциями INSERT, UPDATE и DELETE) отправки этой информации в другую таблицу или табличную переменную (для сохранения в журнале или дополнительной обработки). SQL Server 2008 вводит расширение этой функции, которое дает нам еще бóльшую гибкость — INSERT OVER DML, и мы изучим его сразу после завершения нашего рассмотрения инструкции MERGE.

## Выбор метода соединения

SQL Server исследует MERGE с различными комбинациями выражений слияния и автоматически выполняет соответствующий вид соединения (чтобы вернуть исходные данные и данные назначения, необходимые на обеих сторонах). Существуют четыре типа соединений, и SQL Server выбирает правильный вариант для данной задачи в зависимости от выражений слияния, которые он находит в инструкции MERGE (табл. 2.4).

**Таблица 2.4.** Типы соединений таблиц сервером SQL в зависимости от MERGE

| Выражение в MERGE                                                  | Тип соединения | Возвращает                                                                                                           | Допустимые действия           |
|--------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------|
| WHEN MATCHED                                                       | Внутреннее     | Совпадающие данные в источнике и назначении                                                                          | Обновление, удаление          |
| WHEN NOT MATCHED [BY TARGET]                                       | Левое внешнее  | Данные источника, которые не обнаружены в назначении                                                                 | Вставка                       |
| WHEN NOT MATCHED BY SOURCE                                         | Правое внешнее | Данные назначения, которые не обнаружены в источнике                                                                 | Обновление, удаление          |
| WHEN NOT MATCHED [BY TARGET] в сочетании с любым другим выражением | Полное внешнее | Данные источника, которые не обнаружены в назначении, а также другие совпадающие или несовпадающие данные назначения | Вставка, обновление, удаление |

Выражение `WHEN MATCHED` возвращает строки, которые обнаружены в обеих таблицах (то есть те строки обеих таблиц, у которых совпадают значения в соединяющих ключевых столбцах, определенных при помощи `ON`). Обработчик запросов обрабатывает это как внутреннее соединение, поскольку вы извлекаете только те строки, которые существуют на обеих сторонах. В выражении `WHEN MATCHED` для совпадающих строк назначения разрешаются операции `UPDATE` и `DELETE`.

Выражение `WHEN NOT MATCHED [BY TARGET]` обрабатывается как левое внешнее соединение, поскольку оно извлекает только те строки источника, которые не существуют в назначении. Поэтому в `WHEN NOT MATCHED [BY TARGET]` может быть выполнена только вставка `INSERT` для "заполнения дыры" в назначении (отсутствующими в нем данными источника).

Выражение `WHEN NOT MATCHED BY SOURCE` является дополнением для `WHEN NOT MATCHED [BY TARGET]` и приводит к правому внешнему соединению для извлечения только тех строк назначения, которые не существуют в источнике. Как и в случае `WHEN MATCHED`, вы можете выполнить операции `UPDATE` или `DELETE` по этим строкам назначения (отсутствующим в источнике).

И наконец, сочетание `WHEN NOT MATCHED [BY TARGET]` с одним или с двумя остальными выражениями извлекает те данные источника, которые не обнаружены в назначении, а также данные назначения, которые обнаружены (или не обнаружены) в источнике. В этом случае между двумя таблицами

выполняется полное внешнее соединение. Вы можете выполнить INSERT в выражении WHEN NOT MATCHED [BY TARGET] для добавления данных источника (отсутствующих в назначении), а также UPDATE или DELETE в другом выражении (или других выражениях) для модификации или удаления данных назначения.

### **ПРЕДУПРЕЖДЕНИЕ**

Будьте очень внимательны в отношении значений NULL в ключевых столбцах, по которым вы делаете соединение. Как и при обычном соединении, они ни с чем не совпадают, и в итоге может получиться так, что вы вставите в назначение те строки, которые вы вставлять не собирались.

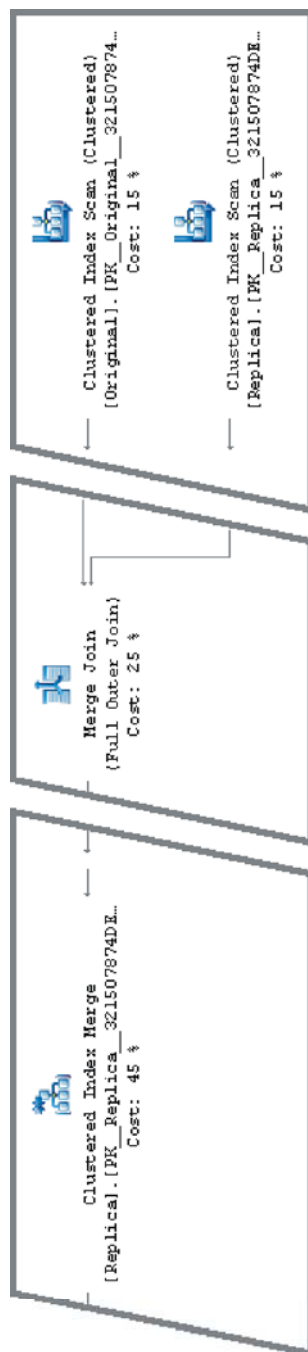
### **ВАЖНО**

Единственной допустимой инструкцией в WHEN MATCHED или WHEN NOT MATCHED BY SOURCE является UPDATE или DELETE, а единственной допустимой инструкцией в WHEN NOT MATCHED [BY TARGET] является INSERT. В этих выражениях слияния не допускаются никакие другие инструкции T-SQL, в том числе и вызовы хранимых процедур. Однако инструкции INSERT и UPDATE в выражениях слияния могут ссылаться на определяемые пользователем функции (UDF).

## **Поведение инструкции *MERGE* в DML**

Как вы только что узнали, инструкция MERGE комбинирует четыре различных инструкции DML (SELECT, INSERT, UPDATE и DELETE) для выполнения слияния. Внутренняя реализация MERGE точно такая же, как и у инкапсулируемых ею отдельных инструкций DML. Это был верный шаг со стороны компании Microsoft, поскольку разработчики могут сразу же начать применять MERGE и не опасаться повреждения функциональности уже существующего кода.

Все триггеры AFTER и INSTEAD OF (которые уже были определены в существующих таблицах или обновляемых представлениях) продолжают срабатывать и тогда, когда эти таблицы или обновляемые представления указываются как назначение для инструкции MERGE. Например, выражение WHEN NOT MATCHED THEN...INSERT заставляет срабатывать все триггеры вставки, определенные для назначения; аналогичным же образом выражение WHEN MATCHED THEN...DELETE приводит к срабатыванию триггеров удаления. Не существует новой концепции "триггера слияния", и это означает, что у вас нет препятствий для того, чтобы начать использовать MERGE в вашем коде T-SQL.



**Рис. 2.2.** План запроса для операции MERGE для синхронизации реплики с оригиналом (экранное изображение было подрезано для размещения на печатной странице)

В конечном результате триггеры срабатывают точно так же, как это происходило бы и при использовании вместо `MERGE` отдельных инструкций. Существующая бизнес-логика, ограничения и правила — все продолжает работать, как раньше.

С `MERGE` можно применять те же самые опции и функции, которые вы привыкли использовать с отдельными инструкциями DML. Например, вы можете использовать CTE для источника слияния (определив его при помощи выражения `WITH` до инструкции `MERGE` и последующей ссылки на него посредством выражения `USING`). Поддерживается также и выражение `TOP`, и традиционные подсказки в запросах. И конечно, обе таблицы (источник и назначение) могут иметь псевдонимы (посредством использования `AS`) — точно так же, как таблицы в обычных запросах.

Мы можем увидеть, как SQL Server реализует инструкцию `MERGE` хранимой процедуры, которая делает репликацию таблицы. Для этого необходимо изучить план выполнения запроса ядра базы данных, который показан на рис. 2.2.

Если вы еще не знакомы с планами запросов, прежде всего вам нужно знать то, что читать их следует справа налево. (В системе помощи Books Online дается объяснение планов выполнения запросов SQL Server Management Studio). Работа начинается с просмотра таблиц источника и назначения.

На рис. 2.3 мы видим преимущества создания индексов по столбцам, соединяющим таблицы `Original` и `Replica` для слияния. Поскольку предикат соединения берет первичные ключи обеих таблиц, оптимизатор запросов для достижения максимальной производительности использует просмотр кластеризованного индекса. Всегда следует создавать индекс (кластеризованный, а в идеальном случае — уникальный) по соединенным столбцам двух таблиц.

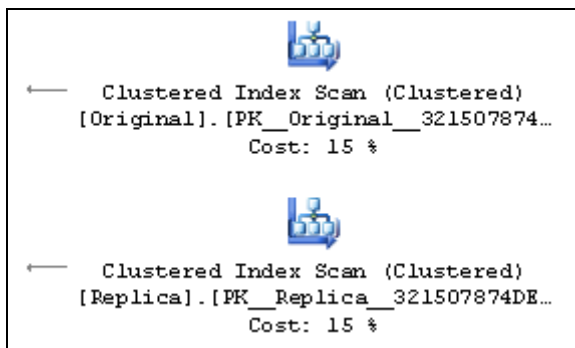
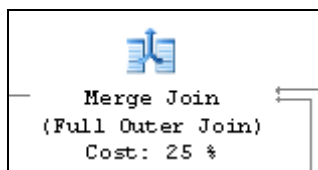


Рис. 2.3. Просмотры кластеризованных индексов

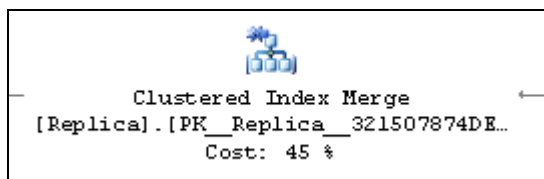


Из плана также становится ясно, что выполняется полное внешнее соединение между двумя таблицами (как показано на рис. 2.4).



**Рис. 2.4.** Полное внешнее соединение в плане запроса для MERGE

Как уже упоминалось ранее, тип соединения зависит от того, какие выражения слияния указаны (или не указаны) в инструкции MERGE. В данном случае SQL Server выполняет полное внешнее соединение, поскольку наша инструкция MERGE использует все три возможных выражения слияния. Следующая далее операция фильтрации (рис. 2.5) реализует предикат первого выражения слияния, который выбирает совпадающие строки только в том случае, когда изменился какой-либо неключевой столбец таблиц (исходной и назначения).



**Рис. 2.5.** Операция слияния с кластеризованным индексом, которая обрабатывает для MERGE обновления, вставки и удаления

Слияние с кластеризованным индексом — это новая операция, которая принимает поток из полного внешнего соединения и для каждой строки решает, следует ли переданную в данную операцию строку обрабатывать как обновление, как вставку или как удаление (в зависимости от синтаксиса инструкции MERGE и данных в таблицах источника и назначения).

## Выполнение вставки/обновления

Инструкция MERGE определенно предназначена для использования в операциях с наборами данных (для обработки множества строк за один заход), ко-

торые мы и выполняли до настоящего момента. Однако она удобна также и в стандартных задачах для единственной строки, когда обновление выполняется в том случае, если строка уже существует в назначении, а вставка — если строки еще нет. Эта операция, часто называемая `upsert`, поскольку она выполняет либо `update` (обновление), либо `insert` (вставку), включает в себя проверку существования строки (чтобы определить конкретную необходимость, делать обновление или вставку).

Мы рассмотрим две реализации хранимой процедуры для `upsert` (с использованием `MERGE`). Первая — это простая версия, которая не поддерживает выходных параметров, идентификационных значений или проверки блокировок. После того как вы поймете ее работу, мы перейдем ко второй версии, которая поддерживает все три этих элемента и гораздо более пригодна для любого реального приложения.

Как мы и обещали, мы продемонстрируем в операции слияния использование подзапроса для конструирования исходной таблицы "на ходу". Обе версии нашей хранимой процедуры создают исходную таблицу динамически (при помощи вложенной инструкции `SELECT`). Исходная таблица содержит одну строку, созданную из входных параметров хранимой процедуры (которую необходимо вставить или обновить в таблице назначения). Давайте же приступим.

Сначала мы создадим таблицу `Customer` (листинг 2.60).

#### Листинг 2.60. Создание таблицы `Customer`

```
CREATE TABLE Customer(  
    CustomerId int PRIMARY KEY,  
    FirstName varchar(30),  
    LastName varchar(30),  
    Balance decimal)
```

Затем мы создадим хранимую процедуру, которую мы можем вызвать для вставки нового клиента или обновления существующего (листинг 2.61).

#### Листинг 2.61. Простая хранимая процедура с использованием `MERGE`

```
CREATE PROCEDURE uspUpsertCustomer(  
    @CustomerId int,  
    @FirstName varchar(30),  
    @LastName varchar(30),
```

```
@Balance decimal)
AS
BEGIN
    MERGE Customer AS tbl
    USING (SELECT
            @CustomerId AS CustomerId,
            @FirstName AS FirstName,
            @LastName AS LastName,
            @Balance AS Balance) AS row
    ON tbl.CustomerId = row.CustomerId
    WHEN NOT MATCHED THEN
        INSERT(CustomerId, FirstName, LastName, Balance)
        VALUES(row.CustomerId, row.FirstName, row.LastName, row.Balance)
    WHEN MATCHED THEN
        UPDATE SET
            tbl.FirstName = row.FirstName,
            tbl.LastName = row.LastName,
            tbl.Balance = row.Balance
;
END
```

Инструкция `MERGE` очень проста. Сначала таблица `Customer` объявляется как назначение для слияния и получает псевдоним `tbl`. После этого при помощи выражения `USING` объявляется источник для слияния (как одна строка, составленная из значений входящих параметров с вложенной инструкцией `SELECT` и с псевдонимом `row`). Таблица и строка затем соединяются для слияния (по столбцу `CustomerId` при помощи выражения `ON`).

Затем следуют два выражения слияния: одно для обработки вставки, а другое — для обновления. Выражение `WHEN NOT MATCHED` выполняется в том случае, если переданный `CustomerId` строки не найден в таблице `Customer`. В этом случае добавляется новая строка с теми значениями, которые были предоставлены другими переданными параметрами. Выражение `WHEN MATCHED` выполняется в том случае, когда переданный `CustomerId` строки уже существует в таблице `Customer`. В этом случае существующая строка обновляется значениями из таблицы-источника.

Давайте теперь вызовем эту хранимую процедуру для добавления двух новых клиентов с указанными кредитовыми остатками:

```
-- Добавить Клиента 1
EXEC uspUpsertCustomer 1, 'Mark', 'Boyer', 100
```

```
-- Добавить Клиента 2
EXEC uspUpsertCustomer 2, 'Ted', 'Bremer', 120
SELECT* FROM Customer
GO
```

| CustomerId | FirstName | LastName | Balance |
|------------|-----------|----------|---------|
| 1          | Mark      | Boyer    | 100     |
| 2          | Ted       | Bremer   | 120     |

Как и ожидалось, хранимая процедура вставила обоих клиентов как новые строки, поскольку значения идентификатора клиента 1 и 2 ранее не существовали — что и привело к срабатыванию выражения `WHEN NOT MATCHED` инструкции `MERGE`.

Когда клиент 1 в следующий раз посетил наш магазин, он сделал покупку в кредит на 10 долларов, что увеличило его остаток со 100 до 110 долларов. Он обратил также внимание кассира на то обстоятельство, что вводивший его данные сотрудник ошибочно записал его имя как Mark, а правильное написание его имени — Marc. Клиента можно обновить в базе данных при помощи точно такого же вызова хранимой процедуры, который использовался для его первоначальной вставки в базу:

```
-- Изменить имя Клиента 1 и его остаток
EXEC uspUpsertCustomer 1, 'Marc', 'Boyer', 110
SELECT* FROM Customer
GO
```

| CustomerId | FirstName | LastName | Balance |
|------------|-----------|----------|---------|
| 1          | Marc      | Boyer    | 110     |
| 2          | Ted       | Bremer   | 120     |

На этот раз хранимая процедура выполнила обновление, поскольку клиент с идентификатором 1 уже существует (было выполнено выражение `WHEN MATCHED` инструкции `MERGE`).

Теперь давайте перейдем ближе к реальности. Мы очень часто позволяем серверу SQL Server автоматически генерировать новые целочисленные первичные ключи с использованием значений `IDENTITY`. В такой ситуации идентификатор клиента определяется базой данных, а не приложением. Поэтому нам понадобится реализовать `@CustomerId` как входной/выходной параметр, чтобы можно было передать `NULL` в случае нового клиента и вернуть значение `IDENTITY`, присвоенное вновь добавленному клиенту. В случае обновления передается идентификатор существующего клиента, и его значение не

изменяется (то есть переданное значение @CustomerId будет возвращено обратно).

Для аудита часто нужны дата и время сервера (время создания и модификации строк). Так же, как и в случае с @CustomerId, мы добавим в нашу хранимую процедуру параметры @Created и @Modified, которые могут вернуть эти присвоенные сервером значения. Однако в отличие от @CustomerId это будут параметры "только для выхода", поскольку клиент не может давать им значения. Нам не нужна точность до долей секунды, поэтому мы будем использовать тип данных `datetime2(0)` компактного хранения времен создания и модификации.

### **ПРИМЕЧАНИЕ**

Точность и тип данных `datetime2` подробно описаны ранее в этой главе.

И наконец, нам необходимо разобраться с версиями строк (если мы хотим обеспечить проверки *оптимистического параллелизма* для обновлений). Такая стратегия часто реализуется при помощи столбца `rowversion`. (Помните, что тип `rowversion` раньше назывался `timestamp`, термин из ранних версий SQL Server). SQL Server сохраняет уникальное 8-байтовое двоичное значение отслеживания изменений строк (для проверок оптимистического параллелизма): клиент должен сначала извлечь текущее значение `rowversion` при выборке строки для внесения изменений, а потом вернуть это значение при попытке обновить строку. Обновление может быть отвергнуто, если возвращенное при обновлении значение `rowversion` отличается от текущего значения `rowversion` в таблице, поскольку это означает, что другой пользователь или процесс успел модифицировать эту строку уже после чтения ее данных клиентом.

Для поддержки такой функциональной возможности в нашей хранимой процедуре мы добавим @Version в качестве входного/выходного параметра типа `rowversion`. Для выполнения обновления клиент передаст исходное значение `rowversion`, которое было получено, когда строка была выбрана. Хранимая процедура затем будет использовать это значение для обеспечения проверок параллелизма перед обновлением строки в таблице. Если обновление произойдет успешно, то обратно клиенту через параметр @Version будет передано новое значение `rowversion`. Для вставки проверка параллелизма не производится. В этом случае клиент передает NULL для @Version и получает назад значение `rowversion`, присвоенное новой добавленной строке.

Наша улучшенная версия хранимой процедуры `uspUpsertCustomer` поддерживает все эти требования. Но сначала нам необходимо пересмотреть схему

таблицы Customer. Мы хотим задействовать использование значений IDENTITY для первичного ключа CustomerId, добавить два столбца datetime2 для аудита времени создания и модификации, а также добавить столбец rowversion для параллелизма (листинг 2.62).

**Листинг 2.62. Создание новой версии таблицы Customer для поддержки расширенных возможностей**

```
CREATE TABLE Customer(  
    CustomerId int IDENTITY(1, 1) PRIMARY KEY,  
    FirstName varchar(30),  
    LastName varchar(30),  
    Balance decimal,  
        Created datetime2(0),  
        Modified datetime2(0),  
        Version rowversion)  
ALTER TABLE Customer ADD CONSTRAINT  
    DF_Customer_Created DEFAULT (SYSDATETIME()) FOR Created  
ALTER TABLE Customer ADD CONSTRAINT  
    DF_Customer_Modified DEFAULT (SYSDATETIME()) FOR Modified
```

Обратите внимание, что мы установили также и значения по умолчанию для Created и Modified (как значения текущей даты и времени сервера, возвращаемых функцией SYSDATETIME). Как вы увидите, это избавит вас от необходимости предоставлять значения Created и Modified для новых вставляемых клиентов — SQL Server просто автоматически установит в них по умолчанию текущие дату и время.

Теперь давайте переделаем саму хранимую процедуру uspUpsertCustomer (для поддержки всех наших новых требований), как показано в листинге 2.63.

**Листинг 2.63. Усовершенствованная хранимая процедура upsert с использованием MERGE**

```
CREATE PROCEDURE uspUpsertCustomer(  
    @CustomerId int OUTPUT, -- Для нового клиента передается NULL  
    @FirstName varchar(30),  
    @LastName varchar(30),  
    @Balance decimal,  
        @Created datetime2(0) OUTPUT,  
        @Modified datetime2(0) OUTPUT,
```

```
@Version rowversion OUTPUT)
AS
BEGIN
    -- Слить единственную строку источника, созданную из параметров,
    -- с таблицей Customer
    MERGE Customer AS tbl
    USING (SELECT
            @CustomerId AS CustomerId,
            @FirstName AS FirstName,
            @LastName AS LastName,
            @Balance AS Balance) AS row
    ON tbl.CustomerId = row.CustomerId
    -- Вставить новую строку, если она не найдена
    -- (в @CustomerId был передан NULL)
    WHEN NOT MATCHED THEN
        INSERT(FirstName, LastName, Balance)
        VALUES(row.FirstName, row.LastName, row.Balance)
    -- Обновить существующую строку, если она найдена -
    -- но только в том случае, если совпадают их rowversion
    WHEN MATCHED AND tbl.Version = @Version THEN
        UPDATE SET
            tbl.FirstName = row.FirstName,
            tbl.LastName = row.LastName,
            tbl.Balance = row.Balance,
            tbl.Modified = SYSDATETIME()
    ;
    -- Если обновления не произошло из-за изменения rowversion
    IF @@ROWCOUNT = 0 AND @CustomerId IS NOT NULL
        RAISERROR('Optimistic concurrency violation', 18, 1)
    -- Если это была вставка, вернуть новое значение IDENTITY
    IF @CustomerId IS NULL
        SET @CustomerId = SCOPE_IDENTITY()
    -- Вернуть предназначенные только для чтения времени
    -- создания/модификации и новую rowversion
    SELECT @Created = Created, @Modified = Modified, @Version =
Version
        FROM Customer
        WHERE CustomerId = @CustomerId
END
```

Здесь происходит много разных вещей, так что давайте изучим эту хранимую процедуру шаг за шагом.

Обратите внимание, что в сигнатуре `@CustomerId` объявляется как выходной параметр. Для нового клиента здесь будет передано значение `NULL`, а обратно будет передано новое значение первичного ключа клиента, сгенерированное базой данных при помощи `IDENTITY`. Для существующего клиента в `@CustomerId` будет передано и возвращено одно и то же значение, используемое в идентификации обновляемого клиента.

Мы также добавили выходные параметры `@Created` и `@Modified` типа `datetime2(0)`. Для нового клиента будут возвращены одни и те же дата и время (когда была вставлена строка данного клиента). Для существующего клиента через эти параметры возвращаются исходное время создания и обновленное время модификации.

И наконец, мы добавили выходной параметр `@Version` типа `rowversion`. Для нового клиента он передается как `NULL`. Для существующего клиента он передается как исходное значение `rowversion`, полученное при выборке клиента из базы данных (оно будет использоваться в проверке оптимистического параллелизма). И для нового, и для существующего клиента обратно через этот параметр передается обновленное значение `rowversion`.

Предикаты назначения, источника и соединения инструкции `MERGE` реализуются точно так же, как и в нашей первой версии хранимой процедуры. В качестве назначения указывается таблица `Customer`, источник создается (как таблица из одной строки) из вложенного запроса `SELECT` с входными параметрами, а предикат соединения сопоставляет строку источника с таблицей назначения по столбцу `CustomerId`.

Выражение `WHEN NOT MATCHED` практически не изменилось, за исключением того, что для инструкции `INSERT` не указан идентификатор. Определив `CustomerId` как столбец `IDENTITY`, мы возлагаем на SQL Server ответственность за присваивание значений новых идентификаторов, поэтому не сможем указать значение первичного ключа для вставки, даже если мы хотели бы этого. Если мы попытаемся это сделать, получим ошибку. Это означает, что клиентская часть не знает, каким будет ID нового клиента (до тех пор, пока строка не будет добавлена в таблицу). Поэтому клиентская часть передает для `@CustomerId` значение `NULL`, которое, конечно же, не будет найдено в таблице, а это приведет к тому, что выражение `WHEN NOT MATCHED` вставит новую строку клиента. Присвоенное значение первичного ключа `CustomerId` будет получено при помощи функции `SCOPE_IDENTITY` (следующей за инструкцией `MERGE`). Обратите также внимание, что вставка не предоставляет значений



Created или Modified, а использует значения по умолчанию SYSDATETIME (это текущие дата и время сервера). Несмотря на то, что столбец Modified будет изменяться при обновлении существующих строк, мы реализуем столбец Created как неизменное значение, которое обозначает исходное время вставки строки в течение всей ее жизни.

Для существующих строк выражение слияния WHEN MATCHED квалифицируется предикатом, препятствующим обновлению данных, которые были модифицированы другим пользователем или процессом после того, как данная клиентская часть получила их более раннюю версию. Это достигается путем проверки совпадения текущего значения rowversion в таблице со значением rowversion, переданным клиентской частью. Отличающиеся значения совершенно определенно означают, что данные таблицы за это время были изменены и их не следует перезаписывать вторым обновлением (которое делается на основе предыдущей версии данных). Выражение слияния получает значение true только тогда, когда такая проверка параллелизма выполняется успешно, что позволяет выполниться инструкции UPDATE. Это обновление почти идентично первой версии хранимой процедуры (за исключением того, что оно изменяет еще и столбец Modified).

После инструкции MERGE хранимая процедура генерирует ошибку в том случае, если она понимает, что обновление в выражении WHEN MATCHED не работало ни одной строки. Это обнаруживается проверкой встроенной переменной @@ROWCOUNT сервера SQL Server на равенство 0. Эта переменная в результате успешной вставки или обновления строки получает значение 1. Если она возвращает значение 0 и в @CustomerId не было передано значение NULL, а предикат WHEN MATCHED обнаружил разные значения rowversion (либо другой пользователь удалил клиента), что отменило операцию обновления, то нам надо выдать ошибку:

```
IF @@ROWCOUNT = 0 AND @CustomerId IS NOT NULL
    RAISERROR('Optimistic concurrency violation', 18, 1)
```

### **ПРИМЕЧАНИЕ**

Если вы считаете, что было бы гораздо лучше поместить в выражение WHEN MATCHED AND tbl.Version <> @Version THEN инструкцию RAISERROR, то вы совершенно правы. Однако в выражениях слияния не допускаются никакие другие инструкции T-SQL, кроме INSERT, UPDATE и DELETE.

Если в @CustomerId передается NULL (а это означает, что вставка была затребована клиентской частью), то мы сохраняем новое значение IDENTITY (сгенерированное сервером SQL Server и предоставляемое через его функцию

SCOPE\_IDENTITY в выходной параметр @CustomerId, возвращаемый клиентской части):

```
IF @CustomerId IS NULL
    SET @CustomerId = SCOPE_IDENTITY()
```

И наконец, время создания/модификации и значение rowversion извлекаются из строки клиента в выходные параметры @Created, @Modified и @Version для возвращения в клиентскую часть:

```
SELECT @Created = Created, @Modified = Modified, @Version = Version
FROM Customer
WHERE CustomerId = @CustomerId
```

Теперь давайте посмотрим, как работает эта новая реализация. Мы напишем код T-SQL для первой версии этой хранимой процедуры (две вставки и одно обновление). Сначала в этой новой версии хранимой процедуры нам необходимо объявить несколько локальных переменных хранения возвращенных выходными параметрами значений:

```
DECLARE @C1Id int
DECLARE @C2Id int
DECLARE @Crt datetime2(0)
DECLARE @Mod datetime2(0)
DECLARE @C1Ver rowversion
DECLARE @C2Ver rowversion
```

Поскольку первые два клиента — новые, то мы присваиваем NULL переменным, содержащим значения их идентификатора:

```
SET @C1Id = NULL
SET @C2Id = NULL
```

Затем мы вызываем хранимую процедуру, передаем переменные с идентификатором клиента (которым мы только что присвоили значения NULL) и получаем обратно новые присвоенные значения первичного ключа IDENTITY (через первый входной/выходной параметр). Имя, фамилия и значения остатка передаются как обычные входные параметры (как и раньше). Мы возвращаем также время создания, время модификации и rowversion (при помощи трех дополнительных выходных параметров) и сохраняем их в других локальных переменных (объявленных ранее). После каждого вызова хранимой процедуры мы делаем выборку выходных параметров (чтобы увидеть их значения). Для имитации мы добавляем также задержку в одну секунду между вызовами:

```
-- Добавить Клиента 1
EXEC uspUpsertCustomer
```

```

@C1Id OUTPUT, 'Mark', 'Boyer', 100, @Crt OUTPUT, @Mod OUTPUT, @C1Ver
OUTPUT
SELECT @C1Id AS C1Id, @Crt AS Crt, @Mod AS Mod, @C1Ver AS C1Ver
WAITFOR DELAY '00:00:01'
-- Добавить Клиенте 2
EXEC uspUpsertCustomer
@C2Id OUTPUT, 'Ted', 'Bremer', 120, @Crt OUTPUT, @Mod OUTPUT, @C2Ver
OUTPUT
SELECT @C2Id AS C2Id, @Crt AS Crt, @Mod AS Mod, @C2Ver AS C2Ver
WAITFOR DELAY '00:00:01'

```

В соответствии со значениями `IDENTITY`, сгенерированными для первичных ключей, двум клиентам были присвоены значения идентификатора 1 и 2. Из-за задержки `WAITFOR DELAY` два клиента имеют отличающееся на одну секунду время создания. И наконец, мы получаем уникальные значения `rowversion`, присвоенные каждому из двух клиентов:

| C1Id  | Crt                 | Mod                 | C1Ver               |
|-------|---------------------|---------------------|---------------------|
| ----- |                     |                     |                     |
| 1     | 2007-11-22 14:14:21 | 2007-11-22 14:14:21 | 0x00000000000001026 |

| C2Id  | Crt                 | Mod                 | C2Ver               |
|-------|---------------------|---------------------|---------------------|
| ----- |                     |                     |                     |
| 2     | 2007-11-22 14:14:22 | 2007-11-22 14:14:22 | 0x00000000000001027 |

Далее следует сценарий, который мы уже описали ранее: остаток клиента 1 увеличивается со 100 долларов до 110, изменяется также написание его имени (с Mark на Marc). Как и прежде, клиента можно обновить в базе данных при помощи точно такого же вызова хранимой процедуры, какой делался для первоначальной вставки; только на этот раз содержащееся в вызове исходное значение `rowversion` должно совпасть с текущим значением в базе данных (для защиты от нарушений параллелизма). Вспомним, что мы сохранили это исходное значение `rowversion` в нашей локальной переменной `@C1Ver` (как только получили его из первоначальной операции вставки две секунды тому назад). Поэтому мы можем использовать ту же самую переменную `@C1Ver` для передачи исходного значения `rowversion` и для получения обратно нового значения `rowversion` после обновления:

```

-- Изменить имя и сальдо первого клиента, передав исходное
-- значение rowversion из первоначальной операции вставки
EXEC uspUpsertCustomer
@C1Id OUTPUT, 'Marc', 'Boyer', 110,
@Crt OUTPUT, @Mod OUTPUT, @C1Ver OUTPUT
SELECT @C1Id AS C1Id, @Crt AS Crt, @Mod AS Mod, @C1Ver AS C1Ver

```

На этот раз переданный через @C1Id идентификатор клиента имеет значение 1, которое было присвоено при первоначальной вставке. (Ключевое слово OUTPUT по-прежнему необходимо, несмотря на то, что значение при обновлении никогда не изменится). Переданное через @C1Ver значение rowversion в этом примере равно 0x1026, это значение также было присвоено при первоначальной вставке. Если предположить, что никакой другой пользователь или процесс не модифицировал этого клиента в течение двух последних секунд, то обновление закончится успешно и в @C1Ver будет возвращено новое значение rowversion:

| C1Id  | Crt                 | Mod                 | C1Ver               |
|-------|---------------------|---------------------|---------------------|
| ----- |                     |                     |                     |
| 1     | 2007-11-22 14:14:21 | 2007-11-22 14:14:23 | 0x00000000000001028 |

Этот результат подтверждает, что обновление выполнилось успешно. Несмотря на то, что время создания 14:14:21 не изменилось (и оно никогда не будет меняться), время модификации 14:14:23 указывает на наше обновление, которое произошло через две секунды. Мы видим также, что значение rowversion изменилось с 0x1026 на 0x1028.

Если бы другой пользователь (или процесс) изменил строку в течение двух секунд между временем создания клиента (14:14:21) и временем нашей попытки обновления (14:14:23), то ее значение rowversion не было бы равно 0x1026. В этом случае наша попытка обновления закончилась бы неудачей (выполнением инструкции RAISERROR, которую мы предусмотрели в хранимой процедуре):

```
Msg 50000, Level 18, State 1, Procedure uspUpsertCustomer, Line 36
Optimistic concurrency violation
```

## Синтаксис INSERT OVER DML

SQL Server 2005 ввел (для инструкций INSERT, UPDATE и DELETE) выражение OUTPUT как способ захвата измененных данных, получившихся в результате этих операций DML. До и после этого моментальные снимки изменившихся данных предоставляются через специальные псевдотаблицы INSERTED и DELETED (которые обычно связаны с триггерами), так что нам нет больше нужды обращаться к триггерам (чего по возможности вообще следует избегать) для сбора такой информации. Как вы уже видели в предыдущем разделе, новая инструкция MERGE в SQL Server 2008 также поддерживает выражение OUTPUT для этой цели. При использовании совместно с MERGE выражение OUTPUT может делать выборку из псевдотаблиц INSERTED и DELETED, а также

из специального виртуального столбца `$action` типа `nvarchar(10)`, который возвращает значение `'INSERT'`, `'UPDATE'` или `'DELETE'` в зависимости от действия, выполненного для данной строки операцией слияния.

Сгенерированные выражением `OUTPUT` результаты могут быть сохранены в другой таблице или табличной переменной при помощи `OUTPUT...INTO`, что позволяет без труда поддерживать историческую регистрацию изменений в базе данных. В SQL Server 2008 у нас теперь есть гораздо более высокая гибкость в том, как мы обрабатываем сохраненные данные, особенно в отношении фильтрации. Поскольку использование `OUTPUT...INTO` выводит все собранные выражением строки в таблицу назначения или табличную переменную, то эти данные отфильтровать невозможно.

## Расширение инструкции **OUTPUT...INTO**

Эта неспособность фильтровать данные изменений из `OUTPUT...INTO` была устранена в SQL Server 2008, но не путем добавления в `OUTPUT...INTO` нового выражения `WHERE`, как можно было бы подумать. Новый инструмент с названием **INSERT OVER DML** предоставляет специальный синтаксис, при помощи которого вы обертываете в инструкцию `INSERT INTO... SELECT` любую инструкцию DML (`INSERT`, `UPDATE`, `DELETE` или `MERGE`), которая имеет выражение `OUTPUT` (вместо того, чтобы использовать `OUTPUT...INTO` в самой инструкции DML). Тонкая, но принципиальная разница состоит в том, что `OUTPUT...INTO` собирает изменения, в то время как новый синтаксис **INSERT OVER DML** потребляет захваченные выражением `OUTPUT` изменения. Используя изменения `OUTPUT` как источник для стандартной инструкции `INSERT INTO...SELECT`, мы можем применить к инструкции `INSERT INTO...SELECT` любое необходимое нам выражение `WHERE`.

### **ПРИМЕЧАНИЕ**

**INSERT OVER DML** — это данное компанией Microsoft название новой способности SQL Server 2008 использовать и фильтровать изменения данных, собранные выражением `OUTPUT` любой инструкции DML. В T-SQL нет инструкции **INSERT OVER DML**.

За исключением этого, при помощи **INSERT OVER DML** вы не сможете сделать ничего сверх того, что вы можете сделать при помощи `OUTPUT...INTO`. Но недооценить важность способности фильтрации измененных данных означает полностью упустить смысл **INSERT OVER DML**. Имея возможность фильтровать при помощи выражения `WHERE`, вы теперь можете применять бизнес-логику для

принятия решения о том, какие собранные измененные данные направляются в таблицу назначения, а какие — нет. Для этой возможности имеется множество практических применений, в том числе процессы извлечения, преобразования и загрузки (extract, transform, and load, ETL) для хранилищ данных, отслеживание истории медленно меняющихся измерений или любой другой сценарий, при котором вы хотите фильтровать изменения из выхода DML, которые сбрасываются во вспомогательную таблицу для обработки.

Рассмотрим простой пример аудита всех изменений, которые публикуются из одной таблицы в другую. В таблице `Book` у нас поддерживается главный список цен на книги и их местоположений на полках, причем каждая книга идентифицируется кодом ISBN (International Standard Book Number). Каждую неделю мы получаем новую таблицу с названием `WeeklyChanges`, которая содержит обновления данных по книгам (с ключом по коду ISBN). Кроме изменений цен и местоположений для существующих книг, таблица еженедельных обновлений может содержать также и новые книги. Вы уже знаете, что инструкция `MERGE` способна обработать такой сценарий (фактически она была создана специально для него) и может эффективно выполнить соответствующие обновления и вставки из таблицы `WeeklyChanges` в таблицу `Book` (при помощи соединения по ISBN и использования выражений `WHEN MATCHED` и `WHEN NOT MATCHED`). Мы знаем также, что мы можем использовать выражение `OUTPUT...INTO` для сбора всех изменений в данных, выполненных при слиянии, и сохранения их в исторической таблице. Поэтому давайте начнем с использования `OUTPUT...INTO` для вывода этих изменений в таблицу `BookHistory`. Затем мы изменим наш подход (используем `INSERT OVER DML`) и посмотрим, как это улучшит наши возможности управления процессом архивирования изменений в данных.

Давайте начнем с создания таблиц `Book` и `WeeklyChange`, имеющих одинаковую схему (листинг 2.64):

#### Листинг 2.64. Создание таблиц `Book` и `WeeklyChange`

```
CREATE TABLE Book(  
    ISBN varchar(20) PRIMARY KEY,  
    Price decimal,  
    Shelf int)  
CREATE TABLE WeeklyChange(  
    ISBN varchar(20) PRIMARY KEY,  
    Price decimal,  
    Shelf int)
```

Затем мы создаем таблицу BookHistory, которая будет принимать изменения, собранные выражением `OUTPUT INTO` инструкции `MERGE` (листинг 2.65):

**Листинг 2.65. Создание таблицы BookHistory**

```
CREATE TABLE BookHistory(  
    Action nvarchar(10),  
    NewISBN varchar(20),  
    NewPrice decimal,  
    NewShelf int,  
    OldISBN varchar(20),  
    OldPrice decimal,  
    OldShelf int,  
    ArchivedAt datetime2)
```

Теперь мы можем создать нашу хранимую процедуру `uspUpdateBooks`, которая выполняет операцию слияния для ежедневного обновления и записывает все изменения данных в таблицу BookHistory (листинг 2.66):

**Листинг 2.66. Использование OUTPUT...INTO вместе с MERGE**

```
CREATE PROCEDURE uspUpdateBooks AS  
BEGIN  
    MERGE Book AS B  
        USING WeeklyChange AS WC  
        ON B.ISBN = WC.ISBN  
    WHEN MATCHED AND (B.Price <> WC.Price OR B.Shelf <> WC.Shelf) THEN  
        UPDATE SET B.Price = WC.Price, B.Shelf = WC.Shelf  
    WHEN NOT MATCHED THEN  
        INSERT VALUES(WC.ISBN, WC.Price, WC.Shelf)  
        OUTPUT $action, inserted.*, deleted.*, SYSDATETIME()  
        INTO BookHistory;  
END
```

Подробные пояснения для этого кода не нужны, поскольку мы подробно описали `MERGE` ранее в этой же главе. Однако следует обратить внимание на выражение `OUTPUT...INTO`. Это выражение записывает действия, новые значения, старые значения, а также текущие дату и время сервера в таблицу BookHistory (из виртуального столбца `$action`, столбцов псевдотаблиц

INSERTED и DELETED, а также функции SYSDATETIME). В нашем конкретном сценарии \$action может вернуть только строку 'INSERT' или 'UPDATE' (поскольку мы не обрабатываем удаления).

Теперь мы заполняем таблицы Book и WeeklyChange (для подготовки нашего обновления):

```
INSERT INTO Book VALUES('A', 100, 1)
INSERT INTO Book VALUES('B', 200, 2)
INSERT INTO WeeklyChange VALUES('A', 101, 1)
INSERT INTO WeeklyChange VALUES('C', 300, 3)
```

Таблица WeeklyChange показывает изменение цены книги A со 100 долларов до 101 доллара, а также добавляет новую книгу C (стоимостью 300 долларов) на стеллаж 3. Когда мы выполняем хранимую процедуру uspUpdateBooks, эти две операции происходят в полном соответствии с нашими ожиданиями (что подтверждается сообщением о количестве обработанных строк):

```
EXEC uspUpdateBooks
GO
(2 row(s) affected)
```

Кроме того, действия и выполненные этими операциями вставки и обновления изменения данных (а также даты и время их выполнения) сохранены в таблицу BookHistory (в результате выполнения выражения OUTPUT...INTO):

```
SELECT* FROM BookHistory
GO
Action NewISBN NewPrice NewShelf OldISBN OldPrice OldShelf ArchivedAt
-----
UPDATE A          101          1          A          100          1          2007-11-25
14:47:23.9907552
INSERT C          300          3         NULL         NULL         NULL         2007-11-25
14:47:23.9907552
```

Пока все хорошо, не так ли? Но что случится, если наши требования несколько изменятся? После некоторого раздумья мы видим, что для сохранения сведений о вставках (которые ранее не имели старых значений и поэтому не имеют особой исторической ценности, кроме даты и времени создания, которые мы могли бы при желании сохранить в таблице Book) будет расходоваться слишком много места. Поскольку все книги необходимо вставлять, но большинство книг не обновляется, то в результате в таблице BookHistory получится совершенно непропорциональное преобладание вставок над обновлениями. Получается большой объем данных, имеющих минимальную ценность (или не имеющих ее вовсе). К несчастью, не существует способа отфильтровать вставки и сохранить только обновления из OUTPUT...INTO.



Решение для этой проблемы предоставляет INSERT OVER DML в SQL Server 2008. В нашем сценарии нас интересует сбор только обновлений (но не вставок). Для того чтобы сделать все еще интереснее, мы добавим исторические данные в саму таблицу Book (вместо отдельной таблицы BookHistory, хотя это тоже возможно — при помощи OUTPUT...INTO). Мы добавим в таблицу Book новый столбец с названием ArchivedAt, который будет содержать дату и время сервера для исторических записей (добавляемых при обновлении книг). Для каждой книги всегда будет существовать одна уникальная строка, содержащая текущие значения и значение NULL в столбце ArchivedAt.

Начинаем мы так же, как и в прошлый раз — с создания наших таблиц (листинг 2.67):

**Листинг 2.67. Создание новых версий таблиц Book и WeeklyChange для использования с INSERT OVER DML**

```
CREATE TABLE Book(  
    ISBN varchar(20),  
    Price decimal,  
    Shelf int,  
    ArchivedAt datetime2)  
CREATE UNIQUE CLUSTERED INDEX  
    UI_Book ON Book(ISBN, ArchivedAt)  
CREATE TABLE WeeklyChange(  
    ISBN varchar(20) PRIMARY KEY,  
    Price decimal,  
    Shelf int)
```

Обратите внимание, что на этот раз мы создали уникальный индекс по комбинации столбцов ISBN и ArchivedAt. Мы уже не можем создать первичный ключ только по столбцу ISBN (как раньше), поскольку таблица Book теперь будет содержать множество записей с одинаковым номером ISBN (потому что в ней хранятся как текущие данные, так и исторические данные об изменениях). Только одна строка с данным конкретным ISBN будет иметь значение NULL в столбце ArchivedAt — это строка, содержащая текущие значения для книги. Все остальные строки с этим же ISBN будут иметь в столбце ArchivedAt отличные от NULL значения, которые являются временной меткой предыдущего обновления. Эти строки содержат исторические значения, архивированные для данной книги в данный момент времени. Путем обеспечения уникальности по ISBN и ArchivedAt мы гарантировали то, что в таблице Books сохраняется только одна текущая строка для книги, поскольку таблица не потерпит двух строк с одинаковыми кодами ISBN и значениями NULL в столбце ArchivedAt.

## Использование ключевого слова **CHANGES**

В листинге 2.68 мы реализуем модифицированную версию `uspUpdateBooks`, которая использует новый синтаксис `INSERT OVER DML` для сбора и сохранения исторических данных по обновлениям (которые добавляются в таблицу `Book`).

### Листинг 2.68. Использование инструкции **MERGE** с синтаксисом **INSERT OVER DML**

```
CREATE PROCEDURE uspUpdateBooks AS
BEGIN
    INSERT INTO Book(ISBN, Price, Shelf, ArchivedAt)
    SELECT ISBN, Price, Shelf, GETDATE() FROM
    (MERGE Book AS B
    USING WeeklyChange AS WC
    ON B.ISBN = WC.ISBN AND B.ArchivedAt IS NULL
    WHEN MATCHED AND (B.Price <> WC.Price OR B.Shelf <> WC.Shelf) THEN
        UPDATE SET Price = WC.Price, Shelf = WC.Shelf
    WHEN NOT MATCHED THEN
        INSERT VALUES(WC.ISBN, WC.Price, WC.Shelf, NULL)
    OUTPUT $action, WC.ISBN, Deleted.Price, Deleted.Shelf
    ) CHANGES(Action, ISBN, Price, Shelf)
    WHERE Action = 'UPDATE';
END
```

Поскольку теперь у нас в таблице `Book` имеются исторические данные, которые не следует использовать при операциях слияния, то мы добавили критерий в предикат соединения (после ключевого слова `ON`), который проверяет наличие значения `NULL` в столбце `ArchivedAt`. Таким образом, инструкции `MERGE` дается указание рассматривать в качестве целевых строк только текущие книги и полностью игнорировать все архивные записи.

Главным здесь является ключевое слово `CHANGES`. Оно предоставляет столбцы выражения `OUTPUT` (определенного во внутренней инструкции `MERGE`) выражению `WHERE` внешней инструкции `INSERT INTO...SELECT`. В нашем примере в их число входят: виртуальный столбец `$action`, код `ISBN`, а также старые значения цены и местоположения на стеллаже (заменяемые операцией обновления).

Представляя виртуальный столбец `$action` через ключевое слово `CHANGES` как `Action`, инструкция `INSERT INTO...SELECT` может применить выражение `WHERE`

для отфильтровывания действий по новым вставленным книгам и добавления в таблицу Book только действий по изменяемым в текущий момент книгам. Этого нельзя достичь при помощи выражения `OUTPUT INTO` инструкции `MERGE`, но можно сделать при помощи именно такого синтаксиса `INSERT OVER DML`.

В нашем случае мы отфильтровываем действия вставки при помощи простого критерия `WHERE Action = 'UPDATE'`. Конечно, мы могли бы применить и гораздо более сложную логику, изучив любые возвращаемые из выражения `OUTPUT` и предоставляемые ключевым словом `CHANGES` столбцы данных. Именно в этом и состоит мощь синтаксиса `INSERT OVER DML`.

Давайте двигаться по нашему сценарию шаг за шагом. Мы начнем с двух книг (A и B) и одного изменения цены (книга A, со 100 долларов до 110):

```
INSERT INTO Book VALUES('A', 100, 1, NULL)
INSERT INTO Book VALUES('B', 200, 2, NULL)
INSERT INTO WeeklyChange VALUES('A', 110, 1)
SELECT* FROM Book
SELECT* FROM WeeklyChange
GO
```

```
ISBN Price Shelf ArchivedAt
```

```
----
```

```
A      100    1      NULL
```

```
B      200    2      NULL
```

```
(2 row(s) affected)
```

```
ISBN Price Shelf
```

```
----
```

```
A      110    1
```

```
(1 row(s) affected)
```

Когда мы теперь выполняем `uspUpdateBooks`, то внутренняя инструкция `MERGE` обновляет цену книги A в таблице Book и отправляет ее первоначальные значения в выражение `OUTPUT`. Значения выражения `OUTPUT` затем потребляются внешней инструкцией `INSERT INTO...SELECT` при помощи ключевого слова `CHANGES`. Поэтому внешняя инструкция может использовать исходные значения по книге для вставки исторических данных обратно в таблицу Book (причем значение `ArchivedAt` установлено в текущую дату и время сервера, в данном примере это примерно пять часов вечера 25 ноября 2007 года):

```
EXEC uspUpdateBooks
SELECT* FROM Book
GO
ISBN Price Shelf ArchivedAt
```

```

-----
A      110    1      NULL
A      100    1      2007-11-25 16:57:19.8600000
B      200    2      NULL
(3 row(s) affected)

```

Мы видим, что текущие данные по книге А (строка со значением NULL в столбце ArchivedAt) содержат стоимость 110 долларов, и мы видим также предыдущие данные по книге А, которые изменились примерно в пять часов вечера 25 ноября 2007 г. Несколько позже мы получим новый набор обновлений. На этот раз книга А переместилась со стеллажа 1 на стеллаж 6, и добавлена новая книга С:

```

DELETE FROM WeeklyChange
INSERT INTO WeeklyChange VALUES('A', 110, 6)
INSERT INTO WeeklyChange VALUES('C', 300, 3)
GO

```

Как и при первом выполнении хранимой процедуры, текущая строка по книге А обновляется, а моментальный снимок предыдущего содержимого строки добавляется в таблицу (с текущей датой и временем сервера):

```

EXEC uspUpdateBooks
SELECT* FROM Book
GO
ISBN Price Shelf ArchivedAt
-----
A      110    6      NULL
A      100    1      2007-11-25 16:57:19.8600000
A      110    1      2007-11-25 16:58:36.1900000
B      200    2      NULL
C      300    3      NULL
(5 row(s) affected)

```

Теперь книга А имеет две исторических записи, в которых содержатся значения, сохраненные во время двух предыдущих обновлений (которые можно идентифицировать по значениям даты и времени в столбце ArchivedAt). Мы видим также новую строку для книги С, которая была вставлена выражением WHEN NOT MATCHED инструкции MERGE.

Однако обратите внимание, что для книги С нет строки истории, поскольку действие вставки (которое было обнаружено выражением OUTPUT инструкции MERGE) было затем отфильтровано в выражении WHERE внешней инструкции INSERT INTO...SELECT. Если бы мы не отфильтровывали действий по вставке в выражении WHERE, в таблицу была бы добавлена еще одна историческая

запись с бессмысленными значениями NULL (как для Price, так и для Shelf). То есть мы избегаем увеличения количества таких исторических строк, которые появлялись бы в результате каждой новой книги (фильтруя действия OUTPUT при помощи INSERT OVER DML). Мы также ликвидировали лишние столбцы для новых значений, которые мы использовали в таблице BookHistory предыдущего примера (поскольку эти значения сохраняются в каждой обновленной версии, архивируемой в самой таблице Book). Мы исключили также необходимость хранения столбца Action, поскольку мы регистрируем только действия по обновлению. И в конечном итоге мы получили преимущество за счет экономии большого объема хранения и использования единственной инструкции INSERT OVER DML.

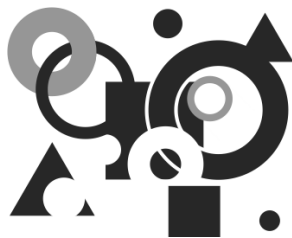
Как вы видите, комбинация новых функциональных возможностей MERGE и INSERT OVER DML в SQL Server 2008 является очень мощным дополнением T-SQL. Мы рекомендуем использовать эти новые функциональные возможности (вместо традиционного подхода с несколькими инструкциями) во всех новых разработках, где их можно применить. Вы несомненно оцените и повышенную управляемость, которая является следствием сведения нескольких инструкций в одну.

## Резюме

В этой главе мы описали несколько основных положений. Версии SQL Server 2005 и 2008 ввели много улучшений в ядро T-SQL. Вы видите, что компания Microsoft предлагает много новых мощных средств для запроса сложных данных при помощи T-SQL, в том числе PIVOT, UNPIVOT, APPLY, функции ранжирования, улучшения TOP, GROUPING SETS и рекурсивные операции при помощи обобщенных табличных выражений (CTE). Вы научились использовать возвращающие табличные значения параметры (TVP) для передачи наборов строк между клиентскими частями, серверами, хранимыми процедурами и определяемыми пользователем функциями (UDF). Для использования в программировании были добавлены новые типы даты и времени. Добавлены также новые возможности языка DML (при помощи MERGE и INSERT OVER DML).

T-SQL по-прежнему остается основным средством вашего взаимодействия с данными. Когда вам необходимо взаимодействовать с операционной системой или с .NET Framework, вы можете использовать CLR и написать ваши запросы на языке Visual Basic .NET или Visual C#. В следующей главе вы узнаете о том, как это делается.

## ГЛАВА 3



# Изучаем SQL CLR

*Эндрю Брест*

Главным рекламным лозунгом Microsoft SQL Server 2005 была интеграция в него общезыковой исполняющей среды Microsoft .NET Common Language Runtime (CLR). Такое архитектурное изменение дало SQL Server возможность применять определенные классы .NET как базовые типы данных, а также использовать языки .NET для создания хранимых процедур, триггеров, функций и даже определяемых пользователем агрегатов. Эти способности были перенесены в SQL Server 2008 и расширены. Они лежат в основе различных новых типов данных SQL Server 2008 (в том числе *geometry* и *geography*, описанных в *главе 9*).

### **ПРИМЕЧАНИЕ**

В этой главе интеграцию CLR и SQL Server мы обозначим как "функции SQL CLR" и будем считать хранимые процедуры, триггеры, функции, агрегаты и определяемые пользователем типы (UDT) пятью основными элементами SQL CLR.

Посмотрим правде в лицо: Transact SQL (T-SQL) является по существу хакерской поделкой. В начале разработки SQL Server компании Microsoft и Sybase взяли SQL (декларативный язык) и добавили объявление переменных, условное ветвление, циклы и параметризированные процедуры (чтобы сделать его квази-процедурным языком). Несмотря на то, что он исключительно удачный и полезный, языку T-SQL всегда не хватало (да и сейчас не хватает) многих возможностей полноценного процедурного языка программирования.

Этот недостаток заставил некоторых людей писать хранимые процедуры T-SQL, которые трудно читать. Других он заставил размещать в коде промежу-

точного уровня ту логику, которую они предпочли бы реализовать в базе данных. Некоторых он даже заставил полностью отказаться от хранимых процедур и использовать в своих приложениях динамический SQL (чего мы делать не рекомендуем). Из-за всех этих сложностей по обходу процедурных ограничений T-SQL интеграция с CLR является желанной функциональной возможностью SQL Server, и она уже привлекла к себе внимание рынка.

В то же время T-SQL (и, вероятно, сам SQL) значительно превосходит процедурные языки в смысле выполнения запросов и манипулирования данными. Его синтаксис и реализация существенно превосходят подход, основанный на процедурных итерациях по строкам данных. Все это остается справедливым в версиях SQL Server 2005 и 2008, причем в последней версии T-SQL был значительно расширен (как это описано в *главе 2*), что делает его еще более ценным.

Несмотря на то, что после выпуска SQL Server 2005 эйфория по поводу SQL CLR значительно улеглась, эта важная технология используется компанией Microsoft для поддержки новых типов данных в SQL Server 2008. Поэтому SQL CLR по-прежнему ставит разработчиков приложений баз данных на перепутье. Мы должны изучать, как работают функциональные возможности SQL CLR, и одновременно вырабатывать в себе непростое понимание того, в какой момент следует (вместо функций SQL CLR) начать использовать T-SQL. Мы должны сопротивляться соблазну полностью перевести на .NET наши базы данных, но учиться использовать преимущества SQL CLR везде, где это имеет смысл. Цель этой главы — помочь вам изучить использование функций SQL CLR и развить инстинкт по их разумному применению.

В этой главе вы узнаете:

- ❑ как включить (отключить) интеграцию SQL CLR на сервере SQL Server;
- ❑ как сервер SQL Server использует код CLR (при помощи загрузки .NET-сборок);
- ❑ как совместно использовать SQL Server 2008 и Microsoft Visual Studio 2008 для простого и быстрого написания кода SQL CLR и его развертывания;
- ❑ как развернуть код SQL CLR без использования Visual Studio (при помощи инструкций T-SQL, с использованием программы SQL Server Management Studio или без нее);
- ❑ как при помощи CLR создать простые хранимые процедуры, триггеры, функции, агрегаты и UDT, как использовать их в ваших базах данных и применять их из T-SQL;

- ❑ как поставщик стандартного клиента SQL Server и новая библиотека серверной стороны могут совместно использоваться для реализации функциональных возможностей SQL CLR;
- ❑ как работает система безопасности SQL CLR, и как настроить разрешения безопасности для ваших сборок;
- ❑ когда использовать функциональные возможности SQL CLR и когда пользоваться вместо них языком T-SQL.

## Приступаем: включаем интеграцию с CLR

Перед тем как начать использовать функциональные возможности SQL CLR, вам нужно сначала узнать, как включить их. Как и у большинства новых продуктов семейства Microsoft Windows Server System, самые передовые функции SQL Server 2008 по умолчанию отключены. Причина вполне разумна: каждая дополнительно включенная функциональная возможность представляет собой потенциальный объект для атак на безопасность продукта, и такая уязвимость совершенно неоправдана в том случае, если эта возможность не используется.

Функции SQL CLR сервера SQL Server 2008 современны и могут быть очень полезными, но в техническом плане они не являются существенными. Можно создавать высокопроизводительные базы данных и логику на стороне сервера и без интеграции с CLR, и именно поэтому она по умолчанию отключена.

Не стоит расстраиваться: включить интеграцию легко. Компания Microsoft предоставляет системную хранимую процедуру для включения/выключения интеграции SQL CLR. Вы можете включить интеграцию SQL CLR при помощи управления на основе политик (Policy-Based Management, PBM), как описано в *главе 4*. Здесь же мы описываем только использование T-SQL.

Откройте SQL Server Management Studio, а затем подключитесь к серверу, который вы хотите сконфигурировать. Потом в окне запросов введите следующие команды и нажмите кнопку **Execute** на панели инструментов **Management Studio SQL Editor**:

```
sp_configure 'clr enabled', 1
GO
RECONFIGURE
GO
```



Вот и все! Для отключения интеграции SQL CLR используйте во втором параметре вызова `sp_configure` значение 0 (вместо 1).

### **СОВЕТ**

Помните, что это сработает отовсюду, что может подключиться к SQL Server, а не только из Management Studio. Фактически вы могли бы выдать предыдущий командный текст из вашего собственного кода при помощи метода `ExecuteNonQuery` объекта `SqlCommand` ADO.NET (если ваш код может подключиться к серверу, а ваш сервер может аутентифицировать вас как пользователя, обладающего серверной ролью `sysadmin`).

После включения интеграции SQL CLR вы можете начинать писать код SQL CLR. Перед этим нам необходимо обсудить интеграцию Visual Studio/SQL Server, и когда ее необходимо использовать.

## **Интеграция Visual Studio/SQL Server**

Visual Studio 2008 и SQL Server 2008 тесно интегрируются между собой сразу несколькими способами. Однако важно понимать, что использование интеграции с Visual Studio совершенно не обязательно, и ее вполне можно заменить языком T-SQL. С выпуском версии SQL Server 2005 язык T-SQL был расширен новыми командами языка определения данных (DDL) обслуживания сборок, типов и агрегатов CLR, а имеющиеся в нем команды хранимых процедур, триггеров и функций были улучшены для распознавания кода внутри развернутых сборок. Visual Studio может выполнять эти команды от вашего имени и облегчить написание классов и функций SQL CLR.

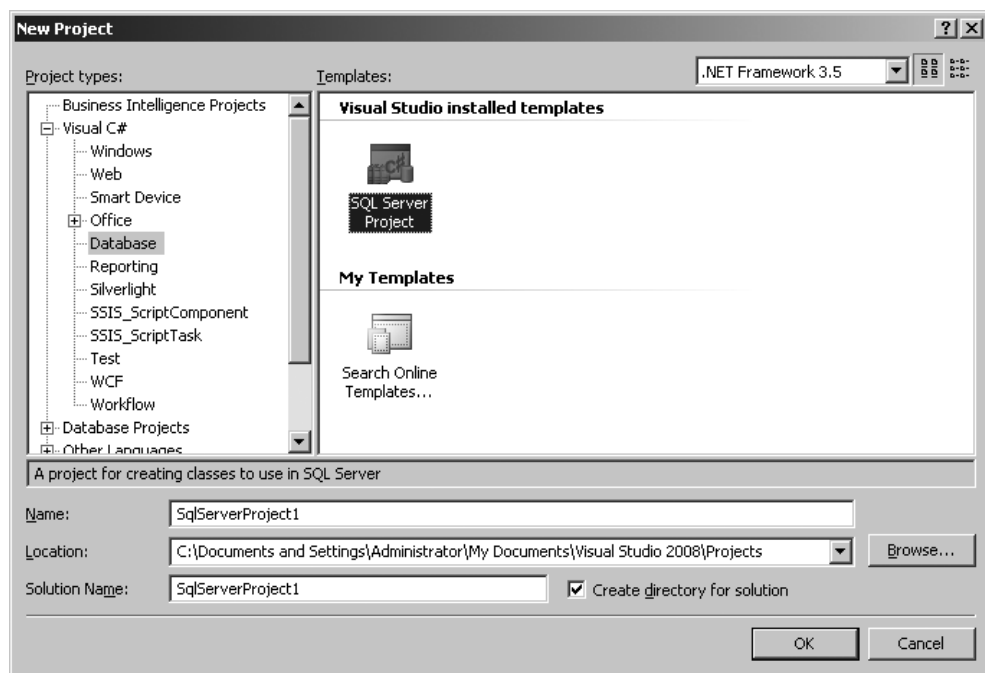
Мы считаем, что все разработчики должны знать об имеющихся возможностях кодирования и развертывания, как с помощью Visual Studio, так и более "ручных". Поскольку мы описываем все основные области программирования SQL CLR, то будем обсуждать развертывание с обеих точек зрения. Сейчас мы опишем некоторые общие вопросы интеграции с Visual Studio, а затем перейдем к описанию разработки для SQL CLR.

## **Проекты SQL Server Project в Visual Studio**

Visual Studio 2008 предоставляет специальный тип проекта SQL Server Project и среди проектов этого типа — предопределенные шаблоны для пяти базовых элементов SQL CLR. Эти шаблоны содержат специфические атрибуты

кода и заглушки функций, которые позволяют легко создавать код SQL CLR. Атрибуты используются в Visual Studio для развертывания сборки и ее хранимых процедур, триггеров и всего остального в вашу базу данных. Некоторые из них используются и сервером SQL Server для правильного применения функций, UDT и агрегатов.

Для тестирования шаблонов и нового типа проектов запустите Visual Studio 2008, а затем создайте новый проект при помощи выбора в меню **File** пункта **New Project** либо щелчка по значку **New Project** панели инструментов (либо комбинации клавиш <Ctrl>+<Shift>+<N>), либо щелчка по ссылке **Create: Project** на стартовой странице **Visual Studio Start Page**). В диалоговом окне **New Project** (рис. 3.1) выберите узел **Database** в древовидном представлении **Project Types** с левой стороны (узел **Database** находится под родительским узлом вашего языка программирования, на рис. 3.1 это язык C#), а затем щелкните по значку **SQL Server Project** в списке **Templates** справа. Введите при желании ваше собственное название для проекта, а затем нажмите **OK**.

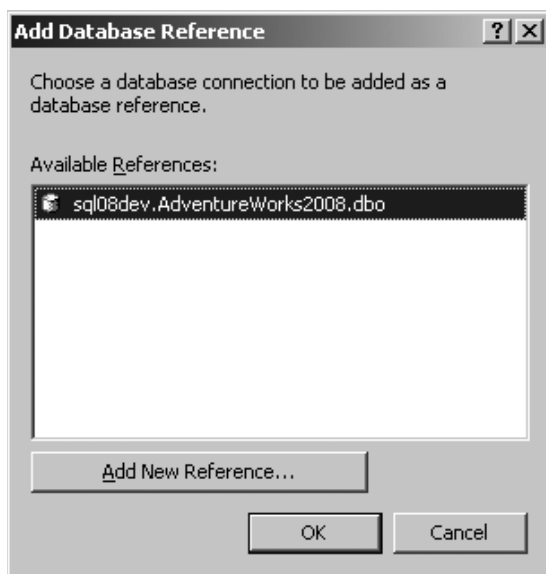


**Рис. 3.1.** Диалоговое окно **New Project** в Visual Studio 2008 с выделенным типом проекта **SQL Server project**

**СОВЕТ**

Если вы используете версию Visual Studio Team Edition for Database Professionals или Visual Studio Team Suite, можете создать проект SQL CLR и выбрать в узле **SQL-CLR** дерева **Project Types** элемент **Visual Basic** или **C# SQL Server Project** в списке **Templates** справа.

Появится диалоговое окно **Add Database Reference** (как показано на рис. 3.2).



**Рис. 3.2.** Диалоговое окно **Add Database Reference**

Visual Studio обеспечивает автоматическое развертывание вашего кода SQL CLR, связав ваш проект с конкретным сервером и базой данных при помощи ссылки (подключения). Все ранее определенные в окне **Server Explorer** подключения к базам данных видны в окне **Add Database Reference**. Здесь же имеется и кнопка **Add New Reference**, которая позволяет вам при необходимости определить новое подключение. Выберите существующее подключение или определите новое, а затем нажмите кнопку **OK**. Проект откроется.

**ПРИМЕЧАНИЕ**

Если в окне **Server Explorer** еще не было определено ни одно подключение, то вместо диалогового окна **Add Database Reference** появится диалоговое окно **New Database Reference**. В этом окне вы можете указать сер-

вер, регистрационную запись, а также подробности нового подключения к базе данных.

### СОВЕТ

После нажатия **OK** в диалоговом окне **Add Database Reference** может появиться напоминание включить отладку SQL CLR для данного подключения. Если это произойдет, выберите **Yes** в двух последующих окнах сообщений.

Вы можете без труда добавить в проект предварительно сконфигурированные классы пяти основных объектов SQL CLR несколькими способами: напрямую из опции **Project** главного меню или из подменю **Add** в контекстном меню **Solution Explorer** вашего проекта (рис. 3.3).

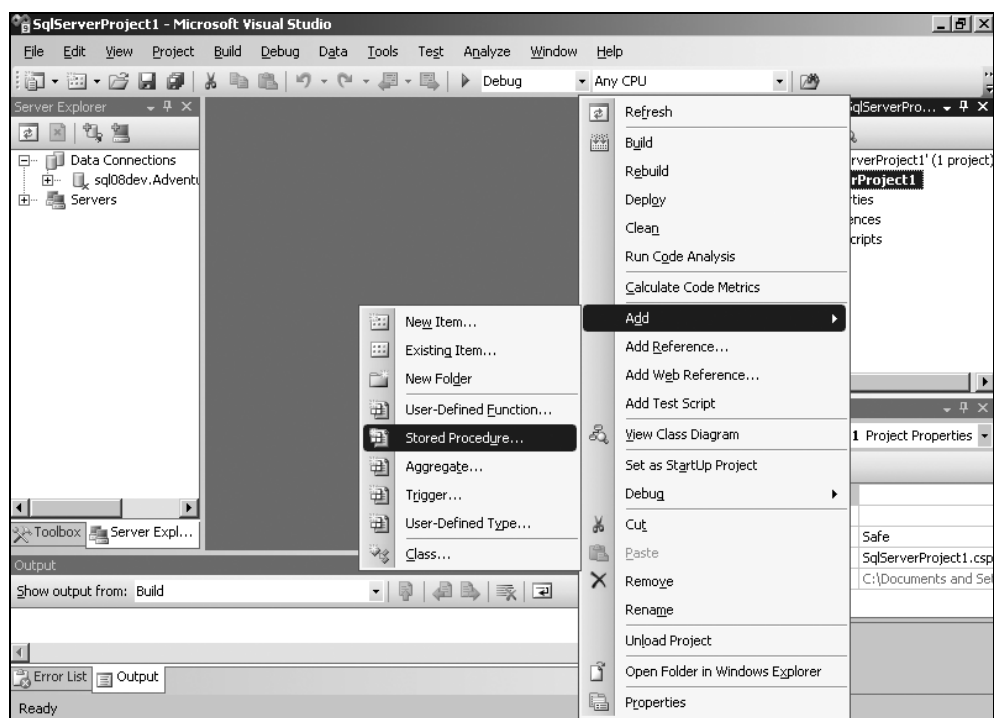


Рис. 3.3. Контекстное меню **Solution Explorer** и его подменю **Add**

Вы можете также добавить предварительно сконфигурированные классы из диалогового окна **Add New Item** (рис. 3.4), которое отображается после выбора **Add New Item** или в контекстном меню **Solution Explorer** вашего проекта.

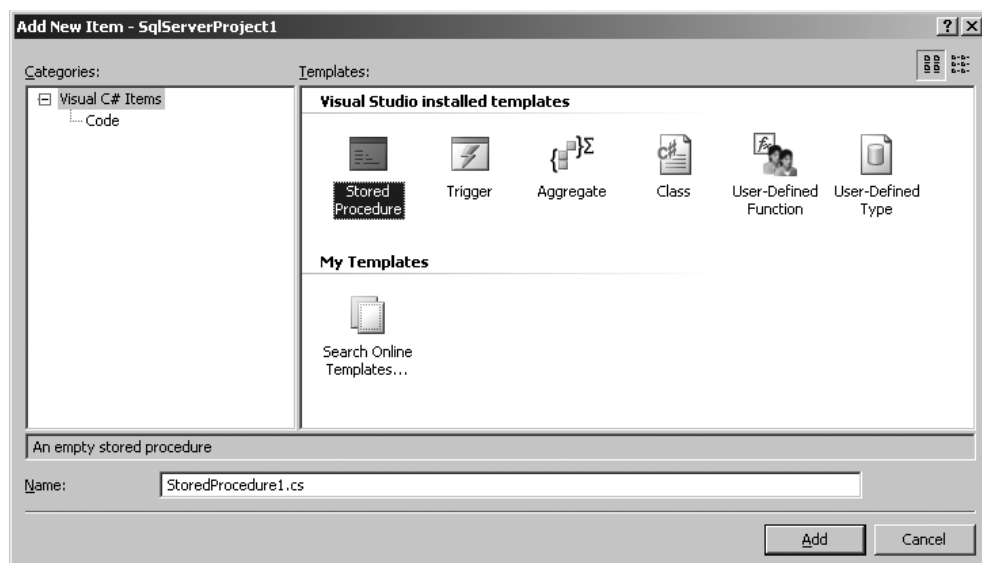


Рис. 3.4. Диалоговое окно **Add New Item** проекта SQL Server project в Visual Studio

## Автоматическое развертывание

После своего открытия проекты SQL Server добавляют опции **Deploy** в меню **Build** главного меню Visual Studio. Кроме того, кнопка **Play (Start Debugging)** и опции **Start Debugging**, **Start Without Debugging**, а также **Step Over** в меню **Debug** (и соответствующие им клавиатурные комбинации <F5>, <Ctrl>+<F5> и <F10>) производят развертывание сборки проекта (в дополнение к выполнению обозначенных в них функций).

По мере того как вы будете учиться, вы получите возможность более тонкого управления процессом развертывания.

## Атрибуты кода SQL CLR

Для разработчиков под SQL CLR предоставляется несколько атрибутов .NET-кода, они содержатся в пространстве имен `Microsoft.SqlServer.Server`. Многие из них вставляются в ваш код при использовании шаблонов в проектах типа SQL Server (например, инструкция `using` для псевдонима самого пространства имен `Microsoft.SqlServer.Server`). Если вы примете решение разрабатывать код без этих шаблонов, то сами должны добавить соответ-

вующие атрибуты и инструкцию `using`. Хотя все атрибуты предоставляются в одном и том же пространстве имен, некоторые из них используются исключительно в Visual Studio, а другие используются и Visual Studio, и в SQL Server.

Мы опишем атрибуты `SqlProcedure`, `SqlFunction`, `SqlTrigger`, `SqlUserDefinedAggregate` и `SqlUserDefinedType` и не будем описывать атрибуты `SqlFacet` и `SqlMethod`.

Точно так же мы описываем не все параметры атрибутов. И иногда описываем только некоторые из возможных значений, которые можно передать этим атрибутам. Например, `SqlFunction` имеет несколько параметров, но мы описываем только `Name`, `FillRowMethodName` и `TableDefinition`. Для `SqlUserDefinedAggregate` и `SqlUserDefinedType` мы опишем только одно значение параметра `Format`.

Того, что мы опишем, будет достаточно для реализации вами простой, средней и даже некоторой продвинутой функциональности (с использованием всех пяти базовых элементов SQL CLR). Те атрибуты и параметры, которые мы не описываем, полезны в основном для оптимизации кода SQL CLR (причем они очень хорошо документированы в SQL Server Books Online и в статьях MSDN).

### **ОТНОСИТЕЛЬНО ПРИМЕРОВ КОДОВ**

Примеры .NET-кодов для этой главы предоставлены в двух версиях на сайте данной книги. Основной материал дан в виде проекта типа SQL Server для Visual Studio, доступ к которому можно получить, если открыть файл решения `Chapter03.sln` в подкаталоге `Chapter03` каталога примеров кодов данной главы. Мы также предоставляем этот код в виде стандартного проекта библиотеки классов `Class Library`, доступ к которому можно получить, если открыть файл решения `Chapter03Manual.sln` в подкаталоге `Chapter03Manual`. Код в обоих проектах практически идентичен, хотя проект библиотеки классов не делает автоматического развертывания при выполнении в Visual Studio различных команд создания и отладки. При описании всех функциональных возможностей SQL CLR мы будем обсуждать, как происходит автоматическое развертывание из проекта SQL Server и как нужно делать командное развертывание для проекта библиотеки классов.

Мы обсудим также и выполнение тестовых скриптов для проекта SQL Server (в Visual Studio) и для проекта `Class Library` (в SQL Server Management Studio). Сопровождать это изучение будет проект `Management Studio`, доступ к которому можно получить, если открыть `Chapter03.ssmssln` в каталоге SSMS данной главы. Этот проект состоит из нескольких скриптов SQL, используемых для тестирования примеров кода SQL CLR, а также скрипта для очистки всего того, что создается в базе

данных примерами кодов и тестами. Проект содержит также файл скрипта с названием `CreateObjects.sql`, который развертывает сборку `Class Library` и находящиеся в ней элементы SQL CLR.

## Ваша первая хранимая процедура SQL CLR

Хотя программирование под SQL CLR может быть очень сложным и запутанным, на самом деле оно предлагает простую модель, которую может использовать любой .NET-разработчик, причем с высокой продуктивностью и в сжатые сроки. Это происходит потому, что суть функциональных возможностей SQL CLR состоит в том, что SQL Server 2008 способен загружать .NET-сборки в вашу базу данных, а затем позволяет вам использовать функции и типы этих сборок при определении столбцов, представлений, хранимых процедур, триггеров и функций.

Для того чтобы дать вам хорошее представление об интеграции SQL CLR, мы должны тщательно изучить ее функциональные возможности и методы. Однако перед этим давайте мы кратко пройдем по сценарию создания и выполнения хранимой процедуры SQL CLR. Это облегчит вам понимание отдельных функциональных возможностей (при их дальнейшем описании).

Строго говоря, любая сборка библиотеки классов .NET (в некоторых случаях необходимо использовать соответствующие атрибуты .NET-кода в ее классах и функциях) может быть загружена в вашу базу данных при помощи простой инструкции T-SQL. Для того чтобы увидеть, как просто все это работает, запустите Management Studio, а затем откройте окно запросов с использованием подключения к образцовой базе данных `AdventureWorks2008`. В каталоге примеров кодов для данной главы убедитесь в том, что файл `Chapter03.dll` находится в подкаталоге `VS\Chapter03Manual\Chapter03\bin\Debug`. Если родительским каталогом является `C:\Projects\Chapter03`, то вы можете загрузить сборку в базу данных `AdventureWorks2008` при помощи следующей команды T-SQL:

```
CREATE ASSEMBLY Chapter03
FROM 'C:\Projects\Chapter03\VS\Chapter03Manual
\Chapter03\bin\Debug\Chapter03.dll'
```

Команда `CREATE ASSEMBLY` имеет и другие синтаксические опции, но мы пока сосредоточимся на предыдущем варианте ее использования.

Те функции сборки, которые находятся внутри класса и выполняют локальные вычислительные задачи и определенные типы доступа к данным, могут быть легко предоставлены в качестве хранимых процедур, триггеров или функций SQL Server. Для обычных хранимых процедур, триггеров и функций для этого достаточно использовать инструкции T-SQL (`CREATE PROCEDURE`, `CREATE TRIGGER` или `CREATE FUNCTION`). В данной главе мы опишем все эти функции, но пока давайте создадим простую хранимую процедуру CLR.

Исходный код сборки Chapter03 вы сможете просмотреть, если откроете файл решения `VS\Chapter03Manual\Chapter03Manual.sln` в каталоге примеров кодов этой главы. В проекте имеется файл `Sprocs.cs` со следующим кодом:

```
using System.Data.SqlClient;
using Microsoft.SqlServer.Server;
public partial class Sprocs
{
    public static void spContactsQuick()
    {
        SqlContext.Pipe.ExecuteAndSend(new SqlCommand("SELECT *
                                                    FROM Person.Person"));
    }
}
```

Метод `spContactsQuick` был создан для подключения к базе данных, в которую загружена его сборка, выполнения `SELECT *` по таблице `Person.Person`, а затем использования специальных объектов на серверной стороне для отправки данных обратно в клиентское приложение. Для того чтобы сделать этот код CLR доступным через SQL Server в виде хранимой процедуры `spContactsQuick`, вы просто выполняете следующую команду из окна запросов Management Studio (которое вы уже открыли):

```
CREATE PROCEDURE spContactsQuick
AS EXTERNAL NAME Chapter03.Sprocs.spContactsQuick
```

### **ВАЖНО**

Убедитесь в том, что вы правильно ввели строку `Sprocs.spContactsQuick`. Эта фраза чувствительна к регистру.

Для проверки хранимой процедуры SQL CLR выполните ее из окна запросов Management Studio так же, как и любую другую хранимую процедуру:

```
EXEC spContactsQuick
```



Или просто так:

```
spContactsQuick
```

Management Studio в ответ должна показать вам содержимое таблицы `Person.Person` (во вкладке **Results** окна запросов).

Как хорошо видно из этого очень простого примера, написание хранимой процедуры CLR может быть очень простым и очень похоже на написание кода (который производит доступ к данным при помощи ADO.NET) клиентской стороны или промежуточного уровня. Самые большие отличия — это предоставление подключения к базе данных и тот факт, что данные должны быть отправлены обратно клиенту (вместо их загрузки в `SqlDataReader` и возвращения, манипулирования или отображения в пользовательском интерфейсе). Присутствие объекта `SqlContext` также отличает код SQL CLR от обычного кода доступа к данным ADO.NET. В следующем разделе мы рассмотрим использование объекта `SqlContext` и его свойства `Pipe`.

Эти только что показанные небольшие фрагменты кодов T-SQL и C#, конечно, не исчерпывают рассказ о SQL CLR. Использование метода `ExecuteAndSend` позволило нам опустить сразу несколько важных концепций. Существует три способа развертывания сборок, а вы видели только упрощенную версию одного из них. Необходимо также учитывать и соображения безопасности. Кроме того, мы даже не начинали рассматривать триггеры, функции, агрегаты или UDT. Несмотря на то, что пример показал простоту программирования под SQL CLR, теперь нам предстоит потратить некоторое время на изучение всех углов и закоулков.

## Хранимые процедуры CLR и доступ к данным на стороне сервера

В нашем предыдущем черновом примере мы рассмотрели разработку хранимой процедуры CLR, но теперь нам необходимо более тщательно изучить эту тему. Мы уже описали механизмы написания и развертывания хранимой процедуры, но теперь давайте вернемся несколько назад, чтобы понять, как хранимые процедуры CLR работают, с концептуальной точки зрения.

Код хранимой процедуры SQL CLR выполняется в экземпляре .NET CLR, хостом для которого является сам SQL Server; он не вызывается как внешний процесс (как расширенные хранимые процедуры (XP) на базе модели Component Object Model (COM)). Поскольку код SQL CLR выполняется в контексте сервера, он обращается с объектами базы данных как с собствен-

ными, локальными объектами. Поэтому он должен обращаться с вызывающим клиентом как с удаленным. Эта среда фактически противоположна той, в которой работает код ADO.NET клиентской стороны и промежуточного уровня. Там для обмена с базой данных требуется удаленное подключение (даже если база данных находится физически на том же самом компьютере), а код ADO.NET выполняется локально. Обратная конфигурация SQL CLR требует некоторого привыкания, но после того, как вы освоитесь с таким положением вещей, вам станет легко писать и понимать код SQL CLR.

В то же время, поскольку .NET не имеет внутреннего способа доступа к локальным объектам сервера или передачи данных и сообщений клиенту, вы должны использовать специальный набор классов для выполнения этих задач. Эти классы содержатся в пространстве имен `Microsoft.SqlServer.Server`.

### **ПРИМЕЧАНИЕ**

Интересно и важно отметить то, что пространство имен `Microsoft.SqlServer.Server` фактически обеспечивается сборкой `System.Data.dll` (из .NET Framework). Это означает, что вам не нужно беспокоиться о добавлении в ваш проект ссылки для использования этого пространства имен. Расположение пространства имен внутри `System.Data.dll` еще больше подчеркивает тесную интеграцию между .NET и SQL Server.

Если хотите, можете представлять себе `Microsoft.SqlServer.Server` как вспомогательную библиотеку для `System.Data.SqlClient`. Она обеспечивает атрибуты кода SQL CLR (о которых мы уже говорили), несколько перечислений, класс исключений, интерфейс и еще пять классов: `SqlContext`, `SqlPipe`, `SqlTriggerContext`, `SqlMetaData`, `SqlDataRecord`. Классы `SqlMetaData` и `SqlDataRecord` мы опишем в конце этого раздела, а класс `SqlTriggerContext` — при обсуждении триггеров CLR далее в этой же главе. Объекты `SqlContext` и `SqlPipe` мы опишем прямо сейчас.

Объект `SqlContext`, который имеет тип `static`, предоставляет дескриптор для контекста серверной стороны (где выполняется ваш код). Он имеет также канал к клиенту для возврата данных и текста — это его свойство `Pipe`, которое в свою очередь обеспечивает доступ к должным образом открытому и инициализированному объекту `SqlPipe`.

Объект `SqlPipe` может посылать данные и сообщения вызывающему клиенту при помощи нескольких методов: `Send`, `SendResultsStart`, `SendResultsRow`, `SendResultsEnd` и `ExecuteAndSend`. В предыдущем образце кода мы использовали метод `ExecuteAndSend` объекта `SqlPipe` для неявного открытия подключения, вызова `ExecuteReader` объекта `SqlCommand` (который использует

это подключение) и передачи содержимого результирующего `SqlDataReader` обратно клиенту. И хоть неявное выполнение методом `ExecuteAndSend` может показаться нам удобным, таких сокращенных методов при детальном изучении программирования под SQL CLR следует избегать.

В общем случае код хранимой процедуры SQL CLR (которая запрашивает таблицы базы данных) должен открыть подключение к этой базе данных, использовать метод `ExecuteReader` объекта `SqlCommand` для запроса данных, а затем применить один из нескольких методов `Send` (или их комбинацию) для отправки их обратно. Методы `Send` не принимают объектов `DataSet`; они принимают только объекты `SqlDataReader`, строки и специальные объекты `SqlDataRecord`. Листинг 3.1 (где показана реализация функции `spContacts` из файла `spTest.cs` примера проекта) является отличным примером того, как это делается.

### Листинг 3.1. Реализация `spContacts` из файла `spTest.cs`

```
[SqlProcedure()]
public static void spContacts()
{
    SqlConnection conn = new SqlConnection("context connection=true");
    SqlCommand cm = new SqlCommand("SELECT * FROM Person.Person", conn);
    conn.Open();
    SqlDataReader dr = cm.ExecuteReader();
    SqlContext.Pipe.Send("Starting data dump");
    SqlContext.Pipe.Send(dr);
    SqlContext.Pipe.Send("Data dump complete");
    dr.Close();
    conn.Close();
}
```

### ПРИМЕЧАНИЕ

Поскольку таблица `Person.Person` имеет столбцы типа `xml`, и поскольку такие столбцы могут вызвать замедление в хранимых процедурах SQL CLR, то запрос в примере кода `spContacts` использует в выражении `SELECT` список столбцов (из которого исключены столбцы `xml`), а не групповой символ `*`. В напечатанном коде для простоты и краткости мы сохранили синтаксис `SELECT *`.

Для того чтобы этот код работал, нам необходимо использовать пространства имен `Microsoft.SqlServer.Server` и `System.Data.SqlClient` (и если вы за-

глянете в пример проекта данной главы (а не в листинг 3.1), то вы увидите, что мы дали псевдонимы обоим этим пространствам имен при помощи инструкций `using`). Мы сделали это потому, что любой обычный объект ADO.NET (такой как `SqlConnection`, `SqlCommand` и `SqlDataReader`) предоставляется нам из `System.Data.SqlClient`. И как уже обсуждалось, для использования таких объектов, как `SqlContext` и `SqlPipe`, нам нужно пространство имен `Microsoft.SqlServer.Server`. Шаблон хранимой процедуры в проектах типа `SQL Server project` в `Visual Studio` автоматически вставляет инструкцию `using` для `Microsoft.SqlServer.Server` и `System.Data.SqlClient`.

Несмотря на то, что код серверной стороны использует объекты `SqlClient`, он делает это специальным способом. В конструктор объекта `SqlConnection` передается строка подключения `context connection=true`. Этим по существу дается указание ADO.NET открыть новое подключение к той базе данных, в которой находится сборка CLR. Обратите также внимание на второй вызов метода `Send` объекта `SqlContext.Pipe`. Здесь (для выталкивания содержимого `SqlDataReader` обратно клиенту) используется перегрузка параметра `SqlDataReader` метода `Send` объекта `SqlPipe`. Вы можете представлять себе этот метод как выполнение цикла `while (dr.Read())` по `SqlDataReader` и возвращение значений каждого столбца для каждой итерации цикла. Но вместо того, чтобы делать все это вам самим, это делает за вас метод `Send`.

Перед тем и после того, как `SqlDataReader` возвращается клиенту, мы используем перегрузку параметра `String` метода `Send` для отправки клиенту сообщений о состоянии. Когда эта хранимая процедура выполняется в `Management Studio`, то возвращаемый текст появляется на вкладке **Results** окна запросов (когда вы используете в `Management Studio` опцию **Results To Text**) или во вкладке **Messages** (когда вы используете опцию **Results To Grid**).

Остальная часть листинга содержит типичный код ADO.NET, где используются объекты из поставщика `SqlClient`. Все это прекрасно иллюстрирует общую идею программирования под SQL CLR: делайте то, что вы обычно делаете в клиенте или на промежуточном уровне, и используйте при этом несколько специальных вспомогательных объектов для работы в контексте SQL Server.

## Пересылка данных при помощи *SqlDataRecord* и *SqlMetaData*

Мы уже упоминали, что метод `Send` объекта `SqlPipe` может принимать объект типа `SqlDataRecord`, и `Microsoft.SqlServer.Server` предоставляет этот объ-

ект, а также и объект с названием `SqlMetaData`. Вы можете совместно использовать эти два объекта в хранимой процедуре CLR (для возвращения результирующего набора по одной строке вместо предоставления `SqlDataReader` методу `Send` объекта `SqlPipe`). Это позволяет вам (но не требует от вас) изучить данные перед отправкой их обратно клиенту. Отправка объектов `SqlDataReader` препятствует изучению данных внутри хранимой процедуры, поскольку объекты `SqlDataReader` являются просто пересылающими структурами результирующих наборов. Использование метода `ExecuteAndSend` и объекта `SqlCommand` подчиняется тем же самым ограничениям.

Объект `SqlDataRecord` позволяет .NET-коду создать отдельную строку, которая возвращается вызывающему клиенту. Ее конструктор принимает массив объектов `SqlMetaData`, которые в свою очередь описывают метаданные для каждого столбца в строке.

Листинг 3.2 (в котором показана реализация функции `spContactCount` из файла `spTest.cs` примера проекта) иллюстрирует то, как необходимо использовать `SqlPipe.Send` совместно с объектами `SqlDataRecord` и `SqlMetaData` для возвращения (из хранимой процедуры) результирующего набора с одним столбцом и одной строкой.

### Листинг 3.2. Функция `spContactCount` из файла `spTest.cs`

```
[SqlProcedure()]
public static void spContactCount()
{
    SqlConnection conn = new SqlConnection("context connection=true");
    SqlCommand cm = new SqlCommand("SELECT COUNT(*)
                                   FROM Person.Person", conn);

    SqlDataRecord drc = new SqlDataRecord(new SqlMetaData("ContactCount",
                                                           SqlDbType.Int));

    conn.Open();
    drc.SetInt32(0, (Int32)cm.ExecuteScalar());
    SqlContext.Pipe.Send(drc);
    conn.Close();
}
```

Этот код объявляет переменную `drc` как объект `SqlDataRecord` и передает ее конструктору один объект `SqlMetaData`. (Передача одного объекта вместо массива допустима в том случае, если объект `SqlDataRecord` будет иметь только один столбец). Объект `SqlMetaData` описывает столбец с названием `ContactCount` типа `SqlDbType.Int`.

**ПРИМЕЧАНИЕ**

Перечисление `SqlDbType` содержится в пространстве имен `System.Data.SqlTypes`. Шаблон SQL Server Stored Procedure вставляет инструкцию `using` для этого пространства имен. Если вы создаете код SQL CLR без использования этого шаблона, то вам придется добавить инструкцию `using` вручную.

Остальная часть кода очень проста. Сначала открывается подключение к контексту и команда контекста, затем выполняется запрос `SELECT COUNT(*)` по таблице `Person.Person`. Поскольку запрос возвращает единственное скалярное значение, он выполняется методом `ExecuteScalar` объекта `SqlCommand`. Затем возвращаемое `ExecuteScalar` значение приводится к `Int32` и загружается в столбец 0 (единственный возвращаемый столбец) объекта `SqlDataRecord` при помощи его метода `SetInt32`. После этого `SqlDataRecord` направляется клиенту при помощи метода `Send` объекта `SqlPipe`.

**ПРИМЕЧАНИЕ**

Если бы мы хотели отправить обратно несколько объектов `SqlDataRecord`, то мы послали бы первый объект при помощи метода `SendResultsStart` объекта `SqlContext`, а затем посылали бы все последующие объекты `SqlDataRecord` при помощи метода `SendResultsRow`. После отправки всех `SqlDataRecord` мы бы вызвали метод `SendResultsEnd`.

После развертывания хранимой процедуры (методы которого мы скоро обсудим) вы можете выполнить ее из SQL Server Management Studio точно так же, как и любую другую хранимую процедуру. Несмотря на то, что результат ее — единственное значение, оно представляется как столбец (и во вкладке **Results** окна запросов показано название столбца `ContactCount`). Помните, что этот результат запроса `COUNT(*)` можно было вернуть и без использования объектов `SqlMetaData` и `SqlDataRecord`; в данном примере показано использование этих объектов как альтернативы пересылке объектов `SqlDataReader` и текста клиенту.

**УКАЗАНИЯ ПО ИСПОЛЬЗОВАНИЮ ХРАНИМЫХ ПРОЦЕДУР CLR**

Важно понимать, как в хранимых процедурах CLR выполнять доступ к данным и их извлечение. Как .NET-разработчик, вы уже знаете, как выполнять в вашем коде вычислительные задачи, поэтому наши примеры кода в основном иллюстрируют доступ к данным на стороне сервера. Как код для иллюстрации концепций они совершенно достаточны.

В то же время вы должны избегать хранимых процедур CLR простых операций CRUD (аббревиатура для Create, Read, Update и Delete). Такие задачи лучше оставить для обычных хранимых процедур T-SQL, которые обычно выполняют такие операции более эффективно, чем это может сделать ADO.NET. Хранимые процедуры CLR работают хорошо тогда, когда вам нужно выполнить вычисления, и вам требуется вся выразительность .NET-языка или богатые функциональные возможности, предоставляемые библиотеками базовых классов .NET (в T-SQL такой выразительности и поддержки библиотек базовых классов нет).

Например, хорошим применением хранимых процедур SQL CLR является реализация поиска на основе нечеткой логики (при помощи бизнес-логики, встроенной в .NET-сборки, для определения того, какие данные имеют сходство с другими данными). Еще одним хорошим применением интеграции SQL CLR является проверка данных (на основе регулярных выражений) в хранимой процедуре обновления или вставки. Общее правило таково: обычный доступ к данным следует оставить для T-SQL. "Более ценные" вычисления являются хорошими кандидатами для интеграции SQL CLR. К вопросу об использовании SQL CLR мы еще будем возвращаться в разных местах этой главы.

## Развертывание

Перед тем как вы сможете протестировать свой код SQL CLR, необходимо развернуть содержащую его сборку и зарегистрировать те функции, которые должны распознаваться как хранимые процедуры. В вашем распоряжении имеются разные способы развертывания; мы опишем их прямо сейчас, перед обсуждением тестирования хранимых процедур и четырех остальных основных элементов SQL CLR.

### Развертывание вашей сборки

Как уже упоминалось ранее, Visual Studio развертывает пример проекта типа SQL Server при сборке, запуске или пошаговом прохождении по проекту; либо вы можете использовать опцию **Build | Deploy** в главном меню Visual Studio. Если вы работаете с примером проекта типа SQL Server, то выберите прямо сейчас опцию **Deploy** либо одну из опций (**Start** или **Build**) в Visual Studio.

Для развертывания версии проекта типа Class Library вы можете выполнить следующую инструкцию T-SQL из Management Studio (предполагая, что

C:\Projects\Chapter03 является родительским каталогом для примеров кодов данной главы):

```
CREATE ASSEMBLY Chapter03
  AUTHORIZATION dbo
  FROM 'C:\Projects\Chapter03\VS\Chapter03Manual\
        Chapter03\bin\Debug\Chapter03.dll'
  WITH PERMISSION_SET = SAFE
GO
```

Выражение `AUTHORIZATION` позволяет вам указать название или роль, которой присваивается владение сборкой. Авторизация по умолчанию — текущий пользователь, и поскольку вы, скорее всего, зарегистрированы как `dbo` для `AdventureWorks2008`, в данном случае это выражение не нужно (именно поэтому мы опустили его в нашем предыдущем примере).

Значение и результат использования выражения `WITH PERMISSION_SET` обсуждаются в конце этой главы. Пока же обратим внимание, что это выражение позволяет вам указать разрешения безопасности, с которыми выполняется ваша сборка. Подобно выражению `AUTHORIZATION`, в данном случае выражение `WITH PERMISSION_SET` технически не нужно, поскольку при выполнении команды `CREATE ASSEMBLY` для `PERMISSION_SET` используется значение по умолчанию `SAFE`. Тем не менее, хорошей практикой является включение обоих выражений.

Если сборка зависит от других сборок, то `SQL Server` смотрит, загружались ли уже эти сборки в базу данных и если загружались — подтверждает, что их владелец тот же самый, что и указанной сборки. Если зависимые сборки еще не загружались в базу данных, то `SQL Server` ищет их в том же самом каталоге, что и указанную сборку. Если он находит все зависимые сборки в этом местоположении, то он загружает их и назначает то же владение, что и первичной сборке. Если он не находит зависимых сборок в этом каталоге или в кэше глобальных сборок (`global assembly cache`, `GAC`), то команда `CREATE ASSEMBLY` завершается с ошибкой.

В выражении `FROM` вы можете указать строковое выражение вместо литерала. Например, вы можете получить ссылку на сборку из таблицы вашей базы данных. Можно также предоставить непосредственно встроенную сборку (указав в выражении `FROM` двоичный поток вместо спецификации файла). Для этого необходимо указать литеральное значение или выражение `varbinary` (либо разделенный запятыми список значений или выражений `varbinary`, когда необходимо указывать зависимые сборки), которое содержит фактическое двоичное содержимое вашей сборки (или сборки). Это позволяет полностью вывести в скрипт создание базы данных и в том числе всех содержащихся в ней сборок `CLR` (без необходимости распространения реальных



файлов сборки). Двоичный поток можно встроить в сам скрипт, либо он может быть выбран из таблицы базы данных (при помощи выражения).

### ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Дополнительную информацию об этой сложной методике развертывания сборок SQL CLR смотрите в SQL Server Books Online.

В дополнение к использованию развертывания Visual Studio и инструкции `CREATE ASSEMBLY` языка T-SQL вы можете выгрузить сборку в вашу базу данных и интерактивно из Management Studio. Просто щелкните правой кнопкой мыши по узлу **servername\Databases\AdventureWorks2008\Programmability\Assemblies** в окне Object Explorer (где `servername` — это имя вашего сервера), а затем выберите в контекстном меню пункт **New Assembly**. Появится диалоговое окно **New Assembly** (рис. 3.5).

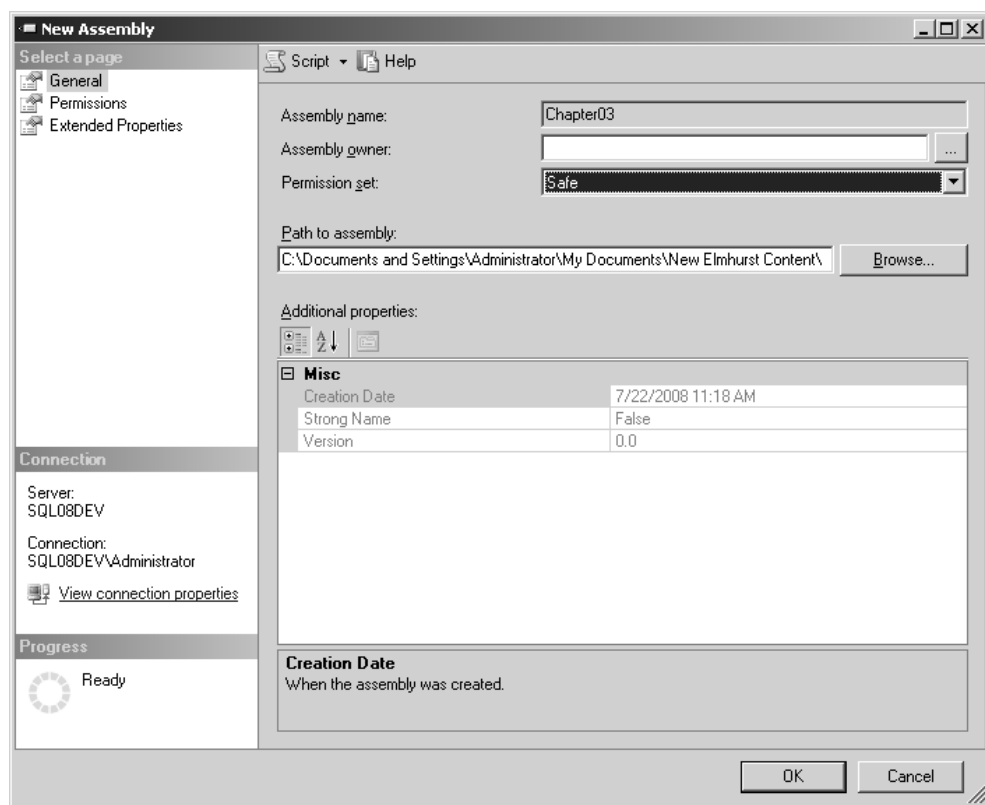


Рис. 3.5. Диалоговое окно **New Assembly** в Management Studio

В текстовом поле **Path To Assembly** введите путь к сборке и название ее файла или используйте кнопку **Browse** для того, чтобы указать его в интерактивном режиме. Вы можете указать подробности **AUTHORIZATION** и **WITH PERMISSION\_SET** в текстовом поле **Assembly Owner** (пользуясь при необходимости кнопкой с многоточием) и в комбинированном окне **Permission Set** соответственно.

Независимо от используемого вами способа развертывания после добавления сборки в вашу базу данных она становится составной частью этой базы данных и ее файла *mdf*. Это означает, что если ваша база данных копируется и восстанавливается (либо развертывается), то все сборки внутри нее двигаются вместе с данными и их не нужно добавлять после этого вручную.

## Развертывание ваших хранимых процедур

В версии примера кода SQL Server развертывание всех хранимых процедур выполняется средой Visual Studio при развертывании самой сборки. Так происходит вследствие применения атрибута *SqlProcedure* к методам класса *StoredProcedures* (находящегося в файле *spTest.cs*). Атрибут *SqlProcedure* принимает необязательный параметр *Name*, значение которого — реальное вызываемое название хранимой процедуры. Если вы не предоставите значения для параметра *Name*, то в качестве названия хранимой процедуры будет использовано название *.NET*-метода.

Атрибут *SqlProcedure* используется только средой Visual Studio в проектах типа SQL Server. Поэтому он был удален из исходного кода проекта типа *Class Library*. Развертывание хранимых процедур из этой версии исходного кода требует выдачи команды *CREATE PROCEDURE* языка T-SQL при помощи выражения *EXTERNAL NAME*, где указываются: сборка, полностью квалифицированное имя класса, а также название метода. Например, для загрузки версии *Class Library* необходимо выдать следующую команду:

```
CREATE PROCEDURE spContacts  
AS EXTERNAL NAME Chapter03.StoredProcedures.spContacts
```

Предыдущая команда указывает, что метод *spContacts* класса *StoredProcedures* в загруженной сборке с названием *Chapter03* должен быть зарегистрирован как хранимая процедура CLR, вызываемая по имени *spContacts*.

### ПРИМЕЧАНИЕ

Все необходимые команды *CREATE PROCEDURE* для версии типа *Class Library* примера кода содержатся в скрипте *CreateObjects.sql* проекта

Management Studio, предоставленного в примерах кодов данной главы на Web-сайте этой книги. Для использования различных реализованных в проекте Class Library элементов SQL CLR вам придется выполнить этот скрипт.

Обратите внимание, что если бы хранимая процедура была написана на Microsoft Visual Basic .NET (а не на C#), то имя класса изменилось бы на `Chapter03.StoredProcedures`. Это потребовало бы также изменения в коде развертывания:

```
CREATE PROCEDURE spContacts  
AS EXTERNAL NAME Chapter03.[Chapter03.StoredProcedures].spContacts
```

В проектах Visual Basic пространство имен по умолчанию для проекта совпадает с названием проекта (и то же самое для названия сборки). На класс внутри проекта необходимо ссылаться с использованием пространства по умолчанию в виде префикса. Поскольку спецификатор класса — это составное название, разделенное точками, то оно должно быть заключено в квадратные скобки, чтобы SQL Server мог распознать его как единое неделимое имя. Поскольку проекты C# обрабатывают пространство имен по умолчанию несколько иначе, то для сборок C# в спецификаторе класса префикс пространства имен не используется.

Еще одно последнее замечание перед тем, как перейти к обсуждению тестирования ваших развернутых хранимых процедур CLR. Важно понимать, что спецификатор класса и имя метода в выражении `EXTERNAL NAME` чувствительны к регистру, и это справедливо даже для сборок, разработанных на языке Visual Basic .NET. Это может сначала показаться странным, но SQL Server ищет ваши методы в сборках, а не в исходном коде. Другими словами, он ищет в промежуточном коде Microsoft Intermediate Language, а не в исходном коде Visual Basic .NET или C#. А язык MSIL является чувствительным к регистру (для обеспечения поддержки чувствительных к регистру языков, таких как C#).

То, что SQL Server и Visual Basic .NET не чувствительны к регистру, не имеет значения! Если вы попытаетесь зарегистрировать метод и получите ошибку о том, что он не найден внутри сборки, то перепроверьте регистр написания вашей команды (соответствует ли он вашему исходному коду).

## Тестирование хранимых процедур

После развертывания сборок и хранимых процедур вы готовы выполнять и тестировать их. Обычно это следует делать из Management Studio; однако

проекты типа SQL Server среды Visual Studio позволяют вам тестировать код SQL CLR из Visual Studio. В Visual Studio в каталоге исходных кодов создается подкаталог Test Scripts. В этом подкаталоге Visual Studio создает файл скрипта с названием Test.sql примеров кода T-SQL для тестирования: хранимых процедур CLR, определяемых пользователем функций, UDT и агрегатов. Он содержит также и некомментированную команду `SELECT`, которая выдает вызывающей стороне текстовый литерал, который поясняет, что текст скрипта необходимо отредактировать.

Visual Studio подключается к вашей базе данных и выполняет этот скрипт сразу же после того, как ваша сборка будет развернута, и вывод из этого скрипта появляется в окне **Output** среды Visual Studio. Это позволяет вам выполнять любое количество команд T-SQL непосредственно из Visual Studio (без необходимости переключаться на другой инструмент). Хотя такой подход гораздо менее интерактивен, чем окно запросов Management Studio, он позволяет вам выполнить быстрое тестирование вашего кода. Он особенно полезен для тестирования регрессий, то есть для подтверждения того, что новая версия вашей сборки не нарушает более старой и критичной функциональности.

Расширение имени файла скрипта должно быть `sql`. В каталоге Test Scripts у вас может быть много файлов скриптов. Для добавления нового скрипта щелкните правой кнопкой мыши по узлу каталога Test Scripts или по узлу проекта в окне **Solution Explorer**, а затем выберите в контекстном меню пункт **Add Test Script**. Только один скрипт может быть активным в каждый момент времени, и как только скриптов у вас становится больше чем один, вы должны указать активный. Для того чтобы сделать скрипт активным, просто щелкните правой кнопкой мыши по его узлу в окне **Solution Explorer**, а затем выберите в контекстном меню пункт **Set As Default Debug Script**. После этого узел будет отображаться жирным шрифтом. Вы можете выполнять или отлаживать даже тот скрипт, который не является активным. Для этого щелкните правой кнопкой мыши по его узлу в окне **Solution Explorer**, а затем выберите в контекстном меню пункт **Debug Script**.

Если вы работаете с версией Class Library, вы должны протестировать хранимые процедуры из Management Studio или другого инструмента запросов SQL Server. И даже если вы работаете с версией SQL Server, то обнаружите в Management Studio большую гибкость.

### **ПРИМЕЧАНИЕ**

Если вы используете версии Visual Studio Team Edition for Database Professionals или Visual Studio Team Suite, то вы можете добавить в ваше

решение проект Team System Database и создавать скрипты там (это будет работать даже для кода SQL CLR проекта типа Class Library).

Файл скрипта `TestStoredProcs.sql` в поставляемом вместе с примером кодов проекте Management Studio запустит обе наши хранимые процедуры CLR (`spContactCount` и `spContacts`). Откройте файл в Management Studio, а затем нажмите кнопку **Execute** на панели инструментов **SQL Editor**, выберите в меню **Query** пункт **Execute** (Вы можете также щелкнуть правой кнопкой мыши в окне запросов и выбрать в контекстном меню пункт **Execute**.)

Когда скрипт запустится, вы сначала увидите результат хранимой процедуры `spContactCount` (рис. 3.6). Обратите внимание, что название столбца `ContactCount` появляется на вкладке **Results** (вспомним, что это прямой результат использования вами в коде CLR объекта `SqlMetaData`). Ниже результата `spContactCount` вы увидите результаты хранимой процедуры `spContacts`. Поскольку таблица `Person.Person` имеет почти 20 000 строк, то на получение этих результатов может потребоваться некоторое время.

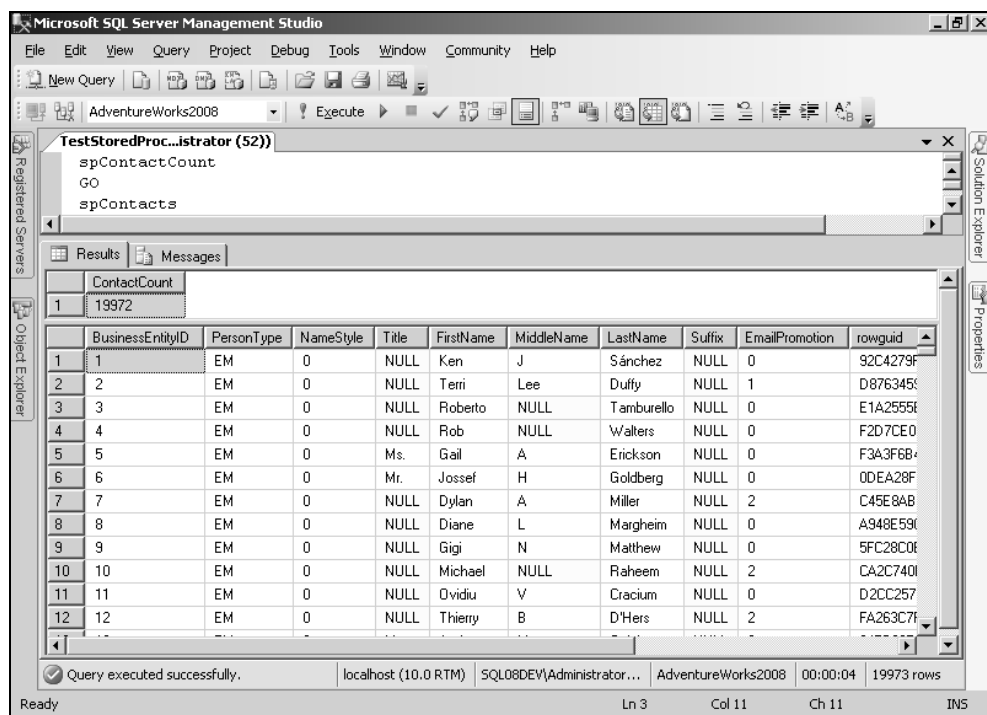


Рис. 3.6. Код скрипта `TestStoredProcs.sql` и его результаты

При поступлении результатов на вкладке **Messages** (или на вкладке **Results**, если вы используете опцию **Results To Text** в Management Studio) должно быть видно сообщение состояния: "Starting data dump". После получения всех строк вы должны будете увидеть сообщение "Data dump complete". Если вы хотите прервать запрос до получения всех строк, то можете использовать кнопку **Cancel Executing Query** на панели инструментов **SQL Editor** или опцию **Query | Cancel Executing Query** в главном меню или комбинацию клавиш <Alt>+<Break>.

Нам предстоит еще описать функции, триггеры, UDT и агрегаты CLR, однако вы уже изучили большую часть того, что необходимо вам для разработки кода SQL CLR. Вы изучили, как создавать в Visual Studio проекты типа SQL Server и использовать их автоматическое развертывание и тестовые скрипты. Вы научились разрабатывать код SQL CLR в стандартных проектах типа Class Library, а также использовать команды T-SQL и Management Studio для развертывания кода. Вы изучили тонкую разницу между развертыванием кода C# и кода Visual Basic .NET. Мы описали еще требования чувствительности к регистру при развертывании с помощью T-SQL.

Имея все это, мы можем достаточно быстро описать оставшиеся четыре основных элемента SQL CLR.

## Функции CLR

Возьмем все, что мы рассказали о хранимых процедурах SQL CLR и их развертывании, и применим это к функциям SQL CLR. Любой программист знает, что *функция* — это процедура, которая возвращает значение (или объект). Большинство .NET-функций возвращает .NET-типы. Функции же SQL CLR должны возвращать `SqlType`. Поэтому для начала нам нужно убедиться, что наши (реализующие функции SQL CLR) классы импортируют пространство имен `System.Data.SqlTypes` (при помощи инструкции `using`). Шаблон SQL Server по умолчанию имеет соответствующую инструкцию `using`, для кода типа Class Library вам придется добавить такую инструкцию вручную.

После импорта пространства имен вы можете писать сами функции. В проектах SQL Server в Visual Studio они должны быть помечены атрибутом `SqlFunction`, который принимает необязательный параметр названия аналогично `SqlProcedure`. Для имеющих скалярные значения функций проектов Class Library атрибут `SqlFunction` необязателен, поэтому он появляется в примере кода Class Library только для нашей возвращающей табличное значение функции (TVF).

Листинг 3.3 (который показывает код функции `fnHelloWorld` из файла `fnTest.cs`) реализует простую функцию "Hello World", которая возвращает значение типа `SqlString`.

**Листинг 3.3. Функция `fnHelloWorld` из файла `fnTest.cs`**

```
[SqlFunction()]
public static SqlString fnHelloWorld()
{
    return new SqlString("Hello World");
}
```

Обратите внимание, что объекты `SqlType` (такие как `SqlString`) требуют явного создания экземпляра и передачи значений конструктора, их нельзя просто объявить и присваивать им значения. Код листинга 3.3 создает экземпляр объекта `SqlString` внутри строки инструкции `return` (чтобы избежать объявления переменной).

Функция, которая не принимает параметров и возвращает жестко запрограммированное значение, имеет весьма ограниченное применение. Функции часто используются в инструкциях T-SQL, как бы расширениями встроенных в язык T-SQL функций. Листинг 3.4 (где показан код функции `fnToCelsius` из файла `fnTest.cs`) реализует функцию преобразования из градусов Фаренгейта в градусы Цельсия.

**Листинг 3.4. Функция `fnToCelsius` из файла `fnTest.cs`**

```
[SqlFunction()]
public static SqlDecimal fnToCelsius(SqlInt16 fahrenheit)
{
    return new SqlDecimal((((Int16)fahrenheit) - 32) / 1.8);
}
```

Функция принимает температуру в градусах Фаренгейта (как `SqlInt16`), преобразует ее в градусы Цельсия и возвращает ее (как `SqlDecimal`). Обратите внимание, что код приводит входной параметр из типа `SqlInt16` в .NET-тип `Int16`, применяет формулу преобразования градусов Фаренгейта в градусы Цельсия, а затем передает результат в конструктор нового объекта `SqlDecimal`.

Развертывание этих функций в версии SQL Server нашего примера кода происходит автоматически. Для версии Class Library используйте инструкцию

CREATE FUNCTION языка T-SQL аналогично тому, как мы использовали инструкцию CREATE PROCEDURE в предыдущем разделе, но добавьте спецификацию типа данных для возвращаемого значения. Например, для развертывания функции fnHelloWorld следует использовать такую инструкцию:

```
CREATE FUNCTION fnHelloWorld()  
    RETURNS nvarchar(4000) WITH EXECUTE AS CALLER  
    AS EXTERNAL NAME Chapter03.UserDefinedFunctions.fnHelloWorld
```

Обратите внимание на использование типа данных nvarchar(4000) для соответствия типу sqlString. Выражением WITH EXECUTE AS CALLER указано, что функция SQL CLR должна выполняться под идентификацией вызывающей стороны.

### СОВЕТ

Вы можете сами набрать эту команду CREATE FUNCTION, однако учтите, что все необходимые команды для функций из примеров кода SQL CLR имеются в файле скрипта CreateObjects.sql проекта Management Studio на Web-сайте этой книги.

Вы можете протестировать эти функции при помощи тестового скрипта проекта SQL Server или в Management Studio. Используйте следующий запрос в вашем тестовом скрипте или окне запросов Management Studio для проверки этих двух функций. (Вы можете также запустить файл скрипта TestScalarFunctions.sql в Management Studio).

```
SELECT  
    dbo.fnHelloWorld() AS HelloWorld,  
    dbo.fnToCelsius(212) AS CelsiusTemp
```

Функции T-SQL могут возвращать как результирующие наборы, так и скалярные значения. Возвращающие результирующие наборы функции называются возвращающими табличные значения функциями (table-valued functions, TVF). Написать SQL CLR TVF вполне возможно, хотя делается это не так, как пишутся скалярные функции SQL CLR или хранимые процедуры SQL CLR. SQL CLR TVF должны возвращать тип, который реализует .NET-интерфейс IEnumerable, а также предоставлять метод "заполнения строки", который интерпретирует этот тип и преобразует экземпляр этого типа в строку таблицы.

Листинг 3.5 (который показывает код функций fnPortfolioTable и FillTickerRow из файла fnTest.cs образцового проекта) реализует TVF с названием fnPortfolioTable.



**Листинг 3.5. fnPortfolioTable и FillTickerRow из файла fnTest.cs**

```
[SqlFunction(
    FillRowMethodName="FillTickerRow",
    TableDefinition="TickerSymbol nvarchar(5), Value decimal")]
public static System.Collections.IEnumerable
    fnPortfolioTable(SqlString tickersPacked)
{
    string[] tickerSymbols;
    object[] rowArray = new object[2];
    object[] compoundArray = new object[3];
    char[] parms = new char[1];

    parms[0] = ';';
    tickerSymbols = tickersPacked.Value.Split(parms);
    rowArray[0] = tickerSymbols[0];
    rowArray[1] = 1;
    compoundArray[0] = rowArray;
    rowArray = new object[2];
    rowArray[0] = tickerSymbols[1];
    rowArray[1] = 2;
    compoundArray[1] = rowArray;
    rowArray = new object[2];
    rowArray[0] = tickerSymbols[2];
    rowArray[1] = 3;
    compoundArray[2] = rowArray;
    return compoundArray;
}

public static void FillTickerRow(object row, ref SqlString
    tickerSymbol, ref SqlDecimal value)
{
    object[] rowArray = (object[])row;
    tickerSymbol = new SqlString((string)rowArray[0]);
    value = new SqlDecimal(decimal.Parse(rowArray[1].ToString()));
}
```

Вместо реализации своего собственного совместимого с `IEnumerable` типа `fnPortfolioTable` использует массив. Это допустимо, поскольку массивы в .NET реализуют принцип `IEnumerable`. Функция `fnPortfolioTable` принима-

ет разделенный точкой с запятой список символов биржевых сводок и возвращает таблицу, где каждый символ появляется в отдельной строке в виде столбца `TickerSymbol` и значения этого символа — столбца `Value`. Структура возвращаемой таблицы объявляется в параметре `TableDefinition` атрибута `SqlFunction` проектов типа `SQL Server` и в команде `CREATE FUNCTION` языка `T-SQL` проектов типа `Class Library`. Присвоенные значения жестко запрограммированы, и возвращается только три строки (независимо от того, сколько биржевых символов передано). Наш пример является скорее инструментом для обучения, чем практическим применением функции `TVF`.

Здесь главное — это массивы. Сначала метод `Split` используется для разбиения списка с разделителями на массив отдельных строк биржевых символов. Затем `TVF` структурирует данные таким образом, чтобы каждый элемент в возвращаемом массиве значений `compoundArray` был двухэлементным массивом, хранящим биржевой символ и его значение. Теперь коду функции нужно вернуть только `compoundArray`. Затем функция `FillTickerRow` (названная в параметре `FillRowMethodName` атрибута `SqlFunction`) принимает двухэлементный массив (передаваемый как первый параметр) и преобразует его элементы в отдельные скалярные значения. Эти скалярные значения затем возвращаются через параметры `ref`, которые начинаются со второй позиции параметров. Названия параметров несущественны, но их местоположение должно соответствовать местоположению столбцов аргумента `TableDefinition` атрибута `SqlFunction`.

Поскольку `SQL Server` требует параметра `FillRowMethodName` атрибута `SqlFunction`, то мы пометили версию `Class Library` функции `fnPortfolioTable` этим атрибутом и предоставили значение параметра. В версии `SQL Server` мы также предоставляем значение для параметра `TableDefinition` (чтобы активировать автоматическое развертывание `TVF`).

Развертывание этой функции (как и других) выполняется `Visual Studio` в примере кода версии `SQL Server`. Для версии `Class Library` вы можете развернуть функцию при помощи следующих команд `T-SQL` (которая также имеется в файле скрипта `CreateObjects.sql`):

```
CREATE FUNCTION fnPortfolioTable(@TickersPacked nvarchar(4000))
RETURNS table (
    TickerSymbol nvarchar(5),
    Value decimal
)
WITH EXECUTE AS CALLER
AS EXTERNAL NAME Chapter03.UserDefinedFunctions.fnPortfolioTable
```

Как и в случае с `fnHelloWorld`, мы поставили `nvarchar(4000)` в соответствие типу данных `SqlString` (на этот раз для одного из входных параметров). Поскольку `fnPortfolioTable` является TVF, то возвращаемый ею тип объявляется как `table` (со вставленным внутрь строки определением таблицы).

Для тестирования TVF используйте следующий запрос в тестовом скрипте Visual Studio или окне запросов Management Studio (можете запустить файл скрипта `TestTableValuedFunction.sql` в проекте Management Studio):

```
SELECT * FROM fnPortfolioTable('IBM;MSFT;SUNW')
ORDER BY TickerSymbol
```

Должны быть возвращены следующие данные:

```
TickerSymbol Value
-----
IBM           1
MSFT          2
SUNW          3
```

SQL Server 2008 дает "подсказку" базе данных о том, что TVF возвращает данные в определенном порядке, оптимизируя запросы или создание индексов, упорядоченных по одному и тому же выражению. Следует, во-первых, убедиться в том, что логика кода CLR TVF возвращает данные в определенном порядке, и, во-вторых, что вы указали этот порядок в выражении `ORDER` команды `CREATE FUNCTION` языка T-SQL. Например, представьте себе, что наш код SQL CLR TVF упорядочил свои результаты по столбцу `TickerSymbol`. В этом случае мы можем модифицировать код T-SQL (который создает эту функцию) следующим образом:

```
CREATE FUNCTION fnPortfolioTable(@TickersPacked nvarchar(4000))
RETURNS table (
    TickerSymbol nvarchar(5),
    Value decimal
)
WITH EXECUTE AS CALLER
ORDER (TickerSymbol)
AS EXTERNAL NAME Chapter03.UserDefinedFunctions.fnPortfolioTable
```

Несмотря на то, что наш код фактически не возвращает данные в порядке `TickerSymbol`, вы по-прежнему можете использовать предыдущую команду T-SQL для версии Class Library TVF. Если вы повторно выполните запрос в файле скрипта `TestTableValuedFunction.sql`, то все будет работать правильно, поскольку входные данные случайно были заданы в порядке `TickerSymbol` следующим образом: `IBM; MSFT; SUNW`. Если же вы измените входную стро-

ку, например, на IBM; SUNW; MSFT, то получите ошибку, поскольку порядок данных в потоке не соответствует подсказке ORDER, указанной для CLR TVF `fnPortfolioTable`. Требуется обновить подсказку ORDER или изменить CLR TVF в соответствии с указанным в ORDER порядком.

Мы приводим текст этой ошибки, поскольку она является единственным доказательством (кроме анализа планов выполнения запросов) того, что эта новая функциональная возможность упорядочивания SQL CLR TVF действительно поддерживается. Обратите внимание, что использовать эту функцию можно на практике только тогда, когда вы регистрируете SQL CLR TVF при помощи кода T-SQL. Это потому, что атрибут `SqlFunction` не имеет параметра для указания выражения ORDER, так что автоматическое развертывание Visual Studio не может выдать такое выражение. Итак, вы должны использовать выражение ORDER только с проектом типа Class Library либо опустить атрибут `SqlFunction` в вашем коде SQL CLR TVF в проекте типа SQL Server и регистрировать TVF вручную (из внешнего скрипта или при помощи специальных скриптов перед развертыванием и после него). Эти специальные скрипты обсуждаются в следующем разделе.

## Триггеры CLR

Триггеры CLR — это просто хранимые процедуры, которые вызываются сервером SQL Server в определенные моменты времени и могут запрашивать значения из псевдотаблиц DELETED и INSERTED (где содержатся моментальные снимки данных "до и после" изменений, внесенных той инструкцией, которая запустила триггер). Триггеры SQL CLR похожи на хранимые процедуры SQL CLR, и они могут создаваться для всех действий языка манипулирования данными (обновлений, вставок и удалений).

SQL Server 2005 ввел концепцию триггеров языка определения данных (DDL), которые могут перехватывать и обрабатывать такие действия, как CREATE TABLE и ALTER PROCEDURE (триггеры DDL описываются в главе 2). Подобно триггерам DML, триггеры DDL могут быть реализованы в коде T-SQL или SQL CLR. Мы опишем в данном разделе и триггеры SQL CLR DML, и триггеры DDL.

Триггеры SQL CLR DML (подобно их аналогам в T-SQL) имеют доступ к псевдотаблицам DELETED и INSERTED и должны объявляться как обрабатывающие одно или несколько конкретных событий для конкретной таблицы (или определенного представления — при некоторых обстоятельствах). Они могут также использовать объект `SqlTriggerContext` (свойство `TriggerContext` объ-

екта `SqlContext`) для определения того, какое конкретное событие (обновление, вставка или удаление) вызвали их срабатывание, и какие столбцы были обновлены.

После того как вы поняли эти концепции, написать триггер SQL CLR DML становится действительно очень просто. Листинг 3.6 функции `trgUpdatePerson` из файла `trgTest.cs` примера проекта показывает код SQL CLR в качестве триггера FOR UPDATE таблицы `Person.Person` базы данных `AdventureWorks2008`.

### Листинг 3.6. `trgUpdatePerson` из файла `trgTest.cs`

```
// [SqlTrigger(Target="Person.Person", Event="for UPDATE")]
public static void trgUpdatePerson()
{
    SqlTriggerContext context = SqlContext.TriggerContext;
    string oldName = string.Empty;
    string newName = string.Empty;
    SqlConnection conn = new SqlConnection("context connection=true");
    SqlCommand cmOld = new SqlCommand(
        "SELECT FirstName FROM DELETED", conn);
    SqlCommand cmNew = new SqlCommand(
        "SELECT FirstName FROM INSERTED", conn);
    conn.Open();
    SqlDataReader drOld = cmOld.ExecuteReader();
    if (drOld.Read())
    {
        oldName = (string)drOld[0];
    }
    drOld.Close();
    SqlDataReader drNew = cmNew.ExecuteReader();
    if (drNew.Read())
    {
        newName = (string)drNew[0];
    }
    drNew.Close();
    conn.Close();
    SqlContext.Pipe.Send("Old Value of FirstName:" + oldName);
    SqlContext.Pipe.Send("New Value of FirstName:" + newName);
    for (int i = 0; i <= context.ColumnCount - 1; i++)
```

```
{  
    SqlContext.Pipe.Send("Column " + i.ToString() + ": " +  
        context.IsUpdatedColumn(i).ToString());  
}  
}
```

Этот триггер CLR DML запрашивает псевдотаблицы DELETED и INSERTED и возвращает значения столбца FirstName "до и после" обновления строки. Это делается посредством выборки значений из псевдотаблиц и возвращением значений в виде текста при помощи метода Send объекта SqlPipe. Код триггера использует также метод TriggerContext.IsUpdatedColumn для возвращения списка всех столбцов таблицы Person.Person и информации о том, были ли они обновлены.

Для автоматического развертывания триггера вам обычно нужно сконфигурировать атрибут SqlTrigger и применить его к .NET-функции, которая реализует этот триггер. Поскольку триггеры DML применяются к объекту назначения (таблице или представлению) и событию (например, для UPDATE или вместо INSERT), то атрибут SqlTrigger имеет параметры каждого из этих элементов информации, и вы должны предоставить значения обоих. Атрибут SqlTrigger развертывает только одну копию триггера, но вы можете использовать T-SQL для развертывания того же самого кода как отдельного триггера другого события и таблицы. Для каждого отдельного развертывания одного и того же кода должно быть уникальное имя триггера.

К несчастью, имеющаяся в Visual Studio ошибка препятствует использованию атрибута SqlTrigger для объектов назначения, не входящих в схему dbo. (Например, наша таблица Person.Person находится в схеме Person, а не в схеме dbo). Это потому, что значение параметра Target окружается квадратными скобками тогда, когда Visual Studio генерирует его код T-SQL (генерируя например [Person.Person], что приведет к ошибке). Именно по этой причине в листинге 3.6 код атрибута SqlTrigger закомментирован. Обходным путем решения этой проблемы является использование скриптов перед развертыванием и после него, которое мы вскоре и обсудим.

### **ВАЖНО**

Обойти ошибку в схеме Visual Studio (по значению Target из Person]. [Person вместо Person.Person) тоже нельзя. Вы можете начать трассировку в SQL Server Profiler для того, чтобы понаблюдать за ошибочным кодом T-SQL, сгенерированным Visual Studio в обоих сценариях.

Вы можете создать единый фрагмент кода, который работает как триггер обновления и триггер вставки для данной таблицы. Затем вы сможете использовать свойство `TriggerAction` объекта `TriggerContext` для точного определения, какое именно событие вызвало срабатывание триггера, и выполнить несколько отличающийся код. Если вы захотите развернуть такой триггер CLR при помощи атрибута `SqlTrigger`, то вы должны будете установить его параметр `Event` в значение `FOR UPDATE, INSERT`.

Вот команда T-SQL для регистрации .NET-функции как триггера SQL CLR (только для события обновления):

```
CREATE TRIGGER trgUpdatePerson
ON Person.Person
FOR UPDATE
AS EXTERNAL NAME Chapter03.Triggers.trgUpdatePerson
```

### ПРИМЕЧАНИЕ

Все необходимые команды `CREATE TRIGGER` для версии Class Library примера кода содержатся в скрипте `CreateObjects.sql` проекта Management Studio данной главы на Web-сайте книги.

Помимо использования этого кода T-SQL в Management Studio, есть также способ выполнить эту команду T-SQL из Visual Studio и таким образом обойти ошибку `SqlTrigger` со схемой `dbo`. Есть почти недокументированная функциональная возможность проектов типа SQL Server: они позволяют вам создать два специальных скрипта T-SQL, которые будут выполняться до и после развертывания вашей сборки. Для использования этой возможности просто создайте в корневом каталоге вашего проекта (а не в каталоге Test Scripts) два скрипта с названиями `PreDeployScript.sql` и `PostDeployScript.sql`. Хотя названия файлов и не имеют чувствительности к регистру, написать их нужно правильно.

### СОВЕТ

Вы можете создать скрипты `PreDeployScript.sql` и `PostDeployScript.sql` вне Visual Studio, а затем добавить их в ваш проект при помощи функции **Add Existing Item** в Visual Studio. Вы можете также добавить их и напрямую (при помощи щелчка правой кнопкой мыши по узлу проекта или узлу каталога Test Scripts в **Solution Explorer**, выбора в контекстном меню пункта **Add Test Script**, переименования новых скриптов и последующего перетаскивания их из каталога Test Scripts в корневой каталог вашего проекта).

Для использования этой функциональной возможности с целью обхода ошибки `SqlTrigger` (связанной со схемой `dbo`) вставьте предыдущий код `CREATE TRIGGER` в ваш файл `PostDeployScript.sql`, а затем вставьте в файл `PreDeployScript.sql` следующий код T-SQL:

```
IF EXISTS (SELECT * FROM sys.triggers WHERE object_id =  
          OBJECT_ID(N' [Person] . [trgUpdatePerson] '))  
DROP TRIGGER Person.trgUpdatePerson
```

Независимо от способа развертывания, вы можете использовать для тестирования триггера следующий запрос. Этот код T-SQL можно найти в файле скрипта `TestTriggers.sql` в проекте `Management Studio`:

```
UPDATE Person.Person  
SET FirstName = 'Gustavo'  
WHERE BusinessEntityId = 1
```

Если вы поместите свойство `TriggerAction` объекта `TriggerContext` в инструкцию сравнения, то технология `Microsoft IntelliSense` покажет вам, что имеется множество перечислимых констант, которым может равняться данное свойство, и что большинство этих констант соответствует триггерам DDL. Это четко демонстрирует, что код SQL CLR может быть использован как для триггеров DDL, так и для триггеров DML.

В случае триггеров DDL может потребоваться большое количество информации о среде для того, чтобы точно определить, какое событие вызвало срабатывание триггера, системный процесс с каким идентификатором ID (SPID) его вызвал, в какое время произошло событие, а также прочей информации, характерной для данного типа события (например, какая команда T-SQL). Для извлечения этой информации можно запросить свойство `EventData` объекта `SqlTriggerContext`. Свойство `EventData` имеет тип `SqlXml`, поэтому оно в свою очередь имеет метод `CreateReader` и свойство `Value`, которые вы можете использовать для извлечения (форматированных в виде XML) данных о событии.

Код листинга 3.7 (взятый из функции `trgCreateTable` файла `trgTest.cs` примера проекта) показывает код SQL CLR для триггера DDL с названием `trgCreateTable`, который зарегистрирован для срабатывания при выполнении любой инструкции `CREATE TABLE` в базе данных `AdventureWorks2008`.

### Листинг 3.7. `trgCreateTable` из файла `trgTest.cs`

```
[SqlTrigger(Target = "DATABASE", Event = "FOR CREATE_TABLE")]  
public static void trgCreateTable()
```



```

{
    SqlTriggerContext context = SqlContext.TriggerContext;
    if (!(context.EventData == null))
    {
        SqlContext.Pipe.Send("Event Data: " +
                               context.EventData.Value.ToString());
    }
}

```

Код запрашивает свойство `Value` объекта `SqlContext.TriggerContext.EventData`, преобразует его в строку, а затем направляет эту строку обратно клиенту. Обратите внимание, что атрибут `SqlTrigger` в данном случае не закомментирован, поскольку в значении параметра `Target` не используется префикс схемы. То есть вы можете использовать развертывание на основе атрибута в проекте типа SQL Server или использовать следующую команду для версии Class Library:

```

CREATE TRIGGER trgCreateTable
ON DATABASE
FOR CREATE_TABLE
AS EXTERNAL NAME Chapter03.Triggers.trgCreateTable

```

Используйте для тестирования триггера DDL следующую DDL-команду языка T-SQL в тестовом скрипте Visual Studio или окне запросов Management Studio. (Вы можете найти этот код в файле скрипта `TestTriggers.sql` примера проекта Management Studio).

```

CREATE TABLE Test (low int, high int)
DROP TABLE Test

```

Ваш результат должен выглядеть примерно так:

```

<EVENT_INSTANCE>
  <EventType>CREATE_TABLE</EventType>
  <PostTime>2008-07-21T11:12:35.720</PostTime>
  <SPID>58</SPID>
  <ServerName>SQL08DEV</ServerName>
  <LoginName>SQL08DEV\AndrewB</LoginName>
  <UserName>dbo</UserName>
  <DatabaseName>AdventureWorks2008</DatabaseName>
  <SchemaName>dbo</SchemaName>
  <ObjectName>Test</ObjectName>
  <ObjectType>TABLE</ObjectType>

```

```
<TSQLCommand>
  <SetOptions ANSI_NULLS="ON" ANSI_NULL_DEFAULT="ON" ANSI_PADDING="ON"
    QUOTED_IDENTIFIER="ON" ENCRYPTED="FALSE" />
  <CommandText>CREATE TABLE Test (low int, high int)</CommandText>
</TSQLCommand>
</EVENT_INSTANCE>
```

### **ПРИМЕЧАНИЕ**

Фактический вывод будет состоять из сплошного неформатированного текста. Для облегчения чтения XML-кода `EventData` здесь были добавлены переводы строк и отступы.

## **Агрегаты CLR**

T-SQL имеет некоторое количество встроенных агрегатов, таких как `SUM`, `AVG` и `MAX`, но этого набора встроенных функций не всегда хватает. К счастью, функциональные возможности SQL CLR в SQL Server 2008 позволяют нам реализовать определяемые пользователем агрегаты в .NET-коде и использовать их из T-SQL. Определяемые пользователем агрегаты могут быть реализованы только в коде SQL CLR, эквивалента в T-SQL они не имеют. Поскольку агрегаты обычно выполняют только вычисления, то они являются отличным вариантом использования для кода SQL CLR. Оказывается также, что их очень легко создавать.

Во-первых, агрегаты похожи на функции (поскольку они принимают и возвращают значения). Фактически, если вы используете агрегат в произвольном запросе данных вызове T-SQL (например, `SELECT SUM(8)`), вы обращаетесь с агрегатом как с функцией. Необходимо помнить, что передаваемый агрегату аргумент обычно является столбцом, поэтому каждое дискретное значение этого столбца (для любой примененной области действия — `WHERE`, `HAVING`, `ORDER BY` и/или `GROUP BY`) передается в агрегат. Задача агрегата — обновлять переменную (которая в конечном итоге станет возвращаемым значением) при передаче в него каждого дискретного значения.

Для агрегатов CLR требуется применение к ним атрибута `SqlUserDefinedAggregate`. Этот атрибут принимает несколько параметров, но все они необязательны (кроме `Format`). В нашем примере мы будем использовать для параметра `Format` значение `Format.Native`. Для более сложных сценариев вам, возможно, понадобится изучить руководство SQL Server

Books Online. Значение `Format.Native` подходит для параметра `Format` во многих случаях.

### ПРИМЕЧАНИЕ

Теперь в SQL Server 2008 поддерживаются определяемые пользователем агрегаты размером более 8000 байтов. Однако они требуют использования для параметра `Format` значения `Format.UserDefined` (а не значения `Format.Native`).

В отличие от атрибутов `SqlProcedure`, `SqlFunction` и `SqlTrigger`, атрибут `SqlUserDefinedAggregate` требуется серверу SQL Server для того, чтобы ваш класс можно было использовать как агрегат. Проекты типа SQL Server используют этот атрибут для развертывания, он включается в шаблон агрегата, но должен использоваться также и в коде проекта типа Class Library (для того, чтобы регистрация агрегата прошла успешно).

Классы агрегатов должны иметь четыре метода: `Init`, `Accumulate`, `Merge`, `Terminate`. Метод `Init` используется для запуска нового вычисления агрегата. Метод `Accumulate` принимает SQL-тип для обработки. Метод `Terminate` возвращает SQL-тип, представляющий результат. И последний метод `Merge` принимает объект, типизированный как класс агрегата (чтобы его можно было объединить с выполняющимся экземпляром).

Метод `Accumulate` занимается объединением дискретного значения со значением агрегата, а метод `Terminate` возвращает окончательное агрегированное значение (после обработки всех дискретных значений). Метод `Init` обеспечивает код запуска, в нем обычно происходит инициализация приватной переменной уровня класса, которая будет использоваться методом `Accumulate`. Метод `Merge` вызывается в многопоточном сценарии, который мы опишем далее в этом же разделе.

Для полной ясности: класс вашего агрегата не будет реализовывать интерфейс для обеспечения этих методов; вы должны создать их таким образом, чтобы они удовлетворяли тому, что мы можем назвать ожидаемыми от классов агрегатов SQL CLR "соглашениями" (в противовес "контракту", которому они должны соответствовать). Когда вы разрабатываете код проекта типа SQL Server, то шаблон `Aggregate` содержит заглушки для этих четырех методов, а также правильно применяет атрибут `SqlUserDefinedAggregate`.

Создавать свои агрегаты очень просто, но продумывание логики агрегации сначала может несколько все запутать. Представьте себе, что вы хотите создать специальный агрегат с названием `BakersDozen`, который увеличивает свое накопленное значение на единицу для каждых 12 единиц дискретного

значения (как пекарь, который добавляет бесплатный тринадцатый пончик тогда, когда вы заказали 12). Используя все, что вы к настоящему моменту изучили об агрегатах, и сочетая с целочисленным делением, вы сможете легко реализовать агрегат `BakersDozen`. Листинг 3.8 (код из структуры `BakersDozen` в файле `aggTest.cs` образцового проекта) содержит всю реализацию агрегата `BakersDozen`.

### Листинг 3.8. Структура `BakersDozen` из файла `aggTest.cs`

```
[Serializable]
[Microsoft.SqlServer.Server.SqlUserDefinedAggregate (Format.Native)]
public struct BakersDozen
{
    private SqlInt32 DonutCount;
    public void Init()
    {
        DonutCount = 0;
    }
    public void Accumulate(SqlInt32 Value)
    {
        DonutCount += Value + ((Int32)Value) / 12;
    }
    public void Merge(BakersDozen Group)
    {
        DonutCount += Group.DonutCount;
    }
    public SqlInt32 Terminate()
    {
        return DonutCount;
    }
}
```

Этот код очень прост. Переменная `DonutCount` используется для отслеживания откорректированного структурой `BakersDozen` количества заказанных позиций, при этом увеличивается текущий итог (целочисленным делением заказанного количества на 12). При такой логике бонусные позиции добавляются только тогда, когда отдельное значение равно (или превышает) 12 или кратное ему число. Двенадцать содержит полную дюжину, 13 — тоже. Двадцать четыре содержит две дюжины, 27 — тоже. Два отдельных заказа по 6 позиций каждый не дадут никаких бонусных позиций, поскольку

ку для получения бонуса в дискретном значении должно содержаться минимум 12 позиций.

Для развертывания агрегата используйте развертывание на базе атрибутов для проектов типа SQL Server или следующую команду для версии типа Class Library:

```
CREATE AGGREGATE BakersDozen (@input int)
    RETURNS int
    EXTERNAL NAME Chapter03.BakersDozen
```

Обратите внимание, что название метода не указано, поскольку агрегат реализован отдельным классом, а не функцией. Обратите также внимание и на то, что тип данных возвращаемого значения должен быть объявлен как тип данных значений, которые будет обрабатывать эта агрегатная функция. Параметр @input работает как указатель места заполнения, поэтому его название несущественно. Обратите внимание, что агрегаты могут строиться как на типах SQL CLR (описаны в следующем разделе), так и на скалярных типах SQL.

### **ПРИМЕЧАНИЕ**

Предыдущая инструкция CREATE AGGREGATE для версии Class Library примера кода содержится в файле скрипта CreateObjects.sql в проекте Management Studio, на Web-сайте этой книги.

Для того чтобы увидеть работу агрегата, сначала запустите файл скрипта CreateTblAggregateTest.sql в примере проекта Management Studio (для создания таблицы с названием AggregateTest со столбцами OrderItemId, OrderId и ItemsOrdered и несколькими строками данных):

```
CREATE TABLE tblAggregateTest (
    [OrderItemId] [int] IDENTITY(1,1) NOT NULL,
    [OrderId] [int] NULL,
    [ItemsOrdered] [int] NOT NULL
)
GO

INSERT INTO tblAggregateTest VALUES (1,2)
INSERT INTO tblAggregateTest VALUES (1,4)
INSERT INTO tblAggregateTest VALUES (2,1)
INSERT INTO tblAggregateTest VALUES (2,12)
INSERT INTO tblAggregateTest VALUES (3,3)
INSERT INTO tblAggregateTest VALUES (3,2)
```

После того как эта таблица построена, используйте для тестирования функции агрегирования (в тестовом скрипте Visual Studio или окне запросов Management Studio) следующую инструкцию T-SQL DDL:

```
SELECT
    OrderId,
    SUM(ItemsOrdered) AS SUM,
    dbo.BakersDozen(ItemsOrdered) AS BakersDozen
FROM tblAggregateTest
GROUP BY OrderId
```

Для каждого отдельного значения в столбце `OrderId` этот запрос использует наш код CLR по следующему алгоритму:

1. Вызвать `Init()`.
2. Вызвать `Accumulate` по одному разу для всех строк с одним и тем же значением `OrderId`, передавая ей значение столбца `ItemsOrdered` этой строки.
3. Вызвать `Terminate` для получения агрегированного значения, которое будет возвращено запросом клиенту.

Результаты будут такими:

| OrderId | SUM | BakersDozen |
|---------|-----|-------------|
| 1       | 6   | 6           |
| 2       | 13  | 14          |
| 3       | 5   | 5           |

Включив в наш запрос встроенный агрегат `sum` языка T-SQL, мы видим, сколько бонусных позиций было добавлено. В данном случае (для `OrderId 2`) была добавлена одна бонусная позиция благодаря строке таблицы со следующими значениями:

| OrderItemId | OrderId | ItemsOrdered |
|-------------|---------|--------------|
| 4           | 2       | 12           |

Все остальные строки содержат такие значения `ItemsOrdered`, которые меньше 12, поэтому по ним бонусные позиции добавлены не были.

Поскольку SQL Server иногда разделяет работу по обработке запроса на несколько потоков, то обработчику запросов понадобится для одного запроса выполнить функцию агрегирования много раз, а затем объединить результаты. Для того чтобы при таком сценарии агрегат работал правильно, необходимо реализовать метод `Merge`.

Метод `Merge` берет результат агрегирования одного потока и объединяет его с агрегатом текущего потока. В нашем случае мы просто добавили значение `DonutCount` из агрегата вторичного потока (доступного через входной параметр `Group`) в наш поток. Добавлять бонусные позиции нет необходимости, поскольку они будут добавлены в вызовах `Accumulate` вторичного потока. Простое сложение — это все, что требуется. Агрегат, который вычисляет, например, некое среднее значение, потребовал бы более сложного кода `Merge`.

Не забывайте, что агрегатам можно передавать скалярные значения, их можно использовать из кода T-SQL без ссылки на набор строк. Ваш агрегат должен учитывать и такой сценарий, даже если это кажется нецелесообразным. В случае `BakerDozen` обработать одиночные скалярные значения легко. Для того чтобы убедиться в этом, попробуйте выполнить следующий "бес табличный" запрос T-SQL:

```
SELECT dbo.BakersDozen(13)
```

Вы увидите, что он вернет значение 14.

### **ПРИМЕЧАНИЕ**

Файл скрипта `TestAggregate.sql` в проекте `Management Studio` содержит оба запроса для тестирования агрегатов.

### **ПРИМЕЧАНИЕ**

Новая функциональная возможность `SQL Server 2008` поддерживает агрегаты, которые принимают много входных параметров. Для реализации таких функций агрегирования один или два параметра принимаются в методе `Accumulate`, обрабатываются в коде, а затем объявляются в инструкции `CREATE AGGREGATE` языка T-SQL (если вы не используете автоматическое развертывание `Visual Studio`). Принимающие множество входных параметров агрегаты по-прежнему должны возвращать единственное значение.

Агрегаты являются превосходным использованием программирования SQL CLR. Поскольку им передаются для обработки значения данных, то они обычно выполняют только вычислительные задачи и не производят доступа к данным. Они состоят из скомпилированного кода CLR, поэтому имеют хорошую производительность. В отличие от хранимых процедур, триггеров и функций, они совершенно не могут быть реализованы на языке T-SQL. При всем при этом вы все равно должны делать код ваших агрегатов (особенно метода `Accumulate`) как можно более "легким". Встраивание вашего собст-

венного кода в последовательность действий обработчика запросов является честью, привилегией и большой ответственностью. Воспринимайте эту ответственность всерьез и убедитесь в том, что ваш код оказывает минимально возможное влияние.

## Типы SQL CLR

Последняя функциональная возможность SQL CLR, которую нам предстоит изучить — это определяемые пользователем типы (*user-defined types, UDT*). Эта функциональная возможность, вероятно, является самой интересной, хотя и самой спорной тоже. Она интересна потому, что технически позволяет хранить объекты в базе данных. Она спорна потому, что чревата ошибками. Типы SQL CLR не были реализованы таким образом, чтобы позволить разработчикам создавать объектно-ориентированные базы данных; они были созданы для того, чтобы позволить хранить, извлекать и легко манипулировать типами данных с множественными значениями и множеством поведений.

Типы SQL CLR имеют определенные ограничения в плане индексирования; их значение должно обновляться при обновлении любого из значений их свойств/полей.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Дополнительная информация по SQL CLR UDT имеется в статье MSDN под названием "Using CLR Integration in SQL Server 2005". Вы можете найти эту статью по адресу: <http://msdn.microsoft.com/en-us/library/ms345136.aspx>. Написанная для SQL Server 2005, она применима и к SQL Server 2008. При ее чтении просто помните о некоторых небольших изменениях SQL CLR, произошедших после версии SQL Server 2005 (они описаны в этой главе).

Методы типов SQL CLR должны быть статическими. Поэтому вы не можете вызывать методы из T-SQL как методы экземпляров; вместо этого вы должны использовать специальный синтаксис `TypeName::MethodName()`. Вы можете реализовать свойства (как в любом обычном классе) и читать из них или писать в них из T-SQL при помощи стандартного синтаксиса с разделяющей точкой: `variable.property/column.property`.

В листинге 3.9 (код из структуры `typPoint` файла `typTest.cs` примера проекта) показана реализация `typPoint` (типа SQL CLR, который может быть использован для хранения декартовых координат в базе данных).



**Листинг 3.9. Структура typPoint из файла typTest.cs**

```
[Serializable]
[SqlUserDefinedType (Format.Native)]
public struct typPoint : INullable
{
    private bool m_Null;
    private double m_x;
    private double m_y;
    public override string ToString()
    {
        if (this.IsNull)
            return "NULL";
        else
            return this.m_x + ":" + this.m_y;
    }
    public bool IsNull
    {
        get
        {
            return m_Null;
        }
    }
    public static typPoint Null
    {
        get
        {
            typPoint pt = new typPoint();
            pt.m_Null = true;
            return pt;
        }
    }
    public static typPoint Parse(SqlString s)
    {
        if (s.IsNull)
            return Null;
        else
        {
            // разобрать здесь входную строку для выделения точек
            typPoint pt = new typPoint();
```

```

        char[] parms = new char[1];
        parms[0] = ':';
        string str = (string)s;
        string[] xy = str.Split(parms);
        pt.X = double.Parse(xy[0]);
        pt.Y = double.Parse(xy[1]);
        return pt;
    }
}

public static double Sum(typPoint p )
{
    return p.X + p.Y;
}

public double X
{
    get { return m_x; }
    set { m_x = value; }
}

public double Y
{
    get { return m_y; }
    set { m_y = value; }
}
}

```

Через свойства класса `x` и `y` вы можете обрабатывать координаты как единый столбец или переменную базы данных. Вы можете присваивать значения координат экземпляру типа как разделенную двоеточием строку, например, `3:4`, при помощи метода `Parse` (неявным образом). Затем вы можете прочитать их обратно в том же самом формате при помощи метода `ToString`. После присваивания вы можете прочитать или модифицировать по отдельности их часть `x` или `y` (при помощи отдельных свойств `x` и `y`). Класс реализует интерфейс `INullable` и его свойство `IsNull`. Метод `Sum` демонстрирует, как предоставить статический член и позволить ему получить доступ к свойствам экземпляра (путем принятия экземпляра типа `SQL CLR`, членом которого он является).

Обратите внимание, что класс является `struct`, и что к нему были применены атрибуты `Serializable` и `SqlUserDefinedType`. Подобно атрибуту `SqlUserDefinedAggregate`, атрибут `SqlUserDefinedType` требуется сервером

SQL Server и появляется как в примере кода типа Class Library, так и в версии типа SQL Server. Как и в случае с `SqlUserDefinedAggregate`, мы просто присваиваем параметру `Format` значение `Format.Native` и не используем остальные параметры.

### ПРИМЕЧАНИЕ

Подобно определяемым пользователем агрегатам, в SQL Server 2008 допускаются и типы CLR с размером более 8000 байтов, но реализующий их код должен использовать значение `Format.UserDefined` для параметра `Format` специального атрибута `SqlUserDefinedType`. Можно обратиться к руководству SQL Server Books Online за информацией относительно использования и других параметров этого атрибута.

В листинге 3.10 (код из `struct typBakersDozen` файла `typTest.cs` образцового проекта) повторно реализована логика `BakersDozen`, которую мы использовали в нашем примере агрегирования. На этот раз она выполнена как UDT.

#### Листинг 3.10. Функция `typBakersDozen` из файла `typTest.cs`

```
[Serializable]
[SqlUserDefinedType(Format.Native)]
public struct typBakersDozen : INullable
{
    private bool m_Null;
    private double m_RealQty;
    public override string ToString()
    {
        return (m_RealQty + (long)m_RealQty / 12).ToString();
    }
    public bool IsNull
    {
        get
        {
            return m_Null;
        }
    }
    public static typBakersDozen Null
    {
        get
        {
```

```
        typBakersDozen h = new typBakersDozen();
        h.m_Null = true;
        return h;
    }
}

public static typBakersDozen Parse(SqlString s)
{
    if (s.IsNull)
        return Null;
    else
    {
        typBakersDozen u = new typBakersDozen();
        u.RealQty = double.Parse((string)s);
        return u;
    }
}

public static typBakersDozen ParseDouble(SqlDouble d)
{
    if (d.IsNull)
        return Null;
    else
    {
        typBakersDozen u = new typBakersDozen();
        u.RealQty = (double)d;
        return u;
    }
}

public double RealQty
{
    get { return m_RealQty; }
    set { m_RealQty = value; }
}

public double AdjustedQty
{
    get
    {
        return (m_RealQty + (long)m_RealQty / 12);
    }
    set
```

```

    {
        if (value % 12 == 0)
            m_RealQty = value;
        else
            m_RealQty = value - (long)value / 13;
    }
}
}

```

Свойства `RealQty` и `AdjustedQty` позволяют присвоить значение заказанному количеству и автоматически вычислить откорректированное количество (или наоборот). Фактическое значение является по умолчанию "входным" значением, откорректированное значение является по умолчанию "выходным" значением типа, а методы `Parse` и `ToString` работают так, как и должны. Если свойству `AdjustedQty` присваивается значение, кратное 12 (что было бы неверно), то это значение присваивается свойству `RealQty`, что заставляет `AdjustedQty` устанавливаться в переданное значение плюс его целое частное от деления на 12.

Для развертывания UDT используйте развертывание на основе атрибутов для проектов типа SQL Server. Файл скрипта `CreateObjects.sql` в проекте Management Studio, поставляемом вместе с примерами кодов, содержит код T-SQL, необходимый для развертывания версий Class Library нашего UDT. Вот команда, которая развертывает `typPoint`:

```

CREATE TYPE typPoint
EXTERNAL NAME Chapter03.typPoint

```

Файл скрипта `TestTypPoint.sql` в проекте Management Studio содержит код T-SQL, который тестирует `typPoint`. Запустите его, а затем изучите результаты, чтобы достичь понимания того, как следует работать с этим типом. Скрипт `CreateTblPoint.sql` создает таблицу со столбцом, который типизирован по `typPoint`. Запустите его, а затем выполните файл скрипта `TestTblPoint.sql` для того, чтобы увидеть, как манипулировать таблицами, которые используют SQL CLR UDT.

Файл скрипта `TestTypBakersDozen.sql` содержит код T-SQL, который тестирует `typBakersDozen`. Метод `ParseDouble` демонстрирует, как реализовать метод разбора без `SqlString`. Мы назвали его `ParseDouble`, поскольку сам метод `Parse` перегружен быть не может. Вы должны явно вызвать `ParseDouble` следующим образом:

```

DECLARE @t AS dbo.typBakersDozen
SET @t = typBakersDozen::ParseDouble(12)

```

Это эквивалентно неявному использованию метода по умолчанию `Parse` и присваиванию строки 12:

```
DECLARE @t AS dbo.typBakersDozen  
SET @t = '12'
```

Обратите внимание, что по существу функция `typBakersDozen` хранит значение реального количества, а его свойства на самом деле являются функциями, которые принимают или выражают это значение в его собственной форме или как откорректированное количество. Для свойства `AdjustedQty` нет резервной переменной, блок `get` свойства `AdjustedQty` только применяет формулу к резервной переменной для `RealQty` и возвращает результат.

Как показывают оба этих результата, вам следует представлять себе CLR UDT не как объекты, хранимые в базе данных, а как классы, которые оборачивают одно скалярное значение (или набор таких значений) и предоставляют службы и функции преобразования для манипулирования ими. Именно поэтому компания Microsoft реализовала типы данных `geometry` и `geography` как SQL CLR UDT. Эти типы не хранят сложных объектов, но управляют объектами, которые нельзя представить себе как простые значения.

Точнее говоря, вы не должны думать о SQL CLR UDT как об объектно-реляционных сущностях. Хотя это может показаться противоречащим здравому смыслу, считайте более подходящими средствами для хранения сериализованных .NET-объектов в базе данных использование типа данных `xml` и (де)сериализации.

Мы изучили все пять элементов SQL CLR. Перед тем как закончить наше обсуждение, нам необходимо рассмотреть безопасность сборок и обслуживание объектов SQL CLR в ваших базах данных.

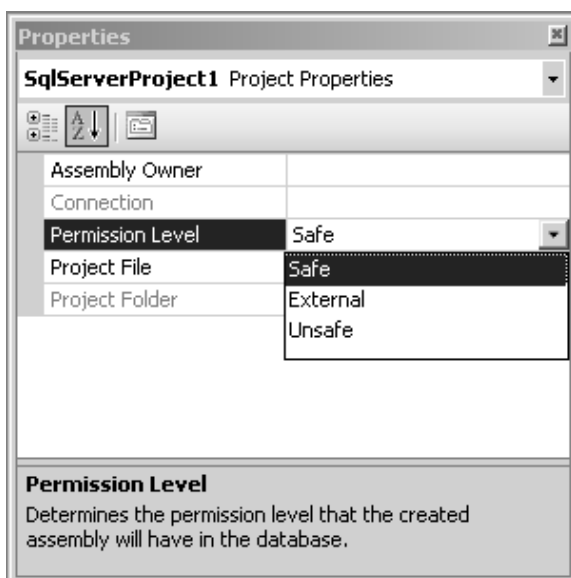
## Безопасность

В зависимости от метода развертывания у вас есть множество способов указать, какой уровень безопасности должен быть обеспечен сборке CLR. Все они требуют, чтобы вы указали один из трех наборов разрешений:

- ☐ `Safe` — сборка может выполнять только локальный доступ к данным и вычислительные задачи;
- ☐ `External_Access` — сборка может выполнять локальный доступ к данным и вычислительные задачи, а также обращаться к сети, файловой системе, реестру и переменным окружения. Несмотря на то, что набор

External\_Access менее строгий, чем Safe, он все равно защищает стабильность сервера;

- ❑ **Unsafe** — сборка имеет неограниченные разрешения и может даже вызывать неуправляемый код. Такая настройка может представлять собой значительную угрозу безопасности SQL Server; только обладатели роли `sysadmin` могут создавать (загружать) такие небезопасные сборки. Обратите также внимание, что для той базы данных, в которую будет загружаться небезопасная сборка, свойство `TRUSTWORTHY` должно быть установлено в значение `ON`, а `dbo` должно иметь разрешение `UNSAFE ASSEMBLY` (либо сборка должна быть специальным образом подписана). В верхней части скрипта `CreateObjects.sql` имеется закомментированный код, который устанавливает свойство `TRUSTWORTHY` базы `AdventureWorks2008` в значение `ON`.

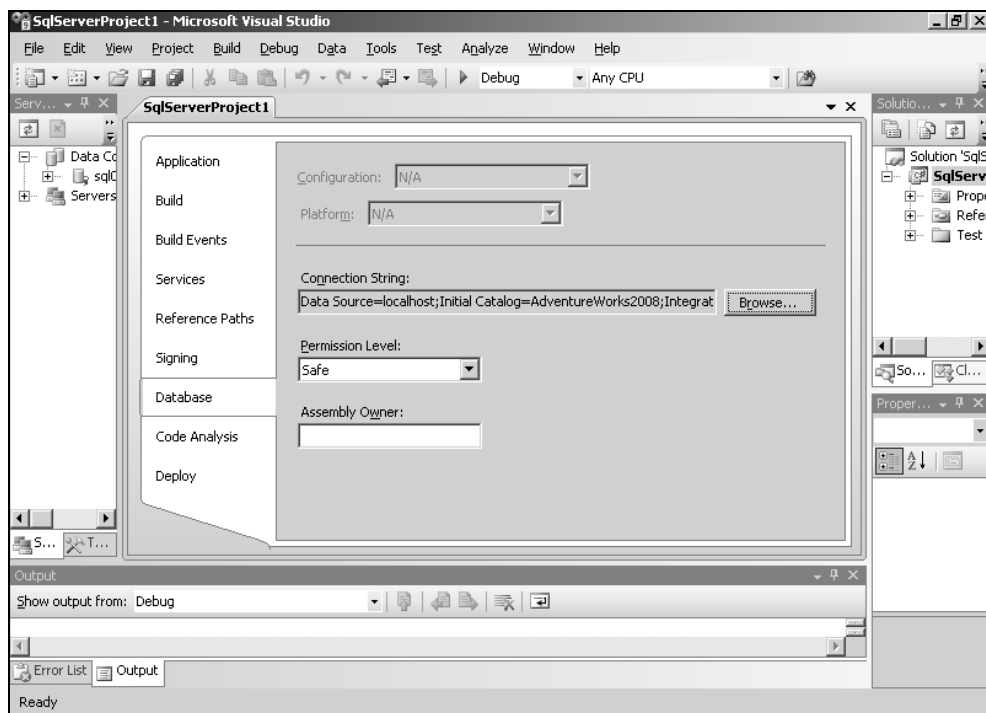


**Рис. 3.7.** Свойство **Permission Level** и варианты его значений в окне **Properties** среды разработки Visual Studio 2008

Когда вы развертываете сборку из Visual Studio, то ее уровень безопасности по умолчанию устанавливается в значение `Safe`. Для того чтобы это изменить, вы можете выбрать узел проекта в окне **Solution Explorer** и настроить свойство **Permission Level** в окне **Properties** (при помощи выбора в комби-

нированном окне одного из значений: **Safe**, **External** или **Unsafe**, как показано на рис. 3.7).

Вы также можете щелкнуть правой кнопкой мыши по узлу проекта в окне **Solution Explorer**, а затем выбрать в контекстном меню пункт **Properties**. Вы можете также дважды щелкнуть по узлу **Properties** (или по узлу **My Project** в проектах Visual Basic). Любой из этих вариантов открывает визуальный конструктор свойств проекта. Выберите в конструкторе вкладку **Database**, а затем выберите в комбинированном окне **Permission Level** набор разрешений, как показано на рис. 3.8 (здесь имеются те же самые три варианта, что и в окне **Properties**).



**Рис. 3.8.** Вкладка **Database** визуального конструктора свойств проекта в среде Visual Studio

Для назначения набора разрешений при помощи T-SQL просто укажите в выражении `WITH PERMISSION_SET` инструкции `CREATE ASSEMBLY` одно из трех: `SAFE`, `EXTERNAL_ACCESS` или `UNSAFE`, как это уже было описано ранее в этой



главе. Тогда в нашем примере в этом выражении использовалось значение по умолчанию `SAFE`.

И наконец, в диалоговом окне **New Assembly** в Management Studio (см. рис. 3.5) и в диалоговом окне **Assembly Properties** вы также можете выбрать в комбинированном окне **Permission Set** одно из трех значений: **Safe**, **External Access** или **Unrestricted**. Последний из этих трех вариантов эквивалентен выбору набора разрешений `Unsafe` в Visual Studio или T-SQL.

## Изучение типов SQL CLR в базе данных и управление ими

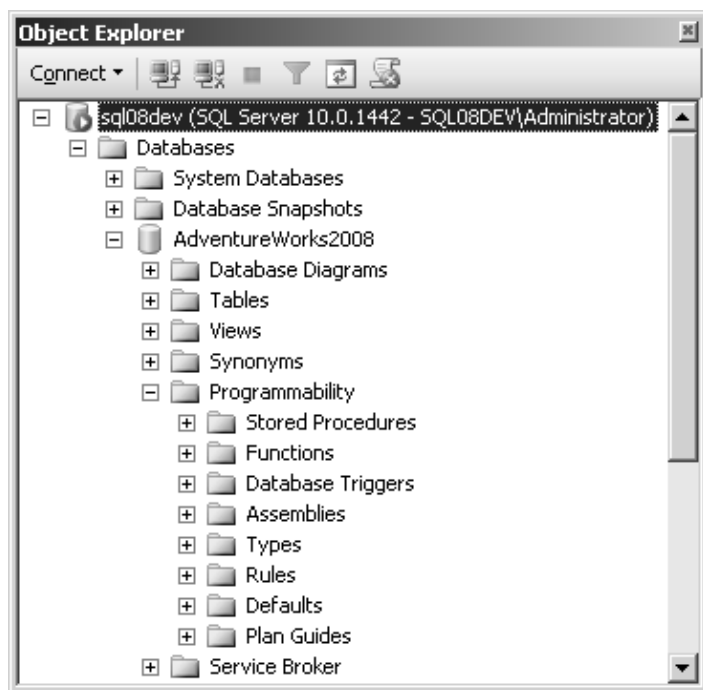
Тяжело удерживать в голове все хранимые процедуры, функции, триггеры, UDT, а также зависимости между ними. К счастью, представляется легким изучение развернутых элементов SQL CLR при помощи пользовательского интерфейса Management Studio. Все элементы SQL CLR (имеющиеся внутри базы данных) можно найти в окне **Object Explorer** программы Management Studio. Для того чтобы найти их в древовидном представлении окна **Object Explorer**, необходимо сначала перейти в узел `\servername\Databases\databasename` (где `servername` и `databasename` — это соответственно имена вашего сервера и базы данных). В табл. 3.1 приведены подузлы этого узла, которые содержат элементы SQL CLR.

Таблица 3.1. Поиск объектов CLR в Object Explorer

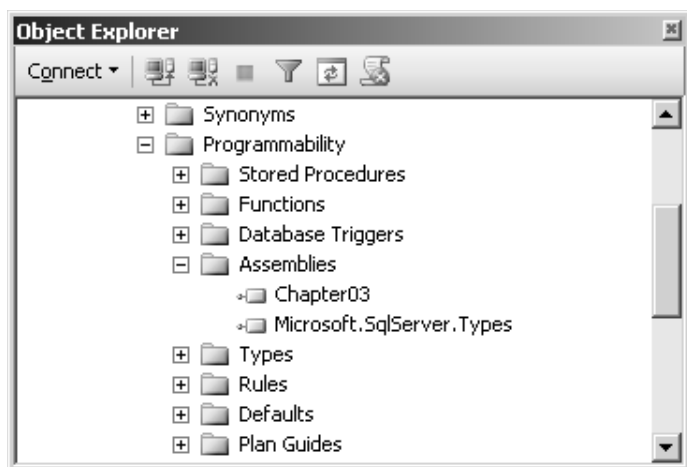
| Чтобы увидеть...                                                                 | Смотрите в...                                                                                                                  |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Родительский узел для хранимых процедур, триггеров DDL, функций, агрегатов и UDT | <b>Programmability</b> (рис. 3.9)                                                                                              |
| Сборки                                                                           | <b>Programmability\Assemblies</b> (рис. 3.10)                                                                                  |
| Хранимые процедуры                                                               | <b>Programmability\Stored Procedures</b> (рис. 3.11)                                                                           |
| Функции                                                                          | <b>Programmability\Functions\Scalar-Valued Functions</b> и <b>Programmability\Functions\Table-Valued Functions</b> (рис. 3.12) |

Таблица 3.1 (окончание)

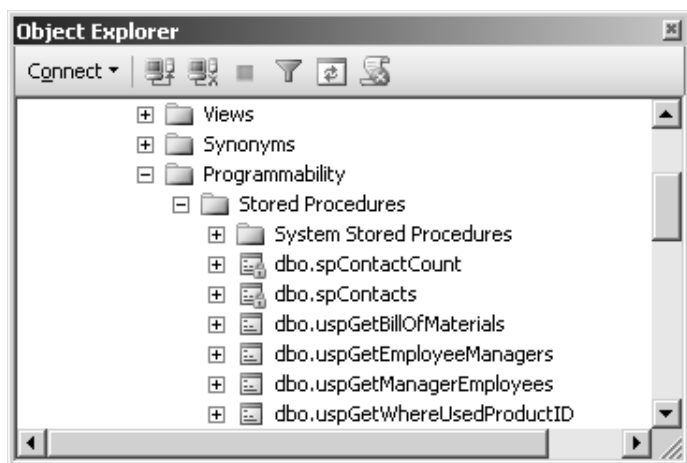
| Чтобы увидеть... | Смотрите в...                                                                                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Агрегаты         | <b>Programmability\Functions\Aggregate Functions</b><br>(рис. 3.12)                                                                                                         |
| Триггеры DML     | <b>Tables\tablename\Triggers</b> , где tablename — это имя таблицы базы данных (включая имя схемы), для которой определен триггер (рис. 3.13)                               |
| Триггеры DDL     | <b>Programmability\Database Triggers</b> (рис. 3.14). (Для триггеров уровня сервера — <b>\servername\Server Objects\Triggers</b> , где servername — это имя вашего сервера) |
| UDT              | <b>Programmability\Types\User-Defined Types</b><br>(рис. 3.15)                                                                                                              |



**Рис. 3.9.** Окно **Object Explorer** в SQL Server Management Studio с развернутым узлом **Programmability**



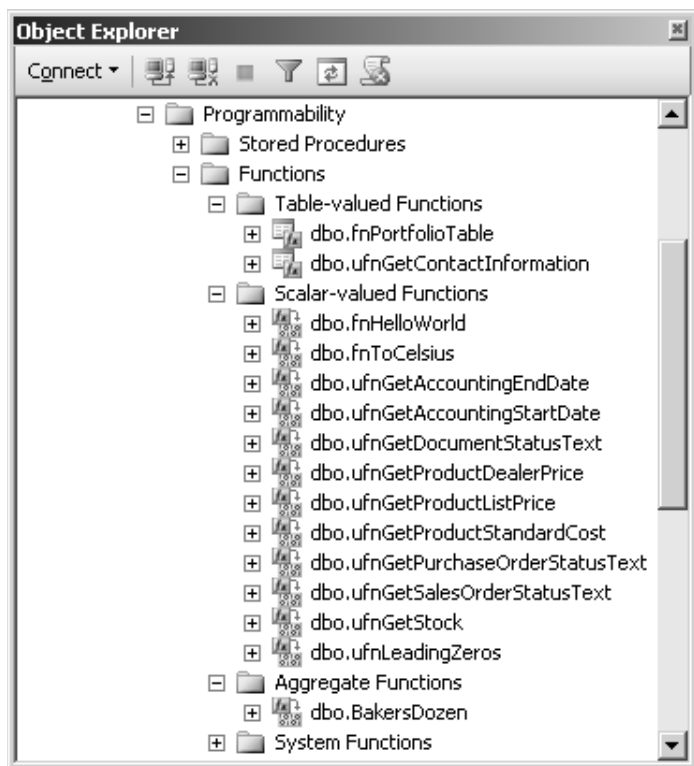
**Рис. 3.10.** Окно **Object Explorer** с выделенным узлом **Assemblies** (обратите внимание на присутствие сборки `Microsoft.SqlServer.Types`, которая является сборкой Microsoft SQL CLR для новых типов данных SQL 2008)



**Рис. 3.11.** Окно **Object Explorer** с выделенными хранимыми процедурами SQL CLR

Помните, что для того, чтобы увидеть ваши объекты SQL CLR, вам может понадобиться использовать для перечисленных в таблице узлов пункт **Refresh** их контекстного меню. Если вы развернули или удалили какие-то объекты SQL CLR (скоро мы это обсудим) уже после открытия подключения **Object Explorer** к вашей базе данных, то древовидное представление станет

неактуальным, и его надо будет обновить. Обратите внимание, что в древовидном представлении значки для хранимых процедур SQL CLR и триггеров SQL CLR DML несколько отличаются от своих аналогов в T-SQL — у них есть маленький желтый замочек в правом нижнем углу.



**Рис. 3.12.** Окно **Object Explorer** с выделенными скалярными, агрегатными и возвращающими табличные значения функциями SQL CLR

После того как вы нашли элемент SQL CLR в окне **Object Explorer**, вы можете щелкнуть правой кнопкой мыши по его узлу в дереве и посредством выбора пункта **Script object type As** в контекстном меню сгенерировать для него скрипт CREATE, DROP или (в некоторых случаях) ALTER. Текст скрипта можно вставить в окно нового запроса, в файл или в буфер обмена, либо его можно использовать для создания задания агента SQL Server Agent.

Для хранимых процедур вы можете также создать скрипты Execute или (выбрав пункт **Execute Stored Procedure** контекстного меню) выполнить их ин-

терактивно и сгенерировать соответствующий скрипт в диалоговом окне **Execute Procedure** программы Management Studio. Это диалоговое окно явным образом запрашивает у вас все входные параметры, определенные для выбранной хранимой процедуры.

Кроме генерирования скриптов для ваших элементов SQL CLR, вы можете просмотреть их зависимости (как зависящие от них объекты, так и те объекты, от которых они сами зависят). Просто щелкните по объекту правой кнопкой мыши и выберите в контекстном меню пункт **View Dependencies**.

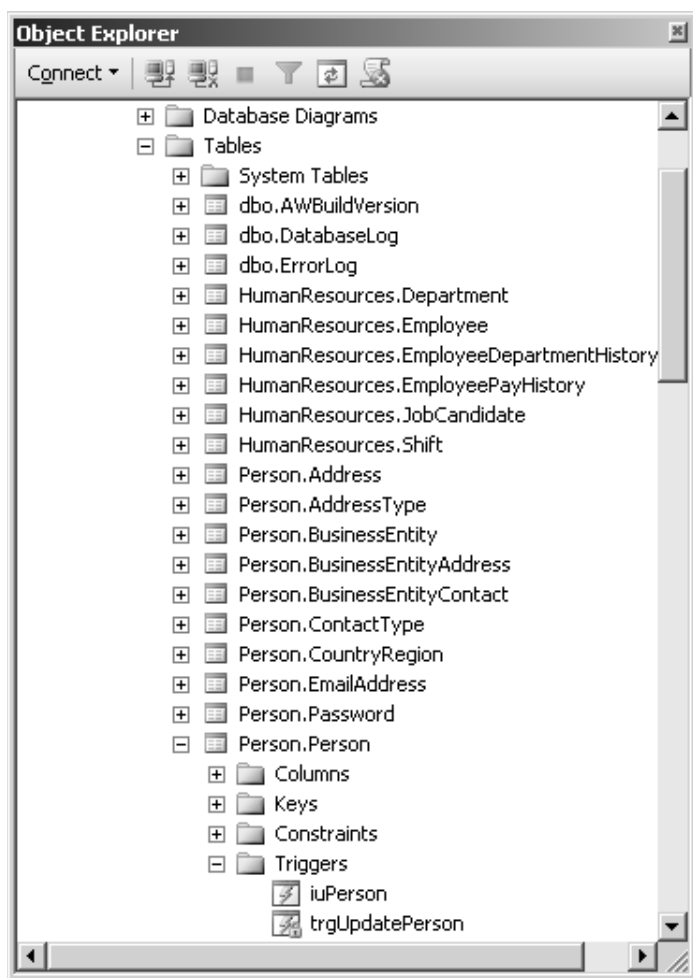


Рис. 3.13. Окно **Object Explorer** с выделенным триггером SQL CLR DML

Для удаления ваших объектов SQL CLR (либо с целью подготовки к загрузке новой версии вашей сборки, либо для удаления совсем) у вас есть сразу несколько возможных вариантов. Для проектов типа SQL Server повторное развертывание вашей сборки приводит к тому, что Visual Studio удаляет ее и все объекты SQL CLR внутри нее (которые были ранее развернуты средой Visual Studio). Это означает, что новые версии можно развертывать из Visual Studio без всяких подготовительных шагов.

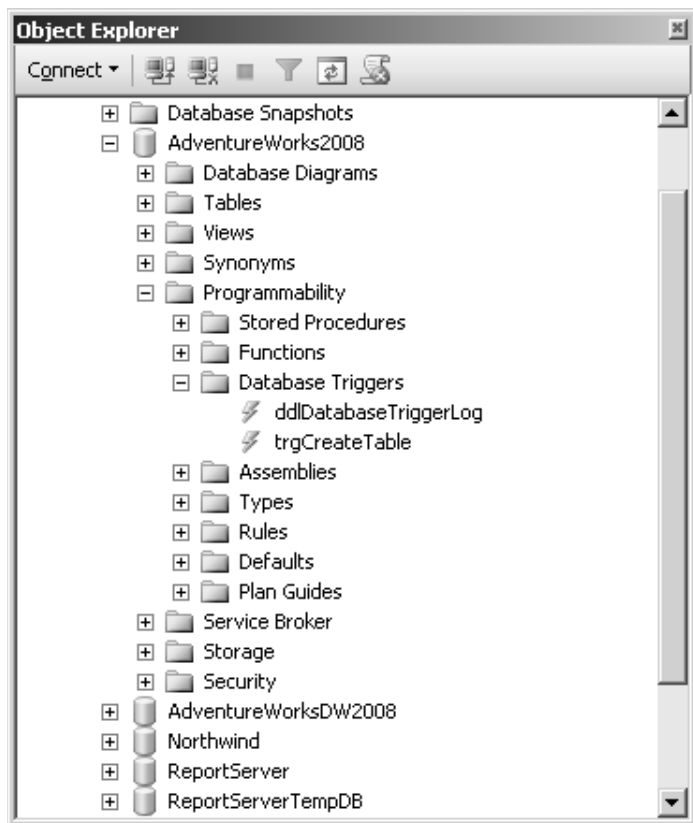


Рис. 3.14. Окно **Object Explorer** с выделенным триггером SQL CLR DDL

Для проектов типа Class Library вы должны выдать команды DROP языка T-SQL для каждого из ваших объектов SQL CLR, а затем и для самой сборки. Перед удалением элемента SQL CLR вы должны удалить все зависимые объекты. Например, перед удалением объекта `typPoint` вы должны удалить `tblPoint`. Вы можете написать эти скрипты DROP вручную или сгенерировать

их при помощи пунктов контекстного меню **Script object type As, DROP To** (в окне **Object Explorer** программы Management Studio).

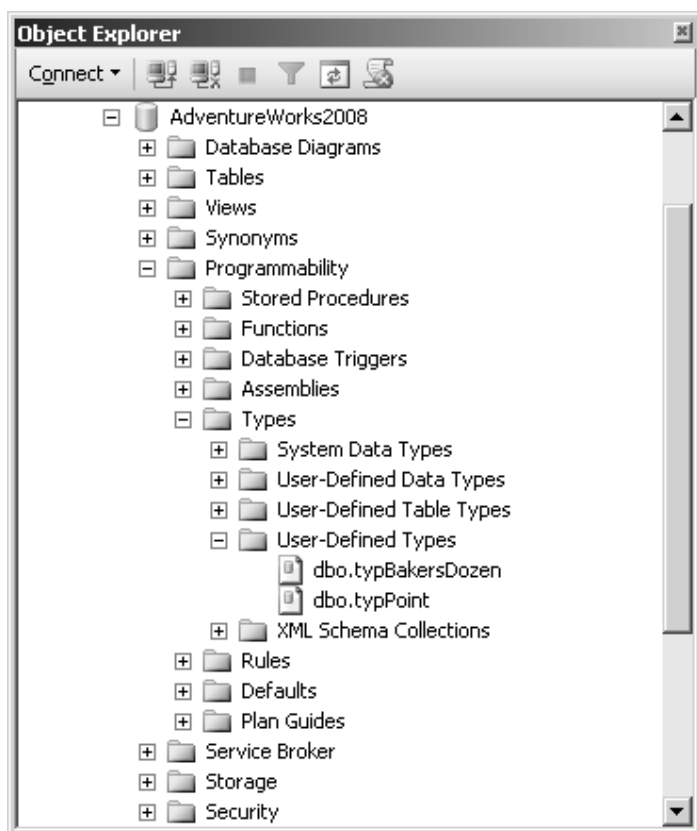


Рис. 3.15. Окно **Object Explorer** с выделенным SQL CLR UDT

Вы можете также использовать для удаления объекта пункт **Delete** контекстного меню любого объекта SQL CLR в окне **Object Explorer** программы Management Studio. При этом вы увидите диалоговое окно **Delete Object**, показанное на рис. 3.16.

Файл скрипта Cleanup.sql в проекте Management Studio содержит все необходимые команды DROP (в правильном порядке) для удаления всех следов нашего проекта типа SQL Server или типа Class Library из базы данных AdventureWorks2008. Для проекта типа SQL Server вам необходимо выполнить этот скрипт только в том случае, если вы хотите удалить объекты совсем.

Для проекта типа Class Library запускайте его перед развертыванием обновленной версии вашей сборки или если вы хотите удалить эти объекты совсем.

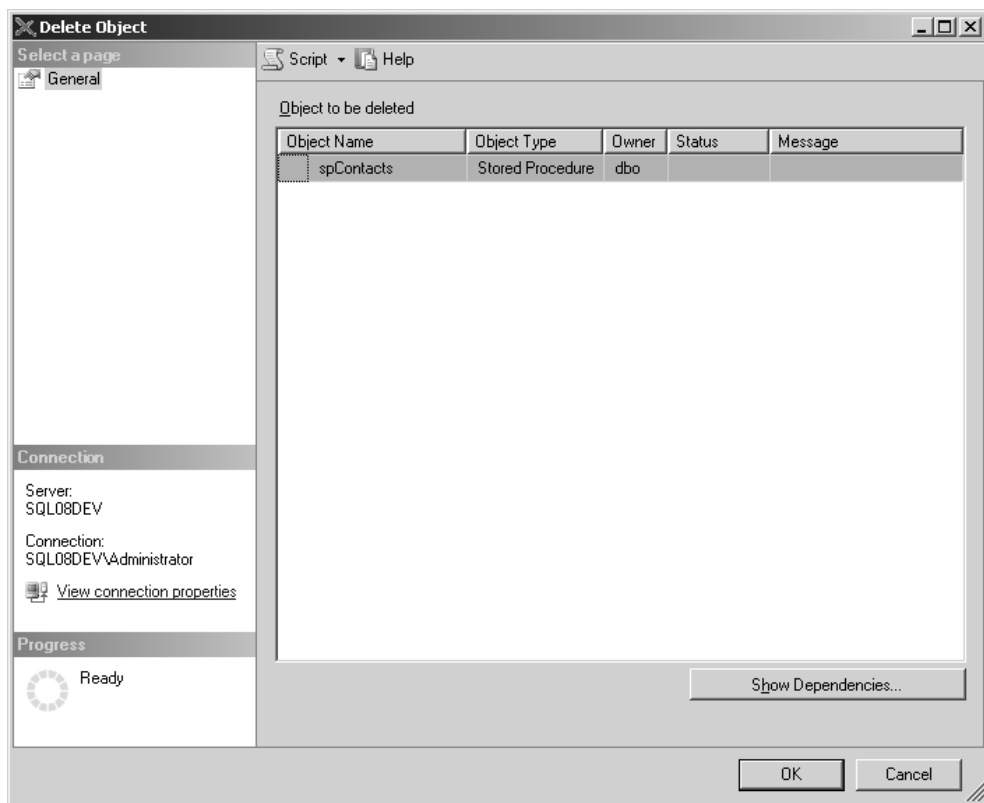


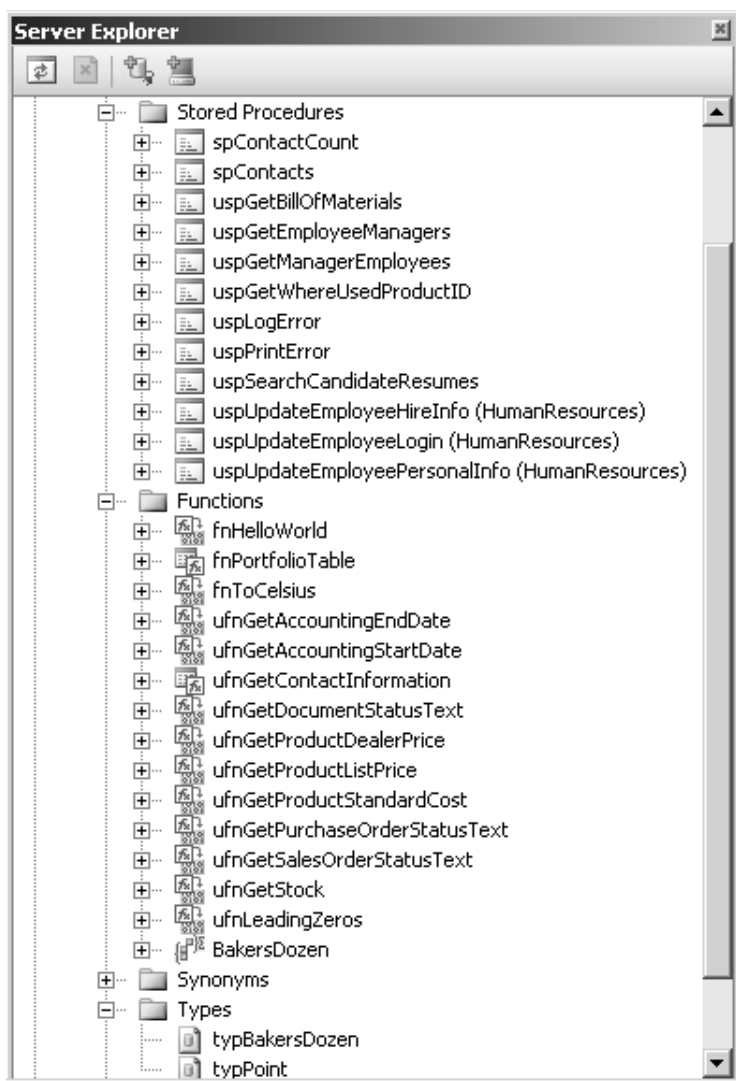
Рис. 3.16. Диалоговое окно **Delete Object** программы Management Studio

### ПРИМЕЧАНИЕ

Скрипт Cleanup.sql удаляет также таблицы tblAggregateTest и tblPoint, которые создаются соответственно скриптами CreateTblAggregateTest.sql и CreateTblPoint.sql. Если вы запускали CreateTblPoint.sql, то для того, чтобы удалить typPoint и развернуть снова проект типа SQL Server, вам придется удалить tblPoint (либо вручную, либо при помощи запуска Cleanup.sql).

Объекты SQL CLR (за исключением триггеров DDL) можно просмотреть и в окне **Server Explorer** в Visual Studio 2008 (рис. 3.17).

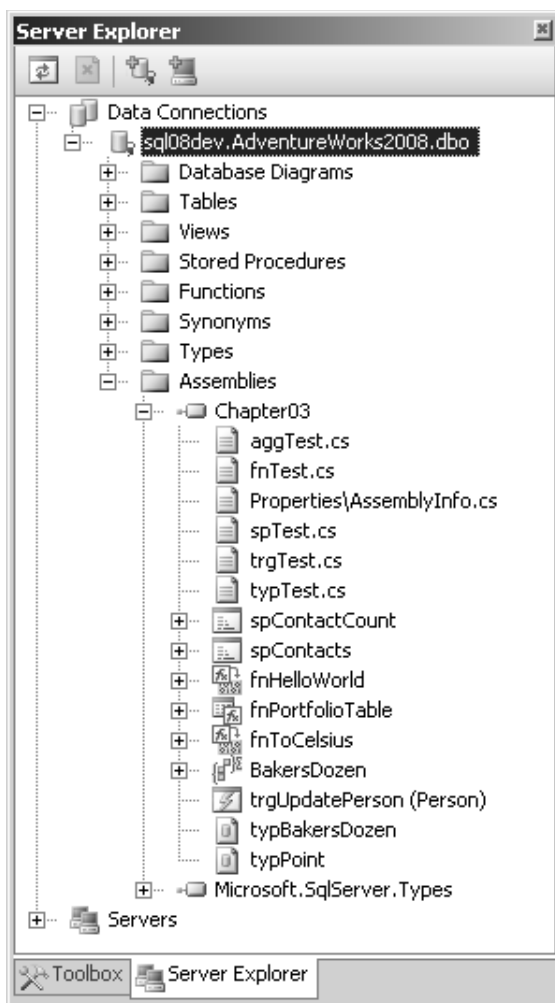




**Рис. 3.17.** Окно **Server Explorer** в Visual Studio с выделенными хранимыми процедурами, функциями, агрегатами и UDT

Большинство объектов вы найдете под их родительскими узлами: хранимые процедуры SQL CLR находятся в узле **Stored Procedures**; скалярные функции, TVF и агрегаты — в узле **Functions**; узлы **Types** и **Assemblies** содержат соответствующие объекты; триггеры DML находятся в узле той таблицы, которой они принадлежат (следом за столбцами этой таблицы).

Вы можете также углубиться в узел конкретной сборки и просмотреть список всех ее объектов SQL CLR (за исключением триггеров DDL), а также файлы исходного кода, которые ее составляют (как показано на рис. 3.18).



**Рис. 3.18.** Окно **Server Explorer** с развернутыми дочерними узлами **Assemblies**

Для созданных (из проектов типа SQL Server) сборок вы можете дважды щелкнуть по любому объекту SQL CLR в окне **Server Explorer** для просмотра его исходного кода. (Вы можете также сделать это и посредством выбора

пункта **Open** в контекстном меню узла объекта SQL CLR или при помощи пунктов **Data | Open** главного меню Visual Studio при выбранном узле). Если проект сборки открыт в тот момент, когда вы открываете исходный код объекта, то код можно будет редактировать, если проект не открыт — исходный код можно будет только читать.

### **ПРЕДУПРЕЖДЕНИЕ**

Поскольку наш триггер SQL CLR DML с именем `trgUpdatePerson` был развернут при помощи языка T-SQL в скрипте `PostDeployScript.sql`, а не посредством атрибута `SqlTrigger`, то его исходный код нельзя просмотреть через узел **Tables\Person (Person)\trgUpdatePerson** подключения окна **Server Explorer** или через узел **Assemblies\Chapter03\trgUpdatePerson (Person)**. Однако вы можете просмотреть его исходный код через узел **Assemblies\Chapter03\trgTest.cs**.

## **Лучшие практики использования SQL CLR**

Перед тем как завершить эту главу, мы хотели бы резюмировать некоторые лучшие практики для надлежащего использования программирования под SQL CLR.

Интеграция CLR с сервером SQL Server 2008 является мощной технологией. В некоторых случаях она позволяет вам делать такие вещи, которые в T-SQL делать нецелесообразно (применение сложной вычислительной логики в хранимых процедурах или триггерах), а в других случаях — чего вы в T-SQL совершенно не сможете сделать (например, создать свои собственные функции агрегирования).

Однако по-прежнему остается фактом то, что выборку и модификацию данных из наборов гораздо лучше выполнять при помощи декларативных конструкций языка T-SQL, чем в процедурных конструкциях объектной модели .NET и ADO.NET. Функциональность SQL CLR нужно приберегать для тех специфических ситуаций, когда требуется мощь .NET как вычислительного ядра.

Вообще говоря, функции и агрегаты являются отличными применениями интеграции SQL CLR. Техника UDT в отслеживании сложных значений (а не объектов как таковых) также хорошо интегрирована с SQL CLR.

Для хранимых процедур и триггеров мы рекомендуем вам начать с такого допущения: они должны быть написаны на T-SQL, а при помощи кода SQL

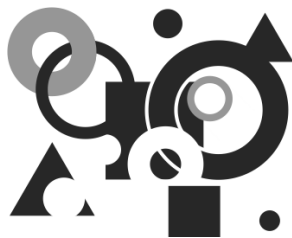
CLR их следует писать только в том случае, когда их не получается написать иначе. И перед этим учтите, что функции, агрегаты и UDT из SQL CLR можно использовать в хранимых процедурах и триггерах T-SQL.

## Резюме

В этой главе вы изучили "механизмы" разработки пяти основных элементов SQL CLR и использование их из T-SQL. Вы видели, как можно использовать преимущества интеграции SQL Server 2008/Visual Studio 2008, а также как разрабатывать код SQL CLR в обычных сборках типа Class Library и развертывать их при помощи T-SQL и SQL Server Management Studio. Вы изучили также большинство атрибутов .NET-кода SQL CLR и их применение в проектах типа SQL Server и стандартных проектах типа Class Library. Вы получили представление о том, как использовать Management Studio и окно **Server Explorer** в Visual Studio 2008 в качестве инструментов управления вашими объектами SQL CLR. Мы обсудили сценарии, при которых использование SQL CLR является хорошим выбором, а также те сценарии, для которых более удачным выбором будет T-SQL.

*Часть I* данной книги в основном описывает создание баз данных, поэтому в этой главе мы намеренно сосредоточились на использовании объектов SQL CLR из языка T-SQL и из программы Management Studio. В предыдущей главе мы отметили некоторые улучшения языка T-SQL, которые вы можете использовать в процессе разработки баз данных. *Часть III* данной книги описывает разработку приложений, которые используют базы данных. В примерах кодов *главы 10* мы демонстрируем, как использовать ваши объекты SQL CLR в .NET-приложениях. Материал этой главы и примеры кодов *главы 10* дают хороший материал для глубокого погружения в разработку под SQL CLR.

## ГЛАВА 4



# Управление сервером

*Стивен Форте*

Работа с сервером Microsoft SQL Server требует от вас выполнения многих задач управления. Будучи разработчиком, вы могли бы подумать, что за управление сервером отвечают администраторы баз данных (DBA), и что вам нужно беспокоиться только о написании кода для создания приложений. Это не всегда верно. Иногда вам приходится выполнять простые задачи по обслуживанию, такие как подключение базы данных как части приложения или программы инсталляции. Например, когда вы развертываете приложение, вам может понадобиться включить в его состав вспомогательную программу, которая подключает базу данных приложения к существующему SQL Server. Вы можете также включить в свое приложение простые функции управления безопасностью базы данных. Иногда может потребоваться обеспечить такое автоматическое резервное копирование и восстановление, которое отсутствует в возможностях агента SQL Server. Корпоративному разработчику может понадобиться программно осуществлять простые административные функции, чтобы обеспечить выполнение некоторых административных задач или автоматизировать простые задачи (такие как выполнение группы подключений и отключений баз данных).

В прошлом вы выполняли такие задачи при помощи объектов SQL Distributed Management Objects (SQL-DMO). Вы создавали приложение (возможно, при помощи Microsoft Visual Basic), которое выполняло стандартные функции обслуживания или административные задачи (в соответствии с вводом пользователя). Если ваши клиенты или пользователи не имели администратора базы данных, то вы могли использовать SQL-DMO для создания простого в использовании инструмента администрирования, который "за кулисами" выполнял достаточно сложные задачи.

Microsoft SQL Server 2005 ввел новую объектную модель под названием "управляющие объекты SQL Server" (SQL Server Management Objects, SMO). SMO позволяет вам делать все, что вы делали при помощи SQL-DMO, включая управление серверами баз данных SQL Server 2000, SQL Server 2005 и SQL Server 2008. SQL Server 2008 ввел управление на основе политик (Policy-Based Management, PBM) — новую инфраструктуру, которая позволяет вам убедиться в том, что конфигурация вашего сервера или группы серверов соответствует настроенной вами политике. Пройдут те времена, когда на одном компьютере не работало то, что работало на другом компьютере. В этой главе мы изучим как SMO, так и PBM, и вы узнаете, как использовать эти функциональные возможности для упрощения управления серверами.

## Что такое SMO

SMO — это объектная модель на основе .NET, включенная вместе с клиентскими инструментами SQL Server 2008 в сборку с названием `Microsoft.SqlServer.Smo.dll`. Прочие вспомогательные сборки обычно находятся в каталоге `Program Files\Microsoft SQL Server\100\SDK\Assemblies` (для 32-битной системы) или в каталоге `Program Files (x86)\Microsoft SQL Server\100\SDK\Assemblies` (для 64-битной системы). Некоторые пространства имен SMO показаны в табл. 4.1 (с указанием их сборки, если это не `Microsoft.SqlServer.Smo.dll`).

**Таблица 4.1.** Некоторые пространства имен SMO

| Класс                                                 | Функция                                                                                                                                                                 |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Microsoft.SqlServer.Management.Smo</code>       | Основное пространство имен SMO, содержит классы экземпляров, служебные классы и перечисления, которые используются для программного манипулирования сервером SQL Server |
| <code>Microsoft.SqlServer.Management.Common</code>    | Классы, которые являются общими для Replication Management Objects (RMO) и SMO (в сборке <code>Microsoft.SqlServer.SqlWmiManagement.dll</code> )                        |
| <code>Microsoft.SqlServer.Management.Smo.Agent</code> | Классы для SQL Server Agent                                                                                                                                             |
| <code>Microsoft.SqlServer.Management.Smo.Wmi</code>   | Классы для поставщика WMI Provider (в <code>Microsoft.SqlServer.SqlWmiManagement.dll</code> )                                                                           |

Таблица 4.1 (окончание)

| Класс                                                        | Функция                                                                                                   |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Microsoft.SqlServer.<br>Management.Smo.<br>RegisteredServers | Классы для идентификации и работы с зарегистрированными серверами (в Microsoft.SqlServer.SmoExtended.dll) |
| Microsoft.SqlServer.<br>Management.Smo.Mail                  | Классы для Database Mail                                                                                  |
| Microsoft.SqlServer.<br>Management.Smo.Broker                | Классы для SQL Server Service Broker                                                                      |

SMO позволяет программно манипулировать версиями SQL Server 2008, 2005, 2000 и 7.0. При помощи SMO доступна вся функциональность SQL Server Management Studio, однако те функции, которые имеются только в SQL Server 2008, не будут работать в более ранних версиях SQL Server. На самом деле SMO предлагает на несколько функций больше, чем Management Studio, и большинством функциональных возможностей SQL Server можно управлять из SMO, причем бóльшим их количеством, чем из Management Studio. Если бы вы захотели, вы могли бы даже написать более функциональную версию Management Studio при помощи Windows Forms или Windows Presentation Foundation (WPF) и SMO.

Выполнение скриптов T-SQL приводит к уязвимости внедрения кода SQL. SMO существенно уменьшает пространство для атаки, поскольку разработчик явно определяет, какие функции предоставляются. Пользователь не сможет внедрить вредоносный код в вашу пользовательскую функцию Backup, использующую SMO и скомпилированный код на языке C#.

## По поводу SQL-DMO

Для выполнения программных задач администрирования разработчикам ранее приходилось использовать SQL-DMO (на базе модели Component Object Model, COM). SMO является логическим развитием SQL-DMO. Чтобы учесть новые функциональные возможности SQL Server 2005-2008 и предоставить максимальную поддержку языка определения данных (DDL) и администрирования SQL Server 2008, в SMO добавлено более 150 новых классов. В документации SQL Server 2008 Books Online есть полный список соответствия объектов и функций SQL-DMO и SMO, а также подробное описание объектной модели SMO.

Основное преимущество SMO состоит в том, что это собственная объектная модель .NET, повышенной производительности и масштабируемости по сравнению с SQL-DMO. SMO имеет кэшируемую объектную модель, которая позволяет вам изменить несколько свойств объекта до того, как они будут обновлены в SQL Server. SMO делает меньше обменов с сервером, имеет также оптимизированное создание экземпляров, а это означает, что вы можете создавать объекты частично или полностью. Вы можете быстро загрузить много объектов (если не будете создавать экземпляры всех свойств объектов). В отличие от этого, SQL-DMO должен при загрузке объекта загрузить и все его свойства, что увеличивает сетевой трафик и использование ресурсов сервера.

При такой объектной модели SMO является предпочтительным интерфейсом прикладного программирования (API) для программного управления и манипулирования экземплярами SQL Server. Для обеспечения обратной совместимости вместе с SQL Server поставляется и SQL-DMO. Поскольку использование SQL-DMO не поощряется (его не следует использовать), лучше для COM-приложений использовать SMO совместно с .NET COM Interop, чем использовать SQL-DMO (чтобы в будущем устранить проблемы с миграцией и гарантировать поддержку со стороны компании Microsoft). Если же вам по какой-то причине необходим SQL-DMO, то помните, что SQL-DMO имеет управляемый объект, который упрощает интерфейс с WMI (для поддержки мониторинга WMI и конфигурирования сервера). Если вы рассматриваете возможность перехода на SMO, то взгляните на табл. 4.2 (адаптированную из руководства Books Online), в которой показано соответствие SQL-DMO и SMO.

**Таблица 4.2. Соответствие SQL-DMO и SMO**

| Объект SQL-DMO                  | Эквивалент в SMO                                                                                                                              |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Alert и AlertSystem             | Пространство имен<br>Microsoft.SqlServer.Management.Agent                                                                                     |
| Application                     | Удален                                                                                                                                        |
| BackupDevice, Backup, и Backup2 | Объекты Backup и BackupRestoreBase                                                                                                            |
| BulkCopy и BulkCopy2            | Заменены объектом Transfer                                                                                                                    |
| Category                        | Перемещен в пространство имен<br>Microsoft.SqlServer.Management.Agent;<br>заменен на объекты AlertCategory,<br>OperatorCategory и JobCategory |



Таблица 4.2 (продолжение)

| Объект SQL-DMO                                                                                                                                                                                                                                                         | Эквивалент в SMO                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Check                                                                                                                                                                                                                                                                  | Объект Check                                                                                                               |
| Column и Column2                                                                                                                                                                                                                                                       | Объект Column                                                                                                              |
| Configuration                                                                                                                                                                                                                                                          | Объекты Configuration<br>и ConfigurationBase                                                                               |
| ConfigValue                                                                                                                                                                                                                                                            | Объект ConfigProperty                                                                                                      |
| Database и Database2                                                                                                                                                                                                                                                   | Объект Database                                                                                                            |
| DatabaseRole и Database-<br>Role2                                                                                                                                                                                                                                      | Объект DatabaseRole                                                                                                        |
| Объект DBFile                                                                                                                                                                                                                                                          | Объект DataFile                                                                                                            |
| DBOption и DBOption2                                                                                                                                                                                                                                                   | Перемещен в объект DatabaseOptions                                                                                         |
| Default и Default2                                                                                                                                                                                                                                                     | Объект Default                                                                                                             |
| DistributionArticle<br>и DistributionArticle2,<br><br>DistributionDatabase<br>и DistributionDatabase2,<br><br>DistributionPublication<br>и DistributionPublication2,<br><br>DistributionSubscription<br>и DistributionSubscription2,<br><br>Distributor и Distributor2 | Часть RMO, пространство имен<br>Microsoft.SqlServer.Replication                                                            |
| DRIDefault                                                                                                                                                                                                                                                             | Перемещен в объект ScriptingOptions                                                                                        |
| FileGroup и FileGroup2                                                                                                                                                                                                                                                 | Объект FileGroup                                                                                                           |
| FullTextCatalog<br>и FullTextCatalog2                                                                                                                                                                                                                                  | Объекты FullTextCatalog и FullTextIndex                                                                                    |
| Index и Index2                                                                                                                                                                                                                                                         | Объект Index                                                                                                               |
| IntegratedSecurity                                                                                                                                                                                                                                                     | Функциональность перенесена<br>в объект ServerConnection<br>пространства имен<br>Microsoft.SqlServer.Management.<br>Common |

Таблица 4.2 (продолжение)

| Объект SQL-DMO                                                                                         | Эквивалент в SMO                                                                                  |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Job, JobFilter,<br>JobHistoryFilter,<br>JobSchedule, JobStep,<br>JobServer и JobServer2,<br>и Operator | Перенесены в пространство имен<br>Microsoft.SqlServer.Management.<br>Smo.Agent                    |
| Key                                                                                                    | Объекты ForeignKey и Index                                                                        |
| LinkedServer<br>и LinkedServer2                                                                        | Объект LinkedServer                                                                               |
| LinkedServerLogin                                                                                      | Объект LinkedServerLogin                                                                          |
| LogFile                                                                                                | Объект LogFile                                                                                    |
| Login и Login2                                                                                         | Объект Login                                                                                      |
| MergeArticle<br>и MergeArticle2                                                                        | Объект MergeArticle;<br>перемещен в пространство имен<br>Microsoft.SqlServer.Replication          |
| MergeDynamicSnapshotJob                                                                                | Пространство имен<br>Microsoft.SqlServer.Replication                                              |
| MergePublication<br>и MergePublication2                                                                | Объект MergePublication; перемещен<br>в пространство имен<br>Microsoft.SqlServer.Replication      |
| MergePullSubscription<br>и MergePullSubscription2                                                      | Объект MergePullSubscription;<br>перемещен в пространство имен<br>Microsoft.SqlServer.Replication |
| MergeSubscription                                                                                      | Объект MergeSubscription; перемещен<br>в пространство имен Micro-<br>soft.SqlServer.Replication   |
| MergeSubsetFilter                                                                                      | Перемещен в пространство имен<br>Microsoft.SqlServer.Replication                                  |
| NameList                                                                                               | Удален; аналогичная функциональность<br>имеется в объекте Scripter                                |
| Permission и Permission2                                                                               | Объекты ServerPermission,<br>DatabasePermission, ApplicationRole<br>и ObjectPermission            |
| Property                                                                                               | Объект Property                                                                                   |

Таблица 4.2 (продолжение)

| Объект SQL-DMO                                           | Эквивалент в SMO                                                                                 |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Publisher и Publisher2                                   | Объект ReplicationServer; перемещен в пространство имен Microsoft.SqlServer.Replication          |
| QueryResults и QueryResults2                             | Заменены системными объектами DataTable или DataSet                                              |
| RegisteredServer и RegisteredSubscriber                  | Перемещен в пространство имен Microsoft.SqlServer.Replication                                    |
| Registry и Registry2                                     | Удалены                                                                                          |
| RemoteLogin                                              | Объект ServerConnection; перемещен в пространство имен Common                                    |
| RemoteServer и RemoteServer2                             | Объект ServerConnection; перемещен в пространство имен Microsoft.SqlServer.Management.Common     |
| Replication, ReplicationDatabase, и ReplicationDatabase2 | Перемещен в пространство имен Microsoft.SqlServer.Replication                                    |
| ReplicationSecurity                                      | Объект ServerConnection; перемещен в пространство имен Microsoft.SqlServer.Management.Common     |
| ReplicationStoredProcedure и ReplicationStoredProcedure2 | Объект ReplicationStoredProcedure; перемещен в пространство имен Microsoft.SqlServer.Replication |
| ReplicationTable и ReplicationTable2                     | Объект ReplicationTable; перемещен в пространство имен Microsoft.SqlServer.Replication           |
| Restore и Restore2                                       | Объекты Restore и BackupRestoreBase                                                              |
| Rule и Rule2                                             | Объект Rule                                                                                      |
| Schedule                                                 | Перемещен в пространство имен Microsoft.SqlServer.Replication                                    |
| ServerGroup                                              | Удален                                                                                           |
| ServerRole                                               | Объект ServerRole                                                                                |
| SQLObjectList                                            | Массив SqlSmoObject                                                                              |

Таблица 4.2 (окончание)

| Объект SQL-DMO                                    | Эквивалент в SMO                                                                                   |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------|
| SQLServer и SQLServer2                            | Объект Server                                                                                      |
| StoredProcedure<br>и StoredProcedure2             | Объекты StoredProcedure<br>и StoredProcedureParameter                                              |
| Subscriber<br>и Subscriber2                       | Объект Subscriber; перемещен<br>в пространство имен Microsoft.<br>SqlServer.Replication            |
| SystemDatatype<br>и SystemDataType2               | Объект DataTypee                                                                                   |
| Table и Table2                                    | Объект Table                                                                                       |
| TargetServer<br>и TargetServerGroup               | Перенесены в пространство имен<br>Microsoft.SqlServer.Management.<br>Smo.Agent                     |
| TransactionLog                                    | Функциональность перенесена в объект<br>Database                                                   |
| TransArticle<br>и TransArticle2                   | Объект TransArticle; перемещен<br>в пространство имен Microsoft.<br>SqlServer.Replication          |
| Метод Transfer<br>и объект Transfer2              | Объект Transfer                                                                                    |
| TransPublication<br>и TransPublication2           | Объект TransPublication;<br>перемещен в пространство имен Microsoft.<br>SqlServer.Replication      |
| TransPullSubscription<br>и TransPullSubscription2 | Объект TransPullSubscription;<br>перемещен в пространство имен Microsoft.<br>SqlServer.Replication |
| Trigger и Trigger2                                | Объект Trigger                                                                                     |
| User и User2                                      | Объект User                                                                                        |
| UserDefinedDatatype<br>и UserDefinedDataType2     | Объект UserDefinedType                                                                             |
| UserDefinedFunction                               | Объекты UserDefinedFunction<br>и UserDefinedFunctionParameter                                      |
| View и View2                                      | Объект View                                                                                        |

## Новейшие функциональные возможности SMO

Для учета новых функций SQL Server 2005 и 2008 в SMO включены следующие функциональные возможности:

- ☐ сбор и оформление в пакеты выполняемых инструкций T-SQL. Инструкции могут собираться и отправляться пакетами (для улучшения производительности работы по сети);
- ☐ управление службами SQL при помощи поставщика WMI. Это означает, что службы SQL можно запускать, останавливать и приостанавливать программным путем;
- ☐ расширенные возможности скриптов. Можно сгенерировать скрипты T-SQL для воссоздания тех объектов SQL Server, которые описывают связи с другими объектами экземпляра SQL Server;
- ☐ передача данных использует службы интерпретации SQL Server Integration Services (SSIS);
- ☐ для разрешения имен удаленных серверов используются уникальные имена ресурсов (unique resource names, URN);
- ☐ имеет богатую модель обработки событий.

SMO имеет также несколько новых объектов и свойств, которые соответствуют новым компонентам SQL Server 2005 и 2008, в том числе:

- ☐ поддержка изоляции моментальных снимков и управления версиями на уровне строк (для повышения параллелизма);
- ☐ секционирование таблиц и индексов для хранения данных с разбиением на секции;
- ☐ конечные точки Hypertext Transfer Protocol (HTTP) для управления запросами Web-служб;
- ☐ полная поддержка технологий XML, в том числе: типа данных XML, коллекций XML-схем и индексов XML;
- ☐ способность создавать копии баз данных "только для чтения" (при помощи баз данных моментальных снимков);
- ☐ полная поддержка брокера SQL Server Service Broker;
- ☐ облегчение управления Database Mail;
- ☐ программное управление зарегистрированными серверами;
- ☐ программное воспроизведение событий SQL Server и работа с SQL Trace;
- ☐ поддержка триггеров DDL.

## Работа с SMO в Microsoft Visual Studio

Вы должны сначала сделать ссылку на сборку SMO в проекте Visual Studio. Для этого необходимо выбрать пункт **Project | Add Reference**, а затем выделить компоненты `Microsoft.SqlServer.Smo`, `Microsoft.SqlServer.SmoExtended`, `Microsoft.SqlServer.Management.Sdk.Sfc`, `Microsoft.SqlServer.ConnectionInfo` (как показано на рис. 4.1).

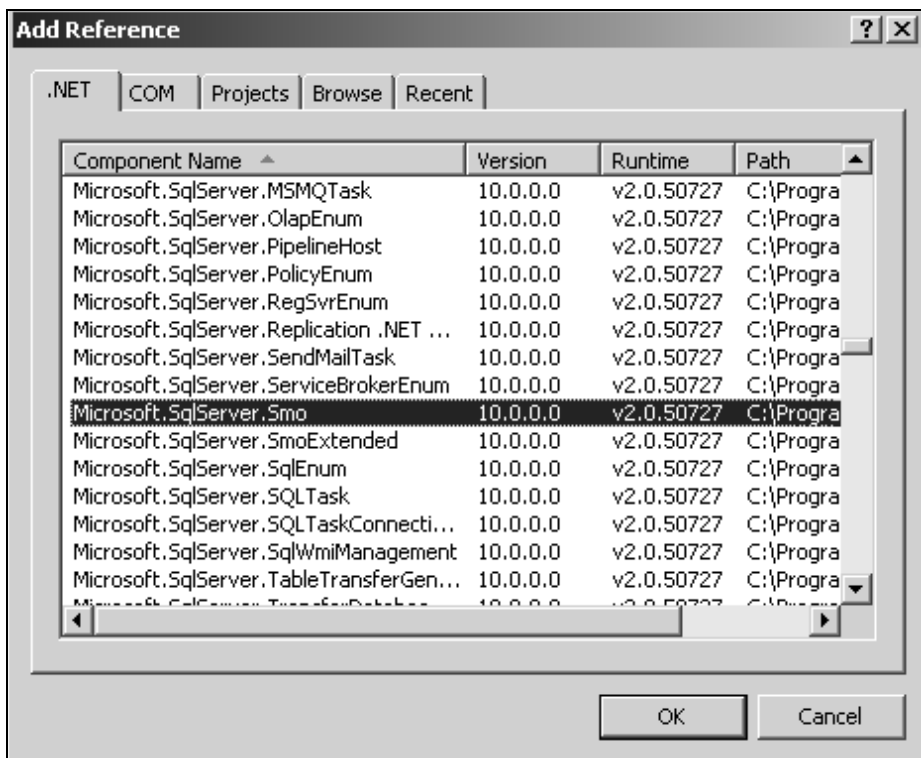


Рис. 4.1. Настройка ссылки на SMO

Затем вы добавляете две инструкции `using` для двух пространств имен, которые вы будете использовать чаще всего:

```
using Microsoft.SqlServer.Management.Common;
using Microsoft.SqlServer.Management.Smo;
```

Объект `ServerConnection` представляет собой подключение к экземпляру SQL Server. Он позволяет вам подключиться к серверу и присвоить это под-

ключению любому объекту `Server` в SMO. Объект `Server` представляет собой фактический экземпляр SQL Server. Этот объект позволяет вам получать свойства сервера базы данных и выполнять такие задачи, как подключение к базе данных. Мы создадим простое приложение для подключения к экземпляру SQL Server с использованием объекта `ServerConnection`, а затем используем объект `Server` для получения версии SQL Server, его редакции и названия экземпляра. Это приложение запросит у пользователя его идентификатор и пароль (с целью использования интегрированной системы безопасности), как это показано на рис. 4.2.

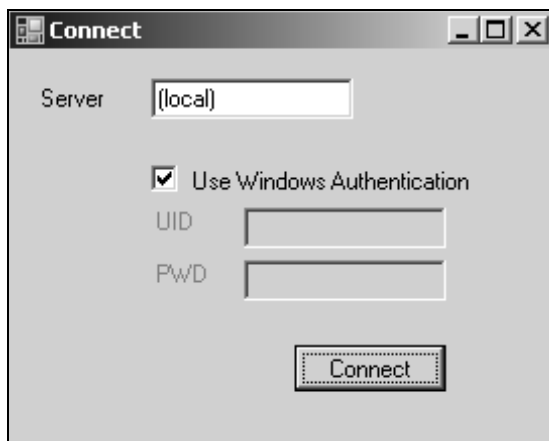


Рис. 4.2. Простое приложение SMO

Код листинга 4.1 показывает, как подключиться к экземпляру SQL Server с использованием интегрированной системы безопасности или системы безопасности SQL Server. Результаты показаны на рис. 4.3.

#### Листинг 4.1. Подключение к базе данных SQL Server при помощи SMO

```
private void btnConnect_Click(object sender, EventArgs e)
{
    // Для подключения используется объект ServerConnection
    ServerConnection conn = new ServerConnection();
    // передать имя server/instance
    conn.ServerInstance = this.txtServer.Text;
    // определить, настроена ли интегрированная безопасность
    if (!chkSSPI.Checked)
```

```
{
    // использование безопасности sql
    conn.LoginSecure = false;
    conn.Login = this.txtUID.Text;
    conn.Password = this.txtPwd.Text;
}
// пробуем подключиться и вернуть информацию о сервере
try
{
    Server svrLocal = new Server(conn);
    // здесь отображаем сообщение пользователю
    // получить информацию при помощи свойства Information
    MessageBox.Show(
        "You are now connected to: " + svrLocal.Name +
        System.Environment.NewLine + " Edition Info: " +
        svrLocal.Information.Edition + System.Environment.NewLine +
        " Version Info: " + svrLocal.Information.Version, "SMO Demos");
}
catch (SmoException exSMO)
{
    // перехватить специфичные для SMO исключения
    MessageBox.Show(exSMO.ToString());
}
catch (Exception ex)
{
    MessageBox.Show(ex.ToString());
}
}
```



**Рис. 4.3.** Результаты простого приложения SMO



**СОВЕТ**

Когда вы ссылаетесь на локальный сервер, который имеет имя экземпляра, вы можете использовать более гибкий синтаксис (local)\instance\_name (вместо server\_name\instance\_name).

## Перебор доступных серверов

Чтобы программно получить список доступных экземпляров SQL Server или перебрать их, вы можете использовать метод `EnumAvailableSqlServers` объекта `SmoApplication`, передав булево значение, которое укажет SMO, нужно ли ограничиться только локальным поиском. Метод `EnumAvailableSqlServers` возвращает `DataTable` и результаты перечисления (с множеством свойств). Следующий код вызывает многократно используемый метод `ListAllKnownInstances` вспомогательного класса. Этот метод делает поиск по сети всех экземпляров SQL Server, которые он может "увидеть" при помощи `EnumAvailableSqlServers`, и возвращает результат `DataTable`. Наш код затем берет `DataTable` и привязывает ее к `DataGridView`, как показано здесь:

```
private void button1_Click(object sender, EventArgs e)
{
    DataTable dtList = SMOUtilities.ListAllKnownInstances();
    dataGridView1.DataSource = dtList;
}
```

Листинг 4.2 делает вызов метода `EnumAvailableSqlServers` и возвращает результаты в `DataTable`.

**ПРЕДУПРЕЖДЕНИЕ**

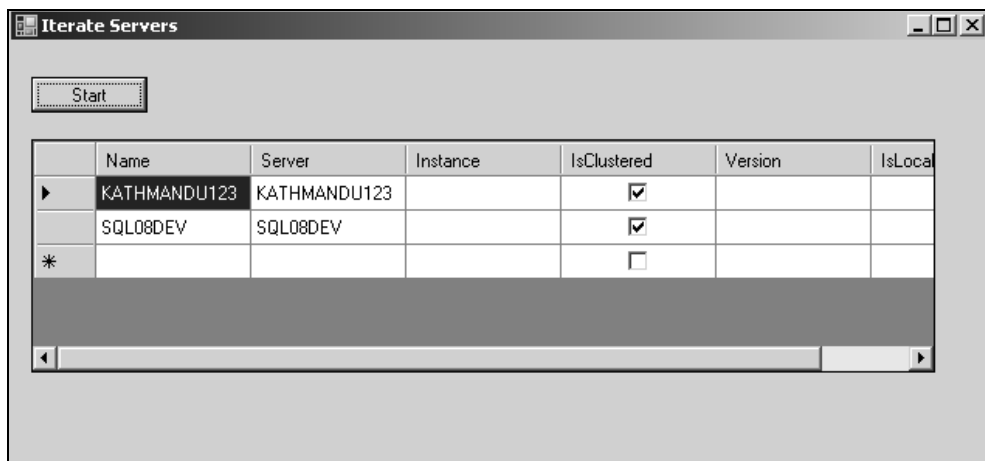
Учтите, что выполнение кода листинга 4.2 будет иногда занимать много времени. Компания Microsoft использует похожий код в мастере SQL Server Import And Export Wizard. В руководстве Books Online написано, что компьютер, на котором работает экземпляр SQL Server, может не получить своевременных откликов на метод `EnumAvailableSqlServers`. Возвращенный список может показать не все доступные на сети экземпляры SQL Server. В последующие вызовы на сети могут стать видимыми дополнительные серверы.

**Листинг 4.2. Перечисление доступных серверов**

```
public static DataTable ListAllKnownInstances()
{
    // этот метод вернет DataTable по всем доступным серверам
```

```
// этот метод может занять определенное время (в зависимости от сети)
// Может подключаться и к компьютерам SQL Server 2000
DataTable dtServers = new DataTable();
try
{
    // Получить список всех доступных экземпляров SQL Server
    // этот метод возвращает DataTable
    dtServers = SmoApplication.EnumAvailableSqlServers(false);
}
catch (SmoException exSMO)
{
    MessageBox.Show(exSMO.ToString());
}
catch (Exception ex)
{
    MessageBox.Show(ex.ToString());
}
return dtServers;
}
```

На рис. 4.4 показаны результаты выполнения кода листинга 4.2, когда заполненная информацией о сервере `DataTable` привязывается к `DataGridView`.



**Рис. 4.4.** Простое перечисление доступных серверов

## Получение настроек сервера

Часто встречается задача получения настроек сервера (конкретного экземпляра SQL Server). Возможно вам придется создать Windows-приложение подобно показанному на рис. 4.5, либо создать Web-приложение для получения настроек сервера, чтобы ваши пользователи могли модифицировать эти настройки. Вы можете выбрать те настройки, которые пользователям будет разрешено изменять (просто не показывая в приложении остальные настройки).

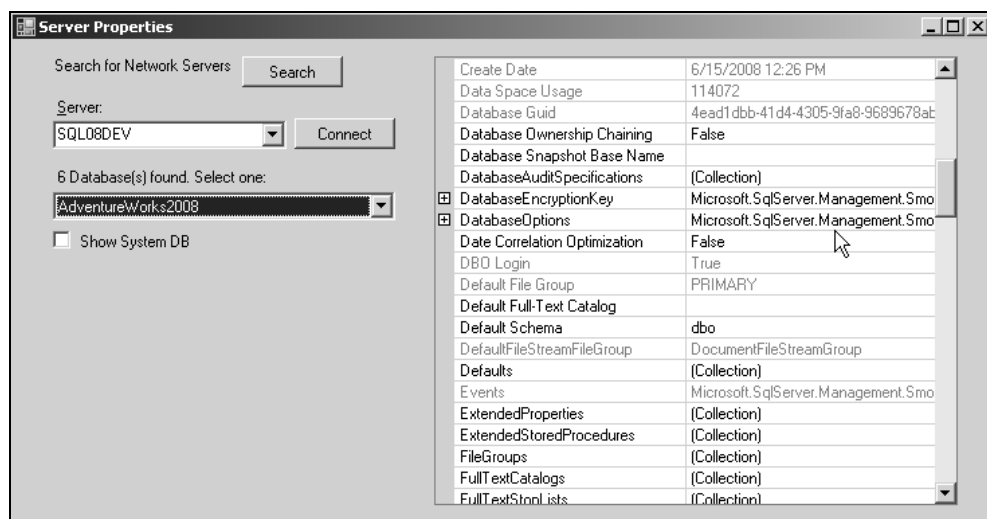


Рис. 4.5. Получение настроек сервера

Несмотря на то, что приложение на рис. 4.5 выглядит сложным, оно простое и основано на том, что вы изучили до настоящего момента. Оно позволяет вам перебрать все доступные серверы, выбрать сервер, а затем выбрать базу данных и отобразить настройки сервера.

Мы можем перечислить все доступные нашему приложению серверы при помощи повторного использования метода `ListAllKnownInstances` (который является частью показанного ранее в листинге 4.2 кода). `ListAllKnownInstances` возвращает `DataTable` со всеми серверами, которые доступны и видны вам. Как показано в листинге 4.3, мы перебираем затем коллекцию `DataRow` для каждой строки в `DataTable`, проверяя, что значение ненулевое и непустое, и добавляя его в комбинированный список с названи-

ем `cboServers` (при помощи метода `Items.Add`). И наконец, после заполнения комбинированного списка всеми доступными серверами мы можем выбрать в качестве значения по умолчанию (в этом комбинированном списке) имя локального экземпляра SQL Server (при помощи свойства `Server.Name`).

#### **Листинг 4.3. Перечисление доступных серверов и заполнение комбинированного списка**

```
private void btnSearch_Click(object sender, EventArgs e)
{
    this.Cursor = Cursors.WaitCursor;
    // получить DataTable с серверами
    DataTable dtList = SMOUtilities.ListAllKnownInstances();
    foreach (DataRow dr in dtList.Rows)
    {
        string serverName = dr["Server"].ToString();
        if (!string.IsNullOrEmpty(dr["Instance"].ToString()))
        {
            serverName += @"\" + dr["Instance"].ToString();
        }
        if (this.cboServers.Items.IndexOf(serverName) < 0)
        {
            this.cboServers.Items.Add(serverName);
        }
    }
    // По умолчанию выбрать локальный сервер
    Server localServer = new Server();
    string localServerName = localServer.Name;
    if (!string.IsNullOrEmpty(localServer.InstanceName))
    {
        localServerName += @"\" + localServer.InstanceName;
    }
    int cboIndex = this.cboServers.FindStringExact(localServerName);
    if (cboIndex >= 0)
    {
        this.cboServers.SelectedIndex = cboIndex;
    }
    this.Cursor = Cursors.Default;
}
```

Затем нам необходимо подключиться к выбранному серверу. Мы можем сделать это при помощи вызова пользовательского метода `ConnectSMO`, который был кратко описан в листинге 4.4. Вы вызываете этот метод путем передачи названия `SQL Server`, к которому вы хотите подключиться, и булева значения, хотите ли вы включать в возвращаемый список доступных баз данных и системные базы данных тоже. Форма приложения имеет флажок, который указывает, хотите ли вы включать системные базы данных. Мы можем вызвать `ConnectSMO` следующим образом:

```
private void btnConnect_Click(object sender, EventArgs e)
{
    this.ConnectSMO(this.cboServers.Text, this.chkSystemDB.Checked);
}
```

Метод `ConnectSMO` отвечает за добавление всех баз данных в комбинированный список `cboDatabase`. Сначала он вызывает универсальный метод с названием `FillComboWithDatabases` в классе `SMOUtilities`. Листинг 4.4 показывает метод `ConnectSMO`, который вызывает `FillComboWithDatabases`. После вызова метода `FillComboWithDatabases` `ConnectSMO` очищает пользовательский интерфейс и возвращает управление пользователю.

#### Листинг 4.4. Пользовательский метод `ConnectSMO`

```
private void ConnectSMO(string serverName, bool includeSystemDB)
{
    try
    {
        // использовать курсор в виде песочных часов
        this.Cursor = Cursors.WaitCursor;
        // сбросить элементы, если вы этого не сделаете, то комбинированный
        // список будет иметь дубликаты после щелчка по флажку показа
        // системных баз данных
        this.cboDatabase.Items.Clear();
        // это многократно используемая функция, которая заполняет
        // комбинированный список доступными базами данных на данном сервере,
        // передавая комбинированный список по ссылке
        // мы будем пользоваться этим в последующих примерах
        SMOUtilities.FillComboWithDatabases
            (serverName, includeSystemDB, ref this.cboDatabase);
        // после обработки комбинированного списка поработаем с UI
        if (this.cboDatabase.Items.Count > 0)
```

```
{
    this.lblDatabase.Text =
        this.cboDatabase.Items.Count.ToString() + " Database(s) found.
        Select one: "; this.cboDatabase.Enabled = true;
}
else
{
    this.lblDatabase.Text = "No Results";
}
this.chkSystemDB.Enabled = true;
}
catch (SmoException exSMO)
{
    MessageBox.Show(exSMO.Message.ToString());
    // очистить текстовые поля формы и комбинированный список
    this.SetFormErrorParms();
}
catch (Exception ex)
{
    MessageBox.Show(ex.ToString());
    this.SetFormErrorParms();
}
finally
{
    this.Cursor = Cursors.Default;
}
}
```

Показанный в листинге 4.5 метод `FillComboWithDatabases` принимает объект `ComboBox`, а также название сервера и флаг (следует ли включать системные базы данных). Метод `FillComboWithDatabases` получает ссылку на экземпляр SQL Server, который вам нужен (при помощи объекта `Server`, последующего перебора коллекции `Databases` и добавления каждой базы данных в комбинированное окно `cboDatabase`).

**Листинг 4.5. Заполнение комбинированного окна базами данных при помощи пользовательского метода `FillComboWithDatabases`**

```
public static void FillComboWithDatabases
(string serverName, bool includeSystemDB, ref ComboBox cbo)
```

```

{
    // Заполнить комбинированное окно базами данных с выбранного сервера
    Server svr = new Server(serverName);
    foreach (Database db in svr.Databases)
    {
        // определить, установлен ли флажок системных баз данных
        // и соответственно включить/исключить их
        if ((includeSystemDB) || (!db.IsSystemObject))
        {
            cbo.Items.Add(db.Name);
        }
    }
}

```

Теперь, когда мы выбрали наш экземпляр SQL Server, пришло время выбрать базу данных, а затем показать все ее свойства. В этом приложении мы привяжем свойства базы данных к объекту `PropertyGrid` из `WindowsForms`. Мы сделаем это в событии `SelectedIndexChanged` комбинированного окна, которое показано в листинге 4.6. Если вы хотите показать только определенные свойства, то можете перебрать коллекцию свойств и показать только те свойства, которые хотите дать увидеть пользователю.

#### Листинг 4.6. Реакция на событие `SelectedIndexChanged`

```

private void cboDatabase_SelectedIndexChanged(object sender, EventArgs e)
{
    try
    {
        // Эта функция получает свойства указанной базы данных
        this.Cursor = Cursors.WaitCursor;
        Server selectedServer = new Server(this.cboServers.Text);
        Database selectedDatabase =
            selectedServer.Databases[this.cboDatabase.Text];
        this.propertyGrid1.SelectedObject = selectedDatabase;
    }
    catch (SmoException exSMO)
    {
        this.DBPropLabel.Text = string.Empty;
        MessageBox.Show(exSMO.ToString());
    }
}

```

```
catch (Exception ex)
{
    this.DBPropLabel.Text = string.Empty;
    MessageBox.Show(ex.ToString());
}
finally
{
    this.Cursor = Cursors.Default;
}
}
```

## Создание приложений резервного копирования и восстановления

Самое часто встречающееся приложение, которое разработчики создавали при помощи SQL-DMO за эти годы — это приложение резервного копирования и восстановления.

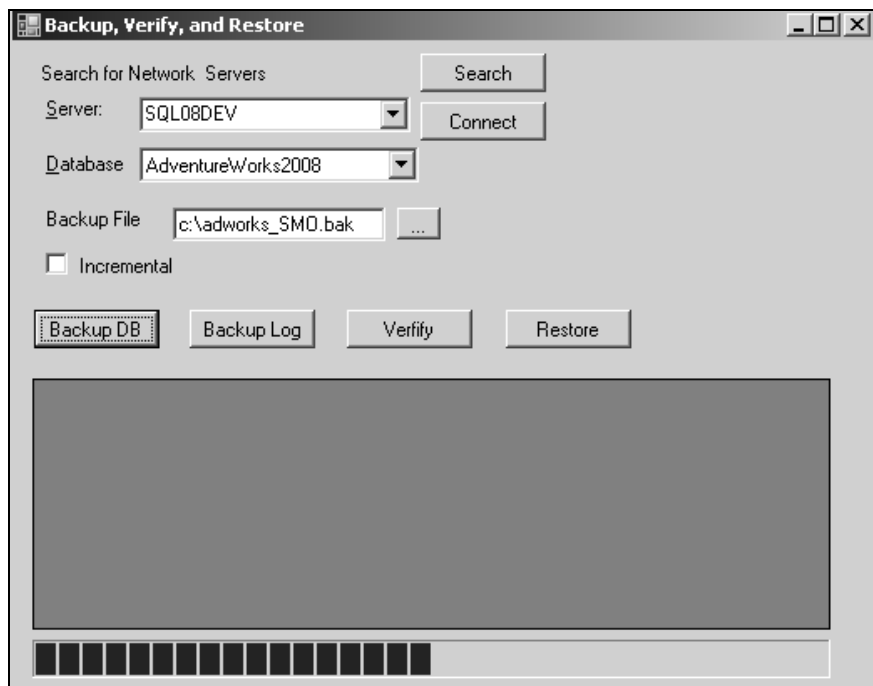


Рис. 4.6. Приложение резервного копирования и восстановления



Иногда это делалось для тех клиентов, у которых не было доступа к базе данных и к Management Studio, а иногда для клиентов, которым было необходимо удаленное управление и контроль. Такие приложения резервного копирования и восстановления в итоге привели к созданию инструментов мониторинга. SMO облегчает создание приложений для резервного копирования, проверки и восстановления. Для демонстрации этих возможностей мы проработаем простое приложение резервного копирования и восстановления, созданное полностью в SMO (как показано на рис. 4.6).

## Резервное копирование

Это приложение позволяет вам искать серверы (повторно используется код из предыдущих примеров). Для получения списка серверов вы будете повторно использовать наш метод `ListAllKnownInstances`, а для списка баз данных сервера — метод `FillComboWithDatabases`. Затем приложение позволит вам выбрать файл резервного копирования, например, `c:\adworks_SMO.bak`. Приложение, которое вы видите на рис. 4.6, совершенно явно является приложением Windows Forms, имеющим в пользовательском интерфейсе кнопку для запуска резервного копирования пользователем. Вы можете также взять код этого приложения и автоматизировать резервное копирование путем создания консольного приложения, приложения WPF или даже Web-страницы ASP.NET.

После выбора сервера и базы данных можно выполнять код резервного копирования. Этот код создает объект `Server` и объект `Backup`, а затем присваивает `BackupActionType.Database` (как выполняемое действие резервного копирования) свойству `Action`. (Через некоторое время мы рассмотрим действия с файлами журнала). Вам необходимо добавить устройство, на которое вы будете производить резервное копирование (обычно это файл, но вы можете также копировать и на ленту, и на другие стандартные устройства SQL). Вы можете использовать булев флаг для указания, делать ли только инкрементальное резервное копирование (листинг 4.7). Сама операция резервного копирования выполняется методом `Backup.SqlBackup`.

### Листинг 4.7. Выполнение резервного копирования программным путем при помощи SMO

```
private void btnBackup_Click(object sender, EventArgs e)
{
    // использовать сервер, выбранный в комбинированном окне Server
    // в комбинированном окне можно ввести и этот сервер
```

```
Server svr = new Server(this.cboServers.Text);
Backup bkp = new Backup();
this.Cursor = Cursors.WaitCursor;
this.dataGridView1.DataSource = string.Empty;
// сделать попытку резервного копирования и показать индикатор
// хода выполнения. Здесь многое может пойти не так (от доступа
// к файлам и до проблем с SQL), так что очень важно обработать
// все ошибки, которые могут возникнуть.
try
{
    string fileName = this.txtFileName.Text;
    string databaseName = this.cboDatabase.Text;
    bkp.Action = BackupActionType.Database;
    bkp.Database = databaseName;
    // настроить устройство: File, Tape, и так далее
    bkp.Devices.AddDevice(fileName, DeviceType.File);
    // указать для случая инкрементального резервного копирования
    bkp.Incremental = chkIncremental.Checked;
    // индикатор хода выполнения
    this.progressBar1.Value = 0;
    this.progressBar1.Maximum = 100;
    this.progressBar1.Value = 10;
    // установить индикатор выполнения в 10% по умолчанию
    bkp.PercentCompleteNotification = 10;
    // вызов обработчика события для увеличения значения
    // индикатора хода выполнения
    bkp.PercentComplete +=
        new PercentCompleteEventHandler(ProgressEventHandler);
    // здесь делается резервное копирование
    bkp.SqlBackup(svr);
    // сообщить пользователю, когда все сделано
    MessageBox.Show("Database Backed Up To: " + fileName, "SMO Demos");
}
catch (SmoException exSMO)
{
    MessageBox.Show(exSMO.ToString());
}
catch (Exception ex)
{

```

```
        MessageBox.Show(ex.ToString());  
    }  
    finally  
    {  
        this.Cursor = Cursors.Default;  
        this.progressBar1.Value = 0;  
    }  
}
```

## Индикатор хода выполнения

Внизу экрана предыдущего примера индикатор хода выполнения показывает продвижение операции резервного копирования. Реализовать его очень легко. Следующий код показывает обработчик события для резервного копирования с параметром `PercentCompleteEventArgs`, который содержит процент выполненного копирования в виде целого числа от 0 до 100. Мы настраиваем значение свойства `Value` индикатора хода выполнения в это значение.

```
public void ProgressEventHandler  
    (object sender, PercentCompleteEventArgs e)  
{  
    // увеличить полосу индикатора хода выполнения на проценты  
    this.progressBar1.Value = e.Percent;  
}
```

Для обработки событий хода выполнения просто присвойте событию `PercentComplete` обработчик события (как уже было показано ранее в листинге 4.7). Вы можете настроить `PercentCompleteNotification` в любое желаемое для вас значение. Это начальное значение вашего индикатора хода выполнения; поэтому 10 означает, что заполнено 10% индикатора хода выполнения.

### СОВЕТ

Для больших баз данных 10% резервного копирования может занять много времени. Использование меньшего значения приведет к тому, что индикатор хода выполнения будет обновляться чаще.

## Резервное копирование журнала

Вы делаете резервное копирование файлов журнала, не так ли? Резервное копирование файла журнала требует точно такого же кода, какой содержится

в листинге 4.3, за исключением того, что вы устанавливаете свойство `Action` в значение `Log` (вместо `Database`):

```
bkp.Action = BackupActionType.Log;
```

## Проверка

После выполнения резервного копирования будет весьма благоразумно проверить как правильность его выполнения, так и то, что устройство резервного копирования исправно. (Можно, конечно, проверить исправность устройства и до выполнения резервного копирования). Вы можете сделать это при помощи использования метода `SqlVerify` объекта `Restore` (листинг 4.8).

### Листинг 4.8. Проверка резервной копии при помощи SMO

```
private void btnVerify_Click(object sender, EventArgs e)
{
    Server svr = new Server();
    Restore rest = new Restore();
    string fileName = this.txtFileName.Text;
    this.Cursor = Cursors.WaitCursor;
    this.dataGridView1.DataSource = string.Empty;
    try
    {
        rest.Devices.AddDevice(fileName, DeviceType.File);
        bool verifySuccessful = rest.SqlVerify(svr);
        if (verifySuccessful)
        {
            MessageBox.Show("Backup Verified!", "SMO Demos");
            DataTable dt = rest.ReadFileList(svr);
            this.dataGridView1.DataSource = dt;
        }
        else
        {
            MessageBox.Show("Backup NOT Verified!", "SMO Demos");
        }
    }
    catch (SmoException exSMO)
    {
        MessageBox.Show(exSMO.ToString());
    }
}
```

```
}  
catch (Exception ex)  
{  
    MessageBox.Show(ex.ToString());  
}  
finally  
{  
    this.Cursor = Cursors.Default;  
}  
}
```

SMO возвращает вам таблицу `DataTable` со всеми свойствами резервной копии, которые вы легко можете привязать к `DataGrid` (рис. 4.7). Это очень полезно в том случае, если вы хотите создать Web-страницу результатов резервного копирования (и просмотреть даже при помощи своего сотового телефона).

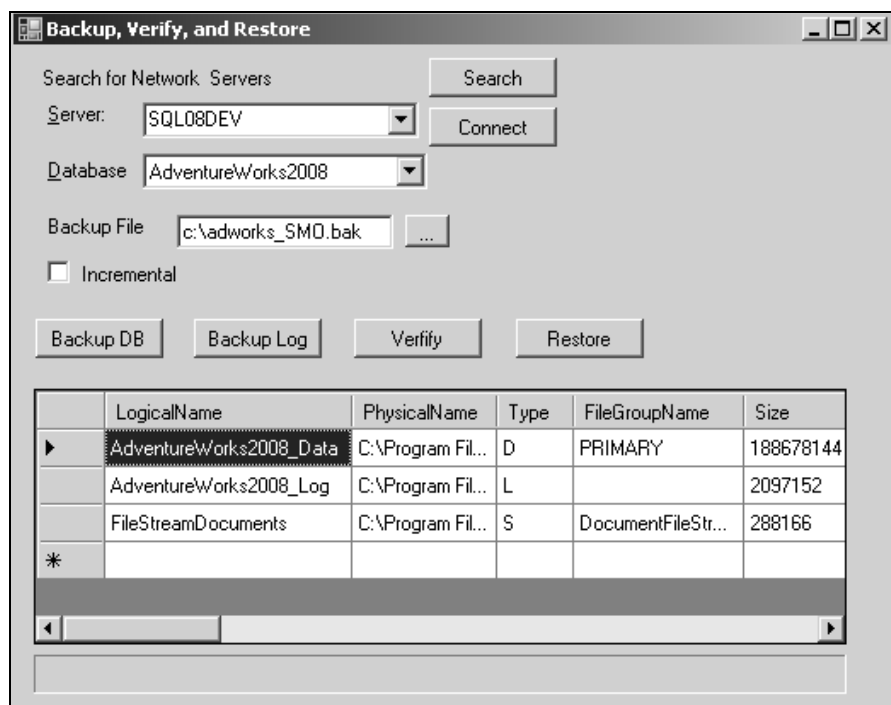


Рис. 4.7. Проверка резервного копирования

## Восстановление

Какая польза от резервного копирования без восстановления? Как показано в листинге 4.9, операцию восстановления вы выполняете посредством создания экземпляров объекта `Server` и объекта `Restore` и предоставления названия файла резервной копии (или устройства), а также имени той базы данных, куда будет происходить восстановление. Для восстановления базы данных установите свойство `Action` объекта `Restore` в значение `RestoreActionType.Log`. Обратите внимание, что мы используем для индикатора хода выполнения тот же самый обработчик события, что и в примере резервного копирования (максимизируя тем самым повторное использование кода).

### Листинг 4.9. Выполнение восстановления при помощи SMO

```
private void btnRestore_Click(object sender, EventArgs e)
{
    Server svr = new Server(this.cboServers.Text);
    Restore res = new Restore();
    this.Cursor = Cursors.WaitCursor;
    this.dataGridView1.DataSource = string.Empty;
    try
    {
        string fileName = this.txtFileName.Text;
        string databaseName = this.cboDatabase.SelectedValue.ToString();
        res.Database = databaseName;
        res.Action = RestoreActionType.Database;
        // использовать для восстановления журнала
        // res.Action = RestoreActionType.Log;
        res.Devices.AddDevice(fileName, DeviceType.File);

        // индикатор хода выполнения
        this.progressBar1.Value = 0;
        this.progressBar1.Maximum = 100;
        this.progressBar1.Value = 10;
        res.PercentCompleteNotification = 10;
        res.ReplaceDatabase = true;
        res.PercentComplete +=
            new PercentCompleteEventHandler(ProgressEventHandler);
    }
}
```

```
// для перемещения файлов mdf использовать следующее:
// res.RelocateFiles.Add(new RelocateFile("aw_data",
//     @"c:\aw_dat.mdf"));

// выполнить восстановление
res.SqlRestore(svr);
MessageBox.Show("Restore of " + databaseName
    + " Complete!", "SMO Demos");
}
catch (SmoException exSMO)
{
    MessageBox.Show(exSMO.ToString());
}
catch (Exception ex)
{
    MessageBox.Show(ex.ToString());
}
finally
{
    this.Cursor = Cursors.Default;
    this.progressBar1.Value = 0;
}
}
```

## Восстановление в другое местоположение

Резервное копирование и восстановление SMO позволяют, например, изменить местоположение файла при помощи метода `RelocateFiles.Add` объекта `Restore`, как показано здесь:

```
res.RelocateFiles.Add
    (new RelocateFile("adventureworks1_data",
        @"c:\adventureworks1_dat.mdf"));
```

## Выполнение программных функций DBCC при помощи SMO

Еще одна полезная функциональная возможность SMO — это проверки целостности базы данных. При помощи SMO вы можете выполнить почти все

функции Database Consistency Check (DBCC) (табл. 4.3). В дополнение к проверкам целостности DBCC может делать много административных проверок.

**Таблица 4.3.** Проверки целостности при помощи SMO

| Метод SMO                | Функция DBCC                             |
|--------------------------|------------------------------------------|
| CheckAllocations         | DBCC CHECKALLOC                          |
| CheckAllocationsDataOnly | DBCC CHECKALLOC('databasename', NOINDEX) |
| CheckCatalog             | DBCC CHECKCATALOG                        |
| CheckTables              | DBCC CHECKDB                             |
| CheckTablesDataOnly      | DBCC CHECKDB('databasename', NOINDEX)    |

В листинге 4.10 простое приложение использует команды DBCC, показанные на рис. 4.8. Это приложение позволяет вам искать серверы (используется код из предыдущих примеров), а затем делать список всех баз данных на сервере. Для получения списка серверов мы будем повторно использовать наш метод `ListAllKnownInstances`, а для получения списка всех баз данных на сервере — метод `FillComboWithDatabases`. Код примера использует метод `CheckCatalog` из SMO (который является командой DBCC); однако с небольшими изменениями вы сможете использовать с похожим кодом любую из перечисленных ранее в табл. 4.3 команд DBCC.

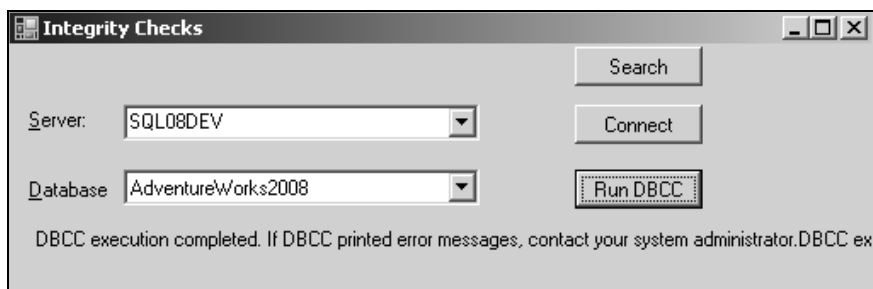
**Листинг 4.10. Выполнение проверки целостности базы данных (DBCC) при помощи SMO**

```
private void btnRunDBCC (object sender, EventArgs e)
{
    Server svr = new Server(this.cboServers.Text);
    Database db = svr.Databases[this.cboDatabase.Text];
    StringCollection results = db.CheckCatalog(); // может выполнять
    // большинство команд DBCC
    // очистить текстовое поле
    this.label1.Text = string.Empty;
    foreach (string result in results)
    {
        if (!string.IsNullOrEmpty(result))
        {

```



```
        this.lblResults.Text += result;
    }
}
```



**Рис. 4.8.** Приложение DBCC с использованием SMO

### **ВАЖНО**

В документации Books Online написано, что `CheckCatalog` выполняет различные проверки целостности между таблицами системных метаданных. На самом же деле, если DBCC `CHECKCATALOG` сообщает о каких-либо ошибках, то это означает, что кто-то вручную либо добавил данные в системные таблицы, либо отредактировал (или удалил) данные. Мы рекомендуем регулярно выполнять эту проверку для базы данных вашего Web-сайта, поскольку модифицированные системные таблицы являются одним из главных симптомов атаки внедрением кода SQL. Вы можете выполнять этот код каждые несколько часов и получать результаты по электронной почте либо разместить на закрытой административной Web-странице и просматривать в любое время.

Метод `CheckCatalog` (подобно всем проверяющим целостность методам SMO) возвращает объект `StringCollection`, заполненный результатами. Как показывает код листинга 4.10, вы можете перебрать результаты и затем предпринять необходимые действия. Если в вашей базе данных все в порядке (как это было в данном примере), то вы получите из DBCC следующее вездесущее сообщение: "DBCC execution completed. If DBCC printed error messages, contact your system administrator."

### **ПРИМЕЧАНИЕ**

Проблема с выполнением проверки DBCC состоит в том, что если вы не являетесь администратором базы данных SQL Server, то вы, скорее всего, не будете знать, что вам делать с ошибками. Административные функции

DBCC выходят за пределы рассмотрения этой книги, но вы можете посоветоваться насчет возможных корректирующих действий с администратором баз данных (DBA). Если же вы одновременно и разработчик, и DBA, то вам следует подробнее изучить данную область.

Вы можете выполнить множество других функций DBCC. Код будет точно таким же, только иногда вам будет необходимо передать имя таблицы или индекса. Дополнительные примеры находятся в табл. 4.3 или в документации Books Online.

## Управление на основе политик

Очень часто разработчик тратит много времени в догадках, почему его приложение работает должным образом на его компьютере, но не работает на тестовом или производственном сервере. Обычно это вызвано некими отличиями между этими двумя средами. Например, возможно, что разработчик написал новую хранимую процедуру SQL CLR на своем компьютере, где интеграция SQL CLR включена, но она может быть выключена на сервере назначения. При все более увеличивающемся количестве функциональных возможностей в новых версиях SQL Server становится все сложнее сконфигурировать сервер базы данных именно так, как вы хотите. Для того чтобы справиться с этими проблемами, SQL Server 2008 вводит "основанное на политике управление" (Policy-Based Management, PBM) для управления сервером. PBM позволяет разработчику или DBA настроить желаемую конфигурацию сервера (то есть политику), а затем проверить конфигурацию этой системы на соответствие этой политике. PBM также обеспечивает выполнение правил, которые вы создаете, например, такого: "все новые таблицы должны находиться либо в схеме `ssn`, либо в схеме `prw`". Кроме того, PBM позволяет разработчику или DBA обеспечить реализацию этой политики на удаленных серверах.

Несмотря на то, что PBM — это в основном функция для администратора баз данных, мы будем рассматривать ее с точки зрения разработчика, который старается обеспечить выполнение политик в сценарии развертывания. При помощи PBM разработчик может обеспечить полное соответствие тестового (или производственного) сервера его собственному компьютеру (который он использует для разработки). Скорее всего, разработчик обратится за помощью к администратору, чтобы обеспечить соответствие своего рабочего компьютера тем компьютерам, которые используются для тестирования и в производственных условиях.

Перед тем как начать простую демонстрацию, давайте мы сделаем обзор основной терминологии PBM:

- ❑ Тип назначения (*target type*) — объекты SQL Server, которые управляются при помощи PBM — например, база данных, индекс, хранимая процедура и схема.
- ❑ Аспект управления (*management facet*) — логический набор предопределенных свойств, которые моделируют поведение или конфигурацию для ваших типов назначения. Существует множество аспектов, которые определяют поведение SQL Server.
- ❑ Условие (*condition*) — желаемое состояние или поведение аспекта управления (устанавливается при помощи булева выражения).
- ❑ Политика (*policy*) — ожидаемое состояние или поведение данного условия.

## Простая политика

Теперь давайте рассмотрим простой пример для демонстрации работы PBM. Нам нужна политика, которая говорит о том, что для базы данных должна быть включена SQL CLR. Затем мы хотим обеспечить выполнение этой политики при помощи PBM. Для этого мы создадим нашу собственную политику с названием *myCLRPolicy*. Эта политика обеспечит, что условие типа назначения *ClrIntegrationEnabled* аспекта *Server Configuration* имеет значение *true*. В табл. 4.4 для данного примера показан тип назначения, аспект управления, а также значения условия и политики.

**Таблица 4.4.** Параметры обеспечения политики CLR

| Наша политика         | Значения PBM     |
|-----------------------|------------------|
| MyCLRPolicy           | Policy           |
| True                  | Condition        |
| Server Configuration  | Management facet |
| ClrIntegrationEnabled | Target type      |

В проводнике **Object Explorer** перейдите с узла сервера по узлам **Management | Policy Management** и **Facets** (как показано на рис. 4.9). Там вы увидите список всех аспектов, которые определены в SQL Server 2008.

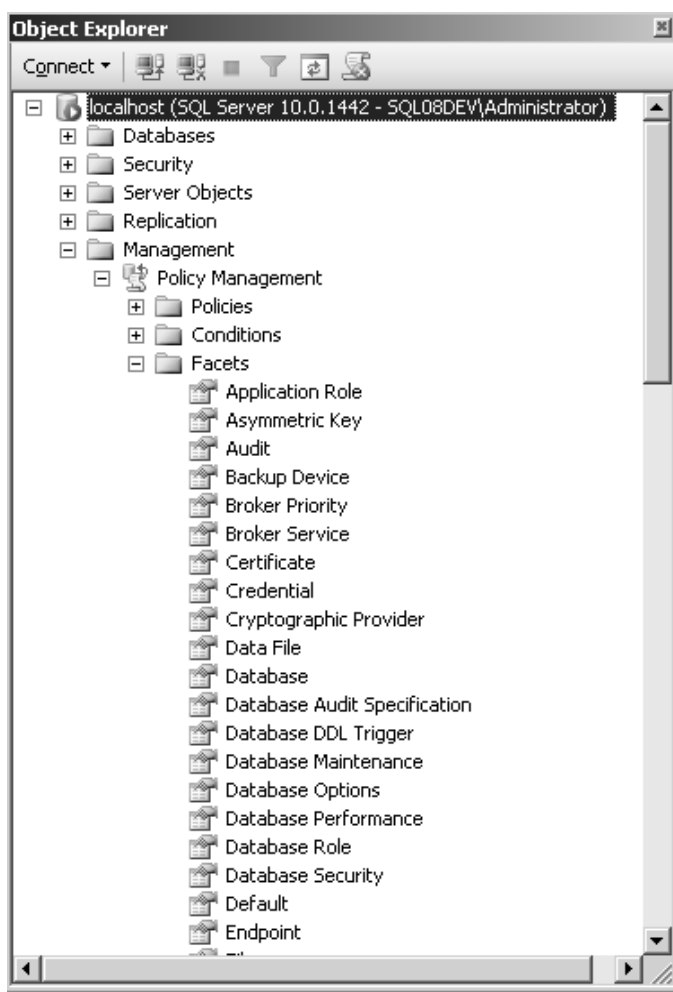
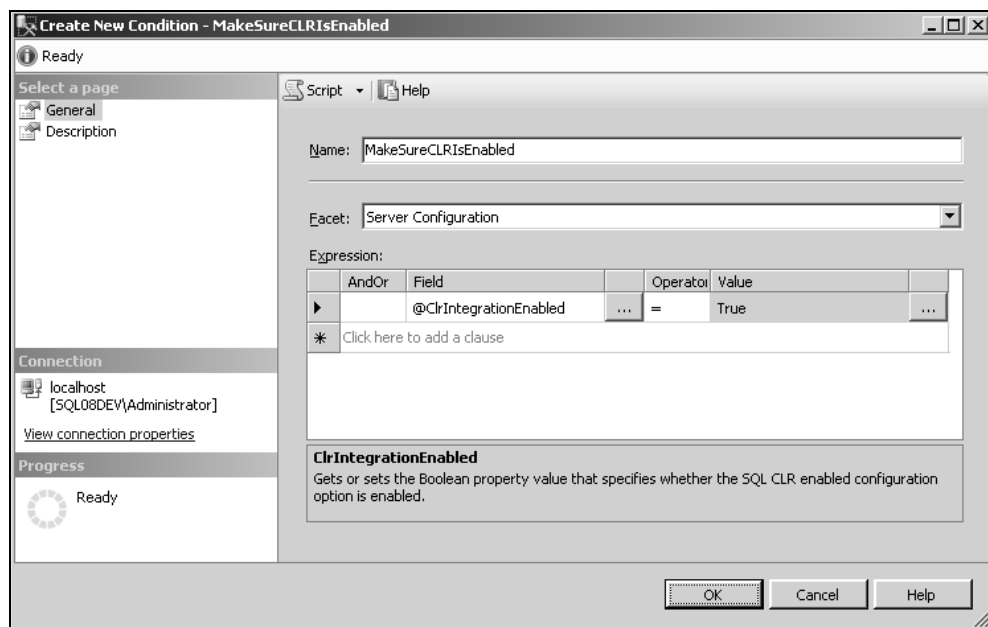


Рис. 4.9. Интерфейс PBM в окне **Object Explorer**

Для того чтобы создать нашу новую политику (которая будет проверять SQL CLR), нам нужно сначала создать новое условие. Для этого щелкните правой кнопкой мыши по узлу **Conditions** (в Policy Management), а затем выберите **New Condition**. После этого появится диалоговое окно **Create New Condition** (рис. 4.10). Дайте условию название — например, `MakeSureClrIsEnabled`. В выпадающем списке **Facet** выберите **Server Configuration**, а затем щелкните в панели Expression на **Click here to add a clause**. После этого вы можете выбрать `@ClrIntegrationEnabled` в поле **Field** и **True** в поле **Value**. После этого нажмите **OK** для создания нового условия.

Рис. 4.10. Диалоговое окно **Create New Condition**

Теперь, когда наше условие полностью настроено, нам необходимо подключить его к политике. Для этого щелкните правой кнопкой мыши по узлу **Policies** в проводнике **Object Explorer** для показа диалогового окна **Create New Policy** (рис. 4.11).

Назовите политику **MyCLRPoicy** и в выпадающем списке **Check Condition** выберите условие **MakeSureCLRIsEnabled** (которое мы только что создали). **Against targets** — это любые удаленные серверы назначения, к которым вы подключены. В данном случае мы собираемся выполнять эту политику локально, поэтому вы можете оставить данное поле пустым. Для режима выполнения у вас есть следующий выбор:

- |                                              |                                                       |
|----------------------------------------------|-------------------------------------------------------|
| <input type="checkbox"/> <b>On Demand;</b>   | <input type="checkbox"/> <b>On Change — Log Only;</b> |
| <input type="checkbox"/> <b>On Schedule;</b> | <input type="checkbox"/> <b>On Change — Prevent.</b>  |

Для нашего примера выберите **On Demand** (это значение по умолчанию) и нажмите **OK** для создания новой политики. Выбрав **On Demand**, вы указали, что политика будет выполняться только тогда, когда вы вручную выполните ее. При выборе **On Shedule** политика будет выполняться при помощи SQL Server Agent в указанное вами время (настраивается в диалоговом окне **Agent**

**Scheduling**). Выбор варианта **On Change — Log Only** выполняет политику тогда, когда изменяется свойство аспекта управления, а нарушения политики направляются в журнал событий Windows. Выбор варианта **On Change — Prevent** будет выполнять политику при изменении свойства в аспекте управления, но не даст выполнить такое изменение (при помощи триггера DDL). Не все изменения можно обнаружить и откатить при помощи RBM, поэтому данный вариант доступен только тогда, когда можно предотвратить такое условие.

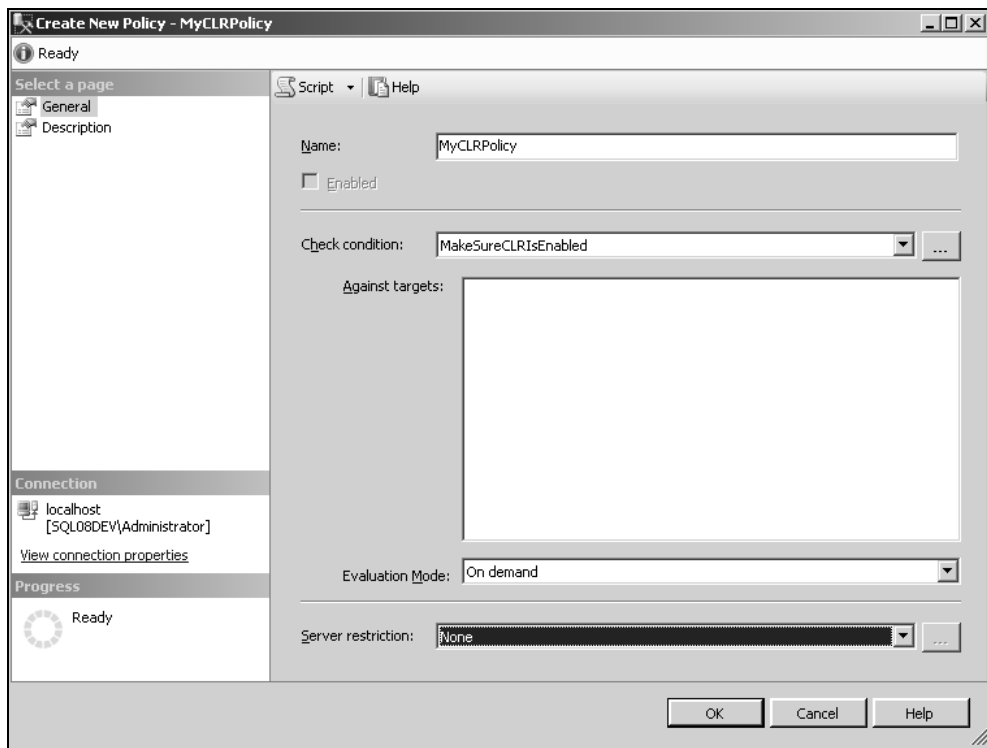


Рис. 4.11. Диалоговое окно **Create New Policy**

Последний шаг — выполнение политики. Для этого щелкните правой кнопкой мыши по вновь созданной политике, а затем выберите в контекстном меню пункт **Evaluate Policies** (как показано на рис. 4.12). SQL Server выполнит политику, оценит условие и получит результаты. Если SQL CLR у вас не включена, то политика завершится с ошибкой (на рис. 4.12 кружок с крестиком в списках).

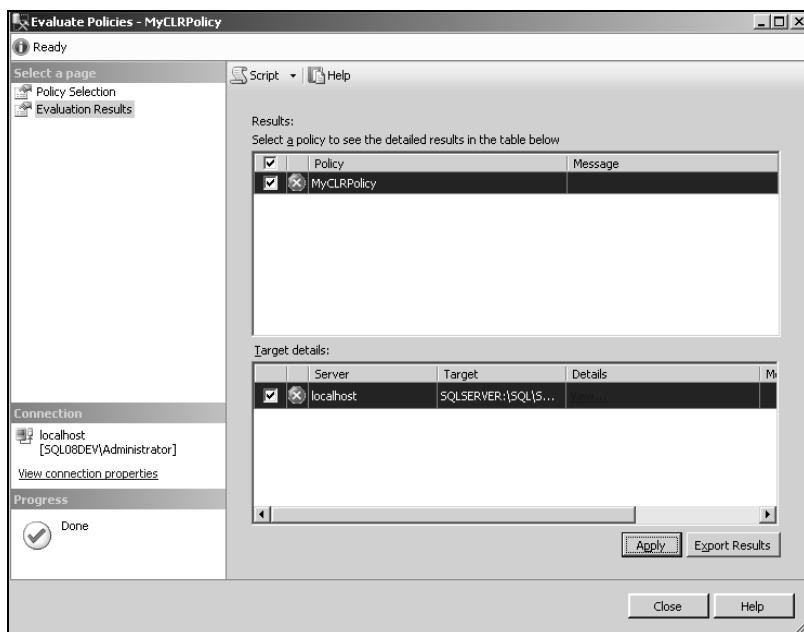


Рис. 4.12. Диалоговое окно **Evaluate Policies** со сбойными результатами

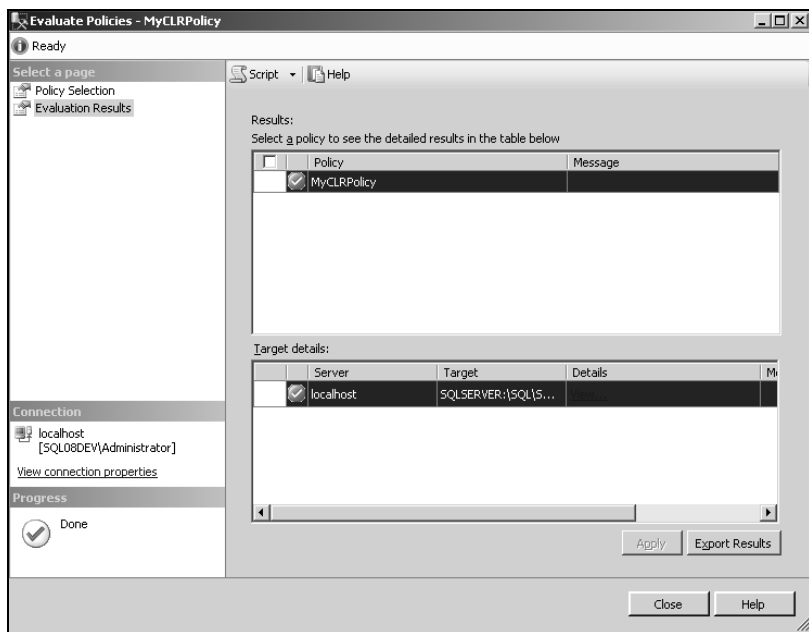


Рис. 4.13. Диалоговое окно **Evaluate Policy**, показывающее успешные результаты после применения политики

Теперь представьте себе, что вы хотите обеспечить эту политику для набора серверов, к которым она относится (смотрите окно **Create New Policy** на рис. 4.11). Обеспечить политики при помощи PBM очень легко. В диалоговом окне **Evaluate Policies** выберите условие, а затем нажмите кнопку **Apply**. Это обеспечит реализацию политики путем выполнения изменений (в нашем случае — при помощи включения CLR). Результаты показаны на рис. 4.13. (Обратите внимание на флажок вместо крестика). Вы можете также экспортировать это условие в виде скрипта T-SQL для выполнения в автономном режиме.

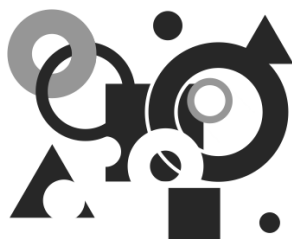
PBM — отличный инструмент для разработчиков и DBA, который используется ими для выравнивания конфигураций сред, в которых они работают.

## Резюме

В этой главе мы лишь поверхностно затронули SMO и PBM, поскольку как SMO, так и PBM имеют огромные возможности и предлагают исключительно богатые программные функции. Вне сомнения, SMO — это прекрасная объектная модель, которая охватывает все, что делает SQL Server, и фактически даже больше того, что охвачено программой SQL Server Management Studio. Как разработчику вам может понадобиться снова и снова выполнять в Management Studio одну и ту же задачу. Если это так, то продумайте возможность написания приложения при помощи SMO. Помните также, что SMO богаче функциональными возможностями и имеет более высокую производительность, чем устаревший SQL-DMO, и поэтому может выполнить многие ваши административные задачи. PBM облегчает настройку идентичной конфигурации вашей рабочей среды и тестовой (или производственной) среды. Это позволяет разработчикам более успешно моделировать тестовую и производственную среду при разработке кода для SQL Server.



## ГЛАВА 5



# Безопасность в SQL Server 2008

*Роб Уолтерс и Леонард Лобел*

С учетом постоянно растущей озабоченности защитой личных данных и распространением компьютерных вирусов разработка методики безопасной обработки данных остается для компании Microsoft жизненно важной задачей.

### Конструктивная безопасность:

- обязательное обучение
- моделирование угроз
- анализ кода и тестирование на проникновение
- автоматизированные инструменты для кода
- Enhanced Security Model

### Безопасность по умолчанию:

- система безопасна в состоянии Default Configuration
- минимальное пространство для атак
- большинство служб SQL настроены как запускаемые вручную
- способность отключать наборы XP

### Безопасность по развертыванию

- автоматическое/неавтоматическое обслуживание программного обеспечения
- инструменты, соответствующие лучшим практикам
- обновления Microsoft Update

### Безопасный обмен:

- возможность написания Secure Code 2.0
- архитектура Web-кастов

**Рис. 5.1.** Инфраструктура безопасности SQL Server 2008

В 2003 г. компания Microsoft создала инициативу Trustworthy Computing, чтобы люди стали чувствовать себя более комфортно при использовании компьютеров и программного обеспечения. (Эту инициативу можно найти по адресу: [http://www.microsoft.com/mscorp/twc/twc\\_whitepaper.mspx](http://www.microsoft.com/mscorp/twc/twc_whitepaper.mspx).) Из этой инициативы Microsoft SQL Server 2008 унаследовал то, что называется Security Framework (инфраструктура безопасности, рис. 5.1).

## **Четыре темы инфраструктуры безопасности**

Безопасность SQL Server 2008 организована по четырем темам: конструктивная безопасность, безопасность по умолчанию, безопасность по развертыванию и безопасный обмен.

### **Конструктивная безопасность**

Безопасность была целью конструирования и для более ранних версий SQL Server. Начиная с версии SQL Server 2005, группа разработчиков продукта достигла общего понимания вопросов безопасности, были написаны и проанализированы модели угроз по всем компонентам продукта. Кроме того, были предприняты серьезные усилия по анализу кода на предмет безопасности всего продукта. Компания Microsoft относится к безопасности очень серьезно, поэтому проектировщики функций SQL Server сделали безопасность высшим приоритетом дизайна продукта.

### **Безопасность по умолчанию**

Идея безопасности по умолчанию является одной из наиболее примечательных в Security Framework. Вы можете познакомиться с ней, если просто устанавливаете SQL Server с установками по умолчанию. Пользователи SQL Server 2000 и более ранних версий заметят, что такие службы, как SQL Server Agent по умолчанию выключены. Кроме того, некоторые функциональные возможности (такие как `xp_cmdshell` и запросы `OPENROWSET`) отключены, чтобы минимизировать возможности для атак, и результаты можно видеть по всему продукту.

## Безопасность по развертыванию

Возможно, одной из наиболее сложных проблем SQL Server является эффективное развертывание в производственной среде. При наличии большого количества конфигураций и функциональных возможностей администраторам сложно отслеживать все новейшие обновления и лучшие практики. SQL Server 2005 и 2008 теперь получают обновления с Microsoft Update.

## Безопасный обмен

Еще до SQL Server 2005 официальные документы, Web-касты, группы новостей обучали и помогали бета-клиентам в освоении продукта. Сегодня большинство этих документов и Web-кастов обновлено и представляет собой богатый материал для изучения.

Все версии SQL Server 2008 имеют функции безопасности, которые помогают пользователям защищать их данные. Эта глава описывает следующие аспекты безопасности SQL Server:

- ☐ обзор безопасности, включая аутентификацию и авторизацию;
- ☐ разделение пользователей и схемы;
- ☐ шифрование данных в базе данных и при передаче;
- ☐ аудит сервера и событий базы данных;
- ☐ защита SQL Server 2008.

### **УМЕНЬШЕНИЕ ПРОСТРАНСТВА ДЛЯ АТАК**

Чем больше дверей в вашем доме, тем больше возможностей имеет злоумышленник для взлома. Даже если эти двери сделаны из сплошной стали и каждая имеет красивый блестящий замок, все равно они считаются пространством, которое подвержено атакам. Одним из очевидных способов ликвидации такой уязвимости является устранение дверей в тех местах, где они не нужны. В компьютерном приложении это означает, что если вы не используете некую функциональную возможность, то ее следует по возможности отключить.

В состоянии поставки SQL Server обеспечивает сокращенное пространство для атак, поскольку у него отключены необязательные функциональные возможности. Это такие функции, как: служба SQL Server Agent, служба SQL Server Browser, `xp_cmdshell`, интеграция с Common Language Runtime (CLR) и т. д.

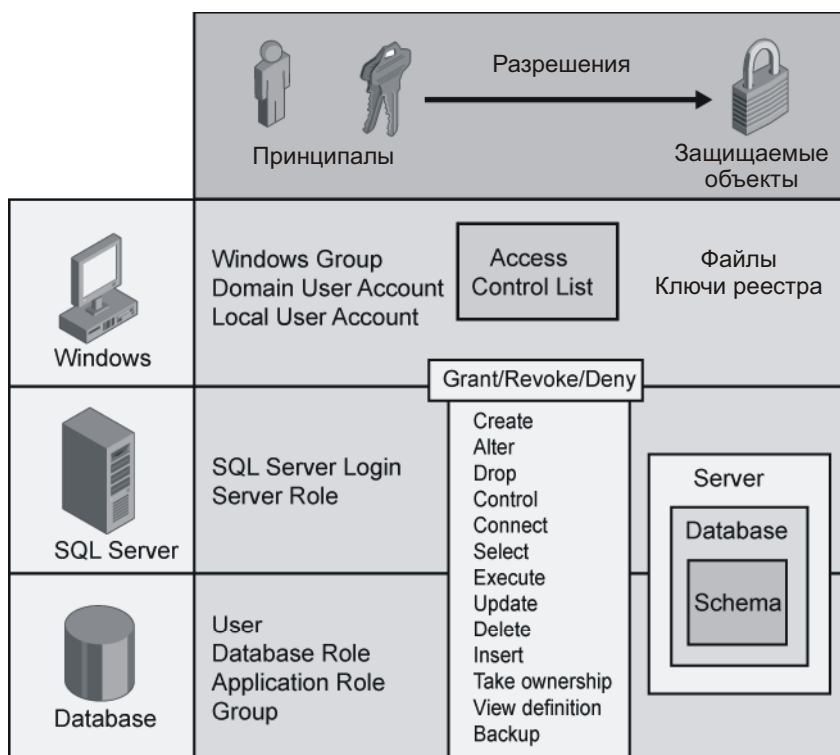
В SQL Server 2005 была введена системная хранимая процедура с названием `sp_configure`, которую можно было использовать для включения/выключения многих функций. В SQL Server 2005 была также введена графическая утилита управления с названием SQL Server Surface Area Configuration Tool. Эта утилита позволяла администраторам включать и выключать такие функции из пользовательского интерфейса. В SQL Server 2008 хранимая процедура `sp_configure` сохранилась, но утилита Surface Area Configuration Tool больше не поддерживается (хотя некоторые функции можно настраивать при помощи SQL Server Configuration Manager). Вместо нее администраторы теперь используют Surface Area Configuration (предоставляемый инфраструктурой Policy-Based Management, PBM).

PBM — это новая функциональная возможность SQL Server 2008, которая позволяет вам объявлять различные политики управления для различных сред назначения и обеспечивать их реализацию. Та функциональность, которую ранее обеспечивал инструмент Surface Area Configuration Tool в SQL Server 2005, теперь является одним из многих аспектов PBM, которые могут быть сконфигурированы в SQL Server 2008. Мы изучаем PBM в *главе 4* (в том числе и Surface Area Configuration для включения интеграции с CLR).

## Обзор безопасности SQL Server 2008

Если вы уже знакомы с концепциями "пользователь", "роль" и "регистрационная запись" SQL Server, то вы, вероятно, можете сразу перейти к следующему разделу данной главы. Но для остальных мы дадим краткое объяснение. Концепции пользователя и роли существуют как в мире операционных систем Microsoft Windows, так и в SQL Server. В абстрактном смысле они называются *принципалами*. Принципалы — это объекты, которые могут требовать ресурсов от приложения или операционной системы. В случае операционной системы Windows принципалы — это такие объекты, как Domain Users and Groups или Local Users and Groups. В SQL Server такими объектами являются регистрационные записи и серверные роли. В базе данных такими объектами являются пользователи базы данных, роли базы данных и роли приложений (и это лишь несколько примеров).

Так что же мы можем сделать с этими объектами? Такие объекты (называемые *защищаемыми объектами*) являются ресурсами, доступ к которым регулируется системой авторизации. Некоторые защищаемые объекты могут содержаться внутри других защищаемых объектов, создавая вложенные иерархии, которые называются защищаемыми областями в ядре базы данных SQL Server: `server`, `database` и `schema`. Каждый защищаемый объект в SQL Server имеет соответствующее разрешение, которое может быть предоставлено принципалу (рис. 5.2).



**Рис. 5.2.** Модель принципал — защищаемый объект — разрешение

## Регистрационные записи SQL Server

Теперь, когда у нас есть понимание безопасности SQL Server на высоком уровне, и мы знаем, как она связана с безопасностью операционной системы Windows, мы можем заняться специфичными для SQL Server вопросами. Для подключения к SQL Server вам необходима регистрационная запись. Регистрационная запись — это комбинация имени пользователя и сложного пароля. Вы можете использовать для регистрации в SQL Server существующую учетную запись Windows и избежать создания специального имени пользователя и пароля. Таким образом, SQL Server поддерживает два типа регистрационных записей: регистрационные записи Windows и регистрационные записи SQL Server.

Сами регистрационные записи не имеют доступа ни к одной конкретной базе данных SQL Server, они позволяют только подключиться к SQL Server. То

есть регистрационные записи являются объектами, которым может быть дано разрешение в масштабе сервера на выполнение определенных действий. Эти действия собираются в серверные роли, такие как `sysadmin`, `diskadmin` и `dbcreator` (это только несколько примеров). В табл. 5.1 показан список серверных ролей и соответствующие им функции. Серверные роли фиксированные: вы не можете удалить их или создать новые в дополнение к существующим девяти ролям.

**Таблица 5.1.** Фиксированные серверные роли

| Серверная роль             | Описание                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>bulkadmin</code>     | Обладатели этой роли могут выполнять инструкцию <code>BULK INSERT</code> . При обладании этой ролью для пользователей, не являющихся <code>sysadmin</code> , все равно требуется доступ к обновляемому объекту                                                                                                                                                                                                                                                                                                                                                    |
| <code>dbcreator</code>     | Обладатели могут создавать, изменять, удалять и восстанавливать любую базу данных                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>diskadmin</code>     | Эта роль используется для управления дисковыми файлами. Большинство возможностей относится к добавлению и удалению устройств резервного копирования                                                                                                                                                                                                                                                                                                                                                                                                               |
| <code>processadmin</code>  | Обладатели могут прерывать процессы, которые работают в экземпляре SQL Server. Эта роль полезна, если вы хотите дать кому-то возможность прервать длительный запрос или потерянное подключение                                                                                                                                                                                                                                                                                                                                                                    |
| <code>public</code>        | Все допустимые регистрационные записи SQL Server являются обладателями этой роли                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>securityadmin</code> | Обладатели могут управлять регистрационными записями и их свойствами. Они могут делать операции <code>GRANT</code> , <code>DENY</code> и <code>REVOKE</code> для разрешений на уровне сервера и на уровне баз данных. Они могут также сбрасывать пароли для регистрационных записей SQL Server. Эта роль не имеет прав для присваивания разрешений на базы данных. Чтобы обладатели прав <code>sysadmin</code> могли это делать, то вы должны сделать эти регистрационные записи частью фиксированной роли <code>db_accessadmin</code> для конкретной базы данных |
| <code>serveradmin</code>   | Обладатели могут изменять конфигурационные настройки на уровне сервера и делать остановку сервера                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>setupadmin</code>    | Обладатели могут добавлять и удалять связанные серверы, а также выполнять некоторые системные хранимые процедуры                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Таблица 5.1 (окончание)

| Серверная роль | Описание                                                                                                                                                                                                                                                                             |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sysadmin       | Обладатели могут выполнять на сервере любые действия. По умолчанию все члены группы BUILTIN\Administrators операционной системы Windows (локальная группа администраторов) являются обладателями серверной роли sysadmin. Учетная запись службы SQL Server также обладает этой ролью |

## Пользователи базы данных

Регистрационные записи можно создавать в SQL Server Management Studio или при помощи инструкции `CREATE LOGIN` языка Transact-SQL (T-SQL). После создания регистрационной записи вы можете предоставить ей доступ к конкретной базе данных. Сами базы данных имеют свой собственный набор ролей, которые определяют доступ и те действия, которые обладатели ролей могут предпринимать в этой конкретной базе данных. Перед тем как мы сможем предоставить регистрационной записи доступ к базе данных, мы должны создать пользователя для этой записи. Пользователя базы данных можно создать в SQL Server Management Studio или при помощи инструкции `CREATE USER` языка T-SQL.

После создания пользователя базы данных вы можете (это необязательно) предоставить ему одну из ролей базы данных. В табл. 5.2 перечислены те роли, которые имеют все базы данных. Подобно серверным ролям, эти роли баз данных фиксированные, и модифицировать их нельзя. Однако в отличие от серверных ролей при желании можно определить дополнительные роли баз данных.

Таблица 5.2. Фиксированные роли баз данных

| Роль базы данных  | Описание                                                                                                                              |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| db_accessadmin    | Обладатели могут добавлять или убирать доступ для регистрационных записей Windows, групп Windows и регистрационных записей SQL Server |
| db_backupoperator | Обладатели могут делать резервное копирование базы данных                                                                             |
| db_datareader     | Обладатели могут читать все данные из всех пользовательских таблиц                                                                    |

Таблица 5.2 (окончание)

| Роль базы данных  | Описание                                                                                                             |
|-------------------|----------------------------------------------------------------------------------------------------------------------|
| db_datawriter     | Обладатели могут добавлять, удалять или изменять данные во всех пользовательских таблицах                            |
| db_ddladmin       | Обладатели могут выполнять любые команды языка определения данных (DDL) по базе данных                               |
| db_denydatareader | Обладатели не могут читать никаких данных из пользовательских таблиц базы данных                                     |
| db_denydatawriter | Обладатели не могут добавлять, удалять или изменять данные в пользовательских таблицах                               |
| db_owner          | Обладатели могут выполнять все действия по конфигурированию и обслуживанию базы данных, включая удаление базы данных |
| db_securityadmin  | Обладатели могут модифицировать владение ролями и управлять разрешениями                                             |

Есть специальная роль базы данных, которую пользователи `sysadmin` не могут явным образом дать другим пользователям: это роль `public`. Все пользователи базы данных неявным образом обладают ролью `public`. Эта роль обладает всеми разрешениями по умолчанию для конкретной базы данных. Вы не можете присвоить ее пользователям, группам или ролям, поскольку по умолчанию этой ролью обладают все. Удалить эту роль нельзя. То есть для защиты от неавторизованного доступа к данным вы должны минимизировать разрешения, предоставляемые роли `public`. Вместо этого лучше предоставлять разрешения другим ролям базы данных и учетным записям пользователей (имеющим свои регистрационные записи).

## Пользовательская учетная запись *guest*

Применительно к теме неавторизованного доступа к данным важно отметить специальную учетную запись пользователя SQL Server с названием `guest`. Эта учетная запись создается по умолчанию в новых определяемых пользователем базах данных, но не существует в `master` и в `tempdb`. Учетная запись `guest` по умолчанию отключена, а это означает, что она не имеет никакого доступа к базе данных. Учетная запись `guest` позволяет получить доступ к базе данных для такой регистрационной записи, которая не имеет учетной



записи пользователя. Регистрационная запись получает удостоверение пользователя `guest` в том случае, когда выполнены все следующие условия:

- ☐ регистрационная запись имеет доступ к экземпляру SQL Server, но не имеет доступа к базе данных (по своей учетной записи пользователя или по членству в группе Windows);
- ☐ база данных имеет учетную запись `guest`;
- ☐ учетная запись `guest` включена в базе данных.

К учетной записи `guest` можно применять разрешения точно так же, как и к любой другой учетной записи пользователя. Однако по возможности следует избегать использования учетной записи `guest`, поскольку все регистрационные записи, не имеющие собственных разрешений по базе данных, получают те разрешения, которые предоставлены учетной записи `guest`. Если вам совершенно необходимо использовать учетную запись `guest`, постарайтесь дать ей минимальные разрешения.

### **ПРИМЕЧАНИЕ**

Несмотря на то, что мы обсудили различные аспекты принципов сервера и баз данных, полное описание этих тем выходит за пределы изложения данной книги. За подробной информацией об управлении пользователями в SQL Server обратитесь к документации SQL Server Books Online.

## **Аутентификация и авторизация**

Перед тем как погрузиться в концепции аутентификации и авторизации, важно обсудить добавившуюся в SQL Server 2005 функциональную возможность — *конечные точки*. В более ранних версиях SQL Server клиенты могли подключаться при помощи протокола Transport Control Protocol (TCP), именованных каналов, общей памяти или Virtual Interface Architecture (VIA). Если один из этих протоколов на сервере был включен, и пользователь имел правильную регистрационную запись, то подключение разрешалось. SQL Server 2005 ввел разделение этого процесса по конечным точкам.

Конечные точки можно считать точками входа в SQL Server. Администраторы могут создать конечную точку не только для TCP, именованных каналов, общей памяти или VIA, но и для Hypertext Transfer Protocol (HTTP). После создания конечной точки вы можете ограничить доступ таким образом, чтобы пользователи подключались только через конечную точку определенного

типа. Например, вы можете создать регистрационную запись Login1, предоставить ей доступ к оконечной точке HTTP и запретить доступ ко всем другим конечным точкам. В этом случае Login1 может получить доступ к SQL Server только через конечную точку HTTP и не может подключиться к SQL Server через TCP или через любой другой протокол. Для того чтобы увидеть, как конечная точка влияет на аутентификацию, давайте изучим процесс установления клиентом подключения.

## Как клиенты устанавливают подключение

Если клиент хочет выполнить подключение к SQL Server по TCP, он должен знать, к какому порту подключаться. До SQL Server 2005 на порту 1434 протокола User Datagram Protocol (UDP) всегда находился в ожидании поток, целью которого было возвращение подробностей по всем работающим экземплярам SQL Server, а также по номерам портов этих экземпляров. Все, что необходимо было сделать клиенту — это подключиться к порту 1434 по протоколу UDP и определить, к какому порту он хочет подключиться (для заданного экземпляра SQL Server). Этот процесс работал до тех пор, пока хакеры не обнаружили способ произвести DoS-атаку (Denial of Service) на SQL Server (путем постоянной отправки на этот порт пакетов с запросом на перечисление). Поскольку этот процесс перечисления был частью службы SQL Server, вирус-червь SQL Slammer создавал серьезные проблемы для SQL Server. Начиная с SQL Server 2005, функция была перенесена в отдельную службу с названием SQL Server Browser, которая может быть включена/выключена без влияния на саму службу SQL Server.

### **ПРИМЕЧАНИЕ**

Конечно, это означает, что служба Browser теперь также уязвима для DoS-атак. Вы можете смягчить эту проблему, если блокируете порт 1434 на своем сетевом экране. Если вы обеспокоены возможностью атак внутри сети, то вы можете вообще отключить службу Browser и явным образом указывать номера портов в строках подключения.

После запроса на сетевое подключение и предварительного согласования параметров соединения SQL Server должен аутентифицировать пользователя (чтобы для начала убедиться, что тот имеет доступ). На рис. 5.3 изображен процесс аутентификации.

В этот момент служба принимает учетные данные, предоставленные пользователем, и пытается проверить их. Если проверка проходит успешно, то ре-

гистрационная запись авторизуется по конечной точке, соответствующей типу произведенного к SQL Server подключения. В нашем примере регистрационная запись проверяется на наличие разрешения `CONNECT` для конечной точки TCP. Если разрешение имеется, то процесс аутентификации продолжается; в противном случае подключение заканчивается ошибкой.

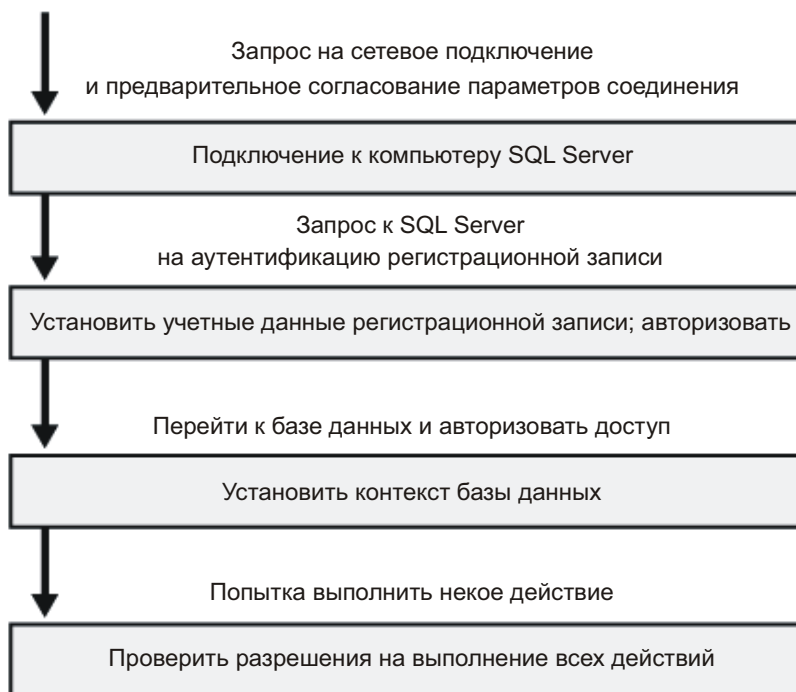


Рис. 5.3. Модель аутентификации SQL Server

### ПРИМЕЧАНИЕ

По умолчанию новым регистрационным записям автоматически предоставляется разрешение `CONNECT` для конечной точки TCP.

После того как регистрационная запись прошла проверку конечной точки, SQL Server переключается в контекст базы данных (база данных по умолчанию, указанная для регистрационной записи или обозначенная в строке подключения). Он пытается аутентифицировать регистрационную запись как пользователя этой базы данных. Если регистрационную запись можно аутен-

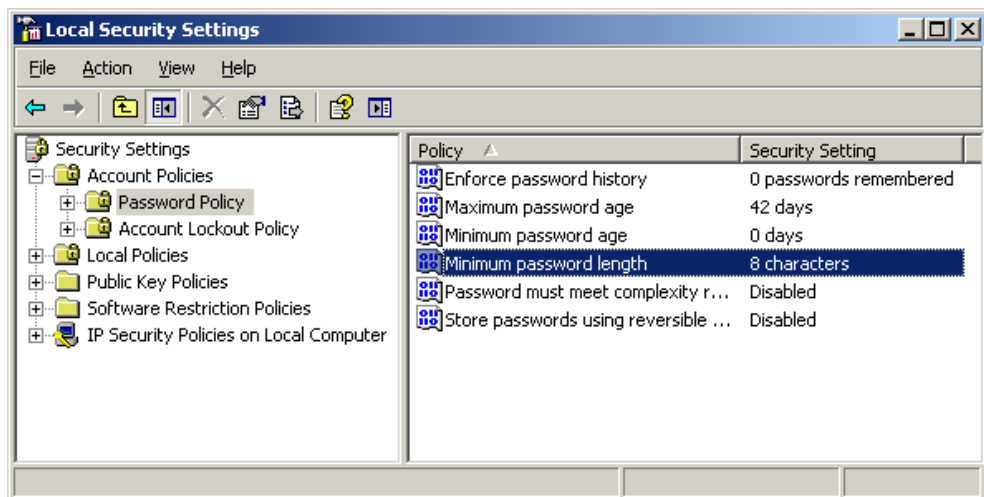
тифицировать, то подключение происходит успешно; в противном случае оно заканчивается ошибкой. Когда контекст базы данных установлен, и регистрационная запись аутентифицирована, пользователь может работать с базой данных на сервере.

## Политики паролей

В мире операционных систем Windows администраторы могут настроить срок окончания действия регистрационной записи и использовать политику паролей (например, потребовать, чтобы пароли имели определенную длину и содержали специальные символы). По традиции регистрационные записи SQL Server никогда не выполняли эти глобальные настройки политик. В SQL Server 2005 и 2008 регистрационные записи (как аутентифицируемые системой Windows, так и записи SQL) соблюдают настройки групповых политик, выполненные в домене Windows.

### ПРИМЕЧАНИЕ

Выполнение парольных политик возможно только для SQL Server на сервере Microsoft Windows Server 2003 и более поздних версий.



**Рис. 5.4.** Узел **Password Policy** в настройках **Local Security Settings**

Для иллюстрации парольных политик давайте определим минимальную длину пароля при помощи апплета **Local Security Settings**, который находится в списке **Administrative Tools** панели управления **Control Panel**. Этот инструмент показан на рис. 5.4.

При помощи этого инструмента мы можем изменить множество параметров. В показанном на рис. 5.4 примере мы установили минимальную длину пароля в восемь символов. После этого изменения мы можем создать новую регистрационную запись. До версии SQL Server 2005 это делалось при помощи системной хранимой процедуры `sp_addlogin`. В SQL Server 2005 и 2008 мы создаем регистрационные записи при помощи инструкции `CREATE LOGIN` языка DDL:

```
CREATE LOGIN foo WITH PASSWORD='123',  
    CHECK_POLICY=ON,  
    CHECK_EXPIRATION=ON
```

При выполнении этой инструкции она заканчивается ошибкой, поскольку наш пароль имеет только три символа, а мы создали парольную политику, которая требует минимум восьми символов:

```
Msg 15116, Level 16, State 1, Line 1
```

```
Password validation failed. The password does not meet policy requirements because it is too short.
```

Системное представление `sys.sql_logins` возвращает информацию о регистрационных записях, такую как наличие политик и сроков окончания. Если вам нужна дополнительная информация по конкретной регистрационной записи (например, сколько ошибочных паролей было введено), то вы можете использовать встроенную функцию `LOGINPROPERTY`:

```
DECLARE @name varchar(20)  
SET @name='sa'  
SELECT  
    LOGINPROPERTY(@name, 'PasswordLastSetTime') AS PasswordLastSetTime,  
    LOGINPROPERTY(@name, 'IsExpired') AS IsExpired,  
    LOGINPROPERTY(@name, 'IsLocked') AS IsLocked,  
    LOGINPROPERTY(@name, 'IsMustChange') AS IsMustChange,  
    LOGINPROPERTY(@name, 'LockoutTime') AS LockoutTime,  
    LOGINPROPERTY(@name, 'BadPasswordCount') AS BadPasswordCount,  
    LOGINPROPERTY(@name, 'BadPasswordTime') AS BadPasswordTime,  
    LOGINPROPERTY(@name, 'HistoryLength') AS HistoryLength,  
    LOGINPROPERTY(@name, 'PasswordHash') AS PasswordHash
```

## Разделение пользователь—схема

Представьте себе, что увольняется работник организации, который создал множество таблиц и хранимых процедур. Как администратор базы данных, вы должны передать владение всеми объектами, которые были созданы бывшим работником. До версии SQL Server 2005 это была утомительная задача. В SQL Server 2005 и 2008 эта задача значительно облегчается благодаря функциональным возможностям схемы.

SQL Server 2005 ввел отличающуюся от более ранних версий SQL Server концепцию схем. В SQL Server 2000 (и более ранних версиях) схемы и пользователи базы данных были эквивалентны. Каждый пользователь базы данных был владельцем схемы с тем же именем, что и у пользователя. Владелец объекта был по существу идентичен владельцу схемы, содержащей объект. Такое однозначное соответствие весьма осложняло изменение владения.

Начиная с SQL Server 2005, пользователи и схемы являются различными объектами, причем схемы являются независимыми контейнерами, которые могут содержать от нуля объектов и больше. Пользователи могут владеть как одной схемой, так и бóльшим их количеством, им может быть присвоена схема по умолчанию. Если схема по умолчанию не указана, то пользователь по умолчанию получает схему `dbo` базы данных. Эта схема по умолчанию может быть использована для разрешения имен защищаемых объектов, ссылки на которые делаются без указания полностью квалифицированного имени. В SQL Server 2000 сначала проверяется схема, принадлежащая вызывающему пользователю базы данных, а за ней — схема, принадлежащая `dbo`. В SQL Server 2005 и 2008 каждый пользователь имеет схему по умолчанию, так что когда SQL Server выполняет разрешение имен защищаемых объектов, то он проверяет схему по умолчанию.

Таким образом, вы можете передать владение многими защищаемыми объектами, а схема может содержать защищаемые объекты, которыми владеет пользователь базы данных, не являющийся владельцем схемы. Это важно отметить, поскольку если вы используете старые системные представления (`sysobjects`), то возвращаемые в запросах результаты будут отображаться некорректно. Вместо них вы должны использовать представление каталога (`sys.objects`).

Инструкция `CREATE SCHEMA` используется для создания новой схемы. Ее можно использовать также для создания таблиц и представлений внутри новой

схемы и для настройки разрешений GRANT, DENY или REVOKE для этих объектов, как это показано в следующем примере:

```
CREATE SCHEMA Sales AUTHORIZATION Rob
CREATE TABLE Leads (id int, name varchar(max), phone nvarchar(20))
GRANT SELECT ON Sales.Leads TO Tammie
DENY SELECT ON Sales.Leads TO Vince;
GO
```

В этом примере создается схема Sales, которая присваивается пользователю Rob. Внутри схемы Sales создается таблица Leads, доступ к таблице для выполнения выборки предоставляется пользователю Tammie, а пользователю Vince такой доступ закрывается.

Для того чтобы пользователю Tammie получить доступ к таблице в версиях до SQL Server 2005, следовало использовать имя схемы Rob, как это показано здесь:

```
SELECT * FROM Production.SalesReps.Rob.Leads
```

В версиях SQL Server 2005 и 2008 можно использовать вместо этого название схемы Sales:

```
SELECT * FROM Production.SalesReps.Sales.Leads
```

Если Rob когда-нибудь уволится или будет уволен, то все, что пользователю sysadmin нужно сделать для передачи владения схемы Sales пользователю Tammie, это следующее:

```
ALTER AUTHORIZATION ON SCHEMA::Sales TO Tammie;
GO
```

Абстрагирование пользователей базы данных от схем предоставляет преимущества, в том числе следующие:

- ☐ одной и той же схемой может владеть множество пользователей (при помощи ролей или членства в группах Windows). Это позволяет ролям и группам владеть объектами;
- ☐ значительно облегчается удаление пользователей баз данных;
- ☐ удаление пользователя базы данных не требует переименования объектов, которые содержатся в схеме этого пользователя. Поэтому вам не нужно перерабатывать и проверять приложения, которые ссылаются только на содержащиеся в схеме защищаемые объекты в случае удаления создавшего их пользователя;
- ☐ пользователи могут совместно использовать одну и ту же схему по умолчанию (для однообразия в разрешении имен);

- ❑ по умолчанию можно хранить совместно используемые объекты в схеме, которая была создана специально для конкретного приложения (вместо схемы `dbo`);
- ❑ разрешениями для схем и содержащихся в схемах защищаемых объектов можно управлять более детально, чем в предыдущих версиях;
- ❑ полностью квалифицированные имена объектов имеют четыре части: `server.database.schema.object`.

## Контекст выполнения

Предоставление и управление разрешениями для пользователей, не являющихся `sysadmin`, всегда было интересной задачей, особенно когда дело касалось пользователей, которые владеют хранимыми процедурами, использующими не принадлежащие им таблицы или другие объекты. Для примера предположим, что имеется три пользователя: `User1`, `User2` и `User3`. `User3` владеет таблицей `CustomerInformation`. `User2` написал хранимую процедуру, которая перечисляет только имена клиентов из этой таблицы. Пользователю `User1` необходимо выполнить хранимую процедуру пользователя `User2`. До версии SQL Server 2005 пользователю `User1` понадобилось бы не только разрешение `EXECUTE` для хранимой процедуры пользователя `User2`, но также и доступ к принадлежащей `User3` таблице `CustomerInformation`. Теперь экстраполируем это на все предприятие, и вы представите, в какой кошмар управления разрешениями это превращалось. Начиная с версии SQL Server 2005, пользователь `User2` может изменить контекст выполнения хранимой процедуры на любой из следующего списка:

- ❑ `EXECUTE AS CALLER` — выполняется с учетными данными вызывающей стороны — то же поведение по умолчанию, что и в предыдущих версиях SQL Server;
- ❑ `EXECUTE AS SELF` — выполняется с учетными данными того пользователя, который последним модифицировал хранимую процедуру;
- ❑ `EXECUTE AS <имя регистрационной записи>` — выполняется с учетными данными указанной регистрационной записи. Для того чтобы это работало, создающему или модифицирующему хранимую процедуру пользователю должно быть предоставлено разрешение `IMPERSONATE` для указанной регистрационной записи;



**ПРИМЕЧАНИЕ**

В `EXECUTE AS` можно указать также имя пользователя базы данных. Если вы используете регистрационную запись, то область действия олицетворения — уровень сервера. Если вы используете пользователя базы данных, то уровень действия — только база данных.

- ❑ `EXECUTE AS OWNER` — выполняется с учетными данными той регистрационной записи, которая владеет хранимой процедурой. Если после создания объекта владелец изменяется, то контекст выполнения автоматически соответствует новому владельцу.

Возвращаясь к нашему примеру, давайте создадим скрипт для нашей проблемы, как это показано в листинге 5.1.

**Листинг 5.1. Пример контекста выполнения**

```
-- Создать наши регистрационные записи
CREATE LOGIN User1 WITH PASSWORD='^*ahfn2@^(K'
GO
CREATE LOGIN User2 WITH PASSWORD='*HABa7s7aas'
GO
CREATE LOGIN User3 WITH PASSWORD='zxd837&^gqF'
GO
CREATE DATABASE MyDB
GO
USE MyDB
GO
-- User3 будет владельцем таблицы
CREATE USER User3 WITH DEFAULT_SCHEMA=User3
GO
CREATE SCHEMA User3 AUTHORIZATION User3
GO
--User2 будет иметь доступ SELECT и напишет процедуру для доступа
CREATE USER User2 WITH DEFAULT_SCHEMA=User2
GO
CREATE SCHEMA User2 AUTHORIZATION User2
GO
--User1 будет иметь право выполнения процедуры
CREATE USER User1 WITH DEFAULT_SCHEMA=User1
```

```
GO
CREATE SCHEMA User1 AUTHORIZATION User1
GO
GRANT CREATE TABLE TO User3
GO
GRANT CREATE PROC TO User2
GO
EXECUTE AS LOGIN='User3'
GO
CREATE TABLE User3.CustomerInformation
    (CustomerName nvarchar(50))
GO
INSERT INTO CustomerInformation VALUES('Bryan''s Bowling Alley')
INSERT INTO CustomerInformation VALUES('Tammie''s Tavern')
INSERT INTO CustomerInformation VALUES('Frank''s Fresh Produce')
GO
GRANT SELECT ON CustomerInformation TO User2
GO
REVERT
GO
EXECUTE AS LOGIN='User2'
GO
-- создать хранимую процедуру, которая будет возвращать строки таблицы
CREATE PROC ViewCustomerNames
AS
    BEGIN
        SELECT * FROM User3.CustomerInformation
    END
GO
GRANT EXECUTE ON ViewCustomerNames TO User1
GO
REVERT
GO

Теперь, после создания всех пользователей и объектов, давайте зарегистри-
руемся как User1 и попробуем напрямую получить доступ к таблице и хра-
нимой процедуре:

EXECUTE AS LOGIN='User1'
-- User1 не может получить доступ к таблице напрямую
SELECT * FROM User3.CustomerInformation
```

```
-- User1 может выполнить процедуру, но не имеет разрешений
-- для соответствующей таблицы
EXEC User2.ViewCustomerNames
GO
```

```
REVERT
GO
```

На этой стадии в версиях до SQL Server 2005 нам пришлось бы дать пользователю User1 какую-то форму доступа к таблице пользователя User3. Начиная с версии SQL Server 2005, пользователь User2 может изменить контекст выполнения хранимой процедуры таким образом, чтобы пользователю User1 не нужно было предоставлять доступа к таблице пользователя User3. В нашем примере User2 изменит контекст выполнения на `EXECUTE AS OWNER`. Это приведет к тому, что хранимая процедура будет выполняться как User2, поскольку User2 является владельцем хранимой процедуры:

```
EXECUTE AS LOGIN='User2'
GO
ALTER PROCEDURE ViewCustomerNames WITH EXECUTE AS OWNER
AS
    BEGIN
        SELECT * FROM User3.CustomerInformation
    END
GO
REVERT
GO
```

Теперь, когда User1 выполняет хранимую процедуру пользователя User2, хранимая процедура будет выполняться с учетными данными пользователя User2 (и иметь доступ к таблице пользователя User3). Такой доступ будет разрешен только в контексте хранимой процедуры. Таким образом, User1 может выполнять хранимую процедуру пользователя User2, не имея явного доступа к соответствующей таблице:

```
EXECUTE AS LOGIN='User1'
-- User1 не может получить доступ к таблице напрямую
SELECT * from User3.CustomerInformation
-- User1 может выполнить процедуру, которая использует
-- таблицу CustomerInformation
EXEC User2.ViewCustomerNames
GO
REVERT
GO
```

В предыдущем примере можно использовать и другую форму переключения контекста. Начиная с версии SQL Server 2005, можно изменить контекст выполнения текущего подключения без необходимости закрывать и вновь открывать это подключение. Для этого пользователь должен быть либо `sysadmin`, либо иметь разрешение `IMPERSONATE` для регистрационной записи. Вы можете использовать эту же инструкцию и с целью переключения пользовательского контекста для пользователя базы данных. При использовании с целью переключения контекста для базы данных область действия олицетворения ограничена текущей базой данных. При переключении контекста для пользователя базы данных разрешения уровня сервера для этого пользователя не наследуются. Кроме того, пользователь должен быть либо `sysadmin`, либо иметь разрешение `IMPERSONATE` для пользователя базы данных.

Выполнение переключения контекста — это мощный и эффективный способ уменьшения количества подлежащих управлению разрешений. Для разработчиков и администраторов оно дает легкий способ тестировать скрипты и вести отладку без необходимости выходить из системы и вновь подключаться к SQL Server.

## Поддержка шифрования в SQL Server

Что и говорить, хранящаяся в базах данных информация интересна не только нам, пользователям, но и многим другим, которых мы не знаем и доверять которым не можем. С учетом постоянно растущего использования реляционных баз данных в бизнесе поставщики баз данных должны все время предлагать новые функции для обеспечения безопасности.

Кроме блокирования доступа к базам данных SQL Server, администраторы и разработчики могут предоставить еще один уровень защиты — *шифрование*. Шифрование берет интересную информацию (такую как номер вашей кредитной карты) и преобразует в двоичное представление, которое могут понять только авторизованные участники данного процесса. Данные могут быть зашифрованы для передачи (как при отправке пароля на Web-сервер) либо сохранены в зашифрованном формате (в файловой системе или базе данных). В версии SQL Server 2008 (только в редакции Enterprise) вся база данных может быть автоматически зашифрована при помощи прозрачного шифрования (Transparent Data Encryption, TDE), которое будет подробно обсуждаться далее в этой же главе.

Данные в SQL Server шифруются при помощи *ключей шифрования*. Эти ключи могут быть симметричными или асимметричными, причем использование

тех и других имеет свои достоинства и недостатки. При шифровании с симметричными ключами отправитель и получатель данных имеют один и тот же ключ. Отправитель шифрует данные при помощи ключа и алгоритма шифрования. Когда данные доходят до получателя, они дешифруются при помощи того же самого алгоритма шифрования и ключа. Главное преимущество такого подхода состоит в том, что производительность шифрования и дешифрования выше, чем при использовании асимметричных ключей. Проблема шифрования возникает тогда, когда наш симметричный ключ шифрования получает кто-то другой. Поскольку мы шифруем только одним ключом, то расшифровать данные может каждый, кто имеет этот ключ.

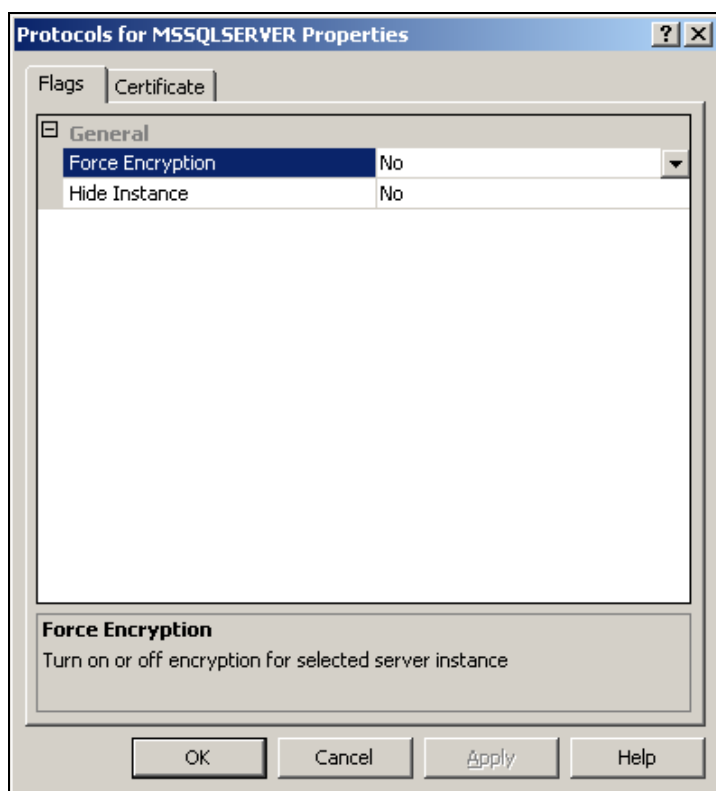
Шифрование с асимметричными ключами использует два ключа: открытый и закрытый. Отправитель шифрует данные при помощи открытого ключа получателя, который свободно может получить кто угодно. Когда пользователь получает данные, он расшифровывает их при помощи своего закрытого ключа. Открытый ключ в этом случае расшифровать данные не может. Таким образом, при использовании асимметричного шифрования ценностью является закрытый ключ.

Еще одна концепция шифрования — *сертификаты*. Сертификаты — это по существу асимметричные ключи, которые содержат дополнительные метаданные. Эти метаданные включают в себя такую информацию, как время окончания и центр сертификации, выдавший данный сертификат. Сертификаты могут создаваться кем угодно, и в некоторых обстоятельствах вы хотите быть уверенными в том, что отправитель или получатель данных реально является тем, за кого он себя выдает. Именно здесь действуют центры сертификации. Эти компании служат посредниками между отправителем и получателем. После уплаты вами некоторой суммы денег и после того, как они проведут проверку вашей идентификации, они дадут вам подписанный ими сертификат. Когда вы будете использовать этот подписанный сертификат для отправки данных, то получатель сможет проверить его в центре сертификации, и поскольку вы оба доверяете центру сертификации, то будет вполне надежно допустить, что сообщение действительно было подписано отправителем.

Есть еще один тип сертификата, называемый *самозаверенным сертификатом*, который может создать каждый. В некоторых случаях вполне допустимо использовать самозаверенные сертификаты. SQL Server автоматически создает самозаверенный сертификат при своем первом запуске. Этот сертификат используется для шифрования подключения при выполнении аутентификации SQL Server.

## Шифрование данных при их перемещении

Если клиент использует интерфейсы прикладного программирования SQL Server Native Access Client, то все запросы подключения к SQL Server шифруются. Это большое улучшение, поскольку в SQL Server 2000 и более ранних версиях SQL Server Authentication имя пользователя и пароль посылались по сети открытым текстом. SQL Server 2005 и 2008 могут автоматически шифровать информацию об имени входа при помощи самозаверенного сертификата, который был создан при первом запуске службы SQL.



**Рис. 5.5.** Диалоговое окно  
**Protocols For MSSQLSERVER Properties**

Пакеты с названием регистрационной записи — это не все, что шифруется при подключении. По желанию можно зашифровать и само подключение на

все время его существования. Запрос на шифрованный канал может выдаваться сервером принудительно (так что по умолчанию будут шифроваться все подключения), либо такой запрос может быть сделан клиентом, выполняющим подключение. Рекомендуется, чтобы во избежание атак посредника администраторы использовали настоящий сертификат (а не самозаверенный).

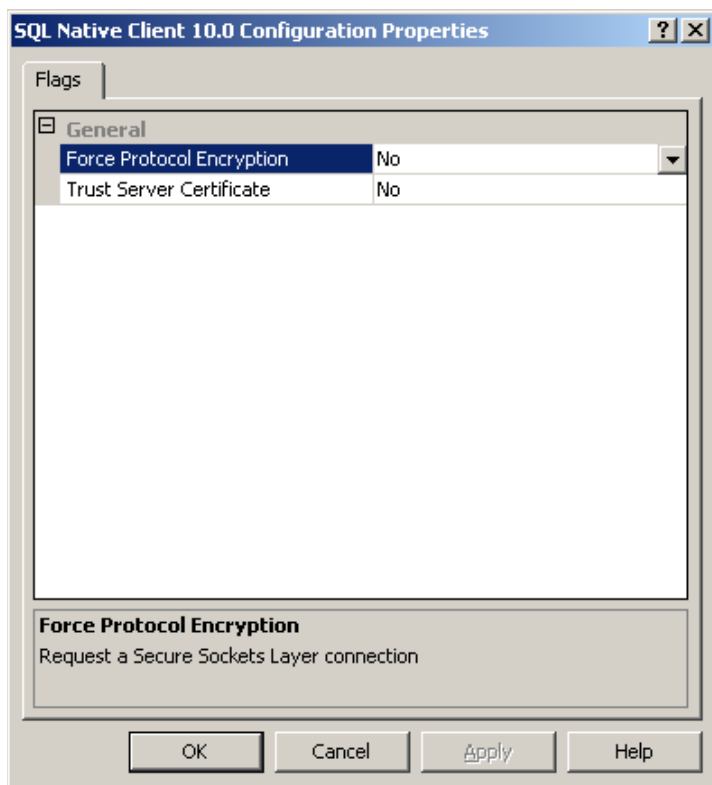


Рис. 5.6. Диалоговое окно  
SQL Native Client 10.0 Configuration Properties

Для того чтобы включить принудительное шифрование на сервере, запустите инструмент SQL Server Configuration Manager (доступен в каталоге **Configuration Tools** пункта меню **SQL Server 2008 Program Files**). Этот инструмент используется для управления протоколами и службами SQL Server. Разверните узел **SQL Server Network Configuration**, щелкните правой кнопкой мыши **Protocols For MSSQLSERVER**, а затем выберите пункт **Properties**

для того, чтобы открыть диалоговое окно **Protocols For MSSQLSERVER Properties** (рис. 5.5).

Это диалоговое окно имеет две вкладки. Вкладка **Flags** позволяет вам принудительно включить шифрование на сервере. Вкладка **Certificate** позволяет вам выбрать сертификат (который уже установлен на локальном компьютере) для использования сервером SQL Server при шифровании данных. Если вы не выберете сертификат и захотите включить шифрование, то SQL Server для шифрования данных будет использовать свой самозаверенный сертификат. Помните, что подключающиеся клиенты не считают самозаверенный сертификат SQL Server надежным. Для того чтобы клиенты могли пользоваться самозаверенным сертификатом SQL Server, они должны настроить эту опцию в диалоговом окне **Properties** узла **SQL Native Client 10.0 Configuration** в Configuration Manager. Это диалоговое окно показано на рис. 5.6.

Это диалоговое окно предлагает два варианта: вариант **Force Protocol Encryption** принудительно заставляет клиентов всегда делать зашифрованные подключения. Второй вариант необходимо выставить тогда, когда клиент хочет делать зашифрованные подключения к SQL Server и использовать при этом самозаверенный сертификат SQL Server.

## Шифрование данных в покое

Номера кредитных карт, информация о заработной плате, пароли — список конфиденциальной информации растет. Доступ к такой информации в базе данных традиционно закрывался при помощи разрешений. Так делается и сейчас, но в SQL Server 2005 и 2008 появился еще один собственный уровень защиты. Конфиденциальные данные можно теперь шифровать при помощи симметричных ключей, асимметричных ключей и сертификатов.

Перед тем как перейти к шифрованию данных столбца, важно определить базовые блоки поддержки шифрования в SQL Server. При первом запуске SQL Server он создает специальный симметричный главный ключ службы (Service Master Key, SMK). Этот ключ используется для шифрования всех главных ключей баз данных (database master keys, DMK), а также всей секретной информации на уровне сервера (учетные данные или пароли связанных серверов). Сам ключ — 128-битный ключ 3DES. Алгоритм 3DES используется потому, что он имеется во всех основанных на операционных системах Windows платформах, которые поддерживаются сервером SQL Server.



**ПРИМЕЧАНИЕ**

Доступность алгоритма шифрования зависит от поставщика службы шифрования той операционной системы, на которой работает SQL Server. Например, операционная система Microsoft Windows XP Service Pack 2 (SP2) поддерживает DES, 3DES, RC2, RC4 и RSA, а операционные системы Windows Server 2003 и 2008 поддерживают еще AES128, AES192 и AES256. Объяснение этих алгоритмов выходит за рамки данной книги, однако имеется масса ресурсов, которые помогут вам выбрать подходящий алгоритм шифрования. Дополнительную информацию можно найти в Интернете по строке "Choosing an encryption algorithm for SQL Server".

SMK шифруется интерфейсом прикладного программирования безопасности (Data Protection API, DPAPI) операционной системы Windows и учетных данных службы SQL Server. Поскольку SMK используется для шифрования всех DMK и прочей секретной информации на уровне сервера, следует регулярно делать его резервные копии. Вы можете сделать резервную копию SMK и восстановить его при помощи инструкций `BACKUP SERVICE MASTER KEY` и `RESTORE SERVICE MASTER KEY`. В случае компрометации SMK, или если вы хотите сменить SMK для соблюдения лучших практик безопасности, вы можете сформировать его повторно при помощи инструкции `ALTER SERVICE MASTER KEY REGENERATE`.

При шифровании данных SMK используется сервером SQL Server для дешифрования DMK (чтобы DMK, в свою очередь, мог дешифровать затребованные клиентом данные). Существует только один DMK для каждой базы данных, причем по умолчанию они не создаются, поскольку DMK используется только для шифрования данных.

Для создания DMK мы можем использовать новую инструкцию DDL:

```
USE <user-defined database>
GO
CREATE MASTER KEY
    ENCRYPTION BY PASSWORD = '<password>'
```

Когда DMK создаются, они шифруются посредством SMK (чтобы SQL Server мог дешифровать данные для клиента) и пароля. Можно убрать шифрование при помощи SMK, но пароль DMK пользователю придется указывать каждый раз, когда он обращается к этому ключу.

После того как мы обсудили главные компоненты шифрования, давайте рассмотрим пример. В этом сценарии есть таблица с названием `SalaryInfo`, которая содержит имена, отделы и зарплату служащих нашей компании. Имеется пользователь `HR_User`, который должен иметь возможность вставлять и просматривать данные этой таблицы.

Поскольку этот пример содержит множество шагов, то лучше всего пройти по листингу 5.2, строка за строкой, и прочитать комментарии. Они объясняют все, что делает каждая инструкция T-SQL.

**Листинг 5.2. Шифрование при помощи ENCRYPTBYKEY и DECRYPTBYKEY**

```
USE master
GO
CREATE LOGIN HR_Login WITH PASSWORD='SomeComplexPassword'
GO
USE MyDB
GO
CREATE USER HR_User FOR LOGIN HR_Login
GO
-- Создать главный ключ базы данных ExampleDB
CREATE MASTER KEY ENCRYPTION BY PASSWORD='AComplexPassword'
GO
--Создать таблицу, которая будет хранить конфиденциальную информацию
--Обратите внимание, что мы используем varbinary для информации о зарплате
--Это потому, что шифрованные данные двоичные
CREATE TABLE SalaryInfo
(Employee nvarchar(50),
 Department nvarchar(50),
 Salary varbinary(60))
GO
-- Предоставить доступ к этой таблице пользователю HR_User,
-- чтобы он мог добавлять данные
GRANT SELECT, INSERT TO HR_User
GO
-- Создать симметричный ключ
-- Зашифровать ключ паролем
-- Предоставить доступ к ключу пользователю HR_User
CREATE SYMMETRIC KEY HR_User_Key
AUTHORIZATION HR_User
WITH ALGORITHM=TRIPLE_DES
ENCRYPTION BY PASSWORD='CompensationPlansRule'
GO
--теперь войдем в систему как HR_User и зашифруем некоторые данные
EXECUTE AS LOGIN='HR_Login'
```

```
GO
-- Сначала нам нужно открыть ключ, который будет использоваться
-- для шифрования данных. Обратите внимание, что мы должны передать
-- пароль для ключа
OPEN SYMMETRIC KEY HR_User_Key DECRYPTION
    BY PASSWORD='CompensationPlansRule'
GO
-- Это системное представление показывает открытые ключи,
-- которые можно использовать для шифрования
SELECT * FROM sys.openkeys
-- Вставить в таблицу конфиденциальные данные
-- ENCRYPTBYKEY принимает GUID ключа и текст данных
-- Поскольку запомнить GUIDs нелегко, то функция KEY_GUID
-- делает для нас поиск
INSERT INTO SalaryInfo VALUES
    ('Bryan', 'Sales', ENCRYPTBYKEY(KEY_GUID('HR_User_Key'), '125000'))
INSERT INTO SalaryInfo VALUES
    ('Tammie', 'Sales', ENCRYPTBYKEY(KEY_GUID('HR_User_Key'), '122000'))
INSERT INTO SalaryInfo VALUES
    ('Frank', 'Development', ENCRYPTBYKEY(KEY_GUID('HR_User_Key'), '97500'))
INSERT INTO SalaryInfo VALUES
    ('Fran', 'Marketing', ENCRYPTBYKEY(KEY_GUID('HR_User_Key'), '99500'))
-- Когда все закончено, всегда закрывайте все ключи
CLOSE ALL SYMMETRIC KEYS
GO
-- Просмотрите таблицу в базе данных
SELECT * FROM SalaryInfo
-- Теперь дешифруем и посмотрим содержимое
-- Мы используем DECRYPTBYKEY и передаем название столбца
-- Нам не нужно указывать GUID ключа, поскольку SQL будет
-- просматривать все ваши открытые ключи и автоматически
-- использует подходящий
OPEN SYMMETRIC KEY HR_User_Key DECRYPTION
    BY PASSWORD='CompensationPlansRule'
GO
SELECT Employee, Department, CONVERT(varchar, DECRYPTBYKEY(Salary))
    FROM SalaryInfo
GO
CLOSE ALL SYMMETRIC KEYS
```

```
GO
-- Вернуться к sysadmin
REVERT
GO
-- При шифровании с паролем надо знать пароль
-- и передавать его каждый раз при шифровании.
-- Либо вы можете создать сертификат и дать доступ пользователю
-- HR User. При этом пользователю не нужно предоставлять пароль,
-- и вы можете легко отозвать доступ к этим зашифрованным данным
-- простым удалением сертификата для данного пользователя
CREATE CERTIFICATE HRCert1
    AUTHORIZATION HR_User
    WITH SUBJECT='Certificate used by the Human Resources person'
-- Открыть ключ, чтобы мы могли модифицировать его
OPEN SYMMETRIC KEY HR_User_Key DECRYPTION BY
    PASSWORD='CompensationPlansRule'
GO
-- Мы не можем удалить пароль, поскольку мы оставим ключ без шифрования,
-- поэтому нам нужно сначала добавить сертификат
ALTER SYMMETRIC KEY HR_User_Key
    ADD ENCRYPTION BY CERTIFICATE HRCert1
GO
-- Теперь мы можем удалить шифрование пароля из ключа
ALTER SYMMETRIC KEY HR_User_Key
    DROP ENCRYPTION BY PASSWORD= 'CompensationPlansRule'
GO
CLOSE ALL SYMMETRIC KEYS
GO
-- Теперь изменим контекст в HR_Login для проверки наших изменений
EXECUTE AS LOGIN='HR_Login'
GO
-- Обратите внимание, что мы открыли ключ без пароля!
-- Это потому, что мы создали сертификат и дали авторизацию
-- явным образом пользователю HR_User
OPEN SYMMETRIC KEY HR_User_Key DECRYPTION BY CERTIFICATE HRCert1
GO
SELECT Employee, Department, CONVERT(varchar, DECRYPTBYKEY(Salary))
    FROM SalaryInfo
GO
```

Предыдущий пример описал основы шифрования и дешифрования в SQL Server. Количество данных этого примера очень просто зашифровать и расшифровать даже на старых (медленных) компьютерах. Производительность шифрования (помимо мощности вашего сервера) зависит еще от двух факторов: размера шифруемых данных и от используемого для шифрования данных алгоритма. Определить зависимость времени от размера сложно ввиду множества факторов. Самый лучший подход — создать тестовую среду, которая имитирует вашу производственную, и провести несколько тестов производительности: зашифровать и расшифровать данные различного размера при помощи разных алгоритмов (или хотя бы при помощи одного того алгоритма, который вы собираетесь использовать).

Еще одна интересная проблема с шифрованными столбцами — это индексирование и поиск. Для SQL Server это двоичные столбцы, поэтому не существует эффективного способа создания по ним индекса (поскольку нельзя предсказать случайный поток байтов). Самое лучшее в этом случае — создать или использовать другой нешифрованный столбец для индексирования по нему. Проблема здесь в том, что вы можете непреднамеренно дать информацию о зашифрованных данных. Представьте, что мы хотим проиндексировать по зарплате и создаем столбец с названием *range*. Каждый, кто имеет разрешение *SELECT* по данной таблице, может догадаться о зарплате сотрудника. Если вам нужно проиндексировать шифрованные данные или делать по ним поиск, будьте изобретательнее при создании нешифрованных столбцов.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Шифрование — вещь непростая, но ее подробное обсуждение выходит за рамки данной книги. В Интернете есть ресурсы, которые помогут вам понять больше о шифровании в системе безопасности SQL Server. Laurentiu Cristofor (один из разработчиков шифрования в SQL Server) имеет массу информации в своем блоге, посвященном шифрованию: <http://blogs.msdn.com/lcris>. Рaul Гарсия также является членом команды разработчиков системы безопасности SQL Server. Его блог по адресу <http://blogs.msdn.com/raulga/> содержит дополнительную полезную информацию, как и блог, посвященный системе безопасности SQL Server по адресу <http://blogs.msdn.com/sqlsecurity/>.

## **Прозрачное шифрование данных в SQL Server 2008**

SQL Server теперь может автоматически зашифровать всю базу данных (как данные, так и файлы журнала), и при этом вам не понадобится вносить ника-

ких изменений в приложение. Это делается при помощи новой функциональной возможности в SQL Server 2008 (она имеется только в редакции Enterprise) — прозрачного шифрования данных (Transparent Data Encryption, TDE). Данные шифруются "на ходу" при записи на диск и расшифровываются при считывании. Шифрование выполняется на уровне страниц и не увеличивает размер базы данных. Поскольку этот процесс совершенно прозрачен, то использовать TDE в SQL Server 2008 чрезвычайно легко. Давайте же посмотрим, как это делается.

### **ПРИМЕЧАНИЕ**

Файловая система NTFS в Windows Server 2000 и более поздних операционных системах имеет функциональную возможность "шифрованная файловая система" (Encrypted File System, EFS). Эта функция также применяет прозрачное шифрование к любым хранящимся на жестком диске данным, но она не сможет защитить базы данных или резервные копии, которые были скопированы на компакт-диски или другие носители. TDE в SQL Server 2008 основана на сертификате, который необходим для расшифровки или восстановления любой зашифрованной базы данных (независимо от того, куда перенесены данные).

## **Создание ключей и сертификатов**

Для того чтобы начать использовать TDE, вам необходимо создать сертификат, а для этого вам нужно создать *главный ключ*. Главный ключ создается только один раз (для каждого экземпляра сервера), и создать его можно только в базе данных master. Хотя в общем случае сертификаты можно создавать в любой базе данных, используемые для TDE сертификаты также должны создаваться в базе данных master, как показано далее:

```
USE master
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'Hrd2GessP@ssw0rd!'
CREATE CERTIFICATE MyEncryptionCert WITH SUBJECT =
                                     'My Encryption Certificate'
```

Для подтверждения создания сертификата вы можете сделать запрос к представлению sys.certificates:

```
SELECT name, pvt_key_encryption_type_desc FROM sys.certificates
WHERE name = 'MyEncryptionCert'
```

Выходные данные подтверждают, что сертификат существует, и его закрытый ключ защищен главным ключом:

```
name                                pvt_key_encryption_type_desc
```

```
-----  
MyEncryptionCert                   ENCRYPTED_BY_MASTER_KEY
```

```
(1 row(s) affected)
```

Затем переключитесь на базу данных, которую вы хотите зашифровать, и создайте ключ шифрования базы данных (database encryption key, DEK):

```
USE MyDB
```

```
GO
```

```
CREATE DATABASE ENCRYPTION KEY
```

```
WITH ALGORITHM = AES_128
```

```
ENCRYPTION BY SERVER CERTIFICATE MyEncryptionCert
```

### **ПРИМЕЧАНИЕ**

Поскольку мы еще не сделали резервной копии сертификата сервера, то SQL Server выдает нам предупреждение о том, что используемый для шифрования DEK сертификат еще не был скопирован. Это предупреждение следует воспринять всерьез, поскольку без этого сертификата вы не сможете получить доступ ни к одной базе данных, зашифрованной ключом DEK. Если сертификат будет поврежден или утрачен, то ваша база данных станет совершенно недоступной. Далее в этом же разделе вы научитесь делать резервные копии сертификата и восстанавливать его.

Эта инструкция создает DEK для базы данных `MyDB`. С использованием этого DEK сервер SQL Server 2008 зашифрует `MyDB` при помощи алгоритма `AES_128`. В выражении `WITH ALGORITHM` в качестве алгоритма шифрования можно указать также `AES_192`, `AES_256` или `TRIPLE_DES_3KEY`. Сам DEK шифруется при помощи сертификата `MyEncryptionCert`, который мы создали в базе данных `master`. (DEK может шифроваться только при помощи сертификата; его нельзя зашифровать одним паролем). DEK будет использоваться не только для того, чтобы шифровать базу данных, но также и для защиты ее резервных копий. Без использованного для шифрования DEK сертификата резервную копию зашифрованной базы данных нельзя будет восстановить нигде — и это конец всему.

## **Включение TDE**

После создания (для базы данных) защищенного сертификатом DEK вы можете начинать процесс шифрования. Шифрование происходит в фоновом

режиме и не мешает приложениям, которые в это время обращаются к базе данных. Для того чтобы начать шифрование базы данных `MyDB`, используйте инструкцию `ALTER DATABASE` и укажите выражение `SET ENCRYPTION ON`:

```
ALTER DATABASE MyDB SET ENCRYPTION ON
```

Вот и все! С этого момента база данных и все ее резервные копии будут зашифрованы. Если неавторизованный субъект каким-то образом получит доступ к физическому носителю с резервными копиями, то без защищающего ключ DEK сертификата они будут совершенно бесполезными.

## Запросы к представлениям TDE

Чтобы увидеть, какие базы данных защищены шифрованием TDE, вы можете сделать запрос к каталогу `sys.databases`:

```
SELECT name, is_encrypted FROM sys.databases
```

Результаты запроса показывают, что `MyDB` является единственной зашифрованной базой данных на сервере:

| name               | is_encrypted |
|--------------------|--------------|
| master             | 0            |
| tempdb             | 0            |
| model              | 0            |
| msdb               | 0            |
| ReportServer       | 0            |
| ReportServerTempDB | 0            |
| AdventureWorks     | 0            |
| AdventureWorksDW   | 0            |
| MyDB               | 1            |

(9 row(s) affected)

### ПРИМЕЧАНИЕ

Шифрование одной (или более) баз данных приводит также к шифрованию `tempdb`. Это может повлиять на производительность нешифрованных баз данных этого экземпляра сервера. Однако поскольку шифрование `tempdb` неявное, то `sys.databases` возвращает для `tempdb` значение 0.

Вы можете также сделать запрос к динамическому административному представлению `sys.dm_database_encryption_keys` для того, чтобы увидеть все ключи DEK и наблюдать за продвижением хода шифрования (или дешифро-



вания, если вы отключили TDE) в фоновых потоках, управляемых сервером SQL Server. Это представление возвращает уникальный идентификатор базы данных, который можно объединить с `sys.databases`, чтобы увидеть фактическое название базы данных. Например, если мы после включения TDE выполним следующий запрос, то мы можем получить информацию о DEK и фоновом процессе шифрования:

```
SELECT
    sys.databases.name,
    sys.dm_database_encryption_keys.encrypted_state,
    sys.dm_database_encryption_keys.percent_complete,
    sys.dm_database_encryption_keys.key_algorithm,
    sys.dm_database_encryption_keys.key_length
FROM
    sys.dm_database_encryption_keys INNER JOIN sys.databases
    ON sys.dm_database_encryption_keys.database_id =
        sys.databases.database_id
WHERE
    sys.databases.name = 'MyDB'
```

Если этот запрос выполняется после включения TDE, но до завершения шифрования базы данных в фоновом режиме, то мы получаем примерно такие результаты:

| name | encrypted_state | percent_complete | key_algorithm | key_length |
|------|-----------------|------------------|---------------|------------|
| MyDB | 2               | 78.86916         | AES           | 128        |

(1 row(s) affected)

Возвращаемое в `encrypted_state` значение дает вам текущее состояние шифрования (или дешифрования):

- ❑ 1 — незашифрована;
- ❑ 2 — ведется шифрование;
- ❑ 3 — зашифрована;
- ❑ 4 — смена ключа;
- ❑ 5 — расшифровка (после инструкции `ALTER DATABASE...SET ENCRYPTION OFF`).

В отдельных состояниях (2, 4 и 5) некоторые операции с базой данных выполнять нельзя. Сюда входят включение или выключение шифрования, удаление или отключение базы данных, удаление файла из группы файлов, вы-

вод базы данных в автономный режим, перевод базы данных (или любой ее группы файлов) в состояние READ ONLY.

## Резервное копирование сертификата

Чрезвычайно важно делать резервное копирование сертификатов сервера, которые вы используете для шифрования ваших баз данных при помощи TDE. Без сертификата вы не сможете получить доступ к зашифрованной базе данных или восстановить резервные копии зашифрованной базы данных (именно для этого и создана TDE). Попытка восстановления шифрованной базы данных без сертификата закончится с такой ошибкой SQL Server:

```
Msg 33111, Level 16, State 3, Line 1
Cannot find server certificate with thumbprint
      '0x6B1FEEEE238847DE75D1850FA20D87CF94F71F33'.
Msg 3013, Level 16, State 1, Line 1
RESTORE DATABASE is terminating abnormally.
```

Для резервного копирования сертификата сервера в файл используйте следующую инструкцию. Кроме самого сертификата необходимо также сохранить в файл закрытый ключ сертификата и защитить его паролем:

```
BACKUP CERTIFICATE MyEncryptionCert TO FILE='C:\MyEncryptionCert.certbak'
WITH PRIVATE KEY (
    FILE='C:\MyEncryptionCert.pkbak',
    ENCRYPTION BY PASSWORD='Pr!vK3yP@ssw0rd')
```

Эта инструкция создает два файла: `MyEncryptionCert.certbak` — резервная копия сертификата сервера, а `MyEncryptionCert.pkbak` — резервная копия закрытого ключа сертификата, защищенная паролем `Pr!vK3yP@ssw0rd`. При резервном копировании закрытого ключа сертификата защита паролем совершенно обязательна. Оба эти файла и пароль понадобятся для восстановления резервной копии шифрованной базы данных на другой сервер или другой экземпляр сервера. Не побоимся сказать очевидную вещь: эти файлы резервных копий и пароль закрытого ключа следует тщательно беречь.

## Восстановление шифрованной базы данных

Перед тем как восстанавливать шифрованную базу в другом месте, необходимо восстановить сертификат сервера, с помощью которого зашифрован ее

ДЕК. И если экземпляр назначения не имеет главного ключа, то его необходимо создать до восстановления сертификата сервера:

```
USE master
```

```
GO
```

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'An0thrHrd2GessP@ssw0rd!'
```

Для восстановления сертификата сервера из сделанных нами ранее файлов резервных копий используйте следующую форму инструкции CREATE CERTIFICATE:

```
CREATE CERTIFICATE MyEncryptionCert
```

```
FROM FILE='C:\MyEncryptionCert.certbak'
```

```
WITH PRIVATE KEY(
```

```
FILE='C:\MyEncryptionCert.pkbak',
```

```
DECRYPTION BY PASSWORD='Pr!vK3yP@ssw0rd')
```

Эта инструкция восстанавливает сертификат сервера MyEncryptionCert из файла резервной копии MyEncryptionCert.certbak и файла резервной копии закрытого ключа сертификата MyEncryptionCert.pkbak. Совершенно очевидно, что указанный в выражении DECRYPTION BY PASSWORD пароль должен совпадать с тем, который был использован при резервном копировании закрытого ключа сертификата (в противном случае сертификат восстановлен не будет). При наличии успешно восстановленного сертификата вы можете восстановить резервную копию любой зашифрованной базы данных, чей ключ DEK основан на сертификате MyEncryptionCert.

## Аудит SQL Server

SQL Server 2008 вводит аудит SQL Server Audit — новую мощную функциональную возможность системы безопасности, которая может отслеживать практически любое действие с сервером или базами данных (выполняемое пользователями) и записывать эти действия в файловую систему или в системный журнал Windows. Подобно шифрованию TDE, аудит SQL Server Audit имеется только в редакции Enterprise версии SQL Server 2008. Эта функциональная возможность помогает выполнять положения нормативных документов, обычно требующих (на инсталляциях масштаба предприятия) реализации строгих правил безопасности, которые часто включают в себя некую форму аудита. Вы можете работать с SQL Server Audit либо при помощи SQL Server Management Studio, либо из T-SQL при помощи нового набора инструкций DDL и представлений каталогов.

### ПРИМЕЧАНИЕ

Здесь мы описываем инструкции аудита DDL языка T-SQL и представления каталогов, а также просмотр журналов аудита в SQL Server Management Studio. Поддержку самого аудита вы найдете в SQL Server Management Studio в узлах ниже **Security** (как на уровне сервера, так и на уровне базы данных).

## Создание объекта аудита

Первым шагом необходимо использовать SQL Server Audit для создания *объекта аудита* (при помощи инструкции `CREATE SERVER AUDIT`). Когда мы создаем объект аудита, то мы, по сути, определяем назначение, в которое SQL Server будет записывать информацию об интересующих нас событиях. Конкретные объекты наблюдения описываются при помощи создания *спецификаций аудита*, которые мы определяем после создания одного или нескольких объектов аудита.

Объект аудита может собирать отслеживаемые события либо в файловую систему, либо в журналы Application или Security. Желаемое место назначения указывается после ключевого слова `TO` инструкции `CREATE SERVER AUDIT`. Например, следующая инструкция создает объект аудита с именем `MyFileAudit`, записывающий все отслеживаемые события (которые будут связаны с данным объектом аудита) в файлы, создаваемые сервером SQL Server в каталоге `C:\SqlAudits` (который должен уже существовать, в противном случае выполнение инструкции закончится ошибкой):

```
USE master
```

```
GO
```

```
CREATE SERVER AUDIT MyFileAudit TO FILE (FILEPATH='C:\SqlAudits')
```

Обратите внимание, что перед созданием объекта аудита необходимо переключиться на базу данных `master`. Если перед выполнением этой инструкции DDL вы не переключитесь с базы данных пользователя на базу данных `master`, то SQL Server выдаст следующую ошибку:

```
Msg 33074, Level 16, State 1, Line 1
```

```
Cannot create a server audit from a user database. This operation must be performed in the master database.
```

При создании объекта аудита он не будет отслеживать никакие события до тех пор, пока не будет явным образом активирован. Невозможно создать и активировать аудит одним шагом при помощи `CREATE SERVER AUDIT`, поэтому после создания аудита следующим шагом будет его активация при помо-

щи инструкции `ALTER SERVER AUDIT`. Следующая инструкция включает объект аудита `MyFileAudit`, который мы только что создали:

```
ALTER SERVER AUDIT MyFileAudit WITH (STATE=ON)
```

Как и при первоначальном создании объекта аудита, перед выполнением инструкции `ALTER SERVER AUDIT` вы опять должны переключиться к базе данных `master` (что в данном случае не нужно, поскольку мы больше не переключались с базы данных `master` после создания объекта аудита).

Инструкцию `ALTER SERVER AUDIT` можно также использовать с выражением `MODIFY NAME` для переименования объекта аудита. Перед переименованием объект аудита следует деактивировать. Например, следующие инструкции переименовывают объект аудита `MyFileAudit` в `SqlFileAudit`:

```
ALTER SERVER AUDIT MyFileAudit WITH (STATE=OFF)
ALTER SERVER AUDIT MyFileAudit MODIFY NAME = SqlFileAudit
ALTER SERVER AUDIT SqlFileAudit WITH (STATE=ON)
```

После создания объекта аудита вы можете определить одну или несколько спецификаций (для отслеживания конкретных событий) и связать эти спецификации с объектом аудита. Отслеживаемые события, собираемые всеми связанными с объектом аудита спецификациями, записываются в назначение (определенное этим объектом аудита). Скоро мы поговорим о спецификациях аудита, но сначала давайте обсудим некоторые более общие опции аудита.

## Опции аудита

Для ваших объектов аудита вы можете указать несколько важных опций. Эти опции (указанные после ключевого слова `WITH` в инструкциях `CREATE SERVER AUDIT` или `ALTER SERVER AUDIT`, а в некоторых случаях — и в обеих инструкциях) не зависят от назначения аудита (то есть они поддерживаются как при записи в файловую систему, так и при записи в журнал событий).

### Опция **QUEUE\_DELAY**

Опция `QUEUE_DELAY` управляет синхронностью/асинхронностью работы аудита. Значение 0 приведет к синхронному аудиту. В противном случае в этой опции указывается любое целое значение от 1000 и выше (для реализации асинхронного аудита — с целью повышения его производительности). Целое значение этой опции указывает максимальное количество времени (в миллисекундах), которое может пройти до того момента, когда в фоновом процессе начнется обработка действий аудита. Значение по умолчанию 1000 приводит

к асинхронной обработке, при которой аудит отслеживаемых событий происходит не позднее 1 секунды с момента их возникновения.

Настройку `QUEUE_DELAY` можно указать и при создании объекта аудита, и позже ее можно при необходимости изменить. Для изменения этой настройки у выполняющегося объекта аудита вам сначала придется деактивировать объект аудита, затем выполнить изменение, а потом опять его активировать.

Следующие инструкции увеличивают временной интервал асинхронной обработки вашего объекта аудита `MyFileAudit` (указывают для опции `QUEUE_DELAY` значение 60 000 миллисекунд, то есть 1 минуту). При выполнении этого изменения объект аудита временно деактивируется.

```
ALTER SERVER AUDIT MyFileAudit WITH (STATE=OFF)
ALTER SERVER AUDIT MyFileAudit WITH (QUEUE_DELAY=60000)
ALTER SERVER AUDIT MyFileAudit WITH (STATE=ON)
```

## Опция **ON\_FAILURE**

Вы можете использовать опцию `ON_FAILURE` для того, чтобы указать, что должен предпринять SQL Server, когда при записи отслеживаемых событий происходит ошибка. Допустимыми значениями этой опции являются `CONTINUE` (по умолчанию) или `SHUTDOWN` (необходимо наличие у регистрационной записи разрешения `SHUTDOWN`). Эту опцию можно указать тогда, когда объект аудита создается, и можно при необходимости позднее изменить. Так же, как и в случае с `QUEUE_DELAY`, выполняющийся объект аудита при внесении такого изменения должен быть временно деактивирован.

Использование `ON_FAILURE=SHUTDOWN` гарантирует, что экземпляр SQL Server полностью прекратит свою работу, если не сможет делать аудит безопасности. Например, следующие инструкции указывают работающему экземпляру SQL Server о необходимости завершить работу в случае ошибки при записи в объект аудита `MyFileObject`.

```
ALTER SERVER AUDIT MyFileAudit WITH (STATE=OFF)
ALTER SERVER AUDIT MyFileAudit WITH (ON_FAILURE=SHUTDOWN)
ALTER SERVER AUDIT MyFileAudit WITH (STATE=ON)
```

## Опция **AUDIT\_GUID**

По умолчанию каждому аудиту присваивается автоматически генерируемый глобально уникальный идентификатор (*globally unique identifier*, GUID). При использовании зеркалирования вам необходимо присвоить конкретный

GUID, который совпадает с содержащимся в зеркальной базе данных GUID. Опция `AUDIT_GUID` позволяет вам это сделать. После создания объекта аудита его GUID изменить нельзя.

## Опция **STATE**

Опция `STATE` допустима только с инструкцией `ALTER SERVER AUDIT`. Она используется для включения или выключения объекта аудита при помощи ключевых слов `ON` и `OFF`. (Вспомним, что создать объект аудита во включенном состоянии нельзя). Как уже демонстрировалось ранее в случае с опцией `QUEUE_DELAY`, опцию `STATE` в инструкции `ALTER SERVER AUDIT` с другими опциями аудита сочетать нельзя.

Когда работающий аудит выключается при помощи `STATE=OFF`, то создается элемент аудита, в котором указываются событие остановки аудита, время остановки и остановивший аудит пользователь.

## Запись аудита в файловую систему

Выражение `TO FILE` используется для записи аудитов в файловую систему (как мы только что указали для нашего объекта аудита `MyFileAudit`). Когда вы делаете аудит в файловую систему, вы можете указать несколько описываемых здесь файловых опций.

## Опция **FILEPATH**

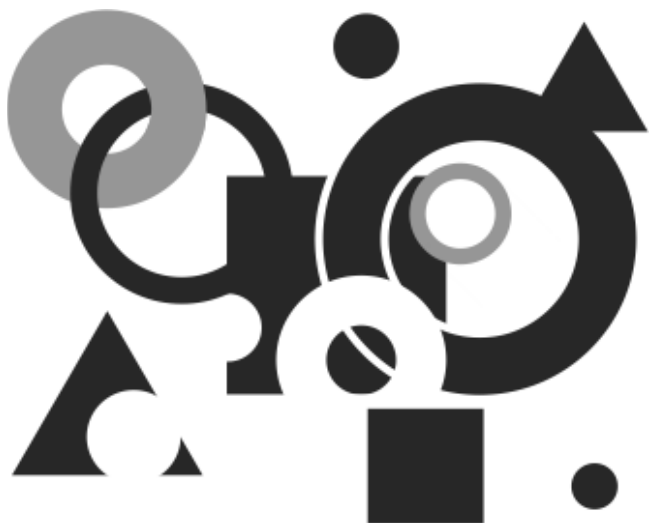
Используйте опцию `FILEPATH`, чтобы указать то место файловой системы, где SQL Server должен создавать файлы записи отслеживаемых событий. Это может быть как локальный путь, так и удаленное местоположение с использованием пути типа Universal Naming Convention (UNC) к разделяемому сетевому ресурсу. Указанный в данном пути каталог должен существовать (иначе произойдет ошибка). Более того, вы должны убедиться, что на каждый используемый вами каталог предоставлены соответствующие разрешения (особенно для разделяемых сетевых ресурсов). Вы не можете управлять именами создаваемых аудитом SQL Server Audit файлов. Эти имена генерируются автоматически на основе имени аудита и идентификатора GUID аудита.

## Резюме

Безопасность — это одна из самых важных забот любого проекта, и время, потраченное на изучение блокировки SQL Server и вашего приложения, в конечном итоге оправдается. Компания Microsoft провела определенную работу по повышению безопасности SQL Server 2008 в состоянии поставки (были отключены такие функциональные возможности, как `xp_cmdshell`, служба SQL Server Agent и многие другие необязательные функции продукта). Он получил также расширенные функции безопасности (например, возможность легкого шифрования данных в покое внутри базы данных и при передаче, более детальные разрешения и легкое переключение контекста).

Эта глава дала обзор некоторых основных концепций безопасности SQL Server. Вы узнали о регистрационных записях, пользователях, ролях и схемах, а также об аутентификации и авторизации. Затем мы перешли к описанию новых расширенных функций безопасности SQL Server 2008, таких как шифрование и аудит. Многие интерактивные ресурсы (такие как документация SQL Server Books Online) описывают эти темы более подробно. Когда вы программируете свое новое приложение, всегда думайте о том, как злоумышленники могут попытаться получить доступ к вашим данным или приложению. В сегодняшнем мире недостаточно простого замка на двери, который поддерживает честность и так честных людей. Каждый из нас — от профессионала из отдела информационных технологий и до разработчика — должен быть внимательным и всегда думать о безопасности.

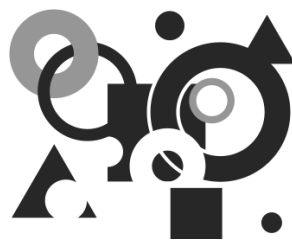




## **ЧАСТЬ II**

**ЗА ПРЕДЕЛАМИ  
РЕЛЯЦИОННОСТИ**

## ГЛАВА 6



# XML и реляционная база данных

*Стивен Форте*

С момента своего появления на мировой сцене в 1998 г. XML (Extensible Markup Language) стал чрезвычайно полезным стандартом для обмена документами и данными между различными системами. XML является форматом (производным от Standard Generalized Markup Language, SGML) для документов, которые содержат структурированную информацию.

Вы можете поинтересоваться, почему может понадобиться работать с XML и хранить его в базе данных. Не предназначены ли для этого синтаксические анализаторы XML и приложения XML? До настоящего времени XML обычно получали из таблиц баз данных и запросов, затем он программно размещался в транспортном формате (таком как физический документ или Web-служба), делался синтаксический разбор, выполнялось чтение на стороне получателя, а затем он помещался в другую базу данных.

Поборники чистоты баз данных, возможно, будут настаивать на том, что не следует хранить XML в базе данных (поскольку это механизм передачи, а не хранения). Они будут спорить, что вам следует использовать его для перемещения данных из одной базы данных (или приложения) в другую, разбирать XML при импорте и сохранять в реляционных таблицах, реконструировать при экспорте из реляционных таблиц обратно в XML для перемещения. На другом конце шкалы спорщиков находятся фанатики XML (особенно блоггеры), которые будут говорить, что базы данных вам больше не нужны, поскольку XML обеспечивает отличный механизм хранения данных с соблюдением их целостности (XML Schema, XML Schema Definition, XSD) и ядро запросов (XQuery или даже XPath). Они рассматривают весь мир как груду файлов XML, редко используют реляционные базы данных и вместо

этого применяют технологии XML для хранения и манипулирования своими данными.

Как это обычно и бывает со всеми крайними точками зрения, и те, и другие ошибаются. Каждая технология имеет свои достоинства. Реляционная база данных располагает такими функциональными возможностями, как первичные ключи, индексы и ссылочная целостность, что дает ей значительные преимущества при хранении необработанных данных и выполнении запросов к ним. Некоторые приложения (и даже сами базы данных) "нарезают" данные XML для сохранения их в базе данных и формируют XML при извлечении данных. В других случаях данные XML просто сохраняются как текст (неструктурированный) в базе данных. Когда появился SQL Server 2000, то он предложил оба этих варианта, но на сегодняшний день ни один из них не является исчерпывающим решением. Было бы гораздо лучше, если бы XML можно было бы хранить в реляционной базе данных в его собственном формате.

Итак, что же вы используете, "чисто реляционный" подход или гибридный вариант, когда вы храните XML в базе данных и работаете с ним там же? Ответ (и в большом количестве других случаев с сервером SQL Server) таков: когда как. Если вы создаете приложение с упором на транзакции (традиционно называемое системой оперативной обработки транзакций — online transaction processing, OLTP), в котором одновременно происходит множество чтений и записей, тогда самым подходящим вариантом является технология реляционной базы данных — первичные ключи, ссылочная целостность и транзакции. Если у вас есть большое хранилище данных, и вы хотите дать пользователям доступ к анализу изменений рынка и алгоритмам интеллектуального анализа данных, то вам, вероятно, следует использовать традиционную реляционную модель в сочетании с технологией оперативной аналитической обработки (online analytical processing, OLAP).

Мы не будем пытаться убедить вас в том, что надо преобразовать все ваши данные в XML только ради использования этой новой технологии, но иногда вам определенно следует рассмотреть возможность использования XML в вашей базе данных. Одна из таких ситуаций (хранить и использовать XML в реляционной базе данных) возникает тогда, когда вы преобразуете данные из базы данных OLTP в другую, "более плоскую" (менее реляционную), которая используется для простых запросов и для показа на Web-страницах. Например, у вас есть высокотранзакционная база данных реального времени со множеством пользователей и высоконормализованными данными. Эти данные также используются и в вашем Web-сайте, причем Web-сайт вполне может перенести некоторый уровень запаздывания (при обновлении данных из

приложения реального времени). Примером является обычное банковское приложение, когда для того, чтобы транзакция банкомата появилась на сайте вашего банковского счета, требуется от 2 минут до одного дня в зависимости от банка. Хорошим вариантом архитектуры системы будет регулярное, по мере необходимости для бизнеса, преобразование данных OLTP в более денормализованную (или плоскую) модель, которая используется для Web-доступа и отчетности. Ваша модель данных затем подгоняется для доступа к Web-страницам и под запросы, которые управляют содержимым страниц; вы можете также создать стратегию индексирования. В этот момент вам может понадобиться хранить некоторые данные в XML, поскольку вы можете легко преобразовать его, привязать и делать с ним на клиентской стороне все, что хотите.

В нашем опыте есть пример проектирования большого Web-сайта для потребителей. Этот сайт был предназначен для хорошо известного бренда, и трафик был огромный — больше миллиарда просмотров уникальных страниц в месяц. Мы "публиковали" версию базы данных OLTP каждые четыре часа (производственную версию превращали в плоскую Web-базу данных). Несколько таблиц были предназначены для пользовательского поиска и содержали все подробности по данным записи (наших основных бизнес-данных) в столбце `ТЕХТ` (в формате XML). Данные XML были иерархическими и содержали данные из нескольких источников, поэтому Web-разработчику было удобно представлять и хранить их в виде XML непосредственно в базе данных. Такой подход был также выгоден и потому, что нам было необходимо поддерживать множество клиентов: Netscape, Microsoft Internet Explorer, а также язык Wireless Markup Language (WML) для доступа с мобильных устройств и сотовых телефонов (еще до Microsoft ASP.NET). Наша клиентская ASP-страница преобразовывала поле XML так, что для Internet Explorer и для Netscape посылались разные коды HTML, а для мобильных клиентов создавался код WML (с теми же данными страницы).

Отрицательным результатом такого подхода были интенсивные преобразования Extensible Stylesheet Language Transformations (XSLT) на уровне клиента. SQL Server 2000 не поддерживал XML в такой мере, как мы хотели бы.

## XML в SQL Server 2000

Как уже упоминалось в предыдущем разделе, SQL Server 2000 предлагал для работы с данными XML два варианта: либо "нарезать" и формировать, либо хранить как текст. В то время это было просто революционно, но на сегодня

таких возможностей недостаточно, и вот почему. Давайте предположим, что разработчик решил хранить XML-данные в текстовом типе данных в базе данных SQL Server 2000. Первая проблема — проверка сохраняемого XML (мы имеем в виду проверку XML по схеме XSD). SQL Server 2000 не имеет таких средств проверки, поэтому XML-данные проверить можно только при помощи внешнего приложения. Это рискованное предложение, так как реальная мощь системы управления реляционными базами данных (RDBMS) состоит в применении правил на уровне сервера.

Вторая проблема — запросы к данным. Конечно, вы можете искать данные по символам и шаблонам при помощи таких функций, как `CharIndex` или `PatIndex`, но они не могут эффективно или надежно находить указанные данные в структурированном XML-документе. На Web-сайте из предыдущего примера нам пришлось для запросов поместить поисковые элементы из столбца данных XML в обычные поля SQL Server. Разработчик может также реализовать полнотекстовый поиск и индексировать текстовые данные, но это даст весьма незначительное улучшение. Будет все так же трудно извлечь данные из конкретного атрибута дочернего элемента XML-содержимого, и сам этот процесс эффективным не будет. Вы не сможете написать запрос, в котором сказано: "Покажите мне все данные, где атрибут `Author` имеет значение `John Smith`".

Третья проблема — модификация XML-данных. Разработчик может просто заменить содержимое всего XML (что не эффективно) или использовать функцию `UpdateText` для внесения изменений на месте. Однако для `UpdateText` необходимо узнать точное местонахождение и длину тех данных, которые вы хотите заменить (а как мы только что сказали, это будет сделать трудно, и процесс этот медленный).

Кроме варианта сохранения в базе данных в его исходном виде и замены по мере необходимости, сложно реализовать какое-либо другое решение для хранения XML при помощи текста. Поэтому разработчики пошли по другому пути и стали "нарезать" и формировать XML-данные при их размещении в базу данных и извлечении из нее. Для облегчения этого процесса SQL Server 2000 предлагал две функции: `OPENXML` для нарезки XML-данных и выражение `FOR XML` инструкции `SELECT` для формирования XML. И `OPENXML` является не самым эффективным механизмом нарезки XML-данных, и `FOR XML` имеет свои ограничения, которые в не очень сложных сценариях требуют использования опции `EXPLICIT`. (Любой использовавший эту опцию разработчик скажет вам, что это не слишком радостная перспектива). Следовательно, разработчикам придется нарезать и формировать XML вне базы данных и передавать в сервер SQL Server (и получать из него) уже реляционные данные.

Естественная эволюция методов сохранения XML-данных в базы данных привела к тому, что в базах данных SQL Server 2005 и 2008 можно сохранять XML в его собственном формате. Теперь SQL Server может не только сохранить XML-данные, но и проиндексировать, запрашивать их при помощи XPath или XQuery и даже эффективно модифицировать их.

## XML в SQL Server 2008 — тип данных *xml*

Для работы с XML-данными SQL Server 2005 ввел новый тип данных `xml`, а SQL Server 2008 улучшил его. При помощи этого нового типа данных вы можете сохранять XML в его собственном формате, запрашивать данные внутри XML, легко и эффективно модифицировать данные в XML (без необходимости заменять все содержимое), а также индексировать данные в XML. Тип `xml` могут иметь данные:

- ☐ переменная;
- ☐ параметр хранимой процедуры или определяемой пользователем функции (UDF);
- ☐ возвращаемое из UDF значение;
- ☐ столбец таблицы.

Есть некоторые ограничения, о которых вы должны знать. Хотя этот тип данных и может содержать неопределенные значения, так что его можно проверить на их наличие, вы не можете непосредственно сравнить экземпляры данных типа `xml` с другим экземпляром данных типа `xml`. (Однако вы можете преобразовать этот экземпляр в данные типа `TEXT` и затем произвести сравнение). Все подобные сравнения требуют предварительного преобразования типа `xml` в символьный тип. Это ограничение означает также, что вы не можете использовать `ORDER BY` или `GROUP BY` с типом данных `xml`. Есть еще несколько ограничений, которые мы более подробно обсудим позже.

Эти ограничения могут показаться очень суровыми, но они не очень влияют на тип `xml` при правильном использовании, а возможности его значительно перевешивают все эти ограничения.

## Работаем с типом *xml* как с переменной

Давайте начнем с написания кода, который использует новый тип данных `xml` как переменную. Как и любую другую переменную Transact-SQL (T-SQL), вы

просто объявляете ее и присваиваете ей данные, как в следующем примере, где блок XML используется для представления данных о торговом представителе:

```
DECLARE @xmlData AS XML
SET @xmlData= '
<Customers>
  <CustomerID>TELRK</CustomerID>
  <CompanyName>Telerik, Inc</CompanyName>
  <ContactName>Stephen Forte</ContactName>
  <ContactTitle>Sales Representative</ContactTitle>
  <Address>5-9 Union Square West</Address>
  <City>New York</City>
  <PostalCode>10028</PostalCode>
  <Country>USA</Country>
  <Phone>030-0074321</Phone>
  <Fax>030-0076545</Fax>
</Customers>'
SELECT @xmlData
```

XML-переменная объявляется, как и любой другой собственный тип данных SQL Server, при помощи инструкции `DECLARE`. Затем переменной присваивается значение. Достаточно странно то, что строка XML-данных присваивается данным типа `xml`, а тип разбирает их в XML. (Кстати, определяемые пользователем типы на базе общезыковой исполняющей среды также поддерживают эту функциональную возможность разбора, и она для них необходима). В примере также производится проверка правильного формирования XML (проверяется совпадение не только имен открывающего и закрывающего тегов элемента, но и их регистров).

Последняя инструкция возвращает XML вызывающей стороне при помощи оператора `SELECT`, и результаты выглядят как одна строка данных XML. Есть еще одно преимущество: поскольку база данных знает о том, что вы работаете с XML (а не с текстом, который является XML), то результаты XML отображаются в SQL Server Management Studio как гиперссылки. Щелчок по гиперссылке открывает новое окно, в котором отображается красиво отформатированный XML (с использованием шаблона Windows Internet Explorer XML Template) с цветовым кодированием и свертываемыми/развертываемыми узлами. Если вы затем щелкнете правой кнопкой мыши по заголовку окна с результатом XML, то вы увидите контекстное меню с некоторыми полезными опциями (например, возможность сохранить XML в файл или скопировать полный путь XML).

## Работаем с XML в таблицах

Вы можете определить столбец таблицы как XML, это показано в следующем примере:

```
USE AdventureWorks2008
```

```
GO
```

```
-- создать таблицу с типом данных XML
```

```
CREATE TABLE OrdersXML
```

```
(OrderDocID int PRIMARY KEY,
```

```
  xOrders xml NOT NULL)
```

Тип данных `xml` имеет некоторые дополнительные ограничения при использовании в качестве столбца таблицы:

- ☐ не может использоваться как первичный ключ;
- ☐ не может использоваться как внешний ключ;
- ☐ не может объявляться с ограничением `UNIQUE`;
- ☐ не может объявляться с ключевым словом `COLLATE`.

Ранее мы утверждали, что нельзя сравнивать два экземпляра данных типа `xml`. Первичные ключи, внешние ключи и ограничения уникальности — для всего этого требуется возможность сравнения типов данных. Поэтому в таких ситуациях XML использовать нельзя. Инструкция `COLLATE` в SQL Server для типа данных `xml` смысла не имеет, поскольку SQL Server не хранит XML как текст; вместо этого он использует другой тип кодирования специально для XML.

Теперь давайте поместим в столбец некоторые данные. В этом примере берется простой статический XML и вставляется в только что созданную нами таблицу `OrdersXML` (с использованием типа данных `xml` как переменной):

```
-- Вставить статический XML через переменную
```

```
DECLARE @xmlData AS XML
```

```
SET @xmlData = '
```

```
<Orders>
```

```
  <Order>
```

```
    <OrderID>5</OrderID>
```

```
    <CustomerID>65</CustomerID>
```

```
    <OrderAmount>25</OrderAmount>
```

```
  </Order>
```

```
</Orders>'
```

```
-- вставить в таблицу
```

```
INSERT INTO OrdersXML (OrderDocID, xOrders) Values (1, @xmlData)
```



Вы можете вставить XML разными способами, такими как XML Bulk Load (мы рассмотрим его позднее в этой же главе), загрузка из переменной XML (как было показано только что) или загрузка из инструкции SELECT при помощи функции FOR XML TYPE, которую мы скоро обсудим. Любая попытка вставить неправильно сформированный XML приведет к исключению:

```
-- Заканчивается ошибкой из-за неправильно сформированного XML
INSERT INTO OrdersXML (OrderDocID, xOrders) VALUES (3, '<nm>steve</NM>')
```

В результате будет получена следующая ошибка SQL Server:

```
Msg 9436, Level 16, State 1, Line 1
XML parsing: line 1, character 14, end tag does not match start tag
```

## Значения по умолчанию и ограничения

Тип данных xml может (подобно другим типам данных) соблюдать режим допустимости неопределенных значений, иметь значения по умолчанию и ограничения. Если вы хотите сделать поле необходимым (NOT NULL) и предоставить значение по умолчанию для столбца XML, то сделайте это так, как вы сделали бы для любого другого столбца:

```
CREATE TABLE OrdersXML
    (OrderDocID INT PRIMARY KEY,
    xOrders XML NOT NULL DEFAULT '<Orders/>')
```

Следующая вставка работает, поскольку она полагается на значение по умолчанию:

```
INSERT INTO OrdersXML (OrderDocID, xOrders) VALUES (2, DEFAULT)
```

Добавление значения по умолчанию не гарантирует, что не будет добавлено просто <Orders>. Например, мы указали в качестве значения по умолчанию узел Orders, но не указали никакого способа обеспечения этого. Следующая вставка работает, даже если мы хотим иметь в нашей таблице только <Orders>, поскольку мы не объявили ограничения и не предоставили схемы столбцов:

```
INSERT INTO OrdersXML (OrderDocID, xOrders)
    VALUES (3, '<blah>steve</blah>')
```

## XML-схемы

Мощной функцией XML является его способность строго типизировать данные в документе XML. XML Schema Definition (XSD) определяет стандарт-

ный набор типов данных, которые должны поддерживаться во всех документах XML. Вы можете использовать XSD для создания XML-схемы данных (требуя соответствия ваших данных тому набору правил, которые вы указываете). Это дает XML преимущество по сравнению почти со всеми прочими методами передачи/описания данных и является главным фактором успеха стандарта XML.

Без XML Schema язык XML был бы просто более современным, более подробным текстовым форматом с разделителями. XML-схема определяет то, как ваши данные должны выглядеть, какие элементы необходимы и какие типы данных эти элементы будут иметь. Аналогично тому, как определение таблицы в SQL Server обеспечивает проверку структуры и типа реляционных данных, XML-схема обеспечивает проверку структуры и типа данных XML.

Мы не будем полностью описывать здесь все функциональные возможности XML Schema — для этого потребовалась бы отдельная книга. Вы можете найти спецификации XSD в организации World Wide Web Consortium (W3C) по адресу: <http://www.w3.org/2001/XMLSchema>. В открытом доступе имеется несколько популярных схем, в том числе схема для Really Simple Syndication (RSS) (протокола, который приводит в действие блоги, блогкасты и прочие формы синдикации текста и двоичных данных) и схема SOAP (которая определяет перенос информации службами XML Web Services).

Вы можете выбрать способ структурирования XSD, создавать необходимые элементы и устанавливать ограничения на допустимые типы данных и их диапазоны. XSD может даже разрешить фрагменты документов.

## Коллекции схем сервера SQL

SQL Server 2005 позволил создавать свои собственные схемы и сохранять их в базе данных как объекты базы данных, а также применять схему для каждого экземпляра XML, в том числе и для столбцов таблиц, и для переменных SQL Server. Это дает улучшение управления кодом XML, который поступает в базу данных, и позволяет также строго типизировать ваш экземпляр XML. SQL Server 2008 расширяет эту поддержку следующими возможностями:

- ☐ сохранение часовых поясов в значениях `xsd:dateTime`
- ☐ нестрогая проверка;
- ☐ типы объединения и просмотра в схемах.

Перед тем как углубиться в эти новые улучшения, мы сделаем краткий обзор того, как схемы работают с сервером SQL Server. Для начала вы можете соз-

дать следующую простую схему и добавить ее в коллекцию `schemas` базы данных `AdventureWorks2008`, как это показано в листинге 6.1.

#### Листинг 6.1. Создание коллекции XML-схем

```
-- синтаксис для добавления схемы
-- схема может не разрешать фрагменты, если вы это укажете
CREATE XML SCHEMA COLLECTION dbo.order_xsd
AS
'<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:simpleType name="orderAmt" >
    <xsd:restriction base="xsd:int" >
      <xsd:maxExclusive value="5000" />
      <xsd:minInclusive value="1" />
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:element name="Order">
<xsd:complexType>
  <xsd:sequence>
    <xsd:element name="CustomerName" type="xsd:string" />
    <xsd:element name="OrderDate" type="xsd:dateTime"/>
    <xsd:element name="OrderAmt" type="orderAmt"/>
  </xsd:sequence>
</xsd:complexType>
  </xsd:element>
</xsd:schema>'
```

GO

Схему `order_xsd` вы можете использовать для любого типа `xml`, в том числе переменных, возвращаемых значений и столбцов таблиц. Эта схема определяет элементы с именами `CustomerName`, `OrderDate` и `OrderAmt` с типом `orderAmt`, определенным как тип данных `int`, минимальное значение которого равно 1, а максимальное — 4999. (Мы знаем, что вы, скорее всего, не будете моделировать свои данные таким образом, но вам может понадобиться преобразовать реляционные данные в XML для выставления счетов или для их отображения; при этом ограничение типа в XSD будет очень ценным). Затем создайте простую таблицу и примените схему к столбцу XML при помощи ссылки на имя схемы (в круглых скобках) после типа данных `xml` в инструкции `CREATE TABLE`, как это показано в листинге 6.2.

**Листинг 6.2. Создание таблицы со схемой**

```
-- создать таблицу со столбцом xml и использовать схему как правило
-- для определенного столбца
CREATE TABLE xml_schematest (
    Order_ID int PRIMARY KEY,
    Order_XML xml (order_xsd) --XML Schema Name
)
GO
```

Как видно из этого примера, столбец `Order_XML` определен не просто как `xml`, но как `xml (order_xsd)`. Тип данных `xml` имеет необязательный параметр, который позволяет вам указать привязанную схему. Его можно использовать и в том случае, когда вы хотите привязать схему к другому использованию типа `xml` (в качестве переменной или параметра). SQL Server теперь разрешает в столбце `Order_XML` только строго типизированный XML-документ. Это гораздо лучше, чем ограничение `СНЕСК` (которое вы все равно можете добавить к этому столбцу, но только при помощи функции). Преимущество использования XML-схемы состоит в том, что вы можете обеспечить типы данных XML (на уровне XML) и попадание в конкретные элементы только правильных XML-данных. Если бы вы использовали ограничение `СНЕСК`, понадобилось бы отдельное ограничение `СНЕСК` для каждой выполняемой проверки. В этом примере (если бы не было созданной в листинге 6.1 схемы) понадобилось бы несколько ограничений `СНЕСК` только для того, чтобы обеспечить минимальное и максимальное значения. Вам понадобилось бы одно ограничение для элемента, затем второе для проверки нижнего конца диапазона (1) и еще одно для проверки верхнего конца допустимого диапазона (4999).

**ПРИМЕЧАНИЕ**

Когда вы создаете столбец `xml`, то все определяемые вами значения по умолчанию не должны конфликтовать с привязанной схемой; в противном случае значение по умолчанию будет всегда выдавать ошибку. Убедитесь также в том, что все определяемые ограничения не конфликтуют с привязанной схемой; в противном случае все попытки вставить или обновить данные будут заканчиваться ошибкой.

Давайте взглянем на схему в действии:

```
-- Это будет работать нормально
INSERT INTO xml_schematest VALUES (1, '
<Order>
```

```
<CustomerName>Bill Gates</CustomerName>
<OrderDate>2008-10-10T14:22:27.25-05:00</OrderDate>
<OrderAmt>100</OrderAmt>
</Order>')
-- Здесь будет ошибка
INSERT INTO xml_schematest VALUES (2, '
<Order>
  <CustomerName>Steve Ballmer</CustomerName>
  <OrderDate>2008-10-10T14:22:27.25-05:00</OrderDate>
  <OrderAmt>10000</OrderAmt>
</Order>')
```

SQL Server обеспечивает реализацию требований схемы при вставках и обновлениях, гарантируя целостность данных. Предоставленные нами данные соответствуют схеме, поэтому вставка работает нормально. Если вы попытаетесь вставить или обновить неверный элемент данных, то вы получите ошибку:

```
UPDATE xml_schematest
SET Order_XML='
<Order>
  <CustomerName>Bill Gates</CustomerName>
  <OrderDate>2008-10-10T14:22:27.25-05:00</OrderDate>
  <OrderAmt>10000</OrderAmt>
</Order>'
WHERE Order_ID=1
```

SQL Server выдаст следующую ошибку:

```
Msg 6926, Level 16, State 1, Line 1
XML Validation: Invalid simple type value: '10000'.
      Location: /*:Order[1]/*:OrderAmt[1]
```

## Улучшения *xsd:dateTime*

Тип данных *xsd:dateTime* в SQL Server 2008 "знает" о часовых поясах. В SQL Server 2005 для данных *dateTime* вам необходимо было сообщать часовой пояс. SQL Server 2005 не сохранял информацию о часовом поясе для данных типа *dateTime* или *time*, он нормализовал их в стандартное время по Гринвичу (Coordinated Universal Time, UTC). Вернемся к нашему примеру: мы добавили дату и время 2008-10-10T14:22:27.25-05:00 для момента времени 2:22PM в восточном часовом поясе США (-5:00 — это время по Гринвичу (Greenwich Mean Time [GMT]) минус 5 часов). Взгляните на XML из базы

данных. SQL Server 2005 нормализует информацию о часовом поясе, и получается 2008-10-10T19:22:27.250Z, причем обратите внимание на Z для указания UTC (Zulu), которое отличается на -5 часов.

```
SELECT * FROM xml_schematest  
<Order>  
  <CustomerName>Bill Gates</CustomerName>  
  <OrderDate>2008-10-10T19:22:27.250Z</OrderDate>  
  <OrderAmt>100</OrderAmt>  
</Order>
```

Выполните эту операцию в SQL Server 2008, и он автоматически сохранит для вас данные по часовому поясу. Те же самые данные будут выглядеть так:

```
<Order>  
  <CustomerName>Bill Gates</CustomerName>  
  <OrderDate>2008-10-10T14:22:27.25-05:00</OrderDate>  
  <OrderAmt>100</OrderAmt>  
</Order>
```

## Нестрогая проверка

SQL Server 2008 добавил в XSD поддержку lax validation. Для описания нестрогой проверки предположим, что мы хотим добавить в наш XML предыдущего примера еще один элемент (после <OrderAmt>), который не является частью той же самой схемы. В SQL Server 2005 схемы могут использовать в качестве групповых символов значения skip и strict атрибута processContents для объявлений any и anyAttribute. (Если вы не знакомы с этими атрибутами и значениями схемы, то знайте, что они используются для указания, как синтаксический анализатор XML должен обращаться с теми элементами XML, которые отсутствуют в схеме). Если в processContents установлено значение skip, то SQL Server полностью пропустит проверку дополнительного элемента (даже если для него есть схема). Если в processContents установлено значение strict, то SQL Server потребует наличия в текущей схеме элемента или пространства имен, по которому будет проверяться элемент. В результате получается схема "все или ничего". SQL Server 2008 добавил поддержку дополнительной промежуточной опции проверки lax. Установив значение атрибута processContents для данного раздела в значение lax, вы сможете обеспечить проверку для всех элементов, имеющих связанную с ними схему, и игнорировать любые элементы, которые не определены в схеме. Например, показанную в листинге 6.3 схему используем в SQL Server 2008. (Если вы выполняете наши упражнения и доба-

вили предыдущую схему из листинга 6.1, вам придется удалить ее перед тем, как добавить ту, которую вы видите в листинге 6.3).

**Листинг 6.3. Схема со слабой проверкой и dateTime**

```
-- Правильная схема SQL Server 2008 (слабая проверка и dateTime)
CREATE XML SCHEMA COLLECTION dbo.order_xsd
AS '
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:simpleType name="orderAmt" >
    <xsd:restriction base="xsd:int" >
      <xsd:maxInclusive value="5000" />
      <xsd:minExclusive value="1" />
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:element name="Order">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="CustomerName" type="xsd:string" />
        <xsd:element name="OrderDate" type="xsd:dateTime"/>
        <xsd:element name="OrderAmt" type="orderAmt"/>
        <xsd:any namespace="##other" processContents="lax"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>'
GO
```

После добавления этой схемы будет разрешено сохранение произвольных XML-элементов следом за <OrderAmt>, причем без ошибки проверки. Для того чтобы увидеть это в действии, обратите внимание на случайный элемент <Notes> в предыдущем примере. Элемент <Notes> (произвольный) не только будет разрешен сервером SQL Server 2008, но он может быть также целой новой структурой элементов, которая поддерживается своей дополнительной схемой (как указано атрибутом xmlns).

```
DECLARE @xml XML(dbo.order_xsd)
SET @xml = '
<Order>
  <CustomerName>Bill Gates</CustomerName>
```

```

<OrderDate>2008-10-10T14:22:27.25-05:00</OrderDate>
<OrderAmt>100</OrderAmt>
<Notes xmlns="sf">My Notes for this order</Notes>
</Order>'
SELECT @xml

```

## Типы объединения и списков

В SQL Server 2005 вы могли определить простой тип `xsd:list` с перечислением, чтобы потребовать элементы списка для любого объекта, который унаследует этот тип. В SQL Server 2008 при помощи `xsd:union` вы можете комбинировать множество списков в один простой тип. Например, в схеме листинга 6.4 мы разрешаем в списке `shiptypeList` строки (такие как `FastShippers`), а также и числовые значения:

### Листинг 6.4. Типы `union` и `list`

```

-- Типы union и list
DROP XML SCHEMA COLLECTION dbo.order_xsd
GO
CREATE XML SCHEMA COLLECTION dbo.order_xsd AS
'<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:simpleType name="orderAmt" >
    <xsd:restriction base="xsd:int" >
      <xsd:maxInclusive value="5000" />
      <xsd:minExclusive value="1" />
    </xsd:restriction>
  </xsd:simpleType>
  <xsd:simpleType name="shiptypeList">
    <xsd:union>
      <xsd:simpleType>
        <xsd:list>
          <xsd:simpleType>
            <xsd:restriction base="xsd:integer">
              <xsd:enumeration value="1"/>
              <xsd:enumeration value="2"/>
              <xsd:enumeration value="3"/>
            </xsd:restriction>
          </xsd:simpleType>
        </xsd:list>
      </xsd:simpleType>
    </xsd:union>
  </xsd:simpleType>
</xsd:schema>'

```



```

</xsd:simpleType>
<xsd:simpleType>
  <xsd:list>
    <xsd:simpleType>
      <xsd:restriction base="xsd:string">
        <xsd:enumeration value="FastShippers"/>
        <xsd:enumeration value="SHL"/>
        <xsd:enumeration value="PSU"/>
      </xsd:restriction>
    </xsd:simpleType>
  </xsd:list>
</xsd:simpleType>
</xsd:union>
</xsd:simpleType>
<xsd:element name="Order">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="CustomerName" type="xsd:string" />
      <xsd:element name="OrderDate" type="xsd:dateTime"/>
      <xsd:element name="OrderAmt" type="orderAmt"/>
      <xsd:element name="ShipType" type="shiptypeList"/>
      <xsd:any namespace="##other" processContents="lax"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
</xsd:schema>'
GO

```

Если вы теперь проверите экземпляр XML числовым значением или строковым значением из списка перечислимых значений, то проверка произойдет следующим образом:

```

-- работает с 1 или с FastShippers из ShipType
DECLARE @xml XML(dbo.order_xsd)
SET @xml = '
<Order>
  <CustomerName>Bill Gates</CustomerName>
  <OrderDate>2008-10-10T14:22:27.25-05:00</OrderDate>
  <OrderAmt>100</OrderAmt>
  <ShipType>1</ShipType>

```

```
<Notes xmlns="sf">Steve Test 1</Notes>
</Order>'
GO
```

Этот пример очень прост, но он полезен в том случае, когда у вас есть больше одного способа описания чего-либо, и вам нужно для этого два списка. Один из таких случаев — это метрические и английские единицы измерения. Этот способ полезен тогда, когда вам нужно ограничить элементы, и вы пишете их из базы данных.

Мы затронули только самые основы использования XML-схем в SQL Server 2008. Эти схемы могут быть очень сложными, и дальнейшее их обсуждение выходит за рамки данной книги. По мере освоения синтаксиса вы сможете реализовать в своей базе данных сложные XML-схемы. Мы считаем, что для обеспечения непротиворечивости ваших XML-данных вам следует всегда использовать XML-схему.

## XML-индексы

Вы можете создать XML-индекс по столбцу XML подобно стандартному индексу SQL Server. Существуют четыре типа XML-индексов: один *первичный XML-индекс* (должен быть создан) и три типа необязательных вторичных XML-индексов (которые создаются по первичному индексу). Однако XML-индекс — это кластеризованный индекс по внутренней таблице, используемой сервером SQL Server для хранения XML-данных. Эта таблица называется *таблицей узлов*, и программисты доступ к ней получить не могут.

Для того чтобы начать работать с XML-индексом, вы должны сначала создать первичный индекс всех узлов. Первичный индекс — это кластеризованный индекс (по таблице узлов, а не по базовой таблице), который связывает каждый узел вашего XML-столбца со столбцом SQL Primary Key. Он делает это путем индексирования одной строки в ее внутреннем представлении (древовидная структура B+) для каждого узла в XML-столбце, генерируя при этом индекс, который обычно в три раза больше, чем ваши XML-данные. Для того чтобы XML-данные работали правильно, ваша таблица должна определить столбец обычного кластеризованного первичного ключа (Primary Key). Этот первичный ключ используется для соединения результатов XQuery с базовой таблицей (XQuery обсуждается далее в этой же главе). Для создания первичного XML-индекса вы сначала создаете таблицу с первичным ключом и столбцом XML (листинг 6.5). Затем мы вставим в нашу новую таблицу данные и научимся создавать запросы XQuery.

**Листинг 6.5. Создание XML-индекса**

```
-- Примеры XML-индекса
-- Удалить таблицу, поскольку мы использовали ее раньше
DROP TABLE OrdersXML
-- Создать таблицу с первичным ключом и столбцом XML
CREATE TABLE OrdersXML
  (OrderDocID int PRIMARY KEY,
   xOrders xml NOT NULL)
-- Вставить в нашу таблицу некие данные
INSERT INTO OrdersXML VALUES (1, '
<Orders>
  <Order>
    <OrderID>5</OrderID>
    <CustomerName>Stephen Forte</CustomerName>
    <OrderAmount>25</OrderAmount>
  </Order>
</Orders>')
INSERT INTO OrdersXML VALUES (2, '
<Orders>
  <Order>
    <OrderID>7</OrderID>
    <CustomerName>Andrew Brust</CustomerName>
    <OrderAmount>45</OrderAmount>
  </Order>
</Orders>')
INSERT INTO OrdersXML VALUES (3, '
<Orders>
  <Order>
    <OrderID>2</OrderID>
    <CustomerName>Bill Zack</CustomerName>
    <OrderAmount>65</OrderAmount>
  </Order>
</Orders>')
```

Для создания первичного XML-индекса мы используем синтаксис `CREATE PRIMARY XML INDEX`:

```
CREATE PRIMARY XML INDEX idx_1
  ON OrdersXML (xOrders)
```

Мы успешно создали новый первичный XML-индекс с названием `idx_1` по столбцу `xOrders` таблицы `OrdersXML`. Первичный XML-индекс `idx_1` теперь заполнил таблицу узлов. Если вы хотите взглянуть на столбцы таблицы узлов, выполните этот код T-SQL:

```
-- показать столбцы таблицы узлов
SELECT col.column_id, col.object_id, col.name, col.system_type_id
FROM sys.columns AS col
    INNER JOIN sys.indexes AS idx
    ON idx.object_id = col.object_id
WHERE idx.name = 'idx_1' AND idx.type = 1
ORDER BY column_id
```

Результаты показаны в табл. 6.1.

**Таблица 6.1.** Столбцы типичной таблицы узлов

| column_id | object_id | name      | system_type_id |
|-----------|-----------|-----------|----------------|
| 1         | 855674096 | id        | 165            |
| 2         | 855674096 | nid       | 56             |
| 3         | 855674096 | tagname   | 231            |
| 4         | 855674096 | taguri    | 231            |
| 5         | 855674096 | tid       | 56             |
| 6         | 855674096 | value     | 231            |
| 7         | 855674096 | lvalue    | 231            |
| 8         | 855674096 | lvaluebin | 165            |
| 9         | 855674096 | hid       | 167            |
| 10        | 855674096 | xsinil    | 104            |
| 11        | 855674096 | xsitype   | 104            |
| 12        | 855674096 | pk1       | 56             |

Три типа вторичных XML-индексов — это `path`, `value` и `property`. Вы можете реализовать вторичный XML-индекс только после того, как создали первичный XML-индекс, поскольку фактически они оба являются индексами таблицы узлов. Эти индексы оптимизируют инструкции XQuery, которые выполняются по XML-данным.

Индекс `path` создается по столбцам `Path ID (hid)` в табл. 6.1) и `Value` первичного XML-индекса (при помощи ключевого слова `FOR PATH`). Такой тип индекса лучше всего для тех случаев, когда у вас есть очень сложный документ, и вы хотите ускорить выражения `XPath` в `XQuery`, которые ссылаются на определенный узел ваших XML-данных явным образом (далее в этой же главе). Если вас больше беспокоят значения узлов, запрашиваемых групповыми символами, можете создать индекс значений при помощи XML-индекса `FOR VALUE`. Индекс `VALUE` содержит те же самые столбцы индекса, что и индексы `PATH`, `Value` и `Path ID (hid)`, но в обратном порядке (нежели в табл. 6.1). Индекс свойств с ключевым словом `PROPERTY` оптимизирует иерархии элементов или атрибуты, которые являются парами имя/значение. Индекс `PROPERTY` содержит первичный ключ базовой таблицы, `Path ID (hid)` и `Value`, причем именно в этом порядке. Синтаксис для создания этих индексов показан в листинге 6.6; вы должны указать, что вы используете первичный XML-индекс (при помощи синтаксиса `USING XML INDEX`).

#### Листинг 6.6. XML-индекс, созданный по первичному индексу

```
-- структурный (Path)
CREATE XML INDEX idx_a ON OrdersXML (xOrders)
    USING XML INDEX idx_1 FOR PATH
-- value
CREATE XML INDEX idx_b ON OrdersXML (xOrders)
    USING XML INDEX idx_1 FOR VALUE
-- property
CREATE XML INDEX idx_c ON OrdersXML (xOrders)
    USING XML INDEX idx_1 FOR PROPERTY
```

Учтите следующие дополнительные ограничения:

- ❑ XML-индекс может содержать только один столбец XML, поэтому вы не можете создать составной XML-индекс (индекс по более чем одному столбцу XML).
- ❑ Использование XML-индексов требует, чтобы первичный ключ был кластеризованным, и поскольку у вас может быть только один кластеризованный индекс на таблицу, то вы не можете создать кластеризованный XML-индекс.

SQL Server 2008 Management Studio позволяет вам просматривать индексы таблиц путем углубления в узле **Database** до имени таблицы и далее до узла **Indexes** в окне **Object Explorer**. Вы можете также создать новый первичный

XML-индекс при помощи щелчка правой кнопкой мыши по узлу **Indexes** и последующего выбора опции **New Index**.

Вооружившись этим новым типом индекса, вы можете написать при помощи XQuery некоторые очень эффективные запросы. Однако перед тем как обратиться к XQuery, давайте взглянем на некоторые другие функциональные возможности XML, которые помогут вам доставить XML-данные в базу данных и получить их оттуда.

## Команды **FOR XML**

SQL Server 2000 ввел улучшение синтаксиса T-SQL, которое позволяет нормальным реляционным запросам выводить их результирующий набор как XML при помощи одного из следующих трех средств:

- ☐ FOR XML RAW;
- ☐ FOR XML AUTO;
- ☐ FOR XML EXPLICIT.

Эти три функции поддерживаются также в SQL Server 2005 и 2008. Сначала мы обсудим те функциональные возможности, которые являются общими для всех трех версий, а затем подробно рассмотрим новые и улучшенные функциональные возможности, имеющиеся в SQL Server 2008.

## Синтаксис **FOR XML RAW**

Оператор FOR XML RAW вырабатывает XML на основе атрибутов. Он по существу создает плоское представление данных, в котором каждая возвращаемая строка становится элементом, а возвращаемые столбцы становятся атрибутами элементов. Оператор FOR XML RAW не интерпретирует соединений каким-либо особенным образом. (Соединения становятся актуальными в FOR XML AUTO). Вот пример простого запроса, который извлекает данные по клиенту и заголовку заказа:

```
SELECT TOP 10 — ограничивает количество строк результата для демонстрации
    Customer.CustomerID, OrderHeader.SalesOrderID, OrderHeader.OrderDate
FROM Sales.Customer Customer
    INNER JOIN Sales.SalesOrderHeader OrderHeader
    ON OrderHeader.CustomerID = Customer.CustomerID
```

```
ORDER BY Customer.CustomerID
```

```
FOR XML RAW
```

Если вы используете SQL Server 2000, то выведется поток текста в окно **Query Analyzer**. Однако в SQL Server 2005 и 2008 вы можете щелкнуть по XML-гиперссылке в возвращенных результатах и увидеть показанные здесь выходные данные:

```
<row CustomerID="11001" SalesOrderID="43860" OrderDate="2001-08-01T00:00:00" />
<row CustomerID="11001" SalesOrderID="44501" OrderDate="2001-11-01T00:00:00" />
<row CustomerID="11001" SalesOrderID="45283" OrderDate="2002-02-01T00:00:00" />
<row CustomerID="11001" SalesOrderID="46042" OrderDate="2002-05-01T00:00:00" />
<row CustomerID="11002" SalesOrderID="46976" OrderDate="2002-08-01T00:00:00" />
<row CustomerID="11002" SalesOrderID="47997" OrderDate="2002-11-01T00:00:00" />
<row CustomerID="11002" SalesOrderID="49054" OrderDate="2003-02-01T00:00:00" />
<row CustomerID="11002" SalesOrderID="50216" OrderDate="2003-05-01T00:00:00" />
<row CustomerID="11002" SalesOrderID="51728" OrderDate="2003-08-01T00:00:00" />
<row CustomerID="11002" SalesOrderID="57044" OrderDate="2003-11-01T00:00:00" />
```

Как видите, каждая строка, возвращенная из запроса, становится элементом с именем `row`, и все столбцы выводятся как атрибуты этого элемента. Однако, вероятнее всего, вам захочется получить более структурированный XML-вывод, что приводит нас к синтаксису `FOR XML AUTO`.

## Синтаксис *FOR XML AUTO*

Оператор `FOR XML AUTO` также вырабатывает XML на основе атрибутов (по умолчанию), но его вывод иерархический, то есть он может создавать вложенные результаты в зависимости от таблиц в выражении соединения в запросе. Например, используя только что продемонстрированный запрос, вы можете просто изменить выражение `FOR XML` на `FOR XML AUTO`:

```
SELECT TOP 10 --ограничивает количество строк результата для демонстрации
    Customer.CustomerID, OrderHeader.SalesOrderID, OrderHeader.OrderDate
FROM Sales.Customer Customer
    INNER JOIN Sales.SalesOrderHeader OrderHeader
    ON OrderHeader.CustomerID = Customer.CustomerID
ORDER BY Customer.CustomerID
FOR XML AUTO
```

Выполните этот запрос, щелкните в результатах по XML-гиперссылке, и вы увидите следующие выходные данные:

```
<Customer CustomerID="1">
    <OrderHeader SalesOrderID="43860" OrderDate="2001-08-01T00:00:00" />
```

```

<OrderHeader SalesOrderID="44501" OrderDate="2001-11-01T00:00:00" />
<OrderHeader SalesOrderID="45283" OrderDate="2002-02-01T00:00:00" />
<OrderHeader SalesOrderID="46042" OrderDate="2002-05-01T00:00:00" />
</Customer>
<Customer CustomerID="2">
  <OrderHeader SalesOrderID="46976" OrderDate="2002-08-01T00:00:00" />
  <OrderHeader SalesOrderID="47997" OrderDate="2002-11-01T00:00:00" />
  <OrderHeader SalesOrderID="49054" OrderDate="2003-02-01T00:00:00" />
  <OrderHeader SalesOrderID="50216" OrderDate="2003-05-01T00:00:00" />
  <OrderHeader SalesOrderID="51728" OrderDate="2003-08-01T00:00:00" />
  <OrderHeader SalesOrderID="57044" OrderDate="2003-11-01T00:00:00" />
</Customer>

```

Как видите, XML-данные имеют главные элементы с названием `Customer` (в соответствии с присвоенным в запросе псевдонимом) и дочерние элементы с названием `OrderHeader` (опять-таки по псевдониму). Обратите внимание, что FOR XML AUTO определяет порядок вложения элементов в зависимости от порядка столбцов в выражении SELECT. Вы можете переписать выражение SELECT таким образом, чтобы столбец `OrderHeader` шел перед столбцом `Customer`, как это показано здесь:

```

SELECT TOP 10 --ограничивает количество строк результата для демонстрации
  OrderHeader.SalesOrderID, OrderHeader.OrderDate, Customer.CustomerID

```

Вывод (в средстве просмотра XML) теперь выглядит таким образом:

```

<OrderHeader SalesOrderID="43860" OrderDate="2001-08-01T00:00:00">
  <Customer CustomerID="1" />
</OrderHeader>
<OrderHeader SalesOrderID="44501" OrderDate="2001-11-01T00:00:00">
  <Customer CustomerID="1" />
</OrderHeader>
<OrderHeader SalesOrderID="45283" OrderDate="2002-02-01T00:00:00">
  <Customer CustomerID="1" />
</OrderHeader>
<OrderHeader SalesOrderID="46042" OrderDate="2002-05-01T00:00:00">
  <Customer CustomerID="1" />
</OrderHeader>
<OrderHeader SalesOrderID="46976" OrderDate="2002-08-01T00:00:00">
  <Customer CustomerID="2" />
</OrderHeader>
<OrderHeader SalesOrderID="47997" OrderDate="2002-11-01T00:00:00">
  <Customer CustomerID="2" />

```



```
</OrderHeader>
<OrderHeader SalesOrderID="49054" OrderDate="2003-02-01T00:00:00">
  <Customer CustomerID="2" />
</OrderHeader>
<OrderHeader SalesOrderID="50216" OrderDate="2003-05-01T00:00:00">
  <Customer CustomerID="2" />
</OrderHeader>
<OrderHeader SalesOrderID="51728" OrderDate="2003-08-01T00:00:00">
  <Customer CustomerID="2" />
</OrderHeader>
<OrderHeader SalesOrderID="57044" OrderDate="2003-11-01T00:00:00">
  <Customer CustomerID="2" />
</OrderHeader>
```

Вероятно, это не то, чего вы хотели. Для того чтобы XML-иерархия совпадала с иерархией таблицы, вы должны вывести как минимум один столбец из родительской таблицы перед тем, как вывести любой столбец из дочерней таблицы. Если есть три уровня таблиц, то, по меньшей мере, еще один столбец из дочерней таблицы должен предшествовать столбцу из таблицы третьего уровня (и т. д.).

## Синтаксис *FOR XML EXPLICIT*

Оператор `FOR XML EXPLICIT` — самый сложный, но самый полезный и гибкий из трех вариантов `FOR XML`. Он вырабатывает XML при помощи создания запроса `UNION` различных уровней выходных элементов. Если у вас есть таблицы `Customer` и `SalesOrderHeader` и вы хотите выработать XML-вывод, то у вас должно быть две инструкции `SELECT` и одна `UNION`. Если вы добавите таблицу `SalesOrderDetail`, то вы должны будете добавить еще одну инструкцию `UNION` и инструкцию `SELECT`.

Как мы уже сказали, запрос `FOR XML EXPLICIT` гораздо сложнее, чем предшественники. Вы должны определить два дополнительных столбца, которые устанавливают иерархические связи XML: столбец `tag` (служит идентификатором строки) и столбец `Parent` (связывает дочерние записи со значением `tag` родительской записи, аналогично `EmployeeID` и `ManagerID`). Вы должны также дать псевдонимы всем столбцам для обозначения элемента `tag` и отображаемого имени для XML-вывода (как показано далее). Помните, что этим правилам должна подчиняться только первая инструкция `SELECT`; псевдонимы во всех последующих инструкциях `SELECT` запроса `UNION` игнорируются.

```

SELECT TOP 2 -- ограничение количества строк результата для демонстрации
  1 AS Tag,
  NULL AS Parent,
  CustomerID AS [Customer!1!CustomerID],
  NULL AS [SalesOrder!2!SalesOrderID],
  NULL AS [SalesOrder!2!OrderDate]
FROM Sales.Customer AS Customer
UNION ALL
SELECT TOP 10 -- ограничение количества строк результата для демонстрации
  2,
  1,
  Customer.CustomerID,
  OrderHeader.SalesOrderID,
  OrderHeader.OrderDate
FROM Sales.Customer AS Customer
  INNER JOIN Sales.SalesOrderHeader AS OrderHeader
    ON OrderHeader.CustomerID = Customer.
CustomerID
ORDER BY [Customer!1!CustomerID], [SalesOrder!2!SalesOrderID]
FOR XML EXPLICIT

```

Выполните этот запрос и щелкните по XML-гиперссылке для того, чтобы увидеть следующие выходные данные:

```

<Customer CustomerID="1">
  <SalesOrder SalesOrderID="43860" OrderDate="2001-08-01T00:00:00" />
  <SalesOrder SalesOrderID="44501" OrderDate="2001-11-01T00:00:00" />
  <SalesOrder SalesOrderID="45283" OrderDate="2002-02-01T00:00:00" />
  <SalesOrder SalesOrderID="46042" OrderDate="2002-05-01T00:00:00" />
</Customer>
<Customer CustomerID="2">
  <SalesOrder SalesOrderID="46976" OrderDate="2002-08-01T00:00:00" />
  <SalesOrder SalesOrderID="47997" OrderDate="2002-11-01T00:00:00" />
  <SalesOrder SalesOrderID="49054" OrderDate="2003-02-01T00:00:00" />
  <SalesOrder SalesOrderID="50216" OrderDate="2003-05-01T00:00:00" />
  <SalesOrder SalesOrderID="51728" OrderDate="2003-08-01T00:00:00" />
  <SalesOrder SalesOrderID="57044" OrderDate="2003-11-01T00:00:00" />
</Customer>

```

Этот вывод напоминает наш пример FOR XML AUTO. Что же мы приобрели путем составления более сложного запроса с FOR XML EXPLICIT? Некоторые

виды вывода, которых нельзя добиться при помощи FOR XML AUTO. Например, вы можете указать, что определенные значения должны составляться как элементы (а не атрибуты), для чего нужно дописать !ELEMENT в конце столбца с псевдонимом:

```
SELECT TOP 2 -- ограничение количества строк результата для демонстрации
  1 AS Tag,
  NULL AS Parent,
  CustomerID AS [Customer!1!CustomerID],
  NULL AS [SalesOrder!2!SalesOrderID],
  NULL AS [SalesOrder!2!OrderDate!ELEMENT] --Render as an element
FROM Sales.Customer AS Customer
UNION ALL
SELECT TOP 10 -- ограничение количества строк результата для демонстрации
  2,
  1,
  Customer.CustomerID,
  OrderHeader.SalesOrderID,
  OrderHeader.OrderDate
FROM Sales.Customer AS Customer
  INNER JOIN Sales.SalesOrderHeader AS OrderHeader
    ON OrderHeader.CustomerID = Customer.CustomerID
ORDER BY [Customer!1!CustomerID], [SalesOrder!2!SalesOrderID]
FOR XML EXPLICIT
```

Было сделано только одно незначительное изменение (мы дописали !ELEMENT к столбцу OrderDate). Однако это изменение существенно влияет на окончательный результат:

```
<Customer CustomerID="1">
  <SalesOrder SalesOrderID="43860">
    <OrderDate>2001-08-01T00:00:00</OrderDate>
  </SalesOrder>
  <SalesOrder SalesOrderID="44501">
    <OrderDate>2001-11-01T00:00:00</OrderDate>
  </SalesOrder>
  <SalesOrder SalesOrderID="45283">
    <OrderDate>2002-02-01T00:00:00</OrderDate>
  </SalesOrder>
  <SalesOrder SalesOrderID="46042">
    <OrderDate>2002-05-01T00:00:00</OrderDate>
```

```

    </SalesOrder>
</Customer>
<Customer CustomerID="2">
    <SalesOrder SalesOrderID="46976">
        <OrderDate>2002-08-01T00:00:00</OrderDate>
    </SalesOrder>
    <SalesOrder SalesOrderID="47997">
        <OrderDate>2002-11-01T00:00:00</OrderDate>
    </SalesOrder>
    <SalesOrder SalesOrderID="49054">
        <OrderDate>2003-02-01T00:00:00</OrderDate>
    </SalesOrder>
    <SalesOrder SalesOrderID="50216">
        <OrderDate>2003-05-01T00:00:00</OrderDate>
    </SalesOrder>
    <SalesOrder SalesOrderID="51728">
        <OrderDate>2003-08-01T00:00:00</OrderDate>
    </SalesOrder>
    <SalesOrder SalesOrderID="57044">
        <OrderDate>2003-11-01T00:00:00</OrderDate>
    </SalesOrder>
</Customer>

```

Обратите внимание, что OrderDate отображается как дочерний элемент для SalesOrder. Несмотря на то, что режим EXPLICIT может давать надежные результаты, требуются более сложные запросы. Например, для добавления нескольких полей из OrderHeader и OrderDetail вам придется написать следующий запрос:

```

--XML EXPLICIT
SELECT TOP 2 -- ограничение количества строк результата для демонстрации
    1 AS Tag,
    NULL AS Parent,
    CustomerID AS [Customer!1!CustomerID],
    NULL AS [SalesOrder!2!SalesOrderID],
    NULL AS [SalesOrder!2!TotalDue],
    NULL AS [SalesOrder!2!OrderDate!ELEMENT],
    NULL AS [SalesOrder!2!ShipDate!ELEMENT],
    NULL AS [SalesDetail!3!ProductID],
    NULL AS [SalesDetail!3!OrderQty],
    NULL AS [SalesDetail!3!LineTotal]

```

```
FROM Sales.Customer AS Customer
WHERE Customer.CustomerID IN (11000, 11001)
UNION ALL
SELECT
    2,
    1,
    Customer.CustomerID,
    OrderHeader.SalesOrderID,
    OrderHeader.TotalDue,
    OrderHeader.OrderDate,
    OrderHeader.ShipDate,
    NULL,
    NULL,
    NULL
FROM Sales.Customer AS Customer
    INNER JOIN Sales.SalesOrderHeader AS OrderHeader
        ON OrderHeader.CustomerID = Customer.CustomerID
WHERE Customer.CustomerID IN (11000, 110012)
UNION ALL
SELECT
    3,
    2,
    Customer.CustomerID,
    OrderHeader.SalesOrderID,
    OrderHeader.TotalDue,
    OrderHeader.OrderDate,
    OrderHeader.ShipDate,
    OrderDetail.ProductID,
    OrderDetail.OrderQty,
    OrderDetail.LineTotal
FROM Sales.Customer AS Customer
    INNER JOIN Sales.SalesOrderHeader AS OrderHeader
        ON OrderHeader.CustomerID = Customer.CustomerID
    INNER JOIN Sales.SalesOrderDetail AS OrderDetail
        ON OrderDetail.SalesOrderID = OrderHeader.SalesOrderID
WHERE Customer.CustomerID IN (11000, 11001)
ORDER BY [Customer!1!CustomerID], [SalesOrder!2!SalesOrderID]
FOR XML EXPLICIT
```

Код будет усложняться по мере добавления данных в вывод. И хоть этот запрос совершенно правильный, решение неприемлемо, что приводит нас к новой улучшенной инструкции `FOR XML`.

## Улучшения инструкции *FOR XML*

Как известно, SQL Server 2000 имел большое количество функциональных возможностей XML (включая и те, которые здесь упомянуты не были — например, просмотр результатов по HTTP). Почти вся остальная поддержка XML в SQL Server 2000 вертится вокруг функции `FOR XML`, которая разработчиками используется явно недостаточно. Вот несколько приемов, которыми инструкция `FOR XML` была дополнена в SQL Server 2005 и 2008:

- ❑ при помощи опции `TYPE` команда `FOR XML` может выводить в тип данных `xml` из инструкции `SELECT`, что в свою очередь позволяет вам вкладывать результаты `SELECT...FOR XML` в другую инструкцию `SELECT`;
- ❑ новая опция `FOR XML PATH` позволяет вам более легко формировать данные и вырабатывать XML на основе элементов (чем опция `EXPLICIT`);
- ❑ вы можете явно указать элемент `ROOT` для вашего вывода;
- ❑ вы можете вырабатывать XML на основе элементов при помощи `FOR XML AUTO`;
- ❑ `FOR XML` может вырабатывать XML при помощи схемы;
- ❑ улучшена обработка вложенности, пробельных символов и неопределенных значений.

## Опция *TYPE*

Для SQL Server 2008 тип `xml` является встроенным типом данных, так что мы теперь можем автоматически обеспечить XML-вывод из запроса `FOR XML` к экземпляру данных типа `xml`. Это делается при помощи ключевого слова `TYPE` после инструкции `FOR XML`:

```
DECLARE @xmlData AS xml
SET @xmlData =
    (SELECT Customer.CustomerID, OrderDetail.SalesOrderID,
        OrderDetail.OrderDate
    FROM Sales.Customer Customer
    INNER JOIN Sales.SalesOrderHeader OrderDetail
    ON OrderDetail.CustomerID = Customer.CustomerID
```

```

WHERE Customer.CustomerID < 11200
ORDER BY Customer.CustomerID
FOR XML AUTO, TYPE) — Приводит к типу XML
SELECT @xmlData

```

Данный пример объявляет переменную типа xml, а затем в этой переменной приводит результат запроса FOR XML (при помощи опции TYPE). Вы можете также использовать эту новую переменную типа xml как часть инструкции INSERT (вставки в столбец XML) или передать ее в хранимую процедуру. Результаты этого запроса показаны здесь:

```

<Customer CustomerID="1">
  <OrderDetail SalesOrderID="43860" OrderDate="2001-08-01T00:00:00" />
  <OrderDetail SalesOrderID="44501" OrderDate="2001-11-01T00:00:00" />
  <OrderDetail SalesOrderID="45283" OrderDate="2002-02-01T00:00:00" />
  <OrderDetail SalesOrderID="46042" OrderDate="2002-05-01T00:00:00" />
</Customer>
<Customer CustomerID="2">
  <OrderDetail SalesOrderID="46976" OrderDate="2002-08-01T00:00:00" />
  <OrderDetail SalesOrderID="47997" OrderDate="2002-11-01T00:00:00" />
  <OrderDetail SalesOrderID="49054" OrderDate="2003-02-01T00:00:00" />
  <OrderDetail SalesOrderID="50216" OrderDate="2003-05-01T00:00:00" />
  <OrderDetail SalesOrderID="51728" OrderDate="2003-08-01T00:00:00" />
  <OrderDetail SalesOrderID="57044" OrderDate="2003-11-01T00:00:00" />
  <OrderDetail SalesOrderID="63198" OrderDate="2004-02-01T00:00:00" />
  <OrderDetail SalesOrderID="69488" OrderDate="2004-05-01T00:00:00" />
</Customer>

```

Вы можете использовать FOR XML TYPE в любом выражении SQL. В следующем примере синтаксис FOR XML TYPE используется как выражение в инструкции SELECT:

```

SELECT
  CustomerID,
  (SELECT
    SalesOrderID,
    TotalDue,
    OrderDate,
    ShipDate
  FROM Sales.SalesOrderHeader AS OrderHeader
  WHERE CustomerID = Customer.CustomerID
  FOR XML AUTO, TYPE) AS OrderHeaders

```

```
FROM Sales.Customer AS Customer
WHERE CustomerID IN (11000, 11001)
FOR XML AUTO
```

Вот результаты:

```
1 <OrderHeader SalesOrderID="43860" TotalDue="14603.7393" Order-
Date="2001-08-01T00:00:00" ShipDate="2001-08-08T00:00:00" />
<OrderHeader SalesOrderID="44501" TotalDue="26128.8674" OrderDate="2001-
11-01T00:00:00" ShipDate="2001-11-08T00:00:00" />
<OrderHeader SalesOrderID="45283" TotalDue="37643.1378" OrderDate="2002-
02-01T00:00:00" ShipDate="2002-02-08T00:00:00" />
<OrderHeader SalesOrderID="46042" TotalDue="34722.9906" OrderDate="2002-
05-01T00:00:00" ShipDate="2002-05-08T00:00:00" />
2 <OrderHeader SalesOrderID="46976" TotalDue="10184.0774" Order-
Date="2002-08-01T00:00:00" ShipDate="2002-08-08T00:00:00" />
<OrderHeader SalesOrderID="47997" TotalDue="5469.5941" OrderDate="2002-
11-01T00:00:00" ShipDate="2002-11-08T00:00:00" />
<OrderHeader SalesOrderID="49054" TotalDue="1739.4078" OrderDate="2003-
02-01T00:00:00" ShipDate="2003-02-08T00:00:00" />
<OrderHeader SalesOrderID="50216" TotalDue="1935.5166" OrderDate="2003-
05-01T00:00:00" ShipDate="2003-05-08T00:00:00" />
<OrderHeader SalesOrderID="51728" TotalDue="3905.2547" OrderDate="2003-
08-01T00:00:00" ShipDate="2003-08-08T00:00:00" />
<OrderHeader SalesOrderID="57044" TotalDue="4537.8484" OrderDate="2003-
11-01T00:00:00" ShipDate="2003-11-08T00:00:00" />
<OrderHeader SalesOrderID="63198" TotalDue="4053.9506" OrderDate="2004-
02-01T00:00:00" ShipDate="2004-02-08T00:00:00" />
<OrderHeader SalesOrderID="69488" TotalDue="908.3199" OrderDate="2004-
05-01T00:00:00" ShipDate="2004-05-08T00:00:00" />
```

## Опция **FOR XML PATH**

В XML на основе элементов вы можете использовать опцию FOR XML PATH для указания псевдонимов столбцов выражений XPath, которые будут формировать XML-вывод:

```
SELECT TOP 2 -- ограничение количества строк результата для демонстрации
BusinessEntityID AS [@BusinessEntityID],
FirstName AS [ContactName/First],
LastName AS [ContactName/Last],
EmailAddress AS [ContactEmailAddress/EmailAddress1]
FROM HumanResources.vEmployee FOR XML PATH
```



Результаты выглядят следующим образом:

```
<row BusinessEntityID="263">
  <ContactName>
    <First>Jean</First>
    <Last>Trenary</Last>
  </ContactName>
  <ContactEmailAddress>
    <EmailAddress1>jean0@adventure-works.com</EmailAddress1>
  </ContactEmailAddress>
</row>
<row BusinessEntityID="78">
  <ContactName>
    <First>Reuben</First>
    <Last>D'sa</Last>
  </ContactName>
  <ContactEmailAddress>
    <EmailAddress1>reuben0@adventure-works.com</EmailAddress1>
  </ContactEmailAddress>
</row>
```

Теперь давайте вернемся к недавнему примеру, в котором мы демонстрировали опцию FOR XML EXPLICIT. Используя опцию TYPE в сочетании с FOR XML PATH, можно заменить тот сложный запрос гораздо более простой версией:

```
SELECT
  CustomerID AS [@CustomerID],
  (SELECT SalesOrderID AS [@SalesOrderID],
    TotalDue AS [@TotalDue],
    OrderDate,
    ShipDate,
    (SELECT ProductID AS [@ProductID],
      OrderQty AS [@OrderQty],
      LineTotal AS [@LineTotal]
    FROM Sales.SalesOrderDetail
    WHERE SalesOrderID = OrderHeader.SalesOrderID
    FOR XML PATH('OrderDetail'), TYPE)
  FROM Sales.SalesOrderHeader AS OrderHeader
  WHERE CustomerID = Customer.CustomerID
  FOR XML PATH('OrderHeader'), TYPE)
FROM Sales.Customer AS Customer
```

```
WHERE CustomerID IN (11000, 11001)
FOR XML PATH ('Customer')
```

Гораздо лучше, не так ли? Эта инструкция использует подзапросы с инструкцией FOR XML PATH в сочетании с TYPE для того, чтобы вложить XML на основе элементов в более крупную инструкцию FOR XML PATH. При этом упорядочивается отдельно каждый заказ Order для клиента в виде дочернего узла CustomerID:

```
<Customer CustomerID="1">
  <OrderHeader SalesOrderID="43860" TotalDue="14603.7393">
    <OrderDate>2001-08-01T00:00:00</OrderDate>
    <ShipDate>2001-08-08T00:00:00</ShipDate>
    <OrderDetail ProductID="761" OrderQty="2" LineTotal="838.917800" />
    <OrderDetail ProductID="770" OrderQty="1" LineTotal="419.458900" />
    :
  </OrderHeader>
  <OrderHeader SalesOrderID="44501" TotalDue="26128.8674">
    <OrderDate>2001-11-01T00:00:00</OrderDate>
    <ShipDate>2001-11-08T00:00:00</ShipDate>
    <OrderDetail ProductID="761" OrderQty="1" LineTotal="419.458900" />
    <OrderDetail ProductID="768" OrderQty="3" LineTotal="1258.376700" />
    :
  </OrderHeader>
  <OrderHeader SalesOrderID="45283" TotalDue="37643.1378">
    <OrderDate>2002-02-01T00:00:00</OrderDate>
    <ShipDate>2002-02-08T00:00:00</ShipDate>
    <OrderDetail ProductID="759" OrderQty="1" LineTotal="419.458900" />
    <OrderDetail ProductID="758" OrderQty="3" LineTotal="2624.382000" />
    <OrderDetail ProductID="750" OrderQty="2" LineTotal="4293.924000" />
    :
  </OrderHeader>
  <OrderHeader SalesOrderID="46042" TotalDue="34722.9906">
    <OrderDate>2002-05-01T00:00:00</OrderDate>
    <ShipDate>2002-05-08T00:00:00</ShipDate>
    <OrderDetail ProductID="763" OrderQty="2" LineTotal="838.917800" />
    <OrderDetail ProductID="757" OrderQty="4" LineTotal="3499.176000" />
    :
  </OrderHeader>
</Customer>
:
```

Если вы знакомы с XPath и чувствуете себя в нем уверенно, то вы оцените некоторые дополнительные функциональные возможности FOR XML PATH. Вы можете использовать функции тестирования узлов в XPath для управления формой XML-вывода:

- ☐ data;
- ☐ comment;
- ☐ node;
- ☐ text;
- ☐ processing-instruction.

Следующий пример использует методы data и comment XPath. Метод data берет результаты базового запроса и помещает их внутрь одного элемента. Метод comment берет данные и преобразует их в XML-комментарий:

```
SELECT
    Customer.BusinessEntityID AS [@CustomerID],
    Customer.FirstName + ' ' + Customer.LastName AS [comment()],
    (SELECT SalesOrderID AS [@SalesOrderID],
        TotalDue AS [@TotalDue],
        OrderDate,
        ShipDate,
        (SELECT ProductID AS [data()]
            FROM Sales.SalesOrderDetail
            WHERE SalesOrderID = OrderHeader.SalesOrderID
            FOR XML PATH('')) AS [ProductIDs]
    FROM Sales.SalesOrderHeader AS OrderHeader
    WHERE CustomerID = Customer.BusinessEntityID
    FOR XML PATH('OrderHeader'), TYPE)
FROM Sales.vIndividualCustomer AS Customer
WHERE BusinessEntityID IN (11000, 11001)
FOR XML PATH ('Customer')
```

Как видно из результатов, составное имя контактного лица становится комментарием XML, а подзапрос ProductID преобразуется в один элемент:

```
<Customer CustomerID="11000">
  <!--Jon Yang-->
  <OrderHeader SalesOrderID="43793" TotalDue="3756.9890">
    <OrderDate>2001-07-22T00:00:00</OrderDate>
    <ShipDate>2001-07-29T00:00:00</ShipDate>
    <ProductIDs> 966 934 923 707 881</ProductIDs>
```

```

</OrderHeader>
:
</Customer>
<Customer CustomerID="11001">
  <!--Eugene Huang-->
  <OrderHeader SalesOrderID="43767" TotalDue="3729.3640">
    <OrderDate>2001-07-18T00:00:00</OrderDate>
    <ShipDate>2001-07-25T00:00:00</ShipDate>
    <ProductIDs> 779 878 870 871 884 712</ProductIDs>
  </OrderHeader>
:
</Customer>

```

## Указываем элемент *ROOT*

Опция *ROOT* позволяет вам добавить главный (корневой) элемент в вывод FOR XML. Вы можете сочетать это с другими ключевыми словами FOR XML:

```

SELECT Customer.CustomerID, OrderDetail.SalesOrderID,
       OrderDetail.OrderDate
FROM Sales.Customer AS Customer
  INNER JOIN Sales.SalesOrderHeader OrderDetail
    ON OrderDetail.customerid=Customer.customerid
WHERE Customer.CustomerID<11005
ORDER BY Customer.CustomerID
FOR XML AUTO, ROOT('Orders')

```

Вывод выглядит следующим образом:

```

<Orders>
  <Customer CustomerID="1">
    <OrderDetail SalesOrderID="43860" OrderDate="2001-08-01T00:00:00" />
    <OrderDetail SalesOrderID="44501" OrderDate="2001-11-01T00:00:00" />
    <OrderDetail SalesOrderID="45283" OrderDate="2002-02-01T00:00:00" />
    <OrderDetail SalesOrderID="46042" OrderDate="2002-05-01T00:00:00" />
  </Customer>
:
</Orders>

```

Выведенный здесь код — тот же самый, что и для любого FOR XML AUTO этого запроса, за исключением того, что теперь XML ROOT окружает наши данные. В этом примере мы использовали ROOT('Orders'), поэтому наши выходные данные окружены XML-элементом <Orders>.

## Создаем встроенную схему XSD

Схемы обеспечивают принудительную структуру XML-данных. Когда вы экспортируете данные при помощи синтаксиса FOR XML, можно включить в них встроенную XML-схему для получателя (чтобы тот также мог обеспечить реализацию этих правил на своем конце). В режимах RAW и AUTO вы можете создать встроенную XSD-схему как часть вывода (при помощи ключевого слова XMLSCHEMA):

```
SELECT Customer.CustomerID, OrderDetail.SalesOrderID,
       OrderDetail.OrderDate
FROM Sales.Customer AS Customer
  INNER JOIN Sales.SalesOrderHeader AS OrderDetail
    ON OrderDetail.CustomerID = Customer.CustomerID
WHERE Customer.CustomerID < 11005
ORDER BY Customer.CustomerID
FOR XML AUTO, XMLSCHEMA
```

Вывод выглядит следующим образом:

```
<xsd:schema targetNamespace="urn:schemas-microsoft-
com:sql:SqlRowSet1" xmlns:schema="urn:schemas-microsoft-
com:sql:SqlRowSet1" xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:sqltypes="http://schemas.microsoft.com/sqlserver/2004/sqltypes"
  elementFormDefault="qualified"> <xsd:import namespace=
  "http://schemas.microsoft.com/sqlserver/2004/sqltypes" schemaLocation=
  "http://schemas.microsoft.com/sqlserver/2004/sqltypes/sqltypes.xsd" />
<xsd:element name="Customer">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element ref="schema:OrderDetail" minOccurs="0"
        maxOccurs="unbounded" />
    </xsd:sequence>
    <xsd:attribute name="CustomerID" type="sqltypes:int"
      use="required" />
  </xsd:complexType>
</xsd:element>
<xsd:element name="OrderDetail">
  <xsd:complexType>
    <xsd:attribute name="SalesOrderID" type="sqltypes:int"
      use="required" />
```

```

        <xsd:attribute name="OrderDate" type="sqltypes:datetime"
                      use="required" />
    </xsd:complexType>
</xsd:element>
</xsd:schema>
<Customer xmlns="urn:schemas-microsoft-com:sql:SqlRowSet1"
          CustomerID="1">
    <OrderDetail SalesOrderID="43860" OrderDate="2001-08-01T00:00:00" />
    <OrderDetail SalesOrderID="44501" OrderDate="2001-11-01T00:00:00" />
    <OrderDetail SalesOrderID="45283" OrderDate="2002-02-01T00:00:00" />
    <OrderDetail SalesOrderID="46042" OrderDate="2002-05-01T00:00:00" />
</Customer>

```

SQL Server выводит схему по базовым типам результирующего набора. Например, поле `SalesOrderID` становится необходимым полем `int` в соответствии со встроенной схемой, основанной на свойствах поля в базовой таблице SQL.

## Вырабатываем основанный на элементах XML

Основанный на элементах XML (канонический) представляет свои данные как отдельные элементы (а не отдельные атрибуты). Весьма полезной опцией `RAW` и `AUTO` выступает ключевое слово `ELEMENTS`:

```

SELECT Customer.CustomerID,
       OrderDetail.SalesOrderID, OrderDetail.OrderDate
FROM Sales.Customer Customer
     INNER JOIN Sales.SalesOrderHeader OrderDetail
     ON OrderDetail.CustomerID = Customer.CustomerID
WHERE Customer.CustomerID = 11000
ORDER BY Customer.CustomerID
FOR XML AUTO, ELEMENTS

```

Результаты выглядят следующим образом:

```

<Customer>
  <CustomerID>11000</CustomerID>
  <OrderDetail>
    <SalesOrderID>43860</SalesOrderID>
    <OrderDate>2001-08-01T00:00:00</OrderDate>
  </OrderDetail>
  <OrderDetail>

```

```
<SalesOrderID>44501</SalesOrderID>
<OrderDate>2001-11-01T00:00:00</OrderDate>
</OrderDetail>
<OrderDetail>
  <SalesOrderID>45283</SalesOrderID>
  <OrderDate>2002-02-01T00:00:00</OrderDate>
</OrderDetail>
<OrderDetail>
  <SalesOrderID>46042</SalesOrderID>
  <OrderDate>2002-05-01T00:00:00</OrderDate>
</OrderDetail>
</Customer>
```

Как видите, элементом в результирующем XML становится каждый столбец запроса (в отличие от атрибута одной строки).

Ключевое слово `ELEMENTS` (используемое в сочетании с инструкцией `FOR XML`) преобразует каждый столбец результирующего набора в отдельный XML-элемент посредством `AUTO`, а каждую строку из соединяемой таблицы — в новый XML-элемент. Обратите внимание, что основанный на элементах язык XML более многословен (чем на атрибутах), но с ним легче работать.

## Улучшения *OPENXML* в SQL Server 2008

До настоящего времени мы использовали `FOR XML` для выработки XML из строк данных, но что если у нас уже есть XML-данные и мы хотим "нарезать" их обратно в реляционные данные? Для этой цели в SQL Server 2000 была введена функциональная возможность `OPENXML`. Это системная функция, которая позволяет нарезать XML-документ в строки T-SQL. SQL Server 2005 и 2008 также поддерживают функцию `OPENXML` — с некоторыми улучшениями, естественно.

Для нарезки данных в реляционные строки при помощи `OPENXML` вы должны сначала создать дескриптор XML-документа (при помощи системной хранимой процедуры `sp_xml_preparedocument`). Эта системная процедура берет XML-документ и создает представление со ссылкой в виде дескриптора, который она возвращает при помощи параметра `OUTPUT`. Функция `OPENXML` использует этот дескриптор вместе с указанным путем и ведет себя как представление базы данных для XML-данных, поэтому вы просто выбираете `SELECT` из функции `OPENXML` (точно так же, как вы делаете `SELECT` из таблицы или представления). Код листинга 6.7 показывает пример `OPENXML` в действии.

**Листинг 6.7. Использование OPENXML**

```

DECLARE @handle int
DECLARE @xmlOrder varchar(1000)
SET @xmlOrder = '
<Root>
  <Customer CustomerID="BRU" ContactName="Andrew Brust">
    <Order CustomerID="BRU" EmployeeID="5" OrderDate="2005-11-04">
      <OrderDetail OrderID="10248" ProductID="16" Quantity="12"/>
      <OrderDetail OrderID="10248" ProductID="32" Quantity="10"/>
    </Order>
  </Customer>
  <Customer CustomerID="ZAC" ContactName="Bill Zack">
    <Order CustomerID="Zac" EmployeeID="3" OrderDate="2005-11-16">
      <OrderDetail OrderID="10283" ProductID="99" Quantity="3"/>
    </Order>
  </Customer>
</Root>'
-- Создать внутреннее представление XML-документа
EXEC sp_xml_preparedocument @handle OUTPUT, @xmlOrder
-- поставщик набора строк OPENXML
SELECT *
  FROM OPENXML (@handle, '/Root/Customer', 1)
  WITH (CustomerID varchar(10), ContactName varchar(20))

```

Этот код позволяет вам запрашивать XML-текст (и работать с ним) так, как будто это реляционные данные. Вывод выглядит следующим образом:

```

CustomerID  ContactName
-----
BRU          Andrew Brust
ZAC          Bill Zack
(2 row(s) affected)

```

Вы можете по желанию указать, чтобы OPENXML использовал реляционное сопоставление между столбцами набора строк и узлами XML на основе элементов или на основе атрибутов. Есть два способа управлять этим сопоставлением. Первый — использовать параметр `flags`, когда XML-узлы соответствуют столбцам набора строк с точно такими же именами. Вы можете использовать также параметр `ColPattern` — это выражение XPath, которое позволяет вам использовать схему для выполнения сопоставления (как часть



SchemaDeclaration в выражении WITH). Указанное в ColPattern соответствие перезаписывает то соответствие, которое было указано в параметре flags.

SQL Server 2005 ввел два улучшения для OPENXML, которые касались нового типа данных xml. Первое: тип данных xml поддерживается инструкцией OPENXML как выходной столбец или как дополнительный столбец. Второе: вы можете передать переменную типа xml напрямую в sp\_xml\_preparedocument. Оба этих улучшения дают нам возможность более легко работать с уже существующими XML-данными в столбце XML или с созданными при помощи FOR XML TYPE данными.

## XML Bulk Load

В SQL Server 2000 методика XML Bulk Load позволяла пользователям загружать большие XML-документы на клиентской стороне. XML Bulk Load работает путем чтения XML и выработки инструкций INSERT, которые выполняются на клиенте пакетным образом. SQL Server 2005 и 2008 значительно улучшили XML Bulk Load, позволив выполняться на сервере и загружать документы непосредственно в столбец типа xml.

Требуется использовать функцию OPENROWSET системного поставщика наборов строк и указать поставщика BULK (как показано в листинге 6.8).

### Листинг 6.8. Выполнение XML Bulk Load

```
-- создать таблицу со столбцом xml
CREATE TABLE tblXmlCustomers
  (CustomerID int PRIMARY KEY IDENTITY,
-- этот файл загрузит 1 запись в (SINGLE_CLOB)
-- для большего количества записей используйте файл форматирования
INSERT INTO tblXmlCustomers
  SELECT * FROM OPENROWSET(BULK 'C:\customer_01.xml',
                           SINGLE_CLOB) AS XmlData
```

Этот пример работает таким образом: сначала создается таблица, которая имеет столбец типа xml и значение первичного ключа. Затем мы используем инструкцию INSERT, которая выбирает все данные из XML-файла (при помощи OPENROWSET). OPENROWSET использует поставщика BULK и загружает в столбец типа xml таблицы tblXmlCustomers все содержимое файла customer\_01.xml.

## Запрашиваем XML-данные при помощи XQuery

Хранение XML в базе данных — это одно; выполнение эффективных запросов по этим данным — совсем другое. В SQL Server 2000 для выполнения запроса по XML-данным (находящимся в текстовых столбцах) вам приходилось разбирать XML и переносить данные элементов и атрибутов в реляционные столбцы или применить полнотекстовый поиск. В SQL Server 2005 и 2008 для запроса XML-данных XQuery предоставляет собственный и достаточно элегантный способ.

### Разбираемся с выражениями XQuery и XPath

XQuery — это язык запроса и обработки XML-данных; его спецификация находится по адресу: <http://www.w3.org/TR/xquery/>. Эта спецификация содержит несколько описаний требований, вариантов использования и моделей данных. Пока же мы объясним основы. После прочтения данного раздела вы должны уметь выбирать, фильтровать и обновлять XML-данные при помощи XQuery.

В XQuery действуют все правила XML. XQuery использует строчные буквы в именах элементов (ключевых словах), и поскольку сам XML чувствителен к регистру, вы должны учитывать это. Хотя XQuery содержит некоторые весьма мощные команды форматирования и обработки, он в первую очередь является языком запросов (в полном соответствии со своим названием), и мы сосредоточимся здесь на написании запросов. Тело запроса состоит из двух частей: выражение XPath и выражение `FLWOR` (произносится как "flower". FLOWR — это акроним по первым буквам `for`, `let`, `where`, `order by` и `return`).

### Выражения XPath

XPath (стандарт W3C, описание дано по адресу <http://www.w3.org/TR/xpath>) использует выражения пути для идентификации конкретных узлов XML-документа. Эти выражения пути аналогичны синтаксису, который вы видите при работе с файловой системой компьютера (например, `c:\folder\myfile.doc`). Взгляните на следующий XML-документ:

```
<catalog>
  <book category="ITPro">
    <title>Windows Step By Step</title>
    <author>Bill Zack</author>
```

```
<price>49.99</price>
</book>
<book category="Developer">
  <title>Developing ADO .NET</title>
  <author>Andrew Brust</author>
  <price>39.93</price>
</book>
<book category="ITPro">
  <title>Windows Cluster Server</title>
  <author>Stephen Forte</author>
  <price>59.99</price>
</book>
</catalog>
```

Следующее выражение XPath выбирает каталог корневого элемента:

```
/catalog
```

Это выражение XPath выбирает все элементы книг корневого элемента каталога:

```
/catalog/book
```

А это выражение XPath выбирает все элементы авторов всех элементов книг из корневого элемента каталога:

```
/catalog/book/author
```

XPath позволяет вам указать подмножество данных в XML. Язык XQuery более надежен и позволяет вам выполнять сложные запросы при помощи выражений FLWOR в сочетании с XPath.

## Выражения *FLWOR*

Точно так же, как инструкции SELECT, FROM, WHERE, GROUP BY и ORDER BY являются основой логики выборки языка SQL, так и ключевые слова for, let, where, order by и return (FLWOR) служат основой каждого написанного вами запроса XQuery. Вы используете ключевые слова for и let для присвоения значений переменным и выполнения итераций по данным в контексте запроса XQuery. Ключевое слово let не поддерживается в реализации XQuery для SQL Server 2005, но поддерживается в SQL Server 2008. Ключевое слово where работает как ограничение и выводит значение переменной. Например, следующий простой запрос XQuery использует выражение /catalog/book стандарта XPath для получения ссылки на все узлы <book>, а ключевое слово for инициирует цикл, но только из элементов, у которых атрибут category равен "ITPro". Этот простой фрагмент кода при помощи переменной \$b инструкции for делает итерации по всем узлам /catalog/book, у которых атри-

бут категории равен "ITPro", и возвращает полученную информацию в упорядоченном по имени автора виде (при помощи ключевого слова `order`):

```
for $b in /catalog/book
  where $b/@category="ITPro"
  order by $b/author[1] descending
  return ($b)
```

А вот простой пример с использованием выражения XQuery для переменной типа `xml`. Мы присваиваем XML этой переменной, а затем используем предыдущее выражение XQuery в методе `query` для типа данных `xml`:

```
DECLARE @XML xml
SET @XML='
<catalog>
  <book category="ITPro">
    <title>Windows Step By Step</title>
    <author>Bill Zack</author>
    <price>49.99</price>
  </book>
  <book category="Developer">
    <title>Developing ADO .NET</title>
    <author>Andrew Brust</author>
    <price>39.93</price>
  </book>
  <book category="ITPro">
    <title>Windows Cluster Server</title>
    <author>Stephen Forte</author>
    <price>59.99</price>
  </book>
</catalog>'
SELECT @XML.query('
  for $b in /catalog/book
  where $b/@category="ITPro"
  order by $b/author[1] descending
  return ($b)')
```

Результаты показаны далее. Обратите внимание, что запись `Stephen` — первая по порядку, поскольку у нас убывающий порядок по элементу `author`. Запись `Andrew` в выводе отсутствует, поскольку у нас есть ограничение по "ITPro" в элементе категории.

```
<book category="ITPro">
  <title>Windows Cluster Server</title>
```

```
<author>Stephen Forte</author>
<price>59.99</price>
</book>
<book category="ITPro">
  <title>Windows Step By Step</title>
  <author>Bill Zack</author>
  <price>49.99</price>
</book>
```

Теперь, когда мы познакомились с основами, давайте рассмотрим использование XQuery с данными SQL Server.

## XQuery сервера SQL Server 2008 в действии

SQL Server 2008 имеет основанную на стандартах реализацию XQuery с использованием пяти методов типа данных `xml`:

- ❑ `xml.exist` — использует вход XQuery и возвращает 0, 1 или NULL (в зависимости от результатов запроса). Этот метод возвращает 0 тогда, когда никакие элементы не совпадают, 1 — когда совпадение есть, и NULL — когда нет XML-данных для запроса. Метод `xml.exist` часто используется для предикатов запросов;
- ❑ `xml.value` — принимает запрос XQuery, который сводится к единственному значению на входе и возвращает скалярный тип SQL Server;
- ❑ `xml.query` — принимает запрос XQuery, который сводится к множеству значений на входе и возвращает поток данных типа `xml` в качестве вывода;
- ❑ `xml.nodes` — принимает запрос XQuery как вход и возвращает из XML-документа набор строк с одним столбцом. По существу, этот метод дробит XML на множество более мелких XML-результатов;
- ❑ `xml.modify` — позволяет вставлять, удалять или модифицировать узлы (либо последовательности узлов) в экземпляре данных типа `xml` при помощи языка манипулирования данными запросов XQuery (DML).

Сначала мы создадим простую таблицу докладчиков на конференции и тем занятий. Обычно вы нормализуете данные как "один-ко-многим" между таблицей докладчиков и таблицей занятий. Вместо использования дополнительной нормализованной таблицы мы смоделируем одну таблицу с информацией о докладчиках и один столбец XML с занятиями докладчиков. В реальном мире вы можете встретить такую ситуацию тогда, когда у вас есть докладчик, а его занятия представлены набором таблиц "один-ко-многим" в базе данных администрации. В случае Web-базы данных вы можете достаточно часто

"публиковать" отчеты или преобразовать нормализованные данные для простого HTML-отображения (или XSLT).

Сначала (по причинам, которые скоро станут понятны) мы создаем схему для нашего XML-столбца. Эта схема (показана в листинге 6.9) будет определять тип разрешаемого в столбце XML, в том числе тип данных `xml`.

#### Листинг 6.9. Создание схемы XML

```
CREATE XML SCHEMA COLLECTION dbo.classes_xsd AS
'<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="class">
    <xs:complexType>
      <xs:attribute name="name" type="xs:string" use="required" />
    </xs:complexType>
  </xs:element>
  <xs:element name="classes">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="class" maxOccurs="unbounded" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>'
```

Затем мы создаем нашу таблицу `tblSpeakers` в листинге 6.10. Обратите внимание, что столбец `Speaker_XML` использует только что созданную схему `classes_xsd`.

#### Листинг 6.10. Создание таблицы со схемой

```
CREATE TABLE tblSpeakers
( Speaker_ID int PRIMARY KEY IDENTITY,
  Speaker_NM nvarchar(50),
  Speaker_Country nvarchar(25),
  Speaker_XML xml (classes_xsd) NOT NULL)
```

XQuery работает более эффективно тогда, когда по XML-столбцу есть XML-индекс. Как вы уже знаете, XML-индекс работает только в том случае, если

имеется ограничение первичного ключа (а оно у нас есть). Этот код создает первичный, а затем и структурный (PATH) индекс, поскольку наши примеры используют много ограничений `where` для значений конкретных элементов:

```
-- XML- индекс: первичный
CREATE Primary XML INDEX idx_1
  ON tblSpeakers (Speaker_XML)
-- PATH
CREATE XML INDEX idx_a
  ON tblSpeakers (Speaker_XML)
  USING XML INDEX idx_1 FOR PATH
```

Теперь, когда у нас есть индекс, вспомним, что для наилучшей производительности XQuery следует всегда использовать схему для XML-столбца. Без схемы ядро XQuery просто обращается со всем этим как со строковыми данными.

И наконец, нам необходимо поместить данные в таблицу при помощи неких инструкций `INSERT` (как показано в листинге 6.11). Последняя инструкция `INSERT` в этом листинге 'Bad Speaker' даст ошибку, поскольку она не содержит элемента `<classes>` (как того требует схема `classes_xsd`). Поскольку XML чувствителен к регистру, то его элемент `<CLASSES>` не совпадает с указанным в схеме обязательным элементом `<classes>`.

#### Листинг 6.11. Вставка XML-данных

```
INSERT INTO tblSpeakers VALUES('Stephen Forte', 'USA', '
<classes>
  <class name="Writing Secure Code for ASP .NET"/>
  <class name="Using XQuery to Query and Manipulate
    XML Data in SQL Server 2008"/>
  <class name="SQL Server and Oracle Working Together"/>
  <class name="Protecting against SQL Injection Attacks "/>
</classes>')
INSERT INTO tblSpeakers VALUES('Richard Campbell', 'Canada', '
<classes>
  <class name="SQL Server Profiler"/>
  <class name="Advanced SQL Querying Techniques"/>
  <class name="SQL Server and Oracle Working Together"/>
  <class name="T-SQL Error Handling in Yukon"/>
</classes>')
INSERT INTO tblSpeakers VALUES('John Huckaby', 'USA', '
<classes>
  <class name="Smart Client Stuff"/>
```

```

    <class name="More Smart Client Stuff"/>
</classes>')
INSERT INTO tblSpeakers VALUES('Malek Kemmou', 'Morocco', '
<classes>
    <class name="SmartPhone 2005"/>
    <class name="Office System 2003"/>
</classes>')
INSERT INTO tblSpeakers VALUES('Goksin Bakir', 'Turkey', '
<classes>
    <class name="SmartPhone 2007"/>
    <class name="Office System 2007"/>
</classes>')
INSERT INTO tblSpeakers VALUES('Jan Vasters', 'Germany', '
<classes>
    <class name="SOA"/>
    <class name="Biz Talk Services"/>
</classes>')
INSERT INTO tblSpeakers VALUES('Monica L. Tripp', 'USA', '
<classes>
    <class name="SQL Server Index"/>
    <class name="SQL Precon"/>
</classes>')
INSERT INTO tblSpeakers VALUES('Bad Speaker', 'France', '
<CLASSES>
    <class name="SQL Server Index"/>
    <class name="SQL Precon"/>
</CLASSES>')
```

Теперь, когда данные у нас уже есть, пришло время начать писать выражения XQuery в SQL Server 2008. Для этого мы будем использовать основанные на запросах методы типа данных xml в обычном запросе T-SQL. Затем мы увидим xml.modify в действии.

## Метод *xml.exist*

Наличие XML в базе данных почти бесполезно, если вы не можете делать запросы к элементам и атрибутам XML-данных. XQuery подходит для поиска по значениям определенного элемента или атрибута. Метод xml.exist принимает запрос XQuery как вход и возвращает 0, 1 или NULL в зависимости от результата запроса: 0 возвращается в том случае, когда совпадающих элементов нет; 1 возвращается тогда, когда совпадение имеется; NULL возвраща-



ется тогда, когда данных для запроса нет. Например, мы увидим, существует ли узел в этой конкретной XML-строке занятий:

```
DECLARE @XML xml
SET @XML = '
<classes>
  <class name="SQL Server Index"/>
  <class name="SQL Precon"/>
</classes>'
SELECT @XML.exist('/classes')
```

Код возвращает 1, поскольку элемент `<classes>` существует в XML-переменной. Если вы измените выражение XQuery для поиска несуществующего узла, например, `SELECT @XML.exist('/dogs')`, то он вернет 0. Было бы хорошо использовать это как часть выражения `CHECK CONSTRAINT`. Однако поскольку SQL Server не позволяет вам непосредственно использовать метод `xml.exist` как часть `CHECK CONSTRAINT`, то для выполнения этого действия придется сначала создать определяемую пользователем функцию (UDF), принимающую XML-поле и возвращающую значение метода `xml.exist`, который ищет экземпляр `<Orders>`:

#### Листинг 6.12. Создание функции для использования в CHECK CONSTRAINT

```
CREATE FUNCTION dbo.DoesOrderXMLDataExist(@XML xml)
  RETURNS bit
AS
BEGIN
  RETURN @XML.exist('/Orders')
END
GO
```

Для использования этой функции UDF из листинга 6.12 в качестве `CHECK CONSTRAINT` создайте таблицу и передайте в UDF столбец, к которому вы хотите применить ограничение:

```
-- Используем AdventureWorks2008
-- создать таблицу при помощи функции
CREATE TABLE OrdersXMLCheck
  (OrderDocID int PRIMARY KEY,
   xOrders xml NOT NULL DEFAULT '<Orders/>'
  CONSTRAINT xml_orderconstraint
   CHECK (dbo.DoesOrderXMLDataExist(xOrders) = 1))
```

Скорее всего, вы будете использовать возвращаемое из `xml.exist` значение (0, 1 или NULL) как часть выражения `WHERE`. Только подумайте: вы можете запустить запрос T-SQL и ограничить его по значению определенного XML-элемента! Возвращаясь к нашему главному примеру, давайте поищем значение SQL Server and Oracle Working Together в элементе `<class>`. Вот выражение XQuery для этого:

```
/classes/class[@name="SQL Server and Oracle Working Together"]
```

Вот как вы можете использовать это выражение:

```
SELECT * FROM tblSpeakers
WHERE
    Speaker_XML.exist('/classes/class[@name=
                        "SQL Server and Oracle Working Together"]') = 1
```

Результат выглядит следующим образом:

```
1 Stephen Forte USA <classes>...</classes>
2 Richard Campbell Canada <classes/>...</classes>
```

Возвращенный в этих результатах XML выглядит для Stephen следующим образом:

```
<classes>
  <class name="Writing Secure Code for ASP .NET " />
  <class name="Using XQuery to Query and Manipulate
            XML Data in SQL Server 2008" />
  <class name="SQL Server and Oracle Working Together" />
  <class name="Protecting against SQL Injection Attacks " />
</classes>
```

## Метод `xml.value`

Метод `xml.value` принимает выражение XQuery и возвращает указанное вами скалярное значение SQL Server. Предположим, есть XML-столбец, и вы хотите вернуть некоторые данные как внутренний тип данных SQL внутри запроса T-SQL. Вы вызываете метод `xml.value` для этого столбца (путем передачи выражения XQuery и типа данных, к которому вы хотите преобразовать вывод). Это требует от вас знания и понимания данных вашего XML-столбца. Вот пример синтаксиса для нашего текущего XML-документа в базе данных:

```
xml.value('/classes[1]/class[1]/@name', 'varchar(40)')
```

Выражение XPath переходит к атрибуту имени первого занятия и приведено к типу `varchar(40)`.

Вы должны выполнить выражение XQuery как часть обычного запроса T-SQL. Здесь феноменально то, что SQL Server сочетает XQuery и реляционные запросы в одном (поскольку в этом примере мы используем обычное выражение WHERE языка T-SQL для того, чтобы показать только докладчиков из Соединенных Штатов).

```
USE AdventureWorks2008
```

```
GO
```

```
SELECT
```

```
    Speaker_ID,
```

```
    Speaker_NM,
```

```
    Speaker_Country,
```

```
    Speaker_XML.value('/classes[1]/class[1]/@name', 'varchar(40)')
```

```
                as Sessions
```

```
FROM tblSpeakers
```

```
WHERE Speaker_Country = 'USA'
```

Результаты показаны здесь:

| Speaker_ID | Speaker_NM    | Speaker_Country | Session                          |
|------------|---------------|-----------------|----------------------------------|
| 1          | Stephen Forte | USA             | Writing Secure Code for ASP .NET |
| 3          | John Huckaby  | USA             | Smart Client Stuff               |
| 7          | Monica Tripp  | USA             | SQL Server Index                 |

(3 row(s) affected)

Давайте исследуем выражение XQuery. Как вы можете вспомнить из листинга 6.11, XML для Stephen выглядит следующим образом:

```
<classes>
  <class name="Writing Secure Code for ASP .NET" />
  <class name="Using XQuery to Query and Manipulate XML Data
              in SQL Server 2008" />
  <class name="SQL Server and Oracle Working Together" />
  <class name="Protecting against SQL Injection Attacks" />
</classes>
```

Следующее выражение пути XQuery возвращает значение атрибута name первого занятия (Writing Secure Code for ASP.NET) как varchar(40). Поэтому в предыдущем запросе выражение XQuery помещено в метод value XML-столбца Speaker\_XML. Само выражение XQuery делает всю работу по получению имени первого класса:

```
/classes[1]/class[1]/@name
```

Этот подход позволяет извлечь данные из XML-столбца и выдать их как обычные скалярные данные SQL Server.

## Метод `xml.query`

Метод `xml.query` возвращает значение типа `xml` и полезен только тогда, когда вы хотите, чтобы конечный результат столбца в запросе был XML; если вам нужны скалярные данные — используйте метод `xml.value`.

Если вы хотите вернуть те же данные, что и в предыдущем примере, но представить итоговый столбец в формате XML, запустите тот же запрос уже с методом `xml.query`:

```
-- xml.query
-- возвращает тип данных XML
-- то же, что и в предыдущем примере, но возвращает XML
SELECT
    Speaker_ID,
    Speaker_NM,
    Speaker_Country,
    Speaker_XML.query('/classes[1]/class[1]') AS Sessions
FROM tblSpeakers
WHERE Speaker_Country = 'USA'
```

Метод `xml.query` приведет к выдаче XML-вывода. Выражение `XQuery` может вернуть один элемент, вернуть все элементы или использовать выражение `return` (буква R в FLWOR) для полного преобразования результатов. В этом примере возвращается первый экземпляр элемента занятия. Результаты столбца `Sessions` для записей `Stephen` те же самые, что и в предыдущем примере, но форматированы в XML:

```
<class name="Writing Secure Code for ASP .NET " />
```

Вместо использования метода `xml.value` вы можете вернуть XML-результат многих узлов (опустив в выражении пути указатели порядковых номеров [1]):

```
-- то же, что и в предыдущем случае, но возвращает всё
SELECT
    Speaker_ID,
    Speaker_NM,
    Speaker_Country,
    Speaker_XML.query('/classes/class') AS Sessions
FROM tblSpeakers
WHERE Speaker_Country = 'USA'
```

Результаты те же самые, за исключением занятий `Stephen`; здесь мы получаем все XML-результаты как XML-столбец:

```
<class name="Writing Secure Code for ASP .NET " />
<class name="Using XQuery to Query and Manipulate
    XML Data in SQL Server    2008" />
```

```
<class name="SQL Server and Oracle Working Together" />
<class name="Protecting Against SQL Injection Attacks" />
```

Вы можете получить еще больше управления над выражениями пути XQuery при помощи FLWOR:

```
for $b in /classes/class
return ($b)
```

Это выражение использует ключевые слова `for` и `return` для того, чтобы сделать цикл по всем элементам занятий и вернуть их значения, и оно дает те же самые результаты, что и предыдущий пример (в котором в качестве выражения XQuery используется `"/classes/class"`). Вы можете включить это выражение в запрос T-SQL:

```
SELECT
    Speaker_ID,
    Speaker_NM,
    Speaker_Country,
    Speaker_XML.query('
        for $b in /classes/class
        return ($b)
    ') AS Sessions
FROM tblSpeakers
WHERE Speaker_Country = 'USA'
```

Предположим, что вы хотите иметь больше контроля над XML-выводом (при помощи методов `xml.query` или `xml.value`, а также комбинирования XQuery и обычного выражения `WHERE` языка T-SQL, чтобы показать только докладчиков в Соединенных Штатах). Вы можете использовать XML-метод в обоих выражениях (`SELECT` и `WHERE`) одного запроса:

```
SELECT
    Speaker_ID,
    Speaker_NM,
    Speaker_Country,
    Speaker_XML.query('/classes/class') AS Sessions
FROM tblSpeakers
WHERE Speaker_Country = 'USA' AND Speaker_XML.exist
    ('/classes/class[@name="SQL Server and Oracle Working Together"]') = 1
```

Выражение `WHERE` языка T-SQL ограничивает столбец `Speaker_Country`, и наше выражение XQuery фильтрует только по интересующему нас элементу `<class>`:

```
<class name="Writing Secure Code for ASP .NET " />
<class name="Using XQuery to Query and Manipulate
    XML Data in SQL Server 2008" />
```

```
<class name="SQL Server and Oracle Working Together" />
<class name="Protecting against SQL Injection Attacks " />
```

SQL Server 2008 теперь имеет поддержку `let` для присваивания значений переменным в выражении XQuery. Например, для подсчета количества заседаний по докладчикам (при помощи выражения `count`) зададим количество заседаний переменной через `let`:

```
DECLARE @xml xml
SET @xml='
<Speakers>
  <Speaker name="Stephen Forte">
    <classes>
      <class name="Writing Secure Code for ASP .NET" />
      <class name="Using XQuery in SQL Server 2008"/>
      <class name="SQL Server and Oracle Working Together"/>
    </classes>
  </Speaker>
  <Speaker name="Richard Campbell">
    <classes>
      <class name="SQL Server Profiler"/>
      <class name="Advanced SQL Querying Techniques"/>
      <class name="SQL Server and Oracle Working Together"/>
      <class name="From 1 Server to 2"/>
    </classes>
  </Speaker>
</Speakers>'
SELECT @xml.query(
  '<Speakers>
  {
  for $Speaker in /Speakers/Speaker
  let $count :=count($Speaker/classes/class)
  order by $count descending
  return
  <Speaker>
  {$Speaker/@name}
  {$Speaker/count}
  <SessionCount>{$count}</SessionCount>
  </Speaker>
  }
  </Speakers>')
```

Вот результат:

```
<Speakers>
  <Speaker name="Richard Campbell">
    <SessionCount>4</SessionCount>
  </Speaker>
  <Speaker name="Stephen Forte">
    <SessionCount>3</SessionCount>
  </Speaker>
</Speakers>
```

Обратите внимание, что Richard отсортировался на самом верху, поскольку мы сортировали вывод по количеству заседаний в убывающем порядке. Для сравнения выражение сортировки использовало переменную, которую мы присвоили при помощи `let`. Если вы для всех незначительных манипуляций с XML использовали инструкцию `return`, то команда `let` вам покажется очень полезной. Хотя это единственное добавление к XQuery в SQL Server 2008, оно весьма желанное.

## Метод `xml.nodes`

Метод `xml.nodes` берет выражение XQuery (`exist`, `value` или `query`) и возвращает экземпляры специального типа `xml` с контекстом, установленным в значение другого узла. Специальный тип данных XML используется для подзапросов к XML-данным вашего результирующего набора.

## Расширения XQuery в SQL Server

Компания Microsoft сделала некоторые расширения спецификации XQuery в SQL Server 2008, начиная с таких задач, как ссылки на не-XML-данные внутри выражений XQuery, и до инструкций языка манипулирования XML-данными (DML).

### Функция `sql:column`

Функция `sql:column` (специфичная для XQuery) позволяет выражению XQuery сослаться на столбец в текущей строке набора данных. Синтаксис таков `{sql:column("ColumnName")}`:

```
for $b in /classes/class
where $b/@name="SQL Server and Oracle Working Together"
```

```
return (<Sessions>{$b}<Speaker id="{sql:column("Speaker_ID")}">
    {sql:column("Speaker_NM")}</Speaker></Sessions>)
```

Сводя все это воедино, создадим атрибут с названием `Speaker id` и значением из столбца `Speaker_ID` текущей строки. Узел `<Sessions>` является конструктором узла, а это означает, что вы можете создать узел как часть вашего `return` в вашем выражении `for`, как это показано здесь:

```
SELECT Speaker_ID, Speaker_NM, Speaker_Country,
       Speaker_XML.query('
    for $b in /classes/class
    where $b/@name="SQL Server and Oracle Working Together"
    return (<Sessions>{$b}<Speaker id="{sql:column("Speaker_ID")}">
        {sql:column("Speaker_NM")}</Speaker></Sessions>
    ') AS Sessions
FROM tblSpeakers
WHERE Speaker_XML.exist('/classes/class[@name="SQL Server
                        and Oracle Working Together"]')=1
```

В XML-результате записи Stephen выражение `xml.query` в `SELECT` соответствует выражению `WHERE`:

```
<Sessions>
  <class name="SQL Server and Oracle Working Together" />
  <Speaker id="1">Stephen Forte</Speaker>
</Sessions>
```

## Функция *sql:variable*

Функция `sql:variable` также уникальна для реализации XQuery в SQL Server. Она работает точно так же, как и `sql:column`, но ссылается на значение переменной (а не на значение столбца в строке). Для текущего запроса у вас есть единственное значение:

```
DECLARE @Conf varchar(20)
SET @Conf= 'NYC .NET User Group'
SELECT Speaker_ID, Speaker_NM, Speaker_XML.query('
    for $b in /classes/class
    where $b/@name="SQL Server and Oracle Working Together"
    return (<Sessions
        conference="{sql:variable("@Conf")}">
        {$b}<Speaker id="{sql:column("Speaker_ID")}">
            {sql:column("Speaker_NM")}</Speaker></Sessions>
    ') AS Sessions
FROM tblSpeakers
```



```
WHERE Speaker_XML.exist('/classes/class[@name="SQL Server  
and Oracle Working Together"]')=1
```

Результаты выглядят следующим образом:

```
<Sessions conference="NYC .NET User Group">  
  <class name="SQL Server and Oracle Working Together" />  
  <Speaker id="1">Stephen Forte</Speaker>  
</Sessions>
```

## Язык XML DML

Спецификация XQuery организации W3C не предоставляет вам способа модифицировать XML-данные так же, как вы можете модифицировать данные реляционных таблиц при помощи ключевых слов INSERT, UPDATE и DELETE языка T-SQL. W3C имеет рабочий проект, но до момента его принятия в качестве стандарта может пройти еще несколько лет, поэтому компания Microsoft создала свой собственный язык манипулирования XML-данными (XML DML), который включен в реализацию XQuery. Версия компании Microsoft соответствует рабочему проекту W3C, но мы совершенно не знаем, совпадут ли текущая реализация компании Microsoft и окончательный вариант стандарта.

XML DML дает вам три способа манипулирования XML-данными столбца через метод `xml.modify` (который мы уже упоминали ранее):

- ❑ `xml.modify(insert)` — позволяет вставить узел или последовательность узлов в экземпляр типа `xml`, с которым вы работаете;
- ❑ `xml.modify(delete)` — позволяет удалить узлы, которые являются результатом выводной последовательности указанного вами выражения XQuery;
- ❑ `xml.modify(replace)` — модифицирует значение единственного узла.

### Метод `xml.modify(insert)`

Метод `xml.modify(insert)` позволяет вставить узел (или последовательность узлов) в экземпляр типа `xml`. Вы используете метод `xml.modify` в сочетании с инструкцией UPDATE языка T-SQL и выражением `where` (T-SQL или XQuery, или обоих). Вы можете также указать ключевое слово `first` или `last` рядом с порядковым номером. Вот простой пример:

```
-- вставить узел XML  
UPDATE tblSpeakers  
SET Speaker_XML.modify(  
  'insert
```

```
<class name="Ranking and Windowing Functions in SQL Server 2008"/>
into /classes[1]')
```

```
WHERE Speaker_ID=1
```

В SQL Server 2008 метод `xml.modify` позволяет заменить XML-элемент переменной `sql:variable`. Согласитесь, легче использовать такой синтаксис для переменной:

```
DECLARE @newElement xml
```

```
SET @newElement = '<class name="Using Ling to SQL"/>'
```

```
UPDATE tblSpeakers
```

```
SET Speaker_XML.modify(
```

```
'insert
```

```
sql:variable("@newElement")
```

```
into /classes[1]')
```

```
WHERE Speaker_ID=1
```

## Метод `xml.modify(delete)`

Метод `xml.modify(delete)` удаляет узлы (в зависимости от указанного вами критерия). Следующий пример удаляет определенный узел (по его порядковому номеру); вы можете также сочетать это с выражением `WHERE` языков T-SQL или XQuery.

```
UPDATE tblSpeakers
```

```
SET Speaker_XML.modify('delete /classes/class[3]')
```

```
WHERE Speaker_ID = 1
```

## Метод `xml.modify(replace)`

Метод `xml.modify(replace)` позволяет вам заменять XML-данные новой информацией. В этом примере заменяется название занятия:

```
UPDATE tblSpeakers
```

```
SET Speaker_XML.modify('
```

```
replace value of
```

```
/classes[1]/class[3]/@name[1]
```

```
with "Protecting against SQL Injection Attacks: Hackers must die"')
```

```
WHERE Speaker_ID = 1
```

## Преобразование столбца в XML

Если у вас есть существующие столбцы `text` или `varchar` в SQL Server 2008, которые содержат XML-данные, вас обрадует возможность использовать тип

данных xml и XQuery. Поскольку вы можете использовать функции CONVERT и CAST языка T-SQL для преобразования этих столбцов в полноценные XML-данные, то несложно написать простой скрипт для обновления.

Сначала нам нужна таблица. Для этого примера мы создадим простую таблицу:

```
CREATE TABLE tblXMLUpgradeTest
(Field_ID int PRIMARY KEY IDENTITY,
 Field_XMLData nvarchar(4000))
```

Затем мы вставим некие данные, но не забудьте, что они хранятся как текст, а не как XML:

```
-- вставить некие данные как текст
-- помните, что не разрешается никакого XQuery или чего-то подобного!
DECLARE @xmlData AS varchar(8000)
SET @xmlData =
(SELECT Customer.CustomerID, OrderDetail.SalesOrderID,
                                     OrderDetail.OrderDate
FROM Sales.Customer Customer
  INNER JOIN Sales.SalesOrderHeader OrderDetail
    ON OrderDetail.customerid=Customer.customerid
WHERE Customer.CustomerID=11000
ORDER BY Customer.CustomerID
FOR XML AUTO, ROOT('Orders'))
-- сначала вставить в таблицу
INSERT INTO tblXMLUpgradeTest(Field_XMLData) VALUES (@xmlData)
```

Теперь вы можете конвертировать ваши данные из простых строк в XML-столбец при помощи последовательного выполнения шести шагов:

1. В таблице, которая содержит строковое поле с данными, подлежащими конвертированию в XML, добавьте поле varchar(8000) или text (в зависимости от исходного столбца). Это временный столбец для хранения предварительно преобразованных данных. (Позднее мы удалим этот столбец).
2. Добавьте данные текущего столбца на основе текста в новый временный столбец.
3. Удалите исходный столбец на основе текста.
4. Добавьте название исходного столбца обратно, но уже с типом xml.
5. Вставьте данные из временного столбца, используя инструкцию CONVERT для преобразования данных к типу xml.
6. Удалите временный столбец.

Пример кода (со всеми шестью шагами) показан в листинге 6.13.

#### Листинг 6.13. Конвертирование текстовых данных в XML

```
-- во-первых, измените таблицу
ALTER TABLE tblXMLUpgradeTest ADD Field_XMLData_Temp varchar(8000)
GO

-- во-вторых, обновите столбец Field_XMLData_Temp
UPDATE tblXMLUpgradeTest SET Field_XMLData_Temp = Field_XMLData
GO

-- в-третьих, удалите текстовый столбец (Field_XMLData)
ALTER TABLE tblXMLUpgradeTest DROP COLUMN Field_XMLData
GO

-- в-четвертых, добавьте тот же самый столбец (Field_XMLData),
-- но уже как XML
ALTER TABLE tblXMLUpgradeTest ADD Field_XMLData XML
GO

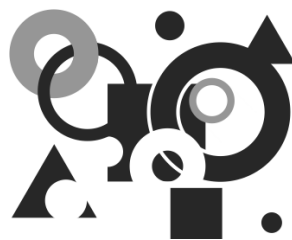
-- в-пятых, вставьте XML в Field_XMLData и преобразуйте
-- к типу данных XML
UPDATE tblXMLUpgradeTest SET Field_XMLData =
                                CONVERT(xml, Field_XMLData_Temp)
GO

-- в-шестых, удалите столбец Field_XMLData_Temp,
-- поскольку в нем текст, а не XML
ALTER TABLE tblXMLUpgradeTest DROP COLUMN Field_XMLData_Temp
GO
```

## Резюме

Мы предприняли чрезвычайно обширное путешествие по типу данных `xml` и механизмам манипуляции его данными (XQuery и XML DML). Как вы видели, SQL Server 2008 предоставляет богатый набор функциональных возможностей XML-технологий. Иногда вы будете хранить XML в базе данных, а тип данных `xml` позволит вам работать с XML собственными средствами, что является значительным улучшением по сравнению с более ранними версиями SQL Server. Вооружившись этим типом данных и возможностью запрашивать его при помощи XQuery, вы можете полностью применять мощь XML внутри ваших баз данных SQL Server 2008 и строить интеллектуальные приложения с использованием XML.

## ГЛАВА 7



# Иерархические данные и реляционная база данных

*Леонард Лобел*

Иерархические данные не являются реляционными. Они организованы как древовидная структура родительских и дочерних элементов (а не как плоские списки строк в связанных таблицах). Самым часто встречающимся форматом иерархических данных на сегодняшний день является XML, и в *главе 6* мы описали богатую поддержку, предоставляемую в Microsoft SQL Server. XML особенно полезен, когда клиентские приложения используют эти данные именно в XML-формате, либо в тех обстоятельствах, когда вам нужно сохранить и извлечь всю иерархию целиком за один раз.

Но если все, что вам нужно — это возможности иерархического связывания между строками, использование типа `xml` означает отказ от реляционной модели и подчинение иерархической, поскольку ваши данные инкапсулированы внутри единого XML-документа (или фрагмента), состоящего из элементов и атрибутов. Возможно, вам не нужны такие крайности только для того, чтобы получить в базе данных иерархические возможности. Вместо этого вы можете продолжать хранить ваши данные как плоский список строк и столбцов внутри таблицы — если это имеет больше смысла для вашего приложения — и использовать тип данных `hierarchyid` для реализации древовидной организации строк таблицы.

Мы моделируем иерархии реляционных таблиц путем установления связей между родительскими и дочерними объектами (связь "один-ко-многим" от Customer к Order или от Order к Order Detail). Такой подход реализует жесткую иерархию, которая ограничена определенными связями между таблицами, и он не может приспособиться к действительно гибкой древовидной

организации с практически неограниченной шириной и глубиной, как форумы и списки адресатов электронной почты, диаграммы организации бизнеса, категории управления контентом, категории продуктов, управление файлами/каталогами. Это только несколько примеров, смысл в том, что они иерархичны по своей сути, потенциально могут расти вширь и вглубь, и обычно по ним делаются рекурсивные итерации.

Одним из традиционных решений получения древовидной структуры для таблицы является подход самосоединения, когда каждая строка имеет первичный ключ и внешний ключ к первичному ключу своей родительской строки (которая находится где-то в другом месте таблицы). Классический пример — иерархия подчиненности в структуре работников коммерческой организации. Единая таблица содержит работников, которые имеют как первичный ключ `EmployeeId`, так и внешний ключ `ManagerId`, который указывает на `EmployeeId` менеджера каждого сотрудника. (У генерального директора значение `ManagerId` будет `NULL`).

Такой подход с самосоединением требует от вас самостоятельного обслуживания древовидной структуры. В главе 2 вы видели, как обобщенные табличные выражения (СТЕ) поддерживают рекурсивные запросы (которые могут "пройти" вверх по всей иерархии до корня из любого заданного узла) по таблицам. К тому же за управление иерархией при обновлениях все равно отвечаете вы. Вы должны поддерживать структуру при всех манипуляциях с узлами: вставки, модификации и удаления дочерних и родительских строк, а также строк того же уровня таблицы. Манипуляции поддеревьями могут приводить к сложным транзакциям, которые вам придется реализовывать. Отсутствует способ тонкого управления порядком дочерних объектов. Например, вы не можете сказать: "вставить дочерний узел перед, после или между другими существующими узлами того же уровня".

## Тип данных *hierarchyid*

Новый тип данных `hierarchyid` в SQL Server 2008 позволяет реализовать надежную иерархическую структуру, которая освободит вас от большей части этих обязанностей по обслуживанию. Как и при подходе с самосоединением, каждая строка таблицы представляет собой узел дерева. Однако вместо указания родительского значения в столбце внешнего ключа каждая строка теперь имеет уникальное значение `hierarchyid`, которое описывает полный путь от корня дерева до положения в дереве узла данной строки. Поэтому

значение `hierarchyid` каждой строки идентифицирует положение каждого узла в дереве относительно его родителя, узлов того же уровня, а также дочерних узлов (которые являются просто строками в той же таблице со своим собственным значением `hierarchyid`). Таким образом, значения `hierarchyid` связывают строки друг с другом (как узлы, формирующие древовидную структуру).

Используя новые расширения T-SQL, которые поддерживают тип данных `hierarchyid`, вы можете вызывать методы для произвольной вставки, модификации и удаления узлов в любой точке структуры очень эффективным образом. Вы можете даже изменить родителя целого поддерева (одним обновлением). SQL Server определяет затронутые обновлением узлы и автоматически управляет изменениями их значений `hierarchyid`. Кроме того, запрос по таблице древовидной структуры с новым типом данных `hierarchyid` гораздо быстрее, чем выполнение рекурсивных запросов по обычной таблице с самосоединением при помощи CTE.

SQL Server использует алгоритм, который кодирует информацию о пути (в значении `hierarchyid`) в чрезвычайно компактный формат переменной длины. Среднее количество битов для представления узла в дереве зависит от среднего количества дочерних элементов узла, известного как "среднее разветвление, один узел в иерархии из 100 000 узлов со средним разветвлением 6 занимает примерно 38 битов, что при хранении округляется до 40 битов (5 байтов). Закодированное значение, безусловно, не имеет никакого осмысленного отображения, однако метод `ToString` переводит значение `hierarchyid` в читаемый человеком формат, который показывает путь с разделителями (и скоро вы это увидите).

Тип `hierarchyid` реализован как определяемый пользователем тип UDT общезыковой исполняющей среды CLR (в отличие от более примитивных типов данных SQL Server, таких как `int` или `date`). Это означает, что `hierarchyid` размещен в CLR, а не в SQL Server (и именно поэтому вызываемые по нему функции называются методами). Другими словами, вам не нужно включать SQL CLR для того, чтобы использовать новый тип данных `hierarchyid` или любые другие новые системные типы CLR, такие как `geometry` и `geography`, которые мы описываем в *главе 9*. Как говорилось в *главе 3*, интеграция SQL CLR увеличивает вашу уязвимость, потому что позволяет коду Microsoft .NET Framework выполняться на сервере, поэтому включать SQL CLR следует только тогда, когда вы создаете свои собственные объекты SQL CLR (хранимые процедуры, определяемые пользователем функции (UDF) и т. д.).

## ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Тип данных `hierarchyid` доступен клиентам CLR как тип данных `SqlHierarchyId` пространства имен `Microsoft.SqlServer.Types`. Это пространство имен содержится внутри сборки `Microsoft.SqlServer.Types.dll` на вкладке **.NET** в диалоговом окне **Add Reference** Microsoft Visual Studio. В самой базе данных информация об узле (представленная значением `hierarchyid`) хранится как тип данных `varbinary`, который может поддерживать практически неограниченное количество дочерних узлов на один родительский узел.

## Создание иерархической таблицы

Давайте начнем с реализации классического дерева менеджер—сотрудник с использованием нового типа данных `hierarchyid`. Мы начинаем со структуры таблицы `Employee` (листинг 7.1).

### Листинг 7.1. Создание иерархической таблицы

```
CREATE DATABASE MyDB
GO
USE MyDB
GO
CREATE TABLE Employee
(
    NodeId          hierarchyid PRIMARY KEY CLUSTERED,
    NodeLevel       AS NodeId.GetLevel(),
    EmployeeId      int UNIQUE NOT NULL,
    EmployeeName    varchar(20) NOT NULL,
    Title           varchar(20) NULL
)
GO
```

Обратите внимание, что столбец `NodeId` объявляется как новый тип `hierarchyid`, а также как первичный ключ таблицы (кластеризованный индекс). Первичный ключ важен по двум причинам, из них первая — то, что SQL Server не гарантирует уникальности значений `hierarchyid` в таблице. При наличии в нашем примере всего одной иерархии определение первичного ключа является одним из способов обеспечения уникальности значений `NodeId`. Определение обычного ограничения уникальности является еще од-



ним способом, когда какой-то другой столбец (не `hierarchyid`) более подходит для первичного ключа таблицы. Вторая причина важности первичного ключа — это поддержка индексирования в глубину (о которой мы скоро поговорим).

Как мы уже начали объяснять, значение в столбце `NodeId` идентифицирует уникальное положение узла каждой строки в структуре. Оно может служить первичным ключом таблицы. Присваиваемые по мере добавления строк в таблицу столбцу `hierarchyid` значения основаны на положении узлов в дереве. Эти значения `hierarchyid` будут автоматически меняться по мере изменения структуры узлов дерева (при помощи методов, которые вы будете изучать чуть позже). Например, если вы переносите узел и всех его потомков в новое положение на дереве (это изменение родителя поддерева), то значения `hierarchyid` этих узлов обновляются сервером SQL Server). Поэтому мы определяем `EmployeeId` как уникальный целочисленный столбец со значениями, которые мы контролируем и которые не изменятся при манипуляциях со структурой узлов дерева в будущем. Поэтому при добавлении сотрудников значения их ID остаются теми же самыми (независимо от того, как их узлы будут переставляться в будущем).

### **ПРИМЕЧАНИЕ**

Вы можете использовать значения `IDENTITY` для столбца `EmployeeId`, что дает серверу SQL Server возможность управлять значениями ключа, которые не изменятся при модификации древовидной структуры. Для того чтобы не отвлекаться от типа `hierarchyid`, мы в нашем примере не добавляем кода использования `IDENTITY`, а просто явно присваиваем уникальные целые значения `EmployeeId`.

## **Метод *GetLevel***

Второй определенный в таблице `EmployeeId` столбец — это `NodeLevel`, объявленный нами как вычисляемый столбец, который возвращает 16-битный целочисленный результат метода `GetLevel`, примененного к значению `hierarchyid` столбца `NodeId`. Этот необязательный столбец только для чтения возвращает номер, указывающий положение уровня его строки (узла), извлеченный и декодированный из сжатого значения `hierarchyid` столбца `NodeId`. В любой иерархии существует одна (и только одна) корневая строка, и она находится на уровне 0. Ее дочерние строки находятся на уровне 1, внучатые строки — на уровне 2. Определение столбца `NodeLevel` (который вы-

зывает метод `GetLevel`) дает нам удобное, легкое для чтения и простое для запросов значение уровня (из значения `hierarchyid`, находящегося в `NodeId`). Кроме того, наш столбец `NodeLevel` позволит создать индекс таблицы в ширину. Мы скоро обсудим индексы в глубину и в ширину.

Столбец `EmployeeId` определен с ограничением уникальности и не принимает значения `NULL`. Поэтому мы можем считать его целое значение "альтернативным первичным ключом" для наших работников, поскольку значение `hierarchyid` в столбце `NodeId` уже было определено как фактический первичный ключ таблицы. Альтернативный ключ не меняется непредсказуемым образом в результате манипулирования узлами (как значение `hierarchyid`). Вам определенно понадобятся такие идентификаторы, если вы захотите установить связи с другими таблицами, хотя вы можете привести тип иерархической структуры к реляционной таблице при помощи одного только значения `hierarchyid` в качестве первичного ключа (других уникальных столбцов не требуется).

## Заполнение иерархии

Перед тем как начать заполнять таблицу `Employee` иерархическими данными, давайте визуализируем нашу организационную диаграмму. На рис. 7.1 показана иерархия сотрудников нашей организации.

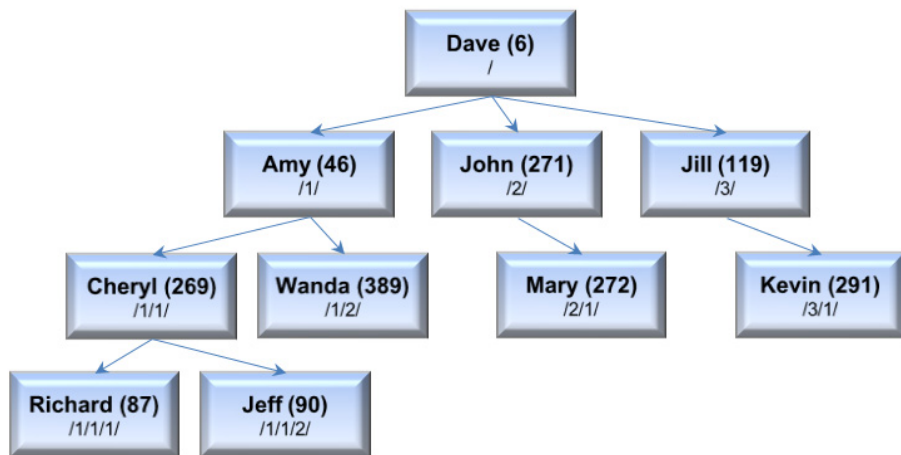


Рис. 7.1. Иерархическая организационная диаграмма

Первая добавляемая в иерархическую таблицу строка всегда является корневым узлом дерева (и корень может быть только один). Затем добавляются дополнительные строки ниже корня (как дочерние и внучатые узлы) и т. д. Узлы можно по мере необходимости переупорядочивать (вызывая методы по значениям `hierarchyid`). Дерево может расти как вширь (с большим количеством узлов на одном уровне), так и вглубь (иметь много потомков).

## Метод *GetRoot*

Мы начнем со вставки нашей первой строки для Dave (менеджер по маркетингу), который находится на самом верху диаграммы с ID 6 (как показано в листинге 7.2).

### Листинг 7.2. Вставка строки для корневого узла в иерархическую таблицу `Employee`

```
INSERT INTO Employee
(NodeId, EmployeeId, EmployeeName, Title)
VALUES
(hierarchyid::GetRoot(), 6, 'Dave', 'Marketing Manager')
GO
```

Значения столбцам `hierarchyid` можно присваивать при помощи вызова методов, которые возвращают значения `hierarchyid`. При вставке первой строки (самый верх диаграммы) мы вызываем метод `GetRoot` по самому типу `hierarchyid`. Специальный синтаксис с двойным двоеточием (::) используется при вызове методов по самому типу данных подобно вызову статического метода в объектно-ориентированных языках, в отличие от вызова методов по значению типа данных (подобно вызову экземпляра метода), в котором используется обычный синтаксис с точкой, как мы уже видели в случае метода `GetLevel` в листинге 7.1. Поскольку это первая строка нашей иерархической таблицы, то единственным допустимым значением `hierarchyid` для `NodeId` является то, которое указывает на корень, и именно это значение возвращает `hierarchyid::GetRoot()`.

Обратите также внимание, что мы не вставляем (и не можем вставлять) значение `NodeLevel`, поскольку это вычисляемый столбец "только для чтения", который возвращает положение уровня каждой строки.

Давайте выберем строку из таблицы, чтобы увидеть, как она выглядит:

```
SELECT * FROM Employee
```

```
GO
```

| NodeId | NodeLevel | EmployeeId | EmployeeName | Title             |
|--------|-----------|------------|--------------|-------------------|
| 0x     | 0         | 6          | Dave         | Marketing Manager |

(1 row(s) affected)

## Метод *GetDescendant*

В предыдущих выходных данных мы видим запись для Dave как корневой узел нашей иерархии (на уровне 0). Значение `hierarchyid` (равное 0x) для `NodeId` позиционирует Dave как узел верхнего уровня (это установлено методом `GetRoot` при вставке строки). Теперь настало время добавить дочерние узлы для сотрудников первого уровня, которые непосредственно подчиняются Dave. Для установки значений `hierarchyid` этих сотрудников мы используем метод `GetDescendant` по значению `hierarchyid` для Dave. Этот метод генерирует и возвращает значение `hierarchyid`, которое является дочерним узлом того же уровня, что и два переданных как параметры метода значения `hierarchyid`.

Теперь давайте сделаем паузу перед добавлением сотрудников под Dave, чтобы дополнительно изучить, как использовать метод `GetDescendant`. Предположим, что мы хотим сконструировать дерево, как показано на рис. 7.2.

Код листинга 7.3 показывает, как нужно использовать `GetDescendant` для дочерних и внучатых узлов корневого узла (созданного методом `GetRoot`).

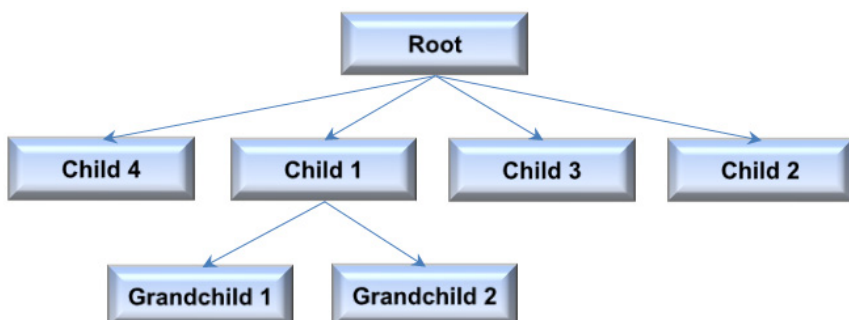


Рис. 7.2. Пример иерархии узлов

**Листинг 7.3. Использование GetDescendant для конструирования иерархии**

```

DECLARE @Root hierarchyid
DECLARE @Child1 hierarchyid
DECLARE @Child2 hierarchyid
DECLARE @Child3 hierarchyid
DECLARE @Child4 hierarchyid
DECLARE @Grandchild1 hierarchyid
DECLARE @Grandchild2 hierarchyid
SET @Root = hierarchyid::GetRoot()
SET @Child1 = @Root.GetDescendant(NULL, NULL)
SET @Child2 = @Root.GetDescendant(@Child1, NULL)
SET @Child3 = @Root.GetDescendant(@Child1, @Child2)
SET @Child4 = @Root.GetDescendant(NULL, @Child1)
SET @Grandchild1 = @Child1.GetDescendant(NULL, NULL)
SET @Grandchild2 = @Child1.GetDescendant(@Grandchild1, NULL)
SELECT
    @Root AS Root,
    @Child1 AS Child1,
    @Child2 AS Child2,
    @Child3 AS Child3,
    @Child4 AS Child4,
    @Grandchild1 AS Grandchild1,
    @Grandchild2 AS Grandchild2

```

Выходные данные выглядят следующим образом:

```

Root Child1 Child2 Child3 Child4 Grandchild1 Grandchild2
-----
0x    0x58    0x68    0x62C0 0x48    0x5AC0        0x5B40
(1 row(s) affected)

```

## Метод ToString

Предыдущие выходные данные показывают шестнадцатеричное представление закодированного значения `hierarchyid` каждого узла, которое для нас практически бесполезно. Для того чтобы просмотреть его в виде текста (ко-

торый содержит путь к каждому узлу дерева), измените инструкцию `SELECT` в листинге 7.3 для вызова метода `ToString` по значениям `hierarchyid`:

```
SELECT
    @Root.ToString() AS Root,
    @Child1.ToString() AS Child1,
    @Child2.ToString() AS Child2,
    @Child3.ToString() AS Child3,
    @Child4.ToString() AS Child4,
    @Grandchild1.ToString() AS Grandchild1,
    @Grandchild2.ToString() AS Grandchild2
GO
Root Child1 Child2 Child3 Child4 Grandchild1 Grandchild2
-----
/      /1/      /2/      /1.1/ /0/      /1/1/      /1/2/
(1 row(s) affected)
```

Мы видим, что метод `ToString` декодирует значение `hierarchyid` в строку с разделителем в виде косой черты. Каждая косая черта обозначает уровень, поэтому вычитание 1 из количества косых черт дает положение уровня каждого узла относительно нулевого (то же самое значение, которое было бы возвращено методом `GetLevel`). `Root` — единственный узел с одной чертой (он представляет уровень 0). Узлы `Child1`, `Child2`, `Child3` и `Child4` имеют по две косых черты и представляют уровень 1. Узлы `Grandchild1` и `Grandchild2` имеют три косых черты и представляют уровень 2.

Мы видим также, как два переданных в метод `GetDescendant` параметра повлияли на присвоение значений `hierarchyid`. Конечно, `Root` был присвоен при помощи `hierarchyid::GetRoot()`, чтобы дать начало дереву (как вы видели в листинге 7.2). Значение `hierarchyid` для каждого из четырех дочерних узлов (под корневым родительским узлом) было присвоено вызовом метода `GetDescendant` по корню. А порядковый номер каждого дочернего узла среди узлов того же уровня (и от того же родителя) определялся двумя параметрами (переданными в `GetDescendant`).

`Child1` — это первый узел, определенный под корневым:

```
SET @Child1 = @Root.GetDescendant (NULL, NULL)
```

Поскольку пока под корневым узлом нет никаких других дочерних узлов, то указать положение относительно других дочерних узлов нельзя. Поэтому мы передаем в метод `GetDescendant` значение `NULL` в обоих позиционных параметрах. В результате `hierarchyid` будет представлен как `/1/`, что обозначает единственный дочерний узел под корневым.

Child2 создается как второй узел под корневым, причем мы хотим, чтобы он разместился после Child1. Представьте себе это как вставку Child2 между Child1 и "концом", что мы и передаем соответственно как два параметра Child1 и NULL при вызове GetDescendant для Child2:

```
SET @Child2 = @Root.GetDescendant(@Child1, NULL)
```

В результате значение hierarchyid представляется как /2/, что обозначает второй дочерний узел под корнем (правее Child1). Затем мы применяем этот же принцип к созданию Child3 как третьего дочернего узла под корнем (отличие только в том, что мы хотим вставить этот дочерний узел между Child1 и Child2, а не добавлять в конец, как мы сделали это с Child2). Мы явно запрашиваем такой порядок при вызове GetDescendant для Child3:

```
SET @Child3 = @Root.GetDescendant(@Child1, @Child2)
```

Этот код присваивает значение hierarchyid для Child3 между Child1 и Child2. Поскольку мы указали положение между Child1 и Child2 со значениями hierarchyid /1/ и /2/ соответственно, то SQL Server присваивает для Child3 такое значение hierarchyid, которое представляется методом ToString как /1.1/ (то есть после /1/ и перед /2/).

И наконец, мы создаем Child4 как еще один дочерний узел под корнем, на этот раз вставляя его перед всеми остальными узлами этого же уровня (то есть перед Child1). Представьте себе это как вставку Child4 между "началом" и Child1, которые мы и передаем как два параметра (NULL и Child1 соответственно) при вызове GetDescendant для Child4:

```
SET @Child4 = @Root.GetDescendant(NULL, @Child1)
```

В результате получается hierarchyid, который представляется для Child4 в виде /0/. Это последний дочерний узел, добавленный под корневым левее Child1. Если бы мы продолжали добавлять узлы левее Child4, то их представления ToString показывали бы отрицательные значения, такие как /-1/, /-2/ и т. д.

Внучатые узлы (второго уровня) ниже Child1 добавляются аналогично:

```
SET @Grandchild1 = @Child1.GetDescendant(NULL, NULL)
```

Этот код присваивает значение /1/1/ для hierarchyid узла Grandchild1, первого дочернего узла под Child1. И точно так же, поскольку это первый дочерний узел под родительским, на этом уровне пока еще нет узлов, по отношению к которым мы могли бы указать положение Grandchild1. Поэтому единственно возможными значениями для передачи в качестве параметров в GetDescendant являются значения NULL и NULL. И наконец, мы добавляем Grandchild2 как второй дочерний узел под Child1, располагая его после Grandchild1:

```
SET @Grandchild2 = @Child1.GetDescendant(@Grandchild1, NULL)
```

Теперь давайте вернемся к нашей реальной иерархической таблице сотрудников. После добавления Dave в корень узлам первого уровня (которые мы теперь будем добавлять ниже него) присваиваются значения `hierarchyid`, возвращаемые вызовом `GetDescendant` для Dave. Первый сотрудник первого уровня — это Amy, ID 46 (как показано в листинге 7.4).

#### Листинг 7.4. Добавление первого дочернего узла под корнем иерархии

```
DECLARE @Manager hierarchyid
SELECT @Manager = NodeId
FROM Employee
WHERE EmployeeId = 6
INSERT INTO Employee
(NodeId, EmployeeId, EmployeeName, Title)
VALUES
(@Manager.GetDescendant(NULL, NULL), 46, 'Amy', 'Marketing Specialist')
GO
```

Давайте внимательно изучим этот код. Сначала нам нужно получить значение `hierarchyid` родителя (Dave, ID 6), мы извлекаем его из таблицы и сохраняем в переменной `@Manager` (также объявленной как тип данных `hierarchyid`). Затем мы можем вставить Amy как первую дочернюю строку (при помощи указания значения `hierarchyid`, возвращаемого методом `GetDescendant` для `@Manager`). Поскольку пока других дочерних узлов под Dave нет, то единственными значениями позиционных параметров для метода `GetDescendant` могут быть только значения `NULL` и `NULL`. Давайте теперь рассмотрим обе строки таблицы, включая и столбец `NodeIdPath`, который транслирует закодированные значения `hierarchyid` при помощи метода `ToString`:

```
SELECT NodeId.ToString() AS NodeIdPath, *
FROM Employee
GO
```

| NodeIdPath | NodeId | NodeLevel | EmployeeId | EmployeeName | Title                |
|------------|--------|-----------|------------|--------------|----------------------|
| /          | 0x     | 0         | 6          | Dave         | Marketing Manager    |
| /1/        | 0x58   | 1         | 46         | Amy          | Marketing Specialist |

(2 row(s) affected)

Действительно, Amy — дочерний узел первого уровня под Dave, на что указывает присвоенное методом `GetDescendant` значение `hierarchyid`, равное `/1/`.



Это показывает, что добавление дочернего узла требует сначала выполнения `SELECT` (получения значения `hierarchyid` родительского узла) до выполнения `INSERT` (фактической вставки новой строки). Таким образом, новой строке может быть присвоено значение `hierarchyid` на основе результата вызова `GetDescendant` по полученному от ее родителя значению `hierarchyid`. Для того чтобы данный процесс стал понятнее, мы написали этот код как две отдельных инструкции. Однако мы могли бы написать это и как одну инструкцию (без временной переменной `@Manager` для хранения родительского значения `hierarchyid`). Для этого необходимо встроить инструкцию `SELECT` как подзапрос в выражение `VALUES` инструкции `INSERT`:

```
INSERT INTO Employee
    (NodeId, EmployeeId, EmployeeName, Title)
VALUES (
    (SELECT NodeId
     FROM Employee
     WHERE EmployeeId = 6).GetDescendant(NULL, NULL),
    46,
    'Amy',
    'Marketing Specialist')
GO
```

Мы хотим упростить процесс добавления дочерних и внучатых узлов в иерархию, поэтому создадим хранимую процедуру `uspAddEmployee`, которая инкапсулирует логику извлечения `hierarchyid` из любого нужного нам родителя и выполнения по нему `GetDescendant` для получения соответствующего значения `hierarchyid` для нового сотрудника. Код хранимой процедуры показан в листинге 7.5.

**Листинг 7.5. Создание хранимой процедуры для автоматической вставки дочерних узлов в иерархию**

```
CREATE PROCEDURE uspAddEmployee(
    @ManagerId int,
    @EmployeeId int,
    @EmployeeName varchar(20),
    @Title varchar(20))
AS
BEGIN
    DECLARE @ManagerNodeId hierarchyid
    DECLARE @LastManagerChild hierarchyid
```

```

DECLARE @NewEmployeeNodeId hierarchyid
-- Получить hierarchyid родителя, переданный в @ManagerId
SELECT @ManagerNodeId = NodeId
FROM Employee
WHERE EmployeeId = @ManagerId
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE
BEGIN TRANSACTION
-- Получить hierarchyid последнего существующего дочернего
-- узла под родителем
SELECT @LastManagerChild = MAX(NodeId)
FROM Employee
WHERE NodeId.GetAncestor(1) = @ManagerNodeId
-- Присвоить новый hierarchyid в конце существующих узлов
-- того же уровня
SELECT @NewEmployeeNodeId =
    @ManagerNodeId.GetDescendant(@LastManagerChild, NULL)
-- Добавить строку
INSERT INTO Employee
(NodeId, EmployeeId, EmployeeName, Title)
VALUES
    (@NewEmployeeNodeId, @EmployeeId, @EmployeeName, @Title)
COMMIT
END
GO

```

Теперь мы можем использовать эту хранимую процедуру для вставки новых строк в таблицу сотрудников на любом уровне иерархии (кроме корневого). Хранимая процедура принимает параметр @ManagerId, который вызывающая сторона предоставляет как идентификатор сотрудника того родительского узла, дочерним элементом которого должен быть добавляемый новый сотрудник. Остальные параметры (@EmployeeId, @EmployeeName и @Title) просто предоставляют значения для нового сотрудника.

После объявления нескольких локальных переменных типа hierarchyid процедура извлекает hierarchyid родителя, указанного параметром @ManagerId, и сохраняет результат в локальной переменной @ManagerNodeId:

```

-- Получить hierarchyid необходимого родителя, переданного в @ManagerId
SELECT @ManagerNodeId = NodeId
FROM Employee
WHERE EmployeeId = @ManagerId

```

Это значение, по которому мы сейчас вызовем `GetDescendant` (установить соответствующее значение `hierarchyid` для нового сотрудника). Нас не волнует порядок следования узлов одного уровня под каждым из родителей, поэтому мы хотим, чтобы процедура просто добавляла нового сотрудника после всех существующих узлов данного уровня. Однако мы по-прежнему должны предоставить два позиционных параметра, которых ждет от нас метод `GetDescendant` и которые явным образом указывают место вставки. Поскольку мы хотим, чтобы новые узлы того же уровня добавлялись в конец, то мы знаем, что второй позиционный параметр должен всегда быть `NULL`. Следовательно, наша задача состоит в том, чтобы установить соответствующее значение для первого позиционного параметра (здесь должно быть либо значение `hierarchyid` последнего уже существующего на данном уровне узла, либо значение `NULL` в том случае, если других таких узлов нет). Мы делаем это при помощи следующей инструкции `SELECT`:

```
-- Получить hierarchyid последнего дочернего элемента под родителем
SELECT @LastManagerChild = MAX(NodeId)
FROM Employee
WHERE NodeId.GetAncestor(1) = @ManagerNodeId
```

## Метод *GetAncestor*

Предшествующий код представляет новый метод с названием `GetAncestor`. Этот метод принимает целочисленный параметр количества уровней вверх от узла (для которого он вызван) до одного из предков этого узла, а затем возвращает `hierarchyid` этого предка. Значение 1 (которое мы здесь используем) возвращает непосредственного родителя того узла, для которого вызывается метод `GetAncestor`. Запрашивая все строки со значениями `hierarchyid`, непосредственный родитель которых тот же, что и родитель, запрошенный вызывающей стороной для нового сотрудника, предыдущее выражение `WHERE` извлекает все существующие узлы того уровня, на который будет добавлен новый сотрудник. Поскольку при этом может быть возвращено множество узлов одного уровня, то мы применяем функцию агрегирования `MAX` для извлечения из них узла с самым большим `hierarchyid` (который является последним в этом наборе) в локальную переменную `@LastManagerChild`. Если узлов на этом уровне нет, то выражение `WHERE` не вернет никаких строк и переменной `@LastManagerChild` будет присвоено значение `NULL`. В любом случае мы установили соответствующее

значение в `@LastManagerChild`, которое будет использовано как первый позиционный параметр для вызова `GetDescendant`:

```
-- Присвоить новый hierarchyid в конце существующих узлов одного уровня
SELECT @NewEmployeeNodeId =
    @ManagerNodeId.GetDescendant(@LastManagerChild, NULL)
```

Теперь мы присвоили `@NewEmployeeNodeId` соответствующее значение `hierarchyid` для нового сотрудника (при помощи вызова `GetDescendant` по родителю `@ManagerNodeId`). Два переданных методу `GetDescendant` параметра указывают, что новый сотрудник должен быть вставлен между последним существующим узлом `@LastManagerChild` и концом списка узлов `NULL`, что по сути означает добавление его в конец списка. Если бы `@LastManagerChild` было присвоено значение `NULL` (по причине отсутствия других узлов того же уровня), то `GetDescendant` был бы вызван с двумя значениями `NULL`, что (как вы уже видели) является правильным способом вызова `GetDescendant` для добавления первого дочернего узла под родителем.

Все, что теперь необходимо — это простая инструкция `INSERT`, которая добавляет нового сотрудника в таблицу с правильным положением узла под родительским элементом (указанным вызывающей стороной):

```
-- Добавить строку
INSERT INTO Employee
    (NodeId, EmployeeId, EmployeeName, Title)
VALUES
    (@NewEmployeeNodeId, @EmployeeId, @EmployeeName, @Title)
```

Последняя заслуживающая пояснения часть этой хранимой процедуры — это использование транзакции вокруг инструкций `SELECT` и `INSERT`. Здесь цель состоит в том, чтобы гарантировать, что ни один узел того же уровня не может быть добавлен другим пользователем между временем установления значения `hierarchyid` последнего узла и временем фактической вставки нового узла. Без заключения этой логики в транзакцию такой гарантии у нас нет, и определенное в качестве последнего узла значение `hierarchyid` может уже и не быть последним узлом данного уровня в момент добавления нового сотрудника. Может произойти конфликт между двумя отдельными параллельными процессами, пытающимися одновременно добавить двух разных новых сотрудников под одним и тем же родительским узлом в одну и ту же позицию. Такая попытка приведет к нарушению ограничения первичного ключа (поскольку обе вставки будут пытаться сохранить одно и то же значение `hierarchyid` в столбце `NodeId`, определенном как первичный ключ). Несмотря на то, что инструкции `SELECT` и `INSERT` можно сочетать в одной сложной инструкции без исполь-

зования явной транзакции (как вы видели ранее при добавлении строки сотрудника Amy), логика гораздо яснее (и поэтому гораздо легче в обслуживании) при кодировании этих инструкций по отдельности.

Теперь, когда мы создали хранимую процедуру `uspAddEmployee` и понимаем то, как она работает, добавить остальных сотрудников из нашей организационной диаграммы несложно. Все, что нам нужно сделать для каждого из них — это указать идентификатор того сотрудника, который является родителем для нового сотрудника, идентификатор нового сотрудника, его имя и должность:

```
EXEC uspAddEmployee 6, 271, 'John', 'Marketing Specialist'
EXEC uspAddEmployee 6, 119, 'Jill', 'Marketing Specialist'
EXEC uspAddEmployee 46, 269, 'Cheryl', 'Marketing Assistant'
EXEC uspAddEmployee 46, 389, 'Wanda', 'Business Assistant'
EXEC uspAddEmployee 271, 272, 'Mary', 'Marketing Assistant'
EXEC uspAddEmployee 119, 291, 'Kevin', 'Marketing Intern'
EXEC uspAddEmployee 269, 87, 'Richard', 'Business Intern'
EXEC uspAddEmployee 269, 90, 'Jeff', 'Business Intern'
```

Метод `ToString` по значению `NodeId` каждой строки подтверждает, что древовидная структура правильно представляет нашу организационную диаграмму:

```
SELECT NodeId.ToString() AS NodeIdPath, *
FROM Employee
ORDER BY NodeLevel, NodeId
GO
```

| NodeIdPath | NodeId | NodeLevel | EmployeeId | EmployeeName | Title                |
|------------|--------|-----------|------------|--------------|----------------------|
| /          | 0x     | 0         | 6          | Dave         | Marketing Manager    |
| /1/        | 0x58   | 1         | 46         | Amy          | Marketing Specialist |
| /2/        | 0x68   | 1         | 271        | John         | Marketing Specialist |
| /3/        | 0x78   | 1         | 119        | Jill         | Marketing Specialist |
| /1/1/      | 0x5AC0 | 2         | 269        | Cheryl       | Marketing Assistant  |
| /1/2/      | 0x5B40 | 2         | 389        | Wanda        | Business Assistant   |
| /2/1/      | 0x6AC0 | 2         | 272        | Mary         | Marketing Assistant  |
| /3/1/      | 0x7AC0 | 2         | 291        | Kevin        | Marketing Intern     |
| /1/1/1/    | 0x5AD6 | 3         | 87         | Richard      | Business Intern      |
| /1/1/2/    | 0x5ADA | 3         | 90         | Jeff         | Business Intern      |

(10 row(s) affected)

Метод ToString конечно отображает значение hierarchyid в гораздо более понятном виде, чем внутреннее значение varbinary, но возвращенная строка по-прежнему остается цифровой и не дает информации. При помощи небольшого кодирования мы можем создать определяемую пользователем функцию (UDF), которая принимает NodeId и возвращает более осмысленное представление пути к данному узлу в стиле "цепочки хлебных крошек" (breadcrumbs). В листинге 7.6 показан код для такой функции fnGetFullDisplayPath с использованием методов GetLevel и GetAncestor:

**Листинг 7.6. Создание UDF, которая строит путь для заданного узла в стиле breadcrumbs**

```
CREATE FUNCTION dbo.fnGetFullDisplayPath(@EntityNodeId hierarchyid)
    RETURNS varchar(max)
AS
BEGIN
    DECLARE @EntityLevelDepth smallint
    DECLARE @LevelCounter smallint
    DECLARE @DisplayPath varchar(max)
    DECLARE @ParentEmployeeName varchar(max)
    -- Начать с указанного узла
    SELECT @EntityLevelDepth = NodeId.GetLevel(),
           @DisplayPath = EmployeeName
    FROM Employee
    WHERE NodeId = @EntityNodeId
    -- Просмотр всех его предков
    SET @LevelCounter = 0
    WHILE @LevelCounter < @EntityLevelDepth BEGIN
        SET @LevelCounter = @LevelCounter + 1
        SELECT @ParentEmployeeName = EmployeeName
        FROM Employee
        WHERE NodeId = (
            SELECT NodeId.GetAncestor(@LevelCounter)
            FROM Employee
            WHERE NodeId = @EntityNodeId)
        -- Приписать имя предка в начало пути
        SET @DisplayPath = @ParentEmployeeName + ' > ' + @DisplayPath
    END
    RETURN(@DisplayPath)
END
```

После объявления нескольких локальных переменных функция начинает с выбора имени сотрудника того узла, который передан как параметр @EntityNodeId (значение hierarchyid узла). Она получает уровень узла при помощи вызова GetLevel и сохраняет его в переменной @EntityLevelDepth. Она также извлекает в переменную @DisplayPath имя сотрудника, которое станет возвращаемым значением функции после того, как к нему будут приписаны спереди имена всех сотрудников из предков данного узла (вверх по всей иерархии до самого корня).

Функция затем создает целочисленную переменную @LevelCounter (инициализированную значением 0), входит в цикл приращения @LevelCounter и прерывает цикл тогда, когда значение счетчика достигает @EntityLevelDepth. При каждой итерации увеличенное значение @LevelCounter подается в метод GetAncestor для извлечения имени следующего сотрудника выше по цепочке, которое затем приписывается спереди к @DisplayPath (вместе с разделяющей уровни строкой '>'). Таким образом код получает родителя, прародителя, пра-прародителя — вверх до самого корня. Когда цикл заканчивается, вызывающей стороне возвращается @DisplayPath. Вот выходные данные функции fnGetFullDisplayPath по нашей таблице сотрудников:

```
SELECT
```

```
    NodeId,
    NodeId.ToString() AS NodeIdPath,
        dbo.fnGetFullDisplayPath(NodeId) AS NodeIdDisplayPath,
    EmployeeName
FROM
    Employee
ORDER BY
    NodeLevel, NodeId
```

```
GO
```

| NodeId | NodeIdPath | NodeIdDisplayPath   | EmployeeName |
|--------|------------|---------------------|--------------|
| 0x     | /          | Dave                | Dave         |
| 0x58   | /1/        | Dave > Amy          | Amy          |
| 0x68   | /2/        | Dave > John         | John         |
| 0x78   | /3/        | Dave > Jill         | Jill         |
| 0x5AC0 | /1/1/      | Dave > Amy > Cheryl | Cheryl       |
| 0x5B40 | /1/2/      | Dave > Amy > Wanda  | Wanda        |
| 0x6AC0 | /2/1/      | Dave > John > Mary  | Mary         |

```

0x7AC0 /3/1/      Dave > Jill > Kevin      Kevin
0x5AD6 /1/1/1/    Dave > Amy > Cheryl > Richard Richard
0x5ADA /1/1/2/    Dave > Amy > Cheryl > Jeff   Jeff
(10 row(s) affected)

```

В этих результатах столбец `NodeIdDisplayPath` показывает имена сотрудников вместе с иерархическими путями.

Как вы видели, заполнение иерархической таблицы требует некоторых предварительных усилий. Однако после того, как ваши данные оказались на своих местах, вы можете использовать всю мощь типа данных `hierarchyid` (при помощи методов запроса и организации древовидной структуры). Мы почти готовы погрузиться в этот процесс, но сначала давайте немного поговорим о схемах индексирования иерархических таблиц.

## Стратегии индексирования иерархических таблиц

Для ваших иерархических таблиц вы можете создать *индекс поиска в глубину* или *индекс поиска в ширину* (либо оба этих индекса). Эти два типа отличаются тем, как SQL Server физически хранит в индексе ссылки на узлы. Определение этих индексов может оказать значительное влияние на производительность при доступе к данным иерархических таблиц.

При индексировании в глубину ссылки на родительские и дочерние узлы хранятся рядом; при индексировании в ширину рядом хранятся ссылки на узлы одного и того же иерархического уровня. Поэтому вы выбираете соответствующий тип индекса на основе своего понимания о том, какую форму имеют иерархические данные вашей таблицы, как она будет расти, как ее будут запрашивать клиентские приложения. Вы можете также создать оба типа индексов (для эффективного доступа к дереву, как по горизонтали, так и по вертикали).

### ПРИМЕЧАНИЕ

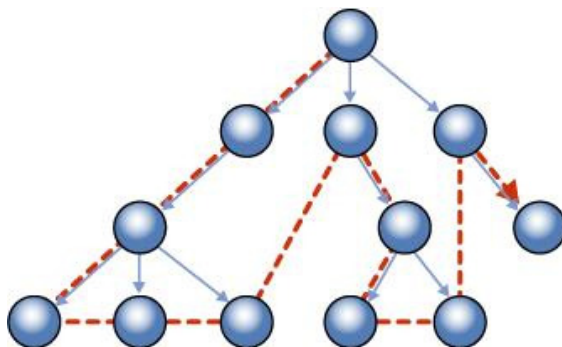
Независимо от того, какую стратегию индексирования вы используете, операторы сравнения выполняются всегда сначала по глубине. То есть  $A < B$  означает, что  $A$  всегда идет раньше  $B$  при прохождении дерева в глубину (либо  $A$  левее  $B$ , если они находятся на одном уровне), даже если используется индексирование в ширину.



## Индексирование в глубину

Определение индекса первичного ключа или уникального индекса по столбцу `hierarchyid` приводит к созданию индекса в глубину. Поскольку мы назначили столбец `NodeId` нашей таблицы как первичный ключ, то мы уже создали индекс в глубину.

При индексировании в глубину (рис. 7.3) SQL Server старается физически сохранять ссылки на узлы поддерева как можно ближе друг к другу, записывает их на диск по возможности на одной странице, чтобы улучшить производительность запросов в том случае, когда у вас очень глубокая иерархия. Создание индекса в глубину для такой иерархии приведет к очень быстрому вертикальному поиску по дереву (при запросах по предкам и потомкам вверх и вниз по цепочке, которая может быть весьма длинной).



**Рис. 7.3.** Физическое хранение строк при оптимизации для поиска в глубину

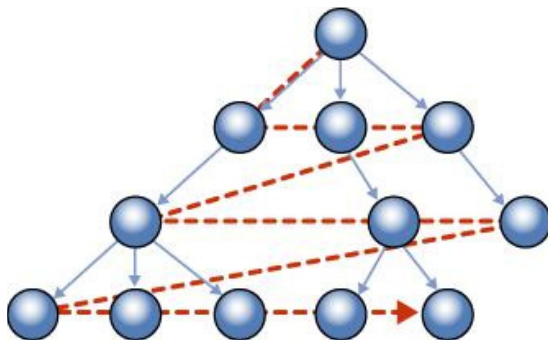
## Индексирование в ширину

При индексировании в ширину (рис. 7.4) ссылки на узлы одного уровня иерархии физически хранятся как можно ближе друг к другу. Такой тип индекса дает высокую производительность горизонтального поиска по большому количеству узлов одного уровня.

Составной индекс по вашей иерархической таблице включает два столбца: целочисленный столбец, который содержит уровень (`level`) узла в иерархии `NodeLevel`, и сам столбец типа `hierarchyid` (это `NodeId`).

Так что для создания индекса в ширину мы выполняем следующий код:

```
CREATE UNIQUE INDEX IX_EmployeeBreadth
ON Employee(NodeLevel, NodeId)
```



**Рис. 7.4.** Физическое хранение строк при оптимизации для поиска в ширину

Как мы уже упоминали, одна таблица может иметь как индекс в глубину, так и индекс в ширину. Для этого необходимо создать один первичный ключ (или уникальный индекс) по столбцу типа `hierarchyid` и другой составной индекс по столбцу уровня узла и `hierarchyid`. Обновления необходимо будет делать в обоих индексах, однако производительность запросов высока как для горизонтального, так и для вертикального поиска по большим иерархиям.

## Запросы к иерархическим таблицам

Теперь, когда наша иерархическая таблица заполнена и проиндексирована, мы готовы начать писать эффективные запросы дополнительными методами типа `hierarchyid`.

### Метод *IsDescendantOf*

Метод `IsDescendantOf` вызывается по одному значению `hierarchyid` и принимает в качестве параметра другие значения `hierarchyid`. Он возвращает булево значение 1 (`true`) в том случае, если объект `hierarchyid` (по которому метод вызван) является потомком (прямым либо непрямым) того объекта `hierarchyid`, который передан в качестве параметра. То есть этот метод по существу возвращает поддерево, чьим корнем является узел, указанный па-

раметром. Вследствие вертикального прохождения по дереву он дает производительность для таблиц с индексом в глубину.

Мы можем легко запросить нашу таблицу сотрудников, чтобы получить всех потомков (дочерние узлы, внучатые узлы и т. д.) конкретного сотрудника при помощи метода `IsDescendantOf`. Например, следующий запрос даст список всех потомков Amy:

```
DECLARE @AmyNodeId hierarchyid
SELECT @AmyNodeId = NodeId
FROM Employee
WHERE EmployeeId = 46
SELECT NodeId.ToString() AS NodeIdPath, *
FROM Employee
WHERE NodeId.IsDescendantOf (@AmyNodeId) = 1
ORDER BY NodeLevel, NodeId
```

Этот запрос выбирает все строки, значения `NodeId` которых являются потомками Amy (сотрудник с идентификатором 46). Вот его результат:

| NodeIdPath | NodeId | NodeLevel | EmployeeId | EmployeeName | Title                |
|------------|--------|-----------|------------|--------------|----------------------|
| /1/        | 0x58   | 1         | 46         | Amy          | Marketing Specialist |
| /1/1/      | 0x5AC0 | 2         | 269        | Cheryl       | Marketing Assistant  |
| /1/2/      | 0x5B40 | 2         | 389        | Wanda        | Business Assistant   |
| /1/1/1/    | 0x5AD6 | 3         | 87         | Richard      | Business Intern      |
| /1/1/2/    | 0x5ADA | 3         | 90         | Jeff         | Business Intern      |

(5 row(s) affected)

Обратите внимание, что результат включает и строку Amy в дополнение ко всем ее потомкам. Сама строка Amy возвращена потому, что она является своим собственным потомком на уровне 0 поддерева. То есть мы выбрали поддерево, которое начинается с Amy и включает в себя всех ее потомков (вне зависимости от того, насколько глубоко они находятся под ней).

Для того чтобы выбрать только непосредственные дочерние строки Amy (которые всего на один уровень ниже нее), вы можете использовать метод `GetAncestor`. (Мы использовали этот метод для создания нашей хранимой процедуры `uspAddEmployee`). Например:

```
SELECT NodeId.ToString() AS NodeIdPath, *
FROM Employee
WHERE NodeId.GetAncestor(1) =
    (SELECT NodeId
     FROM Employee
```

```

WHERE EmployeeId = 46)
ORDER BY NodeLevel, NodeId
GO

```

| NodeIdPath | NodeId | NodeLevel | EmployeeId | EmployeeName | Title               |
|------------|--------|-----------|------------|--------------|---------------------|
| /1/1/      | 0x5AC0 | 2         | 269        | Cheryl       | Marketing Assistant |
| /1/2/      | 0x5B40 | 2         | 389        | Wanda        | Business Assistant  |

(2 row(s) affected)

На этот раз в результатах есть только Cheryl и Wanda, но нет Amy и более дальних потомков Amy. Это потому, что мы запрашиваем только те строки, для которых непосредственным предком является Amy (сотрудник ID 46) — дочерние узлы Amy.

Если бы мы хотели получить всех сотрудников, находящихся на два уровня ниже конкретного менеджера, то мы могли бы передать в метод `GetAncestor` значение 2. Например, для выбора сотрудников, которые подчиняются сотрудникам ниже Dave (чтобы увидеть всех сотрудников на два уровня ниже него), мы можем запросить те строки, прародителем которых является Dave (сотрудник с ID 6), то есть просто внуков Dave:

```

SELECT NodeId.ToString() AS NodeIdPath, *
FROM Employee
WHERE NodeId.GetAncestor(2) =
    (SELECT NodeId
     FROM Employee
     WHERE EmployeeId = 6)
ORDER BY NodeLevel, NodeId
GO

```

| NodeIdPath | NodeId | NodeLevel | EmployeeId | EmployeeName | Title               |
|------------|--------|-----------|------------|--------------|---------------------|
| /1/1/      | 0x5AC0 | 2         | 269        | Cheryl       | Marketing Assistant |
| /1/2/      | 0x5B40 | 2         | 389        | Wanda        | Business Assistant  |
| /2/1/      | 0x6AC0 | 2         | 272        | Mary         | Marketing Assistant |
| /3/1/      | 0x7AC0 | 2         | 291        | Keven        | Marketing Intern    |

(4 row(s) affected)

Запрос вернул всех сотрудников, которые находятся двумя уровнями ниже Dave. Тот факт, что некоторые из них являются двоюродными братьями, а не родными (то есть имеют различных непосредственных родителей), в данном случае к делу не относится. Местоположение точно на два уровня ниже Dave квалифицирует их для выбора этим запросом. И поскольку при сканировании

в этом запросе происходит горизонтальное прохождение дерева, то он дает высокую производительность индекса в ширину.

Для поиска корневого узла иерархии просто выполните метод `GetRoot` по самому типу данных `hierarchyid` при помощи синтаксиса с двойным двоеточием (как показано в следующем коде). Это метод, который мы использовали для создания первого сотрудника на самом верху нашего дерева (Dave, как в листинге 7.2).

```
SELECT NodeId.ToString() AS NodeIdPath, *
```

```
FROM Employee
```

```
WHERE NodeId = hierarchyid::GetRoot()
```

```
GO
```

```
NodeIdPath NodeId NodeLevel EmployeeId EmployeeName Title
```

```
-----
```

| NodeIdPath | NodeId | NodeLevel | EmployeeId | EmployeeName | Title             |
|------------|--------|-----------|------------|--------------|-------------------|
| /          | 0x     | 0         | 6          | Dave         | Marketing Manager |

```
(1 row(s) affected)
```

## Изменение порядка узлов в иерархии

Реорганизация иерархии — это обычная задача ее обслуживания. Вы можете поменять структуру дерева при помощи изменения связей родитель—потомок в иерархии. Тип `hierarchyid` предоставляет метод `GetReparentedValue` такого типа обслуживания. Мы начнем с переноса одного узла (не затрагивая других узлов иерархии). Затем мы перенесем целое поддерево (то есть все узлы-потомки данного узла).

### Метод `GetReparentedValue`

Мы вызываем метод `GetReparentedValue` для того узла, который вы хотите перенести, передавая ему два параметра. Первый указывает исходного родителя (источник), а второй — нового родителя (назначение). Возвращаясь к примеру нашего дерева сотрудников: раньше Wanda подчинялась Amy, но теперь подчиняется Jill. Поэтому мы хотим перенести Wanda из потомков Amy в потомки Jill. Следующий код использует для выполнения такого изменения метод `GetReparentedValue`:

```
DECLARE @EmployeeToMove hierarchyid
```

```
DECLARE @OldParent hierarchyid
```

```
DECLARE @NewParent hierarchyid
```

```
SELECT @EmployeeToMove = NodeId
```

```
FROM Employee
```

```

WHERE EmployeeId = 389 -- Wanda
SELECT @OldParent = NodeId
FROM Employee
WHERE EmployeeId = 46 -- Amy
SELECT @NewParent = NodeId
FROM Employee
WHERE EmployeeId = 119 -- Jill
-- Wanda теперь подчиняется Jill, а не Amy
UPDATE Employee
SET NodeId = @EmployeeToMove.GetReparentedValue(@OldParent, @NewParent)
WHERE NodeId = @EmployeeToMove
GO

```

Иерархия теперь выглядит следующим образом:

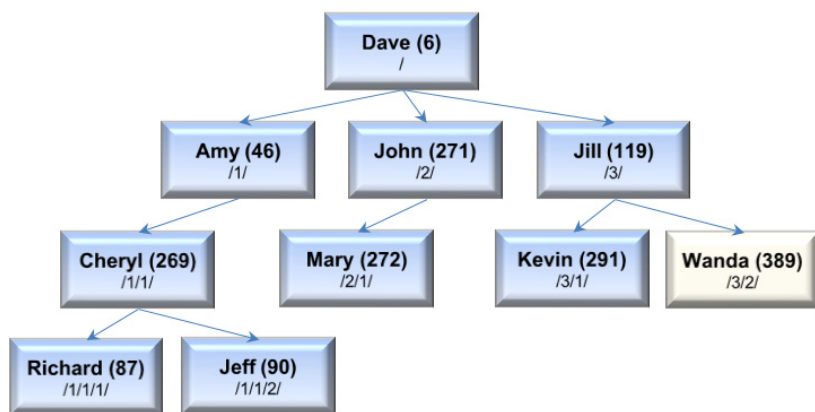


Рис. 7.5. Изменение родителя для одного узла

Просмотр результатов следующего запроса подтверждает, что `NodeIdPath` для Wanda изменился с `/1/2/` (потомок Amy) на `/3/2/` (потомок Jill).

```

SELECT
    NodeId,
    NodeId.ToString() AS NodeIdPath,
    NodeLevel,
    dbo.fnGetFullDisplayPath(NodeId) AS NodeIdDisplayPath,
    EmployeeName
FROM
    Employee

```

```
ORDER BY
  NodeLevel, NodeId
GO
NodeId NodeIdPath NodeLevel NodeIdDisplayPath EmployeeName
-----
0x      /          0          Dave           Dave
0x58    /1/        1          Dave > Amy      Amy
0x68    /2/        1          Dave > John     John
0x78    /3/        1          Dave > Jill     Jill
0x5AC0  /1/1/      2          Dave > Amy > Cheryl Cheryl
0x6AC0  /2/1/      2          Dave > John > Mary Mary
0x7AC0  /3/1/      2          Dave > Jill > Kevin Kevin
0x7B40  /3/2/      2          Dave > Jill > Wanda Wanda
0x5AD6  /1/1/1/    3          Dave > Amy > Cheryl > Richard Richard
0x5ADA  /1/1/2/    3          Dave > Amy > Cheryl > Jeff Jeff
(10 row(s) affected)
```

Изменение положения одного узла при помощи метода GetReparentedValue не влияет на дочерние элементы этого узла. Это означает, что если бы Wanda имела дочерние узлы, то они не только не переместились бы вместе с Wanda в положение новых потомков Jill, они также не переместились бы и на один уровень вверх по иерархии для того, чтобы занять освободившееся положение Wanda как прямые потомки Amy. То же самое будет справедливо и в том случае, если Wanda будет просто удалена (а не перенесена). Бывшие дочерние узлы Wanda поэтому остаются совсем без родителя, у них останется только прародитель (которым в этом случае является Amy, бывший родитель Wanda). SQL Server 2008 не обеспечивает ссылочную целостность иерархии для перехвата такой ситуации. Будущие версии SQL Server, возможно, справятся с этой проблемой, но до той поры обработка осиротевших узлов остается на вашей совести. В нашем примере мы могли бы просто перенести осиротевшие узлы на один уровень выше по иерархии (как прямых потомков Amy), чтобы закрыть оставшийся после Wanda разрыв. Для этого необходимо выполнить трансплантацию поддерева, о которой вы сейчас и узнаете.

## Трансплантация поддеревьев

Мы будем переносить всех потомков узла одним блоком, и осиротевших узлов в результате такой операции не возникнет. В нашем следующем примере мы перенесем всех подчиненных Amy к их новому менеджеру Kevin. Это делается при помощи изменения родителей всех узлов со значениями hierarchyid, начинающимися на /1/ (Amy), кроме собственно Amy. Мы ис-

пользуем метод `IsDescendantOf` возвращения всех потомков `Amy` для обновления. Вспомним из нашего предыдущего обсуждения метода `IsDescendantOf`, что `Amy` является своим собственным потомком на уровне 0 (корнем своего поддерева). Поэтому мы должны исключить сотрудника с ID 46 из подлежащих обновлению строк:

```
DECLARE @OldParent hierarchyid
DECLARE @NewParent hierarchyid
SELECT @OldParent = NodeId
FROM Employee
WHERE EmployeeId = 46 -- Amy
SELECT @NewParent = NodeId
FROM Employee
WHERE EmployeeId = 291 -- Kevin
UPDATE Employee
SET NodeId = NodeId.GetReparentedValue(@OldParent, @NewParent)
WHERE NodeId.IsDescendantOf(@OldParent) = 1
AND EmployeeId <> 46 -- Это исключает Amy
из переноса.
GO
```

### **ПРИМЕЧАНИЕ**

Значения ID для `Amy` и `Kevin` в данном коде закодированы жестко. Так было сделано в демонстрационных целях. В реальности эта логика должна быть в хранимой процедуре, входные параметры которой — любые два узла, инструкции `SELECT` и `UPDATE` — заключены в транзакцию.

Выполнение этого кода переносит всех потомков `Amy` и делает их потомками `Kevin` (рис. 7.6). Обратите внимание, что узлы перенесены не только поперек дерева, но также и на один уровень вниз (поскольку `Kevin` находится на один уровень глубже, чем `Amy`).

Выборка всех сотрудников показывает, что SQL Server обновил значения `hierarchyid` в `NodeId` для `Cheryl`, `Richard` и `Jeff` (чтобы отразить их новое положение под `Kevin`). Впервые наша иерархия достигла глубины в пять уровней (от 0 до 4):

```
SELECT
    NodeId,
    NodeId.ToString() AS NodeIdPath,
    NodeLevel,
    dbo.fnGetFullDisplayPath(NodeId) AS NodeIdDisplayPath,
    EmployeeName
```



FROM  
Employee  
GO

| NodeId   | NodeIdPath | NodeLevel | NodeIdDisplayPath                      | EmployeeName |
|----------|------------|-----------|----------------------------------------|--------------|
| 0x       | /          | 0         | Dave                                   | Dave         |
| 0x58     | /1/        | 1         | Dave > Amy                             | Amy          |
| 0x68     | /2/        | 1         | Dave > John                            | John         |
| 0x78     | /3/        | 1         | Dave > Jill                            | Jill         |
| 0x6AC0   | /2/1/      | 2         | Dave > John > Mary                     | Mary         |
| 0x7AC0   | /3/1/      | 2         | Dave > Jill > Kevin                    | Kevin        |
| 0x7B40   | /3/2/      | 2         | Dave > Jill > Wanda                    | Wanda        |
| 0x7AD6   | /3/1/1/    | 3         | Dave > Jill > Kevin > Cheryl           | Cheryl       |
| 0x7AD6B0 | /3/1/1/1/  | 4         | Dave > Jill > Kevin > Cheryl > Richard | Richard      |
| 0x7AD6D0 | /3/1/1/2/  | 4         | Dave > Jill > Kevin > Cheryl > Jeff    | Jeff         |

(10 row(s) affected)

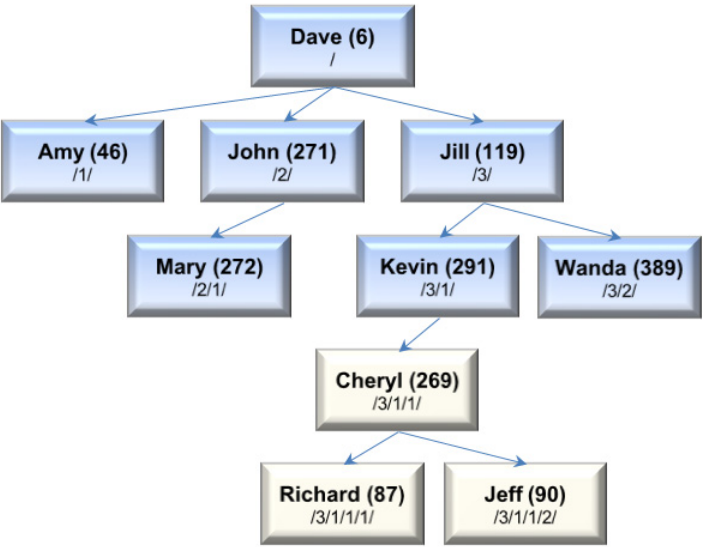


Рис. 7.6. Изменение родителя целого поддерева

Другие методы *hierarchyid*

Тип данных *hierarchyid* предоставляет еще три метода — *Parse*, *Read* и *Write*, которые используются не так часто.

Метод `Parse` — это действие, по существу обратное к `ToString`. Он принимает такую же строку с разделителями в виде косой черты (которая возвращается методом `ToString`) и возвращает эквивалентное сжатое значение `varbinary hierarchyid`, которое SQL Server использует внутренним образом для представления узлов. Это значение может затем быть передано в `ToString`, при этом обратно вы получите исходную строку с разделителями в виде косой черты. Это единственный статический метод, не считая `GetRoot`, поэтому имеет синтаксис с двойными двоеточиями:

```
SELECT hierarchyid::Parse('/2/1/1/') AS NodeId
GO
NodeId
-----
0x6AD6
(1 row(s) affected)
```

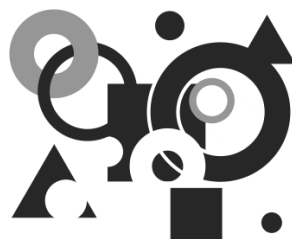
Методы `Parse` и `ToString` позволяют выполнять сериализацию в значения и строки `hierarchyid` и десериализацию из них, но значения `hierarchyid` лучше всего присваивать при помощи вызова метода `GetDescendant` (как мы показывали во всех наших примерах).

Последние два метода, `Read` и `Write`, отсутствуют в T-SQL. Их можно использовать только в .NET-коде с типом `Microsoft.SqlServer.Types.SqlHierarchyId`. Эти два метода используются для передачи значений `hierarchyid` в объекты `BinaryReader` и `BinaryWriter` (и обратно). Они используются также внутренним образом в SQL Server, например, при чтении из столбца `hierarchyid` (или записи в него), для преобразований между `varbinary` и `hierarchyid`. Данные методы предоставляются также для интеграции с вашим .NET-кодом, работающим под SQL CLR. Они не выполняют никаких других важных функций в реализации иерархической структуры.

## Резюме

Теперь мы можем работать с иерархическими данными в нормальных реляционных таблицах. В этой главе вы узнали о том, как использовать новый тип данных `hierarchyid` в SQL Server 2008 для приведения древовидной структуры к данным, хранящимся в обычных таблицах, строках и столбцах баз данных. Тип данных `hierarchyid` предоставляет целый набор методов и стратегий индексирования. Это позволяет нам применять иерархическую модель к традиционным табличным данным и не прибегать к использованию XML в тех ситуациях, когда имеет смысл хранить данные реляционно, а не реструктурировать их для хранения в формате XML.

## ГЛАВА 8



# Хранение неструктурированных данных

*Леонард Лобел*

В наши дни приложения работают с неструктурированными данными гораздо чаще, чем в прошлом. Ускоряющийся "взрыв данных" (беспрестанно подгоняемый падающими ценами на запоминающие устройства) продолжает генерировать все больше и больше неструктурированных данных, которые нам необходимо обрабатывать. В то время как приложениям прошлого не требовалось ничего, кроме установления соответствия между экранами ввода данных и строками со столбцами баз данных плюс возможности выполнения довольно простых запросов, сегодня мы имеем такие данные, как аудио, видео и прочие мультимедийные типы файлов. Вам может понадобиться хранить фотографии сотрудников, записи видеонаблюдения, совещания, сообщения электронной почты (включая вложенные в них приложения), электронные таблицы анализа тенденций, контент в нестандартных форматах, а также прочие объекты. Эти неструктурированные типы содержат двоичные потоки информации, которые обычно называют большими двоичными объектами (binary large object, BLOB). Эти данные BLOB необходимо связать со структурированными данными, обитающими в вашей реляционной базе данных.

Традиционно в Microsoft SQL Server было два решения для комбинирования структурированных табличных данных с неструктурированными данными BLOB: либо хранить BLOB в базе данных вместе со всеми реляционными данными, либо держать их вне базы данных (в файловой системе, например, или в специальном хранилище BLOB, при этом в базе данных хранятся пути). Каждая из этих стратегий имеет свои плюсы и минусы в смысле размера хранения, производительности, управляемости и сложности программирования

(о которых мы еще поговорим), но ни одна из них не является "родной" для ядра базы данных.

FILESTREAM — это новая важная функциональная возможность SQL Server 2008, которая обеспечивает собственную поддержку эффективной работы с BLOB в базе данных. Сначала мы изучим все "за" и "против" хранения BLOB-данных внутри и вне баз данных.

## BLOB в базе данных

Наш первый вариант — это, конечно же, хранение BLOB непосредственно в столбцах таблиц. Вы можете сделать это при помощи объявления столбца как типа данных `varbinary(max)`, в котором можно хранить BLOB размером до 2 Гбайт.

### **ВАЖНО**

Вам не следует больше использовать тип данных `image` версий ранее SQL Server 2005. Вместо `image` следует использовать тип `varbinary(max)`. Тип `image` может быть ликвидирован в будущих версиях SQL Server.

От вас не требуется никаких усилий по связыванию реляционных данных с соответствующими им BLOB-данными. Поэтому управление ими упрощено (поскольку все находится в группе файлов единой базы данных). Операции резервного копирования, восстановления, отключения, копирования и подключения файлов базы данных охватывают все структурированные и BLOB-данные как единое целое. Согласованность транзакций — это еще одно важное преимущество данного подхода. Поскольку BLOB-данные физически являются частью таблиц базы данных, то они могут участвовать в транзакциях. Если вы начинаете транзакцию, обновляете некие данные, а затем откатываете транзакцию назад, то любые обновленные BLOB-данные также откатываются назад, эта смесь структурированных и BLOB-данных обрабатывается данной моделью весьма последовательно.

Однако физическое хранение BLOB в базе данных часто приводит к существенному (и недопустимому) снижению производительности. BLOB-содержимое может расходовать непропорционально большой процент объема хранения базы данных, по сравнению со структурированными данными. В результате сильно страдает производительность запросов, поскольку обработчику запросов нужно просеивать большие объемы данных ваших таблиц (хранящих BLOB-содержимое). Объекты BLOB не могут выводиться в поток из `varbinary(max)` с такой же эффективностью, как если бы они хранились в

файловой системе или в специальном BLOB-хранилище. И наконец, столбцы `varbinary(max)` могут хранить максимум 2 Гбайт. Хотя это и не ограничение для обычных документов, но может стать препятствием для сценариев, например, когда каждая строка таблицы программных продуктов содержит BLOB с ISO-образом (International Organization for Standardization, ISO) дистрибутива программного обеспечения, размер которого может легко превосходить 2 Гбайт.

### **ПРИМЕЧАНИЕ**

Если ваши запросы по размеру хранения BLOB скромные (каждый ваш BLOB имеет размер не более 1 Мбайт), то вам следует рассмотреть возможность хранения их в базе данных при помощи типа данных `varbinary(max)`, а не использовать файловую систему. Многие вещи упрощаются при хранении BLOB внутри ваших таблиц, и при этом работа с небольшим количеством BLOB (или BLOB-данными очень малого размера) обычно не влияет на производительность. Более того, вам следует обдумать возможность кэширования небольших BLOB, к которым часто производятся обращения (вместо повторного считывания их из базы данных).

## **BLOB в файловой системе**

Чтобы справиться с проблемами производительности, вы можете хранить BLOB вне базы данных (как обычные файлы в файловой системе). При таком подходе структурированные данные реляционных таблиц в основном содержат информацию о пути к неструктурированным BLOB-данным. Приложения используют эту информацию о пути как ссылку для отслеживания и нахождения BLOB-содержимого, связанного со строками таблиц базы данных. Поскольку физически они хранятся в файловой системе, то любой BLOB может иметь размер более 2 Гбайт. Фактически их размер ограничен только характеристиками файловой системы и свободным дисковым пространством. Они обеспечивают также гораздо более высокую производительность потоковой передачи, поскольку файловая система предоставляет собственную среду, оптимизированную для потоковой передачи неструктурированных данных (в то время как столбец типа `varbinary(max)` базы данных такой поддержки не предоставляет). И поскольку физическая база данных имеет гораздо меньший размер при отсутствии в ней BLOB, то обработчик запросов может выдавать оптимальную производительность.

Хотя такое физическое разделение структурированного и неструктурированного содержимого и помогает решить проблемы производительности BLOB, оно порождает новые проблемы (поскольку данные теперь разделе-

ны не только физически, но и логически). То есть SQL Server не имеет никакого представления о связях между данными в базе и хранящимися в файловой системе файлами (на которые имеются ссылки в таблицах базы данных). Их связывание существует исключительно на уровне приложения. Поэтому операции (резервного копирования, восстановления, отключения, копирования и подключения) работают только со структурированными табличными данными и не захватывают никаких находящихся в файловой системе BLOB-данных. Преимущества по интегрированному управлению (которые вы имеете при хранении BLOB в базе данных) утрачены, и объем администрирования возрастает из-за необходимости отдельного управления файловой системой.

Разработка приложений по такой модели более сложная (вследствие дополнительных усилий по связыванию базы данных с файловой системой). База данных не предоставляет никакой помощи по установлению и обслуживанию ссылок между структурированными данными и внешними BLOB, всем этим должны заниматься проектировщик базы данных и разработчик приложения. И наконец (хотя, возможно, это и есть самое главное), не существует унифицированного контроля транзакций по базе данных и файловой системе.

## Что такое атрибут

Конечно, эта дискуссия ведет к новой стратегии FILESTREAM, которая сочетает лучшее из обоих описанных миров (и кое-что еще). Во-первых (для ясности), технически это не новый тип данных SQL Server. FILESTREAM реализован как атрибут, который вы применяете к типу данных `varbinary(max)`. Выглядит это достаточно невинно, но применение этого атрибута высвобождает возможности FILESTREAM — исключительно эффективного уровня абстракции хранения для управления неструктурированными данными в базе данных. При наличии этого атрибута мы продолжаем обращаться со столбцом `varbinary(max)`, как будто его содержимое хранится внутри наших табличных данных. На самом же деле данные хранятся вне базы данных в файловой системе NTFS сервера.

При использовании FILESTREAM структурированные и неструктурированные данные логически соединены, но физически разделены. Неструктурированные данные настроены, как еще одна группа файлов в базе данных, и поэтому они участвуют во всех логических операциях (в том числе транзакции и резервное копирование/восстановление). На диске же файлы BLOB хранятся как отдельные физические файлы в системе NTFS. SQL Server создает и поддерживает ссылки между базой данных и файловой системой. Он знает о неструктурированных BLOB-данных в файловой системе и считает содер-

жащие BLOB-данные файлы неотъемлемой частью базы данных. Неструктурированные данные не снижают производительность запросов, поскольку они не хранятся физически внутри табличных данных. Они хранятся в файловой системе, которая высоко оптимизирована для потоковой передачи двоичных данных. Однако логически база данных охватывает и реляционные таблицы, и BLOB-файлы в файловой системе. Поэтому мы будем продолжать обращаться с BLOB-данными (как с точки зрения разработки, так и с точки зрения администрирования) таким образом, как будто мы храним их в самой базе данных. Например, резервное копирование базы данных автоматически включает в себя все BLOB-данные из файловой системы.

### **ПРИМЕЧАНИЕ**

Поскольку BLOB-данные содержатся в своей собственной группе файлов базы данных, то вы легко можете при необходимости исключить их из резервного копирования.

В итоге для обеспечения максимально возможной производительности SQL Server 2008 использует для структурированных и неструктурированных данных соответствующий им тип хранения: реляционные (структурированные) данные он хранит в таблицах, а BLOB-данные (неструктурированные) — в файлах. Поскольку он делает это абсолютно прозрачно, то мы наслаждаемся преимуществами интегрированного управления базой данных. Ядро базы данных обрабатывает за нас ссылки между реляционными таблицами и соответствующими им BLOB-данными в файловой системе. Не нужно беспокоиться о дополнительных сложностях ручного связывания базы данных с файловой системой и поддержания синхронности этих связей (как мы делали это раньше). И наконец, при помощи использования транзакционных возможности файловой системы NTFS обновления BLOB происходят при выполнении транзакций базы данных. Вам начинает все это нравиться, именно этого мы и добивались! Мы готовы погрузиться в реальный код, который заставит FILESTREAM работать на нас.

## **Включение FILESTREAM**

Подобно многим другим функциональным возможностям, для использования FILESTREAM его надо включить. Для этого требуется выполнить два отдельных шага. Сначала функцию необходимо включить для компьютера (как службу Microsoft Windows), а затем ее необходимо активировать для экземпляра сервера. Эти два уровня конфигурирования FILESTREAM созданы

умышленно для разделения ответственности между ролями администратора Windows и администратора SQL Server.

## Включение FILESTREAM для компьютера

Первый шаг — включить FILESTREAM для компьютера (путем установления уровня доступа). Этот шаг можно выполнить тогда, когда SQL Server устанавливается (указать уровень доступа к FILESTREAM при установке), администратор Windows использует SQL Server Configuration Manager из меню **Configuration Tools** группы программ Microsoft SQL Server 2008 (в меню **Start**).

SQL Server Configuration Manager открывается со списком служб, отображаемым в главной панели. В списке служб щелкните правой кнопкой мыши тот экземпляр SQL Server, для которого вы хотите включить FILESTREAM, а затем выберите пункт **Properties**. В диалоговом окне **Properties** выберите вкладку **FILESTREAM**. Три флажка на вкладке **FILESTREAM** позволят вам выбрать различные уровни доступа к FILESTREAM.

Установка первого флажка включает FILESTREAM, но только для доступа через T-SQL. Это дает совершенно прозрачную реализацию FILESTREAM, но не позволяет использовать преимущества поточного доступа к файлам базы данных из ваших клиентских приложений.

Реальная мощь FILESTREAM вступает в игру тогда, когда вы включаете прямой потоковый доступ к файлам BLOB-данных в файловой системе SQL Server. Прямой потоковый доступ файлового ввода-вывода вы включаете при помощи второго флажка. Потоковый доступ к файлам создает также имя разделяемого ресурса Windows, которое используется для создания логических путей Universal Naming Convention (UNC) к BLOB-данным при доступе через FILESTREAM. Мы увидим это чуть позже, когда будем использовать функцию `OpenSqlFilestream` в наших примерах .NET-приложений. Имя разделяемого ресурса указывается в текстовом поле после второго флажка (в данном примере это `MSSQLSERVER`).

В большинстве случаев клиентские приложения не будут выполняться на том же самом компьютере, что и SQL Server, поэтому вам понадобится также выставить и третий флажок, для удаленных клиентов. Исключением из этого общего правила может стать использование версии Microsoft SQL Server 2008 Express, как локального хранилища данных клиентского приложения (когда все работает на одном компьютере). В этом случае вам следует использовать более безопасные настройки и не выставлять третий флажок. При этом будет



предоставлен поточный файловый ввод-вывод локального клиентского приложения, но для удаленных клиентов этот доступ будет выключен.

Мы дадим несколько примеров .NET-приложений, которые работают с FILESTREAM. Эти приложения демонстрируют использование `OpenSqlFilestream` поточного файлового ввода-вывода, и необходимо выставить, по крайней мере, два первых флажка. Если вы выполняете приложения на другом компьютере, то вам понадобится выставить и третий флажок для удаленного доступа.

### **Дополнительная информация**

Не существует эквивалентного скрипта T-SQL, который мог бы настроить уровень доступа FILESTREAM для компьютера. Однако компания Microsoft опубликовала файл VBScript (доступен через Интернет), который позволяет вам включить FILESTREAM из командной строки (альтернатива использования SQL Server Configuration Manager). Страница для скачивания этого скрипта находится по адресу: <http://www.codeplex.com/SQLSrvEngine/Wiki/View.aspx?title=FileStreamEnable&referringTitle=Home> либо поиском в Интернете по строке "How to enable FILESTREAM from the command line".

## **Включение FILESTREAM для экземпляра сервера**

Вторым шагом администратор SQL Server должен включить FILESTREAM для экземпляра сервера. Концепция аналогична первому шагу: определяются различные уровни доступа. FILESTREAM можно включить для экземпляра сервера простым вызовом системной хранимой процедуры `sp_configure`:

```
EXEC sp_configure filestream_access_level, n  
RECONFIGURE
```

Для установления уровня доступа замените в предыдущем коде `n` на число от 0 до 2. Значение 0 полностью выключает FILESTREAM. Значение 1 включает FILESTREAM только для T-SQL, а уровень 2 служит для полного доступа (включая локальный и удаленный потоковый доступ файлового ввода-вывода, как это было сделано для компьютера на первом шаге). Для поддержки наших примеров .NET-приложений, которые будут демонстрировать потоковый доступ файлового ввода-вывода при помощи `OpenSqlFilestream`, вам необходимо будет установить уровень 2 (полный доступ).

### ПРИМЕЧАНИЕ

Естественно, что заданный для экземпляра сервера уровень доступа должен поддерживаться уровнем доступа для компьютера. Поэтому уровни доступа компьютера и экземпляра сервера следует устанавливать одинаковыми.

Вы можете также настроить уровень доступа к FILESTREAM для экземпляра сервера из программы SQL Server Management Studio (из диалогового окна **Advanced Server Properties**). Щелкните правой кнопкой мыши по любому экземпляру сервера в окне **Object Explorer**, выберите пункт **Properties**, а затем страницу **Advanced**. Уровни доступа присутствуют в виде выпадающего списка **Filestream Access Level**.

## Создание базы данных с использованием FILESTREAM

Когда FILESTREAM включен и для компьютера, и для экземпляра сервера, любая база данных экземпляра сервера может поддерживать неструктурированные данные путем определения группы файлов при помощи нового выражения FILEGROUP...CONTAINS FILESTREAM инструкции CREATE DATABASE. Например, инструкция листинга 8.1 создает базу данных PhotoLibrary, которая может хранить изображения с использованием FILESTREAM.

**Листинг 8.1. Создание базы данных при помощи FILEGROUP...CONTAINS FILESTREAM**

```
CREATE DATABASE PhotoLibrary
ON PRIMARY
(NAME = PhotoLibrary_data,
 FILENAME = 'C:\PhotoLibrary\PhotoLibrary_data.mdf'),
FILEGROUP FileStreamGroup1 CONTAINS FILESTREAM
(NAME = PhotoLibrary_group2,
 FILENAME = 'C:\PhotoLibrary\Photos')
LOG ON
(NAME = PhotoLibrary_log,
 FILENAME = 'C:\PhotoLibrary\PhotoLibrary_log.ldf')
```

На этом этапе следует отметить несколько простых, но важных вещей: как и при создании любой базы данных, каталог (или каталоги), указанный для основной группы файлов и группы файлов журнала, должен существовать в

момент создания базы данных. В нашем примере указанный в выражениях `ON PRIMARY` и `LOG ON` каталог `C:\PhotoLibrary` должен уже существовать (либо выполнение инструкции `CREATE DATABASE` завершится ошибкой).

Интересно и несколько странно то, что содержащийся в новом выражении `FILEGROUP...CONTAINS FILESTREAM` параметр `FILENAME` указывает не имя реального файла, а имя каталога. И в отличие от каталогов основной группы файлов и группы файлов журнала, этот каталог не должен существовать в момент создания базы данных (хотя путь к самому каталогу должен существовать), иначе выполнение инструкции `CREATE DATABASE` опять завершится ошибкой. SQL Server берет на себя создание этого каталога и управление им (аналогично тому, как он создает и управляет файлами `mdf` и `ldf` в других группах файлов). В нашем примере SQL Server при выполнении инструкции `CREATE DATABASE` автоматически создаст каталог `C:\PhotoLibrary\Photos` и будет использовать этот каталог для хранения всех BLOB-данных из базы данных `PhotoLibrary`.

Когда мы выполняем эту инструкцию `CREATE DATABASE` при пустом каталоге `C:\PhotoLibrary`, SQL Server создает для нас обычные файлы `mdf` и `ldf`, а также создает подкаталог `Photos` для группы `FILESTREAM`.

SQL Server будет хранить все наши изображения как файлы подкаталога `Photos` и отслеживать ссылки на эти файлы изображений из реляционных таблиц (логически они будут принадлежать столбцу базы данных, определенному как `varbinary(max) FILESTREAM`). Если мы не исключим явным образом группу файлов `FileStreamGroup1` из операции резервного копирования или восстановления, то все наши файлы изображений в подкаталоге `Photos` будут включаться вместе с реляционной базой данных в операции резервного копирования или восстановления.

## Создание таблицы со столбцами FILESTREAM

Теперь мы готовы создать таблицу `PhotoAlbum`. SQL Server требует, чтобы любая использующая `FILESTREAM` таблица имела столбец `uniqueidentifier`, который не может содержать неопределенных значений и указывает атрибут `ROWGUIDCOL`. Вы должны также создать для этого столбца ограничение уникальности. В таблице может быть определен только один столбец `ROWGUIDCOL`, хотя это позволяет вам затем объявить любое количество столбцов `varbinary(max) FILESTREAM` для хранения BLOB-данных. Инструкция листинга 8.2 создает таблицу `PhotoAlbum` со столбцом `Photo`, объявленным как `varbinary(max) FILESTREAM`.

**Листинг 8.2. Создание таблицы с использованием FILESTREAM**

```
CREATE TABLE PhotoAlbum(  
    PhotoId int PRIMARY KEY,  
    RowId uniqueidentifier ROWGUIDCOL NOT NULL UNIQUE DEFAULT  
    NEWSEQUENTIALID(),  
    Description varchar(max),  
    Photo varbinary(max) FILESTREAM DEFAULT(0x))
```

При помощи этой инструкции мы выполняем требование FILESTREAM для столбца ROWGUIDCOL, хотя фактически нам ничего не придется делать для обслуживания этого столбца. Объявляя столбец RowId для вызова функции NEWSEQUENTIALID со значением DEFAULT, мы можем просто притвориться, что этот столбец вообще не существует. При непредоставлении для него значений SQL Server автоматически сгенерирует произвольный глобальный уникальный идентификатор (GUID) столбца. Столбец не принимает значений NULL и определен с необходимым ограничением уникальности.

Мы объявили также целочисленный столбец PhotoId первичного ключа таблицы. Мы будем использовать столбец PhotoId для идентификации фотографий альбома, а SQL Server будет использовать столбец RowId для отслеживания фотографий в файловой системе и ссылок на них в строках таблицы PhotoAlbum. Столбец Photo содержит сам BLOB, который определен как тип данных varbinary(max) с атрибутом FILESTREAM. Это означает, что он обрабатывается, как обычный столбец varbinary(max), но мы знаем, что его BLOB фактически хранится сервером SQL Server в файловой системе. Пока же обратите внимание, что мы определили для столбца Photo двоичное значение 0x по умолчанию. Это понадобится тогда, когда мы начнем потоковую выдачу содержимого, но пока мы до этого еще не дошли.

Манипулирование BLOB-данными — это не то, что легко или практично делать при помощи T-SQL. Конечно, вы можете указать небольшие двоичные потоки, непосредственно встроенные в ваш код T-SQL, либо вставлять и извлекать двоичные потоки при помощи байтовых массивов (как могли бы делать и при помощи обычных столбцов varbinary(max)). Но правильным (и самым быстрым) способом помещения данных в столбцы FILESTREAM и извлечения их оттуда является использование управляемого (или неуправляемого) клиента, который вызывает функцию OpenSqlFilestream, предоставляемую интерфейсом прикладного программирования управляемых клиентов SQL Server 2008.

При помощи OpenSqlFilestream управляемые и неуправляемые приложения могут использовать либо .NET-класс FileStream, либо функции ReadFile и WriteFile интерфейса прикладного программирования

Microsoft Win32 для обеспечения высокопроизводительного потокового доступа к BLOB-данным. В следующем разделе вы увидите, как использовать в языке C# управляемый класс `FileStream` для хранения и извлечения изображений столбца `Photo`. А прямо сейчас мы собираемся сделать нечто совершенно неестественное — использовать T-SQL для приведения строковых данных к типу данных `varbinary(max)` столбца `Photo`. Мы делаем это для того, чтобы вы разбирались с `FILESTREAM` шаг за шагом. Мы скромно начнем со следующей инструкции `INSERT`, которая добавляет в таблицу `PhotoAlbum` нашу первую строку:

```
INSERT INTO PhotoAlbum(PhotoId, Description, Photo)
VALUES(1, 'First pic', CAST('BLOB' As varbinary(max)))
```

Эта инструкция `INSERT` читается точно так же, как и при использовании для `Photo` обычного столбца `varbinary(max)` (без атрибута `FILESTREAM`). Она сохраняет неструктурированные данные столбца `Photo` среди остальных реляционных столбцов, и при возвращении данных из запроса `SELECT` все выглядит точно так же:

```
SELECT *, CAST(Photo AS varchar) AS PhotoText FROM PhotoAlbum
GO
PhotoId RowId                                     Description Photo PhotoText
-----
1      c04a7930-89ab-dc11-91e3-0003ff399330 First pic    0x424C4F42 BLOB
(1 row(s) affected)
```

Однако если мы заглянем "под капот", то увидим, что SQL Server фактически хранит столбец `Photo` в файловой системе (вне базы данных). Поскольку все отслеживается внутренним образом, то нам не нужно точно понимать систему наименования и организации файлов и каталогов, а также ссылок на них из реляционной базы данных. Однако если просто углубиться в подкаталоги каталога `Photos`, мы обнаружим, что в результате инструкции `INSERT` в файловой системе был создан новый файл, который мы можем просмотреть при помощи щелчка правой кнопкой мыши по названию файла и последующего выбора пункта **Open**.

Если мы откроем этот файл программой Notepad (Блокнот), то мы получим доказательство, что неструктурированное содержимое столбца `Photo` хранится вне базы данных в файловой системе. В нашем примере текст "BLOB", который был вставлен в столбец `Photo`, хранится в только что открытом нами при помощи Notepad файле.

Это четко демонстрирует, что данные `FILESTREAM` логически связаны с базой данных, но физически от нее отделены. Поскольку неструктурированные данные хранятся полностью в файловой системе, то мы можем легко из-

менить их содержимое прямым обновлением самого файла в Notepad (даже не привлекая для этого базу данных). Для дальнейшего разъяснения этого положения давайте изменим текст в файле с "BLOB" на "Cool" и сохраним это изменение в файловой системе.

Измененные данные FILESTREAM отражаются уже выполнявшейся нами ранее инструкцией SELECT:

```
SELECT *, CAST(Photo AS varchar) AS PhotoText FROM PhotoAlbum
```

```
GO
```

| PhotoId | RowId                                | Description | Photo      | PhotoText |
|---------|--------------------------------------|-------------|------------|-----------|
| 1       | c04a7930-89ab-dc11-91e3-0003ff399330 | First pic   | 0x436F6F6C | Cool      |

(1 row(s) affected)

### **ВАЖНО**

Мы выполнили это упражнение для проверки того, что для хранения данных FILESTREAM используется файловая система. Поэтому никогда не следует таким непосредственным образом модифицировать файлы в файловой системе. Считайте файловую систему частью групп файлов базы данных (файлов mdf и ldf), ими управляет исключительно SQL Server.

## **Интерфейс прикладного программирования неуправляемых клиентов *OpenSqlFilestream***

Имея внутреннюю организацию FILESTREAM сервером SQL Server, мы готовы двигаться вперед и сохранять двоичные данные настоящих изображений в столбце Photo. Как уже упоминалось ранее, лучше всего это делать при помощи написания клиентского кода, который вызывает функцию `OpenSqlFilestream`, предоставляемую интерфейсом прикладного программирования неуправляемых клиентов SQL Server.

Когда вы работаете с функцией `OpenSqlFilestream`, вы всегда работаете с транзакциями (даже для доступа чтения). Способы избежать их нет, поскольку FILESTREAM намеренно координирует транзакционную целостность структурированного и неструктурированного доступа к данным между SQL Server и файловой системой NTFS. Однако мы должны подчеркнуть, что не стоит пытаться делать внутри транзакции чтение (когда вы не работаете с функцией `OpenSqlFilestream`).

Вот как это работает. Сначала мы начинаем обычную транзакцию базы данных, после чего выполняем любое количество нормальных операций языка манипуляции данными DML, таких как вставки или обновления. Когда мы обращаемся к столбцу `varbinary(max) FILESTREAM`, то SQL Server автоматически инициирует транзакцию файловой системы NTFS и связывает ее с транзакцией базы данных. SQL Server также обеспечивает, что обе транзакции будут либо зафиксированы вместе, либо вместе откатены.

Затем для потоковой передачи BLOB нам понадобятся два ключевых элемента информации. Во-первых, нам нужен контекст транзакции файловой системы, который возвращает функция `GET_FILESTREAM_TRANSACTION_CONTEXT`. (В случае отсутствия транзакции эта функция вернет значение `NULL`). Во-вторых, нам нужен логический путь UNC до файла, содержащего BLOB на сервере (который возвращается методом `PathName`, вызванным по экземпляру значения `varbinary(max) FILESTREAM`). Эти два элемента информации передаются затем как входы в функцию `OpenSqlFilestream`, возвращающую нам дескриптор файла, который может использоваться нами для выполнения эффективных операций потокового ввода/вывода непосредственно BLOB-данных (сохраненных в файловой системе сервера). Только после фиксации транзакции базы данных SQL Server сохраняет изменения как в базе данных (после операций DML), так и в файловой системе (после операций потокового ввода/вывода). Аналогичным же образом откат транзакции отменяет изменения как в базе данных, так и в файловой системе.

### **ПРИМЕЧАНИЕ**

Возвращенная методом `PathName` ссылка UNC не является реальным путем в физической файловой системе сервера. `PathName` возвращает сформированный путь для использования в `OpenSqlFilestream` (включения прямой потоковой передачи между файловой системой и клиентскими приложениями). Имя совместно используемого ресурса в этом UNC-пути основано на том имени совместно используемого ресурса, которое было указано при включении `FILESTREAM` для компьютера (как описывалось ранее в этой же главе). Безопасность файловой системы обеспечена на сервере теми же средствами, что и безопасность группы файлов данных (или группы файлов журнала транзакций). Пользователям никогда не следует предоставлять прямого доступа к файловой системе сервера. Обычные разрешения системы безопасности SQL Server на уровне столбцов применимы также и к столбцам `varbinary(max) FILESTREAM`.

Возвращаемый функцией `OpenSqlFilestream` дескриптор можно использовать с функциями `ReadFile` и `WriteFile` интерфейса прикладного программирования Win32 (для клиентских приложений, написанных на неуправляемом языке, таком как C++). Этот дескриптор может также использоваться .NET-классом `FileStream` для клиентских приложений, написанных на

управляемых языках (таких как языки C# и Visual Basic .NET). Продолжая наш пример библиотеки фотографий, мы начнем с создания приложения Windows Forms на языке C#, которое реализует все ключевые элементы, характерные для приложения FILESTREAM — ни больше и ни меньше. Наше приложение позволит пользователю создать новую фотографию в базе данных (что порождает потоковую передачу BLOB в столбец Photo) и выбрать фотографию (что приводит к передаче BLOB обратно из столбца Photo в элемент управления PictureBox для отображения). OpenSqlFilestream обеспечит нам дескриптор для чтения и записи двоичных данных изображения при помощи обычного класса FileStream, определенного в пространстве имен System.IO.

## Потоковая передача файлов в .NET

Мы начнем с пользовательского интерфейса (user interface, UI) в Windows, который очень прост. Запустите Visual Studio 2008, а затем создайте новое приложение Windows Forms на языке C#. Сконструируйте форму с двумя отдельными полями: одно наверху формы для вставки фотографий, а другое под ним для выбора фотографий. Создайте (в верхнем поле) метки и текстовые поля для ввода идентификатора фотографии, названия файла и описания, а также ссылку на операцию сохранения. В нижнем поле создайте текстовое поле и метку для ввода идентификатора фотографии, а также ссылку на операцию загрузки. Включите метку для отображения описания из базы данных и графическое окно для отображения фотографического BLOB, возвращенного при помощи FILESTREAM.

Для этой формы Windows мы напишем совсем небольшой код, а логику FILESTREAM мы реализуем в отдельном классе доступа к данным, который можно многократно использовать в различных технологиях пользовательского интерфейса (в том числе Windows Forms, ASP.NET и Windows Presentation Foundation, WPF). Давайте добавим код для событий щелчков для ссылок Save и Load этой формы, который перехватывает управление в класс доступа к данным с названием PhotoData (который мы создадим сразу после UI), как показано в листинге 8.3.

**Листинг 8.3. UI делает вызов класса доступа к данным для сохранения и загрузки файлов изображений**

```
private void lnkSave_LinkClicked(object sender,
                                LinkLabelLinkClickedEventArgs e)
{
    int photoId = int.Parse(this.txtSavePhotoId.Text);
```



```
string desc = this.txtDescription.Text;
string filename = this.txtFilename.Text;
PhotoData.InsertPhoto(photoId, desc, filename);
}
private void lnkLoad_LinkClicked(object sender,
                                LinkLabelLinkClickedEventArgs e)
{
    int photoId = int.Parse(this.txtLoadPhotoId.Text);
    string desc;
    Image photo = PhotoData.SelectPhoto(photoId, out desc);
    this.lblDescription.Text = desc;
    this.picImage.Image = photo;
}
```

Когда пользователь нажимает кнопку **Save**, то код извлекает из трех текстовых полей идентификатор новой фотографии, описание и название файла и передает их в метод `InsertPhoto` класса `PhotoData`. Когда пользователь указывает идентификатор фотографии и нажимает кнопку **Load**, то код вызывает метод `SelectPhoto` класса `PhotoData` извлечения необходимого описания и изображения (для их отображения).

## Разбираемся с доступом FILESTREAM к данным

Все волшебство происходит внутри класса `PhotoData`, который является независимым от пользовательского интерфейса (UI) классом доступа к данным. Такой дизайн создает четкое разделение между доступом к данным и UI, при этом приходится обслуживать лишь самое минимальное количество кода UI. В листинге 8.4 показан полный исходный код класса `PhotoData`.

### ПРИМЕЧАНИЕ

Класс `PhotoData` является примером минималистского подхода и служит лишь для цели демонстрации концепций. Строка подключения определена как жестко запрограммированная константа, но в реальном приложении строку подключения следует шифровать и хранить в другом месте (например, в файле параметров конфигурации). В коде используется также конструкция `using` языка `C#` для того, чтобы все размещающие неуправляемые ресурсы объекты (как подключения баз данных и файловые дескрипторы) были должным образом освобождены даже в случае возникновения исключения без всякой дополнительной логики обработки ошибок. Реальные же приложения должны реализовывать надежную стратегию обработки исклю-

чительных состояний, которая включает использование блоков try/catch/finally, журналирование ошибок и проверку.

**Листинг 8.4. Реализация управляемого клиентского класса для доступа к данным FILESTREAM**

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.IO;
using Microsoft.Win32.SafeHandles;
namespace PhotoLibraryApp
{
    public class PhotoData
    {
        private const string ConnStr = "Data Source=.;
            Integrated Security=True;Initial Catalog=PhotoLibrary;";
        #region "Insert Photo"
        public static void InsertPhoto(int photoId,
            string desc, string filename)
        {
            const string InsertCmd =
                "INSERT INTO PhotoAlbum(PhotoId, Description)" +
                " VALUES(@PhotoId, @Description)";
            using (SqlConnection conn = new SqlConnection(ConnStr))
            {
                conn.Open();
                using (SqlTransaction txn = conn.BeginTransaction())
                {
                    using (SqlCommand cmd = new SqlCommand(InsertCmd, conn, txn))
                    {
                        cmd.Parameters.Add("@PhotoId",
                            SqlDbType.Int).Value = photoId;
                        cmd.Parameters.Add("@Description",
                            SqlDbType.VarChar).Value = desc;
                        cmd.ExecuteNonQuery();
                    }
                    SavePhotoFile(photoId, filename, txn);
                    txn.Commit();
                }
            }
        }
    }
}
```

```
    }
    conn.Close();
}
}
private static void SavePhotoFile
    (int photoId, string filename, SqlTransaction txn)
{
    const int BlockSize = 1024 * 512;
    FileStream source = new FileStream(filename, FileMode.Open,
                                       FileAccess.Read);
    SafeFileHandle handle = GetOutputFileHandle(photoId, txn);
    using (FileStream dest = new FileStream(handle, FileAccess.Write))
    {
        byte[] buffer = new byte[BlockSize];
        int bytesRead;
        while ((bytesRead = source.Read(buffer, 0, buffer.Length)) > 0)
        {
            dest.Write(buffer, 0, bytesRead);
            dest.Flush();
        }
        dest.Close();
    }
    source.Close();
}
private static SafeFileHandle GetOutputFileHandle
    (int photoId, SqlTransaction txn)
{
    const string GetOutputFileInfoCmd =
        "SELECT Photo.PathName(), GET_FILESTREAM_TRANSACTION_CONTEXT() +
        " FROM PhotoAlbum" +
        " WHERE PhotoId = @PhotoId";
    SqlCommand cmd = new SqlCommand(GetOutputFileInfoCmd,
                                     txn.Connection, txn);
    cmd.Parameters.Add("@PhotoId", SqlDbType.Int).Value = photoId;
    string filePath;
    byte[] txnToken;
    using (SqlDataReader rdr =
        cmd.ExecuteReader(CommandBehavior.SingleRow))
    {
        rdr.Read();
    }
}
```

```

        filePath = rdr.GetSqlString(0).Value;
        txnToken = rdr.GetSqlBinary(1).Value;
        rdr.Close();
    }
    SafeFileHandle handle =
        NativeSqlClient.GetSqlFileStreamHandle
            (filePath, NativeSqlClient.DesiredAccess.ReadWrite, txnToken);
    return handle;
}
#endregion

#region "Select Photo"
public static Image SelectPhoto(int photoId, out string desc)
{
    const string SelectCmd =
        "SELECT Description, Photo.PathName(),
            GET_FILESTREAM_TRANSACTION_CONTEXT()" +
        " FROM PhotoAlbum" + " WHERE PhotoId = @PhotoId";
    Image photo;
    using (SqlConnection conn = new SqlConnection(ConnStr))
    {
        conn.Open();
        using (SqlTransaction txn = conn.BeginTransaction())
        {
            string filePath;
            byte[] txnToken;
            using (SqlCommand cmd = new SqlCommand(SelectCmd, conn, txn))
            {
                cmd.Parameters.Add("@PhotoId", SqlDbType.Int).Value = photoId;
                using (SqlDataReader rdr =
                    cmd.ExecuteReader(CommandBehavior.SingleRow))
                {
                    rdr.Read();
                    desc = rdr.GetSqlString(0).Value;
                    filePath = rdr.GetSqlString(1).Value;
                    txnToken = rdr.GetSqlBinary(2).Value;
                    rdr.Close();
                }
            }
        }
        photo = LoadPhotoImage(filePath, txnToken);
    }
}

```

```
        txn.Commit();
    }
    conn.Close();
}
return photo;
}
private static Image LoadPhotoImage(string filePath, byte[] txnToken)
{
    Image photo;
    SafeFileHandle handle =
        NativeSqlClient.GetSqlFilestreamHandle
            (filePath, NativeSqlClient.DesiredAccess.Read, txnToken);
    using (FileStream fs = new FileStream(handle, FileAccess.Read))
    {
        photo = Image.FromStream(fs);
        fs.Close();
    }
    return photo;
}
#endregion
}
}
```

В нашем приложении есть еще один небольшой исходный файл `NativeSqlClient`, который инкапсулирует взаимодействие с COM (Component Object Model) Interop и подробности интерфейса вызовов неуправляемого кода `OpenSqlFilestream` из нашего управляемого кода. Фактически `OpenSqlFilestream` вызывается именно этим классом `NativeSqlClient` (наши клиентские приложения на управляемом коде вызывают `NativeSqlClient` для выдачи всех запросов `OpenSqlFilestream`). Мы начнем наше подробное изучение кода с класса `PhotoData` и рассмотрим служебный класс `NativeSqlClient` тогда, когда нам будет нужно его вызвать.

Мы начинаем с включения некоторых необходимых пространств имен. Здесь необходимо отметить `Microsoft.Win32.SafeHandles`, где определяется объект `SafeFileHandle` (возвращаемый из `OpenSqlFilestream`), который мы будем использовать для потоковой передачи BLOB. Для использования класса `Microsoft.Win32.SafeHandles.SafeFileHandle` не нужно делать специальной ссылки на сборку, поскольку он предоставляется основной сборкой .NET-библиотек `mscorlib.dll`. Мы определяем также строку подключения в виде жестко запрограммированной константы (так сделано исключительно в

демонстрационных целях). Реальное приложение должно шифровать строку подключения и хранить ее в другом месте (например, таком как файл параметров конфигурации), мы же стараемся упростить наш пример.

Первый определенный в классе метод — это `InsertPhoto`, который принимает целочисленный ID новой фотографии, строковое описание, а также полный путь к файлу изображения (которое должно быть сохранено в базе данных):

```
public static void InsertPhoto(int photoId, string desc, string filename)
{
    const string InsertCmd =
        "INSERT INTO PhotoAlbum(PhotoId, Description) " +
        " VALUES (@PhotoId, @Description)";
    using(SqlConnection conn = new SqlConnection(ConnStr))
    {
        conn.Open();
        using(SqlTransaction txn = conn.BeginTransaction())
        {
            using(SqlCommand cmd = new SqlCommand(InsertCmd, conn, txn))
            {
                cmd.Parameters.Add("@PhotoId", SqlDbType.Int).Value =
                    photoId;
                cmd.Parameters.Add("@Description",
                    SqlDbType.VarChar).Value = desc;
                cmd.ExecuteNonQuery();
            }
            SavePhotoFile(photoId, filename, txn);
            txn.Commit();
        }
        conn.Close();
    }
}
```

Метод сначала создает и открывает новое подключение `SqlConnection`, а затем иницирует транзакцию базы данных при помощи класса `SqlTransaction`. Затем он создает объект `SqlCommand` (связанный с открытым подключением и иницированной транзакцией) и подготавливает свой командный текст с инструкцией `INSERT` (определенной в строковой константе `InsertCmd`), который и сохраняет ID фотографии и описание в новую запись `PhotoAlbum`. Наша инструкция `INSERT` не предоставляет значения для `RowId` и позволяет SQL Server по умолчанию автоматически сгенерировать и присвоить новое значение `ROWGUID` типа `uniqueidentifier` (точно так же, как мы делали это ранее при использо-

вании T-SQL для вставки первой строки). Мы также не предоставляем значения для столбца `Photo`, и именно здесь в игру вступает значение по умолчанию `0x`, которое мы определили ранее для столбца `Photo`. После выполнения `INSERT` (посредством вызова `ExecuteNonQuery`) транзакция еще не завершена. Несмотря на то, что строка добавлена, она будет откатена назад (и исчезнет) в том случае, если до фиксации транзакции случится ошибка. Поскольку мы не предоставили значение `BLOB` для столбца `Photo` новой строки, то SQL Server использует установленное нами значение по умолчанию `0x` (это было сделано в инструкции `CREATE TABLE` таблицы `PhotoAlbum`). Поскольку это столбец `varbinary(max)` с атрибутом `FILESTREAM`, то это приводит к добавлению пустого файла (связанного с новой строкой) в файловую систему. И так же, как и новая строка, этот пустой `BLOB`-файл исчезает в том случае, когда транзакция базы данных не завершается успешно.

### **ВАЖНО**

Вы не можете открыть дескриптор файла для значения `NULL` столбца. Если вы хотите использовать `OpenSqlFileStream`, то при вставке новых строк необходимо всегда использовать двоичное значение `0x` со столбцами `varbinary(max)` `FILESTREAM`. Это приведет к созданию файла нулевой длины, который может быть перезаписан вызовом `OpenSqlFileStream` (как мы это и делаем).

Именно в этот момент мы вызываем метод `SavePhotoFile` потокового вывода указанного файла изображения в столбец `Photo` только что созданной строки таблицы `PhotoAlbum` (перезаписывая только что созданный пустой файл по умолчанию). Когда управление возвращается из `SavePhotoFile`, то транзакция уже полностью зафиксирована, и подключение закрыто. На этом заканчивается обновление как базы данных, так и файловой системы (структурированным и неструктурированным содержимым новой строки в `PhotoAlbum`).

Метод `SavePhotoFile` читает из файла источника и пишет в базу данных блоками по 512 Кбайт при помощи обычного объекта `FileStream`:

```
private static void SavePhotoFile(int photoId, string filename,
                                  SqlTransaction txn)
{
    const int BlockSize = 1024 * 512;
    FileStream source = new FileStream(filename, FileMode.Open,
                                       FileAccess.Read);
    SafeFileHandle handle = GetOutputFileHandle(photoId, txn);
    using (FileStream dest = new FileStream(handle, FileAccess.Write))
    {
        byte[] buffer = new byte[BlockSize];
```

```

int bytesRead;
while((bytesRead = source.Read(buffer, 0, buffer.Length)) > 0)
{
    dest.Write(buffer, 0, bytesRead);
    dest.Flush();
}
dest.Close();
}
source.Close();
}

```

Метод начинается с определения целочисленной константы `BlockSize`, установленной в разумное значение (512 Кбайт). Изображения большего размера будут отправляться на сервер блоками по 512 Кбайт. Локальный исходный файл изображения сначала открывается в объекте `FileStream` только для чтения. Чтобы получить объект `FileStream` для записи, мы вызываем метод `GetOutputFileHandle` (передавая ему ID фотографии и незавершенную транзакцию) и получаем обратно объект `SafeFileHandle` (определенный в пространстве имен `Microsoft.Win32.SafeHandles`, которое было импортировано инструкцией `using` в верхней части файла исходного кода). Класс `FileStream` предлагает конструктор, принимающий объект `SafeFileHandle`, который мы используем для получения доступа на запись BLOB приемника (в файловой системе NTFS сервера базы данных). Помните, что этот выходной файл включен в транзакцию NTFS и не будет сохранен до тех пор, пока транзакция базы данных не будет зафиксирована кодом, который вызывает `SavePhotoFile`.

Оставшаяся часть метода `SavePhotoFile` реализует простой цикл, который читает из `FileStream` источника и пишет в `FileStream` назначения (до тех пор, пока весь исходный файл не будет обработан), а затем закрывает оба потока.

Давайте теперь изучим метод `GetOutputFileHandle`, который вызывается методом `SavePhotoFile` для получения дескриптора назначения потокового вывода в BLOB-файл:

```

private static SafeFileHandle GetOutputFileHandle(int photoId,
                                                    SqlTransaction txn)
{
    const string GetOutputFileInfoCmd =
        "SELECT GET_FILESTREAM_TRANSACTION_CONTEXT(), Photo.PathName()" +
        " FROM PhotoAlbum" + " WHERE PhotoId = @PhotoId";
    SqlCommand cmd = new SqlCommand(GetOutputFileInfoCmd,
                                     txn.Connection, txn);
    cmd.Parameters.Add("@PhotoId", SqlDbType.Int).Value = photoId;
}

```



```
string filePath;
byte[] txnToken;
using(SqlDataReader rdr =
    cmd.ExecuteReader(CommandBehavior.SingleRow))
{
    rdr.Read();
    txnToken = rdr.GetSqlBinary(0).Value;
    filePath = rdr.GetSqlString(1).Value;
    rdr.Close();
}

SafeFileHandle handle =
    NativeSqlClient.GetSqlFilestreamHandle
        (filePath, NativeSqlClient.DesiredAccess.ReadWrite, txnToken);
return handle;
}
```

Этот код является ключевым в использовании FILESTREAM. Он вызывается в тот момент времени, когда в таблицу PhotoAlbum уже добавлена новая строка и соответствующий ей пустой BLOB уже добавлен в файловую систему, но до фиксирования транзакций, в которых содержатся эти операции. Это именно тот момент времени, когда мы должны подключиться к процессу и выполнить потоковую передачу BLOB-данных в базу данных при помощи OpenSqlFilestream. Вспомните из нашего предыдущего обсуждения, что нам нужно два элемента информации: маркер контекста транзакции и логический путь UNC к самому файлу. Мы получаем оба эти элемента в строке SqlDataReader при помощи инструкции SELECT, возвращая GET\_FILESTREAM\_TRANSACTION\_CONTEXT и Photo.PathName.

Поскольку мы начали транзакцию базы данных до выполнения инструкции INSERT, то SQL Server инициировал транзакцию файловой системы NTFS для данных FILESTREAM столбца Photo новой строки. Функция GET\_FILESTREAM\_TRANSACTION\_CONTEXT возвращает дескриптор транзакции NTFS. SQL Server автоматически зафиксирует эту транзакцию NTFS, когда мы зафиксируем транзакцию базы данных (или откатит транзакцию NTFS, если мы откатим транзакцию базы данных). Когда мы получаем контекст транзакции (значение SqlBinary), мы сохраняем его в массиве байтов txnToken.

Второе возвращаемое нашей инструкцией SELECT значение — это Photo.PathName, сформированный путь (в формате UNC, вместе с именем файла) к BLOB выбранного PhotoId. По существу при помощи выражения WHERE мы читаем ту же самую строку, которую только что добавили (но не зафиксировали) в таблицу PhotoAlbum, чтобы получить полный путь к BLOB

в новом файле, только что созданном (и тоже не зафиксированном) в файловой системе. Затем мы сохраняем его в строковой переменной `filePath`.

При наличии контекста транзакции `FILESTREAM` и полного пути к BLOB-файлу у нас теперь есть все для вызова неуправляемой функции `OpenSqlFilestream` клиента SQL и получения дескриптора выходного файла потоковой передачи нашего содержимого. Однако фактически мы не вызываем `OpenSqlFilestream` напрямую из нашего класса доступа к данным (хотя могли бы). Вместо этого мы вызываем метод `GetSqlFilestreamHandle`, определенный в нашем вспомогательном классе `NativeSqlClient`, который вызывает `OpenSqlFilestream` (как показано в листинге 8.5).

#### Листинг 8.5. Вызов `OpenSqlFilestream`

```
using System;
using System.Runtime.InteropServices;
using Microsoft.Win32.SafeHandles;
namespace PhotoLibraryFilestreamDemo
{
    public class NativeSqlClient
    {
        public enum DesiredAccess : uint
        {
            Read,
            Write,
            ReadWrite,
        }

        [DllImport("sqlncli10.dll", SetLastError = true, CharSet =
CharSet.Unicode)]
        private static extern SafeFileHandle OpenSqlFilestream(
            string path,
            uint access,
            uint options,
            byte[] txnToken,
            uint txnTokenLength,
            Sql64 allocationSize);

        [StructLayout(LayoutKind.Sequential)]
        private struct Sql64
        {
            public Int64 QuadPart;
            public Sql64(Int64 quadPart)
```

```
        {
            this.QuadPart = quadPart;
        }
    }
    public static SafeFileHandle GetSqlFilestreamHandle
        (string filePath, DesiredAccess access, byte[] txnToken)
    {
        SafeFileHandle handle = OpenSqlFilestream(
            filePath,
            (uint)access,
            0,
            txnToken,
            (uint)txnToken.Length,
            new Sql64(0));
        return handle;
    }
}
```

Как вы видите, метод `GetSqlFilestreamHandle` просто обертывает неуправляемую функцию `OpenSqlFilestream`, которая определена с внешней ссылкой на `sqlncli10.dll` (SQL Native Client version 10) при помощи атрибута `DllImport`. Метод `GetSqlFilestreamHandle` принимает маркер контекста транзакции и полный путь к BLOB-файлу. Он также принимает значение из перечисления, которое указывает желаемый режим доступа (это может быть `Read`, `Write` или `ReadWrite`). Функция `OpenSqlFilestream` требует и других параметров, которые в общем случае для стандартного применения `FILESTREAM` не используются (такие как беззнаковые 32-битные опции или 64-битные аргументы размера размещения). Вместо них просто передаются нули.

### **ПРИМЕЧАНИЕ**

Цель нашего клиентского класса `NativeSqlClient` состоит в том, чтобы отделить взаимодействие с COM и 64-битные целые числа SQL от нашего кода доступа к данным, создав таким образом разделение между управляемым и неуправляемым кодом. В результате получается более четкий и легкий в обслуживании код. Мы предоставляем простой метод-оболочку из управляемого кода `GetSqlFilestreamHandle` для неуправляемой функции `OpenSqlFilestream`, чтобы наш класс `PhotoData` и любые другие управляемые классы доступа к данным не нуждались в знании подробностей неуправляемого кода.

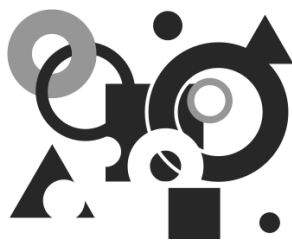
Вот так вставляются новые фотографии. Возвращаясь к классу `PhotoData`: остальные методы делают запросы по `PhotoId` и выполняют потоковую передачу содержимого выбранного файла из базы данных в объект `Image` для его отображения.

## Резюме

Для приложений, которые работают с BLOB-данными, новая функциональная возможность `FILESTREAM` в `SQL Server 2008` значительно улучшает способ хранения и производительность неструктурированного содержимого базы данных путем использования файловой системы `NTFS`. При этом обеспечивается логическая интеграция базы данных и файловой системы (в том числе и поддержка транзакций). В результате нам больше не нужно идти на компромиссы между эффективностью и сложностью, как в прошлом, когда нам приходилось делать выбор между хранением BLOB-данных внутри или вне базы данных.

Вы научились также использовать функцию `OpenSqlFilestream` (интерфейса прикладного программирования неуправляемого клиента) для обеспечения высокопроизводительной потоковой передачи BLOB-содержимого между файловой системой (управляемой сервером `SQL Server`) и вашими `Windows`-, `Web`- и `WPF`-приложениями. Концепции применимы к широкому диапазону приложений, которым требуется интеграция между реляционной базой данных и потоковой файловой системой.

## ГЛАВА 9



# Геопространственные типы данных

*Леонард Лобел*

Геопространственная поддержка — это новое мощное расширение ядра базы данных в версии Microsoft SQL Server 2008. Несмотря на то, что ее называют просто пространственной, эта функциональная возможность позволяет вам хранить фигуры, размеры и местоположения (и манипулировать ими) при помощи новых типов данных `geometry` и `geography`, а также вычислять по этим данным площади, расстояния и пересечения. Эти возможности включают в себя и осведомленность о местоположении для приложений, в которых географические данные теперь могут обрабатываться на уровне базы данных. В этой главе мы изучаем концепции, лежащие в основе пространственных данных, а также и то, как эта новая поддержка в SQL Server 2008 может быть использована для интегрирования анализа местоположения в ваши приложения баз данных.

### **ПРИМЕЧАНИЕ**

В этой главе мы используем термины "пространственный" и "геопространственный" в одном и том же смысле.

Геопространственные возможности соответствуют относительно небольшой нише проектов разработки баз данных, однако с течением времени следует ожидать увеличения потребности бизнеса в таких возможностях, даже если это произойдет косвенным образом. Несмотря на то, что система глобального позиционирования (Global Positioning System, GPS) и картографические приложения (такие как Microsoft Virtual Earth и MapQuest) являются самыми типичными примерами использования геопространственных возможностей, для применения пространственных данных вам не обязательно отображать гра-

фические карты. Практическое применение может быть найдено также и в традиционных бизнес-приложениях. Например, посещающие ваш сайт электронной коммерции покупатели могут искать продукты и получить список местоположений филиалов, где имеются эти продукты (упорядоченный по удаленности от местонахождения покупателя).

## SQL Server 2008 увеличивает возможности

Добавив богатую пространственную поддержку в SQL Server 2008, компания Microsoft выровнялась с другими основными поставщиками (которые уже давно предоставляют такую функциональную возможность) и позволила разработчикам SQL Server использовать пространственные возможности на уровне базы данных. Сама база данных теперь понимает пространственные данные (как собственный тип данных) и может эффективно хранить и обрабатывать для вас экземпляры данных этих типов. Не нужно больше выполнять пространственные вычисления на уровне приложения при помощи собственного кода (который совсем не прост) или при помощи библиотек третьей стороны (что совсем не дешево).

Используемые в геопрограммных операциях алгоритмы очень сложны (мягко выражаясь), поэтому полное изложение данной темы выходит за пределы нашего обсуждения. В то же время абстрагирование сложности является ключевым аспектом этих новых пространственных типов данных.

Геопрограммная поддержка в SQL Server 2008 предназначена специально для быстрой разработки пространственных возможностей ваших баз данных (без необходимости глубоких знаний программирования пространственных вычислений). В этой главе вы научитесь базовым пространственным операциям. Для этого вы сначала разберетесь в двух основных пространственных моделях, а затем пройдете по нескольким примерам кодов, в которых используются новые типы данных `geometry` и `geography`.

### ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

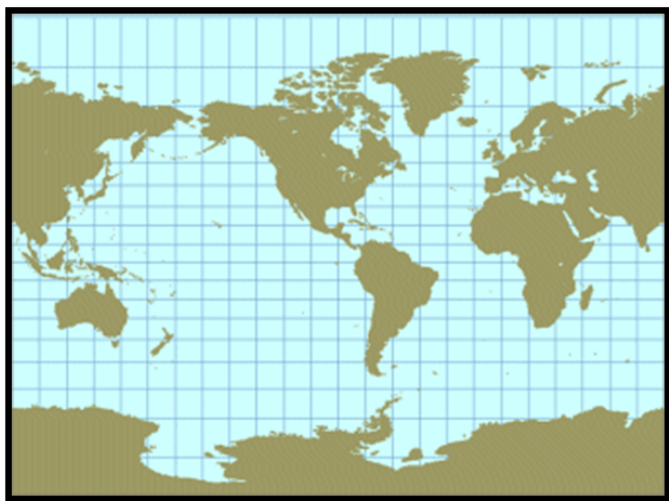
SQL Server 2008 поддерживает OpenGIS Simple Features из стандарта SQL, это опубликованная международным регулирующим органом (Open Geospatial Consortium, OGC) спецификация. В ней представлено более 70 методов работы с новыми пространственными типами SQL Server 2008, хотя в примерах наших кодов мы изучаем только самые основные. Вы можете загрузить pdf-файл спецификации с адреса: <http://www.opengeospatial.org/standards/sfs>.

## Пространственные модели

Давайте начнем наше обсуждение с объяснения двух основных геопространственных моделей: *планарной* и *геодезической*.

### Планарная модель "плоской Земли"

Планарная модель — это плоская поверхность, где фигуры рисуются по двум измерениям (координатам). Эти координаты основаны на произвольной системе измерения, поэтому вы можете определить любую единицу измерения по вашему желанию (например, сантиметры, метры, километры, дюймы, футы, мили, пиксели и т. д.).



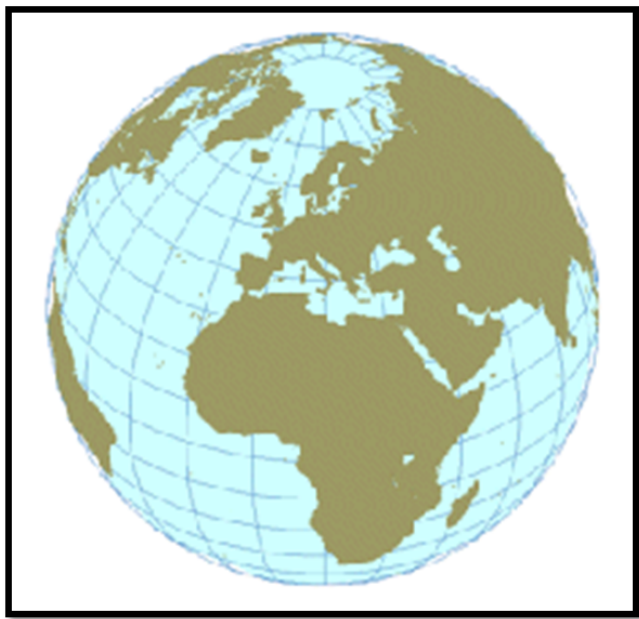
**Рис. 9.1.** Планарная пространственная модель  
(проекция плоской земли)

Работать с планарной моделью можно только тогда, когда имеешь дело с относительно небольшими площадями (такими как строительные планы этажей) или с более крупными площадями, которые либо концептуально плоские, либо достаточно малы для того, чтобы кривизна Земли не искажала результаты вычислений площадей и расстояний. Учтите, что проецирование Земли на плоскую поверхность приводит к пространственным искажениям, которые делают невозможным точное измерение больших пространств по

нашей планете. Например, если вы нарисуете прямую линию из Нью-Йорка в Тель-Авив по карте, показанной на рис. 9.1, то данная линия не даст вам точного расстояния между этими двумя городами. На самом деле маршрут этого полета будет представлен растянутыми и изогнутыми по форме глобуса кусками этой линии. То есть планарная модель определенно не подходит для вычисления дальних траекторий.

## Геодезическая модель "круглой Земли"

Если вам необходимо выполнять пространственные вычисления, которые охватывают большие области Земли, то вашим приложениям базы данных придется принять широко распространенное поверье о том, что Земля круглая. Для правильного рисования и вычисления глобальных фигур и пересечений необходимо учитывать кривизну Земли. Геодезическая модель (показанная на рис. 9.2) представляет местоположения на планете при помощи координатной системы с долготой и широтой.



**Рис. 9.2.** Геодезическая пространственная модель  
(проекция "круглой Земли")



## Пространственные типы данных

Для работы с пространственными данными SQL Server 2008 вводит два новых типа данных — по одному для каждой из пространственных моделей (как вы уже, вероятно, догадались). Тип данных `geometry` предназначен для хранения и обработки пространственных данных планарной модели ("плоская Земля"), а тип `geography` поддерживает геодезическую модель ("круглая Земля").

Замечательно то, что оба этих новых пространственных типа используют большое количество одинаковых методов для общих пространственных вычислений. При работе с геодезической моделью вы будете использовать координаты долготы и широты совместно с типом данных `geography`. Для планарной модели вы будете использовать координаты  $x$  и  $y$  совместно с типом данных `geometry`. Но в любом случае вы будете использовать для работы с вашими пространственными объектами большое количество одних и тех же методов.

## Определяем пространство при помощи Well-Known Text

Фигуры проецируются на пространственные модели при помощи *векторных объектов*, которые являются коллекциями точек, линий и многоугольников (замкнутые фигуры). Оба типа данных (`geometry` и `geography`) поддерживают язык разметки Well-Known Text (WKT), являющийся набором правил выражения векторных объектов, которые вы определяете при помощи синтаксиса, управляемого консорциумом OGC.

В табл. 9.1 показаны некоторые примеры строк WKT.

**Таблица 9.1.** Примеры строк WKT

| Строка WKT                               | Описание                                                                                        |
|------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>POINT(6 10)</code>                 | Одна точка с координатами (6, 10)                                                               |
| <code>POINT(-111.06687 45.01188)</code>  | Одна точка на Земле<br>(координаты: долгота/широта)                                             |
| <code>LINESTRING(3 4,10 50,20 25)</code> | Линия из двух частей, которая нарисована между тремя точками, указанными координатами $x$ и $y$ |

Таблица 9.1 (окончание)

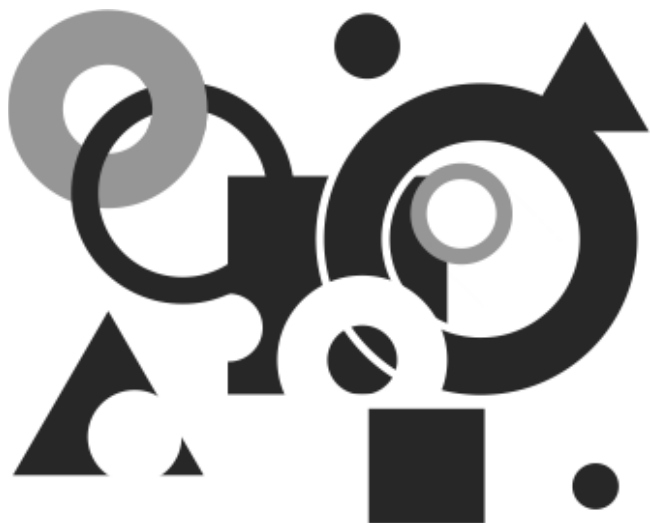
| Строка WKT                                                                                                     | Описание                                                                                  |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| POLYGON(( -75.17031 39.95601, -75.16786 39.95778, -75.17921 39.96874, -75.18441 39.96512, -75.17031 39.95601)) | Замкнутая фигура на Земле, рисуемая между точками, указанными координатами долгота/широта |

Как вы видите, один и тот же синтаксис WKT используется для выражения пространственных элементов как при помощи планарной, так и геодезической модели. Обратите также внимание, что геодезические координаты всегда выражаются в WKT таким образом: сначала идет значение долготы, а затем значение широты.

Имея эти самые основные знания геопространственных концепций, мы готовы начать работать с пространственными данными в SQL Server 2008.

## Резюме

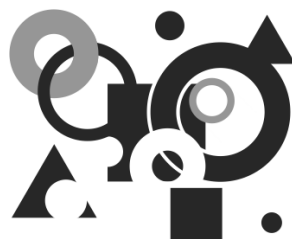
Геопространственные возможности больше не ограничены рамками сложных приложений географических информационных систем. Мощь пространственной обработки теперь доступна всем разработчикам, использующим новые типы данных `geometry` и `geography` в SQL Server 2008. В этой главе вы научились использовать эти типы (путем создания геопространственных приложений, которые интегрируют аналитику местоположения и возможности картографирования). Вы видели, как легко можно выражать и проецировать координаты в планарной ("плоская Земля") или геодезической ("круглая Земля") пространственных моделях при помощи синтаксиса Well-Known Text (WKT).



## **ЧАСТЬ III**

### **ТЕХНОЛОГИИ ДОСТУПА**

## ГЛАВА 10



# Microsoft Data Access Machine

*Эндрю Брест*

Каждый, кто давно работает с технологиями компании, знает, что Microsoft регулярно создает новые интерфейсы прикладного программирования доступа к данным и инструменты для них. Так, во времена до .NET программисты на языке Microsoft Visual Basic сначала работали с Data Access Objects (DAO), затем перешли к Remote Data Objects (RDO), а потом — к ActiveX Data Objects (ADO). Microsoft ADO.NET с его совершенным интерфейсом прикладного программирования данных (в частности, Microsoft SQL Server) и типизированных объектов DataSet стал мощным инструментом быстрой разработки приложений (rapid application development, RAD). Но с выпуском Microsoft Visual Studio 2008, Microsoft .NET Framework 3.5 и последующим выходом для них (в августе 2008 г.) пакета обновлений Service Pack 1 (SP1) количество новых возможностей и опций (появившихся в мире .NET-доступа к данным) стало просто невиданным (даже для компании Microsoft).

Введение интегрированных в язык запросов (language-integrated query, LINQ) для языков C# и Visual Basic .NET, а также технологии LINQ для SQL и ADO.NET Entity Framework, поднимает уровень абстракции данных гораздо выше, чем это обеспечивалось в ADO.NET. В то же время LINQ — это язык ADO.NET (LINQ для DataSet), а ADO.NET можно использовать как интерфейс к Entity Framework поставщика EntityClient. Поэтому ADO.NET по-прежнему фигурирует в числе технологий доступа к данным. Это может привести к еще большей путанице в мире новых технологий.

На этом выбор не заканчивается. Есть еще система ADO.NET Data Services, которая сращивает Entity Framework с технологией Web-служб representational state transfer (REST). И если идея выделенной Web-службы для данных приводит вас в восторг, то скорее всего вас заинтересует и пол-

ностью "облачная" версия этой концепции: служба SQL Server Data Services. Это не просто переварить, особенно когда вы обнаружите, что ADO.NET и типизированные DataSet также были улучшены в .NET Framework 3.5, и что надо разобраться как в старых, так и в новых возможностях ADO.NET.

Наша цель в данной главе — дать вам обзор каждой из этих технологий, обсудив их сходства и различия, а также преимущества и недостатки. Не рассчитывая глубоко погрузиться, мы надеемся, что сможем помочь вам познаться, а также создать хороший задел для подробного изучения наиболее привлекательных для вас моделей. Эта глава посвящена использованию упомянутых технологий подготовки данных и некоторым методам кодирования.

В этой главе мы будем заниматься также и привязкой данных, просто заодно. В следующей главе мы более подробно опишем технологии и инструменты привязки данных в Windows Forms, ASP.NET (в том числе ASP.NET Asynchronous Java and XML [AJAX]), Windows Presentation Foundation (WPF) и Silverlight. Мы опишем также ASP.NET Dynamic Data — технологию, которая позволяет вам генерировать целые Web-сайты для управления данными (созданные в соответствии с моделями, сконструированными при помощи LINQ для SQL или Entity Framework).

## ADO.NET и типизированные наборы *DataSet*

Достижение всех этих целей — трудная задача, поэтому приступим. Лучше всего начать с ADO.NET — технологии, которая лучше всех знакома большинству читателей этой книги. Как уже обсуждалось ранее, нам нужно описать новые функциональные возможности и дать обзор тех, которые существуют уже давно. Но вместо того, чтобы описывать самые простые элементы, обсудим основы непосредственной работы с таблицами и те дополнительные шаги, которые требуются для работы с хранимыми процедурами.

## Основы типизированных объектов *DataSet*

При переходе от классической системы ADO (основанной на Component Object Model, COM) к ADO.NET 1.0, компания Microsoft ввела две радикально новые концепции. Первая состояла в замене подключенного объекта Recordset (ориентированного на один результирующий набор) отключенным объектом DataSet (реляционным объектом, ориентированным на несколько результирующих наборов). Вторая новая концепция (внесенная средой Visual

Studio .NET и основанная на поддержке наследования в .NET) — это способность создавать строго типизированные объекты DataSet (известные также просто, как типизированные объекты DataSet).

Типизированные объекты DataSet добавляют специфичные для схемы вложенные классы, методы и свойства для того, чтобы сделать доступ к данным (извне базы данных) более чувствительным к контексту. Большое преимущество типизированных объектов DataSet в том, что их объектная модель обеспечивает в вашем коде соблюдение точности типов на уровне столбцов (когда вы используете их реализованные свойства). Например, если вы попытаетесь сохранить строковое значение в столбец с числовым типом данных, то вы получите ошибку компилятора. Более того, вложенные классы и свойства (совместно с технологией IntelliSense среды Visual Studio) предотвратят ошибки написания названий таблиц или столбцов. И это далеко не все преимущества.

Типизированные объекты DataSet могут быть отличным инструментом для ваших приложений и объектов промежуточного уровня. Вне зависимости от того, хотите ли вы использовать преимущества технологии привязки данных, типизированные объекты DataSet предоставляют вам элегантный слой инкапсуляции для ваших данных, их схемы и различных запросов по этим данным.

В среде Visual Studio 2008 (как и в более ранних версиях Visual Studio) типизированные объекты DataSet состоят из одного или более объектов DataTable и (необязательно) связей между этими объектами. В визуальном конструкторе Visual Studio для типизированного DataSet вы визуально указываете структуру объектов DataTable, причем вы можете сделать это либо посредством создания их с нуля (при помощи мастера TableAdapter Configuration Wizard), либо при помощи перетаскивания (с использованием окна **Server Explorer**).

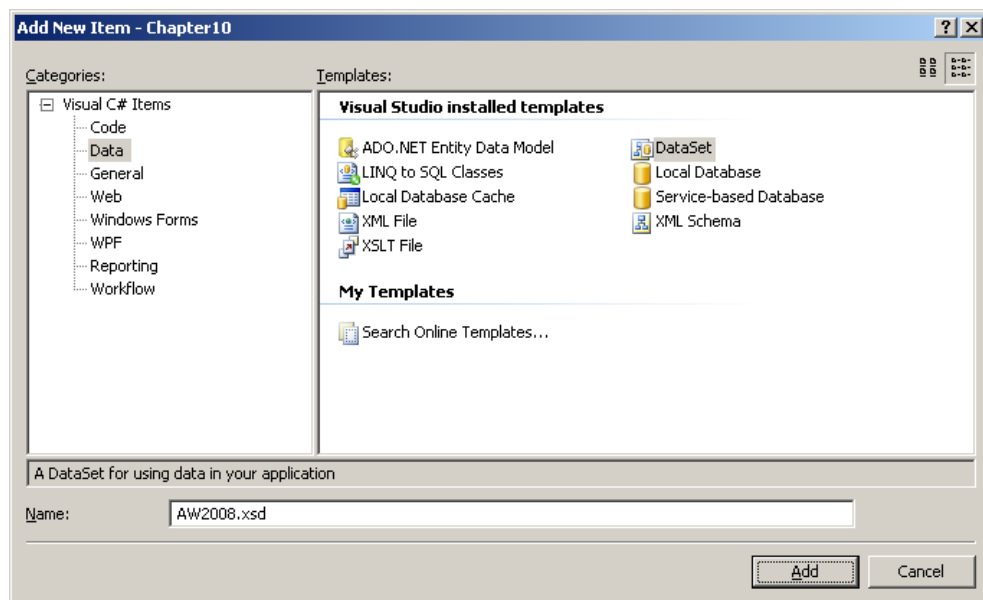
Пример решения для этой главы имеет вид проекта Visual Studio типа Windows Application на языке C# с названием Chapter10. Вы можете изучать типизированный объект DataSet с названием AW2008.xsd и формы этого проекта по мере чтения вами первого раздела данной главы либо создать новый проект типа Windows Application на языке C#, а затем создать типизированный объект DataSet, который мы будем обсуждать в данном разделе.

### **ПРИМЕЧАНИЕ**

Пример решения Visual Studio для данной главы (как и примеры кодов для всей этой книги) доступны на Web-сайте данной книги.

## Использование визуального конструктора DataSet

Для начала запустите Visual Studio 2008 и создайте новый проект Windows Forms. Затем (при помощи клавиатуры, пунктов меню или соответствующего контекстного меню узла проекта в окне **Solution Explorer**) откройте диалоговое окно **Add New Item**. Из списка объектов выберите DataSet и введите название AW2008.xsd (рис. 10.1).



**Рис. 10.1.** Диалоговое окно **Add New Item-Chapter10** в Visual Studio с выбранной опцией DataSet

Нажмите **ОК** и наблюдайте за тем, как объект DataSet открывается в визуальном конструкторе в области документов Visual Studio. Для нового типизированного DataSet визуальный конструктор отображается просто с пустым фоном в сине-белую полоску и текстом, который дает вам указания по добавлению элементов при помощи вкладки **DataSet** окна **Toolbox** (или окна **Server Explorer**).

Начните с добавления в ваш новый типизированный объект DataSet таблицы Sales.Currency. Убедитесь, что вы определили подключение к базе данных AdventureWorks2008 (в окне **Server Explorer**). После определения подключения в узле **Data Connections** окна **Server Explorer** должен появиться узел,

который указывает на базу данных AdventureWorks2008. Разверните узел **AdventureWorks2008**, а затем разверните узел **Tables** внутри него. И наконец, найдите узел **Currency (Sales)** и перетащите его в область конструктора. Ваш набор DataSet должен теперь содержать полностью заданные определения объектов DataTable и TableAdapter для таблицы Currency.

Мы расскажем вам, как добавлять запросы к объекту TableAdapter, и добавим таблицы в типизированный объект DataSet, но сначала давайте проведем инвентаризацию того, что у нас уже есть. Обратите внимание, что большой прямоугольник (представляющий таблицу Sales.Currency) разделен на две части. Верхняя часть (с меткой Currency) представляет собой фактическую таблицу DataTable внутри объекта DataSet. Щелкните по одному из столбцов таблицы и посмотрите в окно **Properties**. Вы увидите, что объект DataColumn (являющийся визуальным представлением обобщенного объекта DataColumn в ADO.NET) имеет редактируемые свойства. Среди них самым примечательным является свойство Caption. Изменение свойства Caption столбца приведет к тому, что все описательные метки (для привязанных впоследствии к данным) элементов управления этого столбца будут показывать заголовок Caption, а не имя физического столбца базы данных.

Интересно также свойство Expression, которое позволяет вам настроить столбец для получения вычисленного значения. Свойства ReadOnly и MaxLength служат для декларативных правил проверки данных (независимых от ограничений базы данных), свойство DefaultValue работает независимо от метаданных физических таблиц, свойства AllowDBNull и NullValue управляют обработкой неопределенных значений (также независимо от базы данных).

## Объекты TableAdapter

Нижняя часть прямоугольника (с меткой CurrencyTableAdapter) представляет собой строго типизированный объект DataAdapter, который управляет заполнением и модификациями данных объекта DataTable с названием Currency. Выберите ее и посмотрите в окно **Properties**. Показано несколько (но не все) стандартных свойств DataAdapter, дано также и свойство Connection, которое позволяет вам указать то подключение, которое будет использоваться свойствами дочерних объектов (SelectCommand, InsertCommand, UpdateCommand, и DeleteCommand) объекта TableAdapter.

Как сегмент DataTable содержит внутри себя объекты Column, так и объект TableAdapter содержит внутри себя объекты Query. Объект



`CurrencyTableAdapter` содержит по умолчанию единственный элемент — строку с меткой `Fill,GetData ()`. Выберите ее и изучите ее свойства в окне **Properties**: вы заметите, что объект `Query` имеет свойства простого объекта `Command` из `ADO.NET`. Далее мы обсудим объекты `Query` подробнее.

Возвращаясь обратно к объекту `TableAdapter`, обратите внимание, что само свойство `Connection` имеет несколько дочерних свойств, которые могут быть сконфигурированы независимо. Вместо редактирования каждого из них по отдельности вы можете выбрать именованную строку подключения из выпадающего списка в столбце `Value` родительского свойства `Connection` либо ввести ее вручную в столбце `Value` дочернего свойства `Connection.ConnectionString`. Возможно, вы хотите узнать, откуда же берутся эти именованные подключения, и как поддерживать их список. Нам предстоит краткий экскурс в конструктор `Visual Studio 2008 Settings`.

## Управление строкой подключения

Чтобы сделать более простым визуальное конструирование типизированных объектов `DataSet`, а также `Web-привязку` данных и привязку данных в `Windows Forms`, `.NET` предоставляет раздел конфигурационного файла специально для строк подключения. Таким образом, вы можете ссылаться на строки подключения по имени и избегать всеми неодобряемого жесткого программирования строк подключения в коде приложения. `Visual Studio` обеспечивает весьма серьезную поддержку этой функциональной возможности стадии проектирования следующими способами.

- ❑ Для многих типов проектов `Visual Studio` специальный визуальный конструктор настроек позволяет создавать строки подключения в файле конфигурации приложения (без необходимости прямого редактирования самого файла) и указывать информацию строки подключения при помощи диалогового окна **Connection Properties**.
- ❑ Как вы только что видели, окно **Properties** (при использовании с объектами `TableAdapter` внутри визуального конструктора типизированного `DataSet`) позволяет выбрать любую именованную строку подключения (определенную конфигурационным файлом).
- ❑ Мастер `TableAdapter Configuration Wizard` (который мы будем изучать далее в этой же главе) позволяет сделать выбор любой именованной строки подключения, определенной конфигурационным файлом, и сохраняет вновь созданные строки подключения в конфигурационный файл (по вашему требованию).

- Вы видите полный список строк подключения сразу же, как только ставите точку после выражения `[ProjectName].Properties.Settings.Default`, где `[ProjectName]` — это название корневого пространства имен вашего проекта.

Для того чтобы все это проверить, откройте визуальный конструктор настроек. В проектах на языке C# вы можете добраться до этого конструктора при помощи разворачивания узла **Properties** в окне **Solution Explorer**, а затем двойного щелчка по его дочернему узлу **Settings.settings**. Вы можете также попасть в визуальный конструктор настроек через вкладку **Settings** визуального конструктора свойств (а для некоторых типов проектов вам придется использовать именно этот способ, чтобы создать для начала файл `Settings.settings`).

В конструкторе настроек вы должны увидеть все произведенные от типизированных объектов `DataSet` строки подключения. Изучив настройку этих строк подключения, вы без труда поймете, как создавать свои собственные. Если вы щелкнете в ячейке `value`, то вы сможете ввести строку подключения вручную. Можно также нажать на кнопку с многоточием справа от ячейки, чтобы открыть диалоговое окно свойств подключения и задать строку подключения в интерактивном режиме.

Закончив ввод, вы можете закрыть визуальный конструктор настроек и вернуться обратно в визуальный конструктор `DataSet`. Если вы теперь выберете объект `CurrencyTableAdapter` и в окне свойств щелкнете по стрелке выпадающего списка (рядом со свойством `Connection`), то увидите в выпавшем списке все строки подключения, которые имелись в визуальном конструкторе настроек (однако не меняйте установленное исходное значение этого свойства).

В этой главе вам встретятся и другие способы настройки строки подключения. Один из них — это мастер `TableAdapter Configuration Wizard`, который мы сейчас и изучим.

## Использование мастера `TableAdapter Configuration Wizard`

Итак, при помощи перетаскивания в окне **Server Explorer** мы поместили таблицу в типизированный объект `DataSet`. Другой (и, возможно, более гибкий способ) — использовать мастер `TableAdapter Configuration Wizard` для добавления улучшенной версии `Currency` в наш типизированный `DataSet`, который использует только хранимые процедуры для выборки, обновления,

вставки и удаления данных. Создайте показанные в листинге 10.1 хранимые процедуры, которые инкапсулируют динамически генерируемый код Transact-SQL (T-SQL), используемый нашим исходным CurrencyTableAdapter. Или скачайте решение для SQL Server Management Studio с названием Chapter10.ssmssln с Web-сайта этой книги, откройте его и выполните имеющийся в нем скрипт CreateCurrencySprocs.sql.

**Листинг 10.1. Создание хранимых процедур для извлечения и модификации данных при помощи TableAdapter**

```
USE AdventureWorks2008
GO
CREATE PROCEDURE dbo.uspSelectCurrency
AS
    SET NOCOUNT ON;
SELECT CurrencyCode, Name, ModifiedDate
FROM Sales.Currency
GO
CREATE PROCEDURE dbo.uspInsertCurrency
(
    @CurrencyCode nchar(3),
    @Name Name,
    @ModifiedDate datetime
)
AS
    SET NOCOUNT OFF;
INSERT INTO [Sales].[Currency] ([CurrencyCode], [Name], [ModifiedDate])
VALUES (@CurrencyCode, @Name, @ModifiedDate);
SELECT CurrencyCode, Name, ModifiedDate FROM Sales.Currency
WHERE (CurrencyCode = @CurrencyCode)
GO
CREATE PROCEDURE dbo.uspUpdateCurrency
(
    @CurrencyCode nchar(3),
    @Name Name,
    @ModifiedDate datetime,
    @Original_CurrencyCode nchar(3),
    @Original_Name Name,
    @Original_ModifiedDate datetime
)
```

AS

```
SET NOCOUNT OFF;

UPDATE [Sales].[Currency] SET [CurrencyCode] = @CurrencyCode, [Name] =
@Name, [ModifiedDate] = @ModifiedDate WHERE (([CurrencyCode] =
@Original_CurrencyCode) AND ([Name] = @Original_Name) AND ([ModifiedDate]
= @Original_ModifiedDate));

SELECT CurrencyCode, Name, ModifiedDate FROM Sales.Currency WHERE
(CurrencyCode = @CurrencyCode)
```

GO

```
CREATE PROCEDURE dbo.uspDeleteCurrency
(
```

```
    @Original_CurrencyCode nchar(3),
    @Original_Name Name,
    @Original_ModifiedDate datetime
```

```
)
```

AS

```
SET NOCOUNT OFF;

DELETE FROM [Sales].[Currency] WHERE (([CurrencyCode] =
@Original_CurrencyCode) AND ([Name] = @Original_Name) AND ([ModifiedDate]
= @Original_ModifiedDate))

GO
```

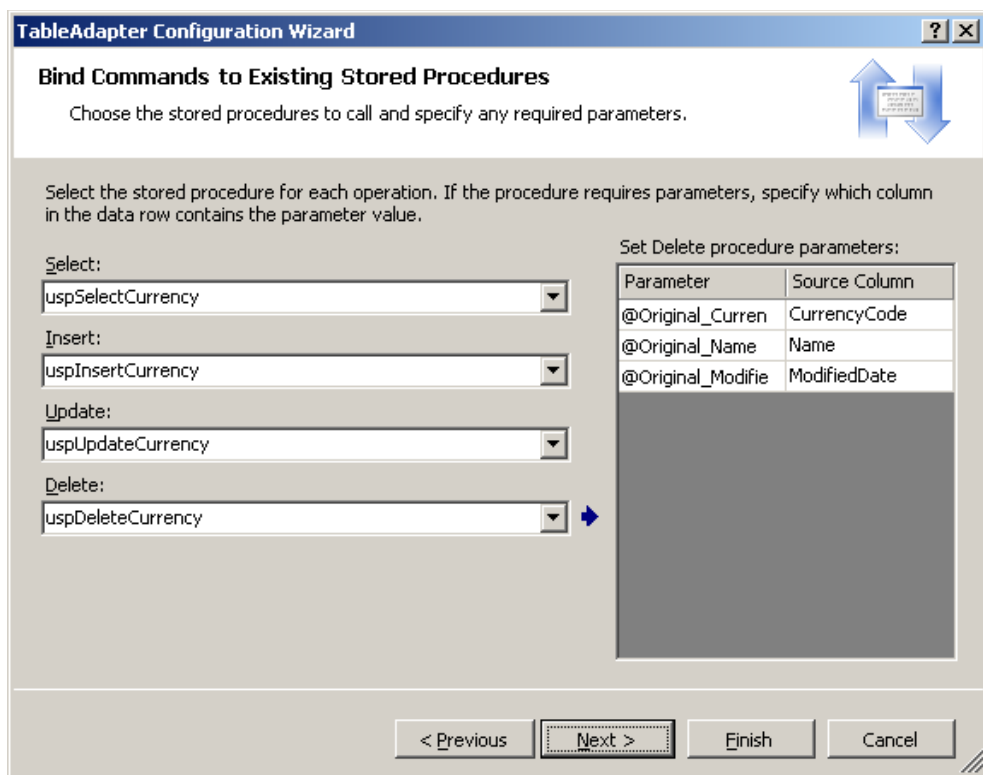
После создания хранимых процедур выполните следующие шаги:

1. Запустите мастер при помощи щелчка правой кнопки мыши в свободной области визуального конструктора DataSet и последующего выбора в контекстном меню пунктов **Add | TableAdapter**. В область визуального конструктора будет добавлена пустая пара DataTable/TableAdapter, и после короткой паузы появится мастер TableAdapter Configuration Wizard.
2. На первой странице мастера (страница **Choose Your Data Connection**) укажите подключение для той пары DataTable/TableAdapter, которую вы хотите добавить к DataSet. Вы сразу же заметите, что мастер позволяет выбрать любую строку подключения из конфигурационного файла, а также те подключения из окна **Server Explorer**, которые в конфигурационном файле не представлены. Щелкните по выпадающему списку, и если вы хотите узнать строку подключения, разверните узел **Connection String** в нижней половине страницы мастера. Выберите строку подключения, указывающую на базу данных AdventureWorks2008 (которую вы использовали для объекта CurrencyTableAdapter), а затем нажмите кнопку **Next**.

Если вы выбрали подключение, которое в конфигурационном файле отсутствует, то страница мастера **Save The Connection String To The Application Configuration File** даст вам возможность сохранения его в файл и позволит задать его название. Согласитесь с установками по умолчанию и нажмите кнопку **Next**.

3. На странице **Choose A Command Type** вы указываете, будут ли свойства `SelectCommand`, `InsertCommand`, `UpdateCommand` и `DeleteCommand` объекта `TableAdapter` основаны на динамических инструкциях SQL или на вызовах хранимых процедур. В последнем случае у вас есть выбор: либо использовать уже существующие хранимые процедуры, либо создать мастером новые процедуры. Вариант существующих хранимых процедур позволяет вам выбирать не только хранимые процедуры, но и возвращающие табличные значения функции (table-valued functions, TVF). Выберите вариант **Use Existing Stored Procedures** и нажмите кнопку **Next**.
4. На странице **Bind Commands To Existing Stored Procedure** (рис. 10.2) вы можете указать используемые для всех четырех команд `TableAdapter` хранимые процедуры. Выберите в выпадающих списках команд `Select`, `Insert`, `Update` и `Delete` соответственно объекты `uspSelectCurrency`, `uspInsertCurrency`, `uspUpdateCurrency` и `uspDeleteCurrency`. Для всех запросов, кроме `Select`, сетка справа от выпадающего списка позволит указать соответствие между столбцами `DataTable` (которые мы создаем, то есть возвращаем хранимой процедурой `Select`) и соответствующими параметрами настраиваемой хранимой процедуры. Согласитесь с назначениями по умолчанию и нажмите кнопку **Next**.
5. На странице **Choose Methods To Generate** просто примите названия по умолчанию (нажав кнопку **Next**).
6. На странице результатов **Wizard Results** мастера вы можете просмотреть выполненные мастером задачи, а затем нажать кнопку **Finish** для завершения работы мастера.

Теперь вы должны увидеть ваши `DataTable` и `TableAdapter` в области конструирования `DataSet`. Набор `DataTable` должен быть назван `uspSelectCurrency`, а набор `TableAdapter` — `uspSelectCurrencyTableAdapter`. Удалите исходные объекты `Currency` и `CurrencyTableAdapter` из `DataSet`, а затем переименуйте новые объекты в `Currency` и `CurrencyTableAdapter` соответственно. Внутри `TableAdapter` вы должны увидеть методы `Fill` и `GetData`. Если бы в мастере вы присвоили этим методам другие имена или сгенерировали бы только один из них, то в этом списке методов были бы отражены конкретные изменения.



**Рис. 10.2.** Страница **Bind Commands To Existing Stored Procedures** мастера TableAdapter Configuration Wizard

Перед тем как продолжить, перетащите таблицы ProductCategory (Production), ProductSubcategory (Production) и Product (Production), а также хранимую процедуру uspGetBillofMaterials из окна **Server Explorer** в типизированный DataSet. Ваш DataSet должен теперь выглядеть так, как показано на рис.10.3. Обратите внимание, что связи между таблицами ProductCategory и ProductSubcategory, ProductSubcategory и Product создаются автоматически. Так происходит потому, что ограничения внешнего ключа между этими таблицами уже имеются в базе данных.

Помните, что вы в любое время можете модифицировать существующий объект TableAdapter при помощи мастера TableAdapter Configuration Wizard (щелчком правой кнопки мыши по объекту и выбором опции **Configure** в контекстном меню).

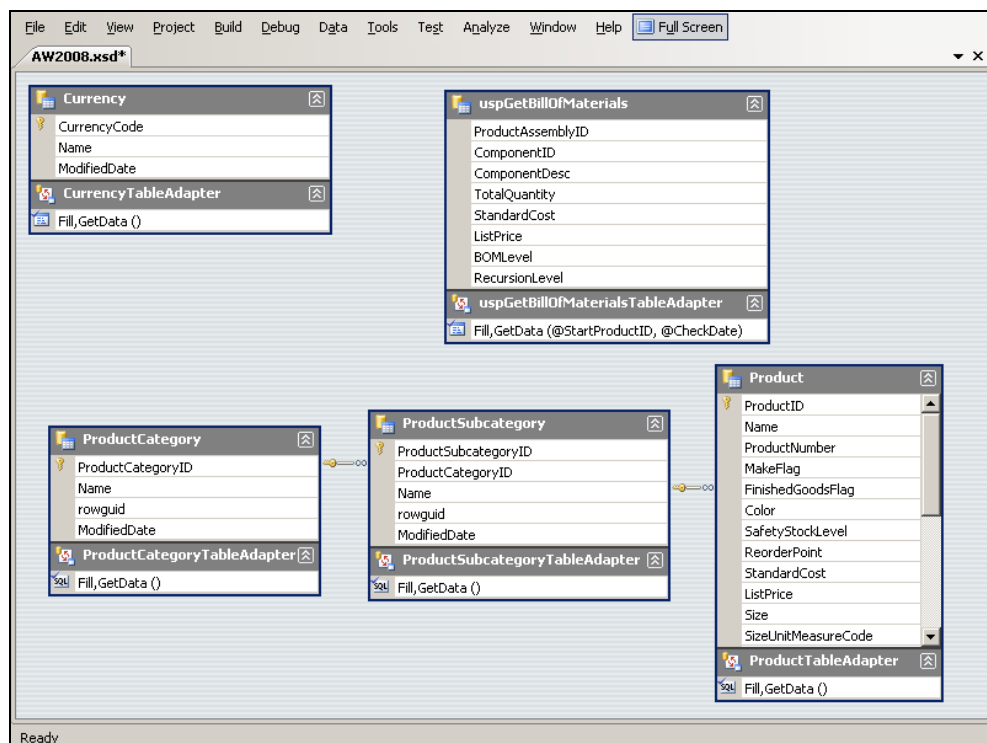


Рис. 10.3. Визуальный конструктор типизированного DataSet с различными определенными параметрами DataTable/TableAdapter

## Дополнительно о запросах и параметрах

Обратите внимание, что список методов для объекта Query в `uspGetBillOfMaterialsTableAdapter` содержит параметры с названиями `@StartProductID` и `@CheckDate`. Они появляются потому, что эти параметры имеет хранящая процедура `uspGetBillOfMaterials`. Хранимые процедуры и параметры запросов SQL предоставляются как коллекция параметров ADO.NET для объектов Command, управляемых вашими объектами TableAdapter, причем они предоставляются как параметры уровня кода в методах `Fill` и `GetData` объекта TableAdapter.

Параметры вашего запроса (так же, как и параметр `@RETURN_VALUE`) представлены в свойстве коллекции `Parameters` объекта Query и свойстве `SelectCommand` объекта TableAdapter. Убедитесь в этом сами, щелкнув по

ячейке значения окна **Properties** для любого свойства коллекции `Parameters`, а затем нажав кнопку с многоточием. Мастер сделает некоторые предположения о типе данных, масштабе, размере и направлении параметров, а также о существовании возвращаемого значения. Вы можете модифицировать эти предположения посредством изменения значений свойств отдельных параметров или путем добавления/удаления параметров из свойств коллекции.

Вернемся к объекту `CurrencyTableAdapter`. Выберем свойство `UpdateCommand.Parameters`, а затем щелкнем по кнопке с многоточием в ячейке его значения. В диалоговом окне **Parameters Collection Editor** последние три параметра в списке **Members** начинаются с префикса `@Original_`, поскольку в команде `UPDATE` этой хранимой процедуры указаны значения всех столбцов выражения `WHERE` (а не только первичный ключ). Щелкните по любому из этих параметров в списке **Members**, а затем посмотрите на свойство `SourceVersion` в сетке свойств справа. Вы заметите, что его значение установлено в `Original`, а для всех тех параметров, которые не начинаются с префикса `@Original_`, это свойство установлено в `Current`. Мы обращаем ваше внимание на это важное обстоятельство для того, чтобы вы знали, как искать эквивалентные опции конфигурации (когда мы будем изучать некоторые другие модели доступа к данным).

### ПРИМЕЧАНИЕ

Мы не являемся приверженцами таких хранимых процедур обновления/удаления, которые параметризуют каждое значение исходного столбца. Обычно все, что нужно — это значение первичного ключа (и, возможно, также исходное значение `rowversion` для параллельного выполнения), и в этом случае обычно нет разницы между исходным и текущим значениями. Предоставленные здесь хранимые процедуры для таблицы `Currency` были фактически сгенерированы мастером `TableAdapter Configuration Wizard` (а затем переформатированы и несколько подчищены), именно поэтому они так структурированы. Тем не менее, работа с исходными значениями параметров научит вас обращаться с ними. И не только в визуальном конструкторе типизированного `DataSet`, но и в визуальных конструкторах `LINQ` для `SQL` и `Entity Framework` (которые мы скоро рассмотрим).

Вы можете без труда добавить запросы в ваши существующие объекты `TableAdapter`. По существу при этом в ваш класс `TableAdapter` добавляются методы (которые можно параметризовать); они в свою очередь работают как оболочки для команд `SQL` или хранимых процедур (которые также можно параметризовать). Можно модифицировать существующий объект `Query` при помощи мастера `TableAdapter Configuration Wizard` (щелчком правой кнопкой мыши по нему и выбором в контекстном меню опции



**Configure).** Можно добавить объекты Query непосредственно в наборе DataSet (а не в объекте TableAdapter), для этого нужно щелкнуть правой кнопкой мыши по области визуального конструктора DataSet, а затем выбрать в контекстном меню пункты **Add | Query**.

## Методы DBDirect и подключенное использование типизированных объектов DataSet

Объекты TableAdapter действуют, как строго типизированные объекты DataAdapter, и идеальны для заполнения отключенных источников данных. Но объекты TableAdapter имеют специальное свойство `GenerateDBDirectMethods`, которое позволяет им функционировать и как подключенным источникам данных. Это булево свойство (значение по умолчанию которого истинно) обеспечивает автоматическое генерирование трех специальных методов (называемых методами DBDirect) класса TableAdapter, которые обрабатывают немедленные обновления, вставки и удаления по физической базе данных, лежащей в основе объекта DataTable.

Используя эти три метода (которые автоматически именуются Update, Insert и Delete), а также метод GetData, разработчики могут использовать схемы и запросы типизированных объектов DataSet без фактического хранения каких-либо данных в объектах DataTable. Как ни странно, но это позволяет вам использовать типизированные объекты DataSet так, как будто они вовсе не являются объектами DataSet. Такое использование сродни тому, как настраиваются четыре отдельных объекта SqlCommand в ADO.NET, объект SqlConnection, а также методы ExecuteReader и ExecuteNonQuery объектов SqlCommand для извлечения и модификации данных (соответственно).

Содержащиеся в объектах DataTable схемы облегчают привязку данных на этапе проектирования (даже для подключенных методов). Вы будете использовать это в следующей главе, где мы описываем привязку данных ASP.NET.

Пока же закройте визуальный конструктор DataSet, а затем нажмите кнопку **Yes** в ответ на вопрос, хотите ли вы сохранить ваши изменения.

## ADO.NET: работаем в коде

Мы потратили много времени на рассмотрение возможностей визуального конструирования объектов DataSet. Эта информация поможет вам стать продуктивным разработчиком для SQL Server 2008, но многие функциональные

возможности ADO.NET требуют написания определенного количества кода без использования визуальных конструкторов Visual Studio. Вы можете использовать код для манипулирования вашими типизированными объектами DataSet и для их заполнения, но пока давайте займемся некоторыми методами чистой разработки кода, которые совсем не используют типизированных объектов DataSet. (К функциональным возможностям визуальных конструкторов мы вернемся в следующей главе).

Мы рассмотрели визуальный конструктор типизированного объекта DataSet потому, что он является важным инструментом для большинства .NET-разработчиков (несмотря на появление большого количества новых инструментов), и хорошее его понимание дает определенный контекст, в котором легче обсуждать другие инструменты. Но пока давайте сделаем обзор основных методов кодирования, используемых в ADO.NET для подключенного доступа к данным и для отключенного доступа к данным (с использованием как типизированных, так и нетипизированных объектов DataSet).

## Простейший запрос

Сначала мы рассмотрим простой запрос по таблице AdventureWorks2008Sales.Currency. Для реализации этого запроса добавьте в проект новую форму и назовите ее frmMain (или просто переименуйте форму Form1, которая была добавлена по умолчанию при создании проекта). Затем перетащите элемент управления DataGridView из вкладки **Data** окна **Toolbox** на форму и назовите этот элемент управления dgvResults. Перетащите элемент управления Button из вкладки **Common Controls** окна **Toolbox** на форму, назовите его btnLoaduntypedDS и установите его свойство Text в значение Untyped DataSet. При необходимости измените размер кнопки (чтобы текст был виден полностью).

Для SQL Server требуется применение поставщика System.Data.SqlClient из ADO.NET, поэтому в вашем коде необходимо будет импортировать это пространство имен. Поэтому в классе выделенного кода формы добавьте следующую строку (сразу же после уже имеющихся строк кода, начинающихся с инструкции using):

```
using System.Data.SqlClient;
```

## Кодирование с нетипизированным объектом DataSet

После того как вы позаботились о пространствах имен, вы можете просто использовать объекты SqlConnection, SqlCommand и SqlDataAdapter, а также

объект `DataSet` (или `DataTable`) и привязанный к ним элемент управления для запросов к вашей базе данных и отображения результирующего набора.

Можно избежать создания и использования объектов `SqlConnection` и `SqlCommand`, поскольку `SqlDataAdapter` имеет такую перегрузку конструктора, которая неявным образом создает для вас эти объекты. Тем не менее, мы считаем важным показать вам, как использовать эти объекты явно. Код листинга 10.2, который содержит реализацию обработчика события `Click` для элемента управления `Button`, выполняет простой запрос по таблице `Sales.Currency`, заполняет результатами объект `DataTable` и привязывает `DataTable` к элементу управления `DataGridView`.

**Листинг 10.2. Выполнение инструкции `SELECT` и привязка результатов к элементу управления `DataGridView`**

```
private string ConnString =
    Properties.Settings.Default.AdventureWorks2008ConnectionString;
private void btnLoadUntypedDS_Click(object sender, EventArgs e)
{
    SqlConnection cn = new SqlConnection(ConnString);
    SqlCommand cm = new SqlCommand
        ("SELECT CurrencyCode, Name, ModifiedDate FROM Sales.Currency", cn);
    SqlDataAdapter daUntyped = new SqlDataAdapter(cm);
    DataTable dt = new DataTable();
    daUntyped.Fill(dt);
    dgvResults.DataSource = dt;
}
```

Обратите внимание, что добавленная ранее в файл настроек строка подключения (которая в предыдущем коде называется `AdventureWorks2008ConnectionString`) сохраняется в переменной уровня класса с названием `ConnString`, которая передается конструктору объекта `SqlConnection`, и можно сослаться на строку подключения по названию (при помощи свойства `Default` объекта `Properties.Settings`). Мы могли бы также передать и строковый литерал, но подход с `Properties.Settings` более разумен, поскольку изменение строки подключения в файле настроек распространится и в этот код (без всяких дополнительных усилий с вашей стороны).

Передача строкового литерала или выражения в конструктор объекта `SqlConnection` имеет тот же эффект, что и настройка свойства `ConnectionString` объекта при создании его экземпляра. В данном листинге

используются подобные "сокращения конструкторов". Свойства `CommandText` и `Connection` объекта `SqlCommand` и свойство `SelectCommand` объекта `SqlDataAdapter` настраиваются посредством передачи соответствующей строки и объектов в конструкторы объектов `cm` и `daUntyped`. Экземпляр объекта `DataTable` создается при помощи конструктора по умолчанию и не требует специфической инициализации.

После создания экземпляров объектов мы выполняем запрос и заполняем таблицу `DataTable` его результирующим набором (вызывая метод `Fill` объекта `SqlDataAdapter` и передавая его нашему объекту `DataTable`). Для отображения результатов мы настраиваем свойство `DataSource` нашего элемента управления `DataGridView` в значение `dt` (наш объект `DataTable`) — и все готово. Отредактируйте файл `program.cs` так, чтобы сделать форму `frmMain` вашей стартовой формой (если только вы не переименовали форму `Form1`, в этом случае `program.cs` был обновлен автоматически), запустите приложение и нажмите кнопку **Untyped DataSet** для проверки, что элемент управления `DataGridView` заполнился правильно.

## Кодирование с типизированным объектом *DataSet*

Теперь добавьте другую кнопку, назовите ее `btnTypedDS` и настройте ее свойство `Text` в значение `Typed DataSet`. В листинге 10.3 показан код для события `Click` кнопки, которое заполняет таблицу `Currency` в нашем типизированном наборе `DataSet` и привязывает ее к объекту `dgvResults`.

### Листинг 10.3. Заполнение типизированного набора `DataSet` и привязка его к одному из его объектов `DataTable`

```
private void btnTypedDS_Click(object sender, EventArgs e)
{
    AW2008TableAdapters.CurrencyTableAdapter da =
        new AW2008TableAdapters.CurrencyTableAdapter();
    AW2008 ds = new AW2008();
    da.Fill(ds.Currency);
    dgvResults.DataSource = ds.Currency;
}
```

Обратите внимание, что в этой версии кода нам не нужно явно объявлять объекты `SqlConnection` или `SqlCommand` (поскольку наш объект `TableAdapter` делает это через свои собственные свойства `SelectCommand` и

Connection), мы не объявляем также объект DataTable (поскольку наш типизированный объект DataSet создает этот объект при помощи своего свойства Currency). Мы уменьшили размер кода с семи строк до четырех, получили поддержку технологии IntelliSense для названий нашего типизированного объекта DataSet, TableAdapter и DataTable, а также избежали необходимости писать в нашем коде запрос SQL. Запустите приложение, а затем нажмите кнопку **Typed DataSet** для тестирования кода.

## Кодирование для подключенного объекта *DataReader*

Вы предпочитаете для подключенного доступа к данным использовать более простые объекты ADO.NET? Добавьте дополнительную кнопку, назовите ее btnConnected и настройте свойство ее текста в значение Connected Reader. В листинге 10.4 показан нужный вам код обработчика события Click.

### Листинг 10.4. Заполнение элемента управления DataGridView вручную из объекта DataReader

```
private void btnConnected_Click(object sender, EventArgs e)
{
    dgvResults.DataSource = null;
    dgvResults.Columns.Add("CurrencyCode", "CurrencyCode");
    dgvResults.Columns.Add("Name", "Name");
    dgvResults.Columns.Add("ModifiedDate", "ModifiedDate");
    SqlConnection cnSqlCli = new SqlConnection(ConnString);
    SqlCommand cmSqlCli = new SqlCommand
        ("SELECT CurrencyCode, Name, ModifiedDate FROM Sales.Currency",
        cnSqlCli);
    SqlDataReader drSqlCli;
    cnSqlCli.Open();
    drSqlCli = cmSqlCli.ExecuteReader(CommandBehavior.CloseConnection);
    while (drSqlCli.Read())
    {
        dgvResults.Rows.Add(drSqlCli["CurrencyCode"], drSqlCli["Name"],
        drSqlCli["ModifiedDate"]);
    }
    drSqlCli.Close();
}
```

В этом коде мы используем простые объекты SqlConnection, SqlCommand и SqlDataReader. Мы подключаемся к базе данных, извлекаем наши данные,

сами загружаем их в сетку, а затем отключаемся — и все. Поскольку мы не привязываем объект `SqlDataReader` к элементу управления `DataGridView` (в Windows Forms мы этого сделать не можем), нам нужно самим добавить столбцы в сетку (а затем загрузить данные каждой строки). Когда мы закончим, мы сможем закрыть `DataReader`, и поскольку его экземпляр был создан при помощи перечислимой константы `CommandBehavior.CloseConnection`, то это закрывает также и наше подключение.

Хотя этот код и длиннее, чем предыдущие примеры (и полагается на дополнительную строку кода, которая объявляет и присваивает значение переменной `ConnString` нашего первого примера), он явно более эффективен (особенно если мы никак не собираемся модифицировать наши данные). Обратите внимание, что перед добавлением столбцов в сетку мы очищаем любые ранее существовавшие привязки данных (путем настройки значения свойства `DataSource` в значение `Null`). Аналогичным же образом, если вы хотите быть в состоянии выполнить любой из наших примеров `DataSet` после выполнения кода подключенного свойства `DataReader`, необходимо вставить в остальные обработчики ошибок следующую строку кода (сразу же перед строкой, которая настраивает свойство `DataSource` сетки):

```
dgvResults.Columns.Clear();
```

Пример приложения (имеющийся в этой главе) уже содержит этот код.

## Добавление логики проверки для типизированного объекта *DataSet*

Давайте рассмотрим еще один метод кодирования, который демонстрирует эффективность типизированных объектов `DataSet`. Мы добавим логику проверки (дополнив нашим собственным кодом сгенерированный визуальным конструктором типизированного объекта `DataSet` код). Для этого откройте типизированный объект `DataSet` в его визуальном конструкторе и дважды щелкните по объекту `Currency`. После этого откроется файл выделенного кода типизированного объекта `DataSet` (а не сгенерированный визуальным конструктором код), и ваш курсор окажется внутри кода частичного класса для класса `CurrencyDataTable`. Внутри этого класса введите код листинга 10.5.

### Листинг 10.5. Встраивание логики проверки в типизированный объект `DataSet` (при помощи частичного класса)

```
protected override void  
    OnColumnChanged(System.Data.DataColumnChangeEventArgs e)
```

```
{  
    CurrencyRow row = (CurrencyRow)e.Row;  
    if (row.Name == "Reserved")  
    {  
        row.SetColumnError(e.Column, "'Reserved' is an invalid name");  
        row.RowError = "Errors have occurred";  
    }  
    else  
    {  
        row.ClearErrors();  
    }  
}
```

Теперь перезапустите приложение, нажмите кнопку **Typed DataSet** и введите слово `Reserved` (именно так, с заглавной буквы) в последнюю строку в столбце `Name`. Нажмите клавишу `<Tab>` для выхода из этой ячейки (но останьтесь в этой строке) и смотрите за тем, что происходит. Вы должны увидеть появление красного значка с восклицательным знаком и в ячейке `Name`, и рядом с заголовком строки.

Наведите курсор мыши на восклицательный знак в ячейке `Name`, и вы увидите (внутри всплывающей подсказки) сообщение об ошибке столбца. (Наведите курсор на второй восклицательный знак, и вы увидите сообщение об ошибке строки). Вернитесь в ячейку `Name` и введите другое название; как только вы введете первый же символ, красный восклицательный знак исчезнет из ячейки. Нажмите клавишу `<Tab>`, чтобы выйти из ячейки, и вы увидите, что красный восклицательный знак исчез и из заголовка строки. Введите уникальный код в столбце `CurrencyCode` и правильную дату в столбце `ModifiedDate`, после этого вы сможете перейти к другим строкам сетки. (Если вы этого не сделаете, то появится очень большое окно сообщения об ошибке).

Если вы нажмете на форме любую другую кнопку и попытаете выполнить те же самые шаги по вводу данных, то вы увидите, что такой проверки не происходит. Проверочное поведение встроено в наш типизированный объект `DataSet` и будет автоматически предоставляться любым элементом управления, к которому он привязан; нетипизированные объекты `DataSet` не имеют такого преимущества.

Как вы видели, типизированные объекты `DataSet` позволяют нам моделировать то подмножество нашей базы данных, с которым мы хотим работать, причем они хорошо совместимы с хранимыми процедурами. Они дают нам

поддержку технологии IntelliSense в коде, позволяют сделать этот код кратким и даже вставлять логику проверки и бизнес-правила в сами типизированные классы `DataTable`. Поэтому, несмотря на то, что появилась масса новых моделей доступа к данным (которые также заслуживают нашего внимания), необходимо помнить о том, чем полезна система ADO.NET (быстрые и эффективные методы чистого кодирования, а также богатая поддержка инструментами Visual Studio).

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Примеры кодов содержат много других примеров того, что вы можете делать при помощи ADO.NET непосредственно в коде, в том числе и программирование с использованием введенных в SQL Server 2005 функциональных возможностей. Сюда входят уведомления о запросах, асинхронные запросы, операции массового копирования `SqlClient`, обработка XML-данных, извлечение определяемых пользователем типов Common Language Runtime (CLR) как собственных объектов CLR. Все эти примеры кодов содержатся в форме `frm2005Demos`, которую можно открыть щелчком по кнопке **2005 Demos** главной формы `frmMain` проекта Chapter 10. Обратите внимание, что для правильной работы демонстрационного кода `2005 Demos` необходимо выполнить скрипты `CreateTblBulkData.sql`, `CreateUspGetNames.sql` и `EnableServiceBroker.sql` в решении `Chapter 10.ssmssln` для Management Studio. Содержимое этих примеров кода приводилось в *главе 8* предыдущего издания данной книги. Для вашего удобства электронную копию этой главы можно скачать с Web-сайта данной книги.

Давайте рассмотрим другие модели доступа к данным, но сначала изучим использование LINQ с `DataSet`.

## **LINQ: новый синтаксический подход для доступа к данным**

Давайте рассмотрим ту проблему, которая наверняка осложнила понимание новых функциональных возможностей LINQ в .NET для тех, кто "не в курсе" — это путаница в брендах и названиях. Так происходит потому, что LINQ (который на самом деле является просто новой языковой возможностью в C# и Visual Basic .NET) обычно используется в моделях LINQ для SQL и в моделях данных Entity Framework (запросы к которой делаются при помощи LINQ для Entities). Поэтому у многих людей LINQ сначала ассоциируется с одним из двух этих инструментов Object Relational Mapping (ORM).



## LINQ для DataSet

Еще раз повторимся, что LINQ является просто языковой возможностью, и его не обязательно использовать с моделями из упомянутых инструментов ORM. Например, LINQ можно использовать для запрашивания XML-файлов (с использованием LINQ для XML), а также и для запросов DataSet (при помощи LINQ для DataSet). Скоро мы рассмотрим LINQ для SQL и Entity Framework, но поскольку мы уже потратили некоторое время на изучение ADO.NET, то для начала мы познакомим вас с механизмом работы LINQ на примере LINQ для объекта DataSet.

В отличие от ADO.NET, где требуется предоставить строку (либо запрос T-SQL, либо имя хранимой процедуры) для свойства CommandText объекта Command, запросы LINQ используют ключевые слова языка и, следовательно, могут быть проверены на правильность компилятором (а не сервером SQL Server). Добавьте дополнительную кнопку, назовите ее btnLinqToDataSet и установите ее текстовое свойство в значение Linq To DataSet. В листинге 10.6 показан код обработчика ошибок Click (который вам понадобится). Этот код показывает, как использовать LINQ для объекта DataSet при выполнении того же самого запроса выборки, который мы уже иллюстрировали в предыдущих листингах.

### Листинг 10.6. Заполнение типизированного объекта DataSet и привязка к запросу LINQ для DataSet

```
private void btnLinqToDataSet_Click(object sender, EventArgs e)
{
    AW2008TableAdapters.CurrencyTableAdapter daL2DS = new
        AW2008TableAdapters.CurrencyTableAdapter();
    AW2008 dsL2DS = new AW2008();
    daL2DS.Fill(dsL2DS.Currency);
    dgvResults.DataSource =
        (from currency in dsL2DS.Currency select currency).AsDataView();
}
```

Сравните это с кодом, показанным ранее в листинге 10.3. Обратите внимание, что код практически совпадает (за исключением последней строки). Вместо привязки ко всей коллекции DataTable мы выполняем по ней запрос:

```
from currency in dsL2DS.Currency select currency
```

Результат этого запроса (коллекция типа EnumerableRowCollection<AW2008.CurrencyRow>) не может быть напрямую привязан к элементу

управления `DataGridView`, поэтому мы преобразовываем его в объект `LinqDataView` при помощи метода `AsDataView`.

## Синтаксис LINQ

Что же делает этот запрос на самом деле? На первый взгляд он может показаться избыточным и путаным. Для того чтобы понять, как он работает, полезно сравнить его с запросом SQL, а затем заменить код C#. С точки зрения SQL, этот запрос очень похож на следующий:

```
SELECT * FROM Currency
```

И поскольку мы знаем, что `SelectCommand` объекта `DataTable` фактически выполняет по базе данных этот запрос, то в этом есть определенный смысл. Но такая точка зрения маскирует реальность, а понимание этой реальности помогает расшифровать преимущество такого кода. Реальность состоит в том, что он эквивалентен следующему (гораздо более длинному) коду C#:

```
List<AW2008.CurrencyRow> myCollection = new List<AW2008.CurrencyRow>();  
foreach(AW2008.CurrencyRow currency in dsL2DS.Currency)  
{  
    myCollection.Add(currency);  
}
```

### ПРИМЕЧАНИЕ

Обобщенный объект `List` (содержащий типизированные объекты `DataRow`) приводит к тому, что определенные свойства `DataRow` (не являющиеся значениями столбцов) добавляются как элементы. Эти дополнительные элементы могут просто подавляться вашим пользовательским интерфейсом, либо запрос LINQ может быть изменен так, чтобы добавлять только настоящие свойства столбцов данных к каждому элементу списка. Для простоты примеры кодов этой главы не учитывают данную аномалию. Помните, что метод `AsView` полностью лишен этой проблемы.

Здесь переменная `myCollection` может быть привязана непосредственно к элементу управления `DataGridView` (поскольку обобщенные объекты `List` фактически являются привязываемыми). Однако хотя обобщенный объект `List` может сначала показаться предпочтительнее сгенерированного запросом LINQ объекта `EnumerableRowCollection`, и в итоге оказывается, что последний имеет массу исключительно полезных методов (таких как `Min`, `Max` и `SelectMany`), которые позволяют выполнять манипуляции с данными даже без использования синтаксиса LINQ.

Все это может по-прежнему казаться избыточным кодом (поскольку сам объект `Currency` уже содержит содержимое, сгенерированное в переменную `myCollection`). Фактически последняя строка кода в `btnLinqToDataSet_Click` может быть заменена следующей (более простой) версией, которая просто привязывает к объекту `Currency`, как мы делали это в нашем примере типизированного объекта `DataSet`:

```
dgvResults.DataSource = dsL2DS.Currency;
```

Однако мы хотели познакомить вас с синтаксисом полного запроса LINQ. Для того чтобы понять ценность этого синтаксиса, рассмотрим следующий вариант запроса LINQ:

```
from currency in dsL2DS.Currency where  
    !currency.CurrencyCode.StartsWith("A") select currency
```

Это выражение после преобразования в объект `List` и присвоения свойству `DataSource` элемента управления `DataGridView` становится тем, что нужно было бы записать следующим образом:

```
List<AW2008.CurrencyRow> myCollection = new List<AW2008.CurrencyRow>();  
foreach (AW2008.CurrencyRow currency in dsL2DS.Currency)  
{  
    if (!currency.CurrencyCode.StartsWith("A"))  
    {  
        myCollection.Add(currency);  
    }  
}  
dgvResults.DataSource = myCollection;
```

Даже если не учитывать необходимость подсчитывать скобки, LINQ дает в этом примере 75-процентное сокращение количества строк кода! Более того, он предоставляет похожий на SQL синтаксис (хотя выражение `select` и находится в конце запроса) для запросов по коллекциям (ориентированным на наборы образом), который избавляет от необходимости применять процедурную логику. Например, LINQ поддерживает также и выражение `order by`: представьте себе, что вам пришлось бы реализовывать это при помощи процедурного кода! Даже если бы вы написали хорошую функцию сортировки, то необходимость ее явного вызова (в качестве отдельного шага) была бы менее удобна и сделала бы код менее читабельным.

Предыдущее описание LINQ годится в качестве рабочего определения: LINQ является расширением синтаксиса языков `C#` и `Visual Basic .NET`, которое позволяет делать запросы на основе наборов и генерировать перечислимые наборы объектов. Он является "подключаемым" средством в том смысле, что

можно создавать поставщиков для выполнения запросов LINQ по конкретным перечислимым типам. LINQ для DataSet делает это для объектов System.Data objects; LINQ для XML делает это для объектов System.XML; LINQ для Objects делает это для обобщенных объектов C#; LINQ для SQL и LINQ для Entities делают это соответственно для моделей данных LINQ SQL и Entity Framework. Два последних расширения мы и рассмотрим далее.

## **LiNQ для SQL и ADO.NET Entity Framework: ORM приходит в .NET**

К запросам LINQ мы через некоторое время вернемся, но сейчас необходимо сосредоточиться на построении моделей. С этой точки зрения, LINQ для SQL и Entity Framework (которые мы будем сокращать как L2S и EF до конца данной главы) и соответствующие визуальные конструкторы необходимо сравнить с типизированным DataSet и визуальным конструктором типизированного DataSet.

Наш типизированный DataSet содержит модель подмножества базы данных и приводил к генерированию строго типизированных классов (для представления объектов DataSet, DataTable и TableAdapter). Модели L2S и EF служат аналогичной цели и дают аналогичные результаты. Типизированный DataSet можно настроить для хранимых процедур и при помощи создания частичного класса встроить в него логику проверки данных. И знаете, что? Модели L2S и EF делают то же самое (более-менее). Типизированные объекты DataSet поддерживают связи и отношение "один-ко-многим" между объектами DataTable. Модели L2S и EF делают то же самое (при помощи своих классов коллекций).

Объекты баз данных можно перетаскивать из окна **Server Explorer** среды Visual Studio в область визуального конструктора DataSet либо создавать вручную. Визуальный конструктор L2S также поддерживает перетаскивание из окна **Server Explorer**, визуальный конструктор EF поддерживает добавление объектов баз данных при помощи мастера. Оба визуальных конструктора (для моделей L2S и EF) поддерживают создание классов вручную.

Визуальный конструктор типизированного DataSet поддерживает SQL Server, есть также и поддержка других баз данных (в том числе Oracle), которая обычно осуществляется инструментами поставщика базы данных. Модель L2S специфична для сервера SQL Server, а EF является моделью на основе поставщиков (что делает возможной поддержку других баз данных). В состоянии поставки EF поддерживает SQL Server; поддержка баз данных

Oracle и IBM (DB2, Informix и UniVerse/UniData) на момент печати данной книги находилась в состоянии разработки.

## Почему нельзя ограничиться технологией ADO.NET

При всей этой схожести между моделями и с учетом того, что LINQ можно использовать для объектов ADO.NET, зачем осложнять себе жизнь новыми технологиями? Обоснованием для этого служат явные функциональные возможности ORM, предоставляемые L2S и EF. В то время как типизированный DataSet генерирует классы в основном для оказания содействия в строгом типизировании (и для получаемой вследствие этого типизирования поддержки IntelliSense и проверки ошибок на этапе компилирования), сами классы типизированных DataSet не предназначены для использования в качестве настоящих бизнес-сущностей.

Классы в моделях L2S и EF предназначены для того, чтобы использоваться именно как классы. Точно так же, как вы можете создавать свои собственные классы бизнес-сущностей (в качестве уровня инкапсуляции над вашей базой данных), чтобы перестать думать о физической структуре таблиц вашей базы данных и хранимых процедурах, так и классы L2S и EF предназначены для использования в качестве подобной абстракции.

Визуальный конструктор типизированного DataSet не позволяет создавать производные классы, то есть встроенной поддержки наследования нет. Но L2S позволяет делать простое наследование, а модели EF позволяют делать очень серьезное наследование.

Соотношение объектов DataTable и физических таблиц баз данных по сути — "один-к-одному" (кстати, использование хранимых процедур и редактирование различных свойств Command объекта TableAdapter позволяет это обойти). Хотя модель L2S также имеет соответствие "один-к-одному" с объектами баз данных, наследование позволяет смягчить это ограничение. А модели EF легко позволяют одной сущности (классу) соответствовать множеству объектов в базе данных.

Хотя технология ADO.NET поддерживает LINQ как альтернативный синтаксис, в основном она продолжает использоваться с SQL. В то же время L2S и EF пригодны к более современным сценариям реализации.

Вы по-прежнему не уверены? Вы и не должны быть уверены. На самом деле не уверены и мы (по крайней мере, не полностью). Факты таковы, что типи-

зированный `DataSet` обеспечивает удивительно обширную модель абстракции, предоставляя в то же время прямой контроль над вырабатываемым кодом SQL при выполнении операций CRUD (`create`, `retrieve`, `update` и `delete`) и первоклассную поддержку привязки данных. Если вы захотите использовать синтаксис LINQ, то такая возможность у вас имеется, и в то же время вы можете использовать нетипизированные объекты ADO.NET (включая объекты `Command` и `DataReader`, которые имеют низкие издержки) для выполнения любых операций в базе данных. И если вы достаточно уверенно чувствуете себя в T-SQL, реляционной теории и т. д., то вы, вероятно, также уверенно чувствуете себя при переходах от этой парадигмы программирования к объектно-ориентированному подходу и обратно.

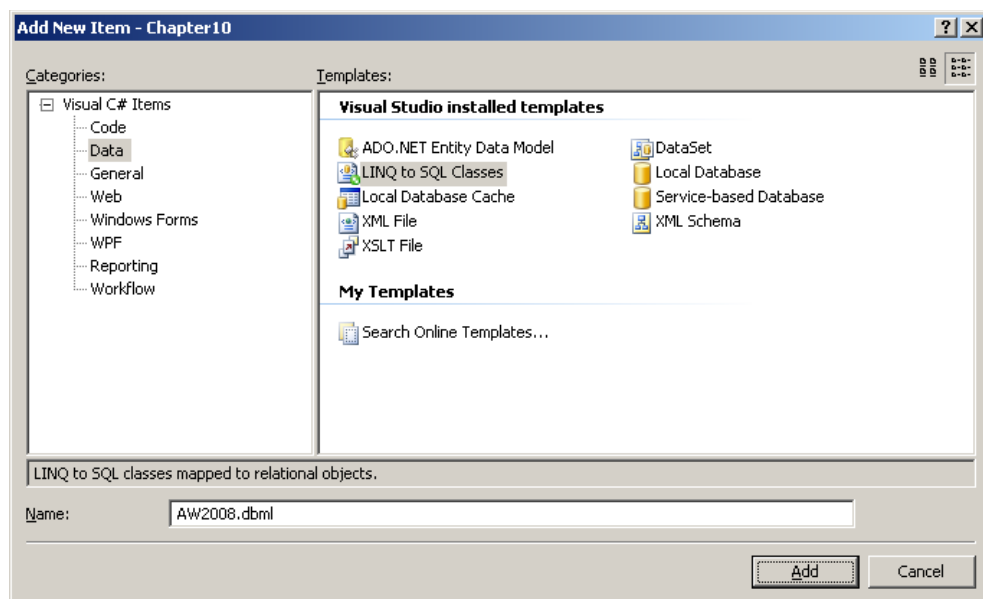
Однако реальность такова, что подход ORM имеет своих сторонников, и тот факт, что компания Microsoft принимает его, означает наличие прямой его поддержки в .NET Framework и смягчает разноречие, когда поддержка какого-то популярного подхода обеспечивается только продуктами сторонних фирм. К тому же, похоже на то, что компании Microsoft подход ORM понравился, и она создает новые технологии, такие как службы ADO.NET Data Services (которые мы изучим далее в этой же главе) поверх EF и ASP.NET Dynamic Data (обсудим в следующей главе) поверх L2S и EF.

Учитывая все это, нам следует кратко рассмотреть построение моделей L2S и EF и использование их в коде. Мы сделаем это прямо сейчас путем сборки моделей, которые структурно отражают созданный нами ранее типизированный `DataSet`. В конце этой главы мы рассмотрим вопрос о том, какую технологию лучше использовать (L2S, EF или другой вариант доступа к данным).

## Создаем модель L2S

С учетом всего, что вы уже узнали о визуальном конструкторе типизированного `DataSet` и нашего концептуального введения в модели L2S и EF, мы сможем очень быстро описать создание модели L2S.

Начнем с использования (для добавления нового элемента в ваш .NET-проект) одного из пунктов меню, панели инструментов или контекстного меню. Когда появится диалоговое окно **Add New Item**, выберите в списке **Templates** (на правой стороне диалогового окна) значение **LINQ to SQL Classes**. Если вы сначала выберете **Data** в окне **Categories** древовидного представления слева, то количество вариантов в списке **Templates** станет гораздо меньше (что позволит вам быстро найти шаблон L2S). Сделанный вами выбор показан на рис. 10.4.



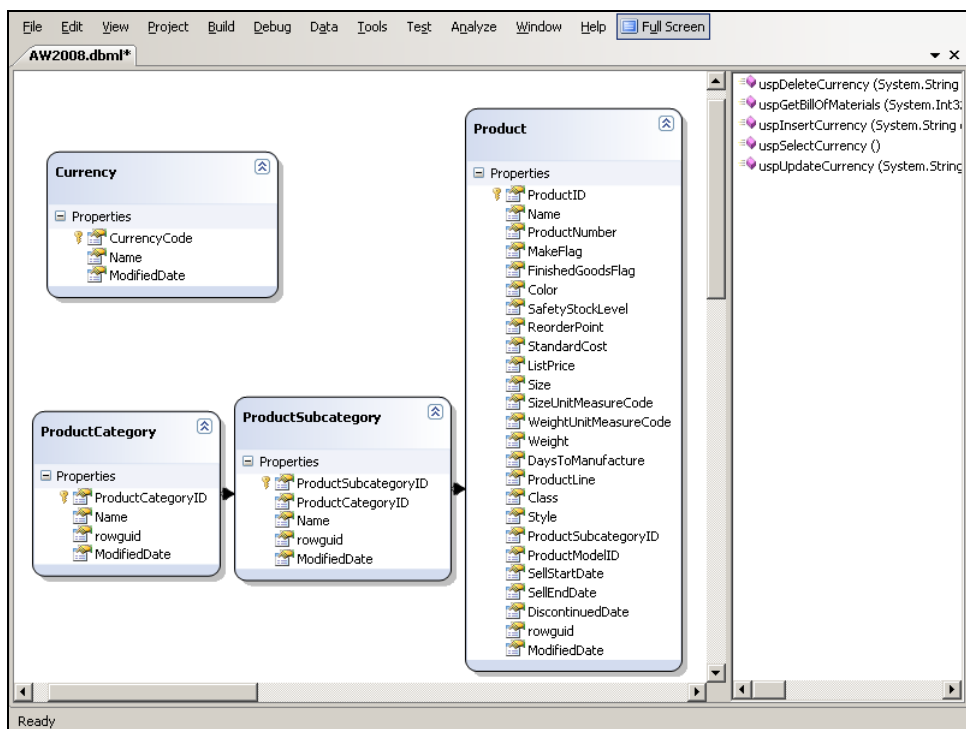
**Рис. 10.4.** Диалоговое окно **Add New Item** при выбранном шаблоне **LINQ To SQL Classes**

Назовите вашу модель `AW2008.dbml`, а затем нажмите кнопку **Add**. После короткой паузы откроется визуальный конструктор модели L2S. Этот конструктор разделен на две панели: слева вы можете конструировать ваши классы (из таблиц или представлений), а справа вы можете управлять методами модели из хранимых процедур и определяемых пользователем функций базы данных (UDF). Мы можем быстро создать нашу модель при помощи перетаскивания (из окна **Server Explorer** на панель классов данных) таблиц `Currency` (`Sales`), `ProductCategory` (`Production`), `ProductSubcategory` (`Production`) и `Product` (`Production`). После этого на панель методов перетащите следующие хранимые процедуры: `uspGetBillOfMaterials`, `uspDeleteCurrency`, `uspInsertCurrency`, `uspSelectCurrency` и `uspUpdateCurrency`.

### **ПРИМЕЧАНИЕ**

Вы можете также перетаскивать хранимые процедуры и на панель классов данных, они все равно будут вставлены на панель методов.

По завершении этих шагов визуальный конструктор модели L2S должен быть похож на рис. 10.5.



**Рис. 10.5.** Визуальный конструктор L2S  
после добавления наших таблиц и хранимых процедур

Ваша модель готова почти на 80%! Сохраните ее.

## Изучение модели

Перед продолжением настройки модели давайте внимательно рассмотрим то, что было сгенерировано нашими действиями по перетаскиванию. Прежде всего, каждая таблица имеет в модели соответствующий класс. Это действительно классы, а не таблицы: внутри каждой фигуры класса на диаграмме вы увидите (под заголовком **Properties**) список свойств, каждое из которых соответствует столбцу той таблицы, на которой основан этот класс. Вы можете вручную добавить дополнительное свойство (при помощи щелчка правой кнопкой мыши по классу и последующего выбора пунктов **Add | Property** контекстного меню).

Затем обратите внимание на стрелку, соединяющую класс `ProductCategory` с классом `ProductSubcategory`. (Аналогичная стрелка есть между



ProductSubcategory и классом Product, но давайте мы рассмотрим первую). Эта стрелка представляет собой *взаимосвязь* (как это называется на языке L2S).

Несмотря на то, что эта взаимосвязь была сгенерирована вследствие наличия в базе данных ограничения внешнего ключа, она представляет собой нечто иное. Для лучшего понимания наведите указатель вашей мыши на стрелку взаимосвязи. Появится подсказка со следующим тестом:

```
ProductCategory.ProductSubcategories -> ProductSubcategory
ProductCategory <- ProductSubcategory.ProductCategory
```

Эта подсказка говорит нам о том, что класс ProductCategory содержит свойство ProductSubcategories, которое указывает на коллекцию объектов ProductSubcategory, и класс ProductSubcategory содержит свойство ProductCategory, которое указывает на единственный объект ProductCategory. Другими словами, каждый родительский объект имеет коллекцию дочерних объектов и свойство, указывающее на эту коллекцию, а каждый дочерний объект имеет свойство, указывающее на его (уникальный) родительский объект.

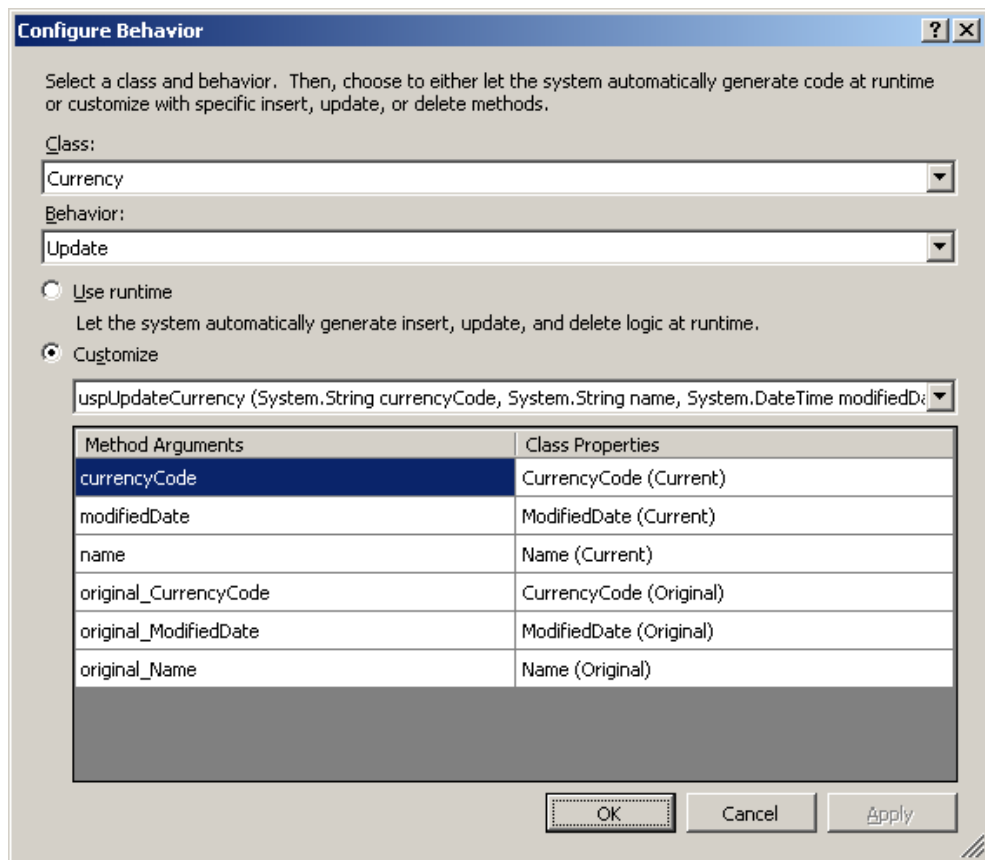
Логично, не так ли? В объектно-ориентированном мире связи "один-многим" представляются дочерними коллекциями и свойствами родитель/потомок. В модели L2S они называются *свойствами взаимосвязи*. Вы увидите, что в L2S коллекции и свойства родитель/потомок заполняются и присваиваются автоматически (при создании экземпляра родительского объекта).

Если окно **Properties** не видно, откройте его. Теперь внимательно щелкните по каждому классу, свойству, взаимосвязи и методу, чтобы посмотреть все редактируемые аспекты объектов модели. И наконец, выберите саму модель (щелкните по свободному пространству визуального конструктора). Обратите внимание, что базовый класс модели — это System.Data.Linq.DataContext, он является объектом AW2008DataContext. Скоро в нашем коде нам понадобится ссылаться на этот объект для того, чтобы делать запросы к данным модели.

## Подключение к хранимым процедурам

Перед тем как перейти к написанию этого кода, нам необходимо сконфигурировать класс Currency. Сейчас (несмотря на то, что хранимые процедуры для обслуживания таблицы были импортированы в модель как методы) этот класс по-прежнему сконфигурирован на использование обычного динамического T-SQL в ответ на выполняемые по нему операции CRUD. Для того чтобы подключить класс к хранимым процедурам, щелкните правой кнопкой мыши

по светло-синей области заголовка класса `Currency`, а затем выберите в контекстном меню пункт **Configure Behavior**. После этого появится диалоговое окно **Configure Behavior** (рис. 10.6 показывает `Update` с хранимой процедурой).



**Рис. 10.6.** Диалоговое окно **Configure Behavior**  
для класса `Currency`

В верхней части диалогового окна имеется комбинированное поле **Class**. По умолчанию здесь выбрано `Currency`, но в выпадающей части этого поля есть все классы модели. Обратите внимание, что комбинированное поле **Behavior** содержит значения `Insert`, `Update` и `Delete`, по умолчанию `Insert`. Для каждого из этих трех значений вы можете установить переключатель **Use Runtime** или **Customize**, по умолчанию выбрано значение **Use Runtime**.

При выбранном в поле **Class** значении `Currency` и в поле **Behavior** значении `Insert` выберите опцию **Customize**. Теперь ниже активируется комбинированное окно, выберите в нем объект `uspInsertCurrency`. Вы настроили модель на использование хранимой процедуры `uspInsertCurrency` (вместо динамического SQL) для выполнения вставок в таблицу `Currency`. Обратите внимание, что сетка внизу диалогового окна теперь заполнена списком параметров метода (соответствуют параметрам хранимой процедуры) и свойствами класса `Currency`, между которыми установлено соответствие. Это соответствие можно редактировать, но необходимости менять его нет. Нажмите кнопку **Apply**.

Затем выберите в поле **Behavior** значение `Update`, выберите опцию **Customize**, а в комбинированном поле под ней объект `uspUpdateCurrency`. Посмотрите на сетку параметров. Обратите внимание, что свойства были присвоены первым трем параметрам (а последним трем — нет). Это потому, что соответствие между параметрами устанавливается только тогда, когда имеются свойства с похожими названиями. Поэтому для параметров `original_CurrencyCode`, `original_ModifiedDate` и `original_Name` мы должны настроить соответствие параметров и свойств самостоятельно.

Вспомним, что свойство `UpdateCommand` объекта `CurrencyTableAdapter` в нашем типизированном наборе `DataSet` имело свою собственную коллекцию параметров. Возможно, вы помните, что когда мы изучали ее содержимое в диалоговом окне редактора **Parameters Collection Editor**, мы заметили, что три параметра (начинающихся с `@Original_`) имели значение свойства `SourceVersion`, равное `Original`. Здесь нам нужно сделать то же самое для трех параметров метода.

Оказывается, что это несложно. Начните со щелчка в ячейке столбца `Class Properties` строки `original_CurrencyCode`. Нажмите стрелку выпадающего списка и обратите внимание на то, что в этом списке имеются как вариант `CurrencyCode (Current)`, так и вариант `CurrencyCode (Original)`. Первый уже был присвоен свойству `CurrencyCode`, поэтому выберите второй, а затем сделайте то же самое для оставшихся двух параметров и соответствующих им свойств. Нажмите кнопку **Apply**, а затем выберите в комбинированном поле **Behavior** значение `Delete`, присвойте ему хранимую процедуру `uspDeleteCurrency` и настройте ее параметры аналогичным образом. Когда вы закончите и будете уверены, что все настроено правильно, нажмите кнопку **Apply**, а затем кнопку **OK**.

Мы почти закончили конфигурировать нашу модель, но у нас до сих пор не было возможности присвоить чему-либо метод `uspSelectCurrency`! Возможно, вас удивило, что в диалоговом окне **Configure Behavior** отсутствует ва-

риант `Select`. Почему это так? Фактически это означает предположение, что разработчики будут предпочитать конвертирование их запросов LINQ в динамический SQL и выполнение его непосредственно по таблице (даже если для обновлений, вставок и удалений используются хранимые процедуры). Однако во многих средах (для пользователей приложения) прямого доступа к таблицам не будет, и, кроме того, мы уверены, что использование хранимых процедур является лучшей практикой, чем применение динамического SQL.

Хранимые процедуры можно использовать для операций выборки. Для того чтобы это работало, необходимо выполнить несколько дополнительных шагов. Сначала выберите в панели методов визуального конструктора метод `uspSelectCurrency`, затем взгляните на его свойство **Return Type** в окне **Properties**, которое установлено в значение **(Auto-Generated Type)**. Щелкните в ячейке значения для этого свойства, а затем нажмите стрелку выпадающего списка. Появится список всех классов модели, из которого вам следует выбрать вариант **Currency** (в ответ на предупреждающее сообщение нажав кнопку **Yes**). Это подсказка для L2S, что при вызове этого метода его результирующий набор должен использоваться для заполнения коллекции объектов `Currency`. Когда эти объекты будут модифицированы и изменения переданы в базу данных, будут использоваться соответственно три другие хранимые процедуры.

Однако этого шага ручной конфигурации можно избежать. Для этого сначала выберите метод `uspSelectCurrency` и удалите его. Затем опять перетащите хранимую процедуру `uspSelectCurrency`, но на этот раз бросьте ее на фигуру класса `Currency` в модели, а не в визуальном конструкторе. Обратите внимание, что метод появляется в визуальном конструкторе еще раз; если вы его выберете, то вы увидите, что его возвращаемый тип был автоматически установлен в значение `Currency`. И последний шаг — измените значение свойства `Name` метода с `uspSelectCurrency` на `SelectCurrency`.

Мы завершили создание модели L2S. Перед тем как изучить запросы в коде к этой модели, сохраните вашу работу. Теперь давайте создадим эквивалентную модель EF.

## Резюме: так много инструментов, так мало времени

Несмотря на то, что в начале этой главы мы объясняли, что количество и разнообразие доступных разработчикам Microsoft технологий доступа к данным велико, возможно, вы все же не были готовы к тому, сколько возможных вариантов

имеется в наличии. А количество вопросов еще больше! Следует ли вообще использовать технологию ORM? И если да, то следует использовать LINQ для SQL или Entity Framework? И если второй вариант, то для запроса моделей следует использовать LINQ для Entities, EntityClient или Object Services? Умер ли обычный ADO.NET? Сочтены ли дни типизированных DataSet? И даже если вы храните верность ADO.NET, следует ли использовать исключительно LINQ для DataSet (вместо прежних прямых SQL-запросов ADO.NET)?

Реальность такова, что ADO.NET по-прежнему здравствует. Обычный код ADO.NET, скорее всего, останется самым эффективным способом доступа к данным. Даже если бы разработчики компании Microsoft толпами начали переходить на ORM-программирование, концепция ADO.NET, скорее всего, сохранила бы свое положение и стала бы аналогом "неуправляемого кода" в области доступа к данным .NET.

В то же время LINQ для SQL и Entity Framework имеют настолько одинаковые подходы, что нам трудно представить себе их продолжительное сосуществование. И в борьбе между ними мы считаем потенциальным победителем технологию EF. Она пришла к нам из команды разработки данных компании Microsoft (в отличие от LINQ для SQL, который был разработан языковой командой); она основана на поставщиках и уже внедряется другими производителями баз данных; для обновления данных службе ADO.NET Data Services нужна технология EF. Мы не можем гарантировать, что EF станет доминирующей ORM-технологией, но мы можем гарантировать, что наш прогноз основан на здравой логике.

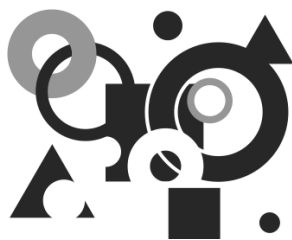
Мы рассматриваем LINQ как определенное достижение в области фильтрации перечислимых коллекций объектов и в области определения значений свойств их объектов. Например, как инструмент XML и ADO.NET Data Services, он дает огромную отдачу при небольших вложениях в его изучение. Компонент LINQ является также отличным способом запроса результирующего набора хранимых процедур на клиенте, причем такими способами, которые хранимыми процедурами не поддерживаются. Это дает как гибкость, так и соответствие рекомендуемым практикам использования хранимых процедур доступа к базам данных. Однако при использовании с реальными таблицами баз данных ценность LINQ значительно падает. Так происходит потому, что нетривиальные запросы LINQ транслируются в запросы SQL, которые могут быть исключительно неэффективными, а также сложными для отслеживания и оптимизации. Для тех, кто хорошо знает SQL, компонент LINQ (как инструмент общего характера для запросов к базе данных) добавляет дополнительный слой, в лучшем случае излишний, а в худшем — вредный для эффективности и производительности.

Разнообразие инструментов доступа к данным в сочетании с методами привязки и представления данных, а также появление WPF и Silverlight параллельно с Windows Forms и ASP.NET, создает нагромождение вариантов для разработчиков, одновременно захватывающее и чрезвычайно хаотичное. Мы думаем, что некоторая перетряска неизбежна, если не по причине избыточности возможностей этих инструментов, то по причине усталости разработчиков от изучения их всех.

Доминирующими платформами приложений будущего нам видятся WPF и Silverlight. Мы видим в качестве популярного набора инструментов для доступа к данным Entity Framework (вместе с ADO.NET Data Services и технологией ASP.NET Dynamic Data), с более основательной поддержкой хранимых процедур и повышенной эффективностью генерируемого SQL. Навыки работы с ADO.NET останутся востребованными и будут полезны.

Типизированные DataSet с течением времени могут стать более специализированными в своем применении, но останутся удобными, эффективными и важными (особенно при разработке Windows Forms). Windows Forms и ASP.NET будут использоваться и сохранят популярность. Вложения коллективного разума в эти технологии, эффективность их визуальных конструкторов и прочих инструментов разработки (не говоря уже о сохраняющихся потребностях в основанных на HTML Web-приложениях) буквально гарантируют это.

Несмотря на то, что мы не можем гарантировать точности этих прогнозов, мы уверены в том, что наш анализ достаточно аккуратен и не сильно расходится с тем, чего ждут другие разработчики. Пока же тщательно выбирайте место вложения своего драгоценного времени, которое вы тратите на изучение новых технологий. В этой главе мы объясняли каждую технологию при помощи других, так что все изученное вами здесь будет полезно вне зависимости от того, как закончится это соревнование между инструментами. И помните, что хотя такая "текучка" в .NET-технологиях доступа к данным может создать некоторые трудности, она гораздо предпочтительнее застоя и недостатка новшеств для платформ.



# Многогранность связывания данных в .NET

*Эндрю Браст*

В *главе 10* мы рассмотрели технологии доступа к данным, имеющиеся в распоряжении .NET-разработчиков, создающих приложения для Microsoft SQL Server 2008. Безусловно, для принятия верного решения относительно способа доступа к вашим данным нужно проделать определенную работу. После того как такое решение будет принято, вам будет необходимо продумать стратегию для отображения данных и их модификации в пользовательском интерфейсе вашего приложения. Несмотря на то, что вы, конечно, можете сами написать код загрузки значений столбцов в элементы управления и считывания их оттуда, компания Microsoft очень усердно потрудились для того, чтобы вам не пришлось этого делать. Разрабатываете ли вы для платформы Microsoft Windows Forms, ASP.NET, Windows Presentation Foundation (WPF) или Silverlight — везде есть богатые технологии связывания данных и методы для выполнения подобной работы.

Связывание данных раньше было сущим наказанием. Так получилось потому, что технологии связывания данных в старых платформах (например, в ранних версиях Microsoft Visual Basic) вынуждали разработчиков использовать двухуровневую архитектуру, которая больше подходила для демонстрационных целей и прототипов, чем для промышленных приложений. Однако с момента выпуска Windows Forms 1.0 в 2002 г. и ASP.NET 2.0 в 2005 г., когда Web-разработчики получили настоящее двустороннее связывание, привязка стала мощной, хорошо настраиваемой, совместимой с n-уровнями и получила очень хорошую поддержку в инструментах Microsoft Visual Studio.

С появлением платформ WPF, Silverlight 2.0, L2S, ADO.NET EF и ADO.NET Data Services связывание данных в .NET неизбежно должно было развиваться (чтобы учесть возможности этих новых платформ) — и это было сделано. Кроме того, появилась новая технология ASP.NET Dynamic Data, которая практически расширяет связывание данных до самого приложения. Поэтому понимание связывания данных не является просто желательным, это настоятельная необходимость.

Наша цель в данной главе — познакомить вас со всеми этими технологиями связывания данных и с той поддержкой, которая имеется для них в Visual Studio. Для одной главы это может показаться слишком обширной задачей. Однако если вы полностью поняли основы связывания данных в Windows Forms и ASP.NET для типизированных DataSet, то понимание связывания данных в L2S, EF и ADO.NET Data Services потребует совсем небольшого объема дополнительного изучения. Более того, это понимание облегчит также объяснение и понимание моделей связывания в WPF и Silverlight.

Держитесь! Если вы способны охватить такой объем материала, то мы можем начать с освоения этих концепций прямо сейчас.

### **ВАЖНО**

В данной главе коды требуют установленных Visual Studio 2008 Service Pack 1 (SP1) и Silverlight 2.0. Некоторые из описываемых методов требуют Microsoft Expression Blend 2.5. На момент печати этой книги Silverlight 2.0 существовал в бета-версии, а Expression Blend 2.5 — только в виде предварительной версии. К тому времени, когда вы будете читать данную книгу, оба этих продукта, скорее всего, уже выйдут в окончательном виде. Код в этой главе может несколько отличаться от программного обеспечения, с которым вам придется работать (особенно Silverlight). Однако мы предполагаем, что эти изменения будут минимальными.

## **Связывание данных в Windows Forms — золотой стандарт**

При появлении на свет Visual Studio .NET и Microsoft .NET Framework технология Windows Forms имела очень хорошую привязку данных, но для ее освоения нужно было повозиться, и поддержка в визуальных конструкторах была слабой. С выходом Visual Studio 2005 и .NET Framework 2.0 гибкость и поддержка в визуальных конструкторах связывания данных Windows Forms стали просто фантастическими. Тем более, что для связывания данных



Windows Forms в Visual Studio 2008 почти ничего не пришлось менять. Тем не менее, мы должны описать, как работает привязка данных Windows Forms и инструменты ее поддержки, поскольку концепции определяют понимание тех новых вещей, которые появляются в других моделях связывания данных.

Самый быстрый способ для демонстрации функциональных возможностей связывания данных Windows Forms — это использовать инструменты проектирования для создания трехуровневого интерфейса связывания для таблиц `ProductCategory`, `ProductSubcategory` и `Product` (базы данных `AdventureWorks2008`) в типизированном `DataSet`. По завершении мы ликвидируем созданные визуальными конструкторами элементы управления, компоненты, настройки и сгенерированный ими код, а затем модифицируем форму так, чтобы она стала работать с нашими моделями L2S и EF (вместо типизированного `DataSet`).

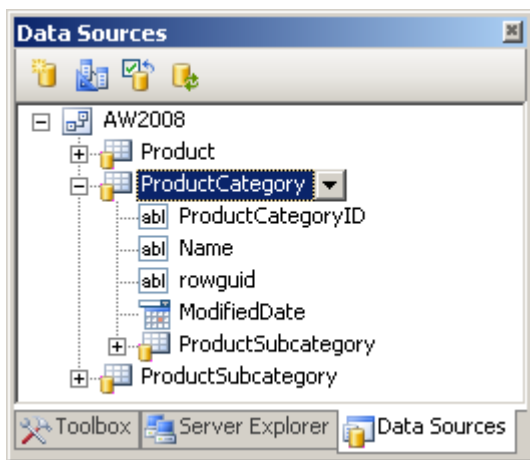
## Подготовка

Воспользуйтесь проектом Windows Forms в примерах кодов данной главы или создайте новый проект и добавьте в него типизированный `DataSet` с названием `AW2008.xsd` (с указанными ранее таблицами). Отредактируйте `DataSet` и переименуйте объект `Relation` (между таблицами `ProductCategories` и `ProductSubcategories`) в `ProductSubcategories`, переименуйте также другой объект `Relation` в `Products`. Это обеспечит согласованность названий объектов `Relation` со свойствами связей и навигации в моделях L2S и EF, которые мы создадим далее. Сохраните и закройте типизированный `DataSet`.

Теперь нам нужно воспользоваться окном **Data Sources**. Если это окно невидимо, отобразите его при помощи выбора в главном меню Visual Studio пунктов **Data | Show Data Sources**. В окне **Data Sources** вы должны увидеть типизированный `DataSet` и составляющие его таблицы. (Для того чтобы увидеть таблицы, вам может понадобиться развернуть узел **DataSet**). Если вы развернете узел любой таблицы, то вы увидите каждый ее столбец. Теперь откройте форму по умолчанию `Form1` (если вы находитесь в новом проекте) или добавьте новую форму (если вы в существующем проекте). При открытой в режиме конструирования форме вы увидите, что значки рядом с узлами таблиц и столбцов в окне **Data Sources** изменились (рис. 11.1).

Каждый значок столбца представляет собой присвоенный данному столбцу тип элемента управления, а каждый значок таблицы является либо представ-

лением в виде сетки, либо детализированным представлением (которое будет сгенерировано при перетаскивании данного узла на форму).

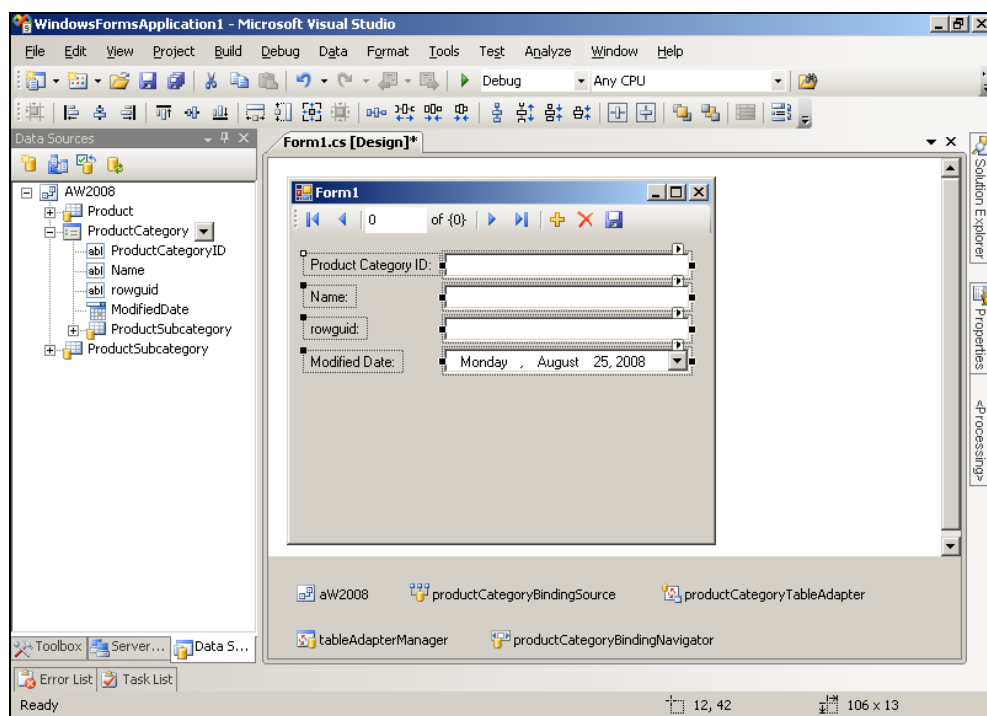


**Рис. 11.1.** Так выглядит окно **Data Sources** при конструировании Windows Form

Нас интересует создание такого иерархического пользовательского интерфейса, в котором данные для категории продуктов представлены как одиночные записи в детализированном представлении, а дочерние подкатегории и продукты представляются внутри сеток. Для подготовки нажмите на выпадающую стрелку узла таблицы `ProductCategory` в окне **Data Sources**, а затем выберите опцию **Details**. Теперь разверните узел для того, чтобы увидеть столбцы таблицы `ProductCategory`. Обратите внимание, что последний "столбец" является фактически узлом, представляющим таблицу `ProductSubcategory`; разверните и этот узел тоже. Затем обратите внимание, что последний дочерний узел в узле таблицы `ProductSubcategory` представляет таблицу `Product`; разверните и его, чтобы была видна вся наша трехтабличная иерархия.

## Генерирование пользовательского интерфейса

Теперь вернемся к узлу таблицы `ProductCategory` и перетащим его на форму в левой части клиентской области. Когда вы отпустите кнопку мыши, на форме и в дереве компонентов появится большое количество элементов (рис. 11.2).



**Рис. 11.2.** Форма Windows Form  
после перетаскивания связывания данных таблицы ProductCategory

Перед тем как изучить то, что было сгенерировано, удалите метку и текстовое поле, которые соответствуют столбцу rowguid, а затем захватите дочерний узел таблицы ProductSubcategory и поместите его справа от всех сгенерированных текстовых полей, выровняв по вертикали с верхней меткой формы. Убедитесь, что вы перетаскиваете тот узел таблицы ProductSubcategory, который находится под узлом таблицы ProductCategory, а не узел верхнего уровня (который находится ниже).

Обратите внимание, что для подкатегорий генерируется элемент управления DataGridView. Несмотря на то, что курсор вашей мыши был помещен на крайнюю правую часть формы, сама форма горизонтально удлинилась для того, чтобы разместить новую сетку. И наконец, перетащите внушительный узел (представляющий таблицу Product, а не узел верхнего уровня, который находится выше) и поместите его сразу же под сеткой подкатегорий (в том же самом положении по горизонтали).

Вы создали полнофункциональную форму обслуживания данных для категорий продуктов (с переходами и прочими необходимыми элементами управления в панели инструментов). Сами данные категорий представляются по одной строке/записи, причем каждый столбец отображается в своем элементе управления, позволяющем выполнить редактирование. Сетка справа сверху будет отображать все подкатегории (принадлежащие к текущей категории), причем выбор строки в этой сетке будет приводить к показу всех ее продуктов в сетке справа внизу. Поскольку мы перетаскили на форму (из окна **Data Sources**) именно дочерние узлы, представляющие эти таблицы (а не узлы верхнего уровня), то отображаются только строки дочерних подкатегорий и продуктов (а не все содержимое этих таблиц).

Запустите приложение (отредактировав при необходимости ваш файл `program.cs`, чтобы сделать вашу новую форму стартовой), а затем просмотрите результаты. Функциональность совершенно полная и весьма впечатляющая.

## Изучаем выходные данные

Легкость связывания данных путем перетаскивания впечатляет, однако большинство разработчиков чувствуют себя неуютно при использовании тех функциональных возможностей, которые они не понимают, вне зависимости от того, насколько они мощные и удобные. Помня об этом, мы даем список того, что было сгенерировано, и взаимосвязи всех настроек свойств объектов:

- в дереве компонентов появился объект `aw2008`, который является экземпляром нашего типизированного `DataSet`. Его свойство `DataSetName` установлено в значение `AW2008`;
- также в дереве компонентов появился компонент `BindingSource` с названием `productCategoryBindingSource`, который работает как посредник между привязанными элементами управления и таблицей `ProductCategory` типизированного `DataSet`. Обратите внимание, что его свойство `DataSource` установлено в значение `aw2008`, а свойство `DataMember` — в значение `ProductCategory`;
- все в том же дереве компонентов находится и компонент `BindingNavigator` (и его панель инструментов) с названием `productCategoryBindingNavigator`. Обратите внимание, что его свойство `BindingSource` установлено в значение `productCategoryBindingSource`. Различные свойства `xxxItem` (например, `AddNewItem`, `MoveFirstItem`) установлены на конкретные

элементы управления `ToolStripButton`, `ToolStripTextBox` и `ToolStripLabel` в панели инструментов `BindingNavigator`;

- ❑ на самой форме два элемента управления `TextBox` и элемент управления `DateTimePicker` привязаны к столбцам `ProductCategoryID`, `Name` и `ModifiedDate` соответственно (и сопутствующие им метки `Label`). Если в окне **Properties** вы выберете свойство `(Data Bindings).Text` элемента управления `productCategoryIDTextBox`, а затем щелкнете по выпадающей стрелке в его столбце `Values`, то вы увидите, что он привязан к столбцу `ProductCategoryID` компонента `productCategoryBindingSource`. Обратите внимание, что для других двух привязанных элементов управления сделаны похожие настройки;
- ❑ в дереве компонентов имеется экземпляр нашего класса `TableAdapter` с названием `productCategoryTableAdapter` (для таблицы `ProductCategory`). Если вы посмотрите на выделенный код формы, то вы заметите, что метод `Fill` этого объекта вызывается в функции обработки события `Load` формы;
- ❑ в дереве компонентов имеются дополнительные объекты `BindingSource` для таблиц `ProductSubcategory` и `Product` (с названиями `productSubcategoryBindingSource` и `productBindingSource` соответственно). Их свойства `DataSource` установлены на родительские компоненты `BindingSource` (`productCategoryBindingSource` и `productSubcategoryBindingSource` соответственно), и категории `DataMember` установлены на дочерние объекты `Relation` родительского объекта `BindingSource` (`ProductSubcategories` и `Products` соответственно). Именно так достигается связывание данных родитель/потомок. Примечательно, что объекты `BindingSource` дочерней таблицы привязаны к другим объектам `BindingSource`, а не к типизированному `DataSet` или какому-либо из составляющих его объектов;
- ❑ на форме элементы управления `productSubcategoryDataGridView` и `productDataGridView` привязаны к дочерним объектам `BindingSource` (через их свойства `DataSource`);
- ❑ в дереве компонентов есть дополнительные объекты `TableAdapter` (это `productSubcategoryTableAdapter` и `productTableAdapter`) для дочерних таблиц, причем их методы `Fill` также вызываются в обработчике события `Load` формы; есть также объект `TableAdapterManager` с названием `tableAdapterManager`, метод `UpdateAll` которого вызывается из обработчика события `Click` кнопки **Save** панели инструментов. Этому методу передается

весь типизированный объект `DataSet` с названием `aw2008`; предварительно вызывается метод `EndEdit` объекта `productCategoryBindingSource`;

- в сгенерированном коде (файл `formname.Designer.cs`) есть строки кода, которые для простых привязанных элементов управления (элементы `TextBox` и `DateTimePicker`) создают новые объекты `Binding` и добавляют их в коллекцию элементов управления `DataBindings`. Например, вот эта строка кода сгенерирована для элемента управления `productCategoryIDTextBox`:

```
this.productCategoryIDTextBox.DataBindings.Add(new  
System.Windows.Forms.Binding("Text",  
this.productCategoryBindingSource,  
"ProductCategoryID", true));
```

Этот код связывает свойство `Text` элемента управления со свойством `ProductCategoryID` компонента `productCategoryBindingSource`. Представленное свойством `(Data Bindings).Text` значение (в окне **Properties**) является упрощенным представлением этого сгенерированного кода.

Разнообразные сгенерированные элементы управления, компоненты, привязывания свойств и код могут показаться сложными и замысловатыми, но в этом существует очень логичная иерархическая организация. С одной стороны — посредники `BindingSource` для источников данных (в данном случае это типизированный `DataSet`), а с другой — привязанные элементы управления, `BindingNavigator` и (необязательно) дочерние компоненты `BindingSource`. Компонент `BindingNavigator` создает интерфейс между элементами панели инструментов и соответствующими действиями по навигации и обслуживанию элемента управления `BindingSource`, с которым он связан. Сложные элементы управления (такие как элемент `DataGridView`) реализуют интерфейс с компонентами `BindingSource` через свойства `DataSource/DataMember` данного элемента; простые элементы (такие как элемент `TextBox`) реализуют интерфейс с компонентами `BindingSource` через свойство своей коллекции `DataBindings`. Экземпляры нашего типизированного `DataSet`, `TableAdapter` и специальные классы `TableAdapterManager` добавляются на форму для улучшения сгенерированного кода, предназначенного для загрузки и сохранения данных.

## Конвертация в LINQ для SQL

Вот собственно и все, причем эти концепции полностью справедливы и для других моделей. Для иллюстрации этого мы можем преобразовать нашу форму под использование модели L2S (вместо типизированного `DataSet`). Вы

можете использовать модель L2S из примеров кодов этой главы или добавить новую. Если вы добавляете модель сами, не забудьте назвать ее `AW2008.dbml` и включить те же три таблицы, что в нашем типизированном `DataSet`, закрыть визуальный конструктор и сохранить ваши изменения.

Вот те шаги, которые необходимо выполнить для преобразования нашей формы под привязку к модели L2S:

1. Удалите из дерева компонентов формы все элементы, кроме типизированного `DataSet`, трех компонентов `BindingSource` и компонента `BindingNavigator`. Сохранение компонента `DataSet` позволит элементам управления `DataGridView` сохранить в неприкосновенности свои коллекции `Columns`.
2. Удалите код внутри обработчика события `Load` формы и последнюю строку кода из события `Click` кнопки **Save** в `BindingNavigator` (то есть ту строку кода, которая вызывает метод `UpdateAll` компонента `TableAdapterManager`). Просто удалите эти строки кода, сами функции удалять не нужно.

3. Вставьте следующую строку кода в начале объявления класса формы (непосредственно перед ее конструктором):

```
AW2008DataContext dc = new AW2008DataContext();
```

4. Вставьте следующую строку кода в обработчике события `Load` формы:

```
productCategoryBindingSource.DataSource = dc.ProductCategories;
```

5. Вставьте следующую строку в конце обработчика события `Click` кнопки **Save** в `BindingNavigator`:

```
dc.SubmitChanges();
```

6. Вот и все! Теперь опять запустите приложение и обратите внимание на то, что все работает, как и прежде.

## Преобразование к Entity Framework

Аналогичные изменения можно сделать для того, чтобы преобразовать нашу форму под использование модели EF. Вы можете воспользоваться моделью EF проекта `Chapter11` (из примеров кодов) или добавить новую. Если вы добавляете модель сами, не забудьте назвать ее `AW2008.edmx` и включить те же три таблицы, что были в типизированном `DataSet` и модели L2S. Затем надо переименовать навигационное свойство `ProductSubcategory` (но не связь!) класса `ProductCategory` в `ProductSubcategories`, а навигационное свойство

Product класса ProductSubcategory — в Products. Выберите модель AW2008.edmx в окне **Solution Explorer**, а затем настройте свойство **Custom Tool Namespace** в значение AdventureWorks2008Model и сохраните ваши изменения. Затем выполните следующие изменения в коде:

1. Добавьте следующую строку кода в конец инструкций using (перед объявлением классов):

```
using EF=AdventureWorks2008Model;
```

2. Следующую строку кода разместите в начале объявления класса формы непосредственно перед его конструктором (заменяв строку, вписанную туда для примера модели L2S):

```
EF.AdventureWorks2008Entities dc = new  
EF.AdventureWorks2008Entities();
```

3. Используйте следующую строку в конце обработчика события click кнопки **Save** в BindingNavigator (заменяв последнюю строку, внесенную туда для примера модели L2S):

```
dc.SaveChanges();
```

4. Следующий блок кода используйте в обработчике события Load формы (заменяв строку, которая была вписана для модели L2S):

```
foreach (EF.ProductCategory cat in dc.ProductCategory)  
{  
    cat.ProductSubcategories.Load();  
    foreach (EF.ProductSubcategory subcat in cat.ProductSubcategories)  
    {  
        subcat.Products.Load();  
    }  
}  
  
productCategoryBindingSource.DataSource = dc.ProductCategory;
```

Вложенные циклы foreach необходимы, поскольку EF не загружает дочерние коллекции автоматически (как это делает L2S). Запустите приложение и убедитесь, что все работает.

### **ПРИМЕЧАНИЕ**

При использовании EF наша форма будет вполне успешно выбирать, обновлять и удалять строки соответствующих таблиц. Однако вследствие специфичных свойств таблицы ProductCategory вставки в нее (без написания дополнительного кода) правильно работать не будут. С другими таблицами вставки с использованием нашего метода связывания данных будут работать хорошо. Тем не менее, мы решили использовать именно



таблицу `ProductCategory` (по причине наличия у нее связей родитель/потомок с `ProductSubcategory` и `Product`). Обратите также внимание, что столбец `ProductCategoryID` в самом верхнем элементе управления `DataGridView` пуст. Это потому, что данное свойство реализовано как навигационное свойство модели EF, а не как скалярное свойство.

## Преобразование для ADO.NET Data Services

Привязка к ADO.NET Data Services также проста. Вы можете просто добавить ссылку на службу **`http://localhost:1863/AW2008Service.svc`** (и настроить пространство имен на `AW2008Service`), которая указывает на службу проекта `Chapter11Service` в примерах кодов этой главы.

### СОВЕТ

Обратите внимание, что номер порта — это статический номер порта, который настроен в проекте `Chapter11Service`, и при необходимости вы можете его изменить (как в ссылке на службу, так и в последующем коде). Если же вы работаете в своем собственном проекте Windows Forms, то необходимо будет убедиться, что служба работает, ее необходимо добавить в ваше решение. После этого она будет запускаться при нажатии клавиши <F5> в Visual Studio.

Вот те изменения кода (заменяющие соответствующие изменения, сделанные для модели EF), которые необходимо выполнить после создания ссылки на службу:

1. Используйте следующую строку кода в начале объявления класса формы непосредственно перед конструктором:

```
AW2008Service.AdventureWorks2008Entities dc =  
new AW2008Service.AdventureWorks2008Entities(  
new Uri("http://localhost:1863/AW2008Service.svc"));
```

2. Используйте следующий блок кода в обработчике события `Load` формы:

```
foreach (AW2008Service.ProductCategory cat in dc.ProductCategory)  
{  
    dc.LoadProperty(cat, "ProductSubcategories");  
    foreach (AW2008Service.ProductSubcategory subcat in  
        cat.ProductSubcategories)  
    {
```

```
        dc.LoadProperty(subcat, "Products");  
    }  
}  
productCategoryBindingSource.DataSource = dc.ProductCategory;
```

И опять вложенные циклы `foreach` необходимы для загрузки дочерних коллекций, хотя способ этой загрузки отличается от того, который использовался для локальной модели EF.

### ПРИМЕЧАНИЕ

Связывание данных будет работать с ADO.NET Data Services для отображения данных иерархическим образом, однако для выполнения обновлений, вставок и удалений потребуется выполнить дополнительную работу. Привязанные элементы управления будут правильно обновлять локальные объекты, но эти изменения не будут автоматически обновлять серверную сторону модели при вызове `SaveChanges`. Поэтому перед вызовом `SaveChanges` необходимо будет явным образом вызывать методы `AddObject` и `DeleteObject` объекта контекста данных (который мы обсуждали в главе 10), а также сопутствующий метод `UpdateObject` (из ADO.NET Data Services) для тех объектов коллекций, которые были соответственно добавлены, удалены или обновлены.

Связывание данных Windows Forms безусловно не исчерпывается тем, что нам удалось показать в этих примерах, но мы дали хороший обзор этой технологии, чтобы помочь вам впоследствии понять связывание данных в WPF и Silverlight. Однако теперь мы переходим к привязке данных ASP.NET.

## Связывание данных в Web-приложениях при помощи ASP.NET

Для иллюстрации простоты связывания создайте новый проект типа ASP.NET Web Application, затем создайте типизированный DataSet с названием `AW2008.xsd`, добавьте в него таблицу `AdventureWorks2008.Currency` и соберите проект. (Это необходимо в проектах ASP.NET, чтобы сделать классы DataSet и TableAdapter доступными для визуальных конструкторов). Теперь откройте форму `Default.aspx` в представлении Design и выполните следующие шаги:

1. Из вкладки **Data** панели инструментов добавьте на форму элемент управления `ObjectDataSource`. Установите его свойство `ID` в значение `odsAW2008`.

- Щелкните по стрелке в правом верхнем углу, а затем выберите в меню **Tasks** пункт **Configure Data Source**.
- На появившейся странице **Choose A Business Object** мастера Configure Data Source Wizard выберите **CurrencyTableAdapter**, затем щелкните по кнопке **Next**, а потом по **Finish**.
- Теперь добавьте на форму элемент управления GridView (также из вкладки **Data** панели инструментов). В его меню **GridView Tasks** выберите из выпадающего списка **Choose Data Source** значение **odsAW2008**, а затем установите флажок **Enable Editing**. Для более привлекательного вида элемента GridView можно использовать также опцию **Auto Format**.

Теперь запустите приложение и убедитесь, что данные отображаются корректно и позволяют выполнять их редактирование внутри столбцов.

## L2S и EF — это просто

Можем мы переделать эту форму под работу с L2S и EF? Разумеется. Выполните следующие шаги:

- Добавьте модель L2S с названием **AW2008.dbml** и добавьте в нее таблицу **Currency**.
- Соберите проект, а затем вернитесь в форму **Default.aspx**.
- Добавьте элемент управления **LinqDataSource** и назовите его **ldsAW2008**.
- Выберите в меню **LinqDataSource Tasks** опцию **Configure Data Source**. На появившейся странице **Choose A Context Object** мастера Configure Data Source Wizard выберите объект **AW2008DataContext**, потом щелкните по кнопке **Next**, а затем по **Finish**.
- Измените источник данных элемента управления GridView (при помощи либо опции **Choose Data Source** меню **GridView Tasks**, либо свойства **DataSourceId** в окне **Properties**) с **odsAW2008** на **ldsAW2008**, а затем настройте свойство **EnableUpdate** последнего в значение **True**. На запрос обновления полей и ключей ответьте кнопкой **No**.
- Запустите приложение, чтобы убедиться в том, что его функциональность не изменилась.

Преобразование формы для работы с EF можно легко выполнить посредством: добавления эквивалентной модели EF с названием **AW2008.edmx** (переименовывать объекты внутри нее не требуется), установки ее опции **Custom Tool Namespace property** в значение **AdventureWorks2008Model**, сборки проекта и

последующего добавления элемента управления `EntityDataSource` на форму с названием `edsAW2008`. Как и в случаях с объектами `ObjectDataSource` и `LinqDataSource`, элемент управления `EntityDataSource` вы настраиваете при помощи мастера `Configure Data Source Wizard`. Выполните следующие шаги:

1. На странице мастера **ConfigureObjectContext** выберите опцию **Named Connection**, выберите в выпадающем списке подключение вашей модели (скорее всего, это будет `AdventureWorks2008Entities`), а затем нажмите **Next**.
2. На странице **Configure Data Selection** выберите объект `Currency` в списке `EntitySetName`, установите флажки **Enable Automatic Inserts**, **Enable Automatic Updates** и **Enable Automatic Deletes**, затем нажмите **Finish**.
3. Настройте элемент управления `GridView` на использование в качестве его источника данных нового элемента управления `EntityDataSource`, а затем запустите приложение и убедитесь, что вся его функциональность сохранилась.

А как насчет ADO.NET Data Services? Оказывается, что из Web-приложения можно использовать Data Services несколькими способами. Один метод простой. Сначала настройте ссылку `Service Reference` на службу (как вы делали это для проекта `Windows Forms`). Затем настройте свойство `DataSourceId` элемента управления `GridView` в значение `(None)` и щелкните **Yes** в ответ на запрос обновления полей и ключей. Установите свойство `AutoGenerateColumns` в значение **True**, а затем в коде запросите службу, получите необходимый результирующий набор и установите свойство `DataSource` элемента управления `GridView` в возвращенный результирующий набор. Затем вызовите метод `DataBind` элемента управления, и все будет готово:

```
AW2008Service.AdventureWorks2008Entities dc = new
    AW2008Service.AdventureWorks2008Entities(new
        Uri("http://localhost:1863/AW2008Service.svc"));
GridView1.DataSource = dc.ProductCategory;
GridView1.DataBind();
```

Вы можете также вызвать ADO.NET Data Services из специальной библиотеки JavaScript, которую мы будем обсуждать далее в этой же главе.

## Не только простые сетки

Для завершения темы связывания данных ASP.NET давайте посмотрим дальше элемент управления `GridView`. Например, если вы предпочитаете работать в построчном режиме детализации, то вы можете воспользоваться

элементом управления `DetailsView`. Несмотря на то, что отображение этого элемента управления значительно отличается от `GridView`, его использование и конфигурирование настолько похожи, что мы оставим их для вас в качестве упражнения. Вместо них давайте взглянем на элемент управления `FormView` и синтаксис связывания данных `ASP.NET`.

Удалите с формы элемент управления `GridView` и добавьте на его место элемент управления `FormView`. Выполните автоматическое форматирование элемента (при желании), а затем настройте его источник данных на `edsAW2008` (наш элемент управления `EntityDataSource`). Включите разбивку на страницы в списке **SmartTag Form View Tasks**, а затем в окне **Properties** настройте свойство `PagerSettings.Mode` в значение `NextPreviousFirstLast`. Закомментируйте или удалите весь код в событии `Load` формы, запустите приложение и изучите его.

Теперь закройте браузер и еще раз выберите элемент управления **FormView** в визуальном конструкторе Web-форм. Щелкните по смарт-тегу, выберите опцию **Edit Templates** в меню **FormView Tasks**, а затем выберите опцию **EditItemTemplate** в появившемся выпадающем списке **Display**. Обратите внимание, что появилась редактируемая область HTML дизайна элемента управления `FormView` при его нахождении в режиме редактирования. Вы увидите, что столбец `CurrencyCode` привязан к элементу управления `Label`, в то время как столбцы `Name` и `ModifiedDate` привязаны к элементам управления `TextBox`.

Щелкните по смарт-тегу элемента управления `Label`, а затем выберите опцию **Edit DataBindings** в меню **Label Tasks**. Появляется диалоговое окно **DataBindings** со снятым флажком **Two-Way Databinding** и неактивным текстом `Eval("CurrencyCode")` в текстовом поле **Code Expression**. Теперь отобразите то же самое диалоговое окно для элемента управления `Name`. На этот раз вы увидите, что флажок **Two-Way Databinding** установлен, и в текстовом поле **Code Expression** отображается неактивный текст `Bind("Name")`. Закройте диалоговое окно, щелкните по смарт-тегу элемента управления `FormView` и выберите в меню **FormView Tasks** опцию **End Template Editing**.

## Связывание данных при помощи разметки

Теперь переключите форму из представления `Design` в представление `Source`. Обратите внимание, что для `EditItemTemplate` элемента управления `FormView` для элемента управления `Label` выдается следующая разметка:

```
<asp:Label ID="CurrencyCodeLabel1" runat="server"
    Text='<%# Eval("CurrencyCode") %>' />
```

А для элемента управления TextBox столбца Name — такая:

```
<asp:TextBox ID="NameTextBox" runat="server"
    Text='<%# Bind("Name") %>' />
```

Эта разметка не вызывает никакого удивления (если учесть то, что мы видели ранее в диалоговых окнах). В обоих случаях атрибут Text устанавливается в значение выражения связывания в виде '<%# action("col") %>', где col — это имя столбца (к которому привязан элемент управления), а action — это либо Eval (для односторонней привязки), либо Bind (для двусторонней привязки).

Знать этот синтаксис связывания данных (и понимать, что диалоговое окно **Edit DataBindings** просто предоставляет удобный пользовательский интерфейс для управления им) очень полезно, поскольку он позволяет вам использовать связывание данных даже тогда, когда нет поддержки визуального конструктора, например, для отдельного элемента управления TextBox (то есть без использования элемента управления FormView).

Вы увидите также, что понимание концепции синтаксиса разметки связывания данных (включая одностороннюю и двустороннюю привязку) и сочетание ее с концепциями объектов BindingSource и Binding в Windows Forms облегчит вам понимание связывания данных в WPF и Silverlight.

Мы не будем изучать остальные элементы связывания данных, но должны упомянуть здесь элемент управления ListView (новый элемент в ASP.NET 3.5), который обеспечивает аналогичную функциональность с повторяющимся шаблоном (напоминающую элемент управления Repeater или даже элемент управления GridView). Теперь у вас должно быть вполне достаточно знаний для того, чтобы работать с этим элементом управления и продуктивно его использовать.

## Использование AJAX для простого доступа к данным

Мы рассмотрели обычную систему ASP.NET. В то же время совместное использование базовых методик AJAX и различных фрагментов инфраструктуры AJAX для доступа к данным в ASP.NET позволяет создавать еще более впечатляющие приложения. Эти методики позволяют обеспечить динамический доступ к данным (который всем нужен) без обновления страниц (которое все ненавидят) при выборке данных с сервера. Если вы хотите, чтобы ваши приложения были похожи на настольные клиенты (а не на тупые экраны терминалов), то вам нужно использовать AJAX.

Исчерпывающее изучение программирования AJAX выходит за рамки данной книги и не смогло бы уместиться в пределах выделенного нам объема. В то же время нам очень важно дать вам представление обо всех имеющихся у вас возможностях, поэтому давайте на них хотя бы взглянем.

По существу ваш диапазон возможностей простирается от использования элемента управления `UpdatePanel` в AJAX ASP.NET (простой вариант) и инструментального набора AJAX Control Toolkit (более сложный для программирования вариант) до JavaScript в сочетании с библиотекой ADO.NET Data Services JavaScript Library (самый сложный для кодирования вариант). В этом разделе мы рассмотрим все три варианта.

## Использование элемента управления *UpdatePanel*

Предпосылка программирования для AJAX состоит в том, что содержимое страницы браузера должно иметь возможность обновляться без необходимости повторной визуализации всей страницы. Обновление "на месте" меньше раздражает пользователя, а также уменьшает объем полезной информации, которую сервер должен возвращать в ответ. Было бы просто здорово, если бы имелся способ декларативно указать ASP.NET, что некоторые части вашей страницы будут часто обновляться, и поэтому вы хотите повторно визуализировать на серверной стороне только это содержимое (когда элемент управления страницы делает возврат формы).

Элемент управления `UpdatePanel` создавался именно для этого сценария; он обеспечивает то, что называется *частичным обновлением страницы*. Просто добавьте на вашу форму один или несколько элементов управления `UpdatePanel`, а затем разместите внутри них те элементы управления, которые будут постоянно повторно визуализироваться (вследствие обновления данных и тому подобного). Содержимое снаружи элементов управления `UpdatePanel` повторно визуализироваться не будет, что позволит достичь более эффективного взаимодействия между клиентом и сервером и улучшить взаимодействие с пользователем.

### **ПРИМЕЧАНИЕ**

В качестве триггеров обновления панели `UpdateControl` вы можете назначить те элементы управления, которые находятся снаружи панели. Например, вы можете указать, что событие `Click` конкретной кнопки должно приводить к возврату формы и обновлению панели. Однако по умолчанию свойство `ChildrenAsTriggers` элемента управления `UpdatePanel` установлено в значение `True`, а его коллекция `Triggers` пуста; при такой кон-

фигурации возвраты форм из элементов управления внутри `UpdatePanel` (только такие возвраты) будут приводить к ее повторной визуализации.

Перейдите на вкладку **AJAX Extensions** панели инструментов **Toolbox**, перетащите с нее элемент управления `ScriptManager` (он требуется для всех операций AJAX) и `UpdatePanel` на нашу страницу, а затем поместите элемент управления `FormView` (но не элементы управления `DataSource`) в элемент управления `ScriptManager` и опять запустите приложение. Внимательно наблюдайте за строкой состояния при навигации по записям/строкам. Вы должны увидеть, что страница не обновляется. Если захотите, верните страницу в ее состояние без `UpdatePanel` и проверьте еще раз: вы увидите, что поведение отличается (даже при доступе к вашему локальному Web-серверу).

### **ПРИМЕЧАНИЕ**

Описанный нами пример содержится в странице `UpdatePanel.aspx` проекта `Chapter11Web`.

### **СОВЕТ**

Если в диалоговом окне **Add New Item** вы добавите в ваш проект AJAX Web Form (вместо Web Form), то элемент управления `ScriptManager` будет автоматически вставлен за вас. Можно также добавить AJAX Master Page со встроенным элементом управления `ScriptManager`.

Элемент управления `UpdatePanel` — это отличный инструмент программирования: он дает мгновенный эффект и не требует почти никакой работы. Но в действительности элементы управления внутри него обслуживаются с Web-сервера (а не просто обновляют свои фрагменты при помощи скриптов на стороне клиента). Вероятно, второй вариант — это именно то, для чего предназначено "настоящее" программирование AJAX. Так как же им воспользоваться без необходимости писать большое количество скриптов? Именно здесь и появляется инструментальный набор ASP.NET Control Toolkit.

## **Добавление резвости при помощи AJAX Control Toolkit**

Инструментальный пакет AJAX Control Toolkit в состав Visual Studio 2008 не входит, фактически это общественный проект, который доступен на сайте CodePlex компании Microsoft. При желании вы можете скачать данный инструментальный набор с этого сайта. (Ссылка на интернет-адрес есть в примерах кодов данной главы). С другой стороны, нужная сборка имеется в нашем проекте `Chapter11Web`, так что вы можете взять ее оттуда.



Откройте страницу `Accordion.aspx` проекта. Эта страница (позаимствованная из одного примера по ASP.NET AJAX одного из отличных пособий, имеющих по адресу: <http://www.asp.net>) для визуализации содержимого таблицы `Currency` сочетает элемент управления `Accordion` (инструментального набора) с элементом `EntityDataSource`. Запустите ее, и вы увидите, что страница сначала показывает все коды валют и полное название первой валюты. Щелкните любой другой код валюты, и панель развернется и покажет ее полное название (а первая панель свернется и будет показывать только код валюты).

Вот фрагмент разметки для элемента управления `Accordion`:

```
<ajaxtoolkit:Accordion ID="accCurrency" runat="server"
                        HeaderCssClass="header"
ContentCssClass="content" Width="300px" DataSourceID="edsCurrency"
                        FadeTransitions="true">

    <HeaderTemplate>
        <b>Currency <%=Eval("CurrencyCode") %></b>
    </HeaderTemplate>
    <ContentTemplate>
        <%=Eval("Name") %>
    </ContentTemplate>
</ajaxtoolkit:Accordion>
```

Похоже на что-то? Вы сообразили! Эта разметка использует тот же синтаксис связывания данных, который мы уже изучали ранее при исследовании выдаваемой визуальным конструктором разметки. Другой достойный примечания аспект: чтобы эта разметка работала правильно, вашему проекту потребуется ссылка на сборку `AjaxControlToolkit.dll` и (соответственно) следующая строка разметки в самом верху страницы:

```
<%@ Register Assembly="AjaxControlToolkit" Namespace="AjaxControlToolkit"
TagPrefix="ajaxtoolkit" %>
```

Если вы устанавливаете данный инструментальный набор и добавите его элементы управления в свой инструментарий, то при добавлении элемента управления из инструментального набора эта строка будет добавляться на страницу автоматически.

## Рассмотрим средства расширения

Кроме элементов управления, данный инструментальный набор включает в себя несколько расширений стандартных элементов управления ASP.NET. Для того чтобы добавить такой расширитель к элементу управления на вашей

странице, необходимо просто щелкнуть по элементу правой кнопкой мыши и выбрать в контекстном меню пункт **Add Extender**. Этот пункт доступен также и через меню **Tasks** элемента управления (нажатием смарт-тега). Еще один простой способ добавить расширитель — перетащить его из инструментального ящика на тот элемент управления, который он должен будет расширить.

Давайте рассмотрим один такой расширитель, для чего изучим страницу `CascadingDropDown.aspx` проекта `Chapter11Web` (которая была позаимствована из примеров кодов учебного видеоролика по адресу <http://www.asp.net>). Элемент управления `CascadingDropDown` в сочетании с двумя стандартными элементами и Web-службой ASP.NET можно использовать для заполнения одного элемента управления `DropDownList` в соответствии со сделанным в другом элементе выбором.

При помощи настройки свойств `ServicePath` и `ServiceMethod` вы можете направить на определенный Web-метод заполнения указанного в его атрибуте `TargetControlID` элемента управления. (Web-метод службы возвращает массив объектов `CascadingDropDownNameValue`). Значение выбранного пункта в другом элементе управления `DropDownList` (который указан в свойстве `ParentControlID`) определяется в Web-методе, что позволяет ограничивать возвращаемые данные сделанным выбором.

На странице `CascadingDropDown.aspx` в элементе управления `DropDownList` отображается список категорий продуктов; выбор в этом списке приведет к заполнению следующего элемента `DropDownList` подкатегориями указанной категории. Выбор подкатегории аналогичным же образом заполняет третий элемент управления `DropDownList` всеми продуктами, соответствующими сделанному выбору. Код Web-службы содержится в `CascadingDropDown.aspx` того же самого проекта. (Web-служба не обязательно должна находиться на другом сервере или в отдельном проекте). Запустите страницу, а затем взгляните на код, чтобы понять, как он работает.

### **ПРИМЕЧАНИЕ**

Функции Web-метода имеют параметры `knownCategoryValues` и `category`. Не путайте их с категориями продуктов базы данных `AdventureWorks2008`, похожие имена являются простым совпадением.

### **ВАЖНО**

Класс Web-службы, содержащей вызываемые элементом управления `CascadingDropDown` Web-методы, должен иметь примененный атрибут `System.Web.Script.Services.ScriptService`.

Инструментальный набор ASP.NET AJAX Control Toolkit не исчерпывается тем, что мы здесь описали. Он имеет больше методов, элементов управления и расширителей. Но эти два коротких примера дают хорошее представление об ориентированных на доступ к данным возможностях платформы.

## Использование библиотеки AJAX Client Library для ADO.NET Data Services

Клиент AJAX для ADO.NET Data Services предоставляет библиотеку скриптов, которая позволяет разработчикам вызывать службу данных из JavaScript почти так же, как это позволяет делать .NET-клиент из .NET-кода. При помощи этой библиотеки мы можем написать код, который вызывает AW2008Service.svc и "на ходу" обновляет даже стандартные элементы управления HTML, и все это при помощи одного только кода на клиентской стороне.

Страница ADONETDatSvcJS.aspx (в примере проекта Chapter11Service) показывает, как это можно сделать. В ней имеется кнопка HTML, которая при нажатии заполняет окно списка HTML категориями продуктов. Выбор в этом окне списка заполняет набор текстовых полей значениями столбцов для данной категории, а дополнительное окно списка — всеми подкатегориями.

Обратите внимание, что клиентская библиотека JavaScript распространяется в виде двух файлов с расширением js (отладочная версия и версия без отладки), один из которых должен быть добавлен в ваш проект. Проект Chapter11Service включает в себя отладочную версию и может запускаться без изменений. Однако при желании вы можете скачать с сайта CodePlex оба эти файла js. (Ссылка на интернет-адрес загрузки имеется в примерах кодов данной главы на Web-сайте этой книги).

После включения в проект этой библиотеки вы сможете создать в коде JavaScript объект `Sys.Data.DataService` и выполнить асинхронный вызов вашей службы данных при помощи обсуждавшегося нами в *главе 10* синтаксиса Uniform Resource Identifier (URI). Вызов должен сослаться на две функции обратного вызова, которые будут обрабатывать как успешные, так и неудачные запросы. Вот код, использованный в ADONETDatSvcJS.aspx:

```
var ds = new Sys.Data.DataService("AW2008Service.svc");  
ds.query("/ProductCategory", cbSuccess, cbFailure);
```

Поскольку страница и служба находятся в одном и том же проекте, то нам не нужен полный (абсолютный) идентификатор URI в первой строке при создании экземпляра объекта `DataService`, относительного URI вполне достаточно.

Вторая строка выбирает коллекцию `ProductCategory` и регистрирует функции `cbSuccess` и `cbFailure` в качестве наших функций обратного вызова.

Код функции `cbSuccess` имеет три входных параметра, причем первый из них предоставляет результаты — в виде массива объектов `ProductCategory`. Код может перебрать массив результатов и использовать стандартный динамический код HTML (DHTML) для заполнения окна списка данными. В примере кода посмотрите также в событии `onchange` окна списка и то, как загружаются данные для отдельных категорий и их подкатегорий.

На данный момент мы описали различные способы, которыми вы можете писать код в ASP.NET для доступа к вашим данным в SQL Server. Нам необходимо остановиться еще на одном аспекте ASP.NET, и это методика, которая совсем не требует кодирования.

## Динамические данные ASP.NET

Элегантность объектов `DataSource` и привязанных к данным элементов управления ASP.NET говорят сами за себя. Они просты, декларативны и очень гибкие. Имеет смысл использовать некую методику шаблонов для генерирования страниц, которые могут привязываться к любой таблице нашей базы данных (или, говоря точнее, к любому классу нашей модели L2S или EF). Такой подход реализуется в ASP.NET Dynamic Data, причем делается это очень хорошо. Конечно, в течение уже многих лет существуют и другие инструменты (на основе шаблонов) для разработчиков, но когда такой инструмент есть внутри Visual Studio 2008, и он может генерировать целые сайты, то ваша производительность буквально подсказывает. А когда этот инструмент позволяет подстраивать сгенерированный сайт при помощи ваших собственных шаблонов и декларативной конфигурации, то жизнь разработчика становится просто замечательной.

Заставить все это работать несложно. Просто создайте или добавьте новый проект в Visual Studio 2008, а затем выберите либо шаблон `Dynamic Data Web Application` (если хотите использовать L2S), либо шаблон `Dynamic Data Entities Web Application` (если хотите использовать EF). Шаблон проекта сгенерирует каталог `DynamicData` с несколькими подкаталогами и файлами внутри обычного проекта ASP.NET Web Application. После этого вам необходимо сделать только две вещи: добавить в проект новую модель (L2S или EF) и сделать два простых изменения в файле `Global.asax`.

Для нашего примера создайте проект `Dynamic Data Entities Web Application`, а затем добавьте в него модель EF с названием `AW2008.edmx`, содержащую таб-

лицы Currency, ProductCategory, ProductSubcategory и Product из базы данных AdventureWorks2008. После этого отредактируйте страницу Global.asax так, как показано в листинге 11.1. (Обратите внимание, что инструкции using в этом листинге опущены, можно использовать генерируемые по умолчанию.)

**Листинг 11.1. Файл Global.asax в проекте Dynamic Data (сконфигурирован для нашей модели EF)**

```
namespace Chapter11DynamicData
{
    public class Global : System.Web.HttpApplication
    {
        public static void RegisterRoutes(RouteCollection routes)
        {
            MetaModel model = new MetaModel();
            model.RegisterContext(typeof(AdventureWorks2008Entities), new
                ContextConfiguration() { ScaffoldAllTables = true });
            routes.Add(new DynamicDataRoute("{table}/{action}.aspx")
            {
                Constraints = new RouteValueDictionary(new { action =
                    "List|Details|Edit|Insert" }),
                Model = model
            });
        }
        void Application_Start(object sender, EventArgs e)
        {
            RegisterRoutes(RouteTable.Routes);
        }
    }
}
```

Теперь запустите приложение и осмотритесь! Сначала вы увидите список всех ваших таблиц; щелкните по любой из них — и вы сможете просмотреть данные в элементе управления GridView (с опцией фильтрации отображенных строк), удалить любую строку на месте, отредактировать или добавить новые строки в элементе управления DetailsView. Вы всегда можете вернуться на начальную страницу и отредактировать другую таблицу.

Посмотрите повнимательнее на код в Global.asax. Этот код использует средство маршрутизации из архитектуры контроллера просмотра моделей (model view controller, MVC) в ASP.NET для интерпретации любого URI в виде

table/action.aspx (где table — это название редактируемой таблицы, а action — это List, Details, Edit или Insert) и последующего направления клиента к List.aspx, Details.aspx, Edit.aspx в каталоге DynamicData/PageTemplates. Например, <http://localhost:20519/Currency/List.aspx> приведет к странице DynamicData/PageTemplates/List.aspx и по существу передаст Currency в качестве таблицы/объекта для связывания.

Откроем страницу DynamicData/PageTemplates/List.aspx в представлении Design. Там вы найдете много элементов управления, которые мы уже изучили: GridView, EntityDataSource, UpdatePanel и ScriptManager (на главной странице). Имеются также Hyperlink и ValidationSummary (которые являются достаточно стандартными элементами). И наконец, есть также некоторые специфичные для ASP.NET Dynamic Data элементы управления. Полное изучение этой страницы выходит за пределы данной книги. Но даже с первого взгляда вы можете видеть, что многие изученные здесь аспекты AJAX и элементов управления источников данных ASP.NET — это те же самые технологии, которые компания Microsoft использовала для создания функциональных возможностей Dynamic Data.

Мы не сможем показать вам все трюки, которые вы можете делать с Dynamic Data, но один из них настолько важен, что мы его опишем. Обратите внимание, что в каталоге DynamicData/PageTemplates есть одна страница, на которую нет ссылок в Global.asax, это ListDetails.aspx. Данная страница позволяет выполнять редактирование по месту в элементе управления GridView и предоставляет элемент управления DetailsView, который позволяет делать вставки (которые GridView делать не может), а также отображает данные для строк, выбранных в GridView.

Это означает, что ListDetails.aspx может охватить все операции List и Details и (при помощи редактирования по месту) обработать также редактирование и вставки. Поэтому мы могли бы отредактировать Global.aspx под использование этой страницы для операций List и Details для всех наших таблиц. Но что будет, если мы захотим обрабатывать таблицу Currency как прежде, а через ListDetails.aspx маршрутизировать только три оставшиеся таблицы? Решение простое: в Global.aspx мы просто меняем функцию RegisterRoutes так, как показано в листинге 11.2.

#### **Листинг 11.2. Модифицированная функция RegisterRoutes в нашем файле Global.asax**

```
public static void RegisterRoutes(RouteCollection routes)
{
    MetaModel model = new MetaModel();
```

```
model.RegisterContext (typeof (AdventureWorks2008Entities), new
    ContextConfiguration() { ScaffoldAllTables = true });
routes.Add(new DynamicDataRoute ("Currency/{action}.aspx")
{
    Constraints = new RouteValueDictionary(new { action =
        "List|Details|Edit|Insert" }),
    Model = model,
    Table = "Currency"
});
routes.Add(new DynamicDataRoute ("{table}/ListDetails.aspx")
{
    Action = PageAction.List,
    ViewName = "ListDetails",
    Model = model
});
routes.Add(new DynamicDataRoute ("{table}/ListDetails.aspx")
{
    Action = PageAction.Details,
    ViewName = "ListDetails",
    Model = model
});
}
```

Сделайте эти изменения и изучите их, затем запустите приложение, чтобы увидеть, как замечательно оно работает.

С такой хорошей основой в области связывания данных Windows Forms и ASP.NET мы можем завершить нашу работу рассмотрением связывания данных в WPF и Silverlight.

## Привязка данных в Windows Presentation Foundation

Windows Presentation Foundation (WPF) — это платформа будущего для разработки под настольные компьютеры Windows. Это может показаться крайностью, но WPF по существу сделала Windows Forms устаревшими. Более того, WPF не просто еще один пакет для работы с формами — переход от Windows Forms к WPF гораздо более резкий, чем был переход, например, от

Visual Basic 6.0 к Windows Forms. Во-первых, WPF использует язык разметки под названием Extensible Application Markup Language (XAML) для управления внешним видом и компоновкой элементов управления на своих формах. Во-вторых, дизайн пользовательского интерфейса типичного приложения WPF — это не просто серый фон и несколько стандартных элементов управления вводом. Эстетические изыски являются практически обязательной составной частью типичного приложения WPF.

В то же время инструментальная поддержка WPF в Visual Studio находится в зачаточном состоянии. Это создает проблему и в то же время смягчает резкость перехода: недостаточная инструментальная поддержка затрудняет переход и замедляет его. Иначе говоря, среда разработки Windows Forms в настоящее время настолько более продуктивна (чем WPF), что подавляющая часть разработок клиентских приложений будет продолжать делаться на Windows Forms в течение еще какого-то времени. Однако факт состоит в том, что Windows Forms в итоге уйдут, и изучение WPF должно стать высокоприоритетной задачей для .NET-разработчиков, пишущих для настольных компьютеров. Это дает нам и мотивацию, и некоторый запас времени для изучения WPF.

В смысле доступа к данным хорошая весть состоит в том, что WPF содержит возможности связывания к данным в самой своей основе. И хотя связывание работает не так, как в Windows Forms, оно разумно сочетает концепции связывания данных как Windows Forms, так и ASP.NET. В несколько меньшей степени это справедливо для Silverlight 2.0. Silverlight задуман как кросс-платформенная версия WPF на основе Web-браузера. Когда Silverlight находился на ранней стадии разработки, он даже назывался WPF/e (где "е" обозначало everywhere, то есть "везде"). Это две разные технологии, но они тесно связаны: если и не сестры, то, безусловно, родственники. То же самое относится и к их механизмам связывания данных. Помня об этом, мы сейчас рассмотрим связывание данных в WPF, а сразу после нее — связывание данных в Silverlight.

## Трудности на этапе конструирования

Одной из самых существенных трудностей программирования доступа к данным в WPF является отсутствие поддержки связывания данных визуальными конструкторами Visual Studio 2008. Есть два варианта: выполнять все связывание данных на XAML или использовать инструмент визуального конструирования компании Microsoft с названием Expression Blend (именуемый далее Blend), который обеспечивает обширную поддержку связывания дан-



ных. Мы начнем с Blend, а затем будем изучать и модифицировать сгенерированный XAML (чтобы понять оба аспекта связывания данных WPF).

Если у вас нет Blend, то вы можете просто следить за нашим изложением, а затем открыть в Visual Studio форму Main.xaml, находящуюся в проекте Chapter11WPF. Blend работает с точно такой же структурой решений, проектов и файлов, что и Visual Studio, так что вы можете создать свой проект в Blend, потом открыть его в Visual Studio, а затем опять открыть его в Blend.

Начните с создания в Visual Studio 2008 нового проекта типа WPF Application project. Добавьте в него модель EF с названием AW2008.edmx (и таблицами ProductCategory и ProductSubcategory). Измените название свойства Name класса ProductCategory на TheName. (Это обходной путь для исправления ошибки, который может стать ненужным к тому времени, как вы это прочтете). Соберите ваш проект и закройте его (или все решение). Откройте проект или решение в Blend, а затем откройте файл Window1.xaml (чтобы мы могли конструировать форму). Отобразите при необходимости панель Blend, а затем щелкните по кнопке **+CLR Object**. В появившемся диалоговом окне **Add CLR Object Data Source** выберите класс ProductCategory (под сборкой проекта и пространством имен по умолчанию), а потом нажмите кнопку **OK**. Теперь класс должен появиться в панели **Data**. Щелкните по стрелке слева от него (для того чтобы развернуть класс и показать его свойства).

Вы должны увидеть свойство ProductCategoryID. Перетащите его на форму в верхнюю левую ее часть. Появится всплывающее меню с запросом типа элемента управления, к которому необходимо привязать свойство — щелкните по TextBox. Затем появится диалоговое окно **Create Data Binding**. В середине этого окна находится стрелка его развертывания (похожая на кнопку **Details>>**, которую вы могли видеть в некоторых других диалоговых окнах); нажмите эту стрелку — и диалоговое окно развернется так, как показано на рис. 11.3.

Диалоговое окно **Create Data Binding** позволяет вам указать поле (в данном случае означающее свойство) элемента управления, к которому будет привязано значение свойства ProductCategoryID, направление связывания, а также несколько других атрибутов связывания. Обратите внимание, что в WPF привязки могут быть как односторонними, так и двусторонними, и односторонняя привязка может идти в любом направлении (как от источника, так и к источнику). Привязка может быть одноразовой; каждый тип элемента управления имеет направление привязки по умолчанию.

Примите варианты по умолчанию для поля/свойства Text и направления привязки, затем нажмите кнопку **OK**. Укажите вертикальный размер тек-

стового поля 25 пикселей, а потом повторите все это для свойств `TheName` и `ModifiedDate` класса `ProductCategory`. Найдите в панели **Data** свойство `ProductSubcategory`, а потом выберите `ListBox` в качестве типа элемента управления привязки. В диалоговом окне **Create Data Binding** обратите внимание на то, что по умолчанию свойство связывания `ListBox` — это `ItemsSource` (то есть именно то, что нам нужно), поэтому нажмите кнопку **OK**.

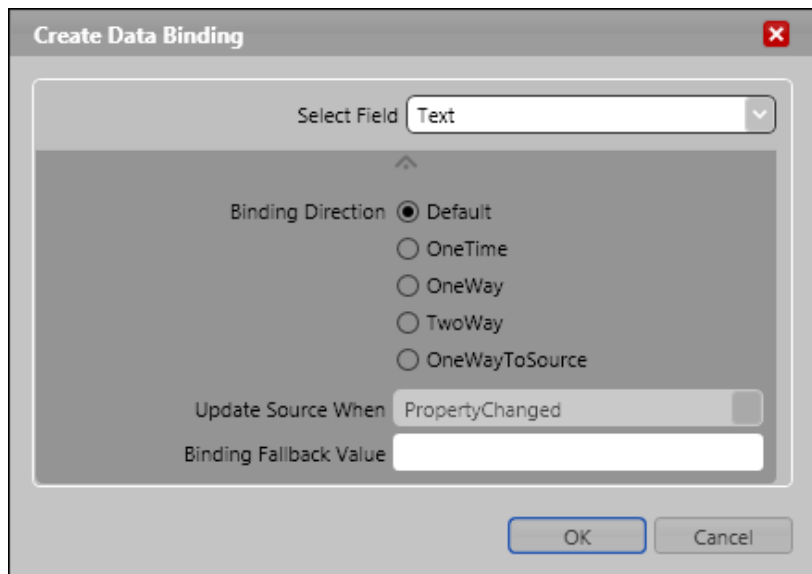
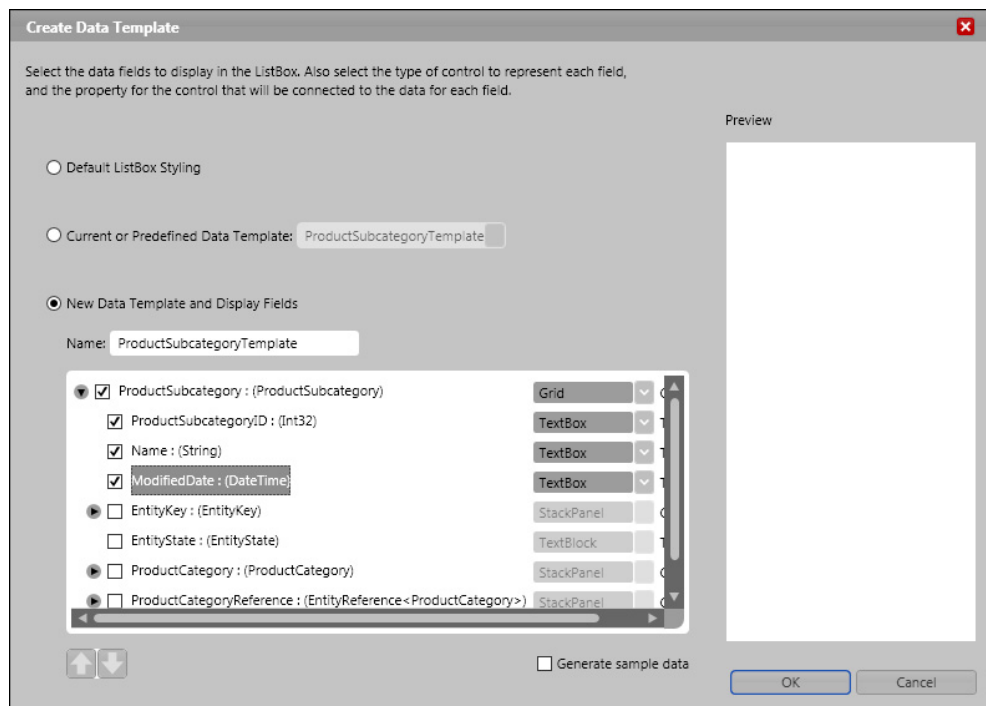


Рис. 11.3. Диалоговое окно **Create Data Binding** в Expression Blend (в развернутом виде)

После этого появится диалоговое окно **Create Data Template**. И вот почему: при помощи шаблона данных `ListBox` в WPF может вести себя и как `ListBox` в Windows Forms, и как элемент управления `Repeater` из ASP.NET (либо как элемент управления `ListView` в ASP.NET), в том смысле, что каждый элемент окна списка может быть контейнером для коллекции составляющих его элементов управления. Шаблон данных содержит XAML-разметку, которая дублируется для каждого элемента списка.

В нашем случае мы хотим создать шаблон данных с тремя привязанными к данным текстовыми полями (по одному для каждого свойства `ProductSubcategory`). Диалоговое окно **Create Data Template** позволяет сделать это очень легко: сначала уберите `ProductSubcategory` из корня списка

полей, затем выберите `ProductSubcategoryId`, `Name`, `ModifiedDate` и кнопками переупорядочивания в нижней левой части диалогового окна переместите эти поля (в заданном порядке) в верхнюю часть списка. И наконец, в комбинированных окнах справа от списка полей выберите `Grid` в качестве типа элемента управления для элемента `ProductSubcategory`, а затем для каждого из трех выбранных свойств укажите `TextBox`. Диалоговое окно должно теперь выглядеть так, как показано на рис. 11.4.



**Рис. 11.4.** Диалоговое окно **Create Data Template** в Expression Blend (настроено для дочерних данных таблицы `ProductSubcategory`)

Нажмите кнопку **ОК**. Мы почти закончили, но, к несчастью, Blend по умолчанию создаст для вашего шаблона данных сетку из трех строк и одного столбца, а не наоборот. Для того чтобы это изменить, щелкните правой кнопкой мыши по элементу управления `ListBox` и в его контекстном меню выберите пункты **Edit Other Templates**, **Edit Generated Items (Item Template)**, **Edit Template**. После этого визуальный конструктор будет редактировать ваш шаблон данных, а не его хост-форму. Здесь вам нужно изменить размер

сетки, чтобы сделать ее шире (выберите Grid щелчком в панели **Objects And Timeline**), создайте два новых столбца, поместите текстовое поле в каждый столбец, а затем удалите вторую и третью строки. Вам придется освоить эти операции Blend. Если вы устали, вы можете пропустить этот шаг и использовать разметку шаблона данных из формы Main.xaml проекта Chapter11WPF.

### ПРИМЕЧАНИЕ

Было бы гораздо более удобно привязывать свойство ProductSubcategory к какой-либо сетке данных (а не к шаблонированному ListBox). WPF (по состоянию на .NET Framework 3.5 SP1) не имеет собственной сетки данных. Наш пример Silverlight использует элемент управления DataGrid, который вы сможете использовать в качестве руководства для элемента управления WPF (после его выпуска).

Для выхода из редактирования шаблона данных и возврата в родительскую форму щелкните по стрелке в верхней левой части панели **Objects And Timeline**. (Стрелка имеет подсказку "Return scope to [Window]."). Сохраните вашу работу, закройте проект и откройте его в Visual Studio.

## Изучение XAML

Откройте форму Window1.xaml (если она не открыта по умолчанию) и просмотрите разметку XAML. Разметка должна быть, как в листинге 11.3.

**Листинг 11.3. Полная разметка XAML для нашей формы WPF, включая наш шаблон данных**

```
<Window x:Class="WpfApplication1.Window1"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Window1" Height="300" Width="429"
    xmlns:WpfApplication1="clrnamespace:
        WpfApplication1" xmlns:d="http://schemas.microsoft.com/expression/
        blend/2008" xmlns:mc="http://schemas.openxmlformats.org/markup-
        compatibility/2006" mc:Ignorable="d">
    <Window.Resources>
        <ObjectDataProvider x:Key="ProductCategoryDS" ObjectType="{x:Type
            WpfApplication1:ProductCategory}" d:IsDataSource="True"/>
        <DataTemplate x:Key="ProductSubcategoryTemplate">
```

```

<Grid Width="207">
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="0.324*"/>
    <ColumnDefinition Width="0.357*"/>
    <ColumnDefinition Width="0.319*"/>
  </Grid.ColumnDefinitions>
  <Grid.RowDefinitions>
    <RowDefinition Height="*"/>
  </Grid.RowDefinitions>
  <TextBox HorizontalAlignment="Stretch" Margin="0,0,0,0"
    Grid.Row="0" Grid.RowSpan="1" Text="{Binding
    Path=ProductSubcategoryID}"/>
  <TextBox Margin="0,0,0,0" Grid.Column="1" Grid.Row="0"
    Grid.RowSpan="1" Text="{Binding Path=Name}"/>
  <TextBox Margin="0,0,0,0" Grid.Column="2" Grid.Row="0"
    Text="{Binding Path=ModifiedDate}" Grid.RowSpan="1"
    VerticalAlignment="Stretch"/>
</Grid>
</DataTemplate>
</Window.Resources>
<Grid>
<TextBox HorizontalAlignment="Left" Margin="19,30,0,0"
  VerticalAlignment="Top" Width="132" Height="23" Text="{Binding
  Path=ProductCategoryID, Mode=Default, Source={StaticResource
  ProductCategoryDS}"/>
<TextBox HorizontalAlignment="Left" Margin="19,57,0,0"
  VerticalAlignment="Top" Width="132" Height="25" Text="{Binding
  Path=TheName, Mode=Default, Source={StaticResource
  ProductCategoryDS}"/>
<TextBox HorizontalAlignment="Left" Margin="19,86,0,0"
  VerticalAlignment="Top" Width="132" Height="25" Text="{Binding
  Path=ModifiedDate, Mode=Default, Source={StaticResource
  ProductCategoryDS}"/>
<ListBox Margin="176,30,45,43" ItemTemplate="{DynamicResource
  ProductSubcategoryTemplate}" ItemsSource="{Binding
  Path=ProductSubcategory, Mode=Default, Source={StaticResource
  ProductCategoryDS}"/>
</Grid>
</Window>

```

Давайте немного разберем эту разметку, а затем мы модифицируем ее и добавим код для того, чтобы получить полностью привязанную к данным форму.

Сначала (на самом верху) в разделе `<Window.Resources>` мы видим определение `<ObjectDataProvider>` и разметку для нашего шаблона данных. Определяя их как локальные ресурсы, разметка внутри самой формы может ссылаться на данные элементы в своих настройках атрибутов. Обратите внимание (ближе к нижней части) на три элемента `<TextBox>` и один элемент `<ListBox>`, соответствующие нашим привязанным элементам управления. Посмотрите на первый элемент `TextBox`, и вы увидите, что его атрибут `Text` настроен следующим образом:

```
Text="{Binding Path=ProductCategoryID, Mode=Default,  
Source={StaticResource ProductCategoryDS}}"/>
```

Другими словами, свойство `Text` элемента управления `TextBox` привязано к свойству `ProductCategoryID` (через атрибут `Path`) текущего элемента, приходящего из определенного через атрибут `Source` ресурса `ProductCategoryDS` `ObjectDataProvider` с направлением привязки по умолчанию типа элемента управления `TextBox` (через атрибут `Mode`). И все это определение (в фигурных скобках) неявно создает объект WPF Binding. (Это можно сделать и в коде, но при помощи синтаксиса скобок мы можем фактически создавать объекты в разметке).

Два других текстовых поля имеют аналогичную разметку (соответствующих свойств). Разметка элемента управления `ListBox` содержит следующее:

```
ItemTemplate="{DynamicResource ProductSubcategoryTemplate}"  
ItemsSource="{Binding Path=ProductSubcategory, Mode=Default,  
Source={StaticResource ProductCategoryDS}}"
```

Иначе говоря, элемент управления `ListBox` привязан к полю `ProductSubcategory` того же ресурса `ObjectDataProvider`, что и текстовые поля. Это свойство является коллекцией/массивом, поэтому такой же коллекцией/массивом является и свойство элемента управления `ItemsSource`, к которому оно привязано, а для управления визуализацией каждого элемента в качестве шаблона данных должен использоваться локальный ресурс `ProductSubcategoryTemplate`. Если вы посмотрите на определение шаблона, то увидите подробности относительно сетки и разметку для трех элементов управления `TextBox`, которая похожа на разметку только что изученных нами элементов управления `TextBox`.

Обратите внимание на то, что элементы управления в шаблоне данных не имеют атрибута `Source` в определениях Binding для своих свойств `Text`. Это потому, что они наследуют свой источник от содержащего их элемента управ-

ления `ListBox` (которым, как мы знаем, является массив `ProductSubcategory` из `ProductCategoryDS`). Источники данных элементов управления WPF всегда наследуются от их родительских элементов управления.

Помня об этом, давайте действительно удалим атрибут `Source` из привязок элементов управления `TextBox` и элемента `ListBox`. По сути это означает, что эти элементы управления корневого уровня будут получать свои данные от свойства `DataContext` (либо корневого элемента управления макета, либо самого родительского `Windows`). В таком случае вы можете удалить из разметки также и весь элемент `ObjectDataProvider`.

Теперь все, что нам необходимо сделать — это установить свойство `DataContext` в значение запроса набора объектов `ProductCategory` модели EF и убедиться, что коллекция `ProductSubcategory` каждого объекта загружена. Для этого создайте обработчик события `Loaded` для объекта `Window` (при помощи двойного щелчка по клиентской области формы в представлении `Design`). Код обработчика показан в листинге 11.4.

#### Листинг 11.4. Обработчик события `Loaded` для нашей формы WPF

```
private void Window_Loaded(object sender, RoutedEventArgs e)
{
    AdventureWorks2008Entities ctx = new AdventureWorks2008Entities();
    var results =
        from cat in ctx.ProductCategory
        select cat;
    foreach (ProductCategory cat in results)
        cat.ProductSubcategory.Load();
    this.DataContext = results;
}
```

Обратите здесь внимание на несколько аспектов:

- ❑ Свойство `DataContext` объекта `Window` во многом похоже на свойство `DataSource` элемента управления `BindingSource` из `Windows Forms`.
- ❑ Дочерние коллекции обрабатываются в привязках данных WPF аналогично тому, как это делается в `Windows Forms`. Свойство `ItemsSource` элемента управления `ListBox` из WPF настраивается так же, как свойства `DataSource` наших элементов управления `DataGridView` (свойство дочерней коллекции). После этого все дочерние данные отображаются правильно.

- Несмотря на то, что мы решили не использовать их здесь, объекты `DataProvider` из WPF ведут себя, как элементы управления `DataSource` из ASP.NET (например, элемент управления `EntityDataSource`), и позволяют выполнить аналогичное связывание данных без кодирования (при помощи разметки).

Перед тем как запустить приложение, добавьте кнопку, создайте для нее обработчик события `Click` и вставьте в него следующий код:

```
CollectionView cv =  
    (CollectionView)CollectionViewSource.DefaultView(this.DataContext);  
if (cv.CurrentPosition < cv.Count - 1)  
    cv.MoveCurrentToNext();
```

Кнопка будет перемещать по строкам коллекции `ProductCategory`. Для целей навигации по коллекции объект `CollectionView` из WPF ведет себя аналогично объекту `BindingSource` из Windows Forms. Объект `BindingSource` имеет методы `MoveNext`, `MovePrevious`, `MoveFirst` и `MoveLast` (мы не обсуждали их ранее). Аналогичным же образом объект `CollectionView` имеет методы `MoveCurrentToNext`, `MoveCurrentToPrevious`, `MoveCurrentToFirst` и `MoveCurrentToLast`. Вам может понадобиться добавить еще три кнопки для работы с тремя оставшимися операциями навигации.

Теперь запустите приложение! Вы должны получить полное представление о всех категориях продуктов и каждой их подкатегории. Если ваш код не заработал с первого раза, посмотрите форму `Main.xaml` в проекте `Chapter11WPF`. Эта форма не только полностью функциональна, она имеет также красивые графические элементы WPF и кнопки загрузки данных (сначала никакие данные не загружены) не только из EF, но также из L2S и типизированного `DataSet`.

## Грандиозный финал: Silverlight

Помните, Silverlight — это кроссплатформенная версия WPF. Поэтому мы можем сразу же погрузиться в разметку (это тоже XAML) версии Silverlight нашего приложения WPF (которую мы создали так: взяли XAML версии WPF, скопировали его и модифицировали). Однако в этой версии есть несколько важных отличий. Во-первых, в Silverlight нет возможности локального доступа (ни ADO.NET, ни EF, ни L2S), поэтому нам необходимо создать определения локальных классов для `ProductCategory` и `ProductSubcategory`, а также XML-файл с фактическими данными. Это означает, что нам понадобится использовать LINQ для XML в нашей объектной модели, а уже затем



мы будем к ней привязываться. Нам придется также по-другому работать с навигацией (поскольку нет объекта `CollectionView`, с которым можно было бы работать).

### ПРИМЕЧАНИЕ

Загрузка данных в Silverlight из службы ADO.NET Data Services и связывание с ними возможны (даже при том, что загрузка из EF невозможна). Форма `Main.xaml` проекта `Chapter11Silverlight` в примерах кодов данной главы загружает данные при помощи LINQ для XML и ADO.NET Data Services. Здесь мы показываем код LINQ для XML. Подробности по ADO.NET Data Services смотрите в примере приложения. Для того чтобы вызов служб ADO.NET Data Services работал правильно, вы должны начать проект Silverlight с запуска тестового проекта `Chapter11SilverlightWeb` (а не запускать сразу проект `Chapter11Silverlight`).

Откройте форму `Main.xaml` проекта `Chapter11Silverlight` и взгляните на нее. Давайте начнем с разметки, фрагмент которой показан в листинге 11.5.

#### Листинг 11.5. Фрагмент XAML нашей страницы Silverlight

```
<TextBox Text="{Binding ProductCategoryID}" Margin="153,166.157,0,0"
    Foreground="#FFFFFFD5" Background="Transparent" Height="24.843"
    VerticalAlignment="Top" Opacity="1" HorizontalAlignment="Left"
    Width="167"/>
<TextBox Text="{Binding Name}" Margin="153,200.493,0,0"
    Foreground="#FFFFFFD5" Background="Transparent" Height="27.507"
    VerticalAlignment="Top" Opacity="1" HorizontalAlignment="Left"
    Width="167"/>
<TextBox Text="{Binding ModifiedDate}" Margin="153,0,0,214.507"
    Foreground="#FFFFFFD5" Background="Transparent" Height="24.843"
    VerticalAlignment="Bottom" Opacity="1" HorizontalAlignment="Left"
    Width="167"/>
<my:DataGrid Name="dgSubcategories" Margin="353,164,10,194" ItemsSource=
    "{Binding ProductSubcategory}" AutoGenerateColumns="True" />
```

Обратите внимание, что объект `Binding` не имеет атрибута `Path`. Вместо этого название свойства привязанного источника просто вставляется в определение объекта. У нас в Silverlight есть также элемент управления `DataGrid`, поэтому мы используем его вместо `ListBox` (и таким образом шаблон данных не нужен). Объекта `DataProvider` также нет, Silverlight его не поддерживает.

Итак, обратимся к коду. В листинге 11.6 показан код загрузки данных для страницы Silverlight.

**Листинг 11.6. Код загрузки данных нашей страницы Silverlight (это LINQ для XML)**

```
RecordNumber = 0;
Results = from r in
    XElement.Load("AW2008.xml").Elements("ProductCategory")
select new ProductCategory
{
    ProductCategoryID = r.Element("ProductCategoryID").Value,
    Name = r.Element("Name").Value,
    ModifiedDate = r.Element("ModifiedDate").Value,
    ProductSubcategory = from rr in r.Elements("ProductSubcategory")
        select new ProductSubcategory
        {
            SubcatID = rr.Element("ProductSubcategoryID").Value,
            Name = (rr.Element("Name").Value)
        }
};
this.DataContext = Results.ElementAt(RecordNumber);
```

Обратите внимание на вложенный запрос LINQ для XML извлечения данных по `ProductCategory` и `ProductSubcategory` из файла `AW2008.xml`. Этот запрос ссылается на определения классов из файла классов `CategoryAndSubcategory.cs` проекта `Chapter11Silverlight`.

Запрос возвращает коллекцию `IEnumerable<ProductCategory>` в переменную `Results`, и мы настраиваем свойство `DataContext` объекта `Page` не на эту переменную, а на один из ее членов, который мы извлекаем при помощи ее метода `ElementAt`. После загрузки данных мы передаем этому методу переменную `RecordNumber`, которая инициализирована значением 0. Здесь не показан код навигационных кнопок, где мы делаем приращение переменной `RecordNumber` (для перехода к следующей записи), уменьшаем ее (для перехода к предыдущей записи), устанавливаем ее в 0 (для перехода к первой записи) или устанавливаем ее в значение `Results.Count() - 1` (для перехода к последней записи), а также обновляем свойство `DataContext` при помощи вызова метода `Results.ElementAt`.

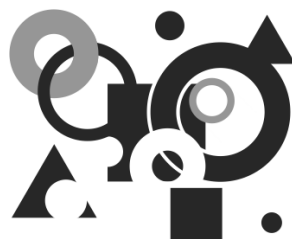
Вот и все. Запустите приложение, чтобы увидеть, как хорошо оно работает. Затем сделайте глубокий выдох, потому что мы закончили!

## Резюме

В этой главе вы увидели много моделей привязки и примеров — вполне достаточно для того, чтобы ваша голова закружилась. Мы знаем, что эта информация может вас запутать, особенно выданная в такой сжатой форме. Однако наша цель была — показать вам, что вне зависимости от выбранного вами варианта (будь это Windows Forms, ASP.NET, ASP.NET AJAX, ASP.NET Dynamic Data, WPF или Silverlight) привязка данных есть, и работает она концептуально схожим образом. Каждая модель привязки заимствует, по крайней мере, от одной другой модели, поэтому при совместном их изучении вы получаете тот максимум, который позволит вам понять их по отдельности.

Возможно, вы захотите перечитать эту главу и внимательно углубиться в примеры, модифицировать их для проверки различных сценариев и убедиться, что вы полностью поняли эти концепции. Если вы это сделаете, в награду вы получите понимание того, как нужно создавать ориентированные на данные приложения (для любой платформы приложений .NET). И кроме всего прочего, вы будете знать, как делать это при помощи элегантных и высокопродуктивных методик связывания данных (а не при помощи ручного кодирования).

## ГЛАВА 12



# Транзакции

*Обновлена Леонардом Лобелом и Кенном Скрибнером*

Разрабатывали вы когда-нибудь приложение с намерением написать ненадежный код? Конечно же, нет. Точно так же было бы совершенно абсурдно, если бы клиенту понравилась написанная вами система, которая полностью испортила его данные. Хватило бы у вас терпения общаться с представителем отдела по обслуживанию клиентов, который не может помочь вам потому, что информация в его компьютере неверна?

Какими бы абсурдными ни казались эти сценарии на первый взгляд, остается фактом то, что многие профессиональные разработчики слишком легко относятся к идее написания надежного программного обеспечения. Большую часть своих усилий они сосредоточивают на создании правильной логики и слишком мало внимания уделяют тому, чтобы эта логика правильно работала в любых обстоятельствах. В то же самое время все большее количество задач автоматизируется, что требует создания все более сложного программного обеспечения, реализовать которое становится возможным при помощи все более мощного аппаратного обеспечения. Точно так же, как мощность настольного компьютера (на котором вы сегодня работаете) в тысячи раз превышает мощность микропроцессоров 25-летней давности, так и сложность программного обеспечения с течением времени сильно выросла. Такое увеличение сложности требует гораздо более высокой надежности нашего кода.

Несмотря на это, большинство программистов продолжает писать ненадежное программное обеспечение. Подумайте о своей заработной плате, о том множестве компьютерных систем, через которые прошел ее расчет, а также о всех программистах, которые работали над его программным обеспечением. Что бы вы подумали, если бы небрежное программирование или недостаточ-

ное тестирование привели к уменьшению вашей зарплаты? Или если бы компьютерная ошибка случайно стерла ваш пенсионный счет?

Многие программисты предполагают, что их небрежность будет обнаружена при тестировании. Но редко удастся оттестировать приложение под надлежащей нагрузкой и воспроизвести различные одновременно происходящие события. Реальность такова, что большая часть проблем с программным обеспечением проявляется в производственной среде, а не при разработке или тестировании. Поэтому мы (разработчики) должны сосредоточиться на том, чтобы логика нашего приложения была правильной и вела себя одинаково во всех сценариях. Когда в логике приложения используются базы данных, то все операции логики и манипуляции данными должны обладать свойствами ACID (atomicity, consistency, isolation, durability): атомарность, целостность, изоляция и отказоустойчивость. Обычно это обеспечивается заключением операций баз данных в транзакции. Поэтому данную главу мы начнем с изучения транзакций.

## Что такое транзакция

Транзакция — это единая операция или набор операций, которые заканчиваются (либо успешно, либо с ошибкой) как единое целое, что обеспечивает целостность данных при возникновении непредвиденных обстоятельств. Возьмем, к примеру, финансовую транзакцию. Предположим, что вы покупаете автомобиль. Единая транзакция покупки автомобиля состоит из трех отдельных операций:

1. Вы выбираете автомобиль.
2. Вы платите за него.
3. Вы уезжаете на нем со стоянки.

Пропуск любого из этих шагов может принести массу проблем для вовлеченных в этот процесс участников. Это простой пример такого набора шагов, который должен всегда происходить совместно (целостным образом).

Транзакции позволяют нам обеспечить целостность наших данных при помощи четырех основных принципов. Эти принципы дают некий набор правил, которые должны быть выполнены для обеспечения успеха транзакции. Эти четыре принципа помогают обеспечить атомарность, целостность, изолированность и отказоустойчивость наших данных (независимо от успеха или неудачи транзакции). Давайте изучим эти свойства.

### ПРИМЕЧАНИЕ

Мы рассматриваем традиционные транзакции с двухфазной фиксацией "в стиле XA" (XA-style). С появлением Microsoft BizTalk и Web Services обычными стали длительные процессы, для которых был создан другой тип — *компенсированная транзакция*. Компенсированные транзакции в этой главе мы рассматривать не будем.

## Свойства ACID

Рассмотрим транзакцию получения с вашего банковского счета 100 долларов через банкомат. Данные для этой транзакции можно представить в базе данных в виде таблицы с двумя столбцами, AccountID и AccountBalance.

Для начала пусть баланс вашего счета будет равен 100 долларам, так что после снятия 100 долларов ваш обновленный баланс должен будет равняться нулю. С точки зрения логики, перед выдачей наличности система должна убедиться, что на вашем счету есть достаточно средств. Это означает, что в данной транзакции должны выполняться два запроса по базе данных. Первый запрос проверяет баланс счета:

```
SELECT AccountBalance FROM Account WHERE AccountID = @AccountID
```

Если запрос возвращает такое значение AccountBalance, которое превышает (или равно) запрошенной сумме, то вы можете снять наличные деньги.

После получения наличности вы должны обновить запись вашего счета обновленным значением баланса. Для этого вы выполняете запрос UPDATE:

```
UPDATE Account  
SET AccountBalance = AccountBalance - 100  
WHERE AccountID = @AccountID
```

Две операции этой транзакции — это два запроса к базе данных, которые поддерживают одну операцию снятия наличных денег. Обе они должны совместно (атомарно) закончиться успехом или неудачей. В противном случае транзакция будет считаться незавершенной. *Атомарность* — это первое свойство ACID.

Теперь давайте несколько изменим характер транзакции. Предположим, что исходный баланс счета равен 150 долларам, а в одной транзакции пользователь делает запрос на снятие 100 долларов и перевод 75 долларов на другой счет. Первый запрос обновления закончится успешно, при этом произойдет изменение баланса счета до 50 долларов. Но вторая операция закончится ошибкой (поскольку на счету недостаточно денег для перевода 75 долларов).

Следовательно, вам нужен способ отменить снятие наличности и вернуть базу данных в ее исходное состояние. Вы не можете оставить базу данных в промежуточном состоянии посередине транзакции, поскольку она находится в противоречивом состоянии. В реальном мире вы не станете заключать в одну транзакцию снятие со счета и перевод денег, но в данном случае мы хотели показать, как данные могут перейти (между операциями) в противоречивое состояние. Откат тех операций, которые не могут быть завершены, демонстрирует второе свойство ACID — *целостность*.

Теперь допустим, что операции снятия денег и перевода разделены в две разные транзакции (вместо одной), но выполняются одновременно. Каждая транзакция должна проверить текущий баланс при помощи запроса вроде следующего:

```
SELECT AccountBalance FROM Account WHERE AccountID = @AccountID
```

Если в вашей системе нет явных проверок для блокировки одновременных чтений, то обе транзакции получат одинаковый результат: 150 долларов. Таким образом, они обе решат, что на счету достаточно средств для транзакции. Одна транзакция выплатит 100 долларов, а другая переведет 75 долларов. Результатом станет общее снятие  $100 + 75 = 175$  долларов (несмотря на то, что имеется только 150 долларов). Во многих системах (особенно финансовых приложений) такие транзакции должны изолироваться друг от друга для предотвращения того, что называется "грязным чтением". Грязное чтение происходит тогда, когда данные читаются в некий переходный момент, и результат этого запроса не отражает истинное состояние данных в момент окончания текущей операции. Это приводит нас к третьему свойству ACID — *изоляция*.

Изоляция означает, что другие запрашивающие общий ресурс транзакции будут заблокированы. Такие блокировки серьезно влияют на время отклика вашего приложения. Вам часто будет необходимо смягчить эти блокировки. Вы можете сделать это при помощи использования уровней изоляции, которые мы обсудим далее в этой же главе.

И наконец, когда вы успешно завершили все операции внутри транзакции, вы не должны потерять все сделанные вами изменения. Иначе говоря, системные сбои не должны влиять на транзакционную целостность ваших операций. Это приводит нас к четвертому свойству ACID — *отказоустойчивости*. *Отказоустойчивость* означает, что участвующие в транзакции системы должны сохранить правильное состояние после транзакции даже в том случае, если с ними произойдет катастрофический сбой буквально через наносекунды после завершения транзакции. И наоборот, если транзакция не была завершена вследствие системного сбоя, то она не должна завершиться (или

отмениться) до сброса системы и перезапуска приложения. Шаги транзакции сохраняются, и приложение может продолжить операции с момента останова без отрицательных последствий (по крайней мере, с точки зрения целостности данных).

Давайте же резюмируем четыре свойства ACID.

- ❑ Атомарность — операции заканчиваются совместно (успешно или неудачно). Транзакция не может считаться завершенной при неудаче хотя бы одного ее шага.
- ❑ Целостность — операции оставляют базу данных в непротиворечивом состоянии. Транзакция переводит базу данных из одного стабильного состояния в другое, причем перед началом или после окончания транзакции не нарушаются никакие правила.
- ❑ Изоляция — транзакция не влияет ни на какую другую транзакцию, выполняющуюся в то же время.
- ❑ Отказоустойчивость — каждая транзакция сохраняется на надежном носителе, который не может быть изменен системными сбоями. Более того, если системный сбой происходит в середине транзакции, то для завершения транзакции необходимо либо отменить выполненные шаги, либо выполнить невыполненные шаги. Обычно это достигается путем использования журнала, который можно "воспроизвести" для возврата системы в целостное состояние.

### **ВАЖНО**

В нашем обсуждении свойств ACID мы вскользь затронули две других важных концепции. Первая относится к реальному процессу снятия наличных денег как части транзакции. То есть транзакции не обязательно ограничиваются одними только объектами баз данных. Далее в этой главе мы изучим и более сложные транзакции (при обсуждении пространства имен `System.Transactions` в Microsoft .NET Framework). Вторая относится к уровням изоляции, которые позволяют подстраивать поведение изоляции под ваши потребности (и это мы также обсудим далее в этой же главе).

Транзакция может работать с единственным ресурсом (база данных) либо со многими ресурсами (несколько баз данных или очередей сообщений). Транзакции с одним ресурсом называются локальными, а транзакции с множеством ресурсов называются распределенными. Сначала мы сосредоточимся на поддержке в Microsoft SQL Server 2008 локальных транзакций, а позднее (в этой же главе) поговорим и о распределенных транзакциях.



## Поддержка локальных транзакций в SQL Server 2008

SQL Server как ядро промышленной базы данных предоставляет встроенную поддержку, которая позволяет вам заключать в транзакцию один или более запросов. Локальные транзакции (которые работают только с одной физической базой данных) действуют в одном из четырех режимов транзакций:

- Autocommit — автоматической фиксации;
- Explicit — транзакции явным образом;
- Implicit — неявных транзакций;
- Batch-scoped — пакетных транзакций.

Давайте изучим по очереди все эти режимы.

### Режим транзакций Autocommit

Режим транзакций с автоматической фиксацией является режимом транзакций по умолчанию. В этом режиме SQL Server обеспечивает неприкосновенность данных во время всего выполнения запроса, независимо от того, указали вы транзакцию или нет. Например, если вы выполняете запрос `SELECT`, то данные во время выполнения запроса не будут меняться. С другой стороны, если вы выполняете запрос языка манипулирования данными DML (запросы `UPDATE`, `INSERT` или `DELETE`), то данные будут автоматически фиксироваться при отсутствии ошибок или откатываться (отменяться) в противном случае. Выполнение отдельного запроса DML никогда не приведет к частичной модификации записей. Два примечательных исключения из этого правила — это рекурсивные обобщенные табличные выражения (СТЕ), для которых все возвращаемые данные заранее не блокируются, а также те ситуации, когда вы явно требуете отменить транзакционную неприкосновенность.

### Режим транзакций Explicit

Режим автоматической фиксации транзакций позволяет вам выполнять отдельные запросы, однако вам часто бывает нужно выполнить в одной транзакции несколько запросов. В этом случае вы используете явные транзакции. В режиме транзакций `Explicit` вы объявляете границы транзакции явным образом. Иначе говоря, вы точно указываете места начала и конца транзакции.

SQL Server продолжает работать в режиме автоматической фиксации транзакций до тех пор, пока вы не потребуете исключения из этого правила, так что если вы хотите выполнить единым пакетом несколько инструкций Transact-SQL (T-SQL), то вам нужно будет использовать режим явных транзакций.

Начало транзакции вы указываете при помощи инструкции `BEGIN TRANSACTION`. После вызова `BEGIN TRANSACTION` ядро базы данных SQL Server пытается включить все последующие операции в одну транзакцию. Инструкция `BEGIN TRANSACTION` имеет следующий синтаксис:

```
BEGIN { TRAN | TRANSACTION }  
    [ { transaction_name | @tran_name_variable }  
      [ WITH MARK [ 'description' ] ] ]  
]
```

В этой инструкции при помощи `transaction_name` или `@tran_name_variable` вы можете указать название транзакции. Вы можете также пометить транзакцию в журнале транзакций (указав ее описание). Это полезно тогда, когда вы хотите восстановить базу данных до именованной отметки. (Дополнительную информацию по этой функциональной возможности смотрите в документации SQL Server Books Online).

Предположим, что вы вызвали `BEGIN TRANSACTION` и радостно выполняете несколько операций DML. Когда вы закончите, вы захотите завершить вашу транзакцию либо сохранением (фиксацией) ваших изменений, либо их отменой (откатом) в случае ошибки. Если вы хотите зафиксировать изменения, то вы выполняете инструкцию `COMMIT TRANSACTION` со следующим синтаксисом:

```
COMMIT { TRAN | TRANSACTION } [ transaction_name | @tran_name_variable ] ]
```

Вот пример, в котором две инструкции DML (`UPDATE` и `INSERT`) обернуты в единую явную транзакцию:

```
BEGIN TRANSACTION  
    UPDATE Table1 SET Column1 = 'One'  
    INSERT INTO Table2 (Column2) VALUES ('Two')  
COMMIT
```

SQL Server следит за количеством активных транзакций. Вы можете получить текущее количество транзакций при помощи функции `@@TRANCOUNT`. При каждом вызове `BEGIN TRANSACTION` значение `@@TRANCOUNT` увеличивается на единицу. Вы можете также вызывать `BEGIN TRANSACTION` внутри другого блока `BEGIN TRANSACTION` (что также приведет к увеличению значения `@@TRANCOUNT`). Соответственно при каждом вызове `COMMIT TRANSACTION` сер-

вер SQL Server уменьшает значение @@TRANCOUNT на единицу. Транзакция остается активной до тех пор, пока значение @@TRANCOUNT не уменьшится до нуля. Когда вы вызываете BEGIN TRANSACTION внутри блока транзакции, то вы по существу создаете *вложенную транзакцию*. Но все это не так просто. Перед тем как вы сможете понять суть вложенных транзакций SQL Server 2008, мы должны рассмотреть такой сценарий, при котором вы хотите отменить изменения (не сохранять их) при возникновении ошибки внутри транзакции.

Если вы не хотите, чтобы изменения стали постоянными, а планируете восстановить базу данных к ее предыдущему состоянию, то вы можете откатить изменения при помощи инструкции T-SQL с названием ROLLBACK TRANSACTION, имеющей следующий синтаксис:

```
ROLLBACK { TRAN | TRANSACTION }  
    [ transaction_name | @tran_name_variable  
    | savepoint_name | @savepoint_variable ]
```

Инструкция ROLLBACK является противоположностью инструкции COMMIT. Вместо сохранения она отменяет все изменения, выполненные в транзакции. Важно понять, что SQL Server никогда не предполагает инструкции COMMIT. Если вы отключитесь от SQL Server без явной выдачи инструкции COMMIT, то SQL Server выберет ROLLBACK. Однако (в качестве хорошей практики) вы никогда не должны оставлять это решение серверу SQL Server. Вы должны явным образом указать SQL Server, какая из двух операций вам нужна. Причина в том, что в любом интерфейсе прикладного программирования (API) приложений (для доступа к данным) реализован пул подключений. Даже если вы закрываете подключение (что автоматически вызовет откат в том случае, когда вы не сделали фиксацию транзакции), то для физического закрытия подключения такому API, как Microsoft ADO.NET, может потребоваться некоторое время. SQL Server будет вынужден поддерживать транзакцию в состоянии выполнения и может при этом блокировать ценные ресурсы дольше, чем этого можно было бы ожидать.

Если при выполнении транзакции происходит серьезная ошибка, то SQL Server откатывает транзакцию. К сожалению, SQL Server не дает четкого определения серьезной ошибки. Ошибка с уровнем серьезности 11 или выше останавливает выполнение текущего пакета и откатывает транзакцию. Ошибки с уровнем серьезности 19 (и более) идут еще дальше и закрывают подключение. Какие из них более серьезны? В обоих случаях выполнение пакета останавливается в некотором неопределенном состоянии. Вследствие такой неоднозначности при возникновении ошибки хорошей идеей будет явно вызвать операцию ROLLBACK.

## СОВЕТ

Всегда выполняйте операцию `ROLLBACK` явным образом и никогда не полагайтесь на то, что API или SQL Server вызовут откат за вас.

Обратите также внимание, что операция `COMMIT TRANSACTION` уменьшает значение `@@TRANCOUNT` на 1, но операция `ROLLBACK TRANSACTION` всегда уменьшает `@@TRANCOUNT` до 0. Что это означает для вложенных транзакций? Сейчас мы это и рассмотрим.

## Вложенные транзакции

Что же означают все эти разговоры об `@@TRANCOUNT` и вложенных транзакциях на практике? Давайте рассмотрим следующий фрагмент кода:

```
BEGIN TRANSACTION OUTERTRAN
  INSERT INTO TEST (TestColumn) VALUES (1)
  BEGIN TRANSACTION INNERTRAN
    INSERT INTO TEST (TestColumn) VALUES (2)
  COMMIT TRANSACTION INNERTRAN
ROLLBACK
```

Интересно, что когда вы выполняете этот код (предположим, что таблица `TEST` существует), то никакие строки в базу данных не вставляются. Это потому, что инструкция `ROLLBACK` автоматически откатывает всю транзакцию к самой внешней инструкции `BEGIN TRANSACTION`, уменьшая значение `@@TRANCOUNT` до нуля. Поэтому внешний `ROLLBACK` отменяет внутренний `COMMIT` — несмотря даже на то, что `COMMIT` был вызван перед `ROLLBACK`. Это тонкий, но очень важный нюанс обработки транзакций, который специфичен для SQL Server.

## Точки сохранения

Еще одна имеющая отношение к транзакциям концепция — это *точки сохранения*. Точка сохранения позволяет вам временно сохранять часть транзакции, которую можно откатить вместо всей транзакции. Она определяется при помощи инструкции `SAVE TRANSACTION`, имеющей следующий синтаксис:

```
SAVE { TRAN | TRANSACTION } { savepoint_name | @savepoint_variable }
```

При помощи вызова (во время транзакции) инструкции `SAVE TRANSACTION` вы помечаете точку в транзакции, до которой вы можете откатиться, чтобы не терять все.

Рассмотрим следующий код:

```
BEGIN TRANSACTION
  INSERT INTO TEST (TestColumn) VALUES (1)
  SAVE TRANSACTION SAVEPOINT1
  INSERT INTO TEST (TestColumn) VALUES (2)
  ROLLBACK TRANSACTION SAVEPOINT1
COMMIT
```

Этот код вставляет строку и устанавливает точку сохранения с названием `SAVEPOINT1`. Затем он выполняет еще одну вставку, но поскольку вы установили точку сохранения до этой второй вставки, то можете откатиться до точки сохранения без потери вашей первой вставки. Как вы, вероятно, догадались, в результате этого блока кода будет вставлена только одна строка с `TestColumn = 1`.

Для того чтобы лучше понять, мы предлагаем вам поэкспериментировать с различными комбинациями `BEGIN TRANSACTION`, `SAVE`, `COMMIT`, `ROLLBACK` и `@@TRANCOUNT` в тестовой таблице вашей базы данных.

## Режим транзакций Implicit

Когда вы подключаетесь к базе данных при помощи SQL Server Management Studio и затем выполняете запрос DML, то все изменения сохраняются автоматически. Так происходит потому, что по умолчанию подключение работает в режиме автоматической фиксации транзакций. Если вы не хотите фиксировать изменения без явного запроса на фиксацию, то настройте подключение в режим неявных транзакций. Вы можете настроить подключение в режим неявных транзакций (или отменить этот режим) при помощи вызова инструкции T-SQL:

```
SET IMPLICIT_TRANSACTIONS {ON | OFF}
```

Когда подключение настроено в режим неявных транзакций (и оно не находится в состоянии транзакции), то транзакция запускается автоматически при выдаче следующих инструкций: `ALTER TABLE`, `CREATE`, `DELETE`, `DROP`, `FETCH`, `GRANT`, `INSERT`, `OPEN`, `REVOKE`, `SELECT`, `TRUNCATE TABLE` или `UPDATE`.

В этом режиме транзакции не фиксируются и не откатываются без вашего явного запроса. Это означает, что если вы выдадите инструкцию `UPDATE`, то SQL Server будет держать блокировку на соответствующих данных до тех пор, пока вы не выдадите `COMMIT` или `ROLLBACK`. Если вы не выдадите `COMMIT` или `ROLLBACK`, то транзакция будет откатена при отключении пользователя.

### **ПРИМЕЧАНИЕ**

Не смущайтесь термином "неявная"; он просто означает, что транзакция начинается неявно (без явной инструкции `BEGIN TRANSACTION`). В итоге вам будет необходимо впоследствии явным образом зафиксировать транзакцию для сохранения изменений.

С практической точки зрения, вам следует избегать настройки подключения на использование неявных транзакций для баз данных с высоким параллелизмом. Например, если при администрировании базы данных вы неявным образом запустите транзакции путем выдачи запросов в интерактивном режиме через SQL Server Management Studio и приведете к блокировке ресурсов базы, то это может серьезно повлиять на производительность всей системы в целом. Примером правильного использования режима неявных транзакций является хранилище данных, где для отчетов нужен доступ к данным только на чтение, причем они могут выполняться с различными уровнями изоляции, которые помогают избегать блокировок (скоро мы это обсудим), и требуется большая осторожность для того, чтобы нечаянно не модифицировать данные.

## **Пакетный режим транзакций**

Подключения SQL Server 2005 и 2008 могут поддерживать несколько активных результирующих наборов (multiple active result sets, MARS) на одном подключении. Обратите внимание, что мы сказали "несколько активных результатов", а не "параллельное выполнение команд". Выполнение команд по-прежнему чередуется, со строгими правилами, которые управляют тем, какие инструкции могут накладываться на другие инструкции.

Подключения с использованием MARS имеют соответствующую среду пакетного выполнения. Эта среда содержит различные компоненты (опции SET, контекст безопасности, контекст базы данных и переменные состояния выполнения), определяющие среду, в которой выполняются команды. Когда включен MARS, вы можете иметь несколько одновременно выполняющихся пакетов, поэтому все изменения среды выполнения имеют область действия в конкретном пакете (до его завершения). По завершении пакета настройки среды выполнения копируются в среду по умолчанию.

Таким образом подключение использует режим пакетных транзакций в том случае, когда оно выполняет транзакцию, включен MARS, и одновременно выполняются несколько чередующихся пакетов.

## MARS и транзакции

MARS не позволяет вам иметь несколько транзакций на одном подключении (только несколько активных результирующих наборов). Транзакции и MARS — это интересное сочетание, и для понимания работы транзакций в MARS вы должны сначала понять правила чередования команд.

В MARS команда, которая читает результаты (`SELECT`, `FETCH` или `READTEXT`), может (в общем случае) свободно чередоваться или прерываться командой, которая пытается модифицировать данные (`UPDATE` или `INSERT`). Таким образом, операция записи может блокировать операцию чтения, но операция чтения не может блокировать операцию записи. Операции чтения продолжают выполняться после завершения операции записи. Если же одновременно возникают две операции записи, то их выполнение упорядочивается. Помните, что выполнение команд в SQL Server всегда последовательное и никогда не бывает параллельным (даже в многопоточных средах). И наконец, инструкции `BULK INSERT` блокируют все прочие операции чтения и записи.

Поэтому теоретически `BULK INSERT` будет блокировать `INSERT`, `UPDATE` и `DELETE`, которые в свою очередь будут блокировать все операции чтения. Однако проблема состоит в том, что в большинстве практических сценариев вы не можете точно предсказать, какая именно команда реально блокирует другую команду. Это потому, что ваша операция чтения могла закончиться до прерывания операцией записи. Вы не можете также точно предсказать момент окончания операции чтения и момент начала операции записи (поскольку операция чтения зависит от множества факторов: скорости процессора, скорости обмена по сети, размера пакетов, сетевого трафика и т. д.). Таким образом, невозможно предсказать, не была ли перед операцией записи начата операция чтения.

В смысле транзакций это означает, что если вы выполняете транзакцию, вставляющую строку, которая в свою очередь запускает триггер, а в триггере есть инструкция `SELECT`, то правила чередования MARS приведут к блокировке инструкции `SELECT` вашего триггера исходной инструкцией `INSERT`. Причем это поведение невозможно достаточно надежно предсказать (из-за разного времени выполнения команд). Если такое происходит, то монитор взаимоблокировок SQL Server обнаружит эту ситуацию и приведет к завершению инструкции `SELECT` с ошибкой. В результате вы можете получить систему, которая работает на компьютере разработчика (нагрузка на котором низкая), но будет давать сбои в производственных условиях (где нагрузка на систему выше). Мы знаем, что проблемы такого типа устранить тяжелее всего. Таким образом, при использовании MARS в вашем проекте вы должны учитывать правила чередования и наличие нескольких пакетов.

## MARS и точки сохранения

Поскольку в одной транзакции у нас работает множество чередующихся команд, то создающие точки сохранения команды могут легко перепутаться. Представьте себе такую ситуацию, когда два чередующихся пакета выдают откат к именованной точке, и совершенно случайно имена этих точек оказываются одинаковыми. Вы не можете предсказать, какой откат произошел раньше, и к какой точке сохранения должен теперь откатиться SQL Server. Кроме того, поскольку вы не можете точно предсказать, какая именно инструкция закончила чередование с другой инструкцией, то вы не можете быть уверенными в том, что точка сохранения вообще была создана до того, как вы выдали откат к ней.

По этим причинам (при выполнении нескольких упорядоченных команд) MARS позволит вам установить точку сохранения, но после начала чередования команд любой запрос `BEGIN TRANSACTION` закончится ошибкой. Поскольку вы не можете предсказать точный порядок чередования команд, то не можете знать наверняка и то, как закончится инструкция `BEGIN TRANSACTION`. С учетом такого непредсказуемого поведения на подключении MARS лучше воздерживаться от использования точек сохранения.

## Использование локальных транзакций в ADO.NET

В ADO.NET команды могут выполняться по источнику данных с использованием класса, который наследует от `DbCommand`. Для SQL Server это объект `SqlCommand`. Предположим, есть таблица, определенная следующим образом:

```
CREATE TABLE TestTable
(
    TestID INT IDENTITY PRIMARY KEY,
    TestColumn INT
)
```

Теперь нужно выполнить для нее эту простую команду `INSERT`:

```
INSERT INTO TestTable (TestColumn) VALUES (100)
```

Вы можете легко заключить эту команду T-SQL в экземпляр `SqlCommand` (как показано в листинге 12.1).

### Листинг 12.1. Использование `SqlCommand` для выполнения простой команды T-SQL

```
using (SqlConnection conn = new SqlConnection(connectionString))
{
```



```
SqlCommand cmd = conn.CreateCommand();
cmd.CommandText = "INSERT INTO TestTable (TestColumn) VALUES (100)";
try
{
    conn.Open();
    cmd.ExecuteNonQuery();
}
finally
{
    conn.Close();
}
}
```

Метод `ExecuteNonQuery` выполняет указанную в свойстве `CommandText` команду по базе данных, указанной в свойстве строки подключения.

Теперь рассмотрим следующий блок кода T-SQL, который содержит два запроса к базе данных в одной транзакции:

```
BEGIN TRANSACTION
    INSERT INTO TestTable (TestColumn) VALUES (100)
    INSERT INTO TestTable (TestColumn) VALUES (200)
COMMIT
```

Этот блок кода фактически состоит из четырех отдельных команд T-SQL. Можно было бы просто вызвать все четыре строки кода как отдельные тексты команд объекта `SqlCommand`. Но это не очень элегантное решение. Хорошей практикой было бы обеспечить выполнение транзакции (путем заключения всей инструкции в один объект `SqlCommand`). Нет гарантий, что при выполнении последующих команд вашему коду будет доступен тот же самый объект `SqlCommand`. Гораздо более удачной реализацией транзакционного кода будет использование двух экземпляров объекта `SqlCommand` и привязка их обоих к одной транзакции (как показано в листинге 12.2).

#### Листинг 12.2. Привязка двух объектов `SqlCommand` к одной транзакции

```
using (SqlConnection conn = new SqlConnection(connectionString))
{
    SqlCommand cmd1 = conn.CreateCommand();
    cmd1.CommandText = "INSERT INTO TestTable (TestColumn) VALUES (100)";
    SqlCommand cmd2 = conn.CreateCommand();
    cmd2.CommandText = "INSERT INTO TestTable (TestColumn) VALUES (200)";
```

```
SqlTransaction tran = null;
try
{
    conn.Open();
    tran = conn.BeginTransaction();
    cmd1.Transaction = tran;
    cmd2.Transaction = tran;
    cmd1.ExecuteNonQuery();
    cmd2.ExecuteNonQuery();
    tran.Commit();
}
catch
{
    if (tran != null)
    {
        tran.Rollback();
    }
    throw;
}
finally
{
    conn.Close();
}
}
```

Как видите, код листинга 12.2 чрезвычайно похож на уже виденный вами ранее код без транзакций. Однако большое отличие состоит в том, что как только мы открываем подключение, мы начинаем на нем транзакцию (при помощи вызова метода `BeginTransaction` объекта подключения). Этот метод возвращает объект `SqlTransaction`, присваиваемый свойству `Transaction` всех объектов `SqlCommand`, которые должны будут выполняться в той же самой транзакции. Кроме того, мы используем конструкцию `try/catch/finally` для обеспечения фиксации в случае успеха и отката в случае неудачи.

Следует отметить, что перед вызовом `BeginTransaction` ваше подключение должно быть открыто, а после вызова `BeginTransaction` все последующие команды (до фиксации или отката) должны находиться в пределах одной транзакции. Таким образом, ADO.NET несколько дисциплинирует вас (чтобы вы писали более четкий код).

## Терминология транзакций

Перед тем как мы выйдем за пределы изложения основ транзакций, давайте сделаем обзор терминологии, которая часто используется при обсуждении транзакций:

- начало транзакции — указание, что все последующие операции подразумеваются находящимися внутри транзакции;
- откат транзакции — отмена операций, которые произошли после начала транзакции, при этом происходит восстановление данных в их исходное состояние. Это делается в случае неудачи;
- фиксация транзакции — сохранение всех операций, выполненных после начала транзакции. Транзакция фиксируется в случае успешного завершения;
- грязное чтение — операция чтения незафиксированных данных. Например, такое происходит тогда, когда транзакция *B* блокируется транзакцией *A*, но поскольку вы изменили поведение изоляции, то транзакция *B* читает изменения транзакции *A* даже несмотря на то, что они еще не зафиксированы;
- неповторяемое чтение — такое условие, когда транзакция *B* во время жизни транзакции *A* модифицирует те данные, с которыми работала транзакция *A*. В результате транзакция *A* читает модифицированные данные, и повторить исходное чтение невозможно;
- фантомное чтение — подобно неповторяемому чтению, за исключением того, что между двумя чтениями в одной и той же транзакции меняется количество строк. Строки, которые отличаются между двумя чтениями, называются *фантомными строками*.

## Уровни изоляции

Как уже упоминалось ранее в этой главе, поведение изоляции при транзакции можно подстроить под потребности вашего приложения. Обычно это делается при помощи настройки уровня изоляции транзакции. Проще говоря, уровни изоляции определяют то, как ведут себя параллельные транзакции. Блокируют ли они друг друга? Пропускают ли они друг друга? Или они предоставляют моментальный снимок предыдущего стабильного состояния данных в случае возникновения состояния пропуска?

Вы можете обнаружить некоторые незначительные несоответствия между уровнями изоляции, определенными в ADO.NET, и уровнями изоляции в SQL Server 2008, поскольку система ADO.NET была написана не только для SQL Server. Мы начнем с рассмотрения поддержки уровней изоляции в SQL Server 2008.

## Уровни изоляции в SQL Server 2008

Вы можете настроить уровни изоляции в SQL Server при помощи инструкции SET TRANSACTION ISOLATION LEVEL, имеющей следующий синтаксис:

```
SET TRANSACTION ISOLATION LEVEL
```

```
{ READ UNCOMMITTED
| READ COMMITTED
| REPEATABLE READ
| SNAPSHOT
| SERIALIZABLE
}
```

Далее показан простой случай использования этой инструкции совместно с инструкцией BEGIN TRANSACTION:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
BEGIN TRANSACTION
    SELECT TestColumn FROM TestTable
COMMIT
```

Как вы видите из синтаксиса, SQL Server 2008 поддерживает пять уровней изоляции:

- ☐ Read Uncommitted;
- ☐ Read Committed;
- ☐ Repeatable Read;
- ☐ Snapshot;
- ☐ Serializable.

### Уровень изоляции Read Uncommitted

При указании этого уровня изоляции вы по существу даете базе данных указание нарушать все блокировки и читать текущее моментальное состояние данных. Но при этом вы можете получить грязное чтение — прочитать еще

незафиксированные данные. Поэтому вам следует избегать этого уровня изоляции тогда, когда ваше приложение требует точных и зафиксированных данных (поскольку использующие этот уровень изоляции транзакции могут возвращать логически неверные данные).

Давайте изучим этот уровень изоляции при помощи примера:

1. Откройте два экземпляра SQL Server Management Studio. В каждом экземпляре используйте базу данных, которая содержит таблицу `TestTable`, созданную ранее в этой же главе. Эти два экземпляра будут использоваться для имитации двух пользователей, выполняющих две параллельные транзакции.

2. В экземпляре 1 выполните команду `UPDATE` для строки данных при помощи следующего блока кода:

```
BEGIN TRANSACTION
```

```
UPDATE TestTable SET TestColumn=200 WHERE TestId=1
```

3. В экземпляре 2 выполните следующий запрос:

```
SELECT TestColumn FROM TestTable WHERE TestId=1
```

Вы заметите, что ваш запрос `SELECT` заблокирован. Это понятно, так как вы пытаетесь читать те же самые данные, которые модифицирует экземпляр 1. Если экземпляр 1 не выдаст `COMMIT` или `ROLLBACK`, то ваш запрос останется заблокированным или просто завершится по истечении времени.

4. Отмените ваш заблокированный запрос комбинацией клавиш `<Alt>+<Break>` или путем нажатия кнопки **Cancel** на панели инструментов. Выполните следующую инструкцию для установки такого уровня изоляции вашего запроса `SELECT`, который позволит читать нефиксированные данные по подключению экземпляра 2:

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED
```

5. Выполните запрос `SELECT` еще раз:

```
SELECT TestColumn FROM TestTable WHERE TestId=1
```

Вы увидите, что запрос не блокируется, он выдает в качестве результата число 200.

6. Вернитесь в экземпляр 1 и выдайте `ROLLBACK`.
7. В экземпляре 2 выполните тот же самый запрос `SELECT` еще раз. В результате вы должны будете получить 100.

Как вы заметили, экземпляр 2 в разные моменты времени возвращал разные результаты для одного и того же запроса. Фактически значение 200 никогда не было зафиксировано в базе данных, но поскольку вы явным образом за-

просили грязное чтение (указав уровень изоляции Read Uncommitted), то вы в результате прочитали те данные, которые не должны были стать окончательными. Минус: в итоге вы прочитали логически неверные данные. Плюс: ваш запрос не блокировался.

## Уровень изоляции Read Committed

Этот уровень изоляции является уровнем по умолчанию в SQL Server потому, что он дает наилучший баланс целостности и производительности. Этот уровень изоляции соблюдает блокировки и предотвращает грязное чтение. В предыдущем примере до того момента, как мы явно потребовали изменить уровень изоляции на Read Uncommitted, подключение работало на уровне изоляции Read Committed. Поэтому наша вторая транзакция (транзакция с автоматической фиксацией в запросе `SELECT`) была заблокирована транзакцией, выполнявшей запрос `UPDATE`.

Уровень изоляции Read Committed предотвращает грязное чтение, но фантомное чтение и неповторяемое чтение при этом уровне изоляции по-прежнему возможны. Это происходит потому, что данный уровень изоляции не предотвращает изменение данных одной транзакцией во время чтения их другой.

Фантомное чтение может произойти в такой ситуации:

1. Начинается транзакция 1.
2. Транзакция 1 читает строку.
3. Начинается транзакция 2.
4. Транзакция 2 удаляет ту строку, которая была прочитана транзакцией 1.
5. Транзакция 2 фиксируется. Транзакция 1 не может повторить свое первоначальное чтение (поскольку строка больше не существует), что приводит к фантомной строке.

Неповторяемое чтение может произойти в следующей ситуации:

1. Начинается транзакция 1.
2. Транзакция 1 читает строку.
3. Начинается транзакция 2.
4. Транзакция 2 изменяет ту строку, которая была прочитана транзакцией 1.
5. Транзакция 2 фиксируется.

6. Транзакция 1 читает строку снова. Транзакция 1 имеет противоречивые данные, поскольку теперь строка содержит другие значения (отличающиеся от предыдущего чтения).

## Уровень изоляции Repeatable Read

В полном соответствии со своим названием, данный уровень изоляции предотвращает неповторяемое чтение. Он делает это при помощи блокирования данных, которые использовались в запросе внутри транзакции. Как и следует ожидать, за это вы платите более высокую цену (параллельные транзакции блокируют друг друга), поэтому данный уровень изоляции следует использовать только при необходимости. Есть и хорошая новость: параллельная транзакция может добавить новые данные, которые удовлетворяют выражению `WHERE` исходной транзакции. Это происходит потому, что первая транзакция блокирует только те строки, которые она первоначально прочитала в свой результирующий набор. Иначе говоря, транзакция с данным уровнем изоляции получает блокировки всех извлеченных данных, но не получает блокировки диапазонов.

Если вы внимательно изучите этот шаблон, то вы увидите, что хотя неповторяемые чтения при использовании этого уровня изоляции и предотвращаются, фантомные чтения все-таки могут происходить. Они могут происходить при следующих условиях:

1. Начинается транзакция 1.
2. Транзакция 1 читает все строки с `TestColumn=100`.
3. Начинается транзакция 2.
4. Транзакция 2 вставляет новую строку с `TestID = 2, TestColumn = 100`.
5. Транзакция 2 фиксируется.
6. Транзакция 1 выполняет запрос `UPDATE` и модифицирует `TestColumn` для строк, в которых `TestColumn=100`. При этом модифицируется также и та строка, которая была вставлена транзакцией 2.
7. Транзакция 1 фиксируется.

Поскольку совместно используемые блокировки не снимаются до конца транзакции, то параллельность здесь ниже, чем при использовании уровня изоляции `Read Committed`, поэтому во избежание получения неожиданных результатов следует соблюдать осторожность.

## Уровень изоляции Serializable

Транзакция, которая выполняется на уровне изоляции Serializable, не допускает грязных, фантомных или неповторяемых чтений. Этот уровень изоляции ставит самые строгие блокировки на читаемые или модифицируемые данные, сохраняя чистоту ваших данных. Это может показаться безупречным уровнем изоляции, но есть веская причина, по которой его не следует использовать часто. В некотором смысле получается безукоризненная транзакция, но транзакции будут блокировать другие выполняющиеся транзакции, что будет снижать производительность параллельного выполнения и даже создавать взаимоблокировки. Таким образом, даже если такая транзакция и сохранит чистоту ваших данных, она будет очень серьезно снижать производительность системы. В большинстве практических ситуаций вы сможете обойтись более низким уровнем изоляции.

## Уровень изоляции Snapshot

Во всех уже обсуждавшихся уровнях изоляции у нас возникал компромисс между логической неприкосновенностью данных и производительностью параллельного выполнения. Поскольку транзакция блокирует те данные, с которыми она работает, то другие транзакции (которые пытаются работать с теми же самыми данными) блокируются до момента фиксации или отката первой транзакции. Конечно, традиционным способом обойти эту проблему является разрешение грязных чтений (и следовательно, получение неверных данных) или просто снижение продолжительности транзакций. Но ни одно из этих решений не позволяет вам читать логически целостные данные при неблокируемом параллельном выполнении.

Архитектуры приложений часто предлагают такие обстоятельства, в которых либо самые мелкие транзакции становятся проблемой, либо транзакции заканчиваются модификацией такого количества данных, что их продолжительность невозможно поддерживать на низком уровне. Для того чтобы обойти эту проблему, в SQL Server 2005 был введен новый уровень изоляции Snapshot (моментальный снимок). Этот уровень изоляции дает вам непротиворечивые чтения без блокировок.

Транзакции с таким уровнем изоляции не создают совмещаемых блокировок на читаемых строках. Кроме того, повторные запросы тех же самых данных внутри такой транзакции гарантируют получение одинаковых результатов. Таким образом обеспечивается повторяемость чтений без всяких блокировок. Это звучит как наилучший из возможных и невозможных вариантов — ско-



рость отклика Read Uncommitted в сочетании с целостностью Repeatable Read. Однако все это не бесплатно.

Такое неблокирующее поведение повторяемого чтения становится возможным путем сохранения ранее зафиксированных версий строк в базе данных tempdb. В результате другие транзакции, которые начались до записи в текущей транзакции и уже прочитали предыдущую версию, будут продолжать читать эту версию. Поскольку предыдущая версия читается из tempdb, то запись может происходить без блокировки, после чего другие транзакции увидят новую версию. Очевидная проблема здесь (конечно же) — накладные расходы на работу с базой данных tempdb. По этой причине перед использованием на SQL Server этого уровня изоляции его необходимо включить. Не следует произвольно включать уровень изоляции Snapshot на базах данных. Но если после тестирования вы принимаете решение, что вашей базе данных нужен этот уровень изоляции, то вы можете включить его при помощи следующей инструкции:

```
ALTER DATABASE MyDB  
    SET ALLOW_SNAPSHOT_ISOLATION ON
```

Как и со всеми остальными уровнями изоляции, после включения для базы данных уровня изоляции Snapshot вы можете использовать его на отдельных подключениях при помощи инструкции SET TRANSACTION ISOLATION LEVEL следующим образом:

```
SET TRANSACTION ISOLATION LEVEL SNAPSHOT
```

## Уровень изоляции Read Committed Snapshot

Изоляция уровня Snapshot предотвращает блокировку чтений записями (при этом для чтений предоставляется ранее зафиксированная версия). Таким образом в течение транзакции вам обеспечиваются повторяемые чтения. Однако такой метод обеспечения повторяемых чтений влечет за собой дополнительные издержки и учет со стороны ядра базы данных SQL Server, которые нужны не во всех ситуациях. Поэтому SQL Server предлагает небольшую модификацию уровня изоляции Read Committed и обеспечивает неповторяемые чтения в течение транзакции, которые не блокируются записями транзакций. Эта модификация называется уровнем изоляции Read Committed Snapshot. Данный уровень изоляции гарантирует целостность данных в течение запроса чтения внутри транзакции, но не в течение всей транзакции, внутри которой находится чтение. Очевидным преимуществом этого уровня изоляции по сравнению с Read Committed является то, что ваши чтения не блокируются. Когда они запрашивают данные, им предлагается либо предыдущее состоя-

ние данных (до операций записи), либо новое состояние данных (после операций записи) — в зависимости от состояния других параллельно выполняющихся транзакций, но им никогда не приходится ожидать снятия с запрошенных данных блокировок параллельных транзакций.

Для использования уровня изоляции Read Committed Snapshot вы должны предварительно включить его на уровне базы данных при помощи следующей команды T-SQL:

```
ALTER DATABASE MyDB  
SET READ_COMMITTED_SNAPSHOT ON
```

После включения этого уровня изоляции для базы данных все запросы с использованием уровня изоляции Read Committed будут демонстрировать поведение типа Snapshot. Несмотря на то, что этот уровень изоляции даст вам поведение типа Snapshot, вы не сможете выполнять повторяемые чтения во время транзакции.

### **ВАЖНО**

Ни уровень Snapshot, ни уровень Read Committed Snapshot не могут использоваться с базой данных, которая поддерживает новую группу файлов FILESTREAM в SQL Server 2008 (описана в *главе 8*). Причина в том, что база данных не может сделать моментальный снимок файловой системы. Если вы попытаетесь включить любой из этих уровней изоляции для базы данных, использующей FILESTREAM, то такая попытка закончится неудачей с выдачей ошибки.

## **Уровни изоляции в ADO.NET**

Как уже упоминалось ранее, уровни изоляции в ADO.NET несколько отличаются от уровней изоляции в SQL Server. Это потому, что ADO.NET, как универсальная технология доступа к данным, поддерживает не только SQL Server, но и другие базы данных. Вот уровни изоляции, которые определены в ADO.NET 2.0 в перечислении `System.Data.IsolationLevel`:

- ❑ `Chaos` — незавершенные изменения от трансляций с более высокими уровнями изоляции не могут быть перезаписаны. Этот уровень не поддерживается в SQL Server и в Oracle;
- ❑ `ReadUncommitted` — аналогичен уровню Read Uncommitted в SQL Server. Этот уровень означает, что совмещаемые блокировки не устанавливаются и монопольные блокировки не соблюдаются;

- ❑ `ReadCommitted` — так же, как и при `Read Committed` в SQL Server, при чтении данных транзакцией удерживаются совмещаемые блокировки. Это исключает грязные чтения, но вы по-прежнему можете получить неповторяемые и фантомные чтения;
- ❑ `RepeatableRead` — совмещаемые блокировки устанавливаются на все данные, которые используются в предикате (критерии) запроса. Как и при `Repeatable Read` в SQL Server, грязные и неповторяемые чтения исключены, но возможны фантомные чтения;
- ❑ `Snapshot` — моментальный снимок предыдущих данных обеспечивает повторяемые чтения с неблокируемыми выборками `SELECT`. Не путайте этот уровень с уровнем `Read Committed Snapshot` в SQL Server 2008 (который должен быть включен на уровне базы данных);
- ❑ `Serializable` — этот уровень можно считать идеальным типом транзакции, при котором на данные устанавливаются монопольные блокировки. Это предотвращает чтение или модификацию данных другими пользователями. Помните, что всегда приходится идти на компромиссы, поэтому монопольные блокировки не следует удерживать в течение длительного времени;
- ❑ `Unspecified` — это универсальный уровень изоляции для баз данных, поддерживающих такие уровни изоляции, которые не охватываются другими вариантами или для таких сценариев, в которых уровень изоляции точно определить невозможно.

Вы можете указать уровень изоляции для транзакции в виде параметра метода `BeginTransaction`. Например, в следующем фрагменте кода мы начинаем транзакцию с уровнем изоляции `ReadUncommitted`:

```
SqlConnection tran =  
    conn.BeginTransaction(IsolationLevel.ReadUncommitted) ;
```

В этом примере `SqlCommand` с установленным в значение `tran` свойством `Transaction` не будет соблюдать никаких монопольных блокировок и позволит вам выполнять грязные чтения тех данных, которые удерживаются другими транзакциями.

## Распределенные транзакции

До настоящего момента наше обсуждение было ограничено транзакциями по одной базе данных. А что, если используется не одна база данных? Что будет, если используется не один сервер баз данных? Что будет, если использу-

ется операция типа модификации кэша в памяти? Можно ли использовать `BEGIN TRANSACTION` для привязки подобных операций к единой транзакции? К сожалению, нет. `BEGIN TRANSACTION` работает только для локальных транзакций, имеющих дело с данными базы данных. Эти транзакции неприменимы к кэшу памяти — поскольку нет механизма журналирования транзакций.

В этом разделе мы более глубоко рассмотрим теорию транзакций и исследуем пространство имен `System.Transactions` в `.NET Framework`. Вы увидите также, почему транзакция (которая и так является дорогой вследствие своих издержек) становится еще более дорогой при управлении от внешнего объекта — координатора транзакций. Это обсуждение относится не только к транзакциям баз данных.

## Терминология распределенных транзакций

При работе с распределенными транзакциями вам будут часто встречаться такие понятия, как диспетчер ресурсов, менеджер транзакций, координатор транзакций и двухфазная фиксация. Давайте более тщательно рассмотрим эти термины.

### Диспетчер ресурсов

Транзакции (баз данных и прочие) управляют ресурсами. Любая операция, которую нужно выполнить в виде транзакции, управляется логическим объектом — процедурой, функцией, библиотекой DLL, выполняемым модулем, компьютером или чем-то еще (что поддерживает транзакции). Любой такой логический объект, который отвечает за управление ресурсом транзакционным образом, называется диспетчером ресурсов (`resource manager, RM`).

Таким образом, `RM` способен (и несет за это ответственность) привлекаться к участию в текущей выполняющейся транзакции и поэтому поддерживает транзакционные возможности.

### Менеджер транзакций или координатор транзакций

Если у вас есть `RM`, который управляет своими ресурсами транзакционным образом (подключившись к текущей выполняемой транзакции), то по определению вам нужен внешний объект, который управляет самой транзакцией. Этот внешний объект отвечает за прослушивание нескольких `RM` (которые

участвуют в одной и той же транзакции) и координацию между ними. Он подтверждает прием запросов на новые транзакции, а также слушает и посылает уведомления в случае успеха и неудачи. Этот объект называется *менеджером транзакций* (transaction manager, TM), *координатором транзакций* (transaction coordinator, TC) или *координатором распределенных транзакций* (distributed transaction coordinator, DTC). С операционной системой Microsoft Windows поставляются Lightweight Transaction Manager (LTM) и Microsoft Distributed Transaction Coordinator (MS DTC).

Обратите внимание, что TC — это не обязательно DTC. Фактически, если сам RM имеет встроенные транзакционные возможности, то TC ему может быть совершенно не нужен. Например, если транзакция ограничена одной базой данных, то SQL Server 2008 может сам управлять такой транзакцией. То есть для локальных транзакций SQL Server не обращается к MS DTC. Для этого есть веские причины, которые станут очевидными после того, как вы прочитаете о типичной реализации распределенной транзакции — о процессе двухфазной фиксации.

## Двухфазная фиксация

Распределенная транзакция может быть реализована несколькими способами. Одним из наиболее часто применяемых способов является использование процесса двухфазной фиксации. Вот типичный ход двухфазной транзакции двух RM и DTC:

1. Инициатор транзакции запрашивает транзакцию у DTC. Этим инициатором транзакции может быть приложение, которое взаимодействует с двумя RM, либо им может быть один из RM.
2. Инициатор транзакции требует, чтобы RM делали свою работу как часть той же самой транзакции. RM регистрируют себя в DTC как часть той же самой транзакции, давая знать о своем желании получать уведомления об успехе или неудаче транзакции в целом. Этот процесс называется *прикреплением к транзакции*.
3. RM продолжают делать свою работу и уведомляют DTC об успехе (держа наготове механизм отката). Это первая фаза процесса двухфазной фиксации, которая называется также подготовительной фазой.
4. После того как DTC получил уведомление об успехе подготовительной фазы от каждого из прикрепленных RM, он выдает уведомление для продолжения и сохранения изменений. При получении такого уведомления

все RM сохраняют свои изменения (при помощи фиксации их переходных состояний). Это называется также *фазой фиксации*. Система перешла из одного стабильного состояния в другое, и распределенная транзакция закончилась.

## Правила и методы прикрепления

Как вы, вероятно, догадались, DTC на компьютере под управлением Windows (это MS DTC) ведет довольно многословный обмен с различными участвующими в транзакции диспетчерами ресурсов. Вследствие того, что при этом происходит обмен по сети, это влияет на производительность приложения. Кроме того, такой обмен может быть заблокирован сетевыми экранами. В дополнение к этому, прикрепившиеся к распределенной транзакции диспетчеры ресурсов часто используют уровень изоляции Serializable. Такую архитектуру реализовать легче всего, поскольку при использовании этого уровня изоляции вы имеете безукоризненную транзакцию — без грязных, фантомных и неповторяемых чтений. Конечно, есть и отрицательная сторона — значительное снижение производительности.

Но SQL Server 2008 сам способен управлять транзакциями, зачем ему при каждом случае повышать уровень изоляции до Serializable? В конце концов (в зависимости от вашей логики) вы можете использовать транзакцию на базе MS DTC (в том и только в том случае, когда ваша транзакция включает в себя более одного RM). Но пока используется только одно подключение к базе данных, вам не следует нести дополнительные издержки на использование DTC. Оказывается, инженеры компании Microsoft уже обдумали эту ситуацию. Для того чтобы исправить такую ситуацию, RM может прикрепиться к транзакции различными способами.

## Прикрепление Volatile

RM с временными (непостоянными) ресурсами является хорошим кандидатом для прикрепления Volatile. Обычно в случае такого прикрепления, если RM не может по какой-либо причине выполнить вторую фазу (фиксацию) распределенной транзакции, ему не нужно явно восстанавливать первую (подготовительную) фазу. Это означает, что если RM дает фатальный сбой в середине транзакции, то ему не нужно обеспечивать явный контракт восстановления для ТС. Прикреплению Volatile не нужна реализация MS DTC, поэтому оно обычно управляется из LTM, более легковесного ТС. Примером

может быть такой диспетчер ресурсов, который управляет кэшем в памяти. Данные кэша не должны быть постоянными, они существуют только в течение относительно короткого времени.

## Прикрепление Durable

Прикрепление Durable необходимо тогда, когда RM имеет постоянные данные, целостность которых зависит от транзакции. Хорошим примером является транзакция, которая делает дисковый ввод/вывод. Например, вы пишете в файл на диск. Если транзакция заканчивается неудачей (или RM дает фатальный сбой), то записанный (как часть подготовительной фазы) файл необходимо будет удалить. То есть RM должен будет подготовить журнал транзакций и записать историю изменений с момента начала транзакции. В случае запроса на восстановление диспетчеру ресурсов потребуются информация для выполнения правильного отката.

### **ПРИМЕЧАНИЕ**

FILESTREAM в SQL Server 2008 реализует прозрачную координацию между транзакциями базы данных и транзакциями файловой системы NTFS. Подробное описание FILESTREAM вы найдете в *главе 8*.

## Прикрепление Promotable Single-Phase

Во многих ситуациях характер транзакции может меняться по мере прикрепления новых RM. Например, база данных SQL Server 2008 может сама прекрасно справляться с управлением транзакциями (пока транзакция ограничена одной базой данных). Или, например, управляющему кэшем в памяти диспетчеру ресурсов не нужна реализация MS DTC, поскольку кэш по своей природе имеет временный характер. Но если есть транзакция с RM для кэша в памяти либо подключение к SQL Server 2008, которое управляется менеджером LTM, и к этой же самой транзакции прикрепляется второе подключение SQL Server 2008, то такая транзакция будет передана в MS DTC, поскольку диспетчеры ресурсов больше не могут самостоятельно управлять ею.

Важно отметить, что вместе с передачей приходит и то, что иногда считают недостатками MS DTC — более высокий уровень изоляции, а также более дорогая и многословная транзакция. Прикрепление Promotable single-phase (PSPE) предлагает огромное преимущество в том смысле, что пока вам ре-

ально не нужен MS DTC, вы его не используете (и не несете его издержек). Однако в тех случаях, когда MS DTC вам нужен, он — отличный подарок.

Существуют вполне определенные правила для передачи транзакции от LTM в MS DTC. Транзакция передается от LTM в MS DTC в том случае, когда случается что-нибудь из следующего списка:

- ❑ к транзакции прикрепляется постоянный ресурс, который не поддерживает однофазных уведомлений;
- ❑ к одной транзакции прикрепляются два постоянных ресурса, которые поддерживают однофазные уведомления;
- ❑ ТС получает запрос на маршалирование транзакции в другой AppDomain в .NET или в другой процесс Windows.

### **ПРИМЕЧАНИЕ**

Подключения SQL Server 2000 всегда передаются в MS DTC, а подключения общезыковой исполняющей среды SQL внутри System.Transactions.TransactionScope передаются в MS DTC даже в том случае, если только одно из них прикреплено в области действия транзакции.

Имея в нашем багаже теорию и терминологию, далее мы рассмотрим поддержку распределенных транзакций в SQL Server 2008 и .NET Framework.

## **Распределенные транзакции в SQL Server 2008**

SQL Server 2008 поддерживает распределенные транзакции при помощи инструкции `BEGIN DISTRIBUTED TRANSACTION`. Эта инструкция требует запустить распределенную транзакцию T-SQL под управлением MS DTC. Она использует следующий синтаксис:

```
BEGIN DISTRIBUTED { TRAN | TRANSACTION }  
    [ transaction_name | @tran_name_variable ]
```

Самый легкий способ прикрепления удаленных экземпляров ядра базы данных SQL Server к распределенной транзакции — это выполнение распределенного запроса, который ссылается на связанный сервер. Например, вы можете связать ServerB и выполнить такой запрос:

```
DELETE FROM ServerB.TestDB.TestTable Where TestID = 1
```



Подключить запрос к распределенной транзакции очень просто:

```
BEGIN DISTRIBUTED TRANSACTION
    DELETE TestDB.TestTable WHERE TestID = 1
    DELETE ServerB.TestDB.TestTable WHERE TestID = 1
COMMIT
```

Очевидным недостатком этой реализации является то, что второй запрос не имеет возможности явного подключения. Это может показаться банальным, но если второй запрос — это хранимая процедура, вызывающая `ServerC`, который в свою очередь вызывает `ServerD`, то все они будут завязаны в одну очень дорогую транзакцию, которая будет управляться одним MS DTC (на инициировавшем сервере).

Эту проблему можно обойти путем конфигурирования поведения по умолчанию, чтобы запросы к связанным серверам не передавались в MS DTC. Вы можете сделать это двумя способами, например, на уровне сервера при помощи следующей команды T-SQL:

```
sp_configure remote proc trans 0
```

Либо вы можете сделать это на уровне подключения при помощи следующего синтаксиса:

```
SET REMOTE_PROC_TRANSACTIONS OFF
```

Затем вы можете использовать `BEGIN TRANSACTION`, и ядро базы данных SQL Server будет управлять транзакциями вместо вас. И наоборот, если вы укажете значение 1 или `ON`, то инструкция `BEGIN TRANSACTION` со связанными серверами будет использовать MS DTC, но здесь либо все, либо ничего.

## Распределенные транзакции в .NET Framework

Концепция распределенных транзакций появилась не в .NET Framework. Вы могли прикрепиться к распределенным транзакциям при помощи координаторов транзакций сторонних компаний, таких как COMITI, COM+ или Microsoft Transaction Server (MTS). Начиная с .NET Framework 1.0, вы могли также использовать для прикрепления к распределенной транзакции пространство имен `System.EnterpriseServices`. Оно по существу является оболочкой для инфраструктуры COM+.

Проблема с решениями на базе `EnterpriseServices` состояла в том, что вам приходилось реализовывать свою операцию как библиотеку классов, помечать ее атрибутом `TransactionOption`, давать ей строгое имя и регистрировать ее в глобальном кэше сборок (GAC). Это усложняло отладку и развер-

тывание. А атрибут `TransactionOption` жестко фиксировал транзакционное поведение вашей операции в одном из следующих значений:

- ☐ `Disabled` — не участвовать в транзакциях. Это значение по умолчанию;
- ☐ `NotSupported` — выполняется вне контекста транзакции;
- ☐ `Supported` — участвует в транзакции, если она существует. Если ее не существует, то операция не будет запрашивать или создавать транзакцию;
- ☐ `Required` — требует транзакции. Если транзакции не существует, то она создается. Если она существует, то операция прикрепляется к транзакции;
- ☐ `RequiresNew` — требует транзакции и создает для себя новую транзакцию.

В .NET Framework 1.0 вы не могли прикрепиться по требованию, отладка и развертывание были трудными. .NET Framework 1.1 дает несколько лучшее решение при помощи класса `ServiceConfig` (как это показано в листинге 12.3).

#### Листинг 12.3. Использование класса `ServiceConfig`

```
ServiceConfig config = new ServiceConfig();
config.Transaction = TransactionOption.Required;
ServiceDomain.Enter(config);
try
{
    // SqlConnection будут автоматически прикрепляться к транзакции.
}
catch
{
    ContextUtil.SetAbort();
}
finally
{
    ServiceDomain.Leave();
}
```

Это решение (известное как "службы без компонентов") не требовало от вас регистрации вашей сборки в GAC и даже строгого именования ее, но в свое время оно было ограничено только операционными системами Microsoft Windows 2003 и Windows XP Service Pack 2, и поэтому не получило серьезного развития. Оно было распространено и на другие операционные системы Windows, но до сих пор остается относительно неизученным. Для преодоле-

ния всех этих проблем .NET Framework 2.0 вводит новое пространство имен `System.Transactions`.

Давайте же рассмотрим поведение транзакции на базе `System.Transactions`. Мы настроим две базы данных и выполним по одному запросу для каждой.

Следующий код создает две базы данных с названиями `Test1` и `Test2`. Затем он создает в `Test1` таблицу с названием `FromTable`, а в `Test2` — таблицу с названием `ToTable`. В обеих таблицах имеется один столбец `int` с названием `Amount`. Затем в таблицу `FromTable` вставляется строка со значением 100, а в таблицу `ToTable` — строка со значением 0.

```
CREATE DATABASE Test1
GO
USE Test1
GO
CREATE TABLE FromTable (Amount int)
GO
INSERT INTO FromTable(Amount) VALUES (100)
GO
CREATE DATABASE Test2
GO
USE Test2
GO
CREATE TABLE ToTable (Amount int)
GO
INSERT INTO ToTable(Amount) VALUES (0)
GO
```

Теперь мы создадим консольное приложение и назовем его `DistributedTrans`. Добавим необходимую ссылку на `System.Transactions`. Затем добавим нужные инструкции `using` (для доступа к пространствам имен `System.Data`, `System.Data.SqlClient` и `System.Transactions`):

```
using System.Data;
using System.Data.SqlClient;
using System.Transactions;
```

Цель данного примера — выполнить два запроса (по одному для каждой базы данных). Один запрос вычитет 50 из `FromTable` в `Test1`, а другой добавит 50 в таблицу `ToTable` базы `Test2`. Обе операции должны быть связаны одной транзакцией. Мы легко можем сделать это при помощи следующего фрагмента кода, который вы можете просто добавить в статический метод `Main`.

(Не волнуйтесь, если вы его пока не понимаете, скоро мы объясним этот код).

```
const string ConnStr1 =
    "Data Source=(local);Initial Catalog=Test1;Integrated Security=SSPI;";
const string ConnStr2 =
    "Data Source=(local);Initial Catalog=Test2;Integrated Security=SSPI;";
const string CmdText1 = "UPDATE FromTable SET Amount = Amount - 50";
const string CmdText2 = "UPDATE ToTable SET Amount = Amount + 50";
using (TransactionScope tsc = new TransactionScope())
{
    using (SqlConnection conn1 = new SqlConnection(ConnStr1))
    {
        SqlCommand cmd1 = conn1.CreateCommand();
        cmd1.CommandText = CmdText1;
        conn1.Open();
        cmd1.ExecuteNonQuery();
    }
    // Операция #1 выполнена, переходим к операции #2
    using (SqlConnection conn2 = new SqlConnection(ConnStr2))
    {
        SqlCommand cmd2 = conn2.CreateCommand();
        cmd2.CommandText = CmdText2;
        conn2.Open();
        cmd2.ExecuteNonQuery();
    }
    tsc.Complete();
}
```

Запустите этот код. Обратите внимание, что два объекта команд (`cmd1` и `cmd2`) выполняются в распределенной транзакции. Если второй запрос закончится неудачей, то первый автоматически откатится назад. Все действительно так просто. Конечно, возможны более сложные реализации, но основной процесс реализации распределенной транзакции при помощи `System.Transaction` действительно так прост. Теперь давайте более внимательно рассмотрим только что написанный нами код.

В начале данного фрагмента кода имеется блок `using`:

```
using (TransactionScope tsc = new TransactionScope())
{
    // ...Делаем здесь транзакционные операции...
}
```

Блок `using` обеспечивает вызов метода `Dispose` для экземпляра `tsc` класса `TransactionScope` тогда, когда область действия инструкции `using` заканчивается. Создание экземпляра `TransactionScope` начинает новую транзакцию, и метод `Dispose` вызывается тогда, когда заканчивается область действия охватывающего блока `using`. Распределенная транзакция фиксируется посредством освобождения `tsc`. Таким образом, внутри блока `using` любой диспетчер ресурсов будет пытаться прикрепиться к текущей выполняющейся транзакции и зафиксирует транзакцию (если метод `Complete` в `TransactionScope` вызывается перед выходом из области действия блока `using` и в самом блоке не происходит исключений).

Как вы могли заметить, процесс выполнения запроса по базе данных реализован обычным кодом ADO.NET:

```
using (SqlConnection conn1 = new SqlConnection(connStr1))
{
    SqlCommand cmd1 = conn1.CreateCommand();
    cmd1.CommandText = CmdText1;
    conn1.Open();
    cmd1.ExecuteNonQuery();
}
```

Волшебство здесь состоит в том, что экземпляр `conn1` класса `SqlConnection` знает, что он работает внутри `TransactionScope`. Когда происходит вход в область действия транзакции, он создает внешнюю транзакцию. Поскольку выполняющийся (в области действия транзакции) транзакционный код теперь имеет активную транзакцию для прикрепления, он прикрепляет себя к соответствующему менеджеру транзакций.

Обратите внимание, что мы сказали "к соответствующему менеджеру транзакций", а не к MS DTC. Это потому, что когда экземпляр `SqlConnection` подключается к базе данных SQL Server 2008, он демонстрирует поведение PSPE: транзакция управляется из LTM, а не из MS DTC (до вызова инструкции `conn2.Open`).

В том фрагменте кода, где мы вызываем `conn2.Open`, настройте точку останова и запустите приложение еще раз. При достижении точки останова изучите значение:

```
Transaction.Current.TransactionInformation.DistributedIdentifier.ToString()
```

В режиме отладки выполните инструкцию `conn2.Open` и проверьте это значение еще раз. Что вы видите? Непосредственно перед открытием `conn2` это значение — неопределенный глобальный уникальный идентификатор (GUID):

```
"00000000-0000-0000-0000-000000000000"
```

Сразу же после выполнения `conn2.Open` это значение меняется на реальное значение GUID, похожее на следующее:

```
"7b3ca1be-6c31-43b9-8fa8-fd759a1a8063"
```

Если вы сразу же откроете панель **Control Panel** и выберете список **Administrative Tools**, в нем **Component Services**, а затем перейдете к списку **Transaction List**, то вы заметите там тот же самый GUID (рис. 12.1).

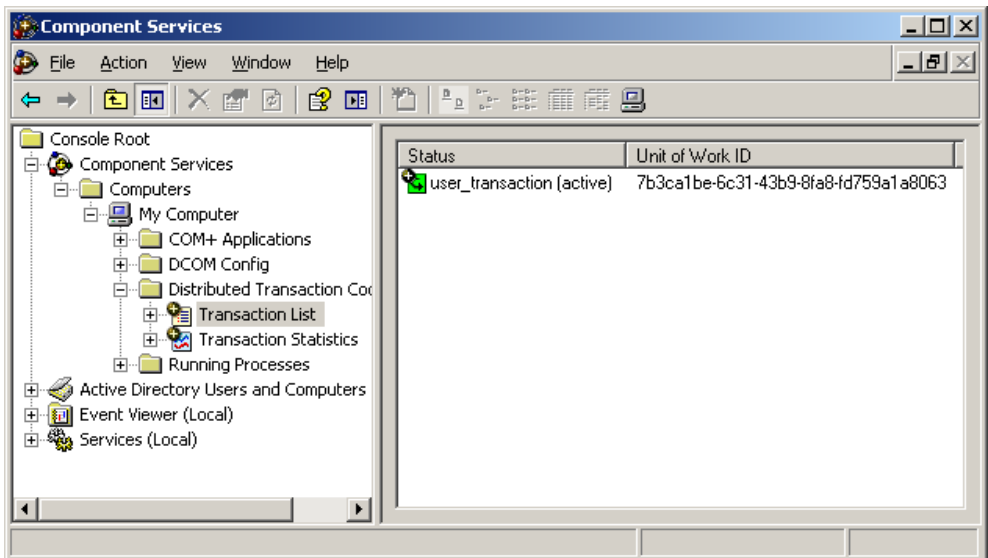


Рис. 12.1. Текущая активная транзакция, управляемая из MS DTC

Это говорит вам о том, что непосредственно перед выполнением `conn2.Open` транзакция управлялась из LTM и поэтому не имела правильного `DistributedIdentifier`. Но после прикрепления второго диспетчера ресурсов транзакция была передана в MS DTC и получила правильное значение `DistributedIdentifier`. Если бы вы выполнили этот пример для базы данных SQL Server 2000, то идентификатор имел бы правильное значение сразу же после выполнения `conn1.Open`. Это подтверждает тот факт, что `SqlConnection` прикрепляется к базе данных SQL Server 2000 в режиме `Durable`, но демонстрирует поведение PSPE при подключении к базе данных SQL Server 2008 (или 2005).

Вероятно, вы могли бы легко добиться того же самого и при помощи инструкции `BEGIN DISTRIBUTED TRANSACTION` (без использования пространства имен `System.Transactions`). Важно понять, что пространство имен

`System.Transactions` предоставляет гораздо более гибкую архитектуру (в которой вы можете логически выбрать, прикрепляться или нет), причем оно работает не только с транзакциями баз данных. Теоретически вы можете написать диспетчер ресурсов, который инкапсулирует любую операцию транзакционным образом (что мы сейчас и сделаем).

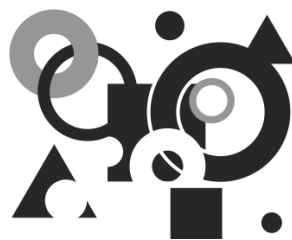
## Резюме

В этой главе мы изложили основы теории транзакций, которая является, вероятно, одной из самых важных тем, связанных с архитектурой приложений. Вы узнали о поддержке транзакций в SQL Server 2008, уровнях изоляции и их поведении. Вы увидели также одновременное параллельное сравнение с ADO.NET.

Мы описали реализацию распределенных транзакций и новую поддержку написания транзакционного кода, который может не иметь ничего общего с базой данных (кроме возможности прикрепления к той же самой транзакции, что и база данных).

И наконец, вы узнали о поддержке и реализации транзакций в SQL CLR. Вы увидели, как объект `SqlConnection` использует реализацию локальных транзакций и реализацию распределенных транзакций, а также и поведение PSPE (при необходимости).

## ГЛАВА 13



# Разработка периодически подключенных систем

*Пол Декоглиано*

Сегодняшние бизнес-приложения должны простираяться дальше настольных компьютеров. Пользователям требуется доступ к данным даже тогда, когда они не сидят за своими столами. Мобильным сотрудникам компаний нужен доступ к данным тогда, когда они находятся в пути.

Торговым представителям, которые посещают своих клиентов, нужен доступ к данным о складских запасах и о продуктах. Их способность оформить заказ в отрыве от офиса может быть решающим фактором для продажи. Персоналу, который занимается доставкой, нужен доступ к адресам клиентов, маршрутам движения и деталям поставки. Наличие такой информации у водителя повышает эффективность его работы.

Чаще всего мобильные сотрудники не имеют постоянного сетевого подключения (во всяком случае в настоящее время доступ по WiFi есть не везде). Поэтому приложения должны обеспечивать пользователей данными даже при отсутствии их подключения к офисной сети. Более того, сделанные пользователем (в период отключения) изменения данных должны синхронизироваться с центральной базой данных при появлении подключения. Такие приложения (которые обеспечивают пользователю возможность эффективной работы в состоянии отключения) называются *периодически подключенными системами*.

В этой главе мы внимательно рассмотрим несколько технологий репликации, которые вы можете использовать для реализации периодически подключенных систем. В них входят: Microsoft SQL Server Merge Replication, а также



две новые замечательные функциональные возможности с названиями Sync Services for ADO.NET и SQL Server Change Tracking.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Данная глава позволит вам быстро начать создавать периодически подключаемые системы при помощи Merge Replication и Sync Services for ADO.NET. Вы можете найти дополнительную информацию об этих технологиях на странице Sync Services For ADO.NET Version 2 на сайте MSDN по адресу: [http://msdn.microsoft.com/en-us/library/bb726002\(SQL.100\).aspx](http://msdn.microsoft.com/en-us/library/bb726002(SQL.100).aspx), а также в документе "Microsoft Virtual Labs: Learn How to Build Occasionally Connected Applications" по адресу:

[http://download.microsoftvirtuallabs.com/download/8/a/7/](http://download.microsoftvirtuallabs.com/download/8/a/7/8a71365b-4c80-4e60-8185-8f12f59bf1d4/LearnHowtoBuildOccasionallyConnectedApplication.pdf)

[8a71365b-4c80-4e60-8185-8f12f59bf1d4/](http://download.microsoftvirtuallabs.com/download/8/a/7/8a71365b-4c80-4e60-8185-8f12f59bf1d4/LearnHowtoBuildOccasionallyConnectedApplication.pdf)

[LearnHowtoBuildOccasionallyConnectedApplication.pdf](http://download.microsoftvirtuallabs.com/download/8/a/7/8a71365b-4c80-4e60-8185-8f12f59bf1d4/LearnHowtoBuildOccasionallyConnectedApplication.pdf).

Merge Replication — это проверенная технология репликации и синхронизации. Она известна в основном как инструмент репликации баз данных (для реализации плана аварийного восстановления). Однако она поддерживает также и репликацию периодически подключаемых приложений. Merge Replication при создании таких приложений предлагает не очень много возможностей программного управления, но все же обеспечивает высокую гибкость (благодаря своим многочисленным опциям конфигурации).

Службы Sync Services for ADO.NET дают вам подмножество функциональных возможностей из Microsoft Sync Framework, предназначенное для работы с синхронизацией баз данных. Этот интерфейс прикладного программирования имеет примерно такой же набор функций, что и Merge Replication, но даст вам больше возможностей программного управления.

### **ПРИМЕЧАНИЕ**

Службы Sync Services for ADO.NET заменяют более старую технологию синхронизации мобильных устройств с названием Remote Data Access (RDA).

Новая функция SQL Server Change Tracking предназначена для хранения и отслеживания информации о модификации данных. При помощи Change Tracking сервер SQL Server сохраняет информацию о произведенных в строках таблицы изменениях, избавляя вас от необходимости создавать собственное подобное решение.

## Сравниваем службы Sync Services и Merge Replication

И Sync Services for ADO.NET, и Merge Replication могут быть использованы для создания периодически подключенных систем. Однако между ними имеются некоторые отличия, о которых вы должны знать при выборе того или другого варианта, в зависимости от ваших потребностей. В табл. 13.1 показаны некоторые важные отличия этих двух технологий.

**Таблица 13.1.** Сравнение возможностей Sync Services и Merge Replication

| Функциональная возможность                          | Sync Services                                                                      | Merge Replication                             |
|-----------------------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------|
| Клиентская база данных                              | SQL Server Compact 3.5                                                             | Любая версия SQL Server                       |
| Серверная база данных                               | Любая реляционная система управления базой данных                                  | Любая версия SQL Server                       |
| Синхронизация через n-уровни или через Web Services | Да                                                                                 | Нет                                           |
| Автоматическое распространение изменений схемы      | Нет                                                                                | Да                                            |
| Программное управление синхронизацией               | Богатый интерфейс прикладного программирования, знакомая модель DataSet из ADO.NET | Ограниченное мастером и хранимыми процедурами |

Короче говоря, если вы создаете приложение, в котором клиенты SQL Server должны синхронизироваться с отличными от SQL Server базами данных, либо если ваше приложение построено поверх n-уровневой или ориентированной на сервисы архитектуры, то выбирайте Sync Services. Для клиент-серверных приложений вы можете использовать как Sync Services, так и Merge Replication.

## Компоненты периодически подключенной системы

Очень часто периодически подключаемая система является расширением традиционного бизнес-приложения для настольного компьютера. Придание мобильности приложению требует определенного пересмотра его архитектуры. Работающие с клиентским приложением устройства обычно небольшие и имеют меньше вычислительной мощности и памяти, чем настольные компьютеры. Им приходится хранить данные локально, поскольку они не всегда имеют подключение к сети. Наличие данных на клиентском устройстве позволяет мобильному пользователю работать в отключенном состоянии.

При наличии периодического подключения пользователи получают копии тех данных, которые нужны им для выполнения их задач. Эти данные извлекаются из центральной базы данных в тот период времени, когда мобильное устройство подключено к офисной сети. Этот процесс называется *синхронизацией*. При наличии подключения устройство синхронизирует свою локальную базу данных с данными центрального сервера, что позволяет мобильному пользователю взять данные с собой в поездку. Таким устройством может быть ноутбук или коммуникатор на базе операционной системы Microsoft Windows Mobile.

Самым основным требованием к периодически подключаемому приложению является предоставление пользователю возможности работать с данными приложения при отсутствии сетевого подключения. Сделанные при отсутствии подключения изменения данных применяются к локальной базе данных; обычно это SQL Server Compact 3.5 (для смартфонов или устройств на базе Windows Mobile), либо SQL Server 2008 Express (для ноутбуков и планшетных компьютеров). Есть и другие требования:

- ☐ синхронизировать изменения между базами данных клиента и сервера;
- ☐ обновлять локальное хранилище данных;
- ☐ разрешать конфликты данных, которые возникают при синхронизации;
- ☐ обрабатывать транзакционные обновления;
- ☐ использовать политики устаревания данных.

Синхронизация данных может быть как односторонней (от сервера к клиенту), так и двусторонней (от сервера к клиенту и обратно). Односторонняя синхронизация обычно используется для справочных данных (таких как спи-

сок продуктов). В этом случае устройство не обновляет данных о продуктах. С точки зрения устройства справочные данные являются данными "только для чтения".

Разумеется, двусторонняя синхронизация требует обновления данных на клиенте и отправки их обратно на сервер. Обновленные данные синхронизируются при установлении сетевого подключения. Например, представьте себе мобильное приложение, которое скачивает данные поставки и сохраняет эти данные локально. В течение всего дня занимающийся доставкой человек делает обновления данных, добавляя статус поставки (аналогично тому, как транспортные компании доставляют посылки). Сохраненные локально (в устройстве) изменения синхронизируются с центральной базой данных при следующем подключении мобильного устройства к сети офиса.

В некоторых случаях требующиеся разным мобильным пользователям данные пересекаются (например, информация о клиентах). Но в других случаях данные весьма специфичны (как адреса клиентов). Например, разным торговым представителям может понадобиться совместное использование данных о продуктах, но разным работающим на доставке людям могут понадобиться только такие адреса клиентов, которые соответствуют обслуживаемым сегодня районам. Все конфликты должны обрабатываться и разрешаться протоколом синхронизации.

Теперь, когда определена наша основная предпосылка, давайте углубимся в технологии Merge Replication и Sync Services for ADO.NET в сочетании с SQL Server Change Tracking (для создания периодически подключенных приложений).

## Merge Replication

Вы можете считать Merge Replication технологией, которая предназначена для репликации и восстановления после аварийных ситуаций. Хотя она чаще всего используется именно для этого, репликационные свойства делают ее первым кандидатом для построения инфраструктуры клиентских приложений, которые подключаются к серверу только время от времени. Merge Replication предоставляет несколько функциональных возможностей, которые облегчают процесс синхронизации, в том числе способность разрешать конфликты данных, управлять частотой синхронизации, очищать устаревшие данные.

Merge Replication исключительно хорошо работает в тех ситуациях, когда вам не нужно серьезное программное управление процессом синхронизации и приложениями типа клиент-сервер. Настройка и конфигурирование Merge Replication в основном осуществляется при помощи мастеров.

## Знакомимся с Merge Replication

Наше обсуждение Merge Replication начинается с краткого словаря терминов репликации. Merge Replication использует топологию репликации издатель/подписчик. Концептуально это похоже на издание газеты. Терминология Merge Replication состоит из издателей, распространителей, подписчиков, статей, публикаций и подписок.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Merge Replication — это огромная тема. В этой главе мы ограничимся обсуждением только тех функциональных возможностей Merge Replication, которые специфичны для создания периодически подключенных приложений. Более подробное описание Merge Replication вы найдете в документации SQL Server 2008 Books Online.

В табл. 13.2 перечислены несколько определений, используемых в этой главе.

**Таблица 13.2.** Часто используемые термины репликации

| Термин           | Описание                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Издатель         | Центральная база данных, данные которой становятся доступны клиентам при помощи репликации. Издатель может предоставлять множество публикаций, каждая из которых будет отдельным набором данных для репликации                                                                                                                                                                                                                                                  |
| Распространитель | База данных, которая хранит метаданные репликации. Распространитель может хранить метаданные для множества издателей. База данных распространения хранит информацию (типа статуса репликации). Распространитель может находиться в том же самом экземпляре базы данных, что и издатель, это называется локальным распространением. Если распространитель находится в отдельном от издателя экземпляре базы данных, то он называется удаленным распространителем |

Таблица 13.2 (окончание)

| Термин              | Описание                                                                                                                                                                                                                                                                                                                                                                                             |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Подписчик           | Клиентская база данных, которая синхронизируется с издателем. Подписчики могут как получать, так и передавать обратно изменения данных                                                                                                                                                                                                                                                               |
| Статья              | Объект базы данных, который является частью публикации. Этим объектом может быть таблица, представление или хранимая процедура. Когда статьей является таблица, то вы можете использовать фильтры для ограничения посылаемых подписчику данных                                                                                                                                                       |
| Публикация          | Группа из одной или более статей издателя. Публикация позволяет реплицировать связанные статьи как единое целое                                                                                                                                                                                                                                                                                      |
| Подписка            | Копия публикации, отправляемая подписчику. Merge Replication поддерживает два типа подписок: принудительную и по запросу. Принудительная подписка — это когда издатель отправляет изменения подписчику без запроса измененных данных со стороны подписчика. Подписка по запросу — это когда подписчик запрашивает изменения. Обычно периодически подключенная система использует подписку по запросу |
| Моментальный снимок | Представление данных на определенный момент времени. Моментальные снимки генерируются и посылаются подписчикам. За инициализацию и доставку моментальных снимков во время синхронизации отвечает "агент" моментальных снимков. Моментальные снимки могут генерироваться по расписанию                                                                                                                |

## Создание периодически подключенного приложения с использованием Merge Replication

Самым лучшим способом изучения функциональных возможностей синхронизации Merge Replication является практика. В этом разделе мы создадим мобильное приложение для карманного ПК, которое имитирует процесс оформления заказов некой компании. В примере для записи статуса доставки заказа используется база данных SQL Server Compact 3.5. Это облегченная база данных, превосходно подходящая для однопользовательских приложений (на карманных ПК).

## Express или Compact?

Не следует считать SQL Server Compact 3.5 пригодным только в качестве базы данных для мобильных устройств. Его облегченный дизайн делает его отличным решением и для однопользовательских приложений на ноутбуках или настольных компьютерах. Конечно, версия SQL Server 2008 Express тоже может работать на ноутбуках и настольных компьютерах, и обе эти версии SQL Server — бесплатные. Так какую же вам следует использовать? Вот некоторые практические советы для принятия решения по выбору между версиями Express и Compact сервера SQL Server для приложений баз данных, работающих на ноутбуках или настольных компьютерах.

Сначала давайте рассмотрим Express. Несмотря на то, что система бесплатная, она поддерживает все функциональные возможности, которые вы (разработчик) полюбили в сервере SQL Server. Это полная реализация ядра реляционной базы данных, ограниченная только по масштабируемости. Версия Express может использовать до 1 Гбайт памяти, только один процессор (в том числе и многоядерный) и поддерживает базы данных размером до 4 Гбайт. Поэтому она легко может справиться с небольшими многопользовательскими средами, такими как небольшие рабочие группы или Web-сайты отделов.

Поскольку нет функциональных возможностей уровня предприятия и способностей бизнес-аналитики (имеющихся в других версиях), то занимаемое при развертывании версии Express дисковое пространство гораздо меньше. Это означает, что вы, безусловно, можете рассматривать Express для однопользовательских приложений, работающих на ноутбуках или настольных компьютерах. Однако здесь вам необходимо остановиться и подумать. Для локального хранилища данных (встроенного в однопользовательское приложение для ноутбука или настольного компьютера) вам следует использовать версию Compact, а не Express. Давайте изучим версию Compact, чтобы понять причину.

Какой бы маленькой ни казалась версия SQL Server 2008 Express, она просто огромна по сравнению с Compact, которая имеет "вес пера" (как в смысле размера, так и в смысле производительности). Помимо значительно сокращенного размера при развертывании (всего 1,5 Мбайт), версия Compact работает в одном процессе с вашим приложением. Это означает, что нет издержек на работу полнофункциональной службы, которые могли бы повлиять на общую производительность приложения. Для однопользовательского приложения (даже для такого, которое работает на компьютере, способном "потянуть" Express) полная реализация SQL Server была бы, скорее всего, совершенно избыточной. Нужна ли вам многопользовательская служба

(работающая в фоновом режиме) для поддержки простейших потребностей хранения данных в типичном однопользовательском приложении? Очень часто не нужна. SQL Server Compact 3.5 предпочтителен в том случае, когда ваши однопользовательские потребности просты, и вы не хотите нести все издержки полноценной службы Microsoft Windows Service (каковой является версия Express).

Однако если вы хотите путешествовать налегке, то будьте готовы отказаться от некоторой части комфорта и удобств. SQL Server Compact 3.5 упрощен и поддерживает только весьма ограниченное подмножество языка Transact-SQL (T-SQL). По существу это означает, что для создания и управления таблицами и индексами, а также для запроса и обновления данных можно использовать только основные инструкции языка определения данных (DDL) и языка манипулирования данными (DML). Не ищите здесь никаких модных функций SQL Server — у вас не будет даже хранимых процедур. Но будет отличная производительность для однопользовательских приложений (и даже тех, которые работают не на мобильных устройствах).

Конечно, версия Express не является вариантом для мобильных устройств (как Compact не является вариантом для многопользовательских приложений). Но для однопользовательских приложений, работающих на ноутбуках или настольных компьютерах, вы должны будете сделать выбор между версиями Compact и Express. Не стоит думать, что версия Compact ограничена мобильными устройствами. Компания Microsoft сделала это очевидным, когда прекратила называть ее Mobile Edition (как это было раньше). Она не называется также Everywhere Edition, как это было в SQL Server 2005. Для того чтобы использовать под однопользовательское приложение на ноутбуке или настольном компьютере версию Express вместо Compact, у вас должны быть очень веские причины, поскольку это будет очень похоже на использование снегоуборочной машины для очистки маленькой тропы (где хватило бы и обычной лопаты). Эти причины могут включать в себя потребность в тех функциях, которые имеются только в Express, например: XML, геопространственные возможности или возможности потоковой передачи файлов.

В нашем примере персонал доставки получает данные по клиентам и заказам при синхронизации своих клиентских устройств с сервером. При доставке заказа сотрудник доставки обновляет статус поставки на мобильном устройстве. Статус хранится на устройстве до тех пор, пока сотрудник доставки не синхронизируется с сервером (при этом статус поставки обновляется на сервере).

База данных примера хранит информацию, которую вы могли бы обнаружить в любой базе данных: клиенты, заказы и т. д. (В промышленной базе данных у



нас было бы гораздо больше таблиц и столбцов, но для нашего примера вполне достаточно такого набора таблиц). Сначала нам нужно создать в SQL Server 2008 базу данных для хранения клиентов и заказов. Запустите SQL Server Management Studio, подключитесь к вашему локальному экземпляру SQL Server 2008 и откройте окно нового запроса. Затем выполните показанный в листинге 13.1 код для создания базы данных и таблиц приложения.

**Листинг 13.1. Создание базы данных и таблиц, используемых в наших примерах**

```
CREATE DATABASE SQL08SampleDB
GO
USE SQL08SampleDB
GO
CREATE TABLE [Customer] (
    CustomerID int NOT NULL IDENTITY(1,1),
    CustomerName varchar(100) NOT NULL,
    CONSTRAINT [PK_Customer]
        PRIMARY KEY CLUSTERED (CustomerID ASC)
)
GO
CREATE TABLE [Order] (
    OrderID int NOT NULL IDENTITY(1,1),
    CustomerID int NOT NULL,
    OrderDate datetime2(0) NOT NULL,
    IsDelivered bit NOT NULL DEFAULT(0),
    CONSTRAINT [PK_Order]
        PRIMARY KEY CLUSTERED (OrderID ASC)
)
GO
CREATE TABLE [Contact] (
    ContactID uniqueidentifier NOT NULL,
    CustomerID int NOT NULL,
    ContactName nvarchar(100),
    ContactPhone nvarchar(100),
    ContactEmail nvarchar(100),
    CONSTRAINT [PK_Contact]
        PRIMARY KEY CLUSTERED(ContactID ASC)
)
GO
```

Теперь выполните код листинга 13.2 для добавления данных в нашу новую базу данных.

### Листинг 13.2. Вставка данных

```
INSERT INTO [Customer] (CustomerName) VALUES
    ('Johnson'), ('Smith'), ('Jones'), ('Davis'), ('Jefferson')
INSERT INTO [Order] (CustomerID, OrderDate, IsDelivered) VALUES
    (1, '01/15/2008', 0), (1, '02/14/2008', 1), (2, '02/13/2008', 0),
    (3, '03/17/2008', 1), (4, '04/12/2008', 0), (5, '04/01/2008', 0)
```

Обратите внимание на использование конструкторов строк в листинге 13.2. Это новая сокращенная запись T-SQL в SQL Server 2008, которая позволяет вам одной инструкцией INSERT вставить множество строк. В нашем случае этот код добавляет пять клиентов и шесть заказов.

## Конфигурирование Merge Replication

Как мы уже упоминали ранее, Merge Replication требует небольшого количества кода и в основном конфигурируется через мастера и диалоговые окна. Для реализации Merge Replication необходимо выполнить пять шагов:

1. Сконфигурировать распространение.
2. Создать публикацию.
3. Подписаться на публикацию.
4. Инициализировать подписку.
5. Синхронизировать данные.
6. Каждый из этих шагов состоит из нескольких более мелких шажков.

## Конфигурирование распространения

Первым делом необходимо сконфигурировать базу данных распространителя. Помните, что эта база данных содержит все необходимые для репликации метаданные. При проектировании и планировании базы данных распространителя помните о следующем:

- используйте сервер с достаточными ресурсами. Подобно любой другой базе данных, база данных распространителя требует дискового пространства и процессорных ресурсов;

- ❑ тщательно выбирайте название базы данных и местоположение файлов. После активации распространителя вы не сможете переименовать базу данных. Для ее переименования вы должны будете деактивировать и переконфигурировать распространитель;
- ❑ не забудьте обо всех издателях, которые будут пользоваться распространением. Издателям, которые не находятся на одном сервере с распространителем, потребуются пароли для удаленного подключения к распространителю;
- ❑ агенту Replication Agent необходимо, чтобы SQL Server Agent был сконфигурирован на автоматический запуск при загрузке сервера.

В нашем примере приложения мы будем использовать для распространителя и издателя один и тот же сервер. Работа распространителя и издателя на одном сервере называется *локальным распространением*.

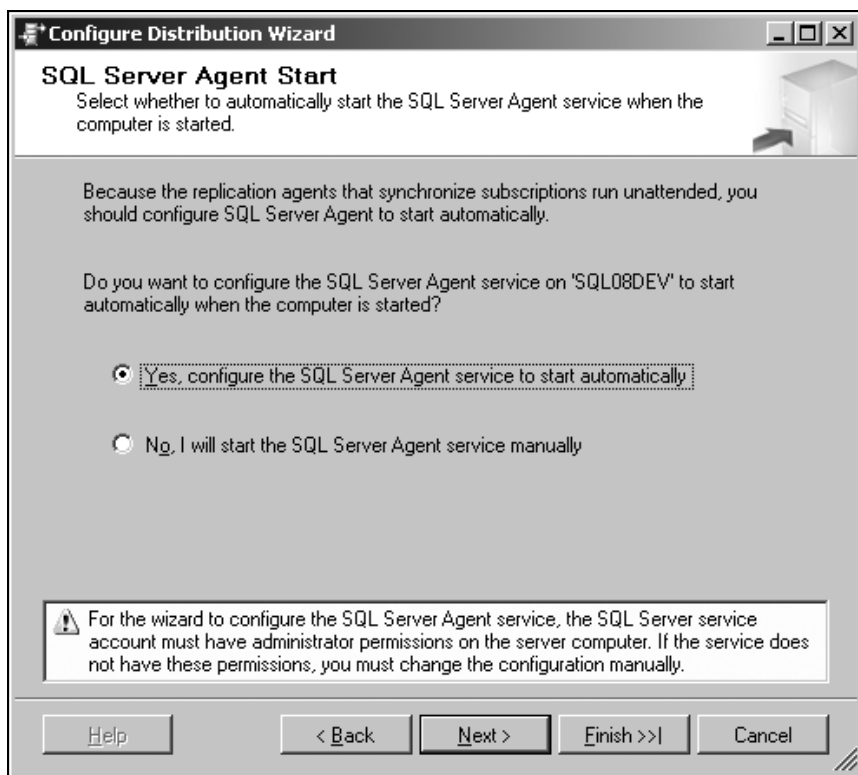
Первый мастер, которым мы здесь будем пользоваться — это **Configure Distribution Wizard**. Этот мастер проведет вас по процессу настройки распространителя для нескольких различных конфигураций. Для нашего примера мы будем конфигурировать распространитель так, чтобы он был издателем, который работает как свой собственный распространитель. Для запуска мастера в окне **Object Explorer** программы **SQL Server Management Studio** перейдите к узлу **Replication**. Щелкните правой кнопкой мыши по узлу **Replication**, а затем выберите пункт **Configure Distribution**. На начальной странице мастера нажмите кнопку **Next** для того, чтобы начать конфигурировать базу данных распространителя. Выберите опцию **<servername> Will Act As Its Own Distributor** (если она еще не выбрана), а затем нажмите кнопку **Next**.

### ПРИМЕЧАНИЕ

Для настройки репликации подключение в **SQL Server Management Studio** должно быть выполнено по фактическому имени сервера. Если подключение было сделано по IP-адресу, псевдониму сервера, **localhost** или просто по символу точки, то вы получите сообщение об ошибке с объяснением данного требования (при попытке запустить мастер **Configure Distribution Wizard**), и **Management Studio** предложит вам установить подключение повторно (чтобы вы могли использовать правильное имя сервера).

Как мы уже упоминали, используемые в **Merge Replication** агенты репликации требуют, чтобы **SQL Server Agent** был сконфигурирован на автоматический запуск при загрузке системы. Если **SQL Server Agent** не будет настроен таким образом (а именно так и сделано по умолчанию в соответствии со стратегией безопасности по умолчанию в **SQL Server**), то вы увидите диало-

говое окно, показанное на рис. 13.1. Выберите вариант **Yes**, а затем нажмите кнопку **Next**.



**Рис. 13.1.** Настройка SQL Server Agent на автоматический запуск для репликации

Затем вы должны указать каталог, в котором SQL Server хранит файлы моментальных снимков и схемы (которые будут запрашиваться подписчиками). Агенты репликации во время синхронизации читают и пишут данные в этот каталог. По умолчанию здесь прописан локальный путь C:\Program Files\Microsoft SQL Server\<instance>\MSSQL\ReplData. Как указано в тексте диалогового окна (рис. 13.2), для приложений, которые используют подписку по запросу или удаленного распространителя, каталог моментальных снимков должен быть разделяемым сетевым ресурсом, записанным в виде маршрута UNC (в форме \\ComputerName\ShareName).

Подписка по запросу — это когда подписчик запрашивает изменения у издателя. Это отличается от принудительной подписки, когда издатель постоянно проталкивает изменения подписчикам. Поскольку наш пример приложения использует подписку по запросу, то мы будем использовать разделяемый сетевой ресурс.

Создайте разделяемый сетевой ресурс для каталога моментальных снимков, для чего откройте Windows Explorer и перейдите в каталог C:\Program Files\Microsoft SQL Server\MSSQL10.MSSQLSERVER\MSSQL. Щелкните правой кнопкой мыши по каталогу **ReplData**, а затем выберите опцию **Sharing And Security**.

### ПРИМЕЧАНИЕ

Для обеспечения безопасности в производственной среде вы должны предоставить к разделяемому ресурсу с моментальными снимками доступ только READ ACCESS для конкретных учетных записей.



Рис. 13.2. Настройка местоположения каталога моментальных снимков

Выберите опцию **Share This Folder**, а затем введите название ReplicationSnapshots (в качестве имени разделяемого ресурса).

### ПРИМЕЧАНИЕ

В разных версиях Windows это диалоговое окно выглядит по-разному (в зависимости от установленных пакетов обновлений).

После настройки разделяемого каталога вернитесь в мастер, введите путь UNC к только что созданному вами каталогу моментальных снимков, в данном примере это \\SQL08Dev\ReplicationSnapshots (рис. 13.2), а затем нажмите кнопку **Next**.

На странице **Distribution Database** введите MyDistributor в качестве названия базы данных распространения. Примите настройки по умолчанию для каталогов данных и журнала транзакций, а затем нажмите кнопку **Next**.



Рис. 13.3. Локальный сервер настроен по умолчанию и как распространитель, и как издатель

На странице **Publishers** (рис. 13.3) указываются издатели, которые будут использовать этого нового распространителя. По умолчанию локальный сервер настроен на работу одновременно и как распространитель, и как издатель. Это все, что нужно нам для нашего приложения, поэтому нажмите кнопку **Next**.

На странице **Wizard Actions** (рис. 13.4) вы указываете шаги, которые мастер должен выполнить по завершении. Здесь вы можете настроить распространение или создать скрипт для настройки распространения (либо сделать и то, и другое). Установите флажок **Configure distribution**, снимите флажок **Generate a script file with steps to configure distribution**, а затем нажмите кнопку **Next**. Если вы решите сгенерировать скрипт, то вы увидите дополнительный шаг мастера (он здесь не показан), который запросит у вас местоположение, в которое будет сохранен файл скрипта.



**Рис. 13.4.** Инструктируем мастера сконфигурировать распространение после нажатия кнопки **Finish**

И наконец, страница **Complete The Wizard** отображает итоги сделанных вами в мастере настроек. Нажмите **Finish** для сборки и настройки распространителя. По завершении мастера вы увидите новую базу данных с названием **MyDistributor**. Эта база создана в узле **System Databases** в SQL Server Management Studio.

### ПРИМЕЧАНИЕ

Если служба SQL Server Agent Service не запущена, вы получите ошибку. Если это случилось, запустите мастер еще раз (при работающем Agent Service).

### ПРИМЕЧАНИЕ

Для того чтобы увидеть базу данных **MyDistributor**, вам может понадобиться обновить узел **System Databases** в окне **Object Explorer**.

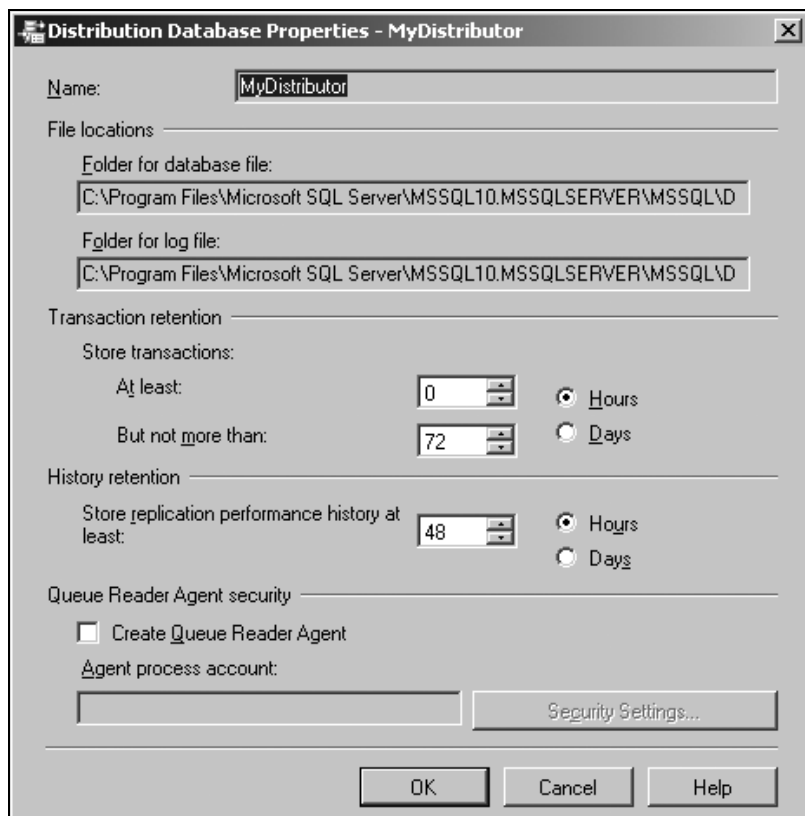


Рис. 13.5. Просмотр свойств распространителя



В диалоговом окне **Distribution Database Properties** (которое можно отобразить при помощи нажатия правой кнопкой мыши узла **Replication**, выбора пункта меню **Distributor Properties** и последующего нажатия кнопки с многоточием справа от базы данных `myDistributor`) можно увидеть несколько свойств. Основным для нашего текущего обсуждения является показанное на рис. 13.5 свойство **Transaction Retention**. Это свойство указывает серверу SQL Server промежуток времени, в течение которого необходимо хранить данные репликации. Вы можете указать минимальное и максимальное значения (кнопки **At least** и **But no more than** соответственно).

## Создание публикации

После настройки распространителя мы можем сосредоточиться на настройке публикации.

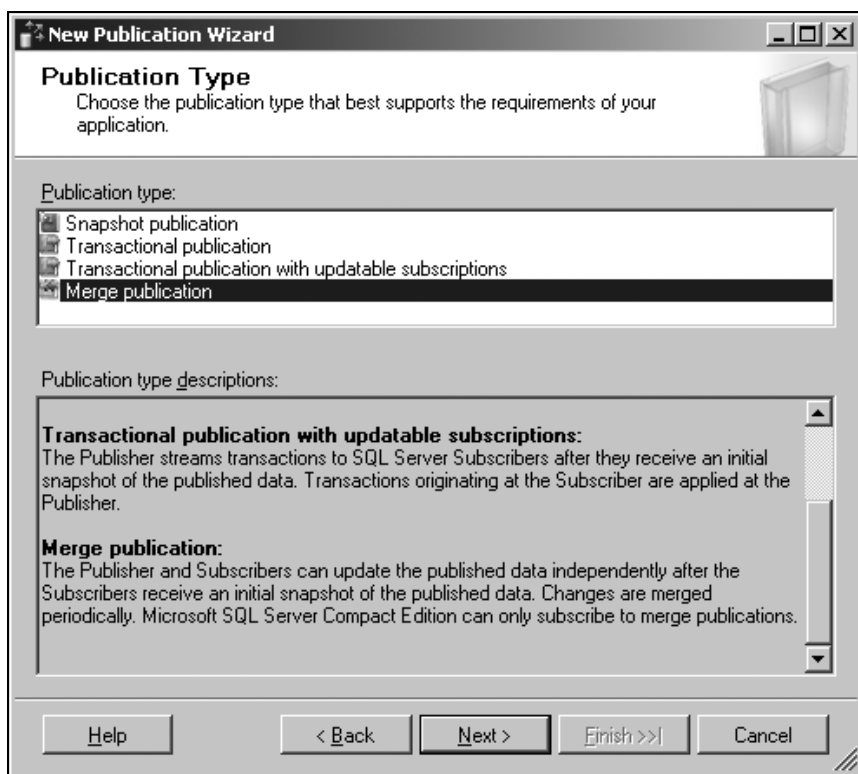


Рис. 13.6. Страница **Publication Type**

В нашем примере мы публикуем таблицы `Customer` и `Order`. Разверните узел **Replication** в окне **Object Explorer** программы SQL Server Management Studio, щелкните правой кнопкой мыши по каталогу **Local Publications** и в контекстном меню выберите опцию **New Publication**. Появится мастер New Publication Wizard, нажмите кнопку **Next** для его запуска.

Первая страница мастера — это страница **Publication Database**. Для создания публикации выберите базу данных, содержащую те данные, которые вы хотите публиковать (в нашем примере это база данных `SQL08SampleDB`), а затем нажмите кнопку **Next**.

На странице **Publication Type** (рис. 13.6) вы указываете тот тип публикации, который лучше всего соответствует требованиям вашего приложения. Для нашего примера это Merge Publication. При такой публикации подписчики инициализируются данными публикации.

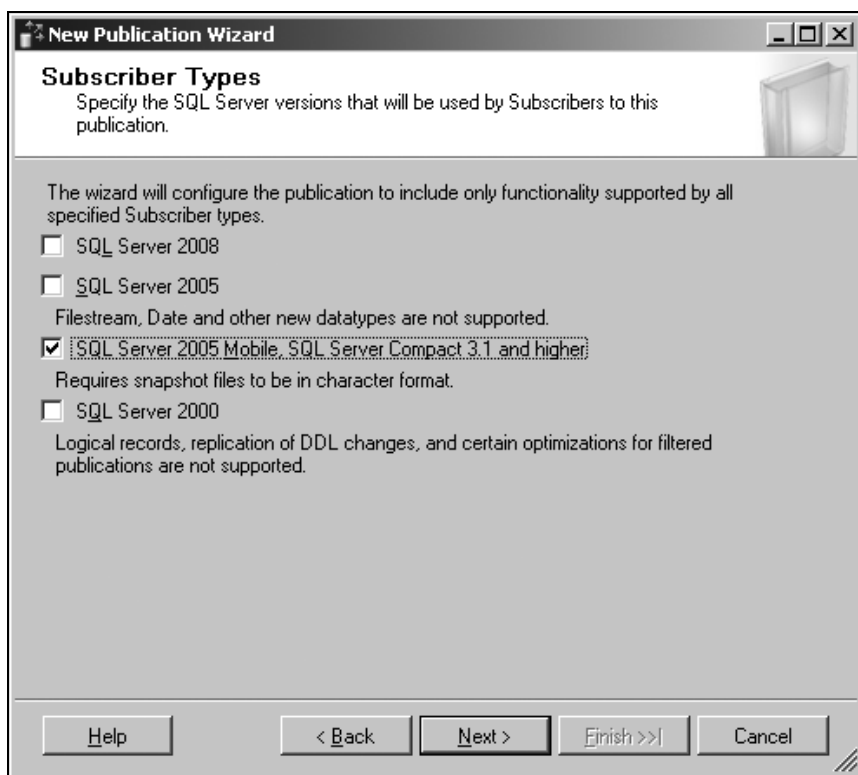


Рис. 13.7. Страница **Subscriber Types**

После инициализации данные время от времени синхронизируются (при появлении подключения клиента). Клиентские приложения вроде нашего примера (который использует SQL Server Compact 3.5) могут подписываться только на публикации типа Merge Publication. Выберите в списке **Merge Publication** и нажмите кнопку **Next**.

На рис. 13.7 показана страница **Subscriber Types**. Здесь вы выбираете версию SQL Server для подписчика. Указанная вами версия определяет функциональные возможности, которые будут доступны для публикации. Поэтому если ваш подписчик использует клиентскую базу данных SQL Server 2005, но вы указываете здесь тип SQL Server 2000, то ваша публикация будет ограничена функциональными возможностями публикаций SQL Server 2000.

Для нашего примера мы используем SQL Server Compact 3.5, поэтому необходимо установить флажок **SQL Server 2005 Mobile, SQL Server Compact 3.1 And Higher**, а затем нажать кнопку **Next**.

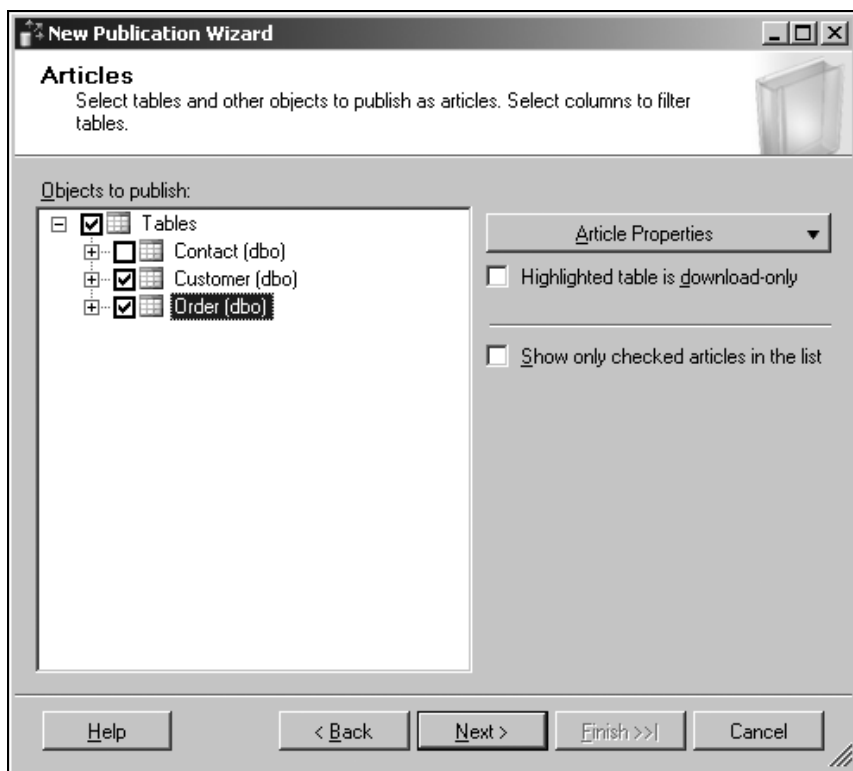


Рис. 13.8. Выбор элементов для репликации

На странице **Articles** (рис. 13.8) вы указываете, какие таблицы базы данных подлежат репликации. Здесь вы выбираете, какие таблицы синхронизировать, а какие должны отслеживаться на появление любых изменений (в том числе и изменений схемы). Вы можете также выбрать для репликации конкретные столбцы. Выберите таблицы `Customer (dbo)` и `Order (dbo)`.

Если вы выбираете в древовидном представлении одну из таблиц (на рис. 13.8 выбрана таблица `Order (dbo)`), то появляется флажок **Highlighted table is download-only**. Если вы установите этот флажок, то вы создадите одностороннюю синхронизацию. В нашем примере используется двусторонняя синхронизация, поэтому не устанавливайте этот флажок. Помните, что при двусторонней синхронизации все изменения языка управления данными DML (которые происходят в базе данных подписки) отслеживаются и распространяются в главную базу данных SQL Server 2008. Нажмите кнопку **Next**.

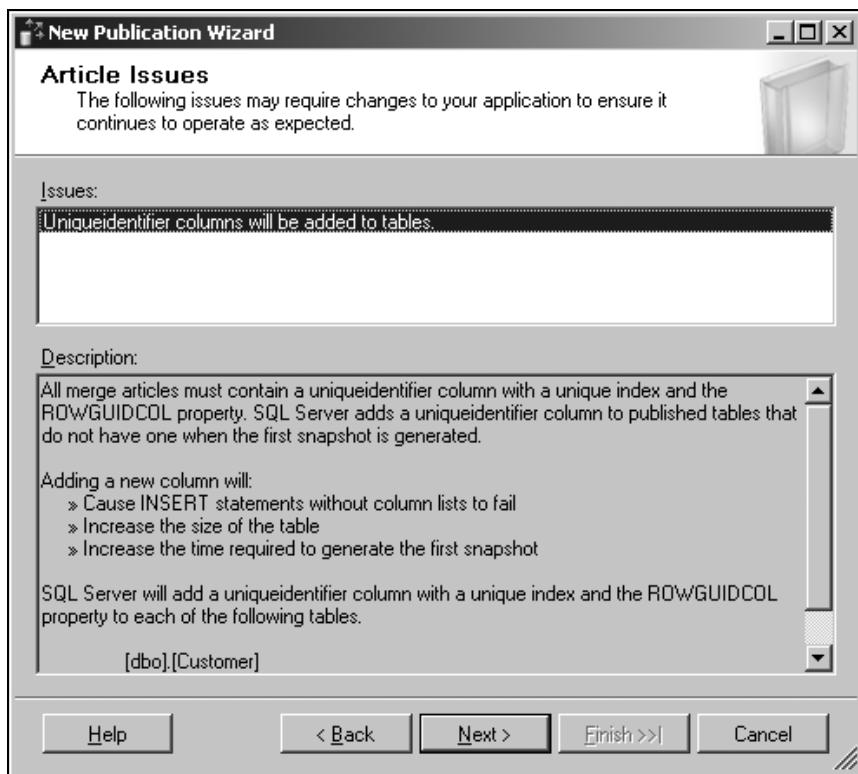


Рис. 13.9. Страница **Article Issues**

Показанная на рис. 13.9 страница **Article Issues** предоставляет список действий, которые будут предприняты. В случае таблиц *Customer* и *Order* вас информируют о том, что мастер создаст в каждой таблице столбцы *uniqueidentifier*, а также и соответствующие уникальные индексы. Merge Replication использует тип данных *uniqueidentifier* (это GUID) для идентификации строк при репликации. Мастер также настроит атрибут *ROWGUIDCOL*, который назначит значение GUID идентификатором каждой вставляемой в таблицу строки.

Для ограничения объема передаваемых при синхронизации данных мы добавим в публикацию фильтр (при помощи страницы **Filter Table Rows** мастера). Нажмите кнопку **Add**, а затем выберите опцию **Add Filter** (чтобы открыть показанное на рис. 13.10 диалоговое окно).

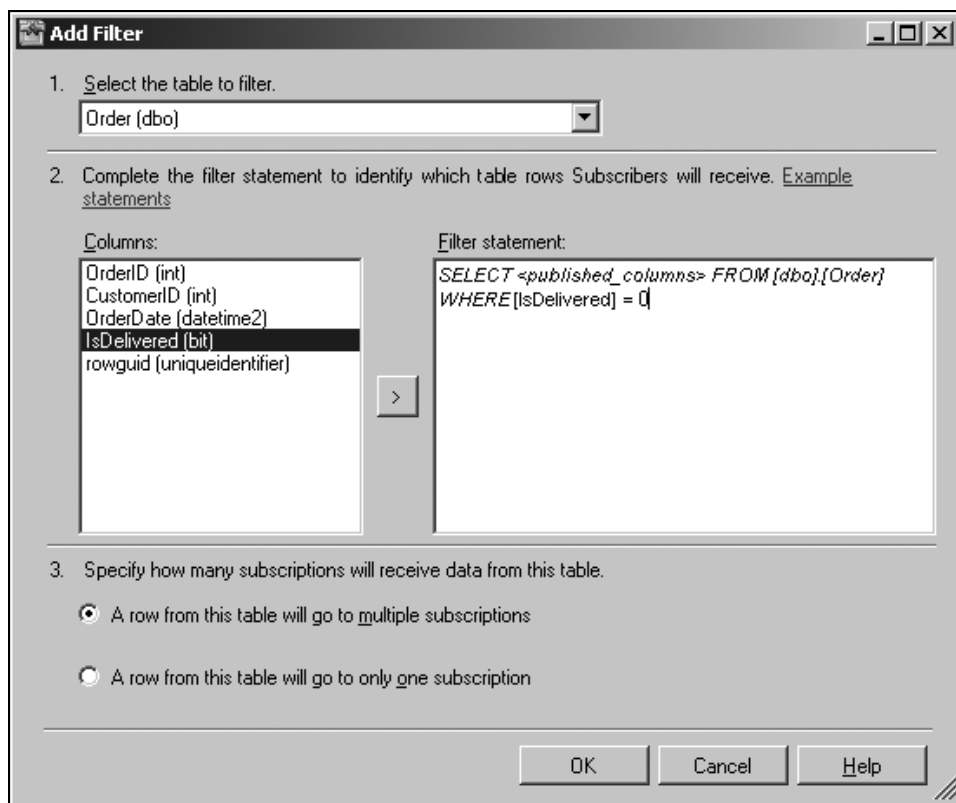


Рис. 13.10. Настройка фильтрации строк для репликации

Выберите в выпадающем списке таблицу `Order (dbo)`, а затем выберите столбец `IsDelivered (bit)` и перетащите его в текстовое поле инструкции фильтра. В текстовом поле **Filter statement** добавьте текст `=0` в конец инструкции. Инструкция фильтра должна выглядеть следующим образом:

```
SELECT <published_columns> FROM [dbo].[Order] WHERE IsDelivered = 0
```

Наш пример для различения доставленных и недоставленных заказов использует нулевое значение в столбце `IsDelivered`. При использовании этого фильтра на мобильное устройство будут загружаться только недоставленные заказы. Нажмите кнопку **OK** для того, чтобы закрыть диалоговое окно **Add Filter**.

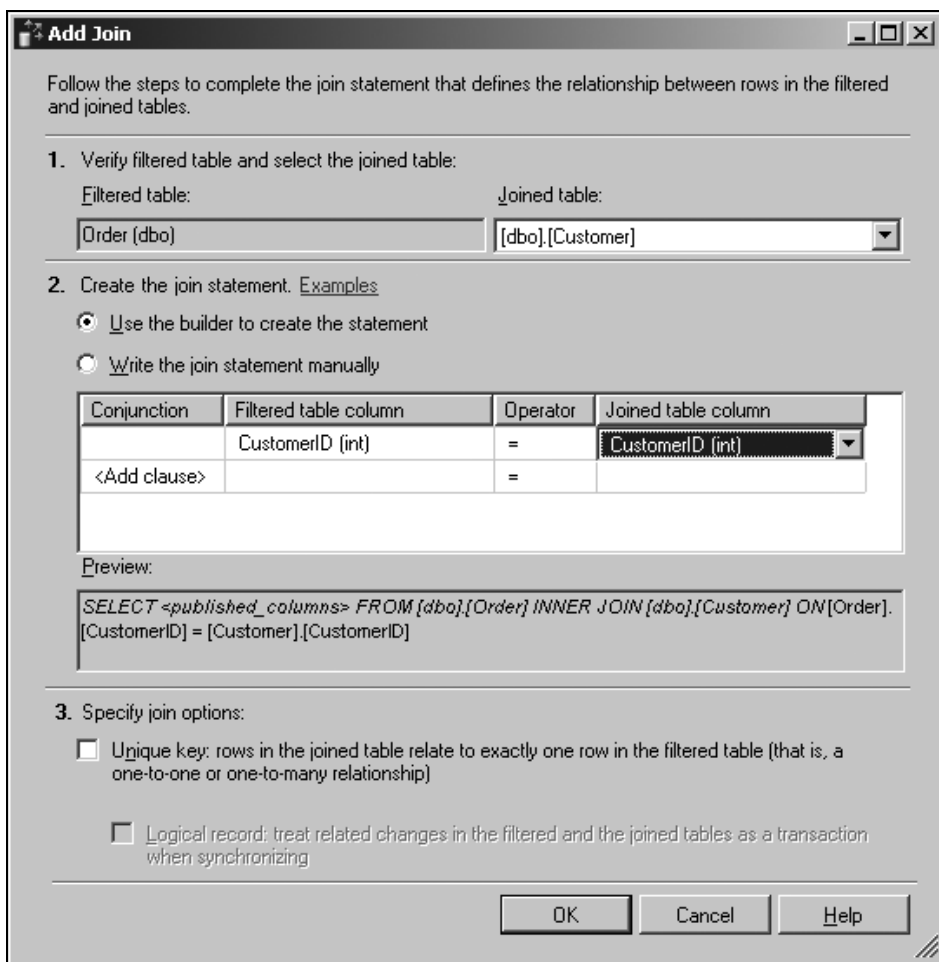


Рис. 13.11. Добавление соединения к фильтру

Мы собираемся также добавить к фильтру и соединение. Соединение дополнительно ограничивает возвращаемые данные только теми клиентами, у которых есть недоставленные заказы. На странице **Filter Table Rows** выберите таблицу `Order (dbo)`, нажмите кнопку **Add** и установите флажок **Add join to extend the selected filter**. В диалоговом окне **Add join** (рис. 13.11) выберите в выпадающем списке **Joined table** таблицу `[dbo].[Customer]`. Затем выберите в столбце **Filtered table** значение `CustomerID(int)` и в столбце **Joined table** также выберите значение `CustomerID(int)`, а потом нажмите кнопку **OK**.

Нажмите кнопку **Next** для перехода на страницу **Snapshot Agent**. На этой странице (рис. 13.12) вы можете выбрать немедленный запуск агента и настроить его работу по расписанию. Для нашего приложения нажмите кнопку **Change**, а затем настройте запуск агента в 4:30 PM ежедневно. После настройки расписания нажмите кнопку **OK** для закрытия диалогового окна **Job schedule properties**, а затем нажмите кнопку **Next**.

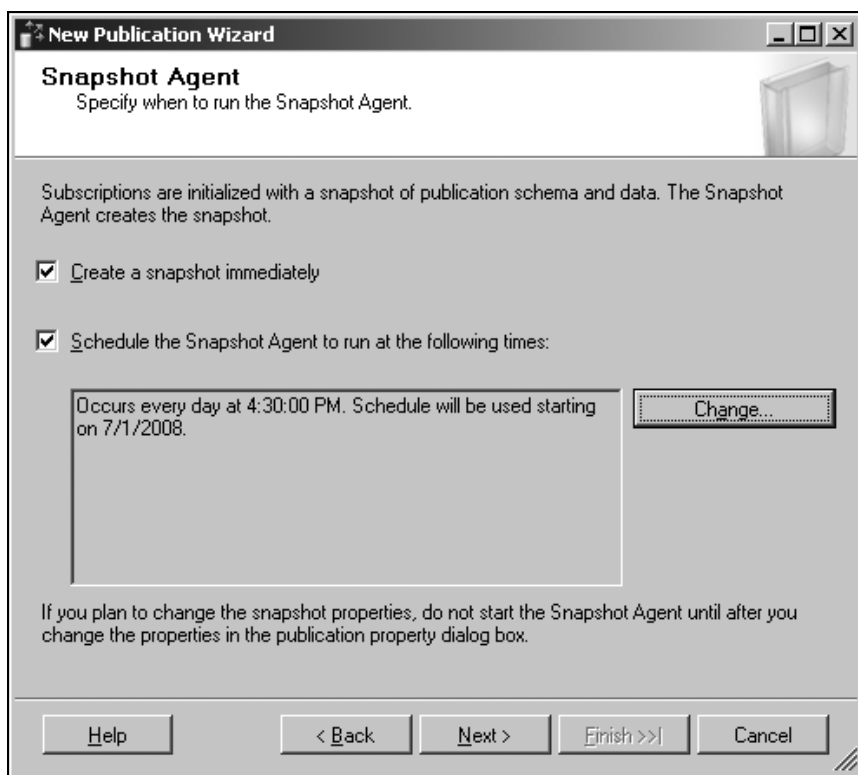


Рис. 13.12. Настройка расписания агента моментальных снимков

На странице **Snapshot Security** вы можете присвоить права доступа, с которыми будет работать агент моментальных снимков. Агент отвечает за создание файлов моментальных снимков, которые используются в процессе синхронизации.

Давайте создадим новую учетную запись Windows, под которой будет работать агент моментальных снимков. На некоторое время мы выйдем из SQL Server Management Studio и воспользуемся одной из форм Windows. Откройте приложение **Computer Management** (через меню **Administrative Tools**) для добавления нового пользователя. Создайте нового пользователя SnapshotAgent. Задайте для него пароль !Agent. Добавьте этого нового пользователя в локальную группу с названием `SQLServerMSSQLUser$<MachineName>$<SQLServerInstanceName>`. В нашем примере это будет группа `SQLServerMSSQLUser$SQL08DEV$MSSQLSERVER`.

### ПРИМЕЧАНИЕ

Для добавления учетных записей пользователей существует несколько методов (в зависимости от используемой вами версии Windows). Выберите тот метод, который вам лучше всего знаком.

Для нашей новой учетной записи нужно регистрационное имя SQL Server. Создайте новое регистрационное имя для группы `SQLServerMSSQLUser$SQL08DEV$MSSQLSERVER`. Это позволит получить доступ к серверу любым пользователям данной группы. Вернитесь в SQL Server Management Studio и запустите показанный в листинге 13.3 скрипт для создания регистрационного имени SQL Server.

#### Листинг 13.3. Создание регистрационного имени SQL Server для локальной группы

```
USE [master]
GO
CREATE LOGIN [SQL08DEV\SnapshotAgent] FROM WINDOWS WITH
DEFAULT_DATABASE=[master]
GO
EXEC master..sp_addsrvrolemember @loginame = N'SQL08DEV\SnapshotAgent',
@rolename = N'sysadmin'
GO
```

Ранее (при создании распространителя) мы создали также и совместно используемый каталог с названием ReplicationSnapshots. Нашей новой учетной записи Windows нужен доступ к этому разделяемому ресурсу на чтение и



запись. Откройте Windows Explorer. Перейдите к каталогу ReplicationSnapshots, а затем щелкните по нему правой кнопкой мыши (для отображения контекстного меню). Выберите пункт меню **Sharing and Security**. Для предоставления учетной записи SnapshotAgent доступа чтения/записи к разделяемому каталогу щелкните по пункту **Permissions**, а затем нажмите кнопку **Add** для добавления учетной записи SnapshotAgent. После ее добавления выберите учетную запись и предоставьте ей разрешения на изменение и чтение. Нажмите дважды на кнопку **OK** для закрытия диалоговых окон **Permissions** и **Properties**. Закройте Windows Explorer.

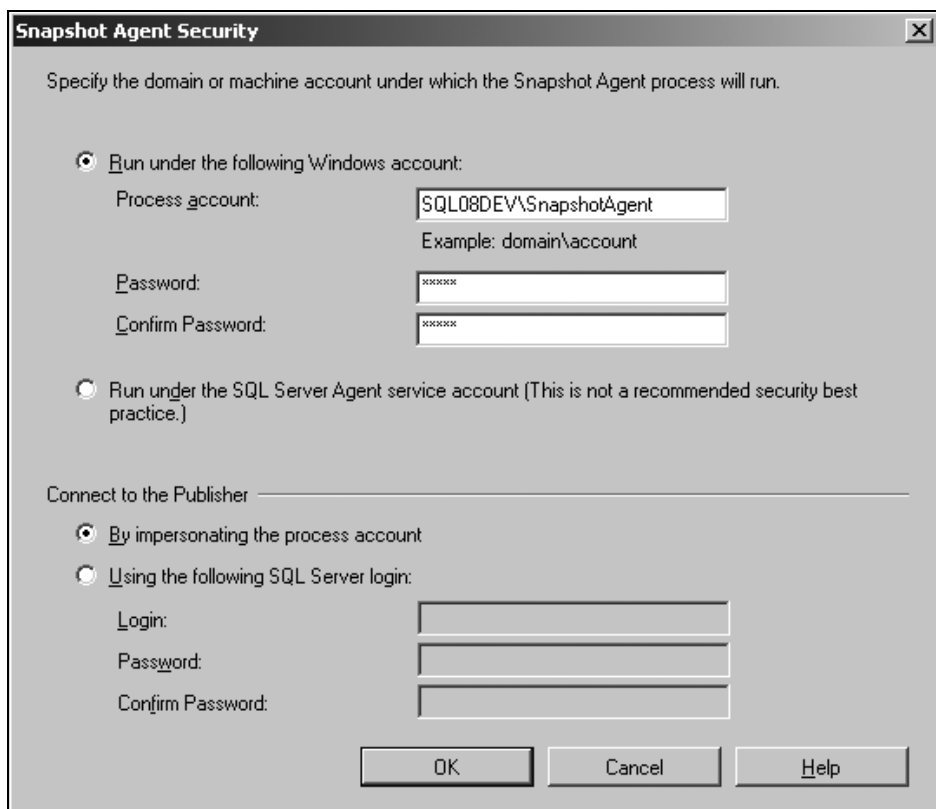


Рис. 13.13. Настройка безопасности агента моментальных снимков

Вернитесь на страницу **Agent Security** и нажмите кнопку **Security Settings** (чтобы открыть диалоговое окно **Snapshot Agent Security**, показанное на рис. 13.13). Выберите для процесса **Snapshot Agent** опцию **Run Under The**

**Following Windows account.** Введите учетную запись <machine name>\SnapshotAgent (обязательно укажите название домена или компьютера) и пароль !Agent (в полях **Process account** и **Password** соответственно). Убедитесь, что опция **Connect to the publisher** установлена в значение **By impersonating the process account**, а затем нажмите кнопку **OK**.

Нажмите кнопку **Next** для перехода к следующей странице мастера. На странице **Wizard Actions** вы можете указать, хотите ли вы создать публикацию прямо сейчас или вывести в скрипт все указанные вами опции (можно сделать и то, и другое). Выберите вариант **Create the publication**, а затем нажмите кнопку **Next**.

### **ВАЖНО**

Перед завершением работы мастера убедитесь, что служба SQL Server Agent запущена. Если она не работает, вы можете получить ошибки или предупреждения при создании мастером публикации.

На странице **Complete The Wizard** вы указываете название для новой публикации. Введите здесь `MyPublication`, а затем нажмите кнопку **Finish**. Мастер строит и выполняет скрипт, который создает публикацию. Теперь мы готовы начать настройку синхронизации.

Для того чтобы увидеть в SQL Server Management Studio новую публикацию `MyPublication`, в окне **Object Explorer** углубитесь в узел **Replication** и раскройте узел **Local Publications**.

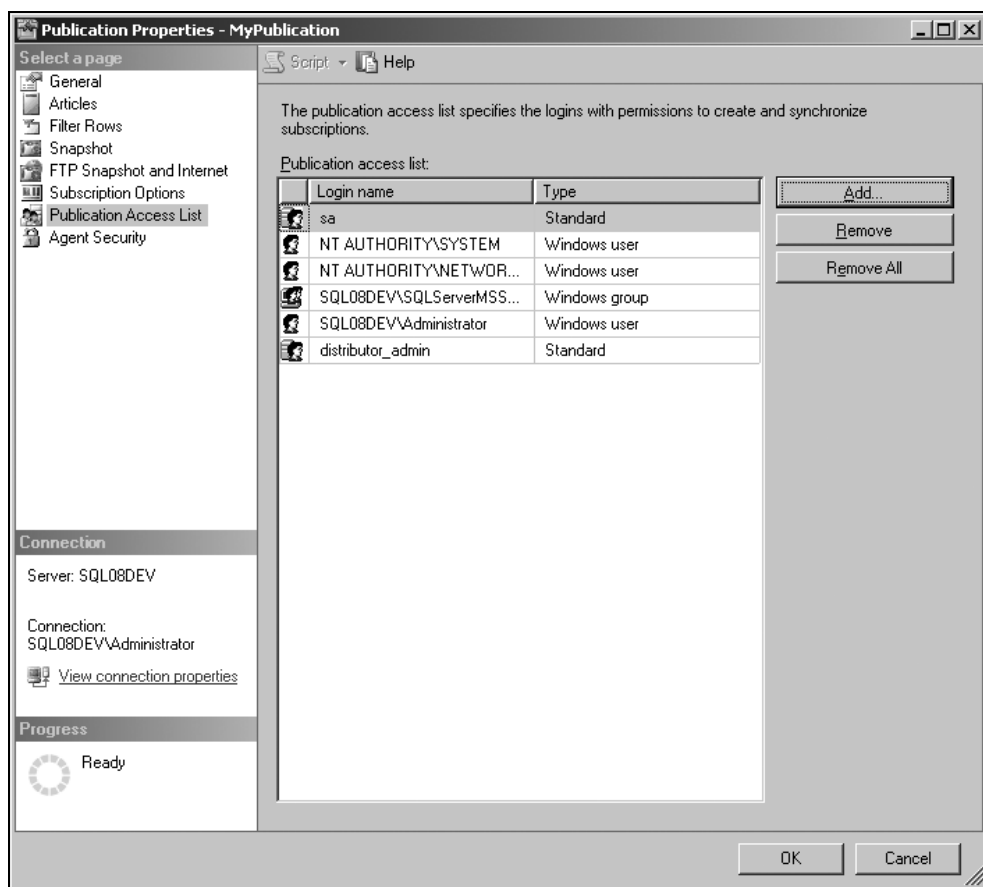
Для завершения настройки публикации нам необходимо выполнить вручную еще несколько пунктов, а именно: нам нужно настроить учетную запись SQL Server, которую подписчик использует для синхронизации с публикацией, а также предоставить этой учетной записи доступ к публикации.

Выполните показанный в листинге 13.4 скрипт для создания новой регистрационной записи SQL Server пользователя `SubscriberAccount` с паролем `account`. Этот скрипт добавляет в базу данных `SQL08SampleDB` нового пользователя и предоставляет ему роль `MSmerge_PAL_role`.

#### **Листинг 13.4. Создание учетной записи пользователя SubscriberAccount в SQL Server**

```
USE [master]
GO
CREATE LOGIN [SubscriberAccount] WITH PASSWORD=N'account',
```

```
DEFAULT_DATABASE=[SQL08SampleDB], CHECK_EXPIRATION=OFF, CHECK_POLICY=OFF
GO
USE [SQL08SampleDB]
GO
CREATE USER [SubscriberAccount] FOR LOGIN [SubscriberAccount]
GO
USE [SQL08SampleDB]
GO
EXEC sp_addrolemember N'MSmerge_PAL_role', N'SubscriberAccount'
GO
```

Рис. 13.14. Диалоговое окно **Publication Properties**

После завершения работы скрипта откройте диалоговое окно **Publication Properties** для MyPublication (для этого необходимо щелкнуть правой кнопкой мыши **Publication**, а затем выбрать в контекстном меню пункт **Properties**). Выберите страницу **Publication Access List** (как показано на рис. 13.14), а затем нажмите кнопку **Add**.

Учетная запись SubscriberAccount сервера SQL Server должна быть добавлена как учетная запись с правами доступа к публикации. Выберите из списка учетную запись SubscriberAccount, а затем нажмите кнопку **OK**. Нажмите кнопку **OK** еще раз (в окне **Publication Properties**, чтобы закрыть его).

Если все настроено правильно, то агент Snapshot Agent сервера SQL Server 2008 запустится сразу же после создания публикации. Первый запуск агента добавит файлы в сетевой разделяемый ресурс публикации. Для инициализации моментального снимка или для ручного запуска Snapshot Agent необходимо открыть диалоговое окно **View Snapshot Agent Status** (рис. 13.15) при помощи щелчка правой кнопки мыши по публикации MyPublication и последующего выбора в контекстном меню пункта **View snapshot agent status**. Нажмите кнопку **Start** для запуска агента.

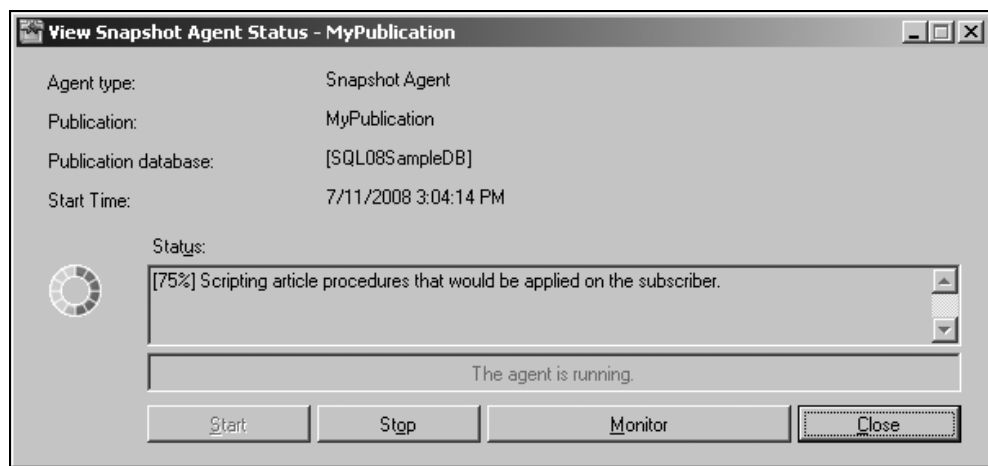
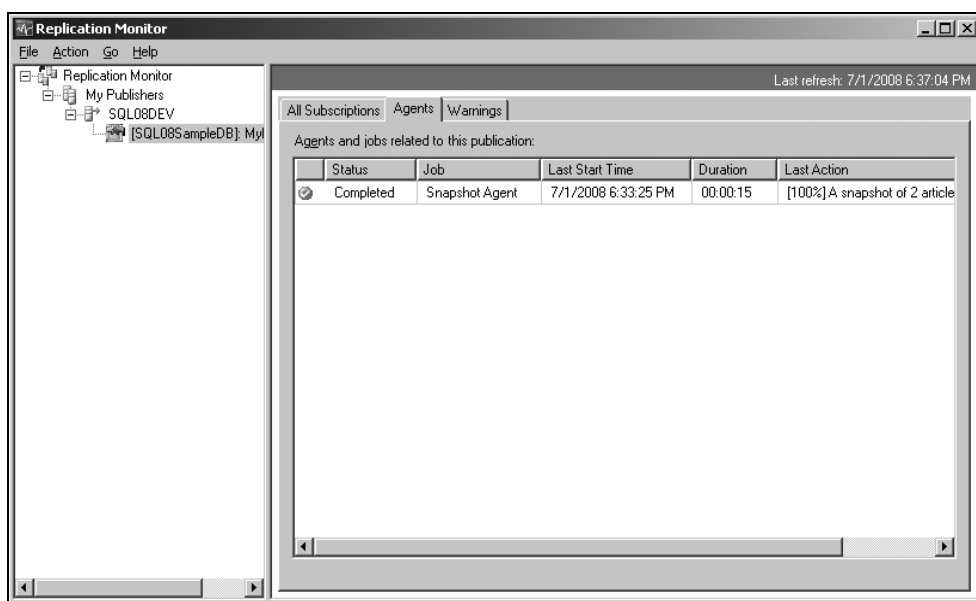


Рис. 13.15. Отслеживание состояния агента моментальных снимков

Вы можете наблюдать за агентом при помощи нажатия кнопки **Monitor**, которая открывает диалоговое окно **Replication Monitor** (рис. 13.16). Этот монитор показывает подробности по всем публикациям сервера, список подписчиков, список состояний агента и т. д.

Рис. 13.16. Диалоговое окно **Replication Monitor**

Теперь мы закончили настройку публикации *MyPublication*. Наш следующий шаг — сконфигурировать моментальный снимок, который будет доставляться через Интернет при помощи службы Internet Information Services (IIS). Merge Replication поддерживает способность доставлять моментальные снимки через Интернет при помощи протокола Hypertext Transfer Protocol (HTTP), это называется Web-синхронизация. Компания Microsoft рекомендует использовать эту синхронизацию, поэтому наш пример применяет Web-синхронизацию для синхронизации баз данных Publisher и Subscriber.

## Web-синхронизация для Merge Replication

Merge Replication поддерживает масштабируемую архитектуру для подписчиков, синхронизирующих данные через Интернет. Web-синхронизация требуется при синхронизации мобильных клиентов через Интернет либо через межсетевые экраны. При помощи протокола HTTP/HTTPS подписчики подключаются к нужному URL и передают данные как XML.

### **ПРИМЕЧАНИЕ**

Web-синхронизация поддерживает доставку моментальных снимков при помощи протокола FTP. Для этого и публикация, и подписки должны быть

настроены на использование FTP. Кроме того, вам понадобится сервер FTP (такой, как IIS). Вы должны также знать, что протокол FTP менее безопасен, поскольку при синхронизации пароль регистрационной записи посылается от подписчика на сервер FTP в незашифрованном виде.

Агент Merge Agent (работающий на компьютере подписчика) упаковывает изменения и отправляет их в виде XML на сервер IIS. Затем сервер IIS конвертирует данные в двоичный формат и направляет их издателю. Передача между IIS и издателем обычно выполняется при помощи протокола TCP/IP. Издатель упаковывает обновления в виде XML и отправляет их обратно подписчику через IIS.

### **ПРИМЕЧАНИЕ**

Web-синхронизация возможна только для подписок по запросу. Все компьютеры, которые участвуют в процессе Web-синхронизации, должны иметь SQL Server 2005 или более позднюю версию.

Web-синхронизация может быть сконфигурирована с одним, двумя или более серверами. В производственной среде (из соображений производительности и безопасности) следует конфигурировать Web-синхронизацию с использованием, по меньшей мере, двух серверов. Вот некоторые полезные практики для Web-синхронизации.

- ❑ Используйте два отдельных сервера: один для IIS, а другой для баз данных Publisher и Distributor. Отдельные серверы увеличивают масштабируемость и безопасность.
- ❑ Для шифрования передачи данных между подписчиком и сервером IIS используйте Secure Sockets Layer (SSL). В документации SQL Server Books Online отмечено, что для Web-синхронизации требуется SSL, однако при создании нашего примера вы увидите, что это не обязательно. Тем не менее, в производственной среде необходимо всегда применять SSL.
- ❑ На подписчике вместо интегрированной аутентификации используйте обычную проверку аутентификации. При помощи обычной аутентификации вы избежите потерь производительности на делегирование Kerberos между IIS и базой данных издателя.
- ❑ Все используемые в Web-синхронизации учетные записи Windows настраивайте с минимальным набором прав.

**ПРИМЕЧАНИЕ**

Более подробную информацию по обеспечению максимальной безопасности вашей производственной среды (в которой используется Web-синхронизация) смотрите в документации SQL Server 2008 Books Online.

Наш пример приложения использует вариант с одним сервером, анонимным доступом и незашифрованными подключениями — это простой вариант для создания прототипа приложения и не может быть рекомендован для развертывания в производственной среде.

Сделанные нами настройки публикации `myPublication` напрямую влияют на нашу способность использовать Web-синхронизацию. Точнее говоря, мы создали публикацию, в которой клиенты используют SQL Server Compact 3.1 (или более позднюю версию) и где тип публикации — `Merge Publication`. Эти настройки позволяют нам сконфигурировать Web-синхронизацию для нашего примера приложения.

**Конфигурирование IIS для Web-синхронизации**

Для конфигурирования Web-синхронизации надо выполнить три основных шага. Первый шаг — сконфигурировать публикацию (мы это уже сделали). Второй шаг — настроить IIS для Web-синхронизации. Третий шаг — сконфигурировать подписчика. В этом разделе описывается конфигурирование IIS для нашего примера.

**ПРИМЕЧАНИЕ**

Web-синхронизация в настоящее время поддерживается в IIS версий 5 и 6, но не в версии 7. Для нее требуется также Microsoft .NET Framework версии 2.0 или более поздняя. Версии .NET ранее 2.0 с сервера IIS рекомендуется удалить. Если на вашем сервере IIS существует несколько версий .NET Framework, то наша реализация Web-синхронизации может выдавать исключения.

IIS можно настроить либо вручную, либо при помощи мастера. Для запуска мастера раскройте узел **Local Publications** в узле **Replication** программы SQL Server Management Studio. Щелкните правой кнопкой мыши по публикации `myPublication`, а затем в контекстном меню выберите пункт **Configure Web Synchronization**.

Мастер проведет вас по настройке нескольких аспектов, включая виртуальный каталог и информацию по безопасности. Для начала нажмите кнопку **Next** на стартовой странице мастера.

Страница **Subscriber Type** запрашивает у вас информацию о версии SQL Server подписчика. Значение по умолчанию — SQL Server. Выберите здесь **SQL Server Compact**, а затем нажмите кнопку **Next**.

На странице **Web Server** вы указываете информацию о сервере IIS, который обеспечивает Web-синхронизацию. Здесь вы вводите имя сервера и даете мастеру указание создать новый виртуальный каталог в IIS.

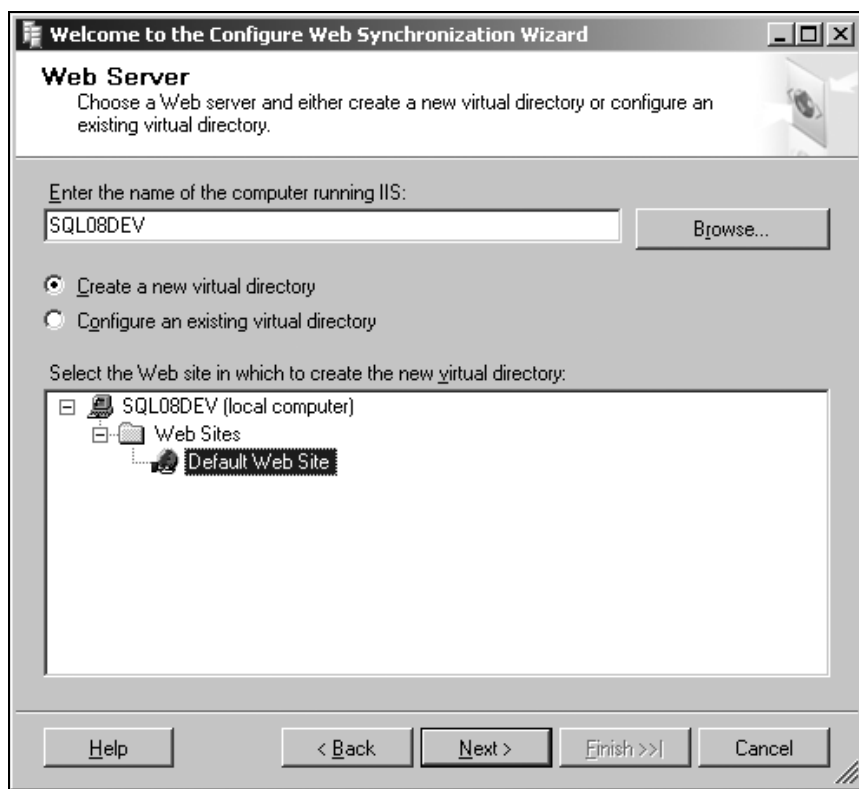


Рис. 13.17. Выбор Web-сайта и виртуального каталога для Web-синхронизации

### ПРИМЕЧАНИЕ

Если у вас не установлены инструменты SQL Server Compact Server Tools, то на данном этапе вы получите указание установить их. При необ-



ходимости установите эти инструменты. Инсталлятор (файл SSCEServer-Tools-ENU.msi) находится в каталоге C:\Program Files\Microsoft SQL Server Compact Edition\v3.5\Tools.

Выберите опцию **Create a new virtual directory**, а затем выберите узел **Default Web Site** в качестве места, где должен быть создан новый виртуальный каталог (рис. 13.17). Затем нажмите кнопку **Next**.

На странице **Virtual Directory Information** введите псевдоним для нового виртуального каталога. Используйте в качестве псевдонима `MyWebSync` и оставьте путь к виртуальному каталогу по умолчанию.

### **ПРИМЕЧАНИЕ**

Если путь не существует, то вы получите подсказку на его создание. Кроме того, вы можете получить подсказку "Copy and register the SQL Server Compact Server Agent". Агент SQL Server Compact Server Agent — это прослушиватель, который ждет запросов на синхронизацию от подписчиков и выполняет эти запросы путем обмена с издателем (для репликации данных). Нажмите **Yes** в обеих этих подсказках.

После создания виртуального каталога его URL будет таким: `http://<servername>/MyWebSync` (в нашем случае — `http://SQL08DEV/MyWebSync`). Сохраните этот URL на некоторое время. Он понадобится нам позже при конфигурировании Web-синхронизации на подписчике. Пока же нажмите кнопку **Next** для продолжения.

На странице **Secure Communications** вы указываете, нужно ли использовать SSL для шифрования передаваемых между подписчиком и сервером IIS данных. Выберите вариант **Do not require [SSL]**, а затем нажмите кнопку **Next**.

Страница **Client Authentication** позволяет вам указать метод, используемый для аутентификации подключающихся через IIS пользователей. Выберите вариант **Clients will connect anonymously**, а затем нажмите кнопку **Next**.

Укажите учетную запись Windows, которая будет использоваться при анонимном подключении. Для нашего примера мы будем использовать учетную запись по умолчанию `IUSR_<MachineName>`. Нажмите кнопку **Next**.

На странице **Snapshot Share Access** (рис. 13.18) вы получаете запрос имени того разделяемого сетевого ресурса, в котором находятся файлы моментальных снимков. Введите путь UNC к разделяемому сетевому ресурсу, который мы уже создали ранее — в нашем примере это будет `\\SQL08DeV\ReplicationSnapshots`.

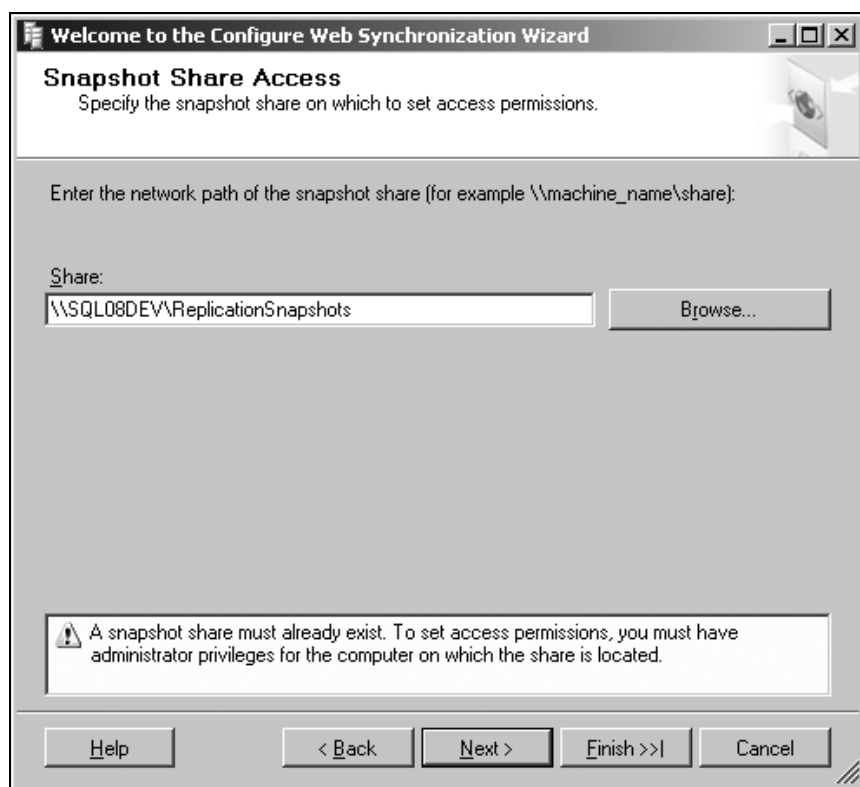


Рис. 13.18. Страница **Snapshot Share Access**

Нажмите кнопку **Next**, а затем **Finish**.

#### **ПРИМЕЧАНИЕ**

Если вы настраиваете Merge Replication в первый раз, то можете получить сообщение о том, что каталог моментальных снимков пуст. Нажмите **Yes** для продолжения работы мастера.

После нажатия **Finish** мастер сконфигурирует Web-синхронизацию по указанным вами параметрам.

#### **ПРИМЕЧАНИЕ**

Если вы планируете реплицировать большие объемы данных, необходимо будет учесть предельный размер пакета агента Merge Agent. Web-синхронизация передает данные между подписчиком и IIS блоками, макси-

мальный размер которых 25 Мбайт. В этот предельный размер входят и издержки на теги XML, которые обертывают реплицируемые данные. Это ограничение означает, что вы не можете реплицировать такие данные столбца, которые могут оказаться больше 25 Мбайт, например, двоичный файл, хранящийся в столбце `varbinary(max)`.

## Конфигурирование базы данных подписки для Web-синхронизации

Итак, мы закончили два из трех шагов, необходимых для настройки нашего приложения под Web-синхронизацию. Третий и последний шаг — настройка базы данных подписки. В этом разделе мы настроим базу данных подписки, работающую под управлением SQL Server Compact 3.5. Перед тем как выполнить этот последний шаг конфигурации, давайте создадим базу данных нашего приложения.

В программе SQL Server Management Studio нажмите кнопку **Connect**, а затем выберите опцию **SQL Server Compact**. Щелкните по комбинированному окну **Database file**, а затем выберите **<New Database...>** в диалоговом окне **Connect To Server**. В диалоговом окне **Create New SQL Server Compact Database** (рис. 13.19) вы вводите информацию для создания новой базы данных сервера SQL Server Compact. Введите в качестве имени файла базы данных `C:\Projects\MergeRepSubscriber.sdf` и укажите пароль для вашей новой базы данных. Вы можете использовать любой пароль либо вовсе его не использовать, хотя и получите предупреждение (которое можете игнорировать) в случае предоставления пустого или слабого пароля. В нашем случае мы укажем в качестве пароля `password`.

Нажмите кнопку **ОК** для создания новой базы данных и закройте диалоговое окно, а затем ответьте **Yes** для того, чтобы игнорировать предупреждение о слабом пароле. (Указанный нами пароль не имеет букв в разных регистрах и символов, а это считается слабостью пароля).

Перейдите к новой базе данных в окне **Object Explorer**, а затем разверните узел **Replication**. Щелкните правой кнопкой мыши по узлу **Subscriptions**, а затем выберите в контекстном меню пункт **New Subscriptions**, чтобы открыть мастер New Subscription Wizard. Нажмите кнопку **Next** для запуска мастера.

Выберите на странице **Choose Publication** публикацию `MyPublication`, а затем нажмите кнопку **Next**.

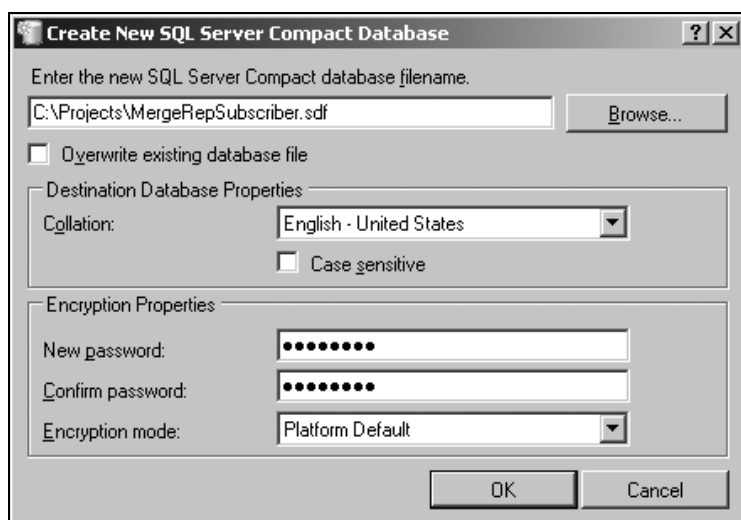


Рис. 13.19. Создание новой базы данных сервера SQL Server Compact

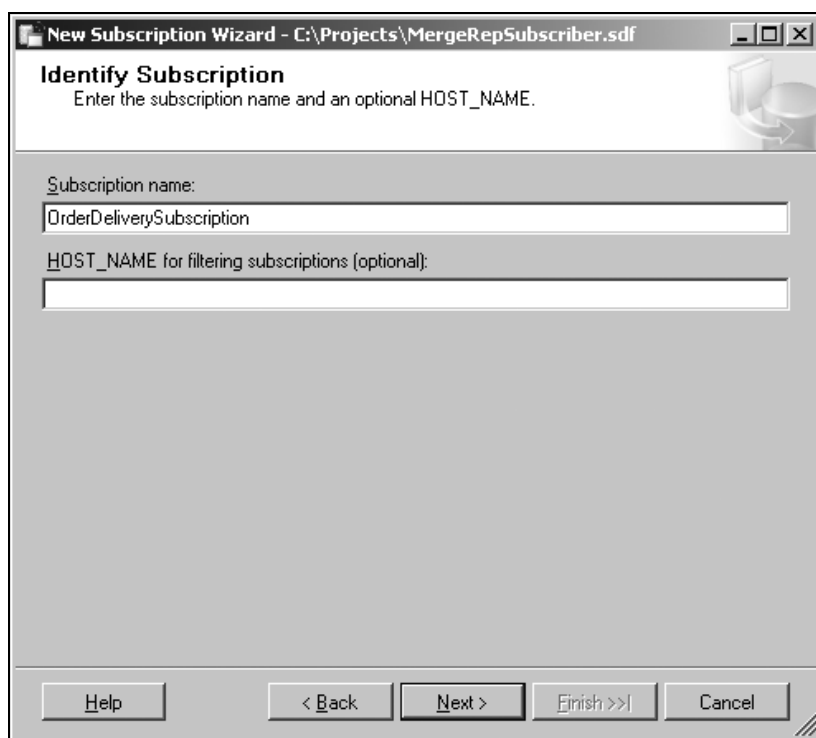


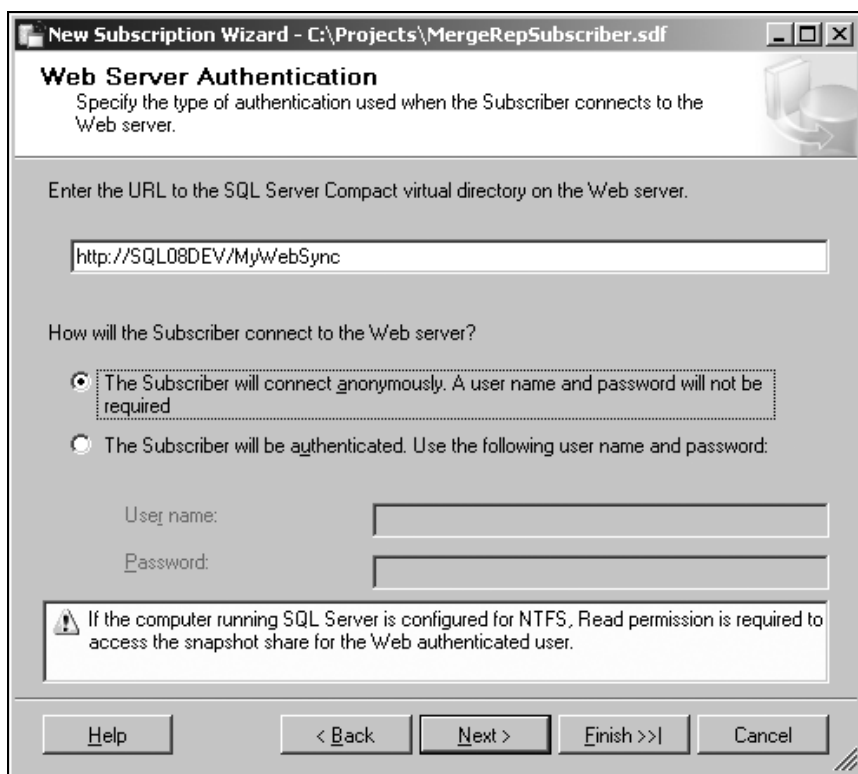
Рис. 13.20. Настройка подписки

**ПРИМЕЧАНИЕ**

Для того чтобы выбрать базу данных публикации, вам может понадобиться сначала перейти к серверу распространения.

На странице **Identify Subscription** (рис. 13.20) введите в качестве имени подписки `OrderDeliverySubscription`. Значение поля **HOST\_NAME** используется для изоляции подписок по именам рабочих станций. Наше приложение не изолирует подписок, поэтому оставьте поле **HOST\_NAME** пустым, а затем нажмите кнопку **Next** для продолжения.

На странице **Web Server Authentication** (рис. 13.21) вы вводите все подробности относительно подключения подписчика к Web-серверу. Введите созданный ранее URL, для нашего примера это `http://SQL08DeV/MyWebSync`. Выберите опцию анонимного подключения к Web-серверу, а затем нажмите кнопку **Next**.



**Рис. 13.21.** Настройка аутентификации Web-сервера для подписки

На странице **SQL Server Authentication** (рис. 13.22) выберите опцию использования аутентификации SQL Server, а затем введите имя пользователя `SubscriberAccount` и пароль `account` (это созданная нами ранее учетная запись SQL).

Нажмите кнопку **Next**. Мастер генерирует и показывает код для использования в нашем клиентском приложении. Скопируйте код на C# со страницы **Complete The Wizard**, начиная чуть ниже текста `Code Sample (C#)` и заканчивая чуть выше текста `Code Sample (Visual Basic.NET)`. Откройте ваш любимый редактор и вставьте в него этот код. Мы вернемся к нему позже. Нажмите кнопку **Finish** для того, чтобы мастер настроил подписчика.

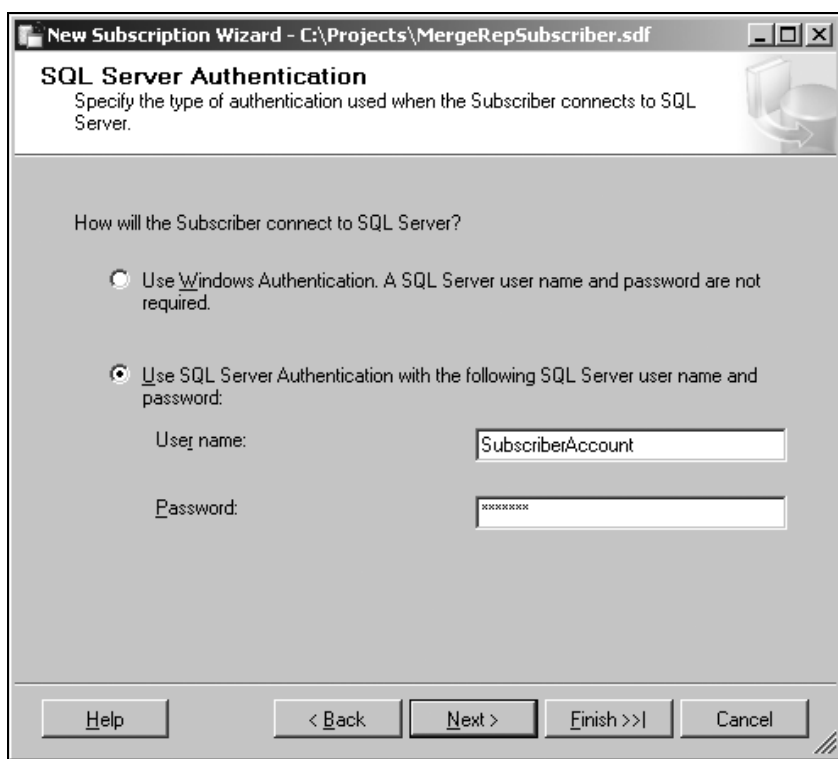


Рис. 13.22. Настройка аутентификации SQL Server для подписчика

## Проверка подписки при помощи Management Studio

Для того чтобы проверить, что подписка создана и синхронизирована, подключитесь к мобильной базе данных `MergeRepSubscriber.sdf` при помощи

SQL Server Management Studio. Там вы увидите, что таблицы `Customer` и `Order` уже были протолкнуты репликацией в базу данных. Кроме того, если вы выполните запрос `SELECT` по таблице `Order`, то вы увидите данные таблицы. Обратите внимание, что в таблице имеются только такие строки, где значение `IsDelivered` равно нулю. Помимо обычных столбцов таблицы `Order` вы увидите также `rowguid` и другие столбцы (используемые репликацией для отслеживания).

Теперь, когда у нас настроены и распространение, и публикация, и подписка, мы можем сосредоточиться на создании мобильного приложения для нашего персонала отдела доставки.

## Создание мобильного приложения при помощи Microsoft Visual Studio 2008

Наш пример — это приложение для карманного ПК, разработанное при помощи Visual Studio 2008. Приложение синхронизирует базу данных SQL Server Compact 3.5 с только что созданной нами публикацией SQL Server 2008.

Запустите Visual Studio 2008. В окне **Server Explorer** щелкните правой кнопкой мыши по пункту **Data Connections**, а затем выберите **Add Connection**. В диалоговом окне **Choose Data Source** выберите **Microsoft SQL Server Compact 3.5**, а затем нажмите **Continue**. Появится диалоговое окно **Add Connection** (рис. 13.23). Введите путь к файлу `MergeRepSubscriber.sdf` (в нашем примере это `C:\Projects\MergeRepSubscriber.sdf`) и пароль (который мы указали ранее как `password`), а затем нажмите **Test Connection** для проверки настроек. Нажмите кнопку **OK** для закрытия диалогового окна **Add Connection**.

Теперь мы создадим проект типа Pocket PC. Выберите **File | New**, а затем выберите пункт **Project**. На форме **New Project** разверните узел **Visual C# node**, а затем щелкните по узлу **Smart Device**. После этого вы увидите несколько шаблонов. Выберите **Smart Device Project**. Назовите новый проект `MyPocketPCDemo` и поместите его в каталог `C:\Projects` (или в другой каталог по вашему выбору). Нажмите кнопку **OK** для создания проекта. После создания проекта вы должны будете выбрать целевую платформу, версию Microsoft .NET Compact Framework, а также шаблон проекта. Выберите соответственно `Windows Mobile 5.0 Pocket PC SDK`, `.NET Compact Framework Version 3.5` и `Device Application` (рис. 13.24). Нажмите кнопку **OK** для продолжения.



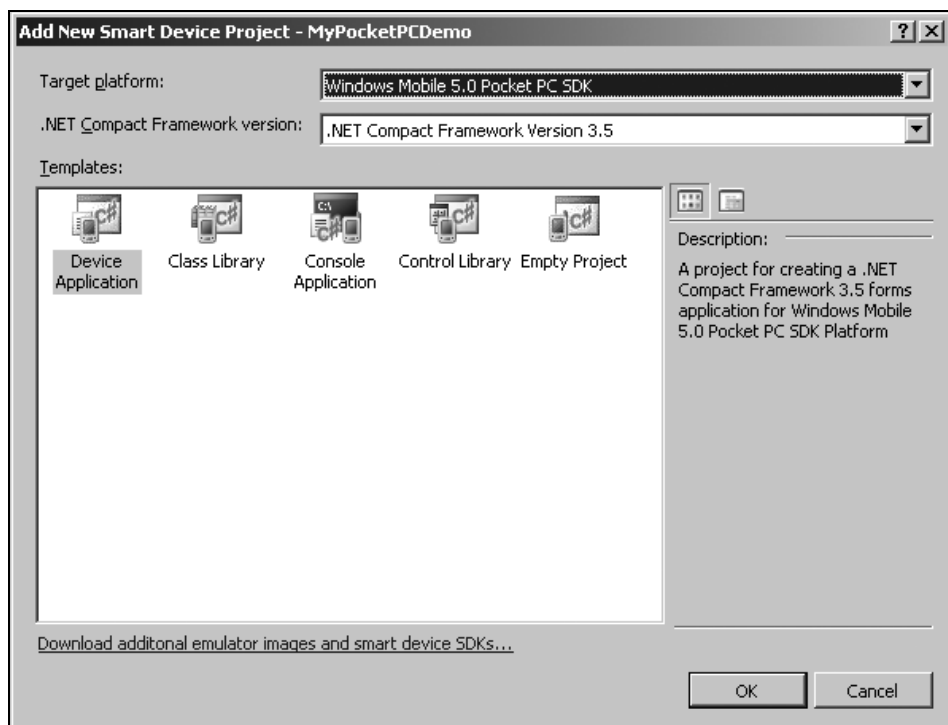
Рис. 13.23. Создание подключения к базе данных SQL Server в Visual Studio 2008

Давайте изменим некоторые настройки проекта по умолчанию. Сначала изменим название формы с `Form1` на `CustomerOrdersForm`. Для этого в окне **Solution Explorer** щелкните правой кнопкой мыши по узлу **Form1.cs**, а затем выберите **Rename**. Измените название на `CustomerOrdersForm.cs`. (На запрос о переименовании ссылок в коде проекта ответьте **Yes**). Затем установите свойство **Text** формы в значение **Customer Orders**.

## Настройка источника данных

Теперь мы создадим источник данных. При наличии источника данных вы можете перетаскиванием создавать на форме объекты `DataGrid`. Выберите в меню **Data** в Visual Studio 2008 пункт **Add New Data Source** для запуска мастера **Data Source Configuration Wizard**. Нажмите значок **Database**, а затем нажмите кнопку **Next**.

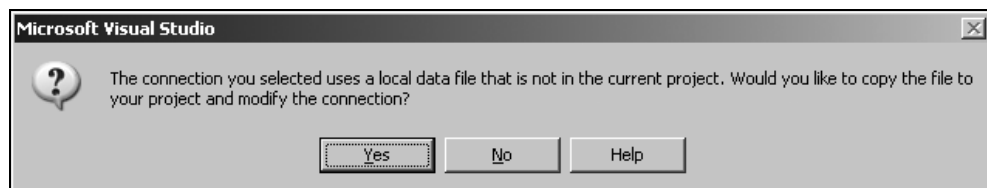




**Рис. 13.24.** Создание нового проекта Smart Device для .NET Compact Framework под устройство Windows Mobile Pocket PC

По умолчанию должно появиться правильное подключение данных. Если оно не появится, выберите базу данных MergeRepSubscriber. Для простоты выберите вариант включения конфиденциальной информации в строку подключения, а затем нажмите кнопку **Next**.

Visual Studio делает запрос на добавление в проект локального файла данных (рис. 13.25). Нажмите **Yes** для копирования файла данных.



**Рис. 13.25.** Запрос Visual Studio на добавление локального файла данных в текущий проект

На странице **Choose Your Database Objects** (рис. 13.26) разверните узел **Tables** для того, чтобы открыть таблицы базы данных MergeRepSubscriber.

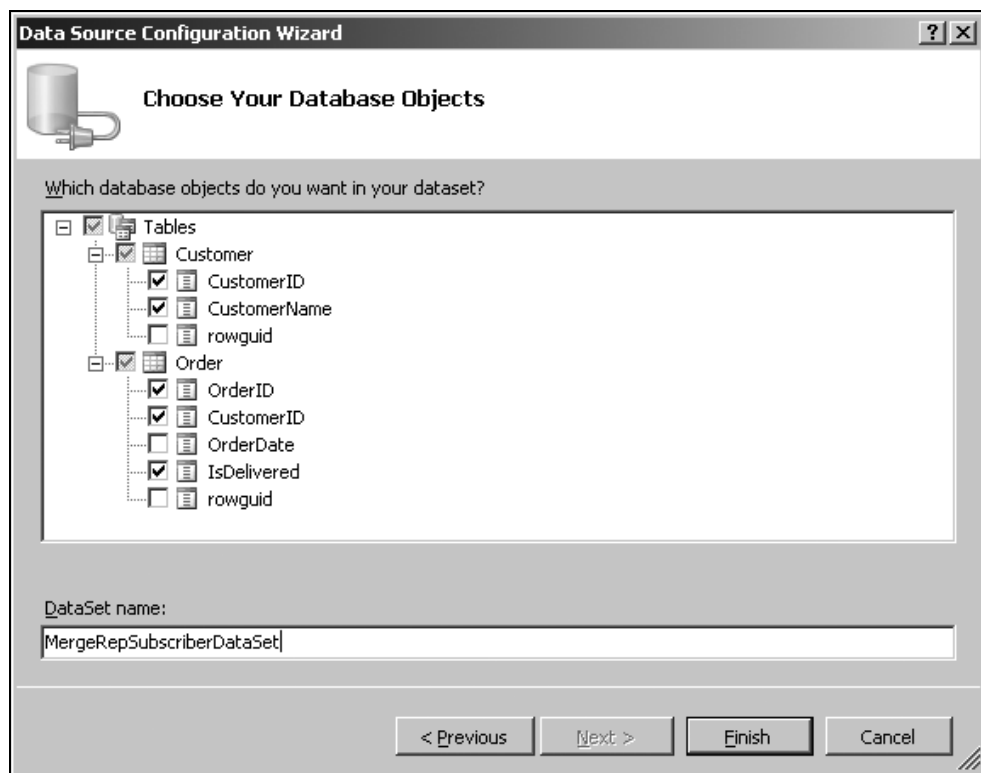
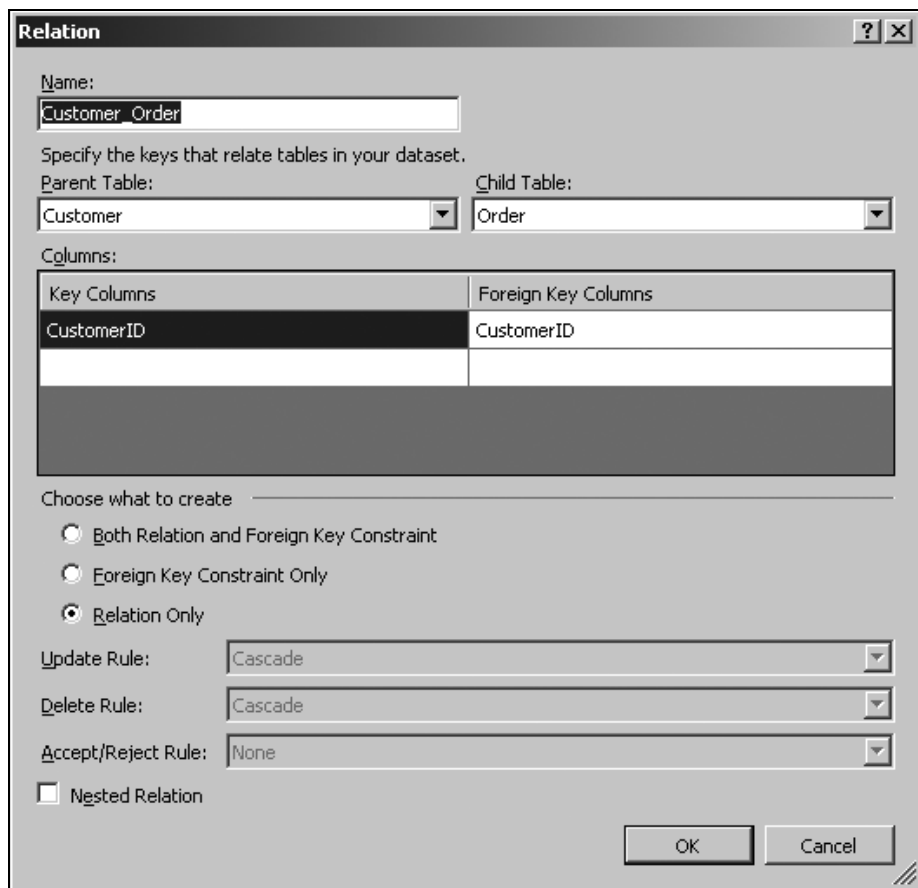


Рис. 13.26. Мастер Data Source Configuration Wizard

Выполните следующие шаги для выбора объектов базы данных, используемых в нашем примере приложения:

1. Разверните узел **Customer**.
2. Выберите столбцы **CustomerID** и **CustomerName**. Вы заметите добавленный сервером SQL Server столбец **rowguid** (для отслеживания репликации), он не нужен при отображении данных для пользователя, поэтому его отмечать не надо.
3. Разверните узел **Order**.
4. Выберите столбцы **OrderID**, **CustomerID** и **IsDelivered**.

5. Оставьте значение **DataSet Name** по умолчанию.
6. Нажмите **Finish**.



**Рис. 13.27.** Определение связи между Customer и Order в наборе данных

Построенный из этих объектов DataSet содержит две таблицы. Мастер Data Source Wizard не подхватывает связь между таблицами Customer и Order, поэтому нам придется добавить ее самим. В окне **Solution Explorer** дважды щелкните по файлу **MergeRepSubscriberDataSet.xsd** для того, чтобы открыть визуальный конструктор. Щелкните правой кнопкой мыши по любому свободному месту, а потом выберите в контекстном меню пункт **Add**, а затем **Relation**. Visual Studio образует связь на основе совпадения типов данных

столбцов. В данном случае она фиксирует совпадение столбцов CustomerID в таблицах Order и Customer.

Нажмите кнопку **ОК** в диалоговом окне **Relation** (рис. 13.27) для создания связи, а затем сохраните и закройте окно визуального конструктора.

## Привязка данных SQL Server Compact к форме карманного ПК

Теперь мы привяжем к нашей форме несколько элементов управления (при помощи имеющейся в Visual Studio 2008 технологии перетаскивания).

Вернемся к форме **CustomerOrdersForm**. В меню **Data** выберем **Show Data Sources**. В окне **Data Sources** выберем таблицу **Customer**, появится выпадающая кнопка. Выберите в выпадающем меню пункт **Details**. Углубитесь в таблицу **Customer** (нажатием на значок плюса слева от таблицы). Выберите столбец **CustomerID** и настройте его элемент управления в значение **None**. Выберите столбец **CustomerName** и настройте его элемент управления в значение **ComboBox**. Углубитесь в таблицу **Order** (под таблицей **Customer**) и разверните узел для того, чтобы увидеть все поля. Установите элементы управления для столбцов **OrderID**, **CustomerID** и **IsDelivered** в значения **Label**, **None** и **CheckBox** соответственно (при помощи смарт-тега каждого столбца).

Перетащите таблицу **Customer** и разместите ее где-нибудь в верхней левой части формы. На форме должны появиться элементы управления **Label** и **ComboBox**. Затем перетащите на форму таблицу **Order** (она также показана как дочерний узел таблицы **Customer**) и поместите ее на форму под элементами управления клиентами. Появится элемент управления **DataGrid** для отображения заказов клиентов.

Затем поместите кнопку в нижний правый угол формы. Назовите кнопку **btnSync** и установите ее свойству **Text** значение **Sync**. При работающем приложении нажатие этой кнопки выполняет ручную синхронизацию между устройством и сервером.

## Настройка свойств элементов управления

Вы могли бы подумать, что после перетаскивания на вашу форму таблицы **Customer** все свойства привязки данных для вас уже настроены. Но это не так. Вам необходимо самостоятельно настроить несколько свойств. Просмотрите свойства элемента управления **ComboBox** и сделайте изменения, перечисленные в табл. 13.3.

**Таблица 13.3.** Значения свойств для элемента управления *customerNameComboBox*

| Название свойства    | Значение свойства                       |
|----------------------|-----------------------------------------|
| DataSource           | customerBindingSource                   |
| DisplayMember        | CustomerName                            |
| ValueMember          | CustomerID                              |
| DropDownStyle        | DropDownList                            |
| (DataBindings), Text | Удалите привязку данных к свойству Text |

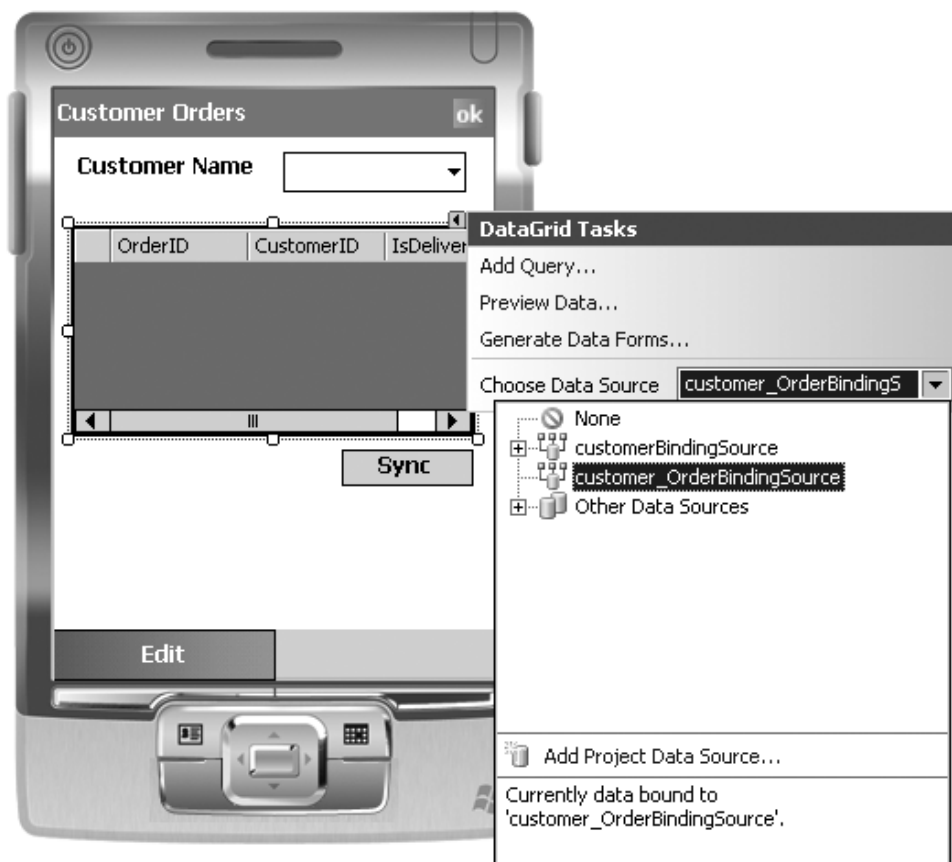
Теперь давайте сосредоточимся на элементе управления `DataGrid`. Первым делом нам необходимо изменить его свойство `DataSource`. По умолчанию `DataSource` настроено на саму таблицу. Однако мы хотим использовать (в качестве источника данных сетки) нашу новую связь. При использовании этой связи строки в сетке будут меняться при перемещении пользователя по клиентам в выпадающем списке (и для этого нам не придется писать никакого кода). Выберите сетку, а затем нажмите ее смарт-тег для показа контекстного меню **DataGrid Tasks**. Убедитесь, что в выпадающем списке **Choose Data Source** указано значение `customer_OrderBindingSource` (рис. 13.28).

Элемент управления `DataGrid` в форме карманного ПК — только "для чтения". Для редактирования данных в сетке вам понадобится добавить в приложение дополнительные формы. Нажмите смарт-тег сетки данных, откройте контекстное меню **DataGrid Tasks**, а затем нажмите кнопку **Generate Data Forms**. Visual Studio добавляет в проект две новых формы: одна для создания новых заказов, а другая — для редактирования существующих заказов. Добавляется также и код для перехода на эти формы с формы `CustomerOrdersForm`. В нашем примере нас интересует только редактирование существующих заказов, поэтому мы несколько модифицируем данные формы. При генерировании этих форм Visual Studio добавляет в главное меню формы `CustomerOrdersForm` кнопку **New** для навигации. По умолчанию эта кнопка добавляет новые заказы. Мы собираемся изменить это поведение, чтобы нажатие данной кнопки приводило нас непосредственно на страницу редактирования.

Сохраните все сгенерированные формы, а затем вернитесь к `CustomerOrdersForm`. Выберите кнопку **New**, а затем клавишу <F4> для показа ее свойств. Измените значение свойства `Text` на `Edit`. Дважды щелкните

по кнопке **Edit** для того, чтобы открыть код ее обработчика событий. Удалите сгенерированный код и замените его следующими строками:

```
MyPocketPCDemo.OrderEditViewDialog orderEditViewDialog =  
    MyPocketPCDemo.OrderEditViewDialog.Instance  
    (this.customer_OrderBindingSource);  
orderEditViewDialog.ShowDialog();
```



**Рис. 13.28.** Привязка сетки к связи `Customer_Order`, которая приводит к автоматической фильтрации заказов по клиентам при перемещении пользователя от клиента к клиенту

Этот код при нажатии пользователем кнопки **Edit** отображает форму `OrderEditViewDialog`. Теперь откройте код файла `OrderEditViewDialog.cs` (при помощи щелчка правой кнопкой мыши по файлу в окне **Solution Ex-**

**plorer** и последующего выбора пункта **View Code**). В конструктор класса добавьте следующую строку кода (следом за вызовом `InitializeComponent`):

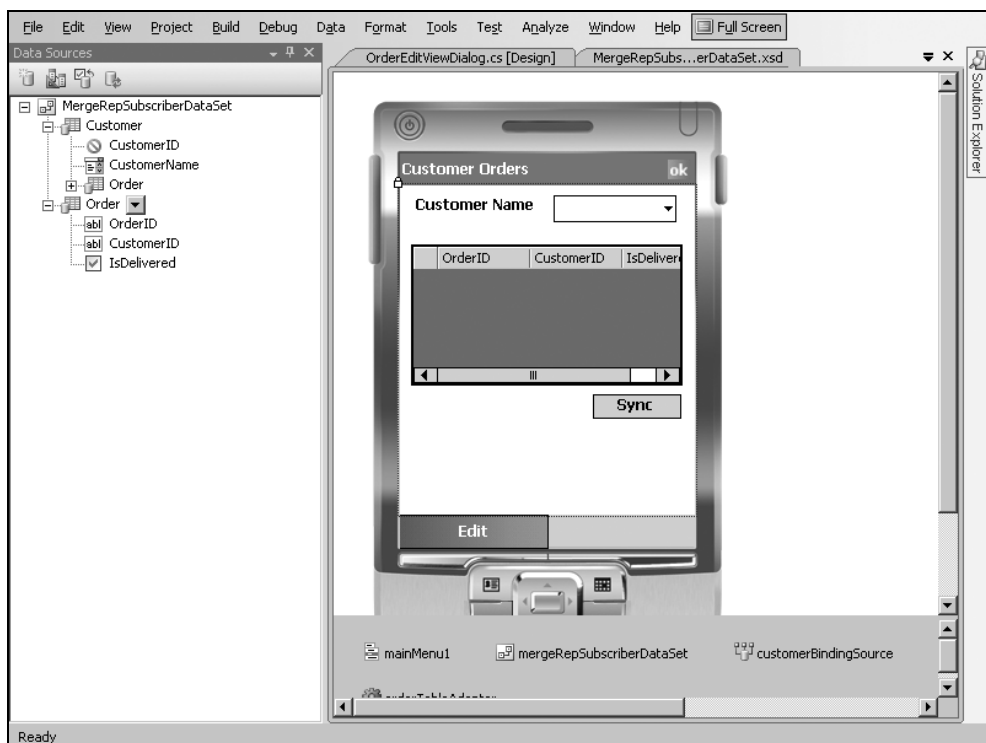
```
this.Closing +=  
    new System.ComponentModel.CancelEventHandler  
(this.OrdersEditViewDialog_Closing);
```

Эта строка кода настраивает обработчик событий при закрытии формы, он сбрасывает все значения из элементов управления пользовательского интерфейса обратно в элемент `DataSet`. Обработчик событий является частью кода листинга 13.5 (который вы должны теперь добавить в файл `OrderEditViewDialog.cs`).

**Листинг 13.5. Одноэлементный метод, который возвращает экземпляр класса `OrderEditViewDialog`**

```
private static OrderEditViewDialog _instance = null;  
public static OrderEditViewDialog Instance  
    (System.Windows.Forms.BindingSource bindingSource)  
{  
    Cursor.Current = Cursors.WaitCursor;  
    if (_instance == null)  
    {  
        _instance = new MyPocketPCDemo.OrderEditViewDialog();  
        _instance.orderBindingSource.DataSource = bindingSource;  
    }  
    _instance.AutoScrollPosition = new Point(0, 0);  
    _instance.orderBindingSource.Position = bindingSource.Position;  
    Cursor.Current = Cursors.Default;  
    return _instance;  
}  
private void OrdersEditViewDialog_Closing(object sender,  
                                           CancelEventArgs e)  
{  
    this.orderBindingSource.EndEdit();  
}
```

На данный момент вы создали привязанную к данным форму сетки для базы данных SQL Server Compact 3.5 (рис. 13.29).



**Рис. 13.29.** Компонка пользовательского интерфейса примера приложения для карманного ПК

## Программная синхронизация

Теперь нам нужно написать код синхронизации приложения в `CustomerOrdersForm`. Для этого потребуются незначительные модификации кода формы (который был сгенерирован Visual Studio), интеграция сгенерированного ранее мастером SQL Server Compact 3.5 Subscription Wizard кода (который вы сохранили в своем любимом редакторе) и добавление небольшого количества дополнительного кода. Полный код формы показан в листинге 13.6. Заглядывайте в него, пока мы шаг за шагом будем описывать реализацию этого кода.

### ПРИМЕЧАНИЕ

Как и все прочие коды примеров, вы можете скачать полный код данного примера с Web-сайта этой книги. Подробности по скачиванию смотрите во введении.



**Листинг 13.6. Код доступа к данным и синхронизации для приложения карманного ПК (выделенный код формы CustomerOrdersForm)**

```
using System;
using System.Linq;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Text;
using System.Windows.Forms;
using System.Data.SqlServerCe;
namespace MyPocketPCDemo
{
    public partial class CustomerOrdersForm : Form
    {
        public CustomerOrdersForm()
        {
            InitializeComponent();
        }
        private void CustomerOrdersForm_Load(object sender, EventArgs e)
        {
            // Замените вставленный мастером код этой единственной строкой:
            this.FillTables();
        }
        private void FillTables()
        {
            // Местоположение файла .sdf на устройстве.
            const string ConnString = @"Data Source=
                ""\Program Files\MyPocketPCDemo\MergeRepSubscriber.SDF"";
                Password=""password"";Max Database Size=128;
                Default Lock Escalation=100;";
            if (MergeRepSubscriberDataSetUtil.DesignerUtil.IsRunTime())
            {
                this.customerTableAdapter.Connection.ConnectionString =
                    ConnString;
                this.orderTableAdapter.Connection.ConnectionString = ConnString;
            }
        }
    }
}
```

```

        this.customerTableAdapter.Fill
            (this.mergeRepSubscriberDataSet.Customer);
        this.orderTableAdapter.Fill
            (this.mergeRepSubscriberDataSet.Order);
    }
}

private void newMenuItemMenuItem_Click(object sender, EventArgs e)
{
    MyPocketPCDemo.OrderEditViewDialog orderEditViewDialog =
        MyPocketPCDemo.OrderEditViewDialog.Instance
            (this.customer_OrderBindingSource);
    orderEditViewDialog.ShowDialog();
}

private void btnSync_Click(object sender, EventArgs e)
{
    this.orderTableAdapter.Update
        (this.mergeRepSubscriberDataSet.Order);
    SqlCeReplication repl = new SqlCeReplication();
    repl.InternetUrl = @"http://sql08dev/MyWebSync/sqlcesa35.dll";
    repl.Publisher = @"SQL08DEV";
    repl.PublisherDatabase = @"SQL08SampleDB";
    repl.PublisherSecurityMode = SecurityType.DBAuthentication;
    repl.PublisherLogin = @"SubscriberAccount";
    repl.PublisherPassword = @"account";
    repl.Publication = @"MyPublication";
    repl.Subscriber = @"OrderDeliverySubscription";
    repl.SubscriberConnectionString =
        @"Data Source=""\Program
        Files\MyPocketPCDemo\MergeRepSubscriber.sdf"";
        Password=""password"";Max Database Size=128;
        Default Lock Escalation =100;";
    try
    {
        // repl.AddSubscription(AddOption.ExistingDatabase);
        repl.Synchronize();
        this.FillTables();
    }
    catch (SqlCeException ex)
    {

```

```

        MessageBox.Show(ex.ToString());
    }
}
}
}

```

Щелкните в любом месте поверхности конструирования формы `CustomerOrdersForm`, а затем выберите пункт **View Code** для открытия окна кода формы. Сначала нам необходимо добавить пространство имен `System.Data.SqlServerCe` (при помощи добавления инструкции `using` в верхнюю часть кода):

```
using System.Data.SqlServerCe;
```

Классы этого пространства имен позволят нам выполнять репликацию между локальным подписчиком `SQL Server Compact` и удаленным издателем. Теперь немного прокрутите вниз до обработчика событий формы с названием `CustomerOrdersForm_Load`. Обратите внимание на несколько строк кода, которые заполняют таблицы `Customer` и `Order` объекта `DataSet` из встроенной базы данных (при загрузке формы). Фактически нам придется делать это дважды: один раз при загрузке формы, а затем при каждой синхронизации пользователем локальной базы данных (подписчика) с центральной базой данных сервера (издателя). Поэтому мы напишем наш собственный метод `FillTables` заполнения объекта `DataSet` из локальной базы данных и заменим код метода `Form Load` вызовом `FillTables`. Затем мы добавим еще один вызов `FillTables` в коде синхронизации (на следующем шаге). Модифицируем `CustomerOrdersForm_Load` и добавим метод `FillTables` следующим образом:

```

private void CustomerOrdersForm_Load(object sender, EventArgs e)
{
    // Заменить вставленный мастером код этой единственной строкой:
    this.FillTables();
}
private void FillTables()
{
    // Местоположение файла .sdf на устройстве.
    const string ConnString = @"Data Source=
        ""\Program Files\MyPocketPCDemo\MergeRepSubscriber.SDF"";
        Password=""password"";Max Database Size=128;
        Default Lock Escalation=100;";
    if (MergeRepSubscriberDataSetUtil.DesignerUtil.IsRunTime())

```

```
{
    this.customerTableAdapter.Connection.ConnectionString = ConnString;
    this.orderTableAdapter.Connection.ConnectionString = ConnString;
    this.customerTableAdapter.Fill
        (this.mergeRepSubscriberDataSet.Customer);
    this.orderTableAdapter.Fill (this.mergeRepSubscriberDataSet.Order);
}
```

В нашем примере приложения синхронизация происходит при нажатии пользователем кнопки **btnSync**. Откройте форму в визуальном конструкторе, а затем дважды щелкните по кнопке для добавления обработчика событий для нее. Добавьте в обработчик событий одну строку кода, которая будет фиксировать изменения (сделанные в форме) в локальной базе данных подписчика (при помощи вызова метода `Update` адаптера таблицы `Order`):

```
private void btnSync_Click(object sender, EventArgs e)
{
    this.orderTableAdapter.Update (this.mergeRepSubscriberDataSet.Order);
}
```

Вызов метода `Update` требуется потому, что если изменения не будут зафиксированы в локальной базе данных подписчика, то агент не найдет никаких изменений для синхронизации. Для синхронизации этих изменений с базой данных сервера вставьте сгенерированный мастером `SQL Server Compact 3.5 Subscription Wizard` код (который находится в вашем любимом редакторе) в обработчик событий следом за только что добавленным нами вызовом метода `Update`.

После вставки кода вы должны сделать несколько изменений. Сначала модифицируйте свойство `SubscriberConnectionString` (изменив источник данных на путь `\ProgramFiles\MyPocketPCDemo\MergeRepSubscriber.sdf`). Изменение пути необходимо, поскольку исходный автоматически сгенерированный код указывает местоположение на компьютере разработчика, а не на самом устройстве. База данных `MergeRepSubscriber` копируется на устройство при развертывании. Строка `SubscriberConnectionString` должна указывать местоположение на реальном устройстве; иначе при запуске вашего приложения вы получите ошибку "Database not found".

Для упрощения нашего примера жестко пропишите пароли для свойств `PublisherPassword` и `SubscriberConnectionString`. Найдите в коде указатели мест заполнения паролей в виде строк `<...>` и замените их соответствующими паролями, которые вы настроили для регистрационной записи

SubscriberAccount (это `account`) и базы данных подписки (это `password`). Безусловно, в производственных условиях информацию для подключения необходимо запрашивать у пользователя или считывать ее из конфигурационного файла.

Нам нужно сделать еще несколько изменений в автоматически сгенерированном коде — в блоке `try`. Мастер создает строку кода, которая в нашем случае не нужна. Он добавляет вызов метода `AddSubscription` объекта `SqlCeReplication`. Вызов этого метода создает подписку. Нам не нужно делать этот вызов, поскольку подписка уже создана в базе данных `MergeRepSubscriber`. Закомментируйте эту строку кода:

```
// repl.AddSubscription(AddOption.ExistingDatabase);
```

Как мы уже упоминали, добавьте еще один вызов метода `FillTables` сразу после вызова метода `Synchronize` для обновления формы после синхронизации:

```
this.FillTables();
```

И наконец, измените переменную `e` на `ex` (в объявлении блока `catch` и в инструкции `MessageBox` внутри блока `catch`). Это необходимо потому, что сгенерированная мастером переменная `e` конфликтует с той переменной `e`, которая объявлена как параметр `EventArgs` обработчика события нажатия кнопки. Выберите кнопку **Sync** на форме визуального конструктора, а затем нажмите клавишу <F4> для отображения окна **Properties**. В этом окне нажмите кнопку **Events**. Будут показаны события кнопки. Рядом с событием `Click` выберите из списка метод с названием `btnSync_Click`.

Теперь мы готовы развернуть приложение на эмулятор.

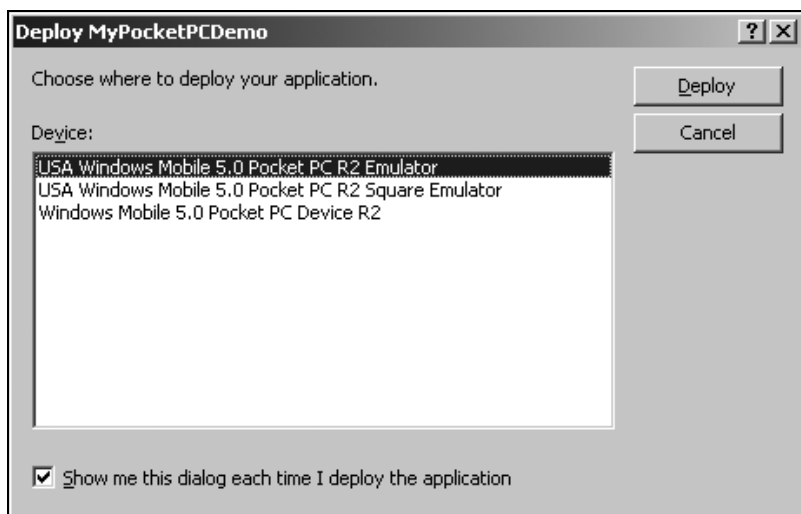
## Развертывание мобильного приложения

Нажмите клавишу <F5> для отображения диалогового окна **Deploy** (рис. 13.30). Для тестирования этого приложения выберите вариант **USA Windows Mobile 5.0 Pocket PC R2 Emulator**, а затем нажмите кнопку **Deploy**.

### ПРИМЕЧАНИЕ

Для того чтобы наш пример приложения заработал, необходимо настроить сеть на эмуляторе устройства. Из-за ограничений по объему мы не описываем здесь шаги настройки сетевого интерфейса на эмуляторе. При работе в виртуальной машине конфигурация сети отличается. В файле помощи эмулятора есть документ, который описывает настройку эмулятора, как в виртуальной машине, так и без нее. Настройку сети вашего эмулятора вы можете проверить в **Pocket Internet Explorer** по адресу

<http://<YourMachineName>/MyWebSync/sqlcesa35.dll?diag>. Если в браузере появится страница, то все настроено. Если нет, то сетевое подключение на вашем эмуляторе не настроено.



**Рис. 13.30.** Развертывание приложения Visual Studio на мобильное устройство или эмулятор

После того как вы выбрали цель развертывания, приложение запускается на устройстве или эмуляторе. При выполнении приложения вы должны понаблюдать за несколькими вещами. Во-первых, в выпадающем списке клиентов отображаются только те клиенты, заказы которых еще не доставлены. Так и задумано. Вспомните, что мы настроили фильтр для подписки, чтобы загружались только те записи о клиентах (и соответствующие им заказы), у которых значение `IsDelivered` равно нулю. Поэтому данные клиента Jones не были даже загружены на устройство. Во-вторых, вы не можете редактировать данные непосредственно в сетке. Щелкните по одной из строк сетки, а затем нажмите **Edit**. Будет показана вторая форма, в которой отображается только выбранная вами строка (рис. 13.31).

Именно сейчас окупятся все ваши усилия. Установите флажок **Is Delivered**. Нажмите кнопку **OK** в верхней правой части формы (чтобы закрыть страницу редактирования и вернуться на главную страницу приложения). Теперь нажмите кнопку **Sync** для синхронизации изменений (внесенных в запись на устройстве) с издателем. Помните, что это подписка по запросу. Подписчик инициирует процесс синхронизации, отправляя изменения данных на сервер

и затем вытягивая новые данные из публикации. После завершения синхронизации обновляется элемент DataGrid. Обратите внимание, что отредактированные вами строки больше не появляются в выпадающем списке клиентов (или в сетке DataGrid заказов), это опять работает фильтр.



**Рис. 13.31.** Форма ввода данных, работающая на эмуляторе устройства

Теперь вы можете вернуться обратно в SQL Server Management Studio и сделать запрос по записям таблицы Order базы данных SQL08DevSample. Вы увидите, что в обновленном вами заказе установлено новое значение IsDelivered, и это очень здорово!

Как вы убедились при конфигурировании этого простого приложения, Merge Replication в основном настраивается при помощи мастеров. Для бо-

лее программируемого подхода мы рассмотрим Sync Services для ADO.NET и SQL Server Change Tracking — новые технологии репликации для SQL Server 2008.

## Службы синхронизации Sync Services для ADO.NET

Sync Services для ADO.NET — это замечательная технология для создания периодически подключенных систем. Она приспособлена для синхронизации данных между данными клиентских устройств и центральным сервером. Службы Sync Services основаны на знакомой модели ADO.NET (с использованием поставщиков, адаптеров и объектов `DbCommand`), которая работает в клиент-серверных, n-уровневых и основанных на службах архитектурах.

Синхронизация производится либо только загрузкой, либо только выгрузкой, либо двусторонним образом. Внутри используется база данных SQL Server Compact 3.5 для отслеживания информации, такой как номер версии последней синхронизации клиентского приложения. Sync Services управляют этой базой данных автоматически.

Службы Sync Services не копируют изменения схемы из таблиц сервера в клиентские таблицы автоматически, но предоставляют возможность управлять изменениями схемы через код.

Среда Sync Services для ADO.NET имеет следующие возможности:

- ☐ синхронизация может выполняться асинхронно в отдельном фоновом потоке;
- ☐ Sync Services предоставляет объекты и методы для нахождения конфликтующих или удаленных строк на сервере (при помощи объектов `DbCommand`);
- ☐ Sync Services может пометить важные таблицы повышенным приоритетом при синхронизации. Такие таблицы будут синхронизироваться раньше менее важных таблиц;
- ☐ передача данных может шифроваться при помощи SSL. Безопасность хранящихся на клиентском устройстве данных может быть обеспечена шифрованием базы данных SQL Server Compact 3.5.

Созданное с использованием Sync Services приложение всегда состоит, по меньшей мере, из двух баз данных (одна на клиенте, а другая на сервере) и



логики приложения (в которой используется несколько классов из интерфейса прикладного программирования Sync Services API). Клиентская база данных — SQL Server Compact 3.5 SP1. С помощью этой базы данных служба Sync Services поддерживает метаданные приложения. Локальные данные приложения также кэшируются в клиентской базе данных. В клиентской базе данных могут быть как интерактивные, так и автономные таблицы.

Базой данных сервера может быть любая база с поддержкой ADO.NET (а не только SQL Server 2008). Это центральное хранилище, где хранятся все данные приложения.

## Объектная модель Sync Services

В табл. 13.4 дан список наиболее часто используемых в приложениях Sync Services классов.

**Таблица 13.4.** Часто используемые классы Sync Services

| Класс                   | Описание                                                                                                                                                        |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SyncAgent               | Управляет синхронизацией. SyncAgent координирует работу поставщика синхронизации клиента и поставщика синхронизации сервера для каждой синхронизируемой таблицы |
| SyncStatistics          | Статистика сеанса синхронизации. Обеспечивает информацию о конфликтах, исключениях, времени синхронизации и количестве изменений                                |
| SyncSession             | Представляет собой сеанс синхронизации. Объекты SyncSession существуют в течение всего времени жизни сеанса и предоставляют информацию о сеансе                 |
| SyncAdapter             | Адаптер синхронизации. Предоставляет команды SQL для взаимодействия с сервером для конкретной таблицы                                                           |
| SqlCeClientSyncProvider | Поставщик синхронизации клиента. Отвечает в первую очередь за обмен с клиентом и абстрагирование клиентской базы данных от агента синхронизации                 |

Таблица 13.4 (окончание)

| Класс                                | Описание                                                                                                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DbServerSyncProvider</code>    | Поставщик синхронизации сервера. В основном делает все то же самое, что и поставщик синхронизации клиента, но для базы данных сервера                                    |
| <code>ServerSyncProviderProxy</code> | Используется для n-уровневых и основанных на службах архитектур. Клиент ссылается на этот прокси (вместо прямой ссылки на поставщика <code>DbServerSyncProvider</code> ) |
| <code>SyncTable</code>               | Представляет собой синхронизированную локальную таблицу                                                                                                                  |
| <code>SyncGroup</code>               | Группа синхронизации. Коллекция взаимосвязанных объектов <code>SyncTable</code>                                                                                          |

## Объекты *SyncAgent* и *SyncStatistics*

Объект `SyncAgent` просматривает каждую синхронизируемую таблицу и вызывает клиентского и серверного поставщиков синхронизации для проведения изменений на сервере.

Событие `SessionProgress` объекта `SyncAgent` происходит при синхронизации. Вы можете использовать это событие для отслеживания сеанса синхронизации или уведомления приложения о необходимости обновить кэшированные данные после завершения сеанса.

Агент `SyncAgent` возвращает информацию о состоянии и ошибках каждого сеанса синхронизации объекта `SyncStatistics` из метода `Synchronize` объекта `SyncAgent`. Вы можете использовать этот объект для отображения информации о состоянии текущего сеанса синхронизации: количество выгруженных и загруженных записей, время синхронизации клиента и сервера, и т. д.

## Объект *SyncSession* и переменные сеанса

Объект `SyncSession` представляет собой реальный сеанс синхронизации и существует в течение всего времени жизни события. В табл. 13.5 перечислены некоторые свойства объекта `SyncSession`.

Таблица 13.5. Неполный список свойств объекта *SyncSession*

| Название свойства | Описание                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClientId          | Значение GUID, которое идентифицирует клиента во время синхронизации. Свойство <code>ClientId</code> можно использовать для обнаружения конфликтов и для подавления "эха изменений" во время синхронизации. Для предотвращения "эха изменений" вы можете пометить выгруженные данные как пришедшие от конкретного клиента (при помощи свойства <code>ClientId</code> ). Когда изменения загружаются, вы можете идентифицировать изменившего данные клиента и не загружать изменения, поступившие от этого клиента |
| SessionId         | Значение GUID, которое уникально идентифицирует текущий сеанс синхронизации                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| SyncParameters    | Коллекция объектов <code>SyncParameter</code> . Свойство <code>SyncParameter</code> используется для передачи значений параметрам запросов аналогично тому, как это делает объект <code>SqlParameter</code> в ADO.NET. Параметры могут использоваться в запросах для обнаружения конфликтов и при фильтрации данных для управления загружаемыми при синхронизации изменениями. Объекты <code>SyncParameter</code> получают свои значения в процессе синхронизации                                                 |

Среда Sync Services использует переменные сеансов для передачи значений параметрам запросов. Во время синхронизации исполняющая среда Sync Services настраивает для вас значения этих параметров (путем сопоставления названия `SyncParameter` с названием параметра запроса). Для облегчения процесса сопоставления объект `SyncSession` предоставляет несколько константных полей (перечислены в табл. 13.6). При этом получается последовательная система именования для обычных параметров (типа `@sync_client_id`).

Таблица 13.6. Неполный список постоянных полей *SyncSession*

| Название поля                | Описание                                                    |
|------------------------------|-------------------------------------------------------------|
| <code>SyncClientId</code>    | Возвращает строковое значение <code>sync_client_id</code>   |
| <code>SyncInitialized</code> | Возвращает строковое значение <code>sync_initialized</code> |

Таблица 13.6 (окончание)

| Название поля          | Описание                                                             |
|------------------------|----------------------------------------------------------------------|
| SyncLastReceivedAnchor | Возвращает строковое значение <code>sync_last_received_anchor</code> |
| SyncNewReceivedAnchor  | Возвращает строковое значение <code>sync_new_received_anchor</code>  |
| SyncRowCount           | Возвращает строковое значение <code>sync_row_count</code>            |
| SyncSessionId          | Возвращает строковое значение <code>sync_session_id</code>           |

**Объект SyncAdapter**

Во многом аналогично тому, как объект `TableAdapter` в ADO.NET предоставляет команды для заполнения и обновления таблицы, объект `SyncAdapter` предоставляет команды для синхронизации таблицы. Каждая синхронизируемая таблица имеет свой объект `SyncAdapter`. Каждый объект `SyncAdapter` предоставляет несколько свойств `DbCommand` запросов выборки и изменений в базе данных сервера. Каждая команда может использовать сеансовые переменные для передачи значений в запросы сервера при синхронизации. Команды `SyncAdapter` выполняются при помощи поставщика `DbServerSyncProvider`. Могут указываться команды в виде хранимых процедур, параметризованных инструкций SQL, представлений, функций и т. д.

**Клиентский и серверный поставщики**

Поставщики абстрагируют подробности реализации базы данных от агента синхронизации точно так же, как ADO.NET использует различных поставщиков для баз данных SQL Server и Oracle (с целью обеспечения общего интерфейса для разных серверных частей).

Клиентский поставщик `Sync Services SqlCeClientSyncProvider` баз данных SQL Server Compact 3.5 — класс, который обменивается непосредственно с клиентской базой данных. Он отвечает за хранение информации о синхронизируемых клиентских таблицах, извлечение изменений с клиента (с момента последней синхронизации), применение серверных изменений к клиенту, а также обнаружение конфликтов данных.

На сервере этим занимается класс `DbSyncServerProvider`. В отличие от объекта `SqlCeClientSyncProvider`, который специфичен для SQL Server Compact 3.5, это поставщик общего типа для серверной стороны, он работает с любой базой данных. Он выполняет те же обязанности на сервере, что и клиентский поставщик на клиенте. Серверный поставщик выполняет запросы в свойствах `DbCommand` объекта `SyncAdapter` для извлечения изменений из таблицы сервера, а также применения изменений из клиентской таблицы к таблице сервера.

## Оболочка `ServerSyncProviderProxy`

В полном соответствии со своим названием, `ServerSyncProviderProxy` является прокси-оболочкой для поставщика `DbServerSyncProvider`. Использование прокси на клиенте позволяет вам создавать n-уровневые приложения. Вместо обмена непосредственно с поставщиком `DbServerSyncProvider` клиент ведет обмен с прокси-сервером. В свою очередь, прокси-сервер ведет обмен со службой, а служба обменивается с серверным поставщиком синхронизации.

## Объекты `SyncTable` и `SyncGroup`

Объекты `SyncTable` представляют собой локальные таблицы в клиентском приложении. Объекты `SyncTable` предоставляют свойства для таких элементов, как название таблицы, способ создания локальной таблицы, направление синхронизации таблицы. В `SyncGroup` может быть добавлено множество объектов `SyncTable`.

Локальные таблицы могут создаваться во время конструирования либо вручную, либо при помощи визуального конструктора `Configure Data Synchronization` в Visual Studio, либо во время выполнения при помощи свойства `CreationOption` объекта `SyncTable`. Свойство `CreationOption` является перечислением типа `TableCreateOption` и используется для управления созданием локальных таблиц. В табл. 13.7 приведены значения перечисления `TableCreateOption`.

**Таблица 13.7.** Значения перечисления `TableCreateOption`

| Значение перечисления             | Описание                                                                                                 |
|-----------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>CreateNewTableOrFail</code> | Создать новую локальную таблицу или сгенерировать ошибку (если таблица с таким названием уже существует) |

Таблица 13.7 (окончание)

| Значение перечисления            | Описание                                                                                                                                |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| DropExistingOrCreateNewTable     | Если именованная локальная таблица существует, то ее надо удалить и создать вновь. Если она не существует, то создать локальную таблицу |
| TruncateExistingOrCreateNewTable | Обрезать существующую локальную таблицу, либо создать новую таблицу (если именованная таблица не найдена)                               |
| UploadExistingOrCreateNewTable   | Выгрузить все строки при начальной синхронизации с сервером, либо создать новую таблицу (если именованная серверная таблица не найдена) |
| UseExistingTableOrFail           | Использовать существующую локальную таблицу. Выдать исключение, если именованная локальная таблица не найдена                           |

Направление синхронизации определяется при помощи свойства `SyncDirection` объекта `SyncTable`. Оно может быть установлено в значения `Bidirectional`, `DownloadOnly`, `Snapshot` или `UploadOnly`.

Объект `SyncGroup` является коллекцией связанных таблиц. Набор таблиц в `SyncGroup` обрабатывается как единый модуль. Опции синхронизации применяются к модулю как к единому целому. Изменения сгруппированных таблиц применяются на сервере как единая транзакция. Неудача любого изменения группы означает, что все изменения откатываются и будут повторены при последующей синхронизации.

### Сбор изменений для синхронизации

Службы `Sync Services` автоматически не отслеживают изменений на сервере (как это делает `Merge Replication`). Вы можете не поверить, но это одна из сильных сторон `Sync Services`. Не привязывая механизм отслеживания изменений `Sync Services` ни к какой конкретной реализации, вы можете использовать на сервере любую поддерживающую `ADO.NET` базу данных, которая способна обработать запросы `SQL` поиска измененных строк (для синхронизации или поиска конфликтов).

Это замечательно, что службы Sync Services независимы от базы данных, но теперь вам нужно будет создавать дополнительные функции в вашем приложении. Вам придется отслеживать изменения, выполненные в таблицах приложения, чтобы выполнить синхронизацию.

Для сохранения и отслеживания произведенных в данных изменений у вас есть несколько вариантов. Первый — создать вашу собственную систему отслеживания. В нее должно будет войти и отслеживание создания или изменения схем таблиц, что потребует механизма (обычно использующего триггеры), который собирает метаданные при изменении строк.

Второй вариант — использовать новую функциональную возможность SQL Server 2008 под названием SQL Server Change Tracking. Использование Change Tracking имеет несколько преимуществ, в том числе:

- ☐ снижение времени кодирования;
- ☐ незначительное влияние на производительность, минимальные издержки для операций языка управления данными (DML);
- ☐ новые функции T-SQL для извлечения информации об изменениях;
- ☐ автоматическая очистка отслеженных изменений (при желании может быть отключена).

Для нашего примера приложения Sync Services мы будем использовать SQL Server Change Tracking.

## Change Tracking в SQL Server 2008

Функция SQL Server 2008 Change Tracking поддерживает информацию о сделанных в таблицах изменениях. Она использует синхронный процесс для отслеживания произведенных инструкциями DML изменений в таблицах. Приложение может запросить эти данные для получения информации об измененных строках и типах произошедших изменений.

### **ПРИМЕЧАНИЕ**

При помощи Change Tracking отслеживаются все операции DML (кроме инструкций `TRUNCATE TABLE` и `UPDATE TEXT`).

Change Tracking не хранит фактических значений измененных строк или столбцов и количества изменений строки, а записывает только самый минимум данных, который необходим для поддержки синхронизации — а именно, какие строки (и столбцы, но это не обязательно) изменились, а также номер

версии. Такой облегченный дизайн снижает влияние использования Change Tracking на производительность.

Когда инструкция DML выполняется по отслеживаемой таблице, SQL Server сохраняет первичные ключи всех строк, затронутых данной инструкцией DML. Информация об изменениях сохраняется во внутренних таблицах Change Tracking. Вы можете запросить представление каталога `sys.internal_tables`, чтобы получить список внутренних таблиц, которыми управляет SQL Server (включая и те, которые используются для Change Tracking).

## Конфигурирование базы данных сервера для Change Tracking

Для использования Change Tracking вы сначала включаете его на уровне базы данных, а затем для всех таблиц, чьи изменения вы хотите отслеживать. Компания Microsoft рекомендует также, чтобы вы включали уровень изоляции Snapshot базы данных (до того, как включить Change Tracking). Использование уровня изоляции Snapshot в таких условиях обеспечивает непротиворечивость информации. Обе этих задачи можно выполнить двумя последовательными инструкциями ALTER DATABASE (листинг 13.7).

### Листинг 13.7. Включение Change Tracking для базы данных

```
ALTER DATABASE SQL08SampleDB
    SET ALLOW_SNAPSHOT_ISOLATION ON
GO
ALTER DATABASE SQL08SampleDB
    SET CHANGE_TRACKING = ON
    (CHANGE_RETENTION = 7 DAYS, AUTO_CLEANUP = ON)
GO
```

Код листинга 13.7 сначала включает уровень изоляции Snapshot для базы данных SQL08SampleDB (которую мы создали для карманного ПК ранее в этой же главе), а затем включает Change Tracking для базы данных. Здесь указан срок хранения изменений 7 дней, а также включена опция автоматической очистки.

### ПРИМЕЧАНИЕ

Несмотря на то, что здесь мы этого не показываем, управление и обслуживание функции SQL Server Change Tracking можно выполнять и через диа-



логовые окна в SQL Server Management Studio. Для доступа к этим диалоговым окнам отобразите окно Properties для базы данных или таблицы, а затем выберите опцию Change Tracking.

При помощи опций `CHANGE_RETENTION` и `AUTO_CLEANUP` (листинг 13.7) вы указываете срок хранения изменений в SQL Server, или следует разрешать SQL Server автоматически очищать отслеженные изменения. Единицами измерения срока хранения могут быть дни, часы или минуты. Простейшее эмпирическое правило: устанавливайте срок хранения не меньше самого длительного периода времени между синхронизациями. Если оно будет меньше, то вы можете внести в приложение неверные данные (из-за выполненных изменений, которые были очищены до того, как произошла синхронизация).

SQL Server имеет задачу, удаляющую ту информацию Change Tracking, которая старше, чем указанное для `CHANGE_RETENTION` значение. Опция автоматической очистки говорит серверу SQL Server о том, следует ли ему выполнять задачу очистки. Установка этого значения в `False` отключает задачу и предотвращает очистку. Это полезно в таких ситуациях, когда какая-то проблема мешает выполнению синхронизации, и вы не хотите, чтобы SQL Server удалил старые версии данных (пока вы не решите имеющуюся проблему).

## Включение Change Tracking для пользовательских таблиц

После включения на уровне базы данных функцию Change Tracking необходимо включить и для каждой таблицы, которую вы хотите отслеживать. Единственное требование к таким таблицам — у них должен иметься первичный ключ.

### ПРИМЕЧАНИЕ

Рекомендуется, чтобы первичный ключ в отслеживаемой таблице был типа `uniqueidentifier`. Это обеспечивает уникальность в распределенной среде.

Для включения Change Tracking таблиц `Customer` и `Contact` выдайте команду `ALTER TABLE`, показанную в листинге 13.8.

### Листинг 13.8. Включение Change Tracking для пользовательских таблиц

```
ALTER TABLE Customer
    ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
```

```
GO
```

```
ALTER TABLE Contact
```

```
    ENABLE CHANGE_TRACKING WITH (TRACK_COLUMNS_UPDATED = OFF);
```

```
GO
```

Опция `TRACK_COLUMNS_UPDATED` говорит серверу SQL Server о том, надо ли записывать информацию о столбцах, которые были изменены при обновлении строки. Когда эта опция установлена в `ON`, то SQL Server сохраняет дополнительную информацию о том, какие столбцы были изменены. При помощи этой опции можно улучшить производительность синхронизации (загружаться будут только те строки и столбцы, которые были изменены). Однако появляются дополнительные издержки на поддержание информации отслеживания измененных столбцов (поэтому эта опция по умолчанию выключена).

Возможно, вы уже задумались о влиянии на общую производительность функции SQL Server Change Tracking. Влияние на производительность будет меняться в зависимости от числа столбцов первичного ключа, количества затронутых инструкцией DML строк и инструкций, выполняемых в транзакции. Однако суммарные издержки будут примерно равны издержкам на поддержание индекса таблицы.

## Управление отслеживаемыми таблицами и базами данных

Представления каталогов `sys.change_tracking_databases` и `sys.change_tracking_tables` дают информацию о базах данных и таблицах с включенной функцией Change Tracking, и какие опции были настроены. Например, следующий запрос возвращает список баз данных с включенной функцией Change Tracking:

```
SELECT
```

```
    d.name as database_name,  
    c.is_auto_cleanup_on,  
    c.retention_period,  
    c.retention_period_units_desc
```

```
FROM
```

```
    sys.databases AS d  
    INNER JOIN sys.change_tracking_databases AS c on  
        d.database_id = c.database_id
```

Выходные данные показывают, что функция Change Tracking включена для нашей базы данных SQL08SampleDb:

```
database_name is_auto_cleanup_on retention_period reten-
tion_period_units_desc
```

```
-----
SQL08SampleDB 1                      7                      DAYS
(1 row(s) affected)
```

Этот запрос может использоваться для определения, какие таблицы конкретной базы данных имеют включенную функцию Change Tracking:

```
SELECT
    o.name as table_name,
    c.is_track_columns_updated_on
FROM
    sys.objects AS o
    INNER JOIN sys.change_tracking_tables AS c ON o.object_id = c.object_id
```

Выходные данные показывают, что функция Change Tracking включена для наших таблиц Customer и Contact:

```
table_name is_track_columns_updated_on
-----
Contact      1
Customer     1
(2 row(s) affected)
```

## Отключение Change Tracking

После того как функция Change Tracking была включена для таблицы, вы можете отключить ее либо при помощи диалогового окна **Properties**, либо при помощи инструкции ALTER TABLE...DISABLE CHANGE\_TRACKING. Например, следующая инструкция останавливает отслеживание изменений для таблицы Customer:

```
ALTER TABLE Customer DISABLE CHANGE_TRACKING
```

Аналогичным же образом, для отключения функции Change Tracking во всей базе данных вы можете выдать команду ALTER DATABASE. Это можно сделать только после того, как функция Change Tracking будет отключена во всех таблицах. Например, после отключения Change Tracking для обеих таблиц (Customer и Order) следующая инструкция отключает эту функцию и для базы данных SQL08SampleDb:

```
ALTER DATABASE SQL08SampleDB SET CHANGE_TRACKING = OFF
```

### Функции Change Tracking

Несколько новых функций Change Tracking в SQL Server 2008 дают возможность определять текущие номера версий, а также получать измененные строки. В табл. 13.8 перечислены некоторые из этих функций, которые мы скоро будем использовать в примере приложения. При написании запросов по отслеживаемым таблицам вы будете делать соединения запросов к этим функциям (для получения измененных строк, номеров версий строк, а также номеров версий системы).

*Таблица 13.8. Часто используемые функции Change Tracking*

| Название функции                  | Описание                                                                                                                                                                      |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CHANGE_TRACKING_MIN_VALID_VERSION | Возвращает самый старый действующий номер версии для извлечения изменений из таблицы                                                                                          |
| CHANGE_TRACKING_CURRENT_VERSION   | Возвращает текущий номер версии изменений. Используется при извлечении изменений из таблицы                                                                                   |
| CHANGETABLE (changes)             | Возвращает информацию отслеживания по изменениям во всех данных таблицы, начиная с конкретной версии. Используется для инициализации клиентских таблиц во время синхронизации |
| CHANGETABLE (version)             | Возвращает информацию отслеживания по изменениям с конкретной версией строки. Полезна для определения конфликтов строк при синхронизации                                      |
| WITH CHANGE_TRACKING_CONTEXT      | Необязательна. Используйте это выражение с запросами DML для уникальной идентификации автора измененных данных                                                                |

Вы можете использовать две связанные с версиями функции CHANGE\_TRACKING\_MIN\_VALID\_VERSION и CHANGE\_TRACKING\_CURRENT\_VERSION для определения момента времени (называемого *якорем*), когда были сделаны изменения.

Функция CHANGETABLE — это основная функция, которую вы будете использовать со службами Sync Services. Она достаточно универсальна и может вернуть информацию как об измененных строках таблицы, так и о конкрет-

ном номере версии записанных данных. В большинстве случаев запросы с `CHANGETABLE` будут соединяться с отслеживаемой таблицей.

Например, для извлечения начального моментального снимка пользовательских данных вы можете написать запрос вроде следующего:

```
SELECT c.[CustomerId], c.[CustomerName]
FROM Customer AS c LEFT OUTER JOIN
    CHANGETABLE(CHANGES Customer, 1) AS CT ON CT.[CustomerId] =
        c.[CustomerId]
```

Такой запрос можно использовать для инициализации таблицы локальной клиентской базы данными из таблицы сервера. Указав `LEFT OUTER JOIN`, вы получите клиентские записи независимо от того, были ли сделаны изменения в таблице `Customer`.

## Установка значения якоря

Функция SQL Server Change Tracking работает путем записи изменений, произведенных в строке в соответствующий момент времени. Как уже упоминалось, службы Sync Services называют этот момент якорем. При синхронизации извлекается новое значение якоря. Новое значение якоря и последнее значение якоря из предыдущей синхронизации становятся верхней и нижней границами диапазона номеров версий. Синхронизироваться должны те наборы изменений, чьи номера версий попадают в диапазон якорей. Для извлечения значения якоря запросите функцию `CHANGE_TRACKING_CURRENT_VERSION`.

## Создание приложения при помощи служб Sync Services

Мы будем использовать Visual Studio 2008 для создания приложения Windows Forms с названием `MySyncServicesDemo`. Это приложение синхронизирует данные (сохраненные локально в базе данных SQL Server Compact 3.5) с нашими таблицами `Customer` и `Contact` в базе данных `SQL08SampleDB` (при помощи функции SQL Server Change Tracking). Такое приложение дает возможность торговым представителям просматривать и обновлять контактные данные клиентов (даже тогда, когда они находятся вне офиса). Мы начнем создавать его как обычное приложение клиент-сервер. Затем мы расширим его до n-уровневой модели (при помощи Windows Communication Foundation, WCF).

**ПРИМЕЧАНИЕ**

Наше обсуждение сосредоточено только на определенных фрагментах приложения, поэтому листинги кодов были соответствующим образом подготовлены. Полные исходные коды работающего приложения `MySyncServicesDemo` доступны на Web-сайте данной книги.

Таблицы `Customer` и `Contact` были созданы ранее в базе данных `SQL Server Compact 3.5` вручную при помощи `Visual Studio 2008` (когда мы создавали наше приложение для карманного ПК с использованием `Merge Replication`). Вспомните, что у нас были варианты создания таблиц на стадии конструирования и на стадии выполнения (программным путем). Свойства объекта `SyncTable` определяют, будут ли службы `Sync Services` пытаться создать таблицы на клиенте. Создание таблиц во время выполнения полезно для сценариев с одной только загрузкой (когда таблицы используются для справочных данных). Для нашего примера мы создали таблицы на стадии конструирования. Локальные таблицы имеют такую же схему, что и серверные.

Приложение использует `DataSet` с названием `LocalCustomerDataset`, полученный из двух локальных таблиц. Мы добавили связь между двумя таблицами по столбцу `CustomerID`. Адаптеры `TableAdapters` в `DataSet` имеют строки подключения, настроенные для использования базы данных `SQL Server Compact 3.5`.

Каждая синхронизируемая таблица приложения использует набор объектов: `SyncAgent`, `SqlCeClientSyncProvider` и `DbServerSyncProvider`. Агент `SyncAgent` и классы поставщика синхронизации клиента для таблицы `Customer` показаны в листинге 13.9. В нашем примере класс `MyClientSyncProvider` отвечает за настройку строки подключения к клиентской базе данных. Он отвечает также и за обработку конфликтов, происходящих при синхронизации. Мы обсудим обработку конфликтов данных далее в этой же главе.

**ПРИМЕЧАНИЕ**

На данный момент приложение сконфигурировано как клиент-серверное. Для обмена с сервером в качестве `DbServerSyncProvider` используется `CustomerServerSyncProvider`. В этом примере клиент обменивается непосредственно с классом `CustomerServerSyncProvider`, который находится на клиенте. Позднее мы заменим этот `DbServerSyncProvider` классом `ServerSyncProviderProxy` и перенесем поставщика `CustomerServerSyncProvider` в службу WCF (чтобы превратить приложение в основанное на службе приложение).

**Листинг 13.9. Класс CustomerSyncAgent и класс MyClientSyncProvider**

```
public class CustomerSyncAgent : SyncAgent
{
    public void Initialize()
    {
        this.LocalProvider = new MyClientSyncProvider();
        this.RemoteProvider = new CustomerServerSyncProvider();
        // Добавить таблицу Customer table: указать направление синхронизации
        // "только загрузка" и что нужно использовать существующую таблицу.
        SyncTable customerSyncTable = new SyncTable("Customer");
        customerSyncTable.CreationOption =
            TableCreationOption.UseExistingTableOrFail;
        customerSyncTable.SyncDirection = SyncDirection.DownloadOnly;
        this.Configuration.SyncTables.Add(customerSyncTable);
    }
}

public class MyClientSyncProvider : SqlCeClientSyncProvider
{
    public MyClientSyncProvider()
    {
        // Указать строку подключения для локальной
        // базы данных SQL Server Compact.
        this.ConnectionString = Helper.ClientConnection();
        this.ApplyChangeFailed +=
            new EventHandler<ApplyChangeFailedEventArgs>
                (this.MyClientSyncProvider_ApplyChangeFailed);
    }
}
```

Рассматривая метод `Initialize` класса `CustomerSyncAgent`, вы сможете лучше понять то, как агент `SyncAgent` управляет синхронизацией. Метод `Initialize` создает экземпляры поставщиков клиента и сервера и присваивает их объектам `LocalProvider` и `RemoteProvider` соответственно. Запросы в этих поставщиках будут выполняться во время синхронизации. Этот метод также создает новую таблицу `SyncTable` с названием `customerSyncTable` для таблицы `Customer`.

Классы `SyncAgent` и `SqlCeClientSyncProvider` очень просты (по сравнению с `DbServerSyncProviders`). Большая часть работы запросов выполняется в классе `DbServerSyncProvider` при помощи метода `SyncAdapter`.

Создание и заполнение таблицы управляется свойством `SelectIncrementalInsertsCommand` объекта `SyncAdapter`. Указанный запрос является, вероятно, самым важным запросом приложения `Sync Services`. Службы `Sync Services` для создания клиентской таблицы извлекают схему таблицы с сервера. Полученная схема основана на столбцах в списке `SELECT` запроса. Создание таблицы зависит от установленного в свойстве `CreationOption` (объекта `SyncTable`) значения.

В нашем примере установка в свойстве `CreationOption` значения `UseExistingTableOrFail` дает методу `SyncAdapter` указание не перезаписывать таблицу при начальной синхронизации. Ранее (при обсуждении объектов `SyncTable`) мы упоминали, что по умолчанию `Sync Services` хочет скопировать схему и данные из базы сервера на клиент (во время начальной синхронизации). Если свойство `CreationOption` установлено в отличное от `UseExistingTableOrFail` значение, то `Sync Services` попытается создать на клиенте локальную таблицу.

Такая возможность имеет свои преимущества, но вы должны знать, что ограничения в локальную базу данных не копируются. Будет производиться также некоторое сопоставление типов данных (вследствие отличий типов данных между базами `SQL Server Compact` и базами "обычных" версий `SQL Server`). В результате этого процесса может получиться несоответствие некоторых типов данных.

### **ПРИМЕЧАНИЕ**

Версия `SQL Server Compact 3.5` на момент выхода из печати этой книги поддерживала не все новые типы данных `SQL Server`. Поэтому для нашего примера была использована бета-версия `SQL Server Compact 3.5 SP1`.

Вы можете скачать `SP1` со следующего URL:

**<http://www.microsoft.com/downloads/details.aspx?FamilyID=68539fae-cf03-4c3b-aeda-769cc205fe5f&DisplayLang=en>**.

Зайдите на Web-сайт этой книги, чтобы узнать о самых последних изменениях.

Вследствие этих и других проблем вам может понадобиться отредактировать в коде схему таблицы (когда `Sync Services` создаст для вас таблицы). Вы можете сделать это при помощи объекта `SqlCeClientSyncProvider`. Этот класс предоставляет два события: `CreatingSchema` и `SchemaCreated`, которые происходят перед и после создания таблицы. При помощи этих событий вы мо-



жете модифицировать схему локальной таблицы во время начальной синхронизации.

Настройка свойства `SyncDirection` в значение `DownloadOnly` по сути сделает таблицу `Customer` копией "только для чтения" серверной таблицы `Customer`. Сделанные в клиентской таблице `Customer` изменения не будут выгружаться на сервер. `Sync Services` загружают данные на клиент во время начальной синхронизации во всех случаях (кроме того, когда свойство `SyncDirection` объекта `SyncTable` установлено в значение `UploadOnly`). В листинге 13.10 показана хранимая процедура для `customerSyncAdapter` нашего примера.

**Листинг 13.10. Хранимая процедура для `SelectIncrementalInsertsCommand`**

```
CREATE PROCEDURE up_CustomerIncrementalInserts
( @sync_initialized bit,
  @sync_last_received_anchor bigint,
  @sync_client_id_binary varbinary(128),
  @sync_new_received_anchor bigint)
AS
IF @sync_initialized = 0
    SELECT c.[CustomerId], c.[CustomerName]
    FROM Customer c
ELSE
    SELECT c.[CustomerId], c.[CustomerName]
    FROM Customer c JOIN
        CHANGETABLE(CHANGES Customer, @sync_last_received_anchor) CT
        ON CT.[CustomerId] = c.[CustomerId]
    WHERE (CT.SYS_CHANGE_OPERATION = 'I' AND CT.SYS_CHANGE_CREATION_VERSION
           <= @sync_new_received_anchor
           AND (CT.SYS_CHANGE_CONTEXT IS NULL OR CT.SYS_CHANGE_CONTEXT
              <> @sync_client_id_binary))
```

Хранимая процедура сначала проверяет параметр `@sync_initialized`. Если процедура вызывается при начальной синхронизации, то его значение будет равно нулю. В этом случае клиенту будут возвращены все строки из таблицы `Customer`. Для последующих синхронизаций `@sync_initialized` равняться нулю не будет, и возвращаться клиенту будут только те строки `Customer`, которые были вставлены (где `CT.SYS_CHANGE_OPERATION = 'I'`), с номером версии меньшим, чем значение нового якоря. Параметр `@sync_client_id_binary` ис-

пользуется для предотвращения "эха изменений", когда выгруженные с клиента изменения возвращаются обратно на клиент во время одной и той же синхронизации.

## Начальная синхронизация

Во время начальной синхронизации клиентской таблицы происходит несколько событий. Сначала таблица создается (это не обязательно), затем заполняется, и клиентской базе данных присваивается уникальный идентификатор. Запрос в свойстве `SelectIncrementalInsertsCommand` используется не только для создания и заполнения таблицы во время начальной синхронизации, но и для применения инкрементальных вставок для тех таблиц, свойство `SyncDirection` которых установлено в значения `DownloadOnly` или `Bidirectional` (при последующих синхронизациях).

После заполнения клиентской таблицы базе данных клиента присваивается уникальное значение. Это GUID, доступ к которому можно получить через свойство `ClientId` объекта `SyncSession`. `ClientId` передается в запросы для идентификации автора изменений в данных. В хранимой процедуре листинга 13.10 параметр `@sync_client_id_binary` получает свое значение из свойства `SyncClientIdBinary`. Это свойство — двоичное значение, представляющее значение `ClientId`.

`ClientId` полезен для идентификации клиента, который выполнил изменение серверных данных. Он помогает также предотвратить "эхо изменений" в сценариях с двусторонней синхронизацией. Во время синхронизации изменения выгружаются с клиента на сервер до загрузки изменений на клиент. При отсутствии идентификации клиента изменения во время сеанса синхронизации выгружаются на сервер, а затем бессмысленно загружаются обратно на клиент.

## Выгрузка изменений

Для настройки запросов выгрузки изменений с клиента на сервер используются три командных свойства. Свойства `InsertCommand`, `UpdateCommand` и `DeleteCommand` — это команды операций `INSERT`, `UPDATE` и `DELETE` с клиента на сервер. Вам нужно настраивать эти свойства только для тех таблиц, которые указаны как `UploadOnly` или `Bidirectional`.

Таблица `Customer` нашего примера является `DownloadOnly`. Однако таблица `Contact` является `Bidirectional`. Изменения данных `Contact` на клиенте выгружаются на сервер при помощи трех запросов листинга 13.11.

**Листинг 13.11. Хранимые процедуры обновления сервера изменениями клиента**

```
CREATE PROCEDURE up_ContactInserts
( @sync_row_count int OUTPUT,
  @sync_client_id_binary varbinary(128),
  @ContactId uniqueidentifier,
  @CustomerId int,
  @ContactName nvarchar(100),
  @ContactPhone nvarchar(100),
  @ContactEmail nvarchar(100))
AS
WITH CHANGE_TRACKING_CONTEXT (@sync_client_id_binary)
INSERT INTO Contact ([ContactId], [CustomerId], [ContactName],
                    [ContactPhone], [ContactEmail])
VALUES (@ContactId, @CustomerId, @ContactName,
        @ContactPhone, @ContactEmail);
SET @sync_row_count = @@rowcount;
GO

CREATE PROCEDURE up_ContactUpdates
( @sync_last_received_anchor bigint,
  @sync_client_id_binary varbinary(128),
  @sync_row_count int OUTPUT,
  @ContactId uniqueidentifier,
  @ContactName nvarchar(100),
  @ContactPhone nvarchar(100),
  @ContactEmail nvarchar(100))
AS
WITH CHANGE_TRACKING_CONTEXT (@sync_client_id_binary)
UPDATE Contact
SET [ContactName] = @ContactName,
    [ContactPhone] = @ContactPhone,
    [ContactEmail] = @ContactEmail
FROM Contact c
JOIN CHANGETABLE(VERSION Contact, ([ContactId]), (@ContactId)) CT
ON CT.[ContactId] = c.[ContactId]
WHERE (CT.SYS_CHANGE_VERSION IS NULL OR CT.SYS_CHANGE_VERSION
      <= @sync_last_received_anchor
```

```

OR (CT.SYS_CHANGE_CONTEXT IS NOT NULL AND CT.SYS_CHANGE_CONTEXT
    = @sync_client_id_binary));
SET @sync_row_count = @@rowcount;
GO

CREATE PROCEDURE up_ContactDeletes
( @sync_row_count int OUTPUT,
  @sync_client_id_binary varbinary(128),
  @ContactId uniqueidentifier,
  @sync_last_received_anchor int)
AS
WITH CHANGE_TRACKING_CONTEXT (@sync_client_id_binary)
DELETE Contact FROM Contact c
    JOIN CHANGETABLE(VERSION Contact, ([ContactId]), (@ContactId)) CT
    ON CT.[ContactId] = c.[ContactId]
WHERE (CT.SYS_CHANGE_VERSION IS NULL OR CT.SYS_CHANGE_VERSION
    <= @sync_last_received_anchor
OR (CT.SYS_CHANGE_CONTEXT IS NOT NULL AND CT.SYS_CHANGE_CONTEXT
    = @sync_client_id_binary));
SET @sync_row_count = @@rowcount;
GO

```

Использованное в этих запросах выражение `WITH CHANGE_TRACKING_CONTEXT` отслеживает клиента, выполнившего изменения (которые применяются на сервере). Клиент передает переменную сеанса `@sync_client_id_binary` в хранимую процедуру (для инструкции `CHANGE_TRACKING_CONTEXT`). Значение идентификатора клиента хранится в таблице, позволяющей приложению Change Tracking отслеживать контекст, в котором было выполнено изменение.

## Загрузка изменений

В листинге 13.10 была показана хранимая процедура для свойства `SelectIncrementalInsertsCommand`, которое извлекает вставленные данные из серверной таблицы. Два других командных свойства извлечения инкрементальных обновлений и удалений сервера, `SelectIncrementalUpdatesCommand` и `SelectIncrementalDeletesCommand`, необходимо настраивать только для тех таблиц, которые указаны как `DownloadOnly` или `Bidirectional`. Листинг 13.12 показывает запросы объекта `customerSyncAdapter` для этих свойств.

**Листинг 13.12. Используемые для SelectIncrementalUpdatesCommand и SelectIncrementalDeletesCommand запросы**

```
CREATE PROCEDURE up_CustomerIncrementalUpdates
( @sync_initialized bit,
  @sync_last_received_anchor bigint,
  @sync_client_id_binary varbinary(128),
  @sync_new_received_anchor bigint)
AS
IF @sync_initialized > 0
BEGIN
    SELECT c.[CustomerId], c.[CustomerName]
    FROM Customer c JOIN
        CHANGETABLE(CHANGES Customer, @sync_last_received_anchor) CT
    ON CT.[CustomerId] = c.[CustomerId]
    WHERE (CT.SYS_CHANGE_OPERATION = 'U' AND CT.SYS_CHANGE_VERSION
           <= @sync_new_received_anchor
           AND (CT.SYS_CHANGE_CONTEXT IS NULL OR CT.SYS_CHANGE_CONTEXT
                <> @sync_client_id_binary));
END
GO

CREATE PROCEDURE up_CustomerIncrementalDeletes
( @sync_initialized bit,
  @sync_last_received_anchor bigint,
  @sync_client_id_binary varbinary(128),
  @sync_new_received_anchor bigint)
AS
IF @sync_initialized > 0
BEGIN
    SELECT CT.[CustomerId]
    FROM CHANGETABLE(CHANGES Customer, @sync_last_received_anchor) CT
    WHERE (CT.SYS_CHANGE_OPERATION = 'D' AND CT.SYS_CHANGE_VERSION
           <= @sync_new_received_anchor
           AND (CT.SYS_CHANGE_CONTEXT IS NULL OR CT.SYS_CHANGE_CONTEXT
                <> @sync_client_id_binary));
END
GO
```

В каждом запросе для поиска конкретных изменений используется столбец `ST.SYS_CHANGE_OPERATION`. Удаления запрос ищет по значению `D`, обновления — по значению `U` (вставки — по `I`, как вы уже видели ранее в листинге 13.10).

Детектирование и разрешение конфликтов данных

Конфликты данных могут возникать при любой синхронизации, поддерживающей изменения на клиенте и на сервере. Предположим, что кто-то удалил строку в серверной таблице, и что именно эта строка была обновлена клиентом в автономном режиме. Когда клиент пытается синхронизировать свои изменения, происходит конфликт (поскольку строки на сервере уже не существует).

В идеале ваше приложение должно совершенно избегать конфликтов (при помощи свойства `ClientId` для фильтрации данных, чтобы данный клиент обновлял только отдельные строки). Проектирование вашего приложения таким образом, чтобы оно избегало конфликтов, имеет еще одно преимущество: при этом увеличивается производительность (из-за уменьшения сетевого трафика). Для тех ситуаций, когда вы не можете избежать конфликтов данных, службы Sync Services предоставляют возможности распознавания и разрешения ошибок и конфликтов.

ПРИМЕЧАНИЕ

Sync Services обрабатывает ошибки при синхронизации — как конфликты.

Конфликты могут происходить либо на сервере, либо на клиенте, во время операций по вставке, обновлению или удалению. В табл. 13.9 описаны шесть типов конфликтов в перечислении `ConflictType`.

Таблица 13.9. Обнаруживаемые службами Sync Services типы конфликтов

| Тип конфликта                         | Описание                                                                                                                                                         |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ClientDeleteServerUpdate</code> | Этот конфликт происходит тогда, когда сервер обновляет строку, которую клиент удалил                                                                             |
| <code>ClientInsertServerInsert</code> | Известен как конфликт первичного ключа. Этот тип конфликта происходит тогда, когда и клиент, и сервер пытаются вставить строку с одним и тем же первичным ключом |

Таблица 13.9 (окончание)

| Тип конфликта            | Описание                                                                                                                                                                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClientUpdateServerDelete | Этот конфликт происходит тогда, когда клиент обновляет строку, которую сервер удалил                                                                                                                                                       |
| ClientUpdateServerUpdate | Этот конфликт происходит тогда, когда клиент и сервер обновляют одну и ту же строку                                                                                                                                                        |
| ErrorsOccurred           | Этот конфликт происходит тогда, когда некая ошибка препятствует выполнению синхронизации строки                                                                                                                                            |
| Unknown                  | Используется поставщиком DbServerSyncProvider только в том случае, когда сервер не может классифицировать тип конфликта. Поставщик SqlCeClientSyncProvider не использует этот тип, поскольку он может классифицировать все типы конфликтов |

В зависимости от того, когда происходит конфликт (при выгрузке или загрузке), событие генерируется соответственно в DbServerSyncProvider или SqlCeClientSyncProvider. Событие ApplyChangeFailed поставщика DbServerSyncProvider генерируется при возникновении конфликта выгрузки. Оно работает совместно со свойствами SelectConflictUpdatedRowsCommand и SelectConflictDeletedRowsCommand (для выборки конфликтующих строк). Хранимые процедуры листинга 13.13 используются в нашем приложении для выборки строк при возникновении конфликта.

**Листинг 13.13. Используемые провайдером DbServerSyncProvider запросы для обнаружения конфликтов синхронизации в таблице Contact нашего приложения**

```
CREATE PROCEDURE up_GetContactUpdateConflicts
    (@ContactId uniqueidentifier)
AS
SELECT c.[ContactId], c.[ContactName],
    CT.SYS_CHANGE_CONTEXT, CT.SYS_CHANGE_VERSION
FROM Contact c JOIN CHANGETABLE(VERSION Contact, ([ContactId]),
    (@ContactId)) CT
ON CT.[ContactId] = c.[ContactId];
```

```

GO
CREATE PROCEDURE up_GetContactDeleteConflicts
    (@ContactId uniqueidentifier, @sync_last_received_anchor bigint)
AS
SELECT CT.[ContactId],
    CT.SYS_CHANGE_CONTEXT, CT.SYS_CHANGE_VERSION
FROM CHANGETABLE(CHANGES Contact, @sync_last_received_anchor) CT
WHERE (CT.[ContactId] = @ContactId AND CT.SYS_CHANGE_OPERATION = 'D');
GO

```

Если конфликт происходит при загрузке, то Sync Services выбирает конфликтующие строки при помощи события `ApplyChangeFailed` объекта `SqlCeClientSyncProvider`.

Параметр `EventArgs` события `ApplyChangeFailed` предоставляет информацию о конфликте и его возможных решениях. Идея в том, что вы можете управлять разрешением конфликта: выигрывают изменения на сервере, выигрывают изменения на клиенте и т. д. В листинге 13.14 показан обработчик события `ApplyChangeFailed` нашего приложения. В производственном приложении вы бы добавили в этот обработчик события еще и код разрешения конфликтов в соответствии с бизнес-логикой.

### **ПРИМЕЧАНИЕ**

После разрешения конфликтов вы должны повторно проверить соблюдение ваших бизнес-правил. Это можно сделать при помощи события `ApplyingChanges` класса `DbServerSyncProvider`.

#### **Листинг 13.14. Тестирование на разные типы конфликтов в событии `ApplyChangeFailed` класса `MyClientSyncProvider`**

```

private void MyClientSyncProvider_ApplyChangeFailed(object sender,
    ApplyChangeFailedEventArgs e)
{
    // добавить здесь логику для обработки конфликтов
    switch (e.Conflict.ConflictType)
    {
        case ConflictType.ClientDeleteServerUpdate:
            break;
        case ConflictType.ClientInsertServerInsert:
            break;
    }
}

```



```
case ConflictType.ClientUpdateServerDelete:
    break;
case ConflictType.ClientUpdateServerUpdate:
    break;
case ConflictType.ErrorsOccurred:
    break;
case ConflictType.Unknown:
    break;
}
```

Объект `ApplyChangeFailedEventArgs` предоставляет свойства для разрешения конфликтов. Вы можете определить тип конфликта при помощи его свойства `ConflictType`. В зависимости от типа вы можете предпринять определенные действия, такие как продолжение синхронизации или повторение обновления. Используйте свойство для применения действия. Свойство `Action` может быть установлено в одно из следующих значений перечисления `ApplyAction`:

- ☐ `Continue` — игнорировать ошибку и продолжить;
- ☐ `RetryApplyingRow` — повторить обновление. Если вы выбираете этот вариант, то вы должны исправить данные до повторения, иначе исключение появится опять;
- ☐ `RetryWithForceWrite` — повторить и принудительно провести обновление. Используйте параметр `@sync_force_write` на сервере (для принудительного обновления на сервере).

Вы можете исследовать проблемные данные при помощи свойства `Context`, возвращающего параметр `SyncContext` объекта `DataSet` с изменениями в текущем сеансе синхронизации. Эти данные — копии реальных данных. Изменение данных в свойстве `Context` не изменяет строк, используемых в текущем сеансе синхронизации.

Класс `SqlCeClientSyncProvider` предоставляет свойство с названием `ConflictResolver`. Вы можете использовать это свойство для разрешения конфликтов на клиенте. Свойство `ConflictResolver` может быть установлено в одно из следующих значений перечисления `ResolveAction`:

- ☐ `ClientWins` — эквивалентно настройке свойства `ApplyAction` в значение `Continue`;

- `ServerWins` — эквивалентно настройке свойства `ApplyAction` в значение `RetryWithForceWrite`;
- `FireEvent` — запускает событие `ApplyChangeFailed` на клиенте.

Если вы не разрешите конфликты и ошибки в том же самом сеансе, где они произошли, то проблемные строки повторно не синхронизируются (если только в последующем не произойдет обновление этих строк). В некоторых случаях вам может понадобиться обновить строки для того, чтобы они ресинхронизировались.

## Собираем все вместе

Давайте посмотрим, как все работает в нашем примере приложения. Когда загружается форма `Form1`, то она инициализирует оба объекта — `customerSyncAdapter` и `contactSyncAdapter`, добавляет обработчики для каждого из событий `SessionProgress` объекта `SyncAdapter`. И наконец, она заполняет таблицы `Customer` и `Contact` в `LocalCustomerDataSet`. Событие `Form1_Load` показано в листинге 13.15.

### Листинг 13.15. Событие `Form1_Load` в приложении `MySyncServicesDemo`

```
private void Form1_Load(object sender, EventArgs e)
{
    this._customerSyncAgent = new CustomerSyncAgent();
    this._customerSyncAgent.SessionProgress +=
        new EventHandler<SessionProgressEventArgs>
            >(this.OnCustomerSyncAgentSessionProgress);
    this._customerSyncAgent.Initialize();
    this._contactSyncAgent = new ContactSyncAgent();
    this._contactSyncAgent.SessionProgress +=
        new EventHandler<SessionProgressEventArgs>
            (this.OnContactSyncAgentSessionProgress);
    this._contactSyncAgent.Initialize();
    // Заполнить LocalCustomerDataSet данными по клиентам и контактными
    // данными из локальной базы данных.
    this.contactTableAdapter.Fill(this.LocalCustomerDataSet.Contact);
    this.customerTableAdapter.Fill(this.LocalCustomerDataSet.Customer);
}
```

Локальная база данных SQL Server Compact 3.5 при первом выполнении приложения пуста. Нажатие на кнопку **Refresh Customer** запускает начальную синхронизацию таблицы Customer (при помощи вызова метода Synchronize объекта customerSyncAdapter). Когда синхронизация завершается, возвращаемый вызовом объект SyncStatistics используется для показа информации о состоянии сеанса.

Во время начальной синхронизации локальная таблица Customer заполняется данными из серверной таблицы Customer. Для извлечения клиентских данных выполняется запрос, который определен в свойстве SelectIncrementalInsertsCommand объекта customerSyncAdapter. Форма отслеживает выполнение синхронизации при помощи обработчика события SessionProgress (листинг 13.16).

**Листинг 13.16. Обработчик события SessionProgress объекта customerSyncAgent**

```
private void OnCustomerSyncAgentSessionProgress
    (object sender, SessionProgressEventArgs e)
{
    // по завершении синхронизации потребовать от локальной базы
    // обновить элементы управления
    if (e.PercentCompleted == 100)
    {
        this.customerTableAdapter.Fill(this.LocalCustomerDataSet.Customer);
    }
}
```

При помощи обработчика события форма Form1 повторно загружает таблицу Customer (при помощи вызова метода Fill объекта customerTableAdapter). Поздравляем! Вы только что синхронизировали свою первую таблицу при помощи служб Sync Services! Теперь сделаем еще один шаг и выгрузим измененные на клиенте данные на сервер.

Процесс синхронизации для таблицы Contact примерно такой же, что и для таблицы Customer. Самая большая разница в том, что таблица Contact поддерживает двустороннюю синхронизацию. Выберите клиента с фамилией Johnson, а затем нажмите кнопку **New Contact** для добавления новых контактных данных. По умолчанию приложение создает контактную запись Customer Contact1. Нажмите кнопку **Save** для сохранения контактных данных в локальную базу данных.

Если в данный момент вы остановитесь и перезапустите приложение, то `Customer Contact1` будет по-прежнему видна в качестве одного из контактов для Johnson. Однако если вы запросите таблицу `Contact` в базе данных `SQL08SampleDB`, то вы не увидите никаких контактов (поскольку данные еще не были синхронизированы). Нажмите кнопку **Sync Contacts** для загрузки нового контакта на сервер. Обработчик события нажатия кнопки вызывает метод `Synchronize` объекта `contactSyncAdapter`. По завершении синхронизации вы увидите окно сообщения со статистикой. Вернитесь в базу данных `SQL08SampleDB` и повторно запросите таблицу `Contact`. На этот раз вы увидите запись `Customer Contact1`.

Теперь пришло время реально поразвлечься с `Sync Services`. Поэкспериментируйте путем добавления или удаления строк непосредственно из серверных таблиц `Customer` или `Contact` и последующей синхронизации клиента. Попробуйте также редактировать или удалять контактные данные на клиенте и синхронизировать изменения с сервером. Не забудьте перед синхронизацией с сервером сохранить изменения на клиенте. Тренируясь с нашим приложением, вы почувствуете поведение `Sync Services` в различных ситуациях.

## Переходим к n-уровням

Обсуждавшийся нами пример приложения является приложением "клиент-сервер". Несмотря на то, что такое приложение является отличным инструментом для демонстрации некоторых функций `Sync Services`, оно не слишком практично. В производственных условиях не вся логика приложения находится на клиенте, она разделена между уровнями (часть кода находится на клиенте, а другая часть — на сервере).

Сервис `Sync Services` предназначен для n-уровневых и основанных на службах архитектур. В этом разделе мы собираемся преобразовать наше клиентское приложение для обмена со службой WCF. Вы оцените гибкость `Sync Services` после того, как увидите, насколько быстро это можно сделать (буквально в несколько простых шагов).

### ПРИМЕЧАНИЕ

Подробное обсуждение WCF выходит за рамки этой главы. Мы сосредоточимся на том, как настроить `Sync Services` в ориентированной на службу архитектуре, и оставим описание подробностей WCF другим ресурсам, например, по адресу <http://msdn.microsoft.com/en-us/netframework/aa663324.aspx>.

Пример проекта службы WCF (включенный в пример решения) называется `MySyncServicesWcfService`. Этот проект — библиотека WCF Service Library, сконфигурированная для работы под IIS. Она содержит несколько классов, многие из которых покажутся вам знакомыми, как классы `CustomerServerSyncProvider` и `ServerHelper`. Это точные копии классов, использованных в проекте `MySyncServicesDemo`. Класс `ServerHelper` является служебным классом, содержащим обычные методы. А вот классы `MyCustomerService` и `ISyncService` — новые, они используются для описания службы и ее контракта.

`ISyncService` — это интерфейс, который определяет наш контракт службы. `MyCustomerService` — это класс службы, который реализует интерфейс `ISyncService`. Код обоих показан в листинге 13.17.

#### Листинг 13.17. Интерфейс `ISyncService` и класс `MyCustomerService`

```
[ServiceContract]
public interface ISyncService
{
    [OperationContract]
    SyncContext ApplyChanges(SyncGroupMetadata groupMetadata,
                            DataSet dataSet, SyncSession syncSession);
    [OperationContract]
    SyncContext GetChanges(SyncGroupMetadata groupMetadata,
                           SyncSession syncSession);
    [OperationContract]
    SyncSchema GetSchema(Collection<string> tableNames,
                          SyncSession syncSession);
    [OperationContract]
    SyncServerInfo GetServerInfo(SyncSession syncSession);
}

public class MyCustomerService : ISyncService
{
    private CustomerServiceProvider _serverSyncProvider;
    public MyCustomerService()
    {
        this._serverSyncProvider = new CustomerServiceProvider();
    }
}
```

```
public virtual SyncContext ApplyChanges(SyncGroupMetadata
    groupMetadata, DataSet dataSet, SyncSession syncSession)
{
    return this._serverSyncProvider.ApplyChanges(groupMetadata,
        dataSet, syncSession);
}
public virtual SyncContext GetChanges(SyncGroupMetadata groupMetadata,
    SyncSession syncSession)
{
    return this._serverSyncProvider.GetChanges(groupMetadata,
        syncSession);
}
public virtual SyncSchema GetSchema(Collection<string> tableNames,
    SyncSession syncSession)
{
    return this._serverSyncProvider.GetSchema(tableNames, syncSession);
}
public virtual SyncServerInfo GetServerInfo(SyncSession syncSession)
{
    return this._serverSyncProvider.GetServerInfo(syncSession);
}
}
```

После настройки службы мы создадим (в проекте `MySyncServicesDemo`) ссылку на нее с названием `SyncServiceReference1`. Сделайте это при помощи щелчка правой кнопкой мыши по узлу **References** (под проектом **MySyncServicesDemo**) и последующего выбора в контекстном меню пункта **Add Service Reference**. Затем найдите службу с названием `MyCustomerService` и выберите ее. Потом повторите этот процесс для добавления ссылки на службу `MyContactService`.

### **ПРИМЕЧАНИЕ**

Для того чтобы вы смогли добавить ссылку на службу в проекте `MySyncServicesDemo`, вам может понадобиться собрать и развернуть проект WCF под названием `MySyncServicesWcfService`.

Ссылки на службы добавляют в клиентское приложение прокси-объекты.

## Включение синхронизации на основе служб

После настройки ссылок на службы необходимо указать объекту `customerSyncAgent` на клиенте, что надо использовать прокси-класс службы (вместо локального класса `CustomerServerSyncProvider`). Здесь вам придется немного поработать. В проекте `MySyncServicesDemo` вы должны отредактировать метод `Initialize` класса `CustomerSyncAgent`, чтобы заменить одну строку кода (которая настраивает свойство `RemoteProvider`) двумя показанными в листинге 13.18 строками.

### Листинг 13.18. Обмен с сервером при помощи прокси (вместо прямого обмена с `DbServerSyncProvider`)

```
SyncServiceReference1.SyncServiceClient remoteCustomer =  
    new MySyncServicesDemo.SyncServiceReference1.SyncServiceClient();  
this.RemoteProvider = new ServerSyncProviderProxy(remoteCustomer);
```

Хотите — верьте, хотите — нет, но это все, что вам нужно сделать. Перезапустите приложение, измените запись в таблице `Customers` на сервере, а затем нажмите кнопку **Refresh Customer**. Ваши изменения будут загружены на клиент при помощи службы WCF.

Остановите приложение, повторите редактирование для класса `ContactSyncAgent`, и все готово. Код для замены в методе `Initialize` показан здесь:

```
ContactServiceReference.SyncServiceClient remoteContact =  
    new MySyncServicesDemo.ContactServiceReference.SyncServiceClient();  
this.RemoteProvider = new ServerSyncProviderProxy(remoteContact);
```

## Дополнительные соображения

Мы завершаем данную главу кратким обсуждением диалогового окна **Configure Data Synchronization** в Visual Studio 2008, применения класса `SqlSyncAdapterBuilder` объектов `SyncAdapter` во время выполнения, а также согласования типов данных.

## Использование визуального конструктора для создания локального кэша данных

Для того чтобы действительно изучить и понять Sync Services для ADO.NET и SQL Server Change Tracking, мы без прикрас описали все подробности на-

стройки и программирования приложения исключительно вручную. Другой подход — использовать Visual Studio 2008 для создания объектов и базового кода. При добавлении в ваш проект элемента `LocalDatabaseCache` Visual Studio покажет диалоговое окно **Configure Data Synchronization** (рис. 13.32), которое создаст необходимые классы синхронизации и добавит их в ваш проект.

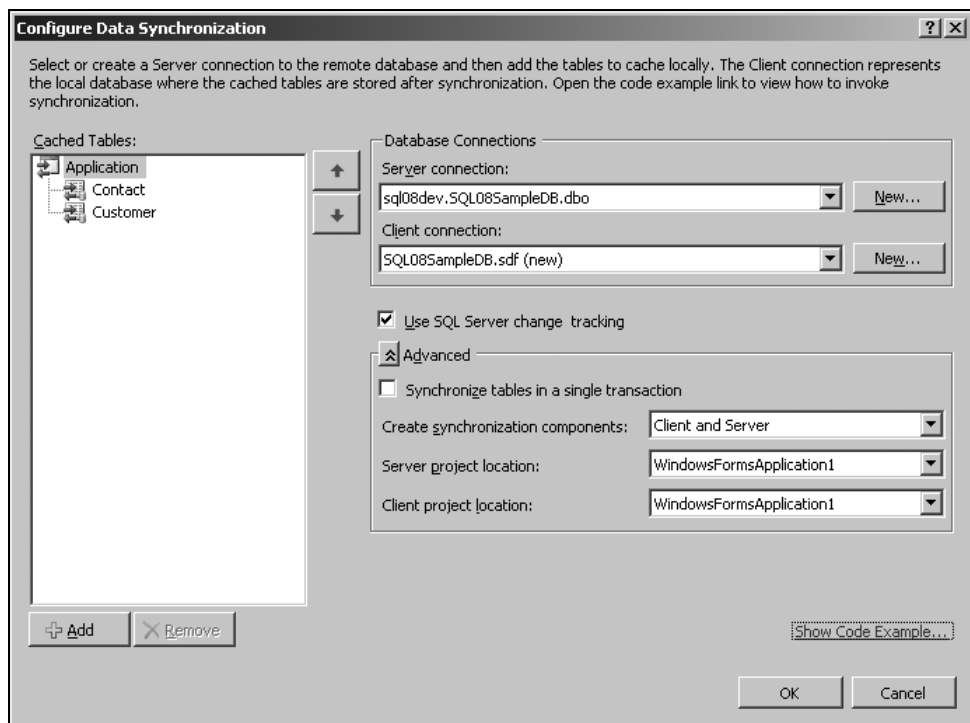


Рис. 13.32. Диалоговое окно **Configure Data Synchronization**

У вас есть возможность выбрать использование SQL Server Change Tracking. В большинстве случаев этим вариантом следует воспользоваться. Однако если ваша серверная база данных отличается от SQL Server 2008, то лучше сбросить этот флажок. После этого диалоговое окно построит пользовательские таблицы Change Tracking в базе данных вашего сервера (рис. 13.33). Сгенерированные скрипты добавляют столбцы в отслеживаемые таблицы, а также создают новые таблицы для отслеживания удаленных строк.

Используя диалоговое окно **Configure Data Synchronization**, вы можете очень быстро настроить приложение (при этом вам даже не нужно знать мно-



гих подробностей о Sync Services). Вы можете использовать созданные здесь таблицы и классы как отправную точку для расширения приложения при помощи двусторонней синхронизации.

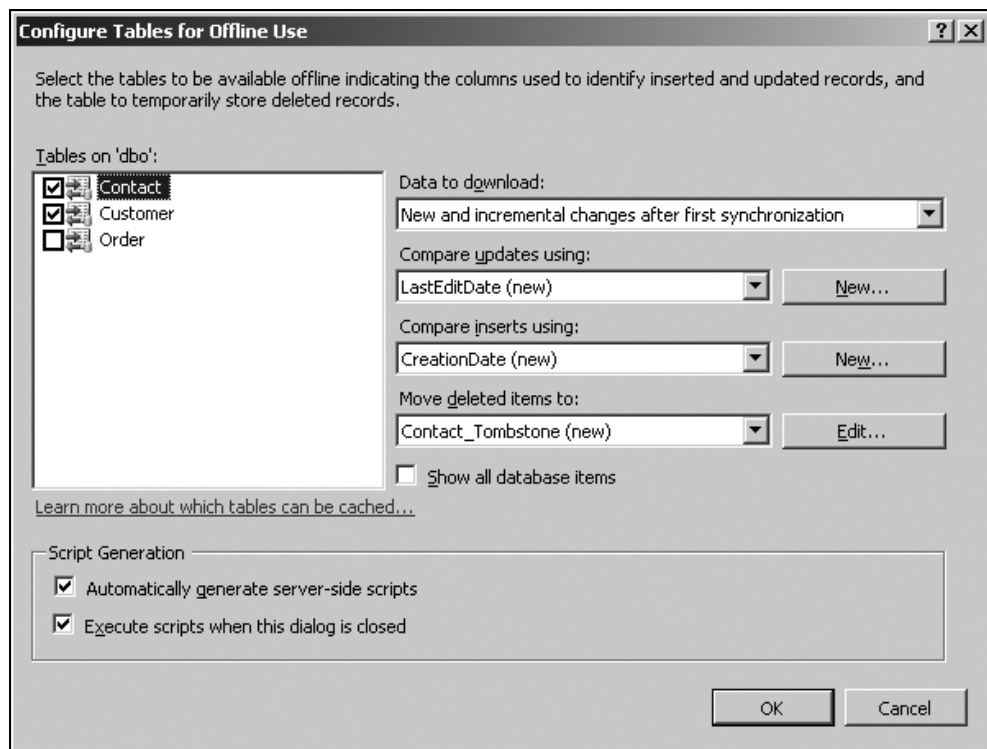


Рис. 13.33. Выбор таблиц для синхронизации и опции генерирования скриптов

## Создание команд при помощи класса *SqlSyncAdapterBuilder*

Класс `SqlSyncAdapterBuilder` создает `SyncAdapter` и необходимые запросы для синхронизации клиентской таблицы с базой данных SQL Server во время выполнения приложения.

`SqlSyncAdapterBuilder` моделируется как объект построителя команд ADO.NET. Построитель создает команды на основе предоставляемой вами информации о синхронизируемых таблицах. Это очень полезная функция, несмотря на то, что у нее есть некоторые недостатки: вы ограничены исполь-

зованием SQL Server на серверной стороне, происходит снижение производительности и безопасности (поскольку построитель использует встраиваемый SQL вместо хранимых процедур).

Указание команд вручную имеет несколько преимуществ перед использованием построителя `SqlSyncAdapterBuilder`. Во-первых, вы можете использовать вместо встраиваемого T-SQL хранимые процедуры. Во-вторых, у вас есть больше возможностей управлять типами изменений, которые выгружаются на сервер. Например, если вы не хотите выгружать на сервер удаления (но используете двустороннюю синхронизацию), вы просто опускаете команду для свойства `DeleteCommand`. В-третьих, вы можете использовать на сервере любую базу данных.

### **ПРИМЕЧАНИЕ**

Построитель `SqlSyncAdapterBuilder` следует использовать в основном для обучения и создания прототипов. Для производственных приложений следует применять команды, которые используют хранимые процедуры везде, где это возможно (из соображений производительности и безопасности).

Если вы решите использовать `SqlSyncAdapterBuilder`, не забудьте соответствующим образом настроить свойство `SyncDirection` объекта `SyncTable`. Свойство `SyncDirection` определяет команды, которые `SqlSyncAdapterBuilder` будет создавать для `SyncAdapter`.

В тех случаях, когда направление вашей `SyncTable` указано как `UploadOnly` или `Snapshot`, вам не нужно делать ничего, кроме вызова метода `ToSyncAdapter` объекта `SqlSyncAdapterBuilder` (как это показано в листинге 13.19).

#### **Листинг 13.19. Создание `SyncAdapter` при помощи класса `SqlSyncAdapterBuilder`**

```
SqlSyncAdapterBuilder customerBuilder =  
    new SqlSyncAdapterBuilder(serverConnection);  
customerBuilder.TableName = "Customer";  
customerBuilder.SyncDirection = SyncDirection.UploadOnly;  
SyncAdapter customerSyncAdapter = customerBuilder.ToSyncAdapter();  
customerSyncAdapter.TableName = "Customer";  
this.SyncAdapters.Add(customerSyncAdapter);
```

Для тех таблиц, направление синхронизации которых `DownloadOnly` или `Bidirectional`, вам понадобится настроить дополнительные свойства, что-

бы `SqlSyncAdapterBuilder` мог сгенерировать правильные команды. Вот небольшой список этих свойств: `CreationTrackingColumn`, `UpdateTrackingColumn`, `TombstoneDataColumns` и `CreationOriginatorIdColumn`.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Для получения дополнительной информации об использовании этих свойств с объектом `SqlSyncAdapterBuilder` рекомендуем материалы по адресу [http://msdn.microsoft.com/en-us/library/microsoft.synchronization.data.server.sqlsyncadapterbuilder\\_properties.aspx](http://msdn.microsoft.com/en-us/library/microsoft.synchronization.data.server.sqlsyncadapterbuilder_properties.aspx).

## **Соображения относительно типов данных**

Вот несколько проблем, которые могут возникнуть, если ваше приложение имеет особые требования к типам данных:

- ❑ службы Sync Services выполняют сопоставление типов данных между базами данных SQL Server Compact 3.5 и SQL Server при их синхронизации. Подробности этого сопоставления обсуждаются в документации SQL Server Books Online. Если ваше приложение имеет другие требования к сопоставлению типов данных, то вы можете использовать для него объект `SyncSchemaColumn`;
- ❑ данные из столбцов `timestamp` не копируются с сервера на клиент;
- ❑ для помеченных атрибутом `ROWGUIDCOL` столбцов `uniqueidentifier` атрибут `ROWGUIDCOL` не применяется в клиентской базе данных при создании таблицы службами Sync Services. Копируются только данные. Вы можете использовать события поставщика `SqlCeSyncClientProvider` для установки атрибута `ROWGUIDCOL` в локальной таблице;
- ❑ столбцы идентификации копируются на клиент с сервера, но на клиенте начальное число всегда 0, а приращение всегда 1;
- ❑ данные вычисляемых столбцов копируются в клиентскую базу данных, но выражения вычисляемых столбцов — нет.

## **Соображения относительно безопасности**

Вы должны отдавать себе отчет в том, что клиентские устройства могут содержать конфиденциальные данные. Вот некоторые шаги, которые вы може-

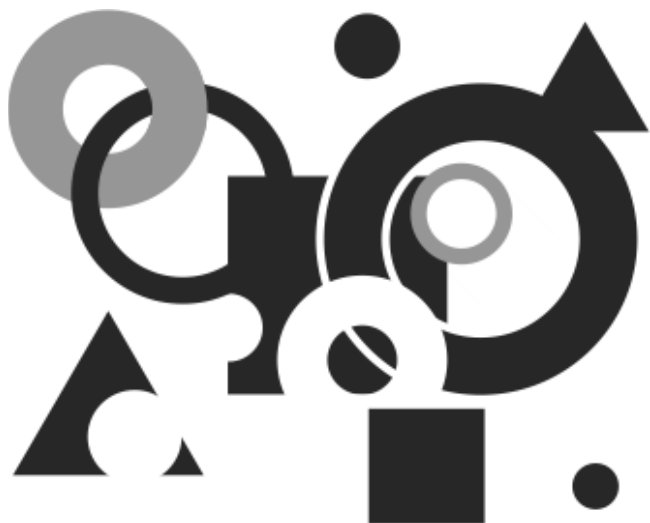
те предпринять для того, чтобы ваша информация не попала в руки злоумышленников (в случае утери или кражи пользовательского устройства):

- ❑ не программируйте имена пользователей и пароли в приложении или файлах конфигурации. Такое допустимо только для демонстрационных приложений. На практике всегда запрашивайте информацию подключения у пользователя (при запуске приложения);
- ❑ используйте хранимые процедуры и предоставляйте доступ к таблицам только через хранимые процедуры. Не разрешайте прямого доступа к таблицам сервера;
- ❑ ограничивайте функциональность серверов и служб (чтобы уменьшить возможности для атаки). Отключайте все службы, которые не используются приложением;
- ❑ SQL Server Compact 3.5 поддерживает шифрование данных, хранящихся в локальной базе данных. По возможности используйте это шифрование.

## Резюме

При создании вашей периодически подключенной системы вы должны будете принять много решений. Выбор между Merge Replication и Sync Services for ADO.NET — это только начало. Merge Replication — это инструмент администратора базы данных, он обеспечивает быструю настройку и конфигурирование, однако ограничен в смысле количества платформ и топологий. Sync Services для ADO.NET — это мечта разработчика: полный контроль плюс надежная объектная модель и масса интересных возможностей для построения периодически подключенных (клиент-серверных или основанных на службах) приложений.

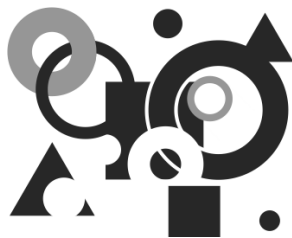
В этой главе мы изучили разработку периодически подключенных систем для SQL Server Compact 3.5 при помощи Merge Replication и Sync Services для ADO.NET в сочетании с SQL Server 2008 Change Tracking. Приложения работают как в интерактивном, так и в автономном режимах, они синхронизируют изменения с сервером при помощи технологий репликации. На основе изложенных в этой главе фундаментальных сведений вы можете расширить ваши бизнес-системы различными новыми "периодически подключенными" возможностями.



# **ЧАСТЬ IV**

## **БИЗНЕС-АНАЛИТИКА**

## ГЛАВА 14



# Хранилища данных

*Марк Фроли*

Эта глава — о хранилищах данных. Если вы до настоящего времени избегали этого понятия (считали слишком сложным или слишком абстрактным для того, чтобы иметь какое-то практическое применение), то данная глава поможет вам отбросить все эти оправдания и охватить тему хранилищ данных. Наши практические советы и указания позволят извлечь больше полезной информации и аналитики из ваших данных. Мы начнем с объяснения, что такое хранилище данных и почему оно вам нужно, а затем покажем, как использовать конкретные функциональные возможности Microsoft SQL Server 2008.

## Определение хранилищ данных

Если вас интересует, что именно означает термин *хранилище данных*, а также имеет ли он какое-нибудь точное значение, то вы не одиноки. Этот термин существует уже более двадцати лет, и, возможно, вы видели самые разные его толкования. Дадим наше определение.

Хранилище данных — это одновременно и представление, и методологический подход к организации и управлению данными предприятия с целью предоставления надежных, непротиворечивых и всесторонних данных, необходимых самому предприятию и использующимся в нем приложениям (тактическим и стратегическим).

Почему в нашем определении нет никаких технических понятий? Именно в этом все дело! Несмотря на то, что для реализации такого представления

действительно необходимы технологии, само хранилище данных — это не технология (и не должно быть технологией). Хранилище — это фундамент данных для работы предприятия. Работа — это принятие информированных решений. И именно для предприятия, а не бизнеса, поскольку хранилища данных есть и в некоммерческих организациях, и в общественном секторе, причем независимо от размера. По сравнению с прошлым, предложения компании Microsoft в сфере хранилищ данных (имеющиеся в семействе SQL Server) сделали хранилища данных доступными и для мелких, и для средних предприятий. Конечно, компания Microsoft продолжает поставлять решения по хранилищам данных и для самых больших предприятий (особенно с выпуском версии 2008 сервера SQL Server).

## Важность хранилищ данных

На сегодняшний день хранилища данных (в какой-либо форме) стали обязательными для работы предприятия сколько-нибудь значительного размера. При полномасштабном применении хранилище данных дает реальное конкурентное преимущество, но даже при использовании его на тактическом уровне (или уровне отделов) оно считается весьма существенным для обеспечения конкурентоспособности (даже при таком простом и необходимом применении, как бухгалтерский учет или начисление заработной платы). Несмотря на то, что часто бывает сложно количественно оценить преимущества хранилищ данных в смысле отдачи от инвестиций в него (return on investment, ROI), сегодня уже никто не подвергает сомнению его важность и необходимость. Как разработчик баз данных, вы, вероятно, будете так или иначе вовлечены в работу с хранилищами данных, если не напрямую, то, по крайней мере, при работе с интерфейсом хранилища данных. Поэтому вам важно понимать, что такое хранилище данных.

Разработка хранилища данных в некоторых отношениях значительно отличается от разработки традиционных систем оперативной обработки транзакций (online transactional processing, OLTP), с которыми вы, вероятно, знакомы лучше. Вот два из наиболее примечательных отличий:

- хранилища данных основаны на данных и их связях (а не на процессах, как в обычных приложениях OLTP);
- многотрудный практический опыт выявил специальные подходы к моделированию данных, которые исключительно полезны для хранилищ данных.

Даже если ваша роль исключительно техническая, вы сможете гораздо лучше работать с интерфейсом хранилища данных, если будете знать об этих отличиях от OLTP и их причины. Это поможет вам также понять точку зрения тех, кто принимает решения (и кто полагается на хранилища данных и их анализ), которая, скорее всего, будет отличаться от подхода пользователей приложений OLTP.

Хранилища данных — это важнейший фундамент того, что известно под названием *бизнес-аналитика* (BI). О тесной связи между хранилищами данных и бизнес-аналитикой мы узнаем далее в этой же главе, но пока примите к сведению, что это не синонимы. В то же время (в соответствии со сделанным нами ранее примечанием) мысленно подставляйте слово "предприятие" во всех случаях, когда слышите слово "бизнес".

Остальная часть данной главы состоит из пяти разделов, которые являются цельным рассказом о хранилищах данных. Вместо того чтобы сразу же сосредоточиться на технических подробностях и пошаговых процедурах в SQL Server 2008, мы даем обзор истории хранилищ данных (которая привела к тому, что на сегодняшний день хранилища данных сильно отличаются от прошлого), а также причины того, почему SQL Server 2008 представляет собой превосходную платформу хранения данных.

Раздел *"Что предшествовало хранилищам данных"* сосредоточивается на истоках хранилищ данных (чтобы понять, почему хранилища данных стали отдельной темой при решении производственных проблем). Второй раздел *"Дизайн хранилищ данных"* описывает два принципиальных подхода к проектированию хранилищ данных. Третий раздел *"Чем хранилища данных не являются"* рассматривает различные термины хранилищ данных (которые очень часто путают) и дает им четкие определения. Четвертый раздел *"Практические советы по хранилищам данных"* рассказывает вам о различных часто встречающихся (но неочевидных проблемах), которые вы можете встретить при создании хранилища данных. И наконец, пятый раздел *"SQL Server 2008 и хранилища данных"* обсуждает специфичные для SQL Server 2008 подробности хранилищ данных.

Имея такие претенциозные планы в этой главе, мы не сможем вам много рассказать о том, как создать безупречное хранилище данных, есть десятки соответствующих книг. Наша цель — дать вам обзор основных сведений, разъяснить термины и указать сложные места, а также изложить некоторые технические подробности о платформе хранения данных, которую предлагает SQL Server 2008.



## **ДАННЫЕ И ИНФОРМАЦИЯ**

Рискнем показаться педантами, но для того, чтобы полностью оценить пользу хранилищ данных, нам потребуется провести границу между данными и информацией. Данные состоят из записанных фактов, например: покупки клиента А в магазине В в день С, за которые заплачено кредитной картой D. Эти факты — суммы покупок (числа). Такие транзакционные данные обычно собираются рабочими приложениями.

Такие факты весьма важны, но информация — это интерпретация фактов, выявление связей между ними и поиск "абстрактного" их значения (если оно существует). Каждая характеристика (клиент, магазин, дата и т. д.) может служить предикатом запроса. Например, каков шаблон покупок по разным магазинам у данного клиента? Или какие магазины имеют наивысшие продажи (по дням)? Конечно, таких запросов может быть бесчисленное множество. Ответы на них позволяют принимать информированные решения на будущее (в отличие от ответов на обычные вопросы, которые продавцы дают по самим фактам, например: "Что это за сумма в моем чеке, которой я не помню?").

Может показаться, что это различие несущественно, но раньше просто не было технических возможностей для представления имеющихся данных в такой форме, которая подходила бы для принятия информированных решений. Вместо этого делались инстинктивные и в некоторой степени обоснованные догадки. В отличие от этого угадывания, хранилища данных организуют, стандартизируют и форматируют факты таким образом, чтобы из них можно было извлечь информацию. В дальнейшем с этой информацией работает бизнес-аналитика.

## **Что предшествовало хранилищам данных**

В зависимости от имеющегося у вас опыта вы, возможно, помните термин "электронная обработка данных" (electronic data processing, EDP или DP), который использовался для описания использования компьютеров в производственных приложениях в течение всех пятидесяти пяти с лишним лет истории компьютеризации. В последние 15 или 20 лет этот термин превратился в "информационные технологии", что часто просто сокращается до аббревиатуры ИТ. Несмотря на то, что это случайное совпадение, время трансформации этих терминов как раз соответствует разделению на периоды "до" и "после" появления хранилищ данных.

До середины 90-х годов прошлого века (в момент расцвета архитектурной парадигмы "клиент-сервер") применение компьютеров в производственных целях имело четкую ориентацию на упрощение или автоматизацию ручных

офисных процессов, а также на регулярные объемные задачи типа выставления счетов, расчета заработной платы и ведения бухгалтерского учета. Такие приложения были первоочередными целями применения компьютеров в промышленных условиях, по крайней мере, по трем причинам:

- повторяемый, весьма ограниченный характер (что облегчало их моделирование и делало их пригодными для автоматизации);
- предполагавшееся уменьшение издержек, связанное с такой автоматизацией;
- техническая реализуемость (с учетом возможностей того времени).

Входные и выходные форматы того времени были весьма несовершенными. В течение длительного времени нормой была пакетная обработка заданий: ввод заданий с перфорированных карт и вывод на бумагу принтера. Кстати, тогда же появилась возможность интерактивной работы (и понятие "он-лайн"). Но использование компьютеров на производстве по-прежнему определялось упомянутыми факторами. Естественным следствием было то, что каждое приложение электронной обработки данных было тесно связано с поддерживаемым им рабочим процессом и практически никак — с другими процессами. Электронная обработка данных записывала основные транзакции предприятия (обеспечивая их целостность), а затем резюмировала результаты в отчетах фиксированной формы. Для обозначения всего этого процесса появился хорошо известный термин "оперативная обработка транзакций".

Электронная обработка данных — вполне подходящее описание для того, что компьютеры и их пользователи делали в период до появления хранилищ данных: они выполняли электронную обработку транзакций, но очень часто не превращали эти данные в информацию (в соответствии с нашим определением).

Несмотря на то, что для работы во многих случаях этого было вполне достаточно, появилось большое количество проблем, которые препятствовали извлечению из накопленных данных дополнительной пользы. Хранилища данных получили свое развитие как ответы на эти проблемы. Давайте же рассмотрим, как им это удалось.

## Отсутствие интеграции с предприятием

Акцент на рабочие процессы неизбежно приводил к созданию неинтегрированных, автономных приложений. Из соображений технических перспектив и целесообразности для предприятия в каждом приложении по-своему опре-

делялись основные объекты, и не только уникальные для данного приложения, но и "основные данные", такие как клиенты и продукты предприятия. Общее понимание значения этих ключевых элементов отсутствовало, поэтому каждое приложение имело свою собственную версию, что приводило к значительному дублированию данных.

При таком состоянии вещей было трудно (или даже невозможно) создать в масштабе предприятия какое-то осмысленное представление о чем бы то ни было. Когда такие попытки делались, они имели характер суммарных отчетов, которые требовали большого количества времени и были дороги, а поэтому создавались нечасто. Принятие решений на уровне предприятия (особенно на операционном и тактическом уровне) по-прежнему в значительной степени зависело от интуиции, опыта и инстинкта. Часто было просто невозможно получить для принятия решения надежную, точную и актуальную информацию. В последние годы перед появлением хранилищ данных предпринимались попытки решить эти проблемы при помощи так называемых информационных систем руководителя (executive information systems, EIS) и систем поддержки принятия решений (decision support systems, DSS). Эти попытки были неэффективны, потому что не давали соответствующей своей стоимости пользы.

### **ОТЧЕТЫ ДЛЯ РУКОВОДСТВА И БУХГАЛТЕРСКИЙ УЧЕТ**

Любое приложение, которое работало с финансовой информацией (а таких было большинство), должно было делать записи в бухгалтерском учете. В результате приложение бухгалтерского учета часто являлось единственным центром интеграции между приложениями. Кроме того, по самому своему характеру оно считалось "истиной в последней инстанции". По этим причинам многие ранние попытки создания отчетности в масштабе предприятия опирались на систему бухгалтерского учета.

Такой подход был полезен, но имелись также и серьезные ограничения. Бухгалтерский учет не слишком хорошо приспособлен к тому, чтобы служить отчетностью для руководства (за исключением самых верхних уровней агрегирования, таких как годовой отчет). Отчетность для руководства — это показатели работы предприятия на более низком уровне, который не интересует бухгалтерский учет. Однако поскольку бухгалтерский учет стал центром интеграции (и, следовательно, источником отчетов для руководства), им стали злоупотреблять. В нем стало увеличиваться количество различных счетов и видов учета (поддерживавших отчетность для руководства), для работы с которыми адаптировались интерфейсы систем бухгалтерского учета. С течением времени такая конфигурация стала разрушаться под тяжестью своего собственного обслуживания (особенно в тех случаях, когда бухгалтерский учет необходимо было реструктурировать при

выполнении слияния компаний). Одним из импульсов к появлению хранилищ данных было желание решить все эти проблемы создания отчетности для руководства.

## **Отсутствие стандартизированных справочных данных**

В связи с таким отсутствием интеграции на предприятии обычно не существовало и единой согласованной "системы записи" основных справочных данных (таких как данные по клиентам или продуктам). Отсюда возникали проблемы неполных и неточных данных, дублирование данных (и соответствующие ошибки), а также потерянное на синхронизацию данных (из разных приложений) время. Но самой главной была невозможность получить (без значительных издержек) всестороннее и актуальное представление о предприятии. Кроме этих совершенно явных проблем, были и менее очевидные, например, возможность по ошибке расторгнуть отношения с клиентом, который неприбылен в одном районе, но очень прибылен в целом (поскольку вы не можете увидеть общую картину работы с этим клиентом по всем районам, продуктам и структурным подразделениям).

Эти проблемы и их причины хорошо осознавались отделами электронной обработки данных и руководством предприятий, что и привело к попыткам создания "основных данных" по самым важным объектам, например: клиентам, тарифам, продуктам и иерархии предприятий. Однако технические ограничения, битвы политического характера и отсутствие понимания вреда такой разрозненности со стороны высшего руководства предприятий не позволили добиться успеха.

## **Отсутствие ретроспективных данных**

Операционные приложения (назовем их ОП) по своему характеру не требуют наличия ретроспективных данных за сколько-нибудь продолжительные периоды времени (и не хранят их), обычно в них содержатся данные за один или два года. Конечно, есть и исключения, такие как приложения по работе с ипотечными кредитами в банке или страхование жизни у страховщиков. Это тоже приложения ОП, но они должны хранить данные за десятки лет. Однако в большинстве случаев ОП хранят минимум ретроспективных данных, чтобы оптимизировать производительность оперативной обработки транзакций и

минимизировать стоимость хранения данных, а также потому, что большего от них и не требуется.

В любом случае ОП предприятия отличаются по продолжительности хранимого в них периода ретроспективных данных, их периодичности (час, день, неделя, месяц и т. д.) и по тому, как обрабатываются изменения ретроспективных данных (хранится ли история изменений, по каким атрибутам и сколько версий). Эти отличия делают интеграцию ретроспективных данных из разных ОП, по меньшей мере, непростой.

## Данные не оптимизированы для анализа

Между ОП и аналитическими приложениями (для краткости АП) существуют и более существенные различия. Как мы уже сказали, ОП (особенно в период до хранилищ данных) в основном занимались регистрацией транзакций. Их почти не интересовала история прошлого или другие ОП, вследствие чего они и получили название "островки автоматизации".

В отличие от ОП, аналитические приложения (АП) "переваривают" данные ОП для того, чтобы дать руководство к действию, прогнозы и представления о предприятии в целом. Иногда такие приложения даже сочетают внутренние и внешние данные (такие как данные по конкурентам), что дает представление о предприятии в более широком контексте. Для достижения этих целей требуется решать проблемы такого рода, которыми операционным приложениям заниматься не приходится. Кроме таких отличий общего характера есть и более специфические.

- В соответствии с характером своей работы ОП физически оптимизированы для операций вставки, обновления и удаления, а приложениям АП требуется оптимизация чтения или запросов.
- Количество данных, используемых для ответа на обычный запрос ОП, очень мало, зато для ответа на обычный запрос АП может потребоваться огромное количество данных. Представьте себе тот объем данных, который необходимо переварить для ответа на вопрос, какие пять клиентов имели максимальный объем закупок в 2007 г. и по каким пяти продуктам они делали максимальные закупки.
- Для интеграции приложений ОП в масштабе предприятия имеется масса препятствий. Вот некоторые из них:
  - объекты, которые обозначают одно и то же, но названы по-разному;
  - объекты, которые обозначают разные вещи, но названы одинаково;

- различные кодировки одних и тех же вещей (например, коды стран);
- разные масштаб и точность измерений;
- разная длина текстов описаний одной и той же вещи;
- разные условные обозначения первичного ключа одного и того же объекта;
- интеллектуальные ключи — в первичных ключах хранится информация.

В результате можно сказать:

- создание представления по данным предприятия (особенно по данным из многих приложений) было очень сложной технической задачей, которую мог выполнить только персонал отдела электронной обработки данных. Обычно такие отчеты делались с большим запозданием;
- очень часто такие отчеты (которые помогли бы в управлении предприятием) даже не запрашивались. Пользователи знали, что к тому времени, как отдел электронной обработки данных сможет их выполнить, поздно использовать это в ведении бизнеса;
- каждый запрос обычно реализовывался в виде нового отчета (даже если от предыдущего он почти не отличался). Вследствие технологических ограничений того времени даже такая простая (для сегодняшнего дня) задача, как агрегирование данных на другом уровне (например, по кварталам, а не по месяцам), приводила к созданию нового отчета. И более того, даже если существовал отчет, который мог дать ответ на запрос, то об этом было невозможно узнать (поскольку не было метаданных по существующим отчетам) — и создавался новый отчет;
- каждый отчет навечно сохранялся в инфраструктуре системы. Часто не было даже возможности отследить, кто и зачем использует данный отчет (если он вообще используется), поэтому после создания отчета проще всего было продолжать хранить его;
- были также и отчеты по отчетам, один отчет становился источником для другого. Отслеживание этих зависимостей было сложным (а иногда и невозможным).

Совершенно понятно, что в эксплуатации все это представляло собой полный кошмар. Но до начала 90-х годов в обычном отделе электронной обработки данных это было совершенно типичным явлением, причем положение продолжало ухудшаться. Стало совершенно очевидно, это был развивающийся кризис, и как ответ на него родились хранилища данных.

Для справедливости следует отметить, что попытки создания хранилищ данных предпринимались задолго до появления самого термина. Но в то время такие создатели по существу просто каждый раз по-новому изобретали колесо. Они не могли воспользоваться достижениями сегодняшнего дня, когда технологии возмужали и благодаря появлению Интернета стали доступны. Верно также и то, что развитие аппаратного обеспечения (радикальное снижение стоимости дискового пространства и фантастически возросшая производительность процессоров) имело существенное влияние на развитие хранилищ данных и их нынешнюю жизнеспособность.

## Проектирование хранилищ данных

Предыдущее изложение дает вам представление о проблемах, которые предстояло решить хранилищам данных. В этом разделе мы дадим лишь поверхностный обзор проектных требований к хранилищу данных и надеемся на то, что это раздразнит ваш аппетит и побудит вас к дальнейшему изучению. К счастью, в наши дни нет ничего проще, чем изучать хранилища данных.

### **ПРИМЕЧАНИЕ**

Польза хранилищ данных признавалась не всегда. В первые годы их существования к ним относились с подозрением, многие сотрудники считали их просто модой и напрасной тратой драгоценного времени. В лучшем случае их считали тем, что "было бы хорошо иметь" и что могут себе позволить только хорошо финансируемые (в основном коммерческие) организации. К счастью, все это в прошлом.

Создание хранилища данных требует решения огромного количества технических и нетехнических проблем, в том числе:

- ☐ определение целей предприятия (которым будет служить хранилище данных) и создание для организационной структуры;
- ☐ идентификация различных групп потребителей данных и их требований;
- ☐ выполнение требований по запаздыванию (при помощи выбора соответствующей архитектуры данных);
- ☐ извлечение, преобразование и загрузка (Extract, transform, and load, ETL) — процесс и инструменты для извлечения данных из ОП, их очистки и преобразования при необходимости, а также для последующей загрузки в хранилище данных. Службы интеграции SQL Server Integration

Services (SSIS) — это основной инструмент ETL компании Microsoft для хранилищ данных;

- ❑ проектирование схем наименования, резервного копирования, мобильности, масштабируемости и доставки для данных, а также схемы обучения;
- ❑ методы доступа конечных пользователей к информации (разделение отчетности и аналитики). Инструментам для этих целей часто уделяется непропорционально большое внимание в проектах хранилищ данных (поскольку они "на виду");
- ❑ организационное воздействие, которое хранилище данных будет оказывать путем поддержки постоянно изменяющихся потребностей предприятия. Это вечный аспект.

Основная цель любого хранилища данных — это интеграция данных из разных источников в централизованное (в логическом смысле) хранилище, причем в такой форме, которая может быть использована для поддержки принятия решений во всем предприятии (и всеми теми, кто в этом нуждается). Такая интеграция не означает простого сброса всех данных из различных автономных приложений в одну общую базу данных. Для хранилища данных требуется определенная схема, к которой приводятся все зачисляемые в него данные. Данные должны быть также "чистыми", это означает, что все различные способы представления одной и той же вещи (в разных источниках данных) преобразованы к единому виду. Обе эти задачи возложены на ETL (как уже упоминалось ранее).

На рис. 14.1 представлен вид хранилища данных "с высоты птичьего полета".



**Рис. 14.1.** Общая архитектура хранилища данных



## Подход сверху вниз

Термин хранилищ данных изобретен в 1990 г. (автор William Inmon). В соответствии с определением, основными характеристиками данных в хранилище являются следующие:

- ❑ ориентация на объект — в разных ОП есть много общих объектов. Клиент, продукт, поставка и счет — вот типичные объекты;
- ❑ интеграция — источники данных не противоречат друг другу по общим объектам;
- ❑ неизменность — загруженные данные обычно не меняются (не обновляются и не удаляются);
- ❑ изменяемость во времени — время всегда является частью ключа, сохраняется состояние на данный момент времени (то есть сохраняется "история").

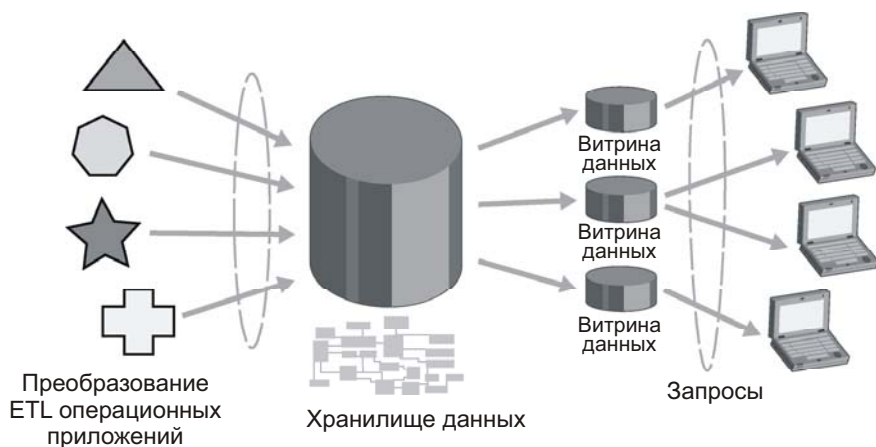
Такое хранилище данных подразумевает, что модель данных предприятия уже создана (или будет создана), и в ней описаны все имеющиеся в разных ОП общие объекты, необходимые численные меры, уровень детализации каждой, а также связи между ними. Постулируется, что логическая модель данных хранилища — это нормализованная реляционная модель такого типа, который используется приложениями оперативной обработки транзакций. Акцент делается на централизованное и нормализованное хранилище данных.

Поскольку сложность нормализованной модели не позволяет делать прямых запросов (по соображениям производительности и простоты использования), то такая архитектура предполагает также наличие различных *витрин данных*, являющихся различными производными базами данных, структура которых оптимизирована для запросов, и которые обычно содержат только агрегированные данные, полученные по хранилищу данных. Ключевым здесь является то, что их архитектура вторична и независима от хранилища данных. На рис. 14.2 показано уточнение рис. 14.1 в соответствии с концепцией хранилища данных.

Поскольку такой подход обычно предполагает, что до начала создания хранилища данных уже существует крупномасштабная модель, то он называется подходом "сверху вниз".

William Inmon написал несколько книг, в которых излагаются принципы и доработки этой архитектуры и в соавторстве с Claudia Imhoff разработал еще более крупную архитектуру — информационную фабрику корпорации (Corporate Information Factory, CIF), составной частью которой является хранилище данных. Ограничения по объему книги не позволяют нам углубиться

в детали подходов Inmon и CIF. Однако перед продолжением мы хотим сделать два замечания.



**Рис. 14.2.** Хранилище данных

О первом вы, наверное, уже и сами догадались, такое требование наличия модели данных предприятия для многих организаций нецелесообразно. Оно успешно выполнялось (обычно на крупных предприятиях), но многим покажется невозможным обосновать расходы на разработку модели (с учетом того, что по ее итогам будет нечего продемонстрировать, кроме документации). Нет никакого сомнения в том, что если его можно выполнить, оно заложит отличный фундамент для информационных приложений, но во многих случаях это невыполнимо.

Второе замечание: многие считают такой подход абстрактным, полезным для акцентирования архитектуры системы, но не слишком полезным в плане практических деталей при реальной разработке. Следующий обсуждаемый нами подход к хранилищам данных появился для того, чтобы ответить на оба этих замечания.

## Подход снизу вверх

С середины 90-х годов и до настоящего времени Ralph Kimball публикует работы по альтернативному подходу к хранилищам данных, ядро которого он называет моделью измерений (Dimensional Model). Если подход Inmon

можно назвать подходом "сверху вниз", то подход Kimball определенно является подходом "снизу вверх", хотя оба предполагают пошаговое продвижение. Один акцентировал и сформулировал некоторые уже использовавшиеся практиками концепции, а другой систематизировал некоторые используемые практики, которым не хватало цельного видения.

Первая из них — модель измерений, которая представляет собой наиболее элегантный компромисс между доступностью для конечного пользователя, простотой, легкой расширяемостью, а также хорошей производительностью как для предопределенных, так и для оперативных запросов. Вторая — идея построения хранилища данных инкрементальным образом. Это то, что большинство предприятий считает более приемлемым, чем подход "большого взрыва" (когда меняется все сразу), как подразумевается архитектурой Inman. Ключевой частью этой идеи является концепция "согласованных измерений" (которую мы опишем чуть дальше), обеспечивающая возможность интегрирования в хранилище данных новых инкрементальных разработок (в отличие от другого подхода, когда каждый новый этап дает нам новый островок автоматизации). И, в-третьих, Kimball подчеркивает практичность такой реализации, давая очень конкретные советы по множеству проблем проектирования данных на страницах своих книг, Web-сайте, регулярных семинарах и курсах подготовки.

Многие считают такой подход весьма желательным, о чем свидетельствует факт, что большинство инструментов анализа данных на сегодняшнем рынке (включая и службы Microsoft SQL Server Analysis Services, которые мы опишем в *главах с 15 по 18*) имеют определенное родство с моделью измерений. По этой причине (а также потому, что такой подход менее абстрактный) оставшуюся часть данного раздела мы посвятим обзору этого подхода.

## **ВАЖНО**

Inmon и Kimball являются самыми известными "зубрами" в области хранилищ данных. Хорошо это или плохо, но поскольку их подходы считаются такими разными, то каждый из них обзавелся целым лагерем сторонников, которые критикуют взгляды своих противников на хранилища данных (причем иногда с излишней горячностью). Тем не менее, оба они придерживаются определенной архитектуры при проектировании хранилищ данных и пошагового подхода к их проектированию и созданию. Фактически большая часть проектов хранилищ данных сочетает в себе элементы обоих подходов (как оно, впрочем, и должно быть), потому что каждый из этих подходов имеет свои превосходные идеи. Вот почему вам следует знать об обоих.

Этот раздел не посвящен научить вас подходу Kimball. Объем книги позволяет только изложить вам несколько ключевых концепций. Это должно будет облегчить ваши дальнейшие исследования.

## Терминология

Вам следует знать несколько полезных терминов из области хранилищ данных, которые (несмотря на то, что они тесно связаны с Kimball и его моделью измерений) стали широко известны благодаря их присутствию во многих инструментах (особенно инструментах OLAP). Большинство этих терминов вы снова встретите в главах, в которых описываются службы SQL Server Analysis Services (*главы с 15 по 18*):

- мера — обычно это интересующее нас числовое значение, такое как цена, баланс счета или складской остаток. Хранящиеся в хранилище данных меры определяются историей предприятия и происходят обычно из операционных приложений (которые являются источником данных для хранилищ). Мера характеризуется также *гранулой*;
- измерение — сердцем модели измерений является измерение, которое часто описывается как "ось анализа" или спецификатор "что". Измерение помогает квалифицировать меру и дать ей контекст. В запросе измерение может быть частью результата запроса или частью ограничений запроса. Самое важное измерение — это Time, которое необходимо практически в любом контексте. Остальные зависят от области применения, но обычно основными являются Customer, Product и Geography. Измерения обычно считаются справочными элементами (элементами основных данных). Измерение — это коллекция связанных значений, называемых *элементами*, например, 2008 может быть элементом измерения Time, а John Smith — элементом измерения Customer. В модели измерений сами измерения считаются независимыми друг от друга (даже если на самом деле это не так). Например, Customer и Product не являются независимыми, поскольку не все клиенты покупают все продукты; однако моделируя их в качестве измерений, мы работаем с ними так, как будто они независимы друг от друга (поскольку это упрощает концептуальную модель, по которой создаются запросы). Очень небольшое количество измерений имеет нулевую корреляцию с другими измерениями (если таковые вообще имеются);
- иерархия — определенная организация "родитель-потомок" элементов в измерении. Каждый набор родителей называется *уровнем* иерархии. Например, измерение Time может иметь уровни с названиями Year и Month.

Уровень Year может иметь элементы вроде 2007 и 2008, а уровень Month может иметь элементы вроде Jan 2007 и Jan 2008 с родительскими элементами 2007 и 2008 на уровне Year. Иерархии естественным образом появляются в самом широком диапазоне приложений и являются всего лишь способом группирования элементов для суммирования. Иерархия отражает тот факт, что разные элементы одного измерения представляют разные уровни детализации;

- таблица измерений — реляционная таблица, содержащая (чаще всего) по одной строке на каждый элемент измерения (их количество зависит от того, какие ретроспективные данные поддерживаются для данного измерения). Таблица измерений обычно имеет как минимум два столбца, один из которых представляет собой ключ (или идентификатор), который уникальным образом определяет элементы измерения, а второй — содержит описательное имя элемента;
- таблица фактов — реляционная таблица, которая с точки зрения моделирования данных работает как ассоциативный элемент между различными измерениями. Она содержит один или более столбцов мер, а также все ключевые столбцы всех связанных измерений. Она заполняется (при помощи ETL) таким образом, что значения мер полностью описываются соответствующими ключами измерений. Таблица фактов характеризуется также своей гранулой. Все меры одной таблицы фактов имеют (должны иметь) одну и ту же гранулу;
- схема звезды — термин для обозначения конфигурации диаграммы отношений сущностей (Entity Relationship, E/R), на которой изображены таблица фактов и соответствующие таблицы измерений;
- гранула — характеристика меры, которая определяется в терминах соответствующих измерений. Гранула имеет два свойства: во-первых, это измерения, которые определяют контекст меры, во-вторых, для каждого такого измерения — это уровень в иерархии измерений, который определяет уровень детализации меры. Эти два свойства определяют гранулу меры. Например, если все меры таблицы фактов относятся к значениям уровня Month иерархии Year-Month измерения Time, то гранула по Time таблицы фактов равна Month. Общая гранула таблицы фактов (называемая ее *гранулярностью*) определяется этими характеристиками всех ее измерений;
- конформное измерение — спроектировано и создано таким образом, чтобы каждая схема звезды (которая включает данное измерение) могла быть логически соединена по такому измерению. Все вхождения такого

измерения в различных таблицах фактов означают одно и то же, каждое измерение включает одни и те же элементы, и каждый элемент имеет совершенно одинаковое значение в отношении тех фактов, контекст которых он помогает определить. Kimball называет такое состояние "шинной архитектурой".

Это не тот случай, когда все таблицы фактов, использующие измерение, должны использовать его на одном и том же уровне (если они имеют иерархию). Например, если одна таблица фактов находится на уровне Year измерения Time, а другая — на уровне Month, то данные из этих таблиц можно скомбинировать, для этого необходимо просто сначала агрегировать данные Month до уровня Year. Без конформных измерений различные схемы звезды нельзя комбинировать по их общим измерениям, и в этом случае инкрементальный подход к построению хранилища данных невозможен. Создание конформных измерений, вероятно, является самой трудной частью подхода модели измерений, и именно здесь он максимально пересекается с подходом Inmap: должно быть достигнуто организационное соглашение о том, какие измерения могут быть конформными (и что они означают). Именно здесь становится очевидной также и нехватка нужных данных (данных необходимой гранулярности) в исходных операционных приложениях.

### **ПРИМЕЧАНИЕ**

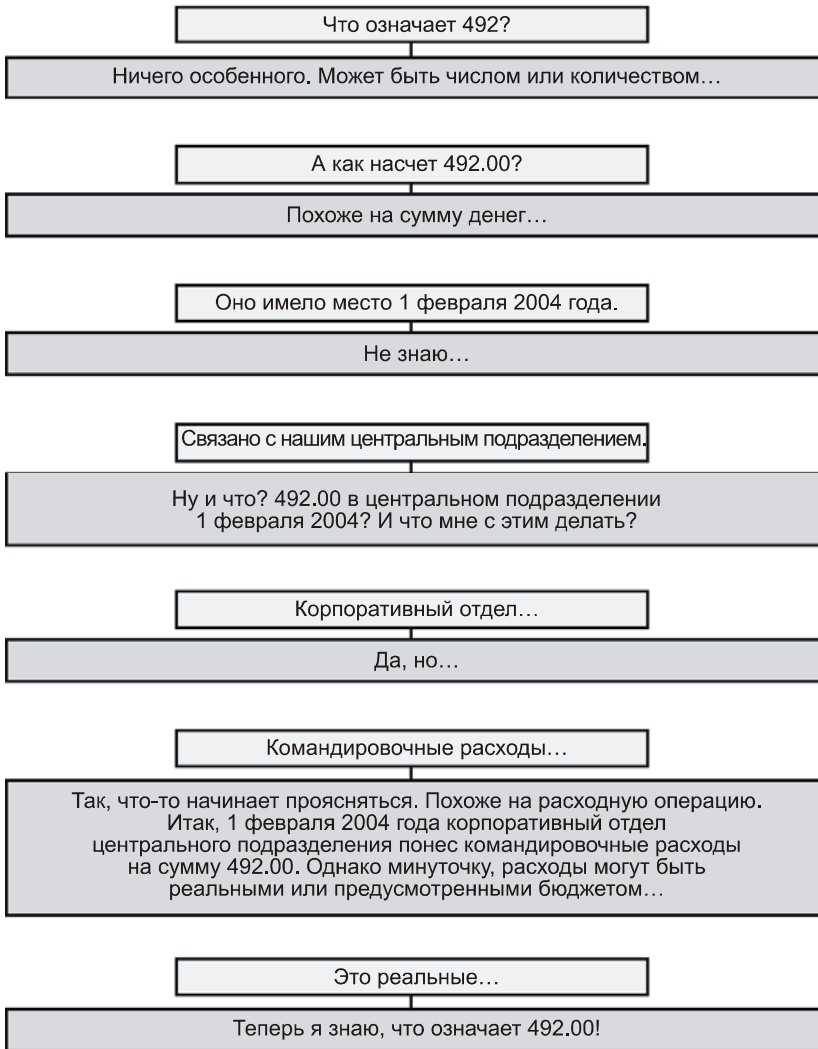
Несмотря на то, что термин "конформное измерение" делает акцент на измерениях, важна также и гранула тех мер, которым дается контекст этими измерениями. Для определения конформных измерений должны существовать определения мер, гранула которых в предлагаемом конформном измерении одинакова во всех существующих или предполагаемых таблицах фактов.

## **Контекст и схема звезды**

Как уже упоминалось ранее, измерения обеспечивают контекст меры. На рис. 14.3 представлен воображаемый диалог, который демонстрирует, как для смысла данных необходим контекст.

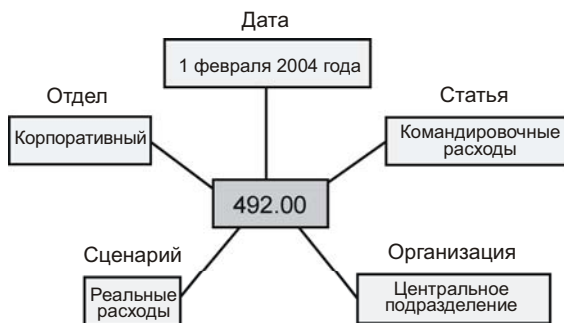
### **ПРИМЕЧАНИЕ**

Знаем ли мы теперь все необходимое, чтобы дать полный контекст числу 492,00? Пока нет, нам нужно еще одно предположение. Можете угадать, какое? Конечно же, относительно валюты этого расхода.

**Рис. 14.3.** Определение контекста меры

Давайте теперь составим диаграмму этого диалога (как показано на рис. 14.4).

Мы можем изучить реальную реализацию этого примера. Выполните показанный в листинге 14.1 код по базе данных AdventureWorksDW2008.



**Рис. 14.4.** Представление того, что нам известно о 492,00

**Листинг 14.1. Запрос значения определенной меры из базы данных AdventureWorksDW2008**

```

USE AdventureWorksDW2008
GO
SELECT
    dd.FullDateAlternateKey,
    do.OrganizationName,
    ddg.DepartmentGroupName,
    da.AccountDescription,
    ds.ScenarioName,
    ff.Amount
FROM
    FactFinance ff
    INNER JOIN DimDate AS dd
        ON ff.DateKey = dd.DateKey
    INNER JOIN DimOrganization AS do
        ON ff.OrganizationKey = do.OrganizationKey
    INNER JOIN DimDepartmentGroup AS ddg
        ON ff.DepartmentGroupKey = ddg.DepartmentGroupKey
    INNER JOIN DimScenario AS ds
        ON ff.ScenarioKey = ds.ScenarioKey
    INNER JOIN DimAccount AS da
        ON ff.AccountKey = da.AccountKey
WHERE
    dd.FullDateAlternateKey = '2/1/2004' AND
    do.OrganizationName = 'Central Division' AND
  
```



```
ddg.DepartmentGroupName = 'Corporate' AND  
da.AccountDescription = 'Travel Lodging' AND  
ds.scenarioName = 'Actual'
```

### ПРИМЕЧАНИЕ

Образцовая база данных AdventureWorksDW2008 реализует хранилище данных по схеме Kimball. Во введении к данной книге есть указания по скачиванию этих данных.

Из этого запроса и из диаграммы E/R, в которой представлены соответствующие таблицы, мы можем увидеть на рис. 14.5 то, что называется схемой звезды.

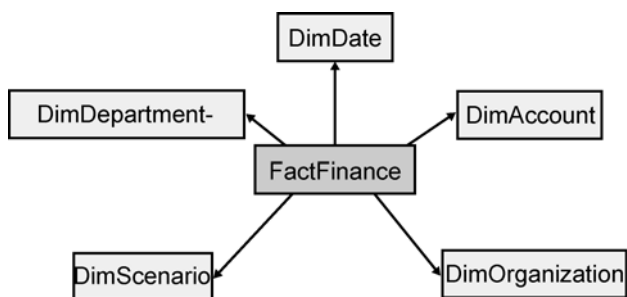


Рис. 14.5. Схема звезды из базы данных AdventureWorksDW2008

## Суррогатные ключи

Концепция *суррогатных ключей* появилась не у Kimball и не в модели измерений, но именно ее они усиленно пропагандируют. Суррогатный ключ — это присваиваемый системой (обычно целочисленный) первичный ключ таблицы. В SQL Server суррогатным ключом обычно является столбец идентификаторов, хотя иногда в конкретной архитектуре может оказаться предпочтительным иметь центральный *генератор ключей*, который выдает суррогатные ключи по мере необходимости. Суррогатные ключи имеют две важные характеристики:

- в них ничто не закодировано — то есть это не "интеллектуальные ключи". Это делает их невосприимчивыми к изменениям исходных данных, которые могут повредить несуррогатные первичные ключи. Исключени-

ем здесь является суррогатный ключ измерения Time, где интеллектуальное представление суррогатного целочисленного ключа в форме YYYYMMDD (когда это применимо к грануле таблиц фактов) может значительно облегчить разбиение таблиц фактов на разделы;

- как целые числа, суррогатные ключи являются самыми эффективными первичными ключами (как с точки зрения производительности, так и с точки зрения объема хранения).

На этом наш краткий обзор подхода Kimball к хранилищам данных заканчивается. Мы вам настоятельно советуем посмотреть ссылки в конце этого раздела, а также поискать в Интернете дополнительную информацию. Рисунок 14.6 иллюстрирует вид построенного по принципам Kimball хранилища данных. Здесь необходимо отметить важный аспект: хранилище данных является коллекцией схем звезды, отдельных витрин данных (как в подходе Inmon) не существует. (И кстати, в хранилище данных по Inmon нет запрета на то, чтобы витрины данных были выполнены по архитектуре Kimball). Хотя на этом рисунке и не показано, но предполагается, что разные схемы звезды не являются непересекающимися: когда у них есть общее функциональное измерение (такое как Customer или Product), то они созданы таким образом, чтобы совместно использовать одну версию измерения. Когда это сделано, данные разных схем звезды можно комбинировать по общим измерениям, а это становится возможным благодаря тому, что они были созданы как конформные измерения (если говорить языком модели измерений).

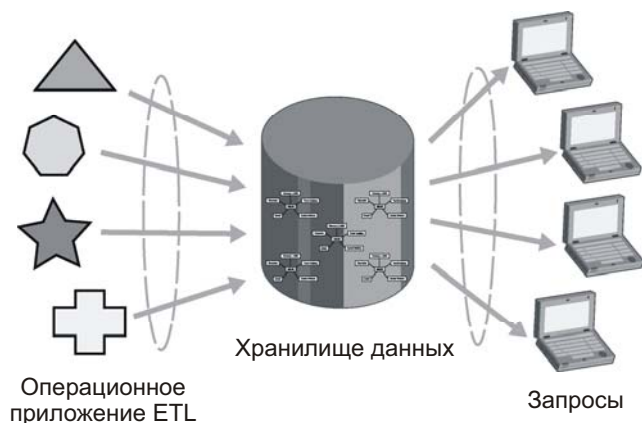


Рис. 14.6. Хранилище данных по Kimball

## Чем хранилище данных не является

В литературе и среди практиков существует много путаницы, поскольку многие термины постоянно примешивают к хранилищам данных, хотя достаточная нынешняя зрелость данной области должна была бы исключать такую путаницу. Снисходительная точка зрения на эту ситуацию состоит в том, что в прошлом это было вполне понятно: тогда данная область быстро развивалась и в области теории, и в области практики, и в смысле выходявших программных продуктов. Однако сегодня должно было бы быть больше четкости, точности и взаимопонимания. Для способствования этому мы должны констатировать наличие существенных различий в некоторых неверно или избыточно используемых терминах. В данном разделе дается обзор некоторых из этих терминов.

## OLAP

Термин "аналитическая обработка данных" (OLAP) был введен доктором E. F. Codd (автором реляционной модели) в 1994 г. для обозначения набора свойств аналитического приложения, в отличие от знаменитой публикации двенадцати правил, которым должна соответствовать система управления реляционной базой данных, по адресу [http://en.wikipedia.org/wiki/Codd's\\_12\\_rules](http://en.wikipedia.org/wiki/Codd's_12_rules). Этот термин должен был прописать различия между уже хорошо известными в то время свойствами приложений оперативной обработки транзакций (OLTP) и менее изученными свойствами аналитических приложений. Основная ценность этого термина в том, что он подчеркивает необходимость проведения такого разграничения. Нынче это понятие можно считать и ответом на ограниченность приложений на базе электронных таблиц. Сегодня с этим термином практически ассоциируются и технологии на базе кубов (несмотря на то, что они не являются частью определения этого термина).

### **ПРИМЕЧАНИЕ**

Так же, как и в случае хранилищ данных, уже задолго до появления термина OLAP существовали подобные проекты (их можно проследить до 60-х годов прошлого века).

Приложение OLAP часто (хотя это и не обязательно) берет данные из некоторой формы схемы звезды. Имеющиеся сегодня на рынке разнообразные инструменты OLAP отличаются по степени необходимости для них схемы

звезды (в качестве источника данных). На одном полюсе находятся те инструменты, которые предоставляют функциональные возможности OLAP (с относительно несложными вычислениями) практически для любого источника данных (с любой его организацией), а на другом — те инструменты, которые могут использовать в качестве источника данных только кубы (структуры данных для облегчения быстрого анализа, они описываются в *главе 15*). Хранилище данных очень полезно в качестве источника данных в любой точке между этими двумя полюсами и совершенно необходимо на том полюсе, где происходит работа с кубами.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Тест Fast Analysis of Shared Multidimensional Information (FASMI) точнее определяет свойства термина OLAP. Подробную информацию по FASMI, а также массу другой превосходной информации по OLAP (причем большую часть — бесплатно) вы можете получить по адресу: <http://www.olapreport.com/fasmi.htm>.

В сервере SQL Server аналитические службы Analysis Services являются полнофункциональным ядром OLAP, они подробно описаны в *главах с 15 по 18*.

## **Интеллектуальный анализ данных**

Для традиционного способа извлечения информации из данных требуется опытный аналитик с глубоким пониманием предприятия, который формулирует запросы, например: каково влияние рекламной акции прошлого месяца на продажи, какие из десяти самых лучших в этом году магазинов были в числе десяти лучших в прошлом году. По существу, аналитик формирует гипотезу причины и следствия, а затем проверяет ее по данным. Для того чтобы этот процесс получения информации методом проб и ошибок был эффективным, требуются такие инструменты, которые позволяют легко формулировать запросы и обеспечивают на них быстрые ответы, чтобы аналитик не сбивался с хода мысли. Технология OLAP для этого приспособлена идеально.

В противоположность этому, интеллектуальный анализ данных — это подход, при котором существующие в наборе данных корреляции автоматически обнаруживаются при помощи специальных моделей данных и статистических алгоритмов. Поскольку процесс автоматизирован, он более тщательно отыскивает корреляции, причем на него не оказывают влияния предрассудки и пробелы, которые непременно будут у аналитика при использовании про-

извольных запросов. Аналитику еще нужно будет оценить каждую корреляцию, чтобы понять, есть ли в ней смысл или она просто коррелятивна.

В принципе, интеллектуальному анализу данных не требуется хранилище данных в качестве источника данных. Однако хорошо спроектированное хранилище данных с очищенными данными было бы идеальным источником данных. Характер анализа и допустимое запаздывание также оказывают влияние на пригодность хранилища данных в качестве источника данных для анализа. Например, для обнаружения мошенничества с кредитными картами частота обновления хранилища данных вряд ли будет достаточной.

Начиная с SQL Server 2000, компания Microsoft приложила массу усилий для того, чтобы дать аналитическим службам SQL Server Analysis Services такие возможности для интеллектуального анализа данных, которые было бы гораздо легче использовать (чем имеющиеся на рынке инструменты) даже не специалисту. Эти возможности подробно описываются в *главе 18*.

## Бизнес-аналитика

Термин "бизнес-аналитика" (Howard Dressner в 1989 г.) оказался чрезвычайно популярным. Сегодня он используется в таких разных контекстах, что можно вполне усомниться, характеризует ли он что-нибудь вообще. Некоторые утверждают, что не характеризует, но мы так не считаем. К несчастью, слово "бизнес" здесь заслоняет собой тот факт, что бизнес-аналитика может быть полезна на любом предприятии (а не только в коммерческих организациях). Поэтому (как мы уже предлагали ранее) когда вы слышите "бизнес-аналитика", думайте об аналитике предприятия.

Самое важное, что бизнес-аналитика (в ее правильном понимании) — это не какая-то конкретная технология (хотя ее реализация несомненно полагается на технологии). Бизнес-аналитика в ее фундаментальном смысле — это подход и философия управления. Подобно большинству других хороших идей, ее базовое положение звучит так просто, что в нем вроде бы и не о чем говорить: управленческие решения должны базироваться на фактах, а не на высококвалифицированном гадании, политике или других субъективных факторах. Разумеется, управление предприятием всегда было основано и на некоторой объективной информации, в числе которой был и бухгалтерский учет. Однако в прошлом такая объективная информация (особенно на уровне предприятия) имела сводный характер, предоставлялась нечасто, была жестко структурирована и не могла дать детализацию тех данных, из которых она была получена.

Бизнес-аналитика стремится изменить это и обеспечить такое положение вещей, когда информация будет точной, надежной, обновляемой по мере необходимости и доступной каждому, кому она нужна (независимо от его уровня в организации). Для достижения этих целей бизнес-аналитике нужны технологии, в состав которых часто входит и хранилище данных. Однако увлечение технологиями (особенно пользовательскими интерфейсами) становится чрезмерным. Не меньше внимания следует уделять и принятию решений на основе фактической информации, которым занимается высшее руководство предприятия, и которое непосредственно влияет на то, как будет работать само предприятие.

Поначалу бизнес-аналитика встречала на предприятиях серьезное сопротивление. Поскольку знание — это власть, то утрата контроля за знаниями напоминает потерю власти (и часто так оно и есть). Бизнес-аналитика была в этом плане определенной угрозой, поскольку она делала информацию доступной гораздо более широкому контингенту. К счастью, в настоящее время ценность бизнес-аналитики признана на большинстве предприятий.

И наконец, мы должны упомянуть, что исторически многие проекты бизнес-аналитики (и поддерживавшие их реализации хранилищ данных) не достигали заявленных ими целей, что создало бизнес-аналитике плохую репутацию дорогой и рискованной затеи. В результате пересматривается необходимость создания хранилищ данных (для поддержки бизнес-аналитики), и вместо них используют отчеты и другие уже существующие источники данных. Несмотря на то, что такой подход имеет определенную привлекательность, только время покажет, станет ли он заметным явлением в бизнес-аналитике.

## **Панели и системы показателей**

Термины "панель" и "система показателей" часто используются как синонимы. Оба они представляют информацию в графическом виде, суммируя ее в виде различных элементов, отображающих относительные значения, тенденции и прочие существенные связи. Но это не синонимы.

### **Панели**

Панель (подобно своей автомобильной тезке) отображает некие меры (без контекста). Она ориентирована на тактическое использование по принципу "только факты" и обновляется так часто, как это требуется для поддерживаемого ею операционного процесса. Она имеет более общий характер, чем сис-

тема показателей, потому что может отображать все, что угодно (в том числе и систему показателей). На рис. 14.7 показана типичная панель.

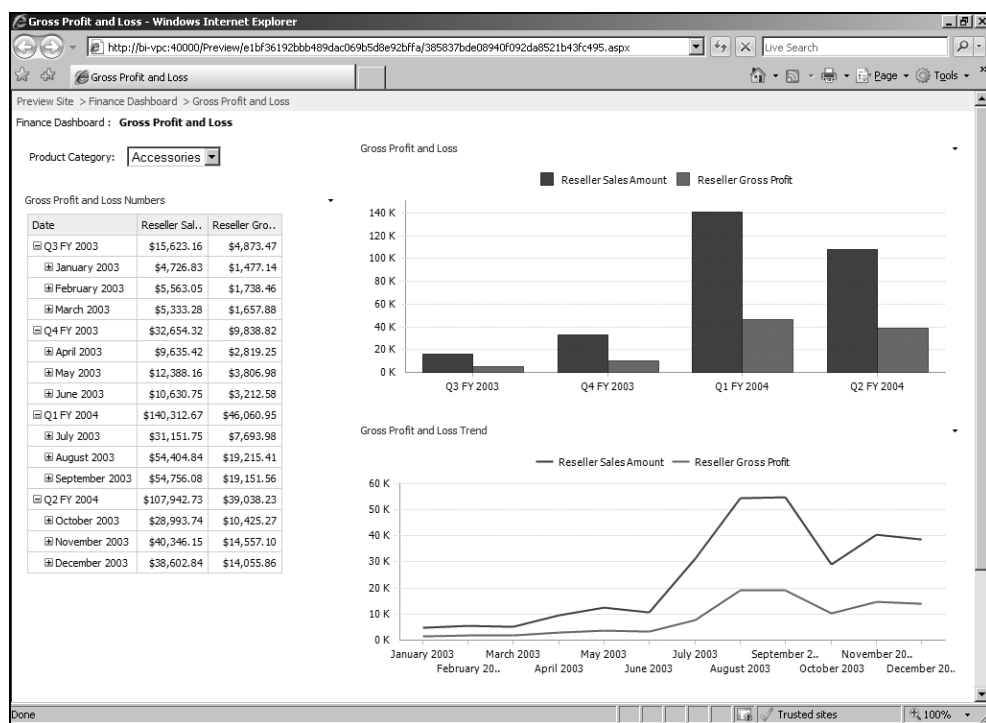


Рис. 14.7. Типичная панель

## Система показателей

Система показателей отображает некие базовые меры в контексте соответствующих целей (или целевых измерений) и дает мгновенное представление о том, соответствует ли данная базовая мера своему целевому значению. Поэтому очевидно, что система показателей не может существовать при отсутствии таких целевых значений. Отсюда следует, что система показателей является стратегическим инструментом, а панель — тактическим и операционным.

Термин "ключевой индикатор производительности" (key performance indicator, KPI) тесно связан с системами показателей. Индикатор типа "светофор" и индикатор тенденции на рис. 14.8 — это индикаторы KPI. Индикатор KPI инкапсулирует меру, соответствующее целевое значение меры, вы-

числение отношения этих двух величин, а также графику, которая отображает "плохое" или "хорошее" состояние индикатора (в зависимости от результата вычисления).

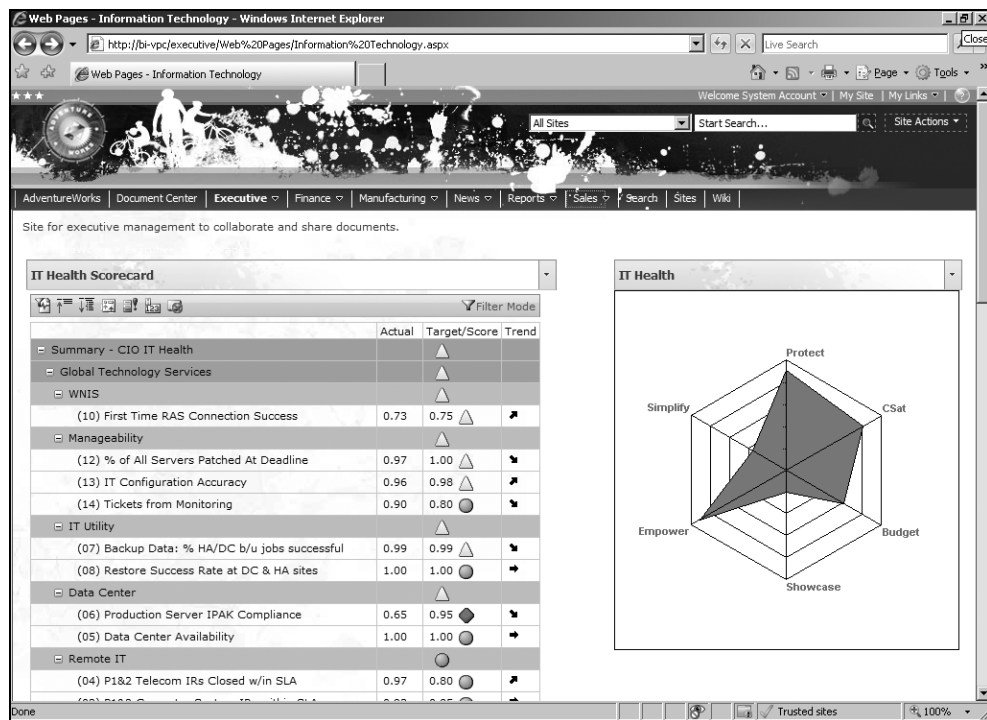


Рис. 14.8. Типичная система показателей

Целевые значения обычно определяются не на нижних уровнях детализации. Рассмотрим разницу в гранулах между мерами Actual и Plan: первая выводится из транзакций, а вторая создается на более высоком уровне суммирования (по крайней мере, по измерению Time). По этой причине системы показателей имеют тенденцию давать информацию на более высоком уровне суммирования, чем панели (что соответствует стратегической ориентации одних и тактической ориентации других). Это в свою очередь означает также, что изменения в них происходят медленнее (поэтому системы показателей обычно обновляются реже, чем панели). В финансовой системе показателей (подобной показанной на рис. 14.8) ключевой индикатор Actual/Plan выполняет все эти принципы и отображается как светофор в столбце Plan. Обратите внимание на индикатор Trend, который также является индикато-



ром KPI и вычисляет соотношение со значениями Actual и Plan предыдущего периода.

С момента появления SQL Server 2005 аналитические службы Analysis Services предоставляют объекты KPI, которые можно сохранять в кубах. Они могут использоваться и отображаться в Microsoft Office Excel 2007, Microsoft Office SharePoint Server и Microsoft Performance Point.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Расширенное описание OLAP (которое включает индикаторы KPI) смотрите в *главе 16*.

## **Управление производительностью**

Управление производительностью — это относительно новый термин, который обозначает специфический вид бизнес-аналитики, но заслуживает отдельного обсуждения (из-за своего присутствия на рынке и в литературе в виде отдельного понятия). Управление производительностью подразумевает использование бизнес-аналитики, однако обратное неверно, поскольку бизнес-аналитика является термином более общего характера. Как уже упоминалось ранее, методики бизнес-аналитики могут быть сосредоточены на многих разных направлениях. Управление производительностью — это конкретная область использования бизнес-аналитики. Сначала здесь устанавливаются организационные цели и способы измерения степени продвижения к ним, при этом часто используются методики бизнес-аналитики (для того чтобы определить цели и показатели). После определения целей собираются данные по производительности в прошлом и настоящем, а также по проектной производительности, которые сравниваются с установленными целями, после чего информация о степени соответствия целевым значениям широко распространяется. Все это обычно делается в виде системы показателей, которая опять-таки использует инструменты и методики бизнес-аналитики.

Система сбалансированных показателей (Balanced Scorecard, BSC) является широко известным примером управления производительностью, она предшественник самого этого термина. Вам следует познакомиться с этим подходом, по крайней мере, потому, что он поможет вам лучше понять факторы, влияющие на стратегию предприятия и на то, как эта стратегия вводится в действие.

## **Практические советы по хранилищам данных**

Для создания хранилища данных необходима и теория, и практические знания. Несмотря на то, что теорию построения хранилищ данных можно считать хорошо изученной, практический опыт по-прежнему много значит. В этом разделе мы рассмотрим некоторые лучшие практики построения хранилищ данных, которые мы считаем наиболее ценными.

### **Предвидим и поощряем изменение операционных процессов**

Почти наверняка проект хранилища данных будет выявлять те элементы данных и связи, которые являются существенными для реализации целей предприятия, но на данный момент в операционных процессах не участвуют. Очень вероятно также и то, что те люди, ежедневная работа которых будет затронута при решении этих проблем, будут считать, что от этого они ничего не приобретают (а иногда даже что-то теряют). Например, целью предприятия может быть определение того, какие группы продавцов должны получить премию (и в какой пропорции) за совместную работу по конкретной продаже, чтобы улучшить распределение премиального фонда продавцов. Для этого необходимо собирать информацию о том, какие группы продавцов были задействованы во время записи транзакции о продаже. Эта информация, скорее всего, не записывается при текущем порядке работы сотрудников операционного отдела (которые регистрируют данную транзакцию), и более того: если запись этой информации будет введена в обязательном порядке, то затрачиваемое на нее время будет уменьшать количество транзакций, которые эти сотрудники могут обработать за час. Скорее всего, они будут сопротивляться данному нововведению (поскольку оно снижает производительность их труда). Поэтому такие новшества следует вводить официально и предусматривать соответствующие стимулы для мотивации сотрудников.

### **Поощряем отказ от управления**

Как уже говорилось ранее, для успешного создания хранилища данных и приложения бизнес-аналитики часто требуется отказ от управления данными тех сотрудников, которые традиционно заведовали ключевыми данными. Ор-

ганизационные перемены являются угрозой для тех, кто вследствие них что-то теряет, и вполне естественно, что такие люди будут сопротивляться этим изменениям. Если предприятие понимает это и мотивирует людей к переменам, то шансы на успех возрастают. Насколько все это выполнимо, зависит в значительной степени от организационной культуры.

## Прототип может не помочь при "продаже идеи"

Создание прототипа или постановка эксперимента (proof of concept, POC) для проекта "хранилище данных/приложение бизнес-аналитики" часто рекомендуется, как способ получить одобрение собственников предприятия. Казалось бы, репрезентативный эксперимент мог бы помочь добиться этой цели. Под словом "репрезентативный" мы подразумеваем, что демонстрируется осуществимость реализации важных технических возможностей (например, возможность моделирования определенных связей), даже если это иллюстрируется такими вымышленными данными, как база данных AdventureWorksDW2008.

Возможно, вы слишком поздно поймете, что собственникам может оказаться очень трудно оценить такой подход, особенно когда эксперимент не основан на таких показателях, которые им знакомы, или если используемые значения нереальны. В эксперименте точность данных не важна, но если ваши собственники потеряют интерес, то его может быть очень тяжело вернуть. Акцент на таких вещах — любимая тактика тех, кто сопротивляется введению хранилища данных (по разным причинам).

Для того чтобы эксперимент имел наилучшие шансы на успех, он должен быть как можно более реалистичным и максимально приближенным к условиям работы собственников. Это часто противоречит идее о том, что эксперимент требует минимальных вложений. Проект внедрения хранилища данных может быть буквально "сбит на взлете" плохо продуманным экспериментом.

## Проблемы суррогатных ключей

Преимущества использования (в хранилище данных) целочисленных суррогатных ключей обсуждались ранее в этой же главе. Но при их использовании существуют и проблемы:

- В общем случае суррогатные ключи не должны быть "интеллектуальными", то есть они не должны иметь никакого закодированного в их значе-

ниях существенного смысла. Однако возможно следует сделать исключение для измерения Time. На физическом уровне в суррогатном ключе Time может находиться значение в виде YYYYMMDD, YYYYMM или YYYYWW (где Y, M, D и W — это год, месяц, день и неделя), причем любая такая форма может быть легко представлена в виде целого числа. Такое нарушение лучших практик оправдывается двумя причинами. Во-первых, если столбец суррогатного ключа Time является первым в составном первичном ключе таблицы фактов (а именно так оно и должно быть), и первичный ключ имеет кластеризованный индекс, то данные фактов будут оптимальным образом организованы для ограничения Time типичного запроса (такое ограничение обычно является моментом или диапазоном времени). Во-вторых, такой интеллектуальный ключ Time существенно облегчит реализацию и обслуживание физического разделения измерения Time на уровне реляционной базы данных.

- Суррогатные ключи могут генерироваться несколькими способами, причем основными являются два метода: первый — это столбец IDENTITY, а второй — построчное присваивание (например, `SELECT MAX(Id) + 1`) с использованием соответствующих механизмов блокировки. Независимо от используемого метода в обычной множественной среде (разработка, контроль качества, производство) могут возникнуть осложнения. Предположим, что в начале цикла разработки ваша среда разработки обновляется из производственной среды. Затем вы через систему ETL копируете входные файлы из производственной среды и запускаете процесс ETL в среде разработки (возможно, в виде части сравнительного испытания). В зависимости от метода присваивания суррогатных ключей существует определенная вероятность того, что одним и тем же данным (с точки зрения бизнес-ключа) присваиваются разные суррогатные ключи (в среде разработки и в производственной среде). Это может существенно осложнить согласование обеих сред.

## Проблемы конвертирования валют

На больших многонациональных предприятиях финансовым приложениям обычно требуется конвертирование валют (для сравнения аналогичных элементов). Учтите, что этот вопрос осложнен бизнес-правилами и проектными головоломками. Начиная с SQL Server 2005, аналитические службы Analysis Services предоставляют такие функциональные возможности, которые могут облегчить реализацию преобразования валют в кубах.

Однако это не решает противоречия между проектированием под произвольные (заранее неизвестные) запросы и необходимостью знать что-то (а возможно, и достаточно много) о похожих запросах (если нужно создать соответствующий им проект). Проблемы конвертирования валют очень хорошо все это иллюстрируют. На следующие вопросы не существует правильных ответов, но если вам предстоит делать преобразования валют, то вам было бы хорошо рассмотреть их все.

- Какая требуется гибкость? Будет ли существовать некая главная валюта (в которой будут выражаться все сравнения), или будет несколько стандартных валют, можно ли будет использовать любую существующую валюту?
- Имеет ли смысл вычислять и сохранять конвертированные значения, либо это должно делаться "на ходу"? Точно так же, как и с любыми тарифами и соотношениями, следует быть осторожными там, где делается агрегирование, чтобы конвертирование валют выполнялось на соответствующем листовом уровне детализации, и только после этого делалось агрегирование до необходимого сводного уровня. На это существенно влияют возможности вашего инструмента OLAP.
- Все суммы должны конвертироваться только по курсу на момент конвертации, или они должны иметь возможность конвертироваться по курсу в любой момент времени?
- По какому курсу должны конвертироваться будущие значения (например, бюджета): по курсу на момент завершения разработки бюджета (и потом никогда больше не корректироваться)? Или следует использовать текущий курс (с корректировкой проектных цифр в каждом периоде)? Должны ли вы иметь возможность определить, какая доля разницы между Actual и Budget приходится на конвертирование валют, а какая — на изменение самого показателя Budget?

Проектирование (с учетом ответов на эти бизнес-вопросы) определяет техническую сложность реализации.

## События и моментальные снимки

Существует два взаимодополняющих подхода к логическому проектированию хранилищ данных: управляемый событиями и моментальных снимков. Оба приводят к компромиссам между сложностью подхода и типом поддерживаемых им запросов.

С одной стороны, можно утверждать, что все (представляющее интерес с точки зрения аналитики) можно представить как *событие*. События — это элементы вроде получаемых платежей (заказов) или доставляемых партий товаров. События по определению происходят в разные моменты времени асинхронно. В принципе, если все соответствующие события удастся идентифицировать и зарегистрировать, можно вывести состояние их на любой момент времени. Для некоторых информационных приложений это критично. Однако такое конструирование (из событий) определенного момента времени может быть чрезвычайно сложным.

С другой стороны, подход на основе моментальных снимков совсем не регистрирует событий. Вместо этого он просто периодически записывает итоговый результат событий. Отвечать на запросы по тем моментам времени, в которых были сделаны моментальные снимки, гораздо проще, чем при подходе на базе событий (когда состояние на данный момент времени необходимо реконструировать).

Эти подходы иногда необходимо комбинировать. Например, для элемента Account часто интересен только баланс счета на определенные моменты времени (такие как конец месяца). С другой стороны, обязательно также иметь возможность запросить каждое событие (приход или расход), которое повлияло на баланс с момента предыдущего моментального снимка.

События и моментальные снимки имеют и дополнительные нюансы (кроме поддерживаемых ими функциональных аспектов). Подход с моментальными снимками буквально всего (независимо от количества изменений с момента последнего моментального снимка) может привести к "разбуханию" данных и быть неэффективным (по сравнению с основанным на событиях подходом) тогда, когда изменений относительно немного.

Вполне оправданно потратить время на стадии проектирования на то, чтобы обдумать последствия использования обоих подходов и определить наилучший для ваших требований вариант.

## SQL Server 2008 и хранилища данных

Ранние версии SQL Server имели новые (связанные с хранилищами данных) функциональные возможности, самыми примечательными из которых были аналитические службы Analysis Services и службы отчетности Reporting Services, а в SQL Server 2005 это были некоторые возможности служб интеграции Integration Services (такие как задача Slowly Changing Dimensions). Но все

эти ранние версии не имели практически ничего (на уровне ядра реляционной базы данных) такого, что было бы специально предназначено для потребностей хранилищ данных. В SQL Server 2008 имеются новые функциональные возможности, которые нацелены непосредственно на хранилища данных, особенно в плане повышения управляемости и экономической эффективности очень больших баз данных. В этом разделе будет дан обзор тех наиболее важных улучшений SQL Server 2008, которые ориентированы на хранилища данных, начиная с улучшений языка Transact-SQL (T-SQL), нацеленных на работу с хранилищами данных.

## Инструкция **MERGE** языка T-SQL

Инструкция **MERGE** более подробно описывается в *главе 2*. Она применима не только к хранилищам данных. Здесь мы также описываем ее, поскольку она имеет непосредственное отношение к хранилищам данных (особенно в контексте процессов ETL).

Для **MERGE** требуется таблица назначения, которая соединена некоторой связью с исходной таблицей. Исходная таблица содержит данные, которые необходимо объединить (или синхронизировать) с таблицей назначения. Инструкция **MERGE** поддерживает до трех типов выражений, определяющих построчное действие, которое необходимо произвести с таблицей назначения (в зависимости от результата сравнения данной строки с исходной таблицей):

- ☐ **WHEN MATCHED** — строка существует как в объединяемой таблице, так и в таблице назначения (выполняется внутреннее соединение, и разрешаются операции **UPDATE** или **DELETE**);
- ☐ **WHEN NOT MATCHED BY TARGET** — строка существует в исходной таблице, но ее нет в таблице назначения (выполняется левое внешнее соединение, и разрешается операция **INSERT**);
- ☐ **WHEN NOT MATCHED BY SOURCE** — строка существует в таблице назначения, но отсутствует в исходной таблице (выполняется правое внешнее соединение и разрешаются операции **UPDATE** или **DELETE**).

### **Дополнительная информация**

Каждое выражение **MERGE** может также в дополнение к неявному соединению устанавливать ограничения (такие как сравнение значений столбцов источника и назначения). Однако существуют весьма конкретные правила,

управляющие использованием нескольких выражений `MERGE` и их различными комбинациями. Мы излагаем их в подробном описании новой инструкции `MERGE` в главе 2.

В контексте хранилищ данных инструкция `MERGE` особенно хорошо приспособлена к обслуживанию таблиц измерений схем звезды. Она очень полезна при обслуживании медленно меняющихся измерений (*slowly changing dimensions*, `SCDs`) первого типа, когда изменения просто накладываются на существующие значения, а также медленно меняющихся измерений второго типа, когда `MERGE` может сделать только часть работы (при обновлении существующей строки все-таки нужна отдельная вставка `INSERT` для создания новой версии). Полное описание `SCD` выходит за рамки этой главы.

В службах интеграции `SQL Server 2008 Integration Services` инструкция `MERGE` может упростить шаблон вставки или обновления. Раньше решение вставить или обновить принималось на основе поиска исходной строки при помощи задачи `Lookup` (в которую загружались строки назначения, имелось два выходных потока данных, один из них делал вставки, а другой — обновления таблицы назначения). При наличии `MERGE` задача `Lookup` больше не нужна, что упрощает пакет служб интеграции `Integration Services` и позволяет избежать проблем с производительностью, памятью и взаимоблокировками (которые могут возникнуть при использовании задачи `Lookup` в том случае, когда таблица назначения большая).

Синтаксис `MERGE` требует наличия двух соединяемых таблиц или эквивалентов таблиц. Назначение должно быть либо таблицей, либо обновляемым представлением, источник может быть любым эквивалентом таблицы. Для служб интеграции это означает, что таблица-источник должна уже существовать или быть создана внутри пакета (как временная таблица, обобщенное табличное выражение `CTE` или другой эквивалент).

Код в листинге 14.2 демонстрирует последовательность выражений T-SQL с использованием `MERGE` по базе данных `AdventureWorksDW2008`. Выполните каждую инструкцию вручную (в соответствии с указаниями в комментариях), после чего выполните инструкцию `MERGE` в конце листинга. Обратите внимание, что `GeographyKey` является столбцом идентификаторов в `DimGeography`, поэтому в инструкции `INSERT` (выражения `WHEN NOT MATCHED BY TARGET` инструкции `MERGE`) список столбцов должен быть явным. Обратите также внимание, что закрывающая точка с запятой необходима для завершения инструкции `MERGE`.



## ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Все инструкции языка манипулирования данными (data manipulation language, DML) в T-SQL (INSERT, UPDATE, DELETE и MERGE) поддерживают выражение OUTPUT, которое может быть очень полезным для архивирования измененных данных. Кроме того, новое средство INSERT OVER DML в SQL Server 2008 улучшает выражение OUTPUT возможностями фильтрации. Подробности относительно выражения OUTPUT и INSERT OVER DML см. в главе 2.

### Листинг 14.2. Использование команды MERGE для обновления хранилища данных

```
USE AdventureWorksDW2008
GO

-- Сделаем копию таблицы.
SELECT * INTO DimGeographyTest FROM DimGeography
-- Создадим таблицу "Changes" как еще одну копию тех же данных.
SELECT * INTO Changes FROM DimGeography
-- Если вы теперь выполните инструкцию MERGE, то не
-- получите никаких сообщений об изменениях. Обратите внимание на условие
-- в выражении UPDATE, которое предотвращает ненужные обновления.
-- Теперь принудительно сделаем UPDATE (53 штуки):
UPDATE Changes
    SET SalesTerritoryKey = 11
    WHERE SalesTerritoryKey = 10
-- Теперь выполнение MERGE сообщает о 53 обновлениях.
-- Теперь принудительно выполним DELETE (пустая таблица по существу
-- удаляет все строки в DimGeographyTest):
DELETE Changes
-- Теперь выполнение MERGE удалит все 653 строки в DimGeographyTest.
-- Тестирование INSERT мы оставим читателю в качестве упражнения.
-- Инструкция MERGE:
MERGE DimGeographyTest AS dg
    USING (SELECT * FROM Changes) AS c
    ON dg.GeographyKey = c.GeographyKey
    WHEN MATCHED and dg.SalesTerritoryKey <> c.SalesTerritoryKey THEN
        UPDATE SET dg.SalesTerritoryKey = c.SalesTerritoryKey
    WHEN NOT MATCHED BY TARGET THEN
        INSERT (City, StateProvinceCode, StateProvinceName,
```

```
CountryRegionCode, EnglishCountryRegionName,  
Spa*nishCountryRegionName, FrenchCountryRegionName,  
PostalCode, SalesTerritoryKey)  
VALUES (c.City, c.StateProvinceCode, c.StateProvinceName,  
c.CountryRegionCode, c.EnglishCountryRegionName,  
c.SpanishCountryRegionName, c.FrenchCountryRegionName,  
c.PostalCode, c.SalesTerritoryKey)  
WHEN NOT MATCHED BY SOURCE THEN  
DELETE  
OUTPUT $action, INSERTED.*, DELETED.*;
```

Использование способностей MERGE к удалению в хранилищах данных является редкостью (за исключением единичных корректировок ошибочных данных), но даже ради этого об этом стоит упомянуть. Вообще же остерегайтесь использовать DELETE совместно с MERGE. Если ваша исходная таблица случайно оказалась пустой (как в нашем примере), то MERGE с выражением WHEN NOT MATCHED BY SOURCE и с указанием DELETE может нечаянно удалить все строки в назначении (в зависимости от прочих условий в выражении WHEN NOT MATCHED BY SOURCE).

## Отслеживание изменений Change Data Capture

Аналогично инструкции MERGE, новая функциональная возможность отслеживания изменений Change Data Capture (CDC) в SQL Server 2008 предназначена для ETL хранилища данных. CDC имеется только в версии Enterprise сервера SQL Server 2008 (и, конечно же, в функционально эквивалентных ей версиях Developer и Evaluation).

### **ПРИМЕЧАНИЕ**

SQL Server 2008 предоставляет несколько функциональных возможностей отслеживания данных, причем каждая из них — для специальной цели. В частности, CDC служит для хранилищ данных, SQL Server Audit — для безопасности (см. главу 5), а SQL Server Change Tracking — для синхронизации периодически подключенных систем и мобильных устройств (при помощи ADO.NET Sync Services, см. главу 13).

CDC эффективно работает с захватом и записью соответствующих изменений контекста хранилища данных. Традиционно для обнаружения изменений в таблице операционного приложения (которые необходимо было применить

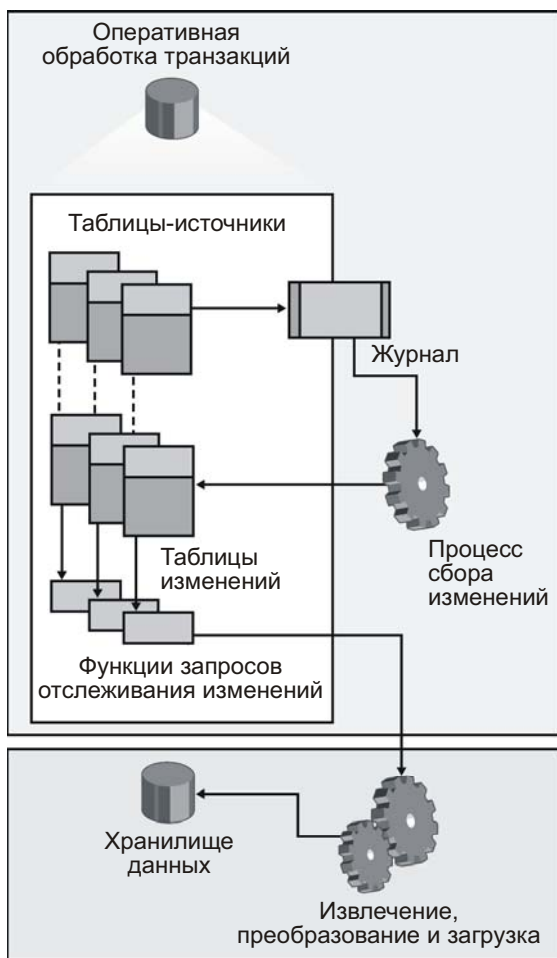
к хранилищу данных) требовались достаточно прямолинейные методы вроде следующих:

- ❑ для обновлений — использовалась либо функция `CHECKSUM` обнаружения неравенства столбцов исходных строк и строк назначения (только в SQL Server), либо сравнивались временные метки;
- ❑ для вставок — производилось внешнее соединение исходных строк и строк назначения и тестирование на значение `NULL` в строках назначения;
- ❑ для вставок и обновлений — реализовывались триггеры в исходной таблице (для выявления изменений и соответствующих действий с таблицей назначения), либо выполнялся поиск (возможно, при помощи задачи `Lookup` служб `Integration Services`) для сравнения источника с назначением и последующего выполнения обновления или вставки (в соответствии с успехом или неудачей поиска);
- ❑ для вставок и обновлений — использовалось выражение `OUTPUT` (в SQL Server 2005 и 2008) или `INSERT OVER DML` SQL Server 2008, которые мы описываем в главе 2.

Введенная в SQL Server 2008 функциональная возможность CDC обеспечивает новый ценный способ вкладки основы для обслуживания меняющихся данных (в хранилище данных). Без обращения к триггерам или другому специальному коду она позволяет записать происходящие в таблице изменения в отдельную таблицу `SQL Server Change Tracking` (таблицу изменений). К этой таблице потом может делать запросы процесс ETL для инкрементального обновления хранилища данных по мере необходимости. Запрос к таблице изменений (вместо самой отслеживаемой таблицы) означает, что процесс ETL никоим образом не влияет на производительность приложений, которые работают с транзакционными таблицами вашей базы данных. CDC управляется заданием `SQL Server Agent` отслеживания журнала транзакций `SQL Server`. Это обеспечивает гораздо более высокую производительность, чем использование триггеров, особенно в характерных для хранилищ данных сценариях массовой загрузки (причем для использования CDC не нужно писать никакого кода). Платой за CDC являются несколько более высокие задержки, которые для хранилищ данных обычно вполне допустимы. На рис. 14.9 показано общее представление архитектуры CDC (иллюстрация взята из документации `SQL Server Books Online`).

Несколько новых системных хранимых процедур и возвращающих табличное значение функций (`table-valued functions`, TVF) предоставлены для включения, отслеживания и использования выдаваемых функцией `SQL Server Change Tracking` данных. Для начала вы выполняете процедуру `sp_cdc_enable_db`

(чтобы включить CDC для текущей базы данных, для этого вы должны иметь роль `sysadmin`). Когда вы выполняете эту процедуру, создаются новые: пользователь `cdc`, схема `cdc` и роль `CDC_admin`. Эти имена жестко фиксированы, так что если у вас уже есть пользователь или схема с таким названием, вам придется переименовать их до начала использования CDC.



**Рис. 14.9.** Архитектура CDC

После включения CDC для базы данных вы включаете CDC и для данной таблицы (путем выполнения `sp_cdc_enable_table`, для этого вы должны иметь роль `db_owner`). В схеме `cdc` создается несколько объектов: таблица

изменений и, по крайней мере, одна (их может быть и две) функция TVF. Давайте рассмотрим эти объекты по очереди.

Когда CDC включается для таблицы, то SQL Server создает в схеме `cdc` таблицу изменений. Таблица изменений будет автоматически заполняться данными изменений. Имя ей присваивается по названию схемы и названию отслеживаемой таблицы. Например, когда вы включаете CDC для таблицы `Employee` в схеме `dbo` (как мы скоро сделаем), то SQL Server создает соответствующую таблицу изменений с названием `cdc.dbo_Employee_CT`, которая будет записывать все изменения таблицы `dbo.Employee`. Схема отслеживаемой таблицы (в данном случае это `dbo`) является частью названия таблицы изменений, так что все таблицы с одинаковыми названиями из различных схем могут однозначно отслеживаться в схеме `cdc`. Можно также явно дать название таблице изменений (при условии, что оно будет уникальным в пределах базы данных).

Процесс ETL будет запрашивать из этой таблицы данные по изменениям (для заполнения вашего хранилища данных), но обычно он не делает для этого выборки непосредственно из таблицы. Вместо этого процесс ETL вызовет специальную функцию TVF запроса таблицы изменений. TVF автоматически генерируется для вас сервером SQL Server при создании таблицы изменений и (подобно таблице изменений) также создается в схеме `cdc` с названием, составленным из названий схемы и отслеживаемой таблицы. То есть если мы отслеживаем таблицу `dbo.Employee`, то SQL Server создает TVF с названием `cdc.fn_cdc_get_all_changes_dbo_Employee`, которая принимает параметры для выборки всех изменений, произошедших в `dbo.Employee` между любыми двумя моментами времени.

Если при вызове `sp_cdc_enable_table` вы укажете `@supports_net_changes=1`, то для таблицы изменений будет создана и вторая функция TVF. Подобно первой, она позволит сделать выборку изменений между любыми двумя моментами времени, однако возвращает только окончательные (*net*) изменения, которые произошли в данный период времени. Например, это означает, что если строка была добавлена, а затем удалена в запрашиваемый при помощи второй функции TVF период времени, то данные по этой строке возвращены не будут, в то время как первая функция TVF вернет данные, отражающие как вставку, так и удаление строки. Эта вторая функция TVF называется аналогичным образом, только вместо слова `all` используется слово `net`. Для `dbo.Employee` эта функция TVF называется `cdc.fn_cdc_get_net_changes_dbo_Employee`. Обратите внимание, что запрос окончательных изменений требует наличия у отслеживаемой таблицы первичного ключа или уникального индекса.

TVF не принимают непосредственно времени начала и окончания, а требуют указать диапазон в виде регистрационных номеров транзакций в журнале (log sequence numbers, LSN), а для этого сначала необходимо вызвать функцию `sys.fn_cdc_map_time_to_lsn`. Поэтому для запроса промежутка между двумя моментами времени вы вызываете функцию `sys.fn_cdc_map_time_to_lsn` дважды: один раз для начального момента времени, а другой — для момента окончания; затем вы используете возвращаемые этой функцией значения LSN как входные значения для TVF (чтобы запросить данные по изменениям). Это может показаться слишком громоздким, однако на самом деле это достаточно обоснованно. Так сделано для поддержки двух таблиц изменений (для одной и той же отслеживаемой таблицы), одна из которых работает на производственные системы, а другая предназначена для поддержки продолжающихся разработок программного обеспечения.

### **СОВЕТ**

Время начала и окончания (с которым вызывается данная функция) не обязательно должно попадать в диапазон представленного в журнале времени. Если то или другое время выходит за пределы представленного в журнале промежутка времени, то функция все делает правильно: она возвращает самый ранний существующий LSN (если указанное время начала раньше, чем самый ранний LSN) и самый поздний LSN (если указанное время окончания позже, чем самый последний LSN). Скоро вы увидите это в листинге 14.3.

Хранимая процедура `sp_cdc_enable_table` имеет несколько необязательных параметров, которые дают вам большую гибкость. Вы можете (среди прочего) указать для таблицы изменений свое собственное название; роль, которой должен обладать пользователь для запросов по изменениям (если он не является `sysadmin` или `db_owner`); какие столбцы таблицы следует отслеживать (не обязательно отслеживать все столбцы); файловую группу, в которой будет создаваться таблицы изменений; а также возможность применения опции `SWITCH_PARTITION` инструкции `ALTER TABLE` к отслеживаемой таблице (что может иметь очень важные последствия). Подробное описание параметров `sp_cdc_enable_table` смотрите в документации SQL Server Books Online.

Если вам больше не нужна CDC для данной конкретной таблицы, вы можете вызвать для нее хранимую процедуру `sp_cdc_disable_table`. Эта процедура удаляет таблицу изменений и TVF, а также обновляет системные метаданные (чтобы указать, что таблица более не отслеживается). Вызовите хранимую процедуру `sp_cdc_disable_db` для того, чтобы полностью отключить CDC для всей базы данных.

**ВАЖНО**

Перед тем как удалять базу данных с CDC, вам следует учесть некоторые обстоятельства. Для удаления такой базы данных вы должны либо остановить SQL Server Agent, либо сначала выключить CDC при помощи запуска `sp_cdc_disable_db` для той базы данных, которую вы хотите удалить. Если вы выберете первый вариант, то все задания SQL Server Agent будут автоматически удалены при следующем его запуске (поскольку будет обнаружено, что отсутствует та база данных, с которой были связаны данные задания). Конечно, при остановленном агенте SQL Server Agent не будет работать и SQL Server Change Tracking для всех баз данных этого экземпляра SQL Server. Предпочтительным вариантом является второй, поскольку он не влияет на другие базы данных с CDC и очищает все связанные с удаляемой базой данных остатки от CDC.

Таблица изменений записывает все изменения в указанных столбцах между двумя моментами времени, включая промежуточные состояния (по каждой инструкции DML). Обратите внимание, что CDC поддерживает *разреженные столбцы*, но не наборы разреженных столбцов. Каждая строка таблицы изменений включает также пять столбцов метаданных. По ним можно определить, какой тип изменения представляет собой каждая строка (вставка, обновление или удаление), а также сгруппировать и упорядочить все изменения, принадлежащие к одной и той же транзакции. Однако нельзя определить, кто именно произвел данное изменение, для обеспечения аудиторских следов. (Мы описываем SQL Server Audit в *главе 5*).

Как мы уже упоминали ранее, для автоматизации процесса регистрации CDC использует SQL Server Agent. При первом выполнении функции `sp_cdc_enable_table` для любой таблицы базы данных SQL Server создает также две задачи SQL Server Agent этой же базы данных. Первая — это задача сбора изменений, отслеживание журнала транзакций. Вторая — задача очистки, которая удаляет строки из таблиц изменений после истечения определенного настраиваемого интервала времени (по умолчанию это три дня) и очищает все остатки CDC при удалении отслеживаемой таблицы. Поэтому при запуске этой процедуры включения CDC для таблицы любой базы данных экземпляра SQL Server агент SQL Server Agent должен быть запущен. Если впоследствии агент SQL Server Agent будет остановлен, то изменения отслеживаемых таблиц будут накапливаться в журнале транзакций, но не будут вноситься в таблицы изменений (до тех пор, пока SQL Server Agent не будет вновь запущен).

Вначале функция CDC может показаться очень громоздкой в использовании, но она очень хорошо продумана в смысле гибкости конфигурирования и

поддержки самых различных сценариев. Код листинга 14.3 показывает полный пример использования CDC.

### Листинг 14.3. Использование CDC

```
-- Создаем тестовую базу данных
CREATE DATABASE CDCDemo
GO
USE CDCDemo
GO

-- Включаем CDC для базы данных
EXEC sp_cdc_enable_db

-- показать базы данных со включенной CDC
SELECT name, is_cdc_enabled FROM sys.databases

-- Просмотреть нового пользователя и схему "cdc"
SELECT * FROM sys.schemas WHERE name = 'cdc'
SELECT * FROM sys.database_principals WHERE name = 'cdc'

-- Создаем таблицу Employee
CREATE TABLE Employee(
    EmployeeId int NOT NULL PRIMARY KEY,
    EmployeeName varchar(100) NOT NULL,
    EmailAddress varchar(200) NOT NULL)

-- Включаем CDC для таблицы (SQL Server Agent должен быть запущен)
EXEC sp_cdc_enable_table
    @source_schema = N'dbo',
    @source_name = N'Employee',
    @role_name = N'CDC_admin',
    @capture_instance = N'dbo_Employee',
    @supports_net_changes = 1

-- показать таблицы со включенной CDC
SELECT name, is_tracked_by_cdc FROM sys.tables

-- Вставить несколько сотрудников...
INSERT INTO Employee VALUES(1, 'John Smith', 'john.smith@ourcorp.com')
INSERT INTO Employee VALUES(2, 'Dan Park', 'dan.park@ourcorp.com')
INSERT INTO Employee VALUES(3, 'Jay Hamlin', 'jay.hamlin@ourcorp.com')
INSERT INTO Employee VALUES(4, 'Jeff Hay', 'jeff.hay@ourcorp.com')

-- Выбрать их из таблицы и из таблицы изменений
SELECT * FROM Employee
SELECT * FROM cdc.dbo_employee_ct
```



```
-- Удалить Jeff
DELETE Employee WHERE EmployeeId = 4
-- Результаты после Delete
SELECT * FROM Employee
SELECT * FROM cdc.dbo_employee_ct
-- (Примечание: для появления результата DELETE в таблице CT
-- может потребоваться несколько секунд)
-- Обновить Dan и Jay
UPDATE Employee SET EmployeeName = 'Dan P. Park' WHERE EmployeeId = 2
UPDATE Employee SET EmployeeName = 'Jay K. Hamlin' WHERE EmployeeId = 3
-- Результаты после Update
SELECT * FROM Employee
SELECT * FROM cdc.dbo_employee_ct -- See note above
-- Дадим заданию CDC возможность инициализироваться
-- (перед доступом к TVF)
WAITFOR DELAY '00:00:20'
-- Для доступа к данным изменений используйте TVF,
-- а не сами таблицы изменений
DECLARE @begin_time datetime
DECLARE @end_time datetime
DECLARE @from_lsn binary(10)
DECLARE @to_lsn binary(10)
SET @begin_time = GETDATE() - 1
SET @end_time = GETDATE()
-- Установить соответствие между интервалом времени и диапазоном LSN
SELECT @from_lsn =
    sys.fn_cdc_map_time_to_lsn('smallest greater than or equal',
                               @begin_time)
SELECT @to_lsn =
    sys.fn_cdc_map_time_to_lsn('largest less than or equal', @end_time)
SELECT @begin_time AS BeginTime, @end_time AS EndTime
SELECT @from_lsn AS FromLSN, @to_lsn AS ToLSN
-- Вернуть изменения, произошедшие в период запроса.
-- Сначала все произошедшие изменения...
SELECT *
    FROM cdc.fn_cdc_get_all_changes_dbo_employee(@from_lsn, @to_lsn, N'all')
-- Затем итоговые изменения, то есть окончательное состояние...
SELECT *
    FROM cdc.fn_cdc_get_net_changes_dbo_employee(@from_lsn, @to_lsn, N'all')
```

Давайте внимательно изучим этот код. После создания базы данных CDCDemo мы включаем для этой базы данных CDC (путем вызова `EXEC sp_cdc_enable_db`). Следующие несколько запросов `SELECT` демонстрируют получение различной связанной с CDC информации. Первый запрос `SELECT` показывает, как столбец `is_cdc_enabled` в `sys.databases` возвращает `true` (1) или `false` (0), что облегчает определение тех баз данных, для которых функция CDC включена. Следующие два запроса `SELECT` показывают, как новая схема и пользователь `cdc` могут быть найдены в `sys.schemas` и `sys.database_principals`.

Затем код создает таблицу `Employee`, в которой (для простоты нашего примера) всего три столбца. Затем для таблицы `Employee` включается CDC (путем вызова `EXEC sp_cdc_enable_table` и передачи параметров, которые идентифицируют таблицу `Employee` в схеме `dbo` для отслеживания изменений). Помните, что в этот момент SQL Server Agent должен быть запущен. Следующая инструкция `SELECT` показывает, как запрашивать столбец `is_tracked_by_cdc` в `sys.tables` для определения тех таблиц, для которых функция CDC включена.

Вспомним, что включение CDC для таблицы `Employee` создает TVF для извлечения всех изменений, произведенных в таблице между любыми двумя моментами времени. Вспомним также, что при указании `@supports_net_changes = 1` создается также вторая функция TVF для извлечения только окончательных изменений, сделанных между любыми двумя моментами времени. Разница между всеми изменениями и окончательными изменениями скоро станет совершенно понятна (когда мы вызовем обе эти функции TVF и сравним их результаты). Но сначала код выполняет операции `INSERT`, `UPDATE` и `DELETE` по таблице `Employee` (для имитации активности базы данных и запуска процесса сбора данных об изменениях). В листинге 14.3 этим операциям сопутствуют инструкции `SELECT`, которые запрашивают таблицу изменений `cdc.dbo_employee_ct`. Это делается исключительно для демонстрации того факта, что данные по происходящим в таблице `Employee` изменениям записываются в таблицу изменений. Однако в нормальном режиме не следует делать таких непосредственных запросов к таблицам изменений, вместо этого для извлечения информации из таблицы `Employee` следует использовать сгенерированные TVF (как это показано в оставшейся части кода).

Затем наш код выполняет оператор `WAITFOR` для того, чтобы сделать паузу в 20 секунд перед вызовом TVF (чтобы дать возможность инициализироваться задаче SQL Server Agent по отслеживанию изменений). Это однократная задержка; ее величина не представляет собой обычной задержки начала записи отслеживаемых при помощи CDC изменений (которая имеет порядок от 2 до

3 секунд). Без этой задержки (или если SQL Server Agent не запущен при вызове TVF) вы получите дезориентирующее сообщение об ошибке, которое совершенно не описывает реальную проблему.

Для вызова любой из сгенерированных функций TVF вам нужно предоставить диапазон значений, определяющий промежуток времени. Как уже объяснялось, этот диапазон выражается при помощи значений LSN, которые вы можете получить посредством вызова функции `sys.fn_cdc_map_time_to_lsn` и передачи в нее соответствующих моментов начала и конца периода времени. Сначала мы настроим временной диапазон для прошедших 24 часов, путем присваивания `GETDATE()-1` и `GETDATE()` соответственно переменным начала и конца периода времени. Затем мы вызываем `sys.fn_cdc_map_time_to_lsn` переменных начала и конца периода времени и получаем значения LSN, которые соответствуют последним 24 часам. (Обратите внимание, что начальный LSN автоматически подстраивается для учета того факта, что 24 часа назад никаких LSN еще не было; точно так же подстраивается и конечный LSN — поскольку их может еще и не быть). Затем мы выдаем две инструкции `SELECT` (чтобы посмотреть значения диапазона времени и LSN):

```
BeginTime                EndTime
-----
2008-07-08 23:42:55.567 2008-07-09 23:42:55.567
(1 row(s) affected)

FromLSN                  ToLSN
-----
0x00000001A0000001E0039 0x0000000200000000A50001
(1 row(s) affected)
```

Имея значения диапазона LSN, мы выдаем еще две инструкции `SELECT` (последние две инструкции листинга 14.3). Первая инструкция запрашивает данный диапазон по TVF для всех изменений, а вторая — по TVF для окончательных изменений. Сравнение результатов этих двух запросов четко демонстрирует разницу между этими TVF:

```
__$start_lsn__$seqval__$operation__$update_mask EmployeeId
EmployeeName EmailAddress

0x00000001E0000007C0013 0x00000001E0000007C0012 2 0x07 1 John
Smith john.smith@ourcorp.com
0x00000001E000000800003 0x00000001E000000800002 2 0x07 2 Dan
Park dan.park@ourcorp.com
0x00000001E000000810003 0x00000001E000000810002 2 0x07 3 Jay
```

```

Hamlin jay.hamlin@ourcorp.com
0x0000001E000000820003 0x0000001E000000820002 2 0x07 4 Jeff
Hay jeff.hay@ourcorp.com
0x0000001E000000850004 0x0000001E000000850002 1 0x07 4 Jeff
Hay jeff.hay@ourcorp.com
0x0000001E0000001AC0004 0x0000001E0000001AC0002 4 0x02 2 Dan P.
Park dan.park@ourcorp.com
0x0000001E0000001AE0004 0x0000001E0000001AE0002 4 0x02 3 Jay K.
Hamlin jay.hamlin@ourcorp.com
(7 row(s) affected)

__$start_lsn__$operation__$update_mask EmployeeId EmployeeName EmailAd-
dress

0x0000001E0000007C0013 2 NULL 1 John Smith john.smith@
ourcorp.com
0x0000001E0000001AC0004 2 NULL 2 Dan P. Park steven.
jones@ourcorp.com
0x0000001E0000001AE0004 2 NULL 3 Jay K. Hamlin jay.hamlin@
ourcorp.com
(3 row(s) affected)

```

Первый результирующий набор включает в себя всю информацию обо всех изменениях, сделанных в течение указанного диапазона LSN (включая и все промежуточные изменения). Таким образом, полученная из первой TVF информация показывает все стадии изменений (всего семь изменений). В нашем сценарии элемент John был вставлен и после этого не изменялся. Поэтому показана только его вставка (значение `__$operation` равно 2). Dan и Jay были вставлены (значение `__$operation` равно 2) и обновлены (значение `__$operation` равно 4), поэтому для каждого из них возвращены оба изменения. Jeff после вставки был удален (значение `__$operation` равно 1), поэтому для него возвращены оба изменения (вставка и удаление).

Второй результирующий набор включает только окончательные изменения, сделанные в течение указанного диапазона LSN. Поэтому для того же диапазона LSN из второй функции TVF мы получаем только три записи об изменениях, каждая из которых дает нам окончательные значения столбцов в указанном диапазоне LSN. John появляется только один раз (как и в предыдущем запросе), поскольку он был вставлен только один раз и после этого не модифицировался и не удалялся (в течение данного диапазона LSN). Несмотря на то, что Dan и Jay были вставлены и обновлены, каждый из них

появляется только по одному разу (со своими окончательными значениями для указанного диапазона LSN), а не по два раза (как в предыдущем запросе). И поскольку Jeff был вставлен и удален в течение указанного диапазоном LSN времени, то никаких данных по Jeff вторая функция TVF (по окончательным изменениям) не возвращает.

## Параллелизм секционированных таблиц

В SQL Server *секционированная таблица* — это таблица, которая при физическом хранении разделена горизонтально (по подмножествам строк) на несколько групп файлов (это остается прозрачным для запросов и языка DML). Делается это для повышения управляемости и изоляции потенциально конфликтующих попыток доступа. Например, разные секции одной таблицы могут иметь разные стратегии резервного копирования и сжатия, а также разные индексы, причем каждый может быть оптимизирован под способ использования данной секции. Учитывая большой объем большинства хранилищ данных, такая гибкость может оказаться просто бесценной.

Типичным ключом секции является *time* (хотя это совершенно не обязательно), поэтому именно он является горизонтальной линией секционирования. Секционирование по *time*, например, позволяет индексировать "старые данные" не так интенсивно, как текущие (доступ к которым производится чаще). Старые данные можно копировать и удалять без всякого влияния на одновременно выполняющиеся запросы по более новым данным. Секционирование является важным инструментом физической реализации, особенно при создании очень большого хранилища данных.

Еще одним преимуществом хорошо спроектированного секционирования является более высокая эффективность планов запросов. Те запросы, которые охватывают только одну секцию, должны просматривать меньше данных, причем индексы лучше оптимизированы (если эта секция относится к новым данным). Кроме того, когда SQL Server работает на многоядерной или многопроцессорной платформе и правильно сконфигурирован, то имеется множество рабочих потоков, при помощи которых можно добиться параллелизма при обработке запроса (назначив для него несколько потоков).

### ПРИМЕЧАНИЕ

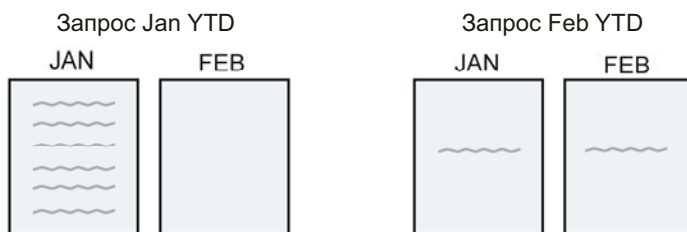
Для получения максимальных преимуществ от секционирования важно физически изолировать секции таблицы друг от друга. Практически это означает, что каждая группа файлов (каждой секции) должна находиться на от-

дельном физическом диске, а в критических случаях — даже на отдельном дисковом контроллере. Однако в данной книге механизмы секционирования таблиц описываться не будут, они хорошо описаны в документации SQL Server Books Online.

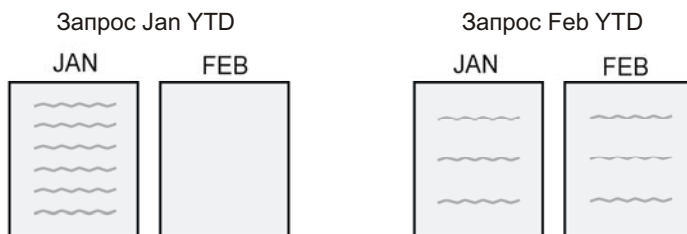
## Управление потоками

SQL Server 2005 оптимизировал параллельное выполнения запросов (охватывающих только одну секцию) путем выделения всех имеющихся потоков для данной секции. Однако для запросов, охватывавших несколько разделов, производительность могла существенно падать — поскольку при этом выделяется только один поток на каждый раздел — что приводит к некоторому параллелизму для всего запроса, но не дает параллельного выполнения по секции. В результате запросы, которые очень мало отличались по ограничению ключа секционирования, могли давать очень разную производительность.

### SQL Server 2005



### SQL Server 2008



**Рис. 14.10.** Разница между SQL Server 2005 и 2008 в том, как для многосекционных запросов выделяются потоки

Новая функциональная возможность Partitioned Table Parallelism в SQL Server 2008 устраняет именно этот недостаток: для запроса по нескольким

секциям выделяются все имеющиеся потоки по круговой схеме. В результате каждая секция (и весь запрос в целом) получает определенную степень параллельного выполнения. Это делается автоматически. Наилучшие результаты достигаются тогда, когда количество потоков (то есть ядер процессора) значительно больше, чем количество разделов таблицы. Разница между SQL Server 2005 и 2008 в выделении потоков для многосекционных запросов показана на рис. 14.10. В SQL Server 2008 над запросом Feb YTD работают в три раза больше потоков на секцию, и при всех прочих равных условиях это должно дать повышение производительности на 200%.

### **ПРИМЕЧАНИЕ**

Функция Partitioned Table Parallelism имеется только в версии Enterprise сервера SQL Server 2008.

## **Укрупнение блокировок**

Еще одна важная возможность Partitioned Table Parallelism относится к блокировкам таблиц. Раньше при принятии решения (для секционированной таблицы) относительно укрупнения блокировки до уровня таблицы ядро базы данных не учитывало то, что одновременно выполняемые инструкции обращаются к разным секциям данной таблицы. При этом каждая такая инструкция была логически независима, и у них не было причин блокировать друг друга. Но поскольку это не учитывалось, и выполнялось укрупнение блокировки до уровня таблицы, то ядро базы данных могло без всякой необходимости заблокировать остальные инструкции, что также повышало вероятность взаимоблокировок. В SQL Server 2008 такое поведение по умолчанию для секционированной таблицы сохраняется, но Partitioned Table Parallelism вводит новую опцию `ALTER TABLE`, которая дает ядру базы данных указание использовать укрупнение блокировок до уровня секции (вместо уровня таблицы). Синтаксис показан здесь:

```
ALTER TABLE MyTable SET (LOCK_ESCALATION = <option>)
```

Опция `LOCK_ESCALATION` может быть указана как `TABLE`, `AUTO` или `DISABLE`. Значение по умолчанию — `TABLE`, укрупнение блокировок будет производиться до уровня таблицы. Если вы укажете `AUTO`, то получите укрупнение до уровня секции для секционированных таблиц и укрупнение до уровня таблицы в остальных случаях. При значении `DISABLE` укрупнение блокировок производиться не будет (в большинстве случаев).

## Оптимизация запросов для соединений типа "звезда"

Оптимизация запросов для соединений типа "звезда" (Star-Join Query Optimization) является новой важной функциональной возможностью SQL Server 2008 (имеется только в версии Enterprise). Эта функция улучшает производительность хранилищ данных, однако она не заслуживает подробного изучения в нашей книге, поскольку не имеет никаких настраиваемых пользователем свойств, и ее работа в основном скрыта внутри ядра базы данных. Хорошая новость состоит в том, что для использования ее преимуществ вам не нужно вообще ничего делать.

Как уже отмечалось ранее, схема звезды является самой обычной физической моделью данных в архитектуре хранилищ данных типа Kimball. Запросы по такой физической модели обычно характеризуются соединением центральной таблицы фактов с множеством таблиц измерений (выполняются как уравнивающие соединения). Причем таблица фактов имеет гораздо более высокое количество элементов, чем таблицы измерений (большее количество строк), а все ограничения запросов сделаны по таблицам измерений. Такой шаблон называется *соединением типа "звезда"*. Поскольку он часто встречается в различных сценариях хранилищ данных, то стало очевидным, что распознающий такой шаблон оптимизатор запросов сможет создавать более эффективные планы запросов.

Вот основная идея. Нужно устранить как можно больше строк-кандидатов из таблицы фактов на самой ранней стадии запроса, поскольку таблица фактов обычно имеет самое большое количество элементов (из участвующих в запросе таблиц). Практически это означает определение сначала кандидатов в ключи соединения из таблиц измерений (используя отсеивание по обычно наложенным на них ограничениям) и последующее использование этой информации для устранения строк-кандидатов из таблицы фактов еще до фактического процесса соединения (что более эффективно). Для эффективности этой стратегии чрезвычайно важна эвристика (то есть правила, по которым оптимизатор распознает запрос соединения типа "звезда").

Эти механизмы сложны и для наших задач почти непрозрачны. SQL Server 2005 ввел некоторые оптимизации соединений типа "звезда", которые основаны на этих принципах, а SQL Server 2008 расширил степень распознавания и оптимизации этого шаблона. Тесты производительности компании Microsoft подтверждают, что повышение производительности для типовых запросов хранилищ данных может составить от 10 до 30%. Улучшения SQL Server 2008 в этой области также содержат более подробную информацию в планах



запросов, которая помогает разработчику понять, применяется ли эта функция к данному конкретному запросу.

### **ПРИМЕЧАНИЕ**

Это улучшение наиболее ценно тогда, когда существенная часть нагрузки SQL Server состоит из произвольных SQL-запросов по схеме звезды. Если же ваша архитектура направляет большинство произвольных запросов к кубу OLAP, то выигрыш будет меньше (или его может вообще не быть), кроме режимов хранения Relational OLAP (ROLAP) или Hybrid OLAP (HOLAP), поскольку в этих случаях значительное количество запросов по кубам становится запросами SQL по схеме звезды.

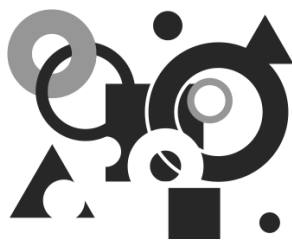
Ограничения по объему данной книги не дают нам возможности обсудить эту функцию подробнее. Мы рекомендуем вам обратиться по ссылкам, приведенным в конце этого раздела.

## **Резюме**

Хранилища данных стали ключевым компонентом любой архитектуры работы с данными в масштабе предприятия и перестали быть уделом только самых крупных предприятий. Хранилища данных развивались как ответ на множество проблем при получении информации (для принятия решений) из собранных операционными приложениями данных, причем эти проблемы далеко не сразу признавались в качестве препятствий, которые существенно снижают потенциал компьютеров в плане превращения данных в информацию. Проблемы существовали не только вследствие исторически имевшихся технических ограничений, но также и вследствие фундаментальных различий оптимального дизайна операционных и информационных приложений.

Хранилище данных служит основой для многих приложений данных. SQL Server 2008 представляет собой полнофункциональную, мощную и рентабельную платформу, на которой можно построить хранилище данных. Вы увидели, что SQL Server 2008 специально приспособлен к решению проблем хранилищ данных и обеспечивает множество долгожданных функций в этой области. Кроме того, компания Microsoft предлагает широкий спектр интегрированных и дополнительных технологий, в том числе Microsoft Office SharePoint, Microsoft Performance Point и 2007 Microsoft Office system, которые позволяют вам создавать информационные приложения поверх вашего хранилища данных на базе SQL Server.

## ГЛАВА 15



# Основы OLAP

*Эндрю Брест*

Эта глава и три последующие посвящены аналитическим службам Analysis Services, основному компоненту бизнес-аналитики сервера Microsoft SQL Server 2008. Такие возможности появились с момента выпуска OLAP Services в версии SQL Server 7.0 (1999 г). Вероятно, вы подумали, что с учетом такой длительной истории компонентов бизнес-аналитики в SQL Server большинство работающих профессионалов хорошо ориентируется в соответствующей терминологии.

Однако это совсем не так. Для многих пользователей SQL Server возможности его бизнес-аналитики оставались невостребованными, не очень понятными и до некоторой степени загадочными. Основной продукт SQL Server (реляционная база данных) обособлен от Analysis Services, поэтому совершенно спокойно можно быть специалистом в первом продукте и совершенно невежественным во втором.

## Для чего существует бизнес-аналитика

Честно говоря, та организация, которая просто собирает свои данные и (возможно) регулярно просматривает их при помощи простых и произвольных отчетов, практически полностью теряет их стратегическую ценность и информацию, которая содержится в этих данных. Более того, использовать базовые возможности Analysis Services совсем не трудно, а сами концепции не настолько сложны, чтобы их не могли освоить специалисты по реляционным базам данных.

Если вы внимательно посмотрите на полный набор функциональных возможностей и компонентов SQL Server 2005 и 2008 в сравнении с SQL Server 2000, то обнаружите, что самые большие достижения сделаны в области бизнес-аналитики. Поймите нас правильно: такие возможности реляционной базы данных, как Service Broker, поддержка XML, а также модель программирования SQL Common Language Runtime (CLR) — все они очень важны. Но их следует понимать как инкрементальные изменения ядра реляционной базы данных, которое было уже вполне зрелым, достаточно совершенным, масштабируемым и хорошо понимаемым на рынке продуктом.

В то же время достижения Analysis Services гораздо более революционны. Выпуск SQL Server 2005 Analysis Services обозначил превращение аналитических служб в массовый продукт, интегрированный в SQL Server, и его инструментальный набор. Аналитические службы SQL Server 2008 Analysis Services добавили некоторый лоск версии 2005 г. и сделали бизнес-аналитику SQL Server еще более дружелюбной к начинающему пользователю (и в то же время еще более мощной для опытных). Поэтому мы твердо уверены, что описать Analysis Services в этой книге очень важно. Кроме того, мы считаем, что каждый работающий с SQL Server профессионал должен иметь, по крайней мере, начальное представление об этой части продукта SQL Server.

Мы также твердо уверены в том, что понимание аналитических служб абсолютно доступно для того, кто является специалистом по реляционной составляющей сервера SQL Server. Наша цель — дать такое описание Analysis Services, которое будет доступным, практичным и интересным. В этой главе содержится относительно короткое введение в аналитическую обработку транзакций (online analytical processing, OLAP). Когда вы закончите эту главу, вы будете готовы строить кубы OLAP и понимать все преимущества этого занятия. Для того чтобы приступить к этому делу, вам не нужно читать три следующих главы; мы советуем вам после прочтения данной главы самостоятельно приступить к этому процессу и познать вкус победы от использования новых знаний по OLAP.

Когда эти практические достижения пробудят ваш интерес к данной технологии (и вашу уверенность в том, что вы сможете изучить ее), вы, вероятно, захотите перейти к *главе 16*, где мы описываем некоторые усовершенствованные функции OLAP (и в том числе новые для версии SQL Server 2008 функции). *Глава 16* гораздо длиннее этой главы, но вы можете читать ее частями и немедленно использовать прочитанное. Когда вы закончите, вы сможете создавать очень сложные кубы OLAP.

На этом вы могли бы и остановиться, но, как разработчик, вы захотите перейти к *главе 17*, где мы описываем разнообразные методики разработки программного обеспечения, которые вы сможете использовать для интегрирования функциональных возможностей OLAP в ваши собственные приложения. Вы научитесь создавать пользовательские интерфейсы (с поддержкой OLAP) при помощи Microsoft Office Excel и Excel Services. Вы также научитесь использовать интерфейсы прикладного программирования и объектные модели, в том числе: ADOMD.NET, XML for Analysis (XMLA) и Analysis Management Objects (AMO). Вы даже узнаете, как использовать собственную модель программирования CLR в Analysis Services для создания кода для серверной стороны (на .NET-языках). *Глава 17* длинная, но ее можно читать и применять постепенно.

После нее вы тоже можете остановиться или перейти к *главе 18*, где мы описываем возможности интеллектуального анализа данных в Analysis Services. Они впервые появились в версии SQL Server 2000 служб Analysis Services, но уже в версии SQL Server 2005 стали вполне зрелыми и еще более улучшились в версии SQL Server 2008. Разработку и проектирование интеллектуального анализа данных освоить проще, чем OLAP. Поэтому мы описываем интеллектуальный анализ данных в одной (хотя и длинной) главе. Многие из методик программирования, описанных в *главе 17*, будут повторены в *главе 18* (что позволит сделать их изложение более кратким). Мы уверены, что мощь этого продукта произведет на вас большое впечатление.

## Начальный курс OLAP

Давайте начнем наше исследование служб Analysis Services с краткого введения в OLAP. В этой главе вы познакомитесь с фундаментальными основами OLAP (включая общие концепции, а также основы построения, обслуживания и запросов кубов OLAP в Microsoft SQL Server 2008). Мы изложим следующие темы:

- определения некоторых терминов, в том числе кубов, мер, измерений, атрибутов, иерархий, уровней, элементов и осей;
- концепции хранилищ данных и обоснование схем звезды и снежинки;
- основы проектов Microsoft Visual Studio Analysis Services, такие как источники данных, представления источников данных, визуальные конструкторы кубов и измерений;
- запросы к кубам на вкладке **Browser** визуального конструктора кубов.

Как мы только что упомянули, SQL Server впервые дал нам функциональные возможности OLAP в версии 7.0. Это были OLAP Services, по существу отдельный от SQL Server продукт. SQL Server 2000 Analysis Services уже имели улучшенные функциональные возможности OLAP и новые возможности интеллектуального анализа данных, но интеграция с реляционной базой данных SQL Server была лишь незначительно выше. Analysis Services в SQL Server 2005 функционально богаче, имеет гораздо более высокую степень интеграции с продуктом, пересмотренную архитектуру и впечатляющую простоту использования. В SQL Server 2008 службы Analysis Services получили новые визуальные конструкторы, а также Best Practice Alerts (BPA), которые еще больше повысили мощь продукта и в то же время понизили порог доступа для новых пользователей, которые хотят строить хорошо спроектированные кубы. В этой главе мы приступим к описанию общей суммы введенных в версиях Analysis Services 2005 и 2008 функциональных возможностей, но начнем с некоторых основных концепций.

OLAP можно представить себе как технологию баз данных, оптимизированную для углубленного анализа. Вот и все! Забудьте все сбивающие с толку объяснения, которые вам приходилось слышать — никакого волшебства здесь нет. OLAP позволяет пользователям неправдоподобно быстро выполнять запросы детализации, причем делает это двумя способами:

- кубы OLAP хранят так много данных, что во многих случаях свертки высокого уровня и цифры среднего или низкого уровня (необходимые для запроса) уже вычислены, и их нужно только вывести;
- даже те вычисления, которые необходимо выполнять "на ходу", можно произвести быстро, поскольку ядро OLAP оптимизировано для этого, и ему не нужно заниматься сложными задачами реляционной базы данных, такими как: управление индексами, выполнение соединений или (кроме особых обстоятельств) работа с обновлениями и параллелизм. Кубы OLAP по существу содержат данные на самых нижних уровнях детализации, вычисляют некоторые (или все) агрегаты более высокого уровня (при построении куба), а также вычисляют и остальные агрегаты во время запроса (путем агрегирования уже имеющихся в кубе значений низкого уровня).

В результате получается технология, которая позволяет пользователям выполнять совершенно невероятный объем исследования данных, что дает им возможность изучить множество произвольных гипотетических сценариев (не беспокоясь при этом о времени и ресурсах, которые потребовались бы ядру реляционной базы данных для ответа на эти запросы). Если пользовате-

ли не боятся задавать вопросы, то они будут задавать их все больше, получать представление о своих данных, принимать лучшие бизнес-решения и получать гораздо более высокую рентабельность реляционных систем, которые поддерживают данные кубов.

## Словарь OLAP

*Куб* (который является в OLAP эквивалентом таблицы реляционной базы данных) состоит из *мер* (являющихся числовыми данными, которые будут анализировать пользователи, например, объем продаж) и *измерений* (являющихся категориями, по которым будет происходить углубление в меры, например, время, география, поставщик или рекламная акция). Измерения могут быть иерархическими. Например, измерение географии, скорее всего, будет иерархическим и может содержать уровни страны, штата, города и почтового кода. Отдельные страны, штаты (и т. д.) называются элементами своих соответствующих уровней. Такая схема позволяет пользователям разбивать продажи по странам, затем углубляться до определенной страны (это хорошо в тех случаях, когда продажи по данной стране особенно высокие или низкие), потом до определенного штата внутри этой страны, затем по конкретным клиентам в этом штате (до почтовых кодов внутри городов). Это может позволить пользователю быстро выяснить, отчего продажи по конкретной стране такие высокие (или низкие).

Все это кажется элементарным и несложным, не так ли? Вы разочарованы? Не стоит. Давайте несколько повысим ставки. Представьте себе электронную таблицу, где столбцы содержат набор стран (в которых работает компания), строки содержат кварталы последнего финансового года, а каждая ячейка содержит объем продаж каждой соответствующей комбинации страны и финансового квартала. Представьте далее, что пользователи могут углубиться в любой квартал (чтобы посмотреть три его месяца) или в любую страну (чтобы увидеть объемы продаж для этих сечений куба). Представьте, что эта электронная таблица размножена в большом количестве экземпляров (по одной таблице на каждого продавца), и что все эти таблицы предлагают точно такие же возможности детализации, возвращая результаты углубления быстро и легко.

Полный набор электронных таблиц можно получить от ядра OLAP при помощи одного очень простого запроса. По-прежнему не впечатляет? Представьте, что каждое число (объем продаж) в каждой электронной таблице находится рядом с соответствующим объемом продаж год назад, и даже при

таким усложнением интерактивный отчет с детализацией по-прежнему можно получить при помощи одного запроса. Надеемся, что мы смогли вызвать ваш интерес и дали вам некоторое представление о мощности и ценности OLAP.

## Измерения, оси, звезды и снежинки

Давайте вернемся к определениям. Поскольку кубы OLAP и даже результирующие наборы запросов OLAP могут содержать данные по многим физическим измерениям (а не только по строкам и столбцам), то применительно к базе данных OLAP часто используется термин "многомерная". Используемый вами язык запросов для кубов OLAP в SQL Server называется MDX (multidimensional expression language, язык многомерных выражений). Используются для написания приложений OLAP объектная модель на базе Component Object Model (COM) ADO MD и ADO MD.NET (ее .NET-эквивалент). Зрительно представить себе многомерные данные может быть очень сложно. Будьте к этому готовы (чтобы вас не постигло уныние), а мы постараемся помочь вам со всем этим справиться.

Кубы OLAP создаются из коллекции специальных таблиц реляционной базы данных: *таблиц фактов* и *таблиц измерений*. Для иллюстрации их содержания представьте себе простой куб, содержащий в качестве мер продажи по подразделениям, общие продажи и скидки, а в качестве измерений — поставщика и местоположение. Представьте себе, что измерение поставщиков плоское (то есть не иерархическое), и измерение местоположения — иерархическое (причем самый нижний его уровень — почтовый код). Таблица фактов должна содержать данные по продажам, которые можно отнести к поставщикам (и их почтовым кодам). Каждая строка содержит почтовый код в первом столбце, ключ по поставщикам во втором столбце и соответствующие данные по измерениям в дополнительных столбцах. Каждая возможная комбинация почтового кода и поставщика соответствует отдельной строке таблицы фактов (за исключением тех почтовых кодов, где поставщики не сделали никаких продаж).

Нам нужны также таблицы измерений. Давайте начнем с таблицы измерений для поставщика. Это просто справочная таблица, содержащая идентификатор поставщика и его название (в разных столбцах). Вы понимаете, что соединение таблицы фактов с этой таблицей по идентификатору поставщика позволит нам получить название поставщика для каждой строки таблицы фактов (для каждого поставщика/почтового кода).

Поскольку местоположение — иерархическое, то связь между таблицей фактов и таблицей измерений по местоположениям более сложна. Информация измерения местоположения может быть записана в единую денормализованную таблицу, которая соответствует каждому уникальному почтовому коду (с соответствующим ему городом, штатом и страной), либо вы можете иметь отдельные (нормализованные) справочные таблицы для каждого уровня иерархии. (У вас может быть также комбинация полуденормализованных таблиц, каждая из которых комбинирует пару иерархических уровней). На рис. 15.1 показан пример данных в таблице фактов и две денормализованные таблицы измерений. Обратите внимание, что таблица фактов содержит только данные измерений и внешние ключи к самому нижнему уровню каждого из двух измерений).

(local).Northwi...impleSample.sql

SELECT \* FROM tblFactTable  
SELECT \* FROM tblDimGeography  
SELECT \* FROM tblDimShippers

Results Messages

|   | PostalCode | ShipperId | Total Sales | Discount         | Unit Sales |
|---|------------|-----------|-------------|------------------|------------|
| 1 | 28001      | 2         | 365.89      | 0                | 11         |
| 2 | 67000      | 1         | 5118.80     | 337.640008449554 | 243        |
| 3 | 97201      | 2         | 516.40      | 0                | 24         |
| 4 | WX3 6FW    | 2         | 3740.00     | 0                | 125        |
| 5 | 59000      | 2         | 4443.00     | 0                | 135        |
| 6 | 05442-030  | 3         | 3433.40     | 161.150007724762 | 270        |
| 7 | 51100      | 2         | 261.40      | 0                | 20         |
| 8 | S-958 22   | 3         | 5967.10     | 189.5            | 259        |
| 9 | W43 10P    | 1         | 400.00      | 0                | 50         |

|   | PostalCode | City           | State    | Country |
|---|------------|----------------|----------|---------|
| 1 | NULL       | Cork           | Co. Cork | Ireland |
| 2 | 01-012     | Warszawa       | NULL     | Poland  |
| 3 | 01307      | Cunewalde      | NULL     | Germany |
| 4 | 02389-673  | Rio de Janeiro | RJ       | Brazil  |
| 5 | 02389-890  | Rio de Janeiro | RJ       | Brazil  |
| 6 | 04179      | Leipzig        | NULL     | Germany |

|   | ShipperId | ShipperName      |
|---|-----------|------------------|
| 1 | 1         | Speedy Express   |
| 2 | 2         | United Package   |
| 3 | 3         | Federal Shipping |

Рис. 15.1. Данные и структура двух гипотетических таблиц измерений, а также простая таблица фактов, которая ссылается на них

После построения куба нормализованный или денормализованный характер его измерений большого значения не имеет, поэтому сделанный вами на этапе проектирования выбор — это по существу просто дело вашего удобства. В случае полностью денормализованных таблиц измерений схема состоит из



таблицы фактов (в центре), из которой исходят "лучи" к каждой таблице измерений (рис. 15.2). Такая геометрия схемы напоминает звезду, поэтому она часто называется "схемой звезды". (Несмотря на то, что эта схема содержит только две таблицы измерений, вы можете представить, откуда берется название). Измерения, которые имеют несколько таблиц, отчасти ослабляют аналогию со звездой (поскольку лучи имеют по несколько узлов). Поэтому измерения с несколькими таблицами чаще называются "схемой снежинки".

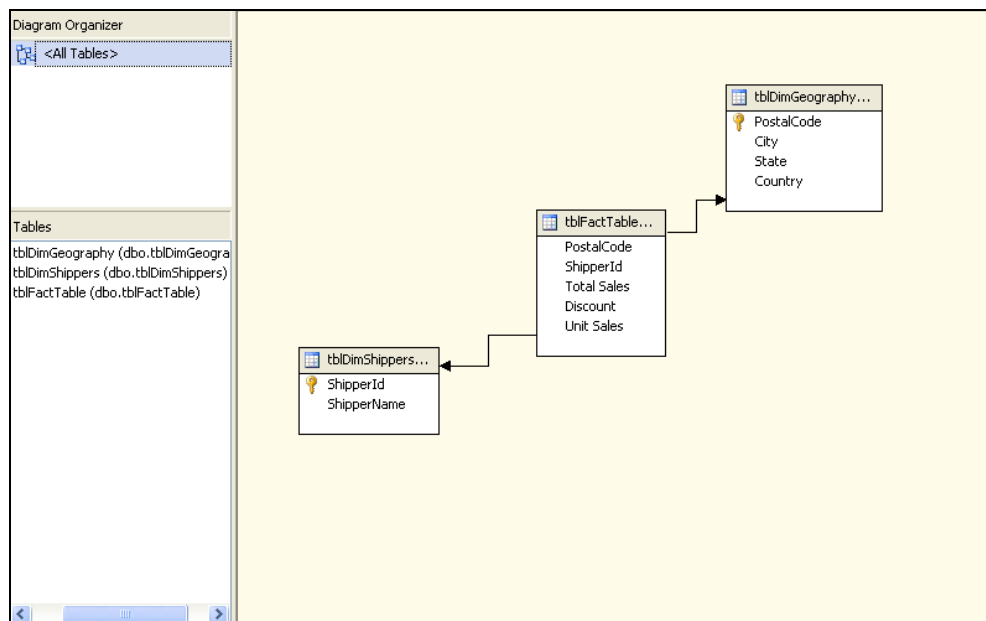


Рис. 15.2. Пример куба OLAP, основанного на схеме звезды

Обычно базы данных со схемами звезда/снежинка создаются из нормализованных баз данных (при помощи выполнения скриптов, хранимых процедур, а также процессов ETL), чтобы создать преобразованные таблицы фактов и измерений. Эти таблицы могут быть созданы в новой отдельной базе данных; фактически они являются основой хранилища данных или витрины данных. В этом виде они могут служить отличными источниками данных и для реляционных отчетов. Поскольку эти таблицы имеют некоторую степень денормализации, то по ним легко создавать отчеты, а поскольку они находятся в отдельной базе данных (или, по крайней мере, существуют в виде отдельных таблиц главной базы данных), то выполнение по ним отчетов (пользователя-

ми) не нагружает производственные таблицы оперативной обработки транзакций (OLTP). Если вы считаете физическое преобразование данных неудобным, то запомните: и таблица фактов, и таблица измерений могут быть представлениями.

## Построение вашего первого куба

Теперь, когда вы уже представляете себе проектирование и реализацию таблиц фактов и измерений, как нам создать куб? Давайте займемся этим.

### Подготовка объектов схемы звезды

Для данной главы мы не будем использовать базу данных AdventureWorks2008, а создадим новую. Это дает нам шанс самим все попробовать. Для этого примера мы будем использовать старую базу данных Northwind (из SQL Server), поскольку в ней нет никаких заготовок для куба. Решение для Management Studio (в примерах кодов для данной главы, имеющих на Web-сайте этой книги) содержит несколько скриптов SQL, создающих таблицу фактов, и несколько представлений, которые можно использовать как таблицы измерений для базы Northwind.

#### **ПРИМЕЧАНИЕ**

Если у вас нет базы данных Northwind, то вы можете скачать ее с сайта компании Microsoft. Указания по скачиванию базы данных Northwind можно найти во введении (в начале этой книги).

После инсталляции (на вашем сервере) базы данных Northwind выполните по ней все скрипты проекта Chapter15 Management Studio project (за исключением Time.sql) в произвольном порядке.

#### **ВАЖНО**

Скрипт Time.sql следует выполнять только в том случае, если вы не будете создавать вручную проект Analysis Services, базу данных и куб. Если вы хотите открыть полностью готовый проект Chapter15 для Visual Studio (имеющийся в примерах кодов этой главы) и развернуть базу данных Chapter15 уже непосредственно из него, то вы должны сначала выполнить скрипт Time.sql. Выполнение этого скрипта создаст заполненную таблицу с названием Time.

После выполнения всех этих шагов ваша копия Northwind должна содержать новую таблицу с названием `tblFact` (которая будет служить таблицей фактов) и следующие представления: `vwEmployees`, `vwGeography`, `vwProducts`, `vwShippers` и `vwSuppliers` (которые будут служить как таблицы измерений). Все эти таблицы и представления создаются скриптами SQL с соответствующими названиями. После создания этих таблиц и представлений вы можете начинать проектировать, строить и запрашивать ваш куб.

## Инструмент (как бы его ни называли)

Теперь вы готовы строить ваш куб, и для этого вам нужны специальные инструменты проектирования. В SQL Server 2000 Analysis Services кубы проектировались и управлялись при помощи инструмента с названием Analysis Manager (модульного компонента программы Microsoft Management Console, MMC), который являлся по существу аналогом SQL Server Enterprise Manager. В конце 1990 гг. модули MMC были популярным средством создания инструментов управления и проектирования. В наши дни гораздо более популярной стала система Visual Studio, которая имеет свой собственный богатый интерфейс прикладного программирования расширений.

Команда разработчиков SQL Business Intelligence приняла решение использовать Visual Studio (а в случае с SQL Server 2008 — Visual Studio 2008) как основу для множества визуальных конструкторов бизнес-аналитики (включая конструкторы для Reporting Services, Integration Services и Analysis Services). В следующей главе вы увидите, что SQL Server Management Studio также может использоваться как инструмент управления баз данных Analysis Services; однако визуальные конструкторы работают внутри Visual Studio.

Несмотря на то, что большинство из вас, вероятно, имеет в своем распоряжении Visual Studio 2008, то же самое нельзя сказать про всех работающих с Analysis Services разработчиков и администраторов. Компания Microsoft понимает это и знает, что им нужно работающее решение. По существу, компании Microsoft нужен способ поставлять вместе с SQL Server некий "огрызок" интегрированной среды разработки Visual Studio (чтобы пользователи, не имеющие лицензионной версии Visual Studio, могли создавать различные визуальные конструкторы и поддерживаемые бизнес-аналитикой SQL Server типы проектов, а также работать с ними).

Когда вы устанавливаете компоненты бизнес-аналитики SQL Server на компьютере, где нет Visual Studio 2008, то инсталлятор SQL Server установит на этот компьютер именно такую "ослабленную" версию Visual Studio; если же

копия Visual Studio 2008 на компьютере уже установлена, то визуальные конструкторы бизнес-аналитики будут использовать именно ее.

### **ПРИМЕЧАНИЕ**

Для работы с различными визуальными конструкторами бизнес-аналитики SQL Server версий Express программы Visual Studio недостаточно. Компьютеры с установленными версиями Visual Studio Express воспринимаются инсталлятором SQL Server как компьютеры, на которых совсем нет Visual Studio 2008.

Все выглядит достаточно разумно, но есть одна деталь, которая может все несколько запутать. Инсталлятор SQL Server 2008 создаст на вашем компьютере ярлык для Visual Studio (для установленной им ослабленной версии или для уже существующей вашей версии). Название этого ярлыка не Microsoft Visual Studio 2008, а SQL Server Business Intelligence Development Studio.

Важно понимать, что этот ярлык фактически указывает на Visual Studio (будь это облегченная версия или полноценный продукт) и что на самом деле нет такой вещи, как Business Intelligence Development Studio. Вам может показаться удобным использовать такое название для ослабленной версии интегрированной среды разработки Visual Studio (которую мы только что обсуждали), но это обоснование становится совершенно неточным, когда вы обнаруживаете, что ярлык с меткой SQL Server Business Intelligence Development Studio фактически связан с полной версией Visual Studio на тех компьютерах, где она имеется.

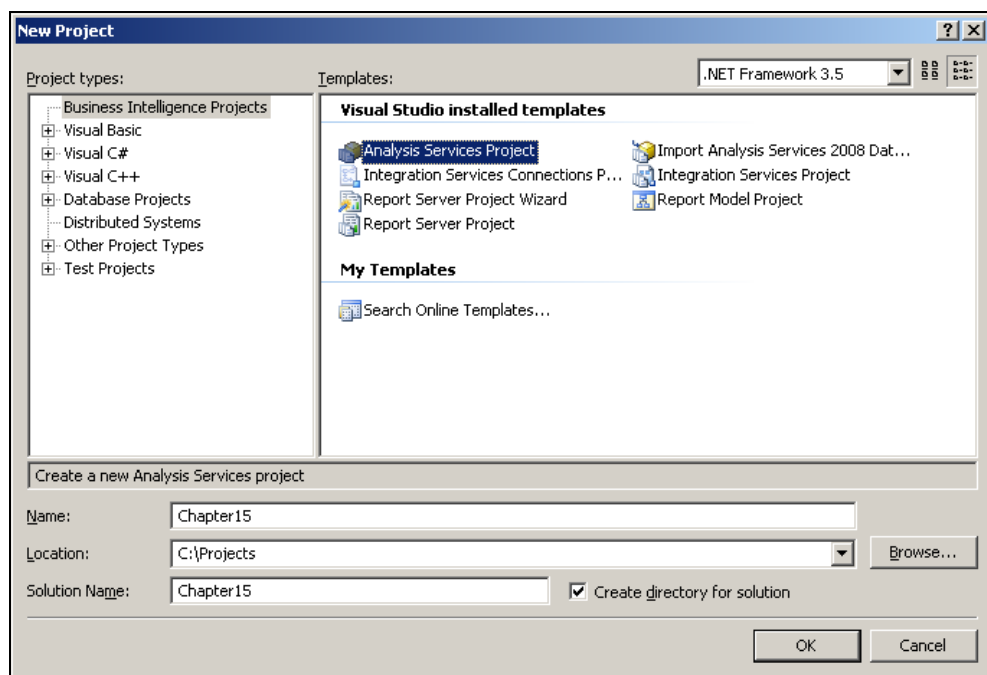
Для того чтобы все было совершенно понятно, в данной книге мы будем называть хост-среду Analysis Services и прочих визуальных конструкторов бизнес-аналитики просто Visual Studio. Мы делаем это замечание не только для прояснения терминологии. Очень важно понимать, что визуальные конструкторы бизнес-аналитики работают в среде Visual Studio (а не в среде Business Intelligence Development Studio), потому что решение Visual Studio может содержать смесь проектов бизнес-аналитики SQL Server и более традиционных проектов приложений Visual Studio .NET.

Например, решение Visual Studio 2008 может содержать проект Analysis Services, проект Reporting Services, а также различные проекты на языках C# или Microsoft Visual Basic .NET (включая приложение Windows Forms, проект Class Library или Web-сайт ASP.NET). Это позволяет нам соблюдать использованный в других главах этой книги стандарт, когда единое решение Management Studio или Visual Studio содержит все коды для данной главы.

Это особенно актуально для главы 18, коды которой включают в себя по одному из всех пяти типов проектов (бизнес-аналитики и .NET).

## Создание проекта

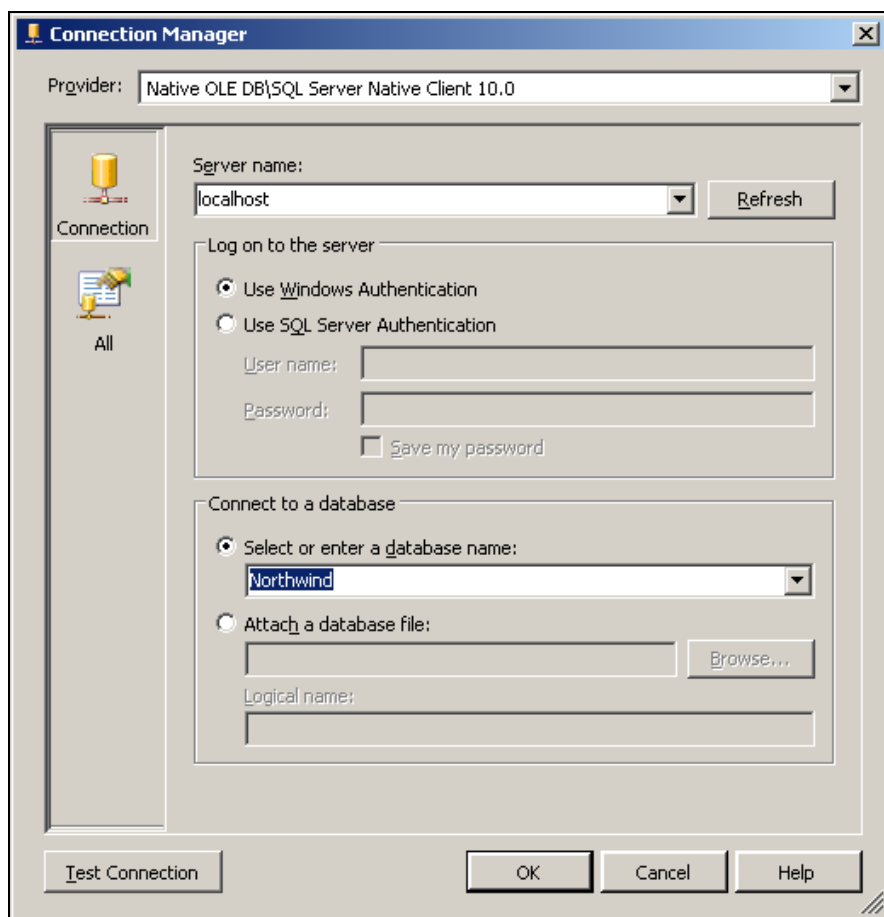
Теперь, когда мы имеем представление о наборе инструментов, пришло время создать проект Analysis Services и спроектировать наш куб. Запустите Visual Studio, а затем выберите **File | New | Project**. В диалоговом окне **New Project** выберите **Business Intelligence Projects** в древовидном представлении **Project types** с левой стороны, а затем выберите **Analysis Services Project** в панели **Templates** справа. Введите в качестве названия проекта Chapter15 и удобный для вас путь к нему. (На рис. 15.3 показан вид этого диалогового окна). Теперь вы готовы нажать кнопку **OK** и создать ваш проект.



**Рис. 15.3.** Создание нового проекта Analysis Services в диалоговом окне **New Project** в Visual Studio

Вы заметите, что ваш новый проект пуст (в отличие от большинства других проектов Visual Studio, где для вас создается и открывается некий объект по

умолчанию). В проектах Analysis Services вы начинаете с чистого листа. Самое лучшее, что можно сделать сразу же — добавить источник данных и представление источника данных. Создайте новый источник данных при помощи щелчка правой кнопкой мыши по пункту **Data Sources** в окне **Solution Explorer**, а затем выберите **New Data Source** из контекстного меню (чтобы запустить мастер Data Source Wizard).



**Рис. 15.4.** Заполнение диалогового окна **Connection Manager** для создания источника данных проекта Analysis Services

На странице **Welcome** мастера нажмите кнопку **Next** для перехода к странице **Select How To Define The Connection**. Нажмите кнопку **New** для показа диа-

логового окна **Connection Manager**. В качестве **Provider** укажите **Native OLE DB\SQL Server Native Client 10.0**. В качестве **Server name** введите название вашего сервера (и через обратную косую черту — название его экземпляра, если есть), укажите данные вашей регистрационной записи и выберите в качестве базы данных **Northwind**. Диалоговое окно должно быть похоже на рис. 15.4.

### ПРИМЕЧАНИЕ

Убедитесь, что учетная запись, под которой работают службы **Analysis Services**, имеет доступ на чтение к базе данных **Northwind**. Если **Analysis Services** работает под учетной записью **Local System**, то все должно быть хорошо. Если это не так, то вам придется добавить соответствующую учетную запись в качестве пользователя **Northwind**.

Нажмите кнопку **OK** для того, чтобы закрыть диалоговое окно **Connection Manager**. В мастере **Data Source Wizard** нажмите кнопку **Next** для перехода на страницу **Impersonation Information**, где вам следует выбрать опцию **Inherit** (последнюю в списке), а затем еще раз нажать кнопку **Next**. На странице **Completing The Wizard** примите название по умолчанию для источника данных **Northwind**, а затем нажмите кнопку **Finish**.

## Добавление представления источника данных

Все, что мы делали до сих пор, похоже на создание подключения к базе данных; нам еще нужно указать, какие объекты из базы данных **Northwind** мы хотим использовать для построения нашего куба. Для того чтобы указать эту информацию, щелкните правой кнопкой мыши по каталогу **Data Source View** в **Solution Explorer** и выберите в контекстном меню пункт **New Data Source View**. После этого будет вызван мастер **Data Source View Wizard**.

На странице приветствия **Welcome** нажмите кнопку **Next** и еще раз нажмите кнопку **Next** на странице **Select A Data Source** (убедившись, что выбран только что созданный вами источник данных **Northwind**). На странице **Select Tables And Views** выберите (то есть переместите из списка **Available objects** в список **Included objects**) таблицу **tblFact** (таблицы перечислены в списке **Included objects** первыми) и представления **vwEmployees**, **vwGeography**, **vwProducts**, **vwShippers** и **vwSuppliers** (которые находятся внизу списка **Included objects**). На рис. 15.5 показано, как в этот момент должны выглядеть списки.

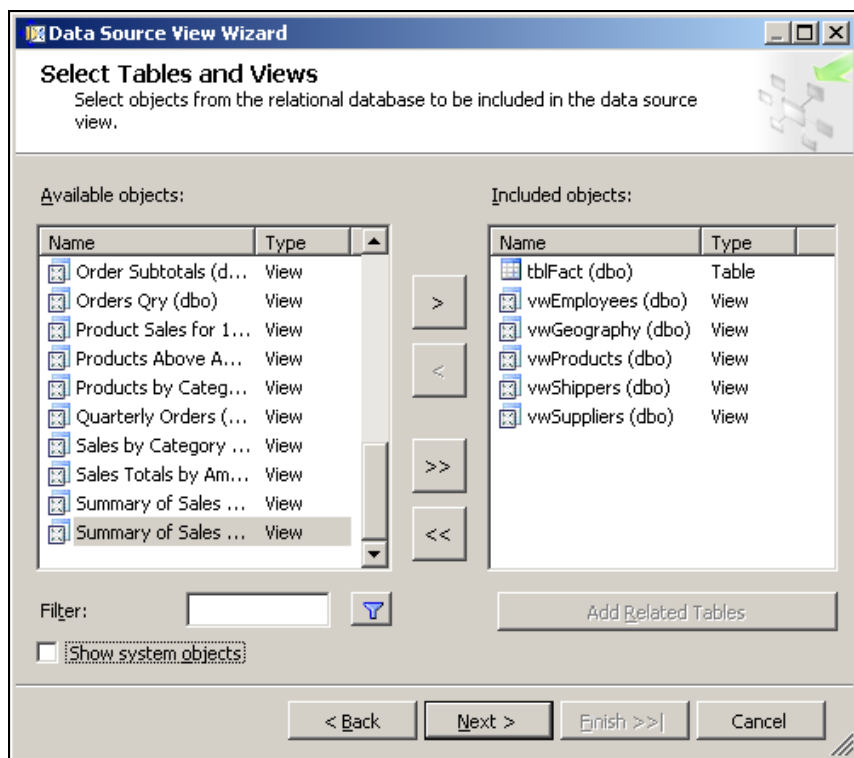


Рис. 15.5. Выбор таблиц фактов и измерений для вашего куба

Если при создании представлений источников данных вы не знаете, какие таблицы вам выбрать — не паникуйте. Вы можете вернуться к этой странице позже, по завершении работы мастера (в виде отдельного диалогового окна **Add/Remove Tables** в визуальном конструкторе представлений источников данных). Далее в этой же главе мы объясним вам, как это делается.

Вы можете ввести выражение фильтра (с групповыми символами) в текстовом поле **Filter** для того, чтобы ограничить таблицы и представления, которые появляются в списке **Available objects**. Например, попробуйте ввести **vw**, а затем нажать клавишу **<Enter>** или щелкнуть по кнопке со значком воронки справа от текстового поля. После этого в списке будут отображаться только таблицы и представления, которые начинаются с букв "vw".

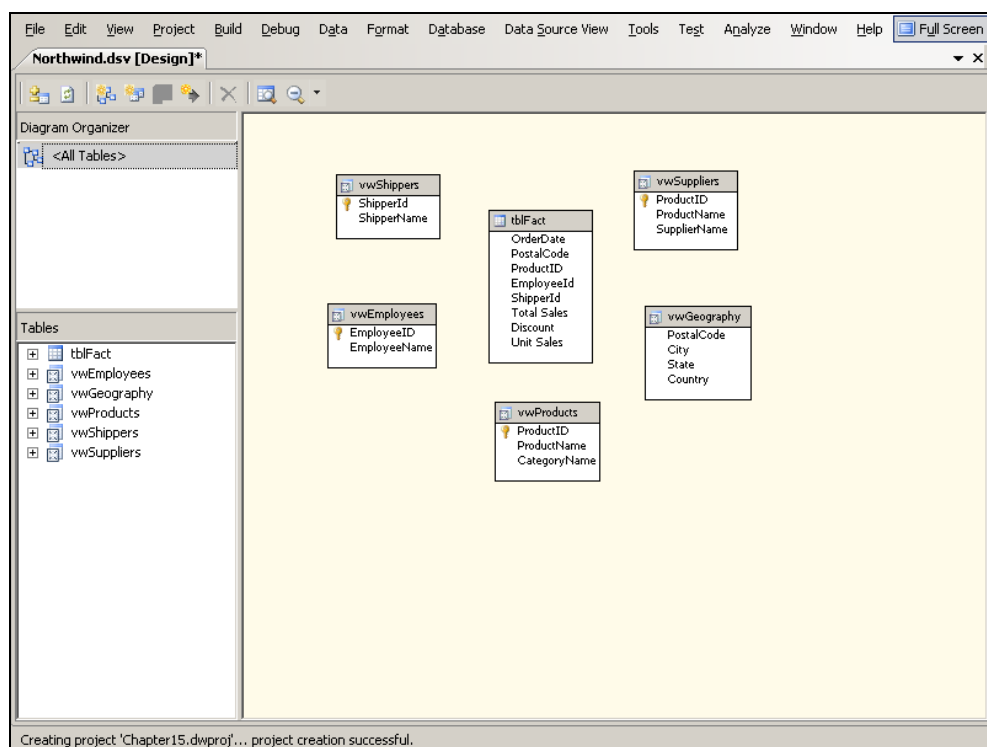
После того как вы перенесете в список **Included objects** хотя бы одну таблицу, вы сможете выбрать таблицу (или таблицы) и нажать кнопку **Add Related Tables** (для того чтобы выбрать все связанные с ней (или с ними) таблицы).



**СОВЕТ**

Это может дать существенное ускорение: если вы сначала включите таблицу фактов и нажмете кнопку **Add Related Tables**, то вы сможете добавить сразу все таблицы измерений (в зависимости от дизайна вашей базы данных со схемой "звезда"). Поскольку мы используем представления как таблицы измерений, то в нашем случае такое ускорение не работает.

Когда вы закончите выбирать таблицы, нажмите кнопку **Next**. На странице **Complete The Wizard** примите присвоенное (вашему представлению источника данных) название Northwind и нажмите кнопку **Finish**. После короткой паузы представление вашего источника данных автоматически откроется в своем визуальном конструкторе. Таблицы в представлении источника данных нужно будет разместить вручную (чтобы все они были видны одновременно, и в центре находилась таблица фактов), как показано на рис. 15.6.



**Рис. 15.6.** Сгенерированное мастером представление источника данных (после некоторого переупорядочивания вручную)

Визуальный конструктор разделен на три панели: сверху слева находится панель **Diagram Organizer**, внизу слева находится панель **Tables**, а справа находится панель макета диаграммы. Изображенная в панели макета диаграммы **All Tables** показывает все таблицы, которые вы выбрали в мастере. В представление источника данных можно добавить диаграммы, которые содержат только подмножество таблиц или представлений в представлении источника данных посредством выбора **Data Source View | New Diagram** (либо щелкнув правой кнопкой мыши в панели **Diagram Organizer** и выбрав пункт **New Diagram** в контекстном меню, а также при помощи щелчка по кнопке **New Diagram** панели инструментов). После любого из этих трех действий вы сможете ввести название диаграммы, а затем перетащить таблицы и представления из панели **Tables** на поверхность конструирования (в панели макета диаграммы). Если вы случайно перетащили таблицу или представление, которое вам на диаграмме не нужно, то просто щелкните по ней правой кнопкой мыши в панели макета диаграммы и выберите пункт контекстного меню **Hide Table** (либо выделите таблицу в панели макета диаграммы и выберите в меню **Data Source View | Hide Table**).

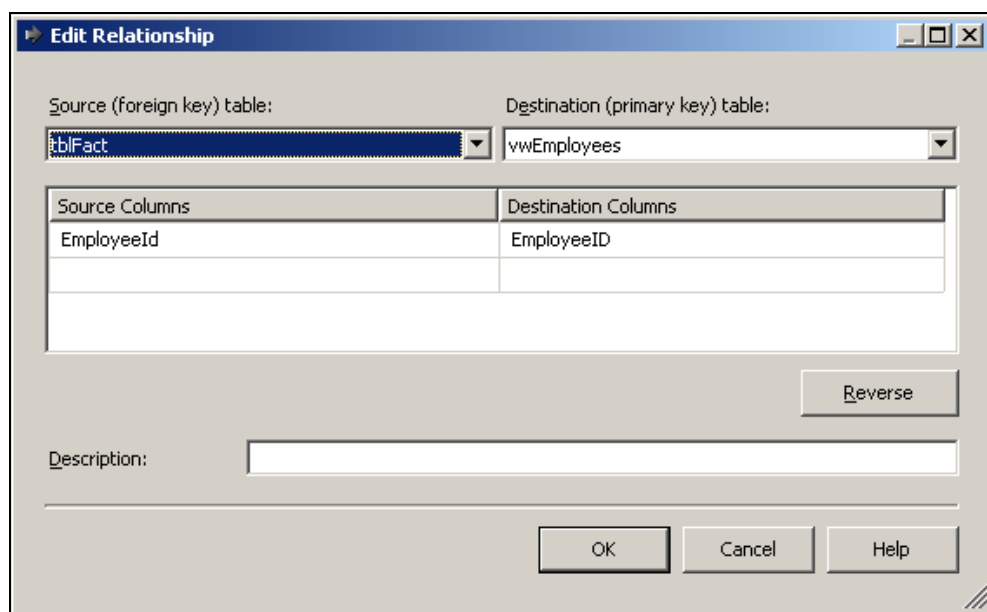
Указанный вами первоначально на странице **Select Tables And Views** мастера **Data Source View Wizard** набор таблиц и представлений (на котором основано представление источника данных) можно изменить внутри визуального конструктора представлений источников данных. Вы можете сделать это в диалоговом окне **Add/Remove Tables** путем выбора **Data Source View | Add/Remove Tables** (либо при помощи щелчка правой кнопкой мыши в панели макета диаграммы и выбора по пункту **Add/Remove Tables** контекстного меню).

По умолчанию все ограничения внешних ключей между таблицами показываются в диаграмме по мере добавления таблиц на диаграмму. В нашем случае таких предопределенных ограничительных связей нет, поэтому вы сами должны добавить связи на диаграмму представления источника данных. Для этого просто щелкните по столбцу внешнего ключа в таблице `tblFact`, а затем тащите курсор мыши к соответствующему столбцу соответствующей таблицы измерений.

Например, вы можете щелкнуть по полю `EmployeeId` в таблице `tblFact` (чтобы выделить его), а затем потащить курсор мыши к полю `EmployeeId` представления `vwEmployees`. Отпустите кнопку мыши — и вы увидите линию со стрелкой, соединяющую таблицу фактов с представлением. Дважды щелкните по этой связи (либо щелкните по ней один раз и выберите **Data Source**

**View | Edit Relationship**) для показа диалогового окна **Edit Relationship** (рис. 15.7), где вы сможете отредактировать свойства этой связи и убедиться, что вы соединили правильные столбцы.

Диалоговое окно **Edit Relationship** — это по существу окно свойств связи между таблицей фактов и "таблицей" измерения сотрудников (фактически это представление с названием `vwEmployees`). Модифицированную версию этого диалогового окна (диалоговое окно **Specify Relationship**) можно использовать для создания связей в том случае, если вам не нравится метод перетаскивания. Вы можете отобразить диалоговое окно **Specify Relationship** при помощи выбора **Data Source View | New Relationship** либо при помощи щелчка правой кнопкой мыши в панели диаграммы и выбора пункта **New Relationship** контекстного меню (либо при помощи щелчка по кнопке **New Relationship** панели инструментов).



**Рис. 15.7.** Диалоговое окно **Edit Relationship**

Для нашего куба нам нужно создать связи между таблицей `tblFact` и каждой из таблиц измерений (каждым представлением). Создайте дополнительные связи (перечисленные в табл. 15.1), а затем сохраните свои изменения.

Таблица 15.1. Дополнительные связи

| Таблица-источник | Столбец-источник | Таблица-назначение | Столбец-назначение |
|------------------|------------------|--------------------|--------------------|
| tblFact          | ProductId        | vwProducts         | ProductId          |
| tblFact          | PostalCode       | vwGeography        | PostalCode         |
| tblFact          | ProductId        | vwSuppliers        | ProductId          |
| tblFact          | ShipperId        | vwSuppliers        | ShipperId          |

Для vwGeography первичного ключа не существует, и визуальный конструктор спросит у вас, не хотите ли вы определить "логический" первичный ключ. Визуальный конструктор задает этот вопрос потому, что ни один столбец данного представления не является первичным ключом в той физической таблице, из которой оно получено. Выберите вариант **Yes**.

Визуальный конструктор представлений источников данных позволяет вам добавлять на диаграммы таблицы, связи и так называемые именованные запросы (и удалять их). Именованные запросы очень похожи на представления баз данных, но они сохраняются в ваших представлениях источников данных (а не в базе данных схемы "звезда"). Щелчок правой кнопкой мыши по таблице (либо в панели макета диаграммы, либо в списке **Tables**) позволяет вам добавить именованное вычисление. Это по существу то же самое, что и добавление вычисляемого поля в представлении, но оно опять-таки сохраняется в представлении источника данных (а не в базе данных источника). Вы можете также просмотреть данные в любой таблице вашего представления источника данных и выполнить для нее анализ PivotTable, Chart и PivotChart (для этого надо щелкнуть по ней правой кнопкой мыши в панели макета диаграммы или списке **Tables** и выбрать пункт **Explore Data** контекстного меню, либо выбрать **Data Source View | Explore Data**).

Кнопка **Find Table** панели инструментов визуального конструктора представлений источников данных (вторая справа), пункты меню **Data Source View | Find Table**, пункт **Find Table** контекстного меню панели макета диаграммы — все они позволяют вам найти и выбрать конкретную таблицу (посредством выбора ее из списка в диалоговом окне **Find Table**). То же самое можно сделать и при помощи выбора нужной таблицы в панели **Tables** — все это несколько избыточно. Кнопка **Zoom** панели инструментов и соответствующая ей кнопка выпадающего списка (крайние справа) позволяют вам изменить масштаб диаграммы. Кнопка **Refresh Data Source View** панели инст-

рументов, опции главного меню **Data Source View | Refresh** и пункт **Refresh** в контекстном меню панели макета диаграммы — все они позволяют вам обновить представление источника данных, и это отличный способ обновить схемы всех таблиц источника данных (если они были модифицированы в базе данных источника) без необходимости удаления представления источника данных (или даже отдельных таблиц внутри него) и перестроения его (или добавления таблиц обратно).

## Создание куба при помощи мастера Cube Wizard

После того как вы закончите совершенствование вашего представления источника данных, закройте его и сохраните изменения. Теперь вы готовы создавать свой куб! Для этого выберите **Project | New Cube** или щелкните правой кнопкой мыши по каталогу **Cubes** в **Solution Explorer**, а затем выберите пункт **New Cube** в контекстном меню.

На странице **Welcome** нажмите кнопку **Next** для перехода к странице **Select Creation Method**. В нашем случае вы выбираете вариант **Use Existing Tables** (который должен быть выбран по умолчанию). Нажмите кнопку **Next** для перехода к странице **Select Measure Group Tables**. Убедитесь, что в выпадающем списке **Data Source View** выбран созданный вами ранее источник данных (**Northwind**), после чего установите флажок у **tblFact** в списке **Measure Group Tables** (вообще здесь можно использовать кнопку **Suggest**, если вы не знаете, какая таблица является таблицей фактов), а затем опять нажмите кнопку **Next**.

На странице **Select Measures** вы получите запрос на создание мер и групп мер. По умолчанию здесь показано по одной мере для каждого числового поля вашей таблицы (или таблиц) фактов (которое не является внешним ключом для таблицы измерений), причем будет также приведена и мера по умолчанию (в нашем случае это **tbl Fact Count**), которая зависит от количества строк. Основанные на полях меры будут иметь такое же название, что и поля (с пробелами в тех местах, где в названиях полей есть внутренние прописные буквы. Например, поле с названием **UnitPrice** создаст меру с названием по умолчанию **Unit Price**).

Вы можете отредактировать название меры (или группы мер), установив фокус ввода на содержащий данное название узел древовидного представления (при помощи мыши или клавиатуры) и нажав затем клавишу **<F2>** (либо по-

вторно щелкнув по узлу, однако не надо щелкать дважды; вы можете также щелкнуть по узлу правой кнопкой мыши). Можно также снять выделение мер (чтобы они не добавлялись в куб). Для нашего примера снимите флажок у меры **Tbl Fact Count** (поскольку она нам не понадобится).

Для каждой указанной на странице **Select Measure Group Tables** таблицы фактов создается одна группа мер. Для нашего примера измените название созданной для **tblFact** группы мер, для этого необходимо отредактировать узел **tblFact** и ввести значение **Main**. Когда вы закончите конфигурирование ваших мер и групп мер, нажмите кнопку **Next**.

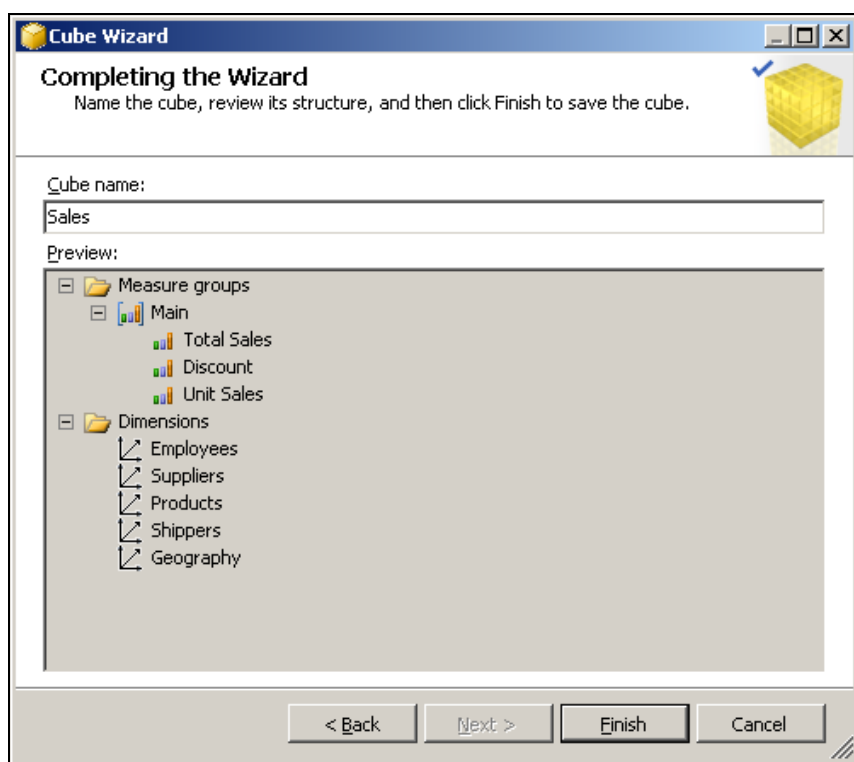


Рис. 15.8. Страница **Completing the Wizard** мастера Cube Wizard

На странице **Select New Dimensions** перечислены все измерения по умолчанию (по одному на каждую таблицу измерений). С отступом под каждым измерением приведено название той таблицы (или представления), по которой оно будет создаваться. Как и с измерениями, вы можете снять выделение лю-

бого измерения (чтобы оно не создавалось). Делается это путем снятия флажка рядом с измерением (или его таблицей измерения). Снятие одного из флажков приводит к снятию обоих флажков. Вы можете изменить название измерения при помощи той же процедуры, которая была описана ранее для мер. Используйте один из этих способов для того, чтобы убрать префикс названия измерения: например, переименуйте измерение `Vw Products` в `Products` и сделайте то же самое для всех остальных измерений.

Когда вы закончите конфигурирование ваших измерений, нажмите кнопку **Next**. На странице **Completing the Wizard** вы можете ввести более подходящее название (чем название по умолчанию). Для нашего примера назовите куб `Sales` (рис. 15.8).

## Использование визуального конструктора Cube Designer

Нажмите кнопку **Finish**. Когда мастер Cube Wizard закончит обработку, то будет сгенерирован и открыт проект вашего куба, при этом вы попадете на вкладку **Cube Structure** визуального конструктора кубов (рис. 15.9).

Конструктор кубов имеет множество вкладок в своей верхней части, а вкладка **Cube Structure** разделена на три панели: слева сверху — **Measures**, слева внизу — **Dimensions**, а справа — **Data Source View**. Панель **Data Source View** изображает то, что является подмножеством таблиц представления источника данных (на котором основан куб), оно содержит только те таблицы и представления, которые были выбраны в мастере Cube Wizard тогда, когда мы указывали таблицу и измерения для нашей группы мер.

По умолчанию панель **Measures** содержит древовидное представление с именем куба в его корне, группами мер в качестве потомков корня и отдельными мерами как потомками своих групп мер. Вы можете изменить древовидное представление на представление в виде сетки (выбрав **Cube | Show Measures In** либо нажав выпадающую кнопку **Show Measures Grid/Show Measures Tree** панели инструментов).

Группы мер и отдельные меры можно добавлять, удалять, переименовывать или перемещать вверх и вниз (при помощи опций главного меню, контекстного меню и панели инструментов). Таким же образом можно переименовать даже сам куб. В представлении сетки перечислены только меры; группами мер можно управлять только в древовидном представлении. Тип данных меры и ее агрегатная функция могут быть изменены в представлении сетки. Эти

(и другие свойства) можно также редактировать и в окне **Properties**. (Мы подробно опишем эти расширенные настройки свойств в следующей главе.)

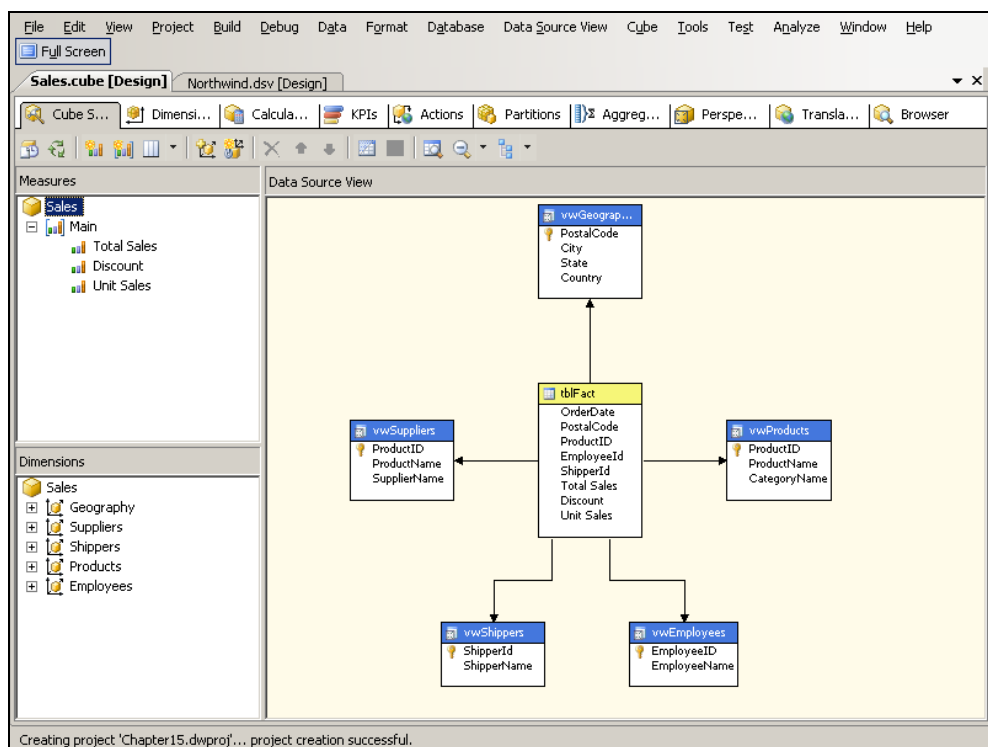


Рис. 15.9. Вкладка **Cube Structure** визуального конструктора кубов

## ДОБАВЛЕНИЕ ТАБЛИЦ В ПАНЕЛЬ DATA SOURCE VIEW

В панель **Data Source View** вы можете добавить любую таблицу, которая присутствует в представлении источника данных. Для этого надо просто выбрать **Data Source View | Show Tables**; либо щелкнуть правой кнопкой мыши в панели **Data Source View**, а затем выбрать пункт **Show Tables** контекстного меню; либо нажать кнопку **Show Table** панели инструментов.

Вы можете также добавить таблицы, которые связаны с определенной таблицей. Для этого необходимо щелкнуть по таблице и выбрать **Data Source View | Show Related Tables**, либо щелкнуть правой кнопкой мыши по таблице и выбрать пункт **Show Related Tables** контекстного меню.

Некоторые кубы используют большое количество таблиц. Если представление загромождено таблицами, то для более ясной картины можно скрыть



отдельные таблицы в панели Data Source View. Для этого необходимо выделить таблицу, а затем выбрать **Data Source View | Hide Table** (либо выбрать пункт контекстного меню **Hide Table**; либо нажать кнопку панели инструментов **Hide Table**).

Вы можете вернуться к показу только тех таблиц, которые использовались для групп мер и измерений. Для этого необходимо использовать пункт **Show Only Used Tables** контекстного меню панели **Data Source View**; либо выбрать **Data Source View | Show Only Used Tables**. Вы можете также показать все таблицы (для этого необходимо использовать пункт контекстного меню **Show All Tables** панели **Data Source View**, либо выбрать **Data Source View | Show All Tables**).

Все эти действия не оказывают никакого воздействия на само представление источника данных (поскольку Data Source View — это просто удобная отфильтрованная диаграмма, основанная на представлении источника данных куба).

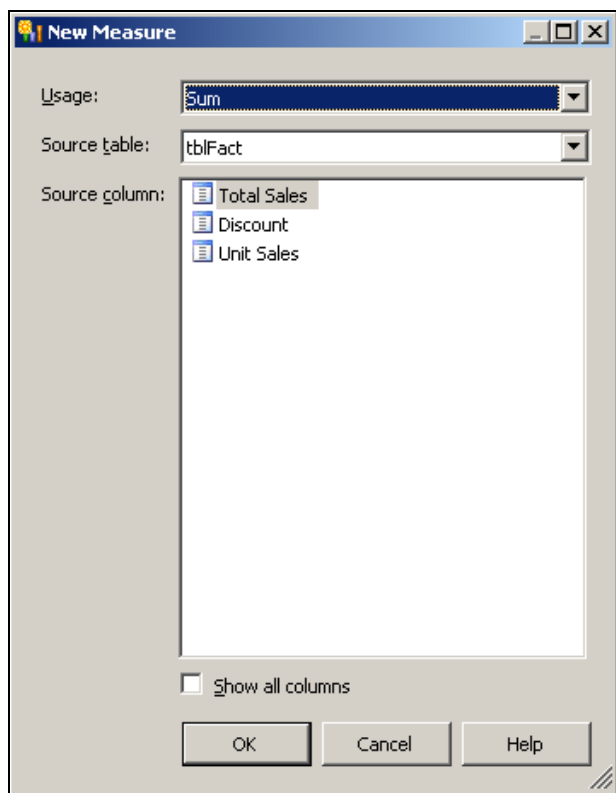
### ПРИМЕЧАНИЕ

Опция **New Linked Object** в меню **Cube**, пункт **New Linked Object** контекстного меню (имеется в панелях **Measures** и **Dimensions**), а также кнопка **New Linked Object** панели инструментов позволяют вам привязать (или импортировать) меры, измерения и прочие объекты из других кубов в редактируемый вами куб. Несмотря на то что мы не будем рассматривать в этой книге связанные объекты, они являются полезным инструментом, который позволяет вам избежать построения новых физических кубов в тех случаях, когда вы хотите скомбинировать элементы других кубов (включая кубы других баз данных Analysis Services и даже те, которые находятся на других серверах). Дополнительную информацию по этой функциональной возможности вы найдете в документации SQL Server Books Online.

Визуальный конструктор кубов — это основной инструмент проектирования и построения кубов; мастер Cube Wizard просто является интерфейсной частью конструктора, которая помогает вам начать работу. Если после выполнения мастера вы хотите добавить в куб новые меры, то вы легко можете сделать это в визуальном конструкторе. Выберите **Cube | New Measure** в главном меню (либо выберите пункт **New Measure** в контекстном меню панели **Measures**, либо нажмите кнопку **New Measure** панели инструментов) для того, чтобы отобразить диалоговое окно **New Measure** (рис. 15.10).

В списке **Source column** (который занимает большую часть диалогового окна) вы можете выбрать то поле таблицы фактов, по которому вы хотите вводить вашу меру, а затем нажать кнопку **ОК** для создания меры. По умолчанию отображаются только те числовые поля, которые не являются

внешними ключами; флажок **Show all columns** под списком **Source column** позволяет вам показать (и выбрать) остальные поля.



**Рис. 15.10.** Диалоговое окно **New Measure**, которое позволяет вам добавлять меры в ваш куб даже после завершения работы мастера Cube Wizard

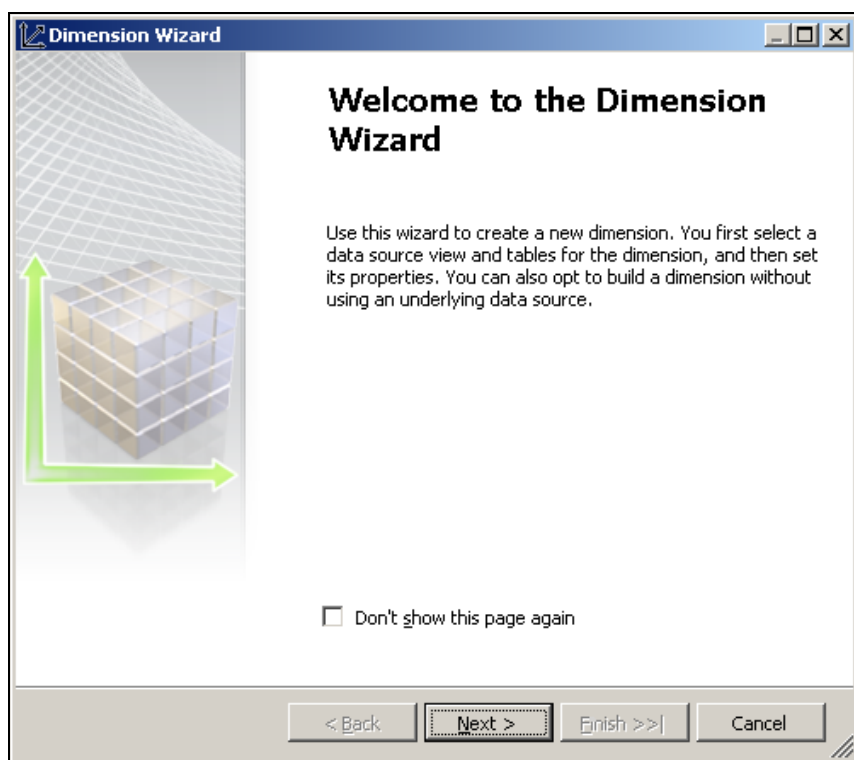
Если существуют таблицы фактов, которые еще не были присвоены существующим группам мер, вы можете добавить также и группы мер. Это более сложная процедура, которую мы в этой книге рассматривать не будем.

Панель **Dimensions** на вкладке **Cube Structure** позволяет вам изучать и обслуживать измерения, атрибуты и иерархии. Она содержит древовидное представление структуры измерений вашего куба (куб является корнем этого дерева, а измерения — потомками куба). Под каждым измерением имеется каталог **Attributes**, в котором перечислены атрибуты измерения; его иерархии (если они есть) перечислены перед каталогом **Attributes**. Вы можете от-

редактировать название измерения, удалить измерение, переместить его вверх или вниз в списке измерений. Вы можете также добавить новое измерение куба или отредактировать существующее измерение в визуальном конструкторе измерений.

## Использование мастера измерений Dimension Wizard

Для того чтобы создать измерение (а нам придется это сделать, так как мы еще не создали измерение времени), выберите в главном меню **Project | New Dimension** (либо щелкните правой кнопкой мыши по каталогу **Dimensions** в окне **Solution Explorer**, а затем выберите пункт **New Dimension** контекстного меню). Любое из этих действий откроет мастер измерений Dimension Wizard (рис. 15.11).



**Рис. 15.11.** Мастер измерений Dimension Wizard, который позволяет определить новое измерение в базе Analysis Services

На странице **Welcome** мастера Dimension Wizard нажмите кнопку **Next**. На странице **Select Creation Method** нажмите **Generate A Time Table In The Data Source**, а затем нажмите кнопку **Next**. На странице **Define Time Periods** введите July 4, 1996 в элементе управления **First Calendar Date** и May 6, 1998 в элементе управления **Last Calendar Day**, а затем убедитесь, что в выпадающем списке **First Day Of The Week** выбрано **Sunday**. В списке **Time Periods** убедитесь, что период **Date** уже выбран; вручную установите флажки **Year**, **Quarter** и **Month**; затем нажмите кнопку **Next**. На странице **Select Calendars** убедитесь, что установлен флажок **Regular Calendar**, а затем нажмите кнопку **Next**. Убедитесь, что страница **Completing the Wizard** выглядит так, как показано на рис. 15.12.



**Рис. 15.12.** Страница **Completing the Wizard** мастера Dimension Wizard (для нашего нового измерения Time)

(Для этого вам придется прокрутить вниз представление **Preview** или изменить размер **Dimension Wizard**, чтобы увидеть все подробности). Кнопку **Finish** пока не нажимайте.

Перед тем как продолжить, давайте посмотрим, что мы делаем в мастере **Dimension Wizard**. Во-первых, мы создаем измерение **Time** с атрибутами **Year**, **Quarter**, **Month** и **Date**; а также иерархию, состоящую из этих атрибутов (именно в таком порядке). (Более подробно мы опишем иерархии в следующем разделе). Мы создаем также атрибуты (подобные **Day Of Quarter**), которые дадут нам новую информацию (если она нам понадобится). Но мы делаем еще одну вещь, которая очень важна: мы создаем таблицу, которая будет служить основой этого измерения, и заполняем ее данными. Для того чтобы правильно выполнить эту задачу, установите флажок **Generate Schema Now** в нижней левой части страницы **Completing the Wizard Page**, после чего нажмите кнопку **Finish**.

Появляется мастер **Schema Generation Wizard**. На странице **Welcome** нажмите кнопку **Next**. На странице **Specify Target** выберите опцию **Use Existing Data Source View** (чтобы использовать представление источника данных **Northwind**), нажмите кнопку **Next** на этой и всех остальных страницах, а затем нажмите кнопку **Finish**. Теперь должно появиться диалоговое окно **Schema Generation Progress**; подождите завершения генерирования схемы — в строке состояния появится сообщение "Generation Completed Successfully" (зелеными буквами) и станет активной кнопка **Close**.

### СОВЕТ

Если вы попытаетесь создать измерение **Time** вторично, то вместо мастера **Schema Generation Wizard** появится диалоговое окно **Schema Generation**. Здесь вы сможете нажать кнопку **OK** и миновать описанные ранее шаги мастера. Вы можете также нажать кнопку **Change Options** для того, чтобы попасть в **Schema Generation Wizard**.

### ПРИМЕЧАНИЕ

Возможно, что в диалоговом окне **Schema Generation Progress** вы заметите несколько предупреждающих сообщений, что для других измерений и самого куба таблицы не генерируются (поскольку они основаны на уже существующих таблицах).

Нажмите кнопку **Close**, после чего появится визуальный конструктор измерений для вашего нового измерения **Time**. Закройте окно визуального конструктора измерений (нажатием на значок закрытия или комбинацией клавиш

<Ctrl>+<F4>). В ответ на подсказку сохранения изменений нажмите кнопку **Save**. Вы заметите, что в визуальном конструкторе кубов новое измерение **Time** не появилось. Это потому, что вы добавили его в базу данных **Analysis Services**, но не в сам куб. Перед тем как добавить измерение в куб, нам нужно убедиться, что его таблица измерений была правильно добавлена в наше представление источника данных.

Откройте или активируйте представление источника данных **Northwind** в визуальном конструкторе представлений источников данных. Вы должны увидеть появившуюся новую таблицу **Time**, но вы должны также заметить и то, что она не связана должным образом с таблицей фактов. Для исправления этого щелкните по столбцу **OrderDate** таблицы **tblFact** и перетащите его на столбец **PK\_Date** таблицы **Time**. Теперь сохраните изменения в представлении источника данных.

В визуальном конструкторе куба для добавления нового измерения **Time** выберите **Cube | Add Cube Dimension** (либо нажмите кнопку **Add Cube Dimension** панели инструментов). В появившемся диалоговом окне **Add Cube Dimension** выберите в списке **Select Dimension** измерение **Time**, а затем нажмите кнопку **OK**. Когда вы вернетесь в визуальный конструктор кубов, то вы заметите, что в древовидное представление панели **Dimensions** добавилось измерение **Time** и что в панели **Data Source View** добавилась таблица **Time** (чтобы схема легко читалась, вам может понадобиться настроить размещение таблицы, для этого используйте пункт **Arrange Tables** контекстного меню панели **Data Source View**).

Если вы начнете выполнять детализацию по новому измерению **Time** (или по любому другому), то вы увидите, что первый дочерний узел — это ссылка, которая открывает визуальный конструктор измерений и позволяет вам редактировать данное измерение. Помимо кнопки ссылки, вы можете открыть визуальный конструктор измерений путем выделения измерения в панели **Dimensions** и выбора **Cube | Edit Dimension** (либо пункта **Edit Dimension** контекстного меню панели). Используйте любой из этих методов для того, чтобы открыть измерение **Geography** в визуальном конструкторе измерений.

## Использование визуального конструктора измерений **Dimension Designer**

Визуальный конструктор измерений (как и визуальный конструктор кубов) имеет вкладки в верхней части и три панели. На вкладке **Dimension Structure**

слева находится панель **Attributes**, в центре — панель **Hierarchies**, а справа — панель **Data Source View**.

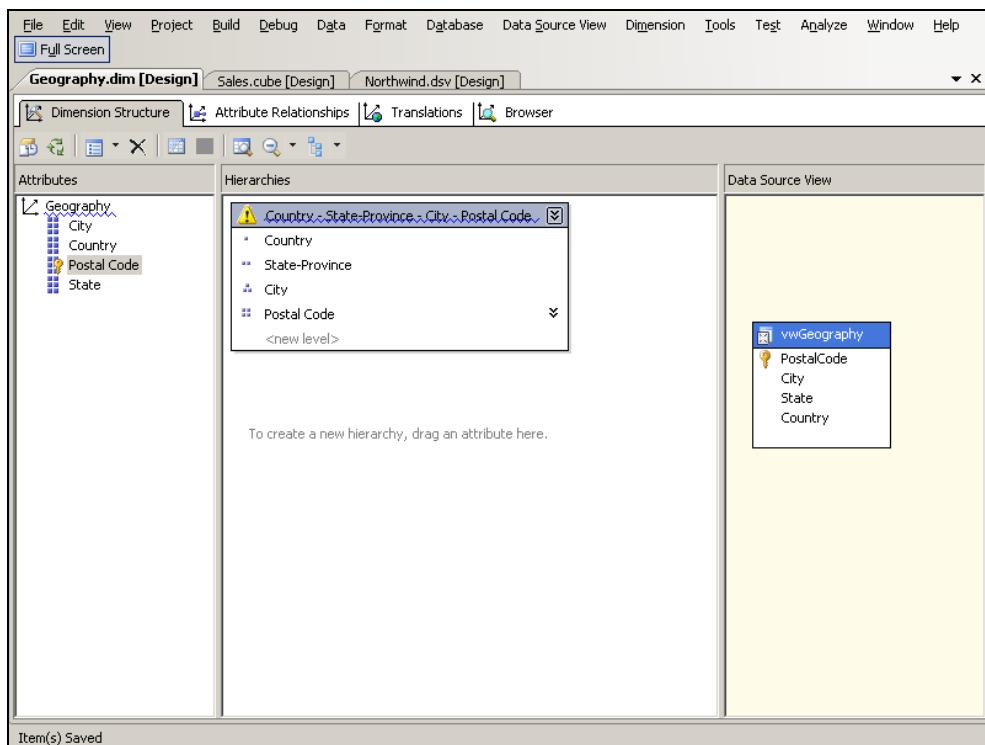
Атрибуты могут отображаться в виде дерева, списка или сеточного представления. Изменять представления можно при помощи выбора в главном меню **Dimension | Show Attributes** либо посредством выбора пункта **Show Attributes** контекстного меню панели **Attributes**. Вы можете также нажать связанную с кнопкой **View Attributes As** кнопку выпадающего списка (третья кнопка слева на панели инструментов визуального конструктора измерений). Обратите внимание, что когда вы переходите на представление в виде списка или сетки, то панель **Hierarchies** смещается из центра в верхний левый угол, а панель **Attributes** — в левый нижний угол. В сеточном представлении можно получить доступ только к некоторым свойствам атрибутов; однако через окно **Properties** доступны все свойства (независимо от того представления, в котором вы находитесь). Мы подробно обсудим окно **Properties** чуть позже.

Обратите внимание, что в панели **Attributes** имеется только почтовый код, но в панели **Data Source View** видны столбцы *City*, *State* и *Country* таблицы измерения. Для создания атрибутов из этих трех столбцов просто перетащите каждый столбец из панели **Data Source View** в панель **Attributes**.

Визуальный конструктор измерений облегчает создание иерархий из множества атрибутов измерений. Давайте создадим иерархию для измерения *Geography*, чтобы мы смогли запрашивать данные нашего куба иерархическим образом (по стране, затем по штату, потом по городу, а затем по почтовому коду). Начнем с перетаскивания атрибута *Country* из панели **Attributes** в панель **Hierarchies**. (Вы можете также щелкнуть правой кнопкой мыши по атрибуту *Country* и выбрать пункт контекстного меню **Start New Hierarchy** либо щелкнуть левой кнопкой мыши по атрибуту *Country* и выбрать **Dimension | Start New Hierarchy**). После этого будет создана новая иерархия с названием по умолчанию *Hierarchy* (в своем собственном прямоугольнике в панели **Hierarchies**, с атрибутом *Country* в качестве верхнего и единственного уровня иерархии).

Для создания дополнительных уровней вы можете перетащить в этот прямоугольник дополнительные атрибуты (или выбрать иерархию, а затем либо щелкнуть правой кнопкой мыши по атрибуту и выбрать пункт **Create Level** контекстного меню, либо щелкнуть по атрибуту левой кнопкой мыши и выбрать пункт **Create Level** меню **Dimension**). Вы можете изменить порядок уровней в иерархии при помощи перетаскивания; а также удалять и переименовывать атрибуты и иерархии (аналогично тому, как это уже описывалось).

При помощи одного из описанных способов добавьте атрибуты *State*, *City* и *Postal Code* как второй, третий и четвертый уровни новой иерархии (оставьте *Country* в качестве первого уровня). После добавления всех уровней переименуйте второй уровень из *State* в *State-Province*, а саму иерархию — в *Country-State-Province-City-Postal Code*, а затем сохраните ваши изменения. (В ответ на запрос о сохранении куба *Sales* нажмите кнопку **ОК**.) Законченный дизайн иерархии показан на рис. 15.13. Пока вы можете без всяких опасений проигнорировать волнистые линии и значок с желтым восклицательным знаком. Мы подробнее объясним их в следующей главе.



**Рис. 15.13.** Визуальный конструктор измерений с нашим полностью спроектированным и сконфигурированным измерением *Geography* (включая его новую иерархию)

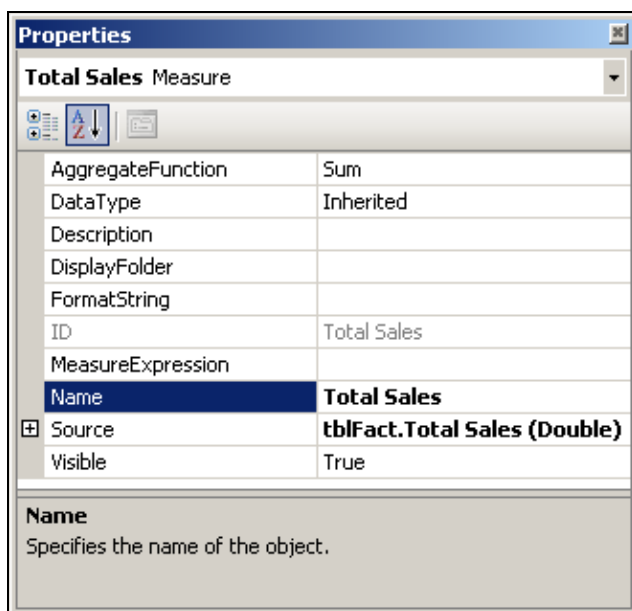
Визуальный конструктор измерений имеет еще три вкладки: **Attribute Relationships**, **Translations** и **Browser**. В следующей главе мы опишем вкладки **Attribute Relationships** и **Translations**. Для изучения всех уровней



измерения и элементов каждого уровня вы можете воспользоваться вкладкой **Browser**, но сделать это можно будет только после обработки куба (очень скоро мы вас с этим познакомим).

## Работаем с окном свойств *Properties* и *Solution Explorer*

Некоторые из уже обсуждавшихся нами настроек (а также и те, которые мы будем обсуждать в следующей главе) можно сделать в окне **Properties**. Для этого вернемся на вкладку **Cube Structure** визуального конструктора кубов, щелкнем по любой мере в окне **Measures**, а затем посмотрим в окно свойств **Properties**. (Если окно **Properties** не видно, то нужно нажать клавишу <F4> или выбрать **View | Properties Window**). Для того чтобы окно **Properties** было легче просматривать, дважды щелкните по его заголовку (чтобы открепить его), отцентрируйте его на экране, а затем измените его размер (чтобы столбцы были широкими и было видно много строк без необходимости прокручивать). Ваше окно **Properties** теперь должно выглядеть, как показано на рис. 15.14.



**Рис. 15.14.** Окно **Properties**, которое позволяет производить сложные манипуляции с мерами, измерениями, кубами и прочими объектами

Обратите внимание, что `AggregateFunction` и `DataType` видны, и их можно редактировать (как и несколько прочих свойств). Щелкните по выпадающей стрелке в верхнем правом углу окна **Properties**, и вы увидите все меры вашего куба в выпадающем списке. Дважды щелкните по заголовку окна **Properties** для того, чтобы закрепить его, а затем выберите в панели **Dimensions** какое-нибудь измерение. Еще раз открепите окно **Properties**, и вы увидите, что свойства этого измерения теперь можно редактировать, и все измерения куба перечислены в выпадающем списке в верхней части окна. Этот способ работает для любого класса объектов вашего куба, в любой панели и на любой вкладке визуального конструктора кубов (кроме вкладок **Calculations** и **Browser**) или конструктора измерений. Поэкспериментируйте с этим способом в различных частях визуальных конструкторов (чтобы вы смогли оценить всю мощь, которую предоставляет окно **Properties** для процесса редактирования куба).

Перед тем как перейти к обработке куба и запросам к нему, заметим, что вы можете манипулировать объектами высокого уровня (такими как источники данных, представления источников данных, измерения и сам куб) при помощи открывания их из окна **Solution Explorer**. Например, для редактирования измерения в визуальном конструкторе измерений вместо того, чтобы нажимать ссылку на него в панели **Dimensions** визуального конструктора кубов, вы можете дважды щелкнуть по его узлу в каталоге **Dimensions** проекта в окне **Solution Explorer**. Попробуйте и убедитесь, что это отлично работает. Если вы щелкнете правой кнопкой мыши по любому редактируемому объекту в окне **Solution Explorer**, а затем выберете **View Code**, то вы увидите, что основой каждого объекта является простой XML-файл. Представьте себе все те возможности по проектированию и созданию кубов, которые это дает пользовательским приложениям сторонних компаний.

### ПРИМЕЧАНИЕ

Используемая в визуальных конструкторах схема XML является стандартом, который называется XML for Analysis (XMLA). В следующих трех главах (особенно в *главе 17*) мы более детально изучим структуру и использование XMLA. Мы опишем также, как различные интерфейсы прикладного программирования Analysis Services используют XMLA "за кулисами".

## Обработка куба

Для обработки куба вы можете выбрать **Database | Process** или **Cube | Process** (либо нажать кнопку **Process** панели инструментов визуально-

го конструктора кубов). Обработка куба или базы данных автоматически обрабатывает все ожидающие обработки изменения в измерениях.

Для обработки куба необходимо собрать и развернуть ваш проект Analysis Services. Если вы решите обработать свой куб (сделайте это сейчас), а он еще не был построен и развернут (а наш куб именно в таком состоянии), то вы получите сообщение о том, что необходимо выполнить эти действия и запрос на выполнение перед обработкой. Нажмите **Yes**. (Вы могли бы также выполнить шаги **Build** и **Deploy** из подменю **Build** главного меню.)

### ПРИМЕЧАНИЕ

При развертывании ваших изменений вы могли заметить несколько предупреждений в окне **Error List**. Это следствие Best Practice Alerts (BPA), данную функцию мы будем обсуждать в *главе 16*. Это не ошибки, а всего лишь предупреждения, которые не повлияют на процессы сборки и развертывания (поэтому вы можете проигнорировать их).

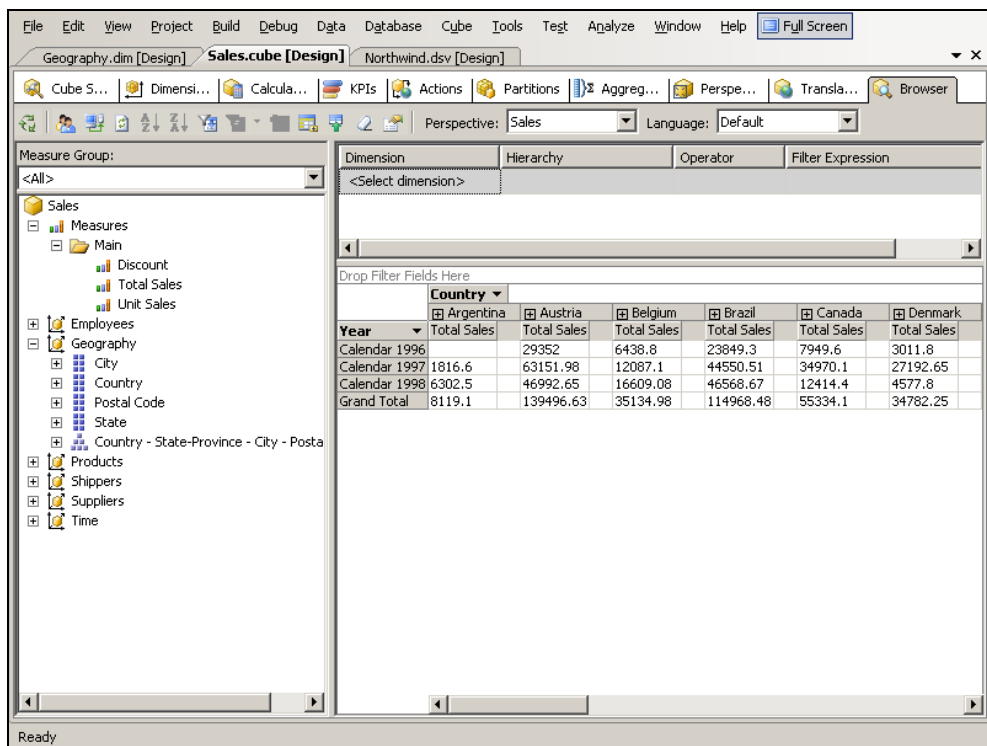
Вы можете наблюдать за продвижением операций сборки и развертывания в окне **Deployment Progress**. По завершении этих операций вы получите диалоговое окно с названием куба. Кнопки **Remove**, **Impact Analysis** и **Change Settings** этого диалогового окна дают вам возможность точного управления. Мы советуем вам изучить возможности этих инструментов по документации SQL Server Books Online. Для нашего примера достаточно просто нажать **Run**. При этом произойдет обработка куба и будет показано диалоговое окно **Process Progress**. После завершения обработки нажмите **Close** в обоих диалоговых окнах — и ваш куб готов к запросам!

## Выполнение запросов

В *главе 17* мы расскажем, как писать .NET-код для запросов ваших приложений к кубу. Пока же для выполнения некоторых быстрых запросов мы будем использовать вкладку **Browser** (в визуальном конструкторе кубов) — это последняя вкладка. Браузер куба — это просто административный инструмент, который ограничен результирующими наборами с двумя осями. По своей функциональности он аналогичен инструменту Office Excel PivotTable и фактически использует элемент управления PivotTable из Office Web Components версии 11.

После загрузки браузера углубитесь в узел **Measures**, а затем в узел **Main measure group** в верхней левой части древовидного представления. Перетащите меру Total Sales в зону **Drop Totals Or Detail Fields Here** области вы-

вода. Затем перетащите атрибут Year измерения Time (углубитесь в измерение для показа его атрибутов) в зону **Drop Row Fields Here**. Теперь вы увидите объемы продаж по годам из базы данных Northwind. Теперь углубитесь в измерение Geography. Перетащите всю иерархию Country-State-Province-City-Postal Code в зону **Drop Column Fields Here**. Теперь вы должны увидеть те же самые данные в корреляции по оси столбцов уровня Country иерархии измерения Geography (рис. 15.15). Вы можете детализировать по любой стране и видеть ее данные с разбивкой по штатам, можете детализировать даже дальше (до городов или почтовых индексов). Обратите внимание на то, как быстро выполняются детализации.



**Рис. 15.15.** Вкладка **Browser** визуального конструктора кубов, которая позволяет выполнять очень сложные запросы к кубам OLAP

Теперь вы можете перетащить дополнительные измерения на любую ось (по желанию можете сначала удалить старые измерения). Размещение двух измерений на одной оси позволит вам углубляться по одному измерению с тем, чтобы выявить элементы другого измерения. Это удобный способ проециро-

вания нескольких логических осей на одну физическую ось. Выполнение запросов, которые по-настоящему проецируют на три или более осей, вполне возможно, но не на вкладке **Browser** визуального конструктора кубов.

Вы можете также добавить в запрос фильтры. Это делается при помощи перетаскивания атрибута или иерархии в зону **Drop Filter Fields Here**, следующего нажатия выпадающей стрелки рядом с названием атрибута или иерархии, а затем установки или снятия флажков элементов, которые будут учитываться или игнорироваться в данных. Вы можете также создать фильтры в виде выражений (в области непосредственно над зоной **Drop Filter Fields Here**). Здесь вы можете ввести простые условные выражения или сложные выражения MDX для фильтрации данных. Мы опишем создание выражений MDX в следующей главе.

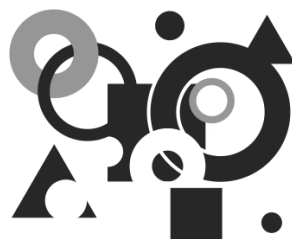
Другие инструменты для запросов и редактирования (компании Microsoft и сторонних организаций) позволяют вам делать в плане составления запросов еще больше. Вы можете также писать свои собственные приложения, которые превосходят вкладку **Browser** по своей функциональности. Однако с учетом того, что это первая глава, посвященная OLAP в этой книге, мы уже сделали достаточно много.

## Резюме

В этой главе мы охватили очень много. Вы изучили основные положения и словарь OLAP, и в том числе все принципы, необходимые для понимания того, как проектировать хранилище данных (со схемой звезда/снежинка) и основанные на нем кубы. В дополнение к этой теории вы изучили практические навыки создания проекта Analysis Services в Visual Studio и добавления в него источника данных, представления источника данных, куба и измерения времени. Вы научились создавать и обслуживать меры и измерения, а также обрабатывать куб и делать к нему запросы.

В единственной этой главе вы научились использовать Data Source Wizard, Data Source View Wizard, Cube Wizard и Dimension Wizard, а также визуальные конструкторы кубов (вкладки **Cube Structure** и **Browser**) и измерений (вкладки **Dimension Structure** и **Browser**). Теперь у вас достаточно знаний для построения простых, но чрезвычайно полезных кубов OLAP по вашим данным транзакций. В следующих двух главах мы изучим реализацию более сложных функциональных возможностей ваших кубов, а затем научимся использовать Office Excel и Excel Services для запросов, а также писать .NET-код для встраивания функционала запросов OLAP в ваши приложения.

## ГЛАВА 16



# Расширенный OLAP

*Эндрю Брест*

В предыдущей главе вы изучили базовые концепции, словарь оперативной аналитической обработки (OLAP) и узнали, как создать свой собственный код на основе базы данных Northwind. Построение куба "с нуля" дает вам такое понимание OLAP, которого трудно было бы достигнуть простым изучением куба Adventure Works, поставляемого в составе Microsoft SQL Server 2008. После того как вы прошли через описанный в *главе 15* процесс создания таблиц фактов и измерений, а также через проектирование и создание куба и запросов к нему (в браузере кубов), у вас должно было появиться чувство относительной простоты и мощи OLAP. В результате вы должны были расстаться с тем трепетом, который многие люди испытывают при изучении OLAP.

Большинство описанных нами в *главе 15* функций были фактически доступны не только в SQL Server 2005 Analysis Services, но уже и в SQL Server 2000 Analysis Services. Способность создать версию вашей базы данных при помощи таблицы фактов и нескольких таблиц измерений, а затем выполнять быстрые запросы детализации — это уже давно не новость. Фактически это можно было делать уже с момента появления служб SQL Server 7.0 OLAP Services. В Analysis Services 2005 было введено (а в Analysis Services 2008 расширено) множество новых функциональных возможностей OLAP, которые продолжают революционизировать рынок средств бизнес-аналитики. Эти функции облегчают построение, запросы и обновление кубов и позволяют кубам отслеживать такую информацию, которая гораздо важнее для бизнес-пользователей, чем в более ранних версиях данного продукта. В этой главе мы опишем некоторые из этих функциональных возможностей.

## **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

В состав примеров кодов этой книги входят два решения Microsoft Visual Studio: Chapter16Begin.sln и Chapter16End.sln (оба в каталоге данной главы). Chapter16Begin.sln — это фактически клон примеров кодов Visual Studio главы 15, однако проект и базы данных переименованы в Chapter16. Если вы хотите следить за нашим изложением, то должны открыть Chapter16Begin.sln и работать в нем. Chapter16End.sln содержит все в уже реализованном виде. Используйте этот код в том случае, если нужно взглянуть на законченную версию проекта. Решение SQL Server Management Studio со скриптами SQL и MDX имеется в виде Chapter16.smsssln в каталоге SSMS данной главы.

## **Что мы расскажем в этой главе**

В данной главе мы расширим наш куб Northwind для применения измерения родитель/потомок, измерений фактов, ролевых измерений, расширенных настроек свойств мер и измерений (включая связи атрибутов), группирования элементов (дискретизации), каталогов отображения, вычисляемых элементов, именованных наборов, действий, ключевых индикаторов производительности, перспектив, переводов, мы рассмотрим также работу ролей безопасности. В следующей главе мы покажем вам, как предоставлять многие из этих функций в ваших собственных приложениях при помощи ADO MD.NET и прочих интерфейсов прикладного программирования OLAP.

Мы обсудим секционирование, варианты хранения, проектирование агрегатов и упреждающее кэширование. Оно является самым значительным введенным в Analysis Services 2005 достижением, которое продолжает сохранять свою важность и в Analysis Services 2008.

### **ПРИМЕЧАНИЕ**

Вот некоторые не описываемые нами здесь концепции и функциональные возможности, которые вы можете изучить самостоятельно: связанные объекты, неоднородные (несбалансированные) иерархии, расширенные опции обработки, шаблоны MDX в визуальном конструкторе кубов, множественные таблицы фактов (со ссылочными измерениями и измерениями "многие-ко-многим"), а также функция Add Business Intelligence визуальных конструкторов аналитических служб Analysis Services.

## MDX в контексте

Мы будем описывать функциональные возможности Analysis Services, где требуется написание выражений на языке многомерных выражений (multidimensional expression language, MDX). Это создает проблему, поскольку мы в данной книге даем лишь самое общее введение в MDX (и этот материал находится в следующей главе). Однако использованный в этой главе код MDX должен быть вполне понятен в контексте изложения (с учетом описания OLAP и вашего опыта в качестве разработчика). Более основательное изложение MDX мы дадим в *главе 17*. После чтения той главы вы можете вернуться к кодам MDX этой главы (чтобы углубить свое понимание представленных здесь примеров).

## А теперь несколько слов о нашем спонсоре

Поставляемая вместе с SQL Server 2008 документация Books Online (и в том числе ее описание аналитических служб) превосходна. Наша книга предназначена служить дополнением к материалам Books Online, облегчать восприятие и давать альтернативные примеры для подкрепления некоторых концепций. Точно так же и наша база данных Northwind в этих посвященных OLAP главах предназначена для того, чтобы дополнять поставляемый вместе с SQL Server 2008 пример AdventureWorksDW2008.

Мы настоятельно рекомендуем, чтобы вы изучили и пример AdventureWorksDW2008, и учебное пособие по Analysis Services, которое имеется в составе Books Online. Это учебное пособие дает исключительную широту и глубину изложения, а пример куба Adventure Works охватывает буквально все функциональные возможности Analysis Services.

### ПРИМЕЧАНИЕ

Вы найдете учебное пособие Analysis Services в Books Online в системе помощи Visual Studio следующим образом: в панели **Contents** (под SQL Server Books Online) выберите узел **SQL Server Tutorials**. На самой странице системы помощи прокрутите вниз до заголовка Analysis Services — **Multidimensional Data**, а затем щелкните под ним по ссылке **Designing And Implementing: Tutorials (Analysis Services—Multidimensional Data)**. И наконец, на появившейся странице щелкните по ссылке **SQL Server Analysis Services Tutorial**.



В то же время мы рекомендуем вам для начала прочитать именно эту главу, поскольку она может помочь вам быстро начать работать. Учебное пособие Books Online и его образцовый куб более последовательны и формальны, поэтому к ним имеет смысл обратиться уже после прочтения обсуждения расширенных функций OLAP в этой главе. Мы начнем с использования расширенных функциональных возможностей измерений и мер в Analysis Services 2008.

## Расширенные измерения и меры

Вы изучили использование мастера Cube Wizard для построения нескольких измерений нашего куба Sales. Мастер Cube Wizard дает удобную возможность строить несколько измерений сразу, но он не дает доступа ко всем функциональным возможностям, которые вы могли бы встроить в ваши измерения.

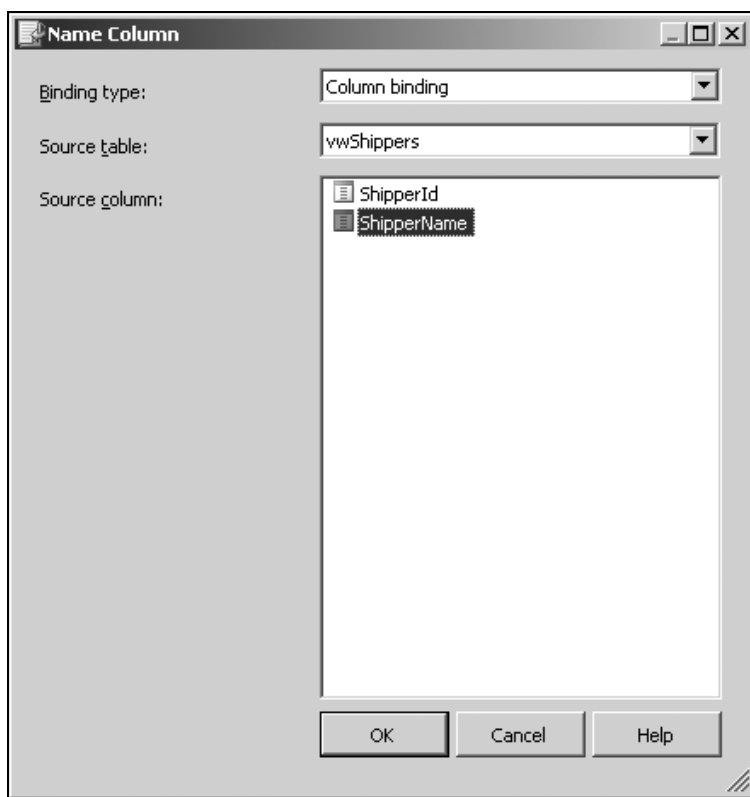
Вы также изучили в *главе 15*, как выйти за пределы Cube Wizard и создать свою собственную иерархию. (Мы сделали это, построив иерархию Country-State-Province-City-Postal Code в измерении Geography при помощи визуального конструктора измерений). Но это было только начало, мы могли бы сделать гораздо больше. Давайте же уточним, что мы построили.

### Ключи и названия

Мы начнем с измерения Shippers. Вспомним, что это измерение плоское (не иерархическое). Атрибут этого измерения отображает не названия, а идентификаторы поставщиков. Это следует исправить. Мы делаем так не просто для элегантности: обсуждая исправление этого недостатка, мы представим некоторые важные концепции, связанные с атрибутами измерения.

Откройте измерение Shippers в визуальном конструкторе измерений. Вы увидите, что для этого измерения определен один атрибут — Shipper Id. Вы также заметите, что в панели **Data Source View** визуального конструктора измерений имеется столбец ShipperName. Может быть очень соблазнительным перетащить этот столбец для создания из него дополнительного атрибута, однако оказывается, что это не нужно. Мы можем скомбинировать два столбца vwShippers.ShipperId и vwShippers.ShipperName в единый атрибут.

Выберите атрибут `Shipper Id` и убедитесь, что окно **Properties** видно. (Если оно не видно, нажмите для его показа клавишу <F4> либо выберите **View | Properties Window**). Посмотрите на настройки значений для свойств `KeyColumns` и `NameColumn` атрибута; они установлены соответственно в `vwShippers.ShipperId` и `(none)`. Посмотрите также на свойство `OrderBy`, и вы увидите, что элементы этого атрибута сконфигурированы на сортировку по их идентификаторам. Этот атрибут измерения наполовину представляет собой то, что нам нужно; его название необходимо изменить на `vwShippers.ShipperName`, а его свойство `OrderBy` необходимо изменить с `Id` на `Name`.



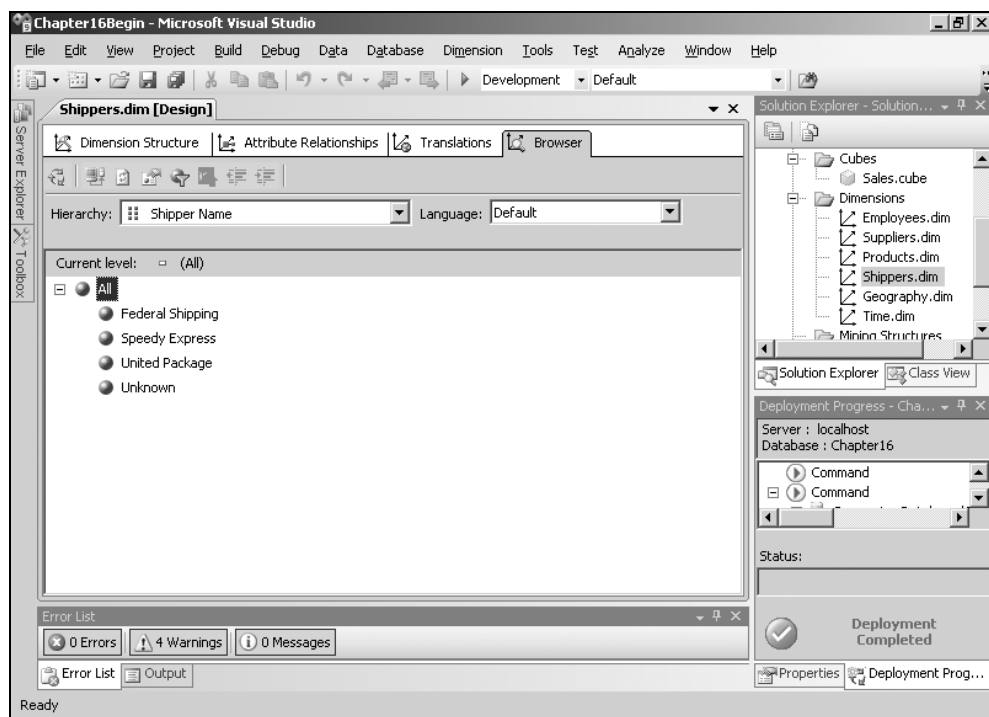
**Рис. 16.1.** Выбор определенного столбца для свойства `NameColumn` атрибута `ShipperName` в диалоговом окне **Name Column**

Для переименования атрибута `Shipper Id` вы можете модифицировать его свойство `Name` в окне **Properties**; вы можете также щелкнуть по атрибуту

правой кнопкой мыши и затем выбрать пункт **Rename** контекстного меню (либо выделить атрибут и нажать клавишу <F2>), а затем отредактировать название прямо на месте. Введите в качестве нового названия Shipper Name.

После переименования атрибута выберите свойство NameColumn, а затем щелкните по кнопке с многоточием. В появившемся диалоге **Name Column** выберите **ShipperName** в списке **Source column** (как показано на рис. 16.1), а затем нажмите кнопку **OK**.

Теперь просто измените значение свойства OrderBy с Key на Name — и мы почти закончили настройку нашего атрибута. Однако перед тем как продолжить, щелкните правой кнопкой мыши по только что переименованному атрибуту Shipper Name. В подменю **Set Attribute Usage** контекстного меню (это альтернативный интерфейс для свойства Usage атрибута) убедитесь, что около опции **Key** выставлен флажок.



**Рис. 16.2.** Изучение переконфигурированного измерения Shippers на вкладке **Browser** визуального конструктора измерений

Мы закончили нашу работу с атрибутом, поэтому теперь пришло время развернуть наши изменения на сервер, а затем использовать вкладку **Browser** визуального конструктора измерений, чтобы убедиться в том, что все выглядит правильно. Нажмите кнопку **Start Debugging** на главной панели инструментов Visual Studio (а не на панели инструментов визуального конструктора измерений), выберите **Debug | Start Debugging** либо нажмите клавишу <F5> (для сохранения изменений и развертывания их на сервер).

### **ВАЖНО**

Вы должны следовать этой процедуре каждый раз, когда мы просим вас сохранить и развернуть ваши изменения (если не указано иное).

После завершения развертывания и обработки щелкните по вкладке **Browser** визуального конструктора измерений (после развертывания вы можете попасть туда автоматически). Углубитесь до узла **All** элемента управления древовидного представления этой вкладки; ваш экран должен выглядеть так, как показано на рис. 16.2.

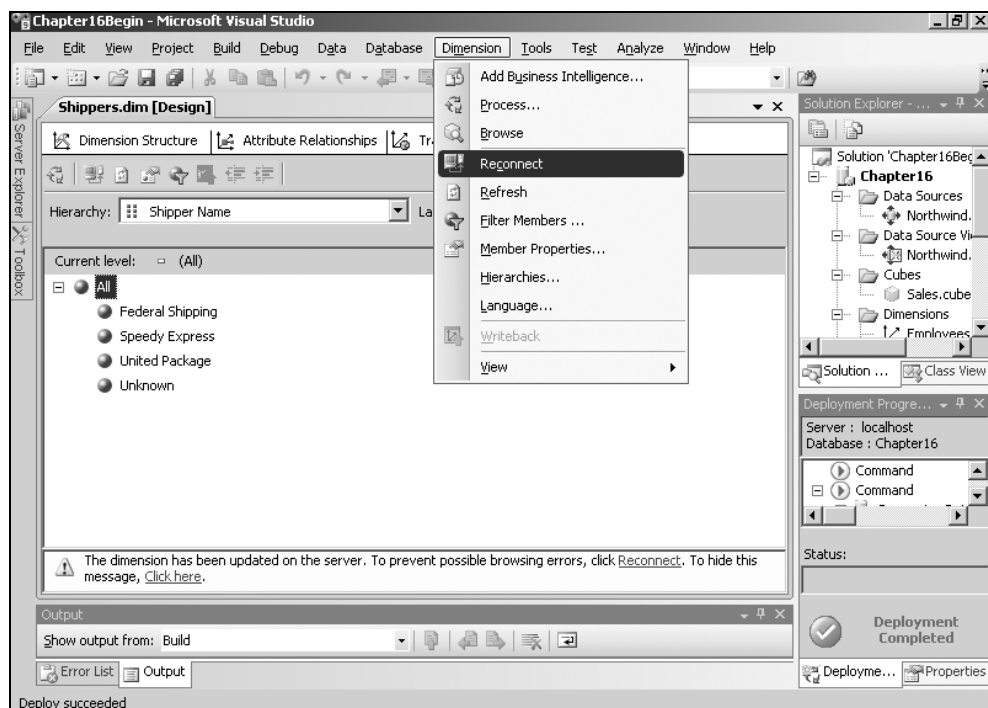
## **Изменение элемента All**

Для того чтобы полностью все организовать, давайте изменим название элемента All измерения Shippers на All Shippers. Для этого вернитесь на вкладку **Dimension Structure** визуального конструктора измерений, выделите корневой узел в древовидном представлении атрибутов (то есть выберите узел, представляющий само измерение) и в окне **Properties** измените значение свойства `AttributeAllMemberName` на `All Shippers`.

Сохраните и разверните ваши изменения. Затем вернитесь на вкладку **Browser** визуального конструктора измерения Shippers. Поскольку структура измерения и куба изменилась, то подключение между браузером и измерением нарушено. Восстановите это подключение посредством нажатия кнопки **Reconnect** (вторая слева) в панели инструментов визуального конструктора (или при помощи выбора **Dimension | Reconnect**, либо посредством нажатия на гиперссылку **Reconnect** в желтой панели ошибок нижней части визуального конструктора (рис. 16.3)).

Название **Shipper Name** должно появиться как иерархия по умолчанию (единственная) в выпадающем списке **Hierarchy** (на вкладке **Browser**), а узел

**All Shippers** — в древовидном представлении браузера (вместо узла **All**, который там был раньше). Если вы раскроете **All Shippers**, то должны будете увидеть имена трех поставщиков и элемент **Unknown** (отсортированные по алфавиту).



**Рис. 16.3.** Браузер измерений  
(так он выглядит в том случае, когда нужно восстановить подключение)

### ПРИМЕЧАНИЕ

Можно привести в порядок и измерение `Employees` (используя такой же подход, который мы применили к `Shippers`). Измерение `Geography` уже было улучшено в главе 15, измерению `Time` изменения не требуются, а измерение `Products` мы скоро приведем в порядок при помощи метода, который можно одинаково успешно применить и к измерению `Suppliers`. В следующем разделе мы добавим новое измерение на основе сотрудников, но оригинальное измерение `Employees` изменять не будем.

## Добавление именованного запроса в представление источника данных

Теперь давайте займемся улучшением других измерений куба и созданием в этом процессе нескольких новых измерений. Давайте начнем с измерения `Employees`. Откройте его при помощи двойного щелчка по его узлу в окне **Solution Explorer**. В панели **Data Source View** визуального конструктора измерений посмотрите на различные поля в `vwEmployees` ("таблице" измерений этого измерения). В представлении имеются только два поля (как и в случае с измерением `Shippers`) — ключ и название. В то же время в исходной таблице имеется еще несколько других полей, одно из которых будет полезно добавить — это столбец `ReportsTo`. Этот столбец содержит ID непосредственного начальника данного сотрудника и таким образом служит внешним ключом самосоединения. Использование этого столбца позволит нам организовать новое измерение родитель/потомок (на основе сотрудников).

Для использования столбца `ReportsTo` мы могли бы просто модифицировать `vwEmployees` в реляционной базе данных и обновить наше представление источника данных. Другой вариант — создать именованный запрос в самом представлении источника данных. Именованный запрос — это по существу представление, но его создание не модифицирует реляционную базу данных. Именованные запросы могут быть единственным возможным вариантом (в том случае, когда вы имеете доступ к базе данных только на чтение). В нашем случае мы сразу же создали представление, так что было бы вполне логично его модифицировать. Однако давайте мы создадим вместо этого именованный запрос, просто чтобы увидеть, как это делается.

Начнем с открытия представления источника данных `Northwind` (дважды щелкнув по его узлу в окне **Solution Explorer**). В панели диаграммы щелкните правой кнопкой мыши по любому свободному месту, а затем выберите пункт **New Named Query** контекстного меню (чтобы открыть диалоговое окно **Create Named Query**). Это диалоговое окно является контейнером для конструктора SQL-запросов (имеющегося в `SQL Server Management Studio`), специализированной панели инструментов, текстовых полей для названия и описания, а также выпадающего списка источников данных. Назовите запрос `Employees` и вставьте в панель **SQL** (большое текстовое поле внизу) следующий текст запроса:

```
SELECT EmployeeID, LastName + ', ' + FirstName AS EmployeeName, ReportsTo
FROM Employees
```

Щелкните внутри любой другой панели (для того, чтобы заставить их обновиться). Должно появиться диалоговое окно (рис. 16.4). Нажмите кнопку **ОК**, а затем сохраните изменения, сделанные в представлении источника данных.

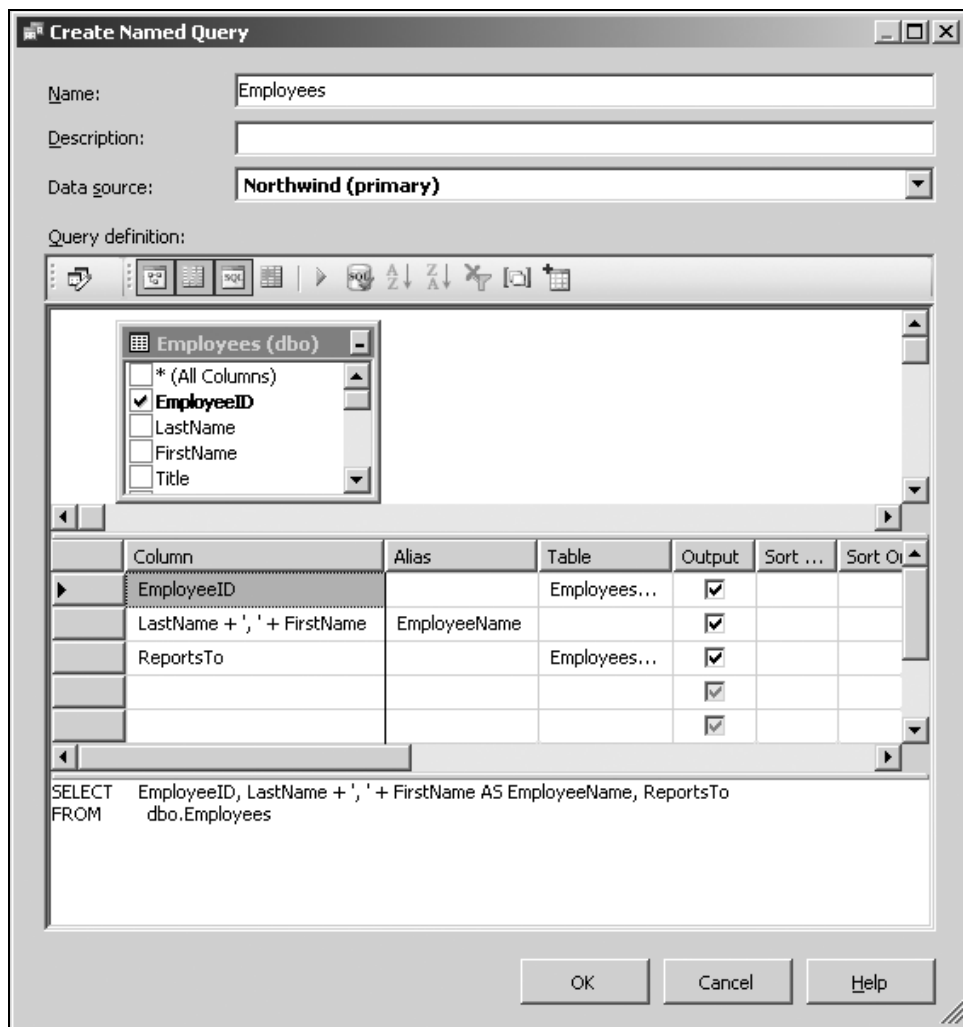
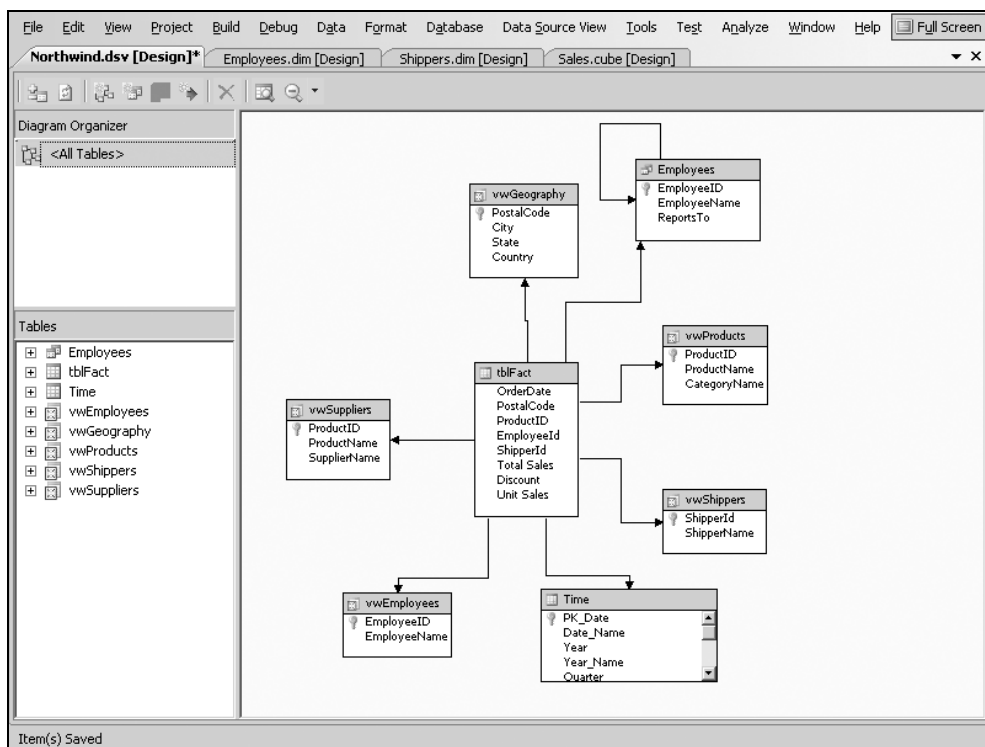


Рис. 16.4. Диалоговое окно **Create Named Query**

Перед тем как использовать этот именованный запрос для создания нового измерения на основе сотрудников, нам нужно создать две связи: одну для самосоединения от столбца `ReportsTo` к столбцу `EmployeeId`, а вторую — от

столбца `EmployeeId` таблицы `tblFact` к столбцу именованного запроса (с аналогичным названием). Для этого нужно щелкнуть правой кнопкой мыши по столбцу `EmployeeId` именованного запроса `Employees` и выбрать пункт **Set Logical Primary Key** контекстного меню (либо щелкнуть по столбцу левой кнопкой мыши и выбрать **Data Source View | Set Logical Primary Key**). Затем создадим связь между таблицей фактов, именованным запросом и самосоединением (при помощи описанной в предыдущей главе процедуры перетаскивания, либо посредством выбора **Data Source View | New Relationship**, либо при помощи выбора пункта **New Relationship** в любом контекстном меню панелей **Diagram** или **Tables**). Вы можете также воспользоваться кнопкой **New Relationship** (четвертая справа) в панели инструментов визуального конструктора представлений источников данных. Когда вы закончите, ваше представление источника данных должно будет выглядеть так, как показано на рис. 16.5. Сохраните ваши изменения.



**Рис. 16.5.** Наше представление источника данных (модифицированное для включения именованного запроса `Employees` и соединения с ним)



## Измерения родитель/потомок

Перед созданием нового измерения на основе сотрудников щелкните правой кнопкой мыши по именованному запросу `Employees` в визуальном конструкторе представлений источников данных, а затем выберите пункт **Explore Data**. Изучите показанные данные. Вы увидите, что `Suyama`, `King` и `Dodsworth` подчиняются `Buchanan`, а также что `Buchanan` и все остальные сотрудники подчиняются `Fuller`. Наша задача — создать специальное измерение, которое представляет эти связи в виде иерархии (без преобразования структуры самой таблицы измерения). Для этого необходимо запустить мастер `Dimension Wizard`.

### СОВЕТ

Как вы уже видели в *главе 15*, добавление нового измерения в куб — это двухшаговый процесс: сначала вы должны создать измерение базы данных, а затем добавить его в куб. Для создания измерения базы данных вам нужно использовать мастер `Dimension Wizard`. Вы можете открыть этот мастер при помощи выбора пункта **New Dimension** в контекстном меню узла **Dimensions** в окне **Solution Explorer** (либо при помощи выбора **Project | New Dimension**).

Существующее измерение базы данных добавляется в куб при помощи диалогового окна **Add Cube Dimension**, которое вы можете показать при помощи выбора **Cube | Add Cube Dimension**; эта опция доступна тогда, когда открыт визуальный конструктор кубов и активна его вкладка **Cube Structure** (или **Dimension Usage**). Вы можете также выбрать опцию **Add Cube Dimension** в контекстном меню вкладки **Cube Structure** панели **Dimensions** либо на вкладке **Dimension Usage**.

На странице **Welcome** нажмите **Next**, а затем еще раз нажмите **Next** для того, чтобы принять выбор по умолчанию на странице **Select Creation Method**. На странице **Specify Source Information** выберите в выпадающем списке **Main Table** именованный запрос `Employees`, примите выбор по умолчанию `EmployeeId` как ключевого столбца, а затем выберите `EmployeeName` в выпадающем списке **Name column** (как показано на рис. 16.6).

Нажмите **Next** для перехода к странице **Select Dimension Attributes** и убедитесь, что флажки установлены возле обоих столбцов — `Employee Id` и `ReportsTo`. Измените название атрибута `Employee Id` на `Employee Name`. Нажмите **Next** для перехода к странице **Completing the Wizard**. Введите `Org Chart` в качестве названия нового измерения, а затем нажмите **Finish**. Теперь новое измерение должно открыться в визуальном конструкторе измерений; если оно не открылось — откройте его.

The screenshot shows the 'Specify Source Information' dialog box in the Dimension Wizard. The dialog has a title bar with the text 'Dimension Wizard' and standard window controls. Below the title bar, the main heading is 'Specify Source Information' with a subtitle 'Select a data source and specify how the dimension is bound to it.' and a small icon of a cube. The dialog contains four sections: 'Data source view:' with a dropdown menu showing 'Northwind'; 'Main table:' with a dropdown menu showing 'Employees'; 'Key columns:' with a list box containing 'EmployeeID' and '(Add key column)'; and 'Name column:' with a dropdown menu showing 'EmployeeName'. At the bottom, there are four buttons: '< Back', 'Next >', 'Finish >>', and 'Cancel'.

**Рис. 16.6.** Страница **Specify Source Information** мастера Dimension Wizard

Измерение `Org Chart` теперь содержит атрибут с названием `Employee Name`, который отображает значения из столбца `EmployeeName` и имеет ключ по столбцу `EmployeeId`. Этот атрибут будет служить в качестве дочернего уровня нашего измерения родитель/потомок.

Обратите внимание, что использование атрибута `Reports To` уже настроено в значение `Parent`. Вы можете убедиться в этом через контекстное меню атрибута (выбрав пункт **Set Attribute Usage** и проверив наличие флажка возле **Parent**) либо через окно **Properties** (свойство `Usage` установлено в значение `Parent`). Однако нам нужно сделать еще одно небольшое улучшение: выберите свойство `NameColumn` атрибута `Reports To`, нажмите кнопку с многоточием, выберите `EmployeeName` в диалоговом окне **Name Column** (в списке **Source Column**), а затем нажмите кнопку **OK**. Это гарантирует, что (подобно атрибуту `Employee Name`) атрибут `Reports To` будет правильно настроен для показа имени сотрудника и будет иметь ключ по идентификатору ID сотруд-

ника (хранящемуся в столбце ReportsTo). Для завершения настройте свойство OrderBy атрибута в значение Name.

Сохраните и разверните ваши изменения, а затем щелкните по вкладке **Browser** визуального конструктора измерений (если вы еще не попали туда автоматически). Убедитесь, что в выпадающем списке **Hierarchy** выбрано **Reports To**, а затем углубитесь по каждому элементу в древовидном представлении. Вы должны увидеть всю иерархию подчинения, причем сотрудники на каждом уровне должны быть отсортированы в алфавитном порядке (рис. 16.7).

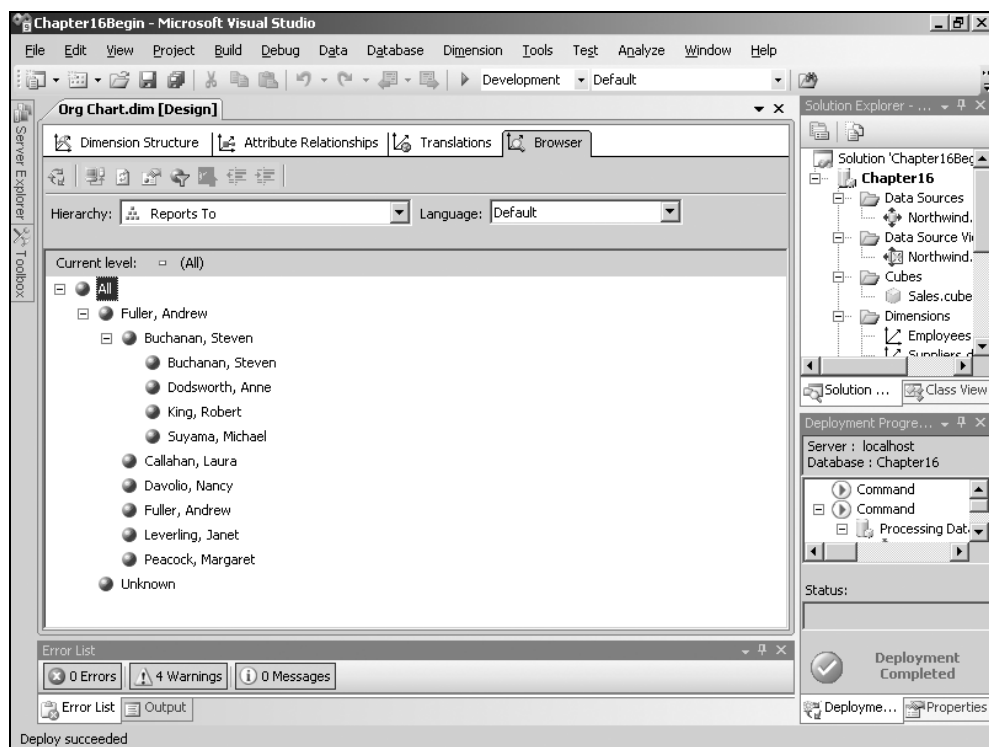


Рис. 16.7. Вкладка **Browser** визуального конструктора измерений с выбранным атрибутом родитель/потомок

Браузер измерений подтверждает, что наш атрибут родитель/потомок был настроен почти правильно. Обратите внимание, что Fuller и Buchanan показаны как подчиняющиеся сами себе. Скоро мы это исправим.

### ПРИМЕЧАНИЕ

Обратите внимание, что отдельной иерархии атрибута `Employee Name` не существует. Это потому, что свойство `AttributeHierarchyVisible` атрибута установлено в значение `False`. Это сохраняет атрибут в структуре измерения (где он должен присутствовать, так как является ключевым), но скрывает атрибут от клиентских приложений.

Теперь выберите первый узел **Fuller, Andrew** в древовидном представлении вкладки **Browser** и обратите внимание на название **Level 02**, находящееся рядом с меткой **Current level** прямо над древовидным представлением (как показано на рис. 16.8).

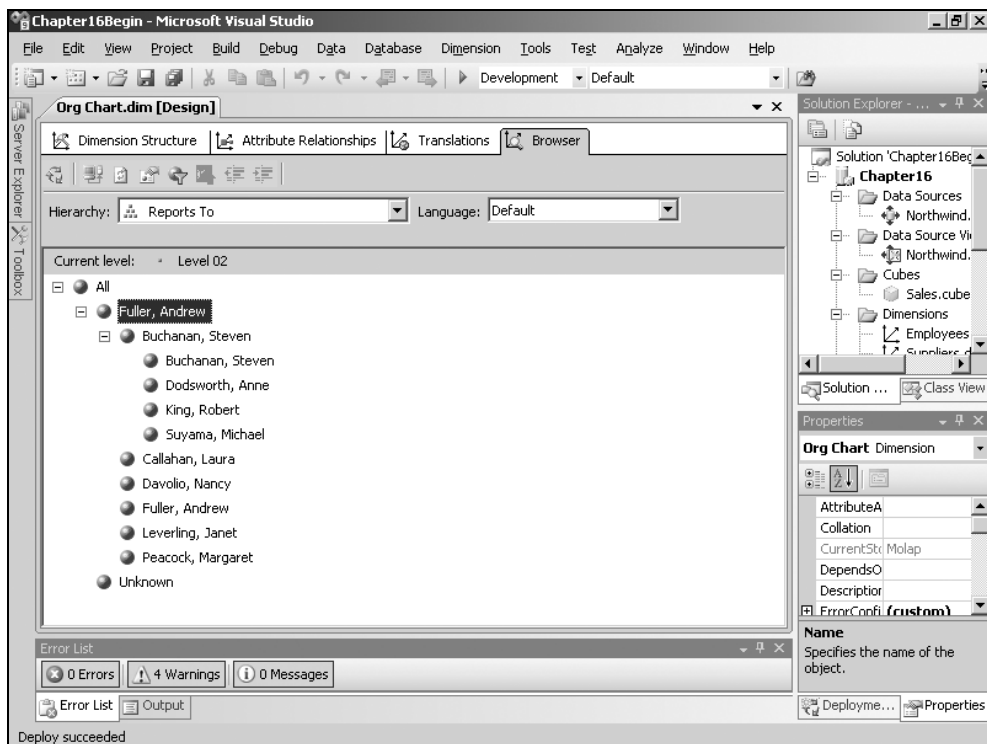
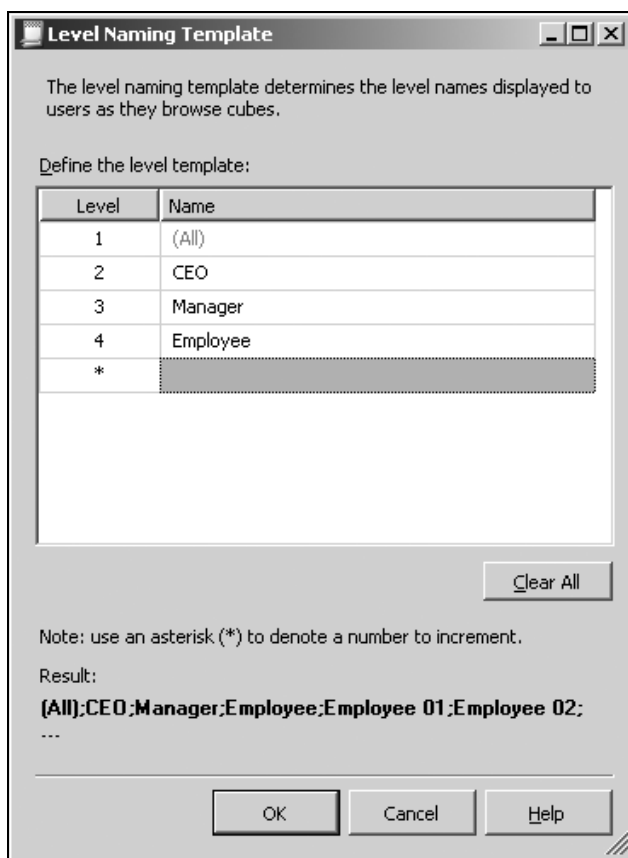


Рис. 16.8. Вкладка браузера визуального конструктора измерений с выделенным обобщенным названием уровня

Щелкните по первому узлу **Buchanan, Steven**, а затем по узлу **Dodsworth, Anne**, обратите внимание на то, что их уровни называются соответственно

**Level 03** и **Level 04**. Эти названия уровней генерируются службами Analysis Services, поскольку иерархия была сгенерирована автоматически (а не была создана вами).

Мы можем изменить эти названия уровней на вкладке **Dimension Structure** при помощи свойства `NamingTemplate` атрибута `Reports To`. В окне **Properties** нажмите кнопку с многоточием для показа диалогового окна **Naming Template**. Заполните это диалоговое окно так, как показано на рис. 16.9, а затем нажмите кнопку **OK**.



**Рис. 16.9.** Диалоговое окно **Level Naming Template**

Диалоговое окно **Level Naming Template** позволяет ввести названия уровней для атрибутов/иерархий родителей/потомков. Дав названия уровням 2, 3 и 4,

мы изменим сгенерированные службами Analysis Services названия **Level 02**, **Level 03** и **Level 04**. И наконец, для того чтобы сотрудники с непосредственным подчинением не появлялись в списках по два раза, установим свойство `MembersWithData` в значение `NonLeafDataHidden`.

Сохраните и разверните ваши изменения. Убедитесь на вкладке **Browser**, что иерархия имеет правильную структуру, и все ее уровни имеют осмысленные названия. (Для появления изменений вам придется подключиться повторно.) После выполнения этих изменений добавьте измерение `Org Chart` в куб `Sales` (при помощи диалогового окна **Add Cube Dimension** визуального конструктора кубов), а затем сохраните и разверните ваши изменения еще раз. Теперь сделайте запрос к новой иерархии `Org Chart.Reports To` и к любой мере по вашему выбору (на вкладке **Browser** куба; при необходимости нужно будет подключиться повторно).

Углубитесь в уровни, чтобы убедиться, что теперь вы можете запрашивать данные куба по CEO, Manager и Employee. Некоторые из сумм по уровням превышают суммы по своим элементам; так происходит потому, что мы скрыли все данные, связанные с нелистовыми элементами иерархии. Настроив свойство `MembersWithData` атрибута `Reports To` в значение `NonLeafDataVisible`, мы ликвидируем проблему суммирования, но это приведет к тому, что Fuller и Buchanan будут появляться в списках по два раза (как вы уже видели ранее).

## Группирование элементов

Следующая наша задача — измерение `Products`. Начнем с переименования атрибута `Product Id` (при помощи ввода `Product Name` в качестве значения свойства `Name`), установки свойства `NameColumn` в значение `vwProducts.ProductName`, а свойства `OrderBy` — в значение `Name`. Затем создадим атрибут из столбца `CategoryName` в `vwProducts` и иерархию с названием `Category-Product` (с использованием в качестве ее уровней атрибутов `Category Name` и `Product Name`, причем именно в этом порядке). Для того чтобы вспомнить создание атрибута и иерархии, обратитесь к *главе 15*. Сохраните и разверните свои изменения, а потом на вкладке **Browser** визуального конструктора измерений выберите **Product Name** в выпадающем списке **Hierarchy**. Углубитесь по элементу `All`, и вы увидите, что количество продуктов очень велико. Использование иерархии `Category-Product` делает список продуктов более управляемым, но вам не всегда нужно разбивать продукты по категориям.

Эту проблему можно решить путем группирования элементов (или дискретизации). По существу эта функция группирует элементы атрибута путем создания нового атрибута/уровня над ним. Каждый элемент нового уровня содержит несколько дочерних элементов из исходного атрибута.

Для реализации группирования элементов для измерения `Products` вернемся на вкладку **Dimension Structure**. Перетащите столбец `ProductName` из панели **Data Source View** в панель **Attributes** и измените его название по умолчанию **Product Name 1** на `Product Group`. Затем установите свойство `DiscretizationBucketCount` этого нового атрибута в значение 10, а свойство `DiscretizationMethod` — в значение `Automatic`. При этом будет создано 10 отдельных родительских элементов, которые можно использовать в новой иерархии совместно с атрибутом `Product Name`. Создайте новую иерархию путем перетаскивания атрибута `Product Group` в панель **Hierarchies**, а затем перетащите атрибут `Product Name` в новую иерархию. Назовите новую иерархию `Group-Product`, а затем сохраните и разверните свои изменения.

Используйте новую иерархию в браузере кубов (сначала надо повторно подключиться) для детализации продаж по группам (используя иерархию `Group-Product`, а не атрибут `Product Group`). Вы увидите, что просмотр ограниченного количества элементов на уровне `Product Group` гораздо проще, чем просмотр всего списка продуктов.

## Пользовательские таблицы и типизирование "измерение/атрибут"

В *главе 15* мы показали вам, как создать измерение времени на основе сгенерированной сервером физической таблицы `time`, которая была добавлена в базу данных `Northwind`. Позднее мы покажем вам, как строить измерение времени на основе сгенерированной сервером псевдотаблицы, которая вообще не имеет представления в виде физической таблицы. Однако в этом разделе мы собираемся показать вам, как строить измерение на основе существующей в базе данных таблицы.

### Измерения времени пользовательских таблиц

Создание измерения времени на основе вашей собственной таблицы, которая уже имеется в базе данных, — это очень типичная задача при создании кубов OLAP, однако, честно говоря, в `Analysis Services 2008` это делать не-

сколько сложнее, чем было в Analysis Services 2005. В этом разделе мы покажем вам, как построить такое измерение. При этом мы объясним вам, как использовать свойство `Type` измерений и атрибутов.

В *главе 15* (при создании измерения времени с использованием сгенерированной сервером таблицы) свойства настраивались автоматически. Для нашей собственной таблицы нам нужно будет настроить свойства самостоятельно, что даст нам хорошую возможность объяснить, как они работают. Создание нашего нового измерения времени позволит нам также изучить две новые функциональные возможности Analysis Services 2008: Best Practice Alerts (BPA) и новую панель **Attribute Relationships** визуального конструктора измерений. Позднее в этой же главе (при изучении переводов) мы вернемся к этому измерению и произведем в нем некоторые модификации.

Для начала запустите скрипты `tblTime.sql` и `tblFact.sql` (в этом порядке), которые имеются в решении Management Studio (в примерах кодов этого проекта). После выполнения скриптов в базу данных Northwind будет добавлена новая таблица измерения `tblTime`, будет модифицирована также и структура `tblFact`. Вернитесь в Visual Studio, добавьте `tblTime` в представление нашего источника данных, обновите его (чтобы добавились изменения `tblFact`), а затем свяжите `tblFact` и `tblTime` по столбцу `TimeKey` обеих таблиц. Нажмите **Yes** в ответ на запрос о создании логического первичного ключа, а затем сохраните изменения. Откройте мастер Dimension Wizard.

На странице **Welcome** нажмите **Next**, затем на странице **Select Creation Method** убедитесь, что выбрана опция **Use An Existing Table**, потом еще раз нажмите **Next**. На странице **Specify Source Information** убедитесь, что в выпадающем списке **Data Source View** выбрана база Northwind, выберите `tblTime` в выпадающем списке **Main Table**, а затем убедитесь, что в списке **Key Columns** имеется только один столбец `TimeKey`. В выпадающем списке **Name Column** выберите `OrderDate`, а затем нажмите **Next**. На странице **Select Dimension Attributes** убедитесь, что в левой колонке установлены флажки для всех атрибутов, кроме `Month Number`. (Флажки активны, несмотря на то, что они выглядят неактивными).

Измените название атрибута `Time Key` на `Order Date` (посредством редактирования ячейки с названием). В столбце `Attribute Type` щелкните в ячейке атрибута `Order Date`, а затем нажмите стрелку выпадающего списка. В появившемся окне раскройте **Date**, а затем **Calendar**, а потом выберите его дочерний узел **Date** (второй в списке, сразу под узлом **Day Of Year**). Теперь сделайте то же самое для настройки типов атрибутов: `Month` для **Month**,



Quarter для **Quarter** и Year для **Year**. Теперь мастер должен выглядеть так, как показано на рис. 16.10.

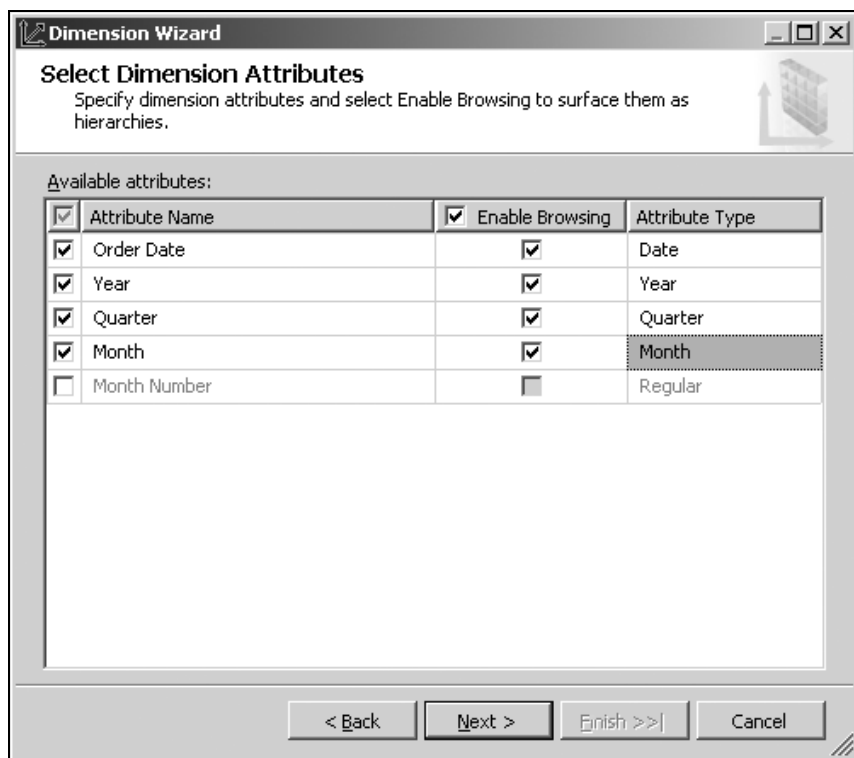


Рис. 16.10. Страница **Select Dimension Attributes** мастера Dimension Wizard

Нажмите **Next**. На странице **Completing the Wizard** назовите новое измерение **TimeInt1** (мы добавим в это измерение перевод, так что будем считать его "международным"), а затем нажмите кнопку **Finish**; новое измерение откроется в визуальном конструкторе измерений.

## Типизирование измерение/атрибут

Перед тем как продолжить настройку элементов измерения **TimeInt1**, давайте посмотрим, что мы построили. Сначала щелкните по каждому из четырех атрибутов и посмотрите значение свойства **Type** в окне **Properties**. Обратите внимание, что это значения **Date**, **Months**, **Quarters** и **Years** (а не **Date**, **Month**,

Quarter и Year). Так происходит потому, что некоторые значения свойств представлены в Dimension Wizard не так, как в окне **Properties**. Теперь щелкните по корневому узлу TimeInt1 и обратите внимание, что его свойство Type установлено в значение Time. Щелкните по этому свойству в окне **Properties**, а затем нажмите стрелку выпадающего списка в столбце его значений. Обратите внимание на богатый выбор вариантов. Обычно вам необходимо явно настроить свойство Type измерения. Однако по выбранным нами типам атрибутов мастер уже определил, что наше измерение должно иметь тип Time.

## Технология Best Practice Alerts

Обратите внимание на волнистую линию под узлом **TimeInt1** в панели **Attributes**, которая похожа на ошибку. Если вы наведете на узел курсор вашей мыши, то вы увидите в виде подсказки следующее сообщение: "Create hierarchies in non-parent child dimensions" (Создайте иерархии в дочерних измерениях, не являющихся родительскими). Это сообщение называется Best Practice Alert (BPA), оно показано на рис. 16.11.

В основе BPA лежит база знаний правил проектирования Analysis Services; эти правила применяются визуальным конструктором в реальном времени (когда он изучает структуру создаваемых вами объектов базы данных Analysis Services).

Изучим эту технологию еще немного: изменим свойство Type измерения на Regular. Волнистые линии теперь появляются под каждым атрибутом (а не только под измерением). Посмотрите на подсказку BPA для атрибута Month, и вы увидите сообщение, что тип атрибута установлен в Months, это не совместимо с типом измерения Regular. Аналогичные сообщения вы увидите и для всех остальных атрибутов. Измените значение свойства Type измерения снова в Time, после короткой паузы вы увидите, что все волнистые линии под атрибутами исчезли.

Волнистая линия под самим измерением, конечно же, осталась. Скоро мы предпримем соответствующие действия для ее ликвидации, но сначала давайте сделаем некоторые другие изменения. Выполните скрипт `tblMonthNames.sql` в решении Management Studio; он добавляет в базу данных Northwind таблицу `tblMonthNames` названий месяцев и кварталов (как на английском, так и на испанском языках). Мы будем использовать содержимое на английском языке для названий столбцов наших атрибутов Month и Quarter; позднее мы создадим перевод, который будет ссылаться на содержимое на испанском языке.

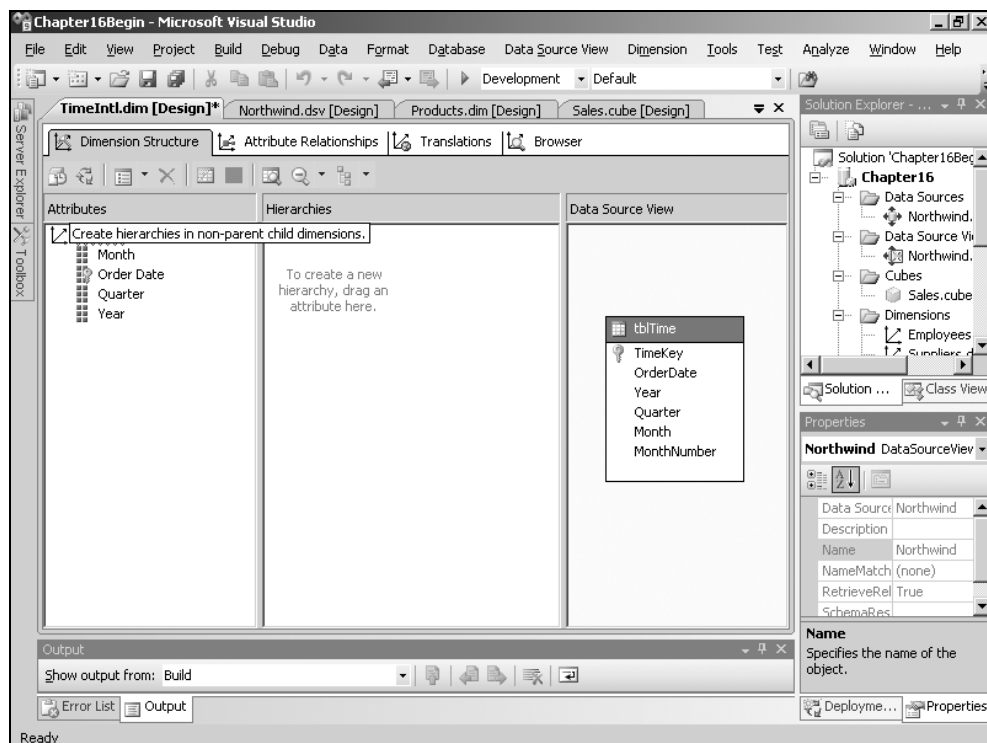


Рис. 16.11. Показанное в виде подсказки сообщение ВРА

Добавим таблицу в наше представление источника данных. Затем создадим связь между `tblTime` и `tblMonthName` по их общему столбцу `MonthNumber`. (В ответ на запрос нажмите **Yes** для создания логического первичного ключа). Сохраните ваши изменения, а если получите запрос на сохранение изменений в измерении `TimeInt1`, нажмите кнопку **OK**. Вернитесь в визуальный конструктор измерений для `TimeInt1` и добавьте `tblMonthNames` в панель **Data Source View** (при помощи пункта **Show Tables** контекстного меню или при помощи выбора **Data Source View | Show Tables**). После добавления таблицы обратите внимание на то, что синий цвет ее заголовка светлее, чем заголовок `tblTime`; это потому, что в таблице `tblMonthNames` ни один столбец не используется как ключевой для атрибутов. И наконец, измените свойства `NameColumn` атрибутов `Month` и `Quarter` на `tblMonthNames.EnglishMonthName` и `tblMonthNames.EnglishQuarterName` соответственно.

Теперь, когда наши атрибуты определены и типизированы, сохраните и разверните свои изменения, а затем просмотрите результаты на вкладке

**Browser.** Изучите каждую иерархию атрибутов, углубляясь по элементу All. Обратите внимание, что в атрибутах Months и Quarters значения повторяются; это потому, что в таблице tblTime у нас данные за несколько лет. Если вы просмотрите данные tblTime, то вы увидите, что фактически в столбцах Quarter и Month находятся уникальные значения типа 1996Q3 и 1996M 7. Таблица tblMonthNames преобразует эти значения соответственно в Q3 и July. Эти названия имеют смысл в контексте иерархии; исходные же значения позволяют определить иерархию (избегая дублирования ключевых значений).

Теперь наша задача проста. Вернитесь на вкладку **Dimension Structure** визуального конструктора измерений и определите новую иерархию с названием Year-Quarter-Month-Order Date, основанную на атрибутах со схожими названиями (именно в таком порядке). Обратите внимание на волнистую линию под названием иерархии и сообщение: "Между одним или несколькими уровнями данной иерархии отсутствуют связи атрибутов. Это может привести к снижению производительности запросов".

ВРА говорит нам о том, что если между атрибутами нашей пользовательской иерархии существует "естественная иерархия", то мы должны сообщить об этом службам Analysis Services. Естественная иерархия — это та, при которой между уровнями существует связь "один-ко-многим" (как в случае данного измерения).

Сообщение службам Analysis Services о том, что имеется естественная иерархия, помогает при создании агрегатов: зачем суммировать все даты года, если вместо этого можно просуммировать четыре квартала?

### **ВАЖНО**

В некоторых случаях не только возможно, но и желательно создавать иерархии со связями "многие-ко-многим". Например, в измерении продуктов вы можете создать иерархию, на верхнем уровне которой находится цвет, а на следующем — категория продукта. Поскольку большинство продуктов имеет разные цвета, то эти уровни не формируют естественной иерархии, однако пользовательская иерархия все равно остается полезной как маршрут детализации.

## **Связи атрибутов**

Указание на существование естественной иерархии делается на вкладке **Attribute Relationships** визуального конструктора измерений. Перейдите на эту

вкладку и обратите внимание, что по умолчанию службы Analysis Services определили только то, что ключевой атрибут является вспомогательным по отношению к трем остальным. По существу визуальный конструктор говорит нам: "Я знаю что годы, кварталы и месяцы состоят из разных дат, но я не имею понятия о том, какова связь между этими атрибутами дат".

Мы можем указать эти связи, например, что кварталы состоят из месяцев. Щелкните по овалу (который представляет атрибут Month в верхней панели **Diagram**), удерживайте нажатой кнопку мыши и тащите этот овал на тот овал, который представляет собой атрибут Quarter. Затем сделайте то же самое для Quarter и Year. При необходимости наведите порядок в компоновке диаграммы (для этого необходимо использовать кнопку **Arrange Shapes** панели инструментов), вкладка **Attribute Relationships** теперь должна выглядеть так, как показано на рис. 16.12.

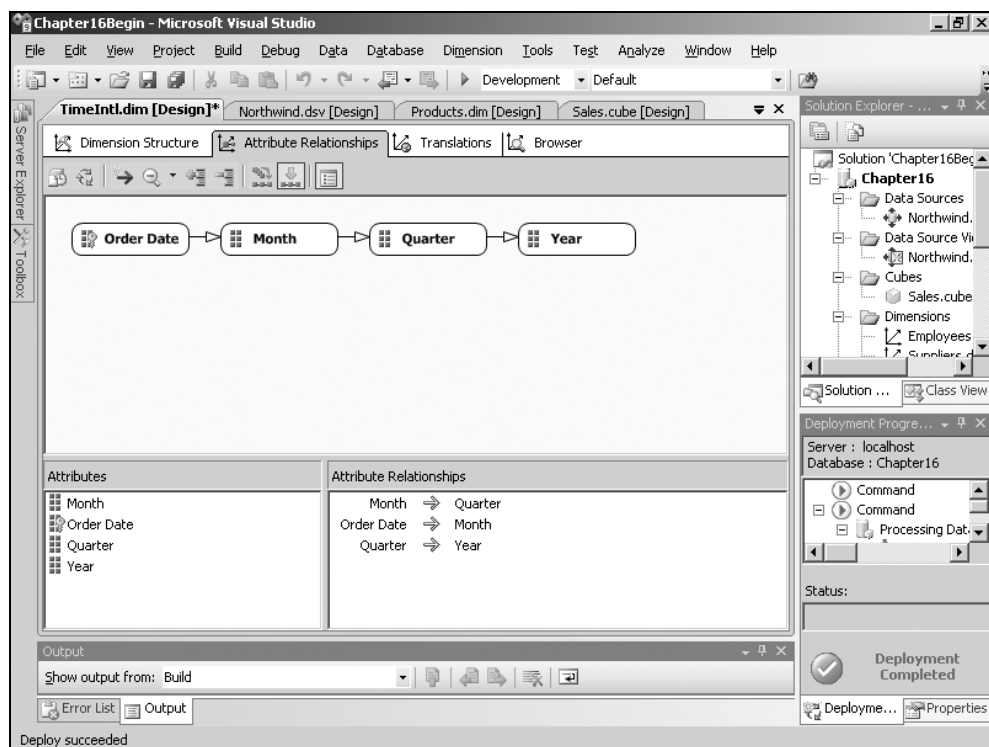


Рис. 16.12. Сконфигурированные связи атрибутов

## СОВЕТ

Существуют два типа связей атрибутов: представляющие естественные иерархии и те, которые представляют неиерархические связи. Для неиерархических связей целевой атрибут находится внутри исходного атрибута, а не соединяется с ним стрелкой. Это очень хорошо демонстрирует измерение *Promotion* базы данных *AdventureWorksDW2008*. Если вы откроете это измерение и посмотрите вкладку **Attributes Relationship**, то вы увидите, что там имеется семь связей атрибутов (только две из которых являются иерархическими).

Мы создали три связи атрибутов. Они представлены стрелками в панели **Diagram** и тремя элементами списка в панели **Attribute Relationships** (внизу слева); выбор одной из них приведет к выделению ее же в другой панели. Для редактирования связи в диалоговом окне **Edit Attribute Relationship** вы можете дважды щелкнуть по одной из стрелок на диаграмме.

Сделайте это прямо сейчас для любой из связей и измените выбор в выпадающем списке **Relationship Type** на **Rigid (Will Not Change Over Time)**, а затем нажмите кнопку **OK**. В визуальном конструкторе измерений обратите внимание на то, что представляющие только что отредактированную вами связь стрелки стали толще. В окне **Properties** обратите также внимание, что значение свойства *RelationshipType* отражает только что сделанное вами изменение. Теперь используйте либо диалоговое окно **Edit Attribute Relationship**, либо окно **Properties** для настройки типов и остальных двух связей в значение **Rigid**. Иногда иерархии с течением времени меняются, в случае измерения времени это не так.

Вернитесь на вкладку **Dimension Structure**, пометка ВРА для иерархии исчезла. Вы заметите также и выпадающие стрелки справа от каждого из уровней (**Quarter**, **Month** и **Order Date**) нашей пользовательской иерархии. Щелкните по любой из этих стрелок для просмотра только что созданных нами связей атрибутов.

Сообщение ВРА для измерения по-прежнему на месте. Наведите курсор мыши на волнистую линию, и вы увидите следующее сообщение: "Avoid visible attribute hierarchies for attributes used as levels in user-defined hierarchies". (Избегайте видимых иерархий для атрибутов, используемых как уровни определенных пользователем иерархий). Иначе говоря, зачем предоставлять для детализации каждый атрибут, когда все они представлены в только что созданной вами иерархии? Сделайте иерархию видимой и скройте атрибуты.

Во-первых, здесь нужно помнить, что подсказки ВРА не всегда можно слепо принимать на веру, в данном случае подсказка как раз не имеет смысла. Не исключено, что нам обязательно нужно детализировать по месяцам (при этом, возможно, фильтруя по годам). Во-вторых, подсказки ВРА — это только предупреждения, это не ошибки. Вы увидите каждую подсказку в окне **Error List** среды Visual Studio в виде предупреждения. Для обновления этого списка надо выбрать **Build | Build Chapter16**. После этого вы увидите, что сборка прошла успешно, и что сообщения ВРА включены в список (рис. 16.13).



Рис. 16.13. Сообщения ВРА в окне **Error List** после успешной сборки

Мы можем сделать с этим сообщением ВРА следующее:

- ☐ игнорировать данное предупреждение (как мы делали все это время для других наших баз данных Analysis Services);
- ☐ разобраться с проблемой для устранения предупреждения;
- ☐ отменить данный экземпляр предупреждения;

- ❑ отменить данное предупреждение глобально (то есть отказаться от проверки этого положения ВРА в текущей базе данных).

Если вы щелкнете по предупреждению в окне **Error List** и выберете пункт **Dismiss** контекстного меню, то появится диалоговое окно **Dismiss Warning**, которое позволит вам отменить данный экземпляр предупреждения, а также сообщит, как его можно отменить на глобальном уровне. Пока нажмите **Cancel**, чтобы не отменять данное предупреждение.

Давайте сначала разберемся с причиной предупреждения, а уже потом посмотрим, как его отменить глобально. Для решения проблемы на вкладке **Dimension Structure** визуального конструктора измерений просто выберите все четыре атрибута (удерживайте нажатой клавишу <Ctrl> и щелкните по каждому атрибуту), а затем измените в окне **Properties** значение их коллективного свойства `AttributeHierarchyVisible` на `False`. Обратите внимание, что предупреждение исчезло (волнистая линия пропала). Теперь измените значение свойства в `True` (для всех четырех атрибутов) и обратите внимание, что волнистая линия вернулась. Мы предпочитаем, чтобы атрибуты были видимыми (и не только в данном случае, но и в других измерениях тоже), поэтому давайте отменим данное предупреждение глобально.

Для этого вам нужно будет открыть редактор баз данных. Вы можете попасть в этот визуальный конструктор при помощи щелчка правой кнопкой мыши по узлу проекта **Chapter16** (это дочерний узел корневого узла **Solution**) в окне **Solution Explorer** и последующего выбора пункта контекстного меню **Edit Database**. Вы можете также просто выбрать **Database | Edit Database**. В редакторе выберите вкладку **Warnings**. В списке **Design Warning Rules** вы увидите пять основных категорий предупреждений. Углубитесь по категории **Dimension Design**, в столбце `Importance` вы увидите, что предупреждения разделены на категории **High**, **Medium** и **Low**. Прокрутите до первого предупреждения **Low**, и вы увидите, что это именно то предупреждение, которое мы хотим глобально отменить. Снимите его флажок, редактор баз данных должен теперь выглядеть так, как показано на рис. 16.14.

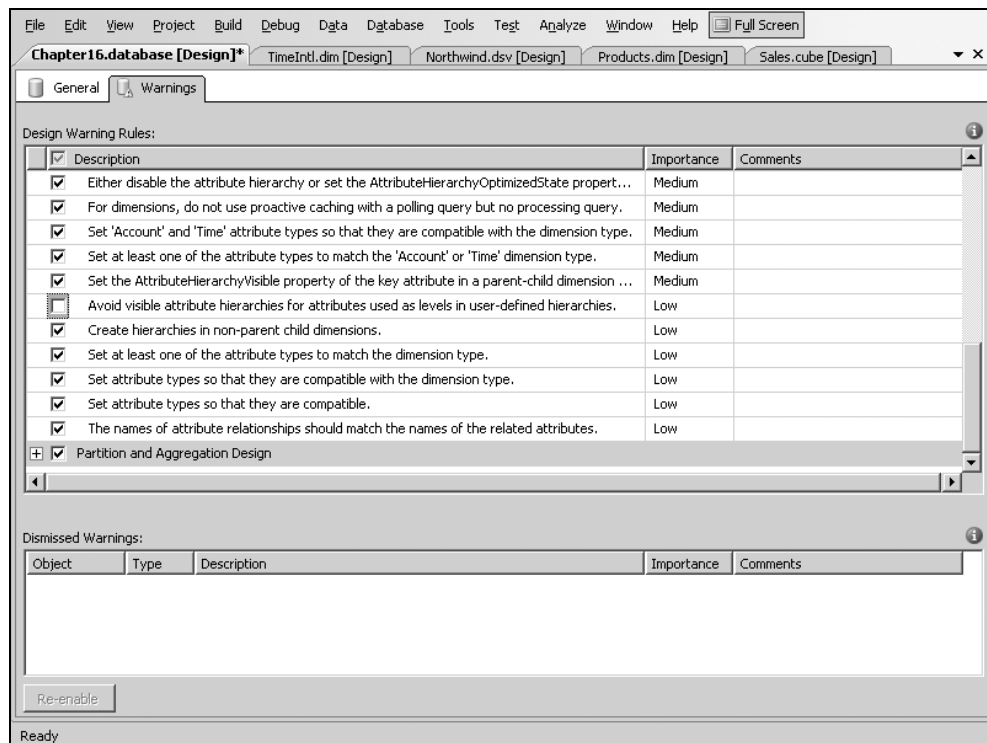
Закройте редактор баз данных, сохраните свои изменения и вернитесь на вкладку **Dimension Structure** визуального конструктора измерений; обратите внимание, что предупреждение сброшено.

### ПРИМЕЧАНИЕ

Мы показали вам, как отменить конкретное предупреждение ВРА для измерения `TimeIntl`, но совершенно очевидно, что в нашем проекте есть и другие. В задачу этой книги не входит разрешение проблем со всеми преду-



предждениями. Однако мы советуем вам просмотреть остальные сообщения ВРА: даже если вы не станете устранять их причины, вы много узнаете о правильном проектировании кубов (которые имеют хорошую производительность).



**Рис. 16.14.** Глобальное отклонение предупреждения ВРА в визуальном конструкторе баз данных

Сохраните ваши изменения в измерении. Мы почти закончили, однако помните, что мы просто определили новое измерение базы данных, но еще не добавили его в наш куб Sales. Для этого откройте куб в его визуальном конструкторе и на вкладке **Cube Structure** добавьте новое измерение TimeInt1 (при помощи любого из упоминавшихся ранее способов **Add Cube Dimension**).

Теперь сохраните и разверните все изменения. На вкладке **Browser** визуального конструктора кубов перетащите меру Total Sales в зону **Drop Totals Or Detail Fields Here**, а затем перетащите иерархию Year-Quarter-Month-

Order Date измерения TimeInt в зону **Drop Row Fields Here**. Углубитесь в иерархию, чтобы увидеть, как она работает.

## Серверные измерения времени

Когда мы строили наш куб Northwind Sales в предыдущей главе, то, возможно, вы подумали, что создание специальной таблицы измерений для измерения Time было несколько избыточным. В конце концов, концепция времени очень универсальна: имеется весьма ограниченный набор возможных атрибутов и иерархий, каждый из которых легко может получить ключ по полю даты таблицы фактов (а не произвольный ключ времени).

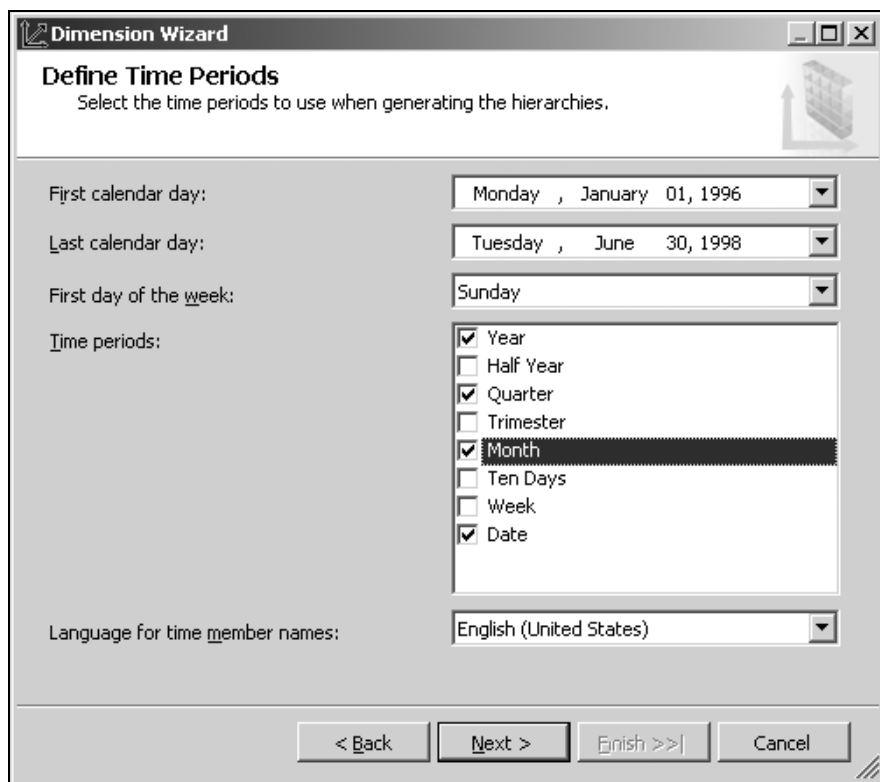


Рис. 16.15. Страница **Define Time Periods** мастера Dimension Wizard

В службах Analysis Services 2000 концепция (производного от столбца даты) измерения времени была очень проста и легко реализуема. Analysis Services 2005 и 2008 расширили эту функциональную возможность, но теперь способ ее использования менее очевиден. Новая реализация таких измерений времени основана на генерируемой сервером таблице псевдоизмерения, а также массиве предопределенных атрибутов и иерархий. Фактически этот тип измерения времени реализуется путем создания специального типа измерения, которое называется *серверным измерением времени*.

Давайте добавим в наш куб серверное измерение времени. Откройте мастер Dimension Wizard и на странице **Select Creation Method** выберите вариант **Generate A Time Table On The Server**. На странице **Define Time Periods** укажите January 1, 1996 и June 30, 1998 в качестве начального и конечного календарных дней и выберите **Year, Quarter, Month** и **Date** в списке **Time Periods** (как показано на рис. 16.15).

Примите параметры по умолчанию для всех остальных настроек, назовите ваше новое измерение *ServerTime*, а затем нажмите **Finish**. Добавьте измерение в ваш куб (как это было описано ранее), а затем щелкните по вкладке **Dimension Usage** визуального конструктора кубов. В этой вкладке вы настраиваете измерения, которые не основаны на обычных связях между таблицами фактов и измерений.

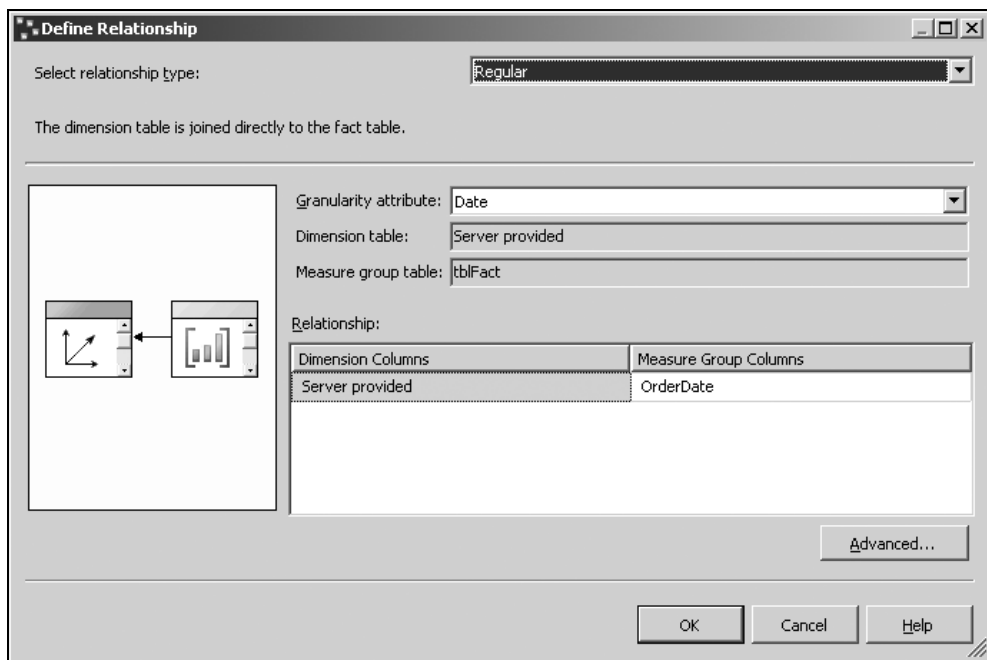
Щелкните по ячейке на пересечении строки измерения *ServerTime* и столбца группы мер *Main*, а затем щелкните по кнопке ячейки с многоточием. (Ячейка будет представлена серым прямоугольником справа от ячейки измерения *ServerTime*). В диалоговом окне **Define Relationship** укажите: связь — типа **Regular**, *Date* — в качестве **Granularity Attribute**, а *OrderDate* — как **Measure Group Column** (рис. 16.16).

Обратите внимание на назначение **Server Provided** (под заголовком **Dimension Columns**): это означает, что таблица измерения сгенерирована сервером и поэтому не требует от вас указания столбца измерения. Нажмите кнопку **OK** и обратите внимание, что ячейка на вкладке **Dimension Usage** теперь показывает *Date* (атрибут гранулярности серверного измерения времени). Сохраните и разверните ваши изменения, а затем просмотрите их в браузере кубов (сначала нужно будет повторно подключиться).

### ПРИМЕЧАНИЕ

Если вы будете углубляться при помощи браузера кубов по измерению *Time* или *ServerTime* в древовидном представлении метаданных, то вы увидите, что оба измерения имеют массив атрибутов по умолчанию. Если

вы откроете измерение `ServerTime` в визуальном конструкторе измерений, то в панели **Time Periods** вкладки **Dimension Structure** вы увидите еще больший массив столбцов (из которых можно создавать новые атрибуты).



**Рис. 16.16.** Диалоговое окно **Define Relationship**  
(настроенное для серверного измерения времени)

## Измерения фактов

Одной из отличительных черт служб Analysis Services является их способность адаптироваться к структуре вашей базы данных (вместо того, чтобы заставлять вас выполнять гимнастические номера по извлечению, преобразованию и загрузке данных). Например, предположим, что вы хотите создать измерение (из данных, которые хранятся в вашей таблице фактов). В Analysis Services 2000 вам пришлось бы продублировать эти данные внутри базы, а затем связать их с вашей таблицей фактов (чтобы использовать их как измерение). Текущая же версия Analysis Services гораздо удобнее: вы просто указываете, что ваши данные измерения находятся в таблице фактов — и службы Analysis Services все организуют. Например, таблица `Orders` в базе

данных Northwind содержит географическую информацию, которая относится к месту поставки (и из нее можно сконструировать полезное измерение). Наше текущее измерение Geography (производное от данных таблицы Customers) было сконструировать несложно. Однако способ использования в качестве уровней нового измерения таких полей нашей таблицы фактов, как ShipCountry и ShipRegion (происходящих из таблицы Orders базы данных Northwind), гораздо менее очевиден.

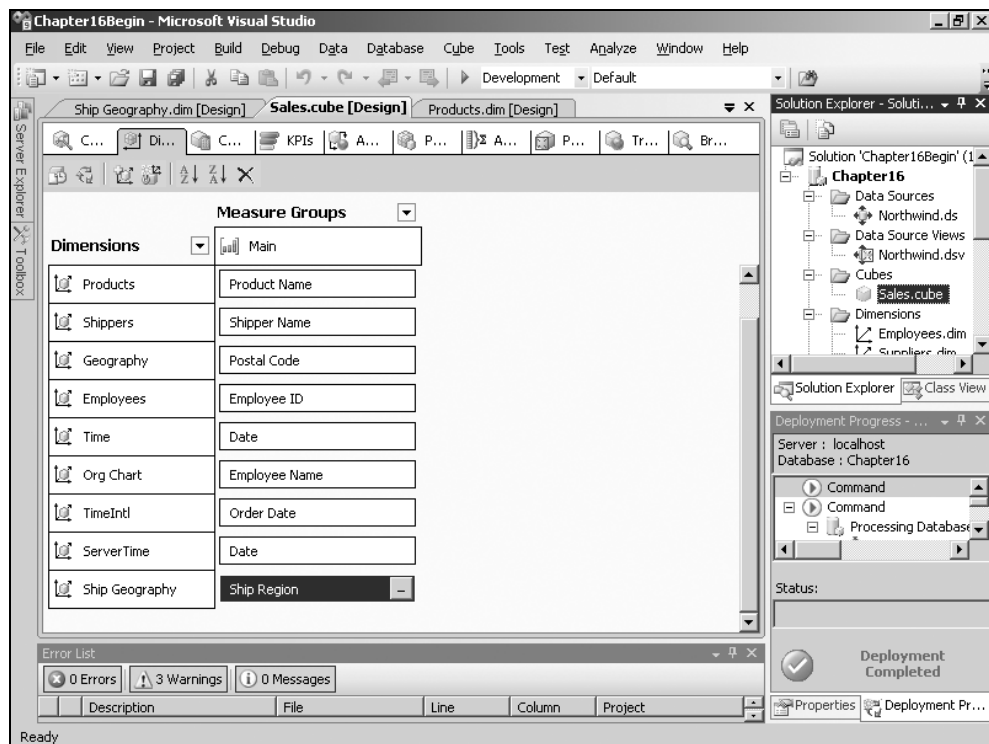


Рис. 16.17. Вкладка **Dimension Usage** с кнопкой многоточия

При помощи Analysis Services 2008 построить это измерение вполне реально. Создайте новое измерение с названием Ship Geography, указав таблицу tblFact как таблицу измерений, Ship Region — как ключевой столбец, а Ship Country — как единственный дополнительный столбец атрибутов. (Снимите выделение всех связанных таблиц). Откройте новое измерение в его визуальном конструкторе, создайте пользовательскую иерархию с названием Country-Region (используйте в качестве ее уровней атрибуты Ship

Country и Ship Region, именно в таком порядке), а затем добавьте новое измерение в ваш куб.

Перед тем как сохранить и развернуть ваши изменения, необходимо сделать еще кое-что. Сначала настроим свойство `ErrorConfiguration.KeyDuplicate` измерения в значение `IgnoreError`. Затем вы должны явно указать Analysis Services, что ваше новое измерение является измерением фактов. Это делается на вкладке **Dimension Usage** визуального конструктора кубов. Настройка очень проста. Найдите ячейку, где строка измерения Ship Geography пересекается со столбцом группы мер Main. В этой ячейке отображается название поля Ship Region. Все как будто бы нормально, но настройка действительно необходима.

Щелкните в ячейке Ship Region, и вы увидите кнопку с многоточием (рис. 16.17).

Поскольку Ship Geography является измерением фактов, то вам придется изменить его использование по умолчанию и дать указание Analysis Services установить для него связь фактов. Нажмите кнопку с многоточием для того, чтобы открыть диалоговое окно **Define Relationship**, измените тип связи с **Regular** на **Fact** (как показано на рис. 16.18), а затем нажмите кнопку **OK**.

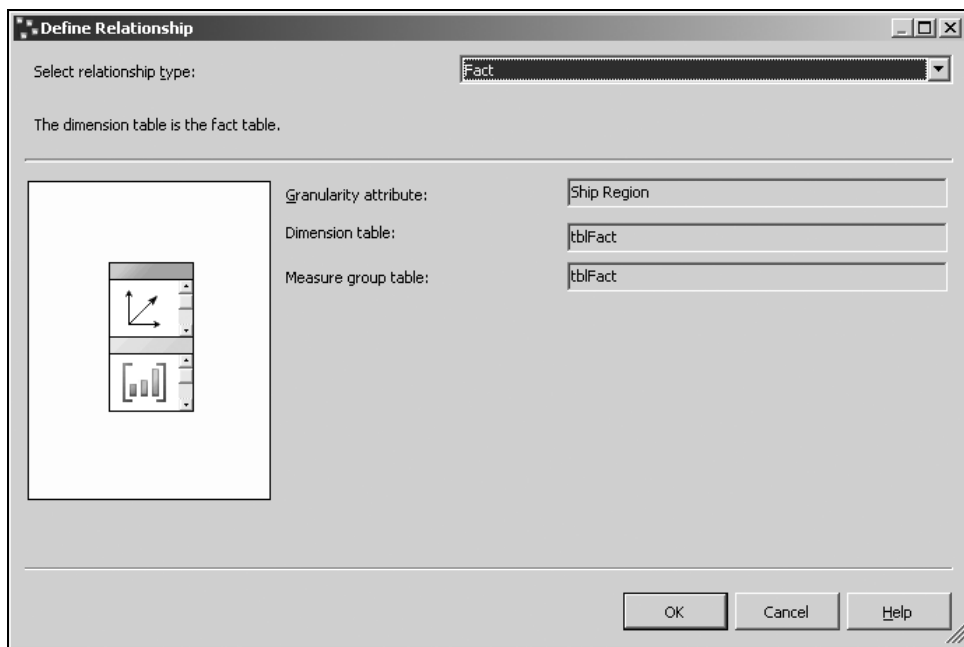
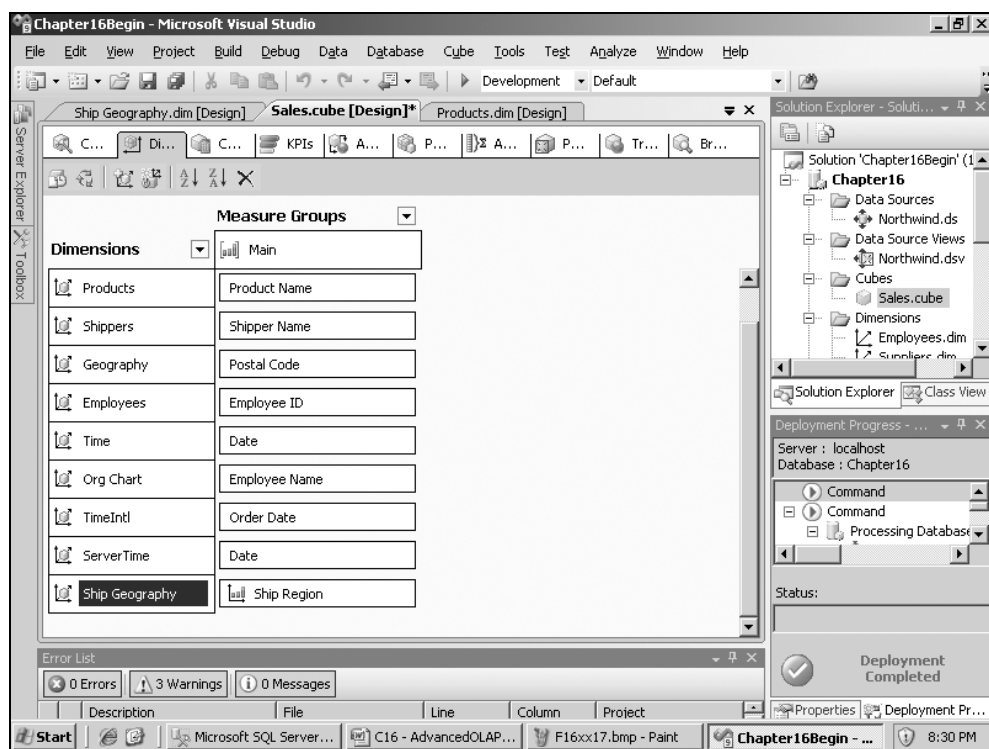


Рис. 16.18. Диалоговое окно **Define Relationship** с выбранным типом связи **Fact**



**Рис. 16.19.** Вкладка **Dimension Usage**  
с выделенным значком измерения фактов Ship Geography

На вкладке **Dimension Usage** визуального конструктора кубов обратите внимание, что в ячейке измерения появился специальный значок измерения фактов (рис. 16.19).

Сохраните и разверните ваши изменения. Подключитесь повторно (при необходимости) на вкладке **Browser** визуального конструктора кубов, и теперь вы должны суметь углубиться в любую меру по атрибутам Ship Country или Ship Region (или через иерархию, которая включает их оба). При этом не забудьте, насколько быстро вы смогли создать это измерение, добавить его в ваш куб и начать использовать. Не требуется никакого ETL, и вам не нужно создавать специальной таблицы измерения, представления или даже именованного запроса. Такая гибкость — которую мы можем назвать произвольной способностью улучшать куб — в Analysis Services 2000 просто не существовала. То, что она была предложена в Analysis Services 2005 и продолжается в Analysis Services 2008, является настоящей революцией в области бизнес-аналитики.

**ВНИМАНИЕ!**

Каждая группа мер куба может иметь не больше одного измерения фактов, а каждое измерение фактов может быть связано только с одной группой мер.

## Ролевые измерения

Измерение фактов — это не единственный специальный тип измерения; существует и много других. Одно из них — *ролевое измерение* — позволяет вам повторно использовать одно измерение базы данных как несколько разных измерений куба. Это особенно полезно с измерениями времени, которые обычно имеют одинаковую структуру.

Наша таблица `tblFact` теперь содержит три поля даты: `OrderDate` (которое было там всегда), `Required Date` и `ShippedDate`. Было бы очень хорошо, если бы мы могли использовать наше измерение `ServerTime` еще два раза, связав новые "экземпляры" соответственно с `RequiredDate` и `ShippedDate`. Оказывается, это можно сделать.

Используйте описанный ранее способ **Add Cube Dimension**, выберите измерение `ServerTime` в диалоговом окне **Add Cube Dimension**, а затем нажмите кнопку **ОК**. Вы должны увидеть добавление нового измерения под названием `ServerTime 1`. Повторите этот процесс для добавления измерения `ServerTime 2`. В панели **Dimensions** вкладки **Cube Structure** визуального конструктора кубов переименуйте `ServerTime 1` и `ServerTime 2` в `Shipped Date` и `Required Date` соответственно, а затем переименуйте исходное измерение `ServerTime` в `Order Date` (для того чтобы обозначить его связь с этими новыми измерениями).

Теперь перейдем на вкладку **Dimension Usage**. Вы должны увидеть все три измерения куба, основанные на измерении базы данных `ServerTime`, включая исходное измерение `ServerTime` (теперь называемое `Order Date`), которые приведены в формате названий `ServerTime (cubedim)`, где `cubedim` — это название отдельного измерения куба. Вы должны также заметить, что ячейки на пересечении двух новых измерений и группы мер `Main` — неактивны и пусты.

Щелкните в одной из пустых ячеек, а затем нажмите кнопку с многоточием для того, чтобы открыть диалоговое окно **Define Relationship**. В выпадающем списке **Select Relationship Type** выберите **Regular**, затем выберите **Date** в выпадающем списке **Granularity Attribute**, потом соответствующий столбец таблицы фактов (`ShippedDate` или `RequiredDate`, в зависимости от вы-



бранного вами измерения куба) в **Measure Group Columns**. Эти настройки явно указывают Analysis Services внешний ключ в таблице фактов, по которому необходимо соединять таблицу серверного псевдоизмерения времени. Нажмите кнопку **ОК**, повторите этот процесс для второго измерения, сохраните и разверните ваши изменения, а затем изучите изменения в браузере кубов.

В Analysis Services 2000 каждое отдельное измерение времени требовало своей собственной таблицы измерения (или, по крайней мере, своего собственного представления). Теперь же, как и в случае с измерениями фактов, службы Analysis Services делают за вас все извлечение данных (так что для реализации трех разных измерений времени от вас не требуется выполнения задач ETL, представлений или именованных запросов). Как вы уже догадываетесь, ролевые измерения так названы потому, что они позволяют одному измерению базы данных играть роль нескольких различных измерений куба при помощи связывания с различными столбцами в группе мер (таблице фактов). Это аналогично использованию одной и той же таблицы в SQL-запросе несколько раз (с присваиванием ей псевдонима): различные ролевые измерения куба являются по существу псевдонимами одного физического измерения базы данных.

## Расширенные меры

При всей этой работе над измерениями нам не следует забывать о мерах. Скоро мы обсудим, как создавать меры, которые полностью вычисляются нашим кубом, но пока давайте сосредоточимся на приведении в порядок имеющихся у нас мер.

### Форматирование мер

При просмотре куба вы заметили, что хранящиеся в мерах долларové суммы отображаются как числа, они не отформатированы как валютные данные. Это обстоятельство может осложнить чтение данных, но эта проблема легко решается.

Откройте куб *Sales* в визуальном конструкторе кубов, щелкните по вкладке **Cube Structure**, выберите меру *Total Sales* в панели **Measures**. Затем в окне **Properties** установите значение свойства *FormatString* в *Currency* и сделайте то же самое для остальных двух мер. Обратите внимание на имеющиеся варианты форматирования. Analysis Services облегчает реализацию гибкого форматирования всех мер на стороне сервера. В следующей главе вы увидите, как легко отображать отформатированные значения мер в ваших приложениях.

## Специальные агрегаты

Окно **Properties** позволяет делать не только простое форматирование ваших мер, но также изменять их формульные определения. Свойство `AggregateFunction` позволяет вам создавать такие меры, агрегаты которых — не просто суммы.

Например, в дополнение к мере `Total Sales` мы захотим получить меру, которая отражает средние итоги продаж для заданного среза измерения. Добавить такую меру в наш куб несложно. На вкладке **Cube Structure** визуального конструктора кубов щелкните по кнопке панели управления **New Measure** либо выберите пункт **New Measure** контекстного меню панели **Measures**. В появившемся диалоговом окне **New Measure** выберите **Average Over Time** в выпадающем списке **Usage**, выберите в списке **Source Column** элемент **Total Sales** и нажмите кнопку **OK**.

Вы увидите появление новой меры `Total Sales — Main`. При помощи контекстного меню, клавиши <F2> или свойства `Name` в окне **Properties** переименуйте ее в `Avg Total Sales`. В окне **Properties** обратите внимание, что свойство `AggregateFunction` установлено в значение `AverageOfChildren`; это результат выбора опции **Average Over Time** в диалоговом окне **New Measure**. Как и с прочими мерами, установите свойство `FormatString` этой новой меры в `Currency`.

### ПРИМЕЧАНИЕ

Щелчок по выпадающей стрелке в ячейке со значением свойства `AggregateFunction` показывает многочисленные функции агрегирования, которые вы можете выбрать. Свойство `MeasureExpression` позволяет ввести выражение MDX значения листовых уровней ваших измерений (до применения функции агрегирования). Подробнее о выражениях MDX вы узнаете в следующей главе.

## Каталоги отображения

Вы можете сгруппировать связанные меры при помощи каталогов отображения. Вводя название каталога в свойство `DisplayFolder` данной меры, вы можете делать разделение мер на множества. Браузер кубов визуального конструктора кубов предлагает непосредственную поддержку каталогов отображения (как Excel 2007). В следующей главе мы опишем поддержку каталогов отображения в приложениях.

После того как название каталога вводится в окне **Properties**, оно превращается в элемент выпадающего списка. Вы можете без труда присвоить то же

самое название каталога отображения другим мерам вашего куба (без необходимости вводить его заново и не рискуя ввести его с ошибками). Введите название `Non-Sum` в качестве значения свойства `DisplayFolder` для созданной вами меры `Avg Total Sales` и введите название `Sum` в качестве значения свойства `DisplayFolder` для меры `Total Sales`. Теперь выберите одну из оставшихся мер и обратите внимание, что в выпадающем списке свойства `DisplayFolder` имеются как `Sum`, так и `Non-Sum`. Выберите `Sum` и сделайте то же самое для последней меры.

Сохраните и разверните ваши изменения, а затем щелкните по вкладке **Browser** визуального конструктора кубов (если вы не попали туда автоматически). Сделайте повторное подключение, а затем просмотрите ваш куб. Исходные три меры должны находиться в каталоге отображения `Sum`, а мера `Avg Total Sales` должна находиться в каталоге отображения `Non-Sum`. При просмотре данные всех мер должны быть отформатированы как значения валют.

### **ВАЖНО**

Пользовательские иерархии внутри измерений также имеют свойство `DisplayFolder`, которое вы можете использовать точно так же, как описано для мер. Вы можете вводить названия каталогов при помощи синтаксиса типа маршрута, например, `\parent folder\child folder`, что позволит вам создавать иерархии каталогов отображения.

## **Вычисления**

Вкладка **Calculations** визуального конструктора кубов предоставляет отличный пользовательский интерфейс для указания именованных наборов и вычисляемых элементов без необходимости писать MDX-код вручную.

### **ПРИМЕЧАНИЕ**

Как уже упоминалось, более подробно мы рассмотрим MDX в следующей главе. Если синтаксис или содержимое выражений для вас пока не ясны, не переживайте по этому поводу — позже все станет понятнее.

Представим себе, что компания *Northwind Trading Company* определила в качестве тестовых рынков города *Albuquerque*, *Boise* и *Kirkland*, и поэтому нужно часто смотреть объемы продаж по этим трем рынкам. При помощи именованного набора (содержащего три этих города) мы можем дать пользо-

вателям простой способ запроса данных только по этим тестовым рынкам. Нам также понадобится видеть чистые объемы продаж, чтобы не обращаться отдельно к двум мерам — Total Sales и Discount. Вычисляемый элемент (определенный как разница между Total Sales и Discount) дает простой способ увидеть чистые продажи без необходимости встраивания данных в таблицу фактов или запросы.

Мы можем также определить дополнительный вычисляемый элемент, который указывает рост наших чистых продаж за любой период времени (сравнением чистых продаж за данный период и за предыдущий). Вы скоро увидите, что вычисления Analysis Services и язык MDX очень легко позволяют сделать это.

## Вычисляемые элементы

Давайте начнем с создания вычисляемых мер (рис. 16.20).

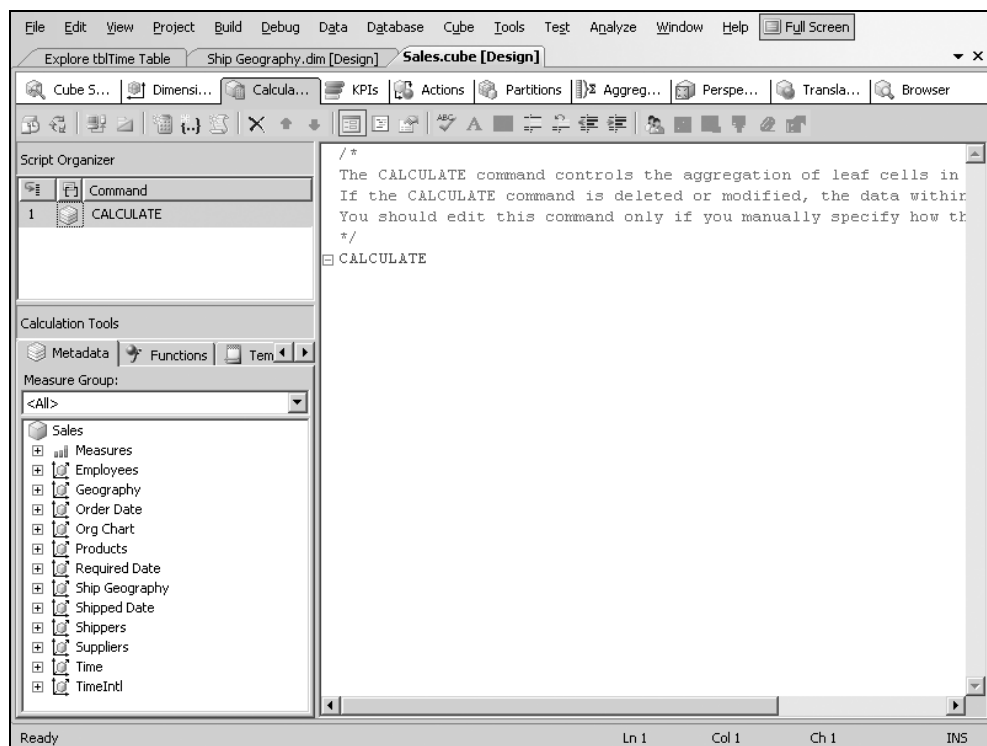


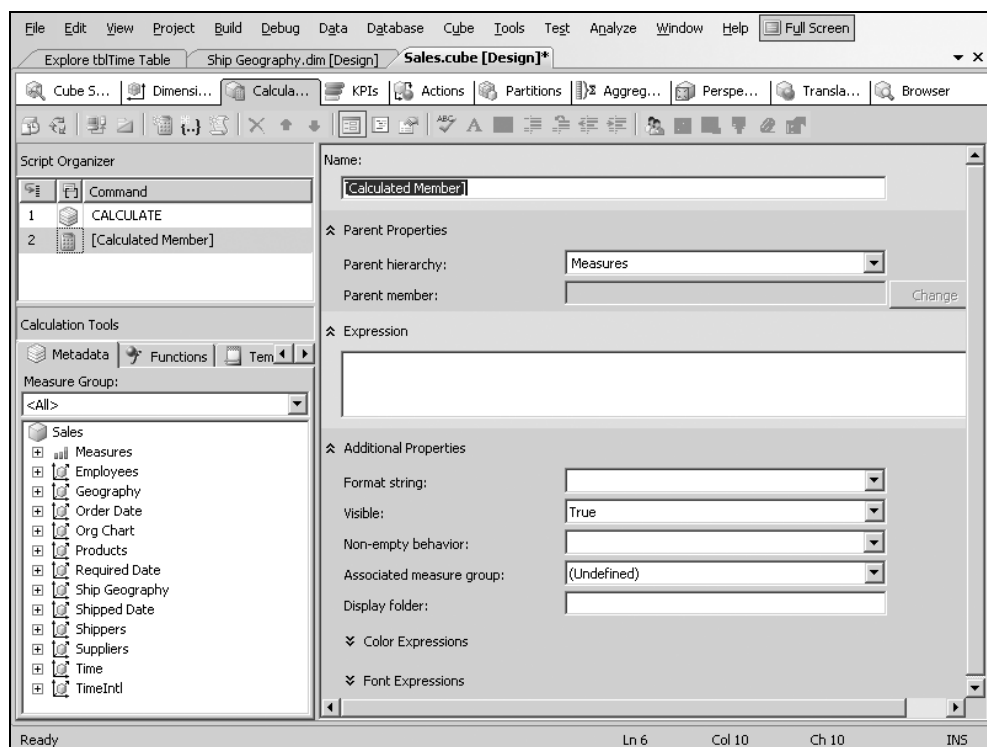
Рис. 16.20. Вкладка **Calculations** визуального конструктора кубов

**ПРИМЕЧАНИЕ**

Поскольку создаваемые в данной главе вычисляемые элементы принадлежат иерархии Measures, то они могут быть названы вычисляемыми мерами. Поэтому термины "вычисляемый элемент" и "вычисляемая мера" используются взаимозаменяемо.

Одно вычисление здесь уже определено; не трогайте его, поскольку правильное вычисление любых новых элементов будет зависеть от него. Нажмите кнопку **New Calculated Member** в панели инструментов визуального конструктора (или выберите пункт **New Calculated Member** контекстного меню панели **Script Organizer**). Это действие добавит в куб новый вычисляемый элемент, который вы теперь должны будете настроить.

Основная область окна визуального конструктора отображает форму, где запрашиваются различные параметры вычисляемого элемента (рис. 16.21).

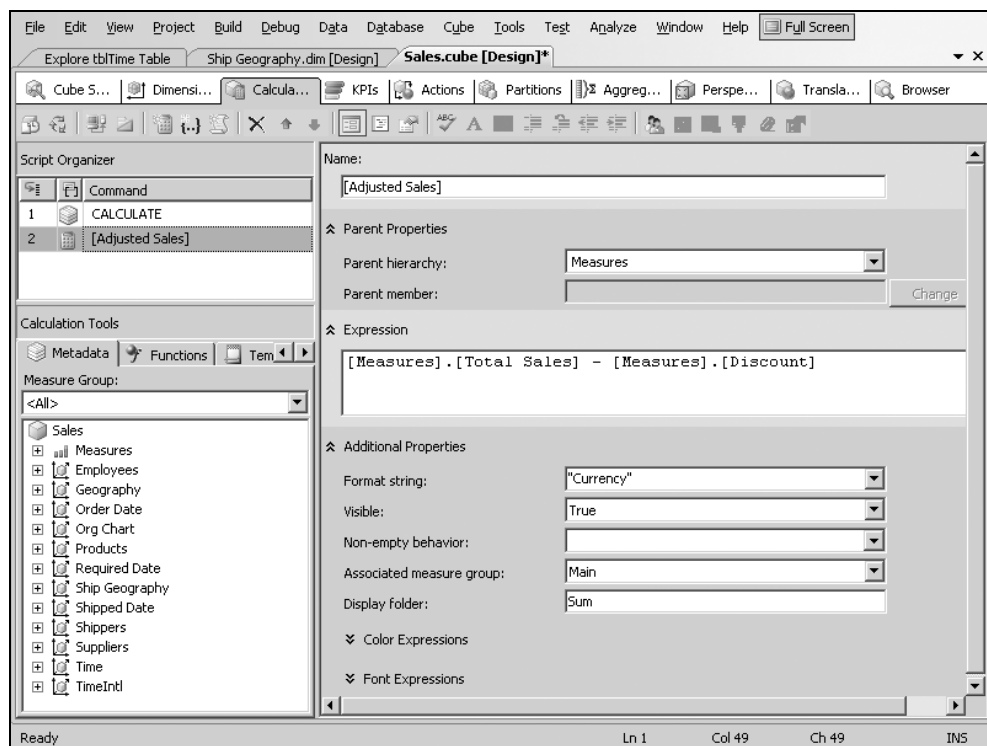


**Рис. 16.21.** Вкладка **Calculations** визуального конструктора кубов (представление в виде формы)

Введите [Adjusted Sales] в поле Name (все названия вычисляемых элементов с пробелами должны заключаться в квадратные скобки); убедитесь, что в выпадающем списке **Parent Hierarchy** выбрана мера Measures, и введите в текстовой области **Expression** следующее выражение MDX:

[Measures].[Total Sales] – [Measures].[Discount]

Теперь выберите формат Currency в выпадающем списке **Format String**. Выберите Main в выпадающем списке **Associated Measure Group**, а затем введите Sum в текстовом поле **Display Folder**. Заполненная форма вычисляемого элемента для [Adjusted Sales] показана на рис. 16.22.



**Рис. 16.22.** Вкладка **Calculations** (представление в виде формы) для вычисляемого элемента Adjusted Sales

Вы только что определили вычисляемую меру с названием Adjusted Sales (которая является разностью между значениями мер Total Sales и Discount). Для того чтобы увидеть сгенерированный визуальным конструктором MDX-код, нажмите кнопку **Script View** панели инструментов (или вы-

берите **Cube | Show Calculations In | Script**). Начиная со второй строки скрипта MDX, вы должны увидеть следующий MDX-код, который представляет собой только что созданную вами вычисляемую меру:

```
CREATE MEMBER CURRENTCUBE.[Measures].[Adjusted Sales]
  AS [Measures].[Total Sales] - [Measures].[Discount],
FORMAT_STRING = "Currency",
VISIBLE = 1 , DISPLAY_FOLDER = 'Sum' ,
      ASSOCIATED_MEASURE_GROUP = 'Main' ;
```

Вы видите, что каждое поле в визуальном конструкторе просто запрашивает у вас выражение для команды `CREATE MEMBER`.

## Сложные выражения

Язык запросов MDX позволяет встраивать в вычисления мощную декларативную аналитическую функциональность. Несмотря на то, что глубокое изучение MDX выходит за пределы данной главы, давайте создадим еще одно вычисление и в этом процессе оценим мощь MDX.

Сначала сохраните и разверните свои изменения, на этот раз при помощи выбора **Build | Deploy Chapter16**, чтобы [Adjusted Sales] можно было использовать в выражении для новой вычисляемой меры.

Нажмите кнопку **Reconnect** панели инструментов вкладки **Calculations** для обновления вкладки **Metadata** панели **Calculation Tools**. Переключитесь в представление формы при помощи нажатия кнопки **Form View** панели инструментов (или при помощи выбора **Cube | Show Calculations In | Form**). В панели **Script Organizer** выберите только что созданный вами вычисляемый элемент [Adjusted Sales] и добавьте новый вычисляемый элемент.

### СОВЕТ

Новые вычисления всегда вставляются после выбранного. Выбор вычисляемого элемента [Adjusted Sales] перед добавлением нового гарантирует, что новый элемент будет добавлен в конец скрипта.

Назовите новый вычисляемый элемент [Adjusted Sales Growth], а затем выберите Measures в выпадающем списке **Parent Hierarchy** (если еще не выбрано) и формат Percent в выпадающем списке **Format String**. Выберите Main в выпадающем списке **Associated Measure Group**, а затем введите Sum в текстовом поле **Display Folder**. Теперь поместите свой курсор в текстовое

поле **Expression**. Выражение MDX для этого вычисления будет сложнее, чем предыдущее, поэтому давайте осваивать его постепенно.

Сначала введите следующий текст:

```
CASE
```

```
    WHEN
```

Не забудьте ввести один пробел после ключевого слова **WHEN**. Это позволит вам вставлять код сразу же после него и обеспечит наличие пробела между ключевым словом **WHEN** и вставленным кодом.

## Использование инструментов вычислений

Следующий раздел кода чреват ошибками (если вводить его вручную), поэтому для продолжения ввода нашего выражения MDX мы воспользуемся панелью **Calculations Tools**. Вы можете представить себе панель **Calculations Tools** и ее вкладки **Metadata**, **Functions** и **Templates** как нечто вроде технологии IntelliSense с перетаскиванием: вы можете перетаскивать объекты куба, прототипы вызовов функций MDX или целые шаблоны кода в зону **Expression**, и соответствующий перетаскиваемому элементу код будет вставлен для вас в эту зону.

### **ВАЖНО**

Панель **Calculation Tools** является инструментом кодировщика. Поэтому она видна не только тогда, когда вкладка **Calculations** визуального конструктора кубов находится в представлении формы, но и тогда, когда она находится в представлении скрипта.

Если вкладка **Metadata** в панели **Calculation Tools** еще не выбрана, щелкните по ней для того, чтобы она стала активной. Затем углубитесь в измерение **Order Date** и найдите иерархию **Year-Quarter-Month-Date**, которая должна появиться как последний дочерний узел в узле измерения. Перетащите эту иерархию в зону **Expression**, разместив в конце второй строки (на один пробел правее ключевого слова **WHEN**). Текст вашего выражения теперь должен выглядеть следующим образом:

```
CASE
```

```
    WHEN [Order Date].[Year — Quarter — Month — Date]
```

Если код выглядит иначе, вы можете его вручную отредактировать.



Продолжая выполнять операции перетаскивания и ручного редактирования, вы должны получить выражение MDX со следующим кодом (однако пока не вводите его):

```
CASE
```

```
    WHEN [Order Date].[Year - Quarter - Month -  
        Date].CURRENTMEMBER.LEVEL.ORDINAL = 0 THEN NULL  
    WHEN ISEMPTY(([Order Date].[Year - Quarter - Month -  
        Date].PREVMEMBER, [Measures].[Adjusted Sales])) THEN "EMPTY"  
    ELSE  
        [Measures].[Adjusted Sales]/  
        ([Order Date].[Year - Quarter - Month - Date].PREVMEMBER,  
        [Measures].[Adjusted Sales]) - 1
```

```
END
```

Скоро мы обсудим, что именно вычисляет это выражение; но сначала мы обсудим то, как использовать элементы на разных вкладках панели **Calculation Tools** для построения нашего кода.

Свойства CURRENTMEMBER и PREVMEMBER можно найти и перетащить с ветви **Navigation** древовидного представления вкладки **Functions**. Однако это может быть не слишком удобно. Проще использовать пункты контекстного меню **List Members** или **Complete Word** в зоне редактирования выражения. Вы можете также использовать аналогично названные опции в меню **Edit | IntelliSense**. Еще более удобными вам могут показаться впоследствии комбинации клавиш <Ctrl>+<J> (для **List Members**), <Ctrl>+<Space> или <Alt>+<→> (для **Complete Word**).

Ключевые слова LEVEL и ORDINAL можно найти в узле **Metadata** вкладки **Functions**. Их также можно перетаскивать, но это тоже не слишком удобно (требует ручной замены сгенерированных префиксов <<Member>> и <<Level>>). Иногда лучше использовать панель **Calculation Tools** как справочное руководство (для напоминания синтаксиса MDX), иногда может быть полезным и перетаскивание. Обычно сложные выражения (с длинными названиями измерений, иерархий, уровней и элементов) лучше перетаскивать, чем набирать вручную.

### ПРИМЕЧАНИЕ

Элементы вкладки **Templates** панели **Calculation Tools** предлагают скелетные версии длинных конструкций кода и дают вам отличное начало для освоения MDX. Элементы из этой вкладки должны перетаскиваться.

Вы обнаружите, что элемент `[Measures].[Adjusted Sales]` можно перетащить из вкладки **Metadata**; для этого разверните узел **Measures** вкладки **Metadata**, потом разверните подузлы **Main** и **Sum**, а затем перетащите узел **Adjusted Sales** в зону **Expression**.

Независимо от того, как вы составляете код выражения, вы должны сделать его синтаксическую проверку. Для этого нажмите кнопку **Check Syntax** (со значком проверки правописания) или выберите **Cube | Check Syntax**. Должно появиться окно с сообщением, что проверка синтаксиса прошла успешно. Если это не так, то проверьте текст формулы или откройте решение Analysis Services с названием `Chapter16End` (имеющееся в примерах кодов данной главы). В нем откройте куб `Sales` в визуальном конструкторе кубов, щелкните по вкладке **Calculatons**, выберите вычисляемый элемент `[Adjusted Sales Growth]` и скопируйте текст выражения оттуда.

## Расшифровка кода

Теперь давайте обсудим, как работает только что созданное нами выражение вычисления. Самый простой способ понять это выражение — это начать с конца и подниматься вверх. Начнем с формулы внутри оператора `ELSE` нашего выражения:

```
[Measures].[Adjusted Sales]/  
    ([Order Date].[Year - Quarter - Month - Date].PREVMEMBER,  
    [Measures].[Adjusted Sales])) - 1
```

Вводная часть этой формулы вычисляет значение меры `Adjusted Sales` для предыдущего периода иерархии `Year-Quarter-Month-Date` измерения `Order Date`. Затем `Adjusted Sales` за текущий период делится на это значение, и, наконец, из частного вычитается число 1. Полученное значение — это прирост вычисляемой меры `Adjusted Sales` по сравнению с предыдущим периодом.

Предыдущий и текущий периоды (на которые ссылается выражение) определяются контекстом, который установлен вызывающим эту функцию запросом. Например, если выражение вычисляется для элемента уровня `Year`, то выражение сосчитает годовой прирост `Adjusted Sales`. Если же оно вычисляется для элемента уровня `Quarter`, то выражение даст прирост по кварталам. Скоро вы узнаете, как можно использовать этот вычисляемый элемент в браузере кубов (чтобы увидеть прирост `Adjusted Sales` на нескольких уровнях иерархии `Year-Quarter-Month-Date` измерения `Order Date`).

Остальную часть выражения понять просто, она обрабатывает сценарии неверных сравнений. Двигаемся по коду вверх: если выражение подсчитывается для самого первого периода на данном уровне (например, January 1996 на уровне Month), то PREVMEMBER будет пустым, и формула вернет запрограммированную строку EMPTY. Если же передаваемый формуле период находится на уровне 0 иерархии (элемент All), то формула вернет NULL (поскольку элемент All по определению не может иметь предшественника).

## Именованные наборы

Перед тем как развернуть и протестировать эти новые вычисления, давайте добавим именованный набор, который ссылается на три города (тестовые рынки). Сделать это очень просто. Убедитесь, что вычисляемый элемент [Adjusted Sales Growth] выбран в панели **Script Organizer** вкладки **Calculations** визуального конструктора кубов, а затем добавьте новый именованный набор при помощи кнопки **New Named Set** панели инструментов вкладки **Calculations**. Назовите набор [Test Markets] и введите в зоне **Expression** следующий текст:

```
{ [Geography].[City].&[Albuquerque] , [Geography].[City].&[Boise] ,  
[Geography].[City].&[Kirkland] }
```

### СОВЕТ

Вам может показаться проще составлять выражение при помощи ввода фигурных скобок, запятых и пробелов и последующего генерирования остального кода посредством перетаскивания соответствующих элементов атрибута City измерения Geography из вкладки **Metadata** панели **Calculation Tools**. Для этого вам придется развернуть узел **Sales\Geography\City\Members\All**, а затем перетащить каждый из трех городов из отсортированного по алфавиту списка в соответствующие точки вставки текстового поля **Expression**.

Использование перетаскивания для указания элементов набора приводит к очень формальному синтаксису сгенерированного кода, например, символ & и квадратные скобки вокруг каждого названия технически не нужны. Скобки избыточны потому, что ни одно из названий не содержит внутренних пробелов. Символ амперсанда используется для ссылок на ключ элемента (а не на его название). В нашем случае название и ключ — это одно и то же, и названия уникальны. Можете оставить скобки и амперсанды в коде выражения, чтобы адаптировать изменения названий или ключей элементов.

## Дополнительная информация по представлению скриптов

Сохраните ваши изменения, но перед их развертыванием переключитесь обратно в представление скриптов для того, чтобы увидеть весь сгенерированный код. Обратите внимание, что каждое введенное вами вычисление (а также и команда `Calculate`, которая была там с самого начала) просто формирует последовательные блоки единого MDX-скрипта. Эти фрагменты кода появляются в скрипте в том же самом порядке, в котором они видны в панели **Script Organizer** представления форм.

Здесь нет .NET-кода, однако представление форм и представление скриптов очень похожи на работу в Visual Studio над .NET-проектом. В MDX не существует концепции событий или "выделенного кода", однако динамический или сгенерированный визуальным конструктором код здесь очень часто используются, а изменения в одном представлении влияют на другое представление.

Более того, редактор кода поддерживает множество таких же функций, что и редактор кода в .NET-проектах: развертывание и свертывание блоков кода; дописывание слов; списки элементов; синтаксические подсказки в виде волнистых линий в реальном времени; информация о параметрах в виде подсказок; ввод комментариев либо при помощи синтаксиса T-SQL (`/*comment*/` и `--comment`), либо в стиле языка C (`//comment`). Комментарии делают ваш MDX-код более внятным и не мешают правильному отображению представления форм.

### Подсказка

Даже тогда, когда вы не находитесь в представлении скриптов, текстовая зона **Expression** поддерживает многие из только что описанных функций.

## Режим отладки

Сходство с .NET-проектами на этом не заканчивается, оно распространяется и на работу в режиме отладки. Вы можете устанавливать и отменять точки останова или выбрать **Debug | Step Over** при попадании на точку останова. Это приводит нас к важному соображению: до настоящего момента мы считали выбор **Debug | Start Debugging** и нажатие кнопки **Start Debugging** панели инструментов эквивалентными развертыванию проекта на сервер

Analysis Services и обработке куба. На деле же это не совсем так. Выбор **Build | Deploy** с именем проекта действительно выполняет такую задачу, но вариант **Start Debugging** из панели инструментов (или меню) разворачивает проект и входит в скрипт вычисления вашего куба (если визуальный конструктор куба открыт и выбрана вкладка **Calculations** при запуске **Start Debugging**).

Попробуйте проверить это сами — нажмите кнопку панели инструментов **Start Debugging** или выберите **Debug | Start Debugging** при открытой вкладке **Calculations**. После сборки и развертывания проекта Visual Studio помещает вас в представление скриптов, даже если при запуске **Start Debugging** вы были в представлении форм, а затем вся интегрированная среда разработки переходит в режим отладки (как в случае .NET-проекта). Становится видимой панель инструментов **Debug**, полностью активируется меню **Debug**, а на вкладке **Calculations** открывается специальная панель **Debug**.

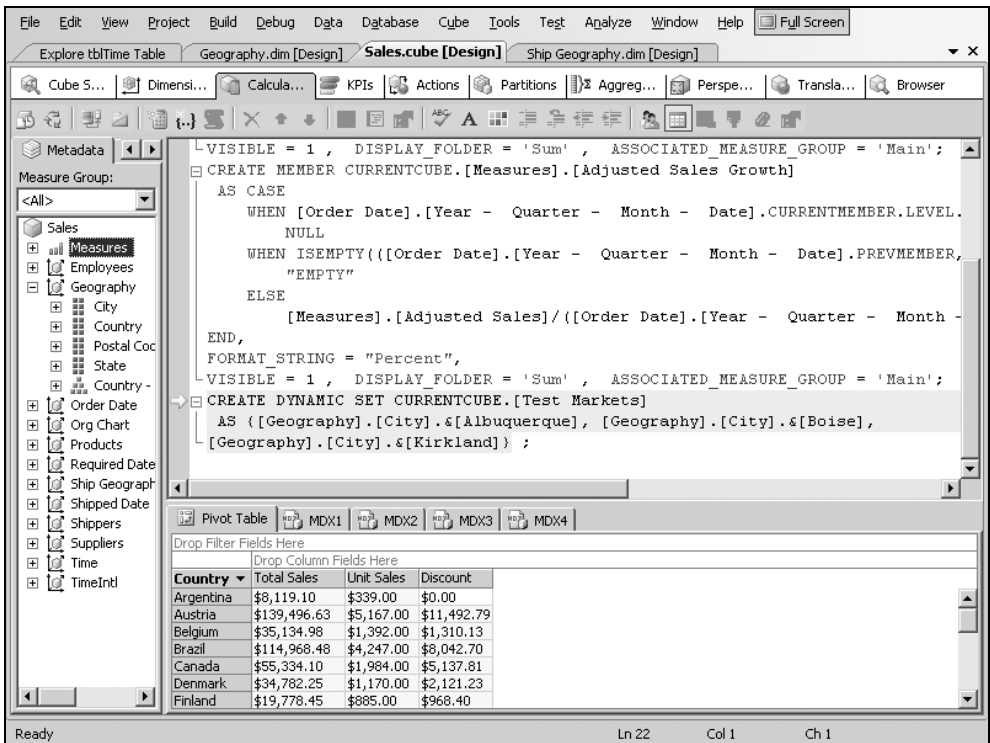


Рис. 16.23. Вкладка **Calculation** в режиме отладки (выделена панель **Debug**)

Панель **Debug** предоставляет вкладку **Pivot Table** (на которую вы можете перетаскивать объекты из вкладки **Metadata**) и четыре интерактивных вкладки **MDX** для диагностических запросов. Режим отладки вкладки **Calculations** показан на рис. 16.23.

Обратите внимание на возможность пошагового прохода по скрипту MDX. Этот режим может сэкономить вам многие часы утомительной отладки (не придется переходить туда и обратно между окнами Visual Studio и SQL Server Management Studio).

## Обратно к браузеру

Перед тем как завершить наше обсуждение вычислений, давайте опишем их. Если вы все еще в режиме отладки, то выберите **Debug | Stop Debugging** или нажмите кнопку **Stop Debugging** на панели инструментов **Debug**. Нажмите вкладку **Browser** визуального конструктора кубов и повторно подключитесь. В древовидном представлении метаданных углубитесь в узел **Measures**, затем в его дочерний узел группы мер **Main**, а потом в его дочерний узел каталога отображения **Sum**. Углубитесь также в узел **Geography**. Эти действия позволят вам увидеть ваши вычисляемые меры и именованный набор.

Теперь сконструируем запрос с использованием вашего нового вычисления. Для этого необходимо выполнить следующие шаги:

1. Перетащите иерархию Year-Quarter-Month-Date измерения Order Date в зону **Row Fields**.
2. Перетащите вычисляемые элементы Adjusted Sales и Adjusted Sales Growth в зону **Detail Fields**. (Разместите Adjusted Sales Growth справа от Adjusted Sales).
3. Перетащите именованный набор Test Markets в панель фильтров (но только не в зону **Drop Filter Fields Here** панели **Pivot Table**).
4. В зоне **Row Fields** углубитесь в узел **Calendar 1997**, а затем в узел **Quarter 1, 1997**.

Результаты этих перетаскиваний показаны на рис. 16.24.

Обратите внимание, что значение Adjusted Sales Growth для Calendar 1996 равно EMPTY (поскольку отсутствует предыдущий период для вычисления прироста). Для других ячеек вы должны увидеть правильные цифры роста (некоторые из которых будут отрицательными).

Отображаемые данные фильтруются для того, чтобы показывать объемы продаж только для трех наших тестовых рынков. Если вы хотите увидеть данные всего куба, то щелкните правой кнопкой мыши по строке именованного набора в панели **Filter** и выберите пункт **Delete** в контекстном меню. Затем вы можете углубиться в один из узлов отображаемых на панели **Pivot Table** месяцев (чтобы увидеть ежедневные данные Adjusted Sales Growth). Если бы вы сделали это при задействованном фильтре именованного набора Test Markets, то увидели бы много пустых ячеек.

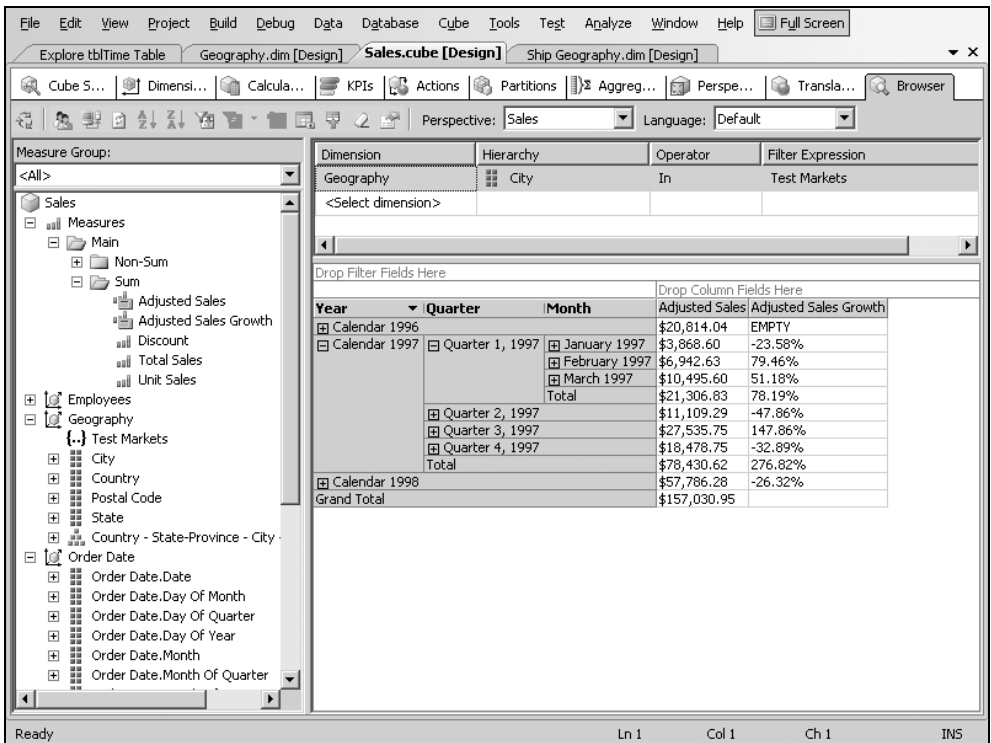


Рис. 16.24. Браузер кубов с вычисляемыми мерами и именованным набором

## Ключевые индикаторы производительности

Меры и измерения определяют то, что куб может показать (давая вам представление о вашем бизнесе, трафике вашего Web-сайта либо об открытиях клинического эксперимента). Вы можете выполнить запрос MDX и посмот-

реть значения меры по некоторому срезу куба, а поскольку вы знакомы с его мерами и измерениями, то вы можете решить, соответствуют ли они вашим целям и ожиданиям. При помощи ключевых индикаторов производительности Analysis Services вы можете встроить эти целевые критерии в сам куб и не оценивать их в голове или в вашем приложении.

Ключевые индикаторы производительности (Key Performance indicators, KPI) состоят из набора выражений MDX (хранящихся на сервере), которые возвращают числовые значения, характеризующие состояние меры в течение некоторого периода времени. Итак, KPI дают инструмент визуализации ваших мер, способ понять, что именно вам говорят данные, и они хорошо подходят для мониторинга бизнес-аналитики.

### ПРИМЕЧАНИЕ

Компонент Monitoring And Analytics сервера Microsoft PerformancePoint Server (который работает в тандеме со службами Analysis Services) предоставляет платформу на базе Microsoft Office SharePoint для разработки панелей управления, систем показателей и KPI для конечных пользователей.

Давайте настроим KPI. К сожалению, вкладка **Browser** куба не отображает их и не позволит делать к ним запросы, но после мы займемся этой проблемой и будем запрашивать значения наших KPI при помощи окна **MDX Query**.

Щелкните по вкладке **KPI** визуального конструктора кубов, а затем нажмите кнопку **New KPI** панели инструментов (или выберите **Cube | New KPI**). Вы должны увидеть открывшийся новый KPI в представлении форм, которое очень похоже на представление форм вкладки **Calculations**. Основная (правая) область вкладки содержит форму для определения KPI. Слева внизу находится панель **Calculation Tools**, а слева сверху — панель **KPI Organizer**, аналогичная панели **Script Organizer** вкладки **Calculations**.

Поля формы KPI содержат название, группу мер, с которой он связан, а также выражения MDX (для отслеживаемого значения и для "целевого" значения). Значение KPI и целевое значение — это числовые индикаторы, которые сами по себе не имеют никакого особенного значения.

## Визуализация KPI: состояние и тенденция

Определенную ценность имеют выражения MDX для вычисления состояния и тенденции KPI. Выражение состояния возвращает значение (обычно в диапазоне от  $-1$  до  $+1$ ), которое указывает состояние KPI. Выражение тенденции



возвращает значение (обычно в том же диапазоне), указывающее на увеличение или уменьшение значения по сравнению с предыдущим периодом времени. Вместе с выражениями вы можете выбрать и соответствующие индикаторные значки.

Значение состояния +1 означает "хорошее" состояние дел (независимо от того, что отслеживает KPI). Значение -1 означает "плохое" состояние, а значение 0 является "нейтральным". Значения тенденции +1, 0 и -1 представляют собой соответственно увеличение, отсутствие изменений и уменьшение значения KPI с течением времени. Выбранные наборы значков состояния и тенденции представляют отдельные дискретные значения, возвращаемые выражениями.

Эти значки как минимум поддерживают значения -1, 0 и 1. Например, светофор показывает красный, желтый или зеленый свет. Другие могут отображать промежуточные значения (обычно -0.5 и +0.5), которым мы можем сопоставить состояния "посредственно" и "удовлетворительно". Эти промежуточные значения (и значки, которым они соответствуют) имеют смысл только для состояния; тенденция по определению всегда положительна, отрицательна или нейтральна.

## Конкретный KPI

Все это весьма абстрактно, но станет понятнее после того, как мы создадим KPI и начнем его использовать. Наш KPI будет основан на мере Avg Total Sales. Настройте его следующим образом:

1. Установите его название **Name** в значение Average Total Sales KPi (квадратные скобки здесь не нужны).
2. В качестве **Associated Measure Group** выберите **Main**.
3. Установите значение **Value Expression** в [Measures].[Avg Total Sales]. Вы можете перетащить узел Measures\Main\Non-Sum\Avg Total Sales из вкладки **Metadata** панели **Calculation Tools** в зону **Value Expression** (вместо того, чтобы вводить все это вручную).
4. Настройте в **Goal Expression** значение 1500.
5. Установите **Status Indicator** в значение Gauge (если это еще не сделано).
6. Введите в зоне **Status Expression** код MDX из листинга 16.1. Возможно, для облегчения выполнения этого шага вы захотите перетащить элементы из каталога KPI вкладки **Functions** панели **Calculation Tools**.

**Листинг 16.1. Код MDX для выражения состояния Average Total Sales**

```

CASE
    WHEN KPIVALUE("Average Total Sales KPI") /
        KPIGOAL("Average Total Sales KPI") >= .95 THEN 1
    WHEN KPIVALUE("Average Total Sales KPI") /
        KPIGOAL("Average Total Sales KPI") < .95
    AND
    KPIVALUE("Average Total Sales KPI") /
        KPIGOAL("Average Total Sales KPI") >= .85 THEN 0
    ELSE -1
END

```

Выражение MDX в листинге 16.1 определяет средние продажи как процент от Goal (1500 долларов) и помечает в качестве положительных значения, которые составляют 95% и выше. Средние продажи свыше 85% и меньше 95% от целевого значения являются нейтральными (0), а ниже 85% — отрицательными (-1).

7. Установите значение **Trend Indicator** в Standard Arrow (если это еще не сделано).
8. Занесите в **Trend Expression** содержимое листинга 16.2 (при желании это можно сделать при помощи перетаскивания из панели **Calculation Tools**).

**Листинг 16.2. Код MDX для выражения тенденции Average Total Sales**

```

CASE
    WHEN ISEMPY(( [Measures].[Avg Total Sales],
        [Order Date].[Year - Quarter - Month - Date].PREVMEMBER)) THEN
    NULL
    WHEN [Measures].[Avg Total Sales] < ([Measures].[Avg Total Sales],
        [Order Date].[Year - Quarter - Month - Date].PREVMEMBER) THEN -1
    WHEN [Measures].[Avg Total Sales] = ([Measures].[Avg Total Sales],
        [Order Date].[Year - Quarter - Month - Date].PREVMEMBER) THEN 0
    ELSE 1
END

```

Выражение MDX в листинге 16.2 использует свойство PREVMEMBER иерархии нашего измерения Order Date для сравнения Adjusted Sales Growth за те-

кущий период с его значением в предыдущий период. Если значение предыдущего периода ниже, то рост положительный, и выражение тенденции возвращает значение +1; если значение выше, то рост отрицательный, и возвращается значение -1. Если значения равны, то роста нет, и возвращается значение 0.

### **ПРИМЕЧАНИЕ**

В реальных условиях лучшим вариантом при отсутствии роста было бы возвращение 0 для некоторого диапазона значений (от "немного меньше" нуля до "немного больше" нуля), но для данного примера мы остановились на разности, равной нулю.

Нажмите выпадающую стрелку рядом с заголовком **Additional Properties** для развертывания этого раздела формы. Мы не будем вводить здесь значения всех полей, но знать имеющиеся варианты нужно. Каталоги отображения позволяют группировать KPI по категориям. Вы можете ввести несколько названий каталогов (через точку с запятой), что позволит вашим KPI появиться более, чем в одном каталоге.

Родительский KPI иерархическим образом устанавливает связь вычислений, когда значение родительского KPI определяется значениями его потомков. Относительный вклад каждого значения дочернего KPI определяется значением, введенным в поле **Weight**. В поле **Current Time Member** вы можете указать исключительный период времени, в течение которого действует данный KPI. Вы должны указать элемент измерения времени, определенный для той группы мер, с которой связан KPI.

### **ПРИМЕЧАНИЕ**

Помните, что значения полей **Value Expression**, **Goal Expression**, **Status Expression**, **Trend Expression**, **Weight** и **Current Time Member** — это полноценные выражения MDX, которые поддерживают все конструкции языка MDX, дописывание слов, списки элементов, подсказки синтаксиса в виде волнистых линий, а также ввод комментариев.

И наконец, поле **Description** позволяет вам документировать KPI посредством сохранения описательной строки текста в самом кубе. Этот текст можно взять из клиентского приложения, что облегчает пользователям просмотр вашего куба. Введите *Monitors health of average total sales* в поле **Description** вашего KPI и оставьте остальные поля раздела **Additional Properties** пустыми.

После завершения определения нашего KPI настало время его протестировать, но наш метод тестирования будет отличаться от тестирования других элементов нашего куба. Мы опишем эту разницу буквально через несколько минут, но сначала сохраните и разверните ваши изменения. Убедитесь, что вы делаете это при открытой вкладке **KPI**.

## Тестирование KPI в представлении браузера

После развертывания обратите внимание, что вкладка **KPI** переключилась из представления форм в представление браузера. Кнопка **Browser View** панели инструментов нажата, и поля формы исчезли, уступив место новому окну. В верхней панели этого окна вы можете указать срез вашего куба, для которого нужно просмотреть данные KPI; в нижней панели отображаются данные KPI. Эти данные включают в себя значение текущего среза, целевое значение, графические изображения значений состояния и тенденции, а также значение веса. Есть также и информационный значок; вы можете навести на него курсор вашей мыши для показа описания KPI в виде подсказки.

Браузер KPI дает легкий способ просмотра определенных подразделов (срезов) вашего куба. Однако по умолчанию браузер отображает агрегированные данные KPI для всего куба, что почти бессмысленно. Для сужения области охвата требуется сделать указания в панели фильтров. Например, мы можем посмотреть данные KPI для Бразилии за четвертый квартал 1996 г., если сделаем следующие установки в панели фильтров, как в табл. 16.1 (сначала надо повторно подключиться, чтобы получить разрешение сделать эти установки).

Таблица 16.1

| Измерение  | Иерархия                | Оператор | Выражение фильтра |
|------------|-------------------------|----------|-------------------|
| Order Date | Year-Quarter-Month-Date | Equal    | {Quarter 4, 1996} |
| Geography  | Country                 | Equal    | {Brasil}          |

Для введения значения фильтра вы должны нажать стрелку выпадающего списка на правом краю столбца. Затем (во всплывающем окне браузера измерений) углубитесь до того элемента, который вас интересует, и нажмите кнопку **ОК**. Состояние выбора элемента {Quarter 4, 1996} измерения Order Date показано на рис. 16.25.

После указания среза вашего куба выйдите из выбранной ячейки таблицы фильтров (можно щелкнуть где угодно в нижней панели), чтобы заставить загрузиться и отобразиться новые данные KPI.

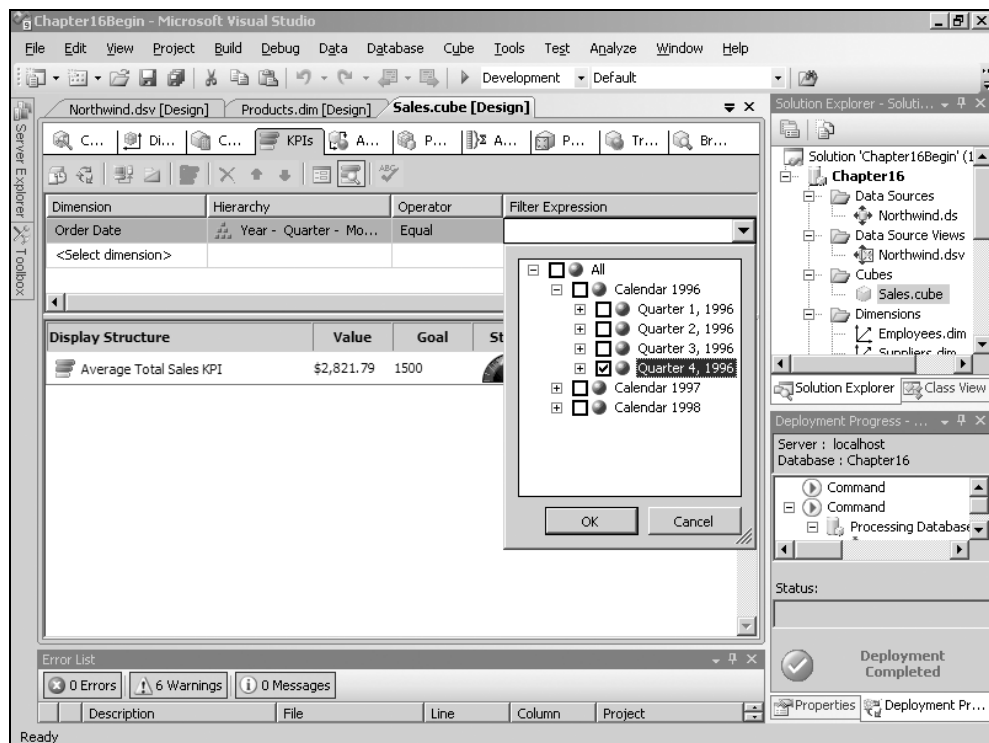
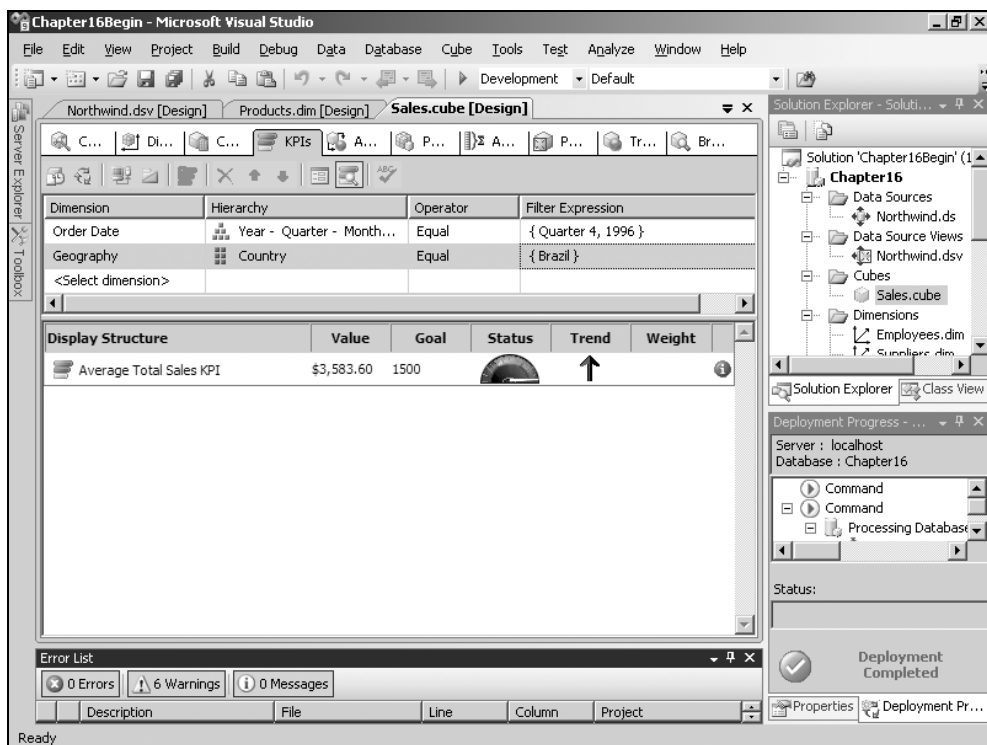


Рис. 16.25. Браузер KPI с показанным всплывающим окном выбора элемента Filter Expression

## Интерпретация результатов и модификация среза

Представление браузера KPI должно выглядеть так, как показано на рис. 16.26.

Обратите внимание, что столбец value показывает 3,583.60 (долларов), что гораздо выше цели. Соответственно графический элемент столбца status показывает стрелку вправо (что обозначает значение состояния +1). Графический элемент столбца Trend показывает стрелку вверх.



**Рис. 16.26.** Браузер KPI по срезу куба (четвертый квартал, Бразилия)

Теперь попытайтесь изменить страну на UK (не забудьте убрать Бразилию перед тем, как нажать кнопку **ОК**), период time оставьте без изменения. Вы должны увидеть уменьшение значения KPI до 1,082.23, график столбца Status укажет плохое состояние (стрелка влево), что, безусловно, правильно, так как это значение меньше 85% цели (1500 долларов), хотя график столбца Trend показывает увеличение по сравнению с предыдущим кварталом. Затем измените выражение фильтра для Order Date на {Calendar 1997}, а выражение фильтра для Geography на {Brazil, Canada}. (Не забудьте очистить старое значение фильтра UK перед нажатием кнопки **ОК**). Вы должны увидеть увеличение значения KPI до 1,420,01, график Status покажет умеренное состояние (стрелка вверх), поскольку это значение составляет примерно 94,7% от требуемого. График Trend показывает стрелку вниз, означая уменьшение по сравнению с 1996 г.

## Запросы KPI в Management Studio

На данный момент у вас должно появиться ощущение эффективности KPI и относительной простоты их создания. Графика может сначала показаться простоватой, но она делает данные вашего куба намного легче усваиваемыми и легкими для понимания.

Однако для доставки данных KPI вашим пользователям вы должны использовать нечто иное, более удобным может быть доступ из браузера кубов. (Браузер кубов может также показаться технологическим инструментом, но на самом деле это компонент PivotTable программы Excel, который доступен пользователям Office Web Components версии 2003).

К сожалению, KPI недоступны в браузере кубов, так что придется использовать другую пользовательскую технологию. Для этого мы сейчас изучим, как писать простые запросы MDX, которые возвращают данные KPI. В следующей главе мы объясним, как создать свой собственный пользовательский интерфейс на базе KPI (при помощи генерирования и выполнения этих запросов). Пока же мы просто спроектируем и выполним эти запросы в интерактивном режиме и сделаем мы это не в Visual Studio, а в Management Studio.

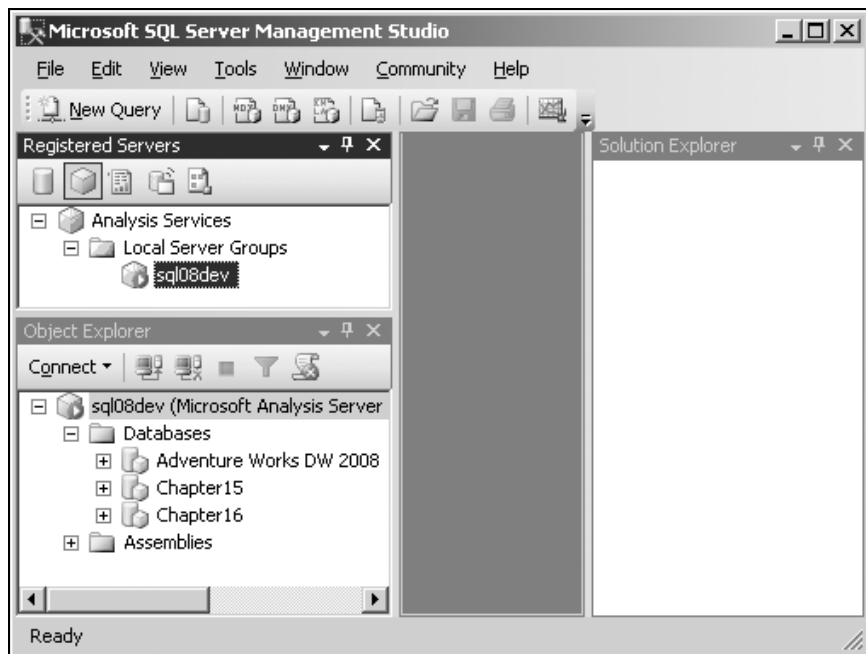
### Management Studio делает бизнес-аналитику

До сих пор мы использовали Management Studio в основном как инструмент для управления реляционными базами данных. Management Studio не только наследник Enterprise Manager и Query Analyzer в SQL Server 2000, но и инструмент управления Analysis Services 2000.

При помощи Management Studio вы можете создавать и редактировать запросы MDX для кубов OLAP. Вы можете сочетать проекты Analysis Services Scripts и SQL Server Scripts в одном решении (для того чтобы управлять всем множеством объектов ваших систем бизнес-аналитики/хранилищ данных).

Откройте Management Studio и убедитесь, что окна **Registered Servers** и **Object Explorer** видны. Если они не видны, то выберите **View | Registered Servers** и **View | Object Explorer** в Management Studio. Теперь нажмите кнопку **Analysis Services** в панели инструментов окна **Registered Servers** (вторая слева, со значком куба), а затем дважды щелкните по узлу древовидного представления вашего сервера. (Если его там нет, добавьте его.) Теперь в окне **Object Explorer** должен быть виден узел, представляющий ваш сервер

**Analysis Services** (а также дочерние узлы **Databases** и **Assemblies**). Разверните узел **Databases**, и вы должны увидеть все базы данных на вашем сервере Analysis Services (рис. 16.27).



**Рис. 16.27.** SQL Server Management Studio с нашим сервером Analysis Services в окнах **Registered Servers** и **Object Explorer**

## Окно запроса MDX

Выберите узел **Chapter16** и нажмите кнопку **New Query** (крайняя слева) в панели инструментов **Standard**. Появится окно запроса MDX. В этом окне слева имеется панель инструментов. Она содержит список кубов базы данных, вкладки **Functions** и **Metadata**, которые перечисляют меры, измерения и KPI выбранного куба. С правой стороны окна запросов MDX находится редактор MDX, который поддерживает кодирование цветом, подсказки синтаксиса в виде волнистых линий, а также смешивание комментариев и кода. Списки элементов и дописывание слов доступны при помощи выбора **Edit | IntelliSense** (или через клавиатурные комбинации).



## СОЗДАНИЕ ЗАПРОСОВ MDX

Нажатие кнопки **New Query** — это только один из нескольких способов открыть окно запроса MDX в Management Studio. Например, вы можете щелкнуть по кнопке **Analysis Services MDX Query** в панели инструментов **Standard** либо выбрать **File | New | Analysis Services MDX Query**.

Любое из этих действий откроет диалоговое окно **Connect To Analysis Services**, где вы укажете названия вашего сервера Analysis Services и базы данных. Для указания названия базы данных нажмите кнопку **Options**, щелкните по вкладке **Connection Properties** и выберите либо введите название базы данных (в выпадающем списке **Connect To Database**). Затем щелкните по **Connect**.

Вы можете избежать диалогового окна **Connect To Analysis Services** при помощи использования кнопки панели инструментов **New Query** (когда в окне **Object Explorer** выбрана конкретная база данных Analysis Services), поскольку подключение и база данных уже подразумевается. Еще одним коротким способом является выбор **New Query | MDX** в контекстном меню узла базы данных в окне **Object Explorer**.

Вы можете изменить параметры подключения открытого запроса MDX путем выбора опции **Connection | Change Connection** контекстного меню (или при помощи выбора **Query | Connection | Change Connection**; или же при помощи нажатия кнопки **Change Connection** в панели инструментов **SQL Server Analysis Services Editors**). Сервер и база данных, к которой вы сейчас подключены, отображаются в строке состояния окна запроса и в окне **Properties** (когда активно окно запроса). Текущая база данных отображается также и в панели инструментов **SQL Server Analysis Services Editors** в выпадающем списке между кнопками **Change Connection** и **Execute** (ее можно там изменить).

Если вы развернете узел **KPI** на вкладке **Metadata**, то вы увидите единственный узел нашего нового KPI; если вы развернете этот узел, то вы должны будете увидеть по одному дочернему узлу для значения, цели, состояния и тенденции данного KPI (рис. 16.28).

Вы можете перетащить любой из этих дочерних узлов в редактор запросов (для генерирования выражений **SELECT** языка MDX). Используя комбинацию ручного ввода и перетаскивания из вкладки **Metadata**, введите код листинга 16.3 в панель редактора окна запроса MDX. (Запрос находится в решении KPIQuery.mdx примеров кодов.)

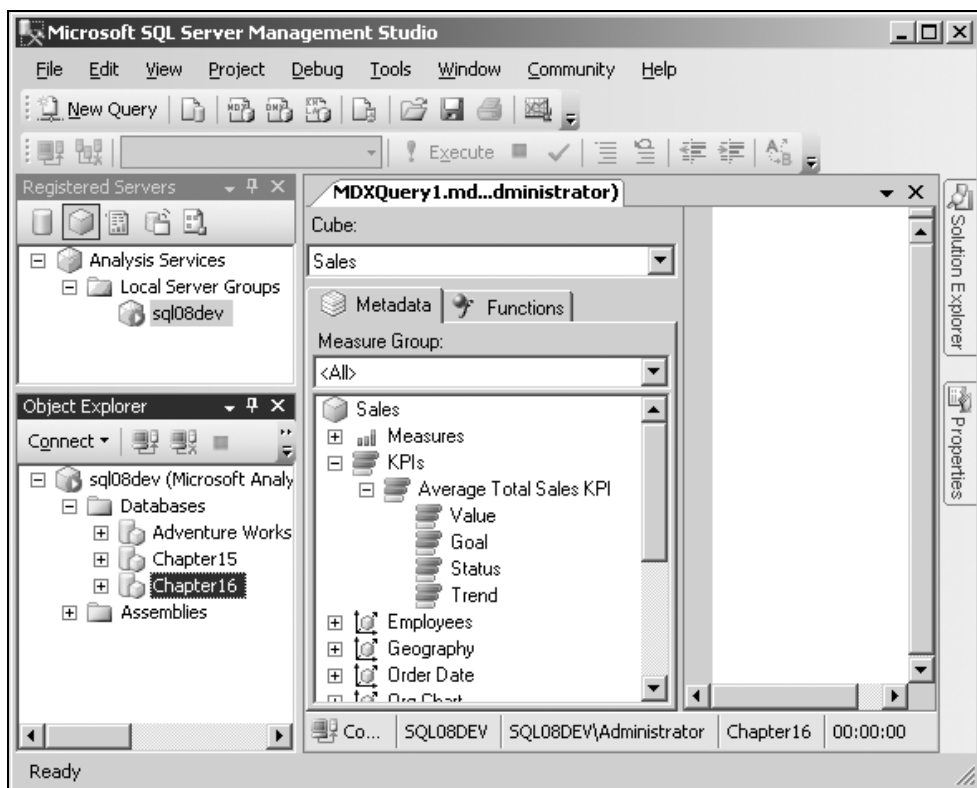


Рис. 16.28. Окно запроса MDX в Management Studio

### Листинг 16.3. Запрос MDX, пригодный для выполнения из Management Studio

```
SELECT
{
  KPIValue("Average Total Sales KPI"),
  KPIGoal("Average Total Sales KPI"),
  KPIStatus("Average Total Sales KPI"),
  KPIStatus("Average Total Sales KPI"),
  KPIStatus("Average Total Sales KPI")
}
ON COLUMNS
FROM [Sales]
WHERE
```

```
(  
[Order Date].[Year - Quarter - Month - Date].[Year].&[1997-01-  
01T00:00:00], { [Geography].[Country].&[Brazil],  
[Geography].[Country].&[Canada] }  
)
```

### СОВЕТ

В листинге 16.3 чрезвычайно непонятное выражение (использующее измерение [Order Date] в выражении WHERE) представляет собой элемент Calendar 1997 этого измерения. При помощи перетаскивания из вкладки **Metadata** вы можете избежать необходимости вручную вводить все это выражение.

Запрос выбирает данные значения, цели, состояния и тенденции по продажам в Бразилии и Канаде (данные по тому же самому срезу, которые извлекались нашим последним запросом в представлении браузера). Вы можете сделать проверку синтаксиса запроса при помощи выбора **Query | Parse** (или кнопки **Parse** со значком проверки правописания) в панели инструментов **SQL Analysis Services Editors**. Появляется панель **Results** с вкладкой **Messages**, на которой должны быть выведены следующие данные:

```
Parsing the query ...
```

```
Obtained object of type: Microsoft.AnalysisServices.AdomdClient.CellSet
```

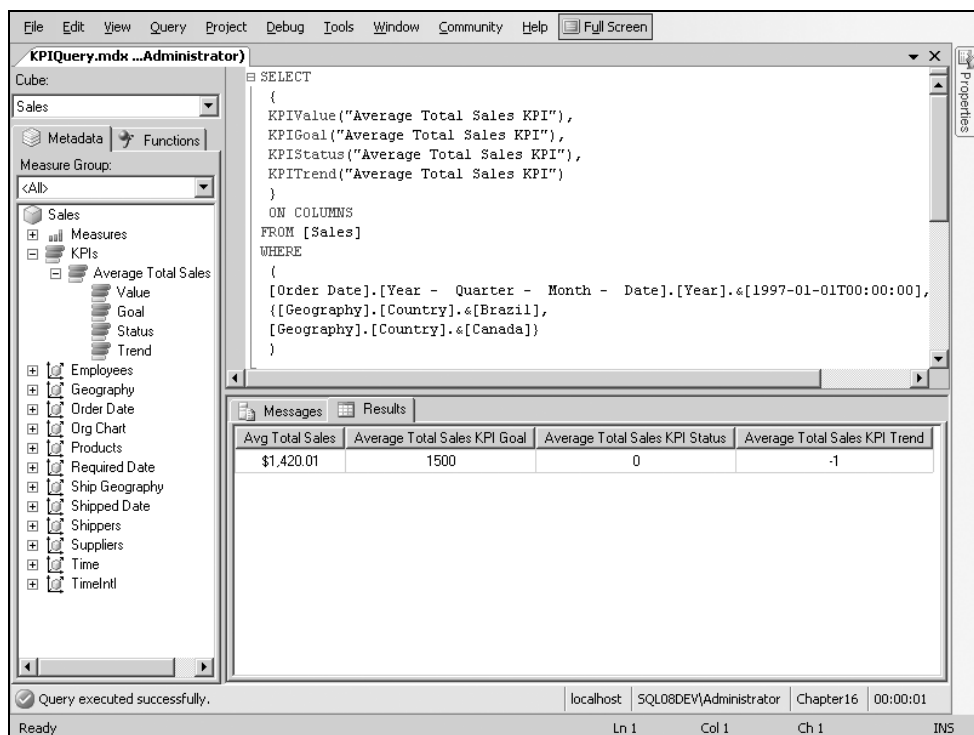
```
Parsing complete
```

Поскольку вы прошли проверку синтаксиса, выполните запрос. Вы можете найти опцию **Execute** в контекстном меню редактора запросов, в меню **Query**, либо в панели инструментов **SQL Server Analysis Services Editors**. Для выполнения запроса MDX вы можете также нажать клавишу <F5> или комбинацию клавиш <Ctrl>+<E>.

### СОВЕТ

Окно запроса MDX выполняет в качестве запроса любой текст (либо все содержимое окна редактора, если текст в нем не выделен). Опция **Parse** ведет себя точно так же. Поэтому при синтаксическом анализе или выполнении вашего запроса убедитесь, что в редакторе запросов MDX не выделен никакой текст (либо выделен весь текст).

После выполнения вашего запроса результат отображается на новой вкладке **Results** в панели **Results** редактора запросов MDX (рис. 16.29).



**Рис. 16.29.** Окно запроса MDX Query  
(показан текст запроса по KPI и его результаты)

Сравните выходные данные с представлением браузера KPI в визуальном конструкторе кубов нашего проекта Analysis Services в Visual Studio, и вы увидите, что по существу это одно и то же. Лишь значения состояния и тенденции отображаются в числовом виде (а не графически). В своем приложении вы можете написать код для выполнения этого запроса MDX и отобразить состояние KPI и данные по его тенденции графическим образом.

## Прочие фокусы бизнес-аналитики в Management Studio

Нам нужно обсудить еще добавление в наш куб действий, перспектив, переводов и ролей, а также управление хранением куба и проектирование агрегатов. Однако перед этим давайте изучим, какие еще задачи OLAP мы можем выполнить в Management Studio.

Для начала разверните в окне **Object Explorer** узел базы данных Chapter16. Вы должны увидеть дочерние узлы источников данных, представлений источников данных, кубов, измерений, структур интеллектуального анализа данных, ролей и сборок. Это все те же узлы верхнего уровня, которые видны в окне **Solution Explorer** проекта Analysis Services в среде Visual Studio. Несмотря на то, что Management Studio не дает полного управления этими элементами, вы можете их перечислять и просматривать их свойства в многостраничных окнах свойств (при помощи щелчка по любому из них и выбора пункта **Properties** в контекстном меню).

Используя контекстное меню объекта, вы можете просматривать измерения и кубы в Management Studio точно так же, как и в Visual Studio (мы опишем это в главе 18). Вы можете также обрабатывать объекты, добавлять и настраивать секции и агрегаты. Скоро мы опишем секции и агрегаты, а сейчас мы их упомянули для того, чтобы вы знали, что они доступны в визуальных конструкторах как Management Studio, так и Analysis Services.

## Скрипты XMLA в Management Studio и проекты Analysis Services Projects в Visual Studio

При помощи контекстных меню объектов вы можете также сгенерировать для них скрипты XMLA. При помощи сгенерированных скриптов объектов (или всей базы Analysis Services) вы можете без труда переносить свои базы данных на другие серверы. Вы можете передать скрипты другим разработчикам или хранить их в системе управления исходными кодами (оформив их в виде проектов Analysis Services Scripts для Management Studio).

### **СОВЕТ**

Management Studio предлагает интеграцию с системами управления исходными кодами через представление Source Control\Plug-in Selection своего диалогового окна **Options** (выберите **Tools | Options**).

Для того чтобы получить представление о скриптах XMLA, углубитесь в окне **Object Explorer** до измерения Products, щелкните его правой кнопкой мыши и выберите в контекстном меню пункт **Script Dimension As | CREATE To | New Query Editor Window**. В новом окне запроса XMLA появится скрипт XMLA для данного измерения. Обратите внимание, что каждый аспект измерения скрупулезно описан при помощи различных элементов XML.

Теперь вернитесь обратно в наш проект Analysis Services, углубитесь в окне **Solution Explorer** до узла измерения `Products.dim` и выберите его. Щелкните по этому узлу правой кнопкой мыши и выберите пункт **View Code** контекстного меню (либо выберите **View | Code**). Вы увидите почти такое же содержимое в формате XML. Сходство показывает, что визуальные конструкторы Analysis Services в Visual Studio — это просто графические интерфейсы скриптов XMLA. Когда вы развертываете ваши проекты из Visual Studio, то вы по существу выполняете сгенерированные визуальными конструкторами скрипты. Скрипты XMLA можно также выполнять и в Management Studio, для этого вы просто используете один из пунктов меню **Execute** или кнопку панели инструментов в окне редактора запросов XMLA.

## Действия

Возможно, вы заметили некий шаблон вычислений и KPI — хранить в вашем кубе разнообразные элементы, которые вы могли бы создать и в своем приложении или запросе. Подумайте: ваши вычисления просто представляют собой коллекцию команд, которые составляют MDX-скрипт, а каждый из ваших KPI — это коллекция MDX-выражений и литеральных значений (описание и названия графических элементов для представления состояния и тенденции), которые вы можете хранить и вычислять на клиентской стороне.

Однако по этой логике представления, функции и хранимые процедуры реляционных баз данных также могут обрабатываться вашими приложениями. Конечно, мы используем эти возможности серверной стороны, поскольку администраторы баз данных проектируют и предоставляют их для приложений и пользователей. Аналогичным же образом Analysis Services предлагают вычисления и KPI для того, чтобы передать управление ими в руки администраторов баз данных и сделать их доступными каждому.

Теперь мы быстро опишем действия, которые являются объектом (на основе MDX) серверной стороны.

## Действия определяются просто

Вы можете представлять себе действия как чувствительные к контексту внешние команды. Иначе говоря, действия позволяют вам сохранять шаблон на основе MDX, который превращается в командную строку той или иной формы. Действия связаны с объектами вашего куба, будь это структурные

объекты (такие как измерения, атрибуты, иерархии, уровни или их элементы) или реальные ячейки куба. Тип команд простирается от запросов детализации (данные ячейки реляционной базы данных) до URL (которые встраивают значение в строки SQL) или строк команд (которые также могут содержать значение). Вы можете даже создать действие, которое передает значение как параметр в отчет Reporting Services.

Действия определяются во вкладке **Actions** визуального конструктора кубов; они видны в контекстных меню браузера кубов. В ваших приложениях вы можете сделать перечисление имеющихся действий и вычислить их значения (при помощи программных способов, которые мы обсудим в следующей главе).

## Проектирование действий

Вкладка **Actions** визуального конструктора кубов (подобно вкладкам **Calculations** и **KPI**) состоит из панели **Organizer** (в данном случае это **Actions Organizer**), панели **Calculation Tools**, а также формы, которую вы должны заполнить для полного описания того действия, которое вы создаете или редактируете. Однако в отличие от вкладок **Calculations** и **KPI** вкладка **Actions** имеет только представление форм; здесь нет представлений скриптов или браузера. К настоящему моменту вы уже должны быть знакомы со значением и способами редактирования полей представления форм, так что мы можем начать создавать новые действия (без длительного изложения основ).

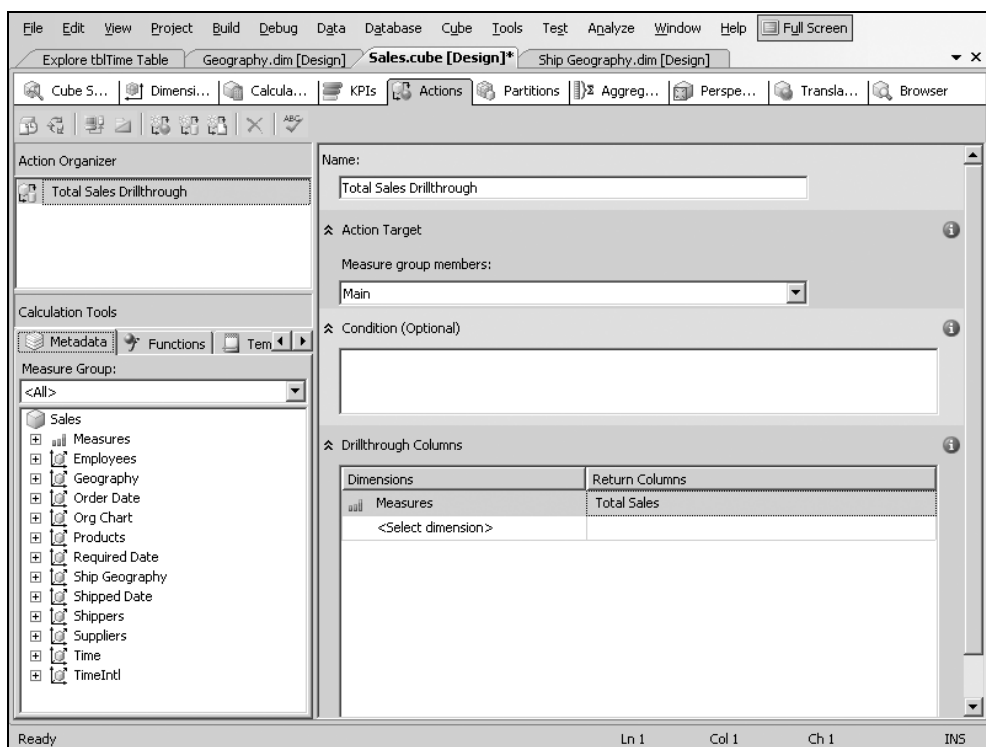
Начните с создания действия детализации (при помощи выбора **Cube | New Drillthrough Action** или нажатия кнопки **New Drillthrough Action** в панели инструментов вкладки **Actions**). В поле **Name** введите `Total Sales Drillthrough` (скобки здесь не нужны), выберите **Main** в выпадающем списке **Measure Group Members**, оставьте поле **Condition** пустым, а в новой строке сетки **Drillthrough Columns** выберите **Measures** в столбце **Dimensions** и `Total Sales` в столбце **Return Columns**. (Последнее вы можете сделать при помощи щелчка по выпадающей стрелке, установки флажка **Total Sales** во всплывшем окне и последующего нажатия кнопки **OK**.) Форма должна выглядеть так, как показано на рис. 16.30.

Этим мы создаем действие, которое позволяет вам щелкнуть правой кнопкой мыши по любой ячейке в браузере куба и увидеть во всплывающем окне данные `Total Sales` по каждой строке таблицы фактов.

Теперь добавьте действие при помощи кнопки **New Action** панели инструментов вкладки **Actions**. Введите `Live Search` в поле **Name**, выберите **Level**

Members в выпадающем списке **Target Type** (если это еще не выбрано), а затем выберите Geography.Country-State-Province-City-Postal Code.City в выпадающем списке **Target Object**. (Нажмите выпадающую стрелку, углубитесь и выберите соответствующий узел во всплывающем окне браузера измерений, а затем нажмите **OK**). Оставьте условное выражение пустым, в разделе **Action Content** выберите **URL** в выпадающем списке **Type** (если он еще не выбран) и введите в поле **Action Expression** следующее:

```
"http://search.live.com/results.aspx?q=" +  
[Geography].[City].CURRENTMEMBER.NAME
```



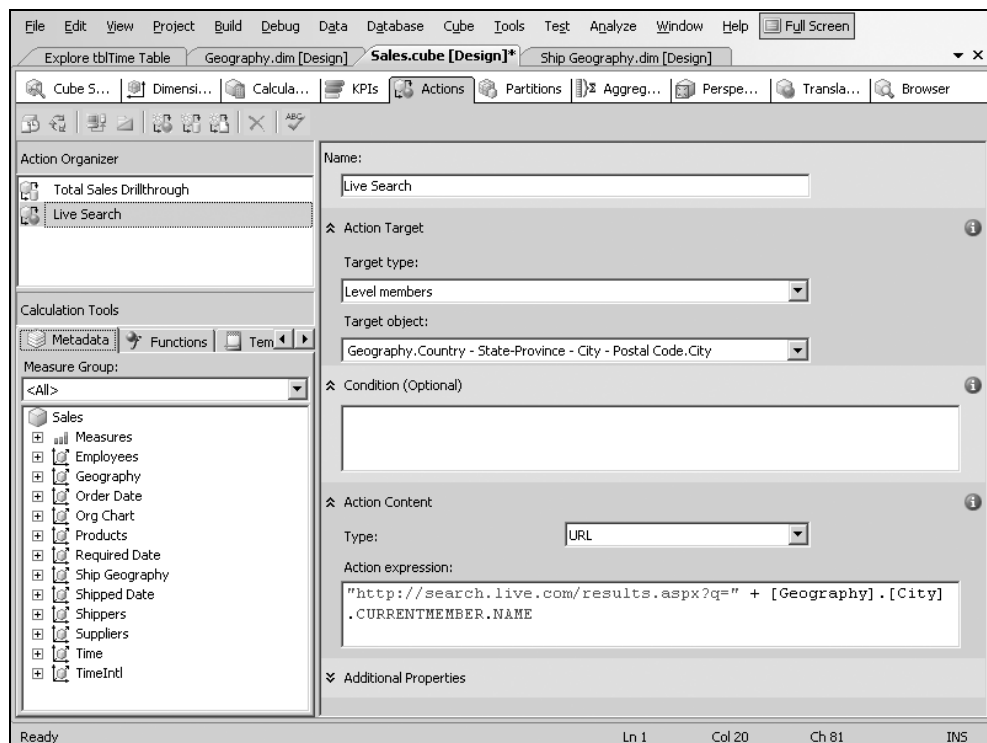
**Рис. 16.30.** Вкладка **Actions** визуального конструктора кубов (с настроенным действием детализации)

Теперь форма должна выглядеть так, как показано на рис. 16.31.

Это действие генерирует URL поиска по выбранному названию города. Щелкнув правой кнопкой мыши по любому городу в заголовке строки или



столбца (вкладки **Browser** визуального конструктора кубов), вы можете выбрать это действие в контекстном меню и перейти на соответствующий URL внутри программы Windows Internet Explorer.

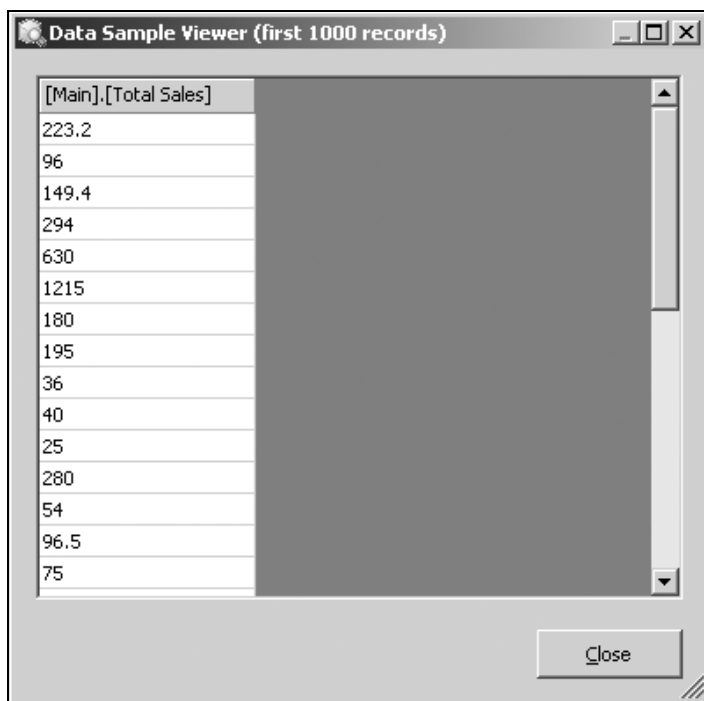


**Рис. 16.31.** Вкладка **Actions** визуального конструктора кубов (с полностью настроенным действием URL)

## Тестирование действий

Вы можете протестировать свои действия при помощи сохранения и развертывания ваших изменений и перехода на вкладку **Browser** визуального конструктора кубов. Перетащите меру Total Sales в зону подробных сведений, а затем перетащите иерархию Country-State-Province-City-Postal Code измерения Geography в зону полей строк. Щелкните правой кнопкой мыши по любой ячейке в зоне данных, а затем выберите в контекстном меню пункт

**Total Sales Drillthrough** (для просмотра результатов запроса для этой ячейки, как показано на рис. 16.32).



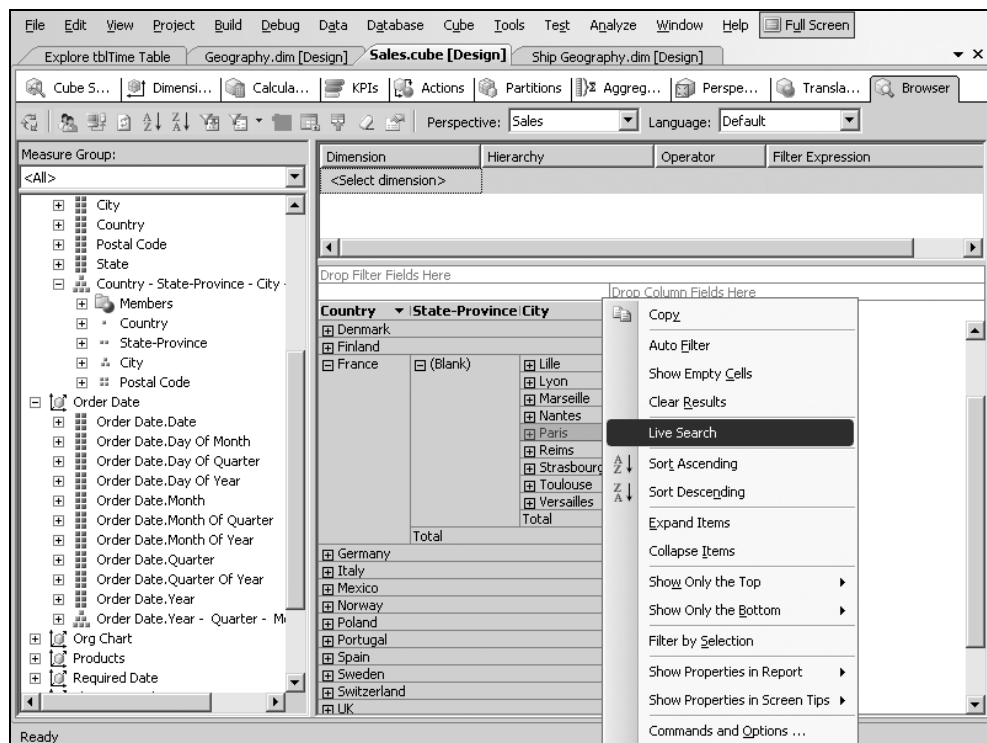
**Рис. 16.32.** Окно **Data Sample Viewer**, открытое по действию детализации для ячейки в браузере кубов

Углубитесь до любого города и щелкните по нему правой кнопкой мыши в области заголовка строки, а затем выберите в контекстном меню пункт **Live Search** (рис. 16.33) для просмотра результатов поиска в Live Search названия этого города.

### **СОВЕТ**

Изменить ваше действие для того, чтобы в нем использовался другой поисковик, очень просто. Например, чтобы использовать Google вместо Live Search, измените строковую часть выражения **Action Expression** с <http://search.live.com/results.aspx?q=> на <http://www.google.com/search?q=> (возможно, вы захотите также изменить и название этого действия).

Поэкспериментируйте с другими типами действий и оцените возможность энергичного использования действий в ваших кубах. OLAP весь посвящен анализу детализаций, а какая функциональность детализации может быть лучше, чем предоставление вашим пользователям подробной информации о данных куба из других приложений, баз данных и даже Интернета?



**Рис. 16.33.** Действия URL появляются в виде пунктов контекстного меню своих целевых объектов (в данном случае это элемент уровня) в браузере кубов

## Секции, настройки хранения и упреждающее кэширование

Кубы — это здорово, и иметь все данные бизнес-аналитики в одном месте — это удобно, но иногда некоторые части вашего куба будут иметь особые тре-

бования к ним. Секции Analysis Services позволяют вам физически разделить ваш куб на срезы и использовать особые настройки хранения и агрегирования, которые соответствуют каждому отдельному подмножеству данных вашего куба и предъявляемым к нему уникальным требованиям.

Например, если пользователям редко нужны данные старше определенной даты, то вы можете хранить эти данные на более медленном и более дешевом диске, реже их обрабатывать и вычислять по ним гораздо меньше агрегатов, чем для свежих данных. Аналогичным же образом, поскольку обычно нужно повторно обрабатывать только самые свежие данные, размещение их в отдельной небольшой секции уменьшит время повторной обработки и таким образом снизит латентность данных вашего куба. Секции Analysis Services позволяют вам определять их источник путем указания либо таблицы фактов (на которой должна основываться секция), либо запроса SQL, который выбирает только подмножество строк из таблицы фактов. Затем вы можете настроить различные аспекты обработки и хранения секции, а также схему ее агрегирования.

Каждый куб состоит, по крайней мере, из одной секции (или если быть точнее, по крайней мере из одной секции на каждую группу мер), так что вы должны знать конфигурацию секций даже в том случае, если вы не хотите секционировать свой куб. В остальной части этого раздела при использовании нашего куба Northwind Sales мы остановимся на конфигурации с одной секцией.

## Редактирование и создание секций

Если вы нажмете вкладку **Partitions** в визуальном конструкторе кубов, то увидите, что для куба Sales уже определена секция с названием `tbl Fact`. Существующий раздел создан на базе нашей таблицы фактов `tblFact` (без всякой фильтрации для выборки строк). Места хранения и обработки установлены в значения по умолчанию Analysis Services; вы можете изменить их в диалоговом окне **Partition Type Selection** (рис. 16.34): выделите строку секции в сетке визуального конструктора, выберите свойство `StorageLocation` в окне **Properties** и нажмите кнопку с многоточием в столбце его значения.

Вы можете модифицировать содержимое секции при помощи щелчка по строке секции в столбце `Source` сетки визуального конструктора и после-

дующего нажатия кнопки с многоточием либо при помощи выбора свойства **Source** секции в окне **Properties** и нажатия кнопки с многоточием в его столбце **Value** (рис. 16.35).

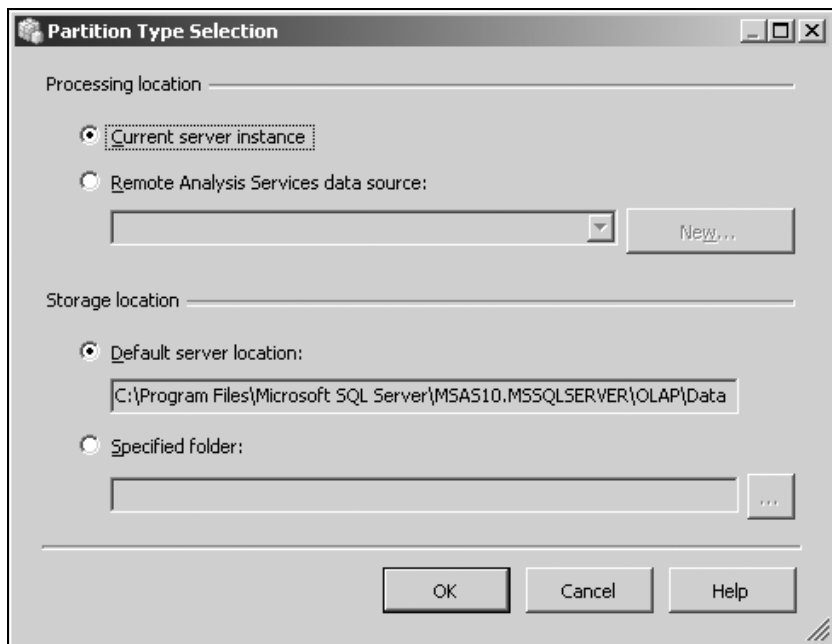


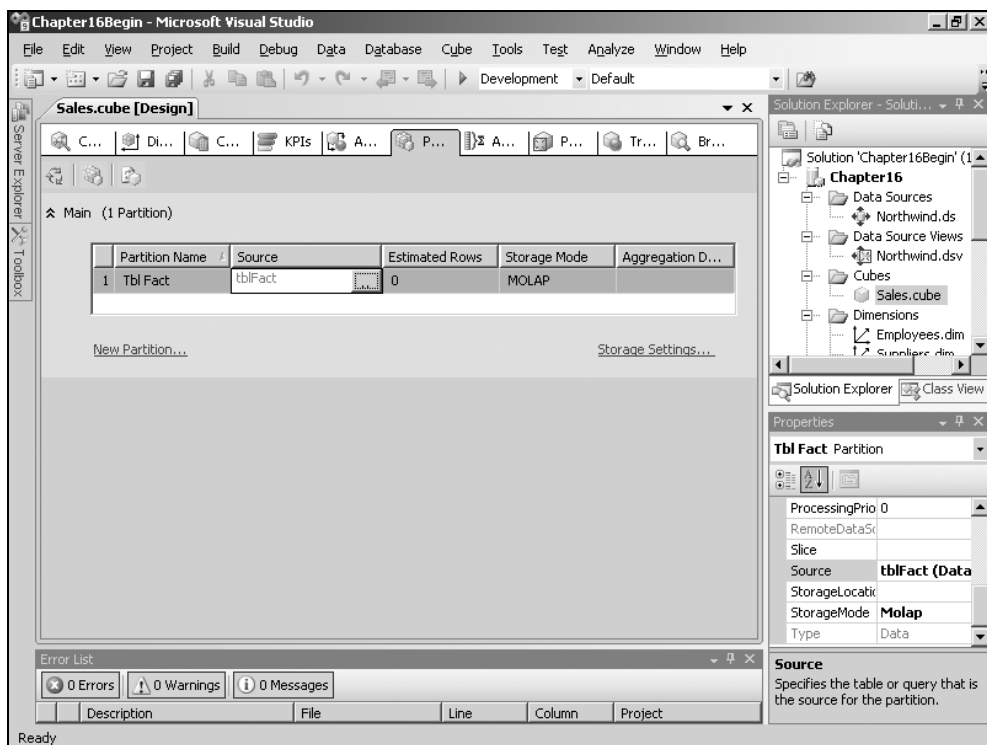
Рис. 16.34. Диалоговое окно выбора  
**Partition Type Selection**

Любой из этих способов открывает диалоговое окно **Partition Source** в показанном на рис. 16.36 представлении привязки таблиц.

Выберите опцию **Query Binding** в выпадающем списке **Binding type** (для показа представления привязки запросов). Представление привязки таблиц позволяет вам определить вашу секцию на базе содержимого всей таблицы фактов, а представление привязки запросов позволяет вам указать SQL-запрос, который будет выбирать только некоторое подмножество строк. Когда вы закончите изучение этих опций, нажмите кнопку **Cancel** диалогового окна (поскольку мы не хотим модифицировать источник нашей секции).

## ПРИМЕЧАНИЕ

Возможно, вас заинтересует мастер Partition Wizard, который обеспечивает альтернативный интерфейс настройки многих упомянутых здесь свойств, но делает он это только для новых секций. Для этого вы можете нажать кнопку **New Partition** панели инструментов. Вам придется удалить существующую секцию, что освободит таблицу фактов для использования в новой секции. Если вы это сделаете, не забудьте после удаления существующей секции закрыть визуальный конструктор кубов и отказаться от сохранения ваших изменений. Это гарантирует, что существующая секция не будет удалена из куба.



**Рис. 16.35.** Вкладка **Partitions** визуального конструктора кубов (выделены кнопки с многоточием диалогового окна **Partition Source**)

В дополнение к этим свойствам секции вы можете также модифицировать и настройки ее хранения, включая и те, которые влияют на упреждающее кэширование (мы обсудим это в следующем разделе).

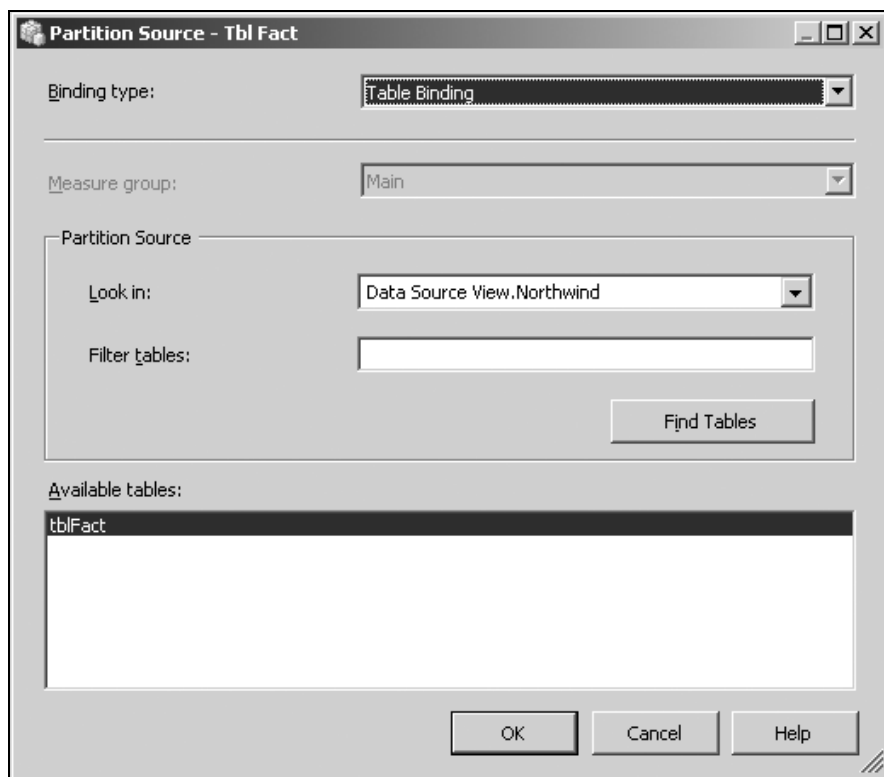


Рис. 16.36. Диалоговое окно **Partition Source** в представлении привязки таблиц

## Опции хранения секции

Во времена SQL Server 7.0 OLAP Services и SQL Server 2000 Analysis Services существовало три типа хранения: Multidimensional OLAP (MOLAP), Relational OLAP (ROLAP) и Hybrid OLAP (HOLAP). При MOLAP (установка по умолчанию) все данные фактов и агрегатов хранятся в многомерном хранилище Analysis Services, которое обеспечивает самое быстрое время отклика на запрос. ROLAP (при котором факты остаются в таблице фактов реляционной базы данных, а агрегаты хранятся в специальных собственных реляционных таблицах ROLAP) обеспечивает более быструю обработку кубов (по очевидным причинам), но более длительное время отклика на запрос. HOLAP (компромиссный вариант) хранит данные в реляционной базе данных, а агрегаты — в многомерном хранилище.

Хранение секций настраивается в диалоговом окне **Partition Storage Settings** (рис. 16.37).

Вы можете открыть диалоговое окно **Partition Storage Settings** при выделенной конкретной секции (при помощи нажатия кнопки **Storage Settings** панели инструментов). Вы можете также выбрать свойство `ProactiveCaching` данной секции в окне **Properties**, а затем нажать кнопку с многоточием в его столбце `Value`.

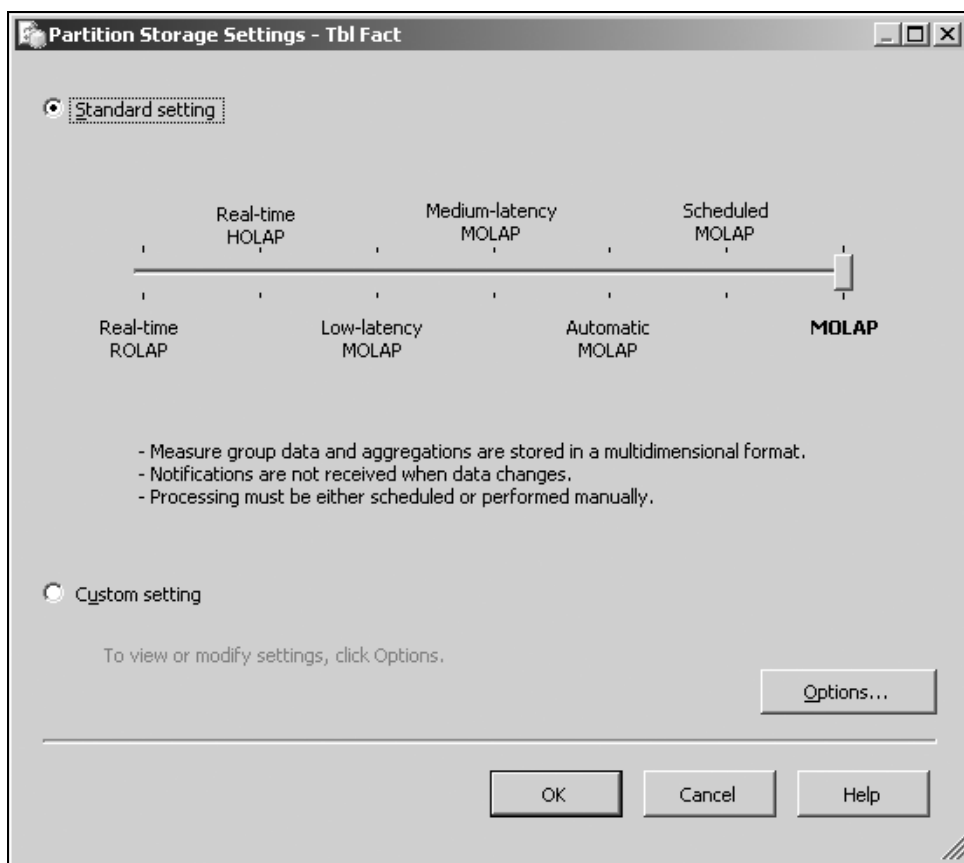


Рис. 16.37. Диалоговое окно **Partition Storage Settings**

То, что опции хранения секции настраиваются в простом диалоговом окне, может несколько ввести в заблуждение. На самом деле диалоговое окно **Partition Storage Settings** предоставляет такое огромное количество опций,



что для их достойной оценки может потребоваться много времени. Это диалоговое окно является также по существу и панелью управления очень важной функцией Analysis Services — упреждающим кэшированием.

## Упреждающее кэширование

MOLAP, ROLAP и HOLAP остались и в версии Analysis Services 2005, и в версии 2008, но их ограничения теперь компенсируются функцией упреждающего кэширования. Например, упреждающее кэширование позволяет вам использовать ROLAP как базовую среду хранения (что обеспечивает более быструю обработку и таким образом позволяет делать OLAP в реальном или почти реальном времени), но дает указание службам Analysis Services кэшировать ваш куб (то есть копировать его) в среду MOLAP (для повышения производительности запросов).

Частоту сброса кэша выбираете вы. В зависимости от настроек кэширования, копия MOLAP может немедленно признаваться недействительной после изменений в базовом хранилище данных. В этом случае запросы выполняются по основному содержимому ROLAP; версия MOLAP проходит повторную обработку в фоновом режиме, а затем вводится в работу. Вы можете также сделать такие настройки, что устаревшие данные MOLAP будут использоваться в течение всего времени обработки обновленных данных MOLAP. Это повышает производительность (поскольку продолжает использоваться хранилище MOLAP), но снижает свежесть данных.

Как вы видите в диалоговом окне **Partition Storage Settings**, упреждающее кэширование имеет семь стандартных настроек, начиная от Real-Time ROLAP и до некэшируемого MOLAP. Все варианты (кроме последнего) настраивают определенную степень кэширования и автоматической обработки вашей секции. По мере движения ползунок между настройками вы видите появление нового текста (с описанием данной настройки) в диалоговом окне под ползунком.

### ПРИМЕЧАНИЕ

Вы можете изменить настройки хранения секции при помощи редактирования отдельных дочерних свойств *ProactiveCaching*. Однако эти свойства доступны для редактирования только после выбора настройки в диалоговом окне **Partition Storage Settings**, которое в любом случае предоставляет гораздо более дружелюбный пользовательский интерфейс.

Для более подробного представления этих настроек нажмите кнопку **Options**, откроется диалоговое окно **Storage Options** (рис. 16.38).

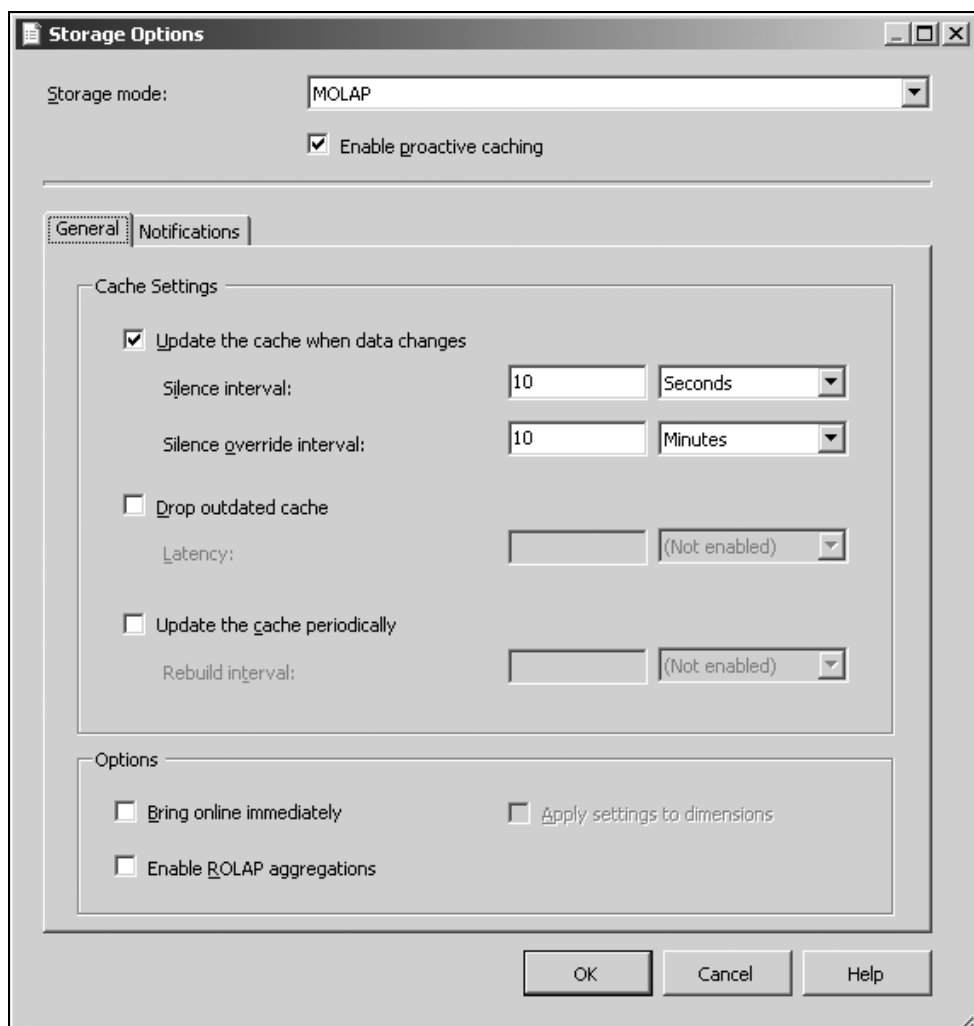


Рис. 16.38. Диалоговое окно **Storage Options**

На вкладке **General** диалогового окна **Storage Options** вы можете просмотреть интервалы запаздывания и обновления кэша. Вкладка **Notifications** позволяет вам указать определенные таблицы или SQL-запросы, за которыми надо наблюдать на предмет наличия изменений данных. В этом диалоговом

окне у вас есть множество детальных возможностей управления. Потратьте некоторое время на ознакомление с ними. Эти опции позволяют вам достичь баланса между производительностью, свежестью данных и доступностью данных в реальном времени. Если вы найдете правильное сочетание параметров, то ваши затраты времени окупятся.

Если вы модифицируете настройки диалогового окна **Storage Options**, то вы автоматически уходите от использования одной из семи стандартных настроек. Проверьте это сами: измените один из параметров, а затем нажмите кнопку **ОК**. Когда вы вернетесь в диалоговое окно **Partition Storage Settings**, то вы увидите, что вместо **Standard Setting** выбрано **Custom Setting**, и ползунок неактивен.

### ПРИМЕЧАНИЕ

Вы можете также настроить хранение и на уровне куба, измерения базы данных или группы мер. Вы можете отредактировать настройки на уровне куба или группы мер в визуальном конструкторе куба: просто выберите куб или группу мер (например, корневые узлы куба и группы мер древовидного представления **Measures** вкладки **Cube Structure**), а затем нажмите кнопку с многоточием в столбце **Value** свойства **ProactiveCaching** в окне **Properties**. Вы можете аналогичным образом настроить опции хранения измерения (через окно **Properties**), но делать это надо в визуальном конструкторе измерений.

Вы можете оставить эти настройки в значениях по умолчанию и успешно работать с кубами **Analysis Services**. Однако мы попытались дать вам краткий обзор этих настроек и способов их работы.

## Дополнительные функциональные возможности и советы

Наше описание секций и упреждающего кэширования было бы неполным без упоминания некоторых функциональных возможностей, которые достойны вашего внимания.

Для начала упомянем доступные для записи группы мер и измерения. При выделенной вкладке **Partitions** в визуальном конструкторе кубов выберите **Cube | Writeback Settings** (или выберите пункт **Writeback Settings** контекстного меню, для чего нужно щелкнуть правой кнопкой мыши по названию секции), чтобы открыть диалоговое окно **Enable Writeback**, где вы можете указать источник данных и таблицу для хранения обратных записей.

В версии Analysis Services 2005 (и более ранних) данные обратной записи должны были храниться в реляционных таблицах обратной записи, создавая таким образом секцию ROLAP для данных. В Analysis Services 2008 эта опция по-прежнему имеется, но есть еще и дополнительная опция (которая также настраивается через диалоговое окно **Enable Writeback**) хранения данных обратной записи в секции MOLAP, что избавляет от необходимости делать взаимно исключающий выбор между функциональностью обратной записи и более высокой производительностью MOLAP по сравнению с ROLAP.

### ПРИМЕЧАНИЕ

Обратную запись могут иметь только те группы мер, которые не содержат специальных агрегатов (то есть агрегатов помимо SUM). Наша группа мер (поскольку она содержит основанную не на суммировании меру Avg Total Sales) не может иметь обратной записи. Однако секция Sales\_Quotas partition группы мер Sales Targets куба Adventure Works базы данных AdventureWorksDW2008 может иметь обратную запись (на тот случай, если вы захотите ее протестировать).

Вы можете также включить обратную запись для измерений, позволив клиентским приложениям обновлять их содержимое (для этого необходимо установить свойство WriteEnabled в значение True в окне **Properties** визуального конструктора измерений).

И еще один совет: в Management Studio вы можете создать секции и изменить настройки хранения и упреждающего кэширования на уровнях измерения, секции или куба. Окна свойств Management Studio позволяют вам модифицировать эти настройки, так что вам не придется использовать Visual Studio для изменения их после развертывания вашего куба. Вы можете также модифицировать настройки обратной записи группы мер при помощи пункта контекстного меню **Writeback Options** (или через пункт **Enable/Disable Writeback** контекстного меню ее дочернего узла **Writeback**, в котором перечислены секции обратной записи).

## Агрегаты

Схема агрегирования указывает, какие предварительно вычисленные агрегаты хранятся в вашем кубе. В Analysis Services 2005 существовало два типа схем агрегирования: алгоритмическая и на основе использования (обе они

создавались на вкладке **Partitions** визуального конструктора кубов). В Analysis Services 2008 конфигурация агрегирования изменилась. Во-первых, теперь схема агрегирования создается на новой вкладке **Aggregations**. Во-вторых, в добавление к предыдущим двум типам агрегирования теперь можно создавать агрегаты вручную, а также модифицировать схемы на основе алгоритмов и использования. Мы коротко опишем создание каждого из этих трех типов агрегирования. Для начала нажмите вкладку **Aggregations** визуального конструктора кубов нашего куба *Sales*.

## Проектирование алгоритмического агрегирования

Схема алгоритмического агрегирования создается при помощи мастера Aggregation Design Wizard, который вы можете открыть при помощи нажатия кнопки **Design Aggregations** (крайняя слева) панели инструментов (или при помощи выбора **Cube | Design Aggregations**; либо при помощи пункта **Design Aggregations** контекстного меню). Этот мастер генерирует агрегаты на основе алгоритмического определения такого типа агрегирования, который будет наиболее благоприятным для производительности запросов.

Давайте создадим схему агрегирования для нашей единственной секции. Начнем с открывания мастера Aggregation Design Wizard (любым из описанных ранее способов). На странице **Welcome** нажмите кнопку **Next**. Появится страница **Review Aggregation Usage**. На этой странице перечислены все атрибуты куба, и для каждого из них вы можете указать, будет ли этот атрибут включаться во все агрегаты (вариант **Full**), не будет включаться в агрегаты (вариант **None**), рассматриваться для включения (вариант **Unrestricted**) или должен обрабатываться по умолчанию (как определено алгоритмом проектирования агрегатов) — вариант **Default**. Оставьте для всех атрибутов вариант **Default** и нажмите кнопку **Next** для перехода на страницу **Specify Object Counts**.

### СОВЕТ

Страница **Aggregation Usage** является альтернативным пользовательским интерфейсом для настройки свойства *AggregationUsage*, доступным через окно **Properties** (при выбранном в визуальном конструкторе куба определенном атрибуте). Значение свойства, присвоенное в окне **Properties**, будет отображаться в мастере Aggregation Design Wizard (и наоборот).

Для проектирования агрегатов мастер должен знать (для куба и секции), сколько строк содержит каждая группа мер и сколько элементов содержит каждый атрибут. Это можно определить, если разрешить мастеру сосчитать строки и элементы в кубе и секции (для этого необходимо нажать кнопку **Count**), либо вы можете ввести эти числа вручную (в сетке). Для случая измерения вам понадобится углубиться в него и ввести числа для его атрибутов. Ручной метод может быть более предпочтительным для очень больших кубов или секций, где мощность атрибута заранее известна. Однако наш куб невелик, поэтому просто нажмите **Count**, а по завершении подсчета нажмите кнопку **Next**.

Хотя алгоритм проектирования агрегатов Analysis Services и определяет, какие агрегаты будут полезны, он не указывает, сколько их будет создано. Вы можете управлять объемом агрегирования на странице мастера **Set Aggregation Options**. Эта страница позволяет вам определить количество создаваемых агрегатов как функцию от дискового пространства или как процент прироста производительности. Вы можете также вручную начать создание агрегатов и остановить этот процесс — в этом случае агрегаты ограничены временем обработки, а не дисковым пространством или приростом производительности.

Для наших небольшого куба и небольшой секции примите выбор по умолчанию **Estimated Storage Reaches** и значение 100 Мбайт, а затем нажмите **Start**. Дождитесь завершения генерирования агрегатов и нажмите кнопку **Next**. На странице **Completing The Wizard** введите Main в текстовом поле **Name**, выберите вариант **Deploy And Process Now**, а затем нажмите **Finish**. При появлении диалогового окна **Process Partition** выберите **Run** для выполнения обработки куба, а затем нажимайте кнопку **Close** во всех диалоговых окнах, которые появятся при завершении обработки. Теперь перейдите на вкладку **Partitions** и обратите внимание, что в столбце Aggregation Design сетки появилось значение Main; вернитесь на вкладку **Aggregations** и обратите внимание, что в столбце Partitions появилось значение Tbl Fact (название нашей секции).

## Проектирование агрегатов на основе использования

Алгоритмическое проектирование агрегатов может дать хорошее повышение производительности, но никакой алгоритм не может предсказать, какие агрегаты будут наиболее полезны (с учетом привычек ваших пользователей). Хо-

рошая новость состоит в том, что проектирование агрегатов на основе использования учитывает именно это: генерируются агрегаты, которые наилучшим образом оптимизируют запросы, записанные в журналах запросов Analysis Services. Схема агрегирования на основе использования создается при помощи мастера Usage-Based Optimization Wizard (UBO Wizard), который можно открыть путем нажатия на кнопку **Usage Based Optimization** панели инструментов (или при помощи выбора **Cube | Usage Based Optimization**; либо при помощи выбора пункта **Usage Based Optimization** контекстного меню).

Мастер UBO Wizard сгенерирует схему агрегирования на основе начальной и конечной дат журнала запросов, списка пользователей, а также процента наиболее частых запросов. Для использования мастера UBO Wizard на вашем сервере должно быть включено журналирование запросов, его можно настроить на странице свойств сервера в SQL Server Management Studio.

## Проектирование агрегатов вручную

Новой в Analysis Services 2008 является способность создавать агрегаты вручную или редактировать их. Можно создавать целые схемы агрегирования; можно добавлять, удалять и модифицировать отдельные агрегаты внутри существующих схем агрегирования. После создания новых схем агрегирования их можно присвоить конкретным секциям.

### **ВНИМАНИЕ!**

Для подавляющего большинства читателей проектирование схем агрегирования вручную — это та возможность, которую им не нужно и даже не следует использовать. Стратегии алгоритмического создания агрегатов и создания их на основе использования достаточно совершенны, и их безопаснее использовать, чем проектирование вручную. Однако представление о проектировании вручную схем агрегирования полезно для лучшего понимания того, как работают стратегии (алгоритмического и основанного на использовании) создания агрегатов, а также дает вам знания, необходимые для просмотра схем агрегирования, созданных этими стратегиями.

Проектирование вручную агрегирования Manual Aggregation Design доступно через активацию представления Advanced вкладки **Aggregations**. Для этого необходимо нажать кнопку **Advanced View** панели инструментов (или выбрать **Cube | Show Aggregations In | Advanced View**).

### СОВЕТ

Если вы закроете визуальный конструктор кубов, а затем опять откроете его и нажмете вкладку **Aggregations**, то вы опять окажетесь в режиме **Standard**. Для возвращения в режим **Standard** без закрывания визуального конструктора нажмите кнопку панели инструментов **Standard View** (четвертая слева) или выберите **Cube | Show Aggregations In | Standard View**.

Для просмотра содержимого нашей существующей схемы агрегирования зайдите в представление *Advanced*, а затем выберите схему агрегирования *Main* в выпадающем списке **Aggregation Design**. Затем углубитесь в каждое измерение. (Вы сэкономите на пролистывании, если углубитесь в последнее измерение, а затем начнете двигаться вверх.)

Когда вы закончите, то вы сможете увидеть подробности каждого агрегата в нашей схеме агрегирования (созданной мастером *Aggregation Design Wizard*). Выпадающие списки **Sorting** и **Range** настраивают сортировку агрегатов и отображение группы агрегатов. Агрегаты отображаются в столбцах с метками (с заголовками *A0*, *A1*, *A2* и т. д.). Включенные в агрегат атрибуты представлены соответствующими флажками в столбце агрегата (рис. 16.39).

### СОВЕТ

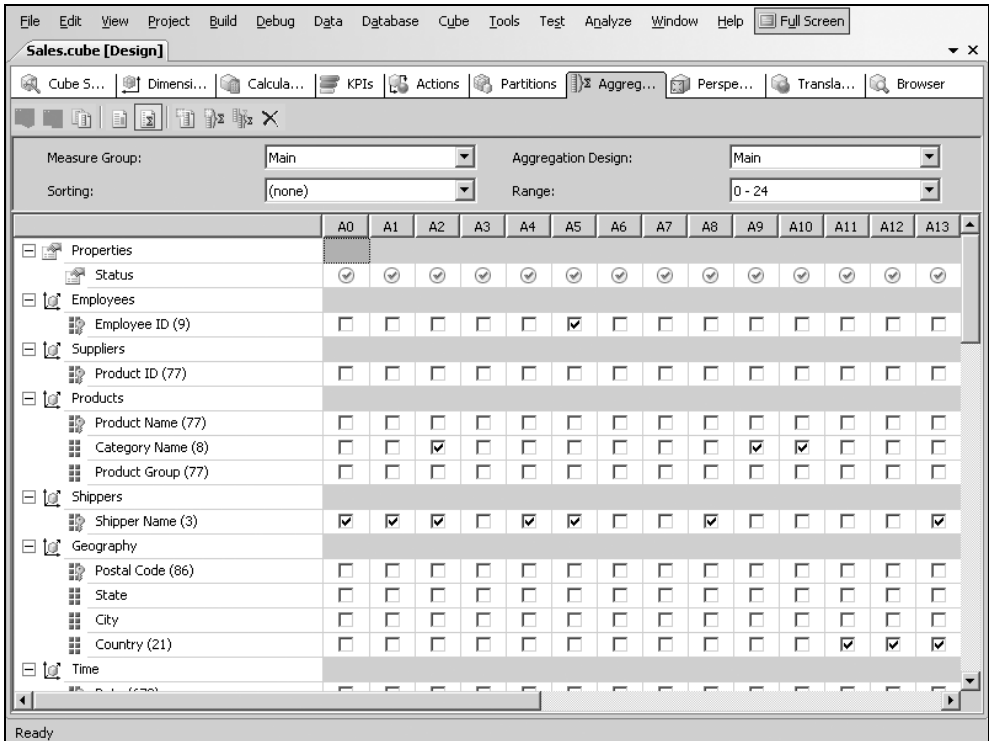
Заголовки столбцов (*A0*, *A1*, *A2* и т. д.) не являются идентификаторами агрегатов, это просто метки столбцов. Название агрегата и его идентификатор *ID* можно увидеть в окне **Properties** (щелкнув где-нибудь в столбце агрегата). Свойства *Name* и необязательное описание *Description* можно редактировать.

Включение или исключение любого атрибута конкретного агрегата можно менять: просто установите или снимите соответствующий флажок. Вы можете также добавить новый (пустой) агрегат или удалить его (при помощи кнопки **New Aggregation** или **Delete** (третья справа и крайняя справа соответственно) панели инструментов либо при помощи пунктов **New Aggregation** или **Delete** контекстного меню). Есть пункт главного меню **Cube | New Aggregation**, но нет пункта главного меню для удаления агрегата.

Агрегат можно также скопировать. Это можно сделать следующим образом: сначала выделите агрегат для копирования (щелкнув где-нибудь внутри его столбца, но не по заголовку). (Щелчок по строке *Properties* или *Status* или аналогичным строкам любого измерения позволяет выбрать агрегат без изменения состояния включения его атрибутов). Затем нажмите кнопку **Copy Aggregation** (вторая слева) панели инструментов (или выберите **Cube | Copy**



**Aggregation**; либо выберите пункт **Copy Aggregation** контекстного меню). Это приведет к копированию выделенного агрегата и вставке его в сетку в самое крайнее левое положение столбцов, все остальные агрегаты сдвинутся на один столбец вправо. Здесь вы можете модифицировать новый вставленный агрегат (чтобы сделать его уникальным).



**Рис. 16.39.** Представление Advanced вкладки **Aggregations** визуального конструктора кубов

## Управление проектированием агрегирования

Схему агрегирования можно создать при помощи нажатия кнопки **New Aggregation Design** панели инструментов (или посредством выбора **Cube | New Aggregation Design**). Оба варианта приводят к открытию диалогового окна **New Aggregation Design**, которое позволяет вам дать название новой схеме агрегирования и указать, будет ли она создана пустой или ско-

пирована с существующей схемы. После нажатия кнопки **ОК** новую схему агрегирования можно будет редактировать (как уже было описано ранее).

Вы можете присвоить схему агрегирования одной или более секциям (путем нажатия кнопки **Assign Aggregation Design** панели инструментов; или посредством выбора **Cube | Assign Aggregation Design**; либо при помощи щелчка в ячейке сетки представления **Standard** на пересечении строки схемы агрегирования и столбца **Partitions** и последующего нажатия кнопки с многоточием). Каждое из этих действий приводит к показу диалогового окна **Assign Aggregation Design** (рис. 16.40), которое позволяет нам выбрать схему агрегирования, а затем все секции, которым вы хотите ее присвоить. Вы можете также использовать этот способ и для удаления присваивания схемы агрегирования секциям, для этого надо указать **(None)** в выпадающем списке **Aggregation designs** и выбрать соответствующие секции.

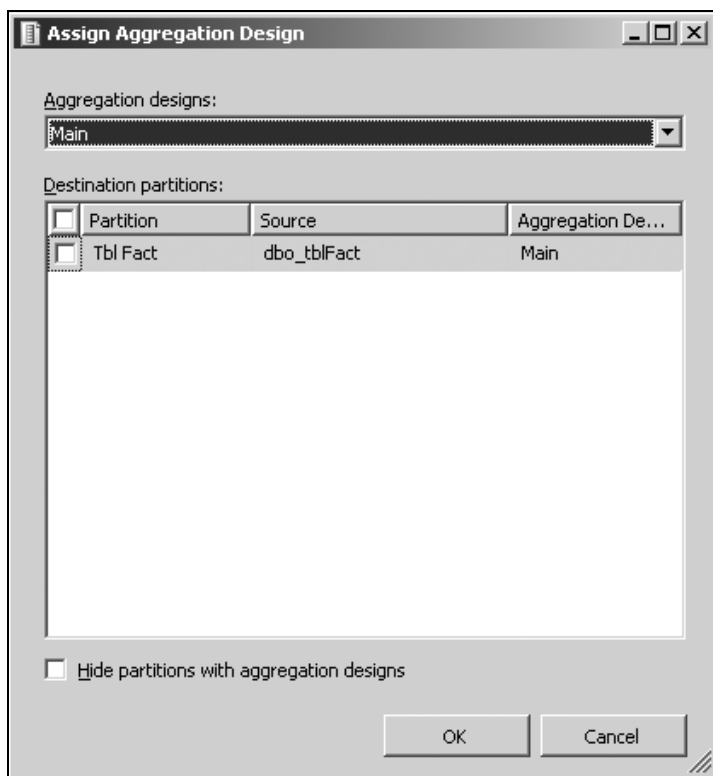


Рис. 16.40. Диалоговое окно **Assign Aggregation Design**

При помощи кнопки **Delete** (крайняя справа) панели инструментов или пункта контекстного меню можно удалить всю схему агрегирования. Если вы хотите удалить схему и находитесь в представлении **Advanced** вкладки **Aggregations**, то перед удалением убедитесь в том, что выделена схема агрегирования, а не отдельный агрегат. Это можно сделать в выпадающем списке **Aggregation design**.

## Схема агрегирования и Management Studio

В Management Studio вы можете создавать алгоритмические и основанные на использовании схемы агрегирования, а также присваивать схемы агрегирования. Эти возможности доступны через **Design Aggregations | Usage Based Optimization**, а также через пункт **Assign Aggregation Design** контекстного меню узла конкретной секции (или ее родительского узла **Partitions**) в окне **Object Explorer**. Опция **Assign Aggregation Design** доступна также через контекстное меню узла схемы агрегирования или его родительского узла **Aggregation Designs**.

## Перспективы

Как мы уже упоминали ранее, отличительной чертой служб Analysis Services после версии 2000 г. является их гибкость. Например, способности иметь действия детализации, ролевые измерения и множество таблиц фактов позволяют вам использовать один куб для множества многомерных и реляционных запросов. В Analysis Services 2000 каждая таблица фактов требовала своего собственного куба, ролевых измерений не было, функциональность детализации была весьма ограничена, а отсутствие упреждающего кэширования означало, что данные куба не могли быть настолько свежими, как это порой было необходимо.

Это понятие о едином кубе, который является единственным источником всех ваших реляционных данных, дает начало концепции Analysis Services 2005/2008 с названием "унифицированная модель измерений" (unified dimensional model, UDM). Парадигма UDM ведет к смягчению различий между OLAP и реляционными данными, и если хотите, продвигает такую точку зрения, что куб — это больше, чем просто куб. Философия UDM рассматривает правильно построенный куб Analysis Services как комплексное хранилище всех агрегированных данных и данных уровня фактов.

Все это очень хорошо, но парадигма UDM имеет свою собственную проблему: если вы затолкаете слишком много всего в один куб, то многие ваши пользователи будут просто подавлены содержимым вашего куба, и он станет для них менее полезен. По этой причине в версии Analysis Services 2005 ввели, а в версии Analysis Services 2008 продолжает предоставляться возможность создания перспектив в ваших кубах.

Перспективы — это просто фильтрованные представления содержимого куба. Вы определяете перспективу путем выбора набора специфических групп мер, измерений или отдельных атрибутов и иерархий, KPI, действий и вычислений — и присваиваете ей название. Клиентские приложения работают с перспективой как с отдельным физическим кубом. Например, вы можете скомбинировать продажи, складские запасы и кадровую информацию в единый куб, а затем опубликовать каждую категорию данных как отдельную перспективу. Это дает вам простоту и мощь парадигмы UDM без отказа от возможности развертывать компоненты данных ваших измерений в виде отдельных модулей, которые выглядят как самостоятельные кубы.

Создать перспективу просто. Щелкните по вкладке **Perspectives** визуального конструктора кубов, а затем нажмите кнопку **New Perspective** панели инструментов (либо выберите пункт **New Perspective** контекстного меню). Любое из этих действий создает новый столбец в сетке вкладки **Perspective**. Там вы вводите название вашей перспективы, а затем выбираете все объекты куба, которые вы хотите в нее включить. Сейчас создайте новую перспективу с названием **Product Info** и присвойте ей всю группу мер **Main**, измерения **Products** и **Suppliers**, KPI с названием **Average Total Sales KPI**, действие **Total Sales Drillthrough** и две вычисляемые меры (**Adjusted Sales** и **Adjusted Sales Growth**).

Сохраните и разверните ваши изменения. В браузере кубов сначала повторно подключитесь, а затем обратите внимание, что выпадающий список **Perspective** содержит два элемента: **Sales** (физический куб) и **Product Info** (только что созданная нами перспектива). Выберите в выпадающем списке **Product Info** и обратите внимание, что в древовидном представлении метаданных перечислены только меры куба (включая две вычисляемых) и два измерения, которые мы выбрали при определении перспективы (рис. 16.41).

Обратите внимание, что обе вычисляемые меры перечислены в каталоге отображения **Sum**, поскольку они были настроены в кубе **Sales** именно таким образом.

В Management Studio окно **Object Explorer** перечисляет только физические кубы базы данных Analysis Services.

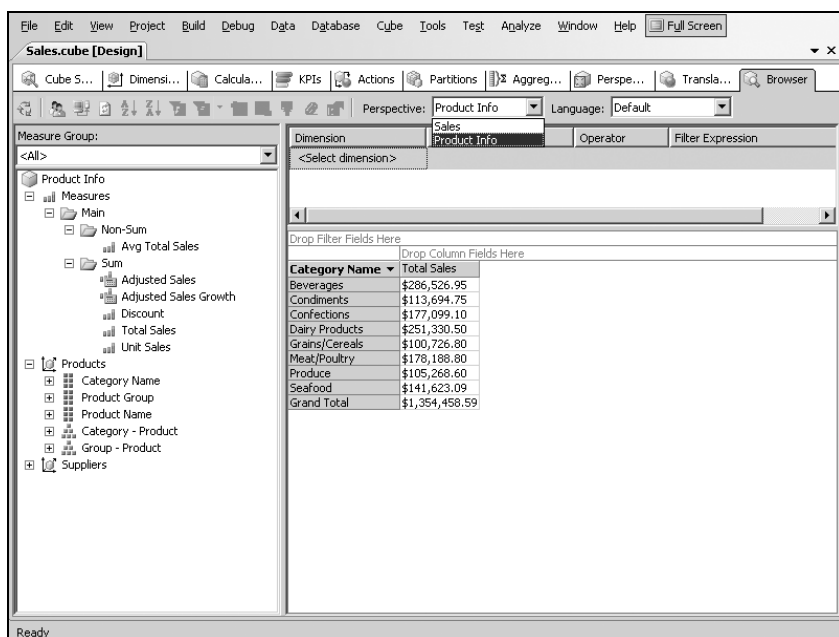


Рис. 16.41. Браузер кубов с выделенной перспективой Product Info

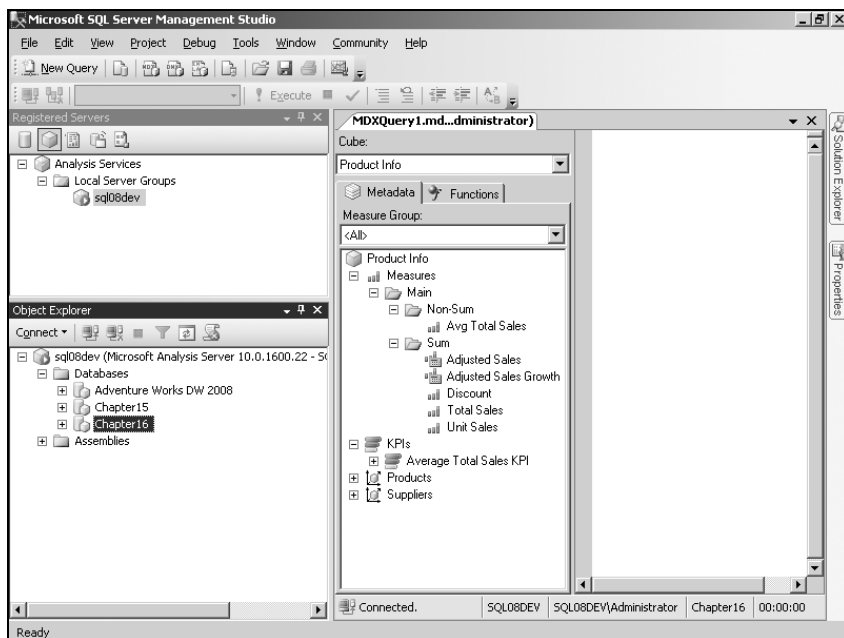


Рис. 16.42. Окно запроса MDX в Management Studio с выделенной перспективой Product Info

Однако в окне запросов MDX перспективы перечислены в выпадающем списке **Cubes** (и поскольку перспектива `Product Info` предшествует кубу `Sales` по алфавиту, то именно она будет выделена по умолчанию). На вкладке **Metadata** перечислены только меры (в том числе и вычисляемые), измерения и KPI (если они есть), которые принадлежат выделенной перспективе или кубу (рис. 16.42).

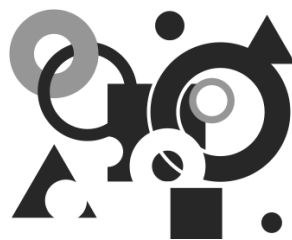
Перспективы предназначены для пользователей ваших кубов, причем настраиваются они на стороне сервера, что облегчает задачу разработчиков приложений. Это понятие обеспечиваемого сервером настраиваемого представления куба можно расширить. Например, языковой локализацией названий объектов вашего куба и его содержимого можно также управлять на стороне сервера. Вы можете сделать это путем создания переводов `Analysis Services`. Обеспечение безопасности в настоящем смысле этого слова становится возможным, посредством создания ролей безопасности и разрешений на уровне объектов куба. Оставшиеся разделы этой главы будут описывать переводы и роли.

## Резюме

Набор функциональных возможностей OLAP в службах `Analysis Services` весьма впечатляющий. Множество типов измерений, мощные вычисления, KPI, перспективы, переводы и упреждающее кэширование — все это значительно облегчает построение кубов и делает их более мощными и соответствующими запросам бизнес-пользователей, чем это было возможно до появления `Analysis Services 2005`. В `Analysis Services 2008` эти функции стали более доступными и управляемыми (при помощи таких новшеств, как `BPA`, вкладка **Attribute Relationships** в визуальном конструкторе измерений, а также вкладка **Aggregations** в визуальном конструкторе кубов).

Теперь вы знаете, как строить сложные кубы. В следующей главе мы покажем вам, как разрабатывать приложения, которые делают запросы к кубам при помощи `Microsoft Office Excel 2007` и `Excel Services`, `ADO MD.NET`, `XML for Analysis (XMLA)` и `Analysis Management Objects (AMO)`. Мы также покажем вам, как создавать логику на стороне сервера (при помощи интеграции `.NET Common Language Runtime (CLR)` и `Analysis Services`).

## ГЛАВА 17



# OLAP: Запросы, инструменты и разработка приложений

Предыдущие две главы ввели вас в основные и расширенные функциональные возможности оперативной аналитической обработки (OLAP) в Microsoft SQL Server Analysis Services 2008. Этот набор возможностей является комбинацией революционных возможностей, введенных версией Analysis Services 2005 и элегантных улучшений версии 2008 г.; остальные функциональные возможности — это в основном перенесенные возможности более ранних версий продукта, но и они теперь стали более доступными. Если вы планируете использовать инструменты OLAP с вашими кубами (в том числе и браузер кубов, с которым мы работали в двух последних главах), то у вас все есть.

Однако реальность такова, что OLAP наиболее полезна тогда, когда ее возможности интегрированы в пользовательские приложения (а не предоставляются через специальные инструменты). Как вам подключиться к кубам из Microsoft Office Excel или встроить запросы OLAP в ваши .NET-приложения? Какие есть инструменты программирования и интерфейсы прикладного программирования (API)? Каковы технологии пользовательских интерфейсов? И как работает язык многомерных выражений (MDX)?

В этой главе мы дадим вам ответы на эти и другие вопросы. Хотите ли вы делать запросы к своему кубу из Excel или написать .NET-код, который запрашивает ваш куб и его метаданные — все это будет описано в данной главе. Мы покажем вам, как подключить PivotTables в Excel к вашим кубам, дадим обзор языка запросов MDX, обсудим использование XML for Analysis (XMLA) как собственного API в Analysis Services. Мы покажем вам, как про-

граммировать непосредственно в XMLA и как использовать ADO MD.NET (ADO.NET Multidimensional) и Analysis Management Objects (AMO) для создания полнофункциональных пользовательских приложений OLAP на управляемом коде. Мы даже покажем вам, как писать .NET-код, который работает на сервере Analysis Services (это похоже на описанное нами в *главе 3* программирование SQL CLR).

Это может показаться слишком большим материалом для одной главы, но вы увидите, что все эти инструменты лежат в одной плоскости и используют друг друга. По мере изучения каждого нового инструмента ваши знания и понимание остальных придут.

### **О ПРИМЕРАХ КОДОВ**

Примеры кодов для этой главы (имеющиеся на Web-сайте данной книги) требуют некоторых пояснений. Они включают в себя: .NET-код, скрипты MDX и XMLA, отдельную страницу HTML и даже файлы Excel. Каталог примеров кодов Chapter17 состоит из четырех подкаталогов с названиями Excel, FrontPage-IE, SSMS и VS. В каталоге Excel имеется два файла (PivotTableChart.xlsx для Excel 2007 и PivotTableChart.xls для Excel 2003), каждый из которых содержит PivotTable и связанную с ней диаграмму (на разных вкладках). Первый файл содержит также вкладки, иллюстрирующие создание системы показателей и использование формул CUBE в ячейках. В каталоге FrontPage-IE имеется файл OWC.htm, в котором содержатся элементы управления PivotTable и ChartSpace для Excel (оба они из компонентов Microsoft Office Web Components). В каталоге SSMS имеется файл решения Chapter17.ssmssln, который содержит три отдельных проекта Analysis Services (MDX Demos, XMLA Demos и CLR Demos). В каталоге VS имеется файл решения Chapter17.sln, который содержит проект Windows Forms (с названием Chapter17), проект Microsoft ASP.NET (с названием Chapter17Web), проект Class Library (с названием Chapter17AdomdServer) и проект Microsoft Visual Studio Tools for Office (с названием Chapter17AddIn). Код надстройки Visual Studio Tools for Office подробно рассматриваться здесь не будет, но мы дадим его обзор далее в этой же главе.

Файл Excel 2003 (в каталоге Excel), файл HTML (в каталоге FrontPage-IE), код проектов Windows Forms и ASP.NET (в каталоге VS) с компонентами Office Web Components — все это взято из предыдущего издания данной книги. В этой главе содержимое упомянутых кодов описываться не будет, но сам код, тем не менее, предоставлен — для тех пользователей, которым нужно делать разработки под среду Excel 2003. Для этих читателей на Web-сайте данной книги выложена соответствующая глава из предыдущего издания. Эта старая версия данной главы содержит подробные описания для Excel 2003 и Office Web Component.

В данной главе мы будем изучать методики разработки (с использованием наших примеров кодов). Вы можете создавать примеры с нуля либо сле-



дить за нашим изложением при помощи предоставленных вам примеров электронной таблицы и кода. Почти все примеры готовы к выполнению (при условии, что на вашем локальном сервере Analysis Services имеется созданная в предыдущей главе база данных Chapter16). Скрипты MDX будут выполняться при условии, что на вашем сервере Analysis Services развернута сборка проекта Chapter17AdomdServer.

По мере разъяснения методик мы будем делать ссылки на те компоненты примеров кодов, которые лучше всего их реализуют. В некоторых случаях примеры кодов изучают такие методики, которые исследуют данную тему даже глубже, чем текст самой главы. Если вы сможете, внимательно изучите весь код; он является ценным источником информации.

## Использование Excel

Excel на сегодняшний день — это клиент номер один для OLAP. Для подразделений информационных технологий вездесущность Microsoft Office (и соответственно простота развертывания и дешевизна) делают Excel весьма популярным. Для пользователей же знакомство с интерфейсом электронных таблиц делает Excel весьма востребованной парадигмой пользовательского интерфейса.

В результате Microsoft обеспечила в Excel 2007 PivotTables поддержку практически всех функциональных возможностей Analysis Services 2005 (и, следовательно, версии 2008 тоже), включая несколько групп мер, ключевые индикаторы производительности KPI, атрибуты измерений (в отличие от пользовательских иерархий), действия, именованные наборы и т. д. Кроме того, компания Microsoft добавила несколько функций в язык формул Excel (чтобы сделать возможными запросы кубов за пределами областей PivotTable электронных таблиц) и существенно облегчила подключение к службам Analysis Services по сравнению с предыдущими версиями.

Более того, с добавлением Excel Services (это по существу интерфейс для просмотра содержимого Excel через Microsoft Office SharePoint) компания Microsoft сделала Excel 2007 инструментом разработки мощных Web-панелей. Это означает, что освоение возможностей OLAP в Excel 2007 дает вам (для работы с вашими кубами Analysis Services) средства создания презентационного уровня, как для настольного компьютера, так и для Web. Кроме того, поскольку PivotTable и интерфейс диаграмм на настольном компьютере обеспечивают списки полей перетаскивания, то вы (и ваши пользователи) можете использовать эти инструменты для запроса ваших кубов без всяких знаний языка MDX.

Все упомянутые возможности доступны без всякого программирования. А для тех, кто желает использовать бизнес-аналитику Excel в своей среде разработки, инструменты Visual Studio Tools for Office позволяют создавать надстройки Excel 2007, которые разрешают программно манипулировать функциями OLAP в Excel.

В этом разделе мы опишем использование Excel как полноценного клиента OLAP (как на настольном компьютере, так и в Web-браузере при помощи Excel Services). После этого мы опишем программирование OLAP в .NET при помощи различных API. Мы также кратко обсудим включенный в примеры кодов данной главы проект Visual Studio Tools for Office. Этот код реализует надстройку Excel 2007, в которой сочетаются обсуждающееся в данной главе программирование API и встроенные возможности бизнес-аналитики Excel. Именно эти возможности мы сейчас и обсудим.

## Подключение к службам Analysis Services

Перед тем как погрузиться в расширенные возможности бизнес-аналитики Excel 2007, давайте опишем основы, как подключить таблицы PivotTable и диаграммы из электронной таблицы Excel к вашим кубам. Для начала вам необходимо запустить Excel 2007, а затем выбрать вкладку **Data** на ленте Ribbon. Затем нажмите кнопку выпадающего меню **From Other Sources** (в группе **Get External Data**) и обратите внимание, что вторая опция выпадающего меню — это **From Analysis Services** (рис. 17.1).

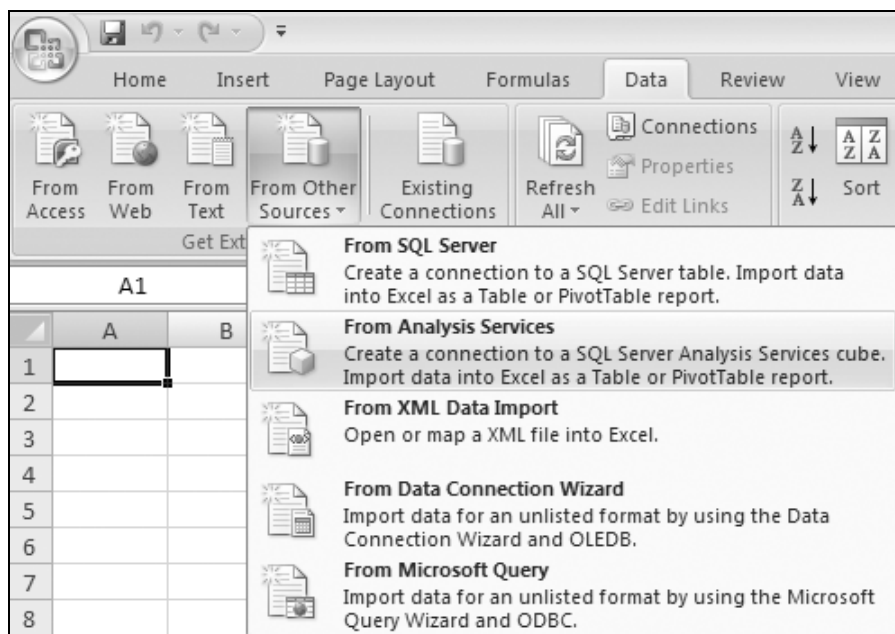
### СОВЕТ

Если вы работаете с низким разрешением экрана, или окно Excel у вас маленькое, то группа **Get External Data** видна не будет. Вместо нее будет видна кнопка **Get External Data** (крайняя слева на вкладке **Data**), которую вам нужно будет нажать. Это приведет к показу группы в выпадающей панели ленты Ribbon, вы сможете затем нажать кнопку меню **From Other Sources** и выбрать пункт **From Analysis Services**.

Выберите вариант **From Analysis Service**, после этого появится мастер Data Connection Wizard. На странице **Connect To Database Server** введите название вашего сервера (может быть localhost или просто .); убедитесь в том, что нажата кнопка **Use Windows Authentication**, а затем нажмите кнопку **Next**.

На странице **Select Database and Table** вы найдете выпадающий список всех баз данных сервера. Выберите вашу базу данных Chapter16, и вы должны

будете увидеть (в сетке под ней) куб Sales и перспективу Product Info (обратите внимание, что в столбце сетки Type программа Excel различает кубы и перспективы). Убедитесь, что флажок **Connect to a specific cube or table** установлен, а затем выберите куб Sales (рис. 17.2).

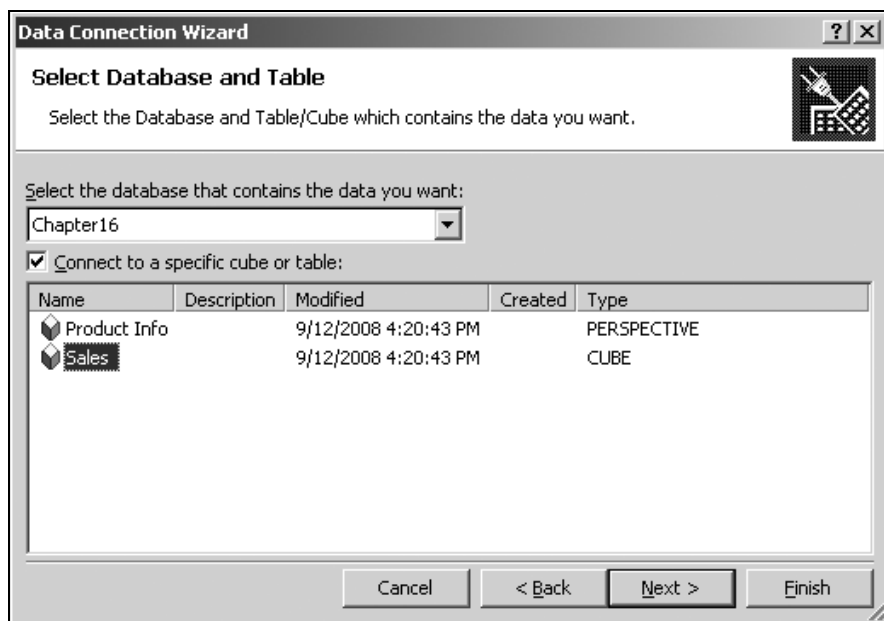


**Рис. 17.1.** Подключение таблиц и диаграмм из электронной таблицы Excel 2007 к вашим кубам

Нажмите кнопку **Next** для перехода на страницу **Save Data Connection File and Finish**. На этой странице вы указываете название файла и местоположение, в котором необходимо сохранить только что предоставленную вами информацию о подключении. Название файла по умолчанию составляется по шаблону `server database cube.odc` (например, `localhost Chapter16 Sales.odc`), а местоположение файла по умолчанию — это каталог My Data Sources (в каталоге My Documents текущего пользователя). Вы можете изменить название по умолчанию при помощи ввода вручную нового названия, также изменить место сохранения по умолчанию при помощи нажатия кнопки **Browse** и перехода к соответствующему каталогу.

Сохраненный этой страницей мастера файл является файлом Office Data Connection (отсюда его расширение `odc`). Если вы грамотно назовете его и

сохраните там, где вы про него не забудете, то сможете использовать его в следующий раз, когда вы захотите подключиться к той базе данных и тому кубу, на который указывает этот файл ods. (Скоро мы покажем вам, как это делается). Есть еще одна причина важности этого файла: для публикации вашего содержимого бизнес-аналитики Excel на сервере SharePoint этот файл должен быть скопирован в библиотеку подключений SharePoint, и электронная таблица должна быть модифицирована таким образом, чтобы она использовала именно эту копию. (Скоро мы покажем вам, как делается и это тоже).



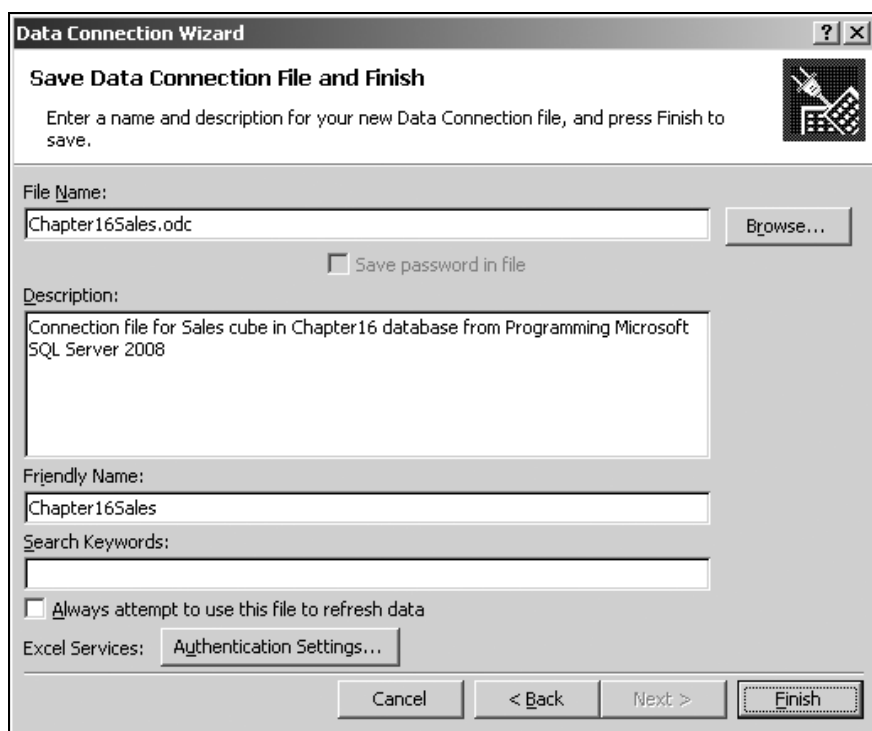
**Рис. 17.2.** Страница **Select Database and Table** мастера Excel Data Connection Wizard

Заполните страницу **Save Data Connection File and Finish** мастера так, как показано на рис. 17.3.

Обратите внимание на изменения полей **File Name** и **Friendly Name**. Поле **Description** необязательно, но вы его заполните именно так, как показано. Не меняйте место сохранения по умолчанию и оставьте пустым поле **Search Keywords**.

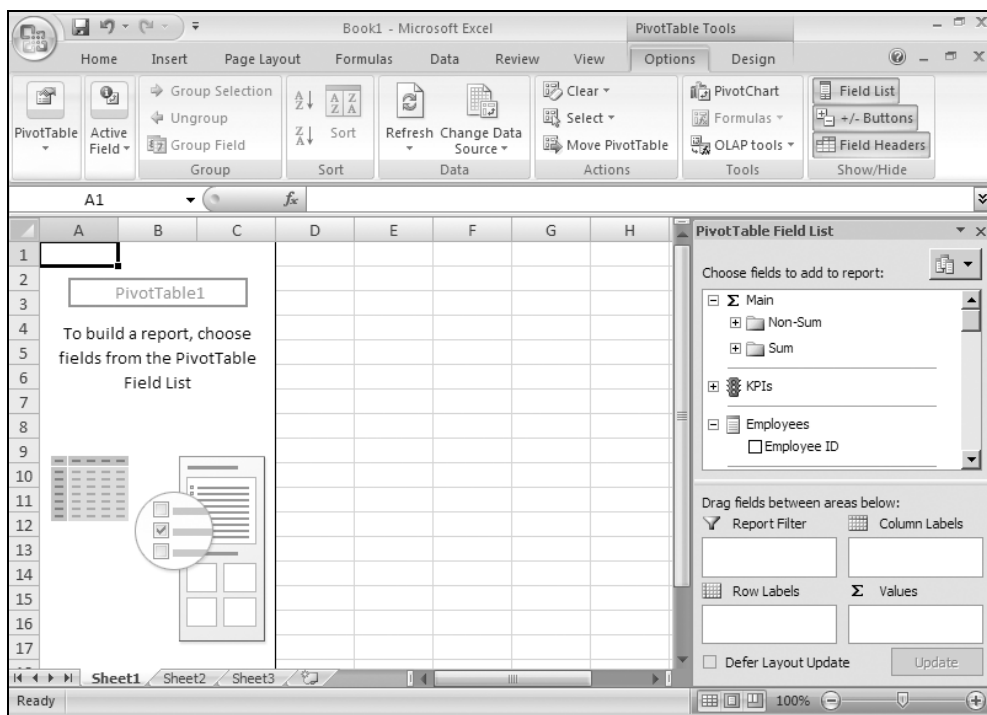
Когда вы закончите, нажмите **Finish**, после чего появится диалоговое окно **Import Data**. Первый набор кнопок позволяет вам выбрать либо создание

только файла подключения (odc), либо файла подключения и PivotTable, либо обоих плюс диаграммы. Если предположить, что вы выбрали создание PivotTable (с диаграммой или без нее), то второй набор кнопок позволяет вам указать, хотите вы вставить новую таблицу PivotTable либо в определенную уже существующую электронную таблицу, либо в новую таблицу (и в какую ячейку). Кнопка **Properties** показывает диалоговое окно **Connection Properties**, которое позволяет вам модифицировать свойства подключения (и в том числе строку подключения).



**Рис. 17.3.** Страница **Save Data Connection File and Finish** мастера Excel Data Connection Wizard

Примите установки по умолчанию и нажмите кнопку **ОК**. Вы увидите пустую таблицу PivotTable на левой стороне вашего экрана и окно **PivotTable Field List** (плавающее или закрепленное) на правой стороне. В нем содержатся все меры (в своих каталогах отображения), КРІ и измерения (их атрибуты, иерархии, именованные наборы) расширенного куба Sales, который мы создали в главе 16 (рис. 17.4).



**Рис. 17.4.** Пустая таблица PivotTable в новой электронной таблице Excel с объектами из нашей базы данных Chapter16 в окне **PivotTable Field List**

Вкладка **PivotTable** файла PivotTableChart.xlsx в кодах этой главы содержит полностью построенную таблицу PivotTable (привязанную к кубу Sales нашей базы данных Chapter16).

### **ПРИМЕЧАНИЕ**

Если бы наш куб имел больше одной группы мер, то в верхней части панели задач **PivotTable Field List** появился бы выпадающий список групп. Чтобы в этом убедиться, попробуйте подключиться к кубу Adventure Works. Обратите внимание, что даже в нашем случае единственная группа мер Main показана в древовидном представлении панели задач как первый родительский узел (слева от нее находится значок с буквой греческого алфавита "сигма").

Перед тем как мы начнем заполнять таблицу, давайте убедимся в том, что созданный нами файл подключения имеется в наличии. Для того чтобы это протестировать, полностью закройте Excel (не сохраняйте никаких изменений в

электронной таблице), а затем опять его запустите. Теперь нажмите кнопку **Existing Connections** в группе **Get External Data** вкладки **Data** ленты Ribbon.

### ПРИМЕЧАНИЕ

Как и прежде, в зависимости от вашего разрешения экрана и размера окна Excel, группа **Get External Data** может и не отображаться. Следуйте нашим предыдущим указаниям для того, чтобы она появилась. Аналогичная процедура может понадобиться и для других описанных в этой главе перемещений по ленте Ribbon. Мы будем надеяться, что вы запомнили эту процедуру, и не будем каждый раз напоминать вам об этом.

После нажатия этой кнопки появляется диалоговое окно **Existing Connections** (рис. 17.5).

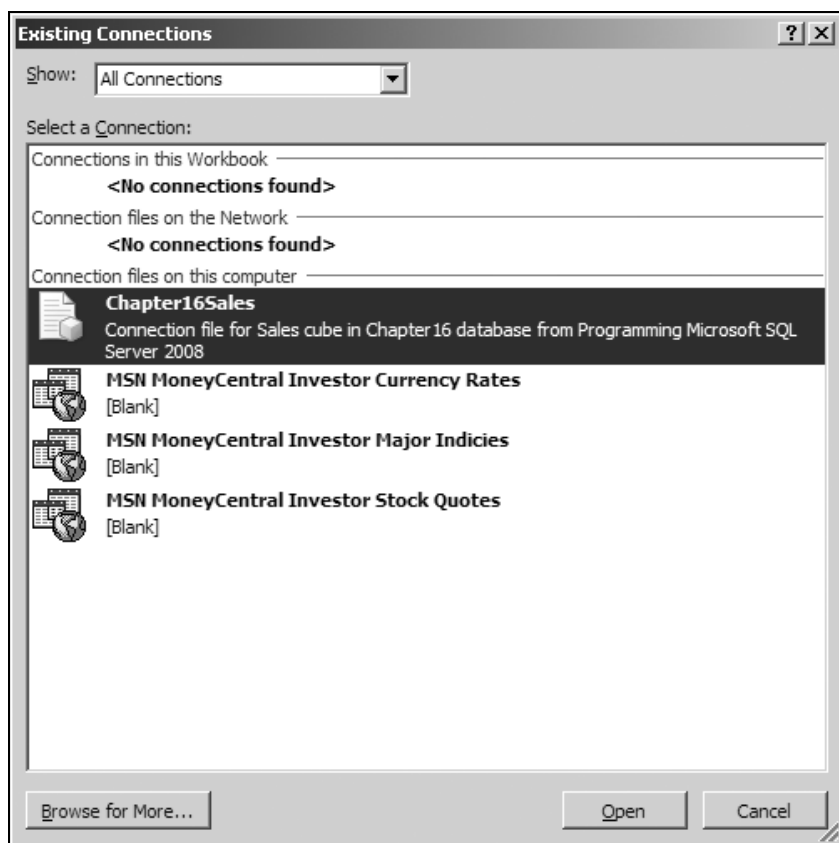


Рис. 17.5. Диалоговое окно **Existing Connections**

Обратите внимание, что созданное нами подключение показано в разделе **Connection files on this computer** данного диалогового окна. Выберите подключение и нажмите **Open** (либо просто дважды щелкните по нему — и появится диалоговое окно **Import Data**). Нажмите кнопку **OK** — и мы снова попадем в пустую таблицу PivotTable, причем все содержимое куба sales будет показано в панели задач **PivotTable Field List** (см. рис. 17.4). Поскольку наш файл *odc* был уже создан и готов, то после открытия Excel нам потребовалось всего пять щелчков мышью (в числе которых был и щелчок выбора вкладки **Data** ленты Ribbon) и никакого ручного ввода.

## Создание PivotTable

Если интерфейс PivotTable и среда, в которой вы сейчас находитесь, кажутся вам знакомыми, то так и должно быть: браузер куба в SQL Server Management Studio и визуальный конструктор куба в проекте Analysis Services используют модифицированную версию PivotTable из Excel 2003.

Понятно, что в Excel 2007 внешний вид значительно изменился, но общий порядок работы в пользовательском интерфейсе сохранился.

Давайте более внимательно рассмотрим показанный на рис. 17.4 список **PivotTable Field List**. Во-первых, обратите внимание, что верхняя половина панели задач содержит древовидное представление полей, а нижняя половина содержит зоны, в которые мы можем поместить поля. Если вы нажмете кнопку выпадающего списка в правой верхней части панели задач, то сможете выбрать показ только одной из этих областей ввода полей либо изменить компоновку совместного представления обеих областей.

В древовидном представлении имеющихся полей обратите внимание на различные значки узлов верхнего уровня (для групп мер, KPI и измерений). В узле группы мер **Main** разверните каждый из двух наших каталогов отображения (для показа наших шести мер, и в том числе двух вычисляемых мер). Раскройте узел KPI для того, чтобы увидеть узел нашего ключевого индикатора производительности **Average Total Sales KPI**, а потом разверните его узел для того, чтобы увидеть узлы его значения, цели, состояния и тенденции. В узле измерения **Geography** углубитесь до иерархии **Country-State-Province-City-Postal Code** для того, чтобы увидеть узлы ее уровней; разверните узел **Other Fields** для того, чтобы увидеть узлы каждого атрибута измерения; раскройте узел **Sets** для того, чтобы увидеть узел нашего именованного набора **Test Markets**.



Обратите внимание, что большинство из листовых узлов имеет флажки. Они для того, чтобы вы могли без труда включить соответствующие объекты в вашу таблицу PivotTable. Попробуйте это сделать прямо сейчас, выставьте флажки (именно в этом порядке) для узлов меры Total Sales, иерархии Country-State-Province-City-Postal Code измерения Geography, а также атрибута Reports To (иерархии родитель/потомок измерения Org Chart).

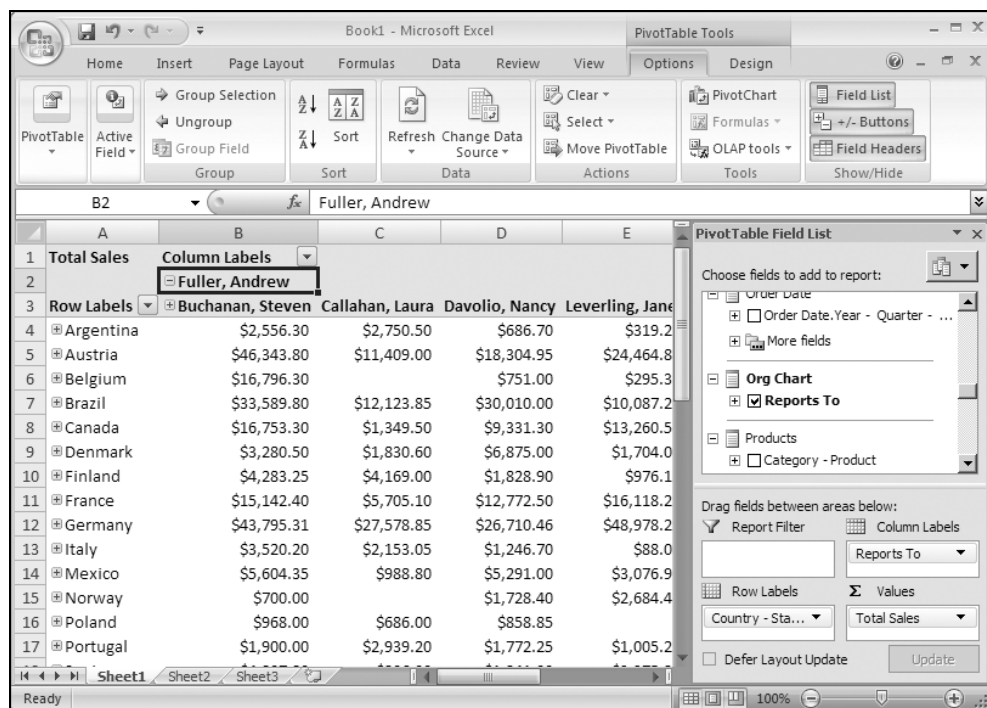


Рис. 17.6. Заполненная таблица PivotTable с запросом к кубу Sales

Если вы посмотрите на PivotTable, то вы заметите, что оба измерения были спроецированы на ось строк, а ось столбцов пуста. Для того чтобы это исправить, посмотрите на зоны полей панели **PivotTable Field List** (расположенные по умолчанию в нижней половине панели **PivotTable Field List**). Обратите внимание, что там есть массив списков, который можно использовать для настройки содержимого зон **Report Filter**, **Row Labels**, **Column Labels** и **Values**. Обратите также внимание, что оба измерения есть в списке **Row Labels**. Для переноса атрибута Reports To на ось столбцов таблицы PivotTable просто перетащите его из списка **Row Labels** в список **Column Labels**. После

этого углубитесь по заголовку столбца Fuller, Andrew. Результат должен выглядеть так, как показано на рис. 17.6.

Кроме выбора элементов в списке **PivotTable Field List** (при помощи установки флажков) вы можете перетаскивать их из древовидного элемента управления в любую из четырех зон для полей. Однако в отличие от браузера кубов в Visual Studio и Management Studio, вы не можете перетащить элемент непосредственно в таблицу PivotTable. В одной зоне может находиться несколько элементов, причем вы можете их удалять путем перетаскивания (из списка, в котором они находятся) за физические пределы области зон панели **Pivot Table Field List**. Когда элементы находятся внутри определенной зоны, вы можете нажимать (в списке зоны) стрелки их выпадающих списков для отображения контекстного меню, которое позволяет вам переносить или удалять их, а также изменять их настройки.

## Изучение данных PivotTable

В древовидном представлении имеющихся полей вы можете нажать стрелку выпадающего списка справа от любой иерархии, уровня или атрибута и получить список элементов, каждый из которых вы можете включить или исключить (рис. 17.7).

Вы можете также перетащить любой из этих элементов в зону **Report Filter**, а затем показать список элементов (щелчком по стрелке выпадающего списка рядом с элементом в зоне фильтров самой таблицы PivotTable, как показано на рис. 17.8).

Эти возможности аналогичны тем, которые предлагает браузер кубов. Фактически перетаскивание позиций в список **Report Filter** и выполнение выбора в списке их элементов в области фильтров позволяет вам использовать эту зону в PivotTable как зону **Filter Fields** браузера кубов.

Для развертывания нелистового элемента в зонах **Row** или **Column** таблицы PivotTable просто щелкните по его значку плюса или дважды щелкните по метке. Для свертывания нелистового элемента щелкните по его значку минуса или еще раз дважды щелкните по метке. В контекстном меню элемента в пункте **Expand/Collapse** есть еще много вариантов развертывания и свертывания.

Наведение курсора на любую ячейку в области данных таблицы PivotTable приведет к показу подсказки. Эта подсказка напомнит вам об отображаемой мере, о ее значении, а также о значениях элементов для данной строки и данного столбца (в которых находится данная ячейка).

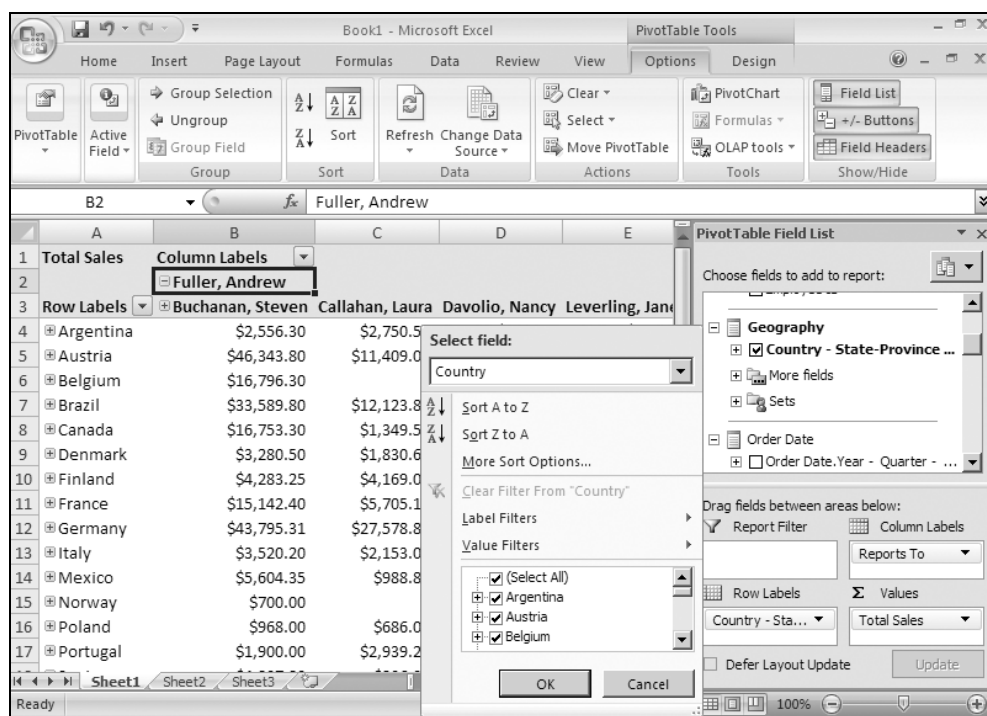


Рис. 17.7. Выбор элементов в панели задач PivotTable Field List

Любые определенные в кубе действия доступны в контекстных меню. Например, щелкните правой кнопкой мыши по любой ячейке с данными и выберите пункт **Additional Actions**, и далее **Total Sales Drillthrough** для выполнения действия детализации, которое мы создали в главе 16. Либо углубитесь по меткам строк до города, а затем щелкните его правой кнопкой мыши и выберите **Additional Actions**, поиск Live Search для выполнения созданного нами действия URL. (По завершении снова сверните родительские узлы города, чтобы на оси строк PivotTable отображались только страны.)

Действия интересны тем, что Excel не нуждается в них для того, чтобы выполнить для вас детализацию. Фактически вы можете дважды щелкнуть по любой ячейке в области данных PivotTable (или выбрать пункт **Show Details** контекстного меню ячейки) для получения всех подробностей детализации. Попробуйте — и вы заметите, что такая обобщенная детализация показывает все столбцы группы мер, в то время как действие детализации покажет только те столбцы, которые были включены при создании данного действия.

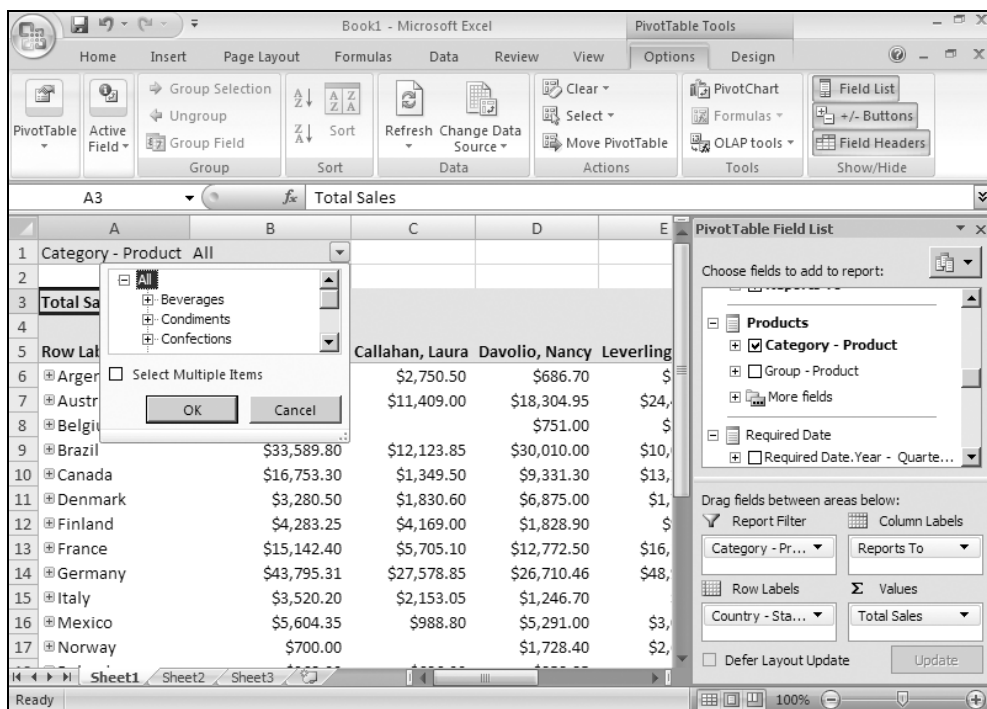


Рис. 17.8. Выбор элементов в зоне фильтров PivotTable

Назовите текущую вкладку PivotTable (чтобы мы всегда могли к ней вернуться). Для этого щелкните правой кнопкой мыши по вкладке **Sheet1**, а затем выберите пункт **Rename** контекстного меню (или просто дважды щелкните по вкладке). Теперь вы можете ввести PivotTable в области названия вкладки. Щелкните вне вкладки (или нажмите клавишу <Enter>) для сохранения измененного названия.

## Системы показателей

Как уже упоминалось ранее, Excel 2007 стал очень хорошим клиентом для построения панелей мониторинга, которые могут содержать системы показателей, использующие KPI. Для того чтобы в этом убедиться, выберите вкладку с названием **Sheet2**, переименуйте ее в **Scorecard**, а затем нажмите кнопку **Existing Connections** в группе **Get External Data** вкладки **Data** ленты Ribbon. Выберите **Chapter16Sales** в разделе **Connections In This Workbook** диалогового окна **Existing Connections** и нажмите **Open**, а затем щелкните по кнопке

The screenshot displays the Microsoft Excel 2010 interface. The main window shows a PivotTable with the following data:

| Row Labels      | Average Total Sales KPI | Average Total Sale | Average Total S |
|-----------------|-------------------------|--------------------|-----------------|
| Calendar 1996   | \$1,754.25              |                    |                 |
| Quarter 1, 1996 |                         |                    |                 |
| Quarter 2, 1996 |                         |                    |                 |
| Quarter 3, 1996 | \$1,340.28              |                    |                 |
| July 1996       | \$1,509.61              |                    |                 |
| August 1996     | \$1,209.52              |                    |                 |
| September 1996  | \$1,316.00              |                    |                 |
| Quarter 4, 1996 | \$2,149.41              |                    |                 |
| Calendar 1997   | \$2,522.56              |                    |                 |
| Calendar 1998   | \$5,219.68              |                    |                 |
| Grand Total     | \$2,821.79              |                    |                 |

The PivotTable Field List task pane on the right shows the following configuration:

- Choose fields to add to report:**
  - ☐ Sets
  - ☒ Order Date
    - ☒ Order Date.Year - Quarte...
    - ☐ More fields
  - ☐ Org Chart
    - ☐ Reports To
- Drag fields between areas below:**
  - Report Filter:** (Empty)
  - Column Labels:** Values
  - Row Labels:** Order Date.Y...
  - Values:** Average T..., Average T...
- ☐ Defer Layout Update
- Update** button

The status bar at the bottom indicates the active sheet is 'Scorecard' and the zoom level is 100%.

Вы заметите, что Excel 2007 отображает состояние и тенденцию совсем не теми графическими элементами, которые были определены в предыдущей главе для Average Total Sales KPI; однако мы можем изменить оба эти графических индикатора при помощи функции Conditional Formatting в Excel 2007. Выберите заголовок столбца с (чтобы выделился весь столбец), а затем нажмите вкладку **Home** на ленте Ribbon. Нажмите кнопку **Conditional For-**

**matting** выпадающего меню (в группе **Styles**), а затем выберите пункт **Icon Sets** для отображения графического выпадающего меню (рис. 17.10).

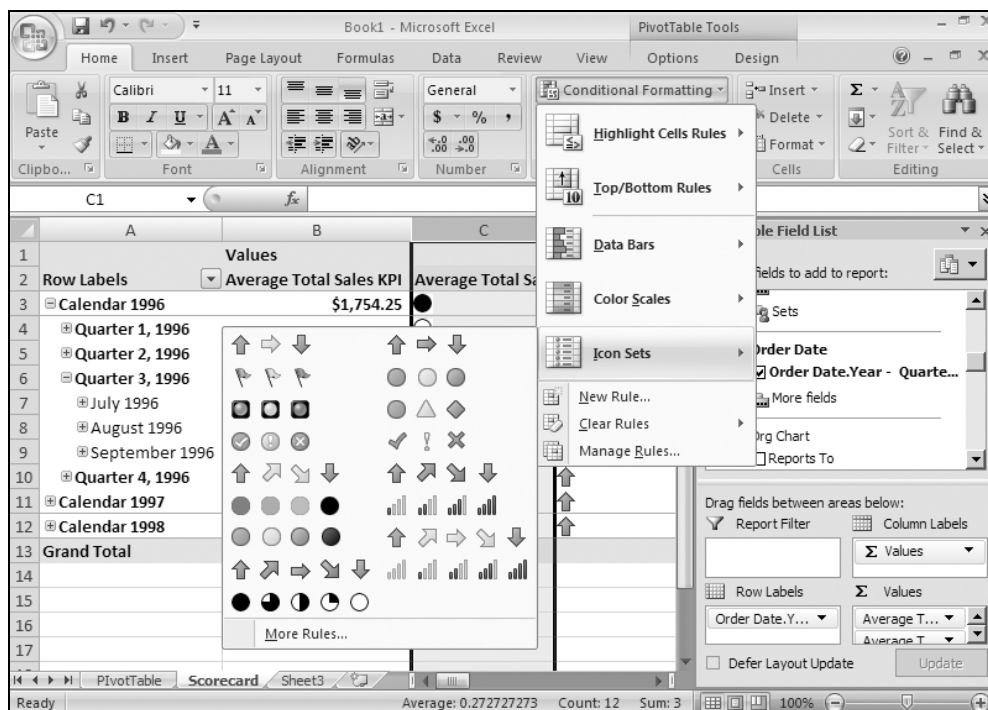


Рис. 17.10. Варианты наборов значков

Когда вы проводите указатель мыши над набором значков, то вы видите предварительное представление выбранного элемента в самой таблице PivotTable и большую подсказку, которая показывает вам название и описание выбранного элемента. Если вы переведете указатель мыши из пункта **Icon Sets** в пункт **Color Sets** и наведете его на имеющиеся там варианты, то вы увидите, что состояние KPI может быть представлено не только значком, но и цветом фона ячейки (см. рис. 17.10). Вернитесь в пункт **Icon Sets** и выберите вариант **3 Traffic Lights (Unrimmed)**, обычно это второй вариант во втором столбце.

В таблице PivotTable вы должны увидеть, что старые черно-белые значки сменились симпатичным зелено-желто-красным светофором (значений состояния). Значки появляются с левого края каждой ячейки, видны также и значения состояния (на правом краю ячейки).

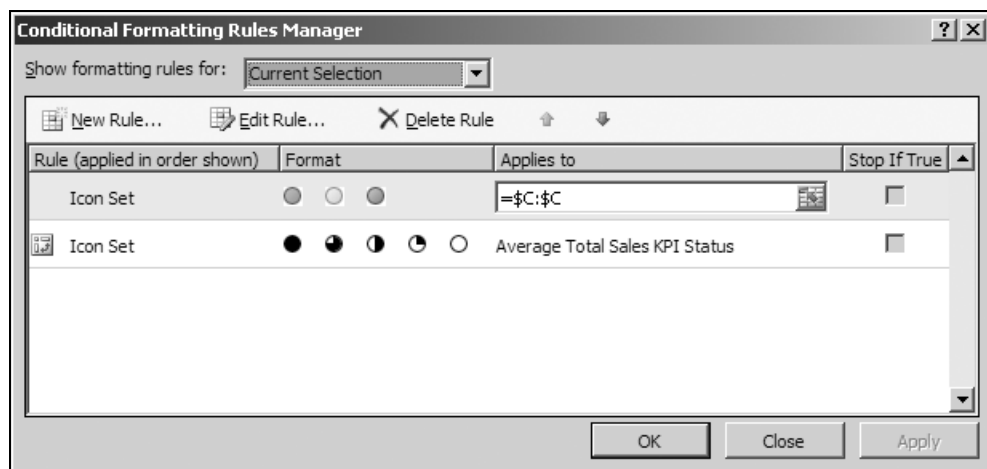


Рис. 17.11. Диалоговое окно Conditional Formatting Rules Manager

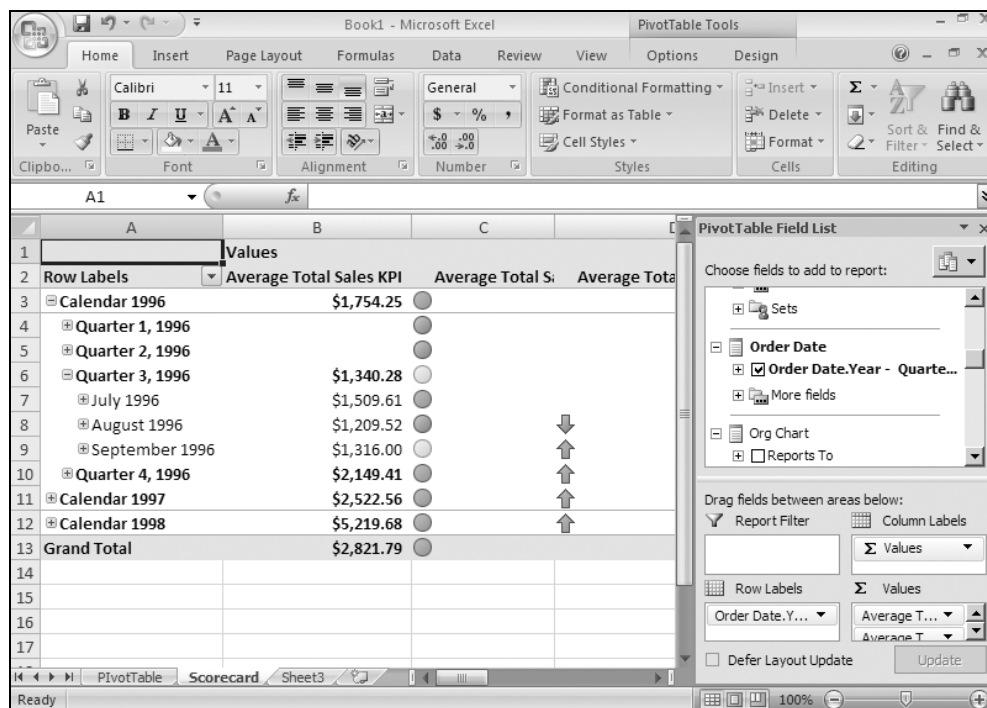


Рис. 17.12. Законченная система показателей с цветными наборами значков

Для того чтобы оставить значки, но скрыть значения, нажмите кнопку выпадающего меню **Conditional Formatting**, а затем щелкните по пункту **Manage Rules**. Это приведет к показу диалогового окна **Conditional Formatting Rules Manager** (рис. 17.11).

В этом диалоговом окне дважды щелкните по строке, соответствующей только что выбранному нами значку трехцветного светофора (или выберите ее и нажмите **Edit Rule** в панели инструментов). После этого появится диалоговое окно **Edit Formatting Rule**; установите флажок **Show Icon Only** и нажмите кнопку **OK**, а затем нажмите **OK** еще раз для того, чтобы закрыть диалоговое окно **Conditional Formatting Rules Manager**. Ваш столбец состояния теперь должен показывать только значки (а значение состояния должно быть скрыто). Теперь выберите заголовок столбца D (чтобы выделить весь столбец тенденции), измените набор его значков на **3 Arrows (Colored)** (первый вариант в первом столбце меню **Icon Sets**), а затем отредактируйте правило столбца (как вы сделали это для столбца **Status**), чтобы скрыть значение тенденции и показывать только значок. Ваша система показателей должна выглядеть так, как показано на рис. 17.12.

## Создание и настройка диаграмм

Мы создали очень полезную таблицу PivotTable и систему показателей и теперь очень близки к тому, чтобы экспортировать все в Excel Services для построения красивого представления панели мониторинга данных по продажам из базы данных Northwind. Однако нам пока не хватает одного элемента — представления наших данных в виде диаграммы. К счастью, Excel с невероятной легкостью позволяет представить данные PivotTable в графическом виде. Для этого вернитесь в ту вкладку нашей электронной таблицы, которую мы назвали **PivotTable**, и щелкните любую ячейку таблицы PivotTable. Теперь щелкните по вкладке **PivotTable Tools/Options** ленты Ribbon, а затем нажмите кнопку **PivotChart** в группе **Tools**. После этого будет показано диалоговое окно **Chart** (рис. 17.13).

По умолчанию выбран тип диаграммы **Clustered Column**, измените его на **Stacked Column** (вариант справа от **Clustered Column**), а затем нажмите кнопку **OK**. Это приведет к вставке диаграммы на вкладку **PivotTable** (как встроенного объекта) и автоматически покажет четыре новых вкладки **PivotChart Tools** в ленте Ribbon, причем первая из них — **Design** — будет активна. Для переноса новой диаграммы в ее собственную электронную таблицу нажмите кнопку **Move Chart** (крайняя справа во вкладке **PivotChart Tools/Design**), чтобы показать диалоговое окно **Move Chart**.



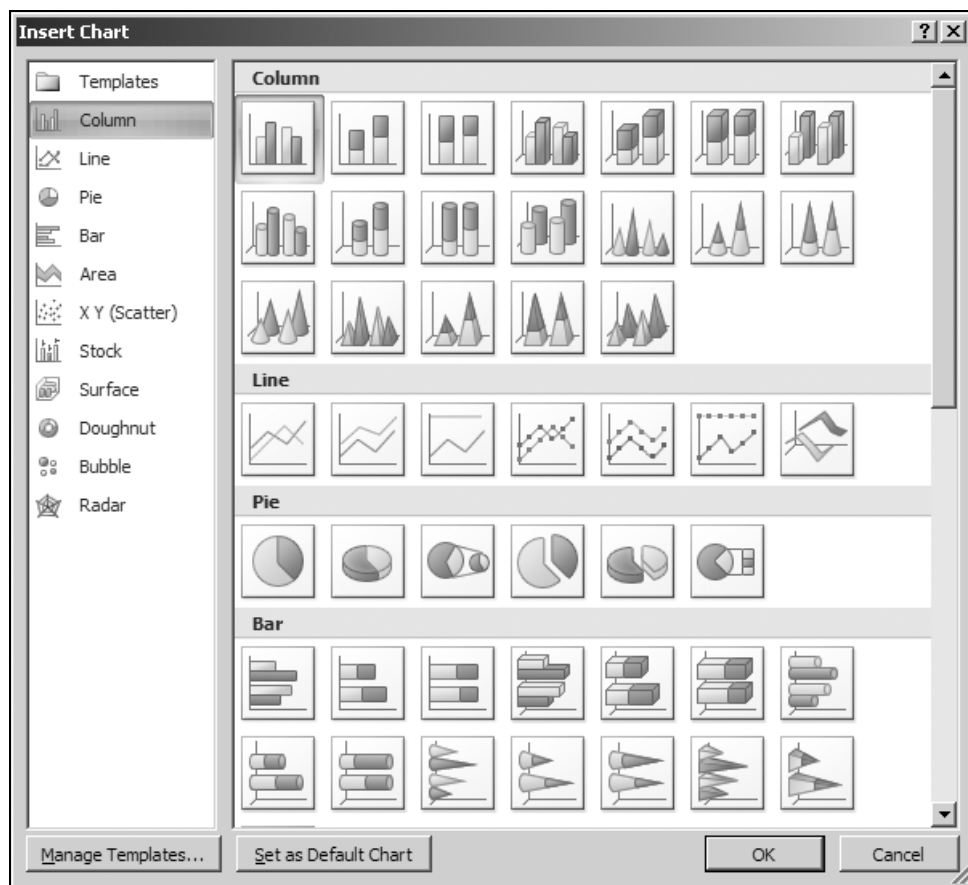


Рис. 17.13. Диалоговое окно Chart

Выберите пункт **New Sheet**, измените название по умолчанию на **PivotChart**, а затем нажмите кнопку **OK**. Теперь диаграмма должна появиться в своей собственной вкладке (с показанной панелью **PivotChart Filter Pane**) — рядом с панелью задач **PivotTable Field List**.

Обратите внимание, что во вкладке **PivotChart Tools/Design** ленты Ribbon слева от кнопки **Move Chart** имеется галерея **Chart Styles**. На самом правом краю галереи имеются кнопки прокрутки вверх и вниз и кнопка **More** под ними. Нажмите кнопку **More** для открытия всей галереи во всплывающем окне (рис. 17.14).

Если вы наведете курсор мыши на вариант с черным фоном и многоцветным передним планом (который находится на рис. 17.14 во втором столбце по-

следней строки галереи), то вы увидите подсказку в виде сообщения "Style 42". Щелкните этот вариант. И наконец, для снижения количества отображаемых в диаграмме столбцов, откройте выпадающий список **PivotChart Filter Pane's Axis Fields**. В появившемся диалоговом окне снимите флажок **Select All** на самом верху, а затем вручную установите флажки для Канады, Мексики и США, потом нажмите кнопку **ОК**. Результат должен выглядеть так, как показано на рис. 17.15 (где мы специально закрыли все панели задач и минимизировали ленту Ribbon, чтобы максимизировать область экрана под диаграмму).

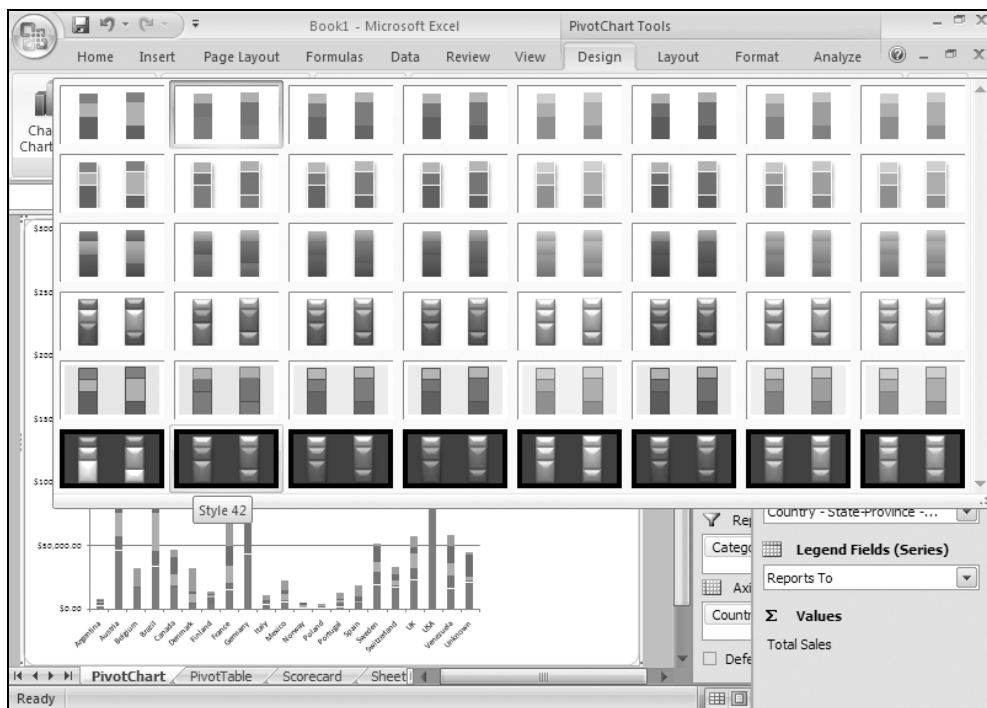


Рис. 17.14. Галерея Chart Styles

Вкладка **PivotChart** файла **PivotTableChart.xlsx** (из кодов данной главы) содержит законченную трехмерную многослойную столбчатую диаграмму, связанную с таблицей на вкладке **PivotTable** файла.

Таблицы **PivotTable** и вкладки **PivotChart** связаны в смысле настроек фильтрации и развертывания/свертывания. Если вы вернетесь обратно во вкладку **PivotTable** после выполнения этих изменений в диаграмме, то увидите, что

изменения в части фильтрации отражены в PivotTable. Если вы раскроете Канаду в PivotTable, а затем вернетесь на вкладку **PivotChart**, то вы увидите, что там тоже произошло развертывание. Поэкспериментируйте еще немного, и вы обнаружите, что и обратное также верно.

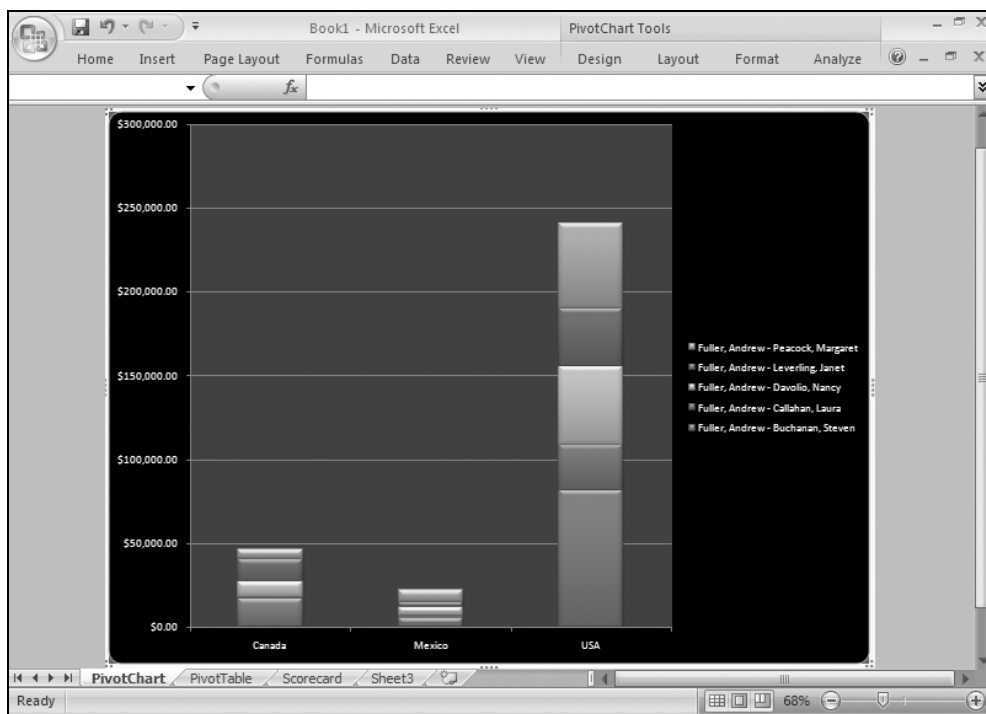


Рис. 17.15. Законченная диаграмма

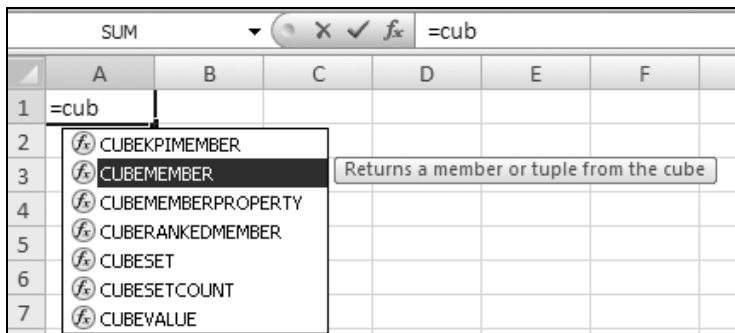
## Запросы к кубам внутри формул

PivotTable представляет собой вполне полноценный клиент OLAP, практически все возможности браузера кубов в ней доступны. Эти функциональные возможности (в сочетании с вездесущностью Excel) делают Excel заманчивым средством развертывания для ваших OLAP-потребностей. Но вместе с Excel 2007 пришла новая интригующая функция: способность доставлять данные OLAP в ячейки электронной таблицы без всякого использования PivotTable.

Перед тем как продолжить, вы можете при помощи перетаскивания переупорядочить вкладки, чтобы вкладка **PivotChart** была расположена между

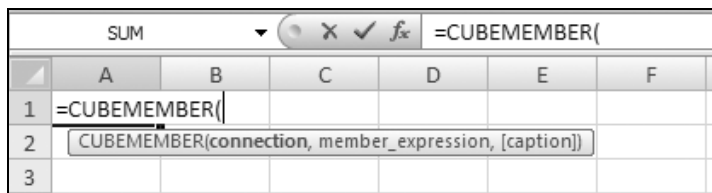
вкладками **PivotTable** и **Scorecard**. После этого щелкните последнюю (пустую) вкладку и измените ее метку с **Sheet3** на **Cell Formulas**.

На этой вкладке мы совсем не будем использовать PivotTable или диаграмм, а вместо этого будем запрашивать кубы при помощи формул ячеек. Для того чтобы вы поняли, что мы имеем в виду, щелкните в любой ячейке и введите знак равенства =, а за ним введите cub. Появится список технологии IntelliSense, в котором перечислены семь новых важных функций ячеек (рис. 17.16).



**Рис. 17.16.** Всплывающий список функций CUBE (технология IntelliSense)

Обратите внимание, что при выборе вами любой из семи функций появляется подсказка, показывающая описание функции. Оказывается, две из этих семи функций — это все, что нам нужно для того, чтобы начать использовать эту новую возможность Excel. Эти две функции (CUBEMEMBER и CUBEVALUE) возвращают элемент измерения и значение ячейки куба. Выберите CUBEMEMBER (вторая функция в списке), а затем нажмите клавишу <Tab>. Появится подсказка синтаксиса IntelliSense (рис. 17.17).



**Рис. 17.17.** Подсказка синтаксиса IntelliSense для функции CUBEMEMBER

Синтаксис этих функций непосвященным может показаться несколько пугающим. Это особенно верно для параметров функций (таких как параметр

member\_expression функции CUBEMEMBER). Несмотря на то, что мы находимся в Excel, этот параметр рассчитывает получить строку MDX-выражения! Но не волнуйтесь. Excel (для облегчения создания формул CUBE) вооружит нас двумя видами оружия: функцией Convert To Formulas и отличной поддержкой IntelliSense при вводе выражений. Давайте же рассмотрим оба эти вспомогательные средства.

## Функция *Convert To Formulas*

Мы начнем с функции Convert To Formulas, поскольку ее очень легко объяснять и использовать. Сначала нажмите клавишу <Esc> для возврата из ввода формулы ячейки (который вы уже начали). Затем щелкните по вкладке **PivotTable**, а потом щелкните внутри самой таблицы PivotTable. Теперь выберите вкладку **PivotTable Tools/Options** ленты Ribbon, а затем нажмите кнопку меню **OLAP Tools** (вторая справа) в группе **Tools**. В выпадающем меню выберите опцию **Convert To Formulas**. Теперь наблюдайте чудо: вся таблица PivotTable преобразуется в обычные ячейки электронной таблицы, заполненные функциями CUBEVALUE и CUBEMEMBER. Щелкните по любой из ячеек и посмотрите на строку формул (чтобы увидеть, как пишутся подобные формулы). Обратите внимание на ссылку на наше подключение Chapter16Sales и в формулах ячеек данных на различные ссылки на ячейки, которые указывают на определенные элементы измерений Geography и Org Chart. Это очень впечатляющая функциональная возможность Excel 2007.

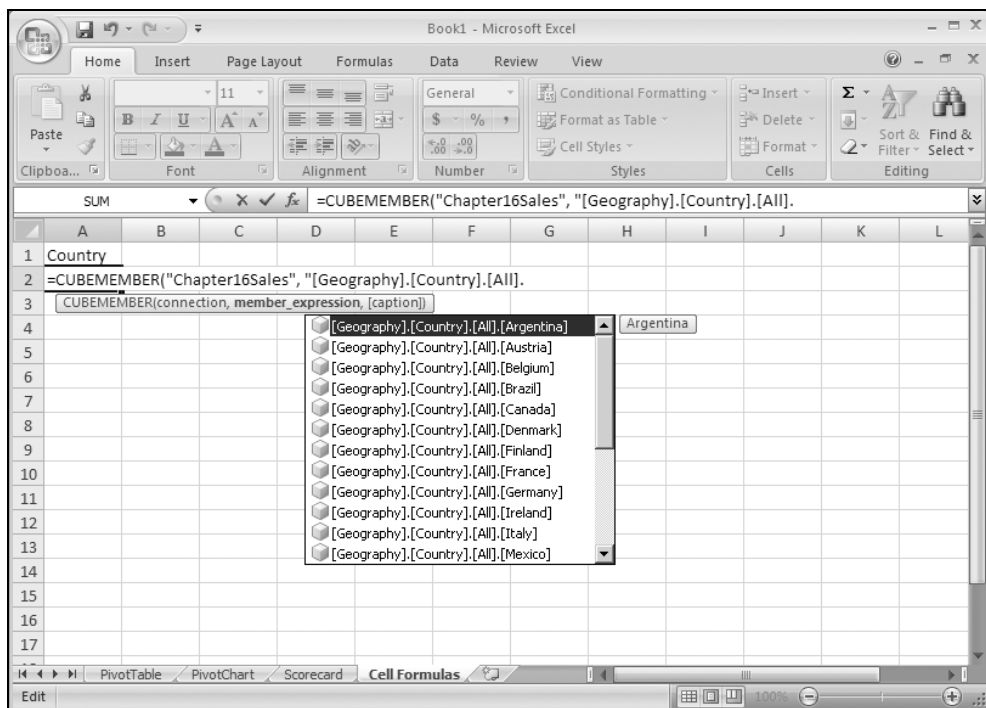
## Excel предоставляет технологию IntelliSense

Теперь давайте изучим самостоятельное создание этих формул (с помощью технологии IntelliSense). Сначала нажмите кнопку **Undo** (или клавиатурную комбинацию <Ctrl>+<Z>), чтобы вернуть таблицу PivotTable обратно в электронную таблицу. Затем перейдите обратно в таблицу Cell Formulas (в ячейку A1), а потом введите слово Country. Перейдите в ячейку A2 и снова введите знак равенства =, а за ним введите cub; затем выберите в выпадающем списке CUBEMEMBER и нажмите клавишу <Tab>.

Подсказка изображает **connection** жирным шрифтом, показывая, что мы должны ввести название подключения. Проблема в том, что мы не хотим вводить это название вручную, а список существующих подключений не выдается. Для того чтобы это исправить, просто введите знак двойных кавычек ". После этого появится список подключений. Поскольку в нашей электронной

таблице есть только одно подключение, то список будет содержать единственный элемент, который будет сразу выделен (**Chapter16Sales**, причем введенное нами для него описание видно в этой подсказке). Просто нажмите клавишу <Tab> для выбора подключения, а затем введите закрывающие двойные кавычки, запятую и пробел.

Теперь подсказка запрашивает у нас выражение для элемента. Вот здесь все может стать непросто, однако все это не слишком сложно (если вы знаете, что надо делать). Начните опять с ввода двойной кавычки (это всегда следует делать в том случае, когда отсутствует список вариантов, и вы хотите, чтобы он появился), после чего появится всплывающий список измерений куба Sales.



**Рис. 17.18.** Всплывающий список элементов атрибута Country для измерения Geography (технология IntelliSense)

Выберите `[Geography]` (щелкнув по нему и нажав клавишу <Tab>), а затем введите точку `.`, чтобы увидеть список атрибутов и иерархий. Выберите `[Geography].[Country]`, а затем введите еще одну точку. Появится список с

единственным элементом [Geography].[Country].[All]. Выберите его (несмотря на то, что элемент All — это не то, что нам нужно) и введите еще одну точку. Теперь появится всплывающий список стран (элементов атрибута Country), как на рис. 17.18.

Выберите [Geography].[Country].[All].[Canada], введите закрывающие двойные кавычки и закрывающую скобку, а затем нажмите клавишу <Enter>.

Теперь в ячейке A2 появится слово Canada. Все это может показаться слишком большой работой для того, чтобы просто вставить название страны, но мы обещаем вам, что ваши труды будут вознаграждены. В качестве следующего шага скопируйте формулу из ячейки A2 в ячейки A3 и A4. Затем отредактируйте формулы в этих двух копиях так, чтобы они ссылались на Mexico и USA (вместо Canada). Обратите внимание, что IntelliSense помогает вам редактировать MDX-выражение (а не только создавать новое).

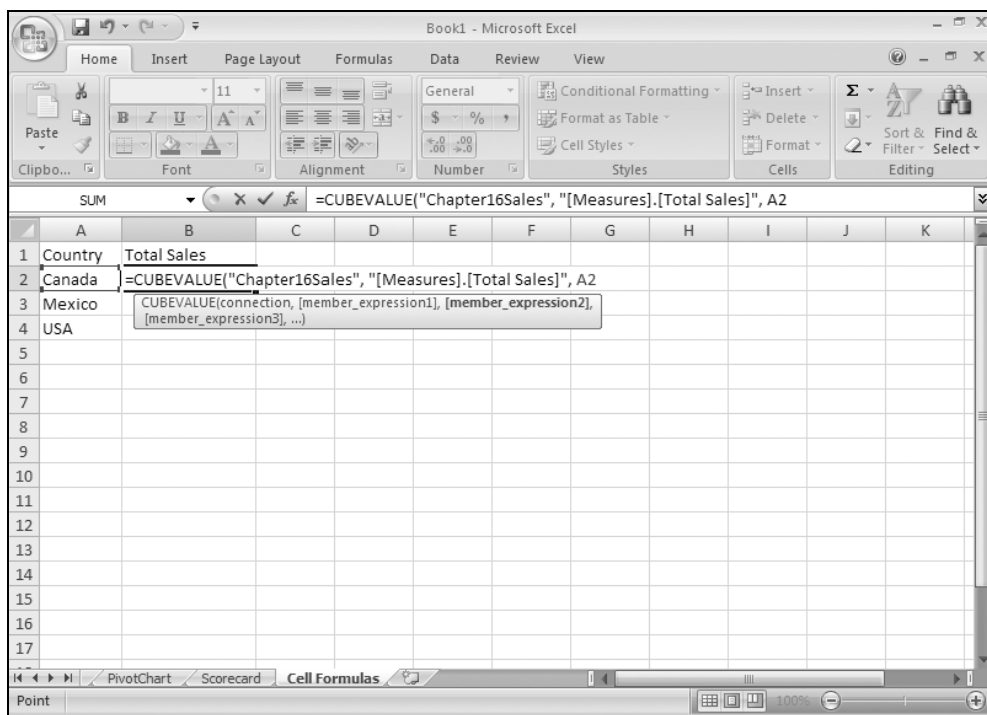
Перейдите в ячейку B1 и введите метку Total Sales. Перейдите в ячейку B2 и начните вводить формулу CUBEVALUE (при помощи примерно той же процедуры, которую мы использовали для ввода CUBEMEMBER). Введите название подключения и начните вводить выражение MDX. Когда появится список измерений, выберите [Measures], введите точку, выберите [Measures].[Total Sales], а затем введите закрывающие двойные кавычки, запятую и пробел.

### **ПРИМЕЧАНИЕ**

В языке MDX меры куба считаются элементами специального измерения Measures. Поэтому в выражениях MDX мы выбираем наши меры из списка измерений, который нам дает IntelliSense в функции CUBEVALUE. Мы обсудим это подробнее далее в этой же главе.

Интересно то, что функция CUBEVALUE может теоретически принять бесконечное количество выражений для элементов. В нашем случае мы хотим ввести еще одно, чтобы мы могли получать объемы продаж именно по Канаде. Сделать это очень просто, поскольку вместо повторного указания выражения элемента для Канады мы можем просто сделать ссылку на ячейку A2. Для этого щелкните по ячейке A2. Вы должны увидеть, как ее адрес добавляется в формулу (в строке формул на рис. 17.19).

Теперь введите закрывающую скобку и нажмите клавишу <Enter>. Возможно, вы увидите появление (на короткое время) в ячейке B2 нескольких решеток (#) или текста #GETTING DATA, после чего будет показано значение \$55,334.10. Это значение является общим объемом продаж по Канаде (по всему кубу).



**Рис. 17.19.** Вставка ссылки на ячейку в функцию CUBEVALUE (при помощи щелчка по ячейке)

Если это все еще не производит на вас впечатления, попробуйте следующее: щелкните ячейку B2, а затем зацепите манипулятор ячейки (жирная квадратная точка в правой нижней части ячейки) и перетащите ее вниз на ячейки B3 и B4. Вы должны будете увидеть появление в этих ячейках общих объемов продаж для Mexico и для USA. Представьте себе, что у вас имеется длинный список стран (а не только две страны) — объединение функционала OLAP, формул ячеек и ссылок на ячейки дает очень мощную комбинацию.

## Форматирование таблиц

Теперь выделите все восемь заполненных ячеек от A1 до B4. Выберите вкладку **Home** ленты Ribbon, а затем нажмите кнопку меню **Format As Table** в группе **Styles**. Выберите **Table Style Medium 3** (четвертая строка, третий столбец). Обратите внимание, что теперь наши данные отформатированы как таблица Excel, мы можем фильтровать и сортировать столбцы. Затем выде-



лите ячейки B2:B4, опять выберите вкладку **Home** ленты Ribbon, нажмите кнопку меню **Conditional Formatting** (в группе **Styles**), выберите вариант **Data Bars**, а потом выберите в подменю опцию **Blue Data Bar** (первая). Вы добавили гистограмму для улучшения визуализации представленных в таблице данных.

## Фильтрация при помощи именованных ячеек

Переместите курсор в ячейку A7, введите метку `Calendar Year`, а затем нажмите клавишу <Enter>. Перейдите в ячейку B7 и введите 1996. Нажмите клавишу <Enter> и назовите эту ячейку `Calendar_Year`. Вы можете либо ввести это название в поле названия Excel (под лентой Ribbon, слева от строки формул), либо использовать вариант **Name A Range** в контекстном меню ячейки.

Теперь вернитесь обратно и отредактируйте формулу ячейки B2. Поместите курсор следом за ссылкой на ячейку A2 (до закрывающей скобки). Введите запятую и пробел, а затем добавьте для нового выражения элемента следующую формулу:

```
CONCATENATE(" [Order Date] . [Year] . [All] . [Calendar ", Calendar_Year, " ] ")
```

Обратите внимание на имеющийся в конце первой строки пробел. Теперь нажмите клавишу <Enter>, и вы должны увидеть обновление значений всех трех ячеек, они будут показывать значения меры `Total Sales` для соответствующих стран за 1996 г. Закончим форматированием ячеек B2:B4, чтобы они показывали значения как валюту.

### СОВЕТ

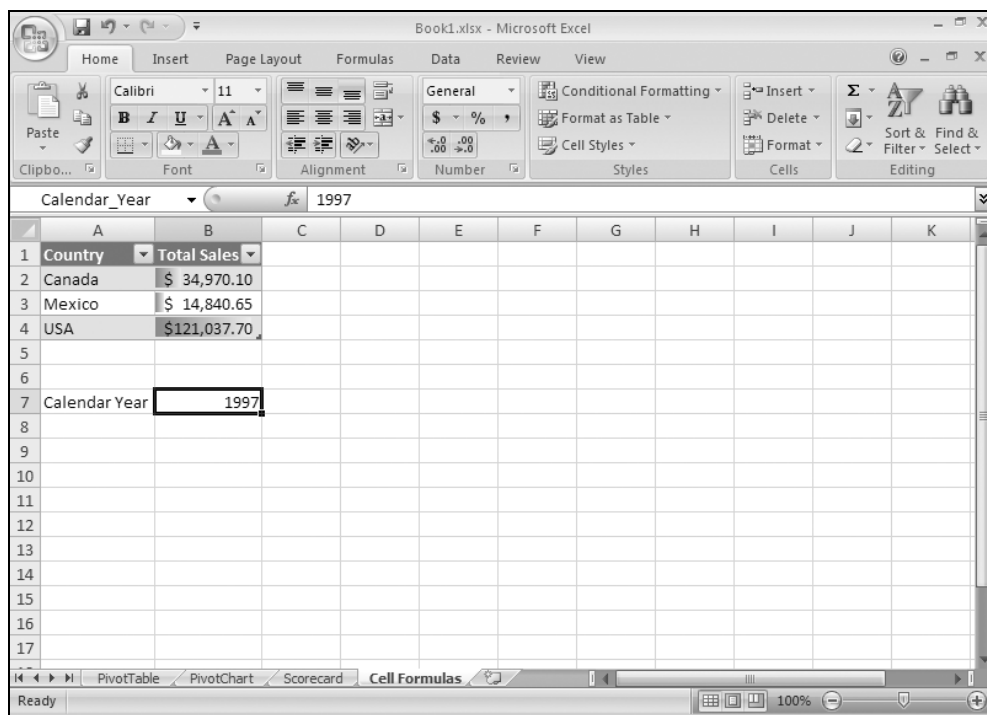
Если по какой-либо причине ячейки B3 и B4 не обновляются, скопируйте в них формулу из ячейки B2.

Измените значение в ячейке B8 на 1997 — и вы опять увидите обновление значений ячеек выше нее. Ваша законченная таблица должна выглядеть, как показано на рис. 17.20.

Давайте посмотрим на то, что мы сделали и построили, чтобы вы все поняли перед тем, как мы экспортируем этот контент бизнес-аналитики Excel в SharePoint (при помощи Excel Services). Сначала мы добавили заголовок столбца `Country` и формулы (под ним), которые перечислили три элемента атрибута `Country` измерения `Geography`. В соседнем столбце справа мы добавили заголовок столбца `Total Sales` и формулы для запроса значений меры `Total Sales` для каждой из стран (которые мы перечислили в первом столб-

це). Второй набор формул ссылается на первый (чтобы указать срез куба, по которому делаются ограничения).

Затем мы добавили в наш контент форматирование таблиц и диаграмму. Закончили мы редактированием формул в столбце Total Sales (чтобы и здесь ввести ограничение по календарному году, значение которого мы поместили в ячейку с названием Calendar\_year). Когда мы редактировали наши формулы Total Sales для ограничения по значению этой ячейки, мы сделали это при помощи ссылки на присвоенное ей название диапазона.



**Рис. 17.20.** Заполненная таблица с формулами ячеек, форматированием и фильтром по календарному году

## Инструменты Visual Studio Tools for Office и надстройки Excel

Одно из самых замечательных свойств, обсуждавшихся нами до настоящего момента функций Excel 2007 — это то, что все они программируемые. При

помощи использования технологии Visual Studio Tools for Office в Visual Studio 2008 вы можете создать надстройку для Excel, которая строит PivotTable, диаграммы, системы показателей и даже формулы ячеек — и все это при помощи кода. Более того, вы можете комбинировать эти методы с использованием ADO MD.NET и АМО (описаны далее в этой же главе) как богатую платформу для разработки приложений бизнес-аналитики.

В примеры кодов этой главы включен проект надстройки Tools for Office Excel 2007 (с названием Chapter17AddIn), который демонстрирует программное создание PivotTable, диаграммы и формул ячеек для базы данных Chapter16. Надстройка вставляет новую вкладку **OLAP** в ленту Ribbon и создает панель задач **OLAP Tools**. Панель задач имеет выпадающие списки, которые перечисляют измерения и меры куба, а также атрибуты выбранного вами измерения. Это позволяет создавать формулы ячеек при помощи кода (для каждого выбранного вами элемента атрибута). Код может определить все элементы, а также заполнить выпадающие списки — для этого надо использовать ADO MD.NET (для чтения метаданных кубов) при помощи методов, которые мы обсудим далее в этой же главе.

Несмотря на то, что ограничения по имеющемуся в нашем распоряжении объему книги не позволяют нам обсудить способы программирования Visual Studio Tools for Office и код примера надстройки, мы все-таки советуем вам изучить этот код и компоновку ленты Ribbon на стадии проектирования. Вы можете запускать, использовать и отлаживать надстройку (выбрав проект надстройки в окне **Solution Explorer** в Visual Studio и затем либо нажав клавишу <F5>, либо выбрав **Debug | Start Debugging**, либо нажав кнопку **Start Debugging** панели инструментов).

## Средства Excel Services

Содержимое формул ячеек было создано на четвертой вкладке нашей электронной таблицы. В нем объединены таблица PivotTable, диаграмма и система показателей (еще одна таблица PivotTable), которые мы создали ранее. Эти четыре средства можно скомбинировать в очень удобное представление панели мониторинга. Сохраните ваши изменения в электронной таблице и давайте теперь используем Excel Services для построения такой панели.

### ПРИМЕЧАНИЕ

Службы Excel Services доступны для пользователей, имеющих лицензию Enterprise Client Access License (Enterprise CAL) для Microsoft Office SharePoint Server 2007. Если предположить, что вы имеете такую лицензию и со-

ответствующий доступ к серверу SharePoint со службами Excel Services, библиотеку подключений, библиотеку отчетов и все это правильно настроено — все описанное в данном разделе будет работать. Если вы не имеете лицензии SharePoint Server Enterprise CAL, все равно можете прочитать этот раздел (чтобы узнать о возможностях данного продукта). Понимая, что начиная с этого момента вы, вероятно, не сможете следить за каждым нашим шагом, мы обеспечили достаточное количество иллюстраций, чтобы показать вам, чего можно ожидать от должным образом настроенной среды SharePoint Server 2007.

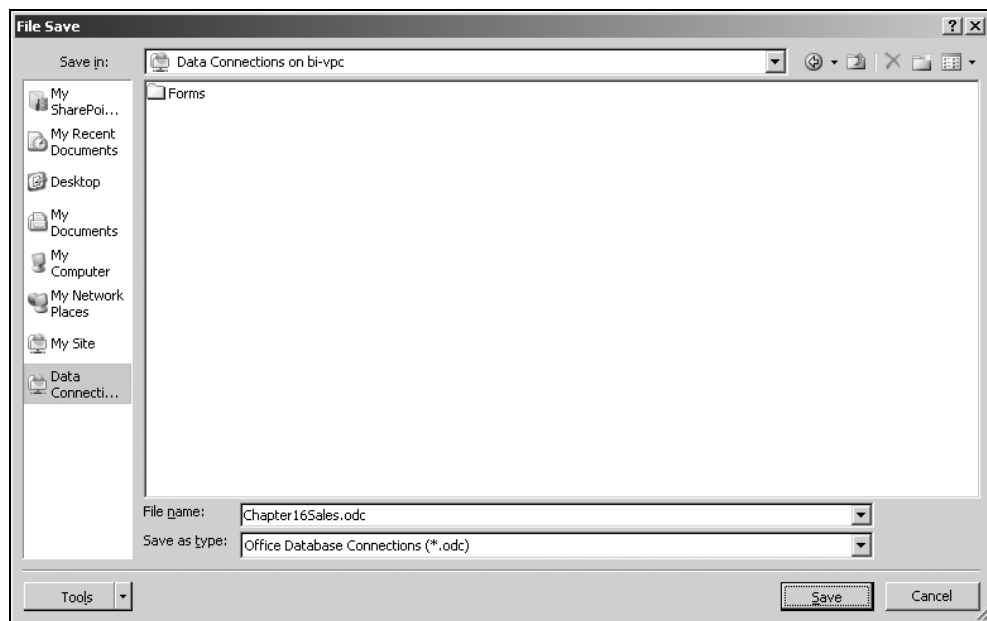
Как мы уже упоминали ранее, перед публикацией нашего контента на сервере SharePoint (при помощи Excel Services) мы должны перенести наше подключение к данным в библиотеку подключений SharePoint. Для этого есть несколько способов, самый очевидный из которых — это переход к библиотеке подключений в браузере и последующее использование пользовательского интерфейса SharePoint для загрузки odc-файла из нашего каталога My Data Sources. Проблема состоит в том, что в дополнение к загрузке копии файла подключения нам нужно также изменить и нашу электронную таблицу (чтобы она использовала загруженную копию). Excel дает нам способ выполнить оба эти шага одновременно.

Отредактируйте текущие свойства подключения электронной таблицы. Начните с вкладки **Data** ленты Ribbon, нажмите кнопку **Connections** в группе **Connections**, убедитесь, что выбрано подключение Chapter16Sales, а затем нажмите кнопку **Properties**. Это приведет к показу диалогового окна **Connection Properties**, где вы должны выбрать вкладку **Definition**, а затем щелкнуть по значку **Export Connection File** (внизу справа). Появится диалоговое окно **File Save**; здесь вы должны перейти в библиотеку подключения SharePoint, а затем сохранить в ней odc-файл (рис. 17.21).

После сохранения файла подключения в библиотеке подключений нам нужно настроить нашу электронную таблицу на использование именно этой копии (а не той, которая находится на нашем локальном диске). Когда мы после этого опубликуем содержимое электронной таблицы на сервере SharePoint, и пользователи станут просматривать данные, они уже будут ссылаться на хранящееся на сервере (и доступное им) подключение к данным.

Для преобразования ссылки на подключение в нашей электронной таблице нам нужно удалить локальную версию и добавить версию из библиотеки подключений. Если вы этого еще не сделали, нажмите **Save** — и вы вернетесь в диалоговое окно **Workbook Connections**. Убедитесь, что выбрана локальная версия Chapter16Sales, и нажмите кнопку **Remove**. Появится сообщение; нажмите кнопку **OK** (невзирая на его грозный текст). Теперь нажмите

кнопку **Add**, перейдите к библиотеке подключений и вашему файлу подключения, а затем нажмите **Open**. И наконец, нажмите кнопку **Close** в диалоговом окне **Workbook Connections** и сохраните ваши изменения.

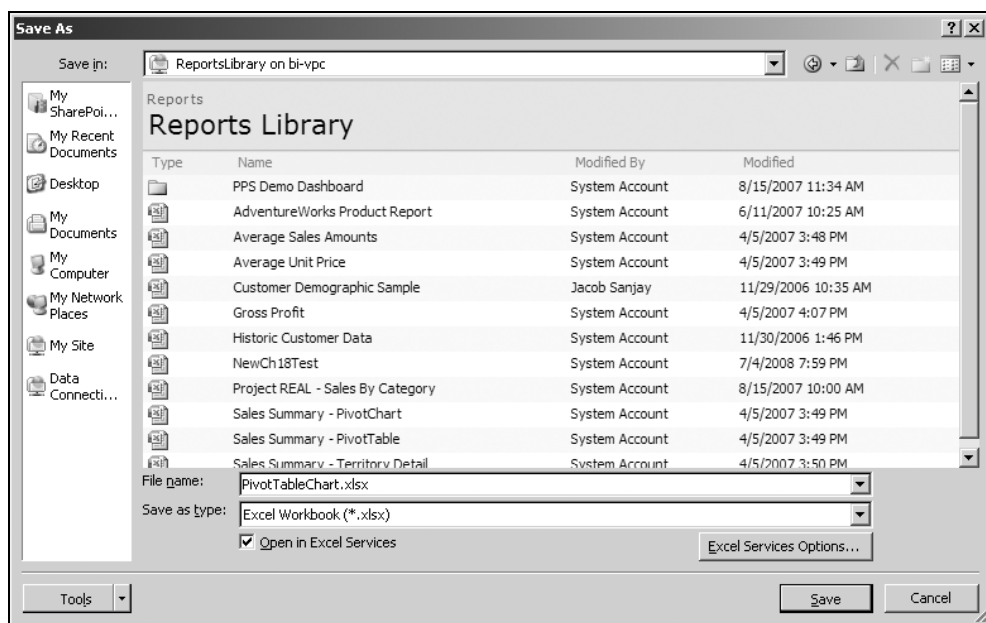


**Рис. 17.21.** Сохранение odc-файла подключения в библиотеке подключений SharePoint

Теперь, когда подключение электронной таблицы мигрировало в библиотеку подключений SharePoint, мы готовы публиковать наш контент в Excel Services. Начните с нажатия кнопки **Microsoft Office Button** (это круглый логотип Microsoft Office в левом верхнем углу окна Excel), а затем выберите в меню **Publish** пункт **Excel Services**. Это приведет к появлению такого окна, которое выглядит как простое диалоговое окно **Save As**. Однако присмотритесь — и вы заметите специальную кнопку **Excel Services Options** справа внизу, как раз над кнопкой **Save** (рис. 17.22).

Нажмите эту кнопку для показа диалогового окна **Excel Services Options**. В этом диалоговом окне мы указываем, что мы хотим опубликовать и как. Выпадающий список предлагает нам простые варианты для публикации: **Entire Workbook**, **Sheets** (то есть отдельные страницы электронной таблицы) или **Items In The Workbook**. Выберите последний вариант, а затем выберите

все наши объекты (кроме именованного диапазона `Calendar_Year`). Диалоговое окно теперь должно выглядеть так, как показано на рис. 17.23.



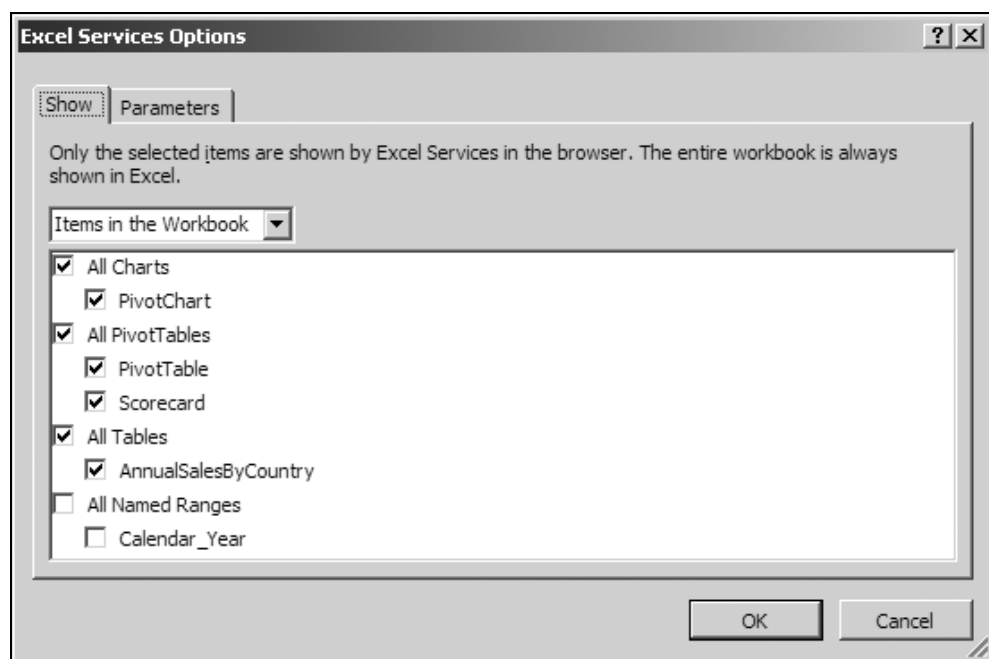
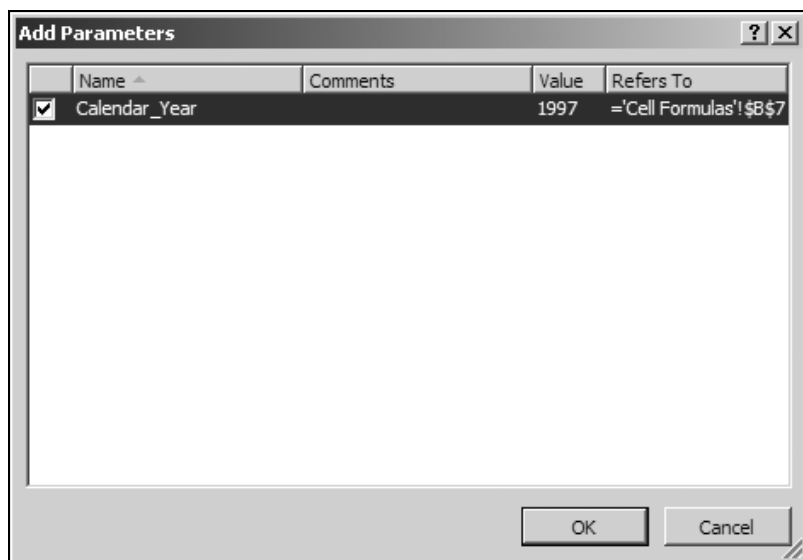
**Рис. 17.22.** Публикация в Excel Services через диалоговое окно **Save As** (при помощи кнопки **Excel Services Options**)

### ПРИМЕЧАНИЕ

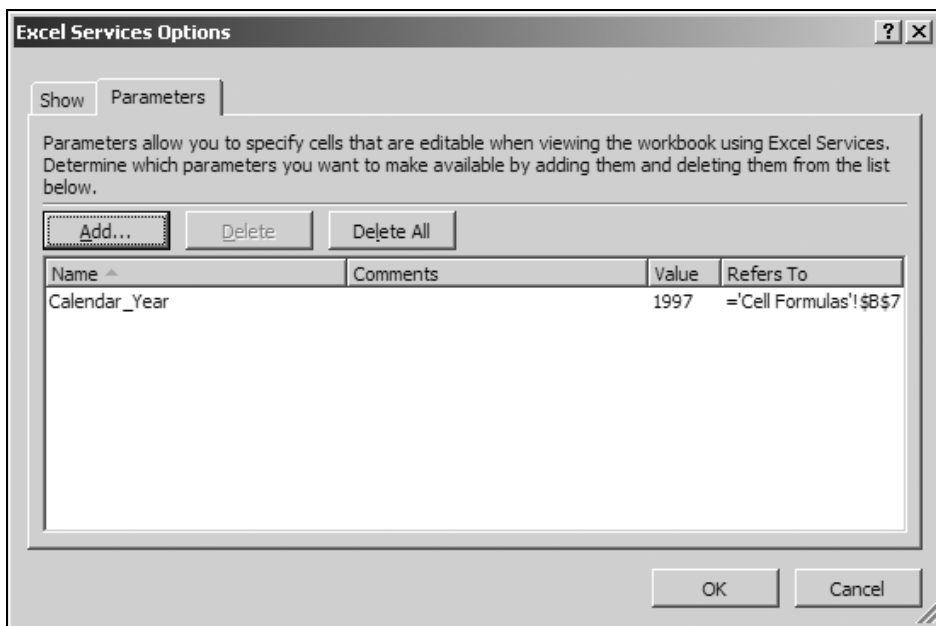
Названия объектов (которые вы видите в таблице и в PivotTable) могут отличаться — мы присвоили эти названия нашим объектам при помощи текстовых полей **PivotTable Name** и **Table Name** на вкладках **PivotTable Tools/Options** и **Table Tools/Design** ленты Ribbon.

Обратите внимание, что каждый из наших объектов показан как помечаемый флажком элемент: две наших таблицы PivotTable (одна из которых — наша система показателей), диаграмма, таблица (с содержимым формул ячеек) и именованный диапазон.

Мы выбрали все эти элементы (за исключением именованного диапазона). Диапазон будет использован, но в особом качестве — для параметризации данных нашей таблицы. Выберите вкладку **Parameters** диалогового окна, а затем нажмите его кнопку **Add**. Это приведет к показу диалогового окна **Add Parameters**, в котором вы должны увидеть наш именованный диапазон (рис. 17.24).

Рис. 17.23. Диалоговое окно **Excel Services Options**Рис. 17.24. Диалоговое окно **Add Parameters**

Установите этот флажок и нажмите кнопку **ОК**. Наш именованный диапазон теперь должен появиться на вкладке **Parameters** (рис. 17.25), которая позволит службам Excel Services отобразить специальную панель задач для запроса его значения.



**Рис. 17.25.** Вкладка **Parameters** диалогового окна **Excel Services Options**

Нажмите кнопку **ОК** для того, чтобы закрыть диалоговое окно **Excel Services Options**. В диалоговом окне **Save As** убедитесь, что установлен флажок **Open In Excel Services**, перейдите к нужной библиотеке отчетов и нажмите **Save**. Это приведет к сохранению электронной таблицы в библиотеке отчетов, после чего откроется некое подобие тестовой страницы браузера, где вы сможете просмотреть каждый из четырех только что опубликованных нами элементов. На рис. 17.26 показана эта страница с изображением нашей таблицы PivotTable.

Если мы перейдем в браузере к Report Library, то мы увидим в ней нашу электронную таблицу и сможем щелкнуть по ней в любой момент времени для показа той же самой тестовой страницы, которая появилась при публикации контента из Excel. Однако страница пока еще не готова для конечных пользователей. Для этой аудитории нам нужна специальная страница панели монито-



ринга (содержащая четыре экземпляра Excel Web Access Web Part). Опция **Dashboard Page** в меню **New** библиотеки Report Library сделает только полдела: она создаст новую страницу Web Part с двумя экземплярами Excel Web Access Web Part плюс несколько других страниц Web Part, которые нам не нужны.

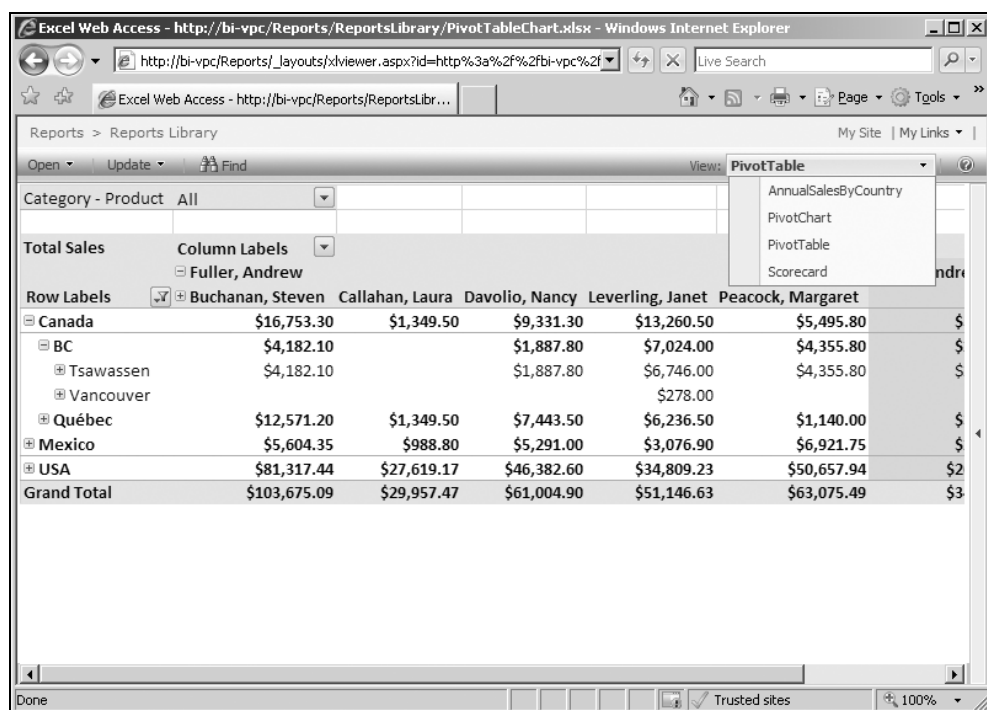


Рис. 17.26. Тестовая страница Excel Services

Необходимые шаги (для того чтобы добавить еще два экземпляра Web Part и сделать в итоговых четырех экземплярах указания на опубликованные нами четыре элемента) займут гораздо больше места, чем мы можем себе здесь позволить. Будьте уверены: если у вас есть базовые навыки работы с SharePoint, то эта процедура будет для вас относительно простой. (Если вы не очень хорошо владеете вставкой Web Part в SharePoint, вам будет полезна следующая ссылка: <http://office.microsoft.com/en-us/sharepointserver/HA011605831033.aspx>). Результат показан на рис. 17.27.

Обратите внимание, что содержимое PivotTable полностью интерактивно: узлы можно разворачивать и сворачивать, а данные можно обновлять в любой момент времени. Кнопки панели инструментов позволяют открыть в клиентском при-

ложении Excel 2007 либо моментальный снимок, либо редактируемую копию. Содержимое формул ячеек также интерактивно. По умолчанию появляется панель задач, в которой можно ввести значение `Calendar_Year`, а затем нажать кнопку **Apply** для обновления ячеек и отображения в них нового значения.

Комбинация возможностей бизнес-аналитики Excel 2007 и служб Excel Services дает исключительно всестороннюю платформу для создания презентационных уровней для наших кубов. Вы можете дополнить ее отчетами служб Reporting Services и другими продуктами компании Microsoft (такими как ProClarity и PerformancePoint). Поскольку все эти продукты показывают свой контент в SharePoint, то вы можете сочетать контент из этих приложений на одной странице панели мониторинга. Польза от такого подхода очень велика. В то же время и сами по себе Excel 2007 и Excel Services чрезвычайно ценны (как вы уже убедились).

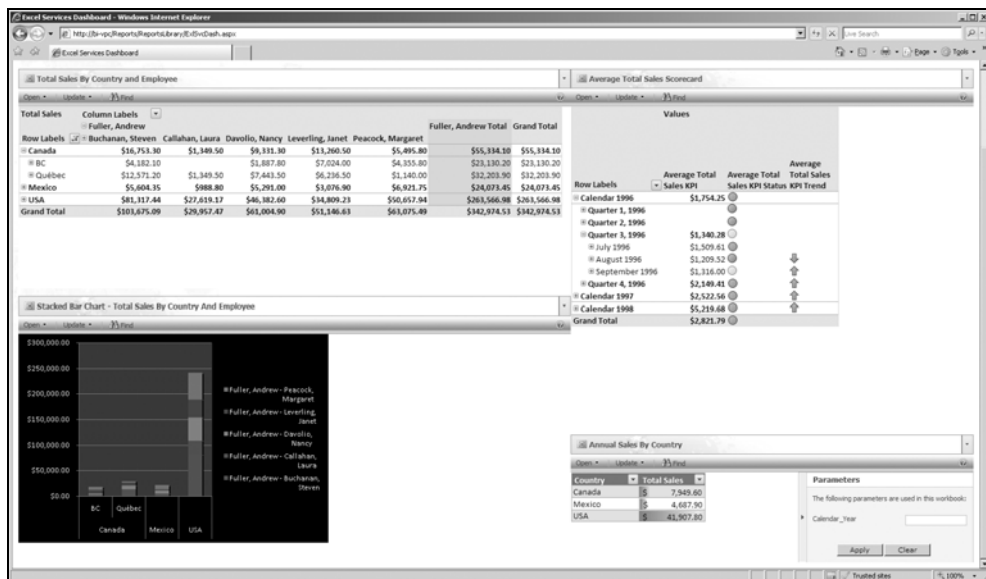


Рис. 17.27. Страница панели мониторинга Excel Services со всеми четырьмя опубликованными элементами

## Разработка OLAP при помощи .NET

Мы подробно описали использование Excel и Excel Services как мощных презентационных инструментов для ваших кубов. Эти инструменты с пользова-

тельским интерфейсом являются очень серьезными вариантами (если учесть вездесущность Microsoft Office и знакомство с ним бизнес-пользователей). Более того, применение Excel позволяет вам создавать впечатляющие пользовательские интерфейсы при минимальном использовании программирования.

Несмотря на то, что вариант с использованием Excel позволяет вам избежать необходимости программировать, он также исключает детальное управление (которое дает подход с программированием). Вариант с использованием Excel требует также наличия Microsoft Office 2007 на пользовательском компьютере (если вы хотите предложить полный набор функциональных возможностей настольного компьютера).

Если вам нужно детальное управление запросами, придется пойти дальше подхода Excel. А если вы .NET-разработчик и хотите использовать в контексте приложения OLAP свои знания по ADO.NET и привязке данных, то вам обязательно надо знать, как программировать непосредственно службы Analysis Services.

## **MDX и интерфейсы прикладного программирования Analysis Services**

Короче говоря: если вы хотите создать высокоспециализированное приложение OLAP, то вам совершенно определенно нужно изучить некоторые основы языка многомерных выражений MDX и понять, как выполнять MDX-запросы при помощи ADO MD.NET (многомерного поставщика данных ADO.NET). Скоро мы обсудим обе этих технологии и продемонстрируем их применение в приложениях Windows Forms.

При изложении MDX и ADO MD.NET мы будем объяснять, как использовать SQL Server Management Studio в качестве MDX-клиента. Мы будем изучать также выполнение запросов XML for Analysis (XMLA) из Management Studio. XMLA можно использовать в Management Studio и как средство языка манипулирования данными (data manipulation language, DML), и как средство языка определения данных (data definition language, DDL). ADO MD.NET работает для XMLA как оболочка вокруг DML. Другая управляемая объектная модель (которую мы будем описывать) — Analysis Management Objects (AMO) — работает для XMLA как оболочка вокруг DDL. Мы изучим программное использование XMLA в вашем коде, а также предоставление "необработанных" ответов XMLA из ADO.NET.

Для завершения нашего описания разработки OLAP мы покажем вам, как использовать возможности программирования CLR на серверной стороне

Analysis Services. При этом вы увидите, как ваши навыки разработки OLAP для клиентской стороны можно перенести на серверную сторону для того, чтобы разрабатывать сложную логику, которая расширяет способности MDX с клиентской стороны.

## Переходим к MDX

Программируете ли вы на клиенте или на сервере, используете ли вы ADO MD.NET или просто работаете с SQL Server Management Studio, вам необходимо знание как минимум основ MDX. Это исключительно богатый язык, сложность которого в этой главе мы можем только оценить. К счастью, использовать MDX для простых запросов действительно просто, а для тех, кто имеет опыт работы с SQL, этот процесс покажется знакомым. Вот прототип запроса MDX:

```
SELECT [column members] ON COLUMNS,  
       [row members] ON ROWS  
FROM [cubename]  
WHERE [measure]
```

Этот схематичный код демонстрирует подробности указания элементов строк и столбцов и слишком упрощает значение выражения WHERE (но с этими недостатками мы скоро справимся). Давайте начнем с реального запроса, который делает выборку из куба Sales нашей базы данных Chapter16 и показывает Total Sales с разбивкой по атрибуту Shipper Name измерения Shippers (по оси COLUMNS) и атрибуту Year измерения Time (по оси ROWS):

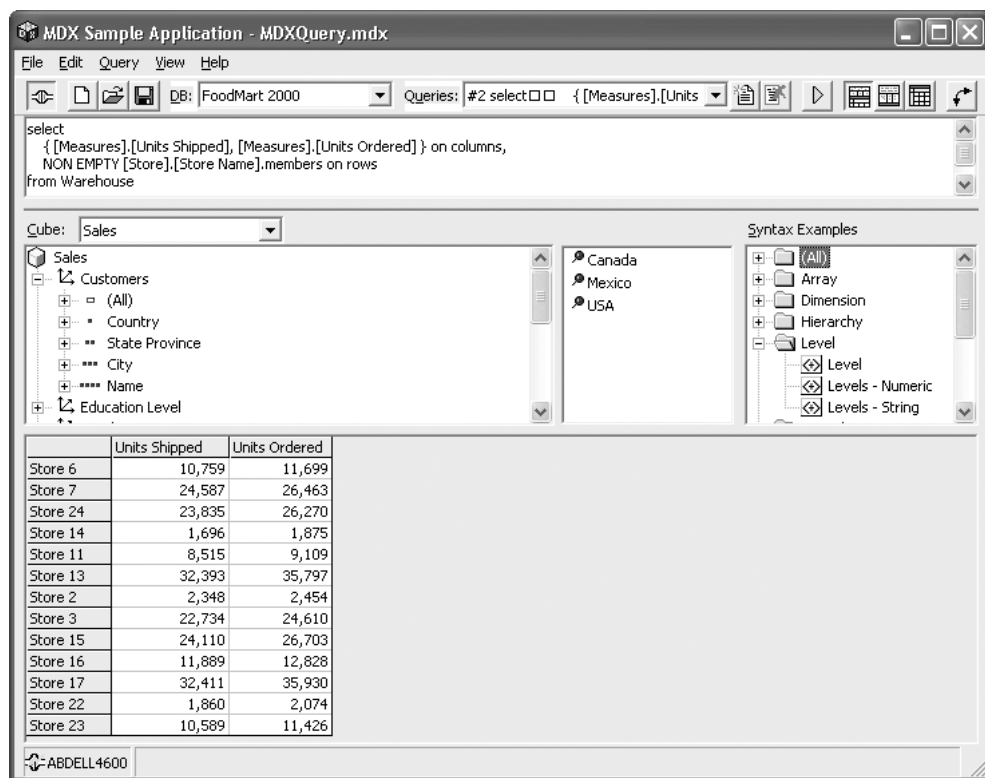
```
SELECT Shippers.[Shipper Name].MEMBERS ON COLUMNS,  
       Time.Year.MEMBERS ON ROWS  
FROM Sales  
WHERE Measures.[Total Sales]
```

Перед тем как продолжить, давайте обсудим использование SQL Server Management Studio как отличного инструмента для выполнения, проектирования и изучения запросов MDX.

## Management Studio как клиент MDX

В *главе 16* мы рассмотрели возможности Management Studio по управлению базами данных Analysis Services и дали краткий обзор возможностей ее окон запросов MDX и XMLA. Теперь мы изучим возможности окна MDX Query более подробно.

SQL Server 2005 ввел Management Studio как замену не только для SQL Enterprise Manager и SQL Query Analyzer, но и инструментов SQL Server 2000 Analysis Services: Analysis Manager и MDX Sample Application. Инструмент MDX Sample Application (рис. 17.28) позволяет выполнять произвольные запросы MDX, а также сохранять их в текстовые файлы (и извлекать оттуда), он содержит также простой пользовательский интерфейс, который позволяет визуальнo составлять эти запросы перетаскиванием. Это полезный инструмент, несмотря на то, что он совершенно обособлен от остальных инструментов SQL Server 2000 и предоставляется как пример приложения (с исходными кодами на языке Visual Basic 6.0), а не продукт промышленного уровня.



**Рис. 17.28.** Приложение MDX Sample Application (предоставляется вместе с SQL Server 2000 Analysis Services)

Как вы видели в предыдущей главе, Management Studio содержит функционал MDX Sample Application (окно **MDX Query**, которое работает практически идентично окну **SQL Query**). Функционал запросов MDX таким образом

интегрирован с остальным функционалом Management Studio, причем Management Studio даже предоставляет некоторые дополнительные функциональные возможности (в том числе технологию IntelliSense). Проекты скриптов Analysis Services (в которых могут содержаться запросы MDX) можно создавать в Management Studio, что позволяет создавать решения, которые состоят из проектов Analysis Services и реляционных проектов SQL Server (со скриптами для соответствующих кубов и таблиц).

### ВНИМАНИЕ!

Несмотря на то, что и MDX Sample Application, и Management Studio используют расширение файла mdx для запросов MDX, эти два формата несовместимы. Приложение MDX Sample Application хранит в одном файле несколько запросов и разделяет их кавычками. Management Studio хранит текст запроса MDX без всякого дополнительного форматирования, как текст SQL в файлах sql.

The screenshot shows the Microsoft SQL Server Management Studio interface. The main window displays an MDX query in the 'SimpleQuery.m...administrator' window. The query is as follows:

```

SELECT Shippers.[Shipper Name].MEMBERS ON COLUMNS,
Time.Year.MEMBERS ON ROWS
FROM Sales
WHERE Measures.[Total Sales]

```

The 'Results' tab is active, showing a table with the following data:

|               | All Shippers   | Federal Shipping | Speedy Express | United Package | Unknown |
|---------------|----------------|------------------|----------------|----------------|---------|
| All           | \$1,354,458.59 | \$407,750.82     | \$373,983.19   | \$572,724.58   | (null)  |
| Calendar 1996 | \$226,298.50   | \$87,133.00      | \$54,193.60    | \$84,971.90    | (null)  |
| Calendar 1997 | \$658,388.75   | \$210,542.35     | \$194,452.99   | \$253,393.41   | (null)  |
| Calendar 1998 | \$469,771.34   | \$110,075.47     | \$125,336.60   | \$234,359.27   | (null)  |

The status bar at the bottom indicates 'Query executed successfully.' and shows the connection details: (local) | SQL08DEV\Administrator | Chapter16 | 00:00:01.

**Рис. 17.29. Окно MDX Query**  
в SQL Server Management Studio (видна вкладка **Results**)

Примеры кодов данной главы включают решение SQL Server Management Studio, которое можно открыть через файл Chapter17\SSMS\Chapter17.ssmssln. Это решение содержит три проекта Analysis Services, один из которых Chapter17\SSMS\MDX Demos\1. MDX Demos.ssmasproj состоит из отдельных mdx-файлов для каждого представленного в этой главе запроса, включая и тот, который уже был обсужден. Откройте это решение и дважды щелкните по файлу запроса SimpleQuery.mdx, чтобы открыть его в Management Studio. Нажмите кнопку **Execute** в панели инструментов **SQL Server Analysis Services Editors** или используйте клавиатуру (главное меню либо пункт контекстного меню), чтобы увидеть результаты запроса MDX в сетке вкладки **Results** (рис. 17.29).

## Богатая поддержка MDX

Помимо того, что Management Studio является удобным текстовым редактором для написания и тестирования ваших запросов, она содержит также множество функций, которые облегчают создание запросов MDX для тех людей, кто с MDX не знаком. В *главе 16* мы кратко описали панель инструментов окна MDX Query и имеющиеся в Management Studio функциональные возможности IntelliSense. Давайте изучим набор функций MDX в Management Studio более подробно.

## Выполняем подключение

Для начала вам нужно понять разные способы открывания нового запроса. Мы описывали некоторые из них в *главе 16*, но включили их еще и сюда для повторения и для полноты изложения. Это пункт меню **File | New | Analysis Services MDX Query**, а также кнопка (третья слева) с этим же названием на панели инструментов **Standard**. Любой из этих вариантов открывает новое окно запроса **MDX Query**, но сначала выдает диалоговое окно **Connect To Analysis Services**, которое требует от вас подключиться (перед тем как поместить вас в редактор MDX). Во многом аналогичное окну **Connect To Database Engine**, это диалоговое окно требует от вас указания названия сервера или экземпляра и позволяет ввести необязательные свойства подключения (при помощи нажатия кнопки **Options** и ввода информации в появляющейся после этого вкладке **Connection Properties**).

Чтобы избежать диалогового окна **Connect To Analysis Services** и попасть непосредственно в окно запроса **MDX Query**, выберите сервер Analysis Services в окне **Registered Servers** или **Object Explorer**, а затем нажмите

кнопку **New Query** (первая слева) в панели инструментов **Standard**. Для просмотра серверов Analysis Services в окне **Registered Servers** вы должны нажать кнопку **Analysis Services** (вторая слева, со значком куба) в панели инструментов окна.

Другие способы быстро попасть в окно запроса MDX Query включают двойной щелчок по подключению Analysis Services в окне **Solution Explorer**, щелчок в нем правой кнопкой мыши по подключению и выбор пункта **New Query**, щелчок правой кнопкой мыши по узлу сервера Analysis Services в окне **Registered Servers** и выбор пункта **New Query** контекстного меню, щелчок правой кнопкой мыши по узлу сервера Analysis Services или базы данных в окне **Object Explorer** и выбор **New Query | MDX**. Когда вы используете последний из этих способов с узлом базы данных, то окно вашего запроса **MDX Query** выбирает эту базу данных по умолчанию. Нажатие кнопки **New Query** панели инструментов или выбор **File | New | Query With Current Connection** (при выбранном в окне **Object Explorer** узле базы данных) даст тот же результат.

В открытом окне MDX Query вы можете изменить информацию подключения посредством нажатия кнопки **Change Connection** (вторая слева) в панели инструментов SQL Server Analysis Services Editors или при помощи выбора пункта меню **Query | Connection | Change Connection** (либо при помощи выбора пункта контекстного меню **Connection | Change Connection**). Любой из этих способов откроет диалоговое окно **Connect To Analysis Services**. Если вы не подключились, то вы можете сделать это при помощи нажатия кнопки **Connect** (крайняя слева) в панели инструментов SQL Server Analysis Services Editors (или посредством выбора пункта меню **Query | Connection | Connect**, или при помощи пункта контекстного меню **Connection | Connect**).

После подключения вы можете использовать другую базу данных того же сервера; для этого необходимо выбрать эту базу данных в выпадающем списке **Available Databases** (справа от кнопки **Change Connection**) в панели инструментов SQL Server Analysis Services Editors. Вы можете специально перевести ваше окно MDX Query в неподключенное состояние при помощи выбора **Query | Connection | Disconnect** (или при помощи выбора пункта **Connection | Disconnect** контекстного меню окна MDX Query).

После открытия окна MDX Query и подключения к соответствующему серверу и базе данных панель **Tools** (область слева от зоны редактирования MDX) станет активной, ее выпадающий список имеющихся кубов или перспектив заполнится, и метаданные для выбранного куба/перспективы (и группы мер при ее наличии) будут показаны в древовидном представлении вкладки **Metadata**.



## Построение запроса

Помня все это, откроем новое окно **MDX Query** (убедившись, что вы подключены к кубу Sales базы данных Chapter16). В этом окне вы создадите новый запрос (похожий на предыдущий), но вместо ввода его заново для составления кода MDX мы будем использовать функции Management Studio.

Одним из отличных способов изучения MDX является использование шаблонов MDX, поставляемых вместе с Management Studio. Чтобы их увидеть, вы должны открыть окно **Template Explorer**. Для этого необходимо выбрать **View | Template Explorer** или нажать клавиши <Ctrl>+<Alt>+<T>. Затем нажмите кнопку **Analysis Services** (вторая слева или справа) в панели инструментов окна **Template Explorer** (чтобы показать шаблоны Analysis Services). Разверните узел **MDX**, а затем его дочерние узлы **Expressions** и **Queries** (рис. 17.30).

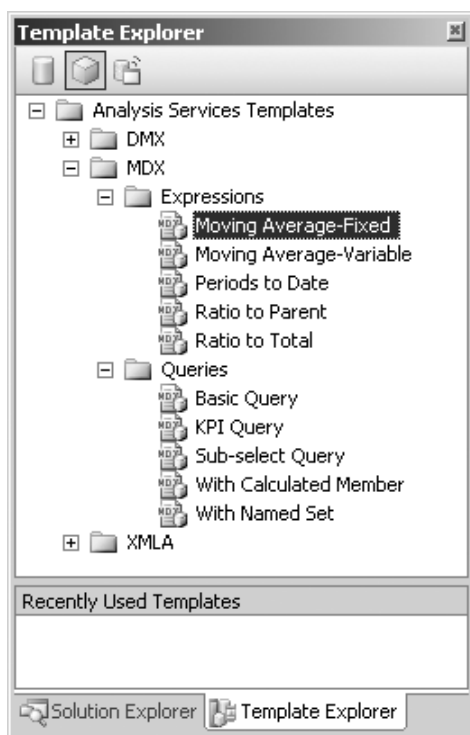


Рис. 17.30. Окно **SQL Server Management Studio Template Explorer** (шаблоны Analysis Services)

Всего SQL Server Management Studio содержит 10 шаблонов MDX, которые помогут вам изучить этот язык. Поскольку мы пишем простые запросы, то мы начнем с шаблона Basic Query. Перетащите этот узел **Basic Query** (он находится под родительским узлом **Queries**) в открытое окно MDX, а затем нажмите стрелку влево или <Home> на клавиатуре (чтобы убрать выделение текста шаблона MDX). Скелетный MDX похож на обсуждавшийся нами ранее схематический запрос MDX:

```
-- =====
-- Basic MDX Query
-- =====
Select <row_axis, mdx_set,> on Columns,
      <column_axis, mdx_set,> on Rows
From <from_clause, mdx_name,>
Where <where_clause, mdx_set,>
```

### **ВАЖНО**

Шаблон Basic Query (в Management Studio) ошибочно помещает местозаполнитель `row_axis` в выражение `on Columns` и местозаполнитель `column_axis` в выражение `on Rows`. Разумеется, все должно быть наоборот. Тем не менее, если вы внимательно будете следовать нашим указаниям, то сгенерированный вами запрос будет верным.

Наша следующая задача — заполнить параметризованные выражения (которые выглядят как текст в угловых скобках). Для того чтобы открыть диалоговое окно **Specify Values For Template Parameters** (рис. 17.31), выберите **Query | Specify Values For Template Parameters** (или нажмите кнопку **Specify Values For Template Parameters** (крайняя правая) в панели инструментов **SQL Server Analysis Services Editors**).

Это диалоговое окно обеспечивает единый пользовательский интерфейс для указания всей конкретики запроса, но оно предназначено для программистов, которые предпочитают вводить выражения без всяких подсказок. Мы же только знакомимся с MDX, поэтому мы будем создавать параметризованные выражения при помощи других средств. Пока же нажмите **Cancel** (чтобы закрыть это диалоговое окно).

Мы будем писать выражения `ON ROWS`, `ON COLUMNS`, `FROM` и `WHERE` по одному. Давайте начнем с `row_axis`: выделите выражение `<row_axis, mdx_set,>` целиком и нажмите кнопку **Delete**. Затем (в древовидном представлении метаданных) разверните измерение `Geography` и перетащите его дочерний узел атрибута `Country` в зону редактирования MDX (разместите его слева от `on`

Columns, оставив пробел справа). Нажмите стрелку вправо для сброса выделения перетянутого текста. Затем введите точку ., и запрос должен выглядеть теперь следующим образом:

```
Select [Geography].[Country]. on Columns,  
    <column_axis, mdx_set,> on Rows  
From <from_clause, mdx_name,>  
Where <where_clause, mdx_set,>
```

Поставьте курсор справа от введенной вами точки и выберите **Edit | IntelliSense | List Members** (или нажмите <Ctrl>+<J>). Должен появиться список технологии IntelliSense; введите members, и должно будет выделиться ключевое слово MEMBERS. Нажмите клавишу <Enter> или <Tab> для вставки этого слова в ваш код.

Повторите этот процесс и для выражения column\_axis, но на этот раз используйте элемент Beverages иерархии Category-Product измерения Products (углубитесь до узла Products\Category — Product\Members\All\Beverages и перетащите его в редактор MDX). После ввода точки (как и в предыдущем случае) нажмите вкладку **Functions** панели **Tools**, углубитесь до узла **Navigation\CHILDREN** и перетащите его правее точки в редакторе. После вставки текста удалите текст <<Member>>. (вместе с точкой) слева от ключевого слова CHILDREN.

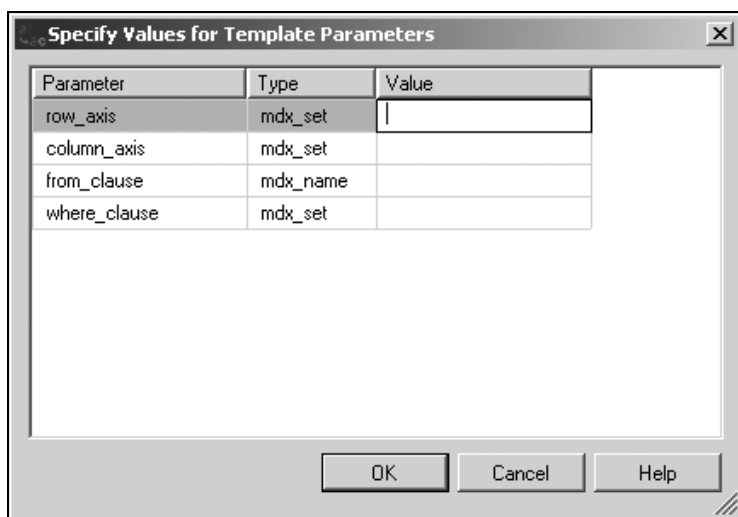


Рис. 17.31. Диалоговое окно **Specify Values For Template Parameters** в SQL Server Management Studio

Удалите из запроса текст `from_clause` (в скобках), а затем нажмите вкладку **Metadata** и перетащите узел `Sales` (корневой) в позицию курсора. Для завершения удалите параметр `where_clause` и перетащите узел **Measures\Main\Sum\Total Sales** на его место. Ваш запрос должен теперь выглядеть следующим образом (если это не так, то вы можете найти его в файле `Children.mdx`):

```
Select [Geography].[Country].MEMBERS on Columns,  
       [Products].[Category - Product].  
       [CategoryName].&[Beverages].CHILDREN on Rows  
From   [Sales]  
Where  [Measures].[Total Sales]
```

Выполните этот запрос. На рис. 17.32 частично показаны результаты, которые вы должны получить.

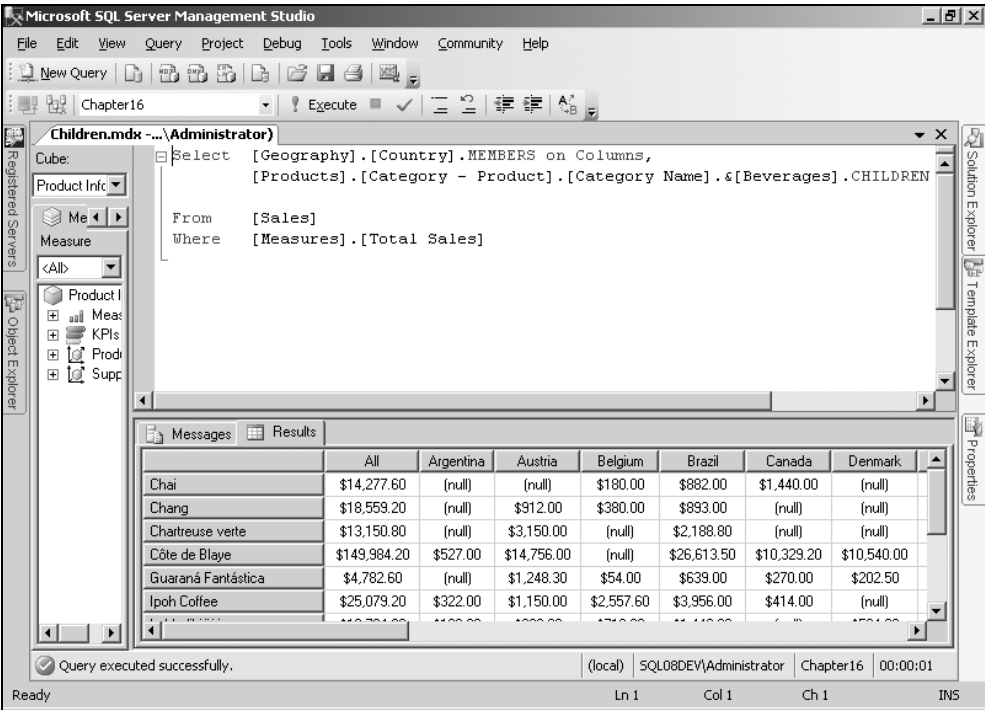


Рис. 17.32. Наш сконфигурированный запрос MDX (с результатами)

## Помимо синтаксиса

Несмотря на то, что до настоящего момента мы написали только два запроса, мы изложили уже очень много материала. Вы изучили основы синтаксиса MDX и использование окна **MDX Query**. Вы изучили также использование окон **Registered Servers** и **Object Explorer** с серверами Analysis Services (в том числе несколько сокращенных способов открытия окон **MDX Query** для конкретных подключений и баз данных). Во время рассмотрения окна **MDX Query** вы узнали, как для пошагового создания ваших запросов MDX можно использовать окно **Template Explorer**, технологию IntelliSense, а также вкладки **Metadata** и **Functions** в панели **Tools**. Однако несмотря на то, что мы познакомили вас с механизмами MDX и функциями MDX в Management Studio, мы не обсудили подробно сам язык. Давайте исправим это и сосредоточимся на некоторых важных концепциях MDX.

Начнем с нашего первого запроса. Обратите внимание, что оба выражения (`ON COLUMNS` и `ON ROWS`) используют синтаксис с точкой (в качестве разделителя) для указания измерения и атрибута. Название нашей меры `Total Sales` заключено в квадратные скобки (это требуется для любого названия, которое имеет внутренние пробелы). Названия без внутренних пробелов могут также заключаться в квадратные скобки (в этом запросе мы так не делали). Однако, как вы видели в предыдущей главе, использование квадратных скобок для названий без внутренних пробелов приветствуется.

Сразу же за названиями атрибутов в выражениях `ON COLUMNS` и `ON ROWS` мы использовали ключевое слово `MEMBERS` (это функция MDX), чтобы потребовать включения каждого элемента атрибута `Country` в виде столбца. Спецификаторы осей (то есть выражения перед ключевыми словами `ON ROWS` и `ON COLUMNS`) должны всегда указывать набор (то есть коллекцию элементов, принадлежащих одному атрибуту или уровням иерархии). Эти элементы должны быть либо явно перечислены, либо указаны при помощи функции MDX (такой как `MEMBERS`).

Элемент набора называется в языке MDX *кортежем*. Кортеж, формальный синтаксис которого мы скоро опишем, указывает элемент в каждом измерении куба. Однако практически кортеж не должен явно указывать элементов больше, чем для одного или нескольких измерений. Это потому, что любой кортеж неявно ссылается на элемент по умолчанию каждого измерения (когда конкретный элемент не указан), и в большинстве случаев — это элемент `[All]`. Поэтому даже если мы идентифицируем кортеж (например) только названием поставщика и ничем больше, то этот кортеж будет ссылаться на подмножество куба для указанного поставщика измерения `Shippers` и элемента `[All]` всех остальных измерений.

Теперь мы должны сказать вам, что данное теоретическое определение кортежа кажется осмысленным с первого взгляда лишь очень немногим. На данный момент вы должны просто принять его и отложить на потом — когда у вас будет больше опыта в OLAP. Практически вы можете представлять себе наборы состоящими просто из указанных элементов (а не из непонятно определенных кортежей). Для более непосредственной иллюстрации концепции набора мы даем здесь вариант нашего первого запроса, в котором мы используем спецификатор явного набора для оси COLUMNS:

```
SELECT { [Speedy Express], [Federal Shipping] } ON COLUMNS,  
       Time.Year.MEMBERS ON ROWS  
FROM   Sales  
WHERE  Measures.[Total Sales]
```

Этот запрос (содержащийся в файле `SelectedShippers.mdx`) позволяет нам запросить ограниченный специфический набор элементов атрибута `Shippers` (вместо отображения их всех). Каждый поставщик технически является кортежем, но пока вы можете думать о них как об элементах.

Как вы видите, явное указание однородных элементов может быть несколько утомительным, а требования синтаксиса очень строгие. Поэтому такие функции (как `MEMBERS`) могут быть очень полезными, они позволяют вам указать большие наборы без необходимости много вводить вручную. Однако разработчикам OLAP не приходится ограничиваться одной только функцией `MEMBERS`! Например, вы можете использовать ключевое слово `CHILDREN` языка MDX для перечисления всех дочерних элементов указанного узла иерархии. Чтобы понять, как это работает, посмотрите наш второй запрос (в файле `Children.mdx`). Ось `ROWS` показывает продукты категории `Beverages` (дочерние элементы для элемента `Beverages` уровня `Category Name` иерархии `Category-Product` измерения `Products`).

## Разбираемся с выражением **WHERE**

Теперь давайте более подробно обсудим выражение `WHERE` этого запроса. До настоящего времени мы использовали выражение `WHERE` как механизм для указания той меры, которую мы хотели видеть в нашем результирующем наборе. Это правильное и весьма обычное использование выражения `WHERE` в запросе MDX, однако оно заслоняет настоящую цель этого выражения: указать в запросе условия фильтрации для всех измерений куба.

При первом изучении это кажется неестественным сразу по нескольким причинам. Например, наше выражение `WHERE` указывало нужную меру, но не указывало никаких фильтров и ничего связанного с измерениями. Однако ока-

зывается, что с технической точки зрения наше выражение `WHERE` имеет фильтр по измерениям. Для того чтобы понять, почему это так, требуется понимание того, как MDX рассматривает меры и как использует элементы измерения по умолчанию.

Проще говоря, MDX считает ваш куб состоящим только из данных и измерений. Вместо рассмотрения мер как отдельных элементов вашего куба он считает каждую меру элементом плоской иерархии одного измерения (с названием `Measures`). По этой причине выражение `[Measures].[Total Sales]` в операторе `WHERE` является фильтром: фактически вы говорите ядру MDX, что надо ограничить запрос анализом только элемента `[Total Sales]` "измерения" `Measures`. Для всех остальных измерений куба фильтры не указаны, поэтому запрос MDX применяет элемент по умолчанию (`[All]`) и возвращает все содержимое всех реальных измерений.

Если вы хотите создать в вашем запросе более "реальный" фильтр, вы можете сделать это добавлением в выражение `WHERE`. Например, чтобы ограничить наш первый запрос продажами продуктов из категории `Condiments`, вам следует изменить предыдущий запрос таким образом (это есть в файле `TrueFilter.mdx`):

```
SELECT Shippers.[Shipper Name].MEMBERS ON COLUMNS,  
       Time.Year.MEMBERS ON ROWS  
FROM   Sales  
WHERE  (Measures.[Total Sales],  
        Products.[Category Name].Condiments)
```

Обратите внимание, что условие фильтрации по измерению `Products` объединено с существующим условием по "измерению" `Measures` (разделителем служит запятая), и все выражение `WHERE` заключено в скобки.

Скобки требуются для любого кортежа, который указан двумя явно названными элементами (или большим их количеством). Поскольку в данном случае мы указываем кортеж при помощи двух элементов `Measures.[Total Sales]` и `Products.[Category Name].Condiments`, то мы должны указать их в списке через запятую и заключить список в скобки. Кортеж нашего выражения `WHERE` является подмножеством куба, состоящим из данных `[Total Sales]` по продуктам из категории `Condiments` и по всем остальным измерениям куба. Остальная часть нашего запроса разбивает это подмножество куба по названию поставщика и календарному году.

### **ПРИМЕЧАНИЕ**

Важно помнить, что выражение `WHERE` предыдущего запроса (которое содержит простое выражение `Measures.[Total Sales]`) также является кор-

тежем. Даже несмотря на то, что оно казалось простым спецификатором элемента, его неявная ссылка на элемент [All] всех измерений куба квалифицирует его как выражение кортежа.

## Мощь осей

Давайте вернемся на шаг назад и обсудим концепцию осей. Рассматривавшиеся нами до настоящего времени запросы возвращали результаты по двум осям: ROWS и COLUMNS. Такие двумерные результирующие наборы удобны, и во многих случаях их вполне достаточно, но MDX может работать с большим количеством осей. Несмотря на то, что первые пять осей имеют названия, вы можете ссылаться на них также и по номеру.

Например, рассмотрим следующий запрос (вы можете найти его в файле SingleAxis.mdx), который возвращает данные по одной оси:

```
SELECT Geography.Country.MEMBERS ON 0
FROM Sales
WHERE Measures.[Total Sales]
```

Здесь мы указываем только ось столбцов (при помощи ее номера). Ось 0 — это ось COLUMNS, а ось 1 — это ось ROWS. Обратите внимание, что в последовательности осей не должно быть пропусков, то есть запрос с одной осью должен использовать ось COLUMNS, а не ось ROWS. Однако оси могут перечисляться и не по порядку, как в следующем запросе (он есть в файле AxesOutOfSequence.mdx), который возвращает данные по двум осям, но указывает ось ROWS до оси COLUMNS и не использует выражение WHERE:

```
SELECT Products.[Category — Product].[Category Name].Beverages.CHILDREN
ON 1, Measures.MEMBERS ON 0
FROM Sales
```

Запрос перечисляет элементы "измерения" Measures по оси COLUMNS, позволяя таким образом не использовать выражение WHERE (которое обычно указывает выбранную нами меру).

А что же делать с запросами с более чем двумя осями? MDX позволяет запросам возвращать теоретически неограниченное количество осей (хотя ресурсы клиента и сервера, несомненно, наложат некоторые ограничения). Вы можете дать названия остальным трем осям (помимо COLUMNS и ROWS). Имена осей после ROWS: PAGES, SECTIONS и CHAPTERS (именно в таком порядке). Они соответствуют номерам 2, 3 и 4 (COLUMNS и ROWS имеют соответственно номера 0 и 1). Названия этих трех осей показывают: несмотря на то, что трудно



представить себе данные более чем по двум осям, гораздо проще представить себе последовательность двумерных результирующих наборов, распределенных по нескольким страницам, разделам и главам книги.

Еще один способ представить оси — это представить себе электронную таблицу Excel. В Excel данные всегда показываются в одной электронной таблице (со столбцами и строками). Но одна таблица может содержать несколько вкладок (page), несколько таблиц могут содержаться в одном каталоге (section), а коллекция каталогов может содержаться в родительском каталоге (chapter). Вы можете без труда представить себе еще больше "осей" (сервер может управлять множеством жестких дисков, несколько серверов может принадлежать одной серверной ферме и т. д.). Таким образом, вы можете представить себе и написание запросов с бóльшим количеством осей, чем пять уже названных.

Понимая использование нескольких осей, можно написать следующий запрос (который имеется в файле ThreeAxes.mdx):

```
SELECT Shippers.[Shipper Name].MEMBERS ON COLUMNS,  
       Geography.Country.MEMBERS ON ROWS,  
       Time.Year.MEMBERS ON PAGES  
FROM   Sales  
WHERE  (Measures.[Total Sales], Products.[Category Name].Condiments)
```

При выполнении запрос возвращает объемы продаж для всех продуктов категории Condiments, разбитые по поставщикам и странам (по осям COLUMNS и ROWS), причем данные по каждому календарному году находятся на отдельной странице.

Однако если вы выполните этот запрос в SQL Server Management Studio, то вкладка **Results** будет пуста, и на вкладке **Messages** панели **Results** появится следующее сообщение:

```
Executing the query ...  
Obtained object of type: Microsoft.AnalysisServices.AdomdClient.CellSet  
Formatting.  
Results cannot be displayed for cellsets with more than two axes.  
Execution complete
```

Вторая строка этого сообщения означает, что запрос верный и выполнен без ошибок. Однако четвертая строка указывает, что запросы, которые возвращают данные более чем по двум осям, не могут отображаться в окне **MDX Query** в SQL Server Management Studio.

### ПРИМЕЧАНИЕ

Несмотря на то, что мы будем рассматривать запросы XMLA гораздо дальше (в этой же главе), вы можете открыть и выполнить запрос в файле `ThreeAxes.xmla`. Этот файл содержит тот же самый запрос, что и `ThreeAxes.mdx`, но он выполнится без ошибки, поскольку данные возвращаются как XML (а не в сеточном представлении). Если вы прокрутите до самого низа возвращаемого на вкладке **Results** кода XML, то вы увидите значения каждой ячейки по всем трем осям. Как читать и обрабатывать такой XML, вам станет ясно к концу данной главы.

## Функции **CROSSJOIN** и **DRILLDOWNMEMBER**

Как же нам обрабатывать запросы, которые возвращают три или более осей? Скоро мы изучим выполнение запросов MDX в .NET-коде и обсудим программные подходы к этой проблеме. Пока же вы можете переписать ваш запрос следующим образом:

```
SELECT Shippers.[Shipper Name].MEMBERS ON COLUMNS,  
       CROSSJOIN([Time].[Year].MEMBERS,  
                 Geography.Country.MEMBERS) ON ROWS  
FROM   Sales  
WHERE  (Measures.[Total Sales], Products.[Category Name].Condiments)
```

Запрос (который имеется в файле `CrossJoin.mdx`) использует функцию `CROSSJOIN` языка MDX для "выравнивания" элементов `Time.Year` и `Geography.Country` по оси `ROWS`. Это дает эффект создания результирующего набора, в котором каждый "заголовок" строки имеет два элемента: год и страну.

### ПРИМЕЧАНИЕ

В нашем запросе перекрестного объединения данные возвращаются по году, а затем по стране. Если бы порядок наборов функции `CROSSJOIN` был обратным, то данные возвращались бы сначала по стране, а потом по году.

Если вы просмотрите данные результата, то вы увидите, что вторая строка в разделе 1996 нашего результирующего набора (25-я строка с начала), где приведены данные объемов продаж по всем приправам, проданным в Аргентине в 1996 г., содержит одни только значения `NULL`. Это означает, что в Аргентине в 1996 г. никакие приправы не продавались; аналогичным же образом последний столбец (для элемента продуктов `Unknown`) имеет пустые ячейки во всех строках. MDX позволяет легко подавить эти пустые элементы осей (при помощи функций `NONEMPTYCROSSJOIN` и `NONEMPTY`). Чтобы увидеть,

как это работает, модифицируйте запрос следующим образом (эта версия имеется в файле `NonEmpty.mdx`):

```
SELECT NONEMPTY(Shippers.[Shipper Name].MEMBERS) ON COLUMNS,  
        NONEMPTYCROSSJOIN(Time.Year.MEMBERS, Geography.Country.MEMBERS)  
        ON ROWS  
FROM     Sales  
WHERE    (Measures.[Total Sales], Products.[Category Name].Condiments)
```

Когда вы выполните этот запрос, то заполненные значениями `NULL` столбцы и строки исчезнут, и вы поймете, что теперь результаты гораздо проще читать.

Мы не сможем описать весь всесторонний набор имеющихся в MDX ориентированных на иерархии функций, но можем дать представление о нем (показав другой запрос). Этот запрос возвращает данные для Канады и США:

```
SELECT NONEMPTY(Shippers.[Shipper Name].MEMBERS) ON COLUMNS,  
        NONEMPTYCROSSJOIN(Time.Year.MEMBERS,  
        DRILLDOWNMEMBER(Geography.[Country - State-Province - City -  
        Postal Code].Country.MEMBERS, {CANADA, USA})) ON ROWS  
FROM     Sales  
WHERE    (Measures.[Total Sales], Products.[Category Name].Condiments)
```

Обратите внимание на использование функции `DRILLDOWNMEMBER` для получения данных `[Total Sales]` для штатов и провинций Канады и США (в дополнение к данным стран). Функция `DRILLDOWNMEMBER` принимает два набора и возвращает объединение первого с дочерними объектами всех тех его элементов, которые содержатся во втором.

## KPI, вычисляемые элементы и именованные наборы

Мы почти закончили с теорией MDX, и теперь у вас есть хорошее представление о его возможностях (несмотря на то, что нам по-прежнему надо много объяснить о том, как выполнять запросы MDX программным путем). Однако все изученные нами до настоящего момента запросы используют только простые компоненты нашего куба — измерения и меры. Не будем забывать, что в предыдущей главе мы реализовали некоторые более сложные функции `Analysis Services`, включая определение вычисляемых элементов, именованные наборы и ключевые индикаторы производительности (KPI). Вопрос в том, как их использовать из MDX?

Давайте начнем с KPI. Вспомним, что KPI (когда он применен к срезу нашего куба) генерирует значение, цель, состояние и тенденцию (последние два обычно возвращают значения в диапазоне от -1 до +1 включительно). Вспомним, что для возвращения всех этих четырех значений существуют функции MDX. Следующий запрос (который имеется в файле скрипта `AverageTotalSalesKPI.mdx`) возвращает все четыре значения для созданного нами в предыдущей главе KPI (это `Average Total Sales KPI`). Он делает это для каждого календарного года, для которого куб содержит данные продаж по UK.

```
SELECT {KPIValue("Average Total Sales KPI"),
        KPIGoal("Average Total Sales KPI"),
        KPIStatus("Average Total Sales KPI"),
        KPITrend("Average Total Sales KPI")} ON COLUMNS,
        [Order Date].[Year - Quarter - Month - Date].Year.MEMBERS ON ROWS
FROM Sales
WHERE Geography.Country.UK
```

MDX не может сообщить нам о графических элементах для отображения состояния и тенденции, которые мы настроили в нашем KPI, но мы можем получить эту информацию из объекта `KPI` в ADO MD.NET, на котором мы остановимся далее в этой же главе.

Что же с вычисляемыми элементами и именованными наборами? Прелесть вычисляемых элементов состоит в том, что на них можно ссылаться из запроса MDX так, как будто они физически являются частью куба (а не вычисляются). Вы можете использовать именованные наборы как замену для явно указанных наборов (таких как мы изучали ранее в запросе `SelectedShippers.mdx`). Все это гораздо легче понять при рассмотрении в запросе, подобном следующему (его можно найти в файле `NamedSetCalcMember.mdx`):

```
SELECT Time.[Year - Quarter - Month - Date].Year.MEMBERS ON COLUMNS,
        [Test Markets] ON ROWS
FROM Sales
WHERE Measures.[Adjusted Sales]
```

Этот запрос возвращает значения для вычисляемой меры `[Adjusted Sales]` с разбивкой по календарным годам по оси `COLUMNS` и по трем городам нашего именованного набора `[Test Markets]` по оси `ROWS`.

Следующий запрос (который можно найти в файле `AdjustedGrowthByState.mdx`) демонстрирует, как запрашивать вычисляемую меру, формула которой основана на сравнении временных рядов. В этом случае мы запрашиваем нашу

вычисляемую меру [Adjusted Sales Growth], которая измеряет прирост продаж по годам в процентах. Эта вычисляемая мера требует использования измерения Order Date в одном из спецификаторов осей:

```
SELECT [Order Date].[Year].MEMBERS ON COLUMNS,  
       [Geography].[Country – State-Province – City – Postal Code].  
       [Country].USA.  
CHILDREN ON ROWS  
FROM Sales  
WHERE Measures.[Adjusted Sales Growth]
```

Этот запрос для всех годов (для которых куб содержит данные) дает цифры роста по оси COLUMNS с разбивкой по отдельным штатам в США по оси ROW.

Помните, что мы описали использование некоторых расширенных функций куба, в том числе KPI и вычислений (именованных наборов и вычисляемых элементов). Мы не рассматривали действия на основе детализации и на основе URL (несмотря на то, что они основаны на хранимых выражениях MDX). Мы опишем их в самом скором времени, необходимо иметь некоторые знания об интерфейсах прикладного программирования (API). Поэтому давайте на некоторое время отложим написание запросов и изучим их использование в наших приложениях.

## Разработка OLAP при помощи ADO MD.NET

Вспомним все, что мы описали в последних двух главах и в этой главе до настоящего момента. Вы научились проектировать куб с нуля и расширять его вычислениями, KPI, действиями и перспективами. Вы научились также запрашивать куб при помощи PivotTable, диаграмм и формул ячеек (как в самом Excel 2007, так и из SharePoint и Excel Services). И только что мы подробно описали, как использовать MDX для запроса ваших кубов из SQL Server Management Studio самыми разными способами, дающими исключительно точный контроль над возвращаемыми результатами.

Однако мы до сих пор не описали, как выполнять запросы MDX из ваших собственных приложений и отображать результаты. Важно знать, как выполнять запросы, которые возвращают данные по двум осям и быстро отображают результаты. Важно также понимать, как выполнять более сложные запросы (с тремя и более осями) и программно изучать их результаты, чтобы элементы третьей и последующих осей можно было перечислять в ваших собственных пользовательских интерфейсах.

С момента появления SQL Server Analysis Services 2005 этот вид программирования стал почти таким же простым, как программирование доступа к данным для реляционных баз данных. Это потому, что компания Microsoft создала управляемого поставщика данных ADO.NET (с названием ADO MD.NET) для использования с базами данных Analysis Services. Фактически ADO MD.NET был представлен в мае 2004 г. для SQL Server Analysis Services 2000. ADO MD.NET реализован как оболочка XMLA, собственного протокола Analysis Services, начиная с версии 2005.

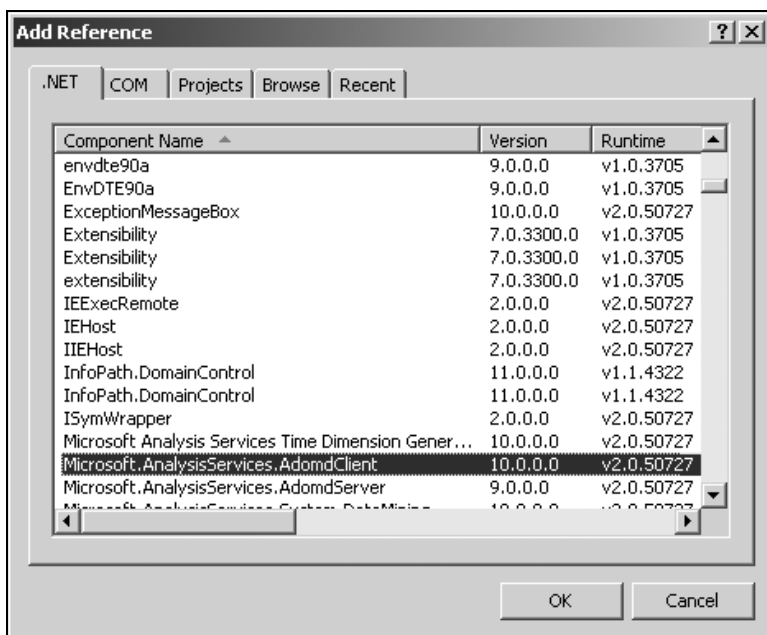
## Использование ваших навыков работы с ADO.NET

В ADO MD.NET замечательно то, что вы можете работать, как с любым другим поставщиком ADO.NET — при помощи объектов `Connection`, `Command`, `DataReader`, `DataAdapter`, `DataTable` и `DataSet` выполнять запросы MDX и обрабатывать результаты. Но вы можете также использовать широкий диапазон специфичных для OLAP объектов — для исследования метаданных куба и тщательного изучения результирующих наборов (в том числе и наборов с более чем двумя осями) — ось за осью и ячейка за ячейкой.

Эти специфичные для OLAP объекты являются по существу клонами объектов, имеющихся в старой (введенной в OLAP Services в SQL Server 7.0) объектной модели ADO MD. Система ADO MD.NET является "сплавом" ADO MD и ADO.NET и основана на XMLA.

Для использования ADO MD.NET в ваших приложениях просто сделайте ссылку на управляемую сборку `Microsoft.AnalysisServices.AdomdClient`, которая имеется в списке на вкладке **.NET** диалогового окна **Add Reference** в Visual Studio. Если вы работаете на персональном компьютере, который имеет более старых поставщиков ADO MD.NET для SQL Server 2000 или Analysis Services 2005, то обязательно выберите новую версию (версия 10.0, рис. 17.33). После настройки ссылки используйте в вашем коде пространство имен с этим же названием.

Теперь у вас есть прямой доступ ко всем объектам библиотеки ADO MD.NET, включая `AdomdConnection`, `AdomdCommand` и `AdomdDataAdapter`. Строки подключения для `AdomdConnection` требуют спецификаторов `Data Source` и `Initial Catalog`; свойство `CommandText` команды `AdomdCommand` может быть после этого настроено в любой запрос MDX для возвращения `AdomdDataReader` (`DataTable` или `DataSet` — при помощи метода `AdomdDataAdapter.Fill`).



**Рис. 17.33.** Диалоговое окно **Add Reference** в Visual Studio (активна вкладка **.NET**)

Помимо стандартной функциональности ADO.NET вы можете также использовать специальный метод `AdomdCommand.ExecuteCellSet` для возврата специфичного для OLAP результирующего набора с названием `CellSet`. Несмотря на то, что с объектами `CellSet` работать не так удобно, как с `DataReader`, `DataTable` и `DataSet`, ими лучше манипулировать программно, особенно когда возвращается больше двух осей.

В дополнение к объектам `CellSet`, ADO MD.NET предлагает и другие уникальные объекты, которые облегчают программирование OLAP. ADO MD.NET облегчает создание .NET-кода, который подключается к кубам OLAP, а затем исследует их метаданные, выполняет запросы MDX и отображает результаты. Теперь мы покажем вам, как использовать ADO MD.NET для написания кода, который запрашивает и данные, и метаданные куба.

## Выполнение запросов MDX

Давайте начнем с данных. Следующий код подключается к базе данных `Chapter16` хоста `Analysis Services` (в данном случае это `localhost`), выполняет

простой запрос MDX по кубу Sales, а потом привязывает результаты к элементу управления DataGridView с названием dgvResults:

```
AdomdConnection conn = new AdomdConnection("DataSource=localhost;Initial  
    " + "Catalog=Chapter16");  
AdomdDataAdapter da = new AdomdDataAdapter("SELECT Shippers.[Shipper  
    Name].MEMBERS ON " + "COLUMNS, Time.Year.MEMBERS ON ROWS FROM Sales  
    WHERE Measures.[Total Sales]", conn);  
DataTable dt = new DataTable();  
da.Fill(dt);  
dgvResults.DataSource = dt;
```

Аналогичный этому код можно найти в функции ExcelMDX в форме frmAPIs проекта Windows Forms (с названием Chapter17). Обратите внимание, что предыдущий код будет работать только в том случае, если в файле класса была размещена следующая директива using:

```
using Microsoft.AnalysisServices.AdomdClient;
```

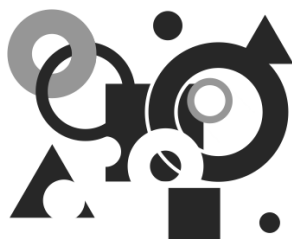
Если вы не считали, то мы вам подскажем: мы подключились, сделали запрос и отображали результаты всего пятью строками кода (не считая директивы using). Конечно, мы воспользовались способностью объекта AdomdDataAdapter принять в своем конструкторе командную строку MDX и таким образом создать за нас объект AdomdCommand. Но это не меняет главного: написание кода OLAP для .NET не обязательно должно быть трудным.

## Резюме

В данной главе изложен обширный материал. Вы научились использовать PivotTable, диаграммы и формулы ячеек из Excel и Excel Services для создания впечатляющих панелей мониторинга для ваших кубов OLAP. Изучив небольшое подмножество языка запросов MDX, вы научились использовать окно **MDX Query** в SQL Server Management Studio для выполнения запросов к вашим кубам (более точных, чем это возможно при помощи Excel).

Платформа OLAP компании Microsoft не только мощная и богатая функциональными возможностями, она также отлично программируется. Этот дружелюбный к разработчикам подход является отличительной чертой технологий компании Microsoft и является важной частью того, что сделало Analysis Services доминирующим продуктом на рынке бизнес-аналитики.





# Расширение бизнес-аналитики при помощи интеллектуального анализа данных

*Элси Пен*

Компонент интеллектуального анализа данных служб Analysis Services в Microsoft SQL Server позволяет разработчикам (даже тем, кто не имеет опыта в области интеллектуального анализа данных) быстро и эффективно интегрировать интеллектуальный анализ данных в приложения. Мастер Mining Model Wizard проведет вас по всем вариантам выбора входов и выходов для моделей, в значительной степени абстрагируя сложность самих алгоритмов. Предоставляются также некоторые параметры алгоритмов, что позволяет более опытным пользователям подстраивать модели (изменяя значения параметров).

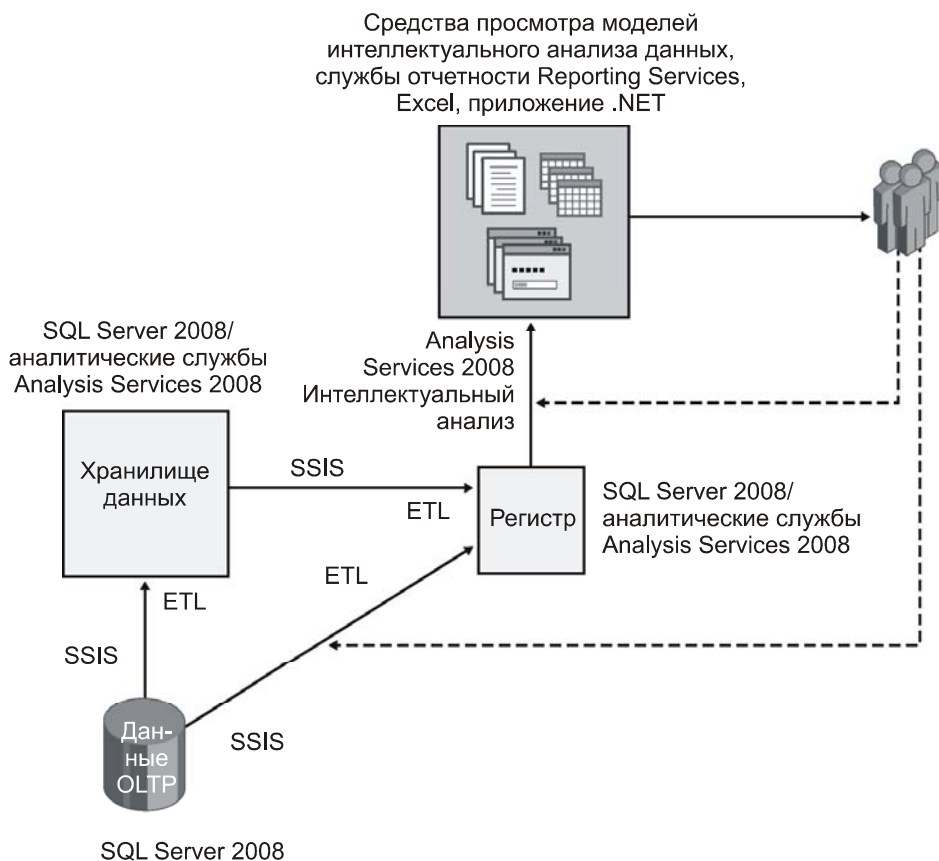
### **ПРИМЕЧАНИЕ**

Некоторые из упоминаемых здесь терминов могут оказаться незнакомыми для вас. Они будут объяснены в данной главе.

## Зачем делать интеллектуальный анализ данных

Возможности бизнеса по сбору и сохранению данных в течение последних 30 лет возросли экспоненциально. Организации встраивают интеллектуаль-

ный анализ данных в свои стратегии анализа бизнеса для обнаружения важных шаблонов в огромных (и обычно растущих) массивах имеющихся данных. Например, можно выявить часы пиковых звонков и соответствующим образом планировать рабочее расписание персонала. Компании розничных продаж могут лучше понять, какие продукты хорошо продаются в разных географических регионах, и оптимизировать сеть поставщиков. Компании продуктов питания выявляют, какие продукты могут испортиться прежде, чем будут использованы, и договариваются о более поздних их поставках. Клинические медицинские данные можно проанализировать для того, чтобы выявить демографические факторы (возраст, пол, этническая принадлежность), которые можно связать с побочными эффектами лечения.



**Рис. 18.1.** Развитая инфраструктура интеллектуального анализа данных с использованием инструментов бизнес-аналитики компании Microsoft

Короче говоря, интеллектуальный анализ данных становится ключевым шагом на пути от бизнес-данных к бизнес-знаниям, чтобы действовать в соответствии с данными, а не просто обслуживать их. На рис. 18.1 показаны компоненты развитой инфраструктуры интеллектуального анализа данных и элементы бизнес-аналитики компании Microsoft, которые реализуют каждый из этих компонентов.

Давайте немного обсудим эти компоненты. Большинство процессов интеллектуального анализа начинается с необработанных данных, чаще всего это данные обычной реляционной системы оперативной обработки транзакций (OLTP). Используемые (для проектов интеллектуального анализа) данные не собирались специально для такого анализа, они были получены при обычном ведении бизнеса.

После этого наступает очередь трудной задачи очистки и преобразования данных в пригодное для анализа состояние (extract, transform, and load, ETL) из множества систем в единое хранилище данных. Затем вы создаете управляемую задачами (или контентом) витрину данных для конкретного инструмента интеллектуального анализа данных. Это те данные, которые будут анализироваться. На рис. 18.1 показаны два возможных маршрута от ваших данных OLTP к витрине данных: путь через хранилище данных и прямой путь (при котором вы избегаете создания хранилища данных). В любом случае службы интеграции SQL Server Integration Services приведут вас к цели. Службы интеграции предлагают полный набор инструментов для извлечения данных из систем-источников, манипулирования данными для приведения их в пригодный для интеллектуального анализа формат, а также для сохранения данных в хранилище или витрине данных.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Хранилища данных обсуждаются в *главе 14*; дополнительно о службах интеграции вы можете узнать по адресу: <http://msdn.microsoft.com/en-us/library/ms141026.aspx>.

При помощи аналитических служб Analysis Services вы можете изучать и анализировать данные. Результаты интеллектуального анализа данных представляются в виде отчетов и диаграмм. Бизнес-пользователи могут получить из анализа новые знания.

На рисунке видны две цепи обратной связи. Цепь к шагу ETL представляет собой использование интеллектуального анализа для проверки данных. Выявляя типичные шаблоны, интеллектуальный анализ помогает находить в

данных аномалии (это может быть частью шага ETL). Это в свою очередь улучшает качество основанного на этих данных анализа. Обратная связь к шагу интеллектуального анализа данных показывает итеративный процесс. Результаты модели интеллектуального анализа могут использоваться для уточнения выборки (содержащейся в витрине данных) или подстройки параметров оценки модели.

Что важнее всего, интеллектуальный анализ данных — это управляемая бизнесом задача. Он обычно определяется как процесс нахождения полезных связей и шаблонов в больших наборах данных. Связи и шаблоны полезны только в такой степени, в которой они помогают решить бизнес-проблему. Вот некоторые задачи, которые можно решить при помощи интеллектуального анализа данных:

- ☐ выдавать рекомендации по продуктам и перекрестным продажам;
- ☐ проводить целевые почтовые рассылки;
- ☐ выявлять мошенничество;
- ☐ создавать профили клиентов;
- ☐ прогнозировать, а также проверять данные.

Бизнес-задача определяет используемый алгоритм интеллектуального анализа данных. На рис. 18.2 показаны некоторые обычные бизнес-задачи и те алгоритмы анализа данных в службах Analysis Services, которые лучше всего подходят для выполнения каждой задачи.

В своей книге "Бизнес со скоростью мысли" Билл Гейтс пишет: "Самой большой ценностью интеллектуального анализа данных является его способность помочь компаниям определить нужные продукты и правильный способ их ценообразования". Те компании, которые смогут определить "нужные продукты" и "правильную цену", смогут лучше других определить своих наиболее вероятных клиентов, а также понять покупательские привычки этих клиентов. Мощь аналитических служб состоит в том, что они дают легкий доступ к такой сложной функциональности интеллектуального анализа данных. Тесная интеграция компонентов Analysis Services с более традиционными компонентами базы данных Microsoft SQL Server 2008 и широко распространенным пакетом Microsoft Office делает интеллектуальный анализ данных доступным в виде естественного расширения любого решения бизнес-аналитики.

|                                      | Перекрестные<br>продажи | Почтовые<br>рассылки | Обнаружение<br>мошенничества | Профилирование<br>клиентов | Прогнозирование | Проверка данных |
|--------------------------------------|-------------------------|----------------------|------------------------------|----------------------------|-----------------|-----------------|
| Правила взаимосвязей                 | X                       |                      |                              |                            |                 |                 |
| Кластеризация                        |                         | X                    | X                            | X                          |                 | X               |
| Деревья решений                      |                         | X                    |                              | X                          |                 |                 |
| Линейная регрессия                   |                         |                      |                              | X                          |                 |                 |
| Логистическая регрессия              |                         |                      |                              | X                          |                 |                 |
| Упрощенный алгоритм Байеса           |                         | X                    |                              | X                          |                 |                 |
| Нейронные сети                       |                         |                      |                              | X                          |                 |                 |
| Кластеризация<br>последовательностей |                         |                      |                              | X                          |                 |                 |
| Временные ряды                       |                         |                      |                              |                            | X               |                 |

**Рис. 18.2.** Матрица алгоритмов  
интеллектуальных данных обычных бизнес-проблем

## Вспоминаем статистику

Не подразумевая глубоких знаний статистики, мы приведем определения для некоторых основных (используемых в этой главе) статистических терминов:

- ☐ среднее — сумма наблюдаемых значений, разделенная на количество наблюдений;
- ☐ медиана — половина чисел набора имеет значение больше медианы, а другая половина меньше;
- ☐ стандартное отклонение — мера рассеивания данных. Стандартное отклонение измеряет разброс значений данных от среднего значения;
- ☐ диапазон — разница между минимальным и максимальным значениями в наборе данных;

- распределение — массив наблюдаемых значений определенного атрибута ваших данных. Распределения данных можно изобразить с помощью гистограмм или секторных диаграмм;
- непрерывные данные — данные, набор возможных значений которых не может быть перечислен. Примеры — рост и возраст, чем точнее вы их измеряете, тем больше существует возможных значений. Например, мы становимся старше буквально каждую секунду;
- дискретные данные — данные, значениями которых являются изолированные точки, или те данные, возможные значения которых можно перечислить. Примеры — страна проживания или семейное положение. Непрерывные данные можно "дискретизировать", сгруппировав возможные значения в наборы. Например, возраст непрерывен, но может быть дискретизирован путем создания таких группировок, как "младше 21 года", "возраст от 21 до 50" и "больше 50".

## Расширения интеллектуального анализа данных в SQL Server 2008

SQL Server 2008 лучше своего предшественника по простоте использования, богатству функциональных возможностей, совершенству клиентских инструментов, а также по программируемости. Некоторые обычные задачи интеллектуального анализа данных теперь встроены в компонент интеллектуального анализа данных SQL Server 2008: создание контрольных наборов данных, обучение и тестирование моделей интеллектуального анализа на подмножествах данных, а также перекрестная проверка моделей. В то время как в SQL Server 2005 создание контрольных наборов данных должно было выполняться вне инструмента интеллектуального анализа данных, в SQL Server 2008 создание контрольного набора встроено в мастер Data Mining Wizard. После этого контрольный набор данных доступен при тестировании и сравнении точности моделей интеллектуального анализа в данной структуре. Если модель интеллектуального анализа данных обучена на специфическом подмножестве данных, то тот же самый фильтр можно применить и к контрольному набору (когда нужно оценить работу модели). К существующим вкладкам **Lift Chart** и **Classification Matrix** визуального конструктора Data Mining Designer добавилась вкладка **Cross Validation**. Эта перекрестная проверка моделей интеллектуального анализа может теперь выполняться при

помощи указания параметров на вкладке **Cross Validation** и последующего нажатия кнопки получения результатов.

Функциональность детализации дает большую гибкость при изучении характеристик вариантов для обучения и тестирования. В то время как в SQL Server 2005 при углублении в детали варианта были доступны только столбцы, включенные в модель интеллектуального анализа, в SQL Server 2008 при детализации может быть показан любой столбец структуры. Полезно увидеть дополнительную информацию о детализации, которая не относится к оценке модели интеллектуального анализа. Например, трудно представить себе добавление в модели анализа такую информацию, как имя или идентификатор; однако она весьма желательна при углублении в детали варианта.

Используемый в SQL Server Analysis Services 2008 алгоритм временных рядов был улучшен. Самое примечательное то, что сейчас можно обучать модели при помощи алгоритма Autoregressive Integrated Moving Average (ARIMA, авторегрессионное интегрированное скользящее среднее). Раньше в алгоритме временных рядов Time Series компании Microsoft был реализован только алгоритм Autoregressive Tree with Cross Prediction (ARTxp, авторегрессионное дерево с перекрестным прогнозированием). По умолчанию Analysis Services использует для оценки взвешенный подход, можно также установить нулевой вес для одного из алгоритмов, чтобы использовался только другой. Это дает гибкость прогнозов (краткосрочных или долгосрочных).

Надстройки интеллектуального анализа для Microsoft Office Excel 2007 были также обновлены, чтобы учесть новые функциональные возможности Analysis Services 2008. И что еще лучше, теперь можно документировать структуры интеллектуального анализа при помощи клиентской надстройки Data Mining Client для Excel.

Мы будем выделять эти новые функции при их появлении в последующих разделах.

## Начало построения

Теперь, когда вы загорелись идеей интеллектуального анализа ваших данных, с чего же вам начать построение моделей интеллектуального анализа и использование их в ваших приложениях? Эта глава использует пример анализа доходности клиентов с применением базы данных AdventureWorksDW2008 хранилища данных Adventure Works. Мы создадим новую базу данных

Analysis Services с моделями интеллектуального анализа, основанными на этих данных, а затем используем скрипты, которые расширяют модель и запрашивают ее содержимое.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Образцы базы данных в состав SQL Server 2008 больше не включаются. Их можно скачать с сайта CodePlex по адресу: <http://www.CodePlex.com/SqlServerSamples>. Во введении к этой книге даны указания по скачиванию базы данных хранилища Adventure Works, используемой в данной главе.

На базе моделей и скриптов мы создадим приложения Microsoft Windows Forms и Microsoft ASP.NET, которые имеют аналогичную (в смысле запросов) функциональность. Мы также загрузим на сервер управляемую сборку, которая реализует хранимую процедуру запроса интеллектуального анализа данных, и выполним запросы при помощи XML for Analysis (XMLA). И наконец, мы перейдем к надстройкам интеллектуального анализа данных для Excel 2007. Давайте же начнем с обзора тех шагов, которые необходимы для создания проекта интеллектуального анализа данных.

## **Подготовка исходных данных**

База данных AdventureWorksDW2008 содержит подмножество данных из нормализованной базы данных оперативной обработки транзакций AdventureWorks2008. Как вы уже знаете, Adventure Works — это вымышленная компания по производству велосипедов, которая продает их как оптовым клиентам, так и индивидуальным заказчикам через Интернет. По индивидуальным клиентам имеются подробные демографические данные, в том числе: доход, семейное положение, количество детей, состояние домовладения. Кроме того, поскольку хранилище данных создано из реляционной базы (содержащей данные из транзакционной системы заказов), оно содержит также подробные данные по поведенческому поведению клиентов. В эти данные входят: наличие недвижимости, количество купленных продуктов, "давность" последней покупки, типы купленных продуктов и т. д. Мы хотим использовать эти данные для того, чтобы получить подробную информацию о клиентах.

Сначала мы выполним показанный в листинге 18.1 запрос, использующий функцию ранжирования NTILE, которая появилась в SQL Server 2005.



## ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Функцию NTILE и другие функции ранжирования мы описывали в главе 2.

### Листинг 18.1. Использование функции NTILE для анализа данных

```
USE AdventureWorksDW2008
GO
SELECT
    count(*) AS NumCustomers
    ,Quartile
    ,min(TotalProfit) AS MinQuartileProfit
    ,max(TotalProfit) AS MaxQuartileProfit
    ,sum(TotalProfit) AS SumQuartileProfit
FROM (SELECT
    Customerkey
    ,ntile(4) OVER(ORDER BY TotalProfit DESC) AS Quartile
    ,TotalProfit
    FROM (SELECT
        Customerkey
        ,sum(SalesAmount) -sum(TotalProductCost) AS TotalProfit
        FROM FactInternetSales
        GROUP BY customerkey) AS c) AS q
GROUP BY Quartile
ORDER BY Quartile
COMPUTE sum(sum(TotalProfit))
GO
```

Запрос возвращает следующие результаты:

| NumCustomers | Quartile | MinQuartileProfit | MaxQuartileProfit | SumQuartileProfit |
|--------------|----------|-------------------|-------------------|-------------------|
| 4621         | 1        | 1077.4324         | 5273.81           | 9149434.8261      |
| 4621         | 2        | 156.2055          | 1077.4324         | 2608879.4859      |
| 4621         | 3        | 26.4672           | 156.1322          | 251766.6393       |
| 4621         | 4        | 1.4335            | 26.4672           | 70802.6937        |
| sum          |          |                   |                   | 12080883.645      |

Вы видите, что 25% клиентов (Quartile 1) генерируют 75% дохода (результат деления 9 149 435 на 12 080 884 равен 0,757). Кто же эти клиенты? Соответствуют ли эти высокодоходные клиенты некому демографическому профилю? Если это так, то мы могли бы планировать рекламные компании, нацеленные на этих клиентов. Или клиенты демонстрируют определенные покупательские привычки? Можно ли каким-то образом поощрять эти высокодоходные привычки, чтобы они распространялись на большее количество клиентов? Для ответа на эти и другие вопросы мы можем использовать интеллектуальный анализ данных.

Для начала выполните четыре SQL-скрипта, имеющихся в проекте Chapter18SQL решения SQL Server Management Studio (включенном в примеры кодов данной главы на Web-сайте этой книги). Выполните скрипты в следующем порядке: `vProfitability.sql`, `vCustomerProfitability.sql`, `vCustomerPurchases.sql` и `vMonthlyProfit.sql`. Скрипты добавляют в базу данных AdventureWorksDW2008 четыре представления: `vProfitability`, `vCustomerProfitability`, `vCustomerPurchases` и `vMonthlyProfit`. Представления `vCustomerProfitability`, `vCustomerPurchases` и `vMonthlyProfit` будут основой моделей интеллектуального анализа. Представление `vCustomerProfitability` имеет по строке на каждого клиента и содержит демографические данные, а также суммарные данные по покупательскому поведению каждого клиента. Представление `vCustomerPurchases` имеет по одной строке на каждый заказ клиента. Представление `vMonthlyProfit` имеет по одной строке на каждый месяц (с января 2001 г. до июня 2004 г.) с суммарной месячной прибылью по каждому из регионов (North America, Europe и Pacific). `vProfitability` имеет по одной строке на каждую покупку клиента. Доход от каждой покупки вычисляется именно в этом представлении. На это представление ссылаются `vCustomerProfitability` и `vMonthlyProfit`. Непосредственно на `vProfitability` в кодах этой главы ссылок нет. Оно используется для упрощения определения представлений `vCustomerProfitability` и `vMonthlyProfit`.

## Создание проекта Analysis Services

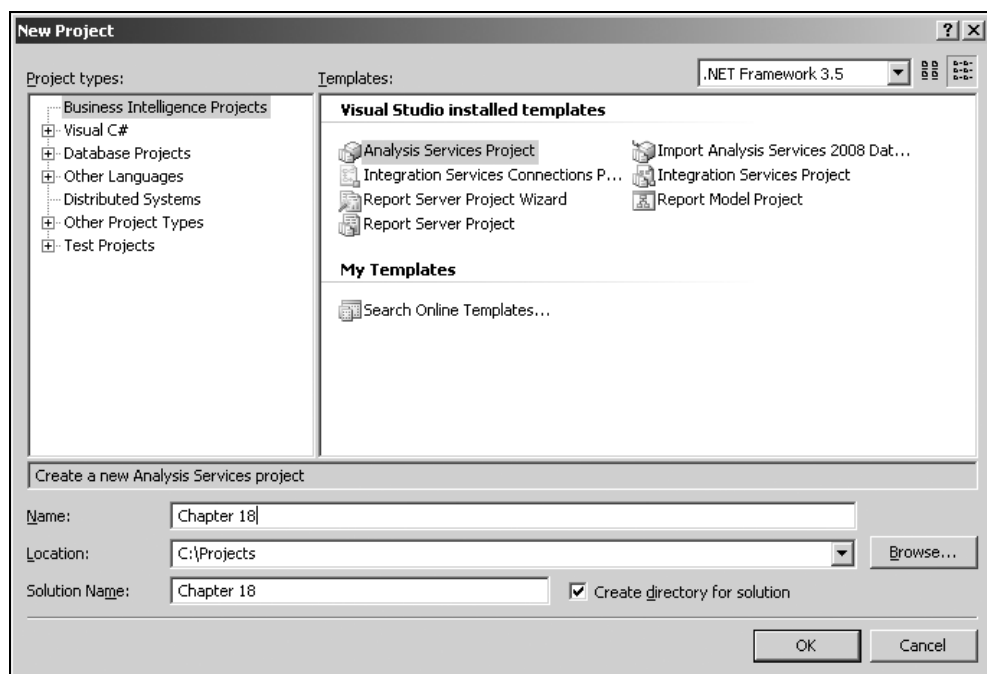
Теперь, когда наши исходные представления созданы, мы готовы создать проект Analysis Services (и базу данных), которые будут содержать наши модели интеллектуального анализа. Обычно на этой стадии мы берем наши исходные данные и делим их на обучающий и контрольный наборы. Раньше мы могли это сделать при помощи задачи преобразования Percentage Sampling

Data Flow служб интеграции Integration Services. Однако в Analysis Services 2008 деление исходных данных на обучающий и контрольный наборы теперь входит в качестве шага в мастере Data Mining Wizard, так что мы можем перейти прямо к созданию нашего проекта Analysis Services.

### ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Проект интеллектуального анализа данных обычно имеет отдельные образцы для обучения (построения) и тестирования моделей интеллектуального анализа данных. Тестирование точности модели интеллектуального анализа данных при помощи иных образцов (не тех, по которым модель обучалась) делается для того, чтобы модель не слишком приспосабливалась к имеющимся у вас данным. В конце концов, настоящая польза от интеллектуального анализа данных состоит в применении модели к новым данным (а не в получении информации о том, что уже произошло).

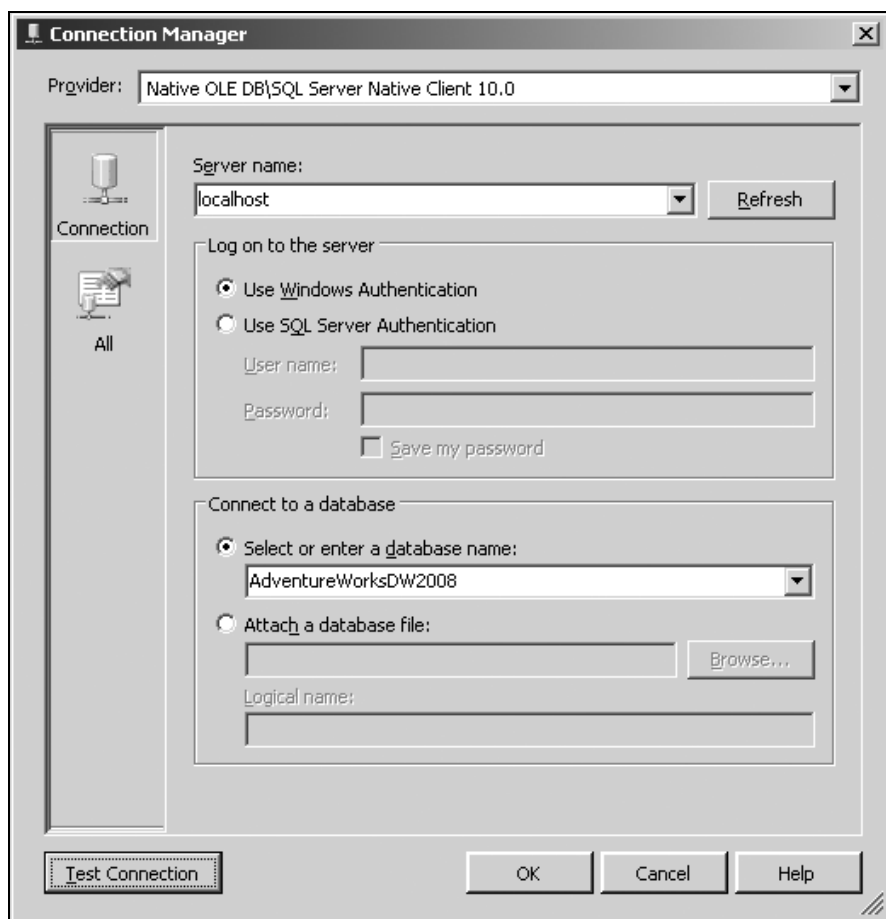
1. В Microsoft Visual Studio выберите пункты **New | Project** (в меню **File**).



**Рис. 18.3.** Диалоговое окно **New Project** для проекта Analysis Services

- В диалоговом окне **Add New Project** выберите тип проекта **Business Intelligence Projects**, а затем — шаблон **Analysis Services Project**. Назовите проект Chapter18. По умолчанию при этом будет также создано одноименное решение (рис. 18.3).
- Нажмите **ОК**, и ваш проект будет создан.

Как и в случае наших содержащих кубы проектов и баз данных Analysis Services в *главах 15 и 16*, мы должны определить в этом проекте источник данных и представление источника данных.



**Рис. 18.4.** Диалоговое окно **Connection Manager** с выделенным поставщиком SQL Native Client Provider

4. Щелкните правой кнопкой мыши по узлу **Data Sources** в окне **Solution Explorer**, а затем выберите пункт **New Data Source**, чтобы открыть мастер Data Source Wizard. Щелкните по кнопке **Next**, а потом по **New** (чтобы появилось диалоговое окно **Connection Manager**).
5. Укажите вашего поставщика (.NET Provider или SqlClient Data Provider), сервер (локальный экземпляр SQL Server, на котором есть база данных AdventureWorksDW2008), любые регистрационные данные, а также базу данных AdventureWorksDW2008. На рис. 18.4 показано заполненное диалоговое окно. Когда закончите, нажмите **OK** для того, чтобы закрыть диалоговое окно.
6. В мастере Data Source Wizard нажмите кнопку **Next**. На странице **Impersonation Information** выберите вариант **Inherit** (последний в списке), а затем еще раз щелкните по кнопке **Next**. Назовите источник данных AdventureWorksDW2008, а затем нажмите **Finish**.

## Data Mining Wizard и визуальный конструктор

Как упоминалось в *главе 15*, добавление в решение источника данных сходно с созданием подключения к базе данных. Для того чтобы указать те объекты из базы данных AdventureWorksDW2008, которые мы хотим использовать для создания наших моделей интеллектуального анализа, нам нужно создать представление источника данных. Для этого щелкните правой кнопкой мыши по узлу **Data Source Views** в окне **Solution Explorer**, а затем выберите пункт **New Data Source View** контекстного меню (чтобы открыть мастер Data Source View Wizard).

На странице **Welcome** нажмите кнопку **Next**, потом еще раз нажмите кнопку **Next** на странице **Select A Data Source** (убедившись, что выбран только что созданный вами источник данных AdventureWorksDW2008). На странице **Select Tables And Views** переместите представление vCustomerProfitability из списка **Available Objects** в список **Included Objects**. Возможно, вам понадобится прокрутить список **Available Objects** до конца, поскольку таблицы в нем перечислены раньше представлений. Нажмите кнопку **Next**, назовите ваш источник данных CustomerProfitability, а затем нажмите кнопку **Finish**.

Для анализа доходности клиентов мы хотим определить общие характеристики высокодоходных и низкодоходных клиентов. Для этого мы должны добавить в наше представление источника данных некий вычисляемый столбец, который определяет "высокую доходность". Мы не хотим хранить его в нашей базе данных, поскольку он специфичен для выполняемого нами анализа.

Для создания вычисляемого столбца в представлении источника данных выполните следующие шаги:

1. Щелкните правой кнопкой мыши по представлению `vCustomerProfitability` на вкладке проектирования представлений источников данных, а затем выберите **New Named Calculation**.
2. В диалоговом окне **Named Calculation** дайте вашему именованному вычислению название `ProfitCategory`, введите описание `High-profit customers in the top quartile of profit`, а затем введите в поле **Expression** следующее:

```
CASE
```

```
    WHEN Profit > 1077 THEN 'High'
```

```
    ELSE 'Low'
```

```
END
```

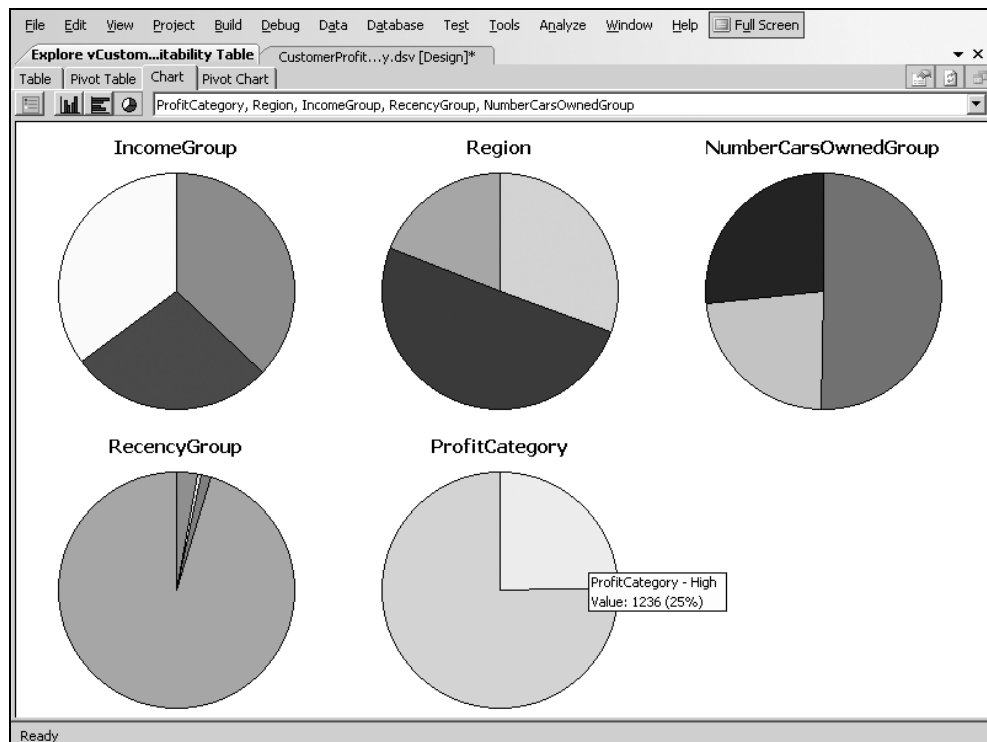
Мы создали две категории клиентов: категорию `High` для клиентов в верхнем квартиле доходов и категорию `Low` для всех остальных клиентов.

3. Нажмите **ОК**. Вы увидите в представлении источника данных созданный вами столбец (со значком калькулятора слева от него).
4. Теперь мы просмотрим данные в представлении источника данных:
5. Щелкните правой кнопкой мыши по представлению `vCustomerProfitability`, а затем выберите пункт **Explore Data**. Откроется окно **Data Explorer**. Прокрутите до самого правого столбца таблицы. Вы увидите только что созданное вами именованное вычисление. Оно будет выглядеть точно так же, как любой другой столбец вашей таблицы.

Вы можете выбрать: просматривать либо  $x$  верхних строк вашей таблицы, либо случайную выборку  $x$  строк из вашей таблицы (значение  $x$  настраивается). По умолчанию окно **Data Explorer** показывает данные верхних 5000 строк источника данных. (Смысл понятия "верхние" может отличаться — в зависимости от сортировки данных в вашей таблице).

6. Для изменения этого представления по умолчанию нажмите кнопку **Sampling Options** (кнопка со значком свойств) в правом верхнем углу окна. После этого откроется диалоговое окно **Data Exploration Options**.

7. Щелкните по опции **Random Sample** (оставьте значение **Sample Count** равным 5000), а затем нажмите **OK**.
8. Для реализации этого изменения конфигурации (чтобы увидеть данные случайного набора ваших записей в `vCustomerProfitability`) вы должны обновить данные. Для этого нажмите кнопку **Resample Data** (кнопка со значком обновления) в правом верхнем углу окна **Data Explorer**.



**Рис. 18.5.** Вкладка **Chart** окна **Data Explorer**, которая позволяет вам увидеть распределение ваших данных

При помощи других вкладок окна **Data Explorer** вы можете создать таблицы `PivotTable` и диаграммы либо простые диаграммы распределения данных. Для использования диаграмм распределения выполните следующие шаги:

1. Щелкните по вкладке **Chart**.

Для подмножества столбцов вашего представления источника данных по умолчанию отображаются столбчатые диаграммы. Вы можете щелкнуть в

выпадающем элементе управления наверху окна, чтобы модифицировать представление для отображения в диаграммах любой комбинации столбцов вашей таблицы.

2. Выберите в выпадающем списке столбец `ProfitCategory` и либо нажмите клавишу `<Enter>`, либо щелкните по вкладке **Chart** (или щелкните в любом месте диаграммы), чтобы закрыть список столбцов. Рядом с остальными диаграммами должна появиться столбчатая диаграмма.
3. Вы можете изменить тип диаграммы на горизонтальную столбчатую диаграмму, либо на секторную диаграмму. Для изменения столбчатых диаграмм на секторные нажмите кнопку секторной диаграммы в левом верхнем углу окна **Data Explorer** (или щелкните правой кнопкой мыши по пустому пространству диаграммы, а затем выберите опцию **Pie Chart**). На рис. 18.5 показаны секторные диаграммы нескольких характеристик клиентов.

Вы можете увидеть детали представленных конкретным элементом вкладки **Chart** данных (для этого надо навести на этот элемент курсор мыши). Например, наведение курсора на сектор диаграммы покажет вам число и процент записей, которые представлены данным сектором (рис. 18.5).

Окно **Data Explorer** позволяет вам легко и быстро увидеть распределение данных (без всякого написания кода). Это может быть полезно на начальной фазе вашего проекта интеллектуального анализа данных (до создания моделей).

## Создание структуры интеллектуального анализа данных

Модели интеллектуального анализа в Analysis Services содержатся внутри *структур интеллектуального анализа*. Такая структура состоит из подмножества столбцов представления вашего источника данных и одной (или более) моделей интеллектуального анализа (которые используют эти столбцы). Тип содержимого каждого столбца настраивается моделью интеллектуального анализа отдельно (для каждой модели вашей структуры вы указываете, является ли данный столбец ключевым, входным или прогнозируемым). Типы содержимого мы обсудим подробнее чуть позже.

Если указать, что данный столбец структуры следует игнорировать в данной модели анализа, вы можете в одной структуре анализа получить две модели, у которых нет общих столбцов (ни входных, ни прогнозируемых). Однако имеет больше смысла группировать в одной структуре такие модели анализа,



которые отвечают на один и тот же бизнес-вопрос и имеют некоторую общность столбцов. Это потому, что созданные в одной структуре модели анализа можно сравнивать при помощи функции Mining Accuracy Chart служб Analysis Services (которую мы скоро рассмотрим). Более того, модели из одной структуры обрабатываются параллельно (за один проход по данным).

Возвращаясь к целям нашего интеллектуального анализа данных, мы хотим знать, могут ли конкретные характеристики или покупательское поведение помочь нам предсказать, кто является высокодоходным клиентом. Мы также хотим кластеризовать клиентов по их демографическим характеристикам и покупательскому поведению. При этом мы будем использовать редактор Data Mining Editor в Visual Studio для создания структуры интеллектуального анализа с моделями, которые используют алгоритмы кластеризации (Clustering), деревьев решений (Decision Tree), а также упрощенный алгоритм Байеса (Bayes). Позднее мы обсудим, как все эти алгоритмы работают, чем они отличаются друг от друга, а также их использование в конкретных сценариях прогнозирования.

## Создание модели интеллектуального анализа

Для создания начальной модели интеллектуального анализа выполните следующие шаги:

1. Щелкните правой кнопкой по узлу **Mining Structures** в окне **Solution Explorer**, а затем выберите пункт **New Mining Structure** для того, чтобы запустить мастер Mining Structure Wizard.
2. На странице **Welcome** нажмите кнопку **Next**.
3. На странице **Select The Definition Method** выберите пункт **From Existing Relational Database Or Data Warehouse**, а затем нажмите кнопку **Next**.
4. На странице **Create The Data Mining Structure** вы можете выбрать методику интеллектуального анализа данных для вашей начальной модели, либо выбрать создание структуры анализа без моделей интеллектуального анализа. (В версии Analysis Services 2005 при создании структуры анализа мастер требовал от вас указать начальную модель анализа). После создания структуры анализа вы можете добавлять в структуру модели анализа. Выберите в выпадающем списке **Microsoft Clustering**, а затем нажмите кнопку **Next**.

5. На странице **Select Data Source View** выберите созданное вами ранее представление источника данных, а затем щелкните по кнопке **Next**.
6. На странице **Specify Table Types** вы увидите представление `vCustomerProfitability` и флажки для указания типа таблицы. Установите флажок **Case** рядом с представлением `vCustomerProfitability`, а затем нажмите кнопку **Next**.

### **ТАБЛИЦЫ ДЛЯ МОДЕЛЕЙ ИНТЕЛЛЕКТУАЛЬНОГО АНАЛИЗА ДАННЫХ**

Службы Analysis Services имеют два типа таблиц, используемых в моделях анализа: таблицы вариантов и вложенные таблицы. Каждая структура анализа должна иметь таблицу вариантов. Проще говоря, таблица вариантов определяет анализируемые элементы (для вашей модели анализа). В анализе доходности клиента основной единицей анализа является клиент; поэтому в качестве таблицы вариантов мы определим `vCustomerProfitability`, по строке на каждого клиента. Вложенные таблицы используются для предоставления дополнительных подробностей. Не каждая структура анализа имеет вложенную таблицу. Пока наша структура анализа тоже будет иметь только таблицу вариантов. Позднее в этой же главе мы расширим модели доходности путем учета типов сделанных клиентом покупок при помощи вложенной таблицы.

7. На странице **Specify The Training Data** вы увидите список всех столбцов вашей таблицы вариантов. Справа от каждого столбца находится набор из трех флажков, чтобы указать, является ли он ключевым, входным или прогнозируемым столбцом вашей модели (рис. 18.6).

Ключевой столбец уникально идентифицирует каждую запись вашей таблицы вариантов. Входные столбцы формируют основу, по которой делается поиск шаблонов данных (как описательных, так и для прогнозирования). Например, алгоритм кластеризации создает кластеры на основе значений входных столбцов. Алгоритм деревьев решений принимает решение по разбиению дерева в зависимости от того, насколько хорошо значения входного столбца прогнозируют конкретный результат.

### **ПРОГНОЗИРУЮЩИЕ И ОПИСАТЕЛЬНЫЕ МОДЕЛИ ИНТЕЛЛЕКТУАЛЬНОГО АНАЛИЗА**

С технической точки зрения маркировка столбца как прогнозируемого означает, что он может быть выбран в модели прогнозирующего запроса расширений интеллектуального анализа (Data Mining Extensions, DMX). Это справедливо для прогнозируемых столбцов любой модели интеллектуального анализа, даже тех, целью которых прогнозирование не является.

Прогнозирующие модели анализа предназначены для формирования значений соответствующих прогнозируемых столбцов. Для описательных моделей анализа прогнозируемые столбцы имеют чисто техническое использование (причем оно может быть различным). В модели кластеризации входные столбцы используются для определения кластеров. Установка для столбца одного только флажка **Predict** означает, что этот столбец не будет использоваться для определения кластеров, однако включение столбца в модель в качестве прогнозируемого позволяет вам просматривать его распределение внутри каждого кластера и сравнивать его распределение по кластерам (после обработки модели).

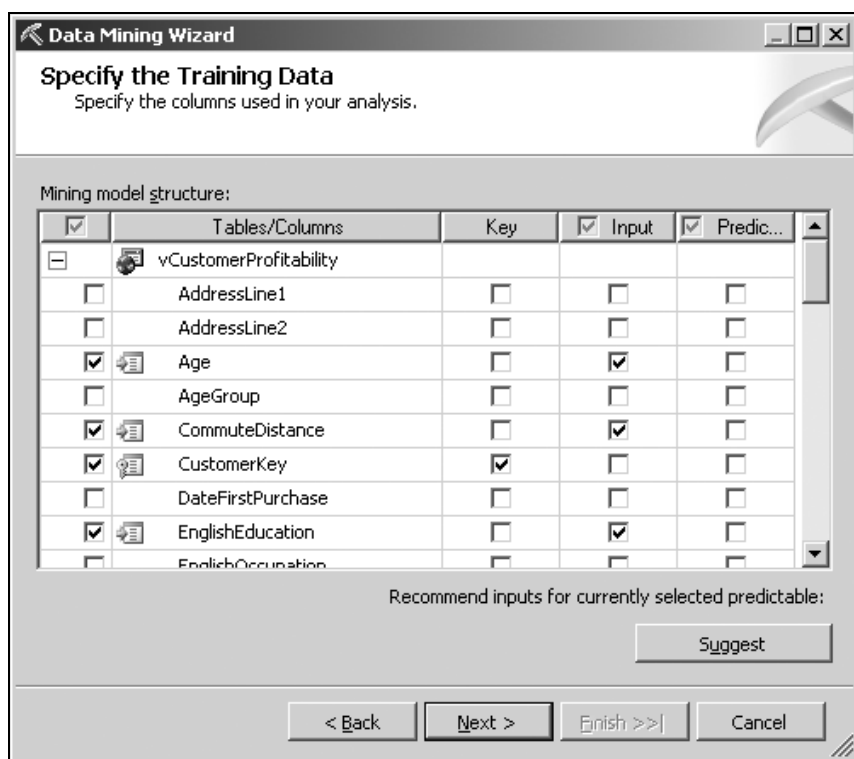
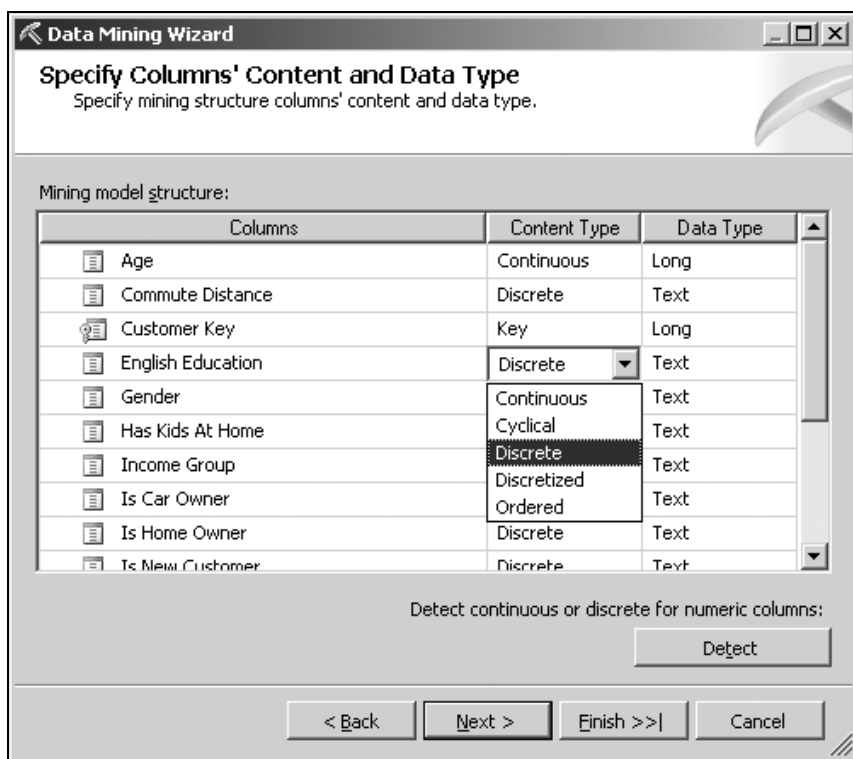


Рис. 18.6. Страница **Specify The Training Data** мастера Data Mining Wizard

- На странице **Specify The Training Data** определите столбец **CustomerKey** в качестве ключа вашей таблицы вариантов и установите флажок **Input** для следующих столбцов: **Age**, **CommuteDistance**, **EnglishEducation**, **Gender**, **HasKidsAtHome**, **IncomeGroup**, **IsCarOwner**, **IsHomeOwner**, **IsNewCustomer**, **MaritalStatus**, **NumProdGroup** и **Region**.

9. Установите флажок **Predict** для столбца ProfitCategory, а затем нажмите кнопку **Next**.
10. На странице **Specify Columns' Data Type And Content** нажмите кнопку **Detect**.

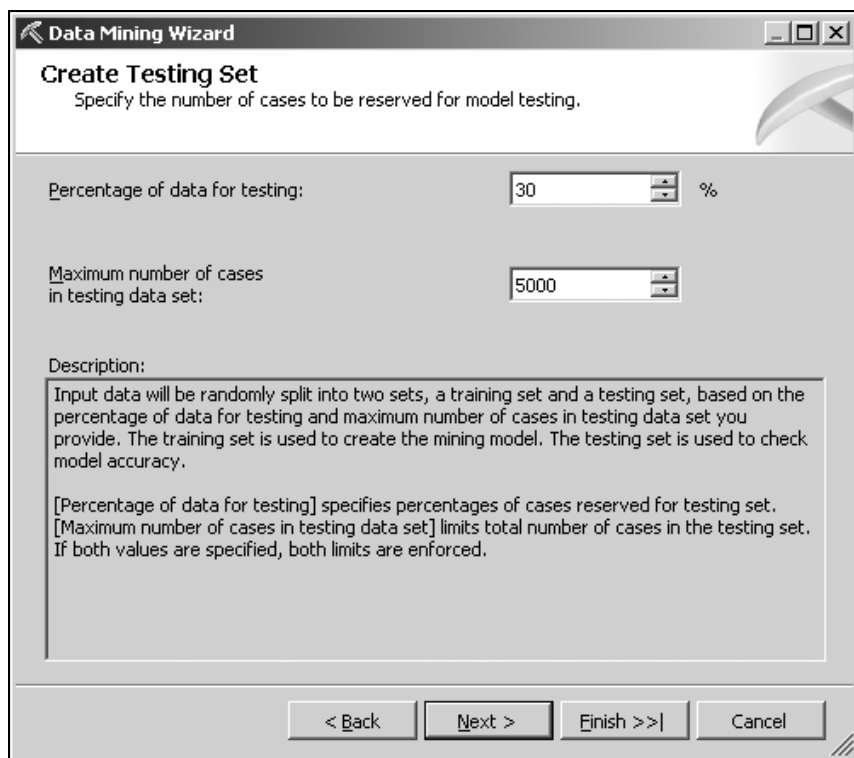
Это заставит службы Analysis Services определить тип ваших входных и прогнозируемых столбцов как дискретный (категорийный) или непрерывный. Вы можете также модифицировать содержимое и тип данных, щелкнув в ячейке и выбрав тип из появившегося выпадающего списка, как показано на рис. 18.7.



**Рис. 18.7.** Тип содержимого и тип данных столбцов указывается на уровне структуры анализа в мастере Data Mining Wizard

11. Проверьте, что тип содержимого для столбца Age установлен как Continuous, для столбца CustomerKey — как Key, а для всех остальных столбцов — как Discrete, а затем нажмите кнопку **Next**.

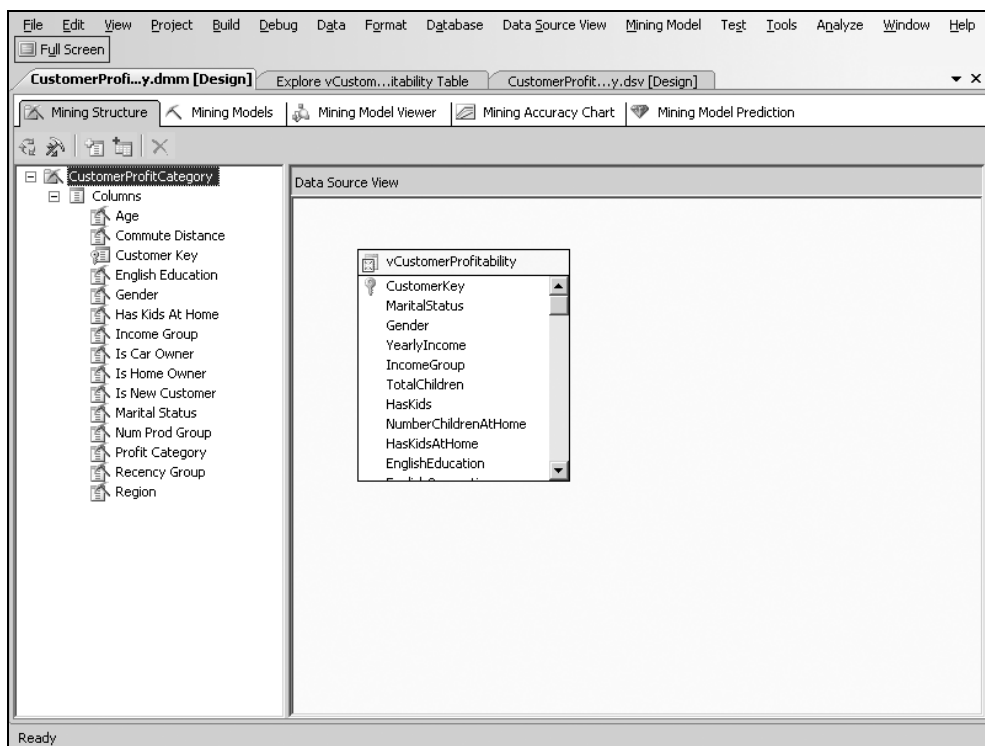
12. Это приводит нас на новую страницу **Split Data Into Training And Test Sets**. До версии SQL Server 2008 обучающий и контрольный наборы создавались еще до создания проекта Analysis Services при помощи такого инструмента, как преобразование Percentage Sampling Data Flow служб интеграции Integration Services. Теперь деление исходных данных на обучающий и контрольный наборы выполняется как шаг работы мастера Data Mining Wizard. По умолчанию процент контрольных вариантов составляет 30. Это означает, что 70% ваших записей будет использовано для обучения моделей анализа, а 30% — для тестирования моделей. Установите максимальное количество контрольных вариантов в 5000. Это означает, что 30% выборки (но не более 5000 вариантов) будет использоваться для контрольного набора. Страница **Create Testing Set** должна быть настроена так, как показано на рис. 18.8. Нажмите кнопку **Next**.



**Рис. 18.8.** Разбиение данных на обучающий и контрольный наборы (теперь часть мастера Data Mining Wizard)

13. Назовите структуру анализа `CustomerProfitCategory`, а модель анализа — `CustomerProfitCategory_CL`, а затем нажмите **Finish**.

Это приводит нас на вкладку **Mining Structure** визуального конструктора структур анализа (рис. 18.9).



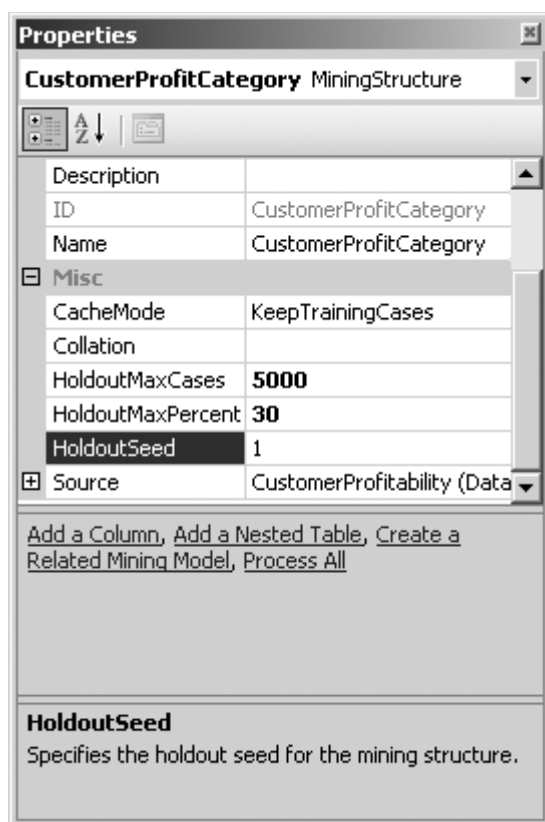
**Рис. 18.9.** Вкладка **Mining Structure** визуального конструктора структур анализа (показывает столбцы структуры анализа)

Левая панель вкладки содержит древовидное представление нашей структуры анализа. Здесь вы можете добавлять и удалять столбцы и вложенные таблицы (на панели инструментов вкладки или правой кнопкой мыши в любом месте панели древовидного представления слева). Панель **Data Source View** (справа) позволяет вам изучать и редактировать представление источника данных.

**ВАЖНО**

Следует настроить параметры разбиения выборки точно так, как указано в тексте. Службы Analysis Services разбивают исходные данные на обучающий и контрольный набор случайным образом. Если будет указано другое начальное число контрольного набора, другое количество или процент контрольных вариантов, то результаты модели анализа не будут идентичными показанным. Конечно, если модели устойчивы, то характер результатов должен быть сходным по разным случайным выборкам, однако они не будут полностью совпадать с показанными здесь примерами.

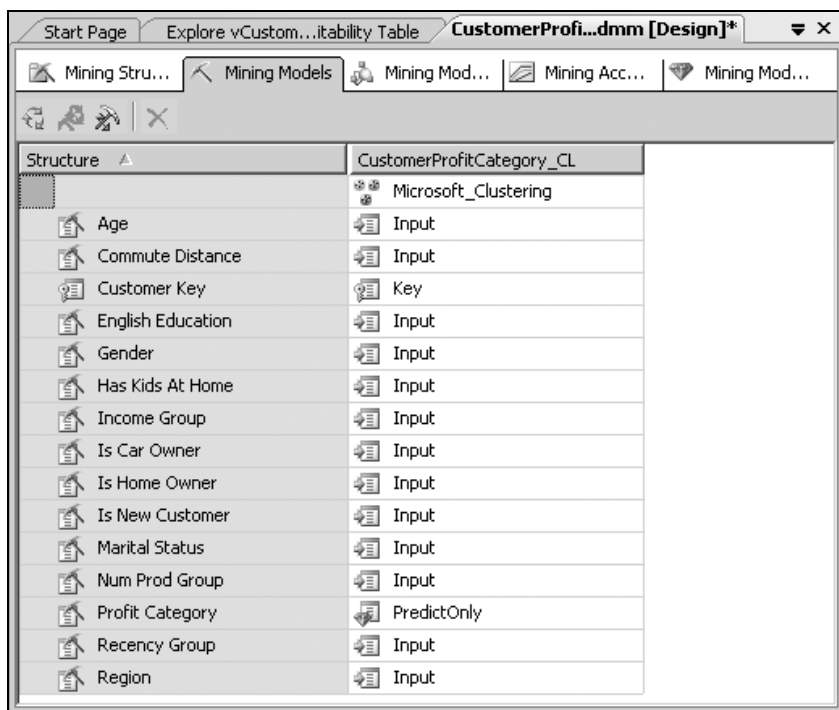
14. Начальные значения должны быть одинаковыми (в этом примере и у вас). Найдите свойство **HoldoutSeed** в панели **Properties** структуры анализа и установите его значение в 1 (рис. 18.10).



**Рис. 18.10.** Свойство **HoldoutSeed**

## Редактирование и добавление моделей интеллектуального анализа данных

После создания вашей структуры анализа и первой модели анализа вы готовы создавать дополнительные модели в этой же структуре. На вкладке **Mining Models** визуального конструктора структур анализа вы можете просмотреть определение вашей модели анализа (рис. 18.11). Вы увидите столбцы вашей структуры анализа, настроенные в соответствии со сделанными вами ранее установками.



**Рис. 18.11.** Вкладка **Mining Model**, которая показывает использование столбцов структуры анализа в моделях анализа

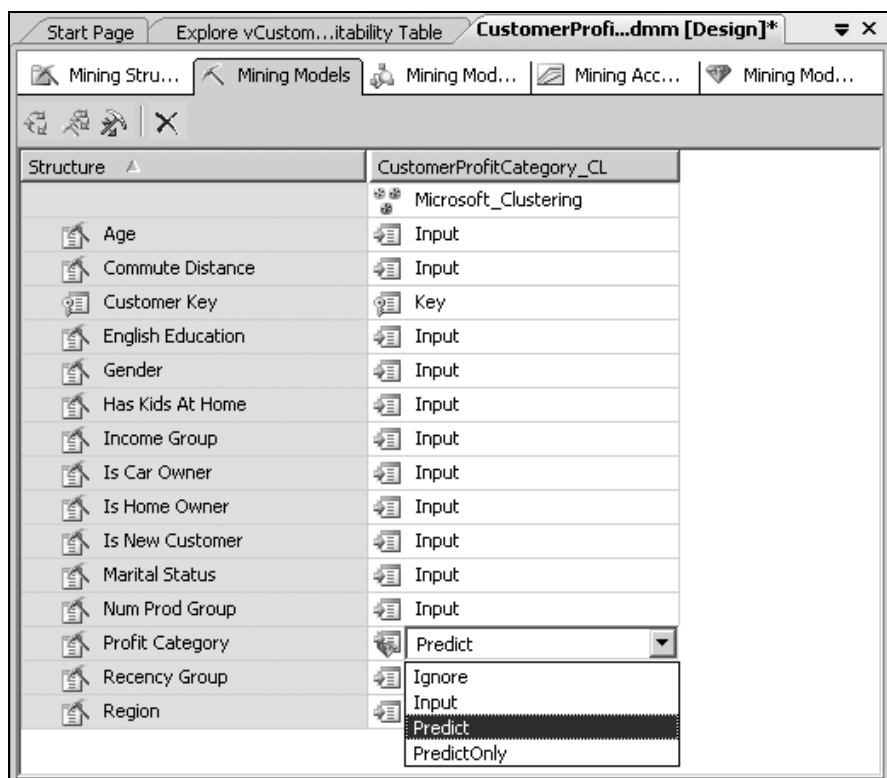
## Редактирование модели интеллектуального анализа данных

Вы можете отредактировать определение вашей модели интеллектуального анализа на вкладке **Mining Model**. Столбец `Profit Category` указан в модели



анализа как PredictOnly. Мы хотим изменить тип использования с PredictOnly на Predict. Для редактирования типа использования столбца сделайте следующие шаги:

1. На вкладке **Mining Models** визуального конструктора структур анализа щелкните по ячейке рядом с Profit Category.
2. В появившемся выпадающем списке измените тип использования столбца на Predict (рис. 18.12).



**Рис. 18.12.** Щелчок по ячейке рядом со столбцом для изменения типа его использования в модели интеллектуального анализа

Изменение типа использования столбца с PredictOnly на Predict позволяет категории доходности клиента стать фактором определения кластеров (а не просто описательной характеристикой, которая анализируется после свершения факта).

## Добавление модели интеллектуального анализа данных

По умолчанию алгоритм кластеризации Microsoft Clustering создает из ваших обучающих данных 10 кластеров. Как вдумчивый аналитик, вы можете быть обеспокоены тем, что пользователям будет трудно уловить нюансы различий между таким большим количеством кластеров. Поэтому вы хотите создать вторую модель кластеризации только на пять кластеров. Для этого выполните следующие шаги:

1. Щелкните правой кнопкой мыши в любом месте вкладки **Mining Models**, а затем выберите **New Mining Model**.
2. В диалоговом окне **New Mining Model** введите CustomerProfitCategory\_CL5 в текстовом поле **Model Name**, выберите **Microsoft Clustering** в выпадающем списке **Algorithm Name**, а затем нажмите **OK**. Новая модель анализа добавляется правее существующей модели.

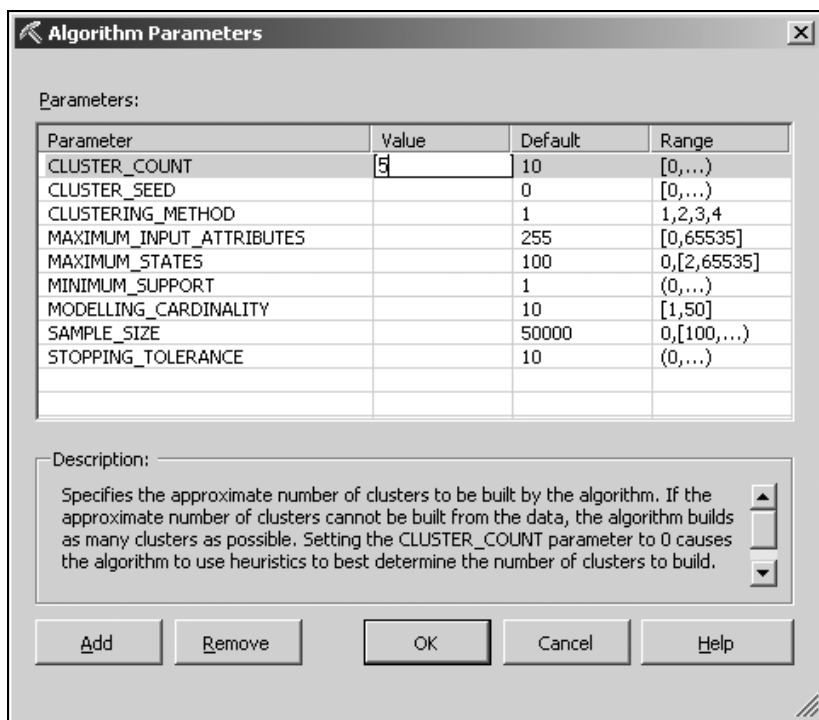


Рис. 18.13. Диалоговое окно **Algorithm Parameters** в модели Microsoft Clustering

3. Для изменения количества создаваемых кластеров с 10 до 5 щелкните правой кнопкой мыши по заголовку столбца `CustomerProfit Category_CL5` (или по любой из ячеек этого столбца), а затем выберите пункт **Set Algorithm Parameters**.
4. В диалоговом окне **Algorithm Parameters** (рис. 18.13) вы можете настроить параметры алгоритма, которые специфичны для выбранной модели интеллектуального анализа.

Сначала это может несколько озадачить, поскольку визуальные конструкторы скрывают большую часть сложностей интеллектуального анализа данных за своим пользовательским интерфейсом и мастерами. А это диалоговое окно позволяет настроить вашу модель, изменив значения по умолчанию для параметров алгоритма. Пощелкайте различные параметры в списке параметров и просмотрите их описания в зоне **Description** (внизу).
5. По завершении изучения параметров щелкните по ячейке на пересечении строки `CLUSTER_COUNT` и столбца `Value`, введите 5 и нажмите **OK**.

## Добавление модели, которая использует другой алгоритм

Теперь в нашей структуре анализа имеется две модели кластеризации. Для проверки точности (модели в структуре анализа) полезно создать дополнительную модель с другим алгоритмом. Мы добавим новую модель, использующую алгоритм деревьев решений:

1. Щелкните правой кнопкой мыши в любом месте вкладки **Mining Models**, а затем выберите **New Mining Model**.
2. В диалоговом окне **New Mining Model** назовите свою модель `CustomerProfitCategory_DT`, выберите алгоритм **Decision Trees** (он должен быть выбран по умолчанию), а затем нажмите **OK**.

Модель `CustomerProfitCategory_DT` появляется правее существующих моделей кластеризации (с теми же столбцами и таким же их использованием).

## Изменение использования столбцов

Добавление модели — это хорошо, но было бы еще лучше, если бы мы могли использовать ее для прогнозирования количества продуктов, которые покупают люди (в дополнение к прогнозированию категории доходности). Для изменения

использования столбца NumProductGroup модели CustomerProfitCategory\_DT, щелкните по ячейке (соответствующей NumProductGroup в CustomerProfitCategory\_DT), а затем выберите Predict в выпадающем списке. Установив тип содержимого обеих переменных (ProfitCategory и NumProdGroup) в Predict, мы позволяем каждой из них быть входом дерева решения для другой. Если бы мы настроили тип содержимого NumProdGroup в PredictOnly, то количество купленных продуктов не могло бы быть фактором при создании дерева решений ProfitCategory, то есть количество купленных продуктов не рассматривалось бы при разбиениях дерева ProfitCategory.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Разбиения — это ветви в узлах дерева решений. Каждый узел дерева решений представляет собой подмножество выборки и характеризуется родителем узла. Разбиения в каждом узле определяются характеристикой входа, по которой распределение прогнозируемой переменной наиболее сильно отличается для определенного в данном узле подмножества. Когда распределение прогнозируемой переменной в узле не имеет значительных отличий ни по одной характеристике входа, разбиений больше нет, и ветка заканчивается. Узел, в котором нет разбиений, называется *листом*.

## **Модели интеллектуального анализа данных и типы данных**

Давайте продолжим строить нашу структуру анализа, добавим еще одну модель, на этот раз по упрощенному алгоритму Байеса:

1. Щелкните правой кнопкой мыши по модели CustomerProfitCategory\_CL, а затем выберите пункт **New Mining Model**.
2. Назовите вашу по модели CustomerProfitCategory\_NB, выберите алгоритм **Naive Bayes**, а затем нажмите **OK**.

Появится окно сообщения с пояснением, что столбец Age будет проигнорирован, поскольку алгоритм Naive Bayes не поддерживает работу с непрерывными столбцами. Нажмите **Yes**. Обратите внимание, что тип содержимого столбца Age установлен в значение Ignore, тип содержимого столбца NumProd — в значение Input, а столбца ProfitCategory — в значение Predict. Для этой модели мы хотим прогнозировать только ProfitCategory, поэтому модифицировать тип содержимого не нужно.

**ВАЖНО**

Поскольку мы создали модель Naive Bayes при помощи щелчка правой кнопкой мыши по модели CustomerProfitCategory\_CL и последующего выбора пункта **New Mining Model**, то настройки типа содержимого CustomerProfitCategory\_CL были использованы и в новой модели, что сделало ProfitCategory единственным прогнозируемым столбцом. Если бы мы щелкнули правой кнопкой мыши по модели CustomerProfitCategory\_DT и выбрали пункт **New Mining Model**, то были бы использованы настройки типа содержимого из этой модели, и в дополнение к ProfitCategory прогнозируемым стал бы и столбец NumProdGroup.

Мы уже упоминали, что в модели Naive Bayes игнорируется возраст (поскольку он является непрерывной переменной). Если возраст является важным фактором при определении того, является ли клиент высокодоходным или низкодоходным, то модель Naive Bayes будет работать хуже, чем другие модели (в состав которых возраст входит). Поэтому мы хотим ввести в модель NB хоть какое-нибудь указание возраста. Для этого мы должны добавить в структуру анализа "дискретизированную" версию столбца возраста и включить ее в нашу модель Naive Bayes.

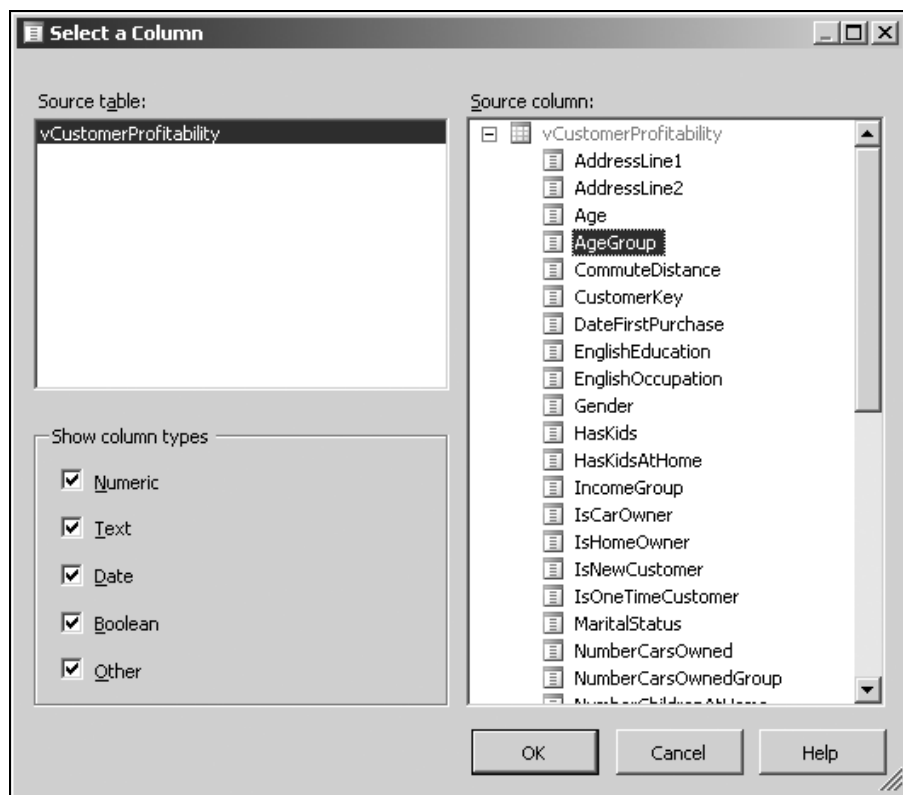
1. Раньше столбцы структуры анализа можно было добавлять только из вкладки **Mining Structure**. В Analysis Services 2008 столбец можно добавить в структуру из вкладки **Mining Models**. Щелкните правой кнопкой мыши в любом месте столбца Structure на вкладке **Mining Models**, а затем выберите **Add A Column**.
2. В диалоговом окне **Select A Column** (рис. 18.14) выберите в списке **Source Column** столбец AgeGroup, а затем нажмите **OK** для добавления AgeGroup в структуру анализа.

**ПРИМЕЧАНИЕ**

Столбец AgeGroup в vCustomerProfitability при помощи оператора CASE категоризирует клиентов по таким возрастным группам, как до 30, от 30 до 35 и от 36 до 45. Тип содержимого столбца discretized можно использовать для категоризирования непрерывного атрибута. Обсуждение этой функции выходит за рамки данной главы, но вы можете прочитать о ней в документации SQL Server Books Online.

3. Вы увидите, что AgeGroup появился в структуре анализа (несмотря на то, что его использование во всех моделях выставлено как тип Ignore). Для включения его в модель Naive Bayes щелкните в ячейке, которая соответ-

ствует столбцу AgeGroup в модели Naïve Bayes, и измените его использование на тип Input.



**Рис. 18.14.** Диалоговое окно **Select A Column** для добавления столбца в вашу структуру анализа

## Фильтрация модели интеллектуального анализа данных

Очень часто вам бывает необходимо оценить модель анализа только для определенного поднабора данных либо оценить в зависимости от определенной характеристики. Например, мы планируем запустить рекламную кампанию, ограниченную клиентами из Северной Америки. Для того чтобы наиболее эффективно спроектировать эту кампанию, мы хотим лучше понять высокодоходных клиентов Северной Америки. Давайте посмотрим, как мы

можем это сделать при помощи фильтрации модели анализа (это новая функциональная возможность SQL Server 2008):

1. Щелкните правой кнопкой мыши по модели анализа `CustomerProfitCategory_DT`, а затем выберите пункт **New Mining Model**. В диалоговом окне **Add New Model** назовите модель `CustomerProfitCategory_DT_NorthAmerica`, убедитесь, что выбран алгоритм **Microsoft Decision Trees**, а потом нажмите **OK**.
2. В панели свойств выберите `CustomerProfitCategory_DT_NorthAmerica` в списке и найдите свойство `Filter` для этой модели анализа. Щелкните в пустой ячейке, соответствующей свойству `Filter`, чтобы появилась кнопка с многоточием, затем нажмите эту кнопку (чтобы показать диалоговое окно **Model Filter**).

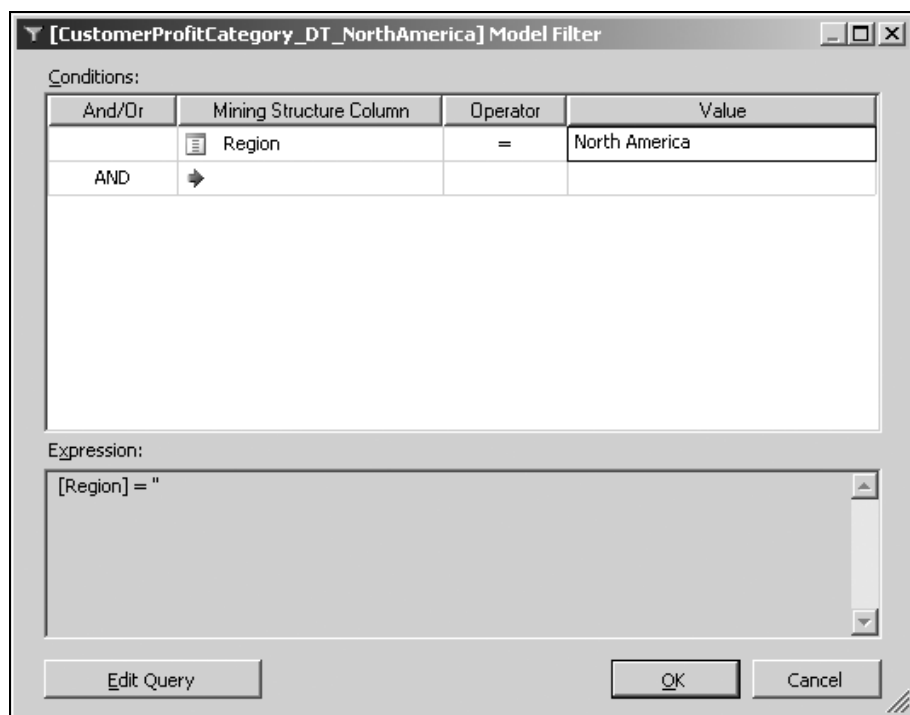


Рис. 18.15. Диалоговое окно **Model Filter**

3. Выберите `Region` в списке **Mining Structure Column**, а затем введите в ячейке значения текст `North America` (рис. 18.15). Нажмите **OK**. Модель

CustomerProfitCategory\_DT\_NorthAmerica будет обучаться только при помощи тех записей, у которых в поле Region стоит North America.

Преимущества фильтрации моделей анализа в SQL Server 2008 наиболее очевидны тогда, когда вы хотите оценить разные модели анализа для разных групп. Например, определяющие факторы доходности клиента в разных регионах отличаются. В этом случае вам нужно оценить разные модели анализа по регионам. В SQL Server 2008 сделать это легко. В то время как в SQL Server 2005 вам пришлось бы добавить отдельную таблицу, представление или именованный запрос в представление источника данных, а потом создать отдельную структуру анализа, в SQL Server 2008 просто нужно настроить фильтр для каждой модели. Фильтрованные модели анализа могут находиться в одной структуре анализа.

Ваша вкладка **Mining Models** показана на рис. 18.16.

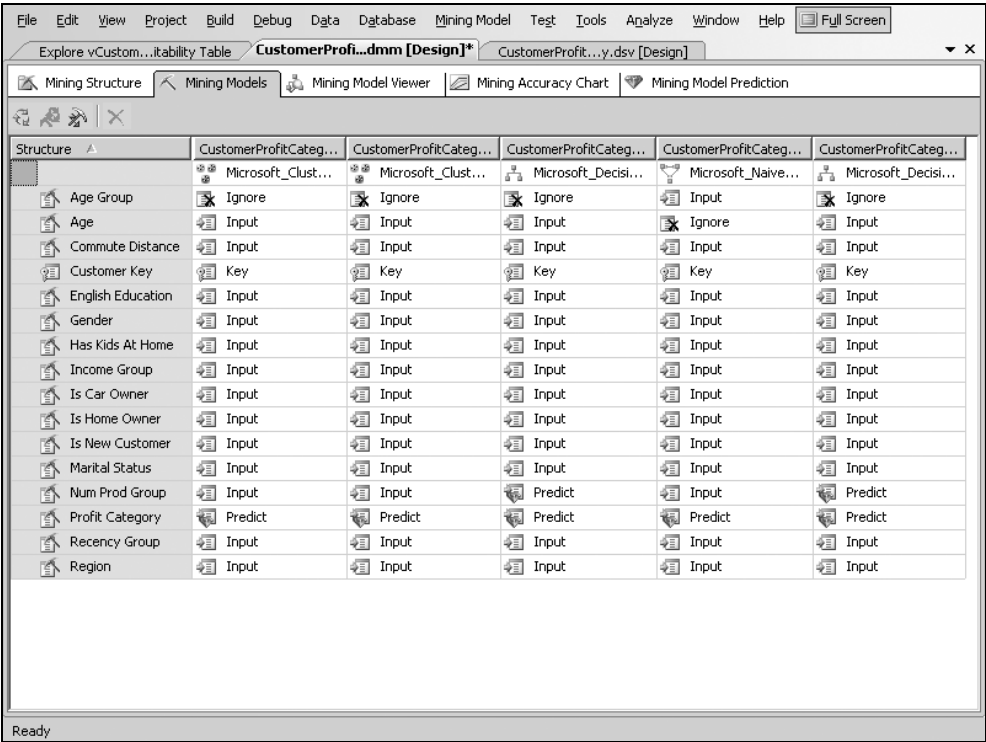


Рис. 18.16. Структура анализа с несколькими моделями анализа



## Развертывание и обработка объектов интеллектуального анализа данных

Мы проделали большую работу, определив структуру анализа с несколькими моделями анализа, готовы просматривать модели и делать к ним запросы, но сначала должны развернуть и обработать ту базу данных Analysis Services, в которой они находятся. Мы можем развернуть базу данных и обработать модели анализа за один шаг, выбрав команду **Start Debugging** в меню **Debug** либо нажав кнопку панели инструментов **Start Debugging** (или функциональную клавишу <F5>).

### **ВНИМАНИЕ!**

Решение Visual Studio (содержащееся в примерах кодов данной главы) настроено таким образом, что кнопки панели инструментов **Start Debugging** и команды меню не приведут к автоматическому развертыванию проекта Chapter18. Мы приняли эту меру предосторожности для того, чтобы не дать вам возможность случайно развернуть эту базу при отладке имеющихся в этом же решении клиентских приложений Windows Forms и ASP.NET. Если вы работаете с примерами кодов, хотите развернуть базу данных Analysis Services и обработать ее модели анализа, то просто щелкните по узлу проекта Chapter18 в окне **Solution Explorer**, а затем выберите команду **Deploy**.

Развертывание и обработка могут занять несколько минут. Вы можете наблюдать за продвижением развертывания в окне **Deployment Progress**, которое по умолчанию появляется как закрепленное окно в правом нижнем углу Visual Studio (после начала развертывания). Если это окно не видно, то вы можете показать его при помощи выбора пункта **Show Deployment Progress** в меню **View** (либо при помощи щелчка правой кнопкой мыши по узлу проекта и последующего выбора пункта **Show Deployment Progress** в окне **Solution Explorer**).

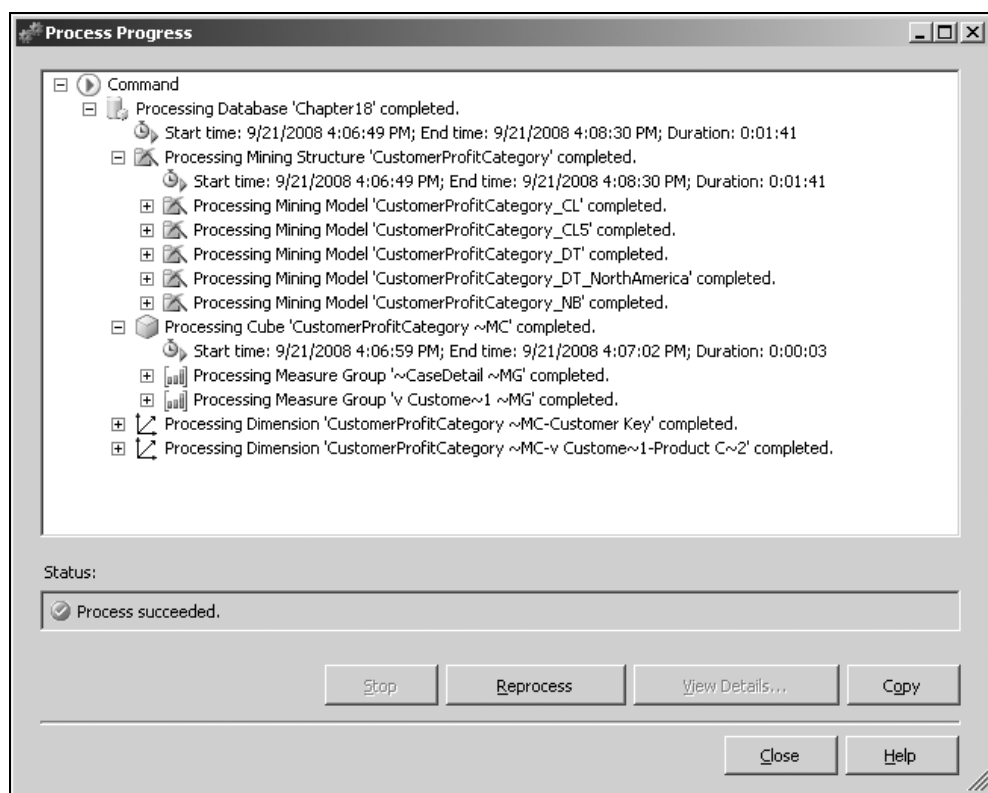
Удобнее всего наблюдать за продвижением развертывания и обработки вашей структуры анализа и моделей в том случае, если вы открепите окно **Deployment Progress** и увеличите его до удобного размера.

### **СОВЕТ**

Самый простой способ открепить окно **Deployment Progress** — это дважды щелкнуть по его заголовку. Вы можете прикрепить его после завершения

обработки (опять дважды щелкнув по его заголовку). Это избавляет от необходимости выполнять скучную процедуру перетаскивания (для открепления и закрепления окна).

После развертывания Visual Studio показывает окно **Process Progress** (на время обработки объектов интеллектуального анализа данных). На рис. 18.17 показано окно **Process Progress** для успешно развернутого и обработанного проекта Analysis Services.



**Рис. 18.17.** Окно **Process Progress** после успешного развертывания и обработки объектов интеллектуального анализа данных

### **ВНИМАНИЕ!**

Если вы получите ошибки обработки с сообщениями о том, что столбцы вашего источника данных не существуют, вам нужно будет изменить ин-

формацию олицетворения источника данных в вашем проекте Analysis Services. Для этого дважды щелкните по узлу источника данных в узле **Data Sources** окна **Solution Explorer**, щелкните по вкладке **Impersonation Information**, а затем выберите соответствующего пользователя. При разработке вы можете выбрать опцию **Use The Service Account**. Перед развертыванием базы данных и моделей анализа в производство вам нужно будет выбрать конкретную учетную запись с соответствующими правами.

После начального развертывания базы данных Analysis Services вы можете обрабатывать модели анализа без развертывания базы данных. Для обработки всех моделей анализа в структуре выберите пункт **Process Mining Structure And All Models** в меню **Mining Model**. Для обработки одной модели структуры анализа выберите модель на вкладке **Mining Models**, а затем выберите пункт **Process Model** в меню **Mining Model**. Обе эти команды (**Process Mining Structure And All Models** и **Process Model**) доступны также и через контекстные меню моделей на вкладке **Mining Models**, а также при помощи двух самых левых кнопок панели инструментов этой вкладки.

## Просмотр моделей интеллектуального анализа данных

Службы Analysis Services предоставляют специфичные для каждого алгоритма средства просмотра моделей в проектах Analysis Services в Visual Studio (а также и в SQL Server Management Studio, подробнее об этом мы скажем чуть позже), которые значительно облегчают задачу интерпретации содержимого моделей, выявленных шаблонов и корреляций. Предоставляемые средствами просмотра инструменты визуализации делают интеллектуальный анализ данных доступным для более широкой аудитории, чем традиционные продукты интеллектуального анализа данных.

Возьмем, например, кластеризацию, которая используется для создания поднаборов (кластеров) ваших данных, которые имеют общие характеристики. Попробуйте себе представить результаты модели кластеризации с 10 клиентскими кластерами (определенными по 8, 10 или 12 характеристикам). Как вы определите идентифицирующие характеристики каждого кластера? Как вы определите, являются ли полученные кластеры похожими? Средство просмотра Microsoft Cluster Viewer делает ответы на такие вопросы исключительно простыми. Давайте посмотрим, как это делается.

## Использование средства просмотра Cluster Viewer

После развертывания и обработки всех моделей анализа структуры CustomerProfitCategory вы попадете на вкладку **Mining Model Viewer**. Выберите CustomerProfitCategory\_CL в выпадающем списке **Mining Model**. Затем выберите **Microsoft Cluster Viewer** в выпадающем списке **Viewer**. Предлагается четыре способа просмотра результатов модели кластеризации: **Cluster Diagram**, **Cluster Profiles**, **Cluster Characteristics** и **Cluster Discrimination**. Для каждого из этих инструментов визуализации имеется отдельная вкладка.

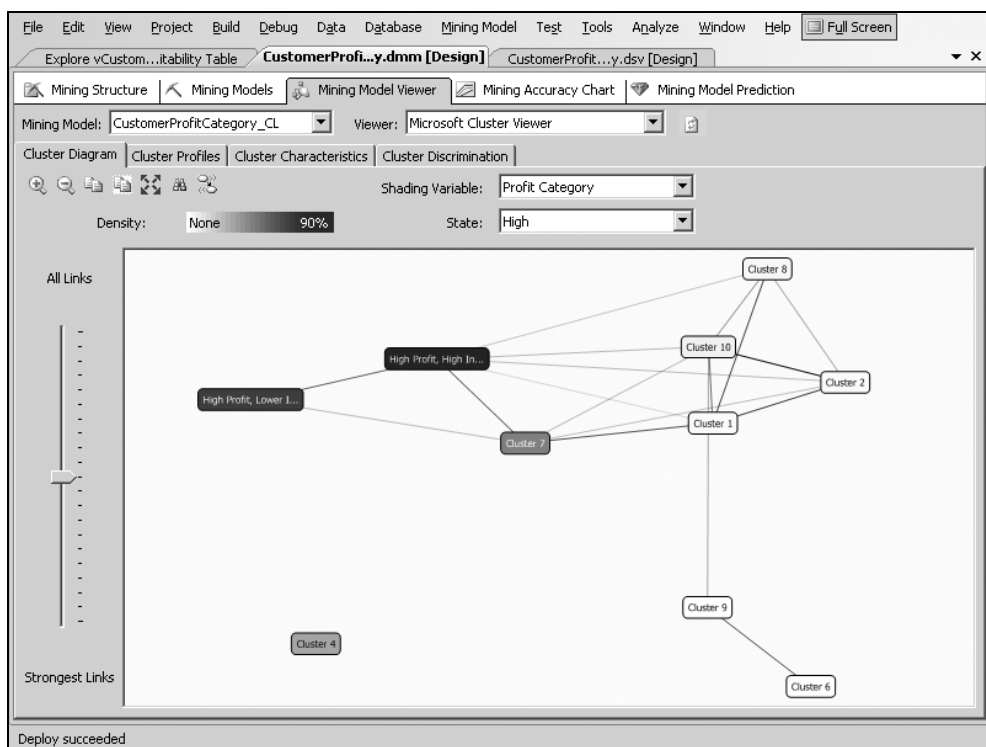


Рис. 18.18. Вкладка **Cluster Diagram** средства просмотра Cluster Viewer

Вкладка **Cluster Diagram** показывает диаграмму кластеров модели с цветовым кодированием для указания плотности данных: чем темнее, тем больше записей содержит кластер. Указываемые закрашиванием переменная и ее значения управляются при помощи выпадающих списков **Shading Variable** и

**State:** вы можете выбрать здесь любой входной или прогнозируемый столбец. Например, если вы выберете атрибут `ProfitCategory` в выпадающем списке **Shading Variable**, а `High` — в выпадающем списке **State**, то диаграмма настроится таким образом, что более темные кластеры будут содержать больше высокодоходных клиентов, чем светлые (рис. 18.18).

### ПРИМЕЧАНИЕ

Ваши номера кластеров должны совпадать с описанными в тексте и имеющимися на рисунках. Однако существует некоторый присущий алгоритму Microsoft Clustering элемент случайности в том, как он определяет кластеры (что может привести к тому, что ваши номера будут отличаться от номеров в тексте и на рисунках). Даже если номера и отличаются, вы все равно должны будете видеть тот же самый качественный характер результатов, который описан в тексте.

Самые темные пузыри (3, 5 и 7) — это кластеры с относительно высокой концентрацией высокодоходных клиентов. Наведение курсора мыши на кластер показывает точный процент высокодоходных клиентов в данном кластере. Теперь выберите столбец `IncomeGroup` в выпадающем списке **Shading Variable** и `High` в выпадающем списке **State**. Теперь самыми темными станут кластеры 7, 8 и 3. Выберите столбец `IsCarOwner` в выпадающем списке **Shading Variable** и `N` — в выпадающем списке **State**. Теперь самыми темными кластерами должны стать 4, 6 и 5.

Изучая таким образом характеристики кластеров, мы можем дать им более осмысленные названия. Вы можете переименовать кластер при помощи щелчка (по кластеру на диаграмме) правой кнопкой мыши и выбора пункта **Rename**. Давайте сделаем это прямо сейчас. Щелкните по кластеру 3 правой кнопкой мыши и переименуйте его в `High Profit, High income, Car Owner`, а затем щелкните правой же кнопкой мыши по кластеру 5 и переименуйте его в `High Profit, Lower income, Less Likely Car Owner`.

Кластеры komponуются на диаграмме по их схожести. Чем короче и темнее линия, соединяющая пузыри, тем более похожи эти кластеры. По умолчанию на диаграмме показаны все связи. Используя ползунок слева от диаграммы, вы можете удалить слабые связи и выявить наиболее похожие кластеры.

## Профили кластеров

Вкладка **Cluster Profiles** (рис. 18.19) показывает распределение входных и прогнозируемых переменных для каждого кластера и для всей выборки кли-

ентов. Эта вкладка особенно полезна для получения общей картины кластеров, созданных моделью анализа. Используя профили кластеров, вы можете легко сравнить атрибуты по кластерам и проанализировать атрибуты кластеров относительно всей выборки данных модели.

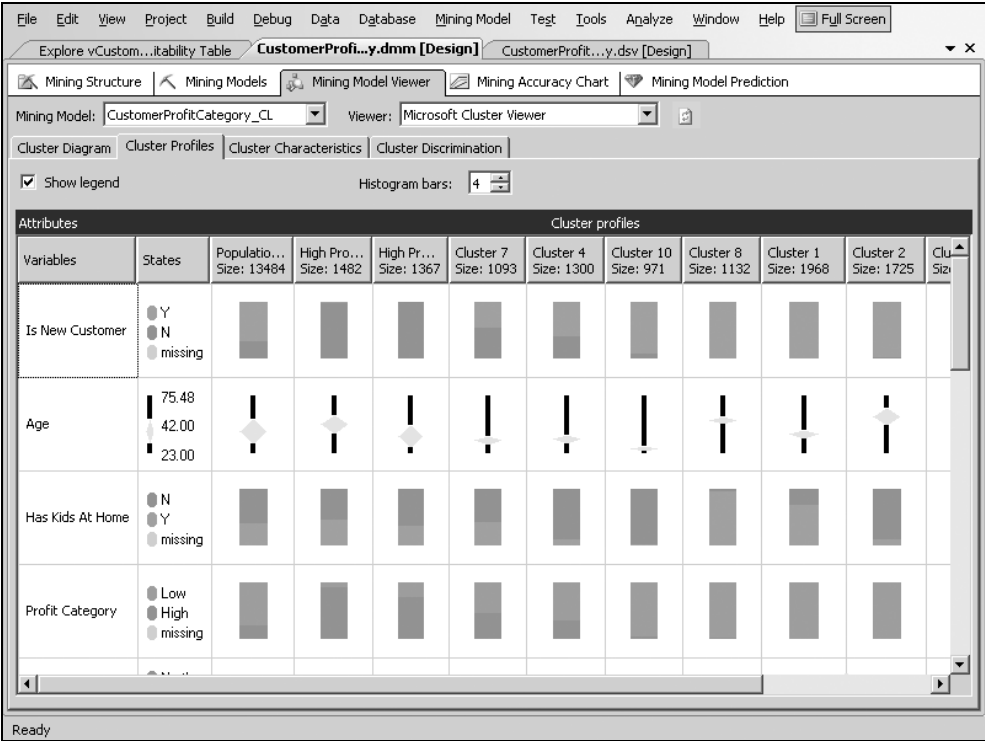


Рис. 18.19. Вкладка **Cluster Profiles** средства просмотра Cluster Viewer

Распределение дискретных атрибутов показывается при помощи раскрашенного столбика. По умолчанию вкладка **Cluster Profile** показывает четыре наиболее часто встречающиеся категории выборки, а все остальные группирует в пятую категорию. Вы можете изменить количество категорий путем увеличения или уменьшения числа в текстовом поле **Histogram Bars**. Перейдите к строке атрибута **IncomeGroup**, а затем прокрутите вправо, чтобы увидеть изменения распределения **IncomeGroup** по кластерам. Сравните столбики гистограммы для кластеров **High Profit**, **High Income**, **Car Owner** и **High Profit, Lower Income**, **Less Likely Car Owner**. (Вам может понадобиться

нажать кнопку **Refresh Viewer Content** в верхней части вкладки **Mining Model Viewer**, чтобы увидеть новые названия ваших кластеров). Столбик гистограммы для всей выборки разделен на три почти равных сегмента, поскольку выборка имеет почти равное количество клиентов в группах с высокими, средними и низкими доходами. Однако сегмент *High Income* столбика гистограммы для кластера *High Profit, High Income, Car Owner* больше, чем сегменты других доходов в этом кластере (поскольку в нем относительно больше клиентов, которые находятся в группе *High Income*). Сегмент *High Income* столбика гистограммы для кластера *High Profit, Lower Income, Less Likely Car Owner* практически неразличим. Если вы наведете курсор вашей мыши на столбик гистограммы, то вы увидите, что сегмент *High Income* составляет 0.3%.

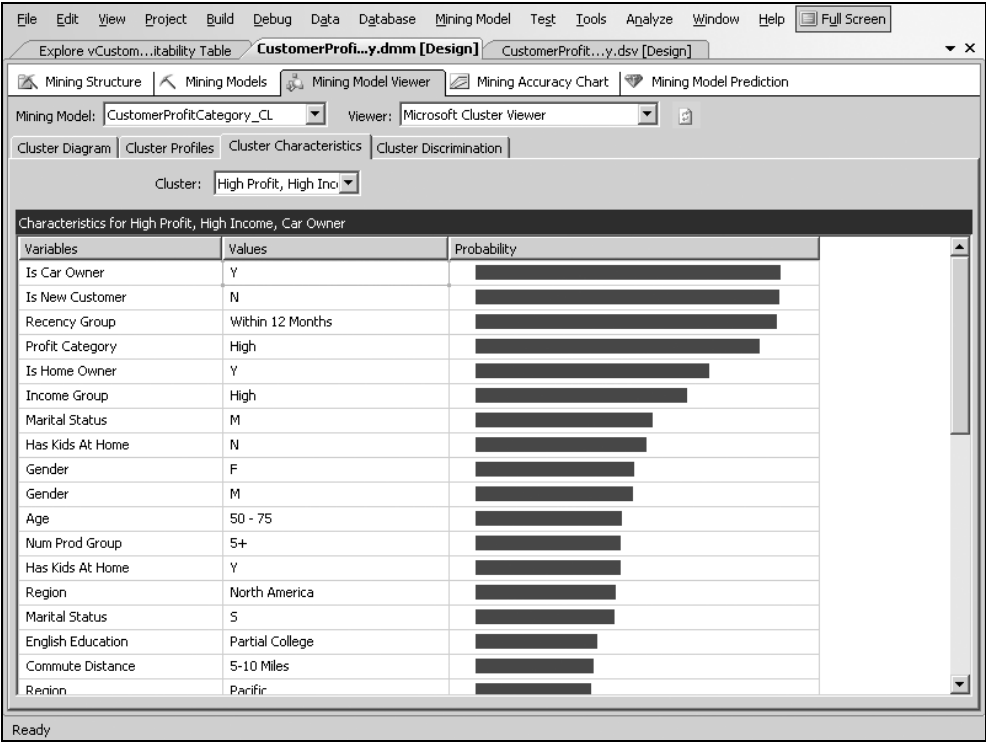
Распределение непрерывных характеристик показано при помощи полосы и ромбовидной диаграммы. Перейдем к строке с атрибутом возраста. Черная полоска представляет диапазон возрастов кластера, причем серединное значение находится в середине полоски. Середина ромба представляет средний возраст клиентов в кластере. Стандартное отклонение (статистический показатель разброса данных) возрастов кластера указано размером ромба. Сравните ромбовидную диаграмму выборки и кластеров *High Profit, High Income, Car Owner* и *High Profit, Lower Income, Less Likely Car Owner*. Более высокий и более плоский ромб означает, что клиенты этого кластера относительно старше, чем клиенты всей выборки, и что вариации возраста в кластере меньше, чем по всей выборке. Вы можете навести указатель мыши на ромбовидную диаграмму и увидеть точный средний возраст и стандартное отклонение по данной диаграмме.

По умолчанию характеристики кластера показываются в убывающем алфавитном порядке. Найдите кластер *High Profit, High Income, Car Owner* и щелкните по заголовку его столбца. Атрибуты кластера будут переупорядочены в соответствии с теми характеристиками, которые наиболее сильно отличают его от общей выборки. Щелкните по кластеру *High Profit, Lower Income, Less Likely Car Owner* — чтобы увидеть, как изменится порядок его характеристик.

Вначале кластеры упорядочиваются по размеру слева направо. Вы можете изменить порядок кластеров при помощи их перетаскивания. На рис. 18.19 показаны упорядоченные по важности атрибуты для кластера *High Profit, High Income, Car Owner* и кластеры, упорядоченные по проценту клиентов в высокодоходной категории.

### Характеристики кластеров

Вкладка **Cluster Characteristics** позволяет вам изучить атрибуты конкретного кластера. По умолчанию показаны атрибуты всей выборки данных модели. Для того чтобы увидеть атрибуты определенного кластера, вы можете выбрать его в выпадающем списке **Cluster**. Например, если вы выберете кластер *High Profit, High Income, Car Owner*, то атрибуты будут переупорядочены в соответствии с вероятностью их появления в этом кластере (рис. 18.20).



**Рис. 18.20.** Вкладка **Cluster Characteristics** средства просмотра Cluster Viewer

Атрибут с самой высокой вероятностью в этом кластере — это владение автомобилем. Иначе говоря, с вероятностью 96% клиент из этого кластера является владельцем автомобилем.



### Сравнение кластеров

Вкладка **Cluster Discrimination** полезна для сравнения двух кластеров (чтобы понять, что их отличает больше всего). Выберите High Profit, High Income, Car Owner в выпадающем списке **Cluster 1** и High Profit, Lower Income, Less Likely Car Owner в выпадающем списке **Cluster 2**. Атрибуты, которые наиболее важны для дифференцирования двух кластеров, показаны в порядке их важности. Полоска обозначает кластер, в котором вы найдете данный атрибут, а размер полоски указывает важность атрибута. На рис. 18.21 два наиболее важных дифференцирующих атрибута выбранных кластеров — это группы с высоким и низким доходом.

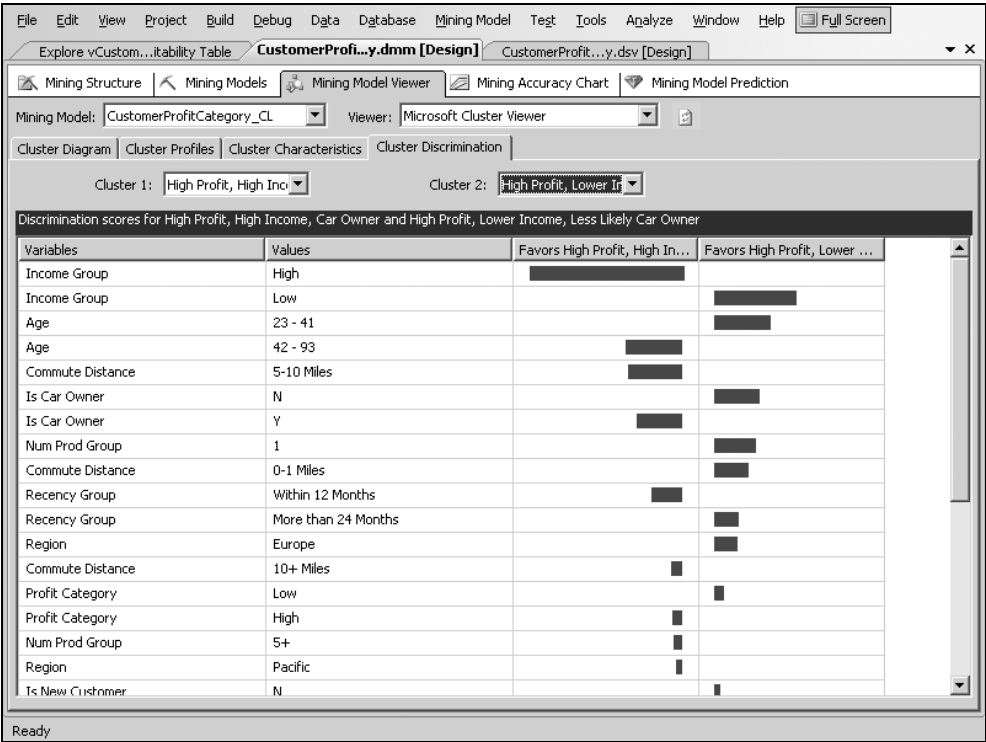


Рис. 18.21. Вкладка **Cluster Discrimination** средства просмотра Cluster Viewer

Мы подробно описали средство просмотра Cluster Viewer и составляющие его вкладки, но не забывайте о том, что в нашей структуре есть модели анализа, которые используют алгоритмы Decision Trees и Naive Bayes (а не алго-

ритм Clustering). Нам необходимо понять и то, как использовать средства просмотра Microsoft Tree Viewer и Naive Bayes Viewer.

## Использование средства просмотра Tree Viewer

Для просмотра результатов модели Decision Trees выберите CustomerProfitCategory\_DT в выпадающем списке **Mining Model**. Microsoft Tree Viewer предлагает два представления результатов оценки моделей: дерево решений и представление сети зависимостей. В представлении дерева решений создается отдельное дерево для каждой прогнозируемой переменной. Вы можете выбрать дерево для просмотра при помощи выпадающего списка **Tree**. В нашей модели у нас есть два дерева, одно из них — для прогнозирования доходности клиента, а другое — для прогнозирования количества покупаемых продуктов.

### Дерево решений

Выберите ProfitCategory в выпадающем списке **Tree** для показа дерева для этой переменной. Средство просмотра Tree Viewer дает поперечное представление дерева решений с корневым узлом слева. Деревья решений ветвятся в каждом узле в соответствии с атрибутом, который в этом узле является самым важным для определения прогнозируемого столбца. Следовательно, первое разбиение после корневого узла **All** является самой важной характеристикой для определения доходности клиента. Как показано на рис. 18.22, первое разбиение в дереве ProfitCategory делается по тому обстоятельству, является ли данный клиент новым клиентом.

Средство просмотра Tree Viewer показывает столбчатую диаграмму для ProfitCategory в каждом узле дерева и использует фоновую заливку узла для указания концентрации вариантов в этом узле. По умолчанию заливка узлов дерева указывает процент выборки в этом узле. Заливка контролируется выбором в выпадающем списке **Background**, где вы можете выбрать конкретное значение для переменной ProfitCategory. Если прогнозируемая переменная является непрерывной, то в каждом узле дерева будет ромбовидная диаграмма с указанием среднего и срединного значений, стандартного отклонения, а также диапазона прогнозируемой переменной.

По умолчанию показывается до трех уровней дерева. Вы можете изменить это значение по умолчанию при помощи выпадающего списка **Default Expansion**. Изменение здесь приводит к изменению значения по умолчанию для

всех деревьев решений, просматриваемых в данной структуре. Для просмотра большего количества уровней дерева без изменения значения по умолчанию используйте движок **Level** или разверните ветви дерева при помощи нажатия на знак плюса в узле дерева. Узел без знака плюса — это листовой узел, у которого нет дочерних ветвей, и его поэтому развернуть нельзя. Ветви заканчиваются тогда, когда больше нет входных атрибутов, которые помогают определить значение прогнозируемой переменной в данном узле. Длина ветвей дерева различна, поскольку это зависит от данных.

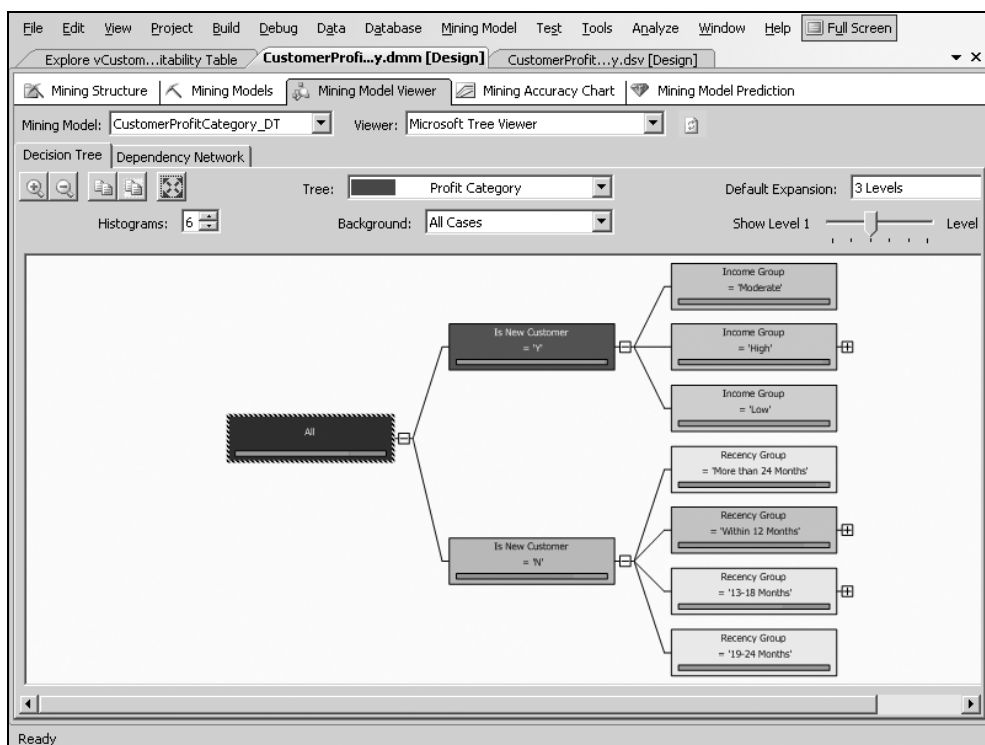


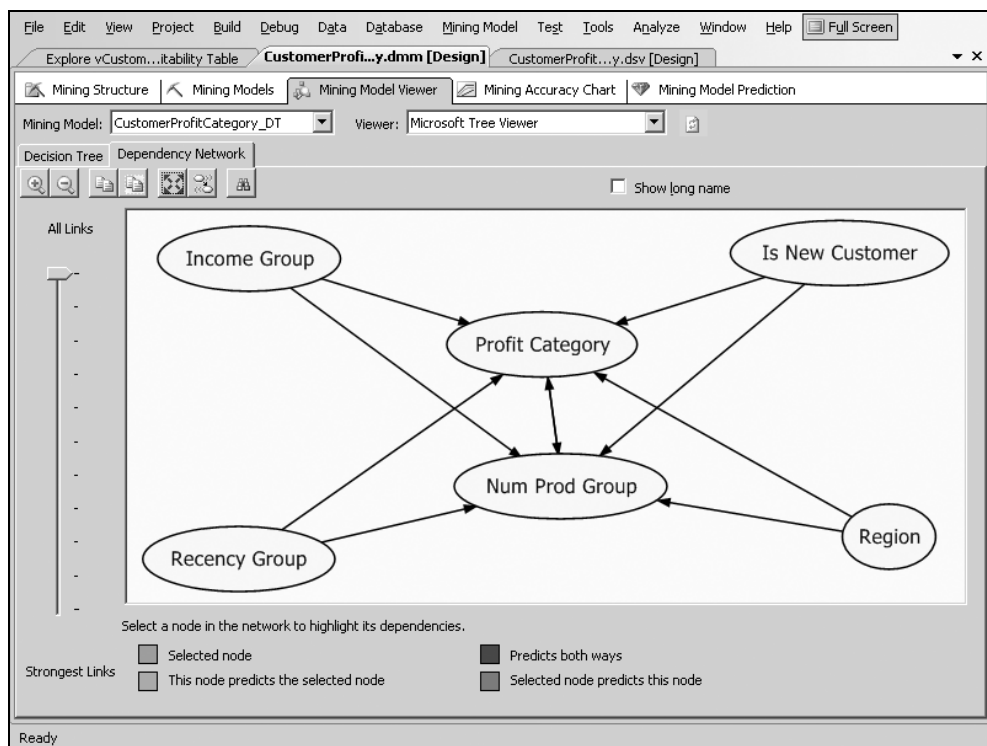
Рис. 18.22. Вкладка **Decision Tree** средства просмотра Tree Viewer

## Сеть зависимостей

Вкладка **Dependency Network** для модели `CustomerProfitCategory_DT` показана на рис. 18.23. Каждый пузырь в сети зависимостей представляет собой прогнозируемую переменную или атрибут, который определил разбиение в

дереве решений для прогнозируемой переменной. Соединяющие атрибуты стрелки указывают направление связи между переменными.

Стрелка к атрибуту указывает, что он прогнозируется, а стрелка от атрибута означает, что он прогнозирует. Двусторонняя стрелка, соединяющая Profit Category и Num Prod Group, означает, что они прогнозируют друг друга (каждый из них разбивает узел в дереве решений другого). Если бы мы настроили тип использования обеих этих переменных в PredictOnly или Input, то мы бы не смогли сделать такое наблюдение.



**Рис. 18.23.** Вкладка **Dependency Network** средства просмотра Tree Viewer

Щелкните по Profit Category. Это активирует цветовое кодирование (описанное в легенде под изображением сети зависимостей) для обозначения прогнозирования данным атрибутом Profit Category (или наоборот). Движок слева от сети зависимостей позволяет вам отфильтровать связи между переменными. Если вы передвинете движок вниз, то вы увидите, что

самым сильным прогнозатором для Profit Category является новизна клиента. Медленно передвиньте движок вверх. Следующим станет красно-оранжевый пузырь Recency Group; это означает, что давность последней покупки является вторым по важности прогнозатором доходности клиента (после новизны клиента).

Теперь переместите движок вниз и щелкните по Num Prod Group. Цветовое кодирование теперь показывает прогнозаторов для этой переменной. Даже если никакие прогнозаторы не отмечены цветом, вы можете начать медленно двигать движок вверх и увидите, что первым становится розовый пузырь Profit Category, показывая, что самым важным определяющим фактором для количества покупаемых продуктов является доходность клиента.

## Средство просмотра Naive Bayes Viewer

Нам еще предстоит просмотреть нашу модель Naive Bayes. Для этого просто выберите CustomerProfitCategory\_NB в выпадающем списке **Mining Model**, что откроет Naive Bayes Viewer. Окно имеет четыре вкладки: **Dependency Network**, **Attribute Profiles**, **Attribute Characteristics** и **Attribute Discrimination**. Наш опыт работы со средствами просмотра Cluster Viewer и Tree Viewer здесь нам очень пригодится, поскольку мы уже видели несколько отличающиеся версии всех этих четырех вкладок.

## Сеть зависимостей

Вкладка **Dependency Network** Naive Bayes Viewer выглядит и работает точно так же, как и вкладка **Dependency Network** Tree Viewer, но дает несколько иные выводы. Как и в предыдущем случае, движок на левой стороне вкладки отфильтровывает самые сильные определяющие характеристики доходности клиента. Если вы переместите его в самое нижнее положение, а затем медленно потянете вверх, то увидите, что в соответствии с моделью Naive Bayes самыми важным прогнозатором доходности клиента является новизна клиента, за которой следует количество купленных клиентом продуктов.

Это отличается от определения доходности клиента по модели дерева решений. Модель дерева решений предполагает, что после новизны клиента самым важным прогнозатором доходности клиента является давность последней покупки. Это подчеркивает важное обстоятельство: даже если средства просмотра для двух моделей используют одну и ту же технологию визуали-

зации, все равно каждая из них использует свой алгоритм и может дать несколько разные выводы.

Профили и характеристики атрибутов,  
а также их сравнение

Вкладки **Attribute Profiles**, **Attribute Characteristics** и **Attribute Discrimination** Naive Bayes Viewer функционально эквивалентны вкладкам **Cluster Profiles**, **Cluster Characteristics** и **Cluster Discrimination** Cluster Viewer. Однако вместо сравнения распределения входных атрибутов по кластерам они позволяют выполнить сравнение распределения входных атрибутов по различным значениям прогнозируемого столбца (в нашем случае это столбец Profit Category).

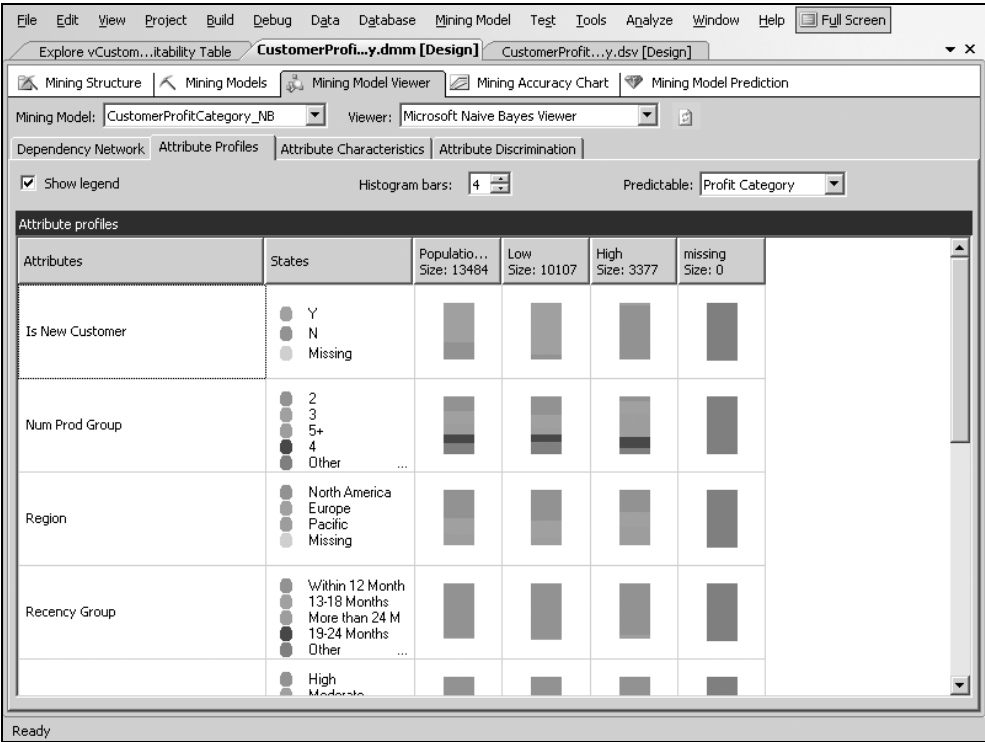


Рис. 18.24. Вкладка **Attribute Profiles** средства просмотра Naive Bayes Viewer

Щелкните по вкладке **Attribute Profiles**, а затем выберите группу **Low profit**. Как показано на рис. 18.24, в соответствии с нашей моделью **Naive Bayes** самыми важными характеристиками для прогнозирования низкой доходности клиента являются новизна клиента, количество купленных товаров, а также регион.

## Проверка и сравнение моделей интеллектуального анализа данных

Мы обработали модели анализа и проанализировали их прогнозирующее содержимое. И хотя легко придти к выводу, что давность клиента является важным определяющим фактором его доходности, модели несколько отличаются в своих выводах относительно остальных характеристик клиентов (или их покупательского поведения), являющихся важными индикаторами доходности. Учитывая наличие этих различий, как определить, какой из этих вторичных прогнозов правильный? Это можно сделать при помощи встроенных "диаграмм точности прогнозов", матрицы классификации и новых средств перекрестной проверки, которые облегчают проверку относительной точности моделей анализа вашей структуры и сравнение их прогнозирующих способностей.

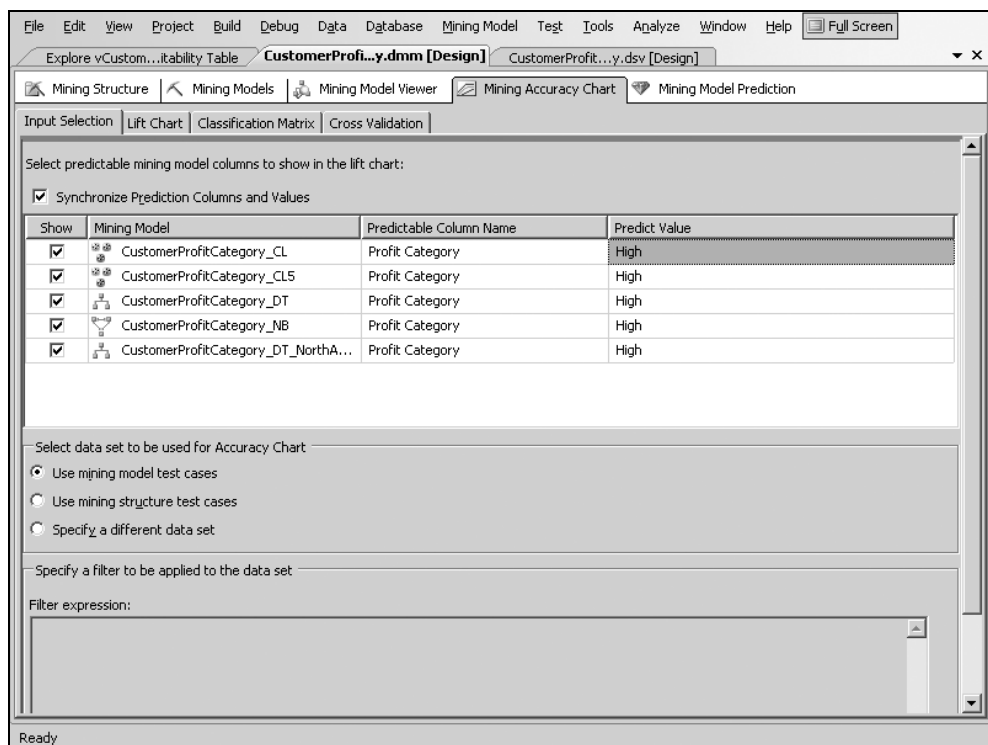
Щелкните по представлению **Mining Accuracy Chart**. Вы окажетесь на вкладке **Input Selection**, которая имеет три части. В верхней части вы указываете модели, прогнозируемые столбцы и значения прогнозируемых столбцов (по которым будут строиться диаграммы точности прогнозов и матрицы классификации, а также выполняться перекрестная проверка). В средней части вы указываете набор данных, который должен использоваться для тестирования ваших моделей анализа. В нижней части вы можете указать фильтры для набора данных, выбранного вами в средней части.

Убедитесь в том, что в верхней части вкладки установлены флажки **Synchronize Prediction Columns and Values**. По умолчанию выбраны все модели интеллектуального анализа данной структуры, а состояние для прогнозируемого столбца не указано. Щелкните по любой ячейке в столбце **Predict Value** сетки, а затем выберите опцию **High** в появившемся выпадающем списке. Поскольку установлено **Synchronize Prediction Columns And Values**, то для всех моделей выставляется значение **high**.

В средней части вкладки у вас есть три варианта для тестового набора: тестовый набор модели, тестовый набор структуры, либо другой набор данных.

В Analysis Services версии 2005 г. выбора не было, единственным вариантом был другой набор данных. Обычно для этого использовался тестовый набор данных, который создавался при помощи задачи Random Sampling Task служб Integration Services.

В Analysis Services 2008 у нас есть возможность использовать контрольный набор, который мы создали при генерации структуры интеллектуального анализа. Более того, если в каких-то моделях указаны фильтры, то мы можем дать указание службам Analysis Services применить эти фильтры и к контрольному набору. Для этого выберите вариант **Use Mining Model Test Cases**. Модель CustomerProfitCategory\_DT\_NorthAmerica будет тестироваться с использованием клиентов из Северной Америки (из контрольного набора). Для тестирования остальных моделей структуры будет использоваться полный контрольный набор.

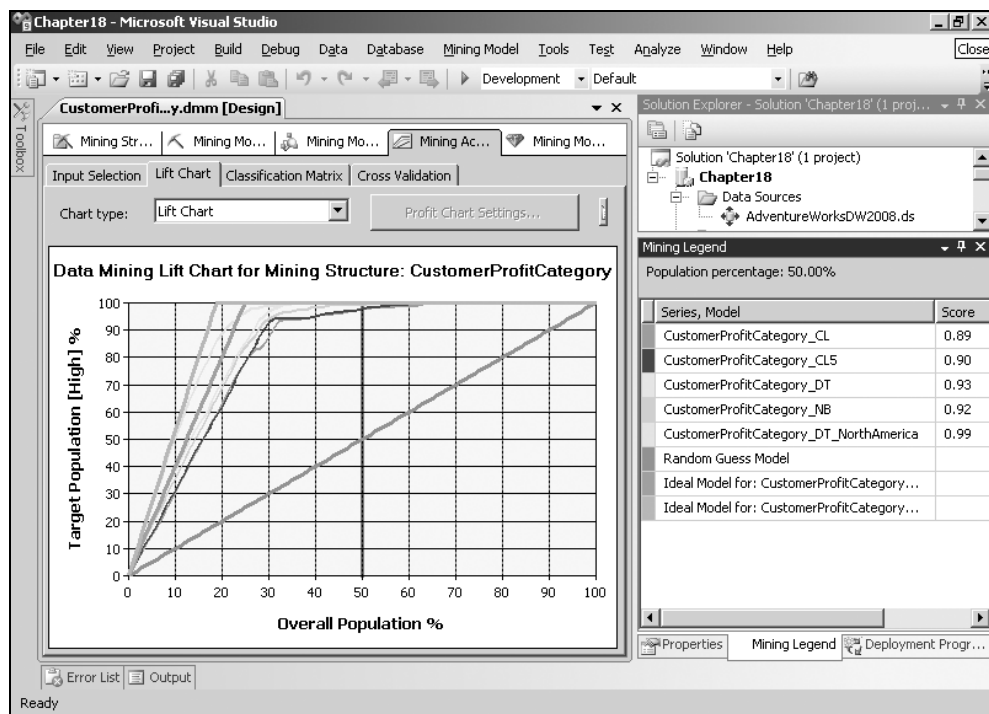


**Рис. 18.25.** Вкладка **Input Selection** визуального конструктора Microsoft Mining Accuracy Chart



Убедитесь, что сделанные вами установки совпадают с показанными на рис. 18.25.

Теперь щелкните по вкладке **Lift Chart**. После загрузки диаграммы убедитесь, что в выпадающем списке **Chart type** выбран вариант **Lift Chart**. Насколько хорошо модели прогнозируют высокодоходных клиентов, показано на рис. 18.26.



**Рис. 18.26.** Диаграмма точности прогнозов со сравнением моделей доходности клиентов

Ось  $x$  диаграммы точности прогнозов показывает процент общей выборки, а ось  $y$  показывает процент выборки, являющийся высокодоходными клиентами. Вспомним, что мы определили как высокодоходных тех клиентов, которые попадают в верхний квартиль клиентов. В идеальной модели (которая абсолютно точно прогнозирует, кто является высокодоходным клиентом) вам нужно просмотреть только 25% общей выборки для того, чтобы определить 100% высокодоходных клиентов. При помощи легенды Mining Legend (в правой части диаграммы) найдите линию, соответствующую идеальной модели. (Это идеальная линия для моделей, которые не фильтруют выборку,

например, для `CustomerProfitCategory_CL`). Обратите внимание, что она достигает верха диаграммы на отметке 25% по оси *x*. Прямая диагональная линия на рисунке представляет собой линию случайного угадывания. При случайном угадывании вам понадобилось бы просмотреть 100% общей выборки, чтобы определить 100% высокодоходных клиентов. Модели интеллектуального анализа дают улучшение по сравнению со случайным угадыванием, модель дерева решений *Decision Tree* (с самым высоким показателем в легенде *Mining Legend*) является самой точной моделью для полной выборки, за ней следует модель *Naive Bayes*. Модель дерева решений лучше прогнозирует доходность для клиентов из Северной Америки, чем для всей выборки.

### ПРИМЕЧАНИЕ

Представление *Mining Accuracy* предлагает также возможность выполнения перекрестной проверки. Перекрестная проверка — это обычный способ тестирования устойчивости ваших моделей анализа. Когда вы выполняете перекрестную проверку, источник данных разделяется на указанное количество поднаборов (разделов). Каждый раздел используется как контрольный набор для тестирования модели, которая обучается по остальной части набора данных. Для каждой обученной и протестированной модели генерируются показатели точности. Для того чтобы модель могла считаться надежной, выдаваемые ею (после обучения на различных случайных образцах данных) результаты должны быть непротиворечивыми. Встроенная возможность перекрестной проверки является новой для версии *SQL Server 2008*, она значительно упрощает процесс перекрестной проверки для разработчиков интеллектуального анализа данных. Подробное обсуждение данной функции выходит за рамки этой книги. Подробности вкладки **Cross Validation** и интерпретацию результатов вы можете найти в документации *SQL Server Books Online*.

## Вложенные таблицы

Вложенные таблицы — это, вероятно, одна из наиболее запутанных функций интеллектуального анализа данных служб *Analysis Services*. Однако если вы их поймете, то они станут для вас неотъемлемым инструментом интеллектуального анализа. Вложенные таблицы позволяют вам включать транзакционные данные в ваши модели анализа (без изменения структуры ваших данных).

Например, предположим, что мы хотим включить в наш анализ доходности данные о типах продуктов, купленных клиентами. Компания *Adventure Works* продает четыре категории продуктов: велосипеды, одежду, аксессуары и комплектующие. Какие продукты покупают высокодоходные клиенты? Яв-

ляются ли покупатели велосипедов высокодоходными клиентами? А покупатели одежды, аксессуаров и компонентов? Хотя мы могли бы добавить (в таблицу вариантов) четыре новых столбца для указания купленных клиентом типов продуктов, что будет в том случае, если впоследствии компания Adventure Works решит продавать 10 категорий продуктов? Или если мы захотим узнать, прогнозируется ли доходность клиента конкретной моделью велосипеда (которых 50 штук)? Добавление в нашу таблицу вариантов отдельного столбца для каждого из этих значений совершенно очевидно было бы неэффективным (как в смысле объема хранения этой таблицы, так и в плане объема работы ETL, необходимой ее создания). Вложенные таблицы избавляют от такой неэффективности, позволяя вам оставить без изменений существующую структуру транзакционных данных.

Для того чтобы лучше разобраться с этим в контексте созданной нами структуры анализа и моделей анализа, давайте рассмотрим инструкцию CREATE для представления vCustomerPurchases (листинг 18.2).

#### Листинг 18.2. Представление с транзакционными данными

```
CREATE VIEW [dbo].[vCustomerPurchases]
AS
SELECT
    f.SalesOrderNumber
    ,f.SalesOrderLineNumber
    ,f.CustomerKey
    ,d.FullDateAlternateKey
    ,pc.EnglishProductCategoryName AS ProductCategory
    ,psc.EnglishProductSubcategoryName AS ProductSubcategory
    ,isnull(p.[ModelName], p.[EnglishProductName]) AS [Model]
    ,f.OrderQuantity
    ,Row_Number ( ) OVER (
        Partition BY f.CustomerKey
        ORDER BY d.FullDateAlternateKey, f.SalesOrderLineNumber)
    AS PurchaseSequence
FROM [dbo].[FactInternetSales] AS f
INNER JOIN dbo.dimProduct AS p
ON f.ProductKey = p.ProductKey
INNER JOIN dbo.dimProductSubCategory AS psc
ON p.ProductSubCategoryKey = psc.ProductSubcategoryKey
INNER JOIN dbo.dimProductCategory AS pc
```

```

ON psc.ProductCategoryKey = pc.ProductCategoryKey
INNER JOIN dbo.dimDate AS d
ON f.OrderDateKey = d.DateKey

```

Это представление создается простой инструкцией `SELECT` из таблицы фактов с соединением с несколькими таблицами измерений (для выборки названий и категорий продуктов). Столбец `PurchaseSequence`, который использует функцию ранжирования T-SQL `ROW_NUMBER`, мы обсудим в следующем разделе.

В табл. 18.1 показаны атрибуты клиентов, а также транзакционные данные о покупках клиентов. Предположим, что мы хотим указать в наших моделях анализа *Melvin* — как покупателя аксессуаров, а *Jennifer* — как покупателя и велосипеда, и аксессуаров (не добавляя явным образом атрибуты *Bike Buyer* и *Accessory Buyer* в данные профиля наших покупателей (которые у нас содержатся в `vCustomerProfitability`). Добавление в структуру анализа транзакционных данных (в нашем случае это `vCustomerPurchases`) в виде вложенной таблицы позволит нам это сделать.

**Таблица 18.1. Вложенные данные**

| CustKey | Имя      | Доход-ность | Пол | Группа доходов | Категория продукта | Название продукта              | Seq |
|---------|----------|-------------|-----|----------------|--------------------|--------------------------------|-----|
| 11580   | Melvin   | Низкая      | М   | Низкие         | Аксессуары         | Sport-100 Helmet, Blue         | 1   |
| 11036   | Jennifer | Низкая      | Ж   | Средние        | Велосипеды         | Mountain-200 Silver, 46        | 1   |
|         |          |             |     |                | Аксессуары         | Sport-100 Helmet, Black        | 2   |
| 11376   | Lance    | Низкая      | М   | Средние        | Одежда             | Half-Finger Gloves, M          | 1   |
|         |          |             |     |                | Одежда             | Short-Sleeve Classic Jersey, M | 2   |
| 11136   | Brianna  | Высокая     | Ж   | Средние        | Велосипеды         | Mountain-200 Black, 38         | 1   |
|         |          |             |     |                | Аксессуары         | Fender Set, Mountain           | 2   |
|         |          |             |     |                | Одежда             | Classic Vest, M                | 3   |

Для использования `vCustomerPurchases` в качестве вложенной таблицы нашей структуры анализа мы должны сначала добавить его в наше представление источника данных:

1. Дважды щелкните по `CustomerProfitability.dsv` в окне **Solution Explorer** (чтобы отобразить его в визуальном конструкторе представлений источников данных).
2. Нажмите кнопку **Add/Remove Objects** панели инструментов (первая слева), либо щелкните правой кнопкой по окну визуального конструктора и выберите пункт **Add/Remove Tables**, либо выберите пункт **Add/Remove Tables** в меню **Data Source View**. Любое из этих трех действий откроет диалоговое окно **Add/Remove Tables**.
3. Найдите `vCustomerPurchases` в списке **Available Objects**, перенесите его в список **Included Objects**, а затем нажмите **OK**.
4. Для завершения изменений в нашем представлении источника данных мы должны установить связь между `vCustomerPurchases` и `vCustomerProfitability`. Щелкните по столбцу `CustomerKey` в `vCustomerPurchases`, а затем перетащите его в столбец `CustomerKey` в `vCustomerProfitability`.
5. Сохраните ваши изменения. Визуальный конструктор представлений источников данных должен теперь выглядеть, как показано на рис. 18.27.
6. В визуальном конструкторе структур анализа щелкните по вкладке **Mining Structure**.
7. Найдите (но не нажимайте) кнопку **Add A Mining Structure Nested Table Column** панели инструментов (четвертая слева).

Службы `Analysis Services` считают вложенную таблицу дополнительным столбцом структуры анализа, то есть считают ее столбцом.

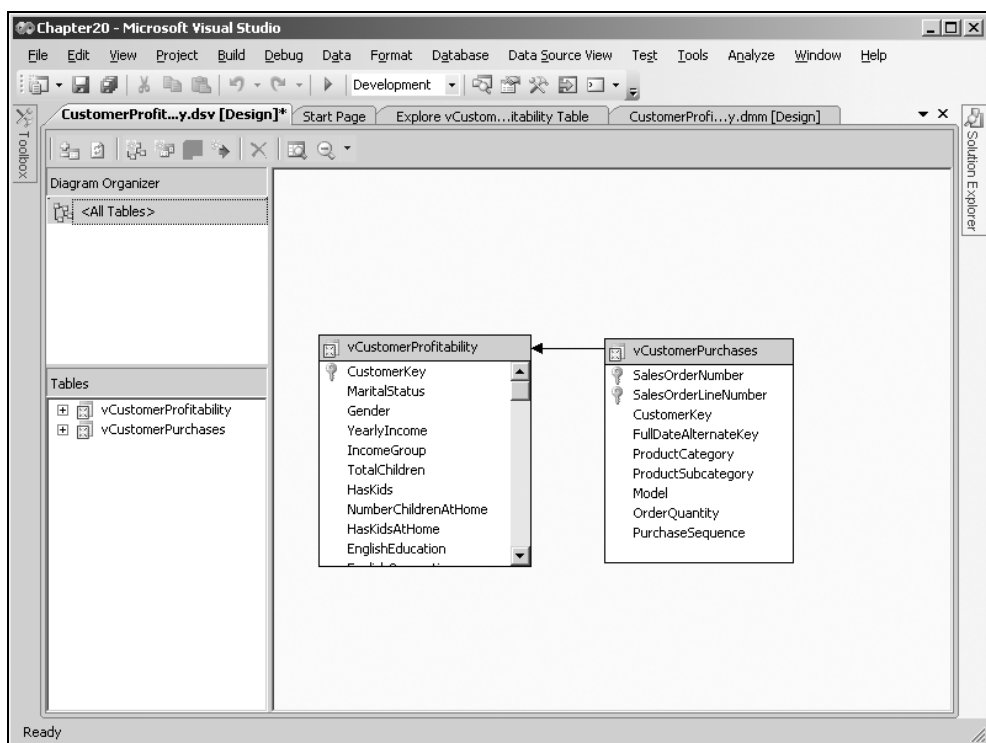
### **ПРИМЕЧАНИЕ**

В соответствии с таким типизированием вложенных таблиц мы изобразили `vCustomerPurchases` в табл. 18.1 как еще один пользовательский атрибут (хотя и иерархический) — наряду с доходностью, полом и группой доходов. Мы сделали так потому, что знали о том, что мы будем использовать `vCustomerPurchases` как столбец вложенной таблицы.

8. Нажмите кнопку **Add A Mining Structure Nested Table Column**. После этого будет показано диалоговое окно **Select A Nested Table Key Column**.

Определение ключа вложенной таблицы может поначалу показаться нелогичным; первым вашим побуждением может быть выбор в качестве ключа вложенной таблицы `CustomerKey`, `SalesOrderNumber`, или комбинации `SalesOrderNumber` и `SalesOrderLineItem`. На самом деле ни один из этих вариантов не является подходящим ключом для вложенной таблицы.

Когда вы указываете столбец в качестве ключевого столбца вложенной таблицы, то различные значения этого столбца становятся атрибутами вашей модели анализа. Поэтому правильным ключом для вложенной таблицы в этой ситуации является `ProductCategory`, возможными значениями которого являются `Bike`, `Clothing` и `Accessory`.



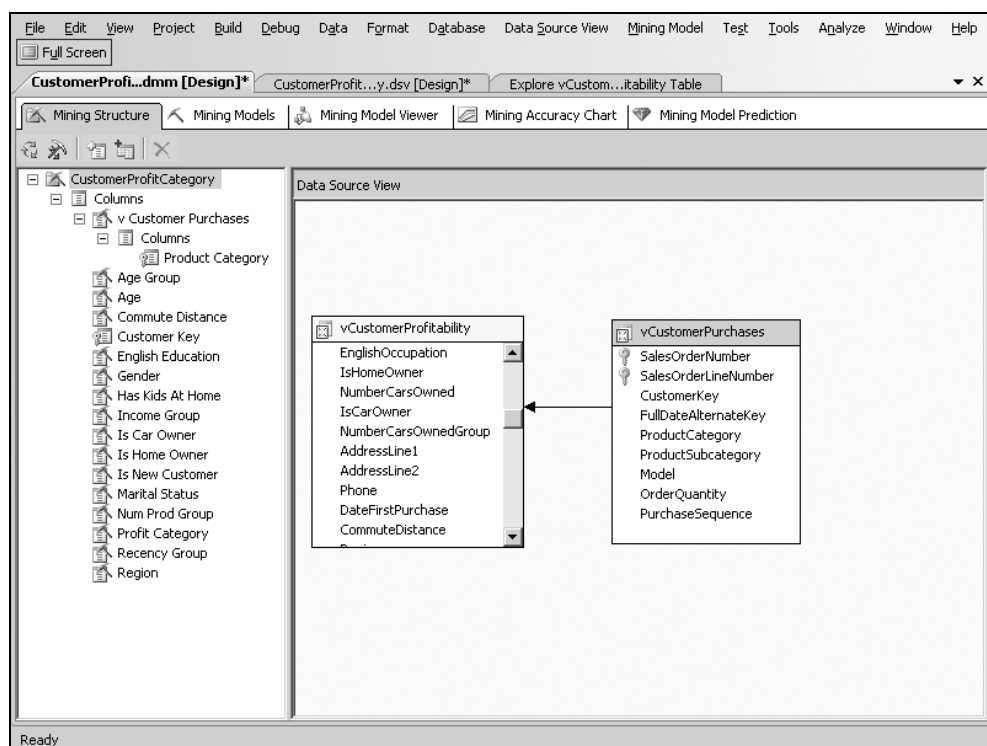
**Рис. 18.27.** Транзакционные данные по покупкам клиентов в представлении источника данных, которое будет использоваться как исходная таблица для вложенной таблицы модели анализа

9. Выберите `vCustomerPurchases` в списке **Nested Table**, а затем выберите `ProductCategory` в списке **Source Column**. Нажмите **OK**.

Как вы увидите после модификации и обработки моделей анализа, выбор `ProductCategory` в качестве ключевого столбца вложенной таблицы добавляет в наши модели анализа такие атрибуты, как `vCustomerPurchases (Bike)` и `vCustomerPurchases (Clothing)`.

Обратите внимание, что вложенная таблица была добавлена в древовидное представление структуры анализа как столбец структуры `CustomerProfitCategory` (под названием `v Customer Purchases`). В отличие от других столбцов структуры анализа рядом с ней имеется значок плюса.

10. Для того чтобы развернуть таблицу, нажмите этот значок плюса. Теперь разверните столбцы вложенной таблицы. Вкладка **Mining Structure** должна выглядеть так, как показано на рис. 18.28.



**Рис. 18.28.** Структура анализа со столбцом вложенной таблицы

Новые столбцы в существующей структуре анализа игнорируются существующими моделями. Поэтому вы должны указать использование новых столбцов в тех моделях, где вы собираетесь их применять:

- 1. Щелкните по вкладке **Mining Models**, а затем щелкните по ячейке на пересечении столбца v Customer Purchases ("родительский" столбец) и модели CustomerProfitCategory\_CL.
- 2. Нажмите стрелку выпадающего списка в ячейке для просмотра элементов этого списка, а затем выберите **Input**. Щелкните по значку плюса рядом с v Customer Purchases. Вы увидите новую строку для дочернего столбца Product Category (с установленным значением Key).

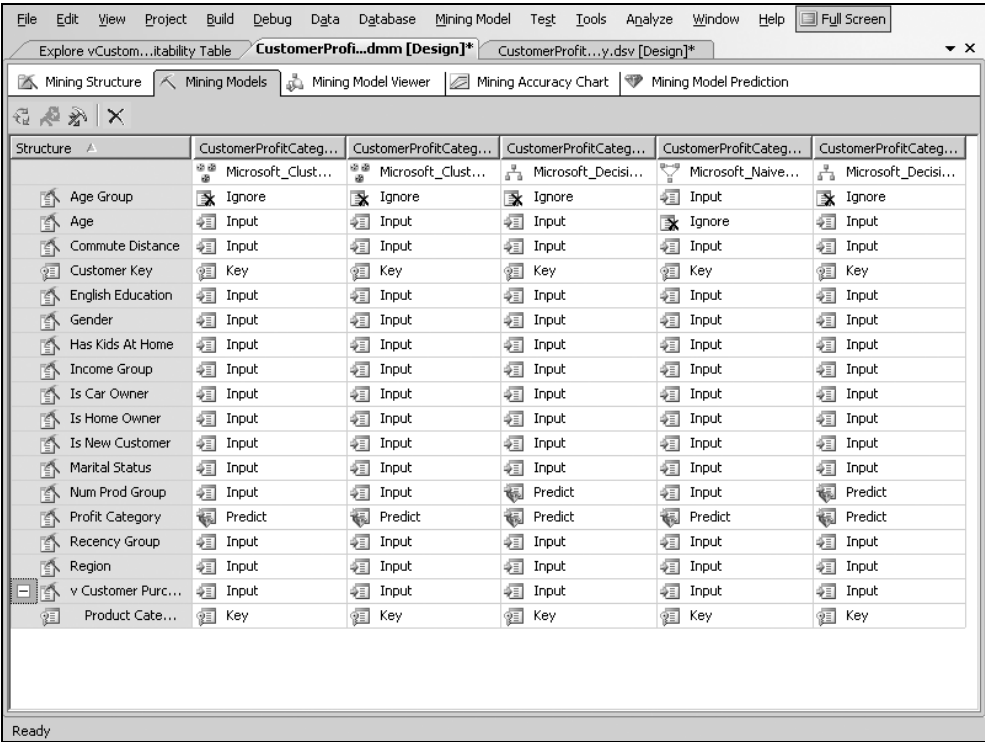


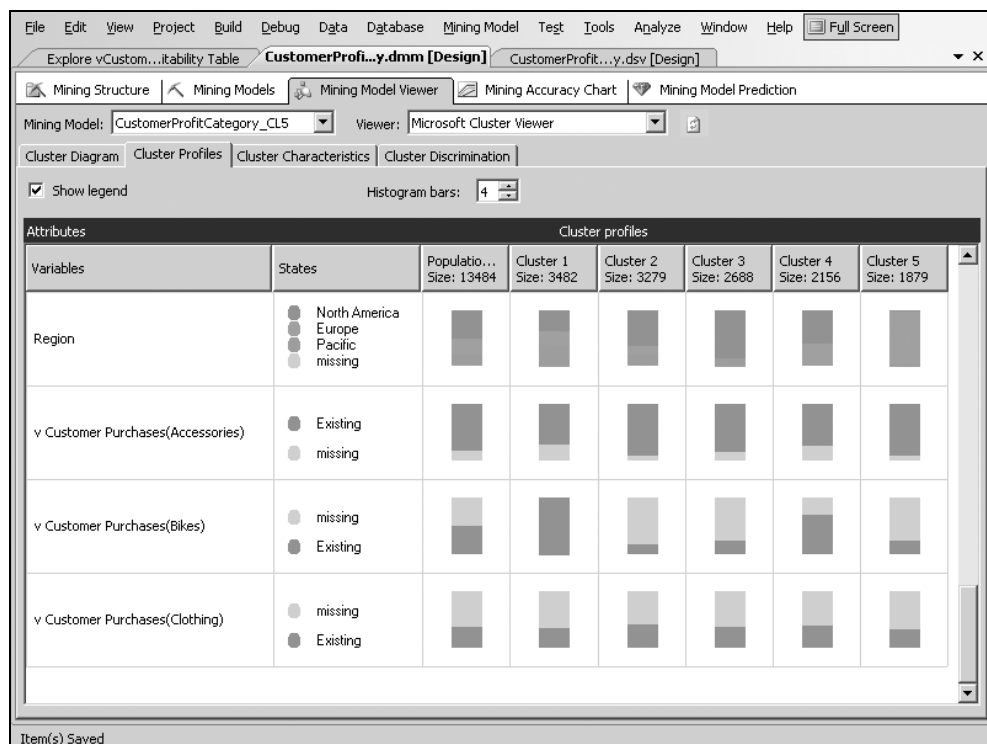
Рис. 18.29. Модели анализа в структуре анализа со столбцом вложенной таблицы

- 3. Повторите процесс изменения использования для v Customer Purchase с **Ignore** на **Input** для всех остальных моделей анализа данной структуры.



Обратите внимание, что каждый раз, когда вы изменяете использование для `v Customer Purchase`, использование для `Product Category` автоматически изменяется с **Ignore** на **Key**. Вкладка **Mining Models** должна выглядеть, как показано на рис. 18.29.

4. Разверните и обработайте ваши модели интеллектуального анализа. Это может занять несколько минут. После завершения обработки щелкните по вкладке **Mining Model Viewer**.



**Рис. 18.30.** Просмотр результатов модели анализа, которая содержит входной столбец вложенной таблицы

5. Щелкните по вкладке **Cluster Diagram** модели `CustomerProfitCategory_CL5` (модель с пятью кластерами). При помощи выпадающего списка **Shading Variable** (обратите внимание на три переменные, которые соответствуют категориям продуктов) измените переменную заливки на `v Customer Purchases(Bikes)`. В выпадающем списке **State** есть два возможных состояния этой переменной: **Existing** и **Missing**. Если ключ `Bikes` существу-

ет во вложенной таблице, то клиент является покупателем велосипеда; если ключа `Bikes` нет, то клиент покупателем велосипеда не является. Выберите **Existing**.

6. Обратите внимание на кластеры 1 и 4, которые имеют самую высокую концентрацию покупателей велосипедов. Измените переменную заливки на `Is New Customer` и выберите состояние `y`. Кластер 4 имеет больше новых клиентов, чем кластер 1. Измените переменную заливки на `Profit Category`, выберите состояние `high` и обратите внимание на то, что два кластера с наивысшими концентрациями покупателей велосипедов имеют также самые высокие концентрации клиентов с высокими доходами.

На рис. 18.30 на вкладке **Cluster Profiles** сравниваются пять кластеров. Когда ключ столбца вложенной таблицы включен в модель анализа, то его значения формируют основу атрибутов существования (которые используются для обучения модели анализа). Значение ключевого столбца (представленное атрибутом существования) показано в скобках после названия столбца вложенной таблицы. Кластер 1 имеет заметно более высокую пропорцию покупателей велосипедов, чем другие кластеры (вторым идет кластер 4). Пропорция покупателей одежды примерно одинакова по всем кластерам.

Используйте средства просмотра моделей анализа для изучения остальных моделей. Обратите внимание на входные атрибуты, добавленные в модели в результате добавления вложенных таблиц. Закройте проект и выйдите из Visual Studio.

## Использование расширений интеллектуального анализа данных

К настоящему моменту вы увидели, как использовать визуальный конструктор структур данных в Visual Studio для создания, обучения и просмотра моделей интеллектуального анализа. Теперь вы увидите, как использовать расширения SQL Data Mining Extensions (DMX) для того, чтобы делать то же самое. DMX является расширением языка SQL для создания и работы с моделями интеллектуального анализа данных в Analysis Services.

Как и в обычном SQL, в DMX есть две категории инструкций: инструкции языка определения данных (data definition language, DDL) и инструкции языка манипулирования данными (data manipulation language, DML). Инструкции DDL используются для создания структур и моделей интеллектуального ана-

лиза данных. Инструкции DML используются для обучения и просмотра моделей анализа, а также для прогнозирования при помощи этих моделей.

Вы можете избежать написания запросов DDL на языке DMX (если будете пользоваться визуальными конструкторами Analysis Services). При помощи этих инструментов можно также автоматически генерировать и запросы прогнозирования на языке DML (скоро мы их изучим). Но так же, как и в случае с реляционными базами данных, не следует полностью полагаться на инструменты при наличии мощного декларативного языка, позволяющего выполнять те же задачи. Сейчас мы опишем написание кода DMX для добавления модели анализа в существующую структуру анализа, создания и обучения моделей кластеризации последовательностей и правил взаимосвязей, а также для выполнения запросов прогнозирования по обученным моделям интеллектуального анализа.

## Моделирование интеллектуального анализа данных при помощи DMX

DMX можно использовать для создания, модификации и обучения структур и моделей анализа точно так же, как они создаются в мастере Data Mining Wizard и модифицируются в визуальном конструкторе интеллектуального анализа данных Data Mining Designer:

1. В SQL Server Management Studio откройте окно **Object Explorer** (если оно не видно). Если ваш сервер Analysis Services не виден в окне **Object Explorer**, добавьте подключение к нему при помощи выпадающего списка **Connect** или соответствующей кнопки панели инструментов.
2. Раскройте узел **Databases**, а затем узел **Chapter18** (нашей базы данных). Обратите внимание на выделенный узел для структур интеллектуального анализа. Раскройте узел **Mining Structures** и убедитесь, что вы видите узел **CustomerProfitCategory** (нашей структуры анализа). Разверните этот узел, а затем раскройте его дочерний узел **Mining Models** (чтобы убедиться, что там имеются узлы для каждой из пяти созданных нами моделей).
3. Нажмите кнопку **Analysis Services DMX Query** на панели инструментов **SQL Server Analysis Services Editors** в Management Studio.

После заполнения диалогового окна **Connect To Analysis Services** откроется специальное окно запросов, в котором вы сможете выполнять инструкции DMX. Окно запросов DMX имеет панель **Tools**, содержащую выпадающий список всех моделей базы данных для всех структур анализа,

вкладку **Metadata** со списком всех столбцов текущей выбранной модели, а также вкладку **Functions**, которая имеет список функций DMX. Кроме того, Template Explorer предлагает несколько шаблонов запросов DMX. (Мы обсудим Template Explorer несколько позже).

- В этой главе мы изучим код нескольких скриптов DMX, содержащихся в проекте Chapter18DMX (принадлежащем решению Management Studio с названием Chapter18.smsssln). Дважды щелкните по файлу **Chapter18.smsssln** в окне Windows Explorer. После этого в SQL Server Management Studio откроется пример решения. Затем откройте первые пять файлов скриптов проекта (пять верхних скриптов в окне **Solution Explorer** в узле **Queries** проекта Chapter18DMX). Ваша среда Management Studio должна теперь выглядеть так, как показано на рис. 18.31.

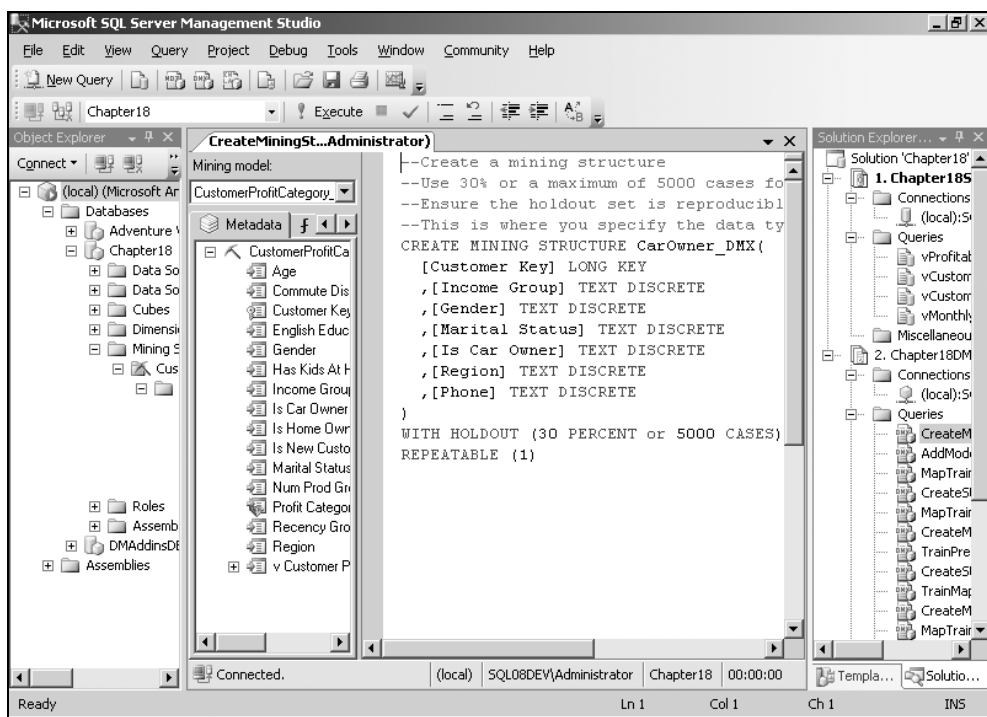


Рис. 18.31. SQL Server Management Studio с открытым окном запроса DMX

При помощи DMX вы можете либо создать структуру анализа (а затем добавить в нее модель анализа), либо создать структуру и модель анализа одним шагом. Скрипты `CreateMiningStructure.dmx`, `AddModelWithDrillthrough.dmx` и

MapTrain.dmx показывают (соответственно), как создать структуру анализа, как добавить в структуру модель и как обработать модель. Скрипты CreateStructureAndModel.dmx и MapTrainOneStep.dmx показывают, как создать структуру и модель анализа одним шагом, а затем обработать структуру и модель.

Зачем же выполнять три скрипта, когда, казалось бы, можно сделать то же самое двумя? Во-первых, вам может понадобиться создать структуру анализа, которая поддерживает множество моделей (когда каждая модель использует только некое подмножество столбцов структуры). Определенные в структуре анализа столбцы формируют домен столбцов, из которого вы можете выбирать столбцы для построения ваших моделей анализа. Однако создание структуры и модели анализа одним шагом заставляет структуру и модель совместно использовать один и тот же набор столбцов.

Простейшая форма инструкции CREATE MINING STRUCTURE такова:

```
CREATE MINING STRUCTURE <structure>
(
    <column name> <data type> <content type>
)
```

Вы должны указать название структуры анализа и определить столбцы для нее. Для каждого столбца структуры анализа вы указываете название, тип данных (например, Text или Long), а также тип содержимого столбца (Key, Continuous или Discrete). Вот пример простой инструкции CREATE MINING STRUCTURE:

```
CREATE MINING STRUCTURE CarOwner_DMx ( [Customer Key] LONG KEY ,
[Income Group] TEXT DISCRETE , [Gender] TEXT DISCRETE , [Marital Status]
TEXT DISCRETE , [Is Car Owner] TEXT DISCRETE , [Region] TEXT DISCRETE,
[Phone] TEXT DISCRETE )
```

Обратите внимание на столбец Phone. Это будет важно при обсуждении детализации модели анализа.

Другая причина использования отдельных инструкций DMX для создания структур и моделей анализа состоит в том, чтобы использовать новую функциональную возможность SQL Server 2008 создания контрольных наборов прямо в Analysis Services. Поддержка создания контрольных наборов есть только в инструкции CREATE MINING STRUCTURE (в инструкции CREATE MINING MODEL ее нет), что (как вы увидите позже) подходит для создания структуры и модели анализа одним шагом.

Параметры контрольного набора указываются при создании структуры анализа следующим образом:

```
CREATE MINING STRUCTURE <structure> ( <column name> <data type> <content type> ) WITH HOLDOUT (<holdout specifier>) REPEATABLE(<holdout seed>)
```

Так же, как и в мастере Data Mining Wizard в Visual Studio, контрольный набор может быть указан как процент от количества вариантов, как максимальное количество вариантов, либо и то, и другое. Если контрольный набор должен быть воспроизводимым, то следует указать для него начальное значение.

Пример инструкции CREATE MINING STRUCTURE с контрольным набором в 30% вариантов (но не более 5000) и начальным значением (равным 1) показан в скрипте CreateMiningStructure.dmx. Откройте скрипт, а затем выполните следующее:

1. Нажмите кнопку **Execute** панели инструментов или выберите команду контекстного меню (пункт **Execute** в меню **Query**), или клавишу <F5>, или комбинацию <Ctrl>+<E>). Появится сообщение о том, что выполнение закончено, и строка состояния в левом нижнем углу окна запросов должна показать, что выполнение запроса закончилось успешно.
2. Перейдите к узлу базы данных **Chapter18** в окне **Object Browser** и обновите ее узел **Mining Structures** (при помощи использования команды **Refresh** ее контекстного меню). Вы должны увидеть только что созданную вами структуру CarOwner\_DMX. Разверните структуру и вы увидите в ней узел **Mining Models**. Если вы раскроете этот узел, то вы увидите, что он пуст; это потому, что в структуре анализа еще нет никаких моделей анализа.

Для добавления модели анализа в существующую структуру анализа вы можете использовать инструкцию ALTER MINING STRUCTURE с выражением ADD MINING MODEL. Давайте рассмотрим простую форму такой инструкции:

```
ALTER MINING STRUCTURE <structure>
ADD MINING MODEL <model> (
    <column name> [<prediction>]
)
USING <algorithm> [(<parameter list>)] [WITH DRILLTHROUGH]
```

Характерный пример инструкции ALTER MINING STRUCTURE дается в скрипте AddModelWithDrillthrough.dmx:

```
ALTER MINING STRUCTURE CarOwner_DMX
ADD MINING MODEL CarOwner_DT (
    [Customer Key]
    , [Income Group]
```

```
, [Gender]
, [Marital Status]
, [Is Car Owner] PREDICT
, [Region]
)
USING Microsoft_Decision_Trees WITH DRILLTHROUGH
```

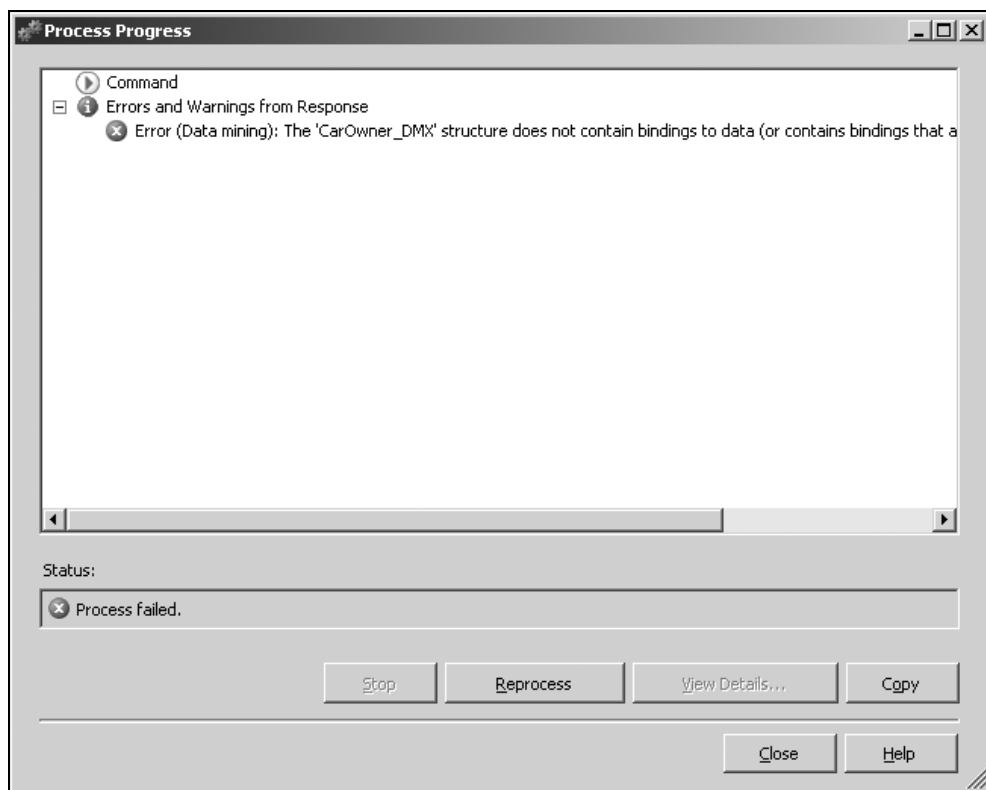
В этом примере в структуру анализа добавляется модель дерева решений для прогнозирования владения автомобилем. Обратите внимание, что в списке столбцов для структуры анализа указан тип данных и тип содержимого столбцов. Тип данных и тип содержимого столбцов являются общими для всех моделей одной и той же структуры анализа. Однако использование столбцов специфично для каждой отдельной модели. Использование столбца (является ли он входным или прогнозируемым) указывается в списке столбцов модели анализа. По умолчанию все столбцы являются входными, поэтому надо указать только те столбцы, которые являются прогнозируемыми. В нашем примере столбец `[Is Car Owner]` помечен как прогнозируемый (поскольку мы хотим использовать алгоритм деревьев решений для прогнозирования владения автомобилем). Вы заметите, что `Phone` (который был определен в структуре анализа) в модели анализа не указан: номер телефона является идентификационной информацией, и это не то, что следует использовать для прогнозирования владения автомобилем. Раздельное создание структуры и модели анализа позволяет структуре иметь идентификационные столбцы (которые не используются в модели анализа). Скоро вы увидите, что это может быть полезно. Давайте продолжим наш пример.

1. Выполните запрос.
2. Обновите и разверните узел **Mining Models** в узле структуры анализа `CarOwner_DMx`. Вы должны увидеть модель `CarOwner_DT`. Щелкните правой кнопкой мыши по этой модели, а затем выберите пункт **Browse**. Вы получите сообщение об ошибке, в котором говорится о том, что модель необходимо обработать. Щелкните по модели правой кнопкой мыши и выберите пункт **Process**.
3. В диалоговом окне **Process Mining Model** нажмите **OK**. Вы увидите сообщение об ошибке, аналогичное показанному на рис. 18.32.

Это сообщение об ошибке появляется потому, что службы Analysis Services не знают исходных данных для структуры анализа. До настоящего момента мы указали некоторые названия столбцов в структуре и модели анализа, но не упомянули об источнике данных для структуры или модели. Привязка столбцов структуры анализа к данным делается при помощи инструкции `INSERT INTO DMx`.

Вот простая форма такой инструкции:

```
INSERT INTO MINING STRUCTURE <structure> (
    <mining structure columns>
)
OPENQUERY(<named datasource>, <query syntax>)
```



**Рис. 18.32.** Сообщение об ошибке, которое появляется при попытке обработать структуру анализа, для которой не указаны привязки столбцов

Для выполнения вашего запроса `INSERT INTO` закройте сообщение об ошибке и диалоговое окно **Process Mining Model**. Закройте пустое окно средства просмотра, а затем откройте файл `MapTrain.dmx` (дважды щелкнув по нему в окне **Solution Explorer**). Пример инструкции `INSERT INTO` для привязки столбцов данных содержится в этом скрипте:

```
INSERT INTO CarOwner_DMx(
    [Customer Key]
```



```

, [Income Group]
, [Gender]
, [Marital Status]
, [Is Car Owner]
, [Region]

, [Phone]
)
OPENQUERY(
    [AdventureWorksDW2008],
    'SELECT
        CustomerKey
        , IncomeGroup
        , Gender
        , MaritalStatus
        , IsCarOwner
        , Region
        , Phone
    FROM AdventureWorksDW2008.dbo.vCustomerProfitability'
)

```

Список столбцов состоит из тех столбцов, которые были указаны в инструкции `CREATE MINING STRUCTURE`. Именованный источник данных был определен в базе данных `Analysis Services` (в нашем случае это `[AdventureWorksDW2008]`). Наш источник данных — это база `SQL Server`, так что код в выражении `OPENQUERY` фактически является инструкцией языка `Transact-SQL (T-SQL)`.

Для продолжения примера сделайте следующее:

1. Выполните скрипт. Инструкция привязывает столбцы структуры анализа и обрабатывает модель анализа.
2. Вернитесь в объект **Object Browser**, щелкните правой кнопкой мыши по узлу **CarOwner\_DT**, а затем выберите **Browse**. После этого в новом окне будет показан **Tree Viewer**. Он идентичен тому, которым вы пользовались в **Visual Studio**.
3. Щелкните правой кнопкой мыши по узлу дерева решений, а затем выберите пункт **Drill Through**. Эта функция была включена при помощи выражения `WITH DRILLTHROUGH`, которое мы указали в скрипте `AddModelWithDrillthrough.dmx`. У вас есть выбор детализации: **Model**

**Columns Only** или **Model And Structure Columns**. В SQL Server 2005 выбора у вас не было, детализацию можно было делать только до столбцов модели анализа. Выберите **Model And Structure Columns**. Это даст нам строки из `vCustomerProfitability` (и в том числе номера телефонов клиентов).

4. Закройте окно средства просмотра и все открытые окна DMX Query.

Давайте теперь изучим, как создать структуру и модель анализа за один шаг. Мы сделаем это при помощи указания в списке названий столбцов, их типа данных, содержимого и использования:

```
CREATE MINING MODEL <model>(  
    <column name> <data type> <Content Type> [<prediction>]  
)  
    USING <algorithm> [(<parameter list>)] [WITH DRILLTHROUGH]
```

Код DMX в файле `CreateStructureAndModel.dmx` является примером такой инструкции:

```
CREATE MINING MODEL CarOwner_DT_OneStep(  
    [Customer Key] LONG KEY  
    , [Income Group] TEXT DISCRETE  
    , [Gender] TEXT DISCRETE  
    , [Marital Status] TEXT DISCRETE  
    , [Is Car Owner] TEXT DISCRETE PREDICT  
    , [Region] TEXT DISCRETE  
)  
    USING Microsoft_Decision_Trees WITH DRILLTHROUGH
```

Поскольку структура и модель создаются за один шаг, то мы опускаем столбец `Phone` (поскольку мы не собираемся использовать его в качестве входного или прогнозируемого столбца нашей модели анализа).

Сделайте дальше следующее:

1. Выполните запрос, а затем обновите узел **Mining Structure** в окне **Object Explorer**. Вы должны увидеть новый узел структуры с названием `CarOwner_DT_OneStep_Structure`. `Analysis Services` прибавляет суффикс `Structure` к указанному в инструкции `CREATE MINING MODEL` названию (для того чтобы отличать структуру анализа от модели анализа).
2. Разверните узел структуры, а затем дочерний узел ее моделей. Вы должны увидеть модель `CarOwner_DT_OneStep`; эта модель и ее родительская структура были созданы одной инструкцией DMX.

3. Для обработки модели CarOwner\_DMx откройте и выполните скрипт MapTrainOneStep.dmx. Вот код DMX из скрипта MapTrainOneStep.dmx:

```
INSERT INTO CarOwner_DT_OneStep(
    [Customer Key]
    , [Income Group]
    , [Gender]
    , [Marital Status]
    , [Is Car Owner]
    , [Region]
)
OPENQUERY (
    [AdventureWorksDW2008] ,
    'SELECT
        CustomerKey
        , IncomeGroup
        , Gender
        , MaritalStatus
        , IsCarOwner
        , Region
    FROM AdventureWorksDW2008.dbo.vCustomerProfitability'
```

Этот запрос похож на запрос в файле MapTrain.dmx, который использовался для обработки структуры анализа CarOwner\_DMx. В MapTrain.dmx в инструкции INSERT INTO была указана структура анализа; в коде DMX файла MapTrainOneStep.dmx указана модель анализа. Указание модели анализа в инструкции INSERT INTO приводит к обработке именно этой конкретной модели. Указание в инструкции структуры анализа приводит к обработке структуры и всех моделей этой структуры. Созданная нами структура анализа имеет только одну модель, поэтому результат получается одинаковым, независимо от того, указываем ли мы структуру или модель.

4. Просмотрите модель CarOwner\_DT\_OneStep (если у вас есть такое желание), а затем закройте средство просмотра и все открытые окна DMX Query.

Инструкции DMX для создания и обработки моделей анализа с вложенными таблицами несколько сложнее, чем для моделей без вложенных таблиц. Как уже упоминалось в предыдущем разделе, службы Analysis Services рассматривают вложенную таблицу как столбец.

Концепция вложенной таблицы как столбца достаточно запутанная. Однако она становится понятнее, когда вы рассматриваете инструкции DMX, создающие структуры и модели анализа, которые используют вложенные таблицы. Вот синтаксис для добавления (в существующую структуру анализа) модели, которая использует вложенную таблицу:

```
ALTER MINING STRUCTURE <structure>
ADD MINING MODEL <model>(
    <column name> [<prediction>],
    <table name> [<prediction>] (
        <column name> [<prediction>]
    )
)
USING <algorithm> [(<parameter list>)] [WITH DRILLTHROUGH]
```

Пример добавления модели анализа с вложенной таблицей показан в файле скрипта CreateModelWithNestedTable.dmx:

```
ALTER MINING STRUCTURE CustomerProfitCategory
ADD MINING MODEL ProductBuyer_DT(
    [Customer Key]
    , [Age]
    , [Commute Distance]
    , [English Education]
    , [Gender]
    , [Income Group]
    , [Is Car Owner]
    , [Region]
    , [v Customer Purchases] PREDICT
    (
        [Product Category]
    )
)
USING Microsoft_Decision_Trees
```

Скрипт CreateModelWithNestedTable.dmx добавляет модель дерева решений (для прогнозирования покупаемых клиентами категорий продуктов) в структуру анализа CustomerProfitCategory (которую мы создали ранее в этой же главе при помощи визуального конструктора Mining Model). Давайте добавим ее в наше решение.

1. Выполните скрипт CreateModelWithNestedTable.dmx, а затем убедитесь, что модель ProductBuyer\_DT добавлена в структуру анализа (в окне **Object Browser**).

2. Щелкните по модели правой кнопкой мыши и выберите **Browse**. Вы получите сообщение об ошибке, в котором сказано, что вы не можете просмотреть модель до ее обработки. Закройте сообщение об ошибке и пустое окно средства просмотра моделей.
3. Для исправления проблемы выполните код DMX скрипта TrainPreMapped.dmx. Этот запрос дает пример использования инструкции INSERT INTO для обработки модели анализа (когда привязки столбцов для структуры уже определены):

```
INSERT INTO ProductBuyer_DT
```

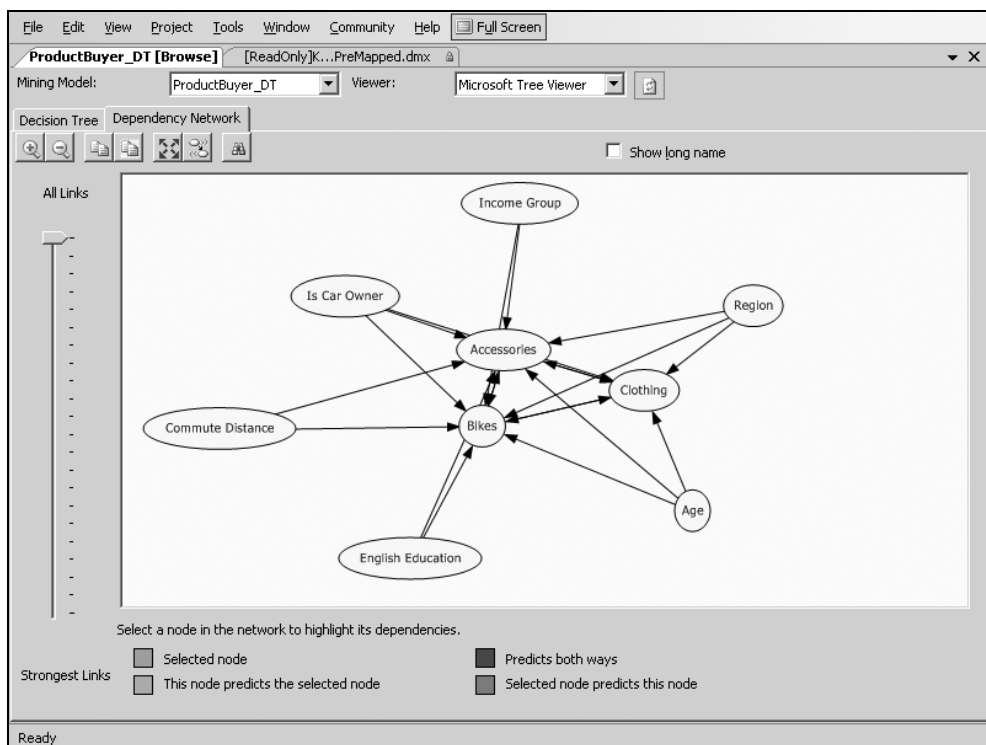
Структура CustomerProfitCategory была определена при помощи визуального конструктора структур интеллектуального анализа данных. Когда вы создаете структуру при помощи этого визуального конструктора, вы выбираете столбцы из вашего представления источника данных для структуры; именно в этот момент привязываются столбцы структуры интеллектуального анализа к исходным данным. Когда столбцы структуры анализа уже привязаны к исходным данным, то инструкция DMX для обработки модели очень проста:

```
INSERT INTO <model>
```

4. После выполнения скрипта TrainPreMapped.dmx вернитесь обратно в **Object Browser**. Щелкните правой кнопкой по модели ProductBuyer\_DT категории CustomerProfitCategory, а затем выберите **Browse**. На этот раз средство просмотра **Tree Viewer** появляется без сообщения об ошибке.
5. Используйте средство просмотра для изучения модели. Модель прогнозирует, является ли клиент покупателем велосипеда, аксессуаров или одежды. На рис. 18.33 показана вкладка **Dependency Network** для этой модели.

Запрос в файле CreateStructureWithNestedTable.dmx создает структуру и модель анализа, использующие алгоритм правил взаимосвязей для прогнозирования тех продуктов, которые пользователи купили:

```
CREATE MINING MODEL PurchaseAssociations (
    [Customer Key] LONG KEY
, [Customer Purchases] TABLE PREDICT
    (
        [Product Subcategory] TEXT KEY
    )
)
USING Microsoft_Association_Rules
```



**Рис. 18.33.** Просмотр модели дерева решений при помощи средства просмотра **Tree Viewer** в Management Studio

Алгоритм правил взаимосвязей Association Rules используется для поиска вклада тех комбинаций элементов, которые часто происходят вместе. Самым обычным применением алгоритма правил взаимосвязей является анализ потребительской корзины. Понимая, какие наборы продуктов покупают клиенты, вы можете делать рекомендации для будущих покупок.

Файл скрипта TrainMapStructureWithNestedTable.dmx содержит код, который показывает, как привязать к исходным данным столбцы структуры анализа, в которой имеется вложенная таблица. Запрос, который указывает столбцы исходных данных, соответствующие столбцам структуры анализа, использует выражение SHAPE. Самый простой случай SHAPE имеет следующую форму:

```
SHAPE {<parent table query>}
APPEND ({ <nested table query> }
    RELATE <master column> TO <child column>
    AS <nested table column name>
```

Выражение `SHAPE` (изначально добавленное для поддержки "иерархических" объектов `RecordSet` в `ActiveX Data Objects (ADO)`) позволяет вам комбинировать данные из нескольких инструкций `SELECT` в единый иерархический результирующий набор (результирующий набор с вложенной таблицей).

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Дополнительную информацию по выражению `SHAPE` и поставщику `MSDataShape OLE DB` можно найти в статье Q189657 базы знаний Microsoft Knowledge Base по адресу: <http://support.microsoft.com/kb/q189657>.

Результирующий набор с вложенной таблицей — это именно то, что требуется для структуры анализа со столбцом вложенной таблицы. Использование выражения `SHAPE` в `DMX` требует, чтобы запрос родительской таблицы и запрос вложенной таблицы были упорядочены по столбцу, который связывает родительскую таблицу с вложенной таблицей. Для запроса `TrainMapStructureWithNestedTable.dmx` это столбец `CustomerKey`. Запрос сочетает использование инструкции `INSERT INTO DMX` и выражения `SHAPE` для связи столбцов структуры анализа с вложенной таблицей.

Вот общая форма такого запроса:

```
INSERT INTO MINING STRUCTURE <structure> (
    <mining structure column> | [SKIP],
    <nested table> (
        <nested table column> | [SKIP]
    )
)
SHAPE {<parent table query>}
APPEND ({ <child table query> }
    RELATE <parent column> TO <child column>
    AS <nested table column name>
```

Вот фактический код файла `TrainMapStructureWithNestedTable.dmx`:

```
INSERT INTO PurchaseAssociations(
    [Customer Key]
    , [Customer Purchases]
    (
        SKIP
        , [Product Subcategory]
    )
)
```

```

SHAPE{
  OPENQUERY([AdventureWorksDW2008],
    'SELECT CustomerKey
      FROM AdventureWorksDW2008.dbo.vCustomerProfitability
      ORDER BY CustomerKey')
}
APPEND ({
  OPENQUERY([AdventureWorksDW2008],
    'SELECT CustomerKey, ProductSubcategory
      FROM AdventureWorksDW2008.dbo.vCustomerPurchases
      ORDER BY CustomerKey'
  ))
RELATE CustomerKey TO CustomerKey) AS CustomerPurchases

```

Указанные в выражении `INSERT INTO` (файла `TrainMapStructureWithNestedTable.dmx`) столбцы структуры анализа являются названиями столбцов структуры. Список столбцов после выражения `INSERT INTO` использует названия столбцов с пробелами (это указанные в инструкции `CREATE MINING MODELS` названия). Обратите внимание, что порядковое положение столбцов в списке соответствует столбцам, возвращаемым запросом исходных данных (которые в данном случае являются результирующим набором, возвращаемым выражением `SHAPE` и его выражениями `APPEND` и `RELATE`). Если запрос исходных данных возвращает столбцы, которые не требуются модели анализа, вы должны использовать ключевое слово `SKIP` (для указания, что данный столбец следует пропустить).

Запрос в файле `TrainMapStructureWithNestedTable.dmx` пропускает столбец `CustomerKey` из запроса к дочерней таблице. Вам нужно включить столбец, который связывает дочернюю (вложенную) таблицу с родительской (в запросах к родительской и дочерней таблицам). Как запрос к родительской, так и запросы к дочерней таблице в файле `TrainMapStructureWithNestedTable.dmx` выбирают столбец `CustomerKey`; однако столбец `[Customer Key]` появляется в структуре анализа только один раз. Поэтому вы должны использовать ключевое слово `SKIP` для пропуска столбца `CustomerKey` для вложенной таблицы.

Выполните файл скрипта `CreateStructureWithNestedTable.dmx`, за ним — `TrainMapStructureWithNestedTable.dmx`, после чего изучите модель `PurchaseAssociation` (выделив правой кнопкой мыши ее узел в окне **Object Explorer**, а затем выбрав **Browse**).

Запросы в файлах скриптов `CreateModelCustomClusterCount.dmx` и `MapTrainSkipColumns.dmx` дают еще один пример того, как использовать мо-



дель анализа с вложенной таблицей, а затем привязать ее структуру к исходным данным.

Скрипт `ModelCustomClusterCount.dmx` создает структуру анализа с моделью кластеризации последовательностей:

```
CREATE MINING MODEL PurchaseSequence (  
    [Customer Key] LONG KEY  
    , [Profit] LONG CONTINUOUS  
    , [Income Group] TEXT DISCRETE  
    , [Customer Purchases] TABLE  
    (  
        [Purchase Sequence] LONG KEY SEQUENCE  
        , [Product Subcategory] TEXT DISCRETE PREDICT  
    )  
)  
USING Microsoft_Sequence_Clustering (Cluster_Count = 0)
```

При использовании алгоритма кластеризации последовательностей интересны не только часто встречающиеся комбинации элементов, но также и порядок (в котором они случаются). В нашем примере модель группирует клиентов в кластеры в соответствии с последовательностью категорий покупаемых продуктов. Модель устанавливает параметр `Cluster_Count` алгоритма в 0. Это приводит к тому, что службы Analysis Services сами определяют оптимальное количество создаваемых кластеров. По умолчанию алгоритм создает 10 кластеров. Выполните запросы в файлах `CreateModelCustomClusterCount.dmx` и `MapTrainSkipColumns.dmx` (именно в таком порядке), а затем просмотрите модель при помощи Cluster Viewer.

## Прогнозы интеллектуального анализа данных при помощи DMX

DMX — это мощный инструмент для запросов DDL к структурам и моделям анализа, но вы можете также использовать его для написания того, что, вероятно, является самым ценным свойством интеллектуального анализа данных: это прогнозирующие запросы. Следующие шесть файлов скриптов DMX в проекте `Chapter18DMX` дают примеры таких прогнозирующих запросов. Используя специальные функции прогнозирования DMX, вы можете применить выявленные (моделями анализа в ваших данных) шаблоны к новым данным.

### ПРИМЕЧАНИЕ

Практически прогнозирующие запросы пишутся для новых образцов данных, к которым вы хотите применить сделанные вами в области интеллектуального анализа открытия. Для простоты последующие прогнозирующие запросы написаны для той же самой базы данных AdventureWorksDW2008, которая использовалась для обучения и тестирования моделей анализа.

Вот базовый формат прогнозирующего запроса:

```
SELECT <select expression list>
FROM <model>
PREDICTION JOIN
<source data query>
ON <join mapping list>
```

Для конкретного запроса замените <model> названием модели анализа (на базе которой вы хотите делать прогнозирующий запрос), а <source data query> замените данными (к которым вы хотите применить прогноз). Достаточно часто вы будете заменять <source data query> инструкцией OPENQUERY (похожей на те, которые используются в инструкциях моделирования SELECT INTO DMX для привязки столбцов интеллектуального анализа к столбцам исходных данных). Вы можете заменить <select expression> любой комбинацией столбцов (возвращаемых запросом к исходным данным), прогнозируемых столбцов модели анализа, а также прогнозирующими функциями DMX (возвращающими столбец данных, в том числе столбец вложенной таблицы). Замените <join mapping list> соответствием столбцов (между исходными данными и входными столбцами модели анализа). Скрипт LeastLikelyCar.dmx содержит следующий код:

```
SELECT TOP 250
    t.FirstName
    ,t.LastName
    ,t.Phone
    ,PredictProbability([Is Car Owner], 'N') AS ProbNotCarOwner
FROM CarOwner_DT
PREDICTION JOIN
OPENQUERY( [AdventureWorksDW2008] ,
    'SELECT
        c.FirstName
        ,c.LastName
        ,tc.*
```

```
FROM vCustomerProfitability tc
INNER JOIN dimCustomer c
ON tc.CustomerKey = c.CustomerKey')
AS t
ON t.Gender = CarOwner_DT.Gender AND
   t.IncomeGroup = CarOwner_dt.[Income Group] AND
   t.MaritalStatus = CarOwner_dt.[Marital Status] AND
   t.Region = CarOwner_dt.Region
ORDER BY PredictProbability([Is Car Owner], 'N') DESC
```

Используя поддерживаемые в DMX выражения TOP и ORDER BY, запрос возвращает имена и номера телефонов 250 человек из vCustomerProfitability, которые с наименьшей вероятностью являются владельцами автомобиля (в соответствии с созданной нами ранее моделью анализа CarOwner\_DT). Запрос использует функцию PredictProbability, которая принимает два аргумента: прогнозируемый столбец из модели анализа и указанное значение для этого столбца. Функция возвращает вероятность отсутствия автомобиля в соответствии с прогнозом модели CarOwner\_DT.

Выполните запрос, и вы увидите, что все 250 клиентов (которые имеют наиболее низкую вероятность владения автомобилем) имеют 78% вероятности отсутствия автомобиля. Модель CarOwner\_DT прогнозирует владение автомобилем по полу, группе доходов, семейному положению и региону. Многие клиенты в этой базе данных имеют одни и те же значения этих характеристик. Те клиенты, у которых совпадают важные для определения владения автомобилем характеристики, будут иметь одинаковую прогнозируемую вероятность владения автомобилем. Для того чтобы увидеть следующее значение вероятности отсутствия автомобиля выберите большее количество клиентов (например, TOP 1250).

Когда названия столбцов модели и запроса к исходным данным совпадают, DMX позволяет использовать соединение NATURAL PREDICTION JOIN. Когда вы используете это соединение, вам не нужно указывать выражение ON для прогнозирующего запроса. Службы Analysis Services соединяют модель и исходные данные на основе названий столбцов. В файле скрипта NaturalPredictionJoin.dmx имеется простой пример:

```
SELECT TOP 250
   t.FirstName
 , t.LastName
 , t.Phone
 , PredictProbability([Is Car Owner], 'N') AS ProbNotCarOwner
```

```

FROM CarOwner_DT
NATURAL PREDICTION JOIN
OPENQUERY(
    [AdventureWorksDW2008],
    'SELECT
        c.FirstName
        ,c.LastName
        ,tc.Phone
        ,tc.Gender
        ,tc.IncomeGroup AS [Income Group]
        ,tc.MaritalStatus AS [Marital Status]
        ,tc.Region
    FROM vCustomerProfitability tc
    INNER JOIN dimCustomer c
    ON tc.CustomerKey = c.CustomerKey')
AS t
ORDER BY PredictProbability([Is Car Owner], 'N') DESC

```

Выполните запрос в файле `NaturalPredictionJoin.dmx` и убедитесь в том, что результаты те же самые, что и для запроса `LeastLikelyCar.dmx`.

Для того чтобы инструкция `NATURAL PREDICTION JOIN` установила соответствие, названия в модели и запросе к исходным данным должны быть идентичными. Если запрос к исходным данным не имеет столбца с идентичным входному столбцу названием, сообщение об ошибке не выдается; вместо этого службы Analysis Services делают прогноз на основе тех столбцов, для которых соответствия найдены. Запрос в файле `NaturalPredictionJoinMissingMappings.dmx` не дает псевдонимов названиям столбцов в запросе к исходным данным (чтобы они совпадали с названиями модели анализа):

```

SELECT TOP 250
    t.FirstName
    ,t.LastName
    ,t.Phone
    ,PredictProbability([Is Car Owner], 'N') AS ProbNotCarOwner
FROM CarOwner_DT
NATURAL PREDICTION JOIN
OPENQUERY([Adventure Works DW],
    'SELECT
        c.FirstName

```

```

    ,c.LastName
    ,tc.Phone
    ,tc.Gender
    ,tc.IncomeGroup
    ,tc.MaritalStatus
    ,tc.Region
FROM vCustomerProfitability tc
INNER JOIN dimCustomer c
ON tc.CustomerKey = c.CustomerKey')
AS t
ORDER BY PredictProbability([Is Car Owner], 'N') DESC

```

В этом запросе прогноз делается только по тем столбцам, которые точно совпадают по названию (в данном случае это `Gender` и `Region`). Выполните файл скрипта `NaturalPredictionJoinMissingMappings.dmx` и обратите внимание, что его результаты отличаются от предыдущих двух прогнозов.

Когда модель имеет столбец вложенной таблицы, запросы несколько более сложны, но имеют все ту же базовую структуру (соединение модели анализа с исходными данными и указание соответствия столбцов исходных данных входным столбцам модели анализа). Сложность запроса состоит в основном в запросе к исходным данным. Запрос должен возвращать результирующий набор со столбцом вложенной таблицы. Как вы помните по изученным нами ранее инструкциям моделирования языка DMX, это делается при помощи выражения `SHAPE`.

Файл `MostLikelyCluster4.dmx` содержит пример прогнозирующего запроса DMX, использующего модель со столбцом вложенной таблицы:

```

SELECT TOP 250
    t.FirstName
    ,t.LastName
    ,t.Profit
    ,ClusterProbability('Cluster 4') AS ProbCluster4
FROM CustomerProfitCategory_CL5
PREDICTION JOIN
SHAPE {
    OPENQUERY ( [AdventureWorksDW2008] ,
        'SELECT
            c.FirstName
            ,c.LastName
            ,tc.*

```

```

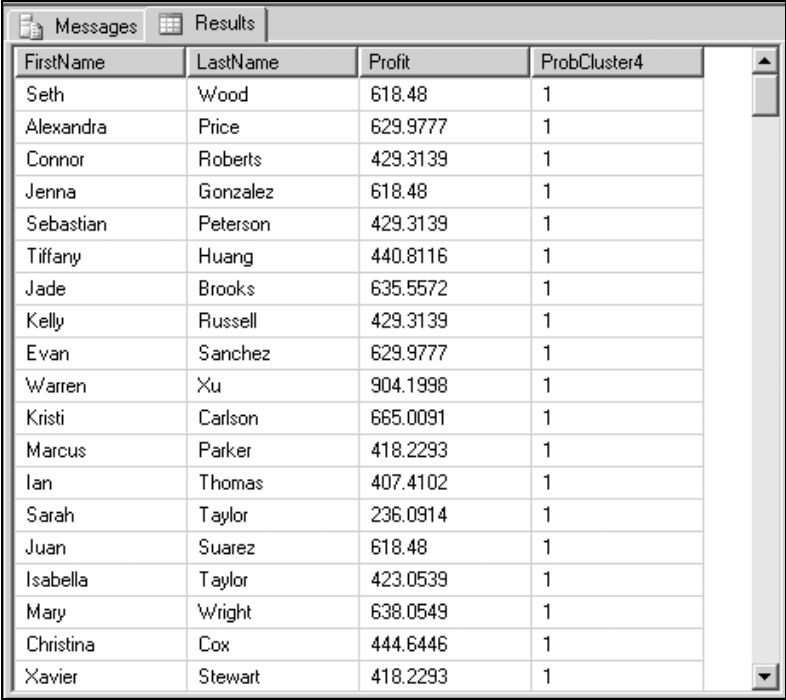
FROM vCustomerProfitability tc
INNER JOIN dimCustomer c
ON tc.CustomerKey = c.CustomerKey
WHERE tc.IsNewCustomer = 'Y'

ORDER BY tc.CustomerKey'
)
}
APPEND ({
    OPENQUERY ([AdventureWorksDW2008] ,
        'SELECT
            CustomerKey
            ,ProductCategory
        FROM vCustomerPurchases
        ORDER BY CustomerKey'
    )}
    RELATE CustomerKey TO CustomerKey)
    AS PurchaseCategory
AS t
ON t.Age = CustomerProfitCategory_CL5.Age AND
    t.CommuteDistance = CustomerProfitCategory_CL5.[Commute Distance] AND
    t.EnglishEducation = CustomerProfitCategory_CL5.[English Education] AND
    t.Gender = CustomerProfitCategory_CL5.Gender AND
    t.HasKidsAtHome = CustomerProfitCategory_CL5.[Has Kids At Home] AND
    t.IncomeGroup = CustomerProfitCategory_CL5.[Income Group] AND
    t.IsCarOwner = CustomerProfitCategory_CL5.[Is Car Owner] AND
    t.IsHomeOwner = CustomerProfitCategory_CL5.[Is Home Owner] AND
    t.IsNewCustomer = CustomerProfitCategory_CL5.[Is New Customer] AND
    t.MaritalStatus = CustomerProfitCategory_CL5.[Marital Status] AND
    t.NumProdGroup = CustomerProfitCategory_CL5.[Num Prod Group] AND
    t.RecencyGroup = CustomerProfitCategory_CL5.[Recency Group] AND
    t.Region = CustomerProfitCategory_CL5.Region AND
    t.PurchaseCategory.ProductCategory =
        CustomerProfitCategory_CL5.[v Customer Purchases].[Product Category]
ORDER BY ClusterProbability('Cluster 4') DESC

```

Этот запрос использует модель кластеризации CustomerProfitCategory\_CL5 для прогнозирования вероятности того, что новый клиент окажется в кластере 4, это кластер с относительно высокой концентрацией высокодоходных клиентов, некоторые из которых вероятно могут стать новыми клиентами.

Нас интересуют новые клиенты, которые попадают в этот кластер, поскольку мы хотели бы сохранить их в качестве своих клиентов. Предыдущий запрос возвращает список клиентов и в том числе: сумму дохода в долларах, полученную от каждого клиента; а также вероятность того, что клиент попадает в кластер 4. Выполните запрос; часть возвращаемого запросом результирующего набора показана на рис. 18.34.



| FirstName | LastName | Profit   | ProbCluster4 |
|-----------|----------|----------|--------------|
| Seth      | Wood     | 618.48   | 1            |
| Alexandra | Price    | 629.9777 | 1            |
| Connor    | Roberts  | 429.3139 | 1            |
| Jenna     | Gonzalez | 618.48   | 1            |
| Sebastian | Peterson | 429.3139 | 1            |
| Tiffany   | Huang    | 440.8116 | 1            |
| Jade      | Brooks   | 635.5572 | 1            |
| Kelly     | Russell  | 429.3139 | 1            |
| Evan      | Sanchez  | 629.9777 | 1            |
| Warren    | Xu       | 904.1998 | 1            |
| Kristi    | Carlson  | 665.0091 | 1            |
| Marcus    | Parker   | 418.2293 | 1            |
| Ian       | Thomas   | 407.4102 | 1            |
| Sarah     | Taylor   | 236.0914 | 1            |
| Juan      | Suarez   | 618.48   | 1            |
| Isabella  | Taylor   | 423.0539 | 1            |
| Mary      | Wright   | 638.0549 | 1            |
| Christina | Cox      | 444.6446 | 1            |
| Xavier    | Stewart  | 418.2293 | 1            |

**Рис. 18.34.** Результаты прогнозирующего запроса DMX

Запрос использует прогнозирующую функцию `ClusterProbability`, которая принимает в качестве аргумента кластер и возвращает вероятность принадлежности к нему входной записи.

В инструкции `SELECT` вы можете также вернуть столбец вложенной таблицы. Например, алгоритм кластеризации поддерживает использование функции `PredictHistogram` с аргументом в виде столбца кластеризации. Будет возвращен табличный столбец с прогнозируемой вероятностью попадания каждой записи результирующего набора в каждый кластер.

Это показано запросом из файла скрипта Cluster5ProbabilityDist.dmx:

```

SELECT TOP 250
    t.FirstName
    ,t.LastName
    ,t.Profit
    ,Cluster()
    ,PredictHistogram(Cluster())
FROM CustomerProfitCategory_CL5
PREDICTION JOIN
SHAPE {
    OpenQuery([AdventureWorksDW2008],
        'SELECT
            c.FirstName
            ,c.LastName
            ,tc.*
        FROM vCustomerProfitability tc
        INNER JOIN dimCustomer c
        ON tc.CustomerKey = c.CustomerKey
        ORDER BY tc.CustomerKey'
    )
}
APPEND({
    OPENQUERY([AdventureWorksDW2008],
        'SELECT
            CustomerKey
            ,ProductCategory
        FROM vCustomerPurchases
        ORDER BY CustomerKey'
    )}
RELATE CustomerKey To CustomerKey)
    AS PurchaseCategory
AS t
ON t.Age = CustomerProfitCategory_CL5.Age AND
    t.CommuteDistance = CustomerProfitCategory_CL5.[Commute Distance] AND
    t.EnglishEducation = CustomerProfitCategory_CL5.[English Education] AND
    t.Gender = CustomerProfitCategory_CL5.Gender AND
    t.HasKidsAtHome = CustomerProfitCategory_CL5.[Has Kids At Home] AND
    t.IncomeGroup = CustomerProfitCategory_CL5.[Income Group] AND

```



```

t.IsCarOwner = CustomerProfitCategory_CL5.[Is Car Owner] AND
t.IsHomeOwner = CustomerProfitCategory_CL5.[Is Home Owner] AND
t.IsNewCustomer = CustomerProfitCategory_CL5.[Is New Customer] AND
t.MaritalStatus = CustomerProfitCategory_CL5.[Marital Status] AND
t.NumProdGroup = CustomerProfitCategory_CL5.[Num Prod Group] AND
t.RecencyGroup = CustomerProfitCategory_CL5.[Recency Group] AND
t.Region = CustomerProfitCategory_CL5.Region AND
t.PurchaseCategory.ProductCategory =
    CustomerProfitCategory_CL5.[v Customer Purchases].[Product Category]

```

Выполните запрос, чтобы увидеть, как SQL Server Management Studio работает с вложенными таблицами. Вы заметите, что каждая ячейка столбца гистограммы прогнозируемого кластера (последний столбец) имеет знак плюс. Щелкните по одному из них, чтобы развернуть вложенную таблицу для данной строки (рис. 18.35). Например, Ruben Torres имеет 99-процентную вероятность попадания в кластер 1 и менее 1% вероятности попадания в кластер 4.

Messages

Results

| FirstName | LastName         | Profit           | \$CLUSTER | Expression                                                                                                                                                                                                                                                                                                                                                                                 |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
|-----------|------------------|------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------------|---------------|-----------|------------------|------------------|-----------|------------------|------------------|-----------|---|---|-----------|---|---|-----------|---|---|
| Jon       | Yang             | 3513.6905        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Eugene    | Huang            | 2795.8839        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Ruben     | Torres           | 3454.8801        | Cluster 1 | ⊖ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
|           |                  |                  |           | <table><tr><th>\$CLUSTER</th><th>\$DISTANCE</th><th>\$PROBABILITY</th></tr><tr><td>Cluster 1</td><td>0.00612816941...</td><td>0.99387183058...</td></tr><tr><td>Cluster 4</td><td>0.99387183058...</td><td>0.00612816941...</td></tr><tr><td>Cluster 3</td><td>1</td><td>0</td></tr><tr><td>Cluster 2</td><td>1</td><td>0</td></tr><tr><td>Cluster 5</td><td>1</td><td>0</td></tr></table> | \$CLUSTER | \$DISTANCE | \$PROBABILITY | Cluster 1 | 0.00612816941... | 0.99387183058... | Cluster 4 | 0.99387183058... | 0.00612816941... | Cluster 3 | 1 | 0 | Cluster 2 | 1 | 0 | Cluster 5 | 1 | 0 |
| \$CLUSTER | \$DISTANCE       | \$PROBABILITY    |           |                                                                                                                                                                                                                                                                                                                                                                                            |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Cluster 1 | 0.00612816941... | 0.99387183058... |           |                                                                                                                                                                                                                                                                                                                                                                                            |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Cluster 4 | 0.99387183058... | 0.00612816941... |           |                                                                                                                                                                                                                                                                                                                                                                                            |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Cluster 3 | 1                | 0                |           |                                                                                                                                                                                                                                                                                                                                                                                            |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Cluster 2 | 1                | 0                |           |                                                                                                                                                                                                                                                                                                                                                                                            |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Cluster 5 | 1                | 0                |           |                                                                                                                                                                                                                                                                                                                                                                                            |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Christy   | Zhu              | 3467.1264        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Elizabeth | Johnson          | 3501.9051        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Julio     | Ruiz             | 3459.8654        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Janet     | Alvarez          | 3453.7156        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Marco     | Mehta            | 3515.577         | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Rob       | Verhoff          | 3454.751         | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Shannon   | Carlson          | 3445.3736        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |
| Jacquelyn | Suarez           | 3435.0441        | Cluster 1 | ⊕ Expression                                                                                                                                                                                                                                                                                                                                                                               |           |            |               |           |                  |                  |           |                  |                  |           |   |   |           |   |   |           |   |   |

Рис. 18.35. Результат запроса DMX со столбцом вложенной таблицы

Результаты запроса со столбцом вложенной таблицы интересны, но ими сложно манипулировать. Для того чтобы справиться с этой проблемой, язык

DMX имеет ключевое слово `FLATTENED`, которое позволяет вам возвращать столбец вложенной таблицы в неиерархическом результирующем наборе (даже при использовании выражения `SHAPE` для получения вложенных данных). Запрос в файле скрипта `Cluster5ProbabilityDistFlattened.dmx` возвращает почти такой же результат, что и файл `Cluster5ProbabilityDist.dmx`, за исключением того, что он использует ключевое слово `FLATTENED` для возвращения неиерархического результирующего набора. В нем также указано, что необходимо вернуть верхние 1250 записей (а не 250). Каждая строка в невыровненном результирующем наборе имеет эквивалент в виде пяти строк (по одной на каждый сегмент гистограммы) в выровненном результирующем наборе. Поэтому для того, чтобы увидеть те же самые данные, нам нужно запросить в пять раз больше записей, чем раньше:

```
SELECT FLATTENED TOP 1250
    t.FirstName
    ,t.LastName
    ,t.Profit
    ,ClusterProbability()
    ,PredictHistogram(Cluster())
FROM CustomerProfitCategory_CL5
PREDICTION JOIN
SHAPE {
    OpenQuery([AdventureWorksDW2008],
        'SELECT
            c.FirstName
            ,c.LastName
            ,tc.*
        FROM vCustomerProfitability tc
        INNER JOIN dimCustomer c
        ON tc.CustomerKey = c.CustomerKey
        ORDER BY tc.CustomerKey'
    )
}
APPEND({
    OPENQUERY([AdventureWorksDW2008],
        'SELECT
            CustomerKey
            ,ProductCategory
        FROM vCustomerPurchases
```

```

ORDER BY CustomerKey'
)}
RELATE CustomerKey To CustomerKey)
AS PurchaseCategory
AS t
ON t.Age = CustomerProfitCategory_CL5.Age AND
t.CommuteDistance = CustomerProfitCategory_CL5.[Commute Distance] AND
t.EnglishEducation = CustomerProfitCategory_CL5.[English Education] AND
t.Gender = CustomerProfitCategory_CL5.Gender AND
t.HasKidsAtHome = CustomerProfitCategory_CL5.[Has Kids At Home] AND
t.IncomeGroup = CustomerProfitCategory_CL5.[Income Group] AND
t.IsCarOwner = CustomerProfitCategory_CL5.[Is Car Owner] AND
t.IsHomeOwner = CustomerProfitCategory_CL5.[Is Home Owner] AND
t.IsNewCustomer = CustomerProfitCategory_CL5.[Is New Customer] AND
t.MaritalStatus = CustomerProfitCategory_CL5.[Marital Status] AND
t.NumProdGroup = CustomerProfitCategory_CL5.[Num Prod Group] AND
t.RecencyGroup = CustomerProfitCategory_CL5.[Recency Group] AND
t.Region = CustomerProfitCategory_CL5.Region AND
t.PurchaseCategory.ProductCategory =
CustomerProfitCategory_CL5.[v Customer Purchases].[Product Category]

```

| FirstName | LastName | Profit    | Expression    | Expression.\$CLUSTER | Expression.\$DISTANCE | Expression.\$PROBABILITY |
|-----------|----------|-----------|---------------|----------------------|-----------------------|--------------------------|
| Jon       | Yang     | 3513.6905 | 0.88167858... | Cluster 1            | 0.118321418143098     | 0.881678581856902        |
| Jon       | Yang     | 3513.6905 | 0.88167858... | Cluster 4            | 0.881678581856902     | 0.118321418143098        |
| Jon       | Yang     | 3513.6905 | 0.88167858... | Cluster 3            | 1                     | 0                        |
| Jon       | Yang     | 3513.6905 | 0.88167858... | Cluster 2            | 1                     | 0                        |
| Jon       | Yang     | 3513.6905 | 0.88167858... | Cluster 5            | 1                     | 0                        |
| Eugene    | Huang    | 2795.8839 | 0.99861576... | Cluster 1            | 0.00138423353037342   | 0.998615766469627        |
| Eugene    | Huang    | 2795.8839 | 0.99861576... | Cluster 4            | 0.998615766469627     | 0.00138423353037336      |
| Eugene    | Huang    | 2795.8839 | 0.99861576... | Cluster 3            | 1                     | 0                        |
| Eugene    | Huang    | 2795.8839 | 0.99861576... | Cluster 2            | 1                     | 0                        |
| Eugene    | Huang    | 2795.8839 | 0.99861576... | Cluster 5            | 1                     | 0                        |
| Ruben     | Torres   | 3454.8801 | 0.99387183... | Cluster 1            | 0.00612816941745731   | 0.993871830582543        |
| Ruben     | Torres   | 3454.8801 | 0.99387183... | Cluster 4            | 0.993871830582543     | 0.0061281694174573       |
| Ruben     | Torres   | 3454.8801 | 0.99387183... | Cluster 3            | 1                     | 0                        |
| Ruben     | Torres   | 3454.8801 | 0.99387183... | Cluster 2            | 1                     | 0                        |
| Ruben     | Torres   | 3454.8801 | 0.99387183... | Cluster 5            | 1                     | 0                        |
| Christy   | Zhu      | 3467.1264 | 1             | Cluster 1            | 0                     | 1                        |
| Christy   | Zhu      | 3467.1264 | 1             | Cluster 2            | 1                     | 0                        |
| Christy   | Zhu      | 3467.1264 | 1             | Cluster 3            | 1                     | 0                        |
| Christy   | Zhu      | 3467.1264 | 1             | Cluster 4            | 1                     | 0                        |

Рис. 18.36. Результат запроса DMX с выровненным столбцом таблицы

Выполните запрос; результаты должны выглядеть так, как показано на рис. 18.36. Каждый столбец вложенной таблицы теперь имеет вид столбца в родительской таблице. Значения в "нетабличных" столбцах родительской таблицы повторяются для каждой строки вложенной таблицы. Важно понимать, что это выровненное представление столбцов вложенной таблицы является совершенно правильным представлением данных, поскольку сама вложенная таблица считается столбцом своего родителя.

## Шаблоны DMX

Management Studio предлагает шаблоны для написания запросов DMX. Для того чтобы увидеть эти запросы, выберите **View | Template Explorer** в Management Studio.

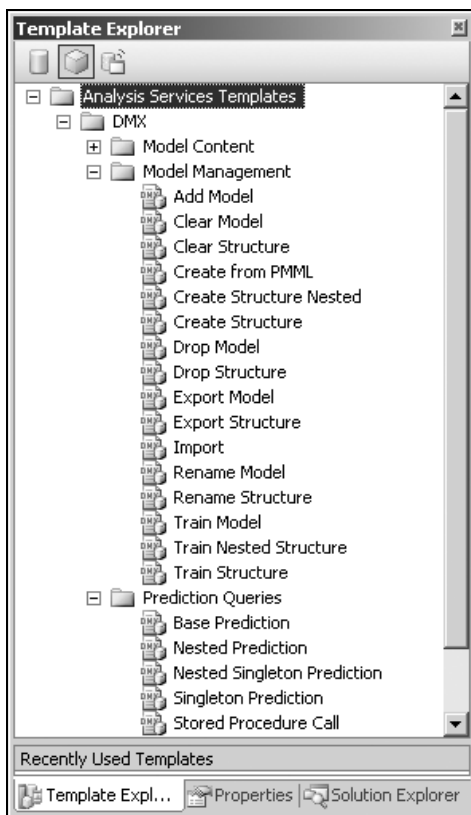


Рис. 18.37. Окно **Template Explorer** в Management Studio

По умолчанию показаны шаблоны SQL. Нажмите кнопку **Analysis Server** в панели инструментов **Template Explorer** (средняя кнопка со значком куба) для показа древовидного представления шаблонов Analysis Services Templates. Разверните узел DMX, чтобы увидеть шаблоны DMX. Окно **Template Explorer** с древовидным представлением шаблонов DMX показано на рис. 18.37.

Analysis Services предоставляет шаблоны для трех типов инструкций DMX: **Model Content**, **Model Management** и **Prediction Queries**. В узле **Model Content** вы можете найти шаблоны запросов для возвращения метаданных о ваших моделях анализа (это по существу та же самая информация, которую предоставляют средства просмотра, но без визуализации). В узле **Model Management** находятся шаблоны для запросов создания и обучения моделей анализа (подобные тем, которые мы писали ранее). В узле **Prediction Queries** находятся шаблоны для написания прогнозирующих запросов. Шаблоны дают вам заготовки для написания некоторых самых обычных запросов DMX и позволяют экономить время (по сравнению с написанием запросов с нуля).

## Применение интеллектуального анализа данных

Возможно, вы хотите узнать, отчего мы целую главу посвятили разработке приложений OLAP и только одну главу — интеллектуальному анализу данных вообще. Интеллектуальный анализ данных в Analysis Services программируется не хуже, чем OLAP, и хотя интеллектуальный анализ иногда проще разработки под OLAP, в целом они имеют сравнимую сложность.

И все-таки разработка интеллектуального анализа данных от программирования OLAP отличается. Во-первых, запросы интеллектуального анализа обычно не используют детализацию, что делает Excel PivotTables гораздо менее востребованным инструментом разработки презентаций. С другой стороны, имеющиеся в Visual Studio и SQL Server Management Studio средства просмотра содержимого моделей доступны разработчикам в виде элементов управления Windows Forms. И на базе этих программируемых визуализаций команда интеллектуального анализа данных компании Microsoft добавила также пример кода, который реализует несколько таких средств просмотра, как серверные элементы управления ASP.NET. Просмотр моделей из ваших приложений при помощи этих элементов управления очень важен и заслуживает отдельного обсуждения.

Но как насчет выполнения прогнозирующих запросов и представления результатов? Или выполнения инструкций DML языка DMX программным путем? И как насчет хранимых на стороне сервера процедур? Или задач управления базами данных, структурами и моделями анализа? Почему эта масса тем заслуживает отдельной главы? Причина в том, что изученный в *главе 17* интерфейс прикладного программирования OLAP дает вам почти все навыки, необходимые для разработки интеллектуального анализа данных.

Почти все, что вы изучили относительно кубов OLAP (ADO MD.NET, Analysis Management Objects (AMO) или XMLA (используемого программно или из Management Studio)), работает и с моделями интеллектуального анализа данных. Это справедливо как для разработки на клиентской стороне, так и для выполняющегося на сервере управляемого кода. Вы можете без труда создавать хранимые процедуры, которые выполняют прогнозирующие запросы DMX и возвращают результирующие наборы клиенту, причем вы можете делать это без использования библиотеки `AdomdClient` и без предоставления сборки разрешений `Unrestricted`.

### **ПРИМЕЧАНИЕ**

Можно также представить результаты запросов содержимого DMX и прогнозирующих запросов DMX при помощи служб отчетности Reporting Services. Точно так же, как Reporting Services поддерживает SQL для реляционных баз данных и MDX для баз OLAP, так же поддерживается и DMX для моделей интеллектуального анализа данных. Несмотря на то, что создание отчета по модели интеллектуального анализа данных в этой главе не описывается, файл решения этой главы содержит проект Report Server, который использует прогнозирующий запрос DMX для создания отчета по матрице классификации.

В этой последней части главы мы дадим несколько прикладных примеров использования средств просмотра содержимого моделей и кода `AdomdClient` в средах Windows Forms, ASP.NET 2.0 и более поздних. Мы также покажем вам, как легко можно создать хранимую на серверной стороне процедуру (которая выполняет прогнозирующий запрос DMX и возвращает результаты) и аналогичный запрос XMLA. Закончим мы демонстрацией того, как использовать надстройки интеллектуального анализа данных для Excel 2007.

## **Интеллектуальный анализ данных и программирование при помощи API**

Как мы только что сказали, ADO MD.NET предоставляет полную поддержку анализа данных. Это справедливо как в отношении метаданных, так и в от-

ношении запросов. Например, объект `AdomdConnection` имеет как коллекцию `MiningStructures`, так и коллекцию `MiningModels`. Каждый элемент коллекции `MiningStructures` имеет свою собственную коллекцию `MiningModels`. Объект `MiningModel` имеет также свойство `Parent`, указывающее на содержащий его объект `MiningStructure`. Такая гибкость объектной модели ADO MD.NET позволяет вашему коду просматривать модели базы данных либо как плоскую коллекцию в масштабе базы данных, либо как иерархическую коллекцию структур и моделей. Объект `MiningModel` также содержит свойство `Algorithm` и коллекцию `Attributes`, позволяя вам программно определить входные и прогнозируемые столбцы модели (а также тот алгоритмический контекст, в котором они используются).

## Использование элементов управления средствами просмотра содержимого моделей в Windows Forms

Хорошим использованием всех этих метаданных было бы предоставление интерфейса, в котором пользователи могли бы выбрать модель и отобразить ее содержимое в соответствующем средстве просмотра. Это почти такая же функциональность, которую предоставляет SQL Server Management Studio в своем окне **Object Explorer** и команде **Browse** контекстного меню, но без необходимости инсталляции и применения самой Management Studio.

Если на вашем компьютере уже инсталлирована Management Studio, то элементы управления уже есть, надо сделать ссылку на сборку `Microsoft.AnalysisServices.Viewers.dll`, которая по умолчанию находится в следующем каталоге: `C:\Program Files\Microsoft Visual Studio 9.0\Common7\IDE\PrivateAssemblies`. Для того чтобы ваше приложение правильно компилировалось и выполнялось, вам нужно также настроить ссылки на следующие сборки (все они находятся в упомянутом каталоге): `Microsoft.AnalysisServices.Controls`, `Microsoft.AnalysisServices.Graphing`, `Microsoft.AnalysisServices.OleDbDM` и `Microsoft.DataWarehouse`.

Кроме того, вам нужно настроить ссылку на библиотеку `Microsoft.DataWarehouse.Interfaces`, которую можно найти на вкладке **.NET** диалогового окна **Add Reference**.

Использовать элемент управления просто: перетащите его на форму, а затем настройте его свойства `ConnectionString` и `MiningModelName` в окне **Prop-**

**ties** или в коде (убедившись, что алгоритм указанной вами модели совпадает с выбранным вами типом средства просмотра). Элементы управления используют OLE DB, поэтому убедитесь, что в присвоенном вами для `ConnectionString` значении есть выражение "Provider=MSOLAP.4;" помимо выражений `Data Source` и `Initial Catalog`. Затем все, что нужно сделать — это просто вызвать метод `LoadViewerData` элемента управления, передав ему `null` в качестве значения обязательного параметра. Следующие три строки кода показывают, как это сделать (подразумевается, что `viewer` — это экземпляр одного из элементов управления средством просмотра, `connectionString` содержит совместимую с ADO MD.NET строку подключения с указанием источника и каталога, а `cboMiningModels` — элемент управления типа `ComboBox`, содержащий список моделей анализа на сервере):

```
viewer.ConnectionString = "Provider=MSOLAP.4;" + connectionString;  
viewer.MiningModelName = cboMiningModels.Text;  
viewer.LoadViewerData(null);
```

Пример решения Visual Studio (предоставленный для этой главы на Web-сайте данной книги) включает проект типа Windows Forms с названием `Chapter18WinClient`.

### **ВАЖНО**

Убедитесь, что все ссылки на сборки проекта `Chapter18WinClient` правильны. Если рядом с некоторыми ссылками появились значки предупреждений, удалите эти ссылки и добавьте их обратно вручную при помощи вкладок **.NET** или **Browse** в диалоговом окне **Add Reference** в Visual Studio.

Проект `Chapter18WinClient` содержит дочернюю форму многодокументарного интерфейса с названием `frmModelViewer` (файл с названием `DMMViewerDemo.cs`), который позволяет пользователю указать сервер, базу данных и модель для просмотра, а затем отображает содержимое модели в элементе управления средством просмотра соответствующего типа. Вместо созданного на этапе разработки элемента управления (и до выполнения показанного ранее кода) код формы объявляет `objViewer` как тип `Microsoft.AnalysisServices.Viewers.MiningModelViewerControl` (класс, от которого наследует каждый элемент управления средством просмотра). Затем он присваивает этой переменной новый экземпляр соответствующего алгоритму средства просмотра (в зависимости от свойства `Algorithm` выбранного объекта `MiningModel`). И наконец, он добавляет объект в коллекцию `Controls` формы и адаптирует его на форме.



Форма позволяет пользователю после указания сервера нажать кнопку **Show Databases**. Обработчик события Click кнопки открывает подключение `AdomdConnection` к серверу и извлекает `DataSet` (содержащий схему со всеми каталогами сервера), который использует для заполнения элемента управления `ComboBox`. Вот упрощенная версия этого кода, в которой `txtServer` является элементом управления типа `TextBox` (содержащим название сервера), а `cboCatalogs` является целевым элементом управления `ComboBox`:

```
AdomdConnection conn = new AdomdConnection("Data Source=" +
    this.txtServer.Text);
conn.Open();
foreach (DataRow dr in conn.GetSchemaDataSet(AdomdSchemaGuid.Catalogs,
    null).Tables[0].Rows)
{
    this.cboCatalogs.Items.Add((string)dr[0]);
}
conn.Close();
```

Обратите внимание, что в строке подключения есть только выражение `Data Source` (а не выражение `Initial Catalog`). Нам нужна только информация на уровне сервера, так что этого достаточно.

Когда пользователь делает выбор в `cboCatalogs`, то код извлекает список названий всех моделей, принадлежащих выбранной базе данных (и заполняет ими элемент управления `ComboBox`). Вот упрощенная версия этого кода (здесь `cboMiningModels` — это целевой `ComboBox`):

```
connectionString =
    "Data Source=" + this.txtServer.Text + ";Initial Catalog=" +
    this.cboCatalogs.Text;
AdomdConnection conn = new AdomdConnection(connectionString);
conn.Open();
foreach (MiningModel mm in conn.MiningModels)
{
    this.cboMiningModels.Items.Add(mm.Name);
}
conn.Close();
```

Как только в `cboMiningModels` будет выбрана модель, можно выполнять код, который создает экземпляр элемента управления средством просмотра и подключает его к конкретной модели.

## Выполнение прогнозирующих запросов при помощи ADO MD.NET

Изученные нами до настоящего момента методики программирования очень просты, но выполнение прогнозирующих запросов еще проще. В *главе 17* вы узнали, что можно использовать объекты `AdomdCommand` и `AdomdDataAdapter` в ADO MD.NET для выполнения запросов MDX и возвращения выровненных результирующих наборов в форме `AdomdDataReader` или `DataSet/DataTable`. Аналогичным же образом вы можете использовать их и для выполнения прогнозирующих запросов DMX и возвращения результирующих наборов в табличной форме. Следующий код (упрощенная версия кода формы `frmDMXAdHoc` проекта `Chapter18WinClient` — файл с названием `DMXAdHoc.cs`) выполняет запрос DMX, извлекает результаты в виде `DataTable`, а затем привязывает `DataTable` к `dvgResults` (элемент управления типа `DataGridView`):

```
AdomdConnection conn = new AdomdConnection
("Data Source=" + this.txtServer.Text + ";Initial Catalog=" +
    this.cboCatalogs.Text);
AdomdCommand cmd = new AdomdCommand(this.txtQuery.Text, conn);
AdomdDataAdapter da = new AdomdDataAdapter(cmd);
DataTable dt = new DataTable();
da.Fill(dt);
this.dvgResults.DataSource = dt;
```

В этом коде `txtServer` и `cboCatalogs` эквивалентны своим тезкам в форме `frmModelViewer` (обсуждалась ранее), а `txtQuery` — это многострочный элемент управления типа `TextBox`, содержащий запрос DMX.

Обратите внимание, что `frmDMXAdHoc` функционально эквивалентна окну DMX в Management Studio. Это означает, что почти все прогнозирующие запросы DMX проекта образцового кода можно выполнить из этой формы. (В текстах запросов можно даже сохранить комментарии). Есть одно исключение: код не сможет правильно обработать результирующие наборы с вложенными столбцами (такие как в `Cluster5ProbabilityDist.dmx`), во избежание этой проблемы используйте ключевое слово `FLATTENED` (как это сделано в `Cluster5ProbabilityDistFlattened.dmx`).

## Запросы содержимого модели

Код ADO MD, используемый в средстве просмотра формы `frmDMXAdHoc`, может обрабатывать другие типы запросов DMX. Например, в этой форме можно выполнить любой из запросов DML (из примера проекта Management Studio). Однако ни один из них не возвращает никаких данных, поэтому было бы разумнее выполнять их при помощи метода `ExecuteNonQuery` объекта `AdomdCommand`. И наконец, можно выполнять также запросы содержимого моделей (которые мы кратко обсуждали ранее в контексте шаблонов DMX). Например, следующий запрос возвращает текстовую информацию о модели кластеризации `CustomerProfitCategory_CL`:

```
SELECT NODE_CAPTION, NODE_DESCRIPTION, NODE_PROBABILITY  
FROM [CustomerProfitCategory_CL].CONTENT
```

Этот запрос можно выполнить из окна DMX Query в Management Studio или из формы `frmDMXAdHoc` проекта `Chapter18WinClient` (попробуйте сами и убедитесь). Данные содержимого в текстовой форме выглядят значительно менее элегантно, чем их визуализация, но их программное использование весьма интересно. Ваше приложение могло бы запросить модель, получить определенные шаблоны (путем разбора столбца `NODE_DESCRIPTION`), а затем выполнить логику бизнес-прогнозирования при вводе новых данных.

## ADO MD.NET и ASP.NET

Большая часть того, что мы изложили до настоящего момента в контексте разработки Windows Forms, может быть аналогичным способом реализована и в ASP.NET. Проект `Chapter18WebClient` (который включен в примеры кодов решения Visual Studio на Web-сайте данной книги) дает пример кодирования эквивалента `Chapter18WebClient` в ASP.NET. Средства просмотра содержимого моделей должны обрабатываться иначе, чем их аналоги в Windows Forms (и скоро мы это обсудим). Однако выполнение запросов DMX производится в значительной степени аналогично.

Например, вы можете заполнить элемент управления `GridView` (с названием `gvResults`) иерархическими результатами запроса DMX следующим образом:

```
AdomdConnection conn = new AdomdConnection ("Data Source=" +  
    this.txtServer.Text + ";Initial Catalog=" + this.ddlCatalogs.Text);  
AdomdCommand cmd = new AdomdCommand(this.txtQuery.Text, conn);
```

```
AdomdDataAdapter da = new AdomdDataAdapter(cmd);  
DataTable dt = new DataTable();  
da.Fill(dt);  
this.gvResults.DataSource = dt;  
this.gvResults.DataBind();
```

В этом фрагменте (который является упрощенной версией кода страницы DMXAdHoc.aspx приложения Chapter18WebClient) ddlCatalogs — это элемент управления DropDownList из ASP.NET, содержащий список баз данных того сервера, название которого указано в элементе управления txtServer. Этот код удивительно похож на свой эквивалент для Windows Forms и универсален в плане обработки запросов DMX.

## Использование веб-элементов управления для интеллектуального анализа данных

Хорошо, что запросы DMX можно обрабатывать во многом аналогично в приложениях и ASP.NET, и Windows Forms, но как насчет визуализации содержимого моделей? Элементы управления средствами просмотра содержимого спроектированы для среды Windows Forms, поэтому мы не можем использовать их в приложении ASP.NET. Однако компания Microsoft предоставляет пример .NET-кода, который реализует версии Asynchronous JavaScript и XML (AJAX) для средств просмотра Cluster, Naive Bayes и Decision Trees (в виде серверных элементов управления ASP.NET). Точнее говоря, реализованы вкладки **Cluster/Attribute Characteristics** и **Cluster/Attribute Discrimination** средств просмотра Cluster и Naive Bayes, а также вкладка **Decision Tree** средства просмотра Tree Viewer. Это означает, что только определенные модели можно просмотреть и только некоторые представления этих моделей можно получить.

Несмотря на то, что они предоставляют только подмножество функциональности, обеспечиваемой элементами управления Windows Forms, эти компоненты ASP.NET чрезвычайно полезны, особенно с учетом распространения Интернета. Однако эти элементы управления предоставляются только как пример кода, и поэтому их несколько сложнее использовать в ваших приложениях, чем их эквиваленты для Windows Forms.

Вы можете установить или просмотреть исходный код этих элементов управления; он доступен в форме проекта библиотеки классов на языке C# как часть примеров Microsoft SQL Server 2008 Samples. Эти примеры больше не поставляются совместно с продуктом. Их можно скачать по адресу

<http://www.CodePlex.com/SqlServerSamples>, как это описано в информации **Documentation and Tutorials** в составе SQL Server 2008. Вы можете либо просмотреть исходный код примеров на сайте, либо установить их. Если вы решите установить примеры, то установщик по умолчанию размещает исходный код Visual Studio для элементов управления в следующий каталог: C:\Program Files\Microsoft SQL Server\100\Samples\Analysis Services\DataMining\Data Mining Web Controls. Если вы откроете в этом каталоге файл решения WebControls.sln и соберете проект, то вы сможете сослаться на сгенерированную сборку в Web-сайте ASP.NET в Visual Studio и добавить три элемента управления средствами просмотра в окно панели инструментов Visual Studio Toolbox.

Вы можете использовать эти элементы управления путем размещения их на Web-форме и настройки их свойств *Server*, *Database* и *Model* (каждое из которых имеет строковый тип), а также сохранять или возвращать название сервера Analysis Services, базы данных и модели анализа (содержимое которой вы хотите показать). Вместо присваивания значений свойствам *Server* и *Database* вы можете создать объект *OleDbConnection*, подключить его к определенной базе данных Analysis Services, а затем присвоить его свойству *Connection* элемента управления.

Средство просмотра Tree Viewer имеет свойство *viewType*, которое вы должны настроить в одну из двух перечислимых констант: *Tree* или *NodeDetails*. Если вы установите его в *Tree*, то вы должны будете также настроить в свойстве *TargetTree* название прогнозируемого столбца, содержащегося в модели анализа. Если вы установите его в *NodeDetails*, то вы должны настроить свойство *TargetNode* на определенный узел дерева решений. Элементы управления Naive Bayes и Cluster Viewer имеют свойство *viewerMode*, которое также необходимо настроить в одну из двух перечислимых констант: *Discrimination* или *Characteristics*, в зависимости от того, функциональность какой вкладки средств просмотра Windows Forms вы хотите продублировать.

## **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

Дополнительную информацию по использованию элементов управления средствами просмотра ASP.NET смотрите в примерах кодов данной главы на Web-сайте этой книги. Приложение Chapter18WebClient реализует кое-какую сложную обработку, обсуждение которой выходит за рамки данной главы. Например, каждое средство просмотра заключено в оболочку как пользовательский элемент управления ASP.NET, чтобы облегчить возможность обновления средств просмотра новым содержимым моделей во время выполнения.

## Разработка управляемых хранимых процедур

Применение ваших навыков разработки OLAP к интеллектуальному анализу данных не заканчивается ADO MD.NET на клиенте; вы можете использовать библиотеку AdomdServer для выполнения на серверной стороне прогнозирования DMX или запросов содержимого и возвращения результатов. Например, в листинге 18.3 приведен код реализации версии серверной стороны для прогнозирующего запроса из скрипта LeastLikelyCar.dmx (который мы рассматривали ранее).

### Листинг 18.3. Использование AdomdServer для запросов DMX

```
using System;
using System.Data;
using Microsoft.AnalysisServices.AdomdServer;
namespace Chapter18ServerAssembly
{
    public class StoredProcedures
    {
        public DataTable GetData()
        {
            DataTable dt = new System.Data.DataTable();
            dt.Columns.Add("FirstName", typeof(string));
            dt.Columns.Add("LastName", typeof(string));
            dt.Columns.Add("Phone", typeof(string));
            dt.Columns.Add("ProbNotCarOwner", typeof(double));
            object[] row = new object[4];
            AdomdCommand comContext = new AdomdCommand
                ("SELECT TOP 250 t.FirstName, t.LastName,t.Phone,
                PredictProbability([Is Car Owner], 'N') AS ProbNotCarOwner
                FROM CarOwner_DT
                PREDICTION JOIN
                OPENQUERY([AdventureWorksDW2008],
                'SELECT c.FirstName,c.LastName,tc.*
                FROM vCustomerProfitability tc INNER JOIN dimCustomer
                c ON tc.CustomerKey = c.CustomerKey')
                AS t
                ON t.Gender = CarOwner_DT.Gender AND
                t.IncomeGroup = CarOwner_dt.[Income Group] AND
```

```

        t.MaritalStatus = CarOwner_dt.[Marital Status] AND
        t.Region = CarOwner_dt.Region
    ORDER BY PredictProbability([Is Car Owner], 'N') DESC");
try
{
    AdomdDataReader dr = comContext.ExecuteReader();
    while (dr.Read())
    {
        row[0] = dr[0];
        row[1] = dr[1];
        row[2] = dr[2];
        row[3] = dr[3];
        dt.Rows.Add(row);
    }
    dr.Close();
    return dt;
}
catch
{
    return null;
}
}
}

```

Несмотря на то, что при помощи `AdomdServer` невозможно выполнить запрос MDX, разрешается выполнить запрос DMX. В случае этой хранимой процедуры мы просто создаем объект `comContext`, передавая запрос DMX в конструктор. Мы типизируем эту функцию как возвращающую `DataTable`, и заполняем локальную переменную `DataTable` содержимым из `AdomdDataReader`. Последний получаем при помощи вызова метода `ExecuteReader` объекта `comContext`. Как и в случае с управляемым кодом *главы 17*, для выполнения этого запроса нет необходимости явно создавать подключение, он неявно использует то подключение, в контексте которого этот код вызывается.

Код листинга 18.3 взят из проекта типа `Class Library` с названием `Chapter18ServerAssembly` (из примера решения Visual Studio). Этот проект можно собрать и развернуть в базу данных `Chapter18` (при помощи методов

развертывания управляемых сборок, обсуждавшихся в *главе 17*). При развертывании сборки вы должны дать ей разрешения `ExtrenalAccess`; присваивание ей разрешений `Unrestricted` не нужно, а присваивание разрешений `Safe` не даст запросу DMX выполниться и вернуть данные должным образом.

После развертывания хранимую процедуру можно вызвать при помощи следующего кода DMX:

```
CALL Chapter18ServerAssembly.Chapter18ServerAssembly.  
    StoredProcedures.GetData()
```

Этот запрос можно выполнить в окне DMX Query в SQL Server Management Studio (он содержится в скрипте `ManagedCode.dmx` проекта Management Studio) или в приложениях `Chapter18WinClient` и `Chapter18WebClient`. Если бы код был более сложным, вам, возможно, понадобилось бы отладить его. Вы могли бы сделать это при помощи описанного в *главе 17* способа (путем прикрепления отладчика Visual Studio к процессу Analysis Services).

Помните, что ваши хранимые процедуры могут также выполнять запросы DDL при помощи языка DMX (используя метод `ExecuteNonQuery` объекта `AdomdCommand`). Объект `Database` в АМО содержит коллекцию `MiningStructures`, каждый элемент которой содержит свою коллекцию `MiningModels`. Объекты `MiningStructure` и `MiningModel` в АМО имеют множество свойств, методов и дочерних объектов, которые полезны для выполнения любых административных задач (или задач DDL) по вашим структурам и моделям интеллектуального анализа данных.

## XMLA и интеллектуальный анализ данных

Для всех задач интеллектуального анализа данных можно использовать XMLA (включая выполнение запросов DMX любого типа или создание объектов из скриптов). Как и в случае объектов OLAP, вы можете щелкнуть правой кнопкой мыши структуру анализа или модель в окне **Object Explorer** в Management Studio и выбрать пункт **Script Mining Structure As** (или **Script Mining Model As**) для генерирования скрипта XMLA (для ее создания, удаления или изменения). Аналогичным же образом, вы можете использовать опцию **View Code** (в окне **Solution Explorer** Visual Studio) для структуры анализа в проекте Analysis Services, чтобы увидеть скрипт XMLA, который будет выполнен для развертывания этой структуры анализа на сервере.



Для примера, следующий запрос (LeastLikelyCar.xmla решения Management Studio) выполняет тот же самый запрос DMX, который содержится в LeastLikelyCar.dmx, и извлекает результат в виде XML:

```
<Execute xmlns="urn:schemas-microsoft-com:xml-analysis">
  <Command>
    <Statement>
      SELECT TOP 250 t.FirstName, t.LastName, t.Phone,
        PredictProbability([Is Car Owner], 'N') AS ProbNotCarOwner
      FROM CarOwner_DT
      PREDICTION JOIN
        OPENQUERY([AdventureWorksDW2008],
          'SELECT c.FirstName, c.LastName, tc.*
          FROM vCustomerProfitability tc
          INNER JOIN dimCustomer c ON tc.CustomerKey = c.CustomerKey')
      AS t
      ON t.Gender = CarOwner_DT.Gender AND
        t.IncomeGroup = CarOwner_dt.[Income Group] AND
        t.MaritalStatus = CarOwner_dt.[Marital Status] AND
        t.Region = CarOwner_dt.Region
      ORDER BY PredictProbability([Is Car Owner], 'N') DESC
    </Statement>
  </Command>
  <Properties>
    <PropertyList>
      <Catalog>Chapter18</Catalog>
    </PropertyList>
  </Properties>
</Execute>
```

Обратите внимание, что структура запроса практически идентична XMLA-скрипту QueryCubeChapter17.xmla из главы 17. Единственная разница — изменение имени базы данных внутри тегов <Catalog> и замена запроса MDX на запрос DMX.

Однако в отличие от QueryCubeChapter17.xmla, данный запрос возвращает свои XML-данные при помощи очень простой схемы. Вот отрывок из выходного потока:

```
<return xmlns="urn:schemas-microsoft-com:xml-analysis">
  <root xmlns="urn:schemas-microsoft-com:xml-analysis:rowset"
```

```
xmlns:xsi=http://www.w3.org/2001/XMLSchema-instance
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<xsd:schema ...>
:
</xsd:schema>
<row>
  <FirstName>Gerald</FirstName>
  <LastName>Moreno</LastName>
  <Phone>1 (11) 500 555-0189</Phone>
  <ProbNotCarOwner>7.795216741405082E-1</ProbNotCarOwner>
</row>
<row>
  <FirstName>Erick</FirstName>
  <LastName>Fernandez</LastName>
  <Phone>1 (11) 500 555-0117</Phone>
  <ProbNotCarOwner>7.795216741405082E-1</ProbNotCarOwner>
</row>
<row>
  <FirstName>Alisha</FirstName>
  <LastName>Nath</LastName>
  <Phone>1 (11) 500 555-0173</Phone>
  <ProbNotCarOwner>7.795216741405082E-1</ProbNotCarOwner>
</row>
:
</root>
</return>
```

Поскольку схема значительно отличается от изучавшейся нами в *главе 17* многомерной схемы OLAP, то обсуждавшаяся там таблица стилей Extensible Stylesheet Language (XSL) для возвращаемых запросами DMX табличных результирующих наборов нам не пригодится. Однако табличная схема настолько прозрачна, что ее можно без труда разобрать при помощи различных объектов пространства System.Xml в .NET или при помощи простой таблицы стилей XSL вашего собственного изготовления.

Перед тем как закончить эту главу, давайте завершим наш обзор методик интеллектуального анализа данных разбором имеющейся в Excel 2007 (специфичной для интеллектуального анализа данных) функциональности.

## Надстройки интеллектуального анализа данных для Excel 2007

Надстройки интеллектуального анализа данных для Excel 2007 можно бесплатно скачать с сайта компании Microsoft. Эти надстройки позволяют пользователям использовать всю мощь ядра интеллектуального анализа служб Analysis Services прямо из такого инструмента, с которым хорошо знаком любой офисный работник.

### **ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ**

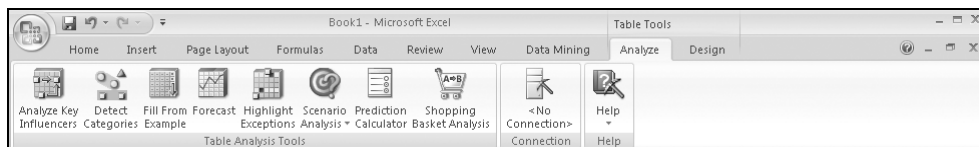
На момент написания этой книги надстройки SQL Server 2005 Data Mining Add-ins для Microsoft Office 2007 можно было скачать по адресу: <http://www.microsoft.com/sql/technologies/dm/addins.mspx>, а надстройки SQL Server 2008 Data Mining Add-ins для Office 2007 можно было скачать по адресу: <http://www.microsoft.com/downloads/details.aspx?FamilyID=896a493a-2502-4795-94ae-e00632ba6de7&DisplayLang=en>. Для инсталляции надстроек выполните инструкции на странице скачивания.

Даже если у вас нет Office 2007, то вы все равно можете экспериментировать с надстройками, для этого необходимо скачать бесплатную ознакомительную версию. Такая версия (а также полный список предварительных требований для инсталляции надстроек) имеется на сайте компании Microsoft по адресу: <http://office.microsoft.com/en-us/products/HA101741481033.aspx>.

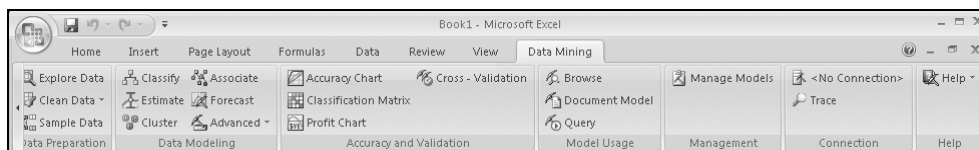
Надстройка для Excel 2007 бывает двух видов: Table Analysis Tools и Data Mining Client. Инсталляция надстройки по умолчанию ставит только версию Table Analysis Tools. Клиент Data Mining Client надо при инсталляции указывать явно. Надстройки интеллектуального анализа данных для Office 2007 содержат также компонент для использования с программой Microsoft Office Visio 2007. (Обсуждение надстройки для Visio выходит за рамки этой главы.) На рис. 18.38 показана лента Ribbon в Excel 2007 для надстройки Table Analysis Tools. На рис. 18.39 для сравнения показана лента Ribbon в Excel 2007 для надстройки Data Mining Client.

Надстройка Table Analysis Tools имеет такие опции меню в ленте Ribbon, которые ориентированы на задачи: пользователь выбирает задачу, и надстройка выбирает алгоритм для выполнения этой задачи. Нельзя явным образом указать алгоритм, который будет использован для задачи анализа в Table Analysis Tools. Несмотря на то, что каждая задача на ленте Table Analysis Tools подразумевает задачу интеллектуального анализа данных, пользователям таких подробностей не сообщают. Так, например, пользователь должен знать только то, что из данных нужно выявить категории (а не то, что для вы-

явления этих категорий следует использовать алгоритм кластеризации Microsoft Clustering). Настройка Table Analysis Tools может быть использована для анализа только тех данных, которые находятся в таблице Excel. Модели анализа создаются как сеансовые (Session Mining models), модель анализа становится недоступной после закрытия подключения (то есть после закрытия электронной таблицы Excel).



**Рис. 18.38.** Лента Ribbon в Microsoft Office Excel 2007 для надстройки Table Analysis Tools



**Рис. 18.39.** Лента Ribbon в Microsoft Office Excel 2007 для надстройки Data Mining Client

### СОВЕТ

Вы не увидите ленты Table Analysis Tools до тех пор, пока ваш курсор не окажется в ячейке таблицы Excel.

Лента клиента Data Mining Client (хотя она более знакомая и дружелюбная для среднего пользователя Office, чем Visual Studio) имеет более технический характер, чем лента Table Analysis Tools. Элементы ленты организованы в группы в соответствии с разными стадиями жизненного цикла проекта интеллектуального анализа данных: **Data Preparation**, **Data Modeling**, **Accuracy And Validation**, **Model Usage** и **Management**. Более того, при помощи опции **Advanced** в группе **Data Modeling** пользователь может указать алгоритм, который должен быть использован для выполнения задачи анализа. Задачи **Explore Data** и **Clean Data** в группе **Data Preparation** можно применить только к данным в таблице Excel, однако все остальные задачи можно применять к любым внешним данным (для которых из Analysis Services

можно выполнить подключение к источнику данных). Модели анализа могут создаваться как сеансовые модели, либо их можно сохранить на сервере (чтобы затем их можно было использовать другим пользователям и применять к новым данным).

В оставшейся части данного раздела мы пройдем по шагам, которые необходимо выполнить для оценки модели прогнозирования доходности при помощи Table Analysis Tools, добавим модель прогнозирования доходности в базу данных при помощи надстройки Data Mining Client, а также просмотрим и сгенерируем документацию для структур и моделей интеллектуального анализа данных (созданных в этой главе).

## Создание модели при помощи инструментов Table Analysis Tools

После инсталляции надстроек интеллектуального анализа данных запустите Excel 2007. В крайней правой позиции панели инструментов **Quick Launch** появилась новая кнопка **Data Mining**. Вам может показаться, что инсталляция надстройки Table Analysis Tools прошла неудачно (поскольку, несмотря на то, что при выборе **Data Mining** в меню **Quick Launch** отображается показанная ранее на рис. 18.39 лента **Data Mining Client**, нет никакой возможности попасть на ленту **Table Analysis Tools**, показанную на рис. 18.38). Это потому, что надстройка Table Analysis Tools может быть использована только с данными таблицы Excel (как уже упоминалось ранее). Поэтому для того, чтобы увидеть **Table Analysis Tools** в меню **Quick Launch**, вы должны создать таблицу Excel.

## Резюме

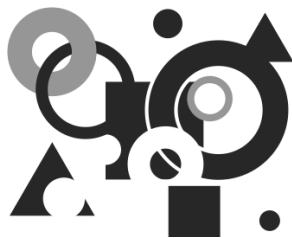
Службы SQL Server Analysis Services 2008 имеют полный набор инструментов для решения проблем интеллектуального анализа данных. Эта глава представила вам данные инструменты и многие способы их применения, начиная от создания случайных выборок и до представления результатов в виде приложения для бизнес-пользователей. Мы представили вам мастер Data Mining Wizard, который ведет вас по шагам создания структуры и модели анализа. Вы увидели, как добавлять и редактировать модели анализа, подстраивать параметры алгоритмов на вкладке **Mining Models** визуального конструктора структур интеллектуального анализа данных.

Для каждого алгоритма имеется несколько средств просмотра, что позволяет проанализировать результаты с разных точек зрения. Вы создали диаграмму точности прогнозирования для сравнения работы разных моделей.

В кратком введении вы увидели, как использовать язык запросов DMX для тех же задач, что и Data Mining Wizard. Затем вы узнали, как применить DMX программным путем в контексте кубов OLAP. ADO MD.NET обеспечивает полную поддержку интеллектуального анализа данных, и вы можете без труда встроить мощные средства просмотра моделей анализа в пользовательское приложение или использовать ядро служб Analysis Services непосредственно из Excel.

Компонент интеллектуального анализа данных служб Analysis Services отличается простотой использования и высокой степенью программируемости. Разработанные к нему дополнения ставят компанию Microsoft на первое место среди поставщиков решений бизнес-аналитики.

## ГЛАВА 19



# Службы отчетности Reporting Services

*Элси Пен и Джефф Болтон*

Службы отчетности Reporting Services сервера Microsoft SQL Server 2008 — это всестороннее решение на базе сервера, которое поддерживает полный жизненный цикл отчетности, от создания и управления отчетами до обеспечения безопасности и доставки конечным пользователям. В этой главе мы изучим все фазы этого цикла, пройдем по нескольким реальным сценариям разработки отчетов. К концу этой главы вы изучите все, что нужно для эффективного использования служб отчетности Reporting Services в SQL Server 2008.

В дополнение к реляционным базам данных SQL Server, службы отчетности Reporting Services поддерживают базы данных Analysis Services, Oracle и DB2. Фактически в отчете может быть использован любой источник данных, для которого есть поставщик OLE-DB или драйвер Open Database Connectivity (ODBC). Можно также комбинировать в один отчет данные из нескольких источников. Разработчики могут управлять форматированием и видимостью элементов отчета, создавать динамические отчеты, которые позволяют конечным пользователям принять решение о направлении и глубине анализа данных. Отчеты допускают встраивание кода скриптов Microsoft Visual Basic .NET или ссылки на .NET-сборку вызова классов и методов управляемого кода.

После создания отчетов они должны быть развернуты на сервере отчетности. Это можно сделать при помощи Microsoft Visual Studio 2008, пользовательского Web-интерфейса Report Manager или инструмента командной строки. Мы изучим и сравним все эти методы развертывания. Кроме имеющегося в

службах Reporting Services Web-интерфейса отчетности, вы можете визуализировать отчеты из любого .NET-приложения (при помощи элемента управления ReportViewer). Две Web-службы, называемые конечными точками SOAP, используются в управлении отчетами и их генерирования.

Огромную роль играют безопасность и надежность как данных, так и самой системы. Этим темам посвящен краткий обзор инструмента конфигурирования (который в версии SQL Server 2008 Reporting Services был значительно улучшен). Поддерживается доставка отчетов пользователям по событию или по расписанию. Обеспечение безопасности отчетов и данных достигается на уровне сайта и на уровне объектов (полный набор настраиваемых ролей и задач). И наконец, для обеспечения единообразной платформы отчетности предприятия Reporting Services легко интегрируется с Microsoft Office SharePoint.

Теперь мы готовы погрузиться в службы отчетности! Жизнь любого отчета начинается с визуального конструктора отчетов Report Designer, именно с него мы и начнем. Сначала мы создадим несколько отчетов, которые демонстрируют мощные функциональные возможности Reporting Services. Затем мы обратим свое внимание на доставку отчетов, которая (как мы уже кратко отметили) включает в себя развертывание, безопасность, разработку приложений и интеграцию.

## Использование конструктора отчетов

Report Designer — инструмент для разработчиков. Графически конструируемые при его помощи отчеты сохраняются в формате специального диалекта XML — RDL (report definition language). Эти файлы RDL в свою очередь развертываются на сервере отчетности и визуализируются для запрашивающих пользователей. Report Designer — это компонент студии Business Intelligence Developer Studio, а следовательно, и компонент Visual Studio. Этот инструмент был значительно улучшен в версии Reporting Services 2008.

### **ПРИМЕЧАНИЕ**

Из Visual Studio 2005 вы можете опубликовать отчет только на сервер отчетности SQL Server 2005 Reporting Services. Из Visual Studio 2008 вы можете публиковать только на сервер отчетности SQL Server 2008 Reporting Services. Для того чтобы модернизировать отчеты версии 2005, необходимо просто открыть файл дизайна отчета (файл rdl) в Visual Studio 2008. Появится окно с запросом вашего подтверждения на обновление отчета до версии 2008 схемы RDL. После обновления отчета для его редактирования



вы должны будете использовать версию Report Designer 2008 г., вернуться к прежней версии невозможно.

Даже при самом поверхностном взгляде видно, что интерфейс Report Designer был преобразован (рис. 19.1). Панель **Report Data** закреплена на левой стороне экрана — снаружи окна самого визуального конструктора. Создание и настройка параметров отчетов и изображений теперь тоже делаются в панели **Report Data**. В нижнюю часть вкладки **Design** добавлена новая панель **Grouping** — для интуитивно понятного перетаскивания при создании и настройке групп.

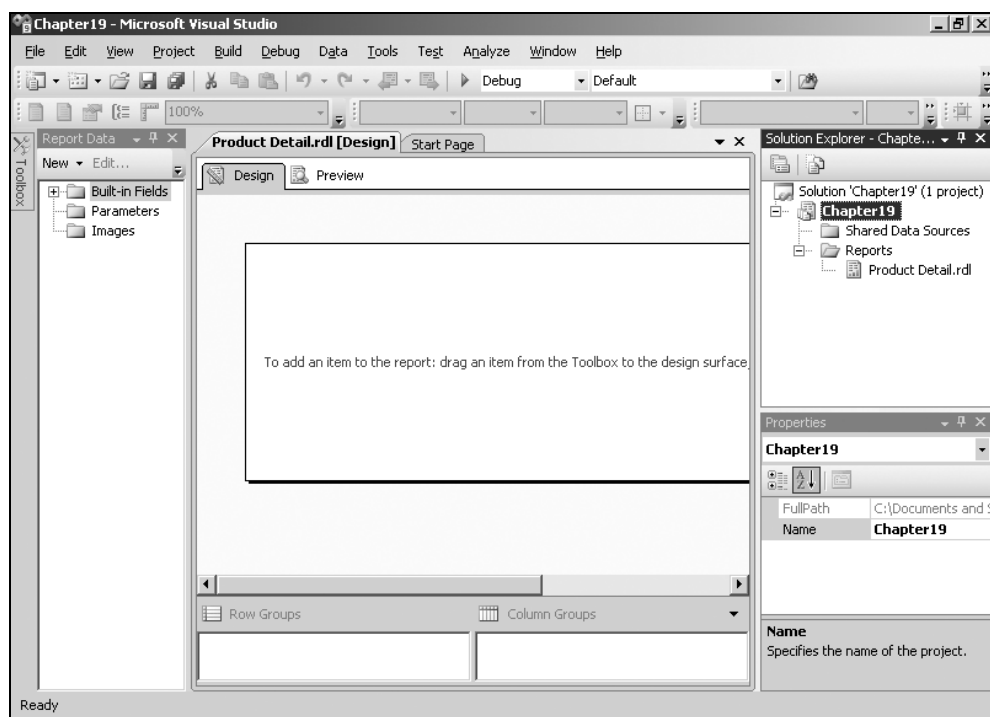


Рис. 19.1. Новый визуальный конструктор Report Designer в Visual Studio 2008

Код для работы с областями данных отчетов был переработан и расширен, чтобы реализовать значительно более гибкие конфигурации таблиц и матриц, а также более сложные и богатые опциями диаграммы. Раньше для областей данных списков, таблиц и матриц использовался разный код, что иногда приводило к отсутствию единообразия при настройке этих типов областей дан-

ных. Сейчас код для работы с областями данных в Reporting Services 2008 унифицирован на основе новой области `tablix` (с шаблоном списка, таблицы или матрицы соответственно). Поэтому, несмотря на то, что в панели инструментов отчетности вы продолжаете выбирать области данных типа списка, таблицы или матрицы, фактически вы создаете область данных `tablix`.

Унификация кода дает также и новую гибкость при создании областей данных. Например, теперь можно добавить в матрицу фиксированный столбец или фиксированную строку и указать, какие столбцы данных должны иметь промежуточные итоги (раньше приходилось включать промежуточные итоги либо для всех столбцов матрицы, либо вовсе их не включать). Конкретные примеры иллюстрации повышенной гибкости областей данных `tablix` даны далее в этой же главе.

В 2007 г. компания Microsoft приобрела программный продукт Dundas Data Visualization, и в результате этого функциональность диаграмм в службах отчетности значительно улучшилась. Если говорить о начальном уровне, то теперь есть поддержка двухосных диаграмм (это то, чего чрезвычайно не хватало в более ранних версиях Report Designer). Пользователи, которым нужны более продвинутые возможности, будут счастливы обнаружить в Reporting Services поддержку пошаговых графиков, пирамид, диаграмм Ганта и др. Полный список имеющихся типов диаграмм имеется в документации SQL Server Books Online.

После выпуска версии SQL Server 2008 визуальный конструктор Report Designer будет поставляться также и как отдельное приложение для настольного компьютера с пользовательским интерфейсом типа Ribbon. Этот интерфейс показан на рис. 19.2. Код автономного Report Designer и визуального конструктора отчетов в Visual Studio один и тот же; поэтому диалоговые окна и возможности одни и те же. Те разработчики, которые привыкли работать в Visual Studio и хотят комбинировать в одном решении проекты Reporting Services и другие проекты, предпочтут работать в Visual Studio. В то же время знакомый по Office интерфейс автономного приложения сделает проектирование отчетов доступным для целой новой группы пользователей.

### **ПРИМЕЧАНИЕ**

В этой главе используется база Adventure Works хранилища данных и база данных Analysis Services. Указания по скачиванию и установке баз данных даны во введении к этой книге.

Теперь мы создадим несколько отчетов и продемонстрируем возможности визуального конструктора Report Designer в Visual Studio 2008.

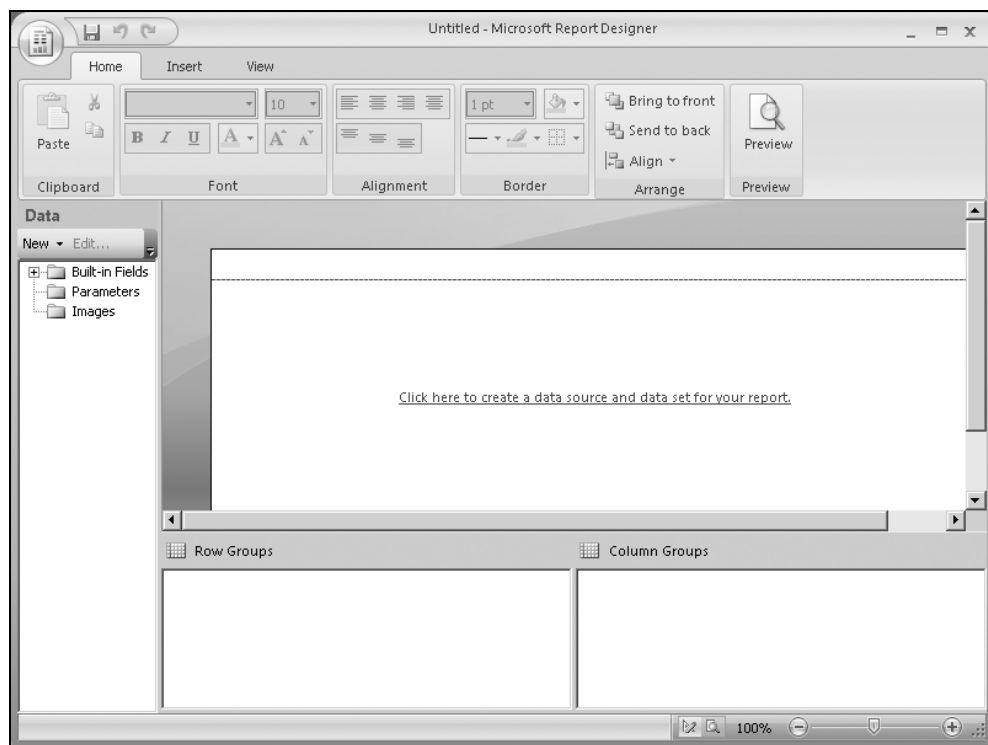


Рис. 19.2. Вид автономного приложения Report Designer

## Создание простого отчета

Мы начнем с очень простого отчета, в котором показаны продукты, продаваемые через Интернет вымышленной велосипедной компанией Adventure Works. Мы полностью создадим (с нуля) и отформатируем его, чтобы понять входящие в отчет элементы. Позднее в этой же главе мы создадим отчет при помощи мастера Report Wizard, который через целый набор диалоговых окон выполнит за нас множество задач по форматированию.

Для начала мы создадим проект и решение Visual Studio для нашего отчета. Запустите Visual Studio 2008, а затем выберите тип проекта Business Intelligence и шаблон Report Server Project. Назовите свой проект Chapter19. По умолчанию будет создано также и решение Chapter19. Щелкните правой кнопкой мыши по узлу **Reports** в окне **Solution Explorer**, добавьте новый отчет посредством контекстного меню. Выберите шаблон **Report**, назовите

отчет `Product Detail.rdl`, а затем щелкните по кнопке **Add**, переходя в интерфейс визуального конструктора Report Designer (см. рис. 19.1).

### **ПРИМЕЧАНИЕ**

Если в визуальном конструкторе Report Designer панель **Report Data** не видна, то вы можете показать ее при помощи пункта **Report Data** в меню **View**.

## **Наборы данных для отчета**

Если вы знакомы с Report Designer в Visual Studio 2005, вы заметите, что в новом Report Designer в Visual Studio 2008 отсутствует вкладка **Data**, весь функционал из нее был перемещен в панель **Report Data**. Раньше в панели данных просто перечислялись поля в отчете, приходилось переключаться между вкладками **Data** и **Layout** (чтобы увидеть источник запроса для наборов данных). Теперь вся информация об источниках и наборах данных легко доступна в панели **Report Data**, причем вы не теряете из вида ваш отчет.

Первый шаг создания отчета — указать набор данных для отчета. Нажмите кнопку **New** в панели **Report Data**, а затем выберите пункт **Dataset**. Назовите набор данных отчета `ProductDetail`. Нам нужно указать источник данных для набора. Поскольку это наш первый отчет, то мы еще не определили ни одного источника данных для отчетов. Нажмите кнопку **New** рядом с выпадающим списком **Data Source**. После этого откроется диалоговое окно **Data Source Properties**.

Назовите источник данных `AdventureWorksDW2008`. У вас есть выбор, создать встроенный или совместно используемый источник данных. Информация подключения для встроенного источника данных хранится как часть RDL в файле отчета и доступна только тому отчету, в котором она определена. Создание же совместно используемого источника данных открывает отдельный файл с информацией о подключении к данным. Совместно используемый источник данных доступен любым отчетам, созданным в том же самом проекте отчета. Совместно используемые источники данных развертываются отдельно от отчетов и после развертывания доступны любым отчетам на том же самом сервере отчетности. Для нашего отчета мы создадим совместно используемый источник данных.

Выберите опцию **Use Shared Data Source Reference**, а затем нажмите кнопку **New**. В диалоговом окне **Shared Data Source Properties** назовите источник данных `AdventureWorksDW2008`, убедитесь, что в качестве типа источника вы-

бран Microsoft SQL Server, а затем нажмите кнопку **Edit** справа от текстового поля **Connection String**.

В диалоговом окне **Connection Properties** введите название сервера (на котором хранится база данных AdventureWorksDW2008) и укажите регистрационные данные этого сервера. Выберите AdventureWorksDW2008 в выпадающем списке **Select Or Enter A Database Name**. (Вы можете также при желании нажать кнопку **Test Connection**, чтобы убедиться в правильности подключения). Затем нажмите кнопку **OK**, чтобы закрыть диалоговое окно **Connection Properties**. Нажмите кнопку **OK** еще раз (чтобы закрыть диалоговое окно **Shared Data Source Properties**). И наконец, в диалоговом окне **Data Source Properties** выберите AdventureWorksDW2008 в выпадающем списке **Use Shared Data Source Reference**, а затем нажмите кнопку **OK** для возвращения в диалоговое окно **Dataset Properties**.

Теперь в диалоговом окне **Dataset Properties** в поле **Data Source** должна быть выбрана база AdventureWorksDW2008. Убедитесь, что опция **Query Type** установлена в значение **Text**, а затем введите в окне следующий код (или нажмите кнопку **Import** и выберите файл SelectProductDetail.sql из файлов кодов этой главы):

```
SELECT
    p.ProductKey,
    pc.EnglishProductCategoryName,
    ps.EnglishProductSubcategoryName,
    p.EnglishProductName,
    p.EnglishDescription,
    p.StandardCost,
    p.ListPrice
FROM dbo.DimProduct AS p
    INNER JOIN dbo.DimProductSubcategory AS ps
        ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
    INNER JOIN dbo.DimProductCategory AS pc
        ON ps.ProductCategoryKey = pc.ProductCategoryKey
```

Этот запрос возвращает список продуктов DimProduct вместе с подкатегориями и категориями продуктов DimProductSubcategory и DimProductCategory. Диалоговое окно **Dataset Properties** теперь должно выглядеть аналогично рис. 19.3.

После ввода запроса в окно запросов нажмите кнопку **OK**. Окно **Dataset Properties** закроется, и вы увидите источник данных AdventureWorksDW2008 и новый набор данных в панели **Report Data**. Под названием набора данных

вы увидите поля, которые были указаны в запросе. Эти поля теперь можно отображать в отчете. Поскольку источник данных был создан как совместно используемый, вы увидите также и новый файл **rds** в узле **Shared Data Sources** окна **Solution Explorer**.

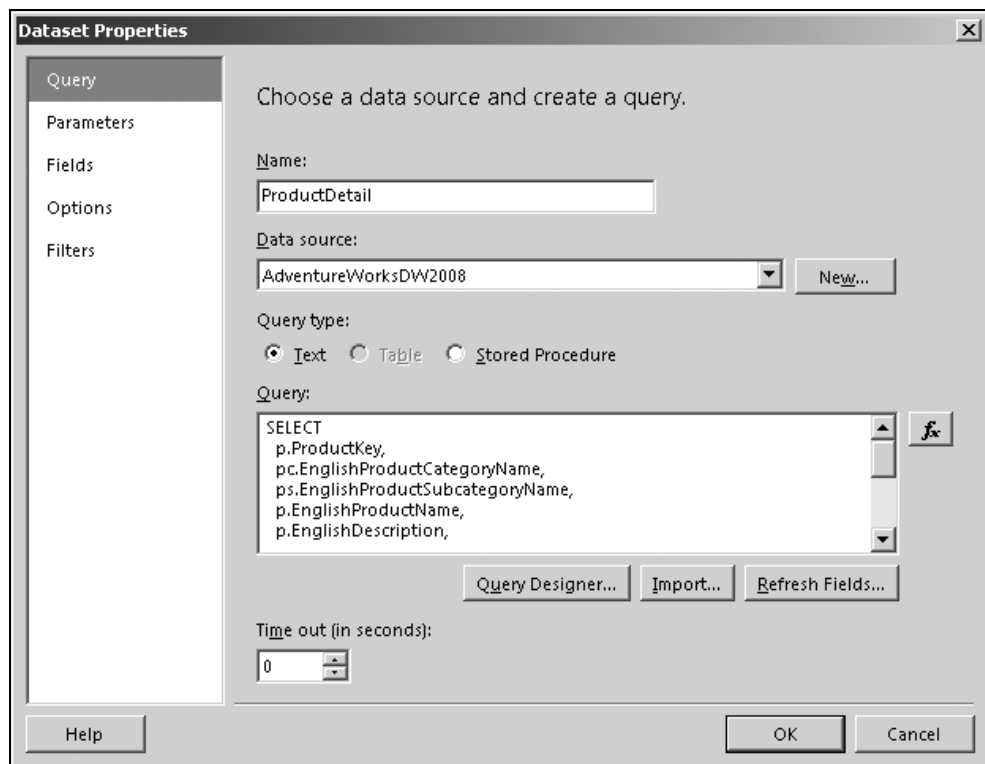


Рис. 19.3. Заполненное диалоговое окно **Dataset Properties**

### СОВЕТ

Визуальный конструктор Report Designer позволяет вам импортировать запросы из скриптов SQL (файлы .sql) или других отчетов (файлы .rdl). Для этого надо просто нажать кнопку **Import**, а затем перейти к файлу скрипта SQL или отчета. Если выбран файл отчета, то визуальный конструктор покажет список всех определенных наборов данных, а также соответствующие им запросы. Это облегчает совместное использование запросов между отчетами, а также копирование запроса (из существующего отчета) для последующей его модификации. В идеальном случае следовало бы исполь-

зовать хранимые процедуры (как с точки зрения производительности запроса, так и в смысле многократного использования); однако в случае отсутствия хранимой процедуры возможность импорта очень удобна для быстрого применения запросов из одного отчета в другом отчете.

Теперь вы успешно определили совместно используемый источник, набор данных для отчета и готовы начать конструирование отчета.

## Добавление области данных

Выбор области данных определяет компоновку (визуализацию) показанных в отчете данных. В Reporting Services существуют два типа областей данных:

- `tablix` — это новая область данных, введенная в Reporting Services 2008. Допускает фиксированные или переменные строки и столбцы (в любой комбинации) с группированием по строкам, столбцам или по строкам и столбцам одновременно;
- `chart` — графическое представление данных.

Как мы уже описывали ранее, область данных типа `tablix` в Reporting Services 2008 — это общая форма для представления следующих трех областей данных (имевшихся ранее в Reporting Services 2005):

- `list` — свободная компоновка данных;
- `table` — фиксированные столбцы и переменные строки с поддержкой группирования по строкам. Количество строк зависит от количества записей в наборе данных;
- `matrix` — переменные столбцы и переменные строки с поддержкой группирования по строкам и столбцам, как в PivotTable и перекрестной компоновке. Количество столбцов и строк зависит от количества уникальных значений в том поле, по которому группируются столбцы или строки.

В Reporting Services 2008 можно легко перейти от табличной области данных к матричной или гибриднему типу `tablix`. Технически можно также изменить тип со списка на таблицу или со списка на матрицу, но на практике это может потребовать дополнительной работы (если поля данных списка еще не упорядочены в подобный сетке формат). Главная идея в том, что выбор области данных в панели инструментов просто указывает стартовую точку для компоновки, при последующей разработке отчета у вас будет большая гибкость при добавлении групп строк и столбцов.

Для этого отчета мы будем отображать данные по продуктам в таблице (столбцы отчета будут фиксированные, но строки — переменные), четыре

фиксированных столбца: название продукта, описание продукта, нормативная себестоимость и прейскурантная цена. Количество строк отчета будет равно общему количеству продуктов, возвращенному запросу.

Выберите **Table** в панели инструментов, а затем перетащите на поверхность конструирования отчета. (Если вы не видите панели инструментов, то сначала выберите **Toolbox** в меню **View**). После перетаскивания таблицы на поверхность конструирования переместите ее таким образом, чтобы левый верхний угол таблицы был вровень с левым верхним углом поверхности конструирования. Вы можете сделать это при помощи выделения таблицы и перетаскивания ее в левый верхний угол. Индикация выравнивания верхней и левой сторон таблицы с верхней и левой сторонами поверхности конструирования имеет синие линии привязки. Вы можете перейти в окно **Properties** таблицы, найти свойство **Location** и настроить **Left** и **Top** в значения 0 (дюймов).

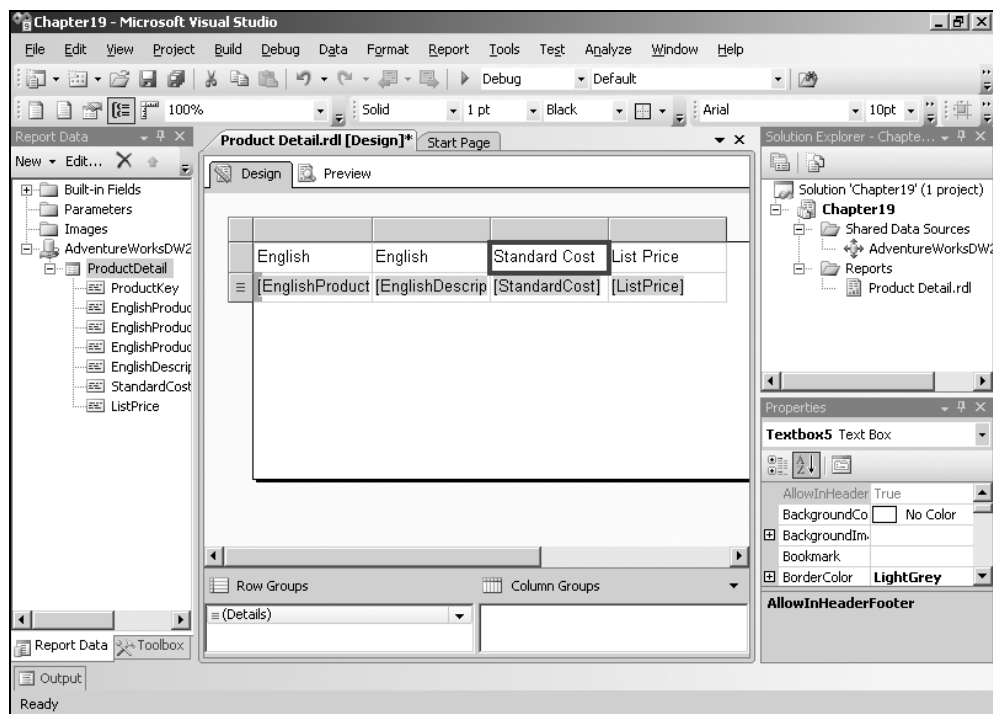


Рис. 19.4. Представление Design отчета Product Detail

Для показа в таблице данных из набора данных **ProductDetail** перетащите поля из набора данных на строку данных таблицы. Для нашего примера отче-



та выберите `EnglishProductName` в панели **Report Data** и перетащите его в самый левый столбец таблицы. Для выбора поля из набора данных вы можете также использовать новые смарт-теги в ячейках таблиц. Наведите курсор мыши на среднюю ячейку в строке данных таблицы. В правом верхнем углу ячейки появится смарт-тег. Нажмите этот смарт-тег, и вам будет показан список имеющихся полей. Выберите `EnglishDescription` из этого списка. Либо перетащите столбец `StandardCost` в самую правую ячейку строки данных, либо используйте смарт-тег для выбора `StandardCost`. Добавьте в таблицу `ListPrice` (при помощи выбора поля в панели **Report Data** и перетаскивания его к правой границе таблицы). На правой границе таблицы появится синяя линия привязки. Для прейскурантной цены в таблицу добавим четвертый столбец. Когда вы закончите добавление в отчет всех четырех полей, поверхность конструирования должна будет выглядеть, как на рис. 19.4.

## Предварительный просмотр отчета

Щелкните по вкладке **Preview**. Отчет генерируется и отображается так, как он будет выглядеть для пользователей. Использование вкладки **Preview** — это удобный способ проверки форматирования и функциональности отчета во время его разработки (без необходимости делать его развертывание).

## Применение форматирования отчетов

Вы успешно создали свой первый отчет, но в эстетическом плане он выглядит не очень хорошо. Текст в ячейках переносится. Заголовки столбцов не отличаются от строк данных, и более того, заголовки столбцов имеются только на первой странице отчета. Иначе говоря, это очень скучный отчет. Давайте поработаем над улучшением внешнего вида отчета.

## Добавление заголовка страницы

Сначала давайте добавим в отчет заголовков. Щелкните по вкладке **Design** для того, чтобы вернуться на поверхность конструирования отчета. Затем щелкните правой кнопкой мыши в любом месте страницы отчета (на поверхности конструирования) и выберите пункты **Insert | Page Header** контекстного меню. На поверхность конструирования будет добавлен заголовок страницы. Щелкните правой кнопкой мыши в любом месте заголовка страницы, а затем выберите пункты **Insert | Textbox** контекстного меню. Выделите и перетащите текстовое поле таким образом, чтобы левый верхний угол поля был вы-

ровнен с левым верхним углом заголовка страницы. Дважды щелкните по текстовому полю (для того чтобы активировать курсор внутри него), а затем введите текст в поле **Product Detail**.

### **СОВЕТ**

Для выбора такого элемента отчета, как текстовое поле или ячейка таблицы, щелкните по ним один раз. Затем для модификации текста внутри текстового поля или ячейки щелкните по этому полю или ячейке дважды.

После выбора текстового поля используйте панель инструментов форматирования (над поверхностью конструирования) для того, чтобы изменить параметры текста на полужирный шрифт, цвет фона **SteelBlue**. Если вы не видите нужный вам цвет в меню **Background Color** панели инструментов форматирования **Formatting**, щелкните по ссылке **More Colors**. Теперь нажмите вкладку **Preview**. Заголовок отчета переносится, но мы собираемся это исправить.

Вернитесь на вкладку **Design**. Щелкните правой кнопкой мыши в любом месте поверхности конструирования вне области страницы, а затем выберите пункты **View | Ruler** контекстного меню. Вдоль верхней и левой сторон поверхности конструирования появятся линейки. Вы можете использовать эти линейки для изменения размеров элементов на поверхности конструирования. Выберите текстовое поле с заголовком отчета в заголовке страницы. Используйте манипуляторы текстового поля для его расширения до размера 3.

## **Использование встроенных полей (глобальных)**

Теперь мы добавим в отчет номера страниц. В панели **Report Data** разверните узел **Built-in Fields**. Вы увидите список имеющихся глобальных переменных отчетов. Найдите поле **Page Number**. Перетащите его на заголовок страницы, под текстовое поле заголовка. При необходимости подстройте положение добавленного текстового поля при помощи синих линий привязки, чтобы оно было вровень с низом текстового поля заголовка. Сделайте предварительный просмотр отчета и убедитесь, что номера страниц добавлены; затем вернитесь на вкладку **Design**.

Если вы посмотрите на список встроенных полей, то заметите имя отчета в виде встроенного поля. Замените жестко запрограммированное вами название отчета встроенным полем (перетащив это поле из панели **Report Data** в текстовое поле заголовка). Текст в текстовом поле заменяется встроенным полем. Если теперь вы измените название вашего файла отчета, то заголовок отчета автоматически изменится, и соответствие сохранится.

## Написание выражений отчета

Для того чтобы отредактировать текст номера строки, щелкните правой кнопкой мыши по текстовому полю, в котором имеется объект `PageNumber`, а затем выберите пункт **Expression** контекстного меню. После этого будет показано диалоговое окно **Expression Editor**. Обратите внимание, что в окне **Expression Editor** переменная `&PageNumber` записана как `Globals!PageNumber`, поскольку номер страницы является членом коллекции `Globals`. Измените выражение отчета таким образом, чтобы оно включало общее количество страниц (при помощи еще одного элемента коллекции `Globals`):

```
= "Page " & Globals!PageNumber & " of " & Globals!TotalPages
```

Выражениям отчета должен всегда предшествовать знак равенства (=), и они всегда написаны на языке Visual Basic. Предыдущий код — это простой, но типичный пример использования выражения отчета для комбинирования статичного текста со встроенными полями коллекции `Globals` (чтобы сделать текст заголовка страницы динамичным). Выражения отчета являются фундаментальным элементом разработки динамичных и многофункциональных отчетов. Нижняя панель диалогового окна **Expression Editor** предоставляет помощь в построении выражений отчетов. Потратьте некоторое время на изучение доступных для построения отчетов коллекций и функций. Далее в этой главе будут даны примеры, иллюстрирующие использование выражений отчетов.

### СОВЕТ

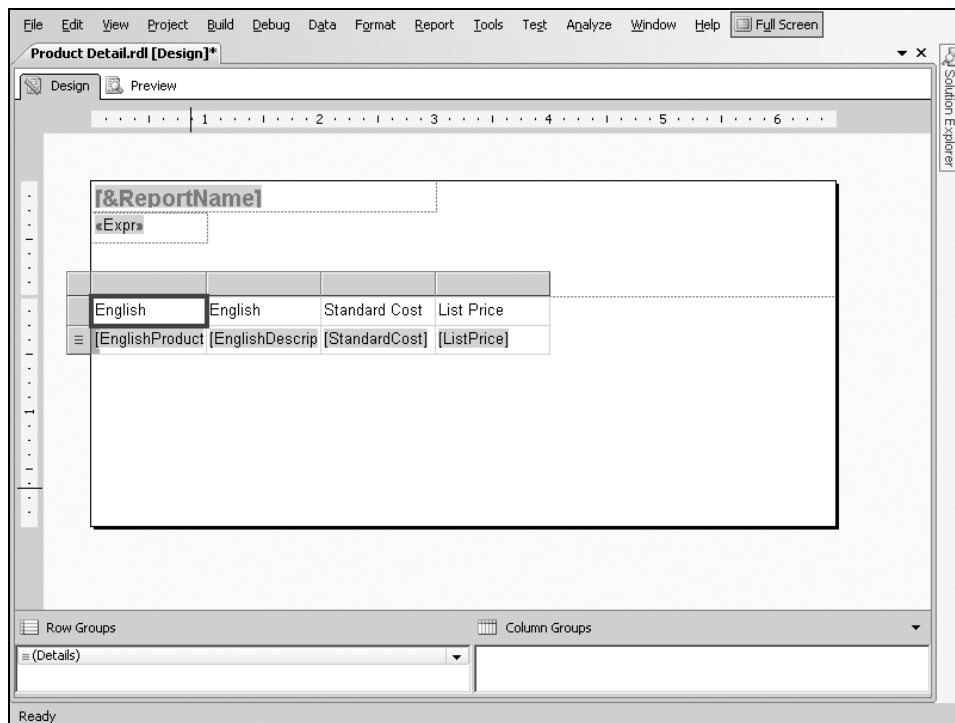
Редактор выражений может быть запущен везде, где вы видите кнопку **Expression** (обозначенную значком функции) либо опцию **Expression** (контекстного меню или выпадающего списка). Выражения позволяют определять значения свойств динамически (а не просто устанавливать в постоянные значения). Условное форматирование в службах отчетности Reporting Services реализовано при помощи использования выражений отчетов.

Нажмите кнопку **ОК** для сохранения выражения отчета и чтобы закрыть диалоговое окно **Expression Editor**. На поверхности конструирования в текстовом поле вы увидите `<<Expr>>`. Так визуальный конструктор Report Designer сообщает вам о том, что в текстовом поле используется выражение.

## Форматирование области данных

Щелкните по ячейке таблицы, чтобы стали видимыми серые манипуляторы столбцов и строк (рис. 19.5). Выберите строку с заголовками столбцов. При

этом будет выбрана вся верхняя строка таблицы. Используйте панель инструментов **Formatting** для изменения атрибутов текста на полужирный шрифт, белый цвет переднего плана, цвет фона SteelBlue.



**Рис. 19.5.** Представление Design, демонстрирующее область данных таблицы с видимыми манипуляторами строк и столбцов

Дважды щелкните по заголовку **English Product Name** и измените этот текст на Product Name. Теперь дважды щелкните по заголовку **English Description** и измените этот текст на Description. Расширьте столбцы (перетащив линию между серыми манипуляторами столбцов вправо). Для более точного управления шириной столбцов выберите столбец таблицы (при помощи щелчка по серому манипулятору столбца), найдите его свойство ширины в панели **Properties**, а затем введите значение ширины.

### СОВЕТ

Панель **Properties** обеспечивает простой доступ к свойствам выбранного в данный момент элемента отчета. Если панель **Properties** не видна, то

вы можете показать ее при помощи выбора пункта **Properties** в меню **View**. Выпадающий список в верхней части панели **Properties** указывает текущий выбранный элемент отчета. Когда на панели конструирования отчета выбраны несколько элементов, то в панели **Properties** показыва-ются общие для выбранных элементов свойства. Поэтому в панели **Prop-erties** можно изменить свойство для всех выбранных элементов одно-временно.

Щелкните по вкладке **Preview** для предварительного просмотра отчета. Про-верьте форматирование и при необходимости вернитесь на вкладку **Design** для выполнения изменений.

## Добавление в отчет групп

Теперь наш отчет показывает по одной строке для каждого продукта. Но продуктов — сотни, и мы хотим помочь пользователям перемещаться по списку продуктов. Мы можем сделать это при помощи добавления в отчет группы строк.

Щелкните правой кнопкой мыши по группе (Details) в поле **Row Groups** пане-ли **Grouping**, а затем выберите пункты **Add Group | Parent Group** контекстного меню. В диалоговом окне выберите группу EnglishProductSubCategoryName в выпадающем списке **Group By**. В таблицу на поверхности конструирова-ния автоматически добавляется столбец. Дважды щелкните по заголовку нового столбца **English Product Sub Category Name**, а затем измените за-головков столбца на Product Subcategory. Сделайте предварительный про-смотр отчета. Вы увидите, что теперь продукты сгруппированы в отчете по подкатегориям.

Теперь повторите этот процесс, добавив EnglishProductCategoryName как родительскую группу для группы EnglishProductSubCategoryName. Дважды щелкните по заголовку вновь добавленного столбца таблицы на поверхности конструирования и измените текст на Product Category.

Теперь мы расширим столбцы категории и подкатегории продуктов. Выдели-те каждый столбец (серые манипуляторы столбцов в верхней части табли-цы). В панели свойств найдите свойство **Size**. Разверните свойство **Size**, что-бы увидеть свойства **Width** и **Height**. Установите в свойстве **Width** значение 2 (рис. 19.6).

Сделайте предварительный просмотр отчета. Обратите внимание, что теперь продукты сгруппированы сначала по категории, а затем по подкатегории.

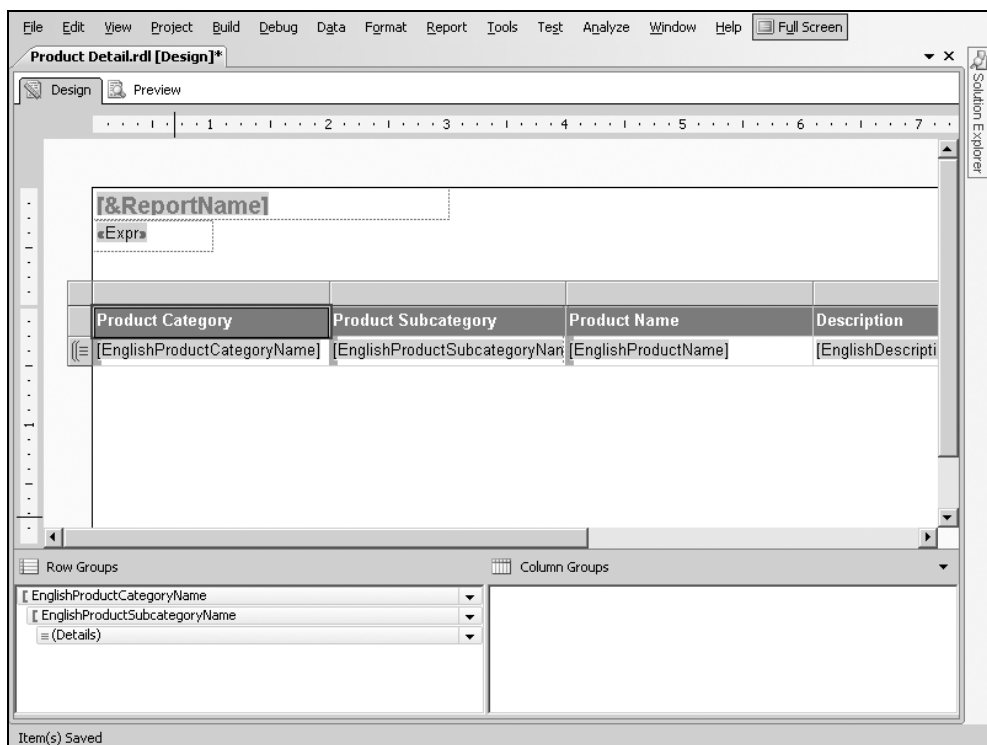
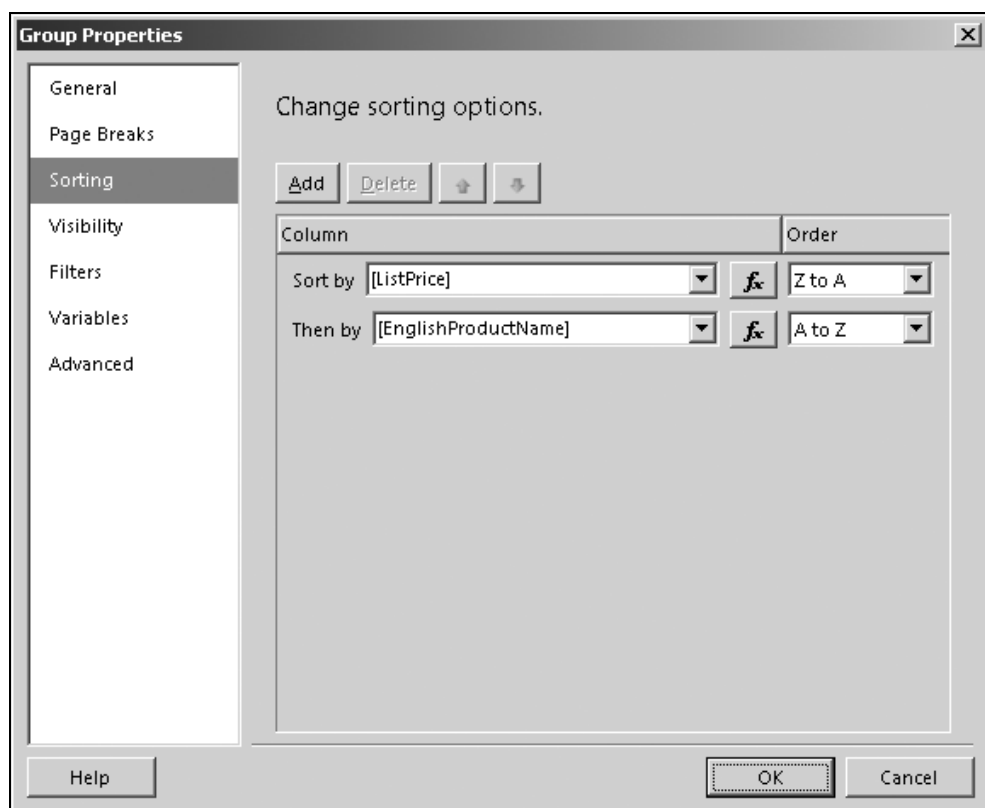


Рис. 19.6. Представление Design с настроенными значениями Row Groups

## Указание порядка сортировки

По умолчанию группы строк сортируются по полю группирования. По отчету по-прежнему трудно перемещаться, поскольку внутри подкатегорий продукты не отсортированы. Для исправления этой проблемы нажмите стрелку справа от группы (Details) в панели **Grouping**, а затем выберите **Group Properties**. В списке переходов слева в диалоговом окне **Group Properties** выберите страницу **Sorting**. Нажмите кнопку **Add**, выберите ListPrice в выпадающем списке и измените **Order** на **Z To A** (то есть убывающая последовательность, чтобы сначала в списке видеть продукты с самыми высокими ценами в каждой подкатегории). Еще раз нажмите кнопку **Add**, а затем выберите EnglishProductName во втором выпадающем списке, порядок **Order** оставьте **A To Z** (возрастающий). Сделанные вами настройки должны выглядеть, как на рис. 19.7. Нажмите кнопку **OK**, чтобы сохранить сделанные вами настройки и закрыть диалоговое окно.



**Рис. 19.7.** Диалоговое окно **Group Properties** с указанными настройками сортировки

Сделайте предварительный просмотр отчета и убедитесь, что порядок сортировки совпадает с только что сделанными вами настройками. Нажмите кнопку **Save All** на панели инструментов (чтобы сохранить файлы решения и проекта).

## Работа с параметрами

Нашим пользователям, скорее всего, не нужно будет видеть сразу все продукты, и мы можем дать более высокую степень управления теми данными, которые предоставляются в отчете.

Отчеты имеют параметры двух типов: параметры запросов и параметры отчетов. Параметры запросов используются в запросе к набору данных (как пе-

редаваемые в хранимые процедуры входные параметры или указанные в параметризированной инструкции T-SQL переменные). Параметры запросов обрабатываются сервером, на котором находится источник данных.

Первый шаг создания параметра запроса — это модификация запроса по набору данных путем добавления параметризованного выражения `WHERE` для фильтрации возвращаемых запросом результатов. Щелкните правой кнопкой мыши по набору данных `ProductDetail` под источником данных `AdventureWorksDW2008` в панели **Report Data**, а затем выберите **Query**. Измените текст запроса в окне **Query Designer** (включите в него параметризованное выражение `WHERE`). Можно импортировать скрипт `ParameterizedSelectProductDetail.sql` из кодов этой главы либо просто добавить выражение `WHERE`:

```
SELECT
    p.ProductKey,
    pc.EnglishProductCategoryName,
    ps.EnglishProductSubcategoryName,
    p.EnglishProductName,
    p.EnglishDescription,
    p.StandardCost,
    p.ListPrice
FROM dbo.DimProduct AS p
    INNER JOIN dbo.DimProductSubcategory AS ps
        ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
    INNER JOIN dbo.DimProductCategory AS pc
        ON ps.ProductCategoryKey = pc.ProductCategoryKey
WHERE pc.EnglishProductCategoryName = @ProductCategory
```

### **ПРИМЕЧАНИЕ**

Количество строк в наборе данных можно ограничить и при помощи указания фильтра для набора данных. Принципиальная разница между параметрами запроса и фильтрами состоит в том, что параметры применяются до того, как данные возвращаются в отчет, а фильтры применяются после того, как данные возвращаются в отчет. Более того, фильтры могут применяться только к тем столбцам, которые возвращаются в наборе данных, в то время как параметры запросов могут фильтровать и по тем столбцам, которые не возвращаются в наборе данных (при условии, что это правильные в контексте данного запроса столбцы). Из соображений производительности рекомендуется по возможности применять параметры, а не фильтры. Это снижает количество записей в результирующем наборе запроса и количество записей, возвращаемых в набор данных отчета по сети. Более того, база данных наверняка имеет индексы, которые позволяют де-



лать более быстрый поиск по значениям параметров (в то время как фильтры после запроса не могут использовать оптимизацию такого рода).

Нажмите кнопку **ОК**, чтобы закрыть окно **Query Designer**. Затем разверните узел **Parameters** в панели **Report Data**. Обратите внимание, что в результате модификации запроса был автоматически добавлен новый параметр с названием `ProductCategory`. Теперь еще раз щелкните по набору данных `ProductDetail`, выберите **Dataset Properties**, а затем выберите **Parameters** в списке переходов слева. Убедитесь, что названию параметра отчета `[@ProductCategory]` соответствует значение параметра запроса `@ProductCategory`. Параметры отчетов в визуальном конструкторе Report Designer обозначены символом @ и заключены в скобки. Затем нажмите кнопку **ОК**, чтобы закрыть диалоговое окно **Dataset Properties**.

Теперь Щелкните на вкладке **Preview**, чтобы выполнить новую параметризованную версию отчета. Вместо немедленного показа отчета (как это было раньше) SQL Server показывает подсказку в верхней части отчета **Product Category**. Мы должны предоставить значение для параметра **Product Category**, а затем нажать **View Report** (чтобы выполнить запрос с этим значением параметра). Для проверки нашего текущего запроса введите в текстовом поле `bikes`, а затем щелкните по **View Report**. Теперь отчет должен показывать только велосипеды.

Как вы видите, когда вы создаете параметр запроса при помощи переменной из вашего запроса к набору данных, то в узле **Parameters** панели **Report Data** автоматически создается соответствующий параметр отчета. Параметры отчетов настраиваются в панели **Parameters** диалогового окна **Dataset Editor**. В нашем примере мы приняли соответствие по умолчанию параметра запроса `@ProductCategory` параметру отчета `[@ProductCategory]`. В следующем разделе вы увидите, как параметры отчетов настраиваются при помощи редактора Report Parameter Editor.

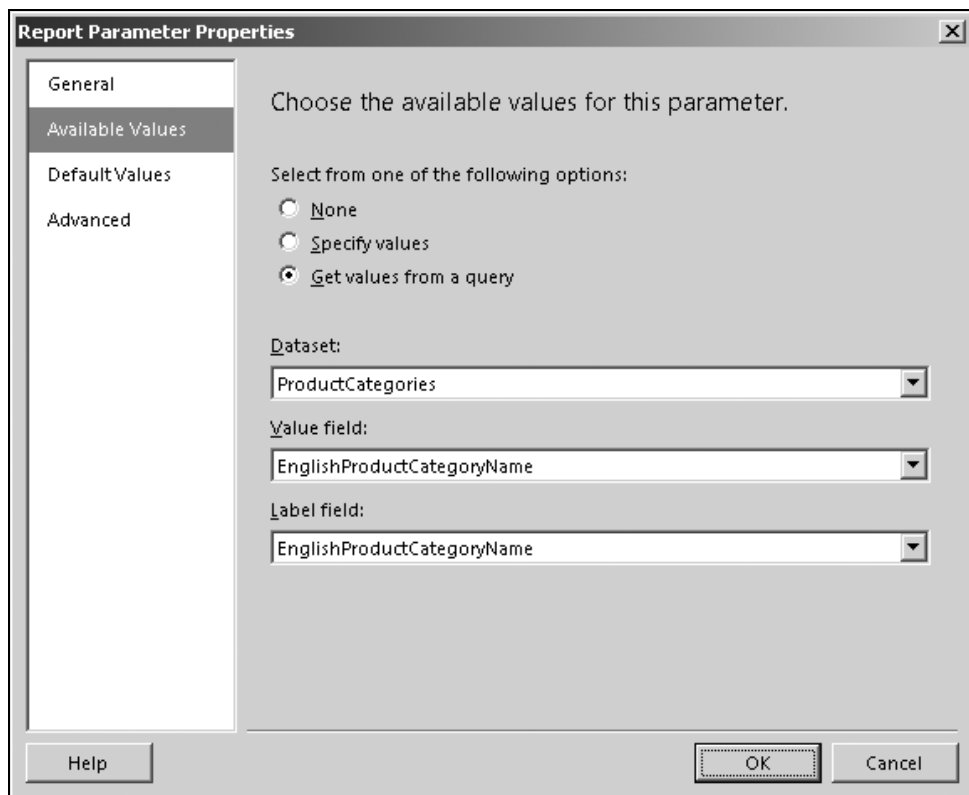
## Настройка параметров отчета

Мы успешно добавили параметр в наш отчет: пользователи просто должны ввести значение, чтобы получить отчет именно по этой категории продуктов. А что же делать тем пользователям, которые не знакомы с категориями продаваемых компанией Adventure Works продуктов? Для этих пользователей мы хотим сделать все еще проще. Вместо текстового поля (которое требует от пользователя не только знания точного названия категории, но и правильного ее написания) мы хотим предоставить выпадающий список имеющихся

категорий продуктов, чтобы просто выбрать желаемую категорию из уже заполненного списка.

Добавить такую функциональность очень легко. Сначала мы создадим набор данных, который возвращает список категорий продуктов. Щелкните правой кнопкой мыши по источнику данных AdventureWorksDW2008 в панели **Report Data**, а затем выберите **Dataset** в меню наверху панели **Report Data**. Назовите набор данных ProductCategories. В качестве источника данных должен по умолчанию быть AdventureWorksDW2008. Введите следующий код в панели запроса (или нажмите кнопку **Import** и импортируйте запрос SelectProductCategories.sql), а затем нажмите кнопку **OK**:

```
SELECT EnglishProductCategoryName  
FROM dbo.DimProductCategory  
ORDER BY EnglishProductCategoryName
```



**Рис. 19.8.** Диалоговое окно **Report Parameter Properties** для параметра отчета ProductCategory

Щелкните правой кнопкой мыши по параметру `ProductCategory` в узле **Parameters** панели **Report Data**, а затем выберите **Parameter Properties**. Выберите страницу **Available Values** в меню переходов слева, а затем выберите вариант **Get Values From A Query**. Появятся три выпадающих списка. Выберите **Product Categories** в выпадающем списке **Dataset**. Выберите `EnglishProductCategoryName` в списках **Value Field** и **Label Field**, но кнопку **OK** пока не нажимайте. На рис. 19.8 показана заполненная страница свойств **Available Values** параметра отчета `ProductCategory`.

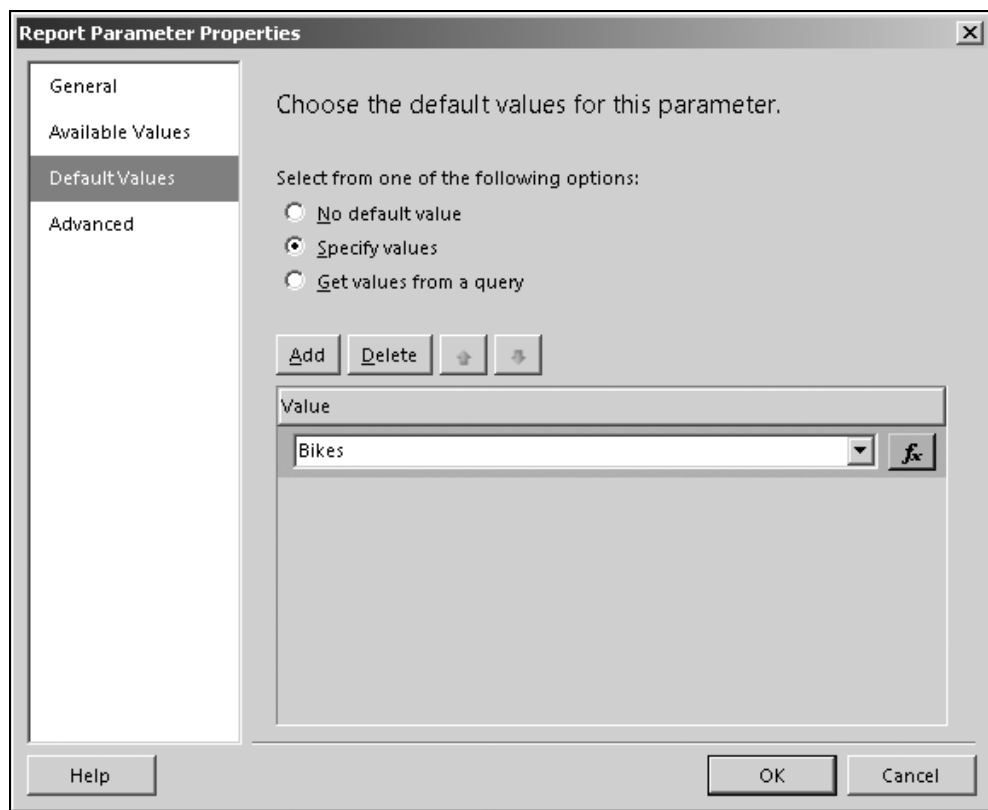


Рис. 19.9. Настройка значения по умолчанию параметра отчета

### ПРИМЕЧАНИЕ

В простых примерах этой главы мы указываем описательное название и как значение, и как метку параметров отчета. На практике значению часто соответствует сокращенный код или идентификатор (в зависимости от того,

что ожидается параметром запроса), а описательное название используется только для метки. Метка — это тот текст, который пользователи видят в выпадающем списке параметров при выполнении отчета.

Возможно, вы заметили, что после добавления параметра сам отчет был сначала пуст (при предварительном просмотре). Это потому, что не было указано значение параметра (категория продуктов). Мы не хотим, чтобы пользователи получали пустой отчет, поэтому изменим это при помощи задания категории продуктов по умолчанию на странице **Default Values** в меню переходов слева. В диалоговом окне **Default Values** выберите вариант **Specify Values**, нажмите кнопку **Add**, а затем введите *Bikes* в появившемся диалоговом окне. На рис. 19.9 показана заполненная страница свойств **Default Values** параметра отчета *ProductCategory*. Нажмите кнопку **OK**, чтобы закрыть диалоговое окно **Report Parameter Properties**.

Сделайте предварительный просмотр отчета. Теперь параметр сконфигурирован как выпадающий список (а не текстовое поле), и по умолчанию отображается отчет по велосипедам. Щелкните по выпадающему списку параметра (чтобы увидеть категории продуктов). Выберите любую категорию, а затем опять сделайте предварительный просмотр отчета, чтобы убедиться, что отчет выполнен для указанной в выпадающем списке категории продуктов.

## Каскадные параметры

Каскадные параметры полезны тогда, когда надо фильтровать по нескольким параметрам и существует иерархическая связь или зависимость между этими параметрами. Например, пользователь хочет фильтровать продукты дальше, до уровня подкатегорий. Мы можем просто добавить второй параметр запроса для поля подкатегорий в наш набор данных *ProductDetail*. Пользователи после этого получают возможность фильтровать по подкатегориям; однако это несколько упрощенный подход, поскольку мы не препятствуем указывать несуществующие комбинации продуктов и подкатегорий. Для создания интуитивно понятного пользовательского интерфейса мы будем использовать связь между категорией продуктов и подкатегориями и реализуем каскадные параметры (чтобы направлять пользователей при выборе ими параметров).

Каскадные параметры реализуются при помощи добавления параметров запроса к набору данных (который используется для заполнения списка значений параметра). Начнем с создания нового набора данных, который возвращает значения подкатегорий продуктов. Щелкните правой кнопкой мыши по источнику данных *AdventureWorksDW2008* в панели **Report Data**, а затем вы-

берите **Add Dataset**. Назовите набор данных `ProductSubCategories` и введите в поле **Query** следующий запрос (или нажмите кнопку **Import** и загрузите запрос `SelectProductSubCategories.sql` из кодов этой главы):

```
SELECT ps.EnglishProductSubcategoryName
FROM dbo.dimProductSubcategory AS ps
INNER JOIN dbo.dimProductCategory AS pc
ON ps.ProductCategoryKey = pc.ProductCategoryKey
WHERE pc.EnglishProductCategoryName = @ProductCategory
ORDER BY ps.EnglishProductSubcategoryName
```

В списке переходов слева выберите **Parameters**, а затем убедитесь, что значению параметра отчета `[@ProductCategory]` соответствует название параметра запроса `@ProductCategory`. Нажмите кнопку **OK**.

Перед тем как мы сможем увидеть наши каскадные параметры в действии, нам нужно модифицировать запрос к набору данных (чтобы он ждал параметр подкатегории). Щелкните правой кнопкой мыши по набору данных `ProductDetail` в панели **Report Data**, а затем выберите **Query**. Модифицируйте выражение `WHERE` запроса следующим образом:

```
WHERE (pc.EnglishProductCategoryName = @ProductCategory)
AND (ps.EnglishProductSubcategoryName = @ProductSubcategory)
```

Нажмите кнопку **OK**. Теперь вы должны увидеть параметр `ProductSubcategory` в узле **Parameters** панели **Report Data**. Щелкните правой кнопкой мыши по параметру `ProductSubcategory`, а затем выберите **Parameter Properties**. На странице **Available Values** выберите вариант **Get Values From A Query**, а затем набор данных `ProductSubcategories`. Выберите `EnglishProductSubcategoryName` в выпадающих списках **Value Field** и **Label Field**. В панели **Default Values** выберите вариант **Specify Values**. Щелкните по кнопке **Add**, а затем введите `Mountain Bikes` в текстовом поле **Values**. Нажмите кнопку **OK**, чтобы сохранить ваши настройки и закрыть диалоговое окно.

Теперь сделайте предварительный просмотр отчета. Убедитесь, что отчет по умолчанию показывает только горные велосипеды. Щелкните по выпадающему списку **Product Subcategory**, чтобы убедиться, что в списке присутствуют только подкатегории велосипедов. Измените параметр **Product Category** на `Clothing`, а затем опять щелкните по выпадающему списку **Product Subcategory**. Убедитесь, что элементы выпадающего списка изменились и показывают подкатегории одежды.

В меню **File** выберите **Save All**, чтобы сохранить файлы решения и проекта.

## Написание кода пользовательского отчета

Ранее в этой главе мы использовали выражения отчета для добавления динамического текста в заголовок отчета. Имеющихся в Visual Basic функций достаточно для большинства выражений отчетов. Однако иногда вам может понадобиться такая функция, которой нет в наличии. В таких случаях можно писать пользовательские функции (которые затем будут использоваться в вашем отчете). Эти пользовательские функции могут быть встроенным в отчет скриптом (на языке Visual Basic .NET) или в .NET-сборку, на которую есть ссылка в вашем отчете. Сейчас мы создадим новый отчет и встроим в него код (который можно будет вызывать из любого выражения отчета).

Мы создадим и добавим в наше решение отчет `Order Detail` и напомним код для изменения фоновой окраски строк отчета (чтобы сделать его более читабельным). На этот раз мы будем использовать для создания отчета мастер `Report Wizard` (а не конструировать его с нуля, как мы делали это для нашего первого отчета). Щелкните правой кнопкой мыши по пункту **Reports** в окне **Solution Explorer**, а затем выберите пункты **Add New Report** контекстного меню (для запуска мастера `Report Wizard`).

Мастер `Report Wizard` имеет несколько встроенных шаблонов отчетов (чтобы вы могли быстро начать работать со службами `Reporting Services`). Теперь, когда вы уже создали вручную свой первый отчет в визуальном конструкторе `Report Designer`, у вас есть понимание составляющих отчет элементов. Имея эти знания, для быстрого создания нашего отчета `Report Detail` мы используем мастер `Report Wizard`.

На странице **Welcome** Щелкните по кнопке **Next**. На странице **Select The Data Source** убедитесь, что выбран совместно используемый источник данных `AdventureWorksDW2008`, а потом нажмите кнопку **Next**.

Введите следующий код в текстовом окне **Query** (или нажмите кнопку **Query Builder**, а потом кнопку **Import** для загрузки запроса `SelectOrderDetail.sql` из примеров кодов этой главы):

```
SELECT
    d.FullDateAlternateKey,
    fs.SalesOrderNumber,
    g.EnglishCountryRegionName,
    g.StateProvinceName,
    p.EnglishProductName,
    fs.SalesAmount
```

```
FROM dbo.FactInternetSales AS fs
INNER JOIN dbo.dimDate AS d
    ON fs.OrderDateKey= d.DateKey
INNER JOIN dbo.dimProduct AS p
    ON fs.ProductKey = p.ProductKey
INNER JOIN dbo.DimProductSubcategory AS ps
    ON p.ProductSubcategoryKey = ps.ProductSubcategoryKey
INNER JOIN dbo.DimProductCategory AS pc
    ON ps.ProductCategoryKey = pc.ProductCategoryKey
INNER JOIN dbo.dimCustomer AS c
    ON fs.CustomerKey = c.CustomerKey
INNER JOIN dbo.DimGeography AS g
    ON c.GeographyKey = g.GeographyKey
WHERE g.EnglishCountryRegionName = @Country
    AND g.StateProvinceName = @State
    AND d.CalendarYear = @CalendarYear
    AND d.CalendarQuarter = @CalendarQuarter
ORDER BY d.FullDateAlternateKey, fs.SalesOrderNumber,
    p.EnglishProductName
```

Нажмите кнопку **Next**. Сохраните установку по умолчанию **Tabular Report Type**, а затем нажмите кнопку **Next**.

Добавьте в раздел **Details** все имеющиеся поля **Available Fields**, а затем нажмите кнопку **Next**. Выберите **Table Style**, а потом нажмите кнопку **Next**. Назовите отчет Order Detail, а потом нажмите **Finish**. Вы увидите представление Design отчета. Поскольку вы уже создали свой первый отчет вручную с нуля, то теперь вы понимаете, как быстро и легко можно создать отчет при помощи мастера Report Wizard. После того как мастер сгенерирует отчет, вы его сможете настроить и расширить точно так же, как будто вы и не использовали мастер вовсе.

Теперь Щелкните по вкладке **Preview**. Там есть четыре параметра отчета (Country, State, Calendar Year и Calendar Quarter), соответствующие четырём параметрам запроса выражения WHERE введенной нами инструкции SELECT. Введите значения этих параметров: Canada (для Country), British Columbia (для State), 2002 (для Calendar Year) и 1 (для Calendar Quarter). Затем нажмите **View Report**.

Вы успешно создали отчет, однако читать его трудно. Нам нужно отформатировать столбцы дат и объемов продаж, а также сделать заливку фона каждой второй строки.

Сначала мы настроим форматирование дат. На вкладке **Design** щелкните правой кнопкой мыши по ячейке [FullDateAlternateKey], а затем выберите пункт **Text Box Properties** контекстного меню. Выберите страницу **Number** в списке переходов на левой стороне диалогового окна **Text Box Properties**. В списке **Category** выберите **Date**. Теперь вы можете выбрать любой формат даты в списке **Type**. Для нашего отчета оставьте формат даты по умолчанию и просто нажмите **OK**. Затем выровняйте даты к левой стороне (при помощи нажатия кнопки **Left** на странице **Formatting**).

Теперь мы отформатируем объемы продаж. Щелкните правой кнопкой мыши по ячейке [SalesAmount], а затем выберите **Text Box Properties**. Выберите страницу **Number** в списке переходов слева. В списке **Category** выберите **Currency**. Затем установите флажок **Use 1000 Separator (,)**, чтобы в значениях SalesAmount отображались запятые, и нажмите **OK**.

Для того чтобы сделать заливку строк, нам нужно написать пользовательскую функцию. Щелкните правой кнопкой мыши в любом месте поверхности конструирования (вне области страницы отчета), а затем выберите пункт **Report Properties** контекстного меню. Просмотрите страницы свойств отчета в списке переходов слева. Обратите, например, внимание на то, что в диалоговом окне **Properties** устанавливается размер печатной страницы отчета. Страница **References** позволяет вам ссылаться на внешние .NET-сборки и классы. В нашем случае мы хотим встроить пользовательскую функцию непосредственно в отчет. Для этого выберите страницу **Code** в списке переходов слева, а затем введите следующий код в текстовом поле **Custom**:

```
Private rowColor1 As Boolean
Function ChangeColor(ByVal color1 As String, _
    ByVal color2 As String, ByVal change As Boolean) As String
    If change Then rowColor1 = Not rowColor1
    If rowColor1 Then
        Return color1
    Else
        Return color2
    End If
End Function
```

Нажмите **OK**, чтобы закрыть диалоговое окно **Report Properties**.

Теперь мы используем нашу только что созданную функцию для настройки фонового цвета строк данных в таблице. Щелкните правой кнопкой мыши по ячейке [FullDateAlternateKey], а затем выберите **Text Box Properties**. Выберите страницу **Fill** в списке переходов слева. Нажмите кнопку **Expression**,



чтобы настроить выражение для BackgroundColor. Введите в диалоговое окно **Expression** следующий код:

```
=Code.ChangeColor("White", "Gainsboro", True)
```

Нажмите **ОК**, чтобы закрыть диалоговое окно **Expression**. В диалоговом окне **Textbox Properties** еще раз нажмите **ОК**, чтобы закрыть диалоговое окно. Затем установите фоновый цвет для всех других ячеек данных таблицы. Для выбора нескольких текстовых полей за один раз удерживайте при щелчках по ним клавишу <Shift> в нажатом состоянии. Найдите свойство BackgroundColor в окне **Properties** в Visual Studio. Выберите ссылку **Expression** в выпадающем списке. Введите следующий код в диалоговом окне **Expression**:

```
=Code.ChangeColor("White", "Gainsboro", False)
```

Нажмите **ОК**, а затем сделайте предварительный просмотр отчета (сделав те же самые настройки параметров, что и раньше). Ваш отчет должен выглядеть так, как показано на рис. 19.10.

| Full Date Alternate Key | Sales Order Number | English Country Region Name | State Province Name | English Product Name   | Sales Amount |
|-------------------------|--------------------|-----------------------------|---------------------|------------------------|--------------|
| 1/1/2002                | SO45081            | Canada                      | British Columbia    | Road-650 Black, 44     | \$699.10     |
| 1/3/2002                | SO45094            | Canada                      | British Columbia    | Road-150 Red, 62       | \$3,578.27   |
| 1/5/2002                | SO45101            | Canada                      | British Columbia    | Road-150 Red, 52       | \$3,578.27   |
| 1/6/2002                | SO45105            | Canada                      | British Columbia    | Road-150 Red, 48       | \$3,578.27   |
| 1/6/2002                | SO45106            | Canada                      | British Columbia    | Mountain-100 Black, 38 | \$3,374.99   |
| 1/6/2002                | SO45108            | Canada                      | British Columbia    | Road-150 Red, 62       | \$3,578.27   |
| 1/9/2002                | SO45123            | Canada                      | British Columbia    | Road-150 Red, 44       | \$3,578.27   |
| 1/11/2002               | SO45134            | Canada                      | British             | Road-150 Red,          | \$3,578.27   |

Рис. 19.10. Отформатированный при помощи пользовательской функции отчет

Нажмите кнопку **Save All** на панели инструментов, чтобы сохранить файлы решения и проекта.

## Создание отчета OLAP

Службы отчетности Reporting Services поддерживают создание отчетов из источников данных Analysis Services. Запросы по источникам данных Analysis Services должны быть написаны на языке многомерных выражений MDX. У нас для вас есть две новости относительно написания кода MDX для источников данных Analysis Services в Reporting Services, плохая и хорошая. Сначала плохая: службам Reporting Services требуется, чтобы возвращаемый запросом набор данных был выровненным. Более того, в столбцах запроса могут возвращаться меры (и только меры). Эти ограничения могут осложнить написание запроса MDX для отчета. Хорошая новость состоит в том, что службы Reporting Services предоставляют графический визуальный конструктор запросов для MDX.

### ПРИМЕЧАНИЕ

В этом разделе используется база данных AdventureWorksDW2008 аналитических служб Analysis Services. Скачав проект для AdventureWorksDW2008, откройте его в Visual Studio, а затем обработайте (щелкните правой кнопкой мыши по базе данных в окне **Solution Explorer**, а затем выберите пункт **Process** контекстного меню). Указания по получению и установке образцовых баз данных смотрите во введении к этой книге.

Для начала мы добавим новый совместно используемый источник данных, указывающий на базу данных Adventure Works DW 2008 аналитических служб. Это база данных Analysis Services, созданная использованием в качестве ее источника реляционной базы данных AdventureWorksDW2008. Щелкните правой кнопкой мыши по узлу **Shared Data Sources** в окне **Solution Explorer**, а затем выберите **Add New Data Source**. Назовите источник данных AdventureWorksCube, а затем выберите Microsoft SQL Server Analysis Services в выпадающем меню **Type**. Нажмите кнопку **Edit**. В диалоговом окне **Connection Properties** укажите название того сервера, на котором находится база данных (или localhost, если она находится на вашем локальном компьютере). Выберите **Adventure Works DW 2008** в выпадающем списке **Database Name**. Проверьте подключение. Нажмите **ОК** два раза, чтобы закрыть диалоговые окна **Connection Properties** и **Shared Data Source Properties**.

Теперь добавьте в решение отчет. Щелкните правой кнопкой мыши по пункту **Reports** в окне **Solution Explorer**, а затем выберите **Add | New Item**. Выберите шаблон **Report**, назовите отчет **Sales Summary by Region and Date.rdl**, а затем щелкните по кнопке **Add**.

В панели **Report Data** выберите **New | Dataset**. Назовите набор данных **SalesSummary**. Нажмите кнопку **New**. В диалоговом окне **Data Source Properties** назовите источник данных отчета **AdventureWorksCube**, а затем выберите вариант **Use Shared Data Source Reference**. Выберите **AdventureWorksCube** в выпадающем списке, а затем нажмите **OK**. В диалоговом окне **Dataset Properties** нажмите кнопку **Query Designer**. После этого загрузится графический визуальный конструктор MDX Query Designer.

## Использование графического визуального конструктора MDX Query Designer

Этот визуальный конструктор является интерфейсом для создания инструкций MDX посредством перетаскивания. Сначала убедитесь в том, что выбран куб **Adventure Works**. Панель **Metadata** на левой стороне визуального конструктора показывает меры и измерения куба (в древовидном представлении). Мы создадим отчет для показа объемов продаж по регионам и времени.

В кубе **Adventure Works** выберите **Internet Sales** в выпадающем списке **Measure Group**. Разверните узел **Customer** в панели **Metadata**, а затем раскройте узел **Customer Geography**. Перетащите уровень **Country** в панель **Query** визуального конструктора. Затем перетащите уровень **State-Province** в панель **Query**. Мы хотим также показать объемы продаж по датам. Разверните узел **Date**, а затем узел **Calendar** в панели **Metadata**. И наконец, раскройте узел **Date.Calendar**. Перетащите уровни **Calendar Year** и **Calendar Quarter** в панель **Query**. Теперь разверните узел **Measures** и перетащите меры **Internet Sales Amount** и **Internet Order Quantity** в панель **Query**.

### СОВЕТ

Если вы планируете показывать в отчете множество уровней определенной пользователем иерархии, то лучше перетащить желаемые уровни из определенной пользователем иерархии, а не использовать отдельные иерархии атрибутов. Службы Reporting Services строят запрос MDX при помощи создания перекрестного соединения всех элементов всех иерархий, имеющих в панели **Query**. Когда используются уровни иерархии, то Reporting Services запрашивает все элементы (**AllMembers**) самого низшего уровня иерархии. В запросе также автоматически возвращаются и родительские

элементы. Поэтому при помощи использования Country и State-Province из иерархии Customer Geography отпадает необходимость в перекрестном соединении (по сравнению с использованием иерархий атрибутов Country и State-Province измерения Customer). Используйте оба метода и сравните инструкции MDX (чтобы уловить разницу).

И наконец, мы добавим параметры в запрос MDX. Это делается при помощи панели фильтрации (над панелью запроса). Мы добавим параметры для года и страны клиента. Когда вы щелкнете в ячейках панели фильтрации, будут активированы выпадающие списки (показаны в табл. 19.1).

Таблица 19.1. Выпадающие списки

| Dimension | Hierarchy         | Operator          | Filter Expression | Выбор параметров |
|-----------|-------------------|-------------------|-------------------|------------------|
| Date      | Date.CalendarYear | Range (Inclusive) | :                 | Оба выбраны      |
| Customer  | Country           | Equals            |                   | Выбран           |

На рис. 19.11 показано заполненное окно **Query Designer**.

Чтобы увидеть сгенерированный визуальным конструктором запроса код MDX, нажмите кнопку **Design Mode** в панели инструментов (самая правая кнопка на панели) в верхней части окна **Query Designer**. Нажатие кнопки **Design Mode** будет переключать вас между графическим и текстовым режимами. В следующем разделе мы будем модифицировать запрос при помощи текстового режима. Пока же нажмите кнопку **Design Mode** еще раз (чтобы вернуться в режим графического конструирования).

Щелкните по **ОК**, чтобы закрыть окно **Query Designer**. Нажмите **ОК** еще раз, чтобы закрыть диалоговое окно **Dataset Properties**. Вы увидите уже знакомую вам вкладку **Report Design**. Разверните узел **Parameters** в панели **Report Data** и обратите внимание на то, что создано три параметра отчета. Параметры отчета соответствуют параметрам запроса. Поскольку мы указали для параметра года диапазон, то службы Reporting Services весьма умно сгенерировали два параметра (один — для начала диапазона годов, а другой — для его окончания).

Сделайте предварительный просмотр отчета. Отчет имеет три запроса параметров (каждый со списком выбора). Щелкните по выпадающему списку для параметра From Date.Calendar Year — этот список заполнен списком годов.

Вспомните, что для отчета `Product Detail` мы создали наборы данных, содержащие имеющиеся значения параметров отчета (`Product Category` и `Product Subcategory`). Эти наборы данных были затем указаны как источники имеющихся значений (когда мы настраивали параметры отчета). В данном случае мы не создавали явно никаких наборов данных с имеющимися значениями календарного года, а выпадающий список параметра заполнен списком имеющихся значений. Как это получилось? Вернемся на вкладку **Design** отчета (чтобы подробнее рассмотреть добавленные визуальным конструктором Report Designer параметры отчета).

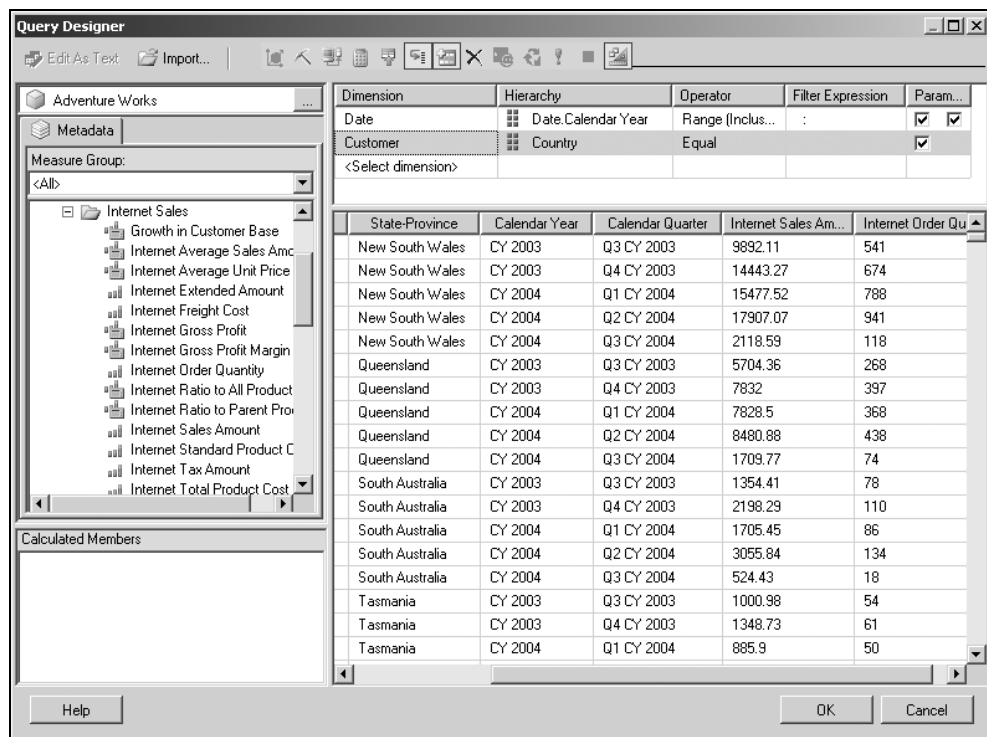


Рис. 19.11. Заполненное окно **Query Designer** для набора данных `SalesSummary`

Щелкните правой кнопкой мыши по `FromDateCalendarYear` в узле **Parameters** панели **Report Data**, а затем выберите пункт **Parameter Properties** контекстного меню. Когда появится диалоговое окно **Parameter Properties**, выберите вариант **Available Values**. Выбран вариант **Get Values From A Query**. Имеющиеся значения взяты из набора данных с названием

FromDateCalendarYear. Мы не создавали этот набор данных, оказывается, это Reporting Services автоматически создают наборы данных имеющихся значений для параметров отчетов соответствующих параметрам запросов для отчета OLAP! До сих пор мы их не замечали (поскольку по умолчанию они скрыты).

Щелкните по **Cancel**, чтобы закрыть диалоговое окно **Parameter Properties**. Щелкните правой кнопкой мыши по источнику данных AdventureWorksCube, выберите пункт **Show Hidden Datasets** контекстного меню и обратите внимание, что в узле AdventureWorksCube появились три дополнительных набора данных. Когда параметры запроса указываются при помощи визуального конструктора Query Designer, то службы Reporting Services автоматически создают наборы данных для заполнения списков имеющихся значений параметров. Как уже упоминалось, эти наборы данных по умолчанию скрыты. После их появления вы можете редактировать их свойства в панели **Report Data** (как и любого другого набора данных для отчета).

Когда мы делали предварительный просмотр отчета, то для параметров отчета не было указано никаких значений по умолчанию, поэтому параметры отчета использовали автоматически присвоенные службами Reporting Services подсказки по умолчанию. Давайте изменим это, чтобы наш отчет стал более дружелюбным к пользователю.

Щелкните правой кнопкой мыши по узлу SalesSummary в панели **Report Data**, а затем выберите пункт **Query** контекстного меню. Наведите курсор мыши на кнопку с символом @ (пятая справа) в панели инструментов **Query Designer**. В подсказке написано **Query Parameters**, но она неактивна. Нажмите самую правую кнопку панели инструментов (чтобы переключиться в текстовый режим). Будет показан запрос MDX (сгенерированный визуальным конструктором MDX Query Designer), и кнопка **Query Parameters** станет активной.

Откройте диалоговое окно **Query Parameters**. Щелкните по соответствующей параметру FromDateCalendarYear ячейке Default. Это активирует выпадающий список ячейки. Выпадающий список имеет элемент управления древовидного представления, показывающий имеющиеся значения параметра. Выберите CY2001, а затем нажмите **OK**. Повторите процесс для параметров ToDateCalendarYear и CustomerCountry, выбрав для ToDateCalendarYear значение по умолчанию CY2003 и для CustomerCountry две страны (Австралия и Канада). Нажмите **OK**, чтобы сохранить значения по умолчанию и закрыть диалоговое окно **Query Parameters**. Затем нажмите **OK**, чтобы закрыть визуальный конструктор Query Designer.

Теперь мы настроим подсказки параметров. Щелкните правой кнопкой мыши по узлу `FromDateCalendarYear` в узле **Parameters** и опять выберите **Property Parameters**. На странице **General** измените значение **Prompt** на **From Year**. В панели **Default Values** проверьте, что значение по умолчанию установлено такое: `[Date].[Calendar Year].&[2001]`. Вместо использования диалогового окна **Parameters** в визуальном конструкторе Query Designer мы могли бы ввести значение по умолчанию в **Default Values** здесь, в диалоговом окне **Parameter Properties**. Однако для этого нам надо было бы знать уникальное название нужного нам элемента по умолчанию, поскольку поле **Value** (используемое для списка **Available Values**) настроено в уникальные названия элементов. Нажмите **ОК** для сохранения новой подсказки.

Сделайте то же самое для того, чтобы изменить подсказки для `ToDateCalendarYear` и `CustomerCountry` на более дружественные к пользователю значения.

## Написание запросов MDX в текстовом режиме

Визуальный конструктор MDX Query Designer отлично подходит для быстрого создания отчета по источнику данных Analysis Services (причем для этого совершенно не нужно знать язык MDX). Однако сгенерированный визуальным конструктором код MDX может и не быть оптимальным, как по его поведению, так и по производительности. Текстовый режим позволяет вам при необходимости модифицировать сгенерированные визуальным конструктором MDX инструкции либо написать с нуля ваши собственные (вообще без использования визуального конструктора).

Настроить параметры для отчета OLAP несложно. Мы просто должны установить флажок **Parameters** в панели фильтров MDX Query Designer — и службы Reporting Services автоматически генерируют параметры отчета и наборы данных с имеющимися значениями для параметров. Однако вас не всегда устроят наборы имеющихся значений, генерируемые для параметров отчета службами Reporting Services.

Возможно, вы заметили, что в выпадающих списках **From Year** и **To Year** есть элемент **All Periods**. По умолчанию набор данных с имеющимися значениями для параметра отчета (сгенерированный службами Reporting Services) включает в себя элемент **all** той иерархии атрибута, для которой создан фильтр. Это не всегда желательно. Сделайте предварительный просмотр отчета `Order Summary` и измените параметр **From Year** на **All Periods**. В панели **Report** появится сообщение, что выполнение запроса закончилось ошибкой, поскольку вариант **All Periods** не является корректным значением для предела диапазона. Поэтому мы хотим удалить вариант **All**

Periods из списков имеющихся значений для параметров **From Year** и **To Year**. Мы можем сделать это путем модификации кода MDX для наборов данных имеющихся значений. Вернемся на вкладку **Design** отчета.

Щелкните правой кнопкой мыши по узлу **FromDateCalendarYear** панели **Report Data**, а затем выберите пункт **Query**. После этого откроется визуальный конструктор Query Designer с запросом MDX для набора данных. Нажмите кнопку с восклицательным знаком (команда run) на панели инструментов для выполнения запроса. Обратите внимание, что элемент All Periods вернулся. Измените в тексте [Date].[Calendar Year].AllMembers на [Date].[CalendarYear].[All Periods].Children и еще раз выполните запрос. Теперь элемента All Periods нет (чего мы и добивались). Это потому, что элемент All Periods возвращается из AllMembers, а не из Children. На рис. 19.12 показан модифицированный запрос и результаты в **Query Designer**.

Нажмите **ОК**, чтобы закрыть Query Designer. Выполните те же самые шаги для удаления элемента all из набора данных ToDateCalendarYear.

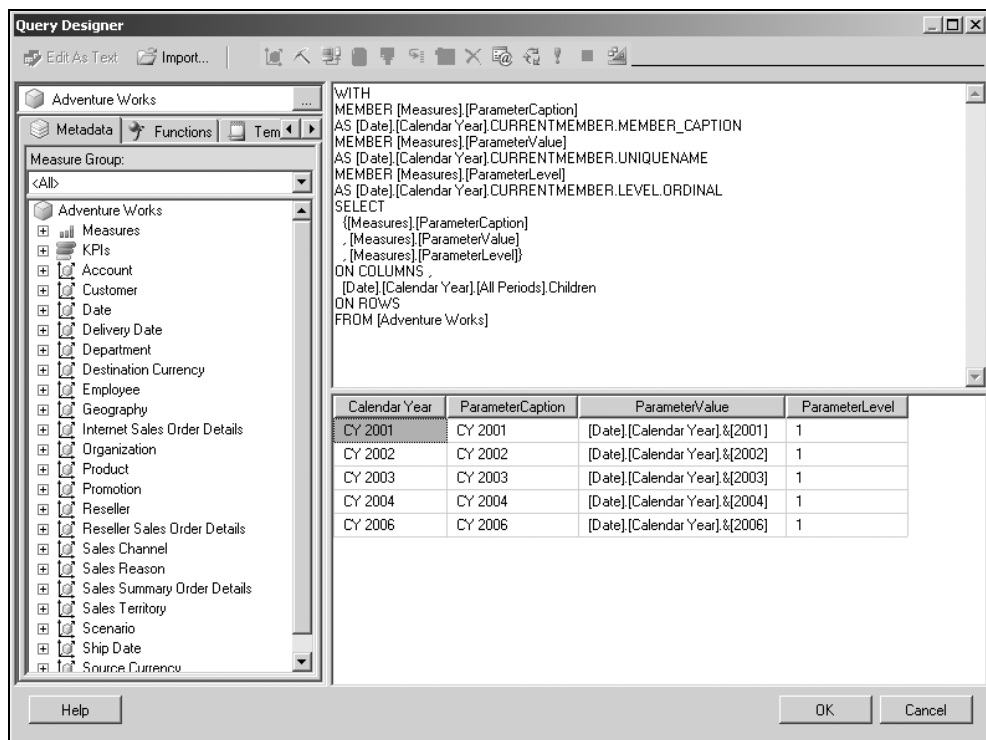


Рис. 19.12. Визуальный конструктор MDX Query Designer в текстовом режиме



**ПРИМЕЧАНИЕ**

Возможно вы заметили, что в этой главе все просматриваемые в текстовом режиме запросы MDX имеют в столбцах только меры. Это не случайность. Службам Reporting Services по возвращении из запросов MDX требуются выровненные наборы данных с мерами (и только мерами) в столбцах. Инструкции MDX, которые возвращают в столбцах элементы измерения или кортежи (коллекции элементов осей измерения) приведут к ошибке.

Мы не хотим также, чтобы элемент `all` появлялся в выпадающем списке параметра `Country`. Когда мы настроили параметр `Country` как параметр множественного выбора, то службы Reporting Services автоматически добавили вариант **Select All** в список имеющихся значений. Вариант `All Customers` избыточен при наличии опции **Select All**.

Для удаления опции `All Customers` из выпадающего списка параметра `Country` сделайте все то же самое, что вы делали для выпадающих списков временных периодов, только в инструкции MDX замените `[Customer].[Country].AllMembers` на `[Customer].[Country].[All Customers].Children`.

Сделайте предварительный просмотр отчета. Убедитесь, что в выпадающих списках больше нет элемента `all`.

Мы использовали текстовый режим визуального конструктора MDX Query Designer для модификации сгенерированного службами Reporting Services кода MDX. Как уже упоминалось, текстовый режим можно также использовать для написания запросов MDX с нуля. Написание запросов с нуля полезно тогда, когда потребности отчетности шире возможностей визуального конструктора Query Designer.

Именно здесь заканчивается различие между отчетами, которые построены по реляционным источникам данных SQL Server, и отчетами по источникам данных Analysis Services. С этого момента создание отчета на основе запроса MDX ничем не отличается от создания отчета на основе запроса T-SQL, поскольку оба типа запросов выдают один и тот же тип набора данных (по которому вы и делаете отчет).

## Создание отчета при помощи матричной области данных

*Матричная область данных* (Matrix data region) — это естественная стартовая точка для создания отчетов по базам данных Analysis Services, данные

группируются как по столбцам, так и по строкам. Точное количество отображаемых в матрице столбцов и строк зависит от различных значений данных в полях (по которым группируются столбцы и строки). Как это чаще всего бывает, матричную область данных проще всего понять при помощи примера.

Начнем с перетаскивания матричной области данных из панели инструментов на поверхность конструирования отчета. Выровняйте матрицу с левым верхним углом поверхности конструирования посредством перетаскивания ее в угол при помощи синих линий привязки или при помощи настройки свойств **Left** и **Top** в значение 0 (под свойством **Location** матричной области данных в панели **Properties**).

Теперь мы настроим группы столбцов. Перетащите поле **Calendar\_Year** из узла **SalesSummary** в ячейку матрицы, содержащую слово **Columns**. Затем перетащите поле **Calendar\_Quarter** под поле **Calendar\_Year** матрицы. Отпустите поле, когда вы увидите синюю линию привязки внизу ячейки, содержащей поле **Calendar\_Year**; при этом будет автоматически создана строка для квартала. Обратите внимание, что **Column Groups** в панели **Grouping** отражают сделанные вами настройки.

Теперь настройте группы строк. Перетащите поле **Country** из узла источника данных **SalesSummary** на ячейку матрицы, содержащую слово **Rows**. Затем перетащите поле **State\_Province** из источника данных **SalesSummary** правее поля **Country**. Помните, что поля обозначаются на поверхности конструирования скобками. Отпустите поле **State\_Province** тогда, когда вы увидите появление синей линии привязки правее ячейки, содержащей поле **Country**; таким образом для поля **State\_Province** автоматически создается столбец.

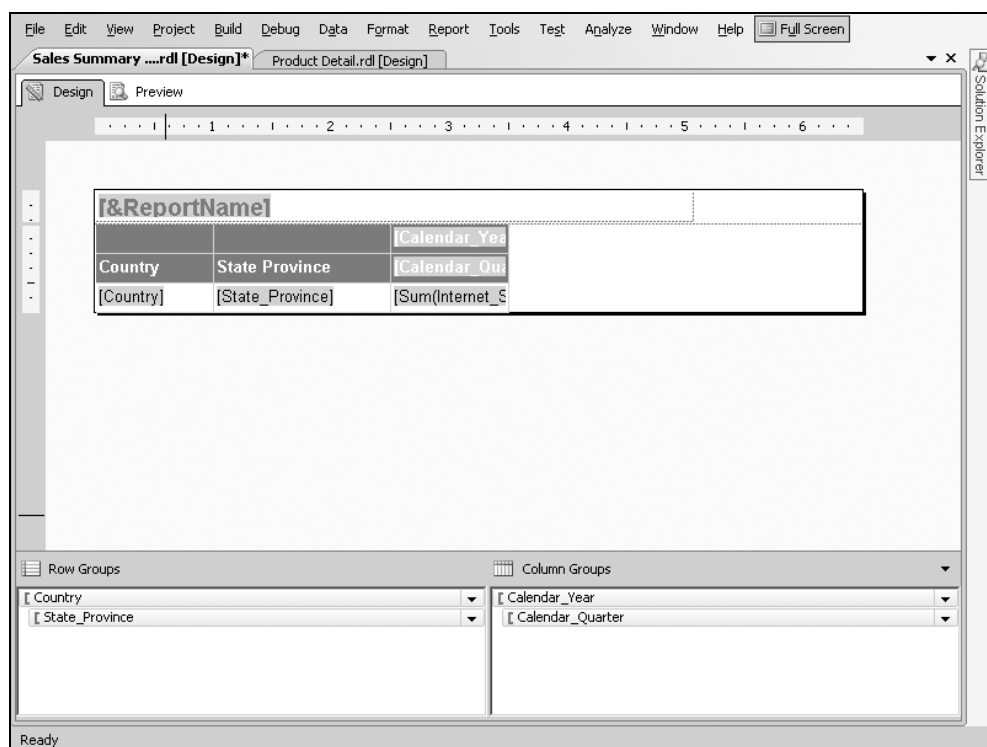
Теперь перетащите поле **Internet\_Sales\_Amount** из панели **Report Data** в ячейку матрицы, содержащую слово **Data**. По умолчанию **Reporting Services** суммируют значения в ячейках **Data**.

Теперь (когда у вас уже есть некоторый опыт создания отчетов) вы знаете, что отчет без форматирования будет выглядеть не очень хорошо. Чтобы это исправить, сделайте в отчете следующие изменения форматирования. (Мы уже подробно рассмотрели шаги выполнения этих изменений ранее в этой же главе).

1. Измените свойство **Fill BackgroundColor** верхних двух строк матрицы на **SteelBlue**.
2. Измените **Font Color** для двух верхних строк матрицы на **White**, а **Font Weight** — на **Bold**.

3. Расширьте столбец **State Province** до 1,5.
4. Измените форматирование **Number** ячейки данных на **Currency** (на **C0** — если вы настраиваете значение свойства **Format** в панели **Properties**).
5. Для показа названия отчета добавьте заголовок отчета. Укажите название отчета при помощи встроенного поля **ReportName** (чтобы заголовок показывал правильное название отчета при его изменении). Отформатируйте шрифт для **Report Name**.
6. Минимизируйте свободное место в заголовке отчета и убедитесь, что текстовое поле для названия отчета достаточно широкое для того, чтобы текст названия отчета помещался без переноса.

Когда вы закончите, ваша поверхность конструирования отчета должна выглядеть так, как на рис. 19.13.



**Рис. 19.13.** Законченное представление **Design** для отчета **Sales Summary by Region and Date**

Сделайте предварительный просмотр отчета. Матричный отчет позволяет пользователям быстро сравнить объемы продаж между кварталами или между штатами (провинциями), данные сгруппированы и по строкам, и по столбцам. Как уже упоминалось, количество строк будет зависеть от количества различных штатов (провинций) в наборе данных, а количество столбцов будет зависеть от количества различных кварталов в наборе данных.

Нажмите кнопку **Save All** на панели инструментов, чтобы сохранить файлы на данной стадии.

## Добавление подытогов и итогов

Часто бывает полезно показать в матричном отчете промежуточные итоги и итоги. На данный момент отчет дает объемы продаж для заданного квартала и конкретного штата (провинции). Скорее всего, ваши пользователи пожелают также узнать общие объемы продаж по стране или за год либо общие объемы продаж за квартал по всем странам. Щелкните по вкладке **Design**, чтобы вернуться на поверхность конструирования (если вы еще не там).

Подытоги и итоги добавляются в матричную область данных через контекстное меню групповой ячейки. Для добавления подытогов по штатам (провинциям) щелкните правой кнопкой мыши по содержащей поле [State\_Province] ячейке, а затем выберите **Add Total**. В отличие от более ранних версий визуального конструктора Report Designer (в которых подытоги и итоги всегда размещались после группы), Report Designer в Visual Studio 2008 предлагает выбор, разместить строку с подытогом перед или после группы. Выберите **After**. Сделайте предварительный просмотр отчета. Обратите внимание на строку Total, добавленную после списка штатов (провинций) каждой страны.

Вернитесь обратно на вкладку **Design** и повторите процесс добавления итогов для групп [Country], [Calendar\_Quarter] и [Calendar\_Year]. Сделайте предварительный просмотр отчета еще раз. В результате добавления Total для [Country] нижняя строка теперь показывает итоги по всем странам. В результате добавления Total для [Calendar\_Year] крайний правый столбец показывает итоги по всем кварталам. Поскольку итоги были также добавлены и для [Calendar\_Quarter], то показаны и столбцы подытогов по годам.

Теперь матрица на поверхности конструирования имеет множество ячеек, в каждой из которых содержится выражение [Sum(Internet\_Sales\_Amount)]. Несмотря на то, что выражение в каждой ячейке одно и то же, область действия каждого выражения различная. Щелчок по ячейке матрицы активирует желтые скобки, которые указывают область действия агрегирования. Щелкните по левой верхней

ячейке, содержащей выражение `[Sum(Internet_Sales_Amount)]`. Это исходная ячейка данных матрицы (до добавления итогов). Желтые скобки видны над заголовком столбца `Calendar_Quarter` и слева от заголовка строки `State_Province`. Это показывает, что сумма вычисляется по календарному кварталу и по штату (провинции). Щелкните по ячейке ниже. Теперь желтые скобки на заголовке строки `Country`. Эта строка представляет подытог для каждой страны в календарном квартале. Теперь щелкните выражение суммы справа внизу матрицы. Желтые скобки не активируются. Это ячейка с общим итогом. Это сумма интернет-продаж по всем странам и всем календарным годам.

Мы успешно добавили в отчет подытоги и итоги. Итоги полезны, но отчет теперь труднее воспринимается (из-за всех этих дополнительных чисел). Мы хотим это изменить, необходимо применить форматирование для выделения в отчете строк подытогов и итогов.

Добавление подытогов и итогов в матричную область данных в Reporting Services всегда было простым, но их форматирование простым не было. Знакомые с визуальным конструктором Report Designer в Visual Studio 2005 помнят пресловутый зеленый треугольник. В старом Report Designer ячейки подытогов форматировались при помощи нажатия на маленький зеленый треугольник в правом верхнем углу содержащей метку подытога ячейки. Нажатие зеленого треугольника приводило к показу свойств ячейки подытогов в панели **Properties**. Теперь же форматировать ячейки подытогов просто, поскольку это делается аналогично форматированию любой другой ячейки матрицы.

Сначала мы добавим полужирное форматирование для общих итогов по всем странам и сделаем фон желтым. Выберите нижнюю строку матрицы при помощи щелчка по серому манипулятору строки. Используйте панель инструментов **Formatting** для того, чтобы сделать шрифт полужирным и фоновый цвет желтым. (Можно настроить свойство **FontWeight** в **Bold**, а свойство **BackgroundColor** в **Yellow** в панели **Properties**).

Затем отформатируем подытоги по странам. Выделите среднюю строку выражений сумм. Используйте панель инструментов **Formatting** или панель **Properties** для того, чтобы сделать шрифт полужирным и цвет фона **Light Blue**. На рис. 19.14 показана отформатированная матрица.

Сделайте предварительный просмотр отчета. При помощи запроса MDX и простого форматирования мы создали отчет, который показывает много информации и все равно легко усваивается.

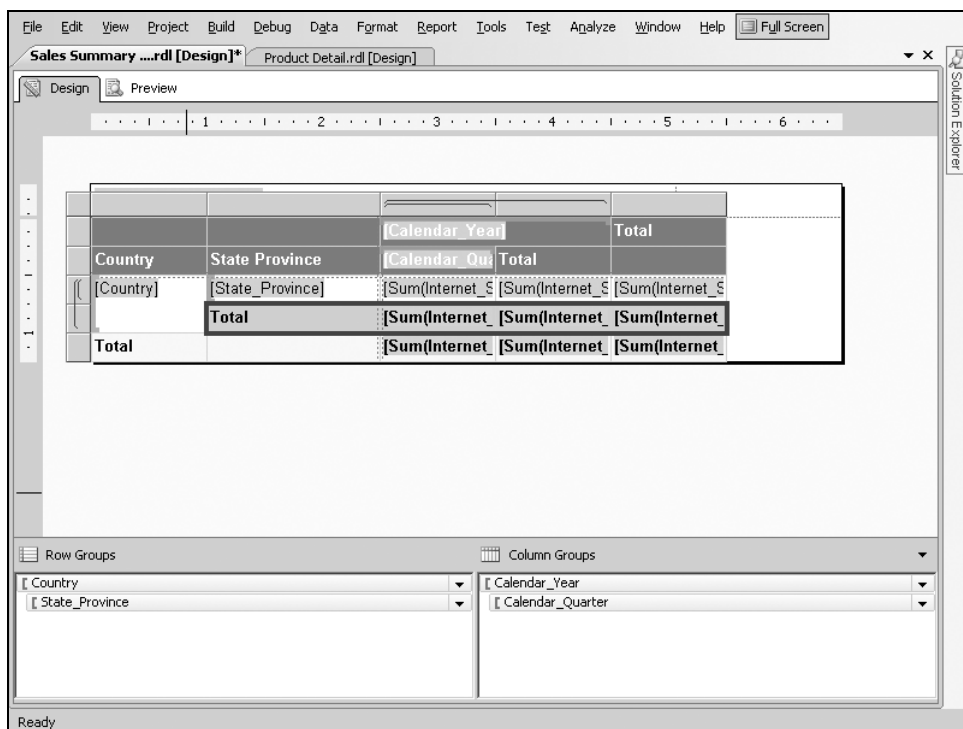


Рис. 19.14. Матричная область данных с отформатированными итогами

## Объясняем тип *tablix*

Все, что мы показывали до настоящего момента, можно было сделать и при помощи визуального конструктора Report Designer в Visual Studio 2005, только более утомительно, особенно в смысле форматирования подытогов. В этом разделе мы собираемся модифицировать наш матричный отчет таким образом, который не был возможен до версии SQL Server 2008 Reporting Services.

Мы уже упоминали несколько раз, что и табличная, и матричная области данных (использованные в примерах до настоящего момента) — это фактически области данных типа *tablix*. Когда мы щелкаем правой кнопкой мыши по табличной или матричной области данных, контекстное меню **Tablix Properties** показывает список пунктов меню. До настоящего момента мы не использовали ту гибкость, которую предоставляет область данных типа *tablix* и команды контекстного меню **Tablix Properties**.

## Добавление групп

Как это часто бывает, повышенная гибкость дает больше возможных вариантов, однако большое количество вариантов может и запутать. В ячейке области данных возможны 16 вариантов расширения области данных (табл. 19.2).

**Таблица 19.2.** Варианты расширения области данных типа *tablix*

|                  |                                                                                        |
|------------------|----------------------------------------------------------------------------------------|
| Add Column Group | Parent Group<br>Child Group<br>Adjacent Left<br>Adjacent Right                         |
| Add Row Group    | Parent Group<br>Child Group<br>Adjacent Above<br>Adjacent Below                        |
| Insert Column    | Inside Group Left<br>Inside Group Right<br>Outside Group Left<br>Outside Group Right   |
| Insert Row       | Inside Group Above<br>Inside Group Below<br>Outside Group Above<br>Outside Group Below |

Эти 16 вариантов по существу представляют собой пять:

- ☐ добавление родительской группы;
- ☐ добавление дочерней группы;
- ☐ добавление смежной группы;
- ☐ добавление фиксированного столбца (или строки) внутри группы;
- ☐ добавление фиксированного столбца (или строки) вне группы.

Добавление новой группы, столбца или строки левее, правее, выше или ниже — это просто вопрос форматирования. Важно понять разницу между добавлением групп и фиксированных столбцов (строк), между добавлением родительских, дочерних или смежных групп, между добавлением фиксированных столбцов (строк) внутри и вне группы. Те варианты, которые неприменимы в контексте ячейки, деактивированы (или не показаны в контекстном

меню). В случае вставки фиксированного столбца или строки (если ячейка не находится в группе) вариантами являются: вставка левее, правее, выше или ниже (без различения того, где должна произойти вставка, внутри или вне группы).

## Родительские и дочерние группы

По существу, единственным вариантом в Visual Studio 2005 было добавление в таблицы и матрицы родительских и дочерних групп. Добавление родительских и дочерних групп создает вложенность групп. В примере с матрицей группа `State_Province` является дочерней для группы `Country`. Записи сначала группируются по `Country` (родительская группа), а внутри страны — по `State_Province` (дочерняя группа). В панели группирования родительские группы находятся над дочерними. Поэтому, глядя на группы **Column Groups**, мы можем легко понять, что группа `Calendar_Year` является родительской, а группа `Calendar_Quarter` — дочерней.

## Смежные группы

Добавление смежных групп дает возможность показать параллельные динамические группы. Например, предположим, что мы хотим показать объемы продаж по регионам сгруппированными по категориям продуктов (в одной и той же матрице). В Visual Studio 2005 это было невозможно, но теперь это можно сделать при помощи выполнения нескольких простых шагов:

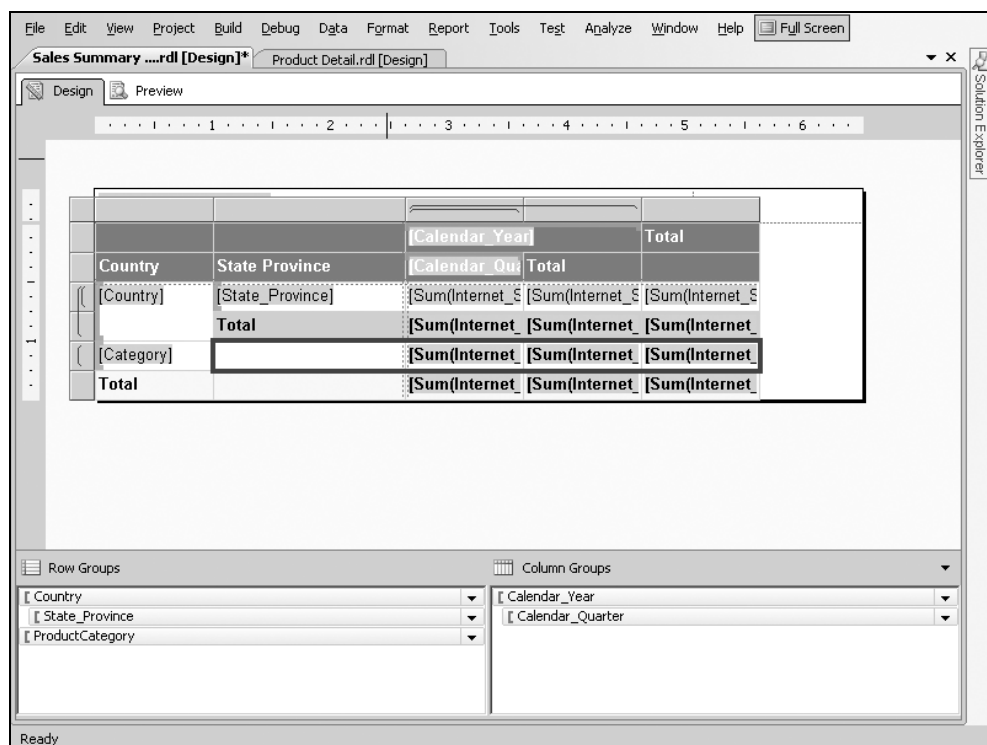
1. Щелкните правой кнопкой мыши по набору данных `SalesSummary` в панели **Report Data**, а затем выберите **Query**.
2. В окне **Query Editor** перетащите иерархию атрибута `Category` из измерения `Product` в окно запроса (левее столбца `Country`). Нажмите **OK** для сохранения изменений, а затем закройте окно **Query Editor**.
3. Щелкните правой кнопкой мыши по текстовому полю `[Country]` (в области данных на поверхности конструирования), а затем выберите в контекстном меню пункты **Add Group | Row Group | Adjacent Below**. Либо нажмите стрелку выпадающего списка группы строк **Country** в панели **Grouping**, а затем выберите **Add Group | Adjacent After**.
4. В диалоговом окне **Tablix Group** выберите поле `[Category]` в выпадающем списке. Щелкните по **OK**. На поверхности конструирования в область данных будет добавлена строка, а в панели **Grouping** появится новая группа.



5. Нажмите стрелку выпадающего списка для вновь добавленной в панели **Grouping** группы, а затем выберите **Group Properties**. Измените название с Group1 на ProductCategory.
6. Скопируйте выражения для сумм из строки State-Province в соответствующие ячейки вновь добавленной строки группы [Category]. Измените цвет фона синих ячеек в строке группы [Category] на **No Color**.

Ваша поверхность конструирования должна теперь выглядеть так, как показано на рис. 19.15.

Сделайте предварительный просмотр отчета. Вы увидите добавленные под заголовком Canada три строки, по одной на каждую категорию продаваемых компанией Adventure Works продуктов. Квартальные объемы продаж по географии и по продуктам показаны в одном отчете.



**Рис. 19.15.** Матричный отчет с родительскими и дочерними группами и смежной группой строк

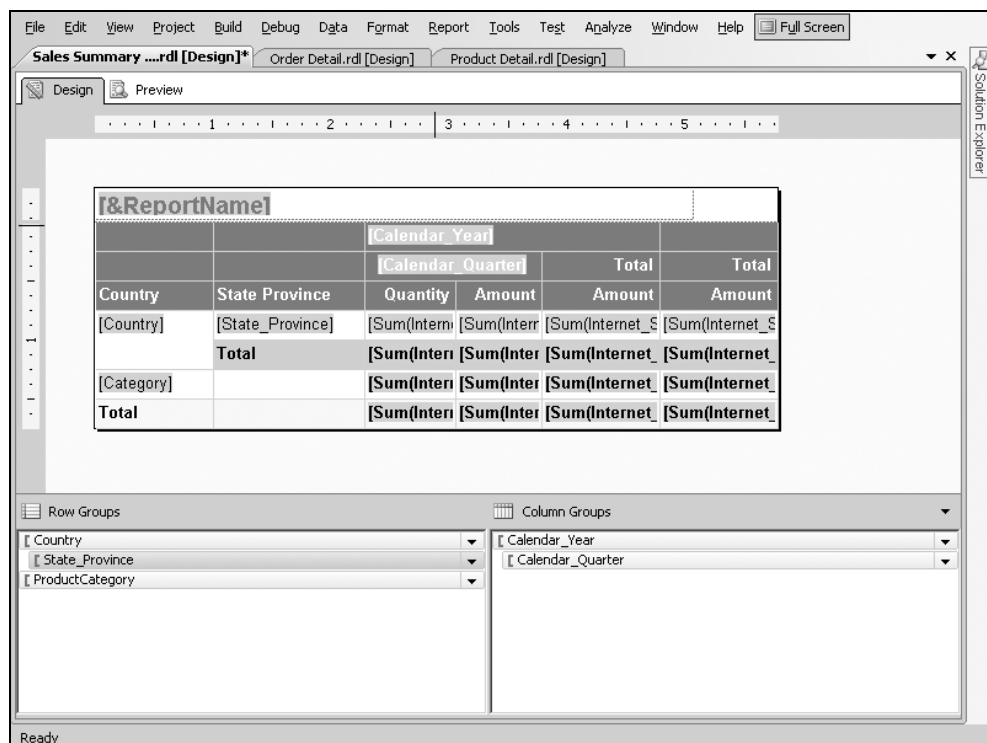
## Добавление фиксированных столбцов и строк

В область данных можно также добавить фиксированные столбцы или строки. Фиксированные столбцы и строки отличаются от групп столбцов и строк тем, что количество добавляемых столбцов или строк не зависит от данных. Когда мы добавляли в область данных `ProductCategory`, то в результате в отчет были добавлены три строки (по одной для каждой категории продуктов в наборе данных). Если компания Adventure Works начнет продавать еще одну категорию продуктов, то в отчете автоматически будет показано четыре строки (и для этого не потребуется делать изменений в отчете). В отличие от этого варианта, при добавлении фиксированного столбца или строки будет добавляться только один столбец или строка.

## Добавление внутри группы

Когда столбец (или строка) добавляется внутри группы, то он имеет область действия этой группы. Например, предположим, что мы хотим показать количество заказов рядом с объемами продаж, но нас интересуют только количества заказов по кварталам, а не по годам. Мы можем показать количество заказов по каждому кварталу путем добавления столбца внутри группы `Calendar_Quarter`. Щелкните правой кнопкой по текстовому полю `[Calendar_Quarter]`, а затем выберите пункты **Column | Inside Group Left** в контекстном меню. В области данных внутри группы `Calendar_Quarter` добавляется новый столбец. Перетащите `Internet_Order_Quantity` из узла **SalesSummary** панели **Report Data** в только что созданный столбец. Каждая ячейка нового столбца будет заполнена `[Sum(Internet_Order_Quantity)]`. Сделайте предварительный просмотр отчета. Вы увидите, что в каждую группу `Calendar_Quarter` отчета добавлено по одному столбцу, показывающему количество заказов по кварталу. Столбцы количества заказов и объема продаж заголовков не имеют. Давайте это исправим.

Вернитесь на вкладку **Design**. Щелкните правой кнопкой мыши по тестовому полю `[Calendar_Quarter]`, а затем выберите пункты **Insert Row | Inside Group Below** в контекстном меню. В таблицу добавляется новая строка. Щелкните по ячейке непосредственно под текстовым полем `[Calendar_Quarter]`, а затем выберите **Split Cells**. Введите `Quantity` в левом текстовом поле, а `Amount` — в правом. Поскольку мы показываем для объемов только подытоги и итоги, то мы хотим также уточнить метку для столбцов `Total`: введите `Amount` в только что добавленной строке над подытогами и итогами. На рис. 19.16 показана законченная поверхность конструирования.



**Рис. 19.16.** Матричный отчет с добавленными фиксированными столбцами и строками

Сделайте в отчете любое нужное вам форматирование. На рис. 19.16 из эстетических соображений было выполнено дополнительное форматирование: уменьшена ширина столбцов, отцентрированы некоторые заголовки столбцов. Когда вас устроит полученный результат, нажмите кнопку **Save All** на панели инструментов (чтобы сохранить файлы по состоянию на данный момент).

## Добавление вне группы

Когда столбец (или строка) добавляется вне группы, то он имеет область действия родительской группы (либо не имеет групп в области своего действия — если родительской группы нет). Щелкните правой кнопкой мыши по текстовому полю [Calendar\_Year], а затем выберите пункты **Insert Column Outside Group Left** в контекстном меню. В области данных левее группы Calendar\_Year (и вне ее) добавляется новый столбец. Мы можем сказать, что

столбец находится снаружи группы `Calendar_Year` также потому, что при щелчке по ячейке столбца не активируется желтый индикатор группирования для группы `Calendar_Year`.

В нашем текущем примере мы не будем добавлять столбец вне группы `Calendar_Year`. Вернитесь во вкладку **Design**, а затем нажмите кнопку **Undo** в панели инструментов (или нажмите комбинацию клавиш `<Ctrl>+<Z>`, либо щелкните правой кнопкой мыши по ячейке только что добавленного столбца и выберите пункт **Delete Columns** в контекстном меню). Нажмите кнопку **Save All** на панели инструментов (чтобы сохранить файлы по состоянию на текущий момент).

## Добавление области данных Chart Data Region

Возможности построения диаграмм в Reporting Services в SQL Server 2008 были значительно улучшены (в результате приобретения компанией Microsoft компонентов Dundas Data Visualization в 2007 г.). Визуальный конструктор Report Designer позволяет отображать несколько областей данных в одном отчете. Это особенно удобно для совместного показа в отчете сетки данных и диаграммы (что дает пользователям два разных способа просмотра одних и тех же данных). Мы собираемся добавить в отчет `Sales Summary by Region and Date` диаграмму тенденций, в которой будут показаны квартальные объемы продаж.

Вернемся на вкладку **Design**. Начнем с расширения тела отчета, чтобы в нем появилось некоторое свободное пространство под имеющейся матрицей: если вы наведете курсор мыши на нижний край тела отчета, то он изменится на двустороннюю вертикальную стрелку, а это означает, что нижний край можно растягивать вниз.

Затем перетащите элемент диаграммы из панели инструментов на свободное пространство под матрицей. Уделите время просмотру имеющихся типов диаграмм, а затем выберите первый тип диаграммы в строке `Line charts` диалогового окна **Chart Type** (самая левая диаграмма с подсказкой **Line**). Нажмите **ОК**. Разместите диаграмму таким образом, чтобы выровнять ее по левой стороне с телом отчета (сразу под матрицей). Разверните диаграмму таким образом, чтобы она была 7 дюймов в ширину и 3,5 дюйма в высоту (либо при помощи растягивания границ диаграммы на панели конструирования, либо при помощи настроек свойств **Width** и **Height** в узле **Size** панели **Properties**).

Настройте данные диаграммы (при помощи перетаскивания полей из панели **Report Data** в область диаграммы). Дважды щелкните по диаграмме. Появятся манипуляторы для полей данных, категории и рядов. Перетащите `Internet_Sales_Amount` и `Internet_Order_Quantity` из панели **Report Data** на манипулятор данных. Перетащите `Calendar_Quarter` на манипулятор категорий, а `Country` — на манипулятор рядов.

Сделайте предварительный просмотр отчета. Если нужно, прокрутите вниз для того, чтобы увидеть диаграмму. Как и в случае добавленных в отчет таблиц и матриц, нам нужно применить к диаграмме некое форматирование (чтобы она лучше выглядела). Во-первых, диаграмма должна иметь заголовок, а ее оси должны быть промаркированы. Более того, диаграмма прижата влево (поскольку место справа занимает легенда). Еще более серьезная проблема в том, что даже несмотря на то, что в легенде указаны четыре элемента, на диаграмме имеется только две линии (соответствующие объемам продаж). Поскольку объемы продаж и количество заказов имеют разные порядки значений, то показ их на одной диаграмме при помощи одной оси не имеет практического смысла. Мы собираемся заняться всеми этими проблемами. Вернемся на вкладку **Design**.

В Visual Studio 2005 все форматирование элементов диаграмм (включая присваивание заголовка диаграмме и меток ее осям) можно было выполнить только при помощи диалоговых окон и свойств. В Visual Studio 2008 форматирование диаграмм интуитивно. В то время как в Visual Studio 2005 диаграммная область данных на поверхности конструирования была просто указателем места заполнения для диаграммы, в Visual Studio 2008 диаграммная область на поверхности конструирования более полно представляет собой диаграмму (элементы диаграммы можно интерактивно настраивать в области диаграммы). Давайте посмотрим, как это делается.

Сначала мы добавим заголовок диаграммы и разметим оси диаграммы. Дважды щелкните по `Chart Title` в диаграммной области данных. Это активирует манипуляторы диаграммы и покажет границы текстового поля заголовка диаграммы. Еще раз дважды щелкните по `Chart Title` (чтобы перейти в режим редактирования текстового поля). Измените заголовок на `Sales Amount And Order Quantity Trend`. Дважды щелкните по **Axis Title** для оси *y* (вертикальная ось). Промаркируйте ось как `Sales Amount`.

Затем мы добавим вторичную ось *y* для значений `Order Quantity`. Диаграммы с двойными осями теперь делаются собственными средствами Reporting Services, раньше такую диаграмму можно было создать только при помощи надстройки Dundas для Reporting Services. Щелкните правой кнопкой мыши элемент данных `Order Quantity` в манипуляторе данных, а затем выберите

пункт **Series Properties** в контекстном меню. Выберите страницу **Axes And Chart Area** в списке переходов на левой стороне диалогового окна. Измените опцию оси **Value** с **Primary** на **Secondary**. Нажмите **OK** для сохранения ваших изменений и закройте диалоговое окно. Обратите внимание, что на правой стороне области диаграммы добавилась ось у. Дважды щелкните по **Axis Title** и измените текст на *Order Quantity*.

Перенесите легенду с правой стороны диаграммы под диаграмму (посредством перетаскивания). Можно щелкнуть правой кнопкой мыши по легенде (или свободное пространство в области диаграммы), а затем использовать диалоговое окно **Legend Properties** для изменения положения легенды. Поскольку каждая часть области диаграммы представляет собой некий реальный элемент диаграммы, то вам может потребоваться не одна попытка, чтобы попасть в то место, которое имеет в контекстном меню пункт **Legend Properties**.

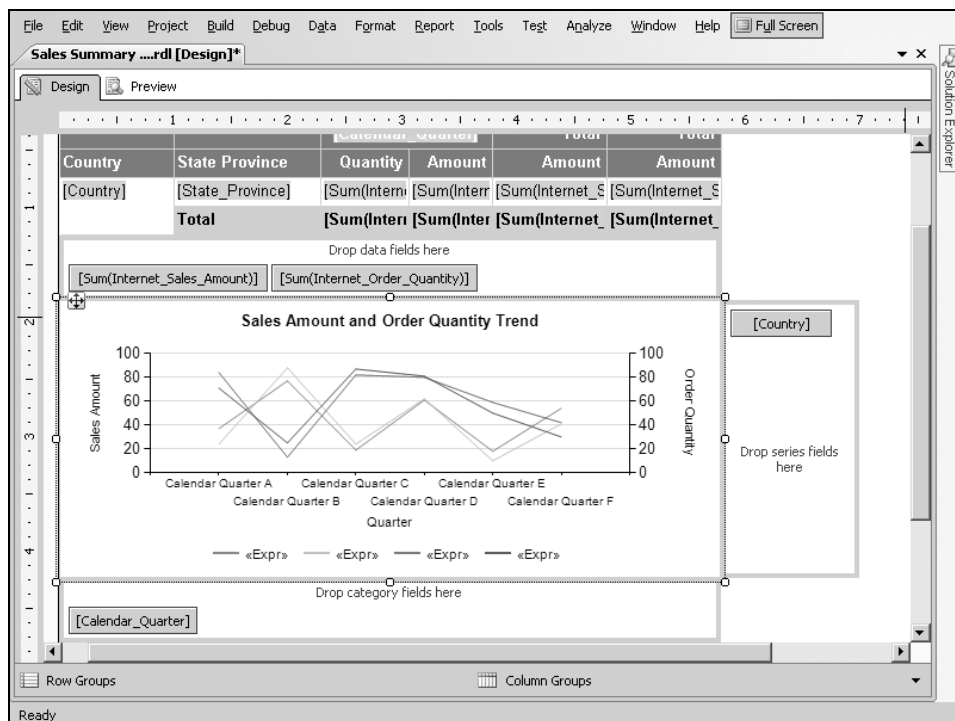


Рис. 19.17. Отформатированная диаграммная область данных

И наконец, мы отредактируем текст легенды (чтобы сделать диаграмму более привлекательной). Щелкните правой кнопкой мыши по элементу данных

Sales Amount в манипуляторе данных, а затем выберите пункт **Select Properties** контекстного меню. Выберите панель **Legend** в списке переходов в левой части диалогового окна. Нажмите кнопку **Expression** для текста легенды Custom. Введите следующее выражение в текстовом поле **Expression Editor**:

```
=Fields!Country.Value & " Sales Amount"
```

Нажмите **ОК** для сохранения изменений и закройте диалоговое окно. Повторите этот процесс для элемента данных Order Quantity и введите в текстовом поле **Expression Editor** следующее выражение:

```
=Fields!Country.Value & " Order Quantity"
```

На рис. 19.17 показана отформатированная область диаграммы в визуальном конструкторе Report Designer.

Сделайте предварительный просмотр страницы. Теперь в диаграмме появятся четыре линии. Измените выбор диапазона дат или выбор страны. Сделанный вами выбор будет отражен как в сетке, так и на диаграмме. Нажмите кнопку **Save All** для сохранения файлов в их текущем состоянии.

## Делаем отчет интерактивным

Службы Reporting Services предлагают множество возможностей для взаимодействия пользователей с отчетом. Мы уже видели возможности параметров, позволяющих пользователям выбирать те данные, которые они хотят видеть в отчетах. Однако иногда мы хотим позволить пользователям управлять не только содержимым отчета, но и глубиной анализа данных. Вы можете дать вашим пользователям возможность скрывать и показывать столбцы отчета, вызывать Web-страницы, углубляться в связанные отчеты — и это еще не все.

### Переключение видимости строк

Наш простой отчет Order Summary by Region and Date стал несколько громоздким (после добавления в него подытогов, групп и столбцов). Мы хотим при просмотре этого отчета пользователем сначала дать ему простой обзор отчета. Мы сделаем это при помощи скрытия столбцов кварталов и добавления переключателя для столбцов по годам (чтобы позволить пользователям углубиться при необходимости до квартала).

Вернемся в представление Design. Нажмите выпадающую стрелку, соответствующую группе Calendar\_Quarter в панели **Grouping**, а затем выберите

**Group Properties.** Выберите страницу **Visibility** в списке переходов на левой стороне диалогового окна. Выберите опцию **Hide** для **When The Report Is Initially Run**. Установите флажок **Display Can Be Toggled By This Report Item**, а затем выберите группу `Calendar_Year` в выпадающем списке. Нажмите **OK** (чтобы закрыть диалоговое окно).

Сделайте предварительный просмотр отчета. Вы увидите, что столбцы кварталов теперь скрыты, и что рядом с каждым годом появился значок плюса. Нажмите значок плюса рядом с годом для того, чтобы показать столбцы кварталов только для этого года.

## Добавление в отчет действий

Службы Reporting Services позволяют настроить три типа действий для щелчка пользователя по ячейке типа `tablix`: **Go To URL**, **Go To Bookmark** и **Go To Report**. Для нашего сводного отчета по заказам мы добавим действие **Go To Report**.

Мы разрешим пользователям из нашего сводного отчета углубляться (то есть открывать его) до отчета `Order Detail`, который мы создали ранее в этой же главе. Поскольку отчету `Order Detail` требуются параметры, то мы должны поставить им в соответствие те значения, которые выбраны пользователем посредством щелчка по сводному отчету. Вернемся в представление `Design`. Щелкните правой кнопкой мыши по ячейке на пересечении групп `Calendar_Quarter` и `State_Province`, а затем выберите пункт **Text Box Properties** контекстного меню. Выберите страницу **Action** в списке переходов в левой части диалогового окна **Text Box Properties**. Затем выберите вариант **Go To Report** и пункт **Order Detail** в выпадающем списке **Select A Report From The List**. Нажмите кнопку **Add**. Выберите элемент `Country` в выпадающем списке **Name** и `[Country]` в выпадающем списке **Value**. Еще раз нажмите **Add**. Во второй строке выберите `State` в выпадающем списке **Name** и `[State_Province]` в выпадающем списке **Value**.

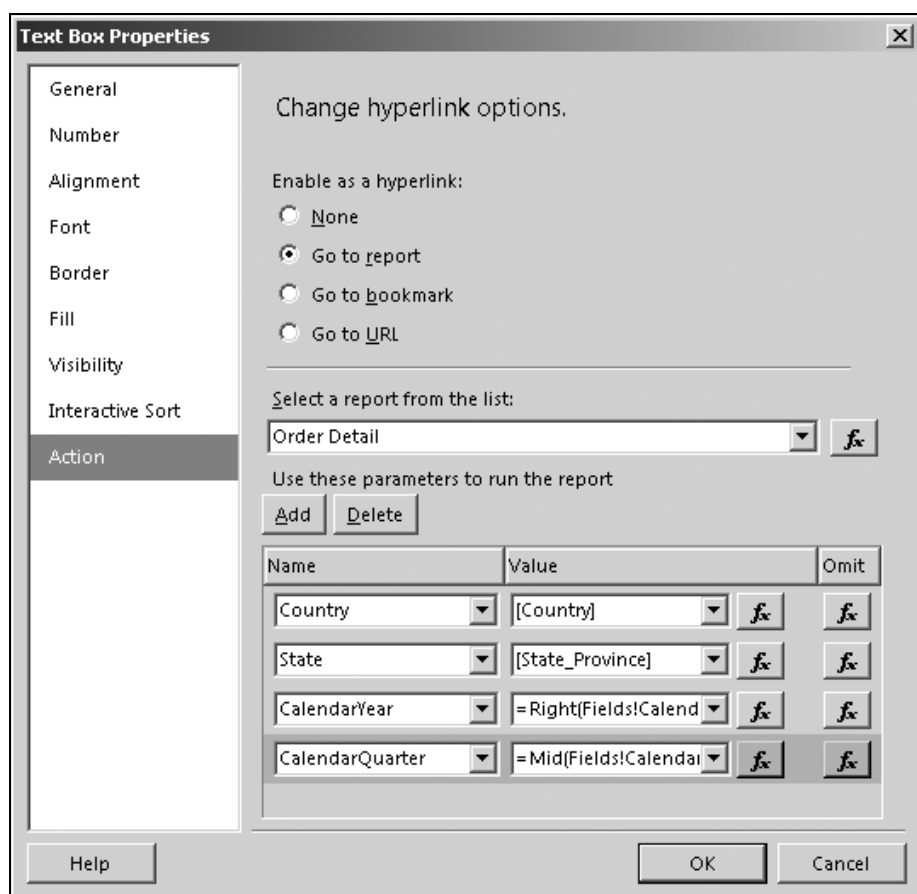
Для параметров `CalendarYear` и `CalendarQuarter` необходимо написать выражения, поскольку названия элементов не совпадают с требующимися для запроса отчета `Order Detail` значениями. Точнее говоря, запрос ожидает год и номер квартала. В сводном же отчете `[Calendar_Year]` отформатирован как `SY yyyy`, а `[Calendar_Quarter]` отформатирован как `Qq SY yyyy`. Перед передачей значений в отчет `Order Detail` лишние символы необходимо удалить. Это можно легко сделать при помощи функций **Right** и **Mid** языка Visual Ba-



sis. Введите показанные в следующей таблице выражения в качестве значений параметров CalendarQuarter и CalendarYear.

| Название        | Выражение для значения                    |
|-----------------|-------------------------------------------|
| CalendarYear    | =Right(Fields!Calendar_Year.Value, 4)     |
| CalendarQuarter | =Mid(Fields!Calendar_Quarter.Value, 2, 1) |

На рис. 19.18 показано заполненное диалоговое окно. Нажмите **ОК** для сохранения сделанных вами настроек и закройте диалоговое окно.



**Рис. 19.18.** Заполненное диалоговое окно **Text Box Properties** для действия **Go To Report**

Сделайте предварительный просмотр отчета. Растяните столбец `SY 2002`, а затем наведите курсор мыши на ячейку, показывающую объем продаж для конкретного квартала и провинции. Курсор изменится на руку, а это означает, что по ячейке можно щелкнуть. Щелкните по ячейке для показа отчета `Order Detail` с указанием заказов для выбранного квартала и провинции (из сводного отчета). Для возвращения в сводный отчет нажмите указывающую влево стрелку на панели инструментов отчета.

Нажмите кнопку **Save All** на панели инструментов для сохранения файлов по состоянию на текущий момент.

## Доставка отчетов

Теперь, когда вы научились создавать разные типы отчетов, оставшаяся часть данной главы будет посвящена доставке этих отчетов вашим пользователям.

## Развертывание на сервер отчетности

После того как вы создали некие отчеты и протестировали их в среде разработки, пришло время сделать их доступными для всех пользователей. Развертывание включает в себя выгрузку файлов RDL ваших отчетов из среды разработки на сервер отчетности, где к ним могут получить доступ все пользователи. Есть три способа развертывания отчетов: непосредственно из Visual Studio; при помощи использования скрипта и утилиты командной строки; из Web-интерфейса Report Manager. Мы изучим все три способа.

### **ПРИМЕЧАНИЕ**

Возможность развертывания отчетов через SQL Server Management Studio в версии SQL Server 2008 отсутствует.

## Развертывание отчетов при помощи Visual Studio 2008

Самый простой способ развернуть отчеты и источники данных — сделать это непосредственно из Visual Studio. Можно развернуть как весь проект отчета, так и один отдельный отчет. Для развертывания целого проекта выберите пункт **Deploy** в меню **Build** либо щелкните правой кнопкой мыши по проекту в окне **Solution Explorer** и выберите **Deploy**.

Целевое местоположение для развертываемых при помощи Visual Studio отчетов определяется настройкой свойств проекта в панели **Properties**. Если панель **Properties** скрыта, то вы можете показать ее при помощи выбора пункта **Properties** в меню **Project**. В табл. 19.3 описаны свойства проекта (связанные с развертыванием).

**Таблица 19.3.** Свойства развертывания в Visual Studio

| Свойство               | Описание                                                                                                                                                                                                                                                                                                                                              |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OverwriteDataSources   | Если установлено в значение true, то развертывание перезаписывает существующие источники данных                                                                                                                                                                                                                                                       |
| TargetDataSourceFolder | Это местоположение, куда развертываются источники данных. Пустое значение приведет к тому, что источники данных будут развертываться в то же самое местоположение, куда развертываются отчеты. Значения вводятся без префикса в виде абсолютного адреса URL (они являются относительными к пути TargetServerURL), например, SampleReports/DataSources |
| TargetReportFolder     | Это местоположение, в которое развертываются отчеты. Значения вводятся без префикса в виде абсолютного адреса URL, например, SampleReports/Reports                                                                                                                                                                                                    |
| TargetServerURL        | Абсолютный URL сервера, на который должны быть развернуты отчеты. Местоположения в TargetDataSourceFolder и TargetReportFolder указаны относительно этого URL — например, http://servername/reportserver                                                                                                                                              |

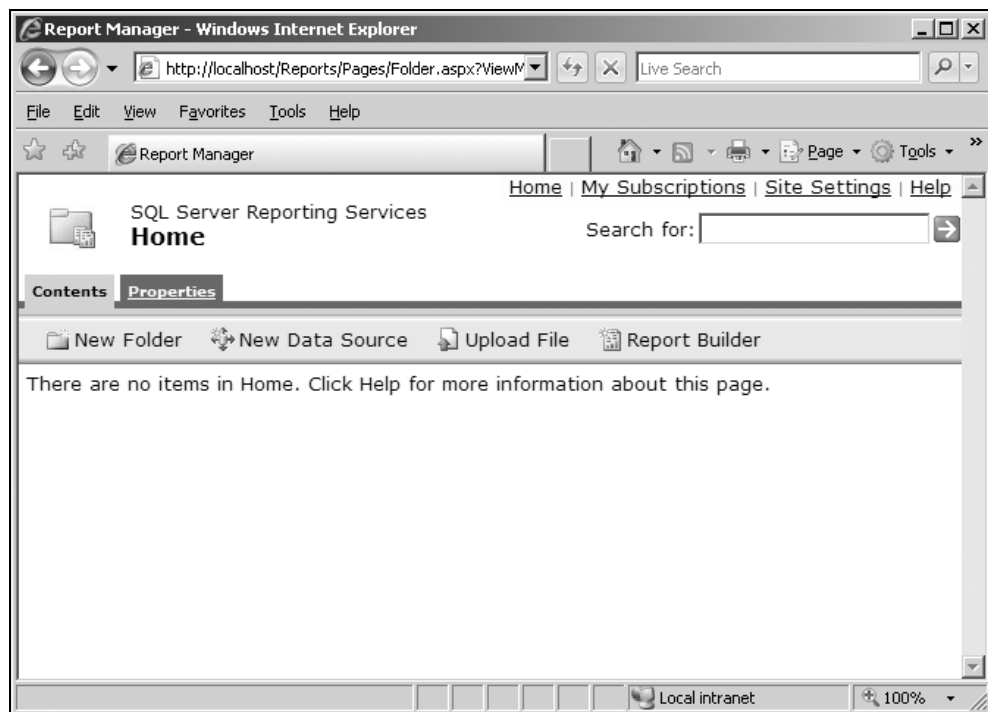
### ПРИМЕЧАНИЕ

При развертывании на сайт под управлением SharePoint (обсуждается далее в этой же главе) все значения местоположений должны вводиться в виде абсолютных URL. Например, свойство TargetReportFolder при развертывании на сайт SharePoint должно быть http://servername/site/subsite/reports.

## Использование Report Manager

Альтернативный способ развертывания отчетов на Web-сайт — это использование интерфейса Report Manager. Доступ к Report Manager можно получить посредством перехода по ссылке **http://servername/reports**. Появляющиеся

элементы меню будут разными для пользователя и для администратора. Администратор получает дополнительные элементы меню с настройками сайта, поддержанием безопасности, а также организацией и выгрузкой отчетов. На рис. 19.19 показано представление для администратора (по умолчанию).



**Рис. 19.19.** Вид браузера Report Manager для администратора

Кнопки на панели инструментов (используемые для развертывания отчетов) описаны в табл. 19.4.

**Таблица 19.4.** Кнопки панели инструментов в Report Manager

| Кнопка                 | Назначение                                                                                                                                         |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>New Folder</b>      | Используйте эту кнопку для создания каталога в текущем каталоге                                                                                    |
| <b>New Data Source</b> | Используйте эту кнопку для создания нового источника данных в текущем каталоге. Информацию по созданию источников данных ищите в начале этой главы |

**Таблица 19.4** (окончание)

| Кнопка             | Назначение                                                                                                                                              |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Upload File</b> | Используйте эту кнопку для загрузки элемента в текущий каталог. Обычно выгружаются файлы RDL, но при помощи этой кнопки можно также выгрузить и ресурсы |

Для развертывания нового отчета в новый каталог выполните следующее:

1. На странице **Home** в Report Manager нажмите **New Folder**. В поле **Name** введите `Sample Reports`, а затем нажмите **OK**.

### **ПРИМЕЧАНИЕ**

Вновь добавленные элементы в Report Manager обозначаются как **!New**.

2. Щелкните по каталогу **Sample Reports**, а затем нажмите **New Data Source** на панели инструментов. В окне **New Data Source** введите значения из табл. 19.5, а затем нажмите **OK**.

**Таблица 19.5.** Поля источника

| Поле                     | Значение                                                        |
|--------------------------|-----------------------------------------------------------------|
| <b>Name</b>              | AdventureWorksCube                                              |
| <b>Data Source Type</b>  | Microsoft SQL Server Analysis Services                          |
| <b>Connection String</b> | Data Source=localhost;Initial Catalog="Adventure Works DW 2008" |
| <b>Connect Using</b>     | Windows Integrated Security                                     |

3. После обновления окна еще раз нажмите **New Data Source**. Введите значения из табл. 19.6, а затем нажмите **OK** для развертывания второго источника данных.

**Таблица 19.6.** Поля второго источника

| Поле                    | Значение             |
|-------------------------|----------------------|
| <b>Name</b>             | AdventureWorksDW2008 |
| <b>Data Source Type</b> | Microsoft SQL Server |

Таблица 19.6 (окончание)

| Поле                     | Значение                                                   |
|--------------------------|------------------------------------------------------------|
| <b>Connection String</b> | Data Source=localhost;Initial Catalog=AdventureWorksDW2008 |
| <b>Connect Using</b>     | Windows Integrated Security                                |

- Нажмите **Upload File** на панели инструментов. Нажмите кнопку **Browse**, а затем перейдите в местоположение созданного нами ранее проекта Reporting Services. Выберите файл Sales Summary by Region and Date.rdl и нажмите **Open**, а затем **OK**.
- Окончательный результат должен выглядеть так, как показано на рис. 19.20.

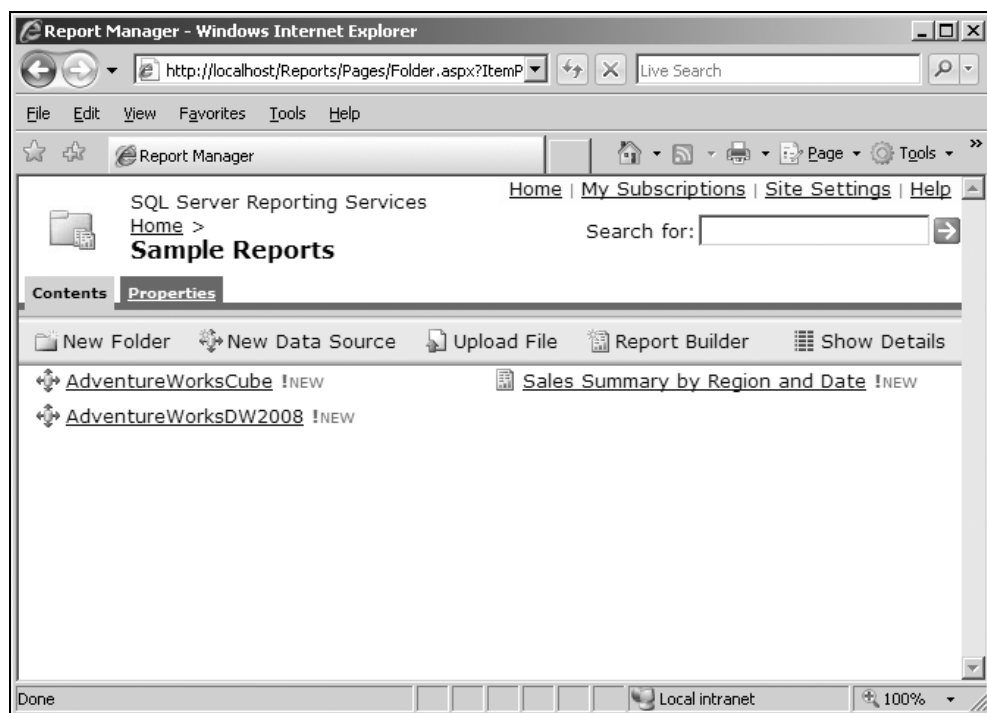


Рис. 19.20. Report Manager после развертывания отчета

### СОВЕТ

Для редактирования существующих элементов нажмите **Show Details** на панели инструментов, а затем нажмите кнопку **Edit** рядом с тем элементом, который вы хотите изменить. Для удаления существующих элементов нажмите **Show Details**, установите флажки (элементов, которые вы хотите удалить), а затем нажмите кнопку **Delete**.

## Использование командной строки

Когда дело доходит до реализации автоматизированного процесса развертывания, ни один из предыдущих методов развертывания не подходит. В современных корпоративных средах развертывания должны быть воспроизводимыми и иметь вид скриптов. Это требование исключает интерактивное использование Visual Studio для развертывания проекта или загрузки файлов на Web-сайт. Утилита командной строки rs.exe предоставляется именно для такой цели. Применяя эту утилиту, вы можете программным образом администрировать сервер отчетов (при помощи скриптов на языке Visual Basic .NET, хранящихся в текстовых файлах rss). Кроме развертывания отчетов эти скрипты можно использовать для выполнения любых действий с Web-службами Reporting Services. (Web-службы Reporting Services предоставляют разработчику всю функциональность Reporting Services и описываются далее в этой же главе).

Вот синтаксис утилиты:

```
rs {-?} [-iinput_file=] [-sserverURL] {-username} {-ppassword}
    {-ltime_out} {-bbatchmode} {-vglobalvars=} {-eendpoint} {-ttrace}
```

### ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ

Подробное изложение утилиты rs выходит за рамки данной главы. Полное описание этой утилиты смотрите в документации SQL Server Books Online.

Скрипт Reporting Services имеет следующие характеристики:

- ☐ написан на языке Microsoft Visual Basic .NET;
- ☐ содержит процедуру main, которая является точкой входа в файл скрипта при его выполнении;
- ☐ использует переменную rs на уровне модуля для выполнения любых операций Web-служб, которые имеются в библиотеках Web-служб. Среда скрипта автоматически подключается к серверу отчетов, создает класс

Web-прокси и генерирует на уровне модуля переменную rs, которая ссылается на прокси-объект Web-служб;

- ❑ может получить доступ к пространствам имен System.Web.Services, System.Web.Services.Protocols, System.Xml и System.IO в .NET Framework (которые становятся доступными благодаря среде скрипта);
- ❑ глобальные переменные настраиваются из командной строки при помощи синтаксиса `-v variable=value`, можно передавать несколько глобальных значений. Следующая строка задает две глобальные переменные (server-Name и databaseName):

```
-v serverName=localhost -v databaseName="Adventure Works DW 2008"
```

Большинство операций Web-служб (которые связаны с задачами развертывания отчетов) перечислены в табл. 19.7.

### **ПРИМЕЧАНИЕ**

Несмотря на то, что названия конечных точек Web-служб заканчиваются на 2005 или 2006, эти конечные точки используются для служб SQL Server 2008 Reporting Services.

**Таблица 19.7.** Операции Web-служб для развертывания отчетов

| Операция         | Описание                                                                                                                                                                                                                                                                             |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rs.Credentials   | Настраивает учетные данные, используемые Web-службой, на объект типа System.Net.NetworkCredential, например:<br>rs.Credentials = System.Net.CredentialCache.DefaultCredentials                                                                                                       |
| rs.CreateFolder  | Создает каталог на сервере отчетности. Названия каталога и родительского каталога указываются как строки. Для настройки свойств каталога может использоваться массив объектов типа ReportingService2005.Property, например:<br>rs.CreateFolder (folderName, folderParent, Nothing)   |
| rs.GetProperties | Возвращает значение одного или более свойств элемента. В качестве аргументов операция принимает полный маршрут к элементу и массив объектов ReportingService2005.Property (для которых вы хотите получить значения), например:<br>rs.GetProperties(folderParent & folderName, props) |



Таблица 19.7 (окончание)

| Операция                         | Описание                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rs.CreateDataSource</code> | Создает источник данных на сервере отчетности. Название источника данных и родительский маршрут задаются как строки, а булево значение указывает, следует ли перезаписывать существующий источник данных. Определение источника данных делается путем создания переменной типа <code>ReportingService2005.DataSourceDefinition</code> и настройки соответствующих свойств нового источника данных. Для настройки свойств источника данных может быть также указан массив объектов <code>ReportingService2005.Property</code> , например: <code>rs.CreateDataSource (dsName, parentPath, True, definition, Nothing)</code> |
| <code>rs.CreateReport</code>     | Создает отчет на сервере отчетов. Название отчета и родительский маршрут — строки, а булево значение указывает, следует ли перезаписывать существующий источник данных. Содержимое файла RDL отчета указывается как массив байтов (который обычно загружается путем чтения файла <code>rdl</code> из местоположения источника). Для настройки свойств источника данных может быть также указан массив объектов <code>ReportingService2005.Property</code> , например: <code>rs.CreateReport (reportName, parentPath, True, definition, Nothing)</code>                                                                    |

В листинге 19.1 показан полный скрипт для развертывания двух дополнительных отчетов и повторного развертывания одного из источников данных в тот же самый каталог, который мы уже создали ранее при помощи Report Manager. Скрипт выполняет эти задачи путем взаимодействия с элементами Web-служб Reporting Services. Кроме взаимодействия с Web-службой, код использует несколько стандартных .NET-объектов для вывода информации о состоянии, чтения файла с диска, а также реализации обработки исключений.

**Листинг 19.1. Скрипт `DeployReports.rss`, который развертывает каталог, источник данных и отчеты**

```
Dim definition As [Byte]() = Nothing
Dim warnings As Warning() = Nothing
Dim filePath As String = "./"
Public Sub Main()
    rs.Credentials = System.Net.CredentialCache.DefaultCredentials
```

```

' Создать корневые каталоги
CreateFolder("Sample Reports", "/")
' Создать совместно используемые источники данных
' для каталога Sample Reports
CreateSQLDataSource("/Sample Reports", "AdventureWorksDW2008",
    serverName, databaseName, "", "")
PublishReport("Order Detail", "Order Detail", "/Sample Reports")
PublishReport("Product Detail", "Product Detail", "/Sample Reports")
End Sub

Public Sub CreateFolder(ByVal folderName As String,
    ByVal folderParent As String)
    If DoesFolderExist(folderName, folderParent) = False Then
        Try
            rs.CreateFolder(folderName, folderParent, Nothing)
            Console.WriteLine("Parent folder created: {0}", folderName)
        Catch e As Exception
            Console.WriteLine(e.Message)
        End Try
    Else
        Console.WriteLine("Folder {0} already exists.",
            folderParent & folderName)
    End If
End Sub

Public Function DoesFolderExist(ByVal folderName As String,
    ByVal folderParent As String) as Boolean
    ' Создать свойство для извлечения.
    Dim retrieveProp As New [Property]()
    retrieveProp.Name = "Description"
    Dim props(0) As [Property]
    props(0) = retrieveProp
    Try
        Dim properties As [Property]() = rs.GetProperties(folderParent
            & folderName, props)

        DoesFolderExist = True
    Catch e As SoapException
        DoesFolderExist = False
    End Try
End Function

```

```
Public Sub CreateSQLDataSource(ByVal parentPath As String,
    ByVal dsName As String, ByVal serverName As String,
    ByVal catalogName As String, ByVal userName As String,
    ByVal password As String)
    Console.WriteLine("CreateSQLDataSource: Name:{0}; Server:{1};
        Catalog:{2}; User:{3};", dsName, serverName, catalogName,
        userName)
    Dim strConnection As String
    strConnection = "Data Source=" & serverName &
        ";Initial Catalog=" & catalogName
    CreateDataSource(parentPath, dsName, strConnection, "SQL",
        userName, password, True)
End Sub

Public Sub CreateOLAPDataSource(ByVal parentPath As String,
    ByVal dsName As String, ByVal serverName As String,
    ByVal catalogName As String, ByVal userName As String,
    ByVal password As String)
    Console.WriteLine("CreateOLAPDataSource: Name:{0}; Server:{1};
        Catalog:{2};", dsName, serverName, catalogName)
    Dim strConnection As String
    strConnection = "Data Source=" & serverName & ";Initial Catalog="
        & catalogName
    CreateDataSource(parentPath, dsName, strConnection, "OLEDB-MD",
        userName, password, True)
End Sub

Public Sub CreateDataSource(ByVal parentPath As String,
    ByVal dsName As String, ByVal strConnection As String,
    ByVal strExtension As String, ByVal userName As String,
    ByVal password As String, ByVal windowsCredentials As Boolean)
    ' Дать определение источника данных.
    Dim definition As New DataSourceDefinition()
    If username = "" Then
        definition.CredentialRetrieval = CredentialRetrievalEnum.Integrated
    Else
        definition.CredentialRetrieval = CredentialRetrievalEnum.Store
        definition.Username = userName
        definition.Password = password
    End If
    definition.WindowsCredentials = windowsCredentials
```

```
definition.ConnectionString = strConnection
definition.Enabled = True
definition.EnabledSpecified = True
definition.Extension = strExtension
definition.ImpersonateUser = False
definition.ImpersonateUserSpecified = True
definition.Prompt = Nothing
Try
    rs.CreateDataSource(dsName, parentPath, True, definition, Nothing)
    Console.WriteLine("Datasource: {0} created successfully with no
                        warnings", dsName)
Catch e As Exception
    Console.WriteLine(e.Message)
End Try
End Sub

Public Sub PublishReport(ByVal reportFile As String,
                        ByVal reportName As String, ByVal parentPath as String)
    If reportName = "" Then
        reportName = reportFile
    End If
    Dim definition As [Byte]() = Nothing
    Dim warnings As Warning() = Nothing
    Try
        Dim stream As FileStream = File.OpenRead(filePath +
                                                    reportFile + ".rdl")
        definition = New [Byte](stream.Length-1) {}
        stream.Read(definition, 0, CInt(stream.Length))
        stream.Close()
    Catch e As IOException
        Console.WriteLine(e.Message)
    End Try
    Try
        warnings = rs.CreateReport(reportName, parentPath, True, definition,
Nothing)
        If Not (warnings Is Nothing) Then
            Dim warning As Warning
            For Each warning In warnings
                Console.WriteLine(warning.Message)
            Next warning
        End If
    End Try
End Sub
```

```
Else
    Console.WriteLine("Report: {0} published successfully with no
                        warnings", reportName)

End If
Catch e As Exception
    Console.WriteLine(e.Message)
End Try
End Sub
```

Для выполнения скрипта развертывания откройте командную строку, а затем перейдите в тот каталог, где расположен файл rss. Например, для выполнения показанного в листинге 19.1 скрипта DeployReports.rss по базе данных AdventureWorksDW2008 на локальном сервере выполните следующую команду:

```
rs -i DeployReports.rss -s http://localhost/reportserver
-v serverName=localhost -v databaseName="Adventure Works DW 2008"
```

### **ПРИМЕЧАНИЕ**

Исполняемый файл rs находится по умолчанию в \Program Files\Microsoft SQL Server\100\Tools\Binn. Этот каталог должен находиться в вашем маршруте поиска. Если команда не выполняется, то либо откорректируйте маршрут поиска, либо добавьте в эту команду полный путь к исполняемому файлу rs и попробуйте еще раз.

## **Программный доступ к отчетам**

Очень часто требования по предоставлению отчетов пользователям бывают очень сложны и выходят за пределы возможностей готовых инструментов представления отчетов. К счастью, у нас есть возможность написания кода для того, чтобы пользователи могли получить доступ к нашим отчетам. Поддерживаемые методы программирования доставки отчетов показаны в табл. 19.8.

**Таблица 19.8.** Поддерживаемые методы программирования доставки отчетов

| Метод                            | Описание                                                                                                                                                                   |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Элемент управления Report Viewer | Элементы управления средством просмотра отчетов позволяют программисту встроить отчет непосредственно в любую Web-страницу ASP.NET или настольное приложение Windows Forms |

Таблица 19.8 (окончание)

| Метод                   | Описание                                                                                                                                                                                                                  |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Интеграция с SharePoint | При помощи настройки Reporting Services можно опубликовать отчеты и прочие элементы в библиотеке SharePoint. Эти отчеты можно затем использовать в решениях на базе SharePoint                                            |
| Доступ через URL        | Этот способ позволяет визуализировать отчет практически из любого приложения. Есть параметры запроса, которые позволяют вызывающей стороне указывать желаемый формат. Основной URL: <b>http://servername/reportserver</b> |
| Web-служба XML          | Этот способ позволяет любому приложению (которое может вызвать Web-службы XML) визуализировать отчеты, а также администрировать сервер отчетности                                                                         |

## Использование URL-доступа для просмотра отчетов

Доступ через URL — это очень удобный вариант для интегрирования просмотра отчетов в ваши приложения (особенно для неуправляемых приложений, которые не могут использовать элементы управления ReportViewer). Например, если вы разрабатываете приложения при помощи классических Active Server Pages (ASP), то доступ через URL можно использовать для встраивания ссылок на отчеты в ваши Web-страницы. Доступ к серверу отчетов будет производиться через URL-запросы. Полный список имеющихся параметров очень длинный и отражает весь набор имеющихся в Reporting Services функциональных возможностей. Полный список вы можете найти в документации SQL Server Books Online. Вот пример URL, который генерирует отчет в виде документа Word:

**http://localhost/ReportServer/Pages/ReportViewer.aspx?/Sample%20Reports/Sales%20Summary%20by%20Region%20and%20Date&rs:Command=Render&rs:format=WORD.**

В табл. 19.9 приведено значение всех частей этого URL.

**Таблица 19.9.** Синтаксис URL для доставки отчетов в формате документов Word 2007

| Синтаксис                                              | Описание                                                  |
|--------------------------------------------------------|-----------------------------------------------------------|
| http://localhost/ReportServer/Pages/ReportViewer.aspx? | URL сервера отчетов и маршрут к Web-странице ReportViewer |

Таблица 19.9 (окончание)

| Синтаксис                                                    | Описание                                                                                 |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------|
| /Sample%20Reports/Sales%20Summary%20by%20Region%20and%20Date | Полный маршрут к визуализируемому отчету. Пробелы должны быть заменены на %20            |
| &rs:Command=Render                                           | Указание серверу отчетности визуализировать отчет                                        |
| &rs:format=WORD                                              | Указание серверу отчетности визуализировать отчет в виде документа Microsoft Office Word |

## Использование элементов управления *ReportViewer*

Для облегчения интегрирования отчетности с .NET-приложениями компания Microsoft предоставляет элемент управления *ReportViewer*. Имеются две версии, одна для приложений Windows Forms, а другая — для приложений Web Forms. Обе версии элемента управления поддерживают обработку и в удаленном, и в локальном режиме.

В режиме удаленной обработки элемент управления настраивается для отображения отчета, развернутого вами на том экземпляре сервера отчетности, который обрабатывает все извлечение данных и все запросы. В режиме локальной обработки элемент управления использует локальное определение отчета для локально обработанных данных (причем есть некоторые ограничения функциональности). Для этого режима не требуется лицензия на SQL Server или на сервер отчетности, поскольку это ваше собственное клиентское приложение запрашивает источники данных (для получения данных для отчета).

Давайте создадим клиентское приложение Windows Forms, которое использует элемент управления *ReportViewer* (в режиме удаленной обработки) для отображения созданного нами ранее отчета Sales Summary by Region and Date.

Для создания приложения выполните внутри Visual Studio следующие шаги:

1. Создайте новый проект Windows Form Application на языке Microsoft Visual C# и назовите его *ReportViewerDemo*.
2. Во вкладке **Reporting** панели инструментов перетащите элемент управления *MicrosoftReportViewer* на форму *Form1*. При этом на форму добавляется элемент управления типа *ReportViewer* с названием **reportViewer1**.

3. В меню смарт-тега элемента управления (появляется автоматически) щелкните по **Dock In Parent Container**.
4. В панели **Properties** установите свойство **ProcessingMode** в значение **Remote**.
5. Для подключения элемента управления к серверу настройте его свойства **ReportPath** и **ReportServerURL** в значения `/Sample Reports/Sales Summary by Region and Date` и `http://localhost/reportserver` (соответственно). Эти свойства находятся под узлом **ServerReport** в панели **Properties**.
6. Нажмите клавишу <F5> для запуска приложения. Форма должна показать отчет Sales Summary by Region and Date внутри встроенного элемента управления ReportViewer (как на рис. 19.21).

Form1

From Year: CY 2001 To Year: CY 2003 Country: Australia, Canada View Report

1 of 1 100%

### Sales Summary by Region and Date

|           |                  | CY 2001            | CY 2002            | CY 2003            |                    |
|-----------|------------------|--------------------|--------------------|--------------------|--------------------|
|           |                  | Total              | Total              | Total              | Total              |
| Country   | State Province   | Amount             | Amount             | Amount             | Amount             |
| Australia | New South Wales  | \$594,415          | \$933,365          | \$1,247,961        | \$2,775,741        |
|           | Queensland       | \$251,864          | \$512,857          | \$650,539          | \$1,415,260        |
|           | South Australia  | \$68,736           | \$143,121          | \$240,446          | \$452,304          |
|           | Tasmania         | \$28,041           | \$51,743           | \$106,911          | \$186,695          |
|           | Victoria         | \$365,991          | \$513,198          | \$787,926          | \$1,667,115        |
|           | <b>Total</b>     | <b>\$1,309,047</b> | <b>\$2,154,285</b> | <b>\$3,033,784</b> | <b>\$6,497,116</b> |
| Canada    | Alberta          | \$3,578            | \$8,203            | \$4,363            | \$16,145           |
|           | British Columbia | \$143,252          | \$613,399          | \$531,384          | \$1,288,035        |
|           | Ontario          |                    |                    | \$37               | \$37               |
|           | <b>Total</b>     | <b>\$146.830</b>   | <b>\$621.602</b>   | <b>\$535.784</b>   | <b>\$1.304.217</b> |

**Рис. 19.21.** Визуализация отчета в приложении Windows Forms при помощи элемента управления ReportViewer



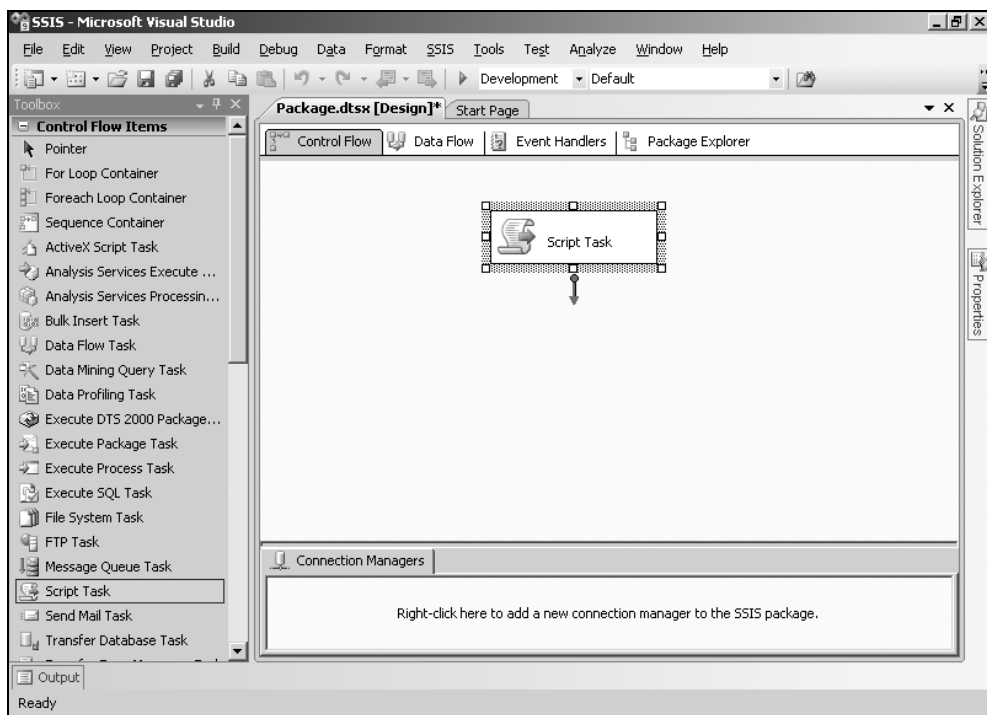
Версия ASP.NET элемента управления ReportViewer (используемая в Web-приложении) реализована очень похоже. Однако между ними есть и отличия. Во-первых, Web-версия элемента управления использует для отображения отчета HTML-форматирование, а версия Windows Forms — интерфейс Graphics Device Interface (GDI). Это может привести к некоторым отличиям в представлении (вследствие неточного характера HTML-визуализации). Второе отличие относится к печати. Приложение Windows Forms использует свойства вывода на печать компьютера пользователя, а приложение Web-форм использует для печати элемент управления ActiveX. И наконец, Web-версия элемента управления имеет минимальные системные требования: для обеспечения полной функциональности достаточно наличия Internet Explorer 6.0 (или более поздней версии) с включенной поддержкой скриптов.

## Вывод генерирования отчетов в скрипты при помощи служб интеграции SQL Server

Для генерирования отчетов вы можете использовать также и службы интеграции SQL Server Integration Services. В нашем следующем примере демонстрируется использование служб интеграции для вывода в скрипт задачи, которая взаимодействует с сервером отчетности при помощи Web-служб. Эта задача вызывает Web-службы Reporting Services для архивирования отчета (при этом вывод форматируется как документ в формате PDF и сохраняется в каталоге назначения).

В более ранних версиях SQL Server задачу Integration Services Script Task можно было написать только на языке Visual Basic .NET (это ограничение по-прежнему существует для задач Reporting Services, как мы только что видели на примере утилиты rs). В SQL Server 2008 вы можете писать задачи Integration Services Script Tasks и на языке C#. Для реализации нашей задачи Integration Services Script Task на языке C# выполните следующие шаги:

1. Запустите Visual Studio 2008 и выберите **File | New Project**.
2. Выберите **Business Intelligence Projects** в **Project Types** слева, а затем выберите шаблон **Integration Services Project**.
3. Назовите проект SSIS, а затем нажмите **OK**. Visual Studio автоматически генерирует в проекте пустой пакет с названием Package.dtsx и открывает его.
4. Из панели инструментов перетащите **Script Task** на вкладку **Control Flow**. Пакет должен теперь выглядеть так, как показано на рис. 19.22.



**Рис. 19.22.** Проект Integration Services после добавления задачи **Script Task**

5. Дважды щелкните по задаче **Script Task**, чтобы открыть ее окно **Properties**. Убедитесь, что свойство **ScriptLanguage** установлено в значение **Microsoft Visual C# 2008**. Нажмите кнопку **Edit Script**, чтобы открыть редактор Integration Services Script Task. Файл с названием **ScriptMain.cs** автоматически создается и открывается в новом редакторе Visual Studio. Этот файл содержит код задачи.
6. В окне **Project Explorer** щелкните правой кнопкой мыши по узлу **References**, а затем выберите **Add Web Reference**.
7. Настройте URL на  
`http://<servername>:80/reportserver/ReportService2005.asmx?wsdl`.  
(местоположение Web-службы управления, замените <servername> названием вашего сервера отчетности).
8. Настройте в названии Web-ссылки значение **ReportManagementProxy**, а затем нажмите **Add Reference**.

9. Повторите этот процесс для добавления второй Web-ссылки на `http://<servername>:80/reportserver/ReportExecution2005.asmx?wsdl` (с названием `ReportExecutionProxy`). Это местоположение Web-службы выполнения замените `<servername>` названием вашего сервера отчетности.
10. Проверьте правильность настройки Web-ссылок. Для этого дважды щелкните по **Settings.settings** под узлом **Properties** в окне **Project Explorer**, а затем убедитесь, что значение **Value** каждого из имеющихся там элементов соответствует указанным в предыдущих шагах URL.
11. Вернитесь на вкладку **ScriptMain.cs** и замените функцию `Main` кодом из листинга 19.2. Объяснение основных элементов кода приведено далее.

**Листинг 19.2. Код задачи Integration Services Script Task**

```
public void Main()
{
    int status = (int)ScriptResults.Success;
    bool refire = false;
    string errMsg = string.Empty;
    string url = "http://localhost/reportserver";
    try
    {
        // Получить ссылку на службу управления SSRS
        ReportManagementProxy.ReportingService2005 mgtSvc =
            GetManagementServiceReference(url);
        if (mgtSvc == null)
        {
            errMsg = "Failed to create instance of SSRS management
                Web Service";
            status = (int)ScriptResults.Failure;
        }
    }
    else
    {
        // Получить ссылку на службу выполнения SSRS
        ReportExecutionProxy.ReportExecutionService execSvc =
            GetExecutionServiceReference(url);
        if (execSvc == null)
        {
            errMsg = "Failed to create instance of SSRS execution
                Web Service";
```

```

        status = (int)ScriptResults.Failure;
    }
    else
    {
        if (ArchiveReport(mgtSvc, execSvc, "/",
            "Sales Summary by Region and Date", ref errMsg) !=
            (int)ScriptResults.Success)
            Dts.Events.FireInformation(2010, "", errMsg, "", 0, ref refire);
    }
}
}
catch (Exception ex)
{
    errMsg = ex.Source + ": " + ex.Message;
    status = (int)ScriptResults.Failure;
}
Dts.Events.FireInformation(2010, "", errMsg, "", 0, ref refire);
Dts.TaskResult = status;
}

private ReportManagementProxy.ReportingService2005
    GetManagementServiceReference(string url)
{
    ReportManagementProxy.ReportingService2005 mgtSvc =
        new ReportManagementProxy.ReportingService2005();
    mgtSvc.Url = url + "/ReportService2005.asmx";
    mgtSvc.Timeout = 60000; //60 seconds
    mgtSvc.Credentials = System.Net.CredentialCache.DefaultCredentials;
    System.Net.WebProxy wp = System.Net.WebProxy.GetDefaultProxy();
    mgtSvc.Proxy = wp;
    return mgtSvc;
}

private ReportExecutionProxy.ReportExecutionService
    GetExecutionServiceReference(string url)
{
    ReportExecutionProxy.ReportExecutionService execSvc =
        new ReportExecutionProxy.ReportExecutionService();
    execSvc.Url = url + "/ReportExecution2005.asmx";
    execSvc.Timeout = 900000; //15 minutes
    execSvc.Credentials = System.Net.CredentialCache.DefaultCredentials;

```



```

        if (GenerateReport(execSvc, item, dest-
File, destFormat) !=
                                (int)ScriptResults.Success)
            status = (int)ScriptResults.Failure;
    }
}
}
catch (Exception ex)
{
    errMsg = ex.Message;
    status = (int)ScriptResults.Failure;
}
}
}
catch (Exception ex)
{
    errMsg = ex.Message;
    status = (int)ScriptResults.Failure;
}
return status;
}
private int GenerateReport(
    ReportExecutionProxy.ReportExecutionService execSvc,
    ReportManagementProxy.CatalogItem item,
    string destFile, string format)
{
    int status = (int)ScriptResults.Success;
    bool refire = false;
    try
    {
        string historyid = null;
        ReportExecutionProxy.ExecutionInfo ei =
            execSvc.LoadReport(item.Path, historyid);
        string deviceInfo = null;
        byte[] results;
        string encoding = string.Empty;
        string mimetype = string.Empty;
        string extension = string.Empty;
        ReportExecutionProxy.Warning[] warnings = null;

```

```

string[] streamIDs = null;
    results = execSvc.Render(format, deviceInfo,
        out extension, out mimetype, out encoding, out warn-
ings, out streamIDs);
    if (results != null)
    {
        System.IO.FileStream stream =
            System.IO.File.Create(destFile, results.Length);
        stream.Write(results, 0, results.Length);
        stream.Close();
    }
    Dts.Events.FireInformation(2010, "", "Successfully archived report "
        + item.Name + " to " + destFile, "", 0, ref refire);
}
catch (Exception ex)
{
    status = (int)ScriptResults.Failure;
    Dts.Events.FireInformation(2010, "", "ERROR: While archiving report "
        + item.Name + ": " + ex.Message, "", 0, ref refire);
}
return status;
}

```

12. Сохраните код и закройте редактор Integration Services Script Task, а затем вернитесь в диалоговое окно **Script Task Editor** и нажмите **OK**.
13. В панели **Solution Explorer** в Visual Studio щелкните правой кнопкой мыши Package1.dtsx, а затем выберите **Execute Package** для выполнения нашей задачи Integration Services Script Task.

Давайте изучим код листинга 19.2. Точка входа по умолчанию — это метод `main`. Сначала создается два прокси Web-служб (один для службы управления, а второй — для службы выполнения) двумя строками кода:

```

ReportManagementProxy.ReportingService2005 mgmtSvc =
    GetManagementServiceReference(url);

:
ReportExecutionProxy.ReportExecutionService execSvc =
    GetExecutionServiceReference(url);

```

Каждый из этих фрагментов кода вызывает находящийся далее в скрипте метод, который создает экземпляр соответствующего прокси Web-службы и настраивает его URL, тайм-аут, учетные данные, а также свойства прокси.

(Прокси — это объект, который доступен нашему коду и взаимодействует непосредственно с Web-службой.) Затем мы вызываем метод `ArchiveReport` и передаем ему два прокси Web-служб, путь на сервере отчетности к архивируемому отчету, а также название отчета:

```
if (ArchiveReport(mgtSvc, execSvc, "/", "Sales Summary by Region and  
Date" ref errMsg) != (int)ScriptResults.Success)
```

Метод `Main` заканчивается настройкой `Dts.TaskResult` (чтобы указать успех или неудачу выполнения задачи).

Метод `ArchiveReport` начинается с проверки существования выходного каталога (в примере он жестко запрограммирован как `C:\ReportOutput\`) и создания его в случае отсутствия. Затем вызывается метод `ListChildren` Web-службы управления в цикле `foreach` для перечисления всех элементов в указанном исходном пути (который в примере передается как корневой каталог `/`) на сервере отчетности. Следом за проверкой (является ли данный элемент нужным нам отчетом `Sales Summary by Region and Date`) мы генерируем отчет при помощи вызова Web-службы:

```
if (GenerateReport(execSvc, item, destFile, destFormat) !=  
(int)ScriptResults.Success)
```

Метод `GenerateReport` делает всю реальную работу по генерированию выходных данных. Сначала он загружает элемент отчета (при помощи вызова метода `LoadReport` Web-службы выполнения):

```
ReportExecutionProxy.ExecutionInfo ei =  
    execSvc.LoadReport(item.Path, historyid);
```

Реальная визуализация отчета выполняется методом `Render` Web-службы выполнения:

```
results = execSvc.Render(format, deviceInfo, out extension, out mimeType,  
    out encoding, out warnings, out streamIDs);
```

Метод `Render` возвращает массив байтов, представляющий собой вывод отчета в формате PDF (в соответствии с указанным параметром `format` — установленным в значение `PDF` в методе `ArchiveReport`), который затем записывается на диск при помощи объекта `FileStream`.

## Администрирование служб Reporting Services

Для выполнения задач администрирования Reporting Services сервер SQL Server 2008 предоставляет несколько инструментов. В этом разделе мы изу-



чим администрирование Reporting Services при помощи Reporting Services Configuration Manager, Reporting Services Web Services, Report Manager и SQL Server Management Studio.

## Использование Reporting Services Configuration Manager

Настройка инсталляции Reporting Services производится при помощи инструмента Reporting Services Configuration Manager. В типичной инсталляции инструмент Reporting Services Configuration Manager находится в меню **Start** операционной системы Windows.

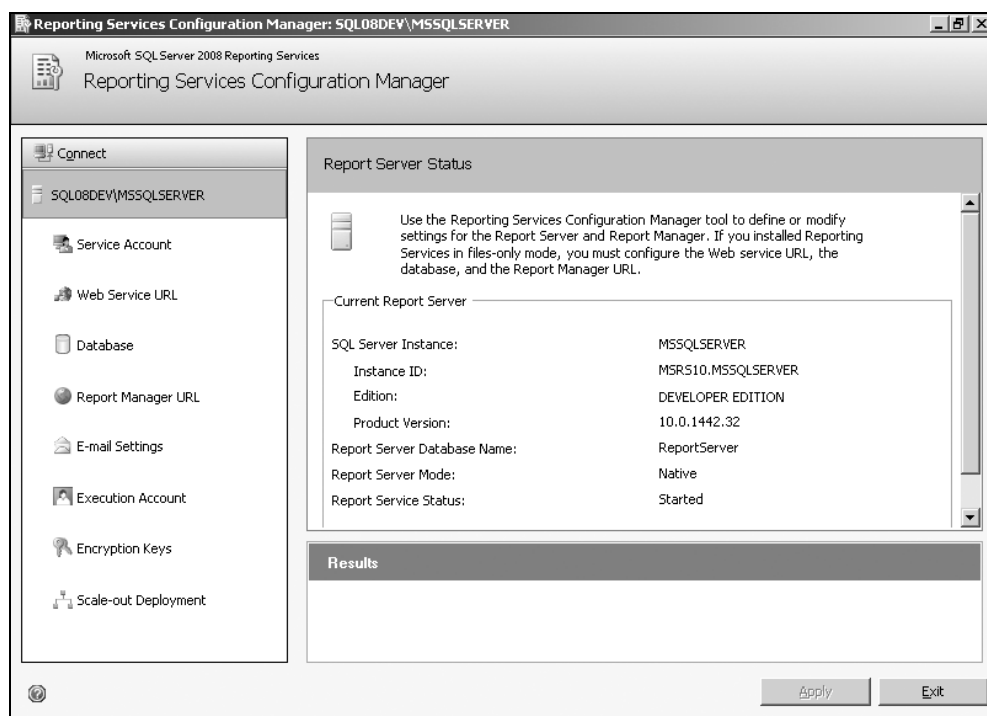


Рис. 19.23. Reporting Services Configuration Manager

При инсталляции Reporting Services по умолчанию выполняется инсталляция Files-Only. После инсталляции сервер отчетности до завершения конфигурации при помощи Reporting Services Configuration Manager использовать нель-

зя. SQL Server 2008 вводит два новых варианта инсталляции: инсталляция в обычном режиме и инсталляция в режиме интеграции с SharePoint. (Интеграцию с SharePoint мы скоро обсудим.) Если сервер был инсталлирован в одном из этих двух режимов, то данный инструмент можно использовать для проверки настроек и внесения изменений. На рис. 19.23 показан инструмент Reporting Services Configuration Manager.

В табл. 19.10 перечислены задачи, которые можно выполнить при помощи Reporting Services Configuration Manager.

**Таблица 19.10.** Задачи Reporting Services Configuration Manager

| Настройка          | Описание                                                                                                                                                                                                                                                                                                                          |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Service Account    | Служебная учетная запись настраивается во время инсталляции, но здесь вы можете изменить пароль или использовать другую учетную запись                                                                                                                                                                                            |
| Web Service URL    | Report Manager — это приложение ASP.NET, доступ к которому осуществляется через URL. Этот URL дает доступ к конечным точкам SOAP сервера отчетности. В каждом приложении может быть настроен один или несколько URL                                                                                                               |
| Database           | Серверу отчетности для его внутренних целей требуется база данных SQL Server. Вы можете использовать этот инструмент для создания и настройки подключения к базе данных сервера отчетности, либо для выбора существующего сервера отчетности                                                                                      |
| Report Manager URL | Report Manager — это приложение ASP.NET, доступ к которому осуществляется через URL. Этот URL используется для запуска Report Manager. В каждом приложении может быть настроен один или несколько URL                                                                                                                             |
| E-Mail Settings    | Службы Reporting Services могут доставлять отчеты или уведомления по обработке отчетов на электронный почтовый ящик (по протоколу Simple Mail Transfer Protocol (SMTP)). Вы можете использовать этот инструмент для указания сервера SMTP и адреса для доставки электронной почты                                                 |
| Execution Account  | Эта учетная запись используется для удаленных подключений во время выполнения запланированных по расписанию операций или при отсутствии учетных данных пользователя                                                                                                                                                               |
| Encryption Keys    | Имеются функции для резервного копирования, восстановления и замены симметричного ключа, который используется для шифрования хранимых строк подключения и учетных данных. Если вы меняете учетные данные службы или переносите базу данных сервера отчетности на другой компьютер, то у вас должна быть копия симметричного ключа |

Таблица 19.10 (окончание)

| Настройка            | Описание                                                                                                                                                                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scale-Out Deployment | Службы Reporting Services поддерживают работу нескольких экземпляров сервера отчетности с одной совместно используемой базой данных. Данный инструмент используется для подключения серверов отчетности к разделяемой базе данных |

## Доступ к Web-службам Report Server

Как вы уже видели на нескольких примерах, полная функциональность Reporting Services предоставляется разработчику через Web-службы. Это позволяет разработчику создавать пользовательские приложения, которые предоставляют всю функциональность интерфейсов Report Manager и SQL Server Management Studio. Эта функциональность простирается от выполнения административных задач (таких как безопасность и обслуживание отчетов) и до представления отчетов (а также обслуживания подписок). В листингах 19.1 и 19.2 мы видели только малую часть того, что можно сделать при помощи Web-служб, но вы должны понимать: все, что можно сделать в интерактивном режиме в существующих пользовательских интерфейсах, можно также сделать программным путем при помощи Web-служб.

Web-службы предоставляют *конечные точки* для управления сервером отчетности, а также для выполнения отчетов и переходов между ними. Название конечной точки управления (используемой для управления такими объектами сервера, как отчеты, источники данных и т. д.) зависит от того, работает ли сервер в режиме интеграции с SharePoint. В обычном режиме работы (то есть когда сервер отчетности не настроен для интеграции с SharePoint) конечная точка называется ReportService2005. В режиме интеграции с SharePoint конечная точка называется ReportService2006. На сервере есть только одна конечная точка, и при использовании неверной конечной точки будет выдаваться ошибка.

URL для доступа к языку Web Services Description Language (WSDL) для конечной точки управления на сервере отчетности, работающем в обычном режиме:

**`http://<ReportServerName>/ReportServer/ReportService2005.asmx?wsdl`**

URL для доступа к языку WSDL для конечной точки управления на сервере отчетности, работающем в режиме интеграции с SharePoint:

**`http://<SharePointServerName>/<SiteName>/_vti_bin/ReportServer/ReportService2006.asmx?wsdl`**

Название конечной точки выполнения (используемой разработчиками для настройки обработки и визуализации отчетов) — `ReportExecution2005`. Эта точка используется как для обычного режима работы, так и для режима интеграции с SharePoint. URL для доступа к языку WSDL для конечной точки выполнения на сервере отчетности, работающем в обычном режиме:

**`http://<ReportServerName>/ReportServer/ReportExecution2005.asmx?wsdl`**

URL для доступа к языку WSDL для конечной точки выполнения на сервере отчетности, работающем в режиме интеграции с SharePoint:

**`http://<SharePointServerName>/<SiteName>/_vti_bin/ReportServer/ReportExecution2005.asmx?wsdl`**

И наконец, в том случае, когда сервер отчетности настроен в режим интеграции с SharePoint и инсталлирована надстройка Reporting Services, то создается дополнительная конечная точка `ReportServiceAuthentication`. Web-служба `ReportServiceAuthentication` используется для аутентификации между сервером отчетности и Web-приложением SharePoint, настроенным на использование аутентификации Forms Authentication. URL для доступа к конечной точке аутентификации на интегрированном с SharePoint сервере отчетности такой:

**`http://<SharePointServerName>/<SiteName>/_vti_bin/ReportServer/ReportServiceAuthentication.asmx`**

## Резюме

Данная глава познакомила вас с жизненным циклом выполнения отчетов при помощи служб Reporting Services сервера SQL Server 2008. Мы начали с создания отчетов и запросов для этих отчетов (при помощи визуального конструктора Report Designer в Visual Studio). После того как вы научились создавать простые отчеты с нуля или при помощи мастера Report Wizard, мы перешли к созданию более сложных отчетов на базе OLAP по базам данных Analysis Services. Попутно мы затронули прочие возможности проектирования (в том числе выражения, форматирование, а также кодирование при помощи встроенных скриптов и ссылок на внешние .NET-сборки).

Затем мы описали все, что следует за созданием безупречного отчета: мы увидели развертывание отчетов при помощи элемента управления ReportViewer в .NET-приложениях. Службы Reporting Services дают вам возможность поставлять данные, которые можно получить при помощи множества программируемых функций SQL Server 2008, описанных нами в этой книге.

# Разработка приложений на основе Microsoft® SQL Server® 2008

*Практическое руководство  
для разработки приложений  
с использованием баз данных*

В книге подробно излагаются основные возможности программирования SQL Server 2008, приводятся практические рекомендации по программированию, образцы кода и примеры приложений, которые помогут при проектировании и разработке надежных приложений баз данных и решений в области бизнес-аналитики.

## Вы научитесь:

- > делать запросы к сложным данным при помощи улучшений языка Transact-SQL;
- > использовать новые нереляционные функции: иерархические таблицы, потоковый вывод файлов, геопространственные возможности;
- > применять XML внутри базы данных для создания приложений, использующих XML;
- > использовать и доставлять данные с помощью Microsoft LINQ, Entity Framework, а также привязки данных;
- > реализовывать шифрование на уровне базы данных и аудит сервера;
- > создавать и обслуживать хранилища данных;
- > использовать Microsoft Excel для создания клиентских интерфейсов для кубов OLAP и язык MDX для запросов к ним;
- > быстро и эффективно интегрировать интеллектуальный анализ данных в приложения.



## Об авторах

**Леонард Лобел (Leonard Lobel)** — руководитель технологического отдела и соучредитель компании Sleek Technologies, Inc. (специализируется на создании решений на базе продуктов Microsoft), главный консультант компании twenty-six New York, которая имеет статус Microsoft Gold Certified Partner.

**Эндрю Дж. Браст (Andrew J. Brust)** — главный специалист по новым технологиям в компании twenty-six New York, региональный директор компании Microsoft по Нью-Йорку и Нью-Джерси, член совета Business Intelligence Partner Advisory Council компании Microsoft, имеет звание MVP по Visual Basic.

**Стивен Форте (Stephen Forte)** — руководитель стратегического отдела компании Telerik, имеющей статус Microsoft Gold Certified Partner, основатель группы пользователей NYC .NET Developer User Group, автор книг по разработке приложений и баз данных, имеет звание MVP.

## РУССКАЯ РЕДАКЦИЯ

Русская Редакция  
Москва,  
Шелепихинская наб., 32  
E-mail: info@rusedit.com  
Internet: www.rusedit.com  
Тел./факс (495) 638-5-638



БХВ-Петербург  
190005, Санкт-Петербург,  
Измайловский пр., 29  
E-mail: mail@bhv.ru  
Internet: www.bhv.ru  
Тел. (812) 251-42-44  
Тел./факс (812) 320-01-79

ISBN 978-5-7502-0393-2



9 785750 203932

Microsoft®  
SQL Server® 2008

Microsoft®